MODUL PEMBELAJARAN PEMROGRAMAN MOBILE

Flutter & Dart

Dari dasar Dart, widget, dan layout hingga penyimpanan lokal Hive, backend Supabase, proyek CRUD dengan login & register, testing, dan rilis aplikasi.

Program Studi S1 Informatika · Fakultas Teknik
Universitas Jenderal Soedirman

Versi 28 Oktober 2026

Daftar Isi

1. **Modul 00** — Silabus Perkuliahan & Pengenalan Mata Kuliah
2. **Modul 01** — Pengenalan Flutter, Arsitektur & Setup Lingkungan
3. **Modul 02** — Fondasi Bahasa Pemrograman Dart Modern (Dart 3)
4. **Modul 03** — Arsitektur Widget & Lifecycle Antarmuka
5. **Modul 04** — Sistem Layouting & Desain Antarmuka Responsif
6. **Modul 05** — Interaktivitas, State Dasar & Validasi Formulir
7. **Modul 06** — Scrollable Views & Optimasi Performa List Data
8. **Modul 07** — Navigasi & Routing Multi-Halaman
9. **Modul 08** — Manajemen State Terstruktur dengan Provider
10. **Modul 09** — Integrasi REST API & Asynchronous Networking
11. **Modul 10** — Penyimpanan Data Lokal (SharedPreferences & SQLite)
12. **Modul 11** — Database NoSQL Lokal dengan Hive
13. **Modul 12** — Backend as a Service dengan Supabase
14. **Modul 13** — Proyek Catatan Kuliah (1): Arsitektur & Autentikasi Supabase
15. **Modul 14** — Proyek Catatan Kuliah (2): CRUD Supabase & Cache Offline Hive
16. **Modul 15** — Debugging, DevTools & Pengujian Otomatis
17. **Modul 16** — Build Rilis APK/AAB & Panduan Capstone Project

MODUL 00 · PENGANTAR

Silabus Perkuliahan & Pengenalan Mata Kuliah

Rencana Pembelajaran Semester (RPS) untuk 17 modul (M00-M16) dalam 16 pertemuan, alur belajar dan alasan urutannya, cara memakai web tutorial, metode belajar yang disarankan, kontrak perkuliahan, serta persiapan perangkat untuk pengembangan aplikasi mobile dengan Flutter & Dart.

Durasi Pertemuan 1 (bersama Modul 01)

Prasyarat Algoritma & Pemrograman Dasar
Konsep Berorientasi Objek (OOP)

Capaian Pembelajaran

- Memahami roadmap pembelajaran Flutter dari dasar, data lokal dan cloud (Hive, Supabase), hingga rilis produksi.
- Menjelaskan alasan urutan modul dan keterkaitan antarmodul.
- Menggunakan tab Materi, Praktikum, dan Kuis serta menjalankan contoh kode dengan benar.
- Mengetahui bobot penilaian praktikum, tugas mandiri, dan proyek akhir (capstone).
- Menyiapkan perangkat keras dan perangkat lunak sesuai standar industri pengembangan mobile.

Daftar Isi Modul

1. Rencana Pembelajaran Semester (RPS)
2. Jadwal 16 Pertemuan + UTS/UAS
3. Alur Belajar: Mengapa Urutannya Seperti Ini?
4. Cara Menggunakan Web Tutorial (Materi, Praktikum, Kuis)
5. Metode Belajar & Cara Menjalankan Contoh Kode
6. Kontrak Perkuliahan & Bobot Penilaian
7. Spesifikasi Perangkat Keras & Lunak yang Disarankan
8. Tugas Praktikum
9. Kuis Pemahaman & Kunci Jawaban

1. Rencana Pembelajaran Semester (RPS)

Selamat datang di mata kuliah **Pemrograman Aplikasi Perangkat Bergerak (Mobile Application Development)** berbasis **Flutter & Dart** untuk mahasiswa **Program Studi S1 Informatika, Fakultas Teknik, Universitas Jenderal Soedirman (UNSOED)**. Kurikulum ini dirancang terstruktur dan komprehensif agar mahasiswa memiliki keterampilan siap industri (industry-ready) dalam membangun aplikasi Android dan iOS berkualitas tinggi dari satu basis kode (single codebase).

> 🎯 **Tujuan Kompetensi Lulusan (CPMK)**
>
> Setelah menyelesaikan rangkaian modul ini, mahasiswa diharapkan mampu merancang arsitektur aplikasi mobile yang bersih, mengimplementasikan antarmuka reaktif yang responsif, mengelola status aplikasi (state management), mengintegrasikan layanan web API (RESTful), mengelola basis data lokal offline-first (SQLite dan Hive), membangun backend cloud dengan autentikasi (Supabase), serta mempublikasikan aplikasi ke format berkas rilis (.apk dan .aab).

Rincian CPMK

Kode	Capaian Pembelajaran Mata Kuliah	Modul Pendukung
CPMK-1	Menyiapkan lingkungan pengembangan dan menulis program Dart 3 yang benar (null safety, OOP, asynchronous).	M01, M02
CPMK-2	Membangun antarmuka Material 3 yang responsif, interaktif, dan mudah dipakai.	M03 - M06
CPMK-3	Menyusun aplikasi multi-halaman dengan navigasi dan state management yang terstruktur.	M07, M08
CPMK-4	Mengelola data dari REST API, penyimpanan lokal (SharedPreferences, SQLite, Hive), dan backend cloud (Supabase).	M09 - M12
CPMK-5	Membangun proyek utuh dengan autentikasi, CRUD, dan cache offline, lalu menguji serta merilisnya.	M13 - M16

Struktur Distribusi Materi Kuliah (17 Modul, 16 Pertemuan)

Seluruh materi dibagi menjadi 17 modul (M00 sampai M16) yang dikelompokkan ke dalam 6 fase. Setiap fase menghasilkan satu capaian nyata yang bisa Anda tunjukkan.

Fase	Modul	Fokus Utama	Output Capaian
Fase 1	M00 - M02	Fondasi: Silabus, Setup Flutter, Bahasa Dart 3	Lingkungan dev siap, aplikasi "Halo Kampus" berjalan, logika Dart dikuasai
Fase 2	M03 - M06	UI: Widget & Lifecycle, Layouting Responsif, Form & Input, List & Scroll	Halaman aplikasi kampus yang rapi dan interaktif
Fase 3	M07 - M08	Arsitektur Aplikasi: Navigasi & Routing, State Management Provider	Aplikasi multi-halaman dengan state terpusat (materi UTS)
Fase 4	M09 - M12	Data: REST API, SharedPreferences & SQLite, Hive NoSQL, Supabase (Database, Auth, RLS)	Aplikasi yang membaca/menyimpan data lokal dan cloud
Fase 5	M13 - M14	Proyek Tutorial Terpadu: Auth login/register Supabase, CRUD + cache offline Hive	Satu aplikasi utuh siap pakai sebagai pola dasar capstone
Fase 6	M15 - M16	Kualitas & Rilis: Debugging, Testing, Build Rilis, Capstone	Proyek portofolio teruji dan siap rilis (.apk / .aab)

Daftar Lengkap Modul

0. **M00 Silabus** - RPS, kontrak kuliah, cara belajar, persiapan perangkat.
1. **M01 Pengenalan Flutter & Setup** - instalasi, CLI, anatomi proyek, Hot Reload, aplikasi pertama.
2. **M02 Bahasa Dart 3** - variabel, null safety, fungsi, OOP, collection, async.
3. **M03 Widget Dasar & Lifecycle** - Stateless vs Stateful, `setState`, siklus hidup.
4. **M04 Layouting & Responsif** - Row, Column, Stack, Expanded, LayoutBuilder.
5. **M05 Form & Input** - TextFormField, validasi, controller.
6. **M06 List & Scroll** - ListView.builder, GridView, pencarian.
7. **M07 Navigasi & Routing** - Navigator, kirim/terima data antar halaman.
8. **M08 State Management Provider** - ChangeNotifier, Consumer, state global.
9. **M09 REST API** - package http, JSON, model, FutureBuilder.
10. **M10 Local Storage** - SharedPreferences dan SQLite.
11. **M11 Hive NoSQL Lokal** - box, adapter, mini proyek CRUD catatan offline.

12. **M12 Supabase** - database Postgres, Auth, Row Level Security (RLS).
13. **M13 Proyek Tutorial 1** - setup arsitektur + Auth login/register Supabase.
14. **M14 Proyek Tutorial 2** - CRUD Supabase + cache offline Hive.
15. **M15 Debugging & Testing** - DevTools, unit test, widget test.
16. **M16 Build Rilis & Capstone** - ikon, signing, build APK/AAB, presentasi proyek.

2. Jadwal 16 Pertemuan + UTS/UAS

Perkuliahan berlangsung **16 pertemuan tatap muka** (masing-masing 150 menit: kira-kira 50 menit penjelasan konsep dan 100 menit praktik), ditambah **UTS** setelah pertemuan 8 dan **UAS** (demo capstone) setelah pertemuan 16. Satu pertemuan umumnya membahas satu modul.

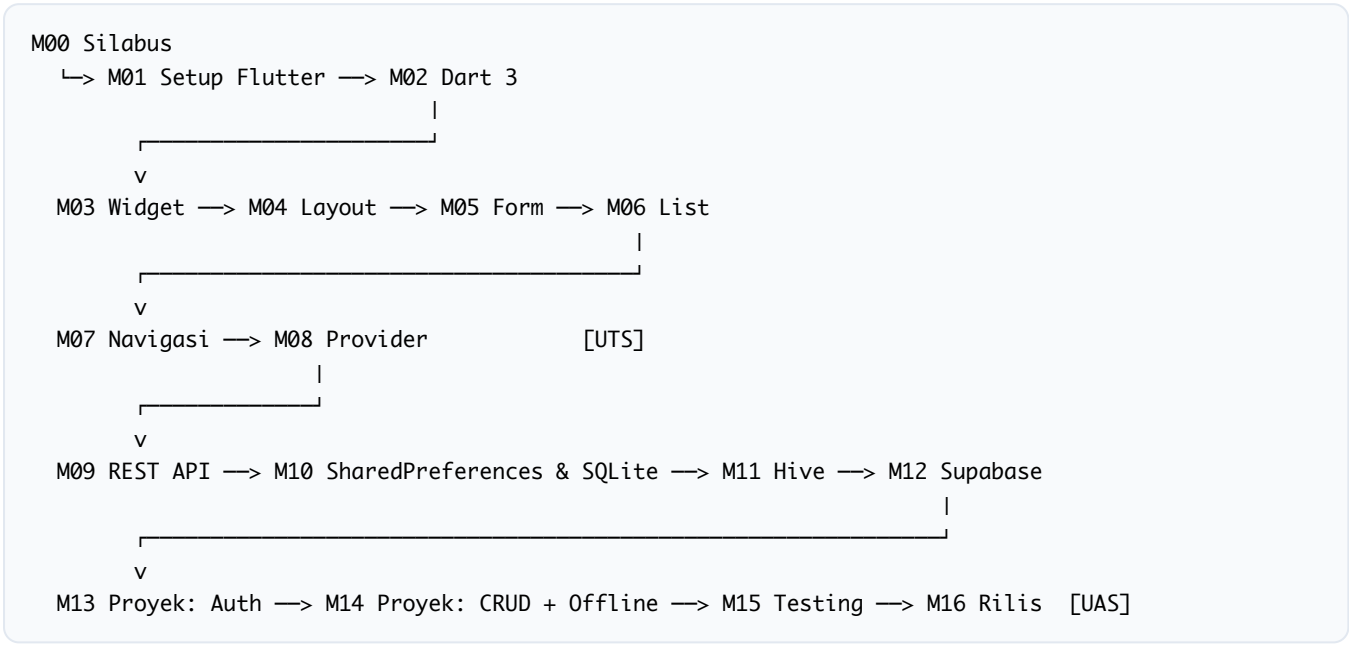
Pertemuan	Modul	Topik	Praktikum / Luaran
1	M00 + M01	Kontrak kuliah, pengenalan Flutter, instalasi SDK, CLI, Hot Reload	Repositori GitHub + aplikasi "Halo Kampus" berjalan
2	M02	Dart 3: tipe data, null safety, fungsi, OOP, collection, async/await	Program Dart konsol (DartPad)
3	M03	Widget dasar, Stateless vs Stateful, lifecycle	Halaman profil interaktif
4	M04	Layouting: Row, Column, Stack, Expanded, layout responsif	Dashboard kampus responsif
5	M05	Form, TextFormField, validasi input	Form pendaftaran mahasiswa
6	M06	ListView, GridView, list dinamis, pencarian	Daftar mata kuliah dengan filter
7	M07	Navigasi, routing, mengirim data antar halaman	Aplikasi multi-halaman
8	M08	State management dengan Provider	Aplikasi KRS dengan state global
UTS	M01 - M08	Ujian praktik: membangun UI multi-halaman + Provider	Aplikasi ujian dikumpulkan via GitHub
9	M09	REST API: http, JSON, model, penanganan error	Aplikasi pembaca data API
10	M10	SharedPreferences dan SQLite	Pengaturan tersimpan + CRUD SQLite
11	M11	Hive NoSQL lokal, box, adapter	Mini proyek CRUD catatan offline
12	M12	Supabase: tabel, query, Auth, Row Level Security	Proyek Supabase + tabel dengan RLS
13	M13	Proyek Tutorial 1: struktur folder, login, register, logout	Aplikasi dengan alur autentikasi lengkap
14	M14	Proyek Tutorial 2: CRUD Supabase + cache offline Hive	Aplikasi CRUD online/offline utuh

Pertemuan	Modul	Topik	Praktikum / Luaran
15	M15	Debugging dengan DevTools, unit test, widget test	Proyek dengan test yang lulus
16	M16	Build rilis APK/AAB, ikon, signing, persiapan capstone	Berkas rilis + draf capstone
UAS	M09 - M16	Presentasi dan *code review defense* capstone kelompok	Aplikasi capstone, laporan, video demo

> 💡 **Satu pertemuan, satu modul**
>
> Sebagian besar modul (terutama M02, M09 - M14) berisi materi yang lebih panjang dari 150 menit. Di kelas, fokus pada konsep inti dan contoh lengkap; label durasi di setiap modul menunjukkan pertemuan ke berapa modul itu dibahas. Bagian yang belum selesai di kelas dilanjutkan sebagai belajar mandiri di rumah sebelum pertemuan berikutnya. Jangan menumpuk: modul berikutnya selalu memakai modul sebelumnya.

3. Alur Belajar: Mengapa Urutannya Seperti Ini?

Urutan modul tidak dibuat asal. Setiap modul adalah "anak tangga" yang dipakai oleh modul sesudahnya. Jika satu anak tangga dilewati, modul berikutnya akan terasa sulit. Berikut alur belajarnya dan alasan di balik urutan tersebut.



Mengapa urutannya seperti ini?

Tahap	Alasan
Setup dulu (M01)	Tanpa SDK yang berjalan, tidak ada satu pun contoh kode yang bisa dicoba. `flutter doctor` harus bersih sebelum masuk materi.
Dart sebelum widget (M02)	Widget ditulis dengan Dart. Konsep class, constructor, named parameter, dan null safety muncul di setiap baris Flutter. Belajar Dart di DartPad lebih ringan daripada langsung di aplikasi.
Widget, layout, form, list (M03 - M06)	Dari yang paling kecil (satu widget) ke yang paling umum dipakai (daftar data). Setelah ini Anda bisa membuat hampir semua tampilan satu halaman.
Navigasi lalu Provider (M07 - M08)	Saat aplikasi punya banyak halaman, data harus bisa dibagi antar halaman. Itulah masalah yang diselesaikan Provider, sehingga Provider baru masuk akal setelah navigasi.
Data: API, lokal, cloud (M09 - M12)	Dimulai dari membaca data (REST API), lalu menyimpan di perangkat (SharedPreferences, SQLite, Hive), terakhir backend lengkap dengan login (Supabase). Hive dibahas sebelum Supabase karena nanti dipakai sebagai cache offline.
Proyek terpadu (M13 - M14)	Semua konsep sebelumnya digabung dalam satu aplikasi nyata: Auth, CRUD, Provider, Hive. Proyek ini menjadi pola dasar capstone Anda.
Testing dan rilis (M15 - M16)	Setelah aplikasinya ada, barulah kita memastikan kualitasnya (test) dan mengemasnya untuk pengguna (build rilis).

> ⚠️ **Jangan lompat ke Supabase dulu**
>
> Banyak mahasiswa ingin langsung membuat aplikasi login. Kode M13 memakai `async/await` (M02), `Form` dan validasi (M05), `Navigator` (M07), dan `ChangeNotifier` (M08) sekaligus. Jika belum paham keempatnya, error yang muncul akan sangat membingungkan.

4. Cara Menggunakan Web Tutorial (Materi, Praktikum, Kuis)

Web tutorial ini adalah "buku ajar interaktif" mata kuliah. Setiap modul memiliki tiga tab di bagian atas halaman modul:

📖 Materi

Isi pelajaran yang dibagi menjadi beberapa section bernomor. Baca berurutan dari section 1. Gunakan daftar isi di samping untuk melompat ke section tertentu.

🛠️ Tugas Praktikum Mandiri

Sasaran, langkah kerja, luaran yang diharapkan, dan rubrik penilaian. Kerjakan setelah membaca semua materi modul, lalu kumpulkan ke repositori GitHub Anda.

✅ Kuis Pemahaman

Soal pilihan ganda dengan pembahasan. Kerjakan tanpa membuka materi untuk mengukur pemahaman, lalu baca pembahasan untuk soal yang salah.

Membaca komponen di dalam materi

Komponen	Artinya
Label 📄 `lib/main.dart` di atas kode	Kode adalah isi **satu file utuh**. Ganti seluruh isi file tersebut dengan kode contoh.
Kode tanpa label	Potongan (snippet) untuk dibaca atau disisipkan ke kode yang sudah ada, bukan program lengkap.
Kode `bash`	Perintah yang diketik di terminal (Terminal macOS/Linux, PowerShell, atau terminal VS Code).
Kotak ℹ️ Info	Penjelasan tambahan atau latar belakang konsep.
Kotak 💡 Tips	Cara kerja yang lebih cepat atau lebih rapi.
Kotak ⚠️ Perhatian	Kesalahan yang sering terjadi. Baca dengan teliti, biasanya ini sumber error mahasiswa.

Contoh membaca label file

Jika Anda melihat dua blok berikut di satu section, artinya Anda harus membuat **dua file** di proyek:

```
lib/main.dart                 -> ganti seluruh isi file bawaan
lib/models/mahasiswa.dart     -> buat folder models/ dan file baru di dalamnya
```

> 💡 **Progres belajar tersimpan otomatis**
>
> Status modul yang sudah Anda selesaikan disimpan di browser (localStorage). Gunakan browser dan perangkat yang sama agar progres tidak hilang. Mode gelap dan fitur pencarian materi tersedia di bagian atas halaman.

5. Metode Belajar & Cara Menjalankan Contoh Kode

Pemrograman tidak bisa dikuasai hanya dengan membaca. Gunakan siklus belajar berikut untuk setiap modul:

1. **Baca konsep** - pahami dulu masalah apa yang diselesaikan materi ini.
2. **Ketik ulang contoh kode** - jangan copy-paste di percobaan pertama. Mengetik membuat Anda memperhatikan setiap tanda kurung, koma, dan titik koma.
3. **Jalankan dan amati** - cocokkan hasil di layar dengan penjelasan materi.
4. **Ubah sedikit** - ganti teks, warna, angka, atau tambahkan satu widget. Tanyakan "apa yang terjadi jika ...?" lalu buktikan.
5. **Sengaja buat error** - hapus satu kurung atau ganti tipe data, lalu baca pesan error-nya. Ini latihan paling berharga untuk debugging.
6. **Kerjakan praktikum dan kuis** - lalu commit hasilnya ke GitHub.

Cara menjalankan contoh kode

Ada dua jenis contoh kode di web ini, dan cara menjalankannya berbeda.

1. Dart murni (terutama Modul 02). Tidak butuh Flutter maupun emulator. Buka dartpad.dev, tempel kode, lalu klik *Run*. Coba sekarang:

```dart
void main() {
  const nama = 'Mahasiswa UNSOED';
  final modul = ['Dart', 'Widget', 'Layout', 'Supabase'];

  print('Halo, $nama!');
  for (final m in modul) {
    print('Saya akan belajar $m');
  }
}
```

Hasil di panel *Console*:

```
Halo, Mahasiswa UNSOED!
Saya akan belajar Dart
Saya akan belajar Widget
Saya akan belajar Layout
Saya akan belajar Supabase
```

2. Aplikasi Flutter (Modul 01 dan seterusnya). Butuh proyek Flutter dan device (emulator, HP fisik, atau Chrome). Buat satu proyek "kelinci percobaan" khusus untuk mencoba contoh:

```
# Sekali saja: buat proyek latihan
flutter create latihan_flutter
cd latihan_flutter

# Setiap mencoba contoh berlabel lib/main.dart:
# 1. Ganti seluruh isi lib/main.dart dengan kode contoh
```

```
# 2. Jalankan aplikasi
flutter run

# Jika contoh memakai package, tambahkan dulu (contoh: package http)
flutter pub add http
```

Alur kerja Git untuk tiap praktikum

Simpan setiap praktikum sebagai commit di repositori yang dibuat pada Praktikum 0:

```
git add .
git commit -m "Praktikum 01: aplikasi Halo Kampus"
git push origin main
```

> ⚠️ **Kesalahan umum saat mencoba contoh**
>
> - **Menempel kode di bawah kode lama** sehingga ada dua fungsi `main()`. Hapus dulu seluruh isi file sebelum menempel.
> - **Lupa** `flutter pub add` sehingga muncul error `Target of URI doesn't exist`.
> - **Hanya menekan Hot Reload** setelah mengganti seluruh `main.dart`. Gunakan Hot Restart (R), penjelasannya ada di Modul 01.
> - **Mengabaikan garis merah di editor.** Jalankan `flutter analyze` dan perbaiki error sebelum `flutter run`.

6. Kontrak Perkuliahan & Bobot Penilaian

Komponen penilaian perkuliahan dirancang seimbang antara pemahaman teoritis konseptual dan keterampilan teknis (hands-on coding practical skill):

20%

Tugas Praktikum Mandiri

Latihan di setiap akhir modul untuk menguji implementasi nyata.

25%

Ujian Tengah Semester (UTS)

Ujian praktik membuat UI kompleks & logika navigasi.

35%

Capstone Project (Tugas Akhir)

Aplikasi mobile lengkap berkelompok dengan arsitektur bersih.

20%

Keaktifan & Kuis Berkala

Kuis interaktif di portal dan partisipasi bedah kode.

> ⚠️ **Kebijakan Kejujuran Akademik (Academic Integrity)**
>
> Mahasiswa diperbolehkan menggunakan dokumentasi resmi Flutter, StackOverflow, dan AI assistant (seperti Antigravity / Gemini) sebagai alat bantu belajar dan pemecahan kendala (debugging). Namun, mahasiswa **wajib memahami setiap baris kode** yang dikumpulkan. Ujian lisan / *code review defense* akan dilakukan untuk setiap proyek besar.

7. Spesifikasi Perangkat Keras & Lunak yang Disarankan

Pengembangan aplikasi mobile membutuhkan sumber daya komputasi yang memadai, terutama saat menjalankan simulator/emulator:

- **Sistem Operasi:** macOS (disarankan jika ingin build iOS), Windows 10/11 64-bit, atau Linux (Ubuntu 20.04+).
- **Prosesor:** Minimal Intel Core i5 / AMD Ryzen 5 generasi modern, atau Apple Silicon (M1/M2/M3/M4).
- **RAM:** Minimal **8 GB** (Sangat disarankan **16 GB** jika menggunakan Android Emulator).
- **Penyimpanan:** Minimal 20 GB ruang kosong SSD untuk instalasi SDK, Android Studio, dan dependencies cache.
- **Opsi Alternatif Perangkat Fisik:** Jika laptop memiliki RAM 8 GB dan terasa lambat menjalankan Android Emulator, gunakan ponsel Android fisik asli yang terhubung kabel USB dengan mode *USB Debugging* aktif. Cara ini jauh lebih hemat memori dan responsif!

Tugas Praktikum: Praktikum 0: Survey Kesiapan Lingkungan & Akun GitHub

Sasaran Praktikum

Menyiapkan akun repositori kode untuk penyerahan seluruh tugas perkuliahan dan mencoba alur menjalankan contoh kode.

Langkah Pengerjaan

1. Buat atau gunakan akun GitHub aktif Anda.
2. Buat satu repositori publik dengan format penamaan: `flutter-praktikum-[NIM]-[NAMA]`.
3. Buat file `README.md` yang memuat biodata mahasiswa dan tabel rekapitulasi praktikum M00 sampai M16 (kolom: modul, judul, status, tautan folder).
4. Jalankan contoh kode Dart pada section 5 di dartpad.dev, ubah daftar modul sesuai topik yang paling ingin Anda pelajari, lalu simpan tangkapan layarnya di repositori.
5. Isi bagian 'Spesifikasi Laptop' di README (OS, prosesor, RAM, ruang SSD kosong) dan tulis rencana device uji Anda: emulator atau HP fisik.
6. Commit dan push semua perubahan, lalu simpan tautan repositori untuk diserahkan ke sistem portal dosen.

Output yang Diharapkan

Repositori GitHub aktif dengan branch utama 'main', README berisi profil mahasiswa, tabel rekap 17 modul, spesifikasi laptop, dan tangkapan layar DartPad.

Rubrik Penilaian

Kelengkapan profil dan README (40%), Format penamaan repositori standar (20%), Bukti menjalankan kode di DartPad (20%), Riwayat commit yang rapi (20%).

Kuis Pemahaman

1. Mengapa pengujian kode pada perangkat fisik Android (Physical Device) sering disarankan untuk laptop dengan RAM 8 GB?

A. Karena Android Emulator mengonsumsi RAM 2-4 GB tambahan di laptop sehingga memperberat sistem

B. Karena Flutter tidak bisa dikompilasi ke emulator komputer

C. Karena Android Studio mewajibkan kabel USB untuk membaca kode Dart

D. Karena perangkat fisik tidak membutuhkan koneksi internet sama sekali

2. Apa peran utama repositori GitHub dalam perkuliahan pemrograman mobile ini?

A. Hanya untuk mengunduh tema editor visual

B. Sebagai pelacak riwayat perubahan kode (version control) dan portofolio digital mahasiswa

C. Untuk menggantikan fungsi Flutter SDK di sistem lokal

D. Untuk menyembunyikan source code agar tidak bisa dilihat siapa pun

3. Mengapa Modul 02 (Bahasa Dart) dipelajari sebelum Modul 03 (Widget)?

A. Karena widget Flutter ditulis dengan Dart, sehingga class, constructor, named parameter, dan null safety harus dipahami lebih dulu

B. Karena Modul 03 tidak membutuhkan Flutter SDK

C. Karena Dart hanya dipakai untuk membuat backend

D. Karena urutan modul hanya disusun berdasarkan abjad

4. Sebuah contoh kode di materi diberi label `lib/main.dart`. Apa yang harus Anda lakukan?

A. Menempelkan kode di bagian paling bawah main.dart bawaan

B. Membuat file baru bernama main2.dart

C. Mengganti seluruh isi lib/main.dart dengan kode contoh, lalu menjalankan aplikasi

D. Menjalankannya di DartPad karena semua kode berlabel bisa jalan di sana

5. Mengapa Modul 11 (Hive) ditempatkan sebelum Modul 13-14 (Proyek Tutorial dengan Supabase)?

A. Karena Hive menggantikan Supabase sepenuhnya

B. Karena Hive dipakai sebagai cache offline pada proyek CRUD Supabase di Modul 14

C. Karena Supabase hanya bisa dipasang setelah Hive dihapus

D. Karena Hive adalah syarat untuk membuat akun GitHub

Kunci Jawaban & Pembahasan

1. A — Android Emulator menjalankan virtualisasi OS Android penuh yang memakan 2 hingga 4 GB RAM komputer. Menggunakan smartphone fisik via USB Debugging memindahkan beban komputasi OS ke ponsel, membuat laptop bekerja jauh lebih ringan.

2. B — GitHub berfungsi sebagai Version Control System (VCS) untuk memantau progres perubahan kode, kolaborasi proyek, dan menjadi etalase portofolio profesional mahasiswa saat melamar magang atau kerja.

3. A — Setiap baris kode Flutter adalah kode Dart. Tanpa memahami class, constructor, parameter bernama, dan null safety, penulisan widget akan terasa membingungkan.

4. C — Label file berarti kode adalah isi satu file utuh. Menempel di bawah kode lama akan menghasilkan dua fungsi main() dan class ganda sehingga terjadi error.

5. B — Proyek tutorial bagian 2 menggabungkan CRUD ke Supabase (online) dengan cache lokal Hive (offline). Karena itu Hive harus dikuasai lebih dulu.

MODUL 01 · FUNDAMENTAL

Pengenalan Flutter, Arsitektur & Setup Lingkungan

Memahami konsep framework Flutter, arsitektur internal (Framework, Engine, Embedder), instalasi Flutter SDK dan Android Studio, mengatasi error setup yang umum, perintah CLI flutter sehari-hari, anatomi proyek, bedah kode counter bawaan baris per baris, perbedaan Hot Reload/Hot Restart/Full Restart, hingga membuat aplikasi pertama 'Halo Kampus'.

Durasi Pertemuan 1 (150 Menit)

Prasyarat Modul 00: Silabus & Persiapan

Capaian Pembelajaran

- Menjelaskan perbedaan arsitektur Native, Hybrid/WebView, dan Flutter Direct Rendering.
- Menginstalasi Flutter SDK, Android SDK, dan ekstensi editor kode, lalu memperbaiki error flutter doctor yang umum.
- Menggunakan perintah CLI flutter: create, devices, emulators, run, pub add, analyze, dan clean.
- Menganalisis peran folder dalam struktur proyek Flutter (android, ios, lib, pubspec.yaml).
- Menjelaskan setiap bagian main.dart bawaan: main, runApp, StatelessWidget, StatefulWidget, setState, dan build.
- Memilih antara Hot Reload, Hot Restart, dan Full Restart sesuai jenis perubahan kode.
- Membuat dan menjalankan aplikasi 'Halo Kampus' pada emulator/perangkat fisik.

Daftar Isi Modul

1. Apa itu Flutter & Mengapa Industri Menggunakannya?
2. Arsitektur Berlapis Flutter (Architecture Layers)
3. Instalasi dan Setup Environment
4. Error Setup yang Sering Muncul & Solusinya
5. Perintah CLI Flutter yang Sering Dipakai
6. Anatomi Struktur Folder Proyek Flutter
7. Bedah main.dart Counter Bawaan Baris per Baris
8. Hot Reload vs Hot Restart vs Full Restart
9. Aplikasi Pertama: Halo Kampus
10. Tugas Praktikum
11. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Flutter & Mengapa Industri Menggunakannya?

Flutter adalah UI Toolkit open-source yang dikembangkan oleh Google untuk membangun aplikasi yang dikompilasi secara native (secara native-compiled) untuk mobile (Android, iOS), web, dan desktop (macOS, Windows, Linux) hanya dari **satu basis kode (single codebase)**.

🚀 Fast Development

Fitur *Stateful Hot Reload* menyuntikkan perubahan kode ke dalam VM Dart dalam hitungan sub-detik tanpa kehilangan status antarmuka aplikasi.

🎨 Expressive & Flexible UI

Flutter mengontrol setiap piksel di layar menggunakan engine render grafis berkinerja tinggi (Skia & Impeller), menjamin tampilan 100% konsisten di semua platform.

⚡ Native Performance

Kode Dart dikompilasi langsung ke kode mesin native ARM (AOT - Ahead-Of-Time compilation) tanpa perantara JavaScript bridge yang lambat.

Perbandingan Pendekatan Pengembangan Mobile

Karakteristik	Native (Kotlin / Swift)	Hybrid (Cordova / Ionic)	Reactive Bridge (React Native)	Flutter (Direct Rendering)
Bahasa	Kotlin / Java (Android), Swift (iOS)	HTML, CSS, JavaScript	JavaScript / TypeScript	**Dart**
Jumlah Codebase	Dua (terpisah Android & iOS)	Satu	Satu (dengan kode native jika perlu)	**Satu (Single Codebase)**
Mekanisme Render	Widget bawaan OS	WebView browser container	Bridge JS memanggil Native Widget	**Render mandiri via Impeller/Skia Engine**
Performa	Sangat Tinggi (100%)	Rendah - Sedang (terbebani WebView)	Tinggi (ada bottleneck bridge)	**Sangat Tinggi (mendekati 60-120 FPS konsisten)**

2. Arsitektur Berlapis Flutter (Architecture Layers)

Flutter dibangun dengan arsitektur modular berlapis-lapis (layered architecture). Ini memberikan abstraksi yang bersih antara logika UI tingkat tinggi dan komunikasi hardware tingkat rendah:

3. Instalasi dan Setup Environment

Berikut adalah langkah-langkah menyiapkan Flutter di komputer pengembangan. Ikuti berurutan dan jangan lanjut ke langkah berikutnya sebelum langkah sebelumnya berhasil.

Langkah 1: Mengunduh Flutter SDK

Unduh paket SDK resmi dari docs.flutter.dev sesuai sistem operasi Anda, lalu ekstrak ke direktori permanen (contoh: `~/development/flutter` di Mac/Linux atau `C:\src\flutter` di Windows).

> ⚠️ **Hindari folder bermasalah**
>
> Jangan ekstrak Flutter ke folder yang butuh hak admin (misal `C:\Program Files\`) atau yang namanya mengandung spasi. Hindari juga folder yang disinkronkan OneDrive/iCloud karena bisa merusak file cache SDK.

Langkah 2: Menambahkan Path ke Environment Variable

macOS / Linux (zsh):

```
# Contoh untuk Mac / Linux (zsh) pada ~/.zshrc:
export PATH="$PATH:$HOME/development/flutter/bin"

# Reload shell configuration
source ~/.zshrc

# Cek: harus menampilkan lokasi folder flutter/bin
which flutter
```

Windows: buka *Start* > ketik "environment variables" > *Edit environment variables for your account* > pilih `Path` > *Edit* > *New* > isi `C:\src\flutter\bin`. Tutup lalu buka ulang terminal, kemudian cek di PowerShell:

```
where.exe flutter
flutter --version
```

Langkah 3: Memasang Android Studio dan Android SDK

Pasang Android Studio, buka sekali agar wizard mengunduh Android SDK. Lalu buka *Settings* > *Languages & Frameworks* > *Android SDK* > tab *SDK Tools* dan pastikan tercentang:

- Android SDK Build-Tools
- Android SDK Command-line Tools (latest)
- Android SDK Platform-Tools
- Android Emulator

Buat emulator lewat *Device Manager* > *Create Virtual Device* (pilih Pixel dengan image Android terbaru).

Langkah 4: Menjalankan flutter doctor

Buka terminal lalu ketik perintah diagnostik utama:

```
flutter doctor

# Versi detail (lokasi SDK, versi Java, dll). Pakai saat mencari penyebab error.
flutter doctor -v
```

Perintah ini akan memindai sistem Anda dan memberi centang hijau jika dependensi sudah terpasang:

```
[✓] Flutter (Channel stable, 3.x.x, on macOS / Windows)
[✓] Android toolchain - develop for Android devices (Android SDK version 3x.x.x)
[✓] Xcode - develop for iOS and macOS (Xcode 16.x)  <-- Opsional untuk pengguna Mac
[✓] Chrome - develop for the web
[✓] Android Studio (version 202x.x)
[✓] VS Code (version 1.9x.x)
[✓] Connected device (2 available)
```

Tanda `[!]` berarti ada peringatan dan `[✗]` berarti komponen belum terpasang. Solusi untuk error yang paling sering muncul dibahas di section berikutnya.

> 💡 **Tips Mahasiswa: Ekstensi VS Code Wajib**
>
> Jika menggunakan Visual Studio Code, buka menu Extensions (`Ctrl+Shift+X` atau `Cmd+Shift+X`) dan pasang:
>
> 1. **Flutter** (oleh Dart-Code) — otomatis memasang dependensi Dart.
> 2. **Awesome Flutter Snippets** — shortcut pengetikan widget cepat (misal ketik `stless` untuk StatelessWidget).
> 3. **Error Lens** — menampilkan error kompilasi langsung di baris kode tanpa perlu membuka tab terminal.

Langkah 5: Menyiapkan HP Android fisik (opsional, disarankan untuk RAM 8 GB)

1. Buka *Setelan* > *Tentang ponsel*, ketuk *Nomor versi / Build number* 7 kali sampai muncul "Anda sekarang developer".
2. Buka *Setelan* > *Opsi pengembang*, aktifkan **USB debugging**.
3. Hubungkan kabel USB (kabel data, bukan kabel charger saja), lalu izinkan dialog "Allow USB debugging" di HP.

4. Cek dari terminal:

```
flutter devices
```

```
Found 3 connected devices:
  SM A155F (mobile) • R58W1234ABC • android-arm64  • Android 14 (API 34)
  macOS (desktop)   • macos       • darwin-arm64   • macOS 15
  Chrome (web)      • chrome      • web-javascript • Google Chrome
```

4. Error Setup yang Sering Muncul & Solusinya

Hampir semua mahasiswa menemui setidaknya satu error saat setup. Jangan panik: baca pesan `flutter doctor`, cari pesan yang sama di tabel ini, lalu jalankan perbaikannya. Setelah memperbaiki, selalu jalankan ulang `flutter doctor`.

1. Android license status unknown

```
[!] Android toolchain - develop for Android devices
    ! Some Android licenses not accepted. To resolve this, run: flutter doctor --android-licenses
```

Penyebab: lisensi Android SDK belum disetujui. **Perbaikan:**

```
flutter doctor --android-licenses
# Ketik y lalu Enter untuk setiap lisensi yang ditanyakan
```

2. cmdline-tools component is missing

```
[✗] Android toolchain - develop for Android devices
    ✗ cmdline-tools component is missing
```

Penyebab: Android SDK Command-line Tools belum dipasang (dan perintah lisensi di atas juga akan gagal karenanya). **Perbaikan:** buka Android Studio > *Settings* > *Languages & Frameworks* > *Android SDK* > tab *SDK Tools* > centang **Android SDK Command-line Tools (latest)** > *Apply*. Jika Flutter tidak menemukan lokasi SDK, beri tahu secara manual:

```
# macOS (lokasi default)
flutter config --android-sdk "$HOME/Library/Android/sdk"

# Windows (lokasi default, ganti NamaUser)
flutter config --android-sdk "C:\Users\NamaUser\AppData\Local\Android\Sdk"
```

3. JAVA_HOME / versi Java tidak cocok

```
ERROR: JAVA_HOME is set to an invalid directory
# atau saat build:
Unsupported class file major version 65
```

Penyebab: Gradle memakai Java yang salah, biasanya JDK lama yang terpasang terpisah. Cara paling aman adalah memakai JDK bawaan Android Studio (folder `jbr`):

```
# Arahkan Flutter ke JDK bawaan Android Studio (macOS)
flutter config --jdk-dir "/Applications/Android Studio.app/Contents/jbr/Contents/Home"

# Atau set JAVA_HOME di ~/.zshrc (macOS), lalu: source ~/.zshrc
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"

# Windows (PowerShell), lalu tutup dan buka ulang terminal
```

```
setx JAVA_HOME "C:\Program Files\Android\Android Studio\jbr"

# Cek hasilnya
flutter doctor -v
```

4. CocoaPods not installed (khusus macOS untuk iOS)

```
[!] Xcode - develop for iOS and macOS
    ✗ CocoaPods not installed.
```

Penyebab: plugin iOS dikelola oleh CocoaPods. **Perbaikan** (pilih salah satu):

```
# Lewat Homebrew (disarankan di Apple Silicon)
brew install cocoapods

# Atau lewat RubyGems
sudo gem install cocoapods

# Jika build iOS gagal setelah menambah package
cd ios
pod repo update
pod install
cd ..
```

Sekalian pastikan Xcode sudah dikonfigurasi:

```
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch
```

5. Error lain yang sering muncul

Gejala	Penyebab	Perbaikan
`'flutter' is not recognized` / `command not found: flutter`	PATH belum berisi `flutter/bin`	Ulangi Langkah 2, lalu **buka terminal baru**
`No devices found`	Emulator belum jalan atau USB debugging belum diizinkan	`flutter emulators --launch <id>` atau cabut-colok kabel dan izinkan dialog di HP
Build pertama sangat lama / `Timeout waiting to lock`	Gradle sedang mengunduh dependensi pertama kali	Tunggu dengan internet stabil. Jika macet, jalankan `flutter clean` lalu `flutter run` lagi
`Waiting for another flutter command to release the startup lock`	Ada proses flutter lain yang masih jalan	Tutup terminal/IDE lain. Jika tetap macet, hapus file `flutter/bin/cache/lockfile`
Emulator hitam / sangat lambat	Akselerasi hardware mati atau RAM kurang	Aktifkan virtualisasi (VT-x/AMD-V) di BIOS, atau gunakan HP fisik

> 💡 **Cara bertanya yang baik saat error**
>
> Saat bertanya ke asisten praktikum atau forum, sertakan: (1) hasil lengkap `flutter doctor -v`, (2) perintah yang Anda jalankan, (3) potongan pesan error **paling atas** (bukan hanya baris terakhir). Pesan error yang lengkap biasanya sudah menyebutkan solusinya.

5. Perintah CLI Flutter yang Sering Dipakai

VS Code dan Android Studio punya tombol untuk semua hal, tetapi memahami perintah CLI (terminal) membuat Anda tahu apa yang sebenarnya terjadi dan sangat membantu saat debugging. Berikut perintah yang akan dipakai sepanjang semester.

1. Membuat proyek: `flutter create`

```
# Bentuk paling sederhana (nama proyek: huruf kecil + garis bawah)
flutter create halo_kampus

# Menentukan organisasi (dipakai sebagai package name Android: id.ac.unsoed.halo_kampus)
flutter create --org id.ac.unsoed halo_kampus

# Hanya membuat folder platform tertentu agar proyek lebih ringan
flutter create --platforms=android,ios,web halo_kampus

# Proyek kosong tanpa contoh counter dan komentar panjang
flutter create --empty halo_kampus
```

> ⚠️ **Aturan nama proyek**
>
> Nama proyek harus huruf kecil dan garis bawah (`snake_case`), tidak boleh diawali angka dan tidak boleh memakai tanda minus. `halo-kampus` dan `HaloKampus` akan ditolak, gunakan `halo_kampus` .

2. Melihat dan menyalakan device

```
# Daftar device yang tersambung (HP, emulator yang sudah menyala, Chrome, desktop)
flutter devices

# Daftar emulator yang sudah dibuat di Android Studio
flutter emulators

# Menyalakan emulator berdasarkan id dari perintah di atas
flutter emulators --launch Pixel_8_API_35
```

3. Menjalankan aplikasi: `flutter run`

```
# Jalankan di device default (jika hanya ada satu)
flutter run

# Pilih device tertentu dengan -d (id atau nama dari 'flutter devices')
flutter run -d chrome
flutter run -d emulator-5554
```

```
# Mode release: lebih cepat, tanpa Hot Reload (untuk uji performa)
flutter run --release
```

Selama `flutter run` berjalan, terminal menerima tombol berikut:

Tombol	Fungsi
r	Hot Reload: suntikkan perubahan kode, state dipertahankan
R	Hot Restart: jalankan ulang aplikasi dari `main()`, state direset
h	Tampilkan semua perintah yang tersedia
d	Detach: lepas terminal tanpa menutup aplikasi
c	Bersihkan layar terminal
q	Quit: hentikan aplikasi

4. Mengelola package: `flutter pub`

```
# Menambah package dari pub.dev (otomatis menulis ke pubspec.yaml + mengunduh)
flutter pub add http
flutter pub add provider

# Menambah package khusus pengembangan (masuk ke dev_dependencies)
flutter pub add dev:mocktail

# Menghapus package
flutter pub remove http

# Mengunduh ulang semua package sesuai pubspec.yaml (misal setelah git clone)
flutter pub get

# Melihat package yang punya versi lebih baru
flutter pub outdated
```

5. Memeriksa dan merapikan kode

```
# Cari error dan peringatan tanpa menjalankan aplikasi
flutter analyze

# Rapikan format (indentasi, spasi) seluruh file Dart di folder lib
dart format lib

# Jalankan semua test di folder test/
flutter test
```

6. Membersihkan dan memperbarui

```
# Hapus folder build/ dan cache proyek. Obat pertama untuk error build yang aneh.
flutter clean
flutter pub get

# Perbarui Flutter SDK ke versi stable terbaru
flutter upgrade

# Membuat berkas rilis (dibahas di Modul 16)
flutter build apk --release
```

Ringkasan perintah

Perintah	Kapan dipakai
`flutter doctor`	Cek kesehatan instalasi
`flutter create`	Membuat proyek baru
`flutter devices` / `flutter emulators`	Melihat dan menyalakan device
`flutter run`	Menjalankan aplikasi (debug)
`flutter pub add` / `remove` / `get`	Mengelola package
`flutter analyze`	Cek error sebelum menjalankan / mengumpulkan
`flutter clean`	Saat build error tanpa alasan jelas
`flutter build`	Membuat berkas rilis

6. Anatomi Struktur Folder Proyek Flutter

Untuk membuat proyek baru, jalankan perintah CLI:

```
flutter create halo_kampus
cd halo_kampus
```

Struktur folder yang dihasilkan memiliki fungsi spesifik:

```
halo_kampus/
├── android/          <-- Konfigurasi proyek native Android (Gradle, Manifest, build.gradle)
├── ios/              <-- Konfigurasi proyek native iOS (Podfile, Info.plist, Xcode project)
├── lib/              <-- KODE UTAMA DART KITA (95% waktu coding di sini!)
|   └── main.dart     <-- Titik masuk utama aplikasi (entry point)
├── test/             <-- Berkas unit test dan widget test otomatis
├── web/ & linux/ & windows/ & macos/ <-- Konfigurasi platform desktop & browser
├── pubspec.yaml      <-- MANIFEST PROYEK: Dependensi package, asset gambar, font & versi
├── pubspec.lock      <-- Berkas pengunci versi dependencies riil
└── README.md         <-- Dokumentasi proyek
```

Memahami pubspec.yaml

File `pubspec.yaml` adalah file konfigurasi sentral dalam Flutter (mirip `package.json` di Node.js atau `composer.json` di PHP Laravel):

```yaml
name: halo_kampus
description: "Aplikasi praktikum pertama mahasiswa"
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ^3.5.0                # Biarkan sesuai hasil 'flutter create' di laptop Anda

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8     # Ikon iOS style
  # Package eksternal ditambahkan lewat perintah 'flutter pub add <nama>', contoh:
  # flutter pub add http

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0       # Aturan coding standard resmi (versi mengikuti flutter create)

flutter:
  uses-material-design: true
  # Hapus tanda '#' di dua baris berikut HANYA setelah folder assets/images/ benar-benar dibuat.
  # Jika foldernya belum ada, build akan gagal: "unable to find directory entry in pubspec.yaml".
  # assets:
  #   - assets/images/
```

⚠️ **Hati-Hati dengan Indentasi YAML!**

File `.yaml` **sangat sensitif terhadap spasi**. Gunakan 2 spasi (bukan tab). Salah 1 spasi saja dapat menyebabkan `flutter pub get` gagal membaca file konfigurasi.

▶️ **Cara Menjalankan Contoh Kode di Modul Ini**

- Contoh berlabel `lib/main.dart` adalah **program lengkap**. Hapus seluruh isi `lib/main.dart` bawaan, tempel kode contoh, lalu jalankan `flutter run`.
- Jika aplikasi sudah berjalan lalu Anda mengganti seluruh isi `main.dart`, tekan `R` **(Hot Restart)**, bukan `r` (Hot Reload). Hot Reload tidak menjalankan ulang `main()` dan `initState()`, sehingga tampilan bisa tetap lama atau kosong.
- Contoh berlabel file lain (misal `lib/providers/krs_provider.dart`) dibuat sebagai file baru di folder yang sama persis, karena di-`import` oleh `main.dart`.
- Contoh Dart murni (Modul 02) tidak butuh Flutter: jalankan di dartpad.dev.
- Setelah mengganti `main.dart`, hapus `test/widget_test.dart` bawaan (berisi tes aplikasi counter) agar `flutter test` tidak gagal.

7. Bedah main.dart Counter Bawaan Baris per Baris

Setiap proyek baru dari `flutter create` berisi aplikasi **counter**: layar dengan angka dan tombol +. Aplikasi kecil ini sudah memuat hampir semua konsep dasar Flutter, jadi kita bedah bagian per bagian. Komentar bawaan sudah dihapus dan diganti nomor penanda.

Sebelum itu: aplikasi Flutter paling kecil

Agar terlihat bahwa "semua adalah widget", ini program Flutter terkecil yang bisa dijalankan. Tidak ada AppBar, tema, atau tombol, hanya satu teks di tengah layar:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Halo, Flutter!',
        textDirection: TextDirection.ltr,
        style: TextStyle(fontSize: 32),
      ),
    ),
  );
}
```

Aplikasi paling kecil: hanya satu Text tanpa MaterialApp

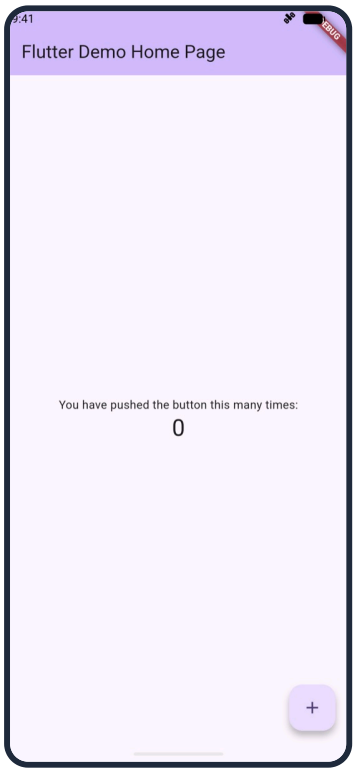
`runApp()` menerima satu widget dan menjadikannya akar (root) seluruh tampilan. Karena tidak memakai `MaterialApp`, kita harus menyebut arah teks (`textDirection`) sendiri. Inilah alasan hampir semua aplikasi dibungkus `MaterialApp`: ia menyediakan arah teks, tema, navigasi, dan banyak hal lain secara otomatis.

Kode counter bawaan

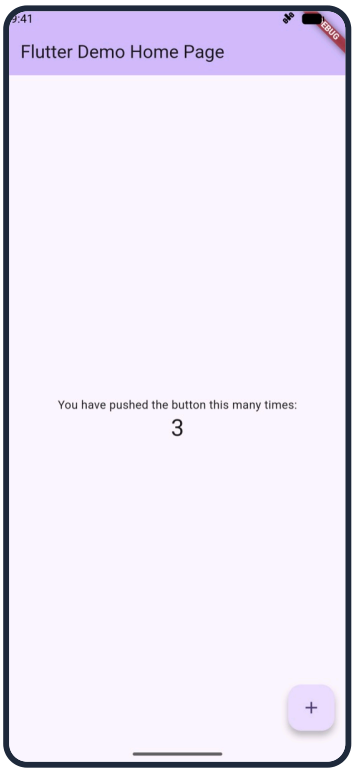
lib/main.dart

```dart
// 1
import 'package:flutter/material.dart';

// 2
void main() {
  runApp(const MyApp());
}

// 3
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // 4
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

// 5
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

// 6
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  // 7
  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  // 8
  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('You have pushed the button this many times:'),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Tampilan awal aplikasi counter



Setelah tombol + ditekan tiga kali

> ℹ️ **Kode Anda sedikit berbeda?**
>
> Versi Flutter terbaru memakai fitur Dart *dot shorthand*, sehingga `flutter create` menulis `.fromSeed(...)` dan `.center` alih-alih `ColorScheme.fromSeed(...)` dan `MainAxisAlignment.center`. Artinya sama persis. Di materi ini kita memakai bentuk lengkap agar asal setiap nilai terlihat jelas.

Penjelasan baris per baris

No.	Kode	Penjelasan
1	`import 'package:flutter/material.dart';`	Mengimpor pustaka Material Design: semua widget seperti `Scaffold`, `AppBar`, `Text`, dan `Icons` berasal dari sini.
2	`void main()` dan `runApp(const MyApp())`	Titik masuk program. `runApp` memasang widget `MyApp` sebagai akar tampilan. `const` berarti objek dibuat sekali saat kompilasi sehingga lebih hemat.
3	`class MyApp extends StatelessWidget`	Widget akar yang tidak punya data berubah. `const MyApp({super.key})` adalah constructor yang meneruskan `key` ke kelas induk.
4	`MaterialApp(...)`	Pembungkus aplikasi: `title` (nama di task switcher), `theme` (warna dari satu `seedColor` menjadi palet Material 3), dan `home` (halaman pertama).
5	`class MyHomePage extends StatefulWidget`	Halaman yang datanya berubah (angka counter). Field `title` bersifat `final`, diisi lewat parameter `required this.title`. `createState()` membuat objek State pasangannya.
6	`class _MyHomePageState extends State<MyHomePage>`	Tempat data yang bisa berubah disimpan: `int _counter = 0`. Garis bawah (`_`) artinya privat, hanya bisa diakses dari file ini.
7	`setState(() { _counter++; })`	Mengubah data **sekaligus** memberi tahu Flutter agar memanggil `build()` lagi. Tanpa `setState`, angka berubah di memori tetapi layar tidak ikut berubah.
8	`Widget build(BuildContext context)`	Menggambarkan tampilan: `Scaffold` (kerangka halaman) berisi `AppBar`, `body` berupa `Center` > `Column` > dua `Text`, dan `FloatingActionButton` yang memanggil `_incrementCounter` saat ditekan. `widget.title` dipakai untuk membaca field dari kelas `MyHomePage`.

Pohon widget (widget tree) aplikasi counter

```
MyApp
└── MaterialApp
    └── MyHomePage
        └── Scaffold
            ├── AppBar
            |   └── Text (judul)
            ├── Center
            |   └── Column
            |       ├── Text ('You have pushed ...')
            |       └── Text ('$_counter')
            └── FloatingActionButton
                └── Icon (Icons.add)
```

Alur ketika tombol + ditekan

1. Pengguna mengetuk `FloatingActionButton`, sehingga `onPressed` memanggil `_incrementCounter()`.
2. `setState` menjalankan `_counter++` lalu menandai widget sebagai "kotor" (perlu digambar ulang).
3. Pada frame berikutnya Flutter memanggil `build()` lagi, dan `Text('$_counter')` kini berisi angka baru.

> ⚠️ **Kesalahan umum**
>
> - Mengubah `_counter++` di luar `setState`: angka tidak muncul di layar.
> - Menaruh variabel `_counter` di dalam `build()`: nilainya kembali ke 0 setiap kali layar digambar ulang.
> - Menulis `title` alih-alih `widget.title` di dalam kelas State: error `Undefined name 'title'`.

8. Hot Reload vs Hot Restart vs Full Restart

Flutter punya tiga cara menerapkan perubahan kode ke aplikasi yang sedang berjalan. Memilih yang tepat menghemat banyak waktu, dan memilih yang salah membuat Anda mengira kode Anda tidak berfungsi.

Aspek	Hot Reload	Hot Restart	Full Restart (stop + run)
Cara	`r` di terminal, atau simpan file (`Ctrl/Cmd+S`) di VS Code	`R` di terminal, atau ikon panah melingkar hijau di VS Code	`q` lalu `flutter run` lagi, atau Stop lalu Run di IDE
Waktu	Kurang dari 1 detik	Beberapa detik	Puluhan detik sampai menit (kompilasi ulang native)
State (data di layar)	Dipertahankan	Direset	Direset
Menjalankan ulang `main()` dan `initState()`	Tidak	Ya	Ya
Kapan dipakai	Mengubah isi `build()`: teks, warna, padding, widget baru	Mengubah `main()`, `initState()`, nilai awal field, variabel global, enum/class	Menambah package berkode native, mengubah file di `android/` atau `ios/`, menambah aset baru

Lab mini: membuktikan perbedaannya

Jalankan kode berikut dengan `flutter run`, lalu tekan tombol di layar sampai angkanya 5. Huruf (A) sampai (E) adalah titik yang akan kita ubah.

`lib/main.dart`

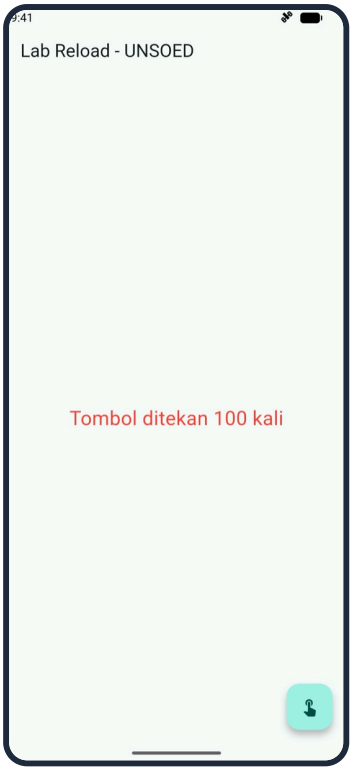
```dart
import 'package:flutter/material.dart';

// Variabel global: nilai awalnya hanya dihitung SEKALI saat pertama dipakai.
final String namaKampus = 'UNSOED';

void main() {
  runApp(const LabReloadApp());
}

class LabReloadApp extends StatelessWidget {
  const LabReloadApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo), // (A)
      ),
      home: const HalamanLab(),
    );
  }
}

class HalamanLab extends StatefulWidget {
  const HalamanLab({super.key});

  @override
  State<HalamanLab> createState() => _HalamanLabState();
}

class _HalamanLabState extends State<HalamanLab> {
  int _klik = 0; // (B)

  @override
  void initState() {
    super.initState();
    debugPrint('initState dipanggil'); // (C)
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Lab Reload - $namaKampus')), // (D)
      body: Center(
        child: Text(
          'Tombol ditekan $_klik kali',
          style: const TextStyle(fontSize: 24, color: Colors.black87), // (E)
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _klik++),
        child: const Icon(Icons.touch_app),
      ),
    );
  }
}
```

Kondisi awal percobaan: tombol sudah ditekan 5 kali

Lakukan percobaan berikut satu per satu dan amati hasilnya:

Percobaan	Ubah menjadi	Hot Reload (r)	Hot Restart (R)
(E) warna teks	`Colors.red`	Teks langsung merah, angka tetap 5	Teks merah, angka kembali 0
(A) warna tema	`Colors.teal`	Tombol dan latar halaman berubah nuansa teal, angka tetap 5	Berubah, angka kembali 0
(B) nilai awal	`int _klik = 100;`	**Tidak berubah**, angka tetap 5 karena objek State lama dipertahankan	Angka mulai dari 100
(C) isi `initState`	`debugPrint('initState versi 2');`	Tidak tercetak, `initState` tidak dipanggil lagi	Tercetak di terminal
Variabel global	`final String namaKampus = 'Universitas Jenderal Soedirman';`	Judul AppBar **tetap** "UNSOED"	Judul berubah



Setelah mengubah (A) teal, (B) 100, dan (E) merah lalu menekan Hot Restart: semua perubahan tampil dan angka mulai dari 100

Percobaan (D) menunjukkan sisi sebaliknya: judul AppBar memakai `namaKampus`, tetapi variabel global hanya dihitung sekali. Hot Reload mengganti kode `build()`, bukan nilai yang sudah tersimpan di memori.

Kapan wajib Full Restart?

```
# Contoh: menambah package yang punya kode native (Android/iOS)
flutter pub add shared_preferences

# Hot Reload / Hot Restart TIDAK cukup. Hentikan aplikasi (tekan q), lalu:
flutter run
```

Jika Anda hanya menekan R setelah menambah package native, biasanya muncul error `MissingPluginException`. Ini bukan bug kode Anda; aplikasinya hanya perlu dikompilasi ulang.

> 💡 **Aturan praktis**
>
> Ubah tampilan: `r`. Ubah data awal atau ganti seluruh `main.dart`: R. Ubah `pubspec.yaml` (package native, aset) atau folder `android/` / `ios/`: stop lalu `flutter run`. Jika ragu dan hasil tampak aneh, naikkan satu tingkat.

9. Aplikasi Pertama: Halo Kampus

Saatnya membuat aplikasi pertama Anda sendiri: **Halo Kampus**, halaman profil mahasiswa dengan tombol presensi. Aplikasi ini memakai konsep yang baru saja dibahas (StatelessWidget, StatefulWidget, `setState`, tema Material 3) ditambah beberapa widget yang akan dipelajari lebih dalam di Modul 03 dan 04.

Langkah 1: Buat proyek

```
flutter create --org id.ac.unsoed halo_kampus
cd halo_kampus
code .            # buka di VS Code (opsional)
```

Langkah 2: Ganti isi `lib/main.dart`

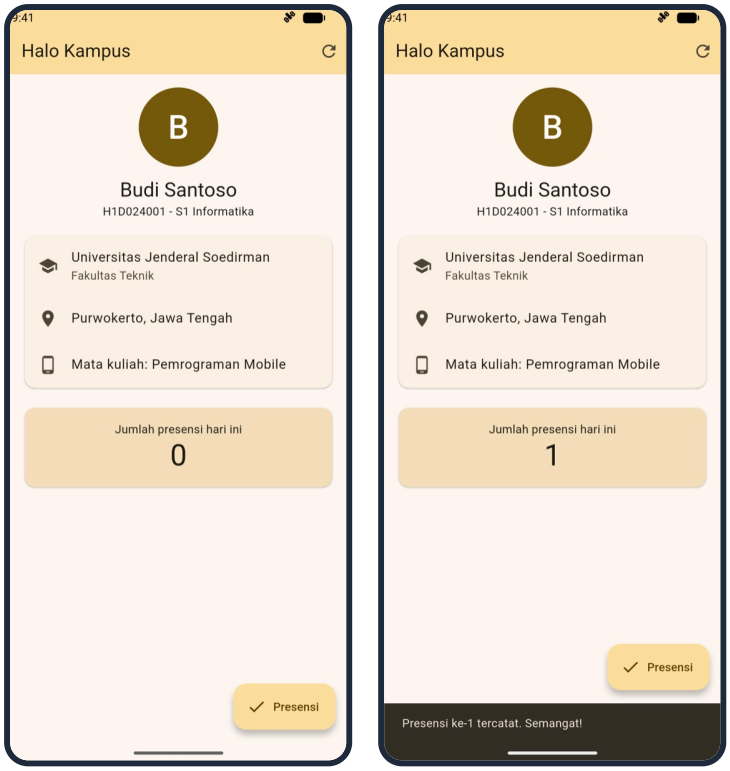
Hapus seluruh isi `lib/main.dart`, lalu ketik (atau tempel) kode berikut:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const HaloKampusApp());
}

/// Akar aplikasi: mengatur tema dan halaman pertama.
class HaloKampusApp extends StatelessWidget {
  const HaloKampusApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Halo Kampus',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFFF2B705)),
      ),
      home: const ProfilPage(),
    );
  }
}

/// Halaman profil mahasiswa. Stateful karena ada data yang berubah (jumlah hadir).
class ProfilPage extends StatefulWidget {
  const ProfilPage({super.key});

  @override
  State<ProfilPage> createState() => _ProfilPageState();
}

class _ProfilPageState extends State<ProfilPage> {
  // Data mahasiswa: ganti dengan identitas Anda sendiri.
  final String _nama = 'Budi Santoso';
  final String _nim = 'H1D024001';
```

```dart
  final String _prodi = 'S1 Informatika';

  int _jumlahHadir = 0;

  void _presensi() {
    setState(() {
      _jumlahHadir++;
    });
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Presensi ke-$_jumlahHadir tercatat. Semangat!')),
    );
  }

  void _reset() {
    setState(() {
      _jumlahHadir = 0;
    });
  }

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Halo Kampus'),
        backgroundColor: warna.primaryContainer,
        actions: [
          IconButton(
            tooltip: 'Reset presensi',
            onPressed: _reset,
            icon: const Icon(Icons.refresh),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // Bagian 1: foto dan nama
          CircleAvatar(
            radius: 48,
            backgroundColor: warna.primary,
            child: Text(
              _nama[0],
              style: TextStyle(fontSize: 40, color: warna.onPrimary),
            ),
          ),
          const SizedBox(height: 12),
          Text(
            _nama,
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.headlineSmall,
          ),
          Text(
            '$_nim - $_prodi',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.bodyMedium,
          ),
          const SizedBox(height: 16),
```

```
          // Bagian 2: kartu informasi
          Card(
            child: Column(
              children: const [
                ListTile(
                  leading: Icon(Icons.school),
                  title: Text('Universitas Jenderal Soedirman'),
                  subtitle: Text('Fakultas Teknik'),
                ),
                ListTile(
                  leading: Icon(Icons.location_on),
                  title: Text('Purwokerto, Jawa Tengah'),
                ),
                ListTile(
                  leading: Icon(Icons.phone_android),
                  title: Text('Mata kuliah: Pemrograman Mobile'),
                ),
              ],
            ),
          ),
          const SizedBox(height: 16),

          // Bagian 3: penghitung presensi
          Card(
            color: warna.secondaryContainer,
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                children: [
                  const Text('Jumlah presensi hari ini'),
                  Text(
                    '$_jumlahHadir',
                    style: Theme.of(context).textTheme.displaySmall,
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _presensi,
        icon: const Icon(Icons.check),
        label: const Text('Presensi'),
      ),
    );
  }
}
```



Tampilan awal Halo Kampus

Setelah tombol Presensi ditekan: angka naik menjadi 1 dan SnackBar muncul di bawah

Langkah 3: Hapus test bawaan dan jalankan

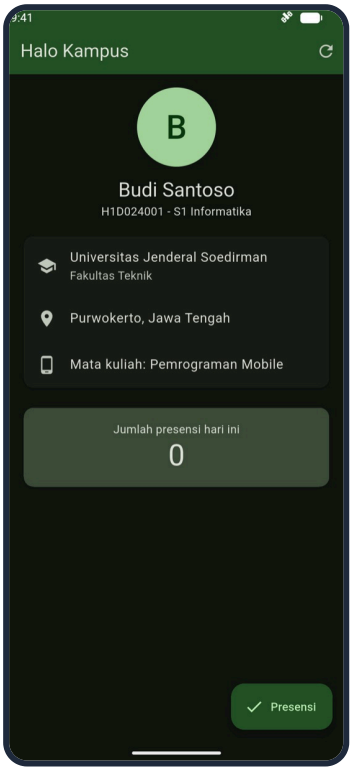
```
# test/widget_test.dart menguji aplikasi counter yang sudah tidak ada
rm test/widget_test.dart        # Windows PowerShell: Remove-Item test\widget_test.dart

flutter analyze                 # pastikan: No issues found!
flutter run
```

Penjelasan bagian penting

- `HaloKampusApp` hanya mengatur tema dan halaman awal, sehingga cukup Stateless. `seedColor: Color(0xFFF2B705)` adalah kuning keemasan; Material 3 otomatis membuat palet lengkap dari satu warna ini. Format `0xFF` + kode hex RGB: `FF` berarti tidak transparan.
- `_ProfilPageState` menyimpan data mahasiswa sebagai `final` (tidak berubah) dan `_jumlahHadir` sebagai data yang berubah.
- `_presensi()` menaikkan angka dengan `setState`, lalu menampilkan `SnackBar` (pesan singkat di bawah layar) lewat `ScaffoldMessenger`.
- `Theme.of(context).colorScheme` mengambil warna dari tema, sehingga mengganti `seedColor` otomatis mengubah warna AppBar, avatar, dan kartu.
- `ListView` dipakai sebagai body agar halaman bisa di-scroll di layar kecil, tidak muncul error overflow garis kuning-hitam.
- `_nama[0]` mengambil huruf pertama nama untuk ditampilkan di avatar.

Langkah 4: Eksperimen

1. Ganti `_nama`, `_nim`, dan `_prodi` dengan data Anda, lalu tekan R (field ini adalah nilai awal State, ingat percobaan di section sebelumnya).
2. Ganti `seedColor` menjadi `Colors.green` lalu tekan `r`. Tekan Presensi beberapa kali dulu agar terlihat bahwa angkanya tidak hilang.
3. Tambahkan satu `ListTile` baru berisi hobi Anda, misalnya dengan ikon `Icons.sports_soccer`.
4. Ubah tema menjadi gelap dengan menambahkan `brightness: Brightness.dark` di dalam `ColorScheme.fromSeed(...)`.



Hasil eksperimen 2 dan 4: seedColor Colors.green dengan brightness: Brightness.dark

⚠️ **Kesalahan umum**

- **Error `Undefined name '_nama'`**: field ditulis di kelas `ProfilPage`, bukan di `_ProfilPageState`.
- **Menambah `ListTile` tidak muncul / error const**: daftar `children: const [...]` hanya boleh berisi widget const. Pastikan widget baru juga const (semua argumennya nilai tetap).
- **Kurung tutup kurang**: aktifkan *Format on Save* di VS Code dan perhatikan koma di akhir argumen (*trailing comma*) agar formatter menyusun kode bertingkat dengan rapi.

Tugas Praktikum: Praktikum 1: Membuat & Menjalankan Proyek 'Halo Kampus'

Sasaran Praktikum

Membuktikan kesiapan environment, memahami alur kerja CLI dan Hot Reload/Hot Restart, serta memodifikasi aplikasi perdana di emulator atau smartphone.

Langkah Pengerjaan

1. Jalankan `flutter doctor -v`. Perbaiki semua tanda `[✗]` pada Flutter, Android toolchain, dan Connected device (lihat section 4). Simpan hasil akhirnya sebagai `doctor.txt`.
2. Buat proyek baru dengan perintah: `flutter create --org id.ac.unsoed praktikum1_kampus`, lalu buka folder proyek di VS Code.
3. Cek device dengan `flutter devices`. Jika memakai emulator, nyalakan lewat `flutter emulators --launch <id>`.
4. Jalankan aplikasi counter bawaan dengan `flutter run`. Di file `CATATAN.md`, tulis penjelasan singkat dengan kata-kata sendiri untuk bagian bernomor 1 sampai 8 dari section 7.
5. Ganti seluruh isi `lib/main.dart` dengan kode Halo Kampus (section 9), hapus `test/widget_test.dart`, lalu ubah nama, NIM, dan prodi menjadi milik Anda. Ubah judul AppBar menjadi `'Portal Mahasiswa – [NIM Anda]'`.
6. Tambahkan minimal dua `ListTile` baru (misal hobi dan angkatan) dan ganti `seedColor` dengan warna pilihan Anda.
7. Lakukan percobaan Hot Reload vs Hot Restart: tekan Presensi sampai 5, ubah warna (tekan r), lalu ubah nilai awal `_jumlahHadir` menjadi 10 (tekan r, kemudian R). Catat hasil tiap langkah di `CATATAN.md` beserta alasannya.
8. Jalankan `flutter analyze` dan pastikan hasilnya *No issues found!*.
9. Ambil tangkapan layar aplikasi yang berjalan (sebelum dan sesudah percobaan Hot Restart).
10. Salin proyek ke folder `praktikum-01/` di repositori GitHub Praktikum 0, sertakan `doctor.txt`, `CATATAN.md`, dan tangkapan layar, lalu commit dan push.

Output yang Diharapkan

Aplikasi Halo Kampus berjalan dengan identitas mahasiswa, NIM di AppBar, minimal dua ListTile tambahan dan warna tema sendiri; repositori berisi doctor.txt, CATATAN.md (penjelasan main.dart dan hasil percobaan reload), serta tangkapan layar.

Rubrik Penilaian

Aplikasi berjalan tanpa error dan flutter analyze bersih (35%), Modifikasi identitas, ListTile, dan tema (20%), Ketepatan catatan percobaan Hot Reload/Hot Restart (25%), Penjelasan main.dart dengan kata-kata sendiri (10%), Kelengkapan berkas di GitHub (10%).

Kuis Pemahaman

1. Mengapa Flutter memiliki performa render grafis yang mendekati aplikasi native murni?

A. Karena Flutter menggunakan WebView berkecepatan tinggi

B. Karena Flutter memiliki Engine grafis sendiri (Skia/Impeller) yang menggambar langsung ke layar tanpa jembatan (bridge) JavaScript

C. Karena semua kode Flutter diubah menjadi file HTML5 sebelum dijalankan

D. Karena Flutter tidak mendukung animasi kompleks

2. File manakah dalam proyek Flutter yang berfungsi untuk mendaftarkan dependensi pihak ketiga (library dari pub.dev) serta aset gambar?

A. lib/main.dart

B. android/build.gradle

C. pubspec.yaml

D. pubspec.lock

3. Fitur Flutter apakah yang memungkinkan developer melihat perubahan kode di antarmuka dalam waktu kurang dari 1 detik tanpa me-restart aplikasi?

A. Cold Restart

B. Ahead-of-Time Compiling

C. Stateful Hot Reload

D. Garbage Collector

4. Anda mengubah `int _counter = 0;` menjadi `int _counter = 50;` lalu menekan Hot Reload, tetapi angka di layar tidak berubah. Apa yang seharusnya dilakukan?

A. Menjalankan flutter clean

B. Menekan Hot Restart (R) karena nilai awal field State hanya dibaca saat objek State dibuat

C. Menginstal ulang Flutter SDK

D. Menambahkan package baru di pubspec.yaml

5. Apa yang terjadi jika `_counter++` dijalankan di luar `setState()` pada aplikasi counter?

A. Aplikasi langsung crash

B. Nilai variabel bertambah di memori, tetapi layar tidak digambar ulang sehingga angka tidak berubah

C. Flutter otomatis memanggil setState

D. Nilai _counter kembali ke 0

6. flutter doctor menampilkan pesan `cmdline-tools component is missing`. Langkah perbaikan yang tepat adalah...

A. Memasang Android SDK Command-line Tools (latest) melalui SDK Manager di Android Studio

B. Menjalankan flutter upgrade berulang kali

C. Menghapus folder lib/ dari proyek

D. Memasang CocoaPods

7. Perintah manakah yang tepat untuk menambahkan package http ke proyek Flutter?

A. flutter install http

B. npm install http

C. flutter pub add http

D. flutter create http

8. Setelah menambah package yang memiliki kode native (misalnya shared_preferences), muncul MissingPluginException saat Hot Restart. Apa penyebabnya?

A. Kode Dart salah ketik

B. Package tersebut tidak mendukung Android

C. Bagian native aplikasi belum dikompilasi ulang, sehingga perlu stop lalu flutter run lagi (Full Restart)

D. Emulator kehabisan baterai

Kunci Jawaban & Pembahasan

1. B — Flutter tidak menggunakan web view atau jembatan komunikasi JS bridge saat runtime. Engine grafisnya (Skia/Impeller) mengontrol setiap piksel dan berkomunikasi langsung dengan GPU perangkat.

2. C — File pubspec.yaml adalah file manifest utama tempat developer mendeklarasikan library/package eksternal, aset gambar, icon font, dan metadata proyek.

3. C — Stateful Hot Reload menyuntikkan kode baru ke dalam Dart Virtual Machine yang sedang berjalan tanpa menghapus state saat ini, memungkinkan iterasi UI super cepat.

4. B — Hot Reload mempertahankan objek State yang sudah ada, sehingga nilai awal field tidak dihitung ulang. Hot Restart membuat ulang seluruh aplikasi dari main(), sehingga nilai awal baru dipakai.

5. B — setState() memberi tahu framework bahwa ada data yang berubah sehingga build() dipanggil lagi. Tanpa itu, data berubah tetapi tampilan tidak ikut diperbarui.

6. A — cmdline-tools adalah komponen Android SDK yang dipasang lewat Android Studio > Settings > Android SDK > SDK Tools. Setelah terpasang, jalankan flutter doctor --android-licenses.

7. C — flutter pub add menulis package ke pubspec.yaml dengan versi terbaru yang kompatibel lalu langsung mengunduhnya.

8. C — Hot Reload dan Hot Restart hanya memperbarui kode Dart. Plugin native harus ikut dikompilasi ke aplikasi, sehingga aplikasi perlu dihentikan dan dijalankan ulang dengan flutter run.

MODUL 02 · BAHASA DART

Fondasi Bahasa Pemrograman Dart Modern (Dart 3)

Belajar Dart dari nol secara berurutan: program pertama, variabel dan tipe data, operator, control flow, fungsi, koleksi, null safety, OOP, fitur modern Dart 3 (records, patterns, sealed class), error handling, sampai pemrograman asinkron dengan Future dan Stream. Ditutup dengan studi kasus sistem nilai dan IPK mahasiswa.

Durasi Pertemuan 2 (150 Menit) + Belajar Mandiri

Prasyarat Modul 01: Setup & Flutter Basics

Capaian Pembelajaran

- Menulis dan menjalankan program Dart murni dengan dart run maupun DartPad.
- Memakai variabel, tipe data, final/const/late, interpolasi string, dan konversi tipe dengan benar.
- Menggunakan operator (aritmatika, logika, null-aware, cascade, ternary) dan control flow termasuk switch expression Dart 3.
- Menulis fungsi dengan parameter posisi, named, required, dan default, serta memanfaatkan closure dan higher-order function.
- Mengolah List, Set, dan Map dengan map, where, fold, sort, spread, collection-if, dan collection-for.
- Menerapkan Sound Null Safety untuk mencegah error null saat runtime.
- Merancang class dengan berbagai constructor, getter/setter, inheritance, abstract class, interface, mixin, extension, dan enhanced enum.
- Memakai records, pattern matching, destructuring, dan sealed class dengan exhaustive switch.
- Menangani error dengan try/catch/finally, throw, dan exception kustom.
- Menulis kode asinkron dengan Future, async/await, Future.wait, Stream, async*, dan StreamController.

Daftar Isi Modul

1. Program Dart Pertama: main(), print & Komentar
2. Variabel, Tipe Data & Konversi Tipe
3. Operator: Aritmatika, Logika, Null-aware & Cascade
4. Control Flow: Percabangan & Perulangan
5. Fungsi: Parameter, Closure & Higher-Order Function
6. Koleksi: List, Set & Map

7. Sound Null Safety
8. Object-Oriented Programming (OOP)
9. Fitur Modern Dart 3: Records, Patterns & Sealed Class
10. Error Handling: try, catch & Exception Kustom
11. Pemrograman Asinkron: Future, async/await & Stream
12. Studi Kasus: Sistem Nilai & IPK Mahasiswa
13. Tugas Praktikum
14. Kuis Pemahaman & Kunci Jawaban

1. Program Dart Pertama: main(), print & Komentar

Sebelum menulis aplikasi Flutter, kita perlu akrab dulu dengan bahasa yang menggerakkannya: **Dart**. Di modul ini semua contoh adalah program Dart murni (tanpa Flutter), jadi bisa dijalankan langsung di terminal atau di DartPad.

Di mana menjalankan kode Dart?

- **DartPad** (`dartpad.dev`): cukup buka browser, tempel kode, klik Run. Cocok untuk mencoba potongan kode kecil.
- **Terminal**: Dart SDK sudah ikut terpasang bersama Flutter (Modul 01). Simpan kode sebagai file `.dart`, lalu jalankan dengan `dart run`.

```
# Cek versi Dart (harus 3.x)
dart --version

# Cara 1: jalankan satu file
dart run halo.dart

# Cara 2: buat proyek console lengkap
dart create -t console belajar_dart
cd belajar_dart
dart run
```

Program pertama

Setiap program Dart dimulai dari fungsi `main()`. Saat program dijalankan, Dart mencari fungsi ini dan mengeksekusi isinya baris demi baris dari atas ke bawah.

`halo.dart`

```
void main() {
  print('Halo, Dart!');
  print('Selamat datang di Praktikum Pemrograman Mobile');
}

// Output:
// Halo, Dart!
// Selamat datang di Praktikum Pemrograman Mobile
```

- `void` artinya fungsi `main` tidak mengembalikan nilai apa pun.
- `print()` menampilkan teks ke konsol, lalu pindah ke baris baru.
- Setiap pernyataan (*statement*) diakhiri titik koma `;`. Lupa titik koma adalah error paling sering bagi pemula.
- Teks (String) ditulis di antara tanda kutip, boleh kutip satu `'...'` atau kutip dua `"..."`.

> ℹ️ **Tentang komentar `// Output:`**
>
> Di seluruh modul ini, hasil yang muncul di konsol ditulis sebagai komentar `// Output: ...`. Komentar tidak ikut dijalankan, jadi kode tetap bisa langsung disalin dan dijalankan.

Komentar

Komentar adalah catatan untuk manusia dan diabaikan oleh compiler. Dart punya tiga jenis komentar:

```dart
// Komentar satu baris: diabaikan oleh compiler.

/*
  Komentar banyak baris.
  Cocok untuk menonaktifkan sementara beberapa baris kode.
*/

/// Komentar dokumentasi (tiga garis miring).
/// Muncul sebagai tooltip saat kursor diarahkan ke nama fungsi di VS Code.
void sapa(String nama) {
  print('Halo, $nama!');
}

void main() {
  sapa('Rani'); // Output: Halo, Rani!
  // sapa('Budi'); baris ini tidak dijalankan karena dikomentari
}
```

Gunakan `///` di atas class atau fungsi yang penting. Editor akan menampilkannya sebagai dokumentasi, sama seperti dokumentasi widget Flutter yang muncul saat Anda mengarahkan kursor ke `Text` atau `Column`.

Struktur file Dart

Sebuah file Dart biasanya berisi (berurutan): `import`, deklarasi tingkat atas (variabel, fungsi, class), dan fungsi `main()`. Urutan fungsi dan class di dalam file tidak berpengaruh, karena Dart membaca seluruh file terlebih dahulu.

```dart
import 'dart:math'; // 1. import library bawaan (opsional)

const namaKampus = 'UNSOED'; // 2. variabel tingkat atas (top-level)

int kuadrat(int x) {
  // 3. fungsi tingkat atas
  return x * x;
}

void main() {
  // 4. titik masuk program
  print(namaKampus); // Output: UNSOED
```

```
  print(kuadrat(7)); // Output: 49
  print(max(3, 9)); // Output: 9
}
```

Fungsi `max()` tidak kita tulis sendiri, melainkan berasal dari library `dart:math` yang di-`import` di baris pertama. Library bawaan lain yang sering dipakai: `dart:async` (Future dan Stream), `dart:convert` (JSON), dan `dart:io` (file dan terminal, tidak tersedia di web).

Argumen dari terminal

`main()` boleh menerima parameter `List<String>` berisi argumen yang diketik setelah nama file. Di Flutter, bentuk ini jarang dipakai, tetapi berguna untuk program console.

```dart
void main(List<String> args) {
  print('Jumlah argumen: ${args.length}');
  print('Isi argumen: $args');
}

// Output:
// Jumlah argumen: 0
// Isi argumen: []
```

Jika dijalankan dengan `dart run argumen.dart halo dunia`, hasilnya menjadi `Jumlah argumen: 2` dan `Isi argumen: [halo, dunia]`.

> ⚠️ **Kesalahan umum pemula**
>
> - Lupa titik koma `;` di akhir statement.
> - Menulis `Main()` atau `Print()`. Dart membedakan huruf besar dan kecil (*case-sensitive*).
> - Kurung kurawal `{ }` tidak berpasangan. Gunakan fitur *Format Document* (`Shift+Alt+F`) di VS Code agar indentasi rapi dan kesalahan mudah terlihat.

2. Variabel, Tipe Data & Konversi Tipe

Variabel adalah "kotak bernama" untuk menyimpan data. Dart adalah bahasa yang *strongly typed* (setiap nilai punya tipe yang jelas), tetapi punya *type inference* sehingga kita sering tidak perlu menulis tipenya secara manual.

Tipe data dasar

Tipe	Isi	Contoh
`int`	Bilangan bulat	`int sks = 3;`
`double`	Bilangan desimal	`double ipk = 3.75;`
`num`	Bisa `int` atau `double`	`num nilai = 80;`
`String`	Teks	`String nama = 'Ayu';`
`bool`	`true` atau `false`	`bool aktif = true;`
`dynamic`	Tipe bebas, pemeriksaan tipe dimatikan	`dynamic data = 1;`

Deklarasi dengan tipe eksplisit dan `var`

```dart
void main() {
  // Tipe ditulis eksplisit
  String nama = 'Ahmad Fikri';
  int semester = 5;
  double ipk = 3.84;
  bool aktif = true;

  // Tipe ditebak otomatis dari nilai awal (type inference)
  var jurusan = 'Informatika'; // dianggap String
  var angkatan = 2023; // dianggap int

  print(nama); // Output: Ahmad Fikri
  print(semester); // Output: 5
  print(ipk); // Output: 3.84
  print(aktif); // Output: true
  print(jurusan.runtimeType); // Output: String
  print(angkatan.runtimeType); // Output: int

  semester = 6; // variabel biasa boleh diubah nilainya
  print(semester); // Output: 6

  // jurusan = 123; // ERROR: jurusan sudah bertipe String
}
```

`var` **bukan** berarti "tipe bebas". Begitu diisi `'Informatika'`, tipe `jurusan` terkunci menjadi `String`. Properti `runtimeType` dipakai untuk melihat tipe sebuah nilai saat program berjalan.

Angka: `int`, `double`, dan `num`

```
void main() {
  int sks = 3;
  double nilai = 87.5;
  num apaSaja = 10; // num bisa menampung int maupun double
  apaSaja = 10.5;

  print(sks + 1); // Output: 4
  print(nilai / 2); // Output: 43.75
  print(apaSaja); // Output: 10.5

  double rata = 7; // literal int otomatis diubah menjadi double
  print(rata); // Output: 7.0
  print(1e3); // Output: 1000.0
  print(0xFF); // Output: 255
  print(1000000 == 1_000_000); // Output: true
}
```

Garis bawah pada `1_000_000` hanya pemisah agar angka besar mudah dibaca (fitur Dart 3.6 ke atas). Nilainya sama persis dengan `1000000`.

> ℹ️ **Perbedaan output di DartPad**
>
> DartPad menjalankan kode sebagai JavaScript, sehingga `double` yang nilainya bulat dicetak tanpa `.0` (misalnya `7`, bukan `7.0`). Output di modul ini mengikuti `dart run` di terminal.

String: kutip, escape, multi-baris, raw

```
void main() {
  String a = 'kutip satu';
  String b = "kutip dua";
  String c = "Jum'at"; // kutip satu di dalam kutip dua
  String d = 'Jum\'at'; // atau di-escape dengan backslash

  String multi = '''
Baris pertama
Baris kedua''';

  String raw = r'C:\folder\baru'; // raw string: backslash tidak diproses

  print(a); // Output: kutip satu
  print(b); // Output: kutip dua
  print(c); // Output: Jum'at
  print(d); // Output: Jum'at
  print(multi);
```

```
  // Output:
  // Baris pertama
  // Baris kedua
  print(raw); // Output: C:\folder\baru
  print('Baris 1\nBaris 2');
  // Output:
  // Baris 1
  // Baris 2
}
```

`\n` adalah karakter baris baru dan `\t` adalah tab. Awalan `r` membuat *raw string* sehingga backslash dicetak apa adanya, berguna untuk path Windows dan pola `RegExp`.

String interpolation

Untuk menyisipkan nilai ke dalam teks, gunakan `$nama` untuk variabel sederhana dan `${ekspresi}` untuk ekspresi (perhitungan, pemanggilan method, akses properti).

```
void main() {
  String nama = 'Nadia';
  int sks = 21;
  double ipk = 3.756;

  print('Nama: $nama'); // Output: Nama: Nadia
  print('SKS semester depan: ${sks + 3}'); // Output: SKS semester depan: 24
  print('Panjang nama: ${nama.length} huruf'); // Output: Panjang nama: 5 huruf
  print('IPK: ${ipk.toStringAsFixed(2)}'); // Output: IPK: 3.76
  print('Huruf besar: ${nama.toUpperCase()}'); // Output: Huruf besar: NADIA

  // Menyambung dengan + juga bisa, tetapi interpolasi lebih disarankan
  print('Halo, ' + nama + '!'); // Output: Halo, Nadia!
}
```

> ⚠️ **Lupa kurung kurawal**
>
> `'$nama.length'` akan mencetak `Nadia.length`, bukan `5`, karena tanpa `{ }` Dart hanya mengambil nama variabelnya. Tulis `'${nama.length}'`.

Method String yang sering dipakai

```
void main() {
  String nim = '  H1D023001  ';
  String bersih = nim.trim(); // buang spasi di awal dan akhir

  print(bersih); // Output: H1D023001
  print(bersih.length); // Output: 9
  print(bersih.toLowerCase()); // Output: h1d023001
  print(bersih.startsWith('H1D')); // Output: true
  print(bersih.contains('023')); // Output: true
  print(bersih.substring(3, 5)); // Output: 02
```

```
  print(bersih[0]); // Output: H
  print(bersih.replaceAll('0', 'o')); // Output: H1Do23oo1
  print('Flutter,Dart,Supabase'.split(',')); // Output: [Flutter, Dart, Supabase]
  print('7'.padLeft(3, '0')); // Output: 007
  print('ab' * 3); // Output: abababab
  print(''.isEmpty); // Output: true
}
```

`substring(3, 5)` mengambil karakter dari indeks 3 sampai **sebelum** indeks 5. Indeks String selalu dimulai dari 0.

`final`, `const`, dan `late`

```
void main() {
  final String namaKampus = 'UNSOED';
  final waktuMulai = DateTime.now(); // nilainya baru diketahui saat program berjalan
  const double nilaiMaksimal = 4.0;
  const jumlahJamSeminggu = 7 * 24; // ekspresi konstan, dihitung saat kompilasi

  // namaKampus = 'UGM'; // ERROR: final hanya bisa diisi sekali
  // nilaiMaksimal = 5.0; // ERROR: const tidak bisa diubah
  // const sekarang = DateTime.now(); // ERROR: DateTime.now() bukan nilai compile-time

  print(namaKampus); // Output: UNSOED
  print(nilaiMaksimal); // Output: 4.0
  print(jumlahJamSeminggu); // Output: 168
  print(waktuMulai.year > 2000); // Output: true
}
```

Kata kunci	Bisa diubah?	Kapan nilainya ditentukan?	Contoh pemakaian
`var` / tipe	Ya	Kapan saja	Penghitung, input pengguna
`final`	Tidak (sekali isi)	Saat program berjalan (runtime)	Data dari API, waktu login
`const`	Tidak	Saat kompilasi (compile-time)	Angka tetap, widget statis Flutter
`late`	Tergantung	Ditunda, diisi sebelum dipakai	Controller di `initState`

`final` mengunci *variabelnya*, bukan isi objeknya. List yang `final` masih bisa ditambah isinya, sedangkan list `const` benar-benar beku:

```
void main() {
  final daftarFinal = ['Dart', 'Flutter'];
  daftarFinal.add('Supabase'); // boleh: isi list bisa berubah
  print(daftarFinal); // Output: [Dart, Flutter, Supabase]
  // daftarFinal = []; // ERROR: variabelnya tidak bisa diganti

  const daftarConst = ['Dart', 'Flutter'];
  print(daftarConst.length); // Output: 2
```

```
  // daftarConst.add('Hive'); // ERROR saat runtime: Unsupported operation
}
```

`late` menandai variabel yang **pasti** diisi nanti sebelum dibaca. `late final` juga bisa dipakai untuk inisialisasi malas (*lazy*): nilai baru dihitung saat pertama kali dipakai.

```
late String token; // akan diisi oleh fungsi login()

void login() {
  token = 'abc123';
}

int hitungBerat() {
  print('Menghitung...');
  return 42;
}

void main() {
  login();
  print('Token: $token'); // Output: Token: abc123

  late final int hasil = hitungBerat(); // belum dihitung di baris ini
  print('Sebelum memakai hasil');
  print(hasil); // baru dihitung di sini

  // Output:
  // Sebelum memakai hasil
  // Menghitung...
  // 42
}
```

`var` vs `dynamic` vs `Object`

```
void main() {
  var a = 10; // tipe terkunci: int
  dynamic b = 10; // tipe boleh berganti, tanpa pemeriksaan compiler
  Object c = 10; // menampung apa saja, tetapi tetap diperiksa compiler

  b = 'sekarang String';
  c = true;
  // a = 'teks'; // ERROR: a bertipe int

  print(a); // Output: 10
  print(b); // Output: sekarang String
  print(c); // Output: true

  // print(c.length); // ERROR compile: Object tidak punya length
  // print(b.panjangSekali); // lolos compile, tetapi CRASH saat runtime
}
```

Hindari `dynamic` kecuali benar-benar perlu (misalnya data JSON mentah). Dengan `dynamic`, salah ketik nama properti baru ketahuan ketika aplikasi sudah berjalan.

Konversi tipe

```dart
void main() {
  // String -> angka
  int umur = int.parse('20');
  double ipk = double.parse('3.75');
  print(umur + 1); // Output: 21
  print(ipk); // Output: 3.75

  // tryParse mengembalikan null jika gagal (tidak crash)
  int? gagal = int.tryParse('dua puluh');
  print(gagal); // Output: null

  // angka -> String
  String teks = 42.toString();
  print(teks + '!'); // Output: 42!
  print(3.14159.toStringAsFixed(2)); // Output: 3.14

  // double -> int (toInt memotong, round membulatkan)
  print(9.7.toInt()); // Output: 9
  print(9.7.round()); // Output: 10
  print(9.2.ceil()); // Output: 10
  print(9.7.floor()); // Output: 9

  // int -> double
  print(5.toDouble()); // Output: 5.0
}
```

Gunakan `int.tryParse()` untuk input dari pengguna (misalnya dari `TextField` di Modul 05), karena pengguna bisa saja mengetik huruf. `int.parse()` akan melempar `FormatException` jika teksnya bukan angka.

> 💡 **Ringkasan**
>
> - Pakai `final` secara default. Ganti ke `var` hanya jika nilainya memang perlu berubah.
> - Pakai `const` untuk nilai yang sudah pasti sejak kode ditulis.
> - Gunakan interpolasi `'$x'` / `'${ekspresi}'` untuk menyusun teks.
> - Selalu pakai `tryParse` untuk input yang belum tentu valid.

3. Operator: Aritmatika, Logika, Null-aware & Cascade

Operator adalah simbol untuk mengolah nilai: menjumlah, membandingkan, menggabungkan kondisi, sampai menangani nilai `null`. Banyak operator Dart sama dengan bahasa C, Java, atau JavaScript, tetapi ada beberapa yang khas Dart seperti `~/`, `??`, dan cascade `..`.

Operator aritmatika

```dart
void main() {
  int a = 17;
  int b = 5;

  print(a + b); // Output: 22
  print(a - b); // Output: 12
  print(a * b); // Output: 85
  print(a / b); // Output: 3.4
  print(a ~/ b); // Output: 3
  print(a % b); // Output: 2
  print(-a); // Output: -17
}
```

- `/` selalu menghasilkan `double`, walaupun kedua angkanya `int`.
- `~/` adalah pembagian bulat (hasil dibulatkan ke bawah menuju nol).
- `%` adalah sisa bagi (modulo), sering dipakai untuk mengecek genap/ganjil: `n % 2 == 0`.

Operator penugasan dan increment

```dart
void main() {
  int poin = 10;
  poin += 5; // sama dengan: poin = poin + 5
  print(poin); // Output: 15
  poin -= 3;
  print(poin); // Output: 12
  poin *= 2;
  print(poin); // Output: 24
  poin ~/= 5;
  print(poin); // Output: 4

  int x = 1;
  print(x++); // Output: 1
  print(x); // Output: 2
  print(++x); // Output: 3
  x--;
  print(x); // Output: 2
}
```

`x++` (postfix) memberikan nilai *lama* lalu menambah, sedangkan `++x` (prefix) menambah dulu lalu memberikan nilai *baru*. Itulah sebabnya `print(x++)` mencetak 1.

Operator perbandingan dan uji tipe

```dart
void main() {
  int nilai = 75;
  print(nilai == 75); // Output: true
  print(nilai != 80); // Output: true
  print(nilai > 80); // Output: false
  print(nilai <= 75); // Output: true

  Object data = 'Dart';
  print(data is String); // Output: true
  print(data is! int); // Output: true
  String teks = data as String; // cast: "anggap data sebagai String"
  print(teks.length); // Output: 4
}
```

`is` mengecek tipe, `is!` kebalikannya, dan `as` melakukan *cast*. Jika cast salah (misalnya `data as int`), program akan crash, jadi biasakan mengecek dengan `is` terlebih dahulu.

Operator logika

```dart
void main() {
  int kehadiran = 85; // persen
  double nilaiAkhir = 72.5;
  bool sudahBayarUkt = true;

  bool bolehUjian = kehadiran >= 75 && sudahBayarUkt;
  bool dapatBeasiswa = nilaiAkhir >= 85 || kehadiran == 100;

  print(bolehUjian); // Output: true
  print(dapatBeasiswa); // Output: false
  print(!sudahBayarUkt); // Output: false
}
```

Operator	Nama	Hasil `true` jika...
`a && b`	AND	keduanya `true`
`a || b`	OR	salah satu `true`
`!a`	NOT	`a` bernilai `false`

Dart memakai *short-circuit evaluation*: pada `a && b`, jika `a` sudah `false`, maka `b` tidak dievaluasi sama sekali. Ini berguna untuk pengecekan aman seperti `teks != null && teks.isNotEmpty`.

Operator ternary

Bentuk singkat dari if-else yang menghasilkan nilai: `kondisi ? nilaiJikaBenar : nilaiJikaSalah`. Di Flutter, ternary sangat sering dipakai di dalam widget, misalnya `isLoading ? CircularProgressIndicator() : Text('Data')`.

```dart
void main() {
  int nilai = 68;
  String status = nilai >= 60 ? 'Lulus' : 'Tidak lulus';
  print(status); // Output: Lulus

  int jumlahPesan = 1;
  print('$jumlahPesan pesan${jumlahPesan > 1 ? ' baru' : ''}'); // Output: 1 pesan
}
```

Operator null-aware: `??`, `?.`, `??=`

Operator ini menangani nilai yang mungkin `null`. Pembahasan lengkapnya ada di section Null Safety; berikut perkenalan singkatnya.

```dart
String? cariNamaPanggilan(String nama) {
  // Mengembalikan null jika nama tidak dikenal
  return nama == 'Muhammad Fikri' ? 'Fikri' : null;
}

void main() {
  String? panggilan = cariNamaPanggilan('Siti Aminah'); // null

  print(panggilan ?? 'Belum diisi'); // Output: Belum diisi
  print(panggilan?.length); // Output: null

  panggilan ??= 'Siti'; // isi hanya jika masih null
  print(panggilan); // Output: Siti

  String? lain = cariNamaPanggilan('Muhammad Fikri');
  lain ??= 'Default'; // tidak berubah karena sudah ada isinya
  print(lain); // Output: Fikri
}
```

- `a ?? b`: pakai `a`, tetapi jika `a` null pakai `b`.
- `a?.b`: akses `b` hanya jika `a` tidak null; jika null, hasilnya `null` (tidak crash).
- `a ??= b`: isi `a` dengan `b` hanya jika `a` masih null.

Cascade `..`

Cascade memungkinkan beberapa operasi berurutan pada **objek yang sama** tanpa menulis nama variabelnya berulang kali. Contoh ini memakai class sederhana; class dibahas detail di section OOP.

```dart
class Pesanan {
  String menu = '';
  int jumlah = 0;
  List<String> catatan = [];
```

```dart
  void tampilkan() => print('$jumlah x $menu, catatan: $catatan');
}

void main() {
  // Tanpa cascade
  var p1 = Pesanan();
  p1.menu = 'Mendoan';
  p1.jumlah = 5;
  p1.tampilkan(); // Output: 5 x Mendoan, catatan: []

  // Dengan cascade
  var p2 = Pesanan()
    ..menu = 'Soto Sokaraja'
    ..jumlah = 2
    ..catatan.add('pedas')
    ..tampilkan(); // Output: 2 x Soto Sokaraja, catatan: [pedas]
  print(p2.jumlah); // Output: 2

  // Cascade juga praktis untuk List
  final angka = <int>[]
    ..add(3)
    ..add(1)
    ..add(2)
    ..sort();
  print(angka); // Output: [1, 2, 3]
}
```

Perhatikan: `..add(3)` mengembalikan **objek list-nya**, bukan hasil dari `add`. Karena itu rangkaian di atas tetap menghasilkan list. Ada juga `?..` untuk objek yang mungkin null.

> ⚠️ **Kesalahan umum**
>
> - Mengira `7 / 2` menghasilkan `3`. Hasilnya `3.5`. Gunakan `7 ~/ 2` untuk mendapatkan `3`.
> - Menulis `=` (penugasan) padahal maksudnya `==` (perbandingan).
> - Membandingkan `double` hasil perhitungan dengan `==`: `0.1 + 0.2 == 0.3` bernilai `false` karena keterbatasan presisi bilangan desimal.

4. Control Flow: Percabangan & Perulangan

Control flow menentukan *baris mana yang dijalankan* dan *berapa kali*. Ada dua kelompok besar: percabangan (`if` , `switch`) dan perulangan (`for` , `while`).

if, else if, else

```dart
void main() {
  double nilai = 78.5;

  if (nilai >= 85) {
    print('A');
  } else if (nilai >= 70) {
    print('B');
  } else if (nilai >= 55) {
    print('C');
  } else {
    print('D/E');
  }
  // Output: B
}
```

Kondisi diperiksa dari atas ke bawah, dan hanya blok **pertama** yang kondisinya benar yang dijalankan. Nilai 78.5 tidak memenuhi `>= 85` , tetapi memenuhi `>= 70` , sehingga pengecekan berhenti di situ. Kondisi di Dart harus bertipe `bool` ; menulis `if (1)` seperti di C adalah error.

switch statement

```dart
void main() {
  String hari = 'Sabtu';

  switch (hari) {
    case 'Sabtu':
    case 'Minggu':
      print('Libur kuliah');
    case 'Jumat':
      print('Kuliah sampai jam 11');
    default:
      print('Kuliah seperti biasa');
  }
  // Output: Libur kuliah
}
```

Sejak Dart 3, `break` tidak wajib lagi: case yang berisi kode otomatis berhenti di akhir case-nya. Case yang kosong (seperti `'Sabtu'`) "jatuh" ke case berikutnya, sehingga Sabtu dan Minggu berbagi kode yang sama.

switch expression (Dart 3)

Switch expression adalah switch yang **menghasilkan nilai**. Bentuknya lebih ringkas: tidak ada `case` , tidak ada `break` , setiap cabang ditulis `pola => nilai` , dan `_` berarti "selain itu".

```dart
void main() {
  String nilaiHuruf = 'B';
  int bobot = switch (nilaiHuruf) {
    'A' => 4,
    'B' => 3,
    'C' => 2,
    'D' => 1,
    _ => 0,
  };
  print('Bobot nilai $nilaiHuruf = $bobot'); // Output: Bobot nilai B = 3

  int nomorHari = 6;
  String jenisHari = switch (nomorHari) {
    6 || 7 => 'Akhir pekan',
    >= 1 && <= 5 => 'Hari kerja',
    _ => 'Nomor hari tidak valid',
  };
  print(jenisHari); // Output: Akhir pekan
}
```

`6 || 7` berarti "6 atau 7", dan `>= 1 && <= 5` berarti "di antara 1 sampai 5". Ini adalah *pattern* yang akan dibahas lebih dalam di section Fitur Modern Dart 3.

Perulangan for

```dart
void main() {
  for (int i = 1; i <= 3; i++) {
    print('Pertemuan ke-$i');
  }
  // Output:
  // Pertemuan ke-1
  // Pertemuan ke-2
  // Pertemuan ke-3

  // Perulangan bersarang (nested): tabel perkalian 3 x 3
  for (int baris = 1; baris <= 3; baris++) {
    String isi = '';
    for (int kolom = 1; kolom <= 3; kolom++) {
      isi += '${baris * kolom}'.padLeft(4);
    }
    print(isi);
  }
  // Output:
  //    1   2   3
  //    2   4   6
  //    3   6   9
}
```

Bagian `for (inisialisasi; kondisi; perubahan)` : `i` mulai dari 1, perulangan berjalan selama `i <= 3`, dan setelah tiap putaran `i` bertambah 1.

for-in dan forEach

```dart
void main() {
  List<String> matkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  // for-in: ambil setiap elemen tanpa indeks
  for (final mk in matkul) {
    print('- $mk');
  }
  // Output:
  // - Pemrograman Mobile
  // - Basis Data
  // - Jaringan Komputer

  // for biasa jika butuh nomor urut
  for (int i = 0; i < matkul.length; i++) {
    print('${i + 1}. ${matkul[i]}');
  }
  // Output:
  // 1. Pemrograman Mobile
  // 2. Basis Data
  // 3. Jaringan Komputer

  // forEach dengan fungsi
  matkul.forEach((mk) => print(mk.toUpperCase()));
  // Output:
  // PEMROGRAMAN MOBILE
  // BASIS DATA
  // JARINGAN KOMPUTER
}
```

while dan do-while

```dart
void main() {
  int saldo = 100000;
  int hari = 0;
  while (saldo >= 25000) {
    saldo -= 25000;
    hari++;
  }
  print('Uang jajan cukup untuk $hari hari, sisa $saldo'); // Output: Uang jajan cukup untuk 4 hari,
sisa 0

  int percobaan = 0;
  do {
    percobaan++;
    print('Percobaan login ke-$percobaan');
  } while (percobaan < 2);
  // Output:
  // Percobaan login ke-1
  // Percobaan login ke-2
}
```

`while` mengecek kondisi **sebelum** menjalankan isi perulangan, sedangkan `do-while` menjalankan isi minimal satu kali, baru mengecek kondisi di akhir. Gunakan `while` jika jumlah

putaran tidak diketahui sejak awal.

break dan continue

```
void main() {
  List<int> daftarNilai = [80, 45, 90, -1, 70];

  for (final n in daftarNilai) {
    if (n < 0) {
      print('Data tidak valid, berhenti.');
      break; // keluar dari perulangan sepenuhnya
    }
    if (n < 60) {
      print('$n dilewati');
      continue; // lompat ke elemen berikutnya
    }
    print('$n lulus');
  }
  // Output:
  // 80 lulus
  // 45 dilewati
  // 90 lulus
  // Data tidak valid, berhenti.
}
```

Nilai 70 tidak pernah diproses karena `break` sudah menghentikan perulangan saat bertemu -1.

> ⚠️ **Kesalahan umum**
>
> - **Infinite loop**: lupa mengubah variabel kondisi di dalam `while` (misalnya lupa `hari++`), sehingga program tidak pernah berhenti.
> - **Off-by-one**: memakai `i <= list.length` padahal indeks terakhir adalah `length - 1`. Akibatnya muncul `RangeError`.
> - Lupa cabang `_` pada switch expression untuk tipe `int` / `String`. Compiler akan menolak karena tidak semua kemungkinan tertangani.

5. Fungsi: Parameter, Closure & Higher-Order Function

Fungsi membungkus sekumpulan perintah agar bisa dipakai berulang kali dengan nama. Di Flutter, fungsi ada di mana-mana: `build()`, `onPressed`, `itemBuilder`, semuanya adalah fungsi. Memahami parameter bernama dan fungsi sebagai nilai adalah kunci membaca kode Flutter.

Fungsi dasar: parameter dan nilai kembali

```
double hitungLuas(double panjang, double lebar) {
  return panjang * lebar;
}

void sapa(String nama) {
  print('Halo, $nama!');
}

void main() {
  double luas = hitungLuas(4, 2.5);
  print(luas); // Output: 10.0
  sapa('Dimas'); // Output: Halo, Dimas!
}
```

Format umum: `TipeKembali namaFungsi(TipeParam param) { ... }`. Fungsi `void` tidak mengembalikan nilai. Fungsi yang punya tipe kembali wajib mengakhiri setiap jalurnya dengan `return`.

Arrow function `=>`

Jika isi fungsi hanya satu ekspresi, tulis dengan panah. `=> ekspresi` sama dengan `{ return ekspresi; }`.

```
int kuadrat(int x) => x * x;
bool isGenap(int x) => x % 2 == 0;
String salam(String nama) => 'Selamat pagi, $nama';

// Fungsi rekursif (memanggil dirinya sendiri)
int faktorial(int n) => n <= 1 ? 1 : n * faktorial(n - 1);

void main() {
  print(kuadrat(9)); // Output: 81
  print(isGenap(7)); // Output: false
  print(salam('Ayu')); // Output: Selamat pagi, Ayu
  print(faktorial(5)); // Output: 120
}
```

Parameter posisi opsional `[ ]`

```
String perkenalan(String nama, [String asal = 'Purwokerto', int? umur]) {
  var teks = 'Saya $nama dari $asal';
  if (umur != null) {
    teks += ', umur $umur tahun';
  }
```

```dart
  return teks;
}

void main() {
  print(perkenalan('Rani')); // Output: Saya Rani dari Purwokerto
  print(perkenalan('Bayu', 'Cilacap')); // Output: Saya Bayu dari Cilacap
  print(perkenalan('Sari', 'Banyumas', 19)); // Output: Saya Sari dari Banyumas, umur 19 tahun
}
```

Parameter di dalam `[ ]` boleh tidak diisi. Isinya bisa diberi nilai default (`asal = 'Purwokerto'`) atau bertipe nullable (`int? umur`) yang otomatis bernilai `null`. Kelemahannya: urutan argumen harus diingat.

Named parameter `{ }`, `required`, dan default

Named parameter dipanggil dengan menyebut namanya, sehingga urutan bebas dan kode mudah dibaca. Gaya inilah yang dipakai hampir semua widget Flutter, misalnya `Text('Halo', style: ..., maxLines: 2)`.

```dart
void daftarMahasiswa({
  required String nama,
  required String nim,
  String prodi = 'Informatika',
  int? semester,
}) {
  print('$nim - $nama ($prodi), semester: ${semester ?? '-'}');
}

void main() {
  daftarMahasiswa(nama: 'Ayu', nim: 'H1D023010');
  daftarMahasiswa(
    nim: 'H1D023011',
    nama: 'Bima',
    semester: 3,
    prodi: 'Teknik Elektro',
  );
  // daftarMahasiswa(nama: 'Cici'); // ERROR: parameter nim wajib diisi

  // Output:
  // H1D023010 - Ayu (Informatika), semester: -
  // H1D023011 - Bima (Teknik Elektro), semester: 3
}
```

Jenis	Deklarasi	Pemanggilan
Posisi wajib	`f(int a)`	`f(1)`
Posisi opsional	`f([int a = 0])`	`f()` atau `f(1)`
Named opsional	`f({int a = 0})` / `f({int? a})`	`f()` atau `f(a: 1)`
Named wajib	`f({required int a})`	`f(a: 1)`

Parameter posisi dan named boleh digabung, asalkan yang posisi ditulis lebih dulu, misalnya `void kirim(String tujuan, {bool penting = false})`.

Fungsi sebagai nilai dan fungsi anonim

Di Dart, fungsi adalah objek: bisa disimpan di variabel, dikirim sebagai argumen, dan dikembalikan dari fungsi lain. Fungsi tanpa nama disebut **fungsi anonim** (lambda).

```
void main() {
  // Fungsi anonim disimpan dalam variabel bertipe fungsi
  int Function(int, int) tambah = (a, b) => a + b;
  print(tambah(2, 3)); // Output: 5

  // Fungsi anonim dengan body blok
  var cetakBintang = (int n) {
    print('*' * n);
  };
  cetakBintang(5); // Output: *****

  // Fungsi anonim langsung dipakai sebagai argumen
  var angka = [3, 1, 2];
  angka.forEach((n) {
    print('angka $n');
  });
  // Output:
  // angka 3
  // angka 1
  // angka 2
}
```

Bentuk `(n) { ... }` dan `(n) => ...` inilah yang Anda tulis di Flutter, misalnya `onPressed: () { ... }`.

Closure: fungsi yang "mengingat" variabel

```
int Function() buatPenghitung() {
  int hitung = 0;
  return () {
    hitung++;
    return hitung;
```

```
  };
}

void main() {
  var klikTombolA = buatPenghitung();
  var klikTombolB = buatPenghitung();

  print(klikTombolA()); // Output: 1
  print(klikTombolA()); // Output: 2
  print(klikTombolA()); // Output: 3
  print(klikTombolB()); // Output: 1
}
```

Fungsi yang dikembalikan tetap bisa mengakses variabel `hitung` walaupun `buatPenghitung()` sudah selesai dijalankan. Setiap pemanggilan `buatPenghitung()` membuat variabel `hitung` sendiri, sehingga penghitung A dan B tidak saling mengganggu.

Higher-order function

Fungsi yang menerima fungsi lain sebagai parameter, atau mengembalikan fungsi, disebut *higher-order function*.

```
int hitung(int a, int b, int Function(int, int) operasi) {
  return operasi(a, b);
}

int Function(int) pengali(int faktor) {
  return (x) => x * faktor;
}

int kuadrat(int x) => x * x;

void main() {
  print(hitung(10, 4, (a, b) => a + b)); // Output: 14
  print(hitung(10, 4, (a, b) => a * b)); // Output: 40

  final kaliDua = pengali(2);
  final kaliTiga = pengali(3);
  print(kaliDua(5)); // Output: 10
  print(kaliTiga(5)); // Output: 15

  // Mengirim fungsi yang sudah ada tanpa tanda kurung (tear-off)
  print([1, 2, 3].map(kuadrat).toList()); // Output: [1, 4, 9]
}
```

`typedef` dan callback

`typedef` memberi nama untuk tipe fungsi agar mudah dibaca. Flutter memakai pola ini, misalnya `VoidCallback` dan `ValueChanged<T>`.

```
typedef SaatDitekan = void Function(String namaTombol);

void buatTombol(String label, {required SaatDitekan onPressed}) {
  print('[Tombol $label dibuat]');
  onPressed(label); // simulasi pengguna menekan tombol
```

```dart
}

void main() {
  buatTombol('Simpan', onPressed: (nama) {
    print('Tombol $nama ditekan, data disimpan.');
  });
  // Output:
  // [Tombol Simpan dibuat]
  // Tombol Simpan ditekan, data disimpan.
}
```

Pola ini sama persis dengan `ElevatedButton(onPressed: () { ... })`: kita "menitipkan" fungsi, lalu widget yang memanggilnya ketika tombol benar-benar ditekan.

> ⚠️ **Kesalahan umum**
>
> - `onPressed: simpan()` memanggil fungsi *saat itu juga* dan mengirim hasilnya. Yang benar `onPressed: simpan` (tear-off) atau `onPressed: () => simpan()`.
> - Lupa `return` pada fungsi blok yang punya tipe kembali.
> - Menulis arrow function dengan blok: `() => { ... }`. Di Dart, ini bukan blok, melainkan Set literal. Gunakan `() { ... }`.

6. Koleksi: List, Set & Map

Koleksi menyimpan banyak nilai dalam satu variabel. Dart punya tiga koleksi utama: **List** (berurutan, boleh duplikat), **Set** (unik), dan **Map** (pasangan kunci-nilai). Hampir semua data di aplikasi, mulai dari daftar produk sampai respon JSON dari API, disimpan dalam koleksi.

List: dasar-dasar

```dart
void main() {
  List<String> buah = ['apel', 'jeruk'];
  buah.add('mangga'); // tambah di akhir
  buah.addAll(['salak', 'durian']); // tambah banyak sekaligus
  buah.insert(0, 'pisang'); // sisipkan di indeks 0

  print(buah); // Output: [pisang, apel, jeruk, mangga, salak, durian]
  print(buah.length); // Output: 6
  print(buah[1]); // Output: apel
  print(buah.first); // Output: pisang
  print(buah.last); // Output: durian
  print(buah.indexOf('mangga')); // Output: 3
  print(buah.contains('nanas')); // Output: false

  buah[1] = 'anggur'; // ubah elemen
  buah.remove('jeruk'); // hapus berdasarkan nilai
  buah.removeAt(0); // hapus berdasarkan indeks
  print(buah); // Output: [anggur, mangga, salak, durian]
  print(buah.sublist(1, 3)); // Output: [mangga, salak]
  print(buah.reversed.toList()); // Output: [durian, salak, mangga, anggur]
  print(buah.isEmpty); // Output: false
}
```

Indeks List dimulai dari 0. Mengakses indeks di luar jangkauan (misalnya `buah[10]`) menyebabkan `RangeError`. Tipe di dalam `< >` disebut *generic*: `List<String>` berarti list yang isinya hanya boleh String.

Membuat List secara otomatis

```dart
void main() {
  var kuadrat = List.generate(5, (i) => i * i);
  print(kuadrat); // Output: [0, 1, 4, 9, 16]

  var kursi = List.filled(3, 'kosong');
  print(kursi); // Output: [kosong, kosong, kosong]

  var kosong = <int>[]; // list kosong bertipe int
  print(kosong.length); // Output: 0
}
```

Set: koleksi tanpa duplikat

```dart
void main() {
  Set<String> hobi = {'coding', 'futsal', 'membaca'};
```

```
  hobi.add('coding'); // diabaikan karena sudah ada
  print(hobi); // Output: {coding, futsal, membaca}
  print(hobi.length); // Output: 3
  print(hobi.contains('membaca')); // Output: true

  var kelasA = {'Ayu', 'Bima', 'Citra'};
  var kelasB = {'Bima', 'Dodi', 'Citra'};
  print(kelasA.union(kelasB)); // Output: {Ayu, Bima, Citra, Dodi}
  print(kelasA.intersection(kelasB)); // Output: {Bima, Citra}
  print(kelasA.difference(kelasB)); // Output: {Ayu}

  // Trik populer: buang duplikat dari List
  var presensi = ['01', '02', '01', '03', '02'];
  print(presensi.toSet().toList()); // Output: [01, 02, 03]
}
```

> ⚠️ **`{}` kosong adalah Map**
>
> `var x = {};` menghasilkan `Map`, bukan `Set`. Untuk Set kosong tulis `<String>{}` atau `Set<String> x = {};`.

Map: pasangan kunci-nilai

```
void main() {
  Map<String, int> stok = {
    'pulpen': 20,
    'buku': 15,
  };

  stok['penggaris'] = 8; // tambah pasangan baru
  stok['buku'] = 12; // ubah nilai
  print(stok); // Output: {pulpen: 20, buku: 12, penggaris: 8}
  print(stok['pulpen']); // Output: 20
  print(stok['penghapus']); // Output: null
  print(stok.containsKey('buku')); // Output: true
  print(stok.keys); // Output: (pulpen, buku, penggaris)
  print(stok.values.toList()); // Output: [20, 12, 8]

  stok.remove('pulpen');
  stok.update('buku', (lama) => lama + 5);
  stok.putIfAbsent('spidol', () => 10);
  print(stok); // Output: {buku: 17, penggaris: 8, spidol: 10}

  for (final entry in stok.entries) {
    print('${entry.key} -> ${entry.value}');
  }
  // Output:
  // buku -> 17
  // penggaris -> 8
  // spidol -> 10
}
```

Mengakses kunci yang tidak ada menghasilkan `null`, bukan error. Karena itu tipe `stok['x']` adalah `int?` (nullable).

Map<String, dynamic>: bentuk data JSON

Data JSON dari API (Modul 09) akan berbentuk `Map<String, dynamic>` karena nilainya bisa bermacam-macam tipe.

```
void main() {
  Map<String, dynamic> mahasiswa = {
    'nama': 'Nadia',
    'nim': 'H1D023005',
    'ipk': 3.75,
    'aktif': true,
    'matkul': ['Mobile', 'Web'],
  };

  String nama = mahasiswa['nama'] as String;
  double ipk = mahasiswa['ipk'] as double;
  List matkul = mahasiswa['matkul'] as List;

  print('$nama, IPK $ipk, ambil ${matkul.length} matkul'); // Output: Nadia, IPK 3.75, ambil 2
matkul
}
```

Method fungsional: map, where, fold, reduce, any, every

Method berikut menerima fungsi sebagai argumen (ingat higher-order function di section sebelumnya) dan membuat kode pengolahan data jauh lebih ringkas daripada perulangan manual.

```
void main() {
  List<int> nilai = [78, 92, 55, 64, 88, 40];

  var lulus = nilai.where((n) => n >= 60).toList();
  print(lulus); // Output: [78, 92, 64, 88]

  var plusBonus = nilai.map((n) => n + 5).toList();
  print(plusBonus); // Output: [83, 97, 60, 69, 93, 45]

  int total = nilai.fold(0, (jumlah, n) => jumlah + n);
  print(total); // Output: 417
  print(total / nilai.length); // Output: 69.5

  int tertinggi = nilai.reduce((a, b) => a > b ? a : b);
  print(tertinggi); // Output: 92

  print(nilai.any((n) => n < 50)); // Output: true
  print(nilai.every((n) => n >= 40)); // Output: true
  print(nilai.firstWhere((n) => n > 80)); // Output: 92
  print(nilai.take(2).toList()); // Output: [78, 92]
  print(nilai.skip(4).toList()); // Output: [88, 40]
  print(nilai.join(' | ')); // Output: 78 | 92 | 55 | 64 | 88 | 40
}
```

Method	Fungsi	Hasil
`where`	Menyaring elemen yang memenuhi syarat	Iterable
`map`	Mengubah setiap elemen	Iterable
`fold`	Mengakumulasi dengan nilai awal	Satu nilai
`reduce`	Mengakumulasi tanpa nilai awal (list tidak boleh kosong)	Satu nilai
`any` / `every`	Ada yang memenuhi / semua memenuhi	`bool`
`firstWhere`	Elemen pertama yang memenuhi syarat	Satu elemen

`where` dan `map` menghasilkan `Iterable` yang bersifat *lazy* (baru diproses ketika dibutuhkan). Tambahkan `.toList()` jika Anda butuh List sungguhan.

Mengurutkan dengan sort

```dart
void main() {
  var angka = [5, 2, 9, 1];
  angka.sort(); // naik (ascending)
  print(angka); // Output: [1, 2, 5, 9]
  angka.sort((a, b) => b.compareTo(a)); // turun (descending)
  print(angka); // Output: [9, 5, 2, 1]

  var nama = ['Citra', 'ayu', 'Bima'];
  nama.sort(); // huruf besar diurutkan lebih dulu
  print(nama); // Output: [Bima, Citra, ayu]
  nama.sort((a, b) => a.toLowerCase().compareTo(b.toLowerCase()));
  print(nama); // Output: [ayu, Bima, Citra]

  var mhs = [
    {'nama': 'Ayu', 'ipk': 3.4},
    {'nama': 'Bima', 'ipk': 3.9},
    {'nama': 'Citra', 'ipk': 3.6},
  ];
  mhs.sort((a, b) => (b['ipk'] as double).compareTo(a['ipk'] as double));
  print(mhs.map((m) => m['nama']).toList()); // Output: [Bima, Citra, Ayu]
}
```

Fungsi pembanding mengembalikan angka negatif jika `a` harus di depan, positif jika `b` di depan, dan 0 jika setara. `compareTo` sudah melakukan itu untuk kita. Perhatikan bahwa `sort()` mengubah list aslinya dan tidak mengembalikan nilai.

Spread, collection-if, dan collection-for

```dart
List<String>? ambilMatkulTambahan() => null; // anggap belum ada data
```

```dart
void main() {
  var wajib = ['Algoritma', 'Basis Data'];
  var pilihan = ['Pemrograman Mobile'];
  var tambahan = ambilMatkulTambahan();
  bool ambilMagang = true;

  var krs = [
    ...wajib, // sebar isi list wajib
    ...pilihan,
    ...?tambahan, // sebar hanya jika tidak null
    if (ambilMagang) 'Magang Industri',
    for (var i = 1; i <= 2; i++) 'Seminar $i',
  ];
  print(krs);
  // Output:
  // [Algoritma, Basis Data, Pemrograman Mobile, Magang Industri, Seminar 1, Seminar 2]

  var nilai = {'Ayu': 90, 'Bima': 55};
  var label = [
    for (final e in nilai.entries) '${e.key}: ${e.value >= 60 ? 'Lulus' : 'Gagal'}',
  ];
  print(label); // Output: [Ayu: Lulus, Bima: Gagal]
}
```

Fitur ini sangat sering dipakai di Flutter untuk menyusun `children` sebuah `Column`, misalnya `children: [const Header(), if (isLogin) const Profil(), ...daftarMenu]`.

Merangkai method (chaining)

```dart
void main() {
  final belanja = [
    {'nama': 'Kopi', 'harga': 15000, 'kategori': 'minuman'},
    {'nama': 'Mendoan', 'harga': 10000, 'kategori': 'makanan'},
    {'nama': 'Es Teh', 'harga': 5000, 'kategori': 'minuman'},
    {'nama': 'Nasi Rames', 'harga': 18000, 'kategori': 'makanan'},
  ];

  final totalMinuman = belanja
      .where((b) => b['kategori'] == 'minuman')
      .map((b) => b['harga'] as int)
      .fold(0, (a, b) => a + b);
  print('Total minuman: Rp$totalMinuman'); // Output: Total minuman: Rp20000

  final namaMahal = belanja
      .where((b) => (b['harga'] as int) >= 15000)
      .map((b) => (b['nama'] as String).toUpperCase())
      .toList();
  print(namaMahal); // Output: [KOPI, NASI RAMES]
}
```

Baca rantai dari atas ke bawah: saring yang kategori minuman, ambil harganya, lalu jumlahkan. Gaya ini lebih mudah dibaca daripada `for` dengan banyak variabel sementara.

⚠️ **Kesalahan umum**

- Lupa `.toList()` setelah `map` / `where`, lalu bingung kenapa tercetak `(...)` bukan `[...]`.
- Memanggil `reduce` atau `first` pada list kosong, sehingga muncul `StateError: No element`. Gunakan `fold` atau cek `isEmpty` dulu.
- Menulis `var hasil = list.sort();`. `sort()` bertipe `void`, list aslinya yang terurut.

7. Sound Null Safety

Di banyak bahasa, error paling sering adalah memakai nilai yang ternyata kosong (*null*), dikenal sebagai `NullPointerException`. Dart 3 menerapkan **Sound Null Safety**: secara default, semua tipe **tidak boleh** null, dan compiler memaksa kita menangani kemungkinan null sebelum program dijalankan.

Tipe non-nullable vs nullable

```dart
void main() {
  String nama = 'Ayu';
  // nama = null; // ERROR: String tidak boleh null

  String? nomorHp; // tanda ? artinya boleh null, nilai awalnya null
  print(nama.length); // Output: 3
  print(nomorHp); // Output: null
  // print(nomorHp.length); // ERROR: nomorHp mungkin null

  nomorHp = '08123456789';
  print(nomorHp.length); // Output: 11
}
```

Setelah `nomorHp` diisi String, Dart tahu nilainya pasti tidak null, sehingga `nomorHp.length` diizinkan. Kemampuan ini disebut **type promotion**.

Ringkasan operator null safety

Sintaks	Arti	Contoh
`Tipe?`	Tipe nullable (boleh null)	`String? email;`
`?.`	Akses hanya jika tidak null	`email?.length`
`??`	Nilai cadangan jika null	`email ?? '-'`
`??=`	Isi hanya jika masih null	`email ??= 'a@b.c';`
`!`	Paksa anggap tidak null (crash jika ternyata null)	`email!.length`
`late`	Diisi nanti, dijanjikan sebelum dipakai	`late String token;`
`required`	Named parameter wajib diisi	`{required String nim}`

Memakai `?.`, `??`, dan `??=`

```dart
String? cariEmail(String nim) {
  final data = {'H1D023001': 'ayu@mhs.unsoed.ac.id'};
  return data[nim]; // null jika nim tidak terdaftar
}

void main() {
  String? email1 = cariEmail('H1D023001');
  String? email2 = cariEmail('H1D099999');

  print(email1?.toUpperCase()); // Output: AYU@MHS.UNSOED.AC.ID
  print(email2?.toUpperCase()); // Output: null
  print(email2 ?? 'Email belum terdaftar'); // Output: Email belum terdaftar
  print(email2?.length ?? 0); // Output: 0

  email2 ??= 'default@unsoed.ac.id';
  print(email2); // Output: default@unsoed.ac.id
}
```

`email2?.length ?? 0` adalah kombinasi yang sangat umum: "ambil panjangnya jika ada, jika tidak pakai 0".

Type promotion dengan pengecekan null

```dart
int hitungPanjang(String? teks) {
  if (teks == null) {
    return 0;
  }
  // Di sini Dart tahu teks pasti tidak null
  return teks.length;
}

class Profil {
  String? bio;
}

void cetakBio(Profil p) {
  // if (p.bio != null) print(p.bio.length); // ERROR: field tidak dipromosikan
  final bio = p.bio; // salin ke variabel lokal dulu
  if (bio != null) {
    print('Panjang bio: ${bio.length}');
  } else {
    print('Bio kosong');
  }
}

void main() {
  print(hitungPanjang(null)); // Output: 0
  print(hitungPanjang('Flutter')); // Output: 7

  cetakBio(Profil()); // Output: Bio kosong
  cetakBio(Profil()..bio = 'Suka Dart'); // Output: Panjang bio: 9
}
```

Promotion berjalan untuk variabel lokal dan parameter. Untuk field class yang publik dan bisa diubah, Dart tidak bisa menjamin nilainya tetap sama di antara pengecekan dan pemakaian, sehingga trik "salin ke variabel lokal" sangat sering dipakai.

Operator `!`: gunakan dengan hati-hati

```dart
void main() {
  Map<String, int> nilai = {'Ayu': 90};

  int nilaiAyu = nilai['Ayu']!; // kita YAKIN kunci 'Ayu' ada
  print(nilaiAyu); // Output: 90

  // int nilaiBudi = nilai['Budi']!;
  // CRASH saat runtime: Null check operator used on a null value

  int nilaiBudi = nilai['Budi'] ?? 0; // cara yang aman
  print(nilaiBudi); // Output: 0
}
```

`!` mematikan perlindungan null safety. Gunakan hanya jika Anda benar-benar yakin nilainya tidak null. Dalam banyak kasus, `??` atau pengecekan `if` adalah pilihan yang lebih aman.

`late` pada field class

```dart
class KoneksiDatabase {
  late String namaDb; // belum ada nilai saat objek dibuat

  void buka(String nama) {
    namaDb = nama;
    print('Database $namaDb dibuka');
  }
}

void main() {
  final db = KoneksiDatabase();
  // print(db.namaDb); // CRASH: LateInitializationError, belum diisi
  db.buka('kampus.db'); // Output: Database kampus.db dibuka
  print(db.namaDb); // Output: kampus.db
}
```

Di Flutter, `late` biasa dipakai untuk controller yang dibuat di `initState()`, misalnya `late TextEditingController _namaController;`.

Null-aware pada List dan Map

```dart
List<String>? ambilDaftar(bool ada) => ada ? ['Dart', 'Flutter'] : null;

void main() {
  final daftar = ambilDaftar(false);
  print(daftar?.length); // Output: null
  print(daftar?[0]); // Output: null
  print(daftar?.isEmpty ?? true); // Output: true
```

```dart
  final daftar2 = ambilDaftar(true);
  print(daftar2?[1]); // Output: Flutter
}
```

⚠️ Kesalahan umum

- Menaburkan `!` di mana-mana agar error compiler hilang. Error-nya hanya pindah menjadi crash saat aplikasi dipakai.
- Membuat semua variabel nullable (`String?`) "supaya aman". Justru sebaliknya: jadikan non-nullable sebisa mungkin, dan nullable hanya untuk data yang memang bisa tidak ada.
- Memakai `late` lalu lupa mengisinya, sehingga muncul `LateInitializationError`.

8. Object-Oriented Programming (OOP)

Flutter sepenuhnya berbasis Object-Oriented Programming (OOP): setiap widget adalah objek dari sebuah class. Section ini membahas OOP di Dart secara bertahap, mulai dari class sederhana sampai mixin, extension, dan enum.

1. Class dan objek

Class adalah cetakan, **objek** adalah hasil cetakannya. Class berisi *field* (data) dan *method* (perilaku).

```dart
class Mahasiswa {
  String nama;
  String nim;
  int semester;

  // Constructor singkat: langsung mengisi field
  Mahasiswa(this.nama, this.nim, this.semester);

  void perkenalan() {
    print('Halo, saya $nama ($nim), semester $semester.');
  }

  void naikSemester() {
    semester++;
  }
}

void main() {
  var ayu = Mahasiswa('Ayu', 'H1D023001', 3);
  var bima = Mahasiswa('Bima', 'H1D023002', 5);

  ayu.perkenalan();
  ayu.naikSemester();
  ayu.perkenalan();
  bima.perkenalan();
  // Output:
  // Halo, saya Ayu (H1D023001), semester 3.
  // Halo, saya Ayu (H1D023001), semester 4.
  // Halo, saya Bima (H1D023002), semester 5.
}
```

Kata kunci `new` tidak diperlukan di Dart modern. `ayu` dan `bima` adalah dua objek terpisah: mengubah semester `ayu` tidak memengaruhi `bima`.

2. Variasi constructor

```dart
class Buku {
  final String judul;
  final String penulis;
  final int tahun;
  final int stok;
```

```
  // a. Constructor utama dengan named parameter
  Buku({
    required this.judul,
    required this.penulis,
    this.tahun = 2024,
    this.stok = 0,
  });

  // b. Named constructor: cara lain membuat objek
  Buku.ebook(this.judul, this.penulis)
      : tahun = 2025,
        stok = 999;

  // c. Redirecting constructor: meneruskan ke constructor utama
  Buku.anonim(String judul) : this(judul: judul, penulis: 'Anonim');

  @override
  String toString() => '$judul oleh $penulis ($tahun), stok $stok';
}

void main() {
  print(Buku(judul: 'Dart Dasar', penulis: 'Rina', stok: 5));
  print(Buku.ebook('Flutter Cepat', 'Andi'));
  print(Buku.anonim('Catatan Kuliah'));
  // Output:
  // Dart Dasar oleh Rina (2024), stok 5
  // Flutter Cepat oleh Andi (2025), stok 999
  // Catatan Kuliah oleh Anonim (2024), stok 0
}
```

Method `toString()` di-*override* agar `print(objek)` menampilkan teks yang informatif, bukan `Instance of 'Buku'`. Named constructor sering ditemui di Flutter, misalnya `ListView.builder` dan `EdgeInsets.all`.

3. Initializer list dan assert

Bagian setelah tanda `:` pada constructor disebut *initializer list*. Bagian ini dijalankan sebelum body constructor dan cocok untuk mengisi field `final` hasil perhitungan.

```
class Mahasiswa {
  final String nim;
  final String nama;
  final String angkatan;

  Mahasiswa(this.nim, String nama)
      : assert(nim.length == 9, 'NIM harus 9 karakter'),
        nama = nama.trim(),
        angkatan = '20${nim.substring(4, 6)}';
}

void main() {
  final m = Mahasiswa('H1D023001', '  Ayu Lestari ');
  print('[${m.nama}]'); // Output: [Ayu Lestari]
  print(m.angkatan); // Output: 2023
}
```

`assert` hanya aktif saat mode debug (Flutter `flutter run`, atau `dart run --enable-asserts`). Jika kondisinya salah, program berhenti dengan pesan yang diberikan.

4. Const constructor

```dart
class Warna {
  final int r, g, b;
  const Warna(this.r, this.g, this.b);
}

void main() {
  const merah1 = Warna(255, 0, 0);
  const merah2 = Warna(255, 0, 0);
  print(identical(merah1, merah2)); // Output: true

  final biru1 = Warna(0, 0, 255);
  final biru2 = Warna(0, 0, 255);
  print(identical(biru1, biru2)); // Output: false
}
```

Objek `const` dengan isi yang sama hanya dibuat **sekali** dan dipakai bersama (`identical` bernilai `true`). Inilah alasan Flutter menyarankan `const Text('Halo')`: widget tidak perlu dibuat ulang setiap kali layar di-*rebuild*. Syaratnya, semua field harus `final`.

5. Factory constructor

`factory` adalah constructor yang boleh mengandung logika dan mengembalikan objek (baru atau yang sudah ada). Pemakaian paling umum adalah `fromJson` untuk mengubah data JSON menjadi objek.

```dart
class Mahasiswa {
  final String nim;
  final String nama;
  final double ipk;

  Mahasiswa({required this.nim, required this.nama, required this.ipk});

  factory Mahasiswa.fromJson(Map<String, dynamic> json) {
    return Mahasiswa(
      nim: json['nim'] as String,
      nama: json['nama'] as String,
      ipk: (json['ipk'] as num).toDouble(),
    );
  }

  Map<String, dynamic> toJson() => {'nim': nim, 'nama': nama, 'ipk': ipk};
}

void main() {
  final json = {'nim': 'H1D023007', 'nama': 'Nadia', 'ipk': 4};
  final m = Mahasiswa.fromJson(json);
  print('${m.nama} - IPK ${m.ipk}'); // Output: Nadia - IPK 4.0
  print(m.toJson()); // Output: {nim: H1D023007, nama: Nadia, ipk: 4.0}
}
```

Perhatikan `(json['ipk'] as num).toDouble()`: server bisa mengirim `4` (int) atau `3.5` (double). Membaca sebagai `num` lalu mengubah ke `double` menghindari error tipe. Pola ini akan dipakai lagi di Modul 09 (REST API).

Factory juga bisa mengembalikan objek yang sama terus-menerus (pola *singleton*):

```
class Pengaturan {
  static final Pengaturan _instance = Pengaturan._internal();
  String tema = 'terang';

  Pengaturan._internal(); // constructor private

  factory Pengaturan() => _instance;
}

void main() {
  final a = Pengaturan();
  final b = Pengaturan();
  a.tema = 'gelap';
  print(b.tema); // Output: gelap
  print(identical(a, b)); // Output: true
}
```

6. Getter, setter, dan private

Nama yang diawali garis bawah `_` bersifat **private**. Akses dari luar diatur lewat *getter* (`get`) dan *setter* (`set`), sehingga data bisa divalidasi.

```
class Mahasiswa {
  final String nama;
  double _ipk = 0;

  Mahasiswa(this.nama);

  double get ipk => _ipk;

  set ipk(double nilai) {
    if (nilai < 0 || nilai > 4) {
      print('IPK $nilai tidak valid, diabaikan.');
      return;
    }
    _ipk = nilai;
  }

  // Getter hasil perhitungan (computed property)
  String get predikat => _ipk >= 3.51 ? 'Cumlaude' : 'Reguler';
}

void main() {
  final m = Mahasiswa('Fikri');
  m.ipk = 3.8; // memanggil setter
  print(m.ipk); // Output: 3.8
  print(m.predikat); // Output: Cumlaude
  m.ipk = 4.5; // Output: IPK 4.5 tidak valid, diabaikan.
```

```
  print(m.ipk); // Output: 3.8
}
```

Contoh lain yang lebih lengkap: rekening dengan saldo dan riwayat yang tidak bisa diubah sembarangan dari luar.

```
class RekeningBank {
  final String pemilik;
  double _saldo = 0;
  final List<String> _riwayat = [];

  RekeningBank(this.pemilik);

  double get saldo => _saldo;
  List<String> get riwayat => List.unmodifiable(_riwayat);

  void setor(double jumlah) {
    if (jumlah <= 0) return;
    _saldo += jumlah;
    _catat('Setor $jumlah');
  }

  bool tarik(double jumlah) {
    if (jumlah > _saldo) {
      _catat('Gagal tarik $jumlah');
      return false;
    }
    _saldo -= jumlah;
    _catat('Tarik $jumlah');
    return true;
  }

  void _catat(String aktivitas) => _riwayat.add(aktivitas); // method private
}

void main() {
  final rek = RekeningBank('Ayu');
  rek.setor(50000);
  rek.tarik(20000);
  rek.tarik(100000);
  print(rek.saldo); // Output: 30000.0
  print(rek.riwayat); // Output: [Setor 50000.0, Tarik 20000.0, Gagal tarik 100000.0]
}
```

> ℹ️ **Private di Dart = private per file**
>
> Dart tidak punya kata kunci `private` / `public`. Awalan `_` membuat anggota hanya terlihat di **file (library) yang sama**. Jadi pada contoh satu file di atas, `main()` secara teknis masih bisa mengakses `_saldo`. Saat class dipindah ke file lain (misalnya `lib/models/rekening.dart`), barulah `_saldo` benar-benar tersembunyi.

7. Anggota static

```
class Mahasiswa {
  static int jumlahDibuat = 0; // milik class, bukan milik objek
  static const String kampus = 'UNSOED';

  final String nama;

  Mahasiswa(this.nama) {
    jumlahDibuat++;
  }

  static String formatNim(String angkatan, int urut) {
    return 'H1D$angkatan${urut.toString().padLeft(3, '0')}';
  }
}

void main() {
  final daftar = [Mahasiswa('Ayu'), Mahasiswa('Bima')];
  print(daftar.length); // Output: 2
  print(Mahasiswa.jumlahDibuat); // Output: 2
  print(Mahasiswa.kampus); // Output: UNSOED
  print(Mahasiswa.formatNim('023', 7)); // Output: H1D023007
}
```

Anggota `static` dipanggil lewat nama class, tanpa membuat objek. Contoh di Flutter: `Colors.blue` dan `Navigator.push(...)`.

8. Inheritance (pewarisan) dan polymorphism

```
class Pengguna {
  final String nama;
  final String email;

  Pengguna(this.nama, this.email);

  String get peran => 'Pengguna';

  void tampilkanProfil() {
    print('[$peran] $nama <$email>');
  }
}

class Mahasiswa extends Pengguna {
  final String nim;

  // super parameter: diteruskan langsung ke constructor Pengguna
  Mahasiswa(super.nama, super.email, this.nim);

  @override
  String get peran => 'Mahasiswa';

  @override
  void tampilkanProfil() {
    super.tampilkanProfil(); // jalankan versi induk dulu
    print('  NIM: $nim');
```

```
  }
}

class Dosen extends Pengguna {
  final String nidn;

  // cara lama: memanggil super(...) di initializer list
  Dosen(String nama, String email, this.nidn) : super(nama, email);

  @override
  String get peran => 'Dosen';
}

void main() {
  final daftar = <Pengguna>[
    Mahasiswa('Ayu', 'ayu@mhs.unsoed.ac.id', 'H1D023001'),
    Dosen('Dr. Budi', 'budi@unsoed.ac.id', '0612345678'),
  ];

  for (final p in daftar) {
    p.tampilkanProfil();
  }
  // Output:
  // [Mahasiswa] Ayu <ayu@mhs.unsoed.ac.id>
  //   NIM: H1D023001
  // [Dosen] Dr. Budi <budi@unsoed.ac.id>
}
```

`extends` mewarisi seluruh field dan method induk. `@override` menandai method yang ditulis ulang. Walaupun list bertipe `List<Pengguna>`, setiap objek menjalankan versi method miliknya sendiri. Inilah **polymorphism**. Di Flutter, `class HomePage extends StatelessWidget` adalah contoh nyata inheritance.

9. Abstract class

Abstract class adalah class "setengah jadi" yang tidak bisa dibuat objeknya. Ia mendefinisikan method yang **wajib** diisi oleh class turunannya.

```
import 'dart:math';

abstract class BangunDatar {
  String get nama;
  double luas(); // method abstrak: tanpa isi
  double keliling();

  // method biasa boleh ada di abstract class
  void info() {
    print('$nama: luas ${luas().toStringAsFixed(2)}, '
        'keliling ${keliling().toStringAsFixed(2)}');
  }
}

class Lingkaran extends BangunDatar {
  final double r;
  Lingkaran(this.r);
```

```dart
  @override
  String get nama => 'Lingkaran';
  @override
  double luas() => pi * r * r;
  @override
  double keliling() => 2 * pi * r;
}

class PersegiPanjang extends BangunDatar {
  final double p, l;
  PersegiPanjang(this.p, this.l);

  @override
  String get nama => 'Persegi panjang';
  @override
  double luas() => p * l;
  @override
  double keliling() => 2 * (p + l);
}

void main() {
  // final b = BangunDatar(); // ERROR: abstract class tidak bisa diinstansiasi
  final daftar = [Lingkaran(7), PersegiPanjang(4, 3)];
  for (final b in daftar) {
    b.info();
  }
  // Output:
  // Lingkaran: luas 153.94, keliling 43.98
  // Persegi panjang: luas 12.00, keliling 14.00
}
```

10. Interface dengan `implements`

Setiap class di Dart otomatis juga merupakan *interface*. Dengan `implements`, sebuah class berjanji menyediakan **semua** method milik class lain, tetapi tidak mewarisi isinya. Sebuah class boleh meng-`implements` beberapa interface sekaligus.

```dart
abstract class BisaDicetak {
  void cetak();
}

abstract class BisaDiekspor {
  String keCsv();
}

class Transkrip implements BisaDicetak, BisaDiekspor {
  final String nama;
  final Map<String, String> nilai;

  Transkrip(this.nama, this.nilai);

  @override
  void cetak() {
    print('Transkrip $nama');
    nilai.forEach((mk, huruf) => print('  $mk: $huruf'));
  }
```

```
  @override
  String keCsv() => nilai.entries.map((e) => '${e.key},${e.value}').join(';');
}

void main() {
  final t = Transkrip('Ayu', {'Mobile': 'A', 'Basis Data': 'B'});
  t.cetak();
  print(t.keCsv());
  // Output:
  // Transkrip Ayu
  //   Mobile: A
  //   Basis Data: B
  // Mobile,A;Basis Data,B
}
```

	`extends`	`implements`	`with` (mixin)
Jumlah	Hanya satu induk	Boleh banyak	Boleh banyak
Mewarisi isi method?	Ya	Tidak, wajib tulis ulang semua	Ya
Hubungan	"adalah" (Mahasiswa adalah Pengguna)	"berjanji bisa"	"punya kemampuan"

11. Mixin

Mixin adalah kumpulan kemampuan yang bisa "ditempelkan" ke banyak class tanpa harus menjadi induknya. Kata kunci `on` membatasi mixin hanya untuk turunan class tertentu, sehingga mixin boleh memakai anggota class tersebut.

```
class Pengguna {
  final String nama;
  Pengguna(this.nama);
}

mixin AksesPerpustakaan {
  int batasPinjam = 3;
  final List<String> _pinjaman = [];

  void pinjamBuku(String judul) {
    if (_pinjaman.length >= batasPinjam) {
      print('Gagal: batas pinjam $batasPinjam buku');
      return;
    }
    _pinjaman.add(judul);
    print('Meminjam "$judul" (${_pinjaman.length}/$batasPinjam)');
  }
}

mixin AksesLab on Pengguna {
  void masukLab(String lab) => print('$nama masuk $lab'); // boleh pakai nama
```

```dart
}

class Mahasiswa extends Pengguna with AksesPerpustakaan, AksesLab {
  Mahasiswa(super.nama);
}

class Dosen extends Pengguna with AksesPerpustakaan {
  Dosen(super.nama) {
    batasPinjam = 10;
  }
}

void main() {
  final m = Mahasiswa('Ayu');
  m.pinjamBuku('Clean Code');
  m.pinjamBuku('Dart Apprentice');
  m.pinjamBuku('Flutter in Action');
  m.pinjamBuku('Refactoring');
  m.masukLab('Lab Komputasi');

  final d = Dosen('Pak Budi');
  d.pinjamBuku('Design Patterns');
  // Output:
  // Meminjam "Clean Code" (1/3)
  // Meminjam "Dart Apprentice" (2/3)
  // Meminjam "Flutter in Action" (3/3)
  // Gagal: batas pinjam 3 buku
  // Ayu masuk Lab Komputasi
  // Meminjam "Design Patterns" (1/10)
}
```

Anda akan menemui mixin di Flutter, misalnya `class _HomeState extends State<Home> with SingleTickerProviderStateMixin` untuk animasi.

12. Extension method

Extension menambahkan method baru ke class yang sudah ada (bahkan class bawaan seperti `int` dan `String`) tanpa mengubah atau mewarisinya.

```dart
extension FormatRupiah on int {
  String get rupiah {
    final s = toString();
    final buffer = StringBuffer();
    for (var i = 0; i < s.length; i++) {
      if (i > 0 && (s.length - i) % 3 == 0) buffer.write('.');
      buffer.write(s[i]);
    }
    return 'Rp$buffer';
  }
}

extension StringKampus on String {
  bool get isNimValid => RegExp(r'^H1D\d{6}$').hasMatch(this);

  String get kapital =>
      isEmpty ? this : this[0].toUpperCase() + substring(1).toLowerCase();
}
```

```dart
void main() {
  print(150000.rupiah); // Output: Rp150.000
  print(1250000.rupiah); // Output: Rp1.250.000
  print('H1D023001'.isNimValid); // Output: true
  print('abc'.isNimValid); // Output: false
  print('fLUTTER'.kapital); // Output: Flutter
}
```

13. Enum dan enhanced enum

Enum mewakili sekumpulan nilai tetap. Kombinasi enum dengan `switch` membuat compiler bisa memastikan semua kemungkinan sudah ditangani.

```dart
enum StatusKrs { draft, diajukan, disetujui, ditolak }

void main() {
  var status = StatusKrs.diajukan;
  print(status); // Output: StatusKrs.diajukan
  print(status.name); // Output: diajukan
  print(status.index); // Output: 1
  print(StatusKrs.values.length); // Output: 4

  final pesan = switch (status) {
    StatusKrs.draft => 'Belum diajukan',
    StatusKrs.diajukan => 'Menunggu dosen wali',
    StatusKrs.disetujui => 'KRS disetujui',
    StatusKrs.ditolak => 'Perlu revisi',
  };
  print(pesan); // Output: Menunggu dosen wali
}
```

Switch di atas tidak butuh `_` karena semua nilai enum sudah ditulis. Jika nanti ditambahkan nilai baru, misalnya `StatusKrs.dibatalkan`, compiler langsung menunjukkan switch mana yang belum lengkap.

Enhanced enum (Dart 2.17+) bisa punya field, constructor, getter, dan method:

```dart
enum NilaiHuruf {
  a('A', 4.0, 85),
  ab('AB', 3.5, 80),
  b('B', 3.0, 70),
  bc('BC', 2.5, 65),
  c('C', 2.0, 60),
  d('D', 1.0, 50),
  e('E', 0.0, 0);

  const NilaiHuruf(this.label, this.bobot, this.batasBawah);

  final String label;
  final double bobot;
  final int batasBawah;

  bool get lulus => bobot >= 2.0;
```

```
  static NilaiHuruf dariAngka(num nilai) {
    return values.firstWhere((h) => nilai >= h.batasBawah);
  }
}

void main() {
  for (final n in [92, 81, 67, 45]) {
    final huruf = NilaiHuruf.dariAngka(n);
    print('$n -> ${huruf.label} (bobot ${huruf.bobot}, lulus: ${huruf.lulus})');
  }
  // Output:
  // 92 -> A (bobot 4.0, lulus: true)
  // 81 -> AB (bobot 3.5, lulus: true)
  // 67 -> BC (bobot 2.5, lulus: true)
  // 45 -> E (bobot 0.0, lulus: false)
}
```

`values` berisi semua nilai enum sesuai urutan penulisan. Karena urutannya dari batas tertinggi, `firstWhere` menemukan huruf pertama yang batas bawahnya terpenuhi. Skala nilai di sini hanya contoh; sesuaikan dengan pedoman akademik yang berlaku.

> ⚠️ **Kesalahan umum OOP**
>
> - Lupa `@override`. Kode tetap jalan, tetapi Anda kehilangan peringatan saat salah ketik nama method induk.
> - Field non-nullable tidak diisi di constructor: error *"Non-nullable instance field must be initialized"*. Isi lewat `this.x`, initializer list, nilai default, atau gunakan `late`/nullable.
> - Membuat semua field publik dan bisa diubah. Biasakan field `final` dan sembunyikan data penting dengan `_`.

9. Fitur Modern Dart 3: Records, Patterns & Sealed Class

Dart 3 menambahkan fitur yang membuat kode lebih ringkas dan lebih aman: **records** (mengelompokkan beberapa nilai tanpa membuat class), **patterns** (membongkar dan mencocokkan bentuk data), serta **sealed class** (switch yang dijamin lengkap oleh compiler).

Records

Record adalah kumpulan beberapa nilai yang dibungkus jadi satu, ditulis dengan tanda kurung. Field-nya bisa berdasarkan posisi (diakses dengan `$1`, `$2`, ...) atau bernama.

```dart
void main() {
  var mhs = ('Ayu', 3.75); // record posisi
  print(mhs.$1); // Output: Ayu
  print(mhs.$2); // Output: 3.75

  ({String nama, int sks}) matkul = (nama: 'Pemrograman Mobile', sks: 3);
  print(matkul.nama); // Output: Pemrograman Mobile
  print(matkul); // Output: (nama: Pemrograman Mobile, sks: 3)

  var campur = ('H1D023001', nama: 'Ayu', aktif: true); // posisi + bernama
  print(campur.$1); // Output: H1D023001
  print(campur.aktif); // Output: true

  // Record dibandingkan berdasarkan isinya
  print(('Ayu', 20) == ('Ayu', 20)); // Output: true
}
```

Fungsi yang mengembalikan beberapa nilai

Sebelum Dart 3, mengembalikan lebih dari satu nilai harus memakai List, Map, atau class khusus. Dengan record, cukup tulis tipe kembalian sebagai record.

```dart
({double rataRata, int tertinggi, int terendah}) hitungStatistik(List<int> nilai) {
  final urut = [...nilai]..sort();
  final total = nilai.fold(0, (a, b) => a + b);
  return (
    rataRata: total / nilai.length,
    tertinggi: urut.last,
    terendah: urut.first,
  );
}

void main() {
  final s = hitungStatistik([80, 65, 90, 75]);
  print('Rata-rata: ${s.rataRata}'); // Output: Rata-rata: 77.5
  print('Tertinggi: ${s.tertinggi}'); // Output: Tertinggi: 90
  print('Terendah: ${s.terendah}'); // Output: Terendah: 65

  // Langsung dibongkar (destructuring) ke variabel
  final (:rataRata, :tertinggi, :terendah) = hitungStatistik([60, 70]);
  print('$rataRata $tertinggi $terendah'); // Output: 65.0 70 60
}
```

`[...nilai]..sort()` menyalin list dulu sebelum diurutkan agar list asli tidak berubah. `:rataRata` adalah singkatan dari `rataRata: rataRata`.

Destructuring

Destructuring berarti membongkar isi record, List, atau Map langsung ke beberapa variabel.

```
void main() {
  var (nama, ipk) = ('Bima', 3.6);
  print('$nama: $ipk'); // Output: Bima: 3.6

  // Menukar dua variabel tanpa variabel bantu
  var a = 1;
  var b = 2;
  (a, b) = (b, a);
  print('a=$a, b=$b'); // Output: a=2, b=1

  // List pattern dengan rest element (...)
  var [pertama, kedua, ...sisa] = [10, 20, 30, 40];
  print('$pertama $kedua $sisa'); // Output: 10 20 [30, 40]

  // Destructuring di for-in: Map entries
  final nilai = {'Ayu': 90, 'Bima': 78};
  for (final MapEntry(:key, :value) in nilai.entries) {
    print('$key mendapat $value');
  }
  // Output:
  // Ayu mendapat 90
  // Bima mendapat 78

  // .indexed menghasilkan record (indeks, elemen)
  for (final (i, mk) in ['Mobile', 'Web'].indexed) {
    print('${i + 1}. $mk');
  }
  // Output:
  // 1. Mobile
  // 2. Web
}
```

Pattern matching di switch

Pattern di `switch` bisa mencocokkan **rentang nilai** (relational pattern), **tipe**, **bentuk List**, dan **record**, ditambah syarat tambahan dengan `when` (guard).

```
String kategoriNilai(int nilai) => switch (nilai) {
      >= 85 && <= 100 => 'Sangat baik',
      >= 70 && < 85 => 'Baik',
      >= 60 && < 70 => 'Cukup',
      >= 0 && < 60 => 'Kurang',
      _ => 'Nilai tidak valid',
    };

void main() {
  print(kategoriNilai(91)); // Output: Sangat baik
  print(kategoriNilai(70)); // Output: Baik
  print(kategoriNilai(59)); // Output: Kurang
```

```
  print(kategoriNilai(120)); // Output: Nilai tidak valid
}
```

```
String jelaskan(Object data) => switch (data) {
      int n when n < 0 => 'Bilangan negatif $n',
      int n => 'Bilangan bulat $n',
      double d => 'Desimal ${d.toStringAsFixed(1)}',
      String s when s.isEmpty => 'Teks kosong',
      String s => 'Teks "$s" (${s.length} huruf)',
      [] => 'List kosong',
      [var x] => 'List berisi satu: $x',
      [var x, ...] => 'List diawali $x',
      (String nama, int umur) => 'Record: $nama umur $umur',
      _ => 'Tipe lain: ${data.runtimeType}',
    };

void main() {
  print(jelaskan(-5)); // Output: Bilangan negatif -5
  print(jelaskan(42)); // Output: Bilangan bulat 42
  print(jelaskan(3.14159)); // Output: Desimal 3.1
  print(jelaskan('')); // Output: Teks kosong
  print(jelaskan('Dart')); // Output: Teks "Dart" (4 huruf)
  print(jelaskan([])); // Output: List kosong
  print(jelaskan([7])); // Output: List berisi satu: 7
  print(jelaskan([1, 2, 3])); // Output: List diawali 1
  print(jelaskan(('Ayu', 20))); // Output: Record: Ayu umur 20
  print(jelaskan(true)); // Output: Tipe lain: bool
}
```

Cabang dicek dari atas ke bawah. `int n when n < 0` artinya "jika data bertipe int, simpan sebagai `n`, *dan* nilainya negatif". Karena itu cabang yang lebih spesifik harus ditulis lebih dulu.

if-case: validasi bentuk data

Pattern juga bisa dipakai di `if` dengan kata kunci `case`. Ini sangat praktis untuk memeriksa JSON: cek kunci yang wajib ada *sekaligus* tipenya, lalu langsung ambil nilainya.

```
void prosesJson(Map<String, dynamic> json) {
  if (json case {'nama': String nama, 'ipk': double ipk}) {
    print('Valid: $nama, IPK $ipk');
  } else {
    print('Format JSON tidak sesuai: $json');
  }
}

void main() {
  prosesJson({'nama': 'Ayu', 'ipk': 3.8}); // Output: Valid: Ayu, IPK 3.8
  prosesJson({'nama': 'Bima'}); // Output: Format JSON tidak sesuai: {nama: Bima}
  prosesJson({'nama': 'Citra', 'ipk': '3.5'}); // Output: Format JSON tidak sesuai: {nama: Citra,
ipk: 3.5}
}
```

Data ketiga ditolak karena `'3.5'` adalah String, bukan double. Tanpa pattern, kita harus menulis beberapa `if` dan `is` secara manual.

Object pattern

Object pattern mencocokkan objek berdasarkan nilai getter/field-nya.

```
class Mahasiswa {
  final String nama;
  final int semester;
  final double ipk;
  const Mahasiswa(this.nama, this.semester, this.ipk);
}

String evaluasi(Mahasiswa m) => switch (m) {
      Mahasiswa(semester: > 14) => '${m.nama}: melewati batas masa studi',
      Mahasiswa(:var nama, ipk: >= 3.5) => '$nama: kandidat cumlaude',
      Mahasiswa(:var nama, :var ipk) when ipk < 2.0 => '$nama: peringatan akademik',
      Mahasiswa(:var nama) => '$nama: normal',
    };

void main() {
  print(evaluasi(const Mahasiswa('Ayu', 5, 3.8))); // Output: Ayu: kandidat cumlaude
  print(evaluasi(const Mahasiswa('Bima', 15, 3.0))); // Output: Bima: melewati batas masa studi
  print(evaluasi(const Mahasiswa('Citra', 4, 1.9))); // Output: Citra: peringatan akademik
  print(evaluasi(const Mahasiswa('Dodi', 6, 3.1))); // Output: Dodi: normal
}
```

Sealed class dan exhaustive switch

`sealed class` adalah abstract class yang turunannya hanya boleh ditulis di file yang sama. Karena compiler tahu **semua** turunannya, switch terhadap sealed class tidak perlu cabang `_`, dan compiler akan memberi error jika ada turunan yang terlewat. Ini disebut *exhaustive checking*.

```
sealed class HasilLogin {}

class LoginBerhasil extends HasilLogin {
  final String nama;
  LoginBerhasil(this.nama);
}

class PasswordSalah extends HasilLogin {
  final int sisaPercobaan;
  PasswordSalah(this.sisaPercobaan);
}

class AkunDiblokir extends HasilLogin {}

HasilLogin login(String username, String password) {
  if (username == 'admin') return AkunDiblokir();
  if (password == 'rahasia') return LoginBerhasil(username);
  return PasswordSalah(2);
}

String pesanUntuk(HasilLogin hasil) => switch (hasil) {
      LoginBerhasil(:final nama) => 'Selamat datang, $nama!',
      PasswordSalah(:final sisaPercobaan) =>
        'Password salah. Sisa percobaan: $sisaPercobaan',
      AkunDiblokir() => 'Akun diblokir, hubungi admin.',
    };
```

```
void main() {
  print(pesanUntuk(login('ayu', 'rahasia'))); // Output: Selamat datang, ayu!
  print(pesanUntuk(login('ayu', '12345'))); // Output: Password salah. Sisa percobaan: 2
  print(pesanUntuk(login('admin', 'apa saja'))); // Output: Akun diblokir, hubungi admin.
}
```

Coba hapus baris `AkunDiblokir() => ...`, lalu jalankan `dart analyze`. Compiler akan menolak dengan pesan bahwa tipe `HasilLogin` tidak tertangani secara lengkap (*not exhaustively matched*).

Pola yang sama sangat cocok untuk status pemuatan data di aplikasi (Memuat, Sukses, Gagal), yang akan Anda pakai bersama Provider dan REST API:

```
sealed class StatusData {}

class Memuat extends StatusData {}

class Sukses extends StatusData {
  final List<String> data;
  Sukses(this.data);
}

class Gagal extends StatusData {
  final String pesan;
  Gagal(this.pesan);
}

String tampilan(StatusData status) => switch (status) {
      Memuat() => 'Menampilkan loading...',
      Sukses(data: []) => 'Belum ada data',
      Sukses(:final data) => 'Menampilkan ${data.length} item',
      Gagal(:final pesan) => 'Error: $pesan',
    };

void main() {
  print(tampilan(Memuat())); // Output: Menampilkan loading...
  print(tampilan(Sukses([]))); // Output: Belum ada data
  print(tampilan(Sukses(['Ayu', 'Bima']))); // Output: Menampilkan 2 item
  print(tampilan(Gagal('Koneksi terputus'))); // Output: Error: Koneksi terputus
}
```

Di Flutter, fungsi seperti `tampilan()` akan mengembalikan widget (misalnya `CircularProgressIndicator()` atau `ListView`), bukan String.

Class modifier (ringkas)

Dart 3 menambahkan kata kunci untuk mengatur bagaimana class boleh dipakai **dari file/library lain**. Di dalam file yang sama, batasan ini tidak berlaku.

Modifier	Boleh dibuat objek?	Boleh `extends` ?	Boleh `implements` ?	Kegunaan
`abstract`	Tidak	Ya	Ya	Kerangka dengan method abstrak
`base`	Ya	Ya	Tidak	Wajib mewarisi implementasi asli
`interface`	Ya	Tidak	Ya	Murni kontrak
`final`	Ya	Tidak	Tidak	Class tertutup, tidak bisa diturunkan
`sealed`	Tidak	Hanya di file sama	Hanya di file sama	Exhaustive switch
`mixin class`	Ya	Ya	Ya	Bisa dipakai sebagai class maupun mixin

Kombinasi yang paling sering dipakai di aplikasi adalah `abstract interface class` untuk kontrak *repository*, sehingga sumber data mudah diganti (misalnya data palsu untuk testing di Modul 15):

```dart
abstract interface class MahasiswaRepository {
  List<String> ambilSemuaNama();
}

final class RepositoryMemori implements MahasiswaRepository {
  final _data = ['Ayu', 'Bima', 'Citra'];

  @override
  List<String> ambilSemuaNama() => List.unmodifiable(_data);
}

void tampilkan(MahasiswaRepository repo) {
  print(repo.ambilSemuaNama().join(', '));
}

void main() {
  tampilkan(RepositoryMemori()); // Output: Ayu, Bima, Citra
}
```

💡 **Ringkasan**

- **Record**: kembalikan beberapa nilai sekaligus tanpa membuat class.
- **Destructuring**: `var (a, b) = ...`, `var [x, ...sisa] = ...`, `for (final (i, e) in list.indexed)`.
- **Pattern** di switch dan `if-case`: cocokkan rentang, tipe, dan bentuk data.
- **Sealed class**: kumpulan status yang terbatas, dan compiler memastikan semuanya ditangani.

10. Error Handling: try, catch & Exception Kustom

Program yang baik tidak langsung "mati" ketika terjadi masalah, seperti input yang salah, data rusak, atau server tidak merespons. Dart menyediakan `try`, `catch`, `on`, `finally`, dan `throw` untuk menangani situasi tersebut dengan rapi.

try dan catch

```
void main() {
  try {
    int angka = int.parse('12a'); // melempar FormatException
    print('Angka: $angka'); // baris ini dilewati
  } catch (e) {
    print('Terjadi error: ${e.runtimeType}');
  }
  print('Program tetap berjalan');
  // Output:
  // Terjadi error: FormatException
  // Program tetap berjalan
}
```

Begitu terjadi error di dalam `try`, sisa baris di blok itu dilewati dan eksekusi melompat ke `catch`. Setelah itu, program berlanjut normal. Tanpa `try/catch`, program akan berhenti dengan pesan *Unhandled exception*.

on, catch, finally, dan throw

```
double bagi(int a, int b) {
  if (b == 0) {
    throw ArgumentError('Pembagi tidak boleh nol');
  }
  return a / b;
}

void main() {
  for (final input in ['10', 'x', '0']) {
    try {
      final b = int.parse(input);
      print('100 / $b = ${bagi(100, b)}');
    } on FormatException {
      print('"$input" bukan angka');
    } on ArgumentError catch (e) {
      print('Argumen salah: ${e.message}');
    } finally {
      print('-- selesai memproses "$input"');
    }
  }
  // Output:
  // 100 / 10 = 10.0
  // -- selesai memproses "10"
  // "x" bukan angka
  // -- selesai memproses "x"
  // Argumen salah: Pembagi tidak boleh nol
```

```
  // -- selesai memproses "0"
}
```

- `on TipeError` menangkap jenis error tertentu saja. Tambahkan `catch (e)` jika butuh objek error-nya.
- `finally` **selalu** dijalankan, baik terjadi error maupun tidak. Cocok untuk menutup koneksi atau mematikan indikator loading.
- `throw` melempar error secara sengaja ketika kondisi tidak valid.
- `catch (e, stackTrace)` memberikan parameter kedua berisi jejak pemanggilan, berguna untuk debugging.

Membuat Exception sendiri

Exception kustom membuat jenis error lebih jelas dan bisa membawa data tambahan. Cukup buat class yang meng-`implements Exception`.

```dart
class SaldoTidakCukupException implements Exception {
  final double saldo;
  final double diminta;

  SaldoTidakCukupException(this.saldo, this.diminta);

  @override
  String toString() => 'SaldoTidakCukupException: saldo $saldo, diminta $diminta';
}

class Dompet {
  double saldo;
  Dompet(this.saldo);

  void bayar(double jumlah) {
    if (jumlah <= 0) {
      throw ArgumentError.value(jumlah, 'jumlah', 'harus lebih dari 0');
    }
    if (jumlah > saldo) {
      throw SaldoTidakCukupException(saldo, jumlah);
    }
    saldo -= jumlah;
  }
}

void main() {
  final dompet = Dompet(50000);
  try {
    dompet.bayar(20000);
    print('Bayar 20000 berhasil, sisa ${dompet.saldo}');
    dompet.bayar(40000);
    print('Baris ini tidak dijalankan');
  } on SaldoTidakCukupException catch (e) {
    print(e);
    print('Kurang: ${e.diminta - e.saldo}');
  }
  // Output:
  // Bayar 20000 berhasil, sisa 30000.0
```

```
  // SaldoTidakCukupException: saldo 30000.0, diminta 40000.0
  // Kurang: 10000.0
}
```

rethrow: mencatat lalu meneruskan error

```
int bacaNilai(String input) {
  try {
    return int.parse(input);
  } on FormatException {
    print('[log] input tidak valid: $input');
    rethrow; // lempar ulang ke pemanggil
  }
}

void main() {
  try {
    bacaNilai('sembilan');
  } catch (e) {
    print('Ditangani di main: ${e.runtimeType}');
  }
  // Output:
  // [log] input tidak valid: sembilan
  // Ditangani di main: FormatException
}
```

`rethrow` berguna ketika sebuah fungsi ingin mencatat error (logging) tetapi keputusan penanganannya, misalnya menampilkan pesan ke pengguna, tetap diserahkan ke pemanggil.

Exception vs Error

	`Exception`	`Error`
Arti	Kondisi tidak normal yang **bisa diantisipasi**	**Bug** di dalam kode
Contoh	`FormatException`, `TimeoutException`, gagal koneksi	`RangeError`, `TypeError`, `LateInitializationError`
Sikap	Tangkap dan tangani (tampilkan pesan, coba lagi)	Perbaiki kodenya, jangan ditangkap diam-diam

⚠️ **Kesalahan umum**

- `catch (e) {}` kosong: error "ditelan" sehingga bug sulit dilacak. Minimal cetak atau catat error-nya.
- Menangkap semua error dengan `catch` padahal yang diharapkan hanya satu jenis. Gunakan `on TipeException` agar lebih spesifik.
- Melempar String (`throw 'gagal'`). Hal ini memang diizinkan, tetapi lebih baik melempar objek `Exception` agar bisa ditangkap berdasarkan tipenya.

11. Pemrograman Asinkron: Future, async/await & Stream

Aplikasi mobile sering harus menunggu: mengambil data dari internet, membaca database, atau menunggu input. Jika program "diam menunggu", layar akan membeku. Dart memakai model *single-threaded event loop* dengan `Future` dan `Stream` agar pekerjaan yang lama berjalan tanpa memblokir tampilan.

Kode sinkron vs asinkron

```
void main() {
  print('1. Mulai');
  Future.delayed(const Duration(seconds: 1), () {
    print('3. Data selesai diambil (setelah 1 detik)');
  });
  print('2. Lanjut mengerjakan hal lain');
  // Output:
  // 1. Mulai
  // 2. Lanjut mengerjakan hal lain
  // 3. Data selesai diambil (setelah 1 detik)
}
```

`Future.delayed` "menjadwalkan" pekerjaan untuk nanti, lalu Dart langsung lanjut ke baris berikutnya. Itulah sebabnya baris 2 tercetak sebelum baris 3. `Future` adalah janji bahwa sebuah nilai (atau error) akan tersedia **di masa depan**.

async dan await

```
Future<String> ambilNamaMahasiswa(String nim) async {
  await Future.delayed(const Duration(milliseconds: 500)); // simulasi jaringan
  return 'Ayu Lestari';
}

Future<void> main() async {
  print('Mengambil data...');
  String nama = await ambilNamaMahasiswa('H1D023001');
  print('Nama: $nama');
  print('Selesai');
  // Output:
  // Mengambil data...
  // Nama: Ayu Lestari
  // Selesai
}
```

- `async` menandai fungsi asinkron. Tipe kembaliannya dibungkus `Future`: `return 'Ayu'` menjadi `Future<String>`.
- `await` menunggu Future selesai lalu mengambil nilainya. `await` hanya boleh dipakai di dalam fungsi `async`.
- Selama menunggu, event loop bebas mengerjakan hal lain, misalnya menggambar ulang layar di Flutter.

then() vs await

```dart
Future<int> ambilJumlahSks() {
  return Future.delayed(const Duration(milliseconds: 300), () => 21);
}

Future<void> main() async {
  // Gaya 1: callback then()
  await ambilJumlahSks().then((sks) => print('then(): $sks SKS'));

  // Gaya 2: async/await (lebih mudah dibaca)
  final sks = await ambilJumlahSks();
  print('await: $sks SKS');
  // Output:
  // then(): 21 SKS
  // await: 21 SKS
}
```

Keduanya setara. `async/await` lebih disarankan karena kode terbaca berurutan dari atas ke bawah, terutama jika ada beberapa langkah yang saling bergantung.

Menangani error pada kode async

```dart
import 'dart:async';

Future<double> ambilIpk(String nim) async {
  await Future.delayed(const Duration(milliseconds: 300));
  if (!nim.startsWith('H1D')) {
    throw Exception('NIM $nim tidak ditemukan');
  }
  return 3.67;
}

Future<void> main() async {
  for (final nim in ['H1D023001', 'X999']) {
    try {
      final ipk = await ambilIpk(nim);
      print('$nim: IPK $ipk');
    } catch (e) {
      print('Gagal: $e');
    }
  }

  // Membatasi waktu tunggu
  try {
    await ambilIpk('H1D023001').timeout(const Duration(milliseconds: 100));
  } on TimeoutException {
    print('Server terlalu lama merespons');
  }
  // Output:
  // H1D023001: IPK 3.67
  // Gagal: Exception: NIM X999 tidak ditemukan
  // Server terlalu lama merespons
}
```

Dengan `await`, error dari Future bisa ditangkap memakai `try/catch` biasa, persis seperti kode sinkron. `.timeout()` melempar `TimeoutException` (dari `dart:async`) jika Future tidak selesai dalam batas waktu.

Menjalankan beberapa Future secara paralel

Jika beberapa permintaan tidak saling bergantung, jangan di-`await` satu per satu. Jalankan bersamaan dengan `Future.wait`.

```
Future<int> ambilNilai(String matkul) async {
  await Future.delayed(const Duration(milliseconds: 400));
  return switch (matkul) {
    'Mobile' => 88,
    'Web' => 79,
    _ => 70,
  };
}

Future<void> main() async {
  final nilai = await Future.wait([
    ambilNilai('Mobile'),
    ambilNilai('Web'),
    ambilNilai('AI'),
  ]);
  print(nilai); // Output: [88, 79, 70]
}
```

Ketiga permintaan berjalan bersamaan sehingga total waktunya sekitar 0,4 detik, bukan 1,2 detik. Urutan hasil selalu sama dengan urutan Future di dalam list.

Jika tipe hasilnya berbeda-beda, Dart 3 menyediakan `.wait` pada record:

```
Future<String> ambilProfil() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return 'Profil Ayu';
}

Future<List<String>> ambilJadwal() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return ['Mobile', 'Web'];
}

Future<int> ambilNotifikasi() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return 3;
}

Future<void> main() async {
  final stopwatch = Stopwatch()..start();

  // Berurutan: total sekitar 1,5 detik
  final profil = await ambilProfil();
  final jadwal = await ambilJadwal();
  final notif = await ambilNotifikasi();
  print('Berurutan: $profil, $jadwal, $notif notifikasi');
```

```
  print('Lebih dari 1,4 detik? ${stopwatch.elapsedMilliseconds > 1400}');

  stopwatch.reset();

  // Paralel dengan record .wait: total sekitar 0,5 detik
  final (p, j, n) = await (ambilProfil(), ambilJadwal(), ambilNotifikasi()).wait;
  print('Paralel: $p, $j, $n notifikasi');
  print('Kurang dari 1 detik? ${stopwatch.elapsedMilliseconds < 1000}');
  // Output:
  // Berurutan: Profil Ayu, [Mobile, Web], 3 notifikasi
  // Lebih dari 1,4 detik? true
  // Paralel: Profil Ayu, [Mobile, Web], 3 notifikasi
  // Kurang dari 1 detik? true
}
```

Stream: rangkaian data dari waktu ke waktu

Jika `Future` menghasilkan **satu** nilai, `Stream` menghasilkan **banyak** nilai yang datang bergantian, misalnya pesan chat, posisi GPS, atau perubahan data realtime dari Supabase (Modul 12).

```
Future<void> main() async {
  final stream = Stream.fromIterable(['Ayu', 'Bima', 'Citra']);

  await for (final nama in stream) {
    print('Hadir: $nama');
  }
  print('Absensi selesai');
  // Output:
  // Hadir: Ayu
  // Hadir: Bima
  // Hadir: Citra
  // Absensi selesai
}
```

`await for` menunggu setiap data yang datang, lalu keluar dari perulangan ketika stream selesai.

Membuat Stream dengan `async*` dan `yield`

```
Stream<int> hitungMundur(int dari) async* {
  for (var i = dari; i >= 1; i--) {
    await Future.delayed(const Duration(milliseconds: 300));
    yield i; // kirim satu nilai ke pendengar
  }
}

Future<void> main() async {
  await for (final detik in hitungMundur(3)) {
    print('$detik...');
  }
  print('Ujian dimulai!');

  // Stream juga punya where, map, take, toList seperti List
  final hasil = await Stream.fromIterable([1, 2, 3, 4, 5, 6])
      .where((n) => n.isEven)
      .map((n) => n * 10)
      .toList();
```

```
  print(hasil);
  // Output:
  // 3...
  // 2...
  // 1...
  // Ujian dimulai!
  // [20, 40, 60]
}
```

Fungsi `async*` mengembalikan `Stream`. Setiap `yield` mengirim satu nilai tanpa mengakhiri fungsi, berbeda dengan `return` yang mengakhiri fungsi.

listen() dan StreamController

`StreamController` dipakai ketika kita ingin "mengirim" data ke stream secara manual, misalnya setiap kali isi keranjang belanja berubah.

```
import 'dart:async';

class Keranjang {
  final _controller = StreamController<int>();
  int _jumlah = 0;

  Stream<int> get jumlahStream => _controller.stream;

  void tambah() {
    _jumlah++;
    _controller.add(_jumlah); // kirim data baru ke stream
  }

  Future<void> tutup() => _controller.close();
}

Future<void> main() async {
  final keranjang = Keranjang();

  keranjang.jumlahStream.listen(
    (jumlah) => print('Isi keranjang: $jumlah'),
    onDone: () => print('Stream ditutup'),
  );

  keranjang.tambah();
  keranjang.tambah();
  keranjang.tambah();
  await keranjang.tutup(); // tunggu sampai semua data terkirim
  // Output:
  // Isi keranjang: 1
  // Isi keranjang: 2
  // Isi keranjang: 3
  // Stream ditutup
}
```

`listen()` mendaftarkan fungsi yang dipanggil setiap ada data baru. Secara default, stream hanya boleh punya satu pendengar; gunakan `StreamController.broadcast()` jika butuh banyak

pendengar. Di Flutter, `StreamBuilder` melakukan `listen` secara otomatis dan membangun ulang widget setiap kali data baru datang.

	`Future<T>`	`Stream<T>`
Jumlah nilai	Satu kali	Nol sampai banyak
Cara menunggu	`await`	`await for` / `listen()`
Membuat fungsi	`async` + `return`	`async*` + `yield`
Widget Flutter	`FutureBuilder`	`StreamBuilder`
Contoh	Login, GET data API	Chat, lokasi GPS, data realtime

> ⚠️ **Kesalahan umum**
>
> - Lupa `await`: variabel berisi `Instance of 'Future<String>'`, bukan datanya.
> - Memakai `await` di fungsi yang tidak ditandai `async`, sehingga terjadi error compile.
> - Meng-`await` satu per satu permintaan yang sebenarnya bisa paralel. Gunakan `Future.wait` atau record `.wait`.
> - Lupa menutup `StreamController` atau membatalkan `StreamSubscription` (di Flutter, lakukan di `dispose()`), sehingga terjadi kebocoran memori.

12. Studi Kasus: Sistem Nilai & IPK Mahasiswa

Saatnya menggabungkan semua yang sudah dipelajari menjadi satu program utuh: **Sistem Nilai dan IPK Mahasiswa**. Program ini mengambil data KHS dari "server" (simulasi), memvalidasi data yang rusak, lalu menghitung IPK, predikat, dan mata kuliah yang perlu diulang.

Spesifikasi

- Setiap mata kuliah punya kode, nama, SKS (1 sampai 6), dan nilai angka (0 sampai 100).
- Nilai angka dikonversi ke nilai huruf dan bobot: A=4.0, AB=3.5, B=3.0, BC=2.5, C=2.0, D=1.0, E=0.0. Nilai di bawah C wajib diulang.
- IPK = jumlah (SKS × bobot) dibagi total SKS.
- Predikat: ≥ 3.51 Dengan Pujian, ≥ 3.01 Sangat Memuaskan, ≥ 2.76 Memuaskan, selain itu Cukup.
- Data dari server bisa rusak. Data yang rusak dilewati dan dilaporkan sebagai peringatan, tanpa membuat program berhenti.

Kode lengkap

Simpan sebagai satu file lalu jalankan dengan `dart run sistem_nilai.dart`. Komentar bernomor menandai bagian-bagian program.

sistem_nilai.dart

```
// ===== 1. ENUM: skala nilai huruf (enhanced enum) =====
enum NilaiHuruf {
  a('A', 4.0, 85),
  ab('AB', 3.5, 80),
  b('B', 3.0, 70),
  bc('BC', 2.5, 65),
  c('C', 2.0, 60),
  d('D', 1.0, 50),
  e('E', 0.0, 0);

  const NilaiHuruf(this.label, this.bobot, this.batasBawah);

  final String label;
  final double bobot;
  final int batasBawah;

  bool get lulus => bobot >= 2.0;

  static NilaiHuruf dariAngka(double nilai) =>
      values.firstWhere((h) => nilai >= h.batasBawah);
}

// ===== 2. EXCEPTION kustom =====
class DataNilaiException implements Exception {
  final String pesan;
  DataNilaiException(this.pesan);
```

```dart
  @override
  String toString() => 'DataNilaiException: $pesan';
}

// ===== 3. CLASS MataKuliah (constructor, validasi, factory, getter) =====
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final double nilaiAngka;

  MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.nilaiAngka,
  }) {
    if (sks < 1 || sks > 6) {
      throw DataNilaiException('SKS $kode harus 1-6');
    }
    if (nilaiAngka < 0 || nilaiAngka > 100) {
      throw DataNilaiException('Nilai $kode di luar rentang 0-100');
    }
  }

  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    if (json
        case {
          'kode': String kode,
          'nama': String nama,
          'sks': int sks,
          'nilai': num nilai,
        }) {
      return MataKuliah(
        kode: kode,
        nama: nama,
        sks: sks,
        nilaiAngka: nilai.toDouble(),
      );
    }
    throw DataNilaiException('Format data ${json['kode']} tidak sesuai');
  }

  NilaiHuruf get huruf => NilaiHuruf.dariAngka(nilaiAngka);
  double get mutu => sks * huruf.bobot;
}

// ===== 4. EXTENSION =====
extension FormatAngka on double {
  String get duaDesimal => toStringAsFixed(2);
}

// ===== 5. SEALED CLASS: hasil pemuatan data =====
sealed class HasilMuat {}

class MuatBerhasil extends HasilMuat {
  final List<MataKuliah> data;
  final List<String> peringatan;
```

```
  MuatBerhasil(this.data, this.peringatan);
}

class MuatGagal extends HasilMuat {
  final String pesan;
  MuatGagal(this.pesan);
}

// ===== 6. CLASS Mahasiswa (private, getter, koleksi, record) =====
class Mahasiswa {
  final String nim;
  final String nama;
  final List<MataKuliah> _khs = [];

  Mahasiswa(this.nim, this.nama);

  List<MataKuliah> get khs => List.unmodifiable(_khs);

  void tambahSemua(List<MataKuliah> daftar) => _khs.addAll(daftar);

  int get totalSks => _khs.fold(0, (total, mk) => total + mk.sks);

  double get ipk {
    if (totalSks == 0) return 0;
    final totalMutu = _khs.fold(0.0, (total, mk) => total + mk.mutu);
    return totalMutu / totalSks;
  }

  String get predikat => switch (ipk) {
        >= 3.51 => 'Dengan Pujian',
        >= 3.01 => 'Sangat Memuaskan',
        >= 2.76 => 'Memuaskan',
        _ => 'Cukup',
      };

  List<MataKuliah> get perluMengulang =>
      _khs.where((mk) => !mk.huruf.lulus).toList();

  ({MataKuliah terbaik, MataKuliah terendah}) get ringkasan {
    final urut = [..._khs]..sort((a, b) => b.nilaiAngka.compareTo(a.nilaiAngka));
    return (terbaik: urut.first, terendah: urut.last);
  }
}

// ===== 7. ASYNC: simulasi mengambil data dari server =====
Future<HasilMuat> muatKhsDariServer(String nim) async {
  await Future.delayed(const Duration(milliseconds: 500));
  if (nim.isEmpty) return MuatGagal('NIM kosong');

  final dataJson = <Map<String, dynamic>>[
    {'kode': 'IF201', 'nama': 'Pemrograman Mobile', 'sks': 3, 'nilai': 88},
    {'kode': 'IF202', 'nama': 'Basis Data', 'sks': 3, 'nilai': 76.5},
    {'kode': 'IF203', 'nama': 'Jaringan Komputer', 'sks': 2, 'nilai': 64},
    {'kode': 'IF204', 'nama': 'Kecerdasan Buatan', 'sks': 3, 'nilai': 81},
    {'kode': 'IF205', 'nama': 'Etika Profesi', 'sks': 2, 'nilai': 55},
    {'kode': 'IF206', 'nama': 'Statistika', 'sks': 3, 'nilai': 'A'}, // rusak
    {'kode': 'IF207', 'nama': 'Kerja Praktik', 'sks': 8, 'nilai': 90}, // rusak
  ];
```

```dart
  final data = <MataKuliah>[];
  final peringatan = <String>[];
  for (final json in dataJson) {
    try {
      data.add(MataKuliah.fromJson(json));
    } on DataNilaiException catch (e) {
      peringatan.add(e.pesan);
    }
  }
  return MuatBerhasil(data, peringatan);
}

// ===== 8. MAIN =====
Future<void> main() async {
  final mhs = Mahasiswa('H1D023001', 'Ayu Lestari');
  print('Memuat KHS ${mhs.nama}...');

  final hasil = await muatKhsDariServer(mhs.nim);
  switch (hasil) {
    case MuatGagal(:final pesan):
      print('Gagal memuat: $pesan');
      return;
    case MuatBerhasil(:final data, :final peringatan):
      mhs.tambahSemua(data);
      for (final p in peringatan) {
        print('Peringatan: $p');
      }
  }

  print('');
  print('KHS ${mhs.nim} - ${mhs.nama}');
  for (final (i, mk) in mhs.khs.indexed) {
    print('${i + 1}. ${mk.kode} ${mk.nama.padRight(20)}'
        '${mk.sks} SKS  ${mk.nilaiAngka.toString().padLeft(4)}  ${mk.huruf.label}');
  }

  final (:terbaik, :terendah) = mhs.ringkasan;
  print('');
  print('Total SKS   : ${mhs.totalSks}');
  print('IPK         : ${mhs.ipk.duaDesimal}');
  print('Predikat    : ${mhs.predikat}');
  print('Terbaik     : ${terbaik.nama} (${terbaik.huruf.label})');
  print('Terendah    : ${terendah.nama} (${terendah.huruf.label})');
  print('Mengulang   : ${mhs.perluMengulang.map((mk) => mk.nama).join(', ')}');
}
```

Output

```
Memuat KHS Ayu Lestari...
Peringatan: Format data IF206 tidak sesuai
Peringatan: SKS IF207 harus 1-6

KHS H1D023001 - Ayu Lestari
1. IF201 Pemrograman Mobile  3 SKS  88.0  A
2. IF202 Basis Data          3 SKS  76.5  B
3. IF203 Jaringan Komputer   2 SKS  64.0  C
4. IF204 Kecerdasan Buatan   3 SKS  81.0  AB
5. IF205 Etika Profesi       2 SKS  55.0  D
```

```
Total SKS   : 13
IPK         : 2.88
Predikat    : Memuaskan
Terbaik     : Pemrograman Mobile (A)
Terendah    : Etika Profesi (D)
Mengulang   : Etika Profesi
```

Penjelasan per bagian

Bagian	Konsep yang dipakai	Section
1. `NilaiHuruf`	Enhanced enum dengan field, getter `lulus`, dan method static `dariAngka` memakai `firstWhere`	8, 6
2. `DataNilaiException`	Exception kustom dengan `implements Exception` dan override `toString()`	10
3. `MataKuliah`	Named parameter `required`, validasi di body constructor dengan `throw`, factory `fromJson` dengan `if-case` pattern, getter terhitung	5, 8, 9
4. `FormatAngka`	Extension method pada `double`	8
5. `HasilMuat`	Sealed class dan exhaustive switch di `main()` dengan object pattern `:final pesan`	9
6. `Mahasiswa`	Field private `_khs`, `List.unmodifiable`, `fold`, `where`, switch expression dengan relational pattern, getter yang mengembalikan record	6, 8, 9
7. `muatKhsDariServer`	`async` / `await`, `Future.delayed`, `try` / `on ... catch` di dalam perulangan	10, 11
8. `main()`	`Future<void> main() async`, destructuring `.indexed` dan record, interpolasi, `padRight` / `padLeft`, `map` + `join`	2, 4, 6, 9

Beberapa hal yang perlu diperhatikan:

- **Data rusak tidak membuat program berhenti.** `IF206` ditolak oleh pattern di `fromJson` karena nilainya String, sedangkan `IF207` lolos pattern tetapi ditolak validasi SKS di constructor. Keduanya melempar `DataNilaiException` yang ditangkap di dalam perulangan, sehingga data lain tetap diproses.
- **Perhitungan IPK**: total mutu = 3×4.0 + 3×3.0 + 2×2.0 + 3×3.5 + 2×1.0 = 37.5. Dibagi 13 SKS menghasilkan 2.8846..., yang dibulatkan menjadi 2.88 oleh extension `duaDesimal`.

- `fold(0.0, ...)` sengaja diberi nilai awal `0.0` (double), karena `mutu` bertipe double. Jika ditulis `0`, Dart menganggap akumulatornya `int` dan terjadi error tipe.
- **Getter** `khs` mengembalikan list yang tidak bisa diubah. Kode di luar class tidak bisa menambah mata kuliah tanpa lewat method `tambahSemua`.

Bonus: progres sinkronisasi dengan Stream

Fitur tambahan yang sering muncul di aplikasi: menampilkan persentase progres saat data dikirim ke server. Ini cocok dibuat dengan `async*`.

```
Stream<int> sinkronkanKrs(int jumlahMatkul) async* {
  for (var i = 1; i <= jumlahMatkul; i++) {
    await Future.delayed(const Duration(milliseconds: 200)); // kirim satu matkul
    yield i * 100 ~/ jumlahMatkul; // persentase selesai
  }
}

Future<void> main() async {
  await for (final persen in sinkronkanKrs(4)) {
    print('Menyinkronkan... $persen%');
  }
  print('Data KRS Ayu Lestari berhasil disinkronkan');
  // Output:
  // Menyinkronkan... 25%
  // Menyinkronkan... 50%
  // Menyinkronkan... 75%
  // Menyinkronkan... 100%
  // Data KRS Ayu Lestari berhasil disinkronkan
}
```

Di Flutter, stream seperti ini bisa dihubungkan ke `StreamBuilder` untuk menggerakkan `LinearProgressIndicator`.

> 💡 **Latihan pengembangan (mandiri)**
>
> 1. Tambahkan field `semester` pada `MataKuliah`, lalu hitung **IPS** (indeks prestasi per semester) memakai `where`.
> 2. Jika satu mata kuliah diambil dua kali, gunakan nilai terbaik saja saat menghitung IPK. Petunjuk: kelompokkan berdasarkan `kode` dengan `Map`.
> 3. Buat `muatKhsDariServer` mengembalikan `MuatGagal` secara acak (gunakan `Random` dari `dart:math`), lalu pastikan pesan gagal tampil dengan benar.
> 4. Tambahkan method `toJson()` pada `MataKuliah` dan cetak seluruh KHS dalam format JSON memakai `jsonEncode` dari `dart:convert`.

Tugas Praktikum: Praktikum 2: Sistem Pengelolaan Nilai & IPK Mahasiswa dengan Dart 3

Sasaran Praktikum

Menerapkan seluruh dasar Dart (variabel, control flow, fungsi, koleksi, null safety, OOP, fitur Dart 3, error handling, dan async) secara bertahap dalam program console pengelolaan nilai mahasiswa.

Langkah Pengerjaan

1. **Persiapan.** Buat proyek dengan `dart create -t console praktikum_dart`. Kerjakan setiap tugas di file terpisah di folder `bin/` (misalnya `bin/tugas1.dart`) dan jalankan dengan `dart run bin/tugas1.dart`. Pastikan `dart analyze` tidak menampilkan error.
2. **Tugas 1 - Variabel & Control Flow (10%).** Simpan daftar nilai `[92, 78, 65, 54, 81, 47]` dalam `List<int>`. Dengan perulangan `for-in` dan *switch expression* ber-relational pattern, cetak setiap nilai beserta huruf mutunya (A/AB/B/BC/C/D/E). Gunakan `continue` untuk melewati nilai negatif jika ada.
3. **Tugas 2 - Fungsi & Koleksi (15%).** Buat fungsi `hitungStatistik(List<int> nilai)` yang mengembalikan *record* `({double rataRata, int tertinggi, int terendah, int jumlahLulus})` memakai `fold`, `reduce`, dan `where`. Buat juga fungsi dengan named parameter `cetakLaporan({required String judul, required List<int> nilai, int batasLulus = 60})`.
4. **Tugas 3 - OOP (25%).** Buat *enhanced enum* `NilaiHuruf` (label, bobot, batas bawah), class `MataKuliah` (kode, nama, sks, nilaiAngka) dengan getter `huruf` dan `mutu`, serta class `Mahasiswa` dengan field private `_khs`, getter `totalSks`, `ipk`, dan `predikat`. Tambahkan `factory MataKuliah.fromJson(Map<String, dynamic> json)` dan override `toString()`.
5. **Tugas 4 - Null Safety & Error Handling (15%).** Buat exception kustom `DataNilaiException`. Lempar exception ini jika SKS di luar 1-6 atau nilai di luar 0-100. Baca input nilai dari List String (misalnya `['85', 'delapan', '70']`) memakai `int.tryParse` dan operator `??`, lalu laporkan data yang tidak valid tanpa menghentikan program.
6. **Tugas 5 - Fitur Dart 3 (15%).** Buat `sealed class HasilMuat` dengan turunan `MuatBerhasil` dan `MuatGagal`. Tangani keduanya dengan *exhaustive switch* dan object pattern. Gunakan `if-case` di `fromJson` untuk memvalidasi bentuk JSON, serta `.indexed` dengan destructuring saat mencetak KHS bernomor.
7. **Tugas 6 - Asynchronous (20%).** Buat fungsi `Future<HasilMuat> muatKhsDariServer(String nim)` dengan `Future.delayed`. Ambil data dua semester secara paralel memakai `Future.wait` atau record `.wait`. Buat juga `Stream<int> simpanKeServer()` dengan `async*` yang mengirim

persentase progres, lalu cetak 'Data KRS Mahasiswa [Nama] berhasil disinkronkan' setelah stream selesai.

8. **Pengumpulan.** Kumpulkan folder proyek (tanpa folder `.dart_tool`) dan tangkapan layar output terminal setiap tugas. Sertakan komentar singkat di setiap file yang menjelaskan konsep yang dipakai.

Output yang Diharapkan

Enam program console yang berjalan tanpa error dan lolos dart analyze: konversi nilai ke huruf mutu, laporan statistik nilai, KHS bernomor lengkap dengan total SKS, IPK dua desimal, dan predikat, peringatan untuk data yang tidak valid tanpa program berhenti, serta simulasi pemuatan data paralel dan progres sinkronisasi asinkron.

Rubrik Penilaian

Tugas 1 Variabel & Control Flow (10%), Tugas 2 Fungsi & Koleksi (15%), Tugas 3 OOP: enum, class, constructor, getter, factory (25%), Tugas 4 Null Safety & Error Handling (15%), Tugas 5 Records/Patterns/Sealed Class (15%), Tugas 6 Future, Future.wait & Stream (20%). Pengurangan nilai jika dart analyze menampilkan error atau output tidak sesuai.

Kuis Pemahaman

1. Apa perbedaan utama antara final dan const di Dart?

A. final untuk angka, const untuk String

B. final diisi sekali dan nilainya boleh ditentukan saat runtime, sedangkan const harus sudah diketahui saat compile-time

C. const boleh diubah, final tidak boleh

D. Tidak ada perbedaan, keduanya sama persis

2. Apa hasil dari print(17 ~/ 5) dan print(17 / 5) di Dart?

A. 3 dan 3

B. 3.4 dan 3

C. 3 dan 3.4

D. 2 dan 3.4

3. Apa fungsi operator ?? (null-coalescing) pada Dart?

A. Memaksa compiler mengabaikan null safety

B. Memberikan nilai cadangan jika ekspresi di sisi kirinya bernilai null

C. Mengecek apakah dua objek identik di memori

D. Menghapus nilai null dari List secara otomatis

4. Perhatikan deklarasi void daftar({required String nim, String prodi = 'Informatika'}). Pemanggilan mana yang VALID?

A. daftar('H1D023001')

B. daftar(prodi: 'Elektro')

C. daftar(nim: 'H1D023001')

D. daftar()

5. Apa output dari kode berikut? var n = [78, 92, 55]; print(n.where((x) ⇒ x >= 60).map((x) ⇒ x + 5).toList());

A. [83, 97, 60]

B. [83, 97]

C. [78, 92]

D. (83, 97)

6. Kapan sebaiknya kita memakai operator ! (null assertion) pada variabel nullable?

A. Setiap kali compiler menampilkan error null, agar error cepat hilang

B. Hanya jika kita benar-benar yakin nilainya tidak null, karena jika ternyata null aplikasi akan crash saat runtime

C. Untuk mengubah variabel menjadi nullable

D. Untuk memberi nilai default ketika variabel null

7. Apa perbedaan extends, implements, dan with pada class Dart?

A. Ketiganya sama, hanya beda gaya penulisan

B. extends mewarisi isi dari satu induk, implements berjanji menyediakan semua method interface tanpa mewarisi isinya, with menempelkan mixin beserta isinya

C. extends boleh banyak induk, implements hanya satu

D. with hanya bisa dipakai pada abstract class

8. Apa keuntungan utama sealed class ketika dipakai bersama switch expression?

A. Program berjalan lebih cepat

B. Objek sealed class tidak bisa bernilai null

C. Compiler mengetahui semua turunannya sehingga memberi error jika ada kasus yang belum ditangani (exhaustive checking)

D. Sealed class bisa dibuat objeknya secara langsung

9. Apa yang terjadi pada kode berikut? try { int.parse('12a'); print('A'); } on FormatException { print('B'); } finally { print('C'); }

A. Mencetak A lalu C

B. Mencetak B lalu C

C. Mencetak A, B, lalu C

D. Program crash tanpa mencetak apa pun

10. Tiga permintaan API yang tidak saling bergantung masing-masing butuh 1 detik. Cara paling efisien untuk menunggu ketiganya adalah...

A. Meng-await ketiganya satu per satu secara berurutan (total sekitar 3 detik)

B. Memakai Future.wait (atau record .wait) agar berjalan paralel (total sekitar 1 detik)

C. Memakai Stream.fromIterable

D. Tidak memakai await sama sekali lalu langsung membaca hasilnya

Kunci Jawaban & Pembahasan

1. B — final hanya bisa diisi sekali tetapi nilainya boleh berasal dari runtime (misalnya DateTime.now()). const harus berupa nilai compile-time, sehingga objek const yang isinya sama bisa dipakai bersama dan dihemat memorinya, seperti const widget di Flutter.

2. C — Operator ~/ melakukan pembagian bulat sehingga hasilnya 3, sedangkan / selalu menghasilkan double, yaitu 3.4. Sisa baginya (17 % 5) adalah 2.

3. B — a ?? b memakai a jika tidak null, dan memakai b jika a bernilai null. Contoh: email ?? 'Belum diisi'.

4. C — nim adalah named parameter yang required, jadi wajib ditulis dengan namanya (nim: ...). prodi punya nilai default sehingga boleh dihilangkan. Named parameter tidak bisa diisi berdasarkan posisi.

5. B — where menyaring nilai >= 60 sehingga tersisa [78, 92], lalu map menambah 5 menjadi [83, 97]. toList() mengubah Iterable menjadi List sehingga dicetak dengan kurung siku.

6. B — Operator ! mematikan perlindungan null safety. Jika nilainya ternyata null, muncul error 'Null check operator used on a null value'. Pilihan yang lebih aman biasanya pengecekan if, ?. atau ??.

7. B — Sebuah class hanya bisa extends satu class, tetapi boleh implements banyak interface (semua method wajib ditulis ulang) dan with banyak mixin (isinya ikut terbawa).

8. C — Turunan sealed class hanya boleh ditulis di file yang sama, sehingga compiler tahu daftar lengkapnya. Switch tanpa cabang _ akan ditolak jika ada turunan yang terlewat, cocok untuk status seperti Memuat, Sukses, dan Gagal.

9. B — int.parse('12a') melempar FormatException sehingga print('A') dilewati. Blok on FormatException mencetak B, dan finally selalu dijalankan sehingga mencetak C.

10. B — Future.wait([...]) atau (f1, f2, f3).wait menjalankan semua Future bersamaan dan menunggu hingga semuanya selesai, sehingga total waktu kira-kira sama dengan permintaan yang paling lama.

MODUL 03 · WIDGET & UI

Arsitektur Widget & Lifecycle Antarmuka

Membedah filosofi 'Everything is a Widget', anatomi MaterialApp/Scaffold/AppBar, widget dasar (Text, Icon, Image, Container, Padding, SizedBox, Align, Card, ListTile), StatelessWidget vs StatefulWidget beserta setState dan lifecycle, cara memecah UI menjadi widget kecil dengan parameter dan callback, const & key, serta tema Material 3 dengan mode gelap.

Durasi Pertemuan 3 (150 Menit) + Belajar Mandiri

Prasyarat Modul 02: Fondasi Dart 3

Capaian Pembelajaran

- Menjelaskan relasi Widget Tree, Element Tree, dan RenderObject Tree.
- Menyusun kerangka aplikasi dengan MaterialApp, Scaffold, dan AppBar.
- Menggunakan widget dasar Text, Icon, Image (asset & network), Container, Padding, SizedBox, Center, Align, Card, dan ListTile.
- Memilih dengan tepat kapan menggunakan StatelessWidget vs StatefulWidget, serta memakai setState dengan benar.
- Menganalisis siklus hidup State (initState, didChangeDependencies, build, didUpdateWidget, dispose) dan membersihkan resource di dispose.
- Memecah UI menjadi widget kecil yang menerima parameter dan callback.
- Menerapkan const, Key, dan ThemeData Material 3 (ColorScheme.fromSeed, textTheme, mode gelap).

Daftar Isi Modul

1. Filosofi 'Everything is a Widget' & Tiga Pohon Flutter
2. Anatomi Aplikasi: MaterialApp, Scaffold, & AppBar
3. Text & TextStyle
4. Icon & IconButton
5. Image: Asset & Network
6. Container & BoxDecoration (Studi Kasus Kartu Mahasiswa)
7. Padding, SizedBox, Center, & Align
8. Card & ListTile
9. StatelessWidget vs StatefulWidget
10. StatefulWidget & setState: Counter Lengkap
11. Lifecycle State dengan Demo Log
12. Memecah Widget: Parameter & Callback
13. const & Key
14. ThemeData Material 3 & Mode Gelap
15. Tugas Praktikum
16. Kuis Pemahaman & Kunci Jawaban

1. Filosofi 'Everything is a Widget' & Tiga Pohon Flutter

Di dalam Flutter, hampir segala sesuatu adalah **Widget**. Bukan hanya tombol atau gambar, tetapi struktur layout (Padding, Center), tema warna (Theme), hingga pendeteksi sentuhan (GestureDetector) semuanya adalah widget.

Di balik layar, Flutter mengelola tiga pohon paralel untuk mencapai performa render 60-120 FPS yang mulus:

Widget adalah "resep", bukan tampilan

Bayangkan widget seperti **resep masakan**. Resep hanya berisi keterangan: bahan apa, berapa banyak, dimasak bagaimana. Flutter yang "memasak" resep itu menjadi piksel di layar. Karena widget hanya resep, membuat ulang widget itu murah sekali. Flutter boleh membuat ulang ribuan widget setiap detik tanpa membuat aplikasi lambat.

Widget disusun bersarang seperti pohon. Setiap widget punya *parent* (induk) dan bisa punya *child* (satu anak) atau *children* (banyak anak). Lihat contoh kecil berikut:

```
// Pohon widget sederhana: Center -> Column -> [Icon, Text]
Center(
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: const [
      Icon(Icons.school, size: 48),
      Text('Halo, Mahasiswa UNSOED!'),
    ],
  ),
)
```

Pohon widget Center → Column → Icon + Text

Jika digambar sebagai pohon, susunannya menjadi seperti ini:

```
Center
 └── Column
      ├── Icon (school)
      └── Text ("Halo, Mahasiswa UNSOED!")
```

Program Flutter paling kecil

Program Flutter selalu dimulai dari fungsi `main()` yang memanggil `runApp()`. Widget yang diberikan ke `runApp()` menjadi akar (root) pohon widget. Coba jalankan kode ini untuk melihat bahwa satu widget `Text` pun sudah bisa menjadi aplikasi.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  // runApp menerima satu widget sebagai akar pohon widget.
  runApp(
    const Center(
      child: Text(
        'Halo, Flutter!',
        // Tanpa MaterialApp, arah teks wajib ditentukan sendiri.
        textDirection: TextDirection.ltr,
        style: TextStyle(fontSize: 32, color: Colors.white),
      ),
    ),
```

```
  );
}
```

Program terkecil: satu Text tanpa MaterialApp, latar hitam

Hasilnya adalah layar hitam dengan tulisan putih di tengah. Perhatikan bahwa kita harus menulis `textDirection` sendiri karena belum ada `MaterialApp` yang menyediakan pengaturan bawaan (arah teks, tema, font). Di section berikutnya kita akan memakai `MaterialApp` supaya hal-hal seperti ini diurus otomatis.

> 💡 **Cara membaca kode Flutter**
>
> Baca kode widget dari luar ke dalam: widget paling luar adalah pembungkus, lalu turun ke `child` atau `children`. Biasakan memberi koma di akhir setiap argumen (*trailing comma*) agar format otomatis (`Shift+Alt+F` di VS Code) menyusun kode menjadi bertingkat rapi.

⚠️ **Kesalahan umum**

- Lupa memanggil `runApp()` di `main()` sehingga layar tetap kosong.
- Menganggap widget sebagai objek yang "diubah-ubah". Widget bersifat *immutable* (tidak bisa diubah). Untuk mengubah tampilan, Flutter membuat widget baru dengan data baru.
- Salah menaruh kurung tutup `)` sehingga widget masuk ke induk yang salah. Gunakan trailing comma dan format otomatis.

2. Anatomi Aplikasi: MaterialApp, Scaffold, & AppBar

Hampir semua aplikasi Flutter yang kamu buat akan memakai tiga widget kerangka: `MaterialApp` (aplikasi), `Scaffold` (halaman), dan `AppBar` (bilah judul). Memahami peran ketiganya membuat kode lain jauh lebih mudah dibaca.

Widget	Analogi	Tugas utama
`MaterialApp`	Gedung kampus	Menyediakan tema, judul aplikasi, navigasi, bahasa, dan arah teks untuk seluruh halaman.
`Scaffold`	Satu ruang kelas	Kerangka satu halaman: tempat untuk `appBar`, `body`, `floatingActionButton`, `drawer`, dan `bottomNavigationBar`.
`AppBar`	Papan nama di pintu kelas	Bilah di bagian atas berisi judul, tombol kembali, dan tombol aksi.

Contoh kecil: kerangka paling minimal

```
MaterialApp(
  home: Scaffold(
    appBar: AppBar(title: const Text('Beranda')),
    body: const Center(child: Text('Isi halaman')),
  ),
)
```

Kerangka minimal: MaterialApp + Scaffold + AppBar

Dari luar ke dalam: `MaterialApp` membungkus `Scaffold` lewat parameter `home`. `Scaffold` punya slot `appBar` dan `body`. Slot `body` adalah area utama di bawah AppBar.

Contoh lengkap: semua slot Scaffold

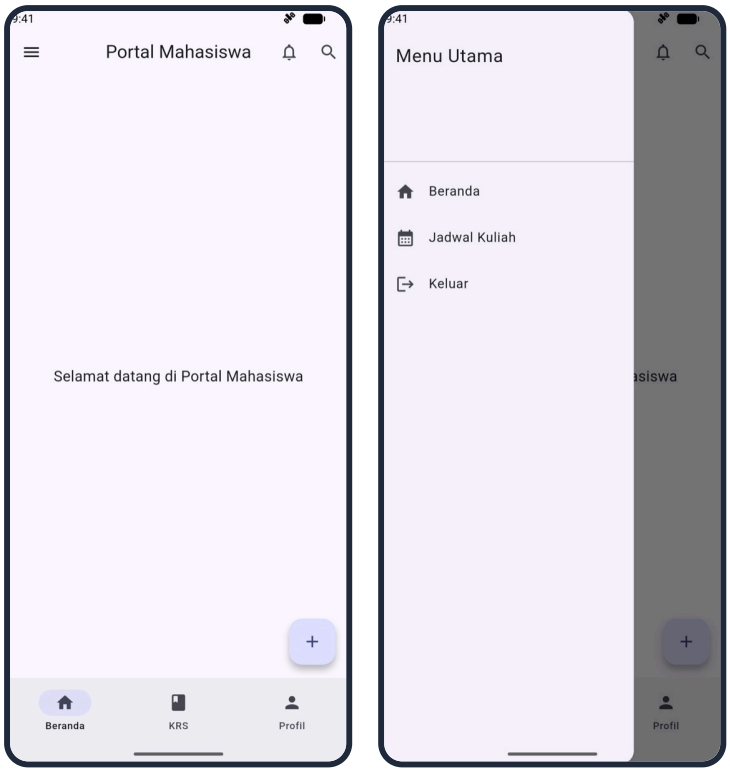
Kode berikut memperlihatkan slot-slot penting `Scaffold` sekaligus. Jalankan, lalu coba tekan ikon menu (garis tiga) dan tombol bulat di kanan bawah.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const AplikasiKampus());
}

class AplikasiKampus extends StatelessWidget {
  const AplikasiKampus({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Portal UNSOED', // nama aplikasi di task manager / tab browser
      debugShowCheckedModeBanner: false, // sembunyikan pita "DEBUG"
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const HalamanBeranda(),
    );
  }
}
```

```dart
class HalamanBeranda extends StatelessWidget {
  const HalamanBeranda({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // 1. AppBar: bilah judul di atas
      appBar: AppBar(
        title: const Text('Portal Mahasiswa'),
        centerTitle: true,
        actions: [
          IconButton(
            icon: const Icon(Icons.notifications_outlined),
            tooltip: 'Notifikasi',
            onPressed: () {},
          ),
          IconButton(
            icon: const Icon(Icons.search),
            tooltip: 'Cari',
            onPressed: () {},
          ),
        ],
      ),
      // 2. Drawer: menu samping. Ikon menu muncul otomatis di AppBar.
      drawer: Drawer(
        child: ListView(
          children: const [
            DrawerHeader(child: Text('Menu Utama', style: TextStyle(fontSize: 22))),
            ListTile(leading: Icon(Icons.home), title: Text('Beranda')),
            ListTile(leading: Icon(Icons.calendar_month), title: Text('Jadwal Kuliah')),
            ListTile(leading: Icon(Icons.logout), title: Text('Keluar')),
          ],
        ),
      ),
      // 3. Body: area utama halaman
      body: const Center(
        child: Text(
          'Selamat datang di Portal Mahasiswa',
          style: TextStyle(fontSize: 18),
        ),
      ),
      // 4. Tombol aksi melayang di kanan bawah
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        tooltip: 'Tambah',
        child: const Icon(Icons.add),
      ),
      // 5. Navigasi bawah (Material 3)
      bottomNavigationBar: NavigationBar(
        selectedIndex: 0,
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home), label: 'Beranda'),
          NavigationDestination(icon: Icon(Icons.book), label: 'KRS'),
          NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    );
```

```
  }
}
```



Semua slot Scaffold: AppBar, body, FAB, NavigationBar

Setelah ikon menu ditekan: Drawer terbuka

Penjelasan kode

- `runApp(const AplikasiKampus())` menjadikan `AplikasiKampus` sebagai akar aplikasi.
- `MaterialApp` mengatur `title`, `theme`, dan halaman pertama lewat `home`. Kita memisahkan halaman ke class `HalamanBeranda` supaya rapi.
- `AppBar.actions` menerima daftar widget (biasanya `IconButton`) yang tampil di sisi kanan.
- Begitu `drawer` diisi, `Scaffold` otomatis menambahkan ikon menu di kiri AppBar.
- `NavigationBar` adalah navigasi bawah gaya Material 3. Di sini indeksnya masih tetap 0; cara mengubahnya dengan `setState` dibahas di section StatefulWidget dan Modul 07.

Properti AppBar yang sering dipakai

Properti	Fungsi
`title`	Widget judul, biasanya `Text`.
`centerTitle`	`true` agar judul di tengah (default Android: kiri, iOS: tengah).
`leading`	Widget di kiri judul (misalnya tombol kembali kustom).
`actions`	Daftar tombol di kanan.
`backgroundColor` / `foregroundColor`	Warna latar dan warna teks/ikon AppBar.
`bottom`	Widget di bawah AppBar, misalnya `TabBar`.

⚠️ **Kesalahan umum**

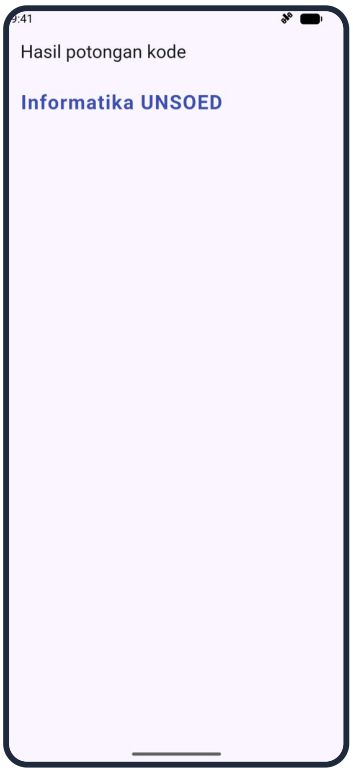
- **Membuat `MaterialApp` di setiap halaman.** Cukup satu `MaterialApp` di akar aplikasi. Setiap halaman cukup memakai `Scaffold`.
- **Lupa `Scaffold`.** Jika `Text` langsung ditaruh di `home` tanpa `Scaffold`, teks tampil dengan garis bawah kuning dan latar hitam karena tidak ada `Material` di atasnya.
- **Menaruh widget besar langsung di `appBar`.** Slot `appBar` mengharapkan `PreferredSizeWidget` seperti `AppBar`, bukan `Container` biasa.

3. Text & TextStyle

Widget `Text` menampilkan tulisan. Tampilan tulisan (ukuran, warna, ketebalan, jarak huruf) diatur lewat objek `TextStyle` yang diberikan ke parameter `style`.

Contoh kecil

```
const Text(
  'Informatika UNSOED',
  style: TextStyle(
    fontSize: 24,                // ukuran huruf (logical pixel)
    fontWeight: FontWeight.bold, // ketebalan: w100 s.d. w900
    color: Colors.indigo,        // warna teks
    letterSpacing: 1.2,          // jarak antar huruf
  ),
)
```



Text dengan TextStyle: besar, tebal, berwarna

Properti penting

Milik	Properti	Fungsi
`Text`	`textAlign`	Perataan: `TextAlign.left`, `center`, `right`, `justify`.
`Text`	`maxLines`	Batas jumlah baris.
`Text`	`overflow`	Perilaku jika teks kepanjangan, misalnya `TextOverflow.ellipsis` (menambah "...").
`TextStyle`	`fontStyle`	`FontStyle.italic` untuk huruf miring.
`TextStyle`	`decoration`	Garis bawah, coret: `TextDecoration.underline`, `lineThrough`.
`TextStyle`	`height`	Tinggi baris sebagai kelipatan ukuran font (misal 1.5).

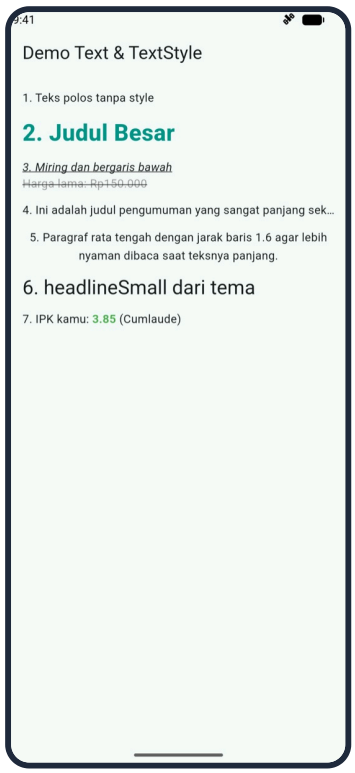
Contoh lengkap: berbagai gaya teks

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const DemoTeks(),
    );
  }
}

class DemoTeks extends StatelessWidget {
  const DemoTeks({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Text & TextStyle')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
```

```dart
// 1. Teks polos tanpa gaya
const Text('1. Teks polos tanpa style'),
const SizedBox(height: 12),

// 2. Ukuran, tebal, warna
const Text(
  '2. Judul Besar',
  style: TextStyle(
    fontSize: 28,
    fontWeight: FontWeight.w800,
    color: Colors.teal,
  ),
),
const SizedBox(height: 12),

// 3. Miring, garis bawah, coret
const Text(
  '3. Miring dan bergaris bawah',
  style: TextStyle(
    fontStyle: FontStyle.italic,
    decoration: TextDecoration.underline,
  ),
),
const Text(
  'Harga lama: Rp150.000',
  style: TextStyle(decoration: TextDecoration.lineThrough, color: Colors.grey),
),
const SizedBox(height: 12),

// 4. Teks panjang dipotong dengan ellipsis
const Text(
  '4. Ini adalah judul pengumuman yang sangat panjang sekali sehingga '
  'tidak muat dalam satu baris layar ponsel mahasiswa',
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
),
const SizedBox(height: 12),

// 5. Rata tengah dan jarak baris
const Text(
  '5. Paragraf rata tengah dengan jarak baris 1.6 agar '
  'lebih nyaman dibaca saat teksnya panjang.',
  textAlign: TextAlign.center,
  style: TextStyle(height: 1.6),
),
const SizedBox(height: 12),

// 6. Memakai gaya dari tema (disarankan)
Text(
  '6. headlineSmall dari tema',
  style: Theme.of(context).textTheme.headlineSmall,
),
const SizedBox(height: 12),

// 7. Text.rich: satu kalimat, beberapa gaya
const Text.rich(
  TextSpan(
    text: '7. IPK kamu: ',
    children: [
```

```
                TextSpan(
                  text: '3.85',
                  style: TextStyle(fontWeight: FontWeight.bold, color: Colors.green),
                ),
                TextSpan(text: ' (Cumlaude)'),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```



Berbagai gaya teks, ellipsis, dan Text.rich

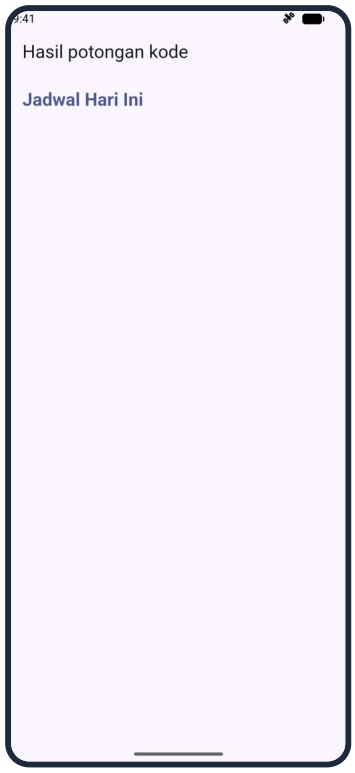
Penjelasan kode

- Teks panjang di Dart bisa ditulis sebagai beberapa string yang ditempel berurutan (*adjacent strings*), seperti pada nomor 4 dan 5. Dart menggabungkannya otomatis.
- `maxLines: 1` + `overflow: TextOverflow.ellipsis` adalah pasangan yang paling sering dipakai untuk judul di daftar.
- `Theme.of(context).textTheme.headlineSmall` mengambil gaya teks bawaan tema Material 3. Cara ini membuat ukuran huruf konsisten di seluruh aplikasi.
- `Text.rich` dengan `TextSpan` dipakai jika satu kalimat butuh beberapa gaya berbeda.

Menggabungkan gaya tema dengan `copyWith`

Sering kita ingin gaya dari tema, tetapi warnanya diganti. Gunakan `copyWith` agar tidak menulis ulang semua properti:

```dart
Text(
  'Jadwal Hari Ini',
  style: Theme.of(context).textTheme.titleLarge?.copyWith(
        color: Theme.of(context).colorScheme.primary,
        fontWeight: FontWeight.bold,
      ),
)
```



Gaya titleLarge dari tema yang diubah dengan copyWith

Tanda `?.` dipakai karena `titleLarge` bertipe `TextStyle?` (boleh null).

⚠️ Kesalahan umum

- Memberi angka bertipe `int` ke tempat yang butuh `double` sebenarnya aman (`fontSize: 24` boleh), tetapi menulis `fontWeight: 700` salah. Gunakan `FontWeight.w700`.
- Memakai `overflow: TextOverflow.ellipsis` tanpa `maxLines` atau tanpa batas lebar. Ellipsis hanya muncul jika teks memang dibatasi.
- Menulis ukuran font berbeda-beda secara manual di setiap halaman. Lebih baik pakai `textTheme` agar konsisten.

4. Icon & IconButton

Widget `Icon` menampilkan simbol dari font ikon. Flutter sudah membawa ribuan ikon Material lewat class `Icons`, jadi kamu tidak perlu mengunduh gambar apa pun.

Contoh kecil

```
const Icon(
  Icons.favorite,      // jenis ikon
  size: 32,            // ukuran (default 24)
  color: Colors.red,   // warna
  semanticLabel: 'Favorit', // dibacakan oleh pembaca layar (aksesibilitas)
)
```

Icon favorit merah ukuran 32

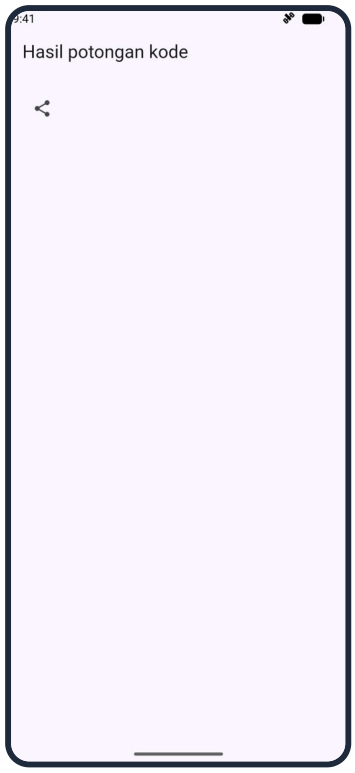
Nama ikon bisa dicari di fonts.google.com/icons. Hampir setiap ikon punya beberapa varian: `Icons.home` (terisi), `Icons.home_outlined` (garis), `Icons.home_rounded`, dan `Icons.home_sharp`.

Icon vs IconButton

`Icon` hanya gambar dan **tidak bisa diklik**. Jika ikon harus bisa ditekan, pakai `IconButton` yang memiliki `onPressed` dan efek sentuhan.

```
IconButton(
  icon: const Icon(Icons.share),
  tooltip: 'Bagikan',
  onPressed: () {
    debugPrint('Tombol bagikan ditekan');
```

```
  },
)
```



IconButton berbentuk ikon bagikan

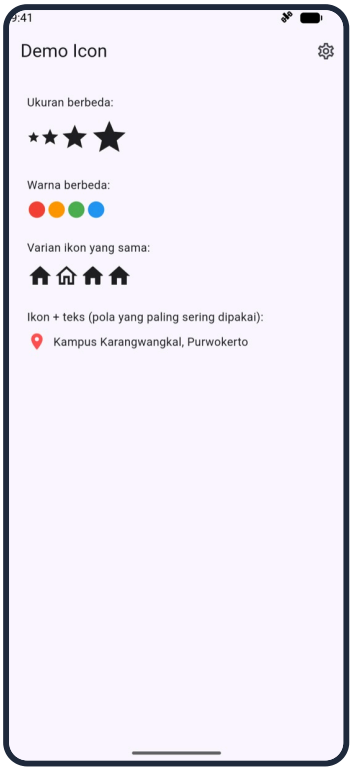
Contoh lengkap: menu ikon portal kampus

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DemoIkon(),
    );
  }
}

class DemoIkon extends StatelessWidget {
  const DemoIkon({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Demo Icon'),
        actions: [
          IconButton(
            icon: const Icon(Icons.settings_outlined),
            tooltip: 'Pengaturan',
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Pengaturan ditekan')),
              );
            },
          ),
        ],
      ),
      body: const Padding(
        padding: EdgeInsets.all(24),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Ukuran berbeda:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.star, size: 16),
                Icon(Icons.star, size: 24),
                Icon(Icons.star, size: 36),
                Icon(Icons.star, size: 48),
              ],
            ),
            SizedBox(height: 24),
            Text('Warna berbeda:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.circle, color: Colors.red),
                Icon(Icons.circle, color: Colors.orange),
                Icon(Icons.circle, color: Colors.green),
                Icon(Icons.circle, color: Colors.blue),
              ],
            ),
            SizedBox(height: 24),
            Text('Varian ikon yang sama:'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.home, size: 32),
                Icon(Icons.home_outlined, size: 32),
                Icon(Icons.home_rounded, size: 32),
                Icon(Icons.home_sharp, size: 32),
              ],
            ),
            SizedBox(height: 24),
            Text('Ikon + teks (pola yang paling sering dipakai):'),
            SizedBox(height: 8),
            Row(
              children: [
                Icon(Icons.location_on, color: Colors.redAccent),
                SizedBox(width: 8),
```

```
                Text('Kampus Karangwangkal, Purwokerto'),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```



Ukuran, warna, dan varian ikon

Penjelasan kode

- Seluruh isi `body` bisa `const` karena tidak ada nilai yang berubah. Satu `const` di depan `Padding` sudah berlaku untuk semua anak di dalamnya.
- `IconButton` di AppBar memakai `ScaffoldMessenger.of(context).showSnackBar(...)` untuk menampilkan pesan singkat di bawah layar.
- Pola **ikon + `SizedBox` + teks** di dalam `Row` akan sangat sering kamu pakai.

⚠️ **Kesalahan umum**

- Membungkus `Icon` dengan `GestureDetector` agar bisa diklik. Bisa, tetapi tidak ada efek sentuhan dan area sentuhnya kecil. Gunakan `IconButton`.
- Salah nama ikon (misalnya `Icons.homes`) akan langsung error saat kompilasi. Cek ejaan di situs Google Fonts Icons.
- Lupa `tooltip` pada `IconButton`. Tooltip membantu pengguna dan pembaca layar memahami fungsi tombol.

5. Image: Asset & Network

Widget `Image` menampilkan gambar. Ada dua sumber yang paling sering dipakai: **asset** (file gambar yang ikut dibundel di dalam aplikasi) dan **network** (gambar dari internet lewat URL).

Konstruktor	Sumber	Kapan dipakai
`Image.asset('assets/...')`	File di folder proyek	Logo, ilustrasi, ikon kustom yang selalu sama.
`Image.network('https://...')`	Internet	Foto profil, gambar berita, data dari server.

Langkah 1: siapkan folder asset

Buat folder `assets/images/` di akar proyek (sejajar dengan `lib/` dan `pubspec.yaml`), lalu salin gambar ke sana, misalnya `logo_unsoed.png`.

```
nama_proyek/
├── assets/
|   └── images/
|       └── logo_unsoed.png
├── lib/
|   └── main.dart
└── pubspec.yaml
```

Langkah 2: daftarkan di `pubspec.yaml`

Buka `pubspec.yaml`, cari bagian `flutter:`, lalu tambahkan `assets:`. Perhatikan indentasi: `assets:` menjorok 2 spasi di bawah `flutter:`, dan daftar di bawahnya menjorok 4 spasi.

`pubspec.yaml` **(potongan)**

```yaml
flutter:
  uses-material-design: true

  # Daftarkan seluruh isi folder (akhiri dengan garis miring)
  assets:
    - assets/images/
```

Setelah menyimpan `pubspec.yaml`, jalankan perintah berikut lalu **hentikan dan jalankan ulang** aplikasi (hot reload tidak cukup untuk asset baru):

```
flutter pub get
flutter run
```

Langkah 3: tampilkan gambar

```
Image.asset(
  'assets/images/logo_unsoed.png',
  width: 120,
  height: 120,
  fit: BoxFit.contain, // gambar utuh, tidak terpotong
)
```

Gambar dari internet

```
Image.network(
  'https://picsum.photos/id/1/400/250',
  height: 180,
  width: double.infinity,
  fit: BoxFit.cover, // penuhi kotak, boleh terpotong
)
```

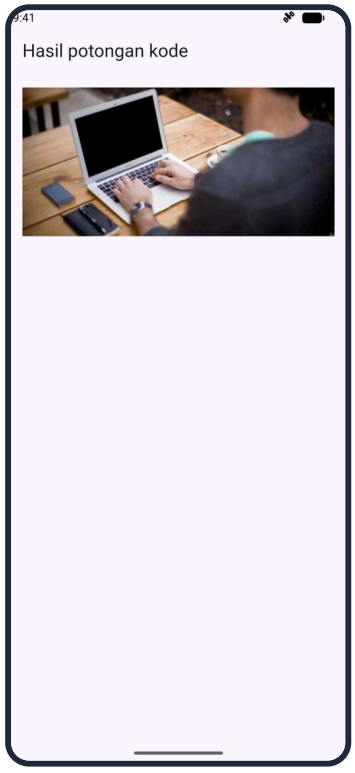


Image.network dengan BoxFit.cover

> ℹ️ **Izin internet di Android**
>
> Saat debug, izin internet sudah aktif. Untuk build rilis Android, pastikan `android/app/src/main/AndroidManifest.xml` berisi `<uses-permission android:name="android.permission.INTERNET"/>`. Detailnya dibahas di Modul 09 dan 16.

Nilai `BoxFit` yang perlu diketahui

Nilai	Hasil
`BoxFit.cover`	Memenuhi kotak, sisi yang berlebih dipotong. Cocok untuk banner dan foto.
`BoxFit.contain`	Gambar utuh di dalam kotak, mungkin ada ruang kosong. Cocok untuk logo.
`BoxFit.fill`	Ditarik memenuhi kotak, gambar bisa gepeng.
`BoxFit.fitWidth` / `fitHeight`	Menyesuaikan lebar atau tinggi saja.

Contoh lengkap: gambar network dengan loading dan error

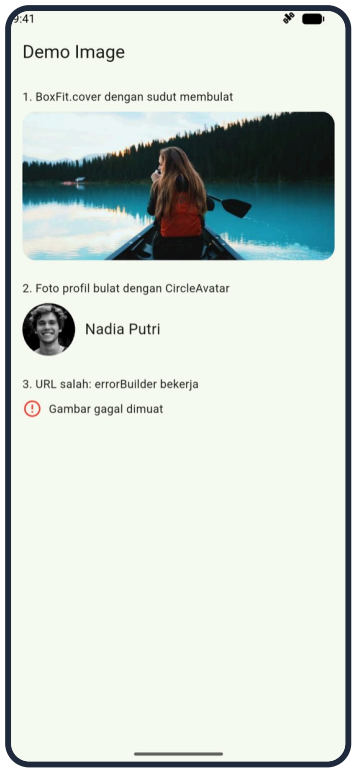
Contoh ini hanya memakai gambar network supaya langsung bisa dijalankan tanpa menyiapkan asset. Gambar dari internet butuh waktu dimuat dan bisa gagal (misalnya saat offline), jadi kita tampilkan indikator loading dan ikon cadangan.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const DemoGambar(),
    );
  }
}

class DemoGambar extends StatelessWidget {
  const DemoGambar({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Image')),
      body: ListView(
        padding: const EdgeInsets.all(16),
```

```
        children: [
          const Text('1. BoxFit.cover dengan sudut membulat'),
          const SizedBox(height: 8),
          // ClipRRect memotong anak menjadi sudut membulat
          ClipRRect(
            borderRadius: BorderRadius.circular(16),
            child: Image.network(
              'https://picsum.photos/id/1011/600/300',
              height: 180,
              width: double.infinity,
              fit: BoxFit.cover,
              // Dipanggil berulang selama gambar diunduh
              loadingBuilder: (context, child, progress) {
                if (progress == null) return child; // selesai dimuat
                return const SizedBox(
                  height: 180,
                  child: Center(child: CircularProgressIndicator()),
                );
              },
              // Dipanggil jika gambar gagal dimuat
              errorBuilder: (context, error, stackTrace) {
                return Container(
                  height: 180,
                  color: Colors.grey.shade300,
                  alignment: Alignment.center,
                  child: const Icon(Icons.broken_image, size: 48),
                );
              },
            ),
          ),
          const SizedBox(height: 24),
          const Text('2. Foto profil bulat dengan CircleAvatar'),
          const SizedBox(height: 8),
          Row(
            children: [
              CircleAvatar(
                radius: 32,
                foregroundImage: const NetworkImage('https://i.pravatar.cc/150?img=12'),
                // child tampil jika foregroundImage belum/gagal dimuat
                child: const Icon(Icons.person),
              ),
              const SizedBox(width: 12),
              const Text('Nadia Putri', style: TextStyle(fontSize: 18)),
            ],
          ),
          const SizedBox(height: 24),
          const Text('3. URL salah: errorBuilder bekerja'),
          const SizedBox(height: 8),
          Image.network(
            'https://contoh.invalid/gambar-tidak-ada.png',
            height: 100,
            errorBuilder: (context, error, stackTrace) => const Row(
              children: [
                Icon(Icons.error_outline, color: Colors.red),
                SizedBox(width: 8),
                Text('Gambar gagal dimuat'),
              ],
            ),
          ),
```

```
        ],
      ),
    );
  }
}
```



Gambar network dengan sudut membulat, CircleAvatar, dan errorBuilder

Penjelasan kode

- `loadingBuilder` menerima `progress`. Jika `null`, artinya gambar sudah selesai diunduh sehingga kita kembalikan `child` (gambar aslinya).
- `errorBuilder` mencegah layar merah dan menggantinya dengan tampilan cadangan.
- `ClipRRect` memotong gambar agar sudutnya membulat. `Image` sendiri tidak punya properti `borderRadius`.
- `CircleAvatar` dengan `foregroundImage` tetap menampilkan ikon `child` jika foto gagal dimuat.

Versi asset dari contoh di atas

Setelah langkah 1 dan 2 selesai, kamu bisa mengganti `Image.network(...)` dengan asset, dan `NetworkImage` dengan `AssetImage`:

```
Column(
  children: [
    Image.asset('assets/images/logo_unsoed.png', height: 100),
    const SizedBox(height: 12),
    const CircleAvatar(
      radius: 32,
      backgroundImage: AssetImage('assets/images/logo_unsoed.png'),
```

```
    ),
  ],
)
```

⚠️ **Kesalahan umum**

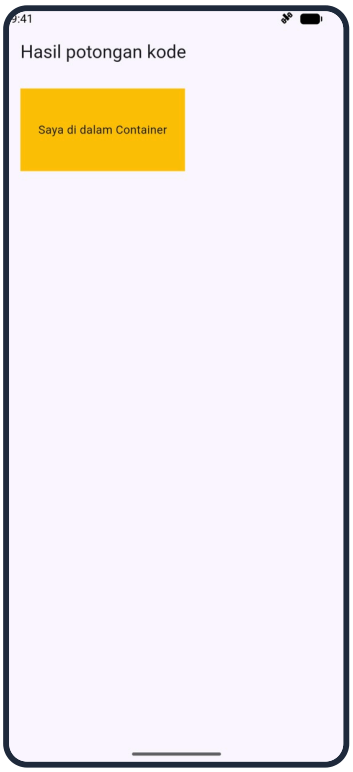
- **"Unable to load asset"**: path di kode berbeda dengan nama file (huruf besar/kecil berpengaruh), atau folder belum didaftarkan di `pubspec.yaml`.
- **Indentasi YAML salah**: `assets:` harus berada di dalam `flutter:` (2 spasi), bukan di tingkat paling kiri.
- **Hanya hot reload** setelah menambah asset. Lakukan `flutter pub get` dan jalankan ulang aplikasi.
- **Gambar network tanpa** `errorBuilder`: saat internet mati, pengguna melihat area kosong atau pesan error.

6. Container & BoxDecoration (Studi Kasus Kartu Mahasiswa)

`Container` adalah "kotak serbaguna". Satu widget ini bisa mengatur ukuran (`width`, `height`), jarak dalam (`padding`), jarak luar (`margin`), posisi anak (`alignment`), dan dekorasi (`decoration`: warna, sudut membulat, border, bayangan, gradasi).

Contoh kecil: kotak berwarna

```
Container(
  width: 200,
  height: 100,
  color: Colors.amber,
  alignment: Alignment.center,
  child: const Text('Saya di dalam Container'),
)
```



Container kuning 200 × 100 dengan teks di tengah

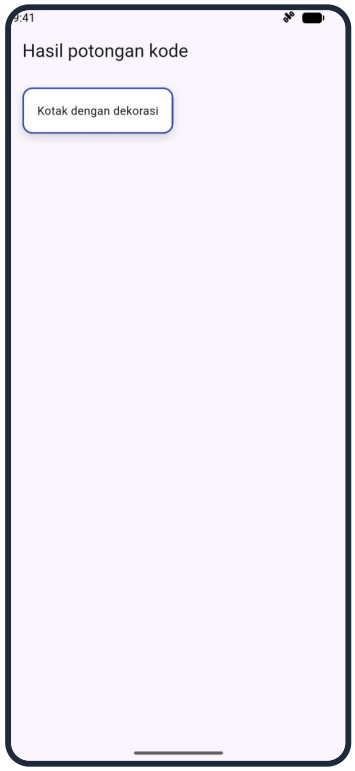
Padding vs margin

```
┌──────────────── margin (jarak ke luar) ────────────────┐
|  ┌──────────── batas Container (warna) ──────────┐     |
|  |   padding (jarak ke dalam)                    |     |
|  |      ┌──────── child ────────┐                |     |
|  |      |  Text('Halo')         |                |     |
|  |      └───────────────────────┘                |     |
|  └───────────────────────────────────────────────┘     |
└────────────────────────────────────────────────────────┘
```

`EdgeInsets` dipakai untuk keduanya: `EdgeInsets.all(16)` (semua sisi), `EdgeInsets.symmetric(horizontal: 16, vertical: 8)`, dan `EdgeInsets.only(left: 8, top: 4)`.

BoxDecoration: menghias kotak

```
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),          // sudut membulat
    border: Border.all(color: Colors.indigo, width: 2), // garis tepi
    boxShadow: [
      BoxShadow(
        color: Colors.black.withValues(alpha: 0.15),  // bayangan transparan
        blurRadius: 8,
        offset: const Offset(0, 4),                   // geser ke bawah 4px
      ),
    ],
  ),
  child: const Text('Kotak dengan dekorasi'),
)
```



BoxDecoration: border, sudut membulat, bayangan

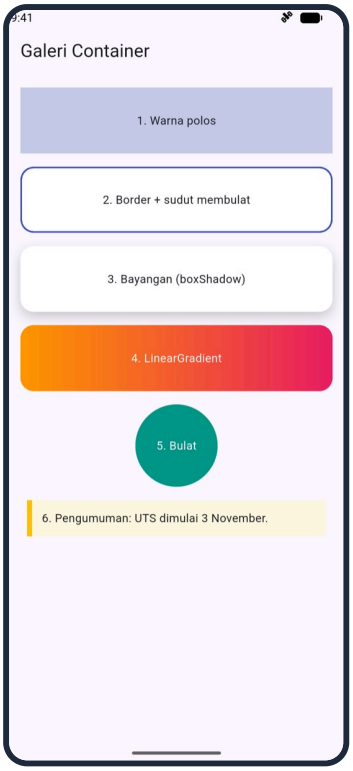
Contoh lengkap: galeri dekorasi Container

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
```

```
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const GaleriContainer(),
    );
  }
}

class GaleriContainer extends StatelessWidget {
  const GaleriContainer({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Galeri Container')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Warna polos + ukuran tetap
          Container(
            height: 80,
            color: Colors.indigo.shade100,
            alignment: Alignment.center,
            child: const Text('1. Warna polos'),
          ),
          const SizedBox(height: 16),

          // 2. Sudut membulat + border
          Container(
            height: 80,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(16),
              border: Border.all(color: Colors.indigo, width: 2),
            ),
            child: const Text('2. Border + sudut membulat'),
          ),
          const SizedBox(height: 16),

          // 3. Bayangan
          Container(
            height: 80,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(16),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withValues(alpha: 0.2),
                  blurRadius: 12,
```

```
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: const Text('3. Bayangan (boxShadow)'),
        ),
        const SizedBox(height: 16),

        // 4. Gradasi warna
        Container(
          height: 80,
          alignment: Alignment.center,
          decoration: BoxDecoration(
            gradient: const LinearGradient(
              colors: [Colors.orange, Colors.pink],
              begin: Alignment.centerLeft,
              end: Alignment.centerRight,
            ),
            borderRadius: BorderRadius.circular(16),
          ),
          child: const Text('4. LinearGradient', style: TextStyle(color: Colors.white)),
        ),
        const SizedBox(height: 16),

        // 5. Lingkaran
        Center(
          child: Container(
            width: 100,
            height: 100,
            alignment: Alignment.center,
            decoration: const BoxDecoration(
              color: Colors.teal,
              shape: BoxShape.circle, // jangan dipakai bersama borderRadius
            ),
            child: const Text('5. Bulat', style: TextStyle(color: Colors.white)),
          ),
        ),
        const SizedBox(height: 16),

        // 6. Border hanya di sisi kiri (gaya kartu pengumuman)
        Container(
          padding: const EdgeInsets.all(12),
          margin: const EdgeInsets.symmetric(horizontal: 8),
          decoration: BoxDecoration(
            color: Colors.amber.shade50,
            border: const Border(
              left: BorderSide(color: Colors.amber, width: 6),
            ),
          ),
          child: const Text('6. Pengumuman: UTS dimulai 3 November.'),
        ),
      ],
    ),
  );
 }
}
```



Galeri dekorasi Container

Penjelasan kode

- `alignment: Alignment.center` menaruh `child` di tengah kotak.
- `Colors.indigo.shade100` adalah tingkat terang warna (50, 100, ... 900).
- `withValues(alpha: 0.2)` membuat warna 20% tidak transparan. Ini pengganti `withOpacity` yang sudah usang.
- `BoxShape.circle` membuat kotak menjadi lingkaran; ukurannya sebaiknya persegi (lebar = tinggi).
- `Border(left: BorderSide(...))` membuat garis hanya di satu sisi.

Studi kasus: Kartu Tanda Mahasiswa

Berikut implementasi kode kartu mahasiswa yang memadukan `Container`, `BoxDecoration`, `Text`, `Icon`, dan `SizedBox`:

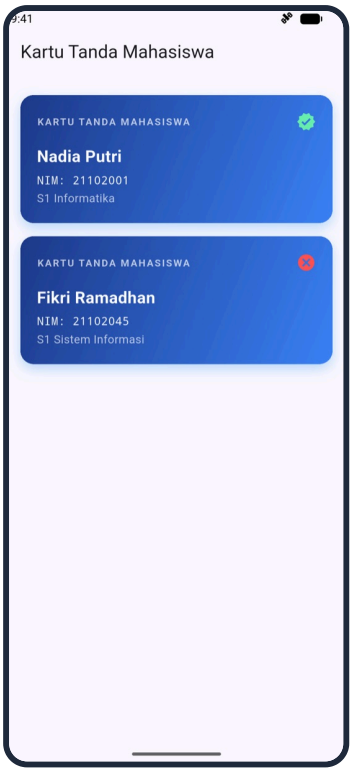
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      title: 'Kartu Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text("Kartu Tanda Mahasiswa")),
        body: const Column(
          children: [
            SizedBox(height: 16),
            KartuMahasiswaWidget(
              nama: "Nadia Putri",
              nim: "21102001",
              jurusan: "S1 Informatika",
            ),
            KartuMahasiswaWidget(
              nama: "Fikri Ramadhan",
              nim: "21102045",
              jurusan: "S1 Sistem Informasi",
              isAktif: false,
            ),
          ],
        ),
      ),
    );
  }
}

class KartuMahasiswaWidget extends StatelessWidget {
  final String nama;
  final String nim;
  final String jurusan;
  final bool isAktif;

  const KartuMahasiswaWidget({
    super.key,
    required this.nama,
    required this.nim,
    required this.jurusan,
    this.isAktif = true,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(20),
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      decoration: BoxDecoration(
        gradient: const LinearGradient(
          colors: [Color(0xFF1E3A8A), Color(0xFF3B82F6)], // Gradasi Biru Kampus
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
        ),
        borderRadius: BorderRadius.circular(16),
        boxShadow: [
          BoxShadow(
            // withValues menggantikan withOpacity yang sudah deprecated
            color: Colors.blue.withValues(alpha: 0.3),
            blurRadius: 12,
```

```
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              // Flexible mencegah overflow jika layar HP sempit
              const Flexible(
                child: Text(
                  "KARTU TANDA MAHASISWA",
                  overflow: TextOverflow.ellipsis,
                  style: TextStyle(
                    color: Colors.white70,
                    fontSize: 12,
                    letterSpacing: 1.5,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
              Icon(
                isAktif ? Icons.verified : Icons.cancel,
                color: isAktif ? Colors.greenAccent : Colors.redAccent,
              ),
            ],
          ),
          const SizedBox(height: 16),
          Text(
            nama,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 20,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 4),
          Text(
            "NIM: $nim",
            style: const TextStyle(
              color: Colors.white,
              fontFamily: 'monospace',
              fontSize: 14,
            ),
          ),
          const SizedBox(height: 2),
          Text(
            jurusan,
            style: const TextStyle(
              color: Colors.white70,
              fontSize: 14,
            ),
          ),
        ],
      ),
    );
```

```
  }
}
```



Studi kasus Kartu Tanda Mahasiswa (aktif dan tidak aktif)

Penjelasan studi kasus

- `KartuMahasiswaWidget` menerima data lewat konstruktor (`nama`, `nim`, `jurusan`, `isAktif`), sehingga satu class bisa dipakai untuk banyak mahasiswa.
- `width: double.infinity` membuat kartu selebar mungkin sesuai ruang dari induknya.
- Ikon berubah berdasarkan `isAktif` memakai operator ternary `kondisi ? a : b`.

> ⚠️ **Kesalahan umum**
>
> - **Memakai `color` dan `decoration` bersamaan.** Ini menimbulkan error *"Cannot provide both a color and a decoration"*. Jika memakai `decoration`, pindahkan warna ke `BoxDecoration(color: ...)`.
> - `shape: BoxShape.circle` + `borderRadius` sekaligus akan error. Pilih salah satu.
> - **Memakai Container hanya untuk jarak.** Jika hanya butuh jarak, `SizedBox` atau `Padding` lebih ringan dan lebih jelas maksudnya (dibahas di section berikutnya).

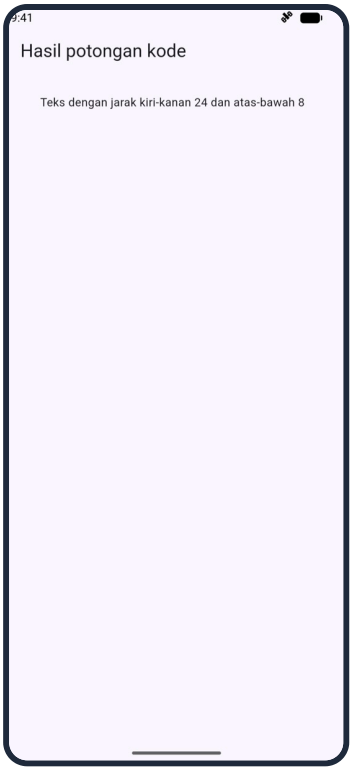
7. Padding, SizedBox, Center, & Align

`Container` memang serbaguna, tetapi untuk tugas tunggal Flutter menyediakan widget yang lebih kecil dan jelas maksudnya: **`Padding`** (jarak dalam), **`SizedBox`** (ukuran tetap atau jarak kosong), **`Center`** (menengahkan), dan **`Align`** (menaruh anak di posisi tertentu).

Widget	Fungsi	Contoh
`Padding`	Memberi ruang di sekeliling anak	`Padding(padding: EdgeInsets.all(16), child: ...)`
`SizedBox`	Kotak berukuran pasti, atau jarak kosong	`SizedBox(height: 12)`
`Center`	Menaruh anak tepat di tengah	`Center(child: ...)`
`Align`	Menaruh anak di titik mana pun	`Align(alignment: Alignment.topRight, child: ...)`

Padding

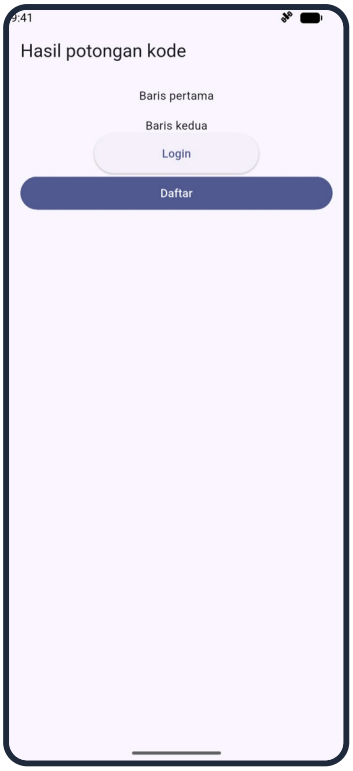
```
const Padding(
  padding: EdgeInsets.symmetric(horizontal: 24, vertical: 8),
  child: Text('Teks dengan jarak kiri-kanan 24 dan atas-bawah 8'),
)
```



Padding horizontal 24 dan vertikal 8

SizedBox: tiga kegunaan

```
Column(
  children: [
    // 1. Jarak vertikal antar widget
    const Text('Baris pertama'),
    const SizedBox(height: 16),
    const Text('Baris kedua'),

    // 2. Memaksa ukuran anak
    SizedBox(
      width: 200,
      height: 48,
      child: ElevatedButton(onPressed: () {}, child: const Text('Login')),
    ),

    // 3. Selebar mungkin
    SizedBox(
      width: double.infinity,
      child: FilledButton(onPressed: () {}, child: const Text('Daftar')),
    ),
  ],
)
```



SizedBox sebagai jarak, ukuran tetap, dan selebar layar

Align dan nilai `Alignment`

`Alignment(x, y)` memakai koordinat dari `-1` sampai `1`. Titik `(0, 0)` adalah tengah, `(-1, -1)` pojok kiri atas, dan `(1, 1)` pojok kanan bawah.

```
(-1,-1) topLeft      (0,-1) topCenter      (1,-1) topRight
(-1, 0) centerLeft   (0, 0) center         (1, 0) centerRight
(-1, 1) bottomLeft   (0, 1) bottomCenter   (1, 1) bottomRight
```

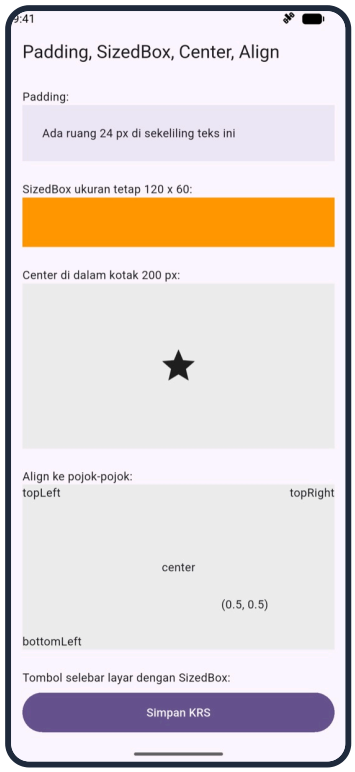
Contoh lengkap: membandingkan keempat widget

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const DemoTataLetakDasar(),
    );
```

```
  }
}

class DemoTataLetakDasar extends StatelessWidget {
  const DemoTataLetakDasar({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Padding, SizedBox, Center, Align')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text('Padding:'),
          Container(
            color: Colors.deepPurple.shade50,
            child: const Padding(
              padding: EdgeInsets.all(24),
              child: Text('Ada ruang 24 px di sekeliling teks ini'),
            ),
          ),
          const SizedBox(height: 24),

          const Text('SizedBox ukuran tetap 120 x 60:'),
          const SizedBox(
            width: 120,
            height: 60,
            child: ColoredBox(color: Colors.orange),
          ),
          const SizedBox(height: 24),

          const Text('Center di dalam kotak 200 px:'),
          Container(
            height: 200,
            color: Colors.grey.shade200,
            child: const Center(child: Icon(Icons.star, size: 48)),
          ),
          const SizedBox(height: 24),

          const Text('Align ke pojok-pojok:'),
          Container(
            height: 200,
            color: Colors.grey.shade200,
            child: const Stack(
              children: [
                Align(alignment: Alignment.topLeft, child: Text('topLeft')),
                Align(alignment: Alignment.topRight, child: Text('topRight')),
                Align(alignment: Alignment.center, child: Text('center')),
                Align(alignment: Alignment.bottomLeft, child: Text('bottomLeft')),
                Align(alignment: Alignment(0.5, 0.5), child: Text('(0.5, 0.5)')),
              ],
            ),
          ),
          const SizedBox(height: 24),

          const Text('Tombol selebar layar dengan SizedBox:'),
          const SizedBox(height: 8),
          SizedBox(
            width: double.infinity,
```

```
            height: 48,
            child: FilledButton(
              onPressed: () {},
              child: const Text('Simpan KRS'),
            ),
          ),
        ],
      ),
    );
  }
}
```



Perbandingan Padding, SizedBox, Center, dan Align

Penjelasan kode

- `Container` berwarna di sekitar `Padding` membuat jarak dalam terlihat jelas.
- `ColoredBox` adalah widget ringan untuk memberi warna latar saja.
- `Stack` dipakai agar beberapa `Align` bisa menempati kotak yang sama (Stack dibahas lengkap di Modul 04).
- `Alignment(0.5, 0.5)` menunjukkan bahwa posisi tidak harus di sudut; nilai pecahan juga boleh.

💡 **Pilih widget yang paling sederhana**

Butuh jarak? `SizedBox`. Butuh ruang di sekitar? `Padding`. Butuh warna + sudut + bayangan sekaligus? Baru `Container`. Kode jadi lebih mudah dibaca dan bisa `const`.

⚠️ **Kesalahan umum**

- Menulis `SizedBox(height: 16)` di dalam `Row` untuk memberi jarak horizontal. Di `Row` yang dipakai adalah `width`.
- Menaruh `Center` di dalam `Column` lalu heran anaknya tidak di tengah layar secara vertikal. `Center` hanya menengahkan di dalam ruang yang diberikan induknya. Gunakan `mainAxisAlignment` pada `Column` (Modul 04).
- Lupa `const` pada `SizedBox` dan `Padding` yang isinya tetap. Linter akan memberi saran *prefer_const_constructors*.

8. Card & ListTile

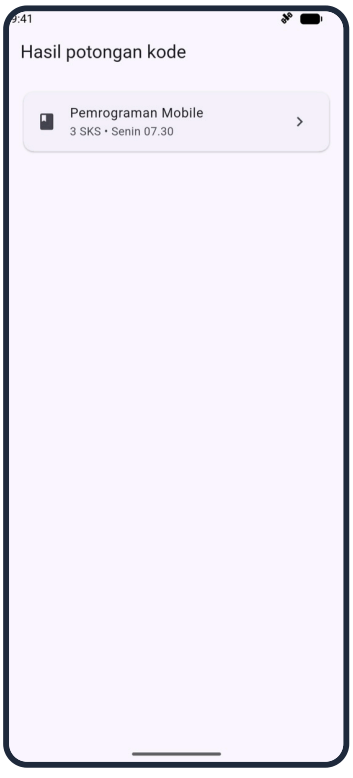
`Card` adalah panel bersudut membulat dengan bayangan/warna permukaan khas Material. `ListTile` adalah satu baris berisi ikon/gambar di kiri, judul, subjudul, dan widget di kanan. Keduanya sering dipakai bersama untuk daftar data seperti mata kuliah, pengumuman, atau kontak.

Anatomi ListTile

```
┌──────────────────────────────────────────────────┐
| [leading]  title (judul)                [trailing] |
|            subtitle (keterangan)                   |
└──────────────────────────────────────────────────┘
```

Contoh kecil

```dart
Card(
  child: ListTile(
    leading: const Icon(Icons.book),
    title: const Text('Pemrograman Mobile'),
    subtitle: const Text('3 SKS • Senin 07.30'),
    trailing: const Icon(Icons.chevron_right),
    onTap: () {
      debugPrint('Mata kuliah dipilih');
    },
  ),
)
```



Card berisi ListTile lengkap

Varian Card di Material 3

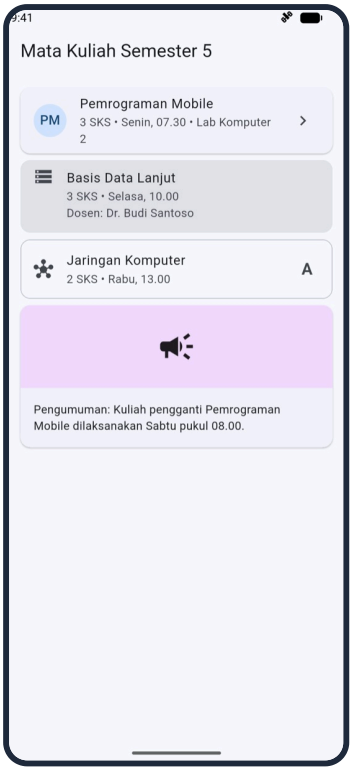
Material 3 menyediakan tiga gaya: `Card()` (elevated, sedikit bayangan), `Card.filled()` (warna permukaan, tanpa bayangan), dan `Card.outlined()` (garis tepi).

Contoh lengkap: daftar mata kuliah

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const DaftarMatkul(),
    );
  }
}

class DaftarMatkul extends StatelessWidget {
  const DaftarMatkul({super.key});

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Semester 5')),
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          // 1. Card biasa + ListTile lengkap
          Card(
            child: ListTile(
              leading: CircleAvatar(
                backgroundColor: warna.primaryContainer,
                child: const Text('PM'),
              ),
              title: const Text('Pemrograman Mobile'),
              subtitle: const Text('3 SKS • Senin, 07.30 • Lab Komputer 2'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Membuka Pemrograman Mobile')),
                );
              },
            ),
          ),
```

```
          // 2. Card.filled + subtitle dua baris
          const Card.filled(
            child: ListTile(
              leading: Icon(Icons.storage),
              title: Text('Basis Data Lanjut'),
              subtitle: Text('3 SKS • Selasa, 10.00\nDosen: Dr. Budi Santoso'),
              isThreeLine: true,
            ),
          ),

          // 3. Card.outlined + trailing berupa teks
          const Card.outlined(
            child: ListTile(
              leading: Icon(Icons.hub),
              title: Text('Jaringan Komputer'),
              subtitle: Text('2 SKS • Rabu, 13.00'),
              trailing: Text('A', style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
            ),
          ),

          // 4. Card berisi konten bebas (bukan ListTile)
          Card(
            clipBehavior: Clip.antiAlias, // agar isi ikut terpotong sudutnya
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Container(
                  height: 100,
                  color: warna.tertiaryContainer,
                  alignment: Alignment.center,
                  child: const Icon(Icons.campaign, size: 48),
                ),
                const Padding(
                  padding: EdgeInsets.all(16),
                  child: Text(
                    'Pengumuman: Kuliah pengganti Pemrograman Mobile '
                    'dilaksanakan Sabtu pukul 08.00.',
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```



Varian Card Material 3 dan Card berisi konten bebas

Penjelasan kode

- `Theme.of(context).colorScheme` memberi akses ke warna tema (misal `primaryContainer`), jadi warna otomatis serasi.
- `onTap` membuat seluruh baris bisa ditekan dan memunculkan efek riak (*ripple*).
- `isThreeLine: true` memberi tinggi ekstra saat subtitle berisi dua baris.
- `clipBehavior: Clip.antiAlias` diperlukan agar `Container` berwarna di dalam Card tidak keluar dari sudut membulat.

> ⚠️ **Kesalahan umum**
>
> - Menaruh `Row` panjang di `title` sehingga overflow. Gunakan `Text` dengan `maxLines` dan `overflow`.
> - Menaruh widget lebar tanpa batas (misal `TextField` atau `Row` tanpa `Expanded`) di `trailing`. Slot `trailing` ukurannya terbatas.
> - Lupa `clipBehavior` sehingga gambar/warna di dalam Card menutupi sudut membulat.

9. StatelessWidget vs StatefulWidget

Setiap widget buatan sendiri adalah turunan dari salah satu dari dua class: `StatelessWidget` (tampilan tidak berubah sendiri) atau `StatefulWidget` (punya data yang bisa berubah dan membuat tampilan digambar ulang).

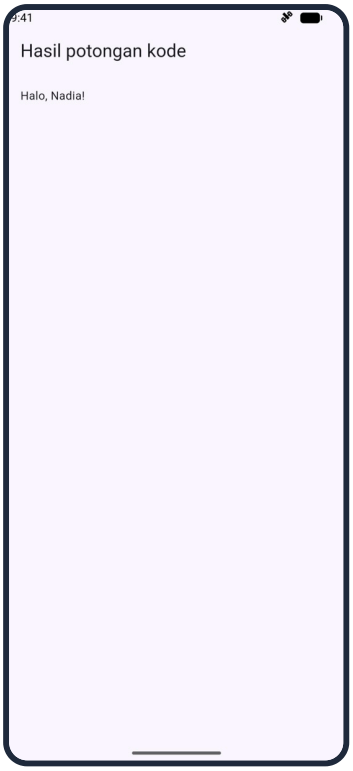
Aspek	StatelessWidget	StatefulWidget
Karakteristik	Statis, tidak berubah sepanjang waktu	Dinamis, dapat berubah berdasarkan interaksi atau data
State	Tidak memiliki objek State internal	Memiliki objek `State<T>` yang persisten
Jumlah class	1 class	2 class: widget + State-nya
Kapan Digunakan?	Label teks statis, icon, layout struktur, kartu profil statis	Formulir input, checkbox toggle, animasi, data hasil request API

Pertanyaan penentu

Tanyakan: *"Apakah widget ini perlu mengingat sesuatu yang bisa berubah saat aplikasi berjalan?"* Jika **tidak**, pakai StatelessWidget. Jika **ya** (angka penghitung, status centang, teks yang diketik), pakai StatefulWidget.

Kerangka StatelessWidget

```
class SapaanMahasiswa extends StatelessWidget {
  // Data masuk lewat konstruktor dan bersifat final (tidak bisa diubah)
  final String nama;

  const SapaanMahasiswa({super.key, required this.nama});

  @override
  Widget build(BuildContext context) {
    return Text('Halo, $nama!');
  }
}
```



Widget SapaanMahasiswa dengan nama Nadia

Tiga bagian wajib: **field** `final` untuk data, **konstruktor** `const` dengan `super.key`, dan **method** `build` yang mengembalikan widget.

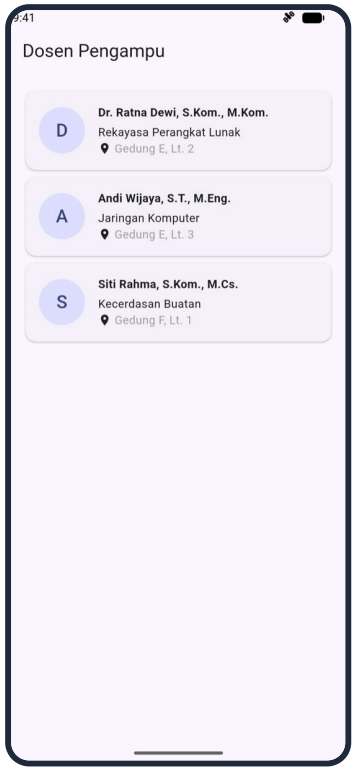
Contoh lengkap StatelessWidget: kartu profil dosen

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text('Dosen Pengampu')),
        body: ListView(
          padding: const EdgeInsets.all(16),
          children: const [
            // Satu class dipakai berulang dengan data berbeda
            KartuDosen(
              nama: 'Dr. Ratna Dewi, S.Kom., M.Kom.',
              bidang: 'Rekayasa Perangkat Lunak',
```

```
              ruang: 'Gedung E, Lt. 2',
            ),
            KartuDosen(
              nama: 'Andi Wijaya, S.T., M.Eng.',
              bidang: 'Jaringan Komputer',
              ruang: 'Gedung E, Lt. 3',
            ),
            KartuDosen(
              nama: 'Siti Rahma, S.Kom., M.Cs.',
              bidang: 'Kecerdasan Buatan',
              ruang: 'Gedung F, Lt. 1',
            ),
          ],
        ),
      ),
    );
  }
}

class KartuDosen extends StatelessWidget {
  final String nama;
  final String bidang;
  final String ruang;

  const KartuDosen({
    super.key,
    required this.nama,
    required this.bidang,
    required this.ruang,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Row(
          children: [
            CircleAvatar(
              radius: 28,
              // Ambil huruf pertama nama sebagai inisial
              child: Text(nama.substring(0, 1), style: const TextStyle(fontSize: 22)),
            ),
            const SizedBox(width: 16),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(nama, style: const TextStyle(fontWeight: FontWeight.bold)),
                  const SizedBox(height: 4),
                  Text(bidang),
                  Row(
                    children: [
                      const Icon(Icons.room, size: 16),
                      const SizedBox(width: 4),
                      Text(ruang, style: const TextStyle(color: Colors.grey)),
                    ],
                  ),
                ],
```

```
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```



Satu class KartuDosen dipakai untuk tiga dosen

Penjelasan kode

- `KartuDosen` tidak menyimpan data yang berubah. Semua datanya datang dari luar (parameter konstruktor), jadi cocok sebagai StatelessWidget.
- Karena konstruktornya `const` dan datanya berupa string literal, daftar `children` di `ListView` bisa diberi `const`.
- `Expanded` mencegah teks nama yang panjang menabrak tepi layar (dibahas di Modul 04).

> ℹ️ **"Stateless" bukan berarti tidak pernah digambar ulang**
>
> StatelessWidget tetap bisa tampil berbeda jika **induknya** membangunnya ulang dengan data baru. Yang tidak bisa adalah mengubah tampilannya **sendiri**. Untuk itu kita perlu StatefulWidget di section berikutnya.

⚠️ **Kesalahan umum**

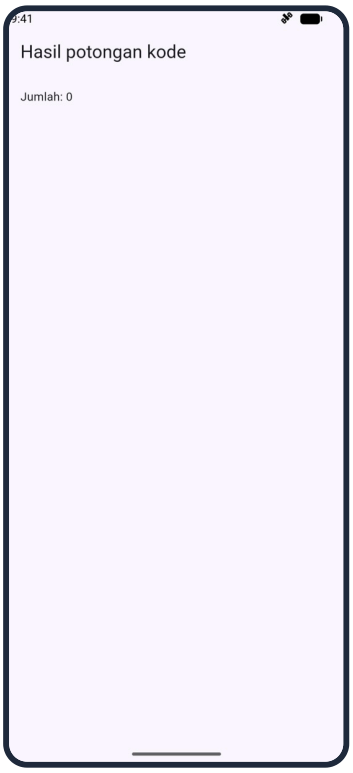
- Mencoba mengubah field di StatelessWidget (`nama = 'baru'`). Field-nya `final` , jadi akan error.
- Langsung memakai StatefulWidget untuk semua hal "berjaga-jaga". Mulailah dari StatelessWidget, ubah ke Stateful hanya jika memang butuh state.

10. StatefulWidget & setState: Counter Lengkap

`StatefulWidget` terdiri dari dua class: class widget (konfigurasi, tetap immutable) dan class `State` (tempat menyimpan data yang berubah). Untuk mengubah data dan memperbarui layar, kita memanggil `setState()`.

Kerangka StatefulWidget

```dart
// Class 1: widget-nya. Hanya bertugas membuat objek State.
class Penghitung extends StatefulWidget {
  const Penghitung({super.key});

  @override
  State<Penghitung> createState() => _PenghitungState();
}

// Class 2: State-nya. Di sinilah data yang berubah disimpan.
class _PenghitungState extends State<Penghitung> {
  int _jumlah = 0; // data (state)

  @override
  Widget build(BuildContext context) {
    return Text('Jumlah: $_jumlah');
  }
}
```



Kerangka Penghitung menampilkan nilai awal 0

Tanda garis bawah (`_`) di depan nama berarti *private*: hanya bisa diakses dari file yang sama. Ini konvensi untuk class State.

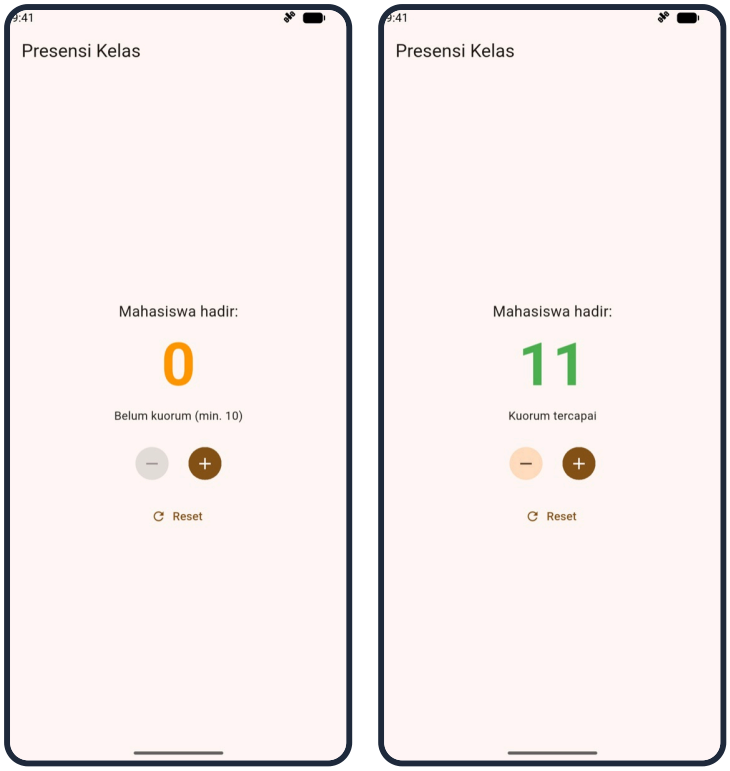
> 💡 **Pintasan VS Code**
>
> Ketik `stful` lalu tekan `Enter` untuk membuat kerangka StatefulWidget, dan `stless` untuk StatelessWidget.

Contoh lengkap: counter dengan tambah, kurang, dan reset

📄 `lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.orange),
      home: const HalamanCounter(),
    );
  }
}

class HalamanCounter extends StatefulWidget {
  const HalamanCounter({super.key});

  @override
  State<HalamanCounter> createState() => _HalamanCounterState();
}

class _HalamanCounterState extends State<HalamanCounter> {
  int _hadir = 0; // jumlah mahasiswa yang hadir

  void _tambah() {
    setState(() {
      _hadir++;
    });
  }

  void _kurang() {
    // Jangan sampai negatif
    if (_hadir == 0) return;
    setState(() {
      _hadir--;
    });
  }
```

```dart
  void _reset() {
    setState(() => _hadir = 0);
  }

  @override
  Widget build(BuildContext context) {
    // build dipanggil ulang setiap setState; print ini membuktikannya
    debugPrint('build() dipanggil, _hadir = $_hadir');

    return Scaffold(
      appBar: AppBar(title: const Text('Presensi Kelas')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Mahasiswa hadir:', style: TextStyle(fontSize: 18)),
            Text(
              '$_hadir',
              style: TextStyle(
                fontSize: 72,
                fontWeight: FontWeight.bold,
                // Warna berubah sesuai state
                color: _hadir >= 10 ? Colors.green : Colors.orange,
              ),
            ),
            Text(_hadir >= 10 ? 'Kuorum tercapai' : 'Belum kuorum (min. 10)'),
            const SizedBox(height: 24),
            Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                IconButton.filledTonal(
                  onPressed: _hadir == 0 ? null : _kurang, // null = tombol nonaktif
                  icon: const Icon(Icons.remove),
                ),
                const SizedBox(width: 16),
                IconButton.filled(
                  onPressed: _tambah,
                  icon: const Icon(Icons.add),
                ),
              ],
            ),
            const SizedBox(height: 16),
            TextButton.icon(
              onPressed: _reset,
              icon: const Icon(Icons.refresh),
              label: const Text('Reset'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Counter presensi: tampilan awal, tombol kurang nonaktif

Setelah tombol tambah ditekan beberapa kali

Apa yang terjadi saat tombol + ditekan?

1. Flutter memanggil `_tambah()`.
2. Di dalam `setState`, nilai `_hadir` bertambah 1.
3. `setState` menandai widget ini sebagai "kotor" (*dirty*), artinya perlu digambar ulang.
4. Pada frame berikutnya, Flutter memanggil ulang `build()`. Karena `_hadir` sudah berubah, `Text` menampilkan angka baru dan warnanya ikut menyesuaikan.

Buka tab **Debug Console** di VS Code: setiap kali tombol ditekan, baris `build() dipanggil` muncul.

Penjelasan kode

- `onPressed: null` membuat tombol nonaktif (abu-abu). Kita memakainya saat nilai sudah 0.
- `setState(() => _hadir = 0)` adalah bentuk singkat (arrow function) dari `setState(() { _hadir = 0; })`.
- Logika yang tidak mengubah tampilan (seperti `if (_hadir == 0) return;`) boleh di luar `setState`.

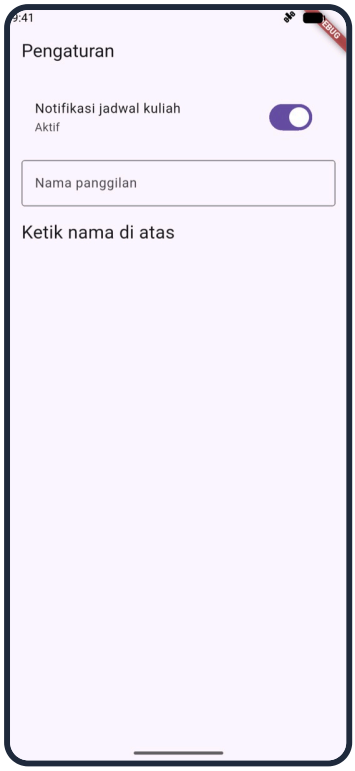
Contoh kedua: toggle dan input teks

State tidak harus angka. Berikut contoh state bertipe `bool` dan `String`:

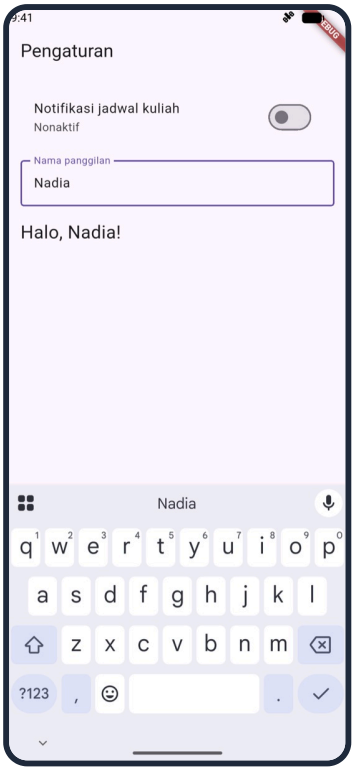
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
```

```dart
void main() {
  runApp(const MaterialApp(home: HalamanPengaturan()));
}

class HalamanPengaturan extends StatefulWidget {
  const HalamanPengaturan({super.key});

  @override
  State<HalamanPengaturan> createState() => _HalamanPengaturanState();
}

class _HalamanPengaturanState extends State<HalamanPengaturan> {
  bool _notifikasiAktif = true;
  String _nama = '';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          SwitchListTile(
            title: const Text('Notifikasi jadwal kuliah'),
            subtitle: Text(_notifikasiAktif ? 'Aktif' : 'Nonaktif'),
            value: _notifikasiAktif,
            onChanged: (nilaiBaru) {
              setState(() => _notifikasiAktif = nilaiBaru);
            },
          ),
          const SizedBox(height: 16),
          TextField(
            decoration: const InputDecoration(
              labelText: 'Nama panggilan',
              border: OutlineInputBorder(),
            ),
            onChanged: (teks) {
              setState(() => _nama = teks);
            },
          ),
          const SizedBox(height: 16),
          Text(
            _nama.isEmpty ? 'Ketik nama di atas' : 'Halo, $_nama!',
            style: const TextStyle(fontSize: 22),
          ),
        ],
      ),
    );
  }
}
```



SwitchListTile dan TextField dengan state



Setelah nama diketik: teks sapaan langsung berubah

Setiap kali switch digeser atau huruf diketik, `setState` memperbarui variabel dan layar langsung menyesuaikan. Form dibahas lebih dalam di Modul 05.

⚠ **Kesalahan umum**

- **Mengubah variabel tanpa `setState`.** Nilainya berubah di memori, tetapi layar tidak diperbarui.
- **Menaruh variabel state di dalam `build()`.** Contoh: `int _hadir = 0;` ditulis di dalam `build`. Nilainya kembali ke 0 setiap build. Variabel state harus menjadi field class State.
- **Memanggil `setState` di dalam `build()`.** Ini memicu build tanpa henti dan error.
- **Menaruh state di class widget** (yang `extends StatefulWidget`). Class widget harus immutable; state disimpan di class `State`.

11. Lifecycle State dengan Demo Log

Objek `State` punya **siklus hidup** (lifecycle): ia dibuat, dipasang ke pohon, digambar berkali-kali, diperbarui, lalu dibuang. Flutter memanggil method tertentu di setiap tahap. Memahami urutannya adalah kunci menghindari kebocoran memori (memory leak) dan error saat aplikasi berjalan.

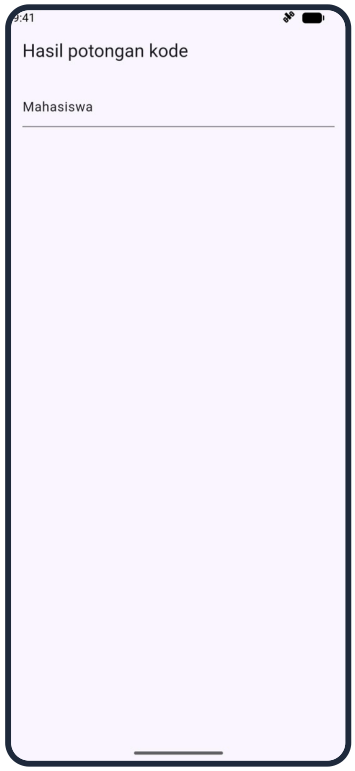
```
createState()
     |
     ▼
initState()              ← sekali, saat State pertama kali dibuat
     |
     ▼
didChangeDependencies()  ← setelah initState, dan tiap kali Theme/MediaQuery yang dipakai berubah
     |
     ▼
build()  ◄──────────────┐  ← berkali-kali
     |                  |
     ├── setState() ────┤
     ├── didUpdateWidget()┤ ← induk membangun ulang dengan parameter baru
     |                  |
     ▼
dispose()                ← sekali, saat widget dibuang dari pohon selamanya
```

1. **`createState()`**: Dipanggil saat framework membuat elemen baru di pohon elemen. Isinya hanya membuat objek State.
2. **`initState()`**: Dipanggil tepat **SATU KALI** saat objek state pertama kali dimasukkan ke tree. Cocok untuk inisialisasi controller, controller animasi, timer, atau mengambil data awal.
3. **`didChangeDependencies()`**: Dipanggil setelah `initState()` dan setiap kali objek yang dibaca lewat `context` (seperti `Theme.of` atau `MediaQuery`) berubah.
4. **`build()`**: Mengembalikan struktur widget tree yang akan digambar. Dipanggil setiap kali ada `setState()`, `didUpdateWidget()`, atau perubahan dependensi.
5. **`didUpdateWidget(oldWidget)`**: Dipanggil saat induk membangun ulang widget ini dengan parameter baru. Kita bisa membandingkan `oldWidget` dengan `widget`.
6. **`setState()`**: Memberi sinyal ke framework bahwa ada data internal yang berubah, memicu pemanggilan ulang `build()`.
7. **`dispose()`**: Dipanggil saat widget dihapus secara permanen dari layar. *Wajib menutup stream, timer, AnimationController, dan TextEditingController di sini agar tidak bocor di memori!*

Contoh kecil: pola initState dan dispose

```dart
class KolomNama extends StatefulWidget {
  const KolomNama({super.key});
```

```dart
  @override
  State<KolomNama> createState() => _KolomNamaState();
}

class _KolomNamaState extends State<KolomNama> {
  late final TextEditingController _controller;

  @override
  void initState() {
    super.initState(); // selalu panggil super.initState() PALING AWAL
    _controller = TextEditingController(text: 'Mahasiswa');
  }

  @override
  void dispose() {
    _controller.dispose(); // bersihkan resource milik kita dulu
    super.dispose();       // super.dispose() PALING AKHIR
  }

  @override
  Widget build(BuildContext context) {
    return TextField(controller: _controller);
  }
}
```



TextField dengan controller yang dibuat di initState

Contoh lengkap: demo log lifecycle

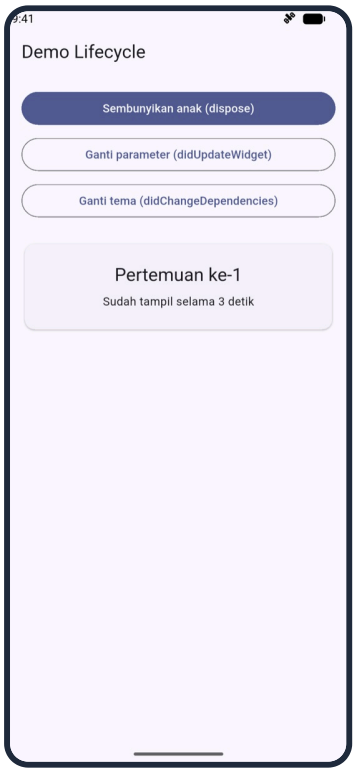
Jalankan kode berikut, buka **Debug Console** (VS Code) atau terminal tempat `flutter run` berjalan, lalu tekan tombol-tombolnya satu per satu sambil membaca log yang muncul.

lib/main.dart

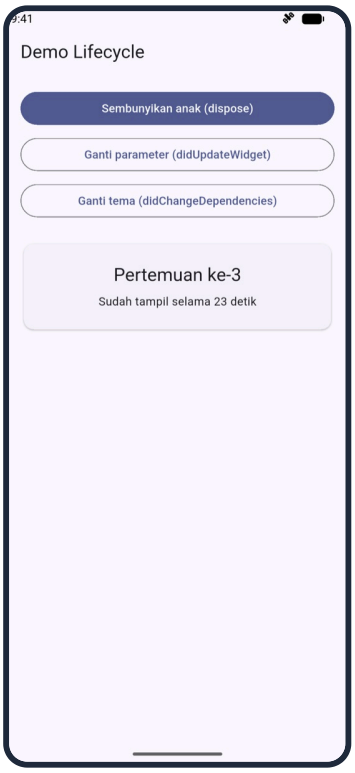
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'dart:async';

import 'package:flutter/material.dart';

void main() {
  runApp(const DemoLifecycleApp());
}

/// Induk: menyimpan 3 state yang mempengaruhi anak.
class DemoLifecycleApp extends StatefulWidget {
  const DemoLifecycleApp({super.key});

  @override
  State<DemoLifecycleApp> createState() => _DemoLifecycleAppState();
}

class _DemoLifecycleAppState extends State<DemoLifecycleApp> {
  bool _tampilkanAnak = true; // untuk memicu initState / dispose
  int _nomorPertemuan = 1;    // untuk memicu didUpdateWidget
  bool _modeGelap = false;    // untuk memicu didChangeDependencies

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      // Matikan animasi pergantian tema agar log tidak tercetak berkali-kali
      themeAnimationDuration: Duration.zero,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: _modeGelap ? Brightness.dark : Brightness.light,
      ),
      home: Scaffold(
        appBar: AppBar(title: const Text('Demo Lifecycle')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              FilledButton(
                onPressed: () => setState(() => _tampilkanAnak = !_tampilkanAnak),
                child: Text(_tampilkanAnak ? 'Sembunyikan anak (dispose)' : 'Tampilkan anak
(initState)'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: () => setState(() => _nomorPertemuan++),
                child: const Text('Ganti parameter (didUpdateWidget)'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: () => setState(() => _modeGelap = !_modeGelap),
                child: const Text('Ganti tema (didChangeDependencies)'),
              ),
              const SizedBox(height: 24),
              if (_tampilkanAnak) KartuLifecycle(nomorPertemuan: _nomorPertemuan),
            ],
```

```
          ),
        ),
      ),
    );
  }
}

/// Anak: mencetak log di setiap tahap lifecycle.
class KartuLifecycle extends StatefulWidget {
  final int nomorPertemuan;

  const KartuLifecycle({super.key, required this.nomorPertemuan});

  @override
  // ignore: no_logic_in_create_state
  State<KartuLifecycle> createState() {
    debugPrint('1. createState()');
    return _KartuLifecycleState();
  }
}

class _KartuLifecycleState extends State<KartuLifecycle> {
  Timer? _timer;
  int _detik = 0;

  @override
  void initState() {
    super.initState();
    debugPrint('2. initState()  -> timer dimulai');
    // Timer berjalan tiap detik. WAJIB dibatalkan di dispose().
    _timer = Timer.periodic(const Duration(seconds: 1), (_) {
      setState(() => _detik++);
    });
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    final gelap = Theme.of(context).brightness == Brightness.dark;
    debugPrint('3. didChangeDependencies()  -> tema gelap: $gelap');
  }

  @override
  void didUpdateWidget(covariant KartuLifecycle oldWidget) {
    super.didUpdateWidget(oldWidget);
    debugPrint(
      '5. didUpdateWidget()  -> pertemuan ${oldWidget.nomorPertemuan} '
      'menjadi ${widget.nomorPertemuan}',
    );
  }

  @override
  void dispose() {
    _timer?.cancel();
    debugPrint('6. dispose()  -> timer dibatalkan setelah $_detik detik');
    super.dispose();
  }

  @override
```

```
  Widget build(BuildContext context) {
    // Dicetak hanya pada build pertama agar log tidak penuh oleh timer
    if (_detik == 0) debugPrint('4. build()');

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            // Akses parameter widget dari State lewat "widget."
            Text(
              'Pertemuan ke-${widget.nomorPertemuan}',
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 8),
            Text('Sudah tampil selama $_detik detik'),
          ],
        ),
      ),
    );
  }
}
```



Demo lifecycle: kartu anak dengan timer



Setelah parameter diganti dua kali (didUpdateWidget)

Contoh log yang muncul

```
// Saat aplikasi dibuka
1. createState()
2. initState()  -> timer dimulai
3. didChangeDependencies()  -> tema gelap: false
4. build()
```

```
// Tekan "Ganti parameter"
5. didUpdateWidget()  -> pertemuan 1 menjadi 2

// Tekan "Ganti tema": induk membangun ulang (didUpdateWidget),
// lalu Theme yang dibaca anak berubah (didChangeDependencies)
5. didUpdateWidget()  -> pertemuan 2 menjadi 2
3. didChangeDependencies()  -> tema gelap: true

// Tekan "Sembunyikan anak"
6. dispose()  -> timer dibatalkan setelah 12 detik

// Tekan "Tampilkan anak" lagi: State BARU dibuat, detik mulai dari 0
1. createState()
2. initState()  -> timer dimulai
3. didChangeDependencies()  -> tema gelap: true
4. build()
```

Penjelasan kode

- `widget.nomorPertemuan`: dari dalam class State, parameter milik widget diakses lewat properti `widget`.
- `initState` tidak boleh memakai `Theme.of(context)` atau `MediaQuery.of(context)`. Hal yang bergantung pada `context` diletakkan di `didChangeDependencies` atau `build`.
- Jika baris `_timer?.cancel()` dihapus, timer tetap berjalan setelah widget hilang dan memanggil `setState` pada State yang sudah dibuang. Hasilnya error *"setState() called after dispose()"*.
- `didUpdateWidget` dipanggil setiap kali induk memanggil `setState`, walaupun nilainya sama (lihat "pertemuan 2 menjadi 2"). Karena itu, di dalamnya biasanya kita membandingkan dulu: `if (oldWidget.nomorPertemuan != widget.nomorPertemuan) { ... }`.
- Saat anak ditampilkan lagi, `_detik` kembali ke 0. Ini membuktikan State lama sudah dibuang dan State baru dibuat.

Pengecekan `mounted` setelah `await`

Jika kamu menunggu proses lama (misalnya request API) lalu memanggil `setState`, cek dulu apakah widget masih terpasang:

```dart
Future<void> _muatData() async {
  await Future.delayed(const Duration(seconds: 2)); // pura-pura request API
  if (!mounted) return; // widget sudah ditutup? jangan setState
  setState(() {
    // perbarui data di sini
  });
}
```

⚠️ **Kesalahan umum**

- Lupa memanggil `super.initState()` di awal atau `super.dispose()` di akhir.
- Membuat controller (`TextEditingController`, `AnimationController`) di dalam `build()`. Controller akan dibuat ulang setiap build dan isinya hilang. Buat di `initState`.
- Tidak membatalkan `Timer` atau `StreamSubscription` di `dispose()`.
- Memanggil `setState` di `dispose()`, atau setelah `await` tanpa cek `mounted`.

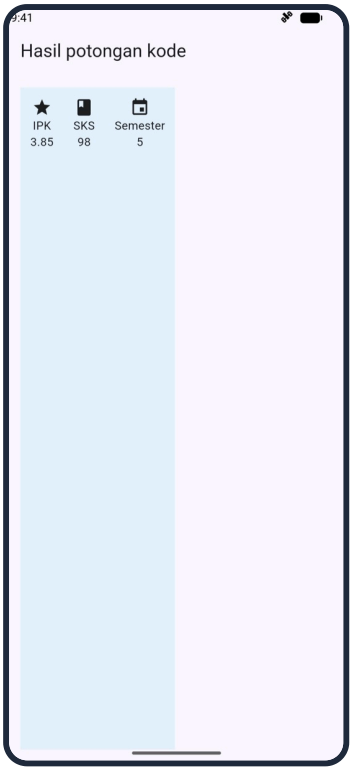
12. Memecah Widget: Parameter & Callback

Method `build()` yang panjangnya ratusan baris sulit dibaca dan sulit diperbaiki. Solusinya: **pecah menjadi widget-widget kecil**. Data dikirim ke widget kecil lewat **parameter konstruktor**, dan widget kecil mengabari induknya lewat **callback** (fungsi yang dikirim sebagai parameter).

Langkah 1: kenali bagian yang berulang

Perhatikan kode "sebelum" berikut. Ada tiga blok `Container` yang hampir sama, hanya berbeda ikon, label, dan nilai:

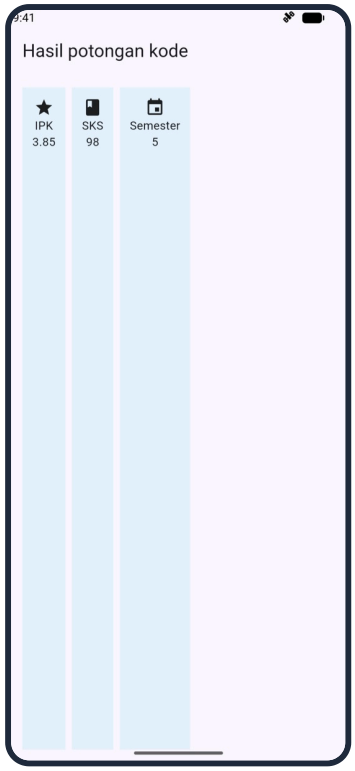
```
// SEBELUM: kode berulang
Row(
  children: [
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.star), Text('IPK'), Text('3.85')]),
    ),
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.book), Text('SKS'), Text('98')]),
    ),
    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: const Column(children: [Icon(Icons.event), Text('Semester'), Text('5')]),
    ),
  ],
)
```



Kode berulang: tiga kotak statistik

Langkah 2: jadikan class dengan parameter

```
// SESUDAH: satu class, dipakai tiga kali
class KotakStatistik extends StatelessWidget {
  final IconData ikon;
  final String label;
  final String nilai;

  const KotakStatistik({
    super.key,
    required this.ikon,
    required this.label,
    required this.nilai,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(12),
      color: Colors.blue.shade50,
      child: Column(children: [Icon(ikon), Text(label), Text(nilai)]),
    );
  }
}

// Pemakaian:
// Row(children: [
//   KotakStatistik(ikon: Icons.star, label: 'IPK', nilai: '3.85'),
//   KotakStatistik(ikon: Icons.book, label: 'SKS', nilai: '98'),
//   KotakStatistik(ikon: Icons.event, label: 'Semester', nilai: '5'),
// ])
```



Hasil yang sama memakai class KotakStatistik

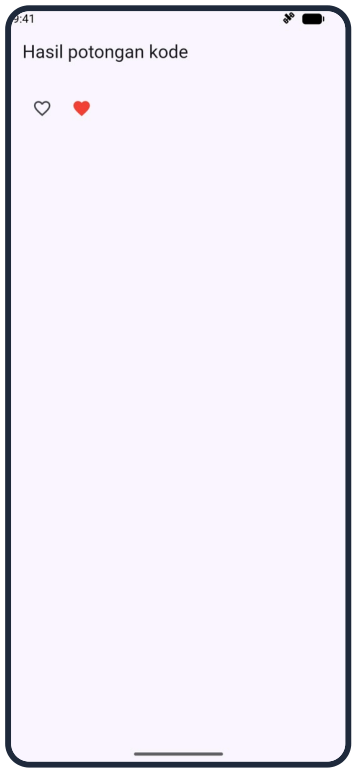
> 💡 **Refactor otomatis di VS Code**
>
> Letakkan kursor pada nama widget (misalnya `Container`), tekan `Ctrl+.` (macOS: `Cmd+.`), lalu pilih **Extract Widget**. VS Code membuatkan class baru untukmu.

Langkah 3: callback, agar anak bisa "melapor" ke induk

Widget kecil sebaiknya tidak menyimpan state yang juga dibutuhkan induk. Biarkan induk yang menyimpan state, lalu kirim fungsi ke anak. Tipe yang paling sering dipakai:

Tipe	Artinya	Contoh pemakaian
`VoidCallback`	Fungsi tanpa parameter, tanpa nilai kembali	Tombol "Hapus" ditekan
`ValueChanged<T>`	Fungsi yang menerima satu nilai bertipe `T`	Checkbox berubah menjadi `true` / `false`
`void Function(String, int)`	Fungsi dengan parameter bebas	Mengirim nama dan jumlah sekaligus

```dart
class TombolSuka extends StatelessWidget {
  final bool disukai;
  final VoidCallback onTekan; // induk yang menentukan apa yang terjadi

  const TombolSuka({super.key, required this.disukai, required this.onTekan});

  @override
  Widget build(BuildContext context) {
    return IconButton(
      onPressed: onTekan, // teruskan ke induk
      icon: Icon(
        disukai ? Icons.favorite : Icons.favorite_border,
        color: disukai ? Colors.red : null,
      ),
    );
  }
}
```



TombolSuka dalam keadaan belum dan sudah disukai

Contoh lengkap: daftar tugas kuliah yang dipecah menjadi widget kecil

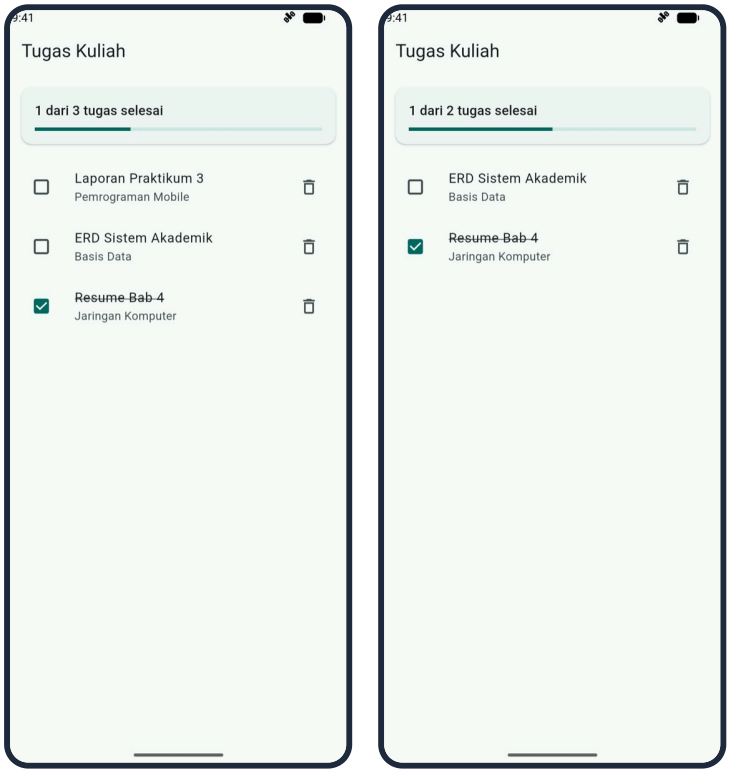
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const HalamanTugas(),
    );
  }
}

/// Model data sederhana untuk satu tugas.
class Tugas {
  final String judul;
  final String matkul;
  bool selesai;

  Tugas({required this.judul, required this.matkul, this.selesai = false});
}

/// INDUK: satu-satunya tempat state disimpan.
class HalamanTugas extends StatefulWidget {
  const HalamanTugas({super.key});

  @override
  State<HalamanTugas> createState() => _HalamanTugasState();
}

class _HalamanTugasState extends State<HalamanTugas> {
  final List<Tugas> _daftar = [
    Tugas(judul: 'Laporan Praktikum 3', matkul: 'Pemrograman Mobile'),
    Tugas(judul: 'ERD Sistem Akademik', matkul: 'Basis Data'),
    Tugas(judul: 'Resume Bab 4', matkul: 'Jaringan Komputer', selesai: true),
  ];

  void _ubahStatus(int index, bool nilai) {
    setState(() => _daftar[index].selesai = nilai);
  }

  void _hapus(int index) {
    setState(() => _daftar.removeAt(index));
  }

  @override
  Widget build(BuildContext context) {
    final jumlahSelesai = _daftar.where((t) => t.selesai).length;

    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Kuliah')),
      body: Column(
        children: [
          // Widget kecil 1: hanya menerima data
          RingkasanTugas(selesai: jumlahSelesai, total: _daftar.length),
          Expanded(
            child: ListView.builder(
              itemCount: _daftar.length,
              itemBuilder: (context, index) {
                // Widget kecil 2: menerima data + 2 callback
                return ItemTugas(
```

```
                  tugas: _daftar[index],
                  onCentang: (nilai) => _ubahStatus(index, nilai),
                  onHapus: () => _hapus(index),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

/// ANAK 1: StatelessWidget yang hanya menampilkan ringkasan.
class RingkasanTugas extends StatelessWidget {
  final int selesai;
  final int total;

  const RingkasanTugas({super.key, required this.selesai, required this.total});

  @override
  Widget build(BuildContext context) {
    final progres = total == 0 ? 0.0 : selesai / total;
    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('$selesai dari $total tugas selesai',
                style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            LinearProgressIndicator(value: progres),
          ],
        ),
      ),
    );
  }
}

/// ANAK 2: menampilkan satu tugas dan melapor ke induk lewat callback.
class ItemTugas extends StatelessWidget {
  final Tugas tugas;
  final ValueChanged<bool> onCentang;
  final VoidCallback onHapus;

  const ItemTugas({
    super.key,
    required this.tugas,
    required this.onCentang,
    required this.onHapus,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: Checkbox(
        value: tugas.selesai,
```

```
        // Checkbox memberi bool? (bisa null), kita ubah jadi bool
        onChanged: (nilai) => onCentang(nilai ?? false),
      ),
      title: Text(
        tugas.judul,
        style: TextStyle(
          decoration: tugas.selesai ? TextDecoration.lineThrough : null,
        ),
      ),
      subtitle: Text(tugas.matkul),
      trailing: IconButton(
        icon: const Icon(Icons.delete_outline),
        tooltip: 'Hapus',
        onPressed: onHapus,
      ),
    );
  }
}
```



Daftar tugas yang dipecah menjadi widget kecil

Setelah tombol hapus pada item pertama ditekan

Penjelasan alur data

```
_HalamanTugasState (punya state _daftar)
   |  data turun lewat parameter (selesai, total, tugas)
   ▼
RingkasanTugas     ItemTugas
                      |  kejadian naik lewat callback (onCentang, onHapus)
                      ▼
              _ubahStatus / _hapus  →  setState  →  build ulang
```

- **Data turun, kejadian naik.** Ini pola paling penting dalam Flutter. Anak tidak mengubah data sendiri, ia memanggil callback dan induk yang memutuskan.
- `RingkasanTugas` dan `ItemTugas` adalah StatelessWidget walaupun tampilannya berubah, karena state disimpan di induk.
- `(nilai) => _ubahStatus(index, nilai)` adalah fungsi anonim yang "mengingat" `index` item tersebut.

Widget class vs method helper

Kamu mungkin melihat kode yang memecah UI dengan method seperti `Widget _buildHeader() { ... }`. Cara ini boleh untuk potongan kecil, tetapi **class widget lebih disarankan** karena bisa `const`, punya `context` sendiri, dan Flutter bisa melewati build-nya jika datanya tidak berubah.

> ⚠️ **Kesalahan umum**
>
> - **Menyimpan salinan state di anak** (misalnya anak punya `bool _selesai` sendiri). Data induk dan anak jadi tidak sinkron.
> - **Memanggil callback saat build**: `onPressed: onHapus()` (dengan kurung) langsung menjalankan fungsi saat build. Yang benar `onPressed: onHapus` tanpa kurung.
> - **Memecah terlalu kecil** sampai setiap `Text` jadi class. Pecah berdasarkan bagian yang bermakna: header, kartu, item daftar.

13. const & Key

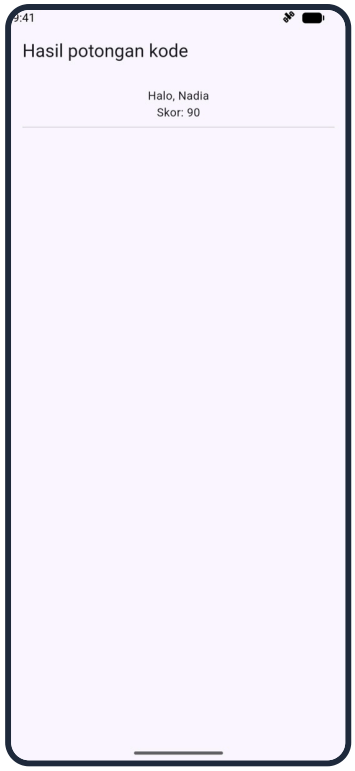
Dua hal kecil yang sering muncul di kode Flutter: kata kunci `const` (supaya widget tidak dibuat ulang tanpa perlu) dan parameter `key` (supaya Flutter bisa mengenali widget yang sama ketika urutannya berubah).

Bagian 1: `const`

Widget yang ditandai `const` dibuat **sekali saat kompilasi** dan dipakai ulang. Saat induknya build ulang, Flutter tahu widget `const` pasti sama sehingga bisa melewatinya. Hasilnya aplikasi lebih hemat kerja.

```
// Boleh const: semua nilainya sudah pasti saat kompilasi
const judul = Text('Jadwal Kuliah');
const jarak = SizedBox(height: 16);
const ikon = Icon(Icons.home, color: Colors.indigo);
const kotak = Padding(padding: EdgeInsets.all(8), child: Text('Halo'));
```

```
// TIDAK boleh const: ada nilai yang baru diketahui saat aplikasi berjalan
Widget contoh(String nama, int skor) {
  return Column(
    children: [
      Text('Halo, $nama'),                  // nama berasal dari variabel
      Text('Skor: $skor'),
      Container(color: Colors.indigo.shade100), // .shade100 bukan konstanta
      // Tetapi anak yang nilainya tetap tetap boleh const:
      const Divider(),
    ],
  );
}
```



Widget dengan nilai dari variabel (tidak bisa const)

Syarat `const`	Contoh
Class punya konstruktor `const`	`const MyWidget({super.key});`
Semua argumen juga konstanta	Literal `'teks'`, angka, `Colors.red`, `Icons.home`, `EdgeInsets.all(8)`
Tidak memakai variabel atau pemanggilan method	Bukan `Text(nama)`, bukan `Theme.of(context)`

> 💡 **Biarkan linter membantu**
>
> Proyek baru Flutter sudah memakai `flutter_lints`. Jika muncul garis biru *"Use 'const' with the constructor to improve performance"*, tekan `Ctrl+.` lalu pilih **Add 'const' modifier**. Perintah `dart fix --apply` bisa menambahkan `const` di seluruh proyek sekaligus.

Bagian 2: `Key`

Flutter mencocokkan widget lama dan baru berdasarkan **tipe dan posisinya**. Biasanya ini cukup. Masalah muncul jika ada beberapa **StatefulWidget bertipe sama** dalam satu daftar lalu

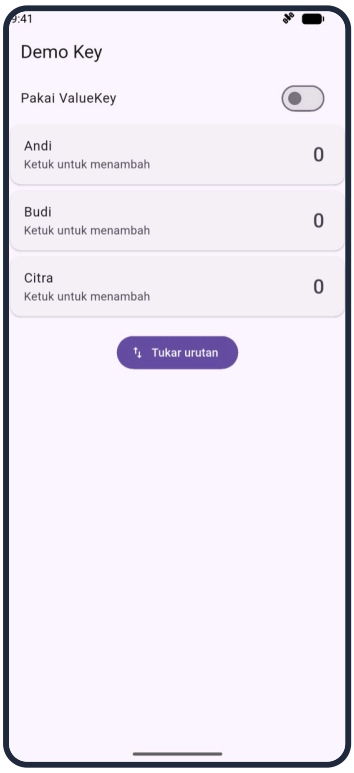
urutannya berubah (dihapus, diurutkan, ditukar). Tanpa key, State bisa "tertinggal" di posisi lama dan menempel ke data yang salah.

Contoh berikut memperlihatkan masalah itu. Setiap kotak menyimpan angka hitungan di State-nya sendiri. Tekan beberapa kotak, lalu tekan tombol **Tukar urutan**.

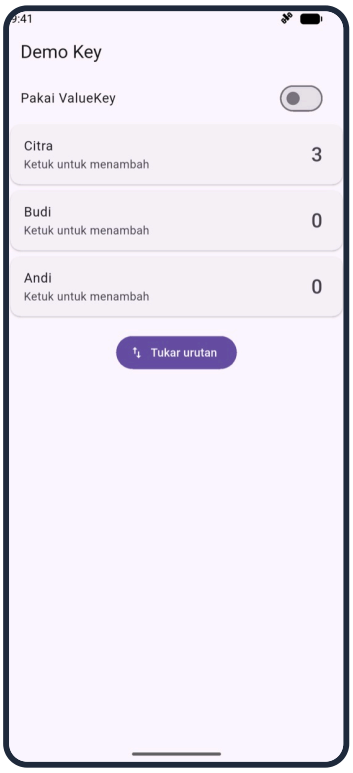
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: DemoKey()));
}

class DemoKey extends StatefulWidget {
  const DemoKey({super.key});

  @override
  State<DemoKey> createState() => _DemoKeyState();
}

class _DemoKeyState extends State<DemoKey> {
  List<String> _nama = ['Andi', 'Budi', 'Citra'];
  bool _pakaiKey = false;

  void _tukar() {
    setState(() => _nama = _nama.reversed.toList());
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Key')),
      body: Column(
        children: [
          SwitchListTile(
            title: const Text('Pakai ValueKey'),
            value: _pakaiKey,
            onChanged: (v) => setState(() => _pakaiKey = v),
          ),
          for (final n in _nama)
            KotakHitung(
              // Tanpa key: State dicocokkan berdasarkan posisi
              // Dengan key: State dicocokkan berdasarkan nama
              key: _pakaiKey ? ValueKey(n) : null,
              nama: n,
            ),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: _tukar,
            icon: const Icon(Icons.swap_vert),
            label: const Text('Tukar urutan'),
          ),
        ],
      ),
    );
  }
```

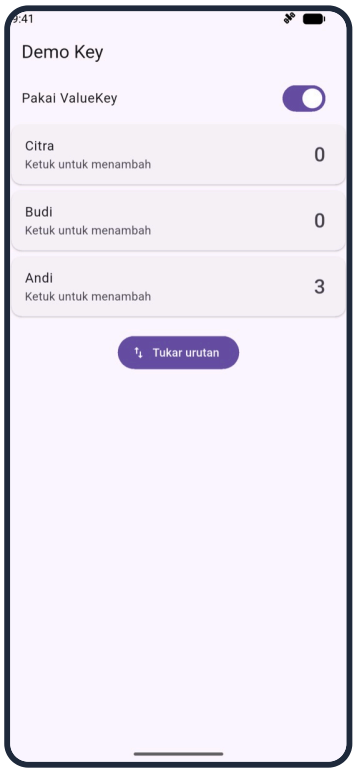
```dart
}

class KotakHitung extends StatefulWidget {
  final String nama;

  const KotakHitung({super.key, required this.nama});

  @override
  State<KotakHitung> createState() => _KotakHitungState();
}

class _KotakHitungState extends State<KotakHitung> {
  int _hitung = 0; // state milik masing-masing kotak

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(widget.nama),
        subtitle: const Text('Ketuk untuk menambah'),
        trailing: Text('$_hitung', style: const TextStyle(fontSize: 24)),
        onTap: () => setState(() => _hitung++),
      ),
    );
  }
}
```



Tampilan awal demo Key



Tanpa key: Andi diketuk 3x lalu ditukar, angka 3 tertinggal di baris pertama



Dengan ValueKey: angka 3 ikut pindah bersama Andi

Apa yang terjadi?

- **Tanpa key:** ketuk Andi 3 kali (Andi = 3), lalu tukar urutan. Sekarang baris pertama berisi "Citra" tetapi angkanya tetap 3. State tetap di posisi pertama, sementara namanya pindah. Ini

salah.

- **Dengan `ValueKey(n)`:** ulangi langkah yang sama. Angka 3 ikut pindah bersama "Andi" ke baris paling bawah. Flutter mencocokkan State berdasarkan key, bukan posisi.

Jenis key yang umum

Key	Kapan dipakai
`ValueKey(id)`	Item daftar yang punya nilai unik (id, NIM, email). Paling sering dipakai.
`ObjectKey(objek)`	Keunikan ditentukan oleh objeknya sendiri.
`UniqueKey()`	Memaksa widget selalu dianggap baru (State di-reset).
`GlobalKey`	Mengakses State dari luar, misalnya `GlobalKey<FormState>` di Modul 05.

⚠️ **Kesalahan umum**

- Memakai **index** sebagai key (`ValueKey(index)`). Index ikut berubah saat urutan berubah, jadi tidak membantu. Pakai id unik dari data.
- Membuat `UniqueKey()` atau `GlobalKey()` di dalam `build()` . Key baru di setiap build membuat State terus di-reset.
- Menambahkan `const` pada widget yang argumennya berasal dari variabel. Compiler akan error *"Invalid constant value"*.

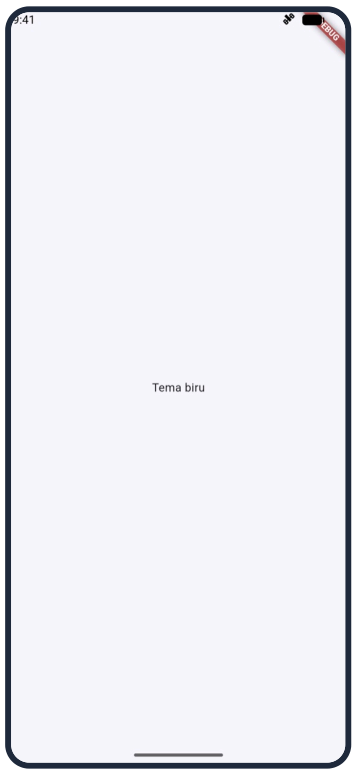
14. ThemeData Material 3 & Mode Gelap

Daripada mengatur warna dan ukuran huruf di setiap widget, Flutter menyediakan `ThemeData`: satu tempat untuk mengatur tampilan seluruh aplikasi. Sejak Flutter 3.16, tema bawaan sudah memakai **Material 3**.

ColorScheme.fromSeed: satu warna, satu palet lengkap

Cukup berikan satu **warna benih** (seed). Flutter menghitung palet lengkap yang serasi: `primary`, `secondary`, `tertiary`, `surface`, `error`, beserta pasangan warna teks di atasnya (`onPrimary`, `onSurface`, dan seterusnya).

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0D47A1)),
  ),
  home: const Scaffold(body: Center(child: Text('Tema biru'))),
)
```



Tema dari ColorScheme.fromSeed

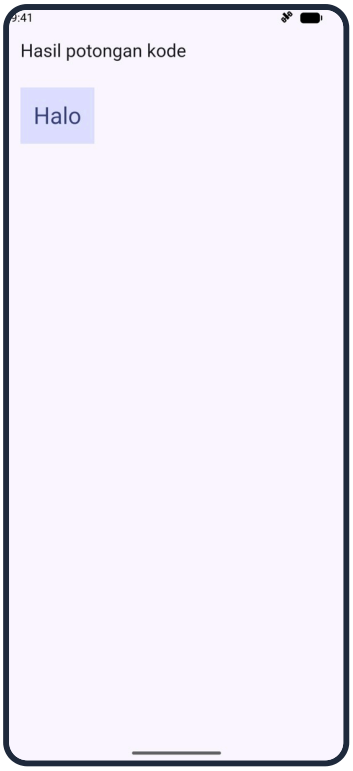
Bentuk singkatnya adalah `ThemeData(colorSchemeSeed: Colors.indigo)`, yang sudah kita pakai di contoh-contoh sebelumnya.

Peran warna di ColorScheme

Warna	Dipakai untuk
`primary` / `onPrimary`	Tombol utama, elemen penting / teks di atasnya.
`primaryContainer` / `onPrimaryContainer`	Latar lembut untuk kartu atau chip yang ditonjolkan.
`secondary`, `tertiary`	Aksen pendukung.
`surface` / `onSurface`	Latar halaman dan kartu / teks umum.
`error`	Pesan kesalahan.

Membaca tema dari widget

```dart
Widget kotakTema(BuildContext context) {
  final warna = Theme.of(context).colorScheme;
  final teks = Theme.of(context).textTheme;

  return Container(
    padding: const EdgeInsets.all(16),
    color: warna.primaryContainer,
    child: Text(
      'Halo',
      style: teks.headlineMedium?.copyWith(color: warna.onPrimaryContainer),
    ),
  );
}
```



Warna primaryContainer dan teks headlineMedium dari tema

Skala teks Material 3 (`textTheme`)

Kelompok	Nama gaya	Contoh pemakaian
Display	`displayLarge/Medium/Small`	Angka besar, hero
Headline	`headlineLarge/Medium/Small`	Judul halaman
Title	`titleLarge/Medium/Small`	Judul kartu, AppBar
Body	`bodyLarge/Medium/Small`	Paragraf
Label	`labelLarge/Medium/Small`	Teks tombol, chip

Contoh lengkap: tema terang, tema gelap, dan tombol ganti tema

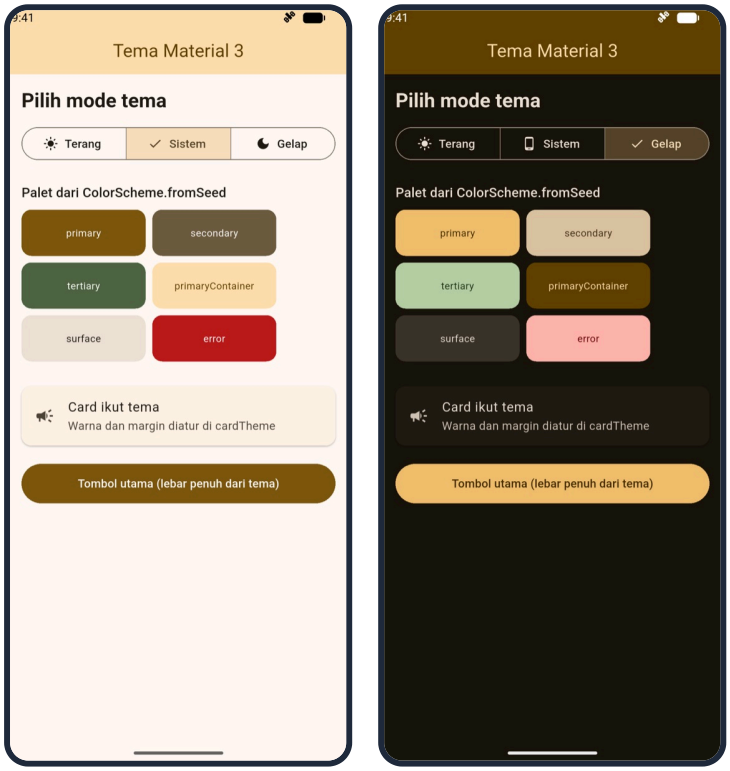
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const AplikasiBertema());
}

class AplikasiBertema extends StatefulWidget {
```

```dart
  const AplikasiBertema({super.key});

  @override
  State<AplikasiBertema> createState() => _AplikasiBertemaState();
}

class _AplikasiBertemaState extends State<AplikasiBertema> {
  // system = ikut pengaturan HP, light = terang, dark = gelap
  ThemeMode _mode = ThemeMode.system;

  static const Color _warnaKampus = Color(0xFFF2A900); // kuning keemasan

  ThemeData _buatTema(Brightness brightness) {
    final skema = ColorScheme.fromSeed(
      seedColor: _warnaKampus,
      brightness: brightness,
    );
    return ThemeData(
      colorScheme: skema,
      // Atur gaya teks tertentu, sisanya ikut bawaan Material 3
      textTheme: const TextTheme(
        headlineSmall: TextStyle(fontWeight: FontWeight.bold),
      ),
      appBarTheme: AppBarTheme(
        backgroundColor: skema.primaryContainer,
        foregroundColor: skema.onPrimaryContainer,
        centerTitle: true,
      ),
      cardTheme: const CardThemeData(
        margin: EdgeInsets.symmetric(vertical: 6),
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: _buatTema(Brightness.light),      // dipakai saat mode terang
      darkTheme: _buatTema(Brightness.dark),   // dipakai saat mode gelap
      themeMode: _mode,                        // menentukan mana yang aktif
      home: HalamanTema(
        mode: _mode,
        onGantiMode: (modeBaru) => setState(() => _mode = modeBaru),
      ),
    );
  }
}

class HalamanTema extends StatelessWidget {
  final ThemeMode mode;
  final ValueChanged<ThemeMode> onGantiMode;

  const HalamanTema({super.key, required this.mode, required this.onGantiMode});

  @override
```

```dart
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    final teks = Theme.of(context).textTheme;

    return Scaffold(
      appBar: AppBar(title: const Text('Tema Material 3')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Pilih mode tema', style: teks.headlineSmall),
          const SizedBox(height: 12),
          SegmentedButton<ThemeMode>(
            segments: const [
              ButtonSegment(value: ThemeMode.light, icon: Icon(Icons.light_mode), label:
Text('Terang')),
              ButtonSegment(value: ThemeMode.system, icon: Icon(Icons.phone_android), label:
Text('Sistem')),
              ButtonSegment(value: ThemeMode.dark, icon: Icon(Icons.dark_mode), label:
Text('Gelap')),
            ],
            selected: {mode},
            onSelectionChanged: (pilihan) => onGantiMode(pilihan.first),
          ),
          const SizedBox(height: 24),
          Text('Palet dari ColorScheme.fromSeed', style: teks.titleMedium),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            runSpacing: 8,
            children: [
              KotakWarna(nama: 'primary', latar: warna.primary, teks: warna.onPrimary),
              KotakWarna(nama: 'secondary', latar: warna.secondary, teks: warna.onSecondary),
              KotakWarna(nama: 'tertiary', latar: warna.tertiary, teks: warna.onTertiary),
              KotakWarna(nama: 'primaryContainer', latar: warna.primaryContainer, teks:
warna.onPrimaryContainer),
              KotakWarna(nama: 'surface', latar: warna.surfaceContainerHighest, teks:
warna.onSurface),
              KotakWarna(nama: 'error', latar: warna.error, teks: warna.onError),
            ],
          ),
          const SizedBox(height: 24),
          const Card(
            child: ListTile(
              leading: Icon(Icons.campaign),
              title: Text('Card ikut tema'),
              subtitle: Text('Warna dan margin diatur di cardTheme'),
            ),
          ),
          const SizedBox(height: 16),
          FilledButton(onPressed: () {}, child: const Text('Tombol utama (lebar penuh dari tema)')),
        ],
      ),
    );
  }
}

class KotakWarna extends StatelessWidget {
  final String nama;
  final Color latar;
```

```
  final Color teks;

  const KotakWarna({super.key, required this.nama, required this.latar, required this.teks});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 150,
      height: 56,
      alignment: Alignment.center,
      decoration: BoxDecoration(color: latar, borderRadius: BorderRadius.circular(12)),
      child: Text(nama, style: TextStyle(color: teks, fontSize: 12)),
    );
  }
}
```



Tema terang Material 3 dengan palet warna

Setelah memilih mode Gelap

Penjelasan kode

- `theme` dan `darkTheme` dibuat dari fungsi yang sama, hanya berbeda `brightness`. Palet gelap otomatis dihitung dari seed yang sama.
- `themeMode` menentukan tema mana yang dipakai. `ThemeMode.system` mengikuti pengaturan mode gelap di HP.
- `appBarTheme`, `cardTheme`, dan `filledButtonTheme` mengatur gaya default semua AppBar, Card, dan FilledButton sekaligus. Itulah sebabnya Card dan tombol di halaman tidak perlu diatur satu per satu.
- Pilihan mode disimpan di `_AplikasiBertemaState` (induk paling atas) dan diubah lewat callback `onGantiMode`, sesuai pola "data turun, kejadian naik".

- `SegmentedButton` adalah tombol pilihan bersegmen dari Material 3.

> ℹ️ **Menyimpan pilihan tema**
>
> Saat aplikasi ditutup, pilihan tema di contoh ini kembali ke "Sistem". Menyimpannya secara permanen akan dipelajari di Modul 10 (SharedPreferences) dan mengelolanya lintas halaman di Modul 08 (Provider).

> ⚠️ **Kesalahan umum**
>
> - **Memberi `brightness` berbeda** antara `ThemeData` dan `ColorScheme`. Atur `brightness` di `ColorScheme.fromSeed` seperti contoh di atas.
> - **Menulis warna langsung** (`Colors.white`, `Colors.black`) di banyak tempat. Di mode gelap teks hitam di latar gelap jadi tidak terbaca. Gunakan `colorScheme.onSurface` dan kawan-kawannya.
> - **Mengisi `darkTheme` tetapi lupa `themeMode`**. Default-nya `ThemeMode.system`, jadi tema gelap hanya muncul jika HP sedang mode gelap.
> - **Memakai `CardTheme`** di `ThemeData.cardTheme`. Versi Flutter terbaru memakai `CardThemeData`.

Tugas Praktikum: Praktikum 3: Kartu Mahasiswa Interaktif dengan Toggle Status Cuti

Sasaran Praktikum

Membuat aplikasi kecil berisi kartu identitas mahasiswa yang dipecah menjadi beberapa widget, memakai StatefulWidget untuk status keaktifan, dan mendukung tema terang/gelap Material 3.

Langkah Pengerjaan

1. Buat project baru `flutter create praktikum_03`, lalu hapus seluruh isi `lib/main.dart`.
2. Atur `MaterialApp` dengan `theme` dan `darkTheme` dari `ColorScheme.fromSeed` memakai satu warna benih pilihanmu, serta `themeMode` yang bisa diganti dari tombol di `AppBar.actions`.
3. Buat `StatefulWidget` bernama `KtmViewerScreen` yang menyimpan `bool _isStatusAktif = true;`.
4. Pecah tampilan menjadi minimal tiga widget kecil: `KartuKtm` (menerima nama, NIM, prodi, status), `BarisInfo` (ikon + label + nilai), dan `TombolStatus` (menerima `VoidCallback onTekan`).
5. Di `KartuKtm`, gunakan `Container` + `BoxDecoration` bergradasi, sudut membulat, dan bayangan (`withValues(alpha: ...)`). Tampilkan foto memakai `CircleAvatar` dengan `NetworkImage` dan ikon cadangan.
6. Tombol status memanggil `setState()` di induk untuk mengubah status (Aktif ↔ Cuti Akademik). Jika Cuti, ubah gradasi kartu menjadi abu-abu dan ikon menjadi peringatan oranye.
7. Di bawah kartu, tampilkan daftar 3 mata kuliah semester ini memakai `Card` + `ListTile` (leading, title, subtitle, trailing).
8. Tambahkan `initState` dan `dispose` pada `KtmViewerScreen` yang mencetak log dengan `debugPrint`, lalu screenshot Debug Console sebagai bukti.
9. Pastikan `flutter analyze` tidak menampilkan error/warning dan gunakan `const` di semua tempat yang memungkinkan.

Output yang Diharapkan

Aplikasi kartu mahasiswa yang rapi, berganti warna dan status saat tombol ditekan, mendukung mode terang/gelap, dan kodenya terpecah menjadi beberapa widget kecil dengan parameter dan callback.

Rubrik Penilaian

Penggunaan StatefulWidget, setState, dan lifecycle yang tepat (30%), Pemecahan widget dengan parameter & callback (25%), Desain kartu dengan Container/BoxDecoration/Card/ListTile (20%), Tema Material 3 & mode gelap (15%), Kerapian kode, const, dan bebas warning analyzer (10%).

Kuis Pemahaman

1. Metode lifecycle manakah pada State yang HANYA dipanggil satu kali saat widget pertama kali diinisialisasi?

A. build()

B. setState()

C. initState()

D. dispose()

2. Mengapa kita tidak boleh memanggil 'setState()' di dalam method 'dispose()'?

A. Karena method dispose() dipanggil saat widget sedang dihancurkan/dihapus dari tree sehingga tidak mungkin me-render ulang lagi

B. Karena compiler Dart akan menghapus seluruh file project

C. Karena dispose() hanya boleh memuat kode styling CSS

D. Karena setState() hanya boleh dipanggil di StatelessWidget

3. Setelah menambahkan folder assets/images/ berisi logo.png, aplikasi menampilkan error 'Unable to load asset'. Penyebab yang paling mungkin adalah...

A. Image.asset hanya mendukung format JPG

B. Folder belum didaftarkan di bagian flutter: assets: pada pubspec.yaml

C. Gambar harus disimpan di dalam folder lib/

D. Perlu menambahkan izin INTERNET di AndroidManifest

4. Apa yang terjadi jika sebuah variabel state diubah TANPA dibungkus setState()?

A. Aplikasi langsung crash

B. Nilai di memori berubah tetapi layar tidak digambar ulang

C. Flutter otomatis memanggil build()

D. Variabel kembali ke nilai awal

5. Menulis Container(color: Colors.red, decoration: BoxDecoration(borderRadius: BorderRadius.circular(8))) menghasilkan error. Bagaimana perbaikannya?

A. Hapus borderRadius

B. Pindahkan color ke dalam BoxDecoration(color: Colors.red, ...)

C. Ganti Container dengan SizedBox

D. Tambahkan const di depan Container

6. Sebuah widget anak perlu memberi tahu induknya bahwa tombol hapus ditekan. Tipe parameter yang paling tepat adalah...

A. String

B. VoidCallback

C. BuildContext

D. Key

7. Daftar StatefulWidget bertipe sama ditukar urutannya, tetapi state (angka hitungan) tertinggal di posisi lama. Solusinya adalah...

A. Memberi setiap item ValueKey berdasarkan id unik datanya

B. Menambahkan const pada setiap item

C. Mengganti semua item menjadi StatelessWidget

D. Memanggil setState dua kali

8. Kode MaterialApp(theme: ..., darkTheme: ..., themeMode: ThemeMode.system). Kapan darkTheme dipakai?

A. Selalu dipakai

B. Tidak pernah dipakai

C. Saat pengaturan perangkat sedang dalam mode gelap

D. Hanya saat aplikasi dijalankan di web

Kunci Jawaban & Pembahasan

1. C — initState() dipanggil tepat satu kali oleh framework ketika objek State dimasukkan ke pohon elemen untuk inisialisasi awal.

2. A — dispose() adalah tahap akhir penghancuran widget dari memori. Memanggil setState() di fase ini akan memicu runtime error karena elemen sudah tidak terpasang di tree.

3. B — Asset harus didaftarkan di pubspec.yaml (di bawah flutter: → assets:), lalu jalankan flutter pub get dan jalankan ulang aplikasi.

4. B — setState() yang memberi tahu framework bahwa widget perlu dibangun ulang. Tanpanya, perubahan nilai tidak terlihat di layar.

5. B — Container tidak boleh menerima color dan decoration sekaligus. Jika memakai decoration, warna ditulis di BoxDecoration.

6. B — VoidCallback adalah fungsi tanpa parameter dan tanpa nilai kembali. Induk mengirim fungsi tersebut, anak memanggilnya saat tombol ditekan (data turun, kejadian naik).

7. A — Tanpa key, Flutter mencocokkan State berdasarkan tipe dan posisi. ValueKey(id) membuat State ikut berpindah bersama datanya.

8. C — ThemeMode.system mengikuti pengaturan sistem. Untuk memaksa, gunakan ThemeMode.dark atau ThemeMode.light.

MODUL 04 · LAYOUTING

Sistem Layouting & Desain Antarmuka Responsif

Menguasai prinsip tata letak Flutter: 'Constraints go down, Sizes go up, Parent sets position'. Mempelajari Row, Column, Expanded, Flexible, Wrap, SizedBox, Spacer, AspectRatio, Stack, Positioned, cara membaca dan memperbaiki overflow, SafeArea, serta desain responsif dengan MediaQuery, LayoutBuilder, breakpoint, dan OrientationBuilder, ditutup studi kasus dashboard portal mahasiswa.

Durasi Pertemuan 4 (150 Menit) + Belajar Mandiri

Prasyarat Modul 03: Arsitektur Widget

Capaian Pembelajaran

- Memahami hukum aturan batas ukuran (Box Constraints): tight, loose, dan unbounded.
- Menguasai perataan elemen dengan Row dan Column (MainAxisAlignment, CrossAxisAlignment, MainAxisSize).
- Menggunakan Expanded, Flexible, Wrap, SizedBox, Spacer, dan AspectRatio untuk membagi ruang secara proporsional.
- Membangun tata letak berlapis menggunakan Stack dan Positioned.
- Membaca pesan RenderFlex overflow dan memperbaikinya dengan teknik yang tepat.
- Menerapkan SafeArea, MediaQuery, LayoutBuilder, breakpoint, dan OrientationBuilder untuk desain yang adaptif.
- Menyusun halaman lengkap yang terpecah menjadi widget-widget kecil dan bebas overflow.

Daftar Isi Modul

1. Aturan Emas Constraints Flutter
2. Row & Column: MainAxis dan CrossAxis
3. Expanded vs Flexible
4. Wrap: Baris yang Turun Otomatis
5. SizedBox, Spacer, & AspectRatio
6. Tata Letak Berlapis (Stack & Positioned)
7. Error Overflow (Garis Kuning-Hitam) & Cara Memperbaikinya
8. SafeArea: Menghindari Notch & Status Bar
9. Desain Responsif (MediaQuery, LayoutBuilder, Breakpoint, OrientationBuilder)
10. Studi Kasus: Dashboard Portal Mahasiswa
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Aturan Emas Constraints Flutter

Untuk menguasai layouting Flutter tanpa frustrasi menghadapi galat *overflow*, setiap mahasiswa wajib menghafal prinsip dasar yang dirumuskan oleh tim inti Flutter:

> 📜 **Tiga Aturan Emas Layout Flutter:**
>
> 1. **Constraints go down:** Widget induk (Parent) memberikan batasan (minimum & maksimum lebar serta tinggi) kepada widget anak (Child).
> 2. **Sizes go up:** Widget anak memutuskan ukurannya sendiri berdasarkan batasan yang diberikan induknya, lalu melaporkan ukurannya kembali ke Parent.
> 3. **Parent sets position:** Widget induk menentukan posisi letak (koordinat x, y) widget anak di layar.

Analogi: pembagian kamar kos

Pemilik kos (induk) berkata: "Lemarimu boleh lebarnya 50 sampai 120 cm." Kamu (anak) memilih lemari 100 cm dan melapor ke pemilik. Pemilik yang menentukan lemari itu ditaruh di pojok mana. Kamu **tidak bisa** memilih lemari 200 cm, dan kamu **tidak bisa** menentukan sendiri posisinya.

Apa itu constraints?

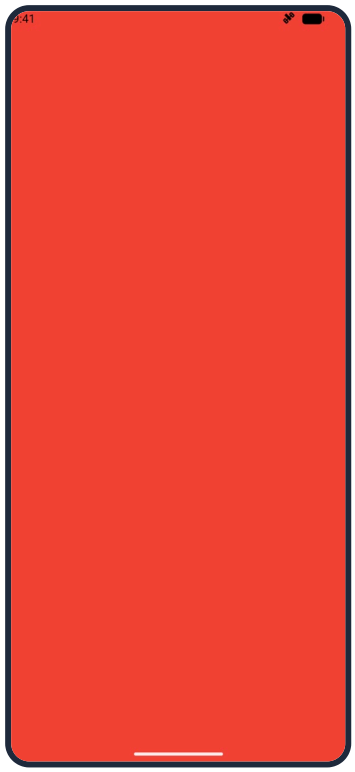
Constraints (`BoxConstraints`) adalah empat angka: `minWidth` , `maxWidth` , `minHeight` , `maxHeight` .

Jenis	Artinya	Contoh pemberi
Tight (ketat)	min = max. Anak *harus* berukuran tepat segitu.	Layar penuh dari `runApp` , `SizedBox(width: 100, height: 100)` , `Expanded`
Loose (longgar)	min = 0, max = sekian. Anak bebas sekecil apa pun asal tidak melebihi max.	`Center` , `Align` , `Scaffold.body`
Unbounded (tak terbatas)	max = tak hingga (`double.infinity`) di satu arah.	Arah utama `Row` / `Column` , `ListView` , `SingleChildScrollView`

Contoh 1: Container yang "tidak patuh"

Pemula sering heran kenapa kode di bawah menghasilkan kotak merah sepenuh layar, padahal ukurannya sudah ditulis 100 × 100:

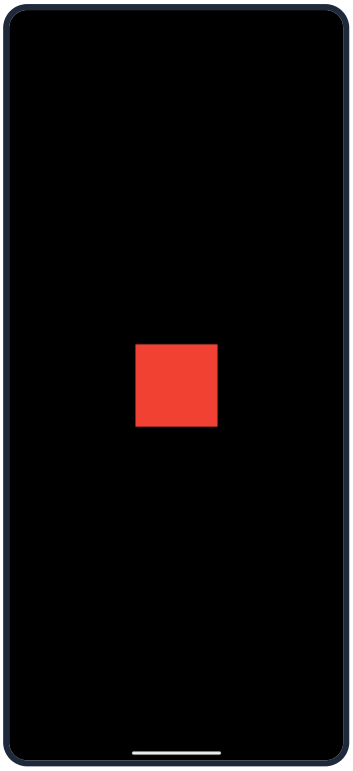
```dart
import 'package:flutter/material.dart';

// Langsung di runApp: layar memberi constraint TIGHT seukuran layar.
// Container dipaksa sebesar layar, width/height 100 diabaikan.
void main() {
  runApp(Container(width: 100, height: 100, color: Colors.red));
}
```



Container 100 × 100 dipaksa memenuhi layar

Solusinya: beri induk yang memberikan constraint **longgar**, misalnya `Center`. Center berkata "kamu boleh sekecil apa pun, maksimal selebar layar", sehingga Container bisa memilih 100 × 100.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(
    Center(
      child: Container(width: 100, height: 100, color: Colors.red),
    ),
  );
}
```



Di dalam Center, Container bisa berukuran 100 × 100

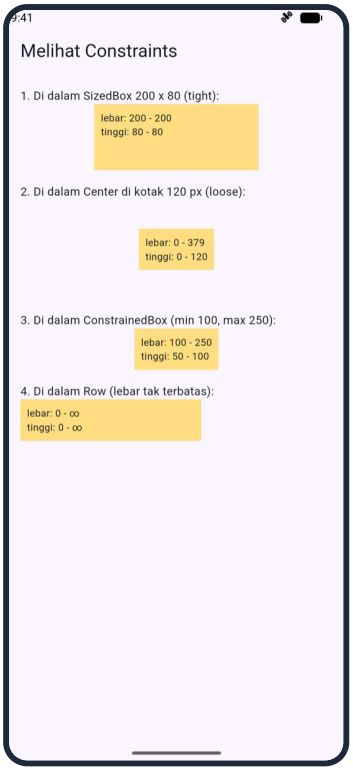
Contoh lengkap: melihat constraints dengan LayoutBuilder

`LayoutBuilder` memberi kita akses ke constraints yang diterima dari induk. Contoh ini menampilkan constraints di beberapa situasi agar aturan di atas terlihat nyata.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text('Melihat Constraints')),
        body: SingleChildScrollView(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text('1. Di dalam SizedBox 200 x 80 (tight):'),
              // Center mencegah stretch dari Column memaksa lebar penuh
              const Center(
```

```
                child: SizedBox(width: 200, height: 80, child: InfoConstraints()),
              ),
              const SizedBox(height: 16),
              const Text('2. Di dalam Center di kotak 120 px (loose):'),
              const SizedBox(
                height: 120,
                child: Center(child: InfoConstraints()),
              ),
              const SizedBox(height: 16),
              const Text('3. Di dalam ConstrainedBox (min 100, max 250):'),
              Center(
                // ConstrainedBox tidak punya konstruktor const
                child: ConstrainedBox(
                  constraints: const BoxConstraints(
                    minWidth: 100,
                    maxWidth: 250,
                    minHeight: 50,
                    maxHeight: 100,
                  ),
                  child: const InfoConstraints(),
                ),
              ),
              const SizedBox(height: 16),
              const Text('4. Di dalam Row (lebar tak terbatas):'),
              const Row(children: [InfoConstraints(lebar: 220)]),
            ],
          ),
        ),
      ),
    );
  }
}

/// Menampilkan constraints yang diterima dari induknya.
class InfoConstraints extends StatelessWidget {
  final double? lebar;

  const InfoConstraints({super.key, this.lebar});

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        String fmt(double v) => v == double.infinity ? '∞' : v.toStringAsFixed(0);
        return Container(
          width: lebar,
          padding: const EdgeInsets.all(8),
          color: Colors.amber.shade200,
          child: Text(
            'lebar: ${fmt(constraints.minWidth)} - ${fmt(constraints.maxWidth)}\n'
            'tinggi: ${fmt(constraints.minHeight)} - ${fmt(constraints.maxHeight)}',
            style: const TextStyle(fontSize: 12),
          ),
        );
      },
    );
  }
}
```



Constraints yang diterima widget di empat situasi

Penjelasan hasil

- **Kotak 1:** lebar 200 - 200, tinggi 80 - 80. Constraint ketat, anak tidak punya pilihan. Kenapa SizedBox perlu dibungkus `Center`? Column memakai `CrossAxisAlignment.stretch` yang memberi constraint ketat selebar layar. Tanpa Center, `width: 200` diabaikan dan kotak menjadi selebar layar (coba hapus Center-nya).
- **Kotak 2:** lebar 0 - (lebar layar), tinggi 0 - 120. `Center` melonggarkan constraint sehingga kotak kuning hanya seukuran teksnya.
- **Kotak 3:** `ConstrainedBox` menambahkan batas minimum dan maksimum sendiri.
- **Kotak 4:** lebar 0 - ∞. Di dalam `Row`, anak menerima lebar tak terbatas. Karena itu kita wajib memberi `lebar: 220`; tanpa batas, widget yang ingin "selebar mungkin" akan error. Ini akar dari banyak error layout yang dibahas di section Overflow.

⚠️ **Kesalahan umum**

- **Mengira `width` / `height` selalu dipatuhi.** Ukuran hanya permintaan; jika constraint induk ketat, permintaan itu diabaikan.
- **Menaruh widget "serakah" di ruang tak terbatas**, misalnya `ListView` di dalam `Column` tanpa `Expanded`, atau `TextField` di dalam `Row` tanpa `Expanded`. Hasilnya error *"unbounded height/width"*.
- **Mengharapkan anak menentukan posisinya sendiri.** Posisi diatur oleh induk (lewat `alignment`, `mainAxisAlignment`, `Positioned`, dan lain-lain).

2. Row & Column: MainAxis dan CrossAxis

Widget `Row` (horizontal) dan `Column` (vertikal) adalah fondasi 80% susunan UI:

Column (Vertikal)

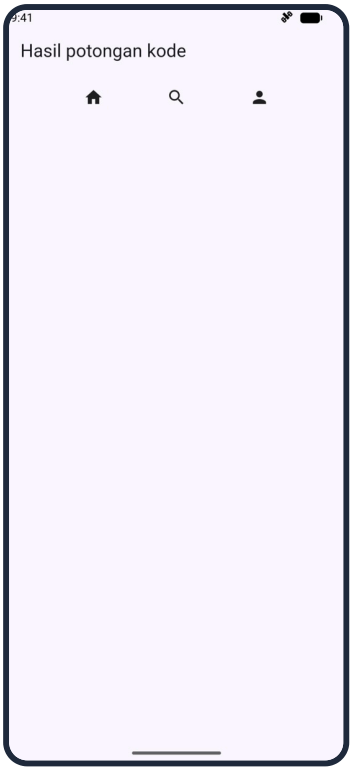
- **MainAxis:** Atas ke Bawah (Y-axis)
- **CrossAxis:** Kiri ke Kanan (X-axis)
- Contoh alignment: `MainAxisAlignment.spaceBetween`, `CrossAxisAlignment.center`

Row (Horizontal)

- **MainAxis:** Kiri ke Kanan (X-axis)
- **CrossAxis:** Atas ke Bawah (Y-axis)
- Contoh alignment: `MainAxisAlignment.spaceEvenly`, `CrossAxisAlignment.stretch`

Contoh kecil

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, // atur di sumbu utama (horizontal)
  crossAxisAlignment: CrossAxisAlignment.center,    // atur di sumbu silang (vertikal)
  children: const [
    Icon(Icons.home),
    Icon(Icons.search),
    Icon(Icons.person),
  ],
)
```



Row dengan
MainAxisAlignment.spaceEvenly

Nilai `MainAxisAlignment`

Gambar di bawah memakai tiga kotak (■) di dalam sebuah `Row` selebar layar:

```
start         |■■■                  |
center        |        ■■■          |
end           |                 ■■■ |
spaceBetween  |■        ■         ■ |   jarak hanya di ANTARA kotak
spaceAround   |  ■      ■      ■    |   jarak tepi = setengah jarak antar kotak
spaceEvenly   |    ■    ■    ■      |   semua jarak sama besar
```

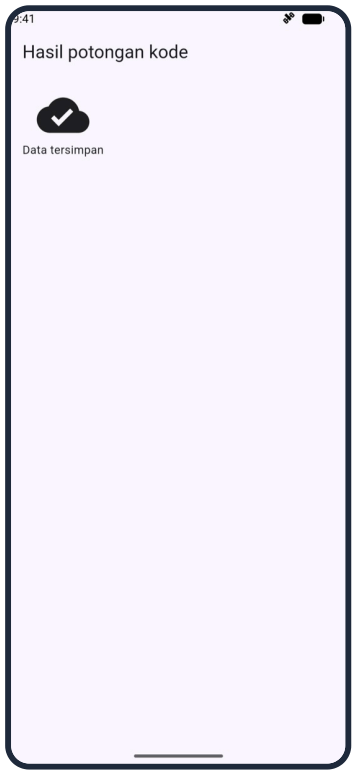
Nilai `CrossAxisAlignment`

Nilai	Pada Row (sumbu silang = vertikal)	Pada Column (sumbu silang = horizontal)
`start`	Rata atas	Rata kiri
`center` (default)	Tengah vertikal	Tengah horizontal
`end`	Rata bawah	Rata kanan
`stretch`	Anak ditarik setinggi Row	Anak ditarik selebar Column

MainAxisSize: seberapa besar Row/Column itu sendiri?

Secara default, `Row` mengambil **seluruh lebar** yang tersedia dan `Column` mengambil **seluruh tinggi** (`MainAxisSize.max`). Dengan `MainAxisSize.min`, ukurannya menyusut sebesar anak-anaknya saja. Ini berguna misalnya untuk tombol berisi ikon dan teks.

```
Column(
  mainAxisSize: MainAxisSize.min, // tinggi Column = total tinggi anak
  children: const [
    Icon(Icons.cloud_done, size: 64),
    Text('Data tersimpan'),
  ],
)
```



Column dengan MainAxisSize.min

Contoh lengkap: playground alignment

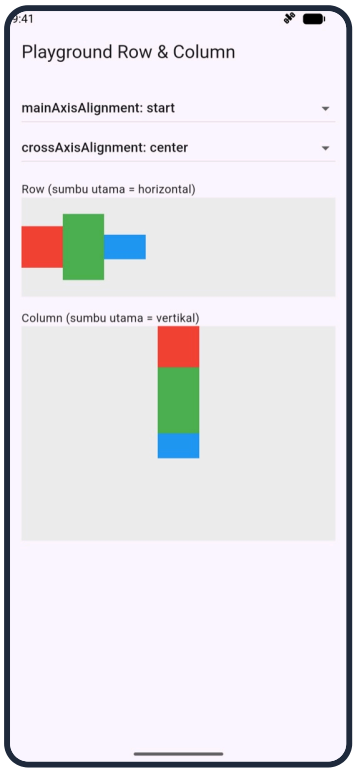
Jalankan contoh ini lalu ganti pilihan pada dropdown. Kamu bisa langsung melihat efek setiap nilai alignment pada `Row` dan `Column`.

`lib/main.dart`

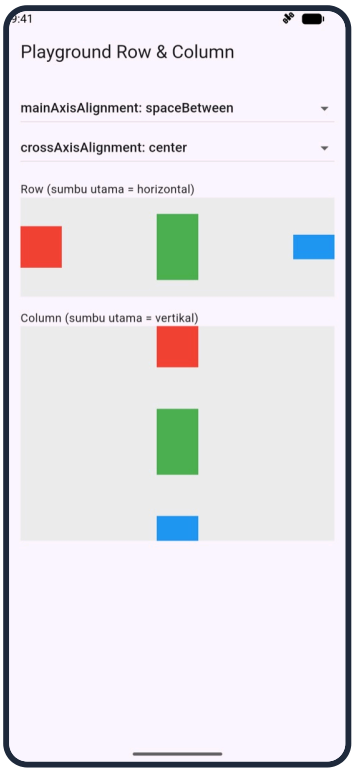
```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: PlaygroundAlignment()));
}

class PlaygroundAlignment extends StatefulWidget {
  const PlaygroundAlignment({super.key});
```

```dart
  @override
  State<PlaygroundAlignment> createState() => _PlaygroundAlignmentState();
}

class _PlaygroundAlignmentState extends State<PlaygroundAlignment> {
  MainAxisAlignment _main = MainAxisAlignment.start;
  CrossAxisAlignment _cross = CrossAxisAlignment.center;

  // Tiga kotak dengan ukuran berbeda agar efek cross axis terlihat
  List<Widget> get _kotak => [
        Container(width: 50, height: 50, color: Colors.red),
        Container(width: 50, height: 80, color: Colors.green),
        Container(width: 50, height: 30, color: Colors.blue),
      ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Playground Row & Column')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // Pilihan mainAxisAlignment
          DropdownButton<MainAxisAlignment>(
            value: _main,
            isExpanded: true,
            items: [
              for (final v in MainAxisAlignment.values)
                DropdownMenuItem(value: v, child: Text('mainAxisAlignment: ${v.name}')),
            ],
            onChanged: (v) => setState(() => _main = v!),
          ),
          // Pilihan crossAxisAlignment (baseline butuh pengaturan khusus, jadi dilewati)
          DropdownButton<CrossAxisAlignment>(
            value: _cross,
            isExpanded: true,
            items: [
              for (final v in CrossAxisAlignment.values)
                if (v != CrossAxisAlignment.baseline)
                  DropdownMenuItem(value: v, child: Text('crossAxisAlignment: ${v.name}')),
            ],
            onChanged: (v) => setState(() => _cross = v!),
          ),
          const SizedBox(height: 16),
          const Text('Row (sumbu utama = horizontal)'),
          Container(
            height: 120,
            color: Colors.grey.shade200,
            child: Row(
              mainAxisAlignment: _main,
              crossAxisAlignment: _cross,
              children: _kotak,
            ),
          ),
          const SizedBox(height: 16),
          const Text('Column (sumbu utama = vertikal)'),
          Container(
            height: 260,
            color: Colors.grey.shade200,
```

```
            child: Column(
              mainAxisAlignment: _main,
              crossAxisAlignment: _cross,
              children: _kotak,
            ),
          ),
        ],
      ),
    );
  }
}
```



Playground alignment: start dan center



Setelah memilih spaceBetween

Penjelasan kode

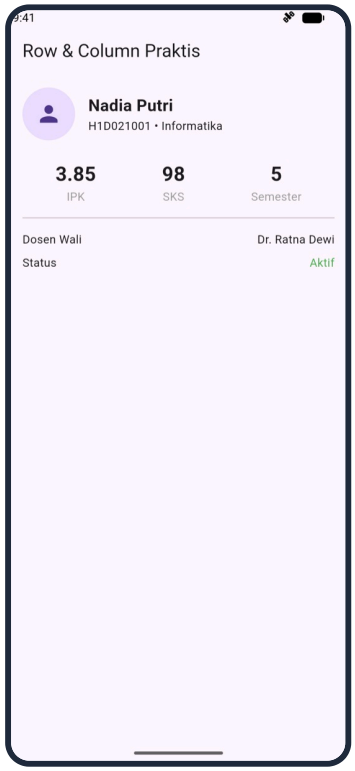
- `MainAxisAlignment.values` berisi semua nilai enum, sehingga dropdown dibuat dengan perulangan `for` di dalam list (*collection for*).
- Kotak hijau lebih tinggi daripada yang lain sehingga perbedaan `start`, `center`, `end` di sumbu silang terlihat jelas pada Row.
- Coba pilih `stretch`: di Column, kotak yang ditulis `width: 50` justru melebar penuh. Stretch memberi constraint *ketat* selebar Column, sehingga permintaan `width: 50` diabaikan. Ini bukti aturan "constraints go down".
- `v!` dipakai karena `onChanged` memberi nilai nullable.

Contoh praktis: baris profil dan kolom statistik

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: ProfilSingkat()));
}

class ProfilSingkat extends StatelessWidget {
  const ProfilSingkat({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Row & Column Praktis')),
      body: const Padding(
        padding: EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start, // semua rata kiri
          children: [
            // Row 1: avatar + nama (Column di dalam Row)
            Row(
              children: [
                CircleAvatar(radius: 32, child: Icon(Icons.person, size: 32)),
                SizedBox(width: 16),
                Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text('Nadia Putri', style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold)),
                    Text('H1D021001 • Informatika'),
                  ],
                ),
              ],
            ),
            SizedBox(height: 24),
            // Row 2: tiga statistik dengan jarak merata
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceAround,
              children: [
                Statistik(nilai: '3.85', label: 'IPK'),
                Statistik(nilai: '98', label: 'SKS'),
                Statistik(nilai: '5', label: 'Semester'),
              ],
            ),
            Divider(height: 32),
            // Row 3: label kiri, nilai kanan
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [Text('Dosen Wali'), Text('Dr. Ratna Dewi')],
            ),
            SizedBox(height: 8),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [Text('Status'), Text('Aktif', style: TextStyle(color: Colors.green))],
```

```
            ),
          ],
        ),
      ),
    );
  }
}

class Statistik extends StatelessWidget {
  final String nilai;
  final String label;

  const Statistik({super.key, required this.nilai, required this.label});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(nilai, style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
        Text(label, style: const TextStyle(color: Colors.grey)),
      ],
    );
  }
}
```



Row dan Column praktis untuk profil singkat

Pola `spaceBetween` untuk "label di kiri, nilai di kanan" dan `spaceAround` untuk deretan statistik akan sangat sering kamu pakai.

⚠️ **Kesalahan umum**

- **Tertukar sumbu.** Ingat: `mainAxis` selalu searah susunan anak. Row = horizontal, Column = vertikal.
- **Memakai `Center` untuk menengahkan anak Column secara vertikal.** Pakai `mainAxisAlignment: MainAxisAlignment.center`.
- **`mainAxisAlignment` tidak berefek** karena Row/Column memakai `MainAxisSize.min` (tidak ada ruang sisa untuk dibagi).
- **Teks rata kiri tidak jalan** di Column: default `crossAxisAlignment` adalah `center`. Tulis `CrossAxisAlignment.start`.

3. Expanded vs Flexible

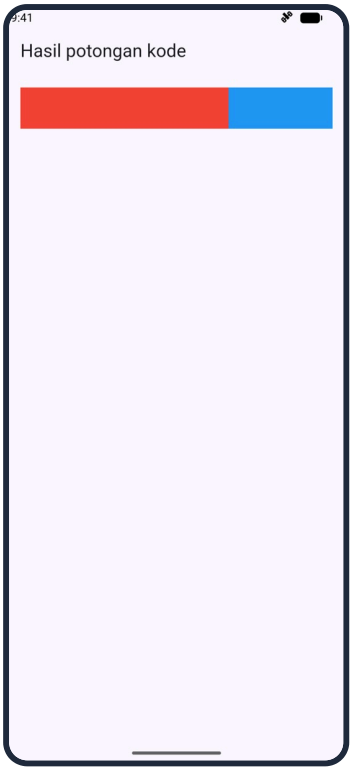
Di dalam `Row` dan `Column`, anak biasanya berukuran sesuai isinya. Jika kita ingin anak **mengisi sisa ruang**, bungkus dengan `Expanded` atau `Flexible`.

	Expanded	Flexible
Sifat	`fit: FlexFit.tight` : anak *dipaksa* mengisi seluruh jatah ruangnya	`fit: FlexFit.loose` : anak *boleh* lebih kecil dari jatahnya
Analogi	Air yang selalu memenuhi gelas	Batu di dalam gelas: maksimal sebesar gelas
Cocok untuk	Kolom isi, kartu sejajar sama lebar, TextField di samping tombol	Teks yang bisa pendek atau panjang di samping ikon

Cara kerja: ruang sisa dibagi menurut `flex`

1. Row menghitung dulu ukuran anak yang **tidak** dibungkus Expanded/Flexible.
2. Sisa ruang dibagi ke anak Expanded/Flexible sesuai angka `flex` (default 1).

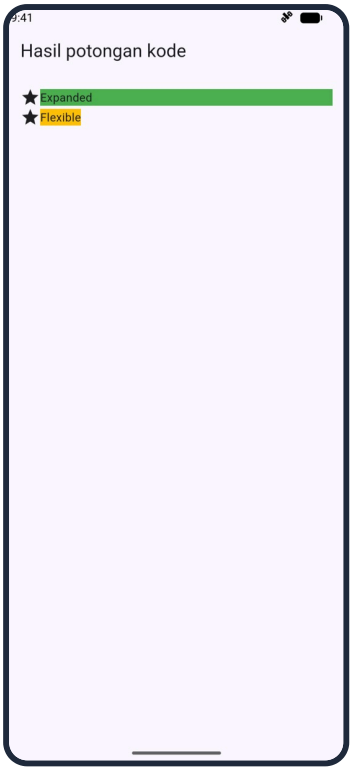
```
Row(
  children: [
    Expanded(flex: 2, child: Container(height: 50, color: Colors.red)),   // 2/3 ruang
    Expanded(flex: 1, child: Container(height: 50, color: Colors.blue)),  // 1/3 ruang
  ],
)
```



Expanded flex 2 : 1

Expanded vs Flexible dalam satu contoh kecil

```
Column(
  children: [
    // Expanded: kotak hijau dipaksa selebar sisa ruang
    Row(children: [
      const Icon(Icons.star),
      Expanded(child: Container(color: Colors.green, child: const Text('Expanded'))),
    ]),
    // Flexible: kotak kuning hanya selebar teksnya
    Row(children: [
      const Icon(Icons.star),
      Flexible(child: Container(color: Colors.amber, child: const Text('Flexible'))),
    ]),
  ],
)
```



Expanded (penuh) vs Flexible
(seukuran isi)

Contoh lengkap 1: membandingkan flex, Expanded, dan Flexible

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: DemoFlex()));
}

class DemoFlex extends StatelessWidget {
  const DemoFlex({super.key});

  Widget _kotak(String teks, Color warna) {
    return Container(
      height: 56,
      color: warna,
      alignment: Alignment.center,
      child: Text(teks, style: const TextStyle(color: Colors.white)),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Expanded vs Flexible')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text('1. Tiga Expanded flex sama (1:1:1)'),
          Row(children: [
```

```
            Expanded(child: _kotak('1', Colors.red)),
            Expanded(child: _kotak('1', Colors.green)),
            Expanded(child: _kotak('1', Colors.blue)),
          ]),
          const SizedBox(height: 16),

          const Text('2. Flex berbeda (1:2:3)'),
          Row(children: [
            Expanded(flex: 1, child: _kotak('1', Colors.red)),
            Expanded(flex: 2, child: _kotak('2', Colors.green)),
            Expanded(flex: 3, child: _kotak('3', Colors.blue)),
          ]),
          const SizedBox(height: 16),

          const Text('3. Lebar tetap + Expanded mengisi sisa'),
          Row(children: [
            SizedBox(width: 80, child: _kotak('80 px', Colors.grey)),
            Expanded(child: _kotak('sisa ruang', Colors.indigo)),
            SizedBox(width: 80, child: _kotak('80 px', Colors.grey)),
          ]),
          const SizedBox(height: 16),

          const Text('4. Expanded (tight) vs Flexible (loose)'),
          Row(children: [
            Expanded(child: Container(color: Colors.teal.shade100, child: const Text('Expanded'))),
            Flexible(child: Container(color: Colors.orange.shade100, child: const
Text('Flexible'))),
          ]),
          const SizedBox(height: 16),

          const Text('5. Expanded di Column (sumbu vertikal)'),
          SizedBox(
            height: 200,
            child: Column(children: [
              _kotak('header tetap', Colors.grey),
              Expanded(child: Container(color: Colors.purple.shade100, alignment: Alignment.center,
child: const Text('isi mengisi sisa tinggi'))),
              _kotak('footer tetap', Colors.grey),
            ]),
          ),
        ],
      ),
    );
  }
}
```

Perbandingan flex, Expanded, dan Flexible

Penjelasan

- **Baris 4:** Expanded dan Flexible sama-sama mendapat jatah separuh lebar. Kotak Expanded memenuhi jatahnya, kotak Flexible hanya selebar teks "Flexible".
- **Baris 5:** Expanded juga bekerja di Column. Pola header-isi-footer seperti ini sering dipakai pada halaman chat atau keranjang belanja.
- Method `_kotak` adalah helper kecil agar kode tidak berulang.

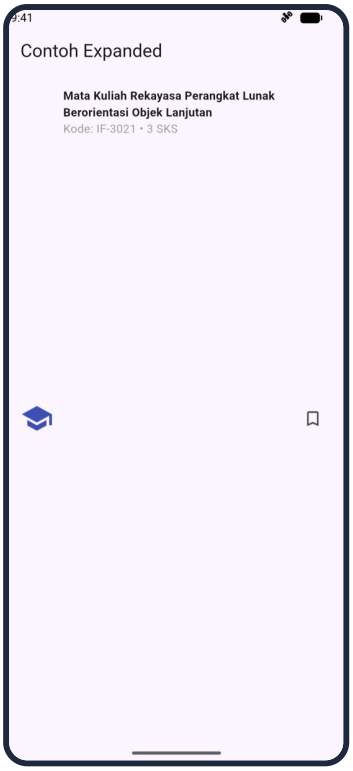
Contoh lengkap 2: teks panjang di samping ikon

Ketika teks atau konten terlalu panjang di dalam Row, layar akan menampilkan garis kuning-hitam *"A RenderFlex overflowed by xxx pixels"*. Solusinya adalah membungkus widget tersebut dengan **`Expanded`**:

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text("Contoh Expanded")),
        body: Padding(
          padding: const EdgeInsets.all(16),
          // Contoh Menghindari Overflow pada Row
          child: Row(
            children: [
              const Icon(Icons.school, size: 40, color: Colors.indigo),
              const SizedBox(width: 12),
              // Expanded memaksa teks mengambil sisa ruang yang tersedia saja!
              // Coba hapus Expanded ini lalu hot reload: muncul garis kuning-hitam overflow.
              const Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      "Mata Kuliah Rekayasa Perangkat Lunak Berorientasi Objek Lanjutan",
                      style: TextStyle(fontWeight: FontWeight.bold),
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis, // Tambahkan '...' jika terpotong
                    ),
                    Text("Kode: IF-3021 • 3 SKS", style: TextStyle(color: Colors.grey)),
                  ],
                ),
              ),
              IconButton(
                icon: const Icon(Icons.bookmark_border),
                onPressed: () {},
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



Teks panjang dibungkus Expanded sehingga tidak overflow

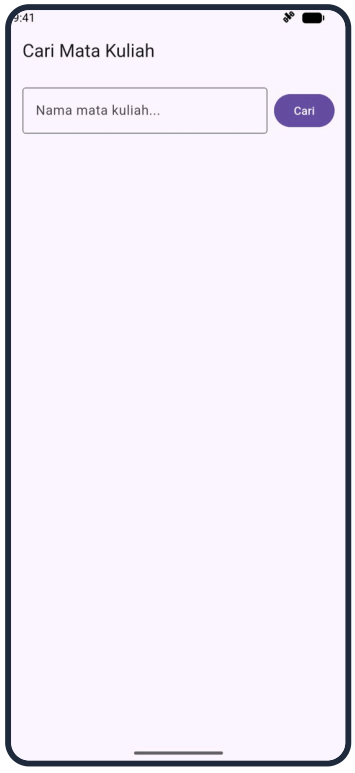
Contoh lengkap 3: TextField di samping tombol

`TextField` ingin selebar mungkin. Di dalam Row (lebar tak terbatas), ini menimbulkan error. `Expanded` memberi batas lebar yang jelas.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: KolomCari()));
}

class KolomCari extends StatelessWidget {
  const KolomCari({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Cari Mata Kuliah')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Row(
          children: [
            // Tanpa Expanded: error "An InputDecorator ... cannot have an unbounded width"
            const Expanded(
              child: TextField(
                decoration: InputDecoration(
                  hintText: 'Nama mata kuliah...',
                  border: OutlineInputBorder(),
                ),
```

```
            ),
          ),
          const SizedBox(width: 8),
          FilledButton(onPressed: () {}, child: const Text('Cari')),
        ],
      ),
    ),
  );
 }
}
```



TextField di dalam Expanded di samping tombol

> ⚠️ **Kesalahan umum**
>
> - **Memakai Expanded di luar Row/Column/Flex.** Contoh: Expanded langsung di dalam `Container` atau `Stack`. Hasilnya error *"Incorrect use of ParentDataWidget"*.
> - **Expanded di dalam Column yang berada di `SingleChildScrollView`.** Tinggi tak terbatas tidak bisa dibagi, sehingga error. Hapus Expanded atau beri tinggi tetap.
> - **Memberi `width` pada anak Expanded di Row.** Lebarnya tetap dipaksa sesuai jatah flex; `width` diabaikan.

4. Wrap: Baris yang Turun Otomatis

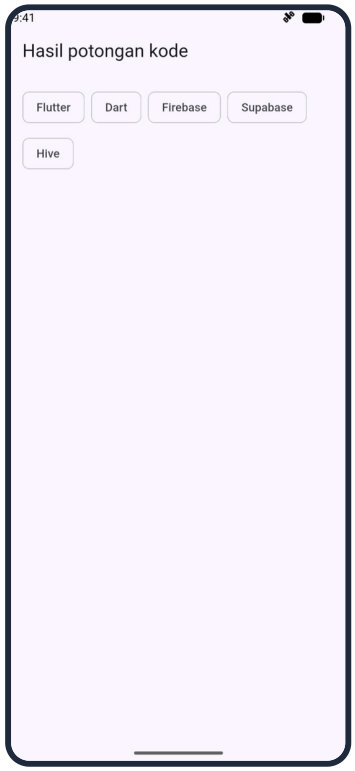
`Row` tidak pernah pindah baris: jika anaknya terlalu banyak, terjadi overflow. `Wrap` menyusun anak berderet seperti Row, tetapi otomatis **turun ke baris baru** saat ruangnya habis, seperti kata-kata dalam paragraf.

Row vs Wrap

```
Row  (overflow):  [Flutter] [Dart] [Firebase] [Supabase] [Hi▓▓▓
Wrap (rapi)    :  [Flutter] [Dart] [Firebase]
                  [Supabase] [Hive] [REST API]
```

Contoh kecil

```dart
Wrap(
  spacing: 8,      // jarak horizontal antar anak
  runSpacing: 8,   // jarak vertikal antar baris
  children: const [
    Chip(label: Text('Flutter')),
    Chip(label: Text('Dart')),
    Chip(label: Text('Firebase')),
    Chip(label: Text('Supabase')),
    Chip(label: Text('Hive')),
  ],
)
```



Chip di dalam Wrap turun baris otomatis

Properti penting

Properti	Fungsi
`direction`	`Axis.horizontal` (default) atau `Axis.vertical`.
`spacing`	Jarak antar anak dalam satu baris.
`runSpacing`	Jarak antar baris.
`alignment`	Perataan anak di setiap baris: `WrapAlignment.start`, `center`, `spaceBetween`, dll.

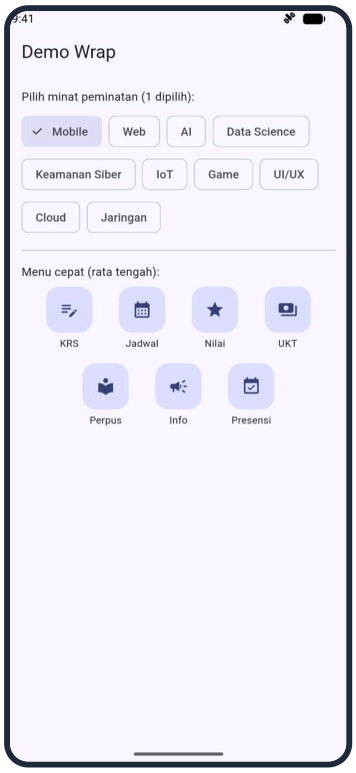
Contoh lengkap: pilih minat dan menu ikon

`lib/main.dart`

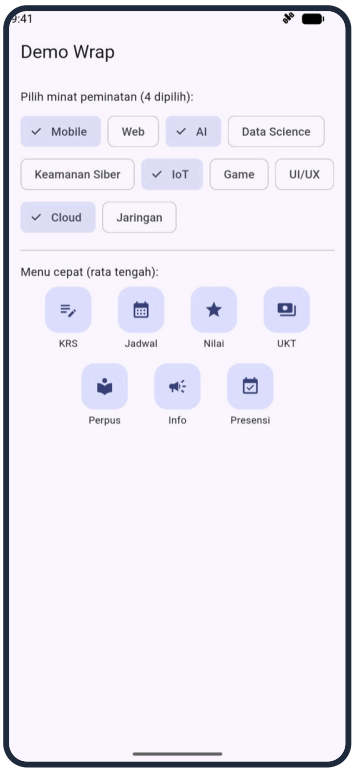
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.indigo),
    home: const DemoWrap(),
  ));
}

class DemoWrap extends StatefulWidget {
  const DemoWrap({super.key});

  @override
  State<DemoWrap> createState() => _DemoWrapState();
}

class _DemoWrapState extends State<DemoWrap> {
  final List<String> _topik = [
    'Mobile', 'Web', 'AI', 'Data Science', 'Keamanan Siber',
    'IoT', 'Game', 'UI/UX', 'Cloud', 'Jaringan',
  ];
  final Set<String> _dipilih = {'Mobile'};

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Wrap')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Pilih minat peminatan (${_dipilih.length} dipilih):'),
          const SizedBox(height: 8),
          // 1. Chip yang bisa dipilih, otomatis turun baris
          Wrap(
            spacing: 8,
```

```
            runSpacing: 4,
            children: [
              for (final t in _topik)
                FilterChip(
                  label: Text(t),
                  selected: _dipilih.contains(t),
                  onSelected: (pilih) {
                    setState(() {
                      if (pilih) {
                        _dipilih.add(t);
                      } else {
                        _dipilih.remove(t);
                      }
                    });
                  },
                ),
            ],
          ),
          const Divider(height: 32),
          const Text('Menu cepat (rata tengah):'),
          const SizedBox(height: 8),
          // 2. Menu ikon dengan lebar tetap, rata tengah
          const Wrap(
            alignment: WrapAlignment.center,
            spacing: 16,
            runSpacing: 16,
            children: [
              MenuIkon(ikon: Icons.edit_note, label: 'KRS'),
              MenuIkon(ikon: Icons.calendar_month, label: 'Jadwal'),
              MenuIkon(ikon: Icons.grade, label: 'Nilai'),
              MenuIkon(ikon: Icons.payments, label: 'UKT'),
              MenuIkon(ikon: Icons.local_library, label: 'Perpus'),
              MenuIkon(ikon: Icons.campaign, label: 'Info'),
              MenuIkon(ikon: Icons.event_available, label: 'Presensi'),
            ],
          ),
        ],
      ),
    );
  }
}

class MenuIkon extends StatelessWidget {
  final IconData ikon;
  final String label;

  const MenuIkon({super.key, required this.ikon, required this.label});

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return SizedBox(
      width: 72,
      child: Column(
        children: [
          Container(
            width: 56,
            height: 56,
            decoration: BoxDecoration(
```

```
              color: warna.primaryContainer,
              borderRadius: BorderRadius.circular(16),
            ),
            child: Icon(ikon, color: warna.onPrimaryContainer),
          ),
          const SizedBox(height: 4),
          Text(label, style: const TextStyle(fontSize: 12)),
        ],
      ),
    );
  }
}
```



FilterChip dan menu ikon di dalam Wrap



Setelah memilih beberapa minat

Penjelasan kode

- `FilterChip` adalah chip yang bisa dipilih. Pilihan disimpan di `Set` agar tidak ada nama ganda.
- Setiap `MenuIkon` lebarnya tetap 72 px. Di HP sempit menjadi 4 per baris, di HP lebar atau landscape lebih banyak per baris, tanpa kode tambahan.
- `WrapAlignment.center` membuat baris terakhir yang tidak penuh tetap di tengah.

⚠️ **Kesalahan umum**

- **Memakai Expanded di dalam Wrap.** Wrap bukan Flex, jadi Expanded tidak bekerja dan memicu error.
- **Memakai Wrap untuk ratusan item.** Wrap membangun semua anak sekaligus. Untuk daftar panjang, gunakan `GridView.builder` (Modul 06).
- **Lupa `runSpacing`** sehingga baris-baris saling menempel.

5. SizedBox, Spacer, & AspectRatio

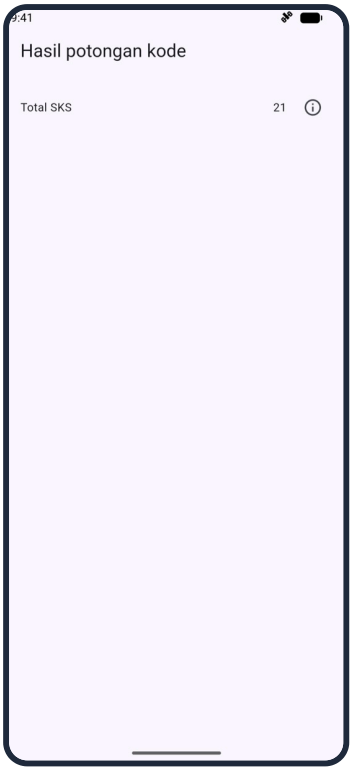
Tiga widget kecil untuk mengatur jarak dan proporsi: `SizedBox` (jarak tetap), `Spacer` (jarak fleksibel yang mendorong widget lain), dan `AspectRatio` (menjaga perbandingan lebar : tinggi).

Widget	Ukuran jarak	Contoh pemakaian
`SizedBox(width: 8)`	Tetap, persis 8 px	Jarak ikon ke teks
`Spacer()`	Mengisi seluruh sisa ruang	Mendorong tombol ke ujung kanan
`AspectRatio(aspectRatio: 16 / 9)`	Tinggi dihitung dari lebar	Banner, thumbnail video

Spacer

`Spacer` sebenarnya adalah `Expanded` yang berisi kotak kosong. Ia hanya bisa dipakai di Row/Column.

```
Row(
  children: [
    const Text('Total SKS'),
    const Spacer(), // dorong widget berikutnya ke ujung kanan
    const Text('21'),
    const SizedBox(width: 8),
    IconButton(onPressed: () {}, icon: const Icon(Icons.info_outline)),
  ],
)
```

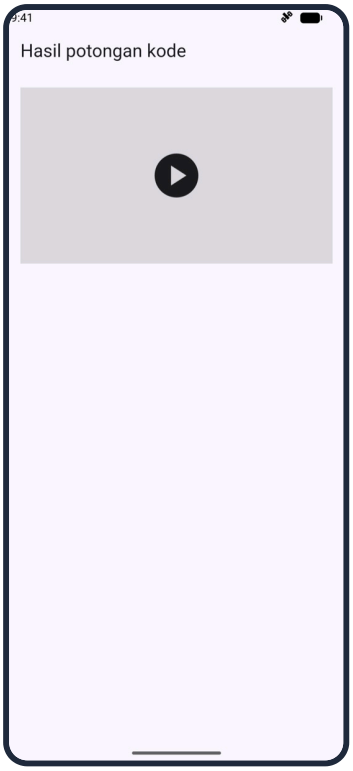


Spacer mendorong nilai dan tombol ke kanan

Seperti Expanded, Spacer punya `flex`. `Spacer(flex: 2)` mengambil jatah dua kali `Spacer()`.

AspectRatio

```
AspectRatio(
  aspectRatio: 16 / 9, // lebar : tinggi
  child: Container(
    color: Colors.black12,
    alignment: Alignment.center,
    child: const Icon(Icons.play_circle, size: 64),
  ),
)
```



AspectRatio 16:9

Jika lebar tersedia 360 px, tinggi otomatis menjadi 360 / (16/9) = 202,5 px. Di tablet yang lebih lebar, tinggi ikut membesar sehingga proporsinya tetap.

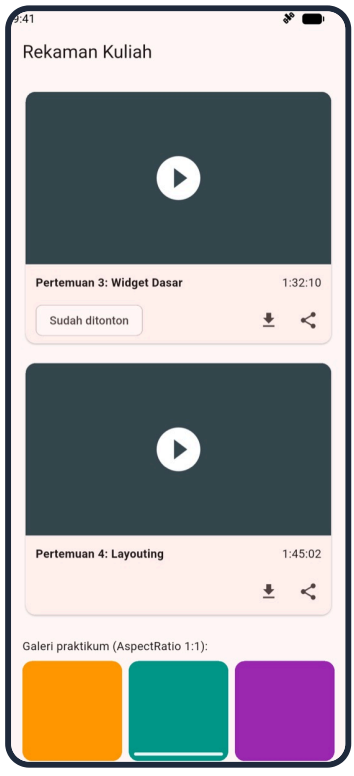
Contoh lengkap: kartu video kuliah

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.red),
    home: const DaftarVideo(),
  ));
}

class DaftarVideo extends StatelessWidget {
  const DaftarVideo({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Rekaman Kuliah')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: const [
          KartuVideo(judul: 'Pertemuan 3: Widget Dasar', durasi: '1:32:10', ditonton: true),
          SizedBox(height: 16),
          KartuVideo(judul: 'Pertemuan 4: Layouting', durasi: '1:45:02'),
          SizedBox(height: 24),
```

```
          // Tiga kotak persegi (1:1) sejajar, sama besar
          Text('Galeri praktikum (AspectRatio 1:1):'),
          SizedBox(height: 8),
          Row(
            children: [
              Expanded(child: KotakPersegi(warna: Colors.orange)),
              SizedBox(width: 8),
              Expanded(child: KotakPersegi(warna: Colors.teal)),
              SizedBox(width: 8),
              Expanded(child: KotakPersegi(warna: Colors.purple)),
            ],
          ),
        ],
      ),
    );
  }
}

class KartuVideo extends StatelessWidget {
  final String judul;
  final String durasi;
  final bool ditonton;

  const KartuVideo({super.key, required this.judul, required this.durasi, this.ditonton = false});

  @override
  Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.antiAlias,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Thumbnail selalu 16:9 berapa pun lebar layar
          AspectRatio(
            aspectRatio: 16 / 9,
            child: Container(
              color: Colors.blueGrey.shade800,
              alignment: Alignment.center,
              child: const Icon(Icons.play_circle_fill, size: 64, color: Colors.white),
            ),
          ),
          Padding(
            padding: const EdgeInsets.all(12),
            child: Row(
              children: [
                Expanded(
                  child: Text(judul, style: const TextStyle(fontWeight: FontWeight.bold)),
                ),
                const SizedBox(width: 8),
                Text(durasi),
              ],
            ),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(12, 0, 4, 4),
            child: Row(
              children: [
                if (ditonton) const Chip(label: Text('Sudah ditonton')),
                const Spacer(), // tombol selalu di kanan, ada chip atau tidak
```

```
                IconButton(onPressed: () {}, icon: const Icon(Icons.download)),
                IconButton(onPressed: () {}, icon: const Icon(Icons.share)),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class KotakPersegi extends StatelessWidget {
  final Color warna;

  const KotakPersegi({super.key, required this.warna});

  @override
  Widget build(BuildContext context) {
    return AspectRatio(
      aspectRatio: 1,
      child: DecoratedBox(
        decoration: BoxDecoration(color: warna, borderRadius: BorderRadius.circular(12)),
      ),
    );
  }
}
```



Kartu video dengan AspectRatio dan Spacer

Penjelasan kode

- **Thumbnail:** `AspectRatio(16 / 9)` menjamin proporsi video tidak gepeng di HP mana pun.

- **Baris tombol:** `Spacer` mendorong tombol unduh dan bagikan ke kanan, baik saat chip "Sudah ditonton" ada maupun tidak.
- **Galeri:** `Expanded` membagi lebar sama rata, lalu `AspectRatio(1)` membuat tinggi sama dengan lebar sehingga kotak benar-benar persegi.
- `DecoratedBox` adalah versi ringan Container yang hanya memberi dekorasi.

> ⚠️ **Kesalahan umum**
>
> - **Spacer di luar Row/Column**, misalnya di dalam `Stack` atau `ListView`, menimbulkan error ParentDataWidget.
> - **Spacer di Row yang berada di ruang tak terbatas** (misalnya di dalam `SingleChildScrollView` horizontal). Tidak ada sisa ruang yang bisa diisi.
> - **AspectRatio di dalam constraint ketat**, misalnya `SizedBox(width: 100, height: 100)`. Ukurannya sudah dipaksa, jadi rasio tidak berlaku.
> - **Menulis `aspectRatio: 16 / 9` sebagai `9 / 16`**. Ingat urutannya lebar dibagi tinggi.

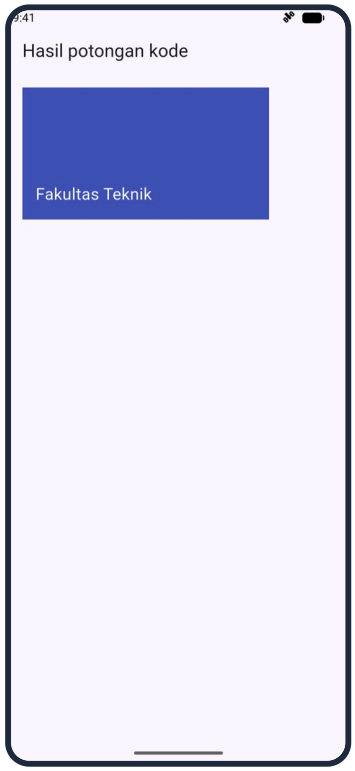
6. Tata Letak Berlapis (Stack & Positioned)

Jika `Row` dan `Column` menyusun widget secara berurutan tanpa tumpang tindih, widget **`Stack`** menumpuk widget di atas widget lainnya (z-index). Anak pertama berada paling bawah, anak terakhir paling atas, seperti layer di aplikasi desain.

```
children: [A, B, C]      tampak dari samping:   C  ← paling atas
                                                B
                                                A  ← paling bawah
```

Contoh kecil: teks di atas gambar

```dart
Stack(
  children: [
    Container(width: 300, height: 160, color: Colors.indigo), // lapisan bawah
    const Positioned(
      left: 16,
      bottom: 16,
      child: Text('Fakultas Teknik', style: TextStyle(color: Colors.white, fontSize: 20)),
    ),
  ],
)
```



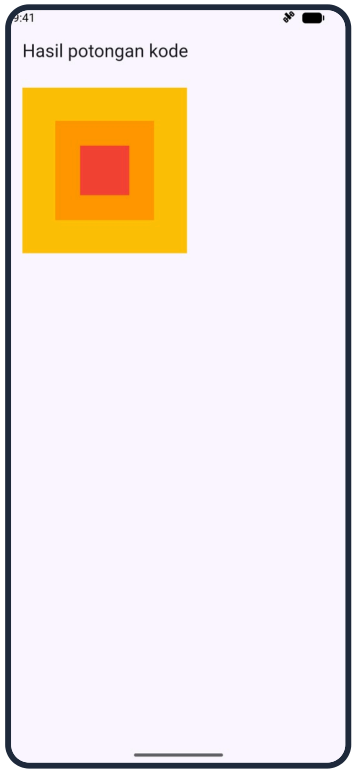
Teks ditumpuk di atas kotak dengan Positioned

Positioned dan alignment

- **Anak tanpa `Positioned`** diletakkan menurut `Stack.alignment` (default `AlignmentDirectional.topStart`, pojok kiri atas).

- `Positioned(top, bottom, left, right, width, height)` mengatur jarak anak dari tepi Stack. Isi dua sisi yang berlawanan (misal `left: 0, right: 0`) untuk merentangkan anak.
- `Positioned.fill()` membuat anak memenuhi seluruh Stack, cocok untuk lapisan gradasi gelap di atas foto.
- **Ukuran Stack** mengikuti anak terbesar yang *tidak* memakai Positioned.

```
Stack(
  alignment: Alignment.center, // anak tanpa Positioned ditaruh di tengah
  children: [
    Container(width: 200, height: 200, color: Colors.amber),
    Container(width: 120, height: 120, color: Colors.orange),
    Container(width: 60, height: 60, color: Colors.red),
  ],
)
```



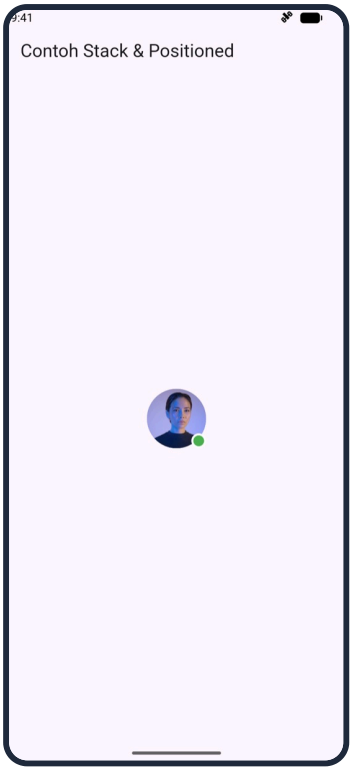
Tiga kotak bertumpuk dengan alignment center

Contoh lengkap 1: badge status online

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text("Contoh Stack & Positioned")),
        body: Center(
          // Contoh Badge Notifikasi pada Foto Profil Mahasiswa
          child: Stack(
            clipBehavior: Clip.none, // Membolehkan elemen keluar sedikit dari batas induk
            children: [
              // Lapisan 1: Foto Profil Lingkaran
              CircleAvatar(
                radius: 36,
                backgroundColor: Colors.indigo.shade100,
                // foregroundImage dipakai agar jika gambar gagal dimuat (offline),
                // CircleAvatar tetap menampilkan child (ikon) sebagai cadangan.
                foregroundImage: const NetworkImage(
                  'https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=150',
                ),
                onForegroundImageError: (error, stackTrace) {},
                child: const Icon(Icons.person, size: 36, color: Colors.indigo),
              ),
              // Lapisan 2: Posisi Indikator Online di Sudut Kanan Bawah
              Positioned(
                bottom: 0,
                right: 0,
                child: Container(
                  width: 18,
                  height: 18,
                  decoration: BoxDecoration(
                    color: Colors.green,
                    shape: BoxShape.circle,
                    border: Border.all(color: Colors.white, width: 2.5),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



Badge status online di foto profil

Contoh lengkap 2: header profil dengan banner dan avatar menumpuk

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.indigo),
    home: const HeaderProfil(),
  ));
}

class HeaderProfil extends StatelessWidget {
  const HeaderProfil({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ListView(
        padding: EdgeInsets.zero,
        children: [
          SizedBox(
            height: 260, // tinggi total: banner 200 + setengah avatar menjorok
            child: Stack(
              children: [
                // Lapisan 1: banner gradasi
                Container(
                  height: 200,
                  decoration: const BoxDecoration(
                    gradient: LinearGradient(
```

```
                  colors: [Color(0xFF1E3A8A), Color(0xFF3B82F6)],
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                ),
              ),
            ),
            // Lapisan 2: lingkaran dekorasi di pojok kanan atas
            Positioned(
              top: -40,
              right: -40,
              child: Container(
                width: 160,
                height: 160,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  color: Colors.white.withValues(alpha: 0.1),
                ),
              ),
            ),
            // Lapisan 3: tombol kembali, di bawah status bar
            Positioned(
              top: MediaQuery.paddingOf(context).top + 8,
              left: 8,
              child: IconButton(
                onPressed: () {},
                icon: const Icon(Icons.arrow_back, color: Colors.white),
              ),
            ),
            // Lapisan 4: avatar menjorok keluar banner
            Positioned(
              left: 24,
              bottom: 0,
              child: Container(
                padding: const EdgeInsets.all(4),
                decoration: const BoxDecoration(color: Colors.white, shape: BoxShape.circle),
                child: const CircleAvatar(
                  radius: 52,
                  child: Icon(Icons.person, size: 56),
                ),
              ),
            ),
            // Lapisan 5: tombol edit di kanan bawah banner
            Positioned(
              right: 16,
              top: 212,
              child: OutlinedButton.icon(
                onPressed: () {},
                icon: const Icon(Icons.edit, size: 18),
                label: const Text('Edit Profil'),
              ),
            ),
          ],
        ),
      ),
      const Padding(
        padding: EdgeInsets.fromLTRB(24, 12, 24, 0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
```

```
                Text('Nadia Putri', style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
                Text('H1D021001 • S1 Informatika'),
                SizedBox(height: 8),
                Text('Universitas Jenderal Soedirman', style: TextStyle(color: Colors.grey)),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

Header profil dengan banner dan avatar menumpuk

Penjelasan kode

- Stack diberi tinggi 260 lewat `SizedBox`, sedangkan banner hanya 200. Sisa 60 px dipakai avatar yang menjorok ke bawah banner.
- `top: -40, right: -40`: nilai negatif menggeser lingkaran keluar dari Stack. Bagian yang keluar terpotong karena default `clipBehavior` Stack adalah `Clip.hardEdge`; di contoh pertama kita memakai `Clip.none` agar badge boleh keluar.
- `MediaQuery.paddingOf(context).top` adalah tinggi status bar, agar tombol kembali tidak tertutup jam dan baterai (dibahas di section SafeArea).
- Urutan anak menentukan lapisan: avatar ditulis setelah banner, maka avatar tampil di atasnya.

⚠️ **Kesalahan umum**

- **Stack berisi hanya anak Positioned.** Stack tidak punya acuan ukuran, sehingga ukurannya mengikuti constraint terbesar atau malah nol. Beri satu anak tidak-Positioned atau bungkus dengan `SizedBox`.
- **Positioned di luar Stack** menimbulkan error *"Incorrect use of ParentDataWidget"*.
- **Urutan terbalik:** widget yang ditulis paling akhir menutupi yang lain, termasuk menghalangi sentuhan ke tombol di bawahnya.
- **Memakai Stack + Positioned untuk seluruh layout halaman** dengan angka piksel tetap. Hasilnya berantakan di layar berbeda. Gunakan Stack hanya untuk bagian yang memang bertumpuk.

7. Error Overflow (Garis Kuning-Hitam) & Cara Memperbaikinya

Garis **kuning-hitam bergaris miring** di tepi layar adalah tanda *overflow*: konten lebih besar daripada ruang yang tersedia. Garis ini hanya muncul di mode debug, tetapi di mode rilis kontennya tetap terpotong. Jadi wajib diperbaiki.

Membaca pesan error

```
══╡ EXCEPTION CAUGHT BY RENDERING LIBRARY ╞═══════════════
A RenderFlex overflowed by 74 pixels on the right.
The relevant error-causing widget was:
  Row Row:file:///.../lib/main.dart:27:20
```

- **RenderFlex** = Row atau Column (keduanya turunan `Flex`).
- **74 pixels on the right** = kelebihan 74 px ke kanan, jadi masalahnya di **Row**. Jika "on the bottom", masalahnya di **Column**.
- **main.dart:27:20** = baris dan kolom kode penyebabnya. Klik di VS Code untuk langsung melompat ke sana.

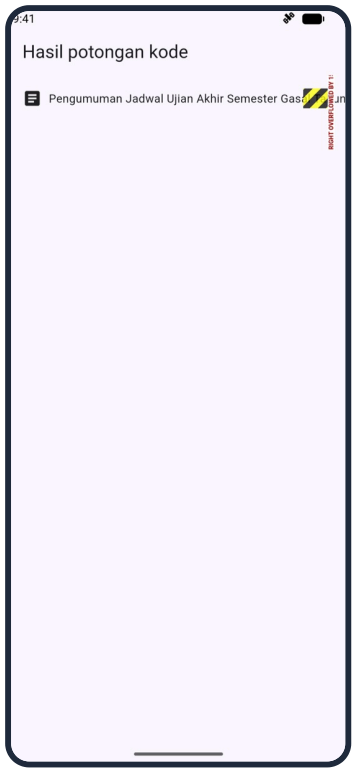
Lima kasus overflow dan obatnya

Kasus	Gejala	Obat
1. Teks panjang di Row	Overflow "on the right"	Bungkus teks dengan `Expanded` / `Flexible`, tambah `overflow: TextOverflow.ellipsis`
2. Terlalu banyak anak di Row	Overflow "on the right"	Ganti `Row` dengan `Wrap`, atau buat bisa digeser horizontal
3. Column lebih tinggi dari layar	Overflow "on the bottom"	Bungkus dengan `SingleChildScrollView` atau ganti dengan `ListView`
4. Keyboard muncul menutupi form	Overflow "on the bottom" saat mengetik	Form dalam `SingleChildScrollView`
5. ListView di dalam Column	Error *"Vertical viewport was given unbounded height"*	Bungkus ListView dengan `Expanded`

Kasus 1: teks panjang (sebelum dan sesudah)

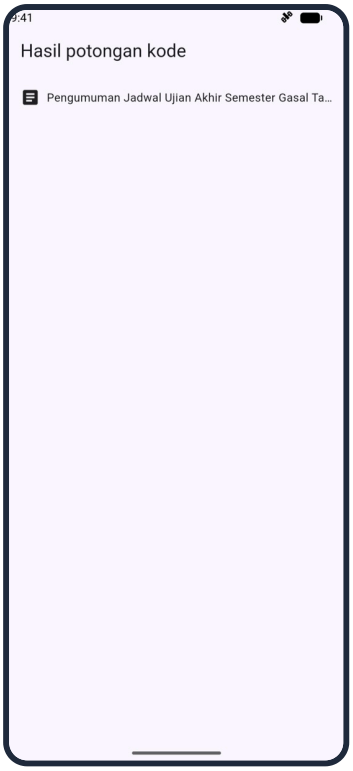
```dart
// SALAH: teks panjang mendorong ikon keluar layar
Row(
  children: const [
```

```
    Icon(Icons.article),
    SizedBox(width: 8),
    Text('Pengumuman Jadwal Ujian Akhir Semester Gasal Tahun Akademik 2026/2027'),
  ],
)
```



SALAH: teks panjang menimbulkan garis kuning-hitam

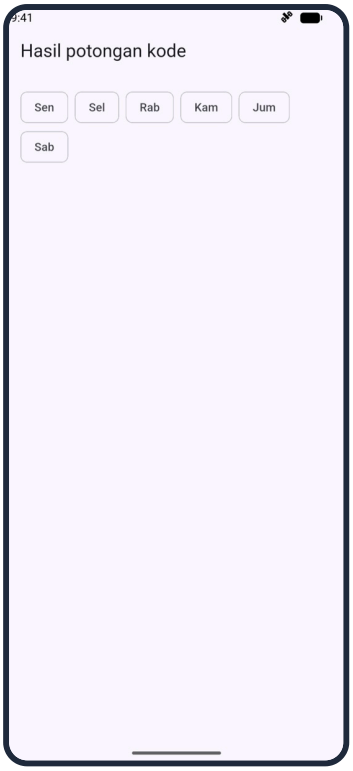
```
// BENAR: teks hanya boleh memakai sisa ruang
Row(
  children: const [
    Icon(Icons.article),
    SizedBox(width: 8),
    Expanded(
      child: Text(
        'Pengumuman Jadwal Ujian Akhir Semester Gasal Tahun Akademik 2026/2027',
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
  ],
)
```



BENAR: teks dibungkus Expanded dan diberi ellipsis

Kasus 2: banyak chip di satu baris

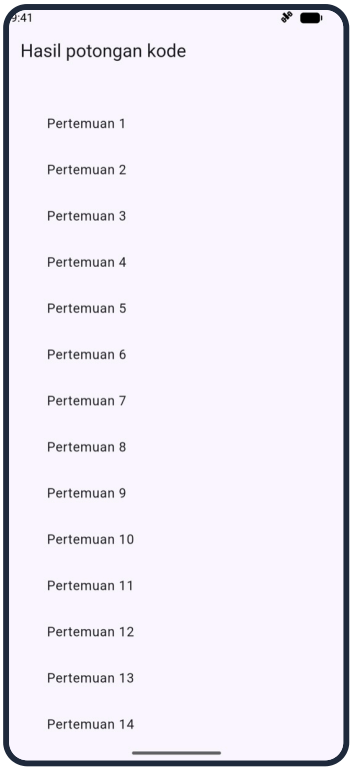
```
// Pilihan A: turun baris otomatis
Wrap(spacing: 8, children: [for (final h in ['Sen', 'Sel', 'Rab', 'Kam', 'Jum', 'Sab']) Chip(label:
Text(h))])

// Pilihan B: tetap satu baris, bisa digeser ke samping
// SingleChildScrollView(
//   scrollDirection: Axis.horizontal,
//   child: Row(children: [...]),
// )
```



Chip hari di dalam Wrap

Kasus 3 & 4: konten lebih tinggi dari layar

```
// SingleChildScrollView membuat isi Column bisa digulir
SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      for (var i = 1; i <= 20; i++)
        ListTile(title: Text('Pertemuan $i')),
    ],
  ),
)
```



Column di dalam SingleChildScrollView

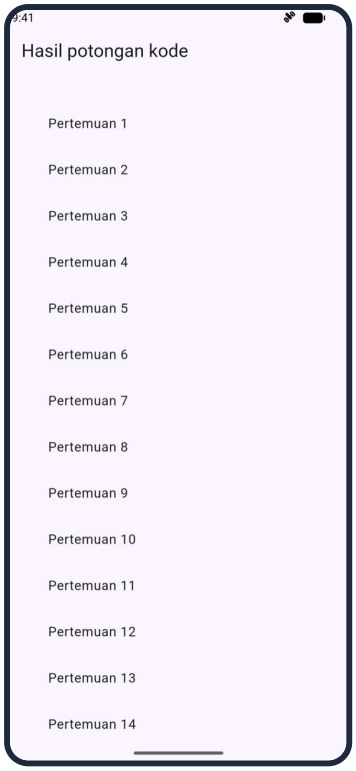
Contoh lengkap: tombol "Rusak" dan "Perbaiki"

Aplikasi berikut sengaja membuat tiga overflow. Tekan saklar **Mode perbaikan** untuk melihat versi yang sudah diperbaiki. Bandingkan kedua versi di kodenya.

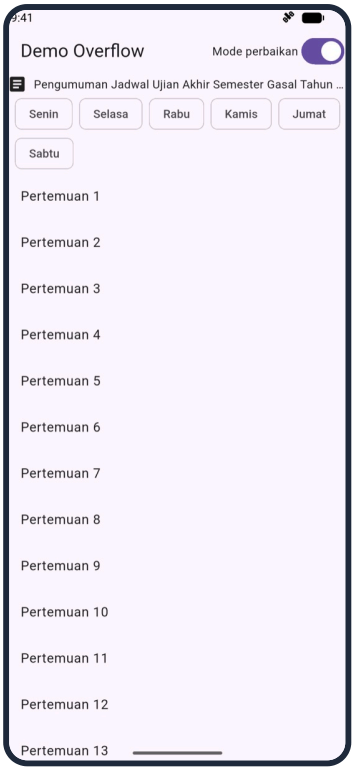
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: DemoOverflow()));
}

class DemoOverflow extends StatefulWidget {
  const DemoOverflow({super.key});

  @override
  State<DemoOverflow> createState() => _DemoOverflowState();
}

class _DemoOverflowState extends State<DemoOverflow> {
  bool _diperbaiki = false;

  static const _judul = 'Pengumuman Jadwal Ujian Akhir Semester Gasal Tahun Akademik 2026/2027';
  static const _hari = ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Demo Overflow'),
```

```
        actions: [
          const Text('Mode perbaikan'),
          Switch(
            value: _diperbaiki,
            onChanged: (v) => setState(() => _diperbaiki = v),
          ),
        ],
      ),
      body: _diperbaiki ? _versiBenar() : _versiRusak(),
    );
  }

  // Versi RUSAK: tiga overflow sekaligus
  Widget _versiRusak() {
    return Column(
      children: [
        const Row(
          children: [
            Icon(Icons.article),
            SizedBox(width: 8),
            Text(_judul), // overflow 1: teks panjang di Row
          ],
        ),
        Row(
          children: [
            for (final h in _hari) Chip(label: Text(h)), // overflow 2: terlalu banyak anak
          ],
        ),
        // overflow 3: total tinggi 20 x 56 px lebih dari tinggi layar
        for (var i = 1; i <= 20; i++) ListTile(title: Text('Pertemuan $i')),
      ],
    );
  }

  // Versi BENAR
  Widget _versiBenar() {
    return SingleChildScrollView( // perbaikan 3: bisa digulir
      child: Column(
        children: [
          const Row(
            children: [
              Icon(Icons.article),
              SizedBox(width: 8),
              Expanded( // perbaikan 1: batasi lebar teks
                child: Text(_judul, maxLines: 1, overflow: TextOverflow.ellipsis),
              ),
            ],
          ),
          Wrap( // perbaikan 2: turun baris otomatis
            spacing: 8,
            children: [for (final h in _hari) Chip(label: Text(h))],
          ),
          for (var i = 1; i <= 20; i++) ListTile(title: Text('Pertemuan $i')),
        ],
      ),
    );
  }
}
```



Versi rusak: tiga overflow sekaligus



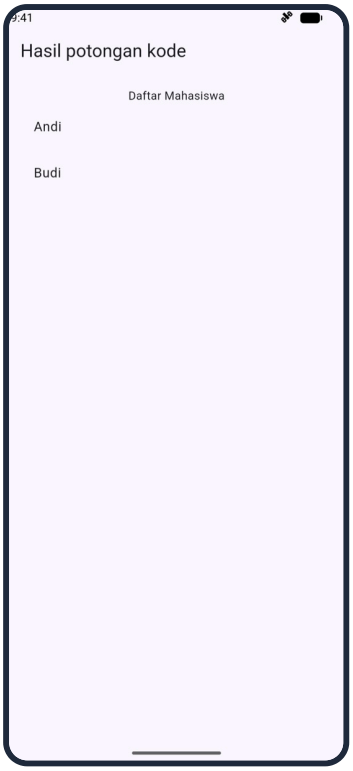
Setelah mode perbaikan diaktifkan

Penjelasan kode

- Di versi rusak, Debug Console menampilkan beberapa pesan *"A RenderFlex overflowed by ... pixels"*. Coba baca dan cocokkan arahnya (right/bottom) dengan Row atau Column penyebabnya.
- Setiap perbaikan diberi komentar nomor yang sama dengan masalahnya.
- Untuk 20 item, `SingleChildScrollView` + `Column` masih wajar. Untuk ratusan item atau data dari server, gunakan `ListView.builder` (Modul 06) yang hanya membangun item yang terlihat.

Kasus 5: ListView di dalam Column

```
Column(
  children: [
    const Text('Daftar Mahasiswa'),
    // Tanpa Expanded: "Vertical viewport was given unbounded height"
    Expanded(
      child: ListView(
        children: const [
          ListTile(title: Text('Andi')),
          ListTile(title: Text('Budi')),
        ],
      ),
    ),
  ],
)
```



ListView di dalam Column dibungkus Expanded

💡 **Flutter Inspector**

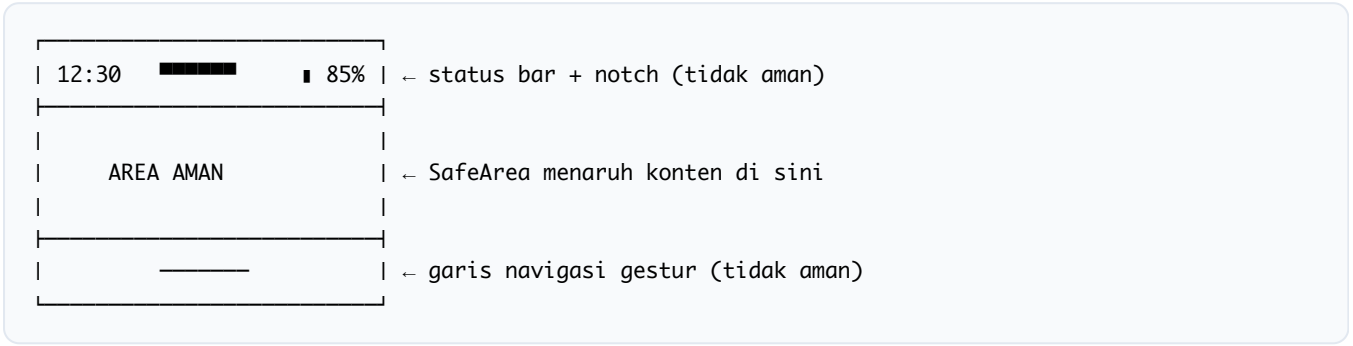
Di VS Code, buka **Flutter DevTools → Widget Inspector** lalu aktifkan *Show Guidelines* atau *Layout Explorer*. Kamu bisa melihat ukuran dan constraints setiap widget secara visual untuk mencari sumber overflow.

⚠️ **Kesalahan umum saat memperbaiki**

- **Membungkus semua halaman dengan SingleChildScrollView** walaupun isinya memakai `Expanded`. Expanded di dalam ruang yang bisa digulir akan error.
- **Mengecilkan ukuran font sampai muat** di HP sendiri. Di HP lain yang lebih sempit atau dengan ukuran font sistem lebih besar, overflow muncul lagi.
- **Mengabaikan overflow karena "hanya muncul di debug".** Di rilis, garisnya hilang tetapi konten tetap terpotong.

8. SafeArea: Menghindari Notch & Status Bar

Layar HP modern tidak lagi persegi panjang sempurna: ada **status bar**, **notch/poni**, **lubang kamera**, sudut membulat, dan **garis navigasi gestur** di bawah. `SafeArea` memberi padding otomatis agar konten tidak tertutup bagian-bagian tersebut.

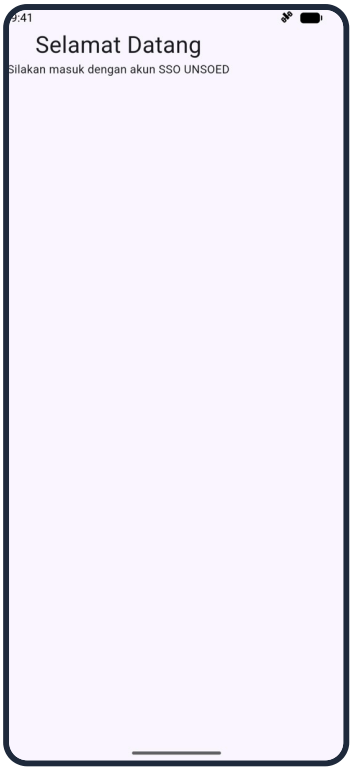


Kapan perlu SafeArea?

- **Halaman dengan `AppBar`:** AppBar sudah otomatis menghindari status bar. `body` tidak perlu SafeArea untuk bagian atas.
- **Halaman tanpa AppBar** (splash screen, halaman login penuh, halaman dengan header gambar): **perlu** SafeArea.
- **Konten menempel di bawah layar** (tombol "Bayar" di bawah): perlu SafeArea bagian bawah agar tidak tertutup garis gestur.

Contoh kecil

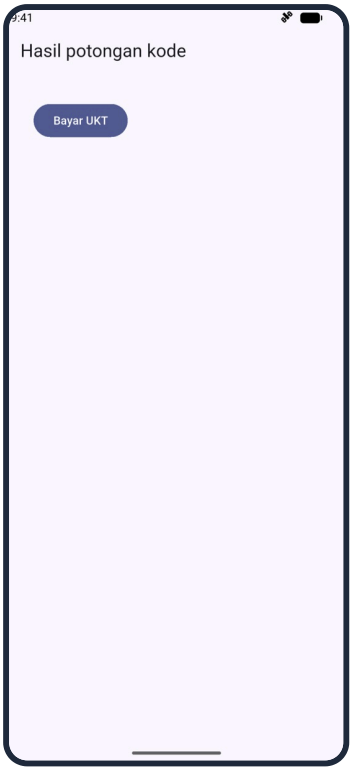
```dart
Scaffold(
  // Tanpa AppBar: teks ini akan tertutup status bar jika tanpa SafeArea
  body: SafeArea(
    child: Column(
      children: const [
        Text('Selamat Datang', style: TextStyle(fontSize: 28)),
        Text('Silakan masuk dengan akun SSO UNSOED'),
      ],
    ),
  ),
)
```



Halaman tanpa AppBar dengan SafeArea

Memilih sisi yang dilindungi

```dart
SafeArea(
  top: false,   // atas boleh menempel (misal gambar header sampai ke status bar)
  bottom: true, // bawah tetap aman dari garis gestur
  minimum: const EdgeInsets.all(16), // padding minimum walau HP tidak punya notch
  child: FilledButton(onPressed: () {}, child: const Text('Bayar UKT')),
)
```



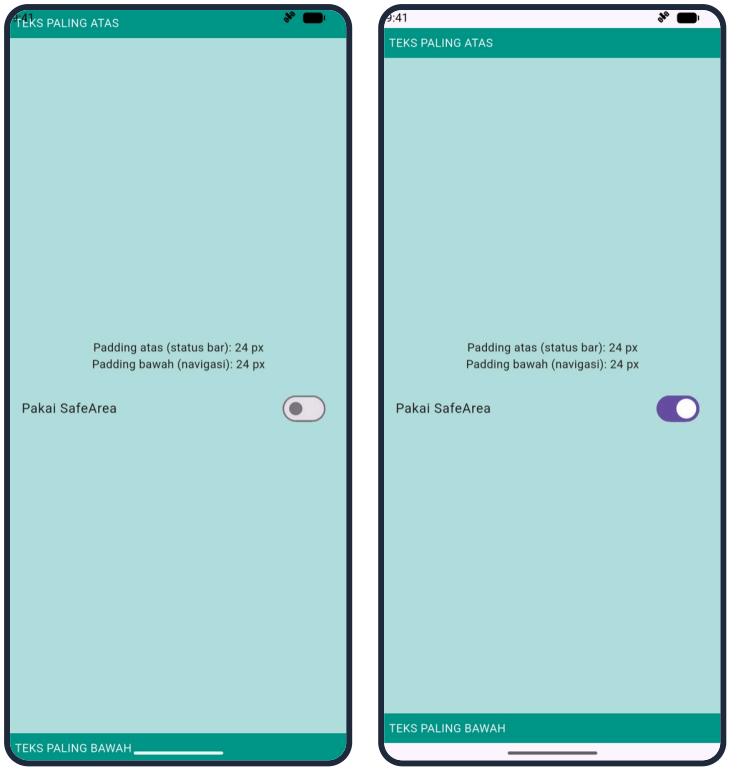
SafeArea bagian bawah dengan padding minimum

Contoh lengkap: membandingkan dengan dan tanpa SafeArea

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: DemoSafeArea()));
}

class DemoSafeArea extends StatefulWidget {
  const DemoSafeArea({super.key});

  @override
  State<DemoSafeArea> createState() => _DemoSafeAreaState();
}

class _DemoSafeAreaState extends State<DemoSafeArea> {
  bool _pakaiSafeArea = false;

  @override
  Widget build(BuildContext context) {
    // Besar area yang tidak aman di HP ini
    final padding = MediaQuery.paddingOf(context);

    final isi = Container(
      color: Colors.teal.shade100,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Container(
```

```
            color: Colors.teal,
            padding: const EdgeInsets.all(8),
            child: const Text('TEKS PALING ATAS', style: TextStyle(color: Colors.white)),
          ),
          Expanded(
            child: Center(
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text('Padding atas (status bar): ${padding.top.toStringAsFixed(0)} px'),
                  Text('Padding bawah (navigasi): ${padding.bottom.toStringAsFixed(0)} px'),
                  const SizedBox(height: 16),
                  SwitchListTile(
                    title: const Text('Pakai SafeArea'),
                    value: _pakaiSafeArea,
                    onChanged: (v) => setState(() => _pakaiSafeArea = v),
                  ),
                ],
              ),
            ),
          ),
          Container(
            color: Colors.teal,
            padding: const EdgeInsets.all(8),
            child: const Text('TEKS PALING BAWAH', style: TextStyle(color: Colors.white)),
          ),
        ],
      ),
    );

    return Scaffold(
      // Sengaja tanpa AppBar
      body: _pakaiSafeArea ? SafeArea(child: isi) : isi,
    );
  }
}
```



Tanpa SafeArea: teks paling atas tertutup status bar

Dengan SafeArea: konten turun ke area aman

Penjelasan kode

- `MediaQuery.paddingOf(context)` berisi ukuran area tidak aman. Nilai inilah yang dipakai SafeArea sebagai padding.
- Saat SafeArea mati, teks "PALING ATAS" bertabrakan dengan jam dan ikon baterai. Saat SafeArea aktif, teks turun ke area aman.
- Widget `isi` disimpan dalam variabel agar bisa dipakai dengan atau tanpa SafeArea tanpa menulis ulang.

⚠️ **Kesalahan umum**

- **Membungkus seluruh `Scaffold` dengan SafeArea.** Latar AppBar jadi tidak menyatu dengan status bar dan muncul strip kosong. Taruh SafeArea di dalam `body`.
- **Hanya menguji di emulator tanpa notch.** Coba juga emulator dengan notch/lubang kamera dan mode landscape (notch pindah ke samping).
- **Lupa sisi bawah** untuk tombol yang menempel di bawah layar pada HP dengan navigasi gestur.

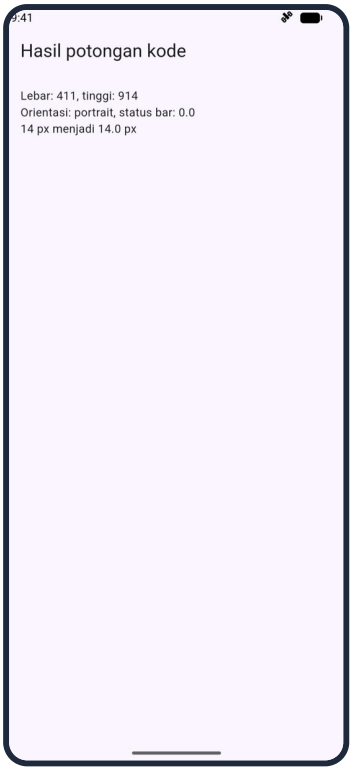
9. Desain Responsif (MediaQuery, LayoutBuilder, Breakpoint, OrientationBuilder)

Aplikasi mobile harus tampil proporsional di smartphone layar kecil (5 inci), tablet (10 inci), maupun saat orientasi layar diputar (landscape). Flutter menyediakan tiga alat utama: `MediaQuery` (info layar), `LayoutBuilder` (info ruang dari induk), dan `OrientationBuilder` (info orientasi).

Alat	Memberi informasi	Cocok untuk
`MediaQuery`	Ukuran **seluruh layar**, padding sistem, orientasi, skala teks	Keputusan tingkat halaman (pakai menu samping atau tidak)
`LayoutBuilder`	Constraints **dari induk langsung**	Widget yang bisa dipakai di mana saja (kartu, grid)
`OrientationBuilder`	Portrait/landscape berdasarkan ruang induk	Mengubah jumlah kolom saat HP diputar

MediaQuery: membaca info layar

```
Widget infoLayar(BuildContext context) {
  final ukuran = MediaQuery.sizeOf(context);          // Size(lebar, tinggi)
  final orientasi = MediaQuery.orientationOf(context); // portrait / landscape
  final padding = MediaQuery.paddingOf(context);      // area status bar, notch
  final skalaTeks = MediaQuery.textScalerOf(context); // pengaturan ukuran font HP

  return Text(
    'Lebar: ${ukuran.width.toStringAsFixed(0)}, tinggi: ${ukuran.height.toStringAsFixed(0)}\n'
    'Orientasi: ${orientasi.name}, status bar: ${padding.top}\n'
    '14 px menjadi ${skalaTeks.scale(14).toStringAsFixed(1)} px',
  );
}
```



Informasi layar dari MediaQuery

> 💡 **Pakai `sizeOf` , bukan `of(context).size`**
>
> `MediaQuery.sizeOf(context)` hanya membangun ulang widget saat *ukuran* berubah. `MediaQuery.of(context).size` membangun ulang setiap kali data MediaQuery apa pun berubah (misalnya keyboard muncul), sehingga lebih boros.

Breakpoint: batas lebar untuk berganti tata letak

Material 3 menyarankan tiga kelas ukuran jendela. Simpan sebagai konstanta agar konsisten di seluruh aplikasi:

Kelas	Lebar	Perangkat umum	Navigasi yang disarankan
Compact	< 600	HP portrait	`NavigationBar` di bawah
Medium	600 - 839	Tablet portrait, HP lipat	`NavigationRail` di samping
Expanded	≥ 840	Tablet landscape, desktop	`NavigationRail` diperluas / drawer permanen

```
class Breakpoint {
  static const double medium = 600;
  static const double expanded = 840;
```

```
}

enum KelasLayar { compact, medium, expanded }

KelasLayar kelasDariLebar(double lebar) {
  if (lebar >= Breakpoint.expanded) return KelasLayar.expanded;
  if (lebar >= Breakpoint.medium) return KelasLayar.medium;
  return KelasLayar.compact;
}
```

Contoh lengkap 1: MediaQuery untuk menu samping

Jalankan di Chrome (`flutter run -d chrome`) lalu ubah lebar jendela browser untuk melihat tata letak berganti antara mode HP dan tablet.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DashboardScreen(),
    );
  }
}

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // Mendapatkan ukuran & orientasi layar perangkat saat ini
    final Size screenSize = MediaQuery.sizeOf(context);
    final Orientation orientasi = MediaQuery.orientationOf(context);
    final bool isTablet = screenSize.width > 600;

    return Scaffold(
      appBar: AppBar(
        title: Text("Dashboard (${orientasi == Orientation.portrait ? 'Portrait' : 'Landscape'})"),
      ),
      body: isTablet
          ? Row(
              children: [
                // Di tablet / layar lebar tampilkan menu samping
                Container(
                  width: 250,
                  color: Colors.blueGrey[50],
                  child: ListView(
```

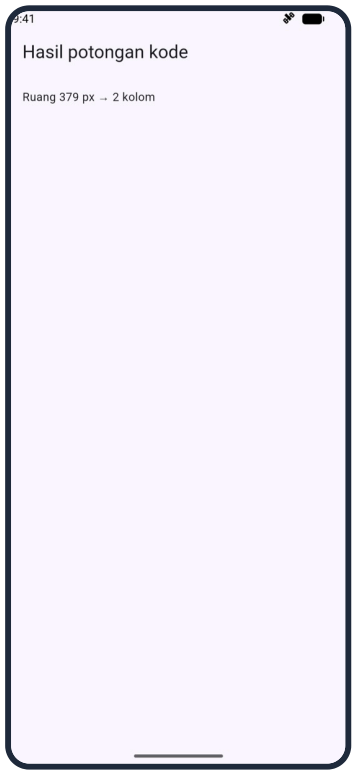
```
                    children: const [
                      ListTile(leading: Icon(Icons.home), title: Text("Beranda")),
                      ListTile(leading: Icon(Icons.calendar_month), title: Text("Jadwal")),
                      ListTile(leading: Icon(Icons.person), title: Text("Profil")),
                    ],
                  ),
                ),
                const Expanded(child: KontenUtama()),
              ],
            )
          : const KontenUtama(), // Di HP tampilkan satu kolom penuh
    );
  }
}

class KontenUtama extends StatelessWidget {
  const KontenUtama({super.key});

  @override
  Widget build(BuildContext context) {
    final lebar = MediaQuery.sizeOf(context).width;
    return Center(
      child: Text(
        "Lebar layar: ${lebar.toStringAsFixed(0)} px\n"
        "${lebar > 600 ? 'Mode Tablet (menu samping tampil)' : 'Mode HP (satu kolom)'}",
        textAlign: TextAlign.center,
        style: const TextStyle(fontSize: 18),
      ),
    );
  }
}
```

Dashboard di layar HP (mode satu kolom)

LayoutBuilder: responsif berdasarkan ruang induk

MediaQuery melihat ukuran *layar*. Masalahnya, sebuah widget belum tentu mendapat seluruh layar. Misalnya `KontenUtama` di contoh tadi hanya mendapat (lebar layar - 250) di mode tablet. `LayoutBuilder` memberi constraints yang benar-benar diterima widget tersebut.

```
LayoutBuilder(
  builder: (context, constraints) {
    // constraints.maxWidth = lebar yang diberikan induk
    final kolom = constraints.maxWidth >= 600 ? 3 : 2;
    return Text('Ruang ${constraints.maxWidth.toStringAsFixed(0)} px → $kolom kolom');
  },
)
```



LayoutBuilder menentukan jumlah kolom

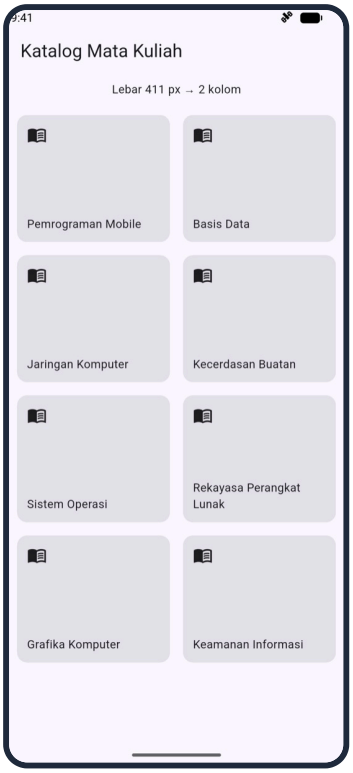
Contoh lengkap 2: grid responsif dengan LayoutBuilder dan breakpoint

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.indigo),
    home: const KatalogMatkul(),
  ));
}

class KatalogMatkul extends StatelessWidget {
```

```
  const KatalogMatkul({super.key});

  static const _matkul = [
    'Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer', 'Kecerdasan Buatan',
    'Sistem Operasi', 'Rekayasa Perangkat Lunak', 'Grafika Komputer', 'Keamanan Informasi',
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Katalog Mata Kuliah')),
      body: LayoutBuilder(
        builder: (context, constraints) {
          final lebar = constraints.maxWidth;
          // Jumlah kolom ditentukan dari breakpoint
          final int kolom;
          if (lebar >= 840) {
            kolom = 4;
          } else if (lebar >= 600) {
            kolom = 3;
          } else {
            kolom = 2;
          }

          return Column(
            children: [
              Padding(
                padding: const EdgeInsets.all(8),
                child: Text('Lebar ${lebar.toStringAsFixed(0)} px → $kolom kolom'),
              ),
              Expanded(
                child: GridView.count(
                  crossAxisCount: kolom,
                  padding: const EdgeInsets.all(8),
                  mainAxisSpacing: 8,
                  crossAxisSpacing: 8,
                  childAspectRatio: 1.2,
                  children: [
                    for (final m in _matkul)
                      Card.filled(
                        child: Padding(
                          padding: const EdgeInsets.all(12),
                          child: Column(
                            crossAxisAlignment: CrossAxisAlignment.start,
                            mainAxisAlignment: MainAxisAlignment.spaceBetween,
                            children: [
                              const Icon(Icons.menu_book),
                              Text(m, maxLines: 2, overflow: TextOverflow.ellipsis),
                            ],
                          ),
                        ),
                      ),
                  ],
                ),
              ),
            ],
          );
        },
      ),
```

```
    );
  }
}
```



Grid responsif: 2 kolom di HP portrait

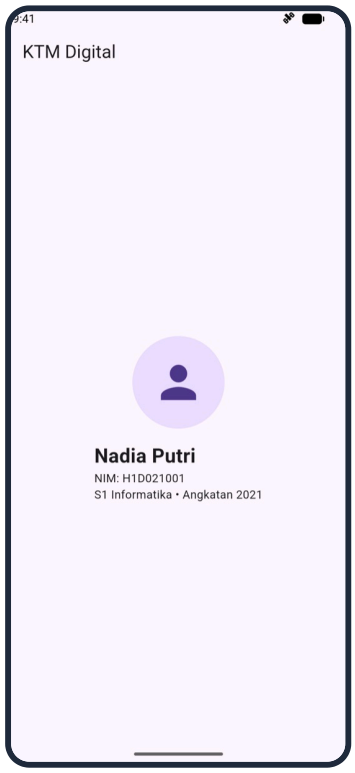
`GridView.count` membuat grid dengan jumlah kolom tetap (dibahas lebih lengkap di Modul 06). Di sini jumlah kolomnya dihitung dari lebar yang diberikan `LayoutBuilder`.

OrientationBuilder: portrait vs landscape

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: HalamanKtm()));
}

class HalamanKtm extends StatelessWidget {
  const HalamanKtm({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('KTM Digital')),
      body: OrientationBuilder(
        builder: (context, orientation) {
          const foto = CircleAvatar(radius: 56, child: Icon(Icons.person, size: 64));
          const data = Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.start,
```

```
            children: [
              Text('Nadia Putri', style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
              Text('NIM: H1D021001'),
              Text('S1 Informatika • Angkatan 2021'),
            ],
          );

          // Portrait: foto di atas data. Landscape: foto di samping data.
          if (orientation == Orientation.portrait) {
            return const Center(
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [foto, SizedBox(height: 16), data],
              ),
            );
          }
          return const Center(
            child: Row(
              mainAxisSize: MainAxisSize.min,
              children: [foto, SizedBox(width: 32), data],
            ),
          );
        },
      ),
    );
  }
}
```



OrientationBuilder pada mode portrait

Putar emulator (tombol rotasi di panel samping emulator) untuk melihat foto berpindah dari atas ke samping. Pastikan rotasi otomatis di emulator aktif.

⚠️ **Kesalahan umum**

- **Menulis ukuran piksel tetap** seperti `width: 360` untuk konten utama. Di HP 320 px terjadi overflow, di tablet tampak kecil di tengah. Gunakan Expanded, persentase dari lebar, atau breakpoint.
- **Memakai MediaQuery untuk widget yang ada di dalam panel sempit.** Lebar layar besar, tetapi ruang widget kecil. Pakai `LayoutBuilder`.
- **Mengecek `Platform.isAndroid` untuk menentukan layout.** Tablet Android dan HP Android sama-sama Android. Tentukan layout dari ukuran, bukan dari jenis platform.
- **Mengabaikan skala teks.** Pengguna dengan font sistem besar membuat teks lebih tinggi; hindari tinggi tetap pada kotak berisi teks.

10. Studi Kasus: Dashboard Portal Mahasiswa

Saatnya menggabungkan semua yang sudah dipelajari menjadi satu halaman utuh: **Dashboard Portal Mahasiswa**. Halaman ini memakai Stack, Row, Column, Expanded, Wrap, AspectRatio, Spacer, SafeArea, SingleChildScrollView, dan LayoutBuilder sekaligus.

Langkah 1: rancang dulu di atas kertas

Sebelum menulis kode, pecah halaman menjadi kotak-kotak. Setiap kotak nantinya menjadi satu widget.

```
Scaffold
└── SafeArea (bagian bawah)
    └── SingleChildScrollView
        └── Column
            ├── HeaderDashboard      ← Stack: banner gradasi + avatar + sapaan
            ├── RingkasanAkademik    ← Row + 3 x Expanded(KartuStatistik)
            ├── JudulBagian "Menu"
            ├── MenuCepat            ← LayoutBuilder + Wrap (jumlah per baris menyesuaikan)
            ├── JudulBagian "Jadwal Hari Ini"
            ├── KartuJadwal x 3      ← Row: jam | garis | detail(Expanded)
            ├── JudulBagian "Pengumuman"
            └── BannerPengumuman     ← AspectRatio 16:7 + Stack
```

Langkah 2: kode lengkap

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const PortalApp());
}

class PortalApp extends StatelessWidget {
  const PortalApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Portal Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: const Color(0xFF1E3A8A)),
      home: const DashboardMahasiswa(),
    );
  }
}

// ========================= HALAMAN =========================

class DashboardMahasiswa extends StatelessWidget {
  const DashboardMahasiswa({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: SafeArea(
        top: false, // header boleh sampai ke belakang status bar
        child: SingleChildScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              HeaderDashboard(nama: 'Nadia Putri', nim: 'H1D021001'),
              RingkasanAkademik(),
              JudulBagian(judul: 'Menu'),
              MenuCepat(),
              JudulBagian(judul: 'Jadwal Hari Ini'),
              KartuJadwal(jam: '07.30', matkul: 'Pemrograman Mobile', ruang: 'Lab Komputer 2',
sedangBerlangsung: true),
              KartuJadwal(jam: '10.00', matkul: 'Basis Data Lanjut', ruang: 'Ruang E201'),
              KartuJadwal(jam: '13.00', matkul: 'Rekayasa Perangkat Lunak Berorientasi Objek',
ruang: 'Ruang E305'),
              JudulBagian(judul: 'Pengumuman'),
              BannerPengumuman(),
              SizedBox(height: 24),
            ],
          ),
        ),
      ),
    );
  }
}

// ======================== HEADER (Stack) ========================

class HeaderDashboard extends StatelessWidget {
  final String nama;
  final String nim;

  const HeaderDashboard({super.key, required this.nama, required this.nim});

  @override
  Widget build(BuildContext context) {
    final atas = MediaQuery.paddingOf(context).top; // tinggi status bar

    return Stack(
      children: [
        // Lapisan 1: latar gradasi (anak non-Positioned menentukan ukuran Stack)
        Container(
          padding: EdgeInsets.fromLTRB(20, atas + 16, 20, 56),
          decoration: const BoxDecoration(
            gradient: LinearGradient(
              colors: [Color(0xFF1E3A8A), Color(0xFF3B82F6)],
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
            ),
            borderRadius: BorderRadius.vertical(bottom: Radius.circular(28)),
          ),
          child: Row(
            children: [
              const CircleAvatar(radius: 28, child: Icon(Icons.person, size: 32)),
              const SizedBox(width: 12),
```

```
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Text('Selamat pagi,', style: TextStyle(color: Colors.white70)),
                    Text(
                      nama,
                      style: const TextStyle(color: Colors.white, fontSize: 22, fontWeight:
FontWeight.bold),
                      overflow: TextOverflow.ellipsis,
                    ),
                    Text(nim, style: const TextStyle(color: Colors.white70)),
                  ],
                ),
              ),
              IconButton(
                onPressed: () {},
                tooltip: 'Notifikasi',
                icon: const Badge(
                  label: Text('3'),
                  child: Icon(Icons.notifications_outlined, color: Colors.white),
                ),
              ),
            ],
          ),
        ),
        // Lapisan 2: lingkaran dekorasi transparan
        Positioned(
          right: -30,
          bottom: -30,
          child: Container(
            width: 120,
            height: 120,
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: Colors.white.withValues(alpha: 0.08),
            ),
          ),
        ),
      ],
    );
  }
}

// ======================= RINGKASAN (Row + Expanded) =======================

class RingkasanAkademik extends StatelessWidget {
  const RingkasanAkademik({super.key});

  @override
  Widget build(BuildContext context) {
    // Transform.translate menggeser kartu ke atas agar menumpuk di tepi header
    return Transform.translate(
      offset: const Offset(0, -32),
      child: const Padding(
        padding: EdgeInsets.symmetric(horizontal: 16),
        child: Row(
          children: [
            Expanded(child: KartuStatistik(nilai: '3.85', label: 'IPK', ikon: Icons.star)),
```

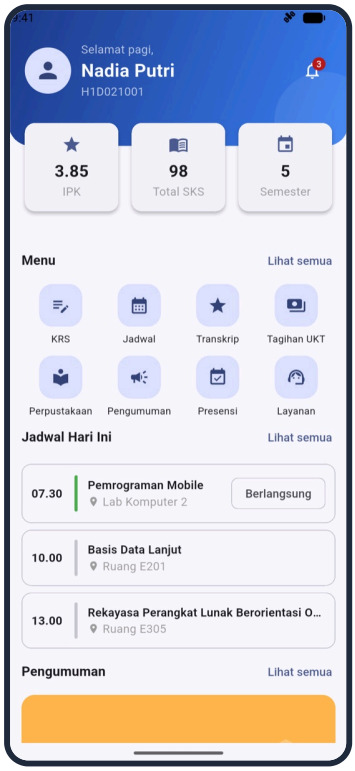
```
            SizedBox(width: 8),
            Expanded(child: KartuStatistik(nilai: '98', label: 'Total SKS', ikon: Icons.menu_book)),
            SizedBox(width: 8),
            Expanded(child: KartuStatistik(nilai: '5', label: 'Semester', ikon: Icons.event)),
          ],
        ),
      ),
    );
  }
}

class KartuStatistik extends StatelessWidget {
  final String nilai;
  final String label;
  final IconData ikon;

  const KartuStatistik({super.key, required this.nilai, required this.label, required this.ikon});

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Card(
      elevation: 3,
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 8),
        child: Column(
          children: [
            Icon(ikon, color: warna.primary),
            const SizedBox(height: 6),
            Text(nilai, style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
            // FittedBox mengecilkan teks jika kartu terlalu sempit
            FittedBox(child: Text(label, style: const TextStyle(color: Colors.grey))),
          ],
        ),
      ),
    );
  }
}

// ======================== JUDUL BAGIAN (Row + Spacer) ========================

class JudulBagian extends StatelessWidget {
  final String judul;

  const JudulBagian({super.key, required this.judul});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 0, 8, 4),
      child: Row(
        children: [
          Text(judul, style: Theme.of(context).textTheme.titleMedium?.copyWith(fontWeight:
FontWeight.bold)),
          const Spacer(),
          TextButton(onPressed: () {}, child: const Text('Lihat semua')),
        ],
      ),
    );
```

```
  }
}

// ======================== MENU CEPAT (LayoutBuilder + Wrap) ========================

class MenuCepat extends StatelessWidget {
  const MenuCepat({super.key});

  static const _menu = [
    (Icons.edit_note, 'KRS'),
    (Icons.calendar_month, 'Jadwal'),
    (Icons.grade, 'Transkrip'),
    (Icons.payments, 'Tagihan UKT'),
    (Icons.local_library, 'Perpustakaan'),
    (Icons.campaign, 'Pengumuman'),
    (Icons.event_available, 'Presensi'),
    (Icons.support_agent, 'Layanan'),
  ];

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: LayoutBuilder(
        builder: (context, constraints) {
          // HP: 4 per baris. Layar lebar: 8 per baris.
          final perBaris = constraints.maxWidth >= 600 ? 8 : 4;
          final lebarItem = constraints.maxWidth / perBaris;
          return Wrap(
            runSpacing: 12,
            children: [
              for (final (ikon, label) in _menu)
                SizedBox(width: lebarItem, child: ItemMenu(ikon: ikon, label: label)),
            ],
          );
        },
      ),
    );
  }
}

class ItemMenu extends StatelessWidget {
  final IconData ikon;
  final String label;

  const ItemMenu({super.key, required this.ikon, required this.label});

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return InkWell(
      onTap: () {},
      borderRadius: BorderRadius.circular(12),
      child: Column(
        children: [
          Container(
            width: 52,
            height: 52,
            decoration: BoxDecoration(
```

```
              color: warna.primaryContainer,
              borderRadius: BorderRadius.circular(16),
            ),
            child: Icon(ikon, color: warna.onPrimaryContainer),
          ),
          const SizedBox(height: 6),
          Text(
            label,
            textAlign: TextAlign.center,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: const TextStyle(fontSize: 12),
          ),
        ],
      ),
    );
  }
}

// ======================= JADWAL (Row + Expanded) =======================

class KartuJadwal extends StatelessWidget {
  final String jam;
  final String matkul;
  final String ruang;
  final bool sedangBerlangsung;

  const KartuJadwal({
    super.key,
    required this.jam,
    required this.matkul,
    required this.ruang,
    this.sedangBerlangsung = false,
  });

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Card.outlined(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
      child: Padding(
        padding: const EdgeInsets.all(12),
        child: Row(
          children: [
            SizedBox(
              width: 52,
              child: Text(jam, style: const TextStyle(fontWeight: FontWeight.bold)),
            ),
            Container(
              width: 4,
              height: 44,
              decoration: BoxDecoration(
                color: sedangBerlangsung ? Colors.green : warna.outlineVariant,
                borderRadius: BorderRadius.circular(2),
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Column(
```

```
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(matkul, maxLines: 1, overflow: TextOverflow.ellipsis,
                      style: const TextStyle(fontWeight: FontWeight.w600)),
                  Row(
                    children: [
                      const Icon(Icons.room, size: 14, color: Colors.grey),
                      const SizedBox(width: 4),
                      Text(ruang, style: const TextStyle(color: Colors.grey)),
                    ],
                  ),
                ],
              ),
            ),
            if (sedangBerlangsung) const Chip(label: Text('Berlangsung')),
          ],
        ),
      ),
    );
  }
}

// ======================= PENGUMUMAN (AspectRatio + Stack) =======================

class BannerPengumuman extends StatelessWidget {
  const BannerPengumuman({super.key});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: ClipRRect(
        borderRadius: BorderRadius.circular(16),
        child: AspectRatio(
          aspectRatio: 16 / 7,
          child: Stack(
            fit: StackFit.expand, // anak non-Positioned memenuhi Stack
            children: [
              Container(color: Colors.orange.shade300),
              const Positioned(
                right: -10,
                bottom: -10,
                child: Icon(Icons.school, size: 140, color: Colors.white24),
              ),
              const Padding(
                padding: EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  mainAxisAlignment: MainAxisAlignment.end,
                  children: [
                    Text('Pendaftaran Wisuda Periode II',
                        style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
                    Text('Batas akhir 30 November 2026'),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
```

```
        ),
      );
    }
  }
```



Dashboard portal mahasiswa (bagian atas)

Dashboard setelah digulir ke bawah

Langkah 3: bedah kode per bagian

Bagian	Teknik layout	Kenapa?
`DashboardMahasiswa`	`SafeArea(top: false)` + `SingleChildScrollView`	Header boleh sampai ke status bar, tetapi bagian bawah aman. Seluruh halaman bisa digulir sehingga tidak overflow di HP kecil atau landscape.
`HeaderDashboard`	`Stack` + `Positioned` + `Expanded`	Lingkaran dekorasi ditumpuk di atas gradasi. Nama panjang tidak mendorong ikon notifikasi keluar.
`RingkasanAkademik`	3 x `Expanded` + `Transform.translate`	Tiga kartu selalu sama lebar, dan digeser ke atas agar menumpuk di tepi header.
`JudulBagian`	`Row` + `Spacer`	Judul di kiri, "Lihat semua" di kanan.
`MenuCepat`	`LayoutBuilder` + `Wrap`	Lebar item dihitung dari ruang yang tersedia: 4 per baris di HP, 8 di tablet.
`KartuJadwal`	`Row`: lebar tetap | garis | `Expanded`	Kolom jam rapi sejajar; nama mata kuliah panjang dipotong dengan ellipsis.
`BannerPengumuman`	`AspectRatio` + `Stack(fit: StackFit.expand)`	Proporsi banner tetap di semua ukuran layar.

- `static const _menu = [(Icons.edit_note, 'KRS'), ...]` memakai **record** Dart 3, lalu dibongkar dengan `for (final (ikon, label) in _menu)` (lihat Modul 02).
- `FittedBox` mengecilkan label statistik secara otomatis jika kartu terlalu sempit, sebagai pengaman tambahan dari overflow.
- `Badge` adalah widget Material 3 untuk angka kecil di pojok ikon.

Langkah 4: uji ketahanan layout

1. Putar emulator ke landscape: halaman tetap bisa digulir tanpa garis kuning-hitam.
2. Jalankan di Chrome (`flutter run -d chrome`) dan lebarkan jendela: menu cepat berubah menjadi 8 per baris.
3. Ganti nama di `HeaderDashboard` dengan nama yang sangat panjang: nama terpotong dengan "...".
4. Perbesar ukuran font di pengaturan HP (Settings → Display → Font size): periksa apakah ada yang terpotong.

⚠️ **Kesalahan umum pada studi kasus**

- **Menulis semua bagian di satu method** `build` sampai ratusan baris. Pecah seperti contoh: satu bagian = satu class.
- **Memakai** `Expanded` **di dalam Column halaman** padahal Column berada di `SingleChildScrollView`. Tinggi tak terbatas tidak bisa dibagi dan akan error.
- **Lupa** `maxLines` **+** `ellipsis` pada teks yang datanya berasal dari server. Panjang data asli sering di luar dugaan.

Tugas Praktikum: Praktikum 4: Halaman Profil Mahasiswa Responsif

Sasaran Praktikum

Membangun halaman profil mahasiswa yang memadukan Stack, Row, Column, Expanded, Wrap, AspectRatio, SafeArea, dan LayoutBuilder, serta bebas overflow di HP kecil, landscape, maupun layar lebar.

Langkah Pengerjaan

1. Buat project baru `flutter create praktikum_04` dan file `lib/profil_mahasiswa_screen.dart`. Panggil halaman tersebut dari `lib/main.dart`.
2. Header (Stack): banner gradasi dengan `AspectRatio(16 / 6)`, foto profil `CircleAvatar` yang menjorok keluar banner memakai `Positioned`, dan tombol kembali yang tidak tertutup status bar (`SafeArea` atau `MediaQuery.paddingOf`).
3. Di bawah header, tampilkan nama, NIM, dan prodi. Nama harus memakai `maxLines: 1` dan `TextOverflow.ellipsis`; uji dengan nama sangat panjang.
4. Buat baris statistik (IPK, SKS, Semester) memakai `Row` dan tiga `Expanded` agar sama lebar.
5. Buat bagian 'Keahlian' berisi minimal 8 `Chip` di dalam `Wrap` dengan `spacing` dan `runSpacing`.
6. Buat bagian 'Informasi' berisi pasangan label-nilai (Dosen Wali, Angkatan, Email, Status) memakai `Row` + `Spacer` atau `MainAxisAlignment.spaceBetween`.
7. Gunakan `LayoutBuilder`: jika lebar ≥ 600 px, tampilkan bagian Keahlian dan Informasi berdampingan (Row dengan dua Expanded); jika lebih kecil, susun bertumpuk (Column).
8. Bungkus konten dengan `SingleChildScrollView`. Uji di emulator portrait, landscape, dan di Chrome dengan berbagai lebar jendela. Lampirkan 3 screenshot bukti tanpa garis kuning-hitam.
9. Pecah halaman menjadi minimal 4 widget kecil (misal `HeaderProfil`, `BarisStatistik`, `BagianKeahlian`, `BagianInformasi`) dan pastikan `flutter analyze` bersih.

Output yang Diharapkan

Halaman profil mahasiswa yang rapi dan profesional, menyesuaikan susunan di layar sempit dan lebar, serta bebas dari error RenderFlex overflow di semua orientasi.

Rubrik Penilaian

Penerapan Stack/Positioned, Row/Column, Expanded, Wrap, dan AspectRatio yang tepat (35%), Responsivitas dengan LayoutBuilder dan uji orientasi (25%), Bebas overflow & SafeArea (20%), Pemecahan widget dan kerapian kode (20%).

Kuis Pemahaman

1. Apa perbedaan utama antara widget 'Expanded' dan 'Flexible' di dalam sebuah Row atau Column?

A. Expanded memaksa widget anaknya untuk mengisi SELURUH ruang sisa yang tersedia (fit: FlexFit.tight), sedangkan Flexible membolehkan anak berukuran lebih kecil dari ruang sisa (fit: FlexFit.loose)

B. Expanded hanya bisa digunakan di dalam Stack, sedangkan Flexible untuk Column

C. Expanded membuat widget menjadi transparan, sedangkan Flexible membuatnya berwarna

D. Tidak ada perbedaan sama sekali

2. Widget manakah yang digunakan untuk menumpuk widget satu di atas widget lainnya seperti layer Photoshop?

A. Row

B. Column

C. Stack

D. Wrap

3. Kode runApp(Container(width: 100, height: 100, color: Colors.red)) menghasilkan kotak merah sepenuh layar. Mengapa?

A. Karena Container tidak mendukung width dan height

B. Karena layar memberi constraint ketat (tight) seukuran layar, sehingga ukuran 100 × 100 diabaikan

C. Karena warna merah selalu memenuhi layar

D. Karena lupa memanggil setState

4. Pada sebuah Column, properti apa yang mengatur perataan anak secara HORIZONTAL (kiri/tengah/kanan)?

A. mainAxisAlignment

B. crossAxisAlignment

C. mainAxisSize

D. textDirection

5. Muncul pesan 'A RenderFlex overflowed by 48 pixels on the bottom' pada halaman berisi form panjang. Perbaikan yang paling tepat adalah...

A. Membungkus Column dengan SingleChildScrollView

B. Membungkus setiap TextField dengan Expanded

C. Mengubah Column menjadi Row

D. Menghapus Scaffold

6. Sederet 10 Chip di dalam Row melebihi lebar layar. Widget pengganti Row yang otomatis memindahkan chip ke baris berikutnya adalah...

A. Stack

B. Wrap

C. Expanded

D. AspectRatio

7. Sebuah widget kartu dipakai di panel sempit pada tablet. Agar jumlah kolomnya dihitung dari ruang yang benar-benar diberikan induknya, alat yang tepat adalah...

A. MediaQuery.sizeOf(context)

B. LayoutBuilder

C. Platform.isAndroid

D. SafeArea

8. Halaman tanpa AppBar menampilkan teks yang tertutup jam dan ikon baterai. Widget yang menyelesaikan masalah ini adalah...

A. SafeArea

B. Spacer

C. Flexible

D. Positioned

Kunci Jawaban & Pembahasan

1. A — Secara default, Expanded memiliki sifat fit: FlexFit.tight yang memaksa anak menghabiskan ruang sisa yang ada. Sementara Flexible memiliki default FlexFit.loose yang membiarkan anak berukuran sesuai keinginannya tanpa dipaksa melebar penuh.

2. C — Widget Stack digunakan untuk menyusun widget secara bertumpuk (layering) di sumbu z, di mana widget anak berikutnya berada di atas widget sebelumnya.

3. B — Constraints go down: root memberi constraint tight seukuran layar. Anak harus mematuhinya. Bungkus dengan Center agar constraint menjadi longgar.

4. B — Sumbu utama Column adalah vertikal, sehingga perataan horizontal diatur oleh crossAxisAlignment.

5. A — Overflow 'on the bottom' berarti Column lebih tinggi dari ruangnya. SingleChildScrollView membuat isinya bisa digulir.

6. B — Wrap menyusun anak berderet dan turun ke baris baru ketika ruang horizontal habis.

7. B — MediaQuery memberi ukuran seluruh layar, sedangkan LayoutBuilder memberi constraints dari induk langsung.

8. A — SafeArea menambahkan padding sebesar area status bar, notch, dan navigasi gestur.

MODUL 05 · INPUT & FORM

Interaktivitas, State Dasar & Validasi Formulir

Membangun antarmuka interaktif dengan tombol Material 3 dan gestur, mengelola input teks dengan TextEditingController dan FocusNode, memvalidasi formulir dengan GlobalKey<FormState>, memakai input pilihan (Checkbox, Switch, RadioGroup, Slider, Dropdown, DatePicker), serta memberi umpan balik lewat SnackBar, AlertDialog, dan BottomSheet.

Durasi Pertemuan 5 (150 Menit) + Belajar Mandiri

Prasyarat Modul 04: Sistem Layouting

Capaian Pembelajaran

- Memilih varian tombol Material 3 (FilledButton, ElevatedButton, OutlinedButton, TextButton, IconButton, FAB) sesuai hierarki aksi dan menonaktifkannya dengan onPressed null.
- Membuat widget kustom dapat disentuh dengan GestureDetector dan InkWell.
- Mengatur tampilan TextField dengan InputDecoration serta mengelola TextEditingController dan FocusNode beserta dispose-nya.
- Menerapkan validasi form lengkap (email, password, konfirmasi password) dengan Form, TextFormField, dan GlobalKey<FormState>.
- Menggunakan Checkbox, Switch, RadioGroup, Slider, DropdownButtonFormField, DatePicker, dan TimePicker.
- Memberi umpan balik kepada pengguna dengan SnackBar, AlertDialog, dan BottomSheet.
- Membangun formulir pendaftaran mahasiswa lengkap beserta halaman hasilnya.

Daftar Isi Modul

1. Ragam Tombol Material 3
2. GestureDetector & InkWell
3. TextField & InputDecoration
4. TextEditingController: Baca, Ubah, Kosongkan
5. Jenis Keyboard, Password & FocusNode
6. Form, TextFormField & Validasi
7. Checkbox, Switch, Radio & Slider
8. Dropdown, DatePicker & TimePicker
9. Umpan Balik: SnackBar, AlertDialog & BottomSheet
10. Studi Kasus: Formulir Pendaftaran Mahasiswa
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Ragam Tombol Material 3

Tombol adalah cara paling dasar pengguna "berbicara" dengan aplikasi. Material Design 3 (M3) menyediakan beberapa jenis tombol dengan **hierarki** yang jelas: aksi paling penting dibuat paling menonjol, aksi pendukung dibuat lebih kalem.

Konsep: memilih tombol sesuai tingkat kepentingan

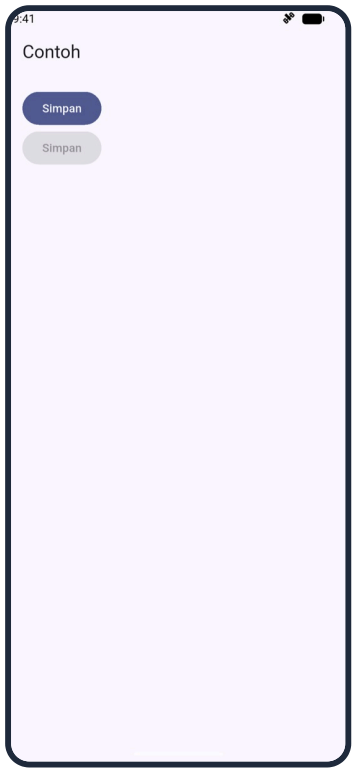
Widget	Kapan dipakai	Contoh di aplikasi kampus
`FilledButton`	Aksi utama, paling menonjol. Idealnya satu per layar.	Simpan KRS, Kirim, Masuk
`FilledButton.tonal`	Aksi penting tapi bukan yang utama.	Simpan sebagai draft
`ElevatedButton`	Tombol berbayang, dipakai jika latar belakang ramai sehingga tombol perlu "terangkat".	Lihat KHS di atas gambar
`OutlinedButton`	Aksi sekunder, sering berpasangan dengan FilledButton.	Batal, Kembali
`TextButton`	Aksi prioritas rendah, tampil seperti tautan.	Lupa password?, Lihat semua
`IconButton`	Aksi berupa ikon saja, di AppBar, kartu, atau baris daftar.	Hapus, Favorit, Bagikan
`FloatingActionButton` (FAB)	Aksi utama sebuah layar yang "melayang" di pojok kanan bawah.	Tambah catatan, Isi KRS

Contoh kecil: tombol aktif dan tombol nonaktif

Semua tombol di atas punya dua parameter utama: `onPressed` (fungsi yang dijalankan saat ditekan) dan `child` (isi tombol, biasanya `Text`). Aturan pentingnya: **jika `onPressed` bernilai `null`, tombol otomatis menjadi nonaktif (disabled)**, warnanya memudar dan tidak bisa ditekan.

```
// Tombol AKTIF: onPressed berisi fungsi
FilledButton(
  onPressed: () {
    debugPrint('Tombol Simpan ditekan');
  },
  child: const Text('Simpan'),
),
```

```
// Tombol NONAKTIF (disabled): onPressed bernilai null
const FilledButton(
  onPressed: null,
  child: Text('Simpan'),
),
```



Atas: tombol aktif. Bawah: tombol dengan onPressed null (disabled)

Contoh lengkap: galeri tombol Material 3

Program berikut menampilkan semua jenis tombol, menghitung jumlah klik, dan punya satu tombol untuk menonaktifkan semua tombol lain sekaligus. Salin ke `lib/main.dart` lalu jalankan.

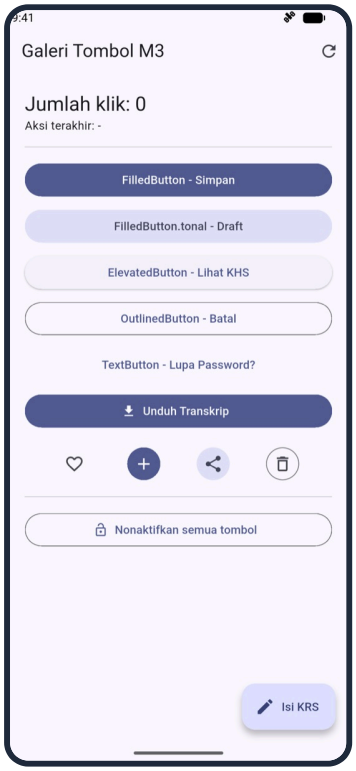
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const GaleriTombolScreen(),
    );
  }
```

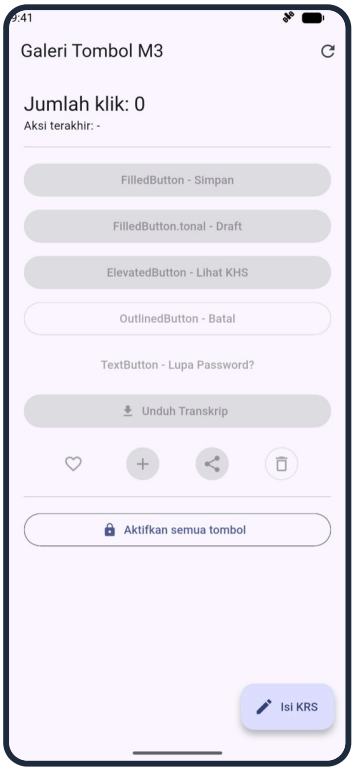
```dart
}

class GaleriTombolScreen extends StatefulWidget {
  const GaleriTombolScreen({super.key});

  @override
  State<GaleriTombolScreen> createState() => _GaleriTombolScreenState();
}

class _GaleriTombolScreenState extends State<GaleriTombolScreen> {
  int _jumlahKlik = 0;
  bool _tombolAktif = true;
  String _aksiTerakhir = '-';

  void _catat(String aksi) {
    setState(() {
      _jumlahKlik++;
      _aksiTerakhir = aksi;
    });
  }

  @override
  Widget build(BuildContext context) {
    // Jika _tombolAktif false, semua callback menjadi null -> tombol disabled
    final VoidCallback? aksiSimpan = _tombolAktif ? () => _catat('Simpan') : null;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Galeri Tombol M3'),
        actions: [
          IconButton(
            tooltip: 'Reset',
            icon: const Icon(Icons.refresh),
            onPressed: () => setState(() {
              _jumlahKlik = 0;
              _aksiTerakhir = '-';
            }),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          Text('Jumlah klik: $_jumlahKlik', style: Theme.of(context).textTheme.headlineSmall),
          Text('Aksi terakhir: $_aksiTerakhir'),
          const Divider(height: 32),

          // 1. Prioritas tertinggi
          FilledButton(onPressed: aksiSimpan, child: const Text('FilledButton - Simpan')),
          const SizedBox(height: 8),
          FilledButton.tonal(
            onPressed: _tombolAktif ? () => _catat('Tonal') : null,
            child: const Text('FilledButton.tonal - Draft'),
          ),
          const SizedBox(height: 8),

          // 2. Tombol dengan bayangan (elevasi)
          ElevatedButton(
            onPressed: _tombolAktif ? () => _catat('Elevated') : null,
```

```
      child: const Text('ElevatedButton - Lihat KHS'),
    ),
    const SizedBox(height: 8),

    // 3. Aksi sekunder
    OutlinedButton(
      onPressed: _tombolAktif ? () => _catat('Batal') : null,
      child: const Text('OutlinedButton - Batal'),
    ),
    const SizedBox(height: 8),

    // 4. Aksi prioritas rendah
    TextButton(
      onPressed: _tombolAktif ? () => _catat('Lupa Password') : null,
      child: const Text('TextButton - Lupa Password?'),
    ),
    const SizedBox(height: 8),

    // 5. Versi .icon: ikon + label
    FilledButton.icon(
      onPressed: _tombolAktif ? () => _catat('Unduh') : null,
      icon: const Icon(Icons.download),
      label: const Text('Unduh Transkrip'),
    ),
    const SizedBox(height: 16),

    // 6. IconButton dan variannya
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        IconButton(
          tooltip: 'Favorit',
          onPressed: _tombolAktif ? () => _catat('Favorit') : null,
          icon: const Icon(Icons.favorite_border),
        ),
        IconButton.filled(
          tooltip: 'Tambah',
          onPressed: _tombolAktif ? () => _catat('Tambah') : null,
          icon: const Icon(Icons.add),
        ),
        IconButton.filledTonal(
          tooltip: 'Bagikan',
          onPressed: _tombolAktif ? () => _catat('Bagikan') : null,
          icon: const Icon(Icons.share),
        ),
        IconButton.outlined(
          tooltip: 'Hapus',
          onPressed: _tombolAktif ? () => _catat('Hapus') : null,
          icon: const Icon(Icons.delete_outline),
        ),
      ],
    ),
    const Divider(height: 32),

    // Saklar untuk mengaktifkan / menonaktifkan semua tombol
    OutlinedButton.icon(
      onPressed: () => setState(() => _tombolAktif = !_tombolAktif),
      icon: Icon(_tombolAktif ? Icons.lock_open : Icons.lock),
      label: Text(_tombolAktif ? 'Nonaktifkan semua tombol' : 'Aktifkan semua tombol'),
```

```
          ),
          const SizedBox(height: 80), // ruang agar FAB tidak menutupi tombol terakhir
        ],
      ),
      // 7. FloatingActionButton: aksi utama layar
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _catat('FAB'),
        icon: const Icon(Icons.edit),
        label: const Text('Isi KRS'),
      ),
    );
  }
}
```



Galeri tombol saat semua tombol aktif



Setelah menekan "Nonaktifkan semua tombol"

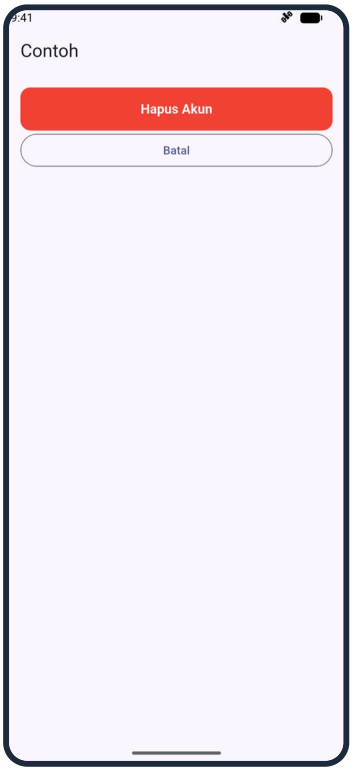
Penjelasan kode:

- `final VoidCallback? aksiSimpan = _tombolAktif ? () => _catat('Simpan') : null;` adalah pola paling umum untuk mengaktifkan atau menonaktifkan tombol berdasarkan kondisi. `VoidCallback?` berarti "fungsi tanpa parameter, boleh null".
- `FilledButton.icon`, `OutlinedButton.icon`, dan sejenisnya menerima `icon` dan `label` sebagai pengganti `child`.
- `IconButton.filled`, `.filledTonal`, dan `.outlined` adalah varian IconButton khas M3. Selalu isi `tooltip` agar pembaca layar (aksesibilitas) tahu fungsi ikon tersebut.
- `FloatingActionButton.extended` adalah FAB dengan ikon dan teks. Karena FAB menutupi sebagian konten, kita beri `SizedBox(height: 80)` di akhir daftar.

Mengubah gaya tombol

Setiap jenis tombol punya fungsi statis `styleFrom()` untuk mengubah warna, ukuran, dan bentuk. Untuk tombol selebar layar, cara paling mudah adalah `minimumSize: Size.fromHeight(...)` atau membungkus tombol dengan `SizedBox(width: double.infinity)`.

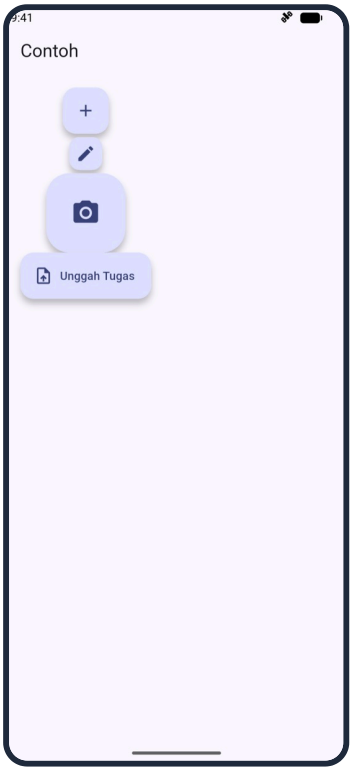
```
// Mengubah gaya satu tombol dengan styleFrom
FilledButton(
  style: FilledButton.styleFrom(
    backgroundColor: Colors.red,          // warna latar
    foregroundColor: Colors.white,        // warna teks & ikon
    minimumSize: const Size.fromHeight(52), // tinggi 52, lebar penuh
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
  ),
  onPressed: () {},
  child: const Text('Hapus Akun'),
),

// Tombol selebar layar tanpa styleFrom: bungkus dengan SizedBox
SizedBox(
  width: double.infinity,
  child: OutlinedButton(onPressed: () {}, child: const Text('Batal')),
),
```



Tombol merah selebar layar dengan styleFrom dan OutlinedButton selebar layar

Ragam FloatingActionButton

```dart
// FAB standar (ikon saja)
FloatingActionButton(
  onPressed: () {},
  tooltip: 'Tambah catatan',
  child: const Icon(Icons.add),
),

// FAB kecil dan besar
FloatingActionButton.small(onPressed: () {}, child: const Icon(Icons.edit)),
FloatingActionButton.large(onPressed: () {}, child: const Icon(Icons.camera_alt)),

// FAB dengan label teks
FloatingActionButton.extended(
  onPressed: () {},
  icon: const Icon(Icons.upload_file),
  label: const Text('Unggah Tugas'),
),
```



FAB standar, small, large, dan extended

> **ℹ️ FAB biasanya hanya satu**
>
> Contoh di atas sengaja menampilkan beberapa FAB untuk perbandingan. Dalam aplikasi nyata, satu layar sebaiknya hanya punya **satu** FAB yang dipasang di parameter `floatingActionButton` milik `Scaffold`. Jika terpaksa memakai beberapa FAB di satu layar, beri masing-masing `heroTag` yang berbeda agar tidak terjadi error saat pindah halaman.

⚠️ **Kesalahan umum**

- **Menulis `onPressed: _simpan()` (dengan tanda kurung).** Ini MEMANGGIL fungsi saat build, bukan saat ditekan. Tulis `onPressed: _simpan` atau `onPressed: () => _simpan()`.
- **Bingung kenapa tombol abu-abu dan tidak bisa ditekan.** Periksa apakah `onPressed` bernilai `null`.
- **Memakai banyak `FilledButton` dalam satu layar.** Pengguna jadi bingung mana aksi utama. Pakai Outlined atau Text untuk aksi pendukung.
- **Lupa `tooltip` pada `IconButton`.** Ikon saja sering tidak jelas maknanya.

2. GestureDetector & InkWell

Tidak semua yang bisa ditekan harus berbentuk tombol. Kartu mata kuliah, foto profil, atau kotak menu juga bisa dibuat responsif terhadap sentuhan menggunakan `GestureDetector` atau `InkWell`.

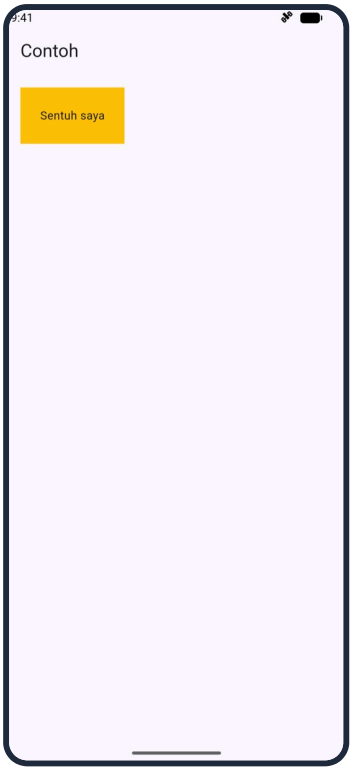
Konsep

	`GestureDetector`	`InkWell`
Efek visual saat ditekan	Tidak ada	Riak air (*ripple*) khas Material
Jenis gestur	Sangat lengkap: tap, double tap, long press, drag, pan, scale	Tap, double tap, long press, hover
Syarat	Tidak ada	Harus ada widget `Material` (atau `Card`, `Scaffold`) di atasnya
Cocok untuk	Gestur khusus (geser, cubit), area tanpa efek	Kartu atau baris yang bisa diklik

Contoh kecil: GestureDetector

Bungkus widget apa pun dengan `GestureDetector` lalu isi callback gestur yang ingin ditangkap.

```
GestureDetector(
  onTap: () => debugPrint('Tap sekali'),
  onDoubleTap: () => debugPrint('Tap dua kali'),
  onLongPress: () => debugPrint('Tekan lama'),
  child: Container(
    padding: const EdgeInsets.all(24),
    color: Colors.amber,
    child: const Text('Sentuh saya'),
  ),
),
```

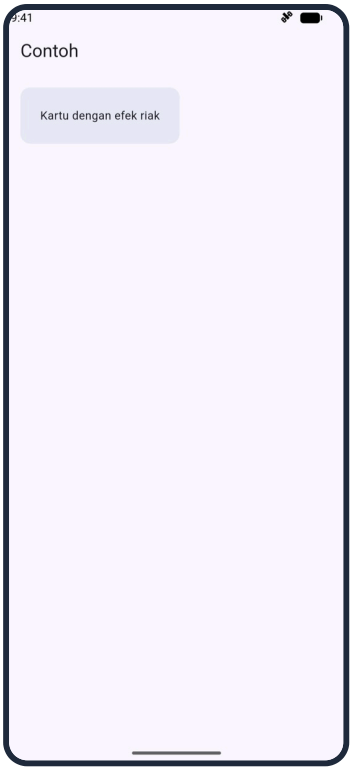


Kotak kuning yang bisa disentuh. Tidak ada efek visual apa pun saat ditekan

Contoh kecil: InkWell

InkWell menggambar efek riak di atas widget `Material` terdekat. Karena itu warna latar sebaiknya diberikan ke `Material`, **bukan** ke `Container` di dalam InkWell (Container berwarna akan menutupi riaknya).

```
// Material memberi "kanvas" tempat efek riak digambar
Material(
  color: Colors.indigo.shade50,
  borderRadius: BorderRadius.circular(12),
  child: InkWell(
    borderRadius: BorderRadius.circular(12), // riak ikut melengkung
    onTap: () => debugPrint('Kartu ditekan'),
    child: const Padding(
      padding: EdgeInsets.all(24),
      child: Text('Kartu dengan efek riak'),
    ),
  ),
),
```



Kartu dengan efek riak saat ditekan

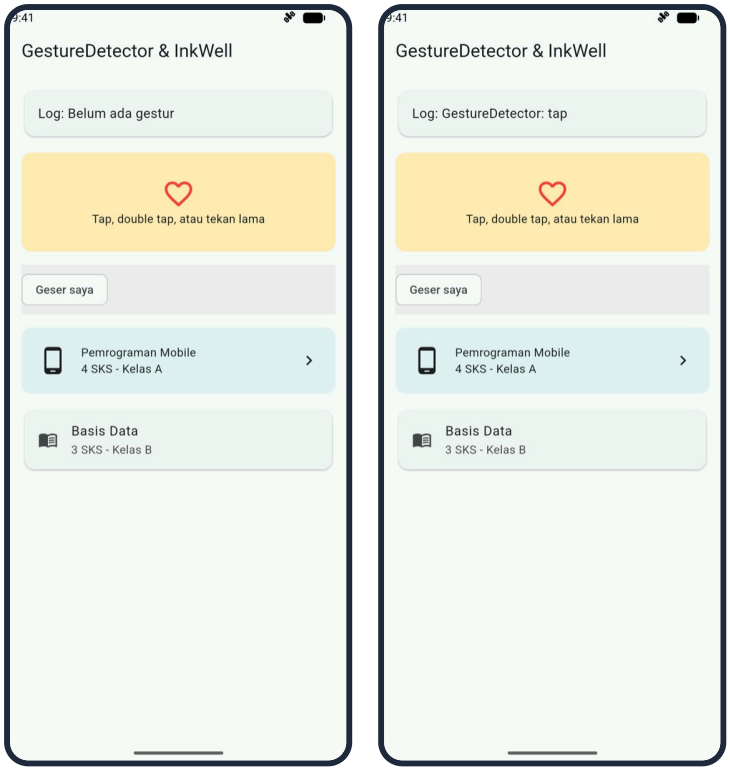
Contoh lengkap: berbagai gestur dalam satu layar

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const GestureDemoScreen(),
    );
  }
}

class GestureDemoScreen extends StatefulWidget {
  const GestureDemoScreen({super.key});

  @override
  State<GestureDemoScreen> createState() => _GestureDemoScreenState();
}

class _GestureDemoScreenState extends State<GestureDemoScreen> {
  String _log = 'Belum ada gestur';
```

```dart
  bool _disukai = false;
  double _geserX = 0;

  void _tulisLog(String pesan) => setState(() => _log = pesan);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('GestureDetector & InkWell')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Card(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Text('Log: $_log', style: const TextStyle(fontSize: 16)),
            ),
          ),
          const SizedBox(height: 16),

          // 1. GestureDetector: banyak jenis gestur, TANPA efek visual
          GestureDetector(
            onTap: () => _tulisLog('GestureDetector: tap'),
            onDoubleTap: () {
              setState(() => _disukai = !_disukai);
              _tulisLog('GestureDetector: double tap -> suka = $_disukai');
            },
            onLongPress: () => _tulisLog('GestureDetector: long press'),
            child: Container(
              height: 120,
              decoration: BoxDecoration(
                color: Colors.amber.shade100,
                borderRadius: BorderRadius.circular(12),
              ),
              alignment: Alignment.center,
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Icon(
                    _disukai ? Icons.favorite : Icons.favorite_border,
                    color: Colors.red,
                    size: 40,
                  ),
                  const Text('Tap, double tap, atau tekan lama'),
                ],
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 2. GestureDetector untuk drag horizontal
          GestureDetector(
            onHorizontalDragUpdate: (details) {
              setState(() => _geserX = (_geserX + details.delta.dx).clamp(0, 200));
            },
            onHorizontalDragEnd: (_) => _tulisLog('Geser selesai di x =
${_geserX.toStringAsFixed(0)}'),
            child: Container(
              height: 60,
```

```
              color: Colors.grey.shade200,
              alignment: Alignment.centerLeft,
              child: Transform.translate(
                offset: Offset(_geserX, 0),
                child: const Chip(label: Text('Geser saya')),
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 3. InkWell: efek riak Material saat ditekan
          Material(
            color: Colors.teal.shade50,
            borderRadius: BorderRadius.circular(12),
            child: InkWell(
              borderRadius: BorderRadius.circular(12),
              onTap: () => _tulisLog('InkWell: kartu Pemrograman Mobile ditekan'),
              onLongPress: () => _tulisLog('InkWell: long press'),
              child: const Padding(
                padding: EdgeInsets.all(20),
                child: Row(
                  children: [
                    Icon(Icons.phone_android, size: 36),
                    SizedBox(width: 16),
                    Expanded(
                      child: Text('Pemrograman Mobile\n4 SKS - Kelas A'),
                    ),
                    Icon(Icons.chevron_right),
                  ],
                ),
              ),
            ),
          ),
          const SizedBox(height: 16),

          // 4. Cara paling ringkas: Card + InkWell (Card sudah berupa Material)
          Card(
            clipBehavior: Clip.antiAlias, // agar riak tidak keluar dari sudut kartu
            child: InkWell(
              onTap: () => _tulisLog('InkWell di dalam Card ditekan'),
              child: const ListTile(
                leading: Icon(Icons.menu_book),
                title: Text('Basis Data'),
                subtitle: Text('3 SKS - Kelas B'),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



Tampilan awal

Setelah kotak kuning diketuk: log berubah

Penjelasan kode:

- **Kotak kuning** memakai `GestureDetector` dengan tiga callback. Double tap membalik nilai `_disukai` sehingga ikon hati berganti.
- **Chip "Geser saya"** memakai `onHorizontalDragUpdate`. `details.delta.dx` adalah jarak geser jari sejak frame sebelumnya; nilainya dijumlahkan lalu dibatasi dengan `clamp(0, 200)`.
- **Kartu Pemrograman Mobile** memakai pola `Material` + `InkWell`. Nilai `borderRadius` diberikan ke keduanya agar riak ikut melengkung.
- **Kartu Basis Data** memakai `Card`, yang sudah berupa Material. `clipBehavior: Clip.antiAlias` memotong riak agar tidak keluar dari sudut kartu.

> 💡 **Tips**
>
> Untuk baris daftar, `ListTile` sudah punya `onTap` dan efek riak sendiri, jadi tidak perlu dibungkus InkWell. Untuk tombol, gunakan widget tombol bawaan. InkWell paling berguna untuk kartu atau tata letak kustom.

⚠️ **Kesalahan umum**

- **Riak InkWell tidak terlihat.** Biasanya karena tidak ada `Material` di atasnya, atau anak InkWell adalah `Container` berwarna yang menutupi riak.
- **Area kosong di GestureDetector tidak bisa ditekan.** Secara bawaan hanya bagian yang "terlihat" yang menerima sentuhan. Tambahkan `behavior: HitTestBehavior.opaque` jika seluruh area harus responsif.
- **Memasang GestureDetector di atas tombol.** Tombol sudah menangani sentuhannya sendiri, sehingga kedua penangan bisa saling berebut.

3. TextField & InputDecoration

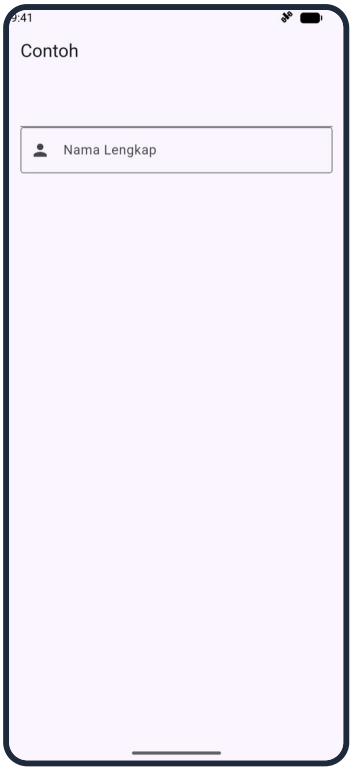
`TextField` adalah kotak tempat pengguna mengetik teks. Tampilannya (label, ikon, garis tepi, pesan error) diatur oleh satu objek bernama `InputDecoration`.

Konsep

- `TextField` mengurus *perilaku*: menerima ketikan, menampilkan kursor, membuka keyboard.
- `InputDecoration` mengurus *tampilan*: `labelText`, `hintText`, `prefixIcon`, `suffixIcon`, `helperText`, `errorText`, `border`, dan lain-lain.
- Untuk membaca isi teks, cara paling sederhana adalah `onChanged`: fungsi yang dipanggil setiap kali isi berubah. Cara yang lebih lengkap (controller) dibahas di section berikutnya.

Contoh kecil

```
// Versi paling sederhana
const TextField(),

// Dengan label, petunjuk, dan ikon
TextField(
  decoration: const InputDecoration(
    labelText: 'Nama Lengkap',        // label yang naik ke atas saat fokus
    hintText: 'Contoh: Nadia Putri',  // teks samar saat kosong
    prefixIcon: Icon(Icons.person),   // ikon di kiri
    border: OutlineInputBorder(),     // kotak bergaris
  ),
  onChanged: (teks) => debugPrint('Diketik: $teks'),
),
```



TextField polos (atas) dan TextField dengan label, hint, ikon, serta kotak garis

Properti InputDecoration	Fungsi
`labelText`	Label di dalam kotak, naik ke atas saat field fokus atau berisi
`hintText`	Contoh isian yang tampil samar saat kosong
`helperText`	Teks bantuan kecil di bawah field
`errorText`	Pesan error merah di bawah field (garis ikut merah)
`prefixIcon` / `suffixIcon`	Ikon di kiri / kanan
`prefixText` / `suffixText`	Teks tetap di kiri / kanan, misalnya "Rp" atau "kg"
`border`	`UnderlineInputBorder()` (bawaan) atau `OutlineInputBorder()`
`filled` + `fillColor`	Memberi warna latar pada field

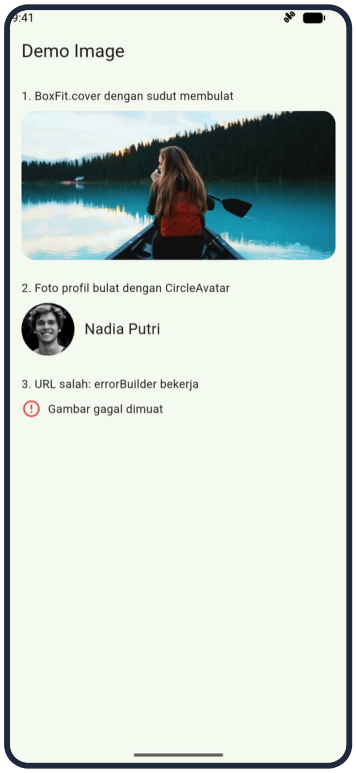
Contoh lengkap: katalog gaya TextField

`lib/main.dart`

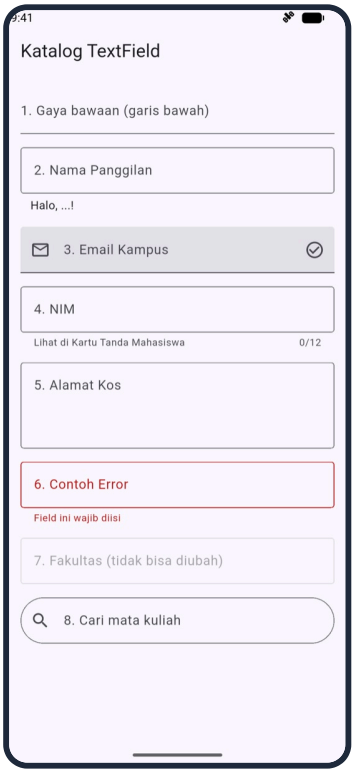
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KatalogTextFieldScreen(),
    );
  }
}

class KatalogTextFieldScreen extends StatefulWidget {
  const KatalogTextFieldScreen({super.key});

  @override
  State<KatalogTextFieldScreen> createState() => _KatalogTextFieldScreenState();
}

class _KatalogTextFieldScreenState extends State<KatalogTextFieldScreen> {
  String _nama = '';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Katalog TextField')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Gaya garis bawah (bawaan)
          const TextField(
            decoration: InputDecoration(labelText: '1. Gaya bawaan (garis bawah)'),
          ),
          const SizedBox(height: 16),

          // 2. Gaya kotak + onChanged untuk membaca teks secara langsung
          TextField(
            decoration: const InputDecoration(
              labelText: '2. Nama Panggilan',
              hintText: 'Ketik nama panggilanmu',
              border: OutlineInputBorder(),
            ),
            onChanged: (value) => setState(() => _nama = value),
          ),
          Padding(
            padding: const EdgeInsets.only(top: 4, left: 12),
```

```
        child: Text('Halo, ${_nama.isEmpty ? '...' : _nama}!'),
      ),
      const SizedBox(height: 16),

      // 3. Latar terisi (filled) + ikon depan dan belakang
      const TextField(
        decoration: InputDecoration(
          labelText: '3. Email Kampus',
          prefixIcon: Icon(Icons.email_outlined),
          suffixIcon: Icon(Icons.check_circle_outline),
          filled: true,
        ),
      ),
      const SizedBox(height: 16),

      // 4. Teks bantuan + batas karakter (counter otomatis muncul)
      const TextField(
        maxLength: 12,
        decoration: InputDecoration(
          labelText: '4. NIM',
          helperText: 'Lihat di Kartu Tanda Mahasiswa',
          prefixText: 'H1D',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

      // 5. Multi-baris untuk teks panjang
      const TextField(
        minLines: 3,
        maxLines: 5,
        decoration: InputDecoration(
          labelText: '5. Alamat Kos',
          alignLabelWithHint: true,
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

      // 6. Tampilan error manual (tanpa Form)
      const TextField(
        decoration: InputDecoration(
          labelText: '6. Contoh Error',
          errorText: 'Field ini wajib diisi',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),

      // 7. Nonaktif dan hanya-baca
      const TextField(
        enabled: false,
        decoration: InputDecoration(
          labelText: '7. Fakultas (tidak bisa diubah)',
          hintText: 'Fakultas Teknik',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),
```

```
          // 8. Sudut membulat + warna garis saat fokus
          TextField(
            decoration: InputDecoration(
              labelText: '8. Cari mata kuliah',
              prefixIcon: const Icon(Icons.search),
              border: OutlineInputBorder(borderRadius: BorderRadius.circular(30)),
              focusedBorder: OutlineInputBorder(
                borderRadius: BorderRadius.circular(30),
                borderSide: const BorderSide(color: Colors.orange, width: 2),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```



Bagian atas katalog TextField



Setelah mengetik "Budi" di field nomor 2

Penjelasan kode:

- **Field 2** memakai `onChanged` untuk menyimpan teks ke `_nama` lalu memanggil `setState`, sehingga sapaan di bawahnya ikut berubah setiap huruf diketik.
- **Field 4** memakai `maxLength: 12`. Flutter otomatis menampilkan penghitung "0/12" dan menolak huruf ke-13.
- **Field 5** memakai `minLines` dan `maxLines` sehingga kotak setinggi 3 baris dan bisa membesar sampai 5 baris. `alignLabelWithHint: true` membuat label sejajar di atas, bukan di tengah.

- **Field 6** menampilkan error secara manual dengan `errorText`. Pada form sungguhan, error lebih mudah diatur dengan `Form` dan `validator` (section 6).
- **Field 8** membedakan garis saat normal (`border`) dan saat fokus (`focusedBorder`).

> 💡 **Tema global**
>
> Agar semua TextField di aplikasi otomatis memakai kotak garis, atur sekali di tema: `ThemeData(inputDecorationTheme: const InputDecorationThemeData(border: OutlineInputBorder()))`. Dengan begitu kita tidak perlu menulis `border:` di setiap field.

> ⚠️ **Kesalahan umum**
>
> - **Error "InputDecorator ... unbounded width".** Terjadi saat TextField diletakkan langsung di dalam `Row`. Bungkus dengan `Expanded` atau beri lebar dengan `SizedBox`.
> - **Error "No Material widget found".** TextField harus berada di bawah `Scaffold` atau `Material`.
> - **Keyboard menutupi field di bagian bawah layar.** Pakai `ListView` atau `SingleChildScrollView` sebagai pembungkus agar halaman bisa digulir.

4. TextEditingController: Baca, Ubah, Kosongkan

`onChanged` cukup untuk membaca teks. Namun sering kita juga perlu **mengubah** isi field dari kode, **mengosongkan** field setelah tombol Simpan ditekan, atau **memberi nilai awal**. Untuk itu Flutter menyediakan `TextEditingController`.

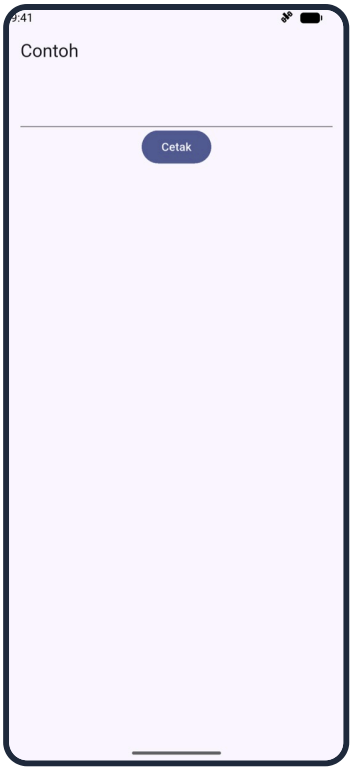
Konsep: tiga langkah wajib

1. **Buat** controller sebagai field di dalam `State` (bukan di dalam `build`).
2. **Hubungkan** ke TextField lewat parameter `controller:`.
3. **Buang** (`dispose()`) controller di method `dispose()` milik State.

Setelah terhubung, kita bisa memakai `controller.text` (membaca atau mengubah isi), `controller.clear()` (mengosongkan), dan `controller.addListener(...)` (dipanggil setiap isi berubah).

Contoh kecil

```
class ContohController extends StatefulWidget {
  const ContohController({super.key});

  @override
  State<ContohController> createState() => _ContohControllerState();
}

class _ContohControllerState extends State<ContohController> {
  // 1. Buat controller sebagai field di State
  final _namaController = TextEditingController();

  // 3. Wajib dibuang saat widget dihapus dari layar
  @override
  void dispose() {
    _namaController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        // 2. Hubungkan controller ke TextField
        TextField(controller: _namaController),
        FilledButton(
          // Baca isi TextField kapan saja lewat .text
          onPressed: () => debugPrint('Nama: ${_namaController.text}'),
          child: const Text('Cetak'),
        ),
      ],
    );
  }
}
```



TextField terhubung controller dan tombol Cetak

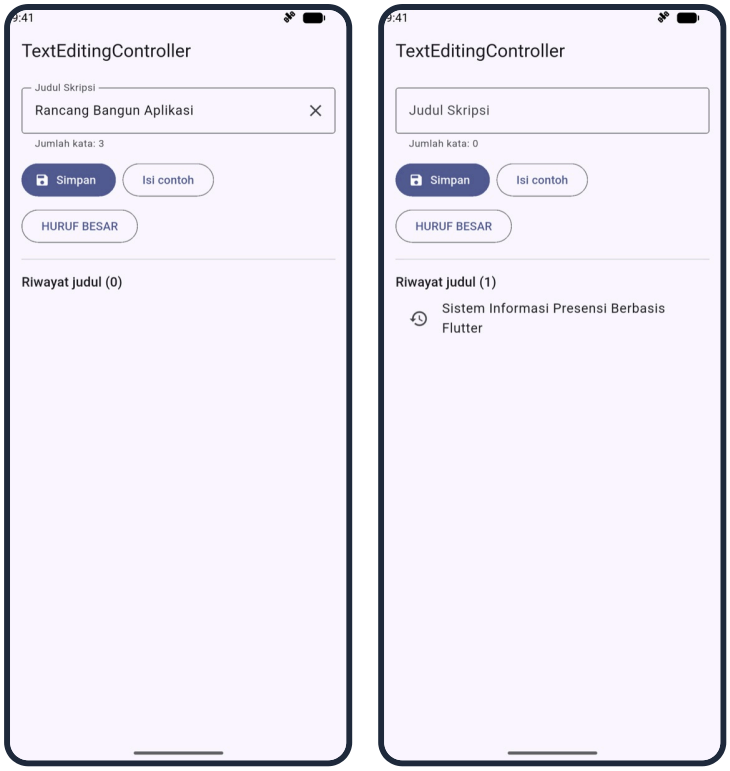
Contoh lengkap: membaca, mengubah, mengosongkan, dan mendengarkan

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ControllerDemoScreen(),
    );
  }
}

class ControllerDemoScreen extends StatefulWidget {
  const ControllerDemoScreen({super.key});

  @override
  State<ControllerDemoScreen> createState() => _ControllerDemoScreenState();
}

class _ControllerDemoScreenState extends State<ControllerDemoScreen> {
```

```dart
  // Nilai awal bisa diisi lewat parameter text
  final _judulController = TextEditingController(text: 'Rancang Bangun Aplikasi');
  final List<String> _riwayat = [];
  int _jumlahKata = 0;

  @override
  void initState() {
    super.initState();
    _jumlahKata = _hitung(_judulController.text);
    // Listener dipanggil SETIAP kali teks (atau posisi kursor) berubah
    _judulController.addListener(_hitungKata);
  }

  int _hitung(String teks) {
    final bersih = teks.trim();
    return bersih.isEmpty ? 0 : bersih.split(RegExp(r'\s+')).length;
  }

  void _hitungKata() {
    setState(() => _jumlahKata = _hitung(_judulController.text));
  }

  @override
  void dispose() {
    _judulController.removeListener(_hitungKata);
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _judulController.text.trim();
    if (judul.isEmpty) return;
    setState(() => _riwayat.insert(0, judul));
    _judulController.clear(); // mengosongkan TextField
  }

  void _isiContoh() {
    // Mengubah isi TextField dari kode
    _judulController.text = 'Sistem Informasi Presensi Berbasis Flutter';
  }

  void _hurufBesar() {
    _judulController.text = _judulController.text.toUpperCase();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('TextEditingController')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            TextField(
              controller: _judulController,
              decoration: InputDecoration(
                labelText: 'Judul Skripsi',
                border: const OutlineInputBorder(),
```

```
                helperText: 'Jumlah kata: $_jumlahKata',
                // Tombol hapus hanya muncul jika ada teks
                suffixIcon: _judulController.text.isEmpty
                    ? null
                    : IconButton(
                        icon: const Icon(Icons.clear),
                        onPressed: _judulController.clear,
                      ),
              ),
            ),
            const SizedBox(height: 12),
            Wrap(
              spacing: 8,
              runSpacing: 8,
              children: [
                FilledButton.icon(
                  onPressed: _simpan,
                  icon: const Icon(Icons.save),
                  label: const Text('Simpan'),
                ),
                OutlinedButton(onPressed: _isiContoh, child: const Text('Isi contoh')),
                OutlinedButton(onPressed: _hurufBesar, child: const Text('HURUF BESAR')),
              ],
            ),
            const Divider(height: 32),
            Text('Riwayat judul (${_riwayat.length})',
                style: Theme.of(context).textTheme.titleMedium),
            Expanded(
              child: ListView(
                children: [
                  for (final judul in _riwayat)
                    ListTile(leading: const Icon(Icons.history), title: Text(judul)),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```



Nilai awal dari TextEditingController(text: ...) dan jumlah kata

Setelah Isi contoh, Simpan, lalu field kosong kembali

Penjelasan kode:

- `TextEditingController(text: '...')` memberi isi awal pada field.
- `addListener(_hitungKata)` di `initState` membuat `_hitungKata` dipanggil setiap teks berubah, baik karena diketik maupun diubah dari kode. Inilah bedanya dengan `onChanged`, yang hanya terpanggil saat pengguna mengetik.
- `_judulController.text = '...'` mengganti isi field dari kode (tombol "Isi contoh" dan "HURUF BESAR").
- `_judulController.clear()` mengosongkan field setelah disimpan. Tombol silang di `suffixIcon` juga memanggil `clear`.
- Di `dispose()` kita melepas listener lalu membuang controller. Urutannya: bersihkan milik kita dulu, `super.dispose()` paling akhir.

	`onChanged`	`TextEditingController`
Membaca teks	Ya, lewat parameter callback	Ya, kapan saja lewat `.text`
Mengubah / mengosongkan dari kode	Tidak bisa	Bisa
Perlu `dispose`	Tidak	Ya, wajib
Cocok untuk	Pencarian sederhana, preview langsung	Form yang dikirim, nilai awal, tombol hapus

⚠️ **Kesalahan umum**

- **Membuat controller di dalam `build()`.** Setiap kali `setState` dipanggil, controller baru dibuat sehingga teks yang diketik hilang. Selalu jadikan field di State.
- **Lupa `dispose()`.** Controller tetap tersimpan di memori setelah layar ditutup (*memory leak*).
- **Memakai controller di StatelessWidget.** StatelessWidget tidak punya `dispose`, jadi controller harus berada di StatefulWidget.
- **Memanggil `setState` di `initState` sebelum layar tampil.** Cukup isi variabel langsung, seperti `_jumlahKata = _hitung(...)` pada contoh.

5. Jenis Keyboard, Password & FocusNode

Form yang nyaman dipakai di HP bukan hanya soal tampilan. Jenis keyboard yang tepat, password yang bisa diintip, dan perpindahan fokus otomatis membuat pengguna bisa mengisi form lebih cepat.

1. keyboardType: memilih jenis keyboard

Parameter `keyboardType` menentukan keyboard yang muncul. Ini tidak membatasi isi (pengguna tetap bisa menempelkan huruf), tapi sangat memudahkan pengisian.

```
TextField(
  keyboardType: TextInputType.emailAddress, // keyboard dengan tombol @
  decoration: InputDecoration(labelText: 'Email'),
),
TextField(
  keyboardType: TextInputType.number, // keyboard angka
  decoration: InputDecoration(labelText: 'IPK Semester Lalu'),
),
TextField(
  keyboardType: TextInputType.phone, // keyboard telepon
  decoration: InputDecoration(labelText: 'Nomor WhatsApp'),
),
TextField(
  keyboardType: TextInputType.multiline, // Enter = baris baru
  maxLines: null,
  decoration: InputDecoration(labelText: 'Catatan'),
),
```

Field IPK sedang fokus; field ini memakai TextInputType.number sehingga HP menampilkan keyboard angka

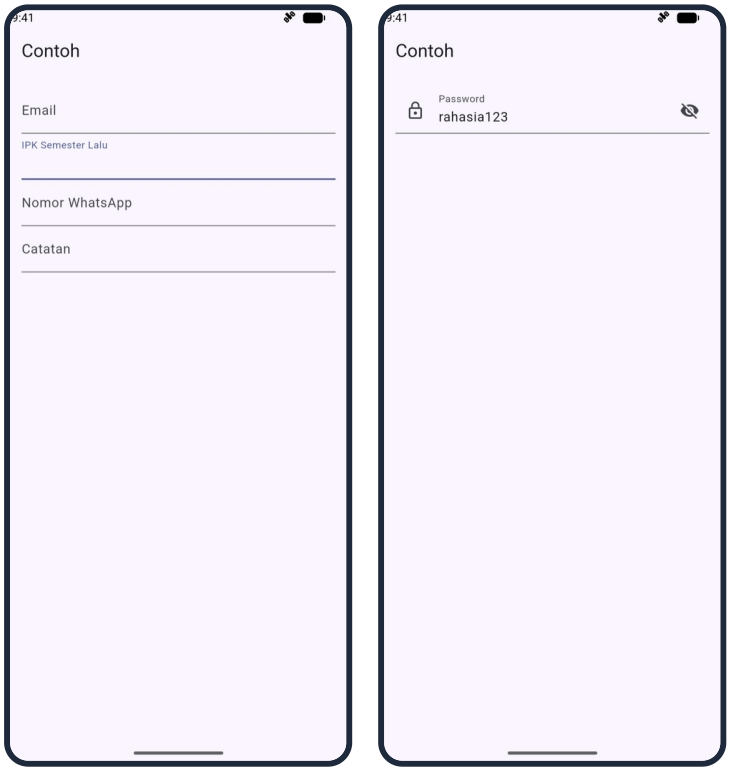
Nilai	Keyboard	Untuk field
`TextInputType.text`	Huruf (bawaan)	Nama, alamat
`TextInputType.emailAddress`	Huruf + tombol @ dan .	Email
`TextInputType.number`	Angka	NIM, umur, nilai
`TextInputType.phone`	Papan telepon	Nomor HP
`TextInputType.url`	Huruf + tombol / dan .com	Alamat website
`TextInputType.multiline`	Enter membuat baris baru	Catatan, deskripsi

2. obscureText: password dengan tombol tampilkan/sembunyikan

`obscureText: true` mengganti setiap huruf dengan titik. Agar pengguna bisa memeriksa ketikannya, kita simpan nilai `obscureText` di variabel State lalu membaliknya lewat `IconButton` di `suffixIcon`.

```
class PasswordField extends StatefulWidget {
  const PasswordField({super.key});
```

```dart
  @override
  State<PasswordField> createState() => _PasswordFieldState();
}

class _PasswordFieldState extends State<PasswordField> {
  bool _sembunyikan = true; // awalnya password disembunyikan

  @override
  Widget build(BuildContext context) {
    return TextField(
      obscureText: _sembunyikan, // true -> teks tampil sebagai titik
      enableSuggestions: false,
      autocorrect: false,
      decoration: InputDecoration(
        labelText: 'Password',
        prefixIcon: const Icon(Icons.lock_outline),
        suffixIcon: IconButton(
          tooltip: _sembunyikan ? 'Tampilkan password' : 'Sembunyikan password',
          icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
          onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
        ),
      ),
    );
  }
}
```



Password tersembunyi (titik-titik)

Setelah ikon mata ditekan, password terlihat

`enableSuggestions: false` dan `autocorrect: false` mencegah keyboard menyimpan atau mengoreksi password.

3. textInputAction dan FocusNode: berpindah antar field

- `textInputAction` mengganti tombol Enter di keyboard menjadi **Next** (`TextInputAction.next`), **Done** (`.done`), **Search** (`.search`), dan lainnya.
- `onSubmitted` dipanggil saat tombol tersebut ditekan.
- `FocusNode` adalah "remote" untuk sebuah field. `focusNode.requestFocus()` memindahkan kursor ke field itu, sedangkan `FocusScope.of(context).unfocus()` melepas fokus dan menutup keyboard. Sama seperti controller, FocusNode wajib di-`dispose`.

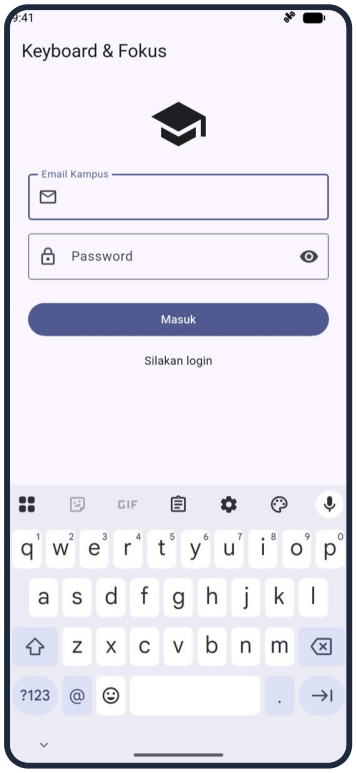
Contoh lengkap: login dengan perpindahan fokus

`lib/main.dart`

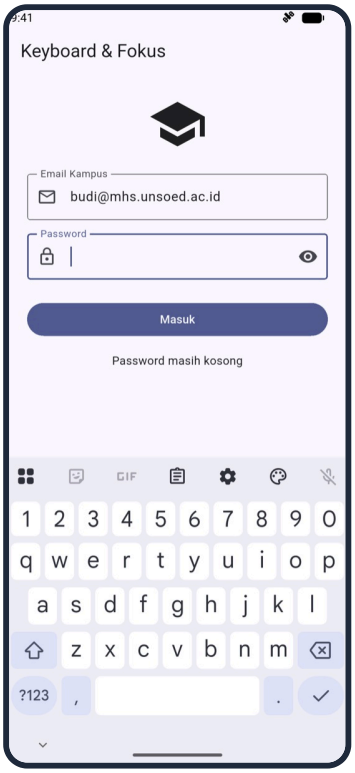
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const LoginSederhanaScreen(),
    );
  }
}

class LoginSederhanaScreen extends StatefulWidget {
  const LoginSederhanaScreen({super.key});

  @override
  State<LoginSederhanaScreen> createState() => _LoginSederhanaScreenState();
}

class _LoginSederhanaScreenState extends State<LoginSederhanaScreen> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  // FocusNode = "remote" untuk mengatur fokus sebuah TextField
  final _emailFocus = FocusNode();
  final _passwordFocus = FocusNode();

  bool _sembunyikan = true;
  String _status = 'Silakan login';

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    _emailFocus.dispose();
```

```
    _passwordFocus.dispose();
    super.dispose();
  }

  void _login() {
    // Tutup keyboard dengan melepas fokus
    FocusScope.of(context).unfocus();

    final email = _emailController.text.trim();
    final password = _passwordController.text;

    if (email.isEmpty) {
      setState(() => _status = 'Email masih kosong');
      _emailFocus.requestFocus(); // arahkan kursor ke field yang bermasalah
      return;
    }
    if (password.isEmpty) {
      setState(() => _status = 'Password masih kosong');
      _passwordFocus.requestFocus();
      return;
    }
    setState(() => _status = 'Login sebagai $email');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keyboard & Fokus')),
      // Ketuk area kosong untuk menutup keyboard
      body: GestureDetector(
        onTap: () => FocusScope.of(context).unfocus(),
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            const Icon(Icons.school, size: 72),
            const SizedBox(height: 24),
            TextField(
              controller: _emailController,
              focusNode: _emailFocus,
              autofocus: true, // langsung fokus saat layar dibuka
              keyboardType: TextInputType.emailAddress,
              textInputAction: TextInputAction.next, // tombol "Next" di keyboard
              // Saat "Next" ditekan, pindah ke field password
              onSubmitted: (_) => _passwordFocus.requestFocus(),
              decoration: const InputDecoration(
                labelText: 'Email Kampus',
                prefixIcon: Icon(Icons.email_outlined),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            TextField(
              controller: _passwordController,
              focusNode: _passwordFocus,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.done, // tombol "Done" di keyboard
              onSubmitted: (_) => _login(), // tekan Done = login
              decoration: InputDecoration(
                labelText: 'Password',
```

```
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
                  onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
                ),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _login, child: const Text('Masuk')),
            const SizedBox(height: 16),
            Text(_status, textAlign: TextAlign.center),
          ],
        ),
      ),
    );
  }
}
```



Field email langsung fokus (autofocus) dan keyboard email terbuka



Email diisi, Masuk ditekan: kursor dipindah ke Password

Penjelasan kode:

- `autofocus: true` pada email membuat keyboard langsung muncul saat layar dibuka.
- Pada email, `onSubmitted: (_) => _passwordFocus.requestFocus()` memindahkan kursor ke password ketika tombol Next ditekan.
- Pada password, `TextInputAction.done` + `onSubmitted: (_) => _login()` membuat pengguna bisa login langsung dari keyboard.
- Di `_login`, jika ada field kosong, kursor diarahkan ke field tersebut dengan `requestFocus()`.

- `GestureDetector` di luar `ListView` menutup keyboard saat area kosong diketuk.

> ⚠️ **Kesalahan umum**
>
> - **Membuat `FocusNode()` di dalam `build`.** Sama seperti controller: buat di State dan dispose.
> - **Memberi `obscureText: true` pada field multi-baris.** Flutter menolak kombinasi ini (`maxLines` harus 1).
> - **Mengandalkan `keyboardType.number` sebagai validasi.** Pengguna tetap bisa menempelkan huruf. Validasi tetap dilakukan di `validator` (section berikutnya).

6. Form, TextFormField & Validasi

Memeriksa setiap field satu per satu dengan `if` cepat menjadi berantakan. Flutter menyediakan `Form`, `TextFormField`, dan `GlobalKey<FormState>` untuk memvalidasi semua field sekaligus. Pola di section ini akan kita pakai lagi untuk layar login dan register pada Modul 13-14.

Konsep: empat bagian

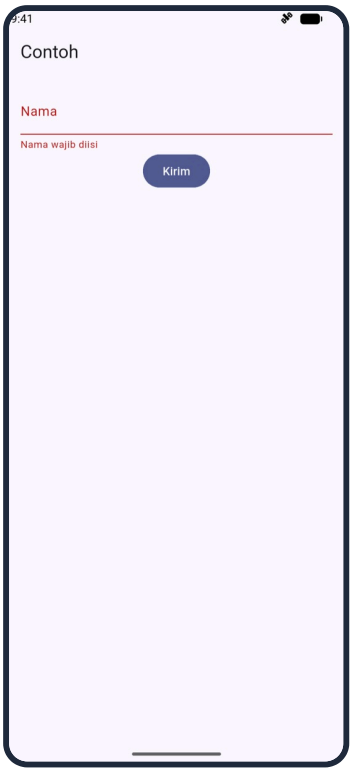
1. `GlobalKey<FormState>`: "kunci" untuk mengakses form dari luar, misalnya dari tombol Kirim.
2. `Form`: pembungkus semua field.
3. `TextFormField`: seperti TextField tetapi punya parameter `validator`.
4. `_formKey.currentState!.validate()`: menjalankan semua validator. Hasilnya `true` jika semua valid.

Aturan validator: **kembalikan `null` jika valid**, atau kembalikan `String` berisi pesan error jika tidak valid. Pesan itu otomatis tampil merah di bawah field.

Contoh kecil

```dart
// 1. Kunci untuk mengakses FormState
final _formKey = GlobalKey<FormState>();

@override
Widget build(BuildContext context) {
  // 2. Bungkus semua field dengan Form
  return Form(
    key: _formKey,
    child: Column(
      children: [
        // 3. Pakai TextFormField (bukan TextField) agar punya validator
        TextFormField(
          decoration: const InputDecoration(labelText: 'Nama'),
          validator: (value) {
            if (value == null || value.trim().isEmpty) {
              return 'Nama wajib diisi'; // String = pesan error
            }
            return null; // null = valid
          },
        ),
        FilledButton(
          onPressed: () {
            // 4. Jalankan SEMUA validator sekaligus
            if (_formKey.currentState!.validate()) {
              debugPrint('Form valid, data boleh diproses');
            }
          },
          child: const Text('Kirim'),
        ),
      ],
    ),
```

```
    );
  }
```



Setelah Kirim ditekan dengan field kosong

Mengumpulkan aturan dalam kelas Validators

Aturan yang sama (email, password) akan dipakai di banyak layar. Lebih rapi jika dikumpulkan dalam satu kelas berisi fungsi `static`. Pada proyek nyata, kelas ini bisa disimpan di `lib/utils/validators.dart`.

```dart
class Validators {
  // Pola email sederhana: teks@teks.domain
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? value, String namaField) {
    if (value == null || value.trim().isEmpty) {
      return '$namaField wajib diisi';
    }
    return null;
  }

  static String? email(String? value) {
    if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(value.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value, {int minPanjang = 8}) {
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < minPanjang) return 'Password minimal $minPanjang karakter';
    if (!RegExp(r'[0-9]').hasMatch(value)) return 'Password harus mengandung angka';
    return null;
```

```
  }

  static String? konfirmasi(String? value, String passwordAsli) {
    if (value == null || value.isEmpty) return 'Ulangi password';
    if (value != passwordAsli) return 'Konfirmasi password tidak sama';
    return null;
  }
}
```

- `RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$')` berarti: satu atau lebih huruf/angka/titik/plus/strip, lalu `@`, lalu nama domain, lalu minimal satu bagian `.xxx`. Awalan `r` membuat string "mentah" sehingga `\` tidak perlu ditulis dua kali.
- `konfirmasi` menerima dua nilai: isian konfirmasi dan password asli. Keduanya dibandingkan dengan `!=`.

Contoh lengkap: form registrasi akun

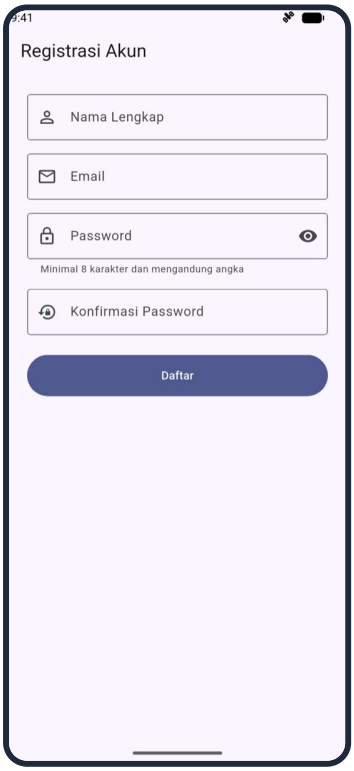
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const RegisterScreen(),
    );
  }
}

// ===== Kumpulan validator (nanti bisa dipindah ke lib/utils/validators.dart) =====
class Validators {
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? value, String namaField) {
    if (value == null || value.trim().isEmpty) return '$namaField wajib diisi';
    return null;
  }

  static String? email(String? value) {
    if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(value.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value, {int minPanjang = 8}) {
```

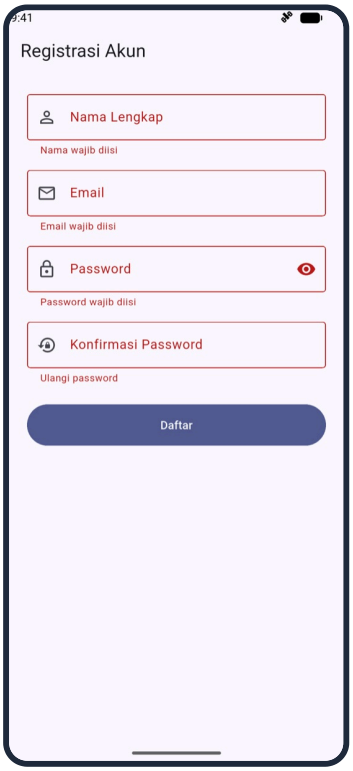
```
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < minPanjang) return 'Password minimal $minPanjang karakter';
    if (!RegExp(r'[0-9]').hasMatch(value)) return 'Password harus mengandung angka';
    return null;
  }

  static String? konfirmasi(String? value, String passwordAsli) {
    if (value == null || value.isEmpty) return 'Ulangi password';
    if (value != passwordAsli) return 'Konfirmasi password tidak sama';
    return null;
  }
}

// ===== Layar registrasi =====
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _namaController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _konfirmasiController = TextEditingController();

  bool _sembunyikanPassword = true;
  bool _sedangMengirim = false;

  @override
  void dispose() {
    _namaController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _konfirmasiController.dispose();
    super.dispose();
  }

  Future<void> _daftar() async {
    // Tutup keyboard lalu validasi semua field
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    setState(() => _sedangMengirim = true);
    // Simulasi mengirim data ke server selama 2 detik
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return; // widget bisa saja sudah ditutup selama menunggu

    setState(() => _sedangMengirim = false);
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Akun ${_emailController.text.trim()} berhasil dibuat')),
    );
    _formKey.currentState!.reset(); // kosongkan error
    _namaController.clear();
    _emailController.clear();
    _passwordController.clear();
    _konfirmasiController.clear();
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Registrasi Akun')),
      body: Form(
        key: _formKey,
        // Validasi ulang otomatis setelah pengguna mulai berinteraksi
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _namaController,
              textInputAction: TextInputAction.next,
              textCapitalization: TextCapitalization.words,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person_outline),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.wajib(v, 'Nama'),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _emailController,
              keyboardType: TextInputType.emailAddress,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Email',
                hintText: 'nama@mhs.unsoed.ac.id',
                prefixIcon: Icon(Icons.email_outlined),
                border: OutlineInputBorder(),
              ),
              validator: Validators.email,
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _passwordController,
              obscureText: _sembunyikanPassword,
              textInputAction: TextInputAction.next,
              decoration: InputDecoration(
                labelText: 'Password',
                helperText: 'Minimal 8 karakter dan mengandung angka',
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikanPassword ? Icons.visibility : Icons.visibility_off),
                  onPressed: () =>
                      setState(() => _sembunyikanPassword = !_sembunyikanPassword),
                ),
              ),
              validator: (v) => Validators.password(v),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _konfirmasiController,
              obscureText: _sembunyikanPassword,
```

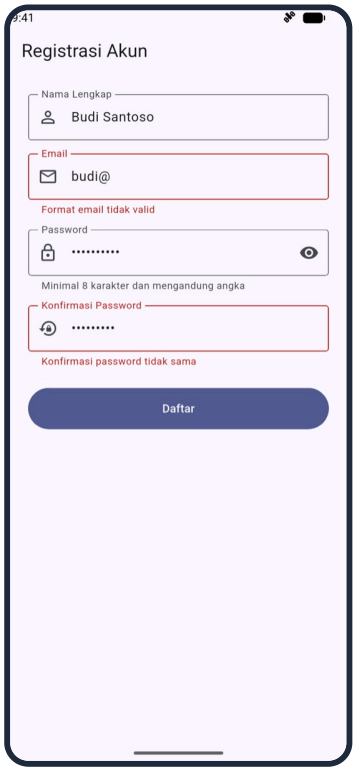
```
              textInputAction: TextInputAction.done,
              onFieldSubmitted: (_) => _daftar(),
              decoration: const InputDecoration(
                labelText: 'Konfirmasi Password',
                prefixIcon: Icon(Icons.lock_reset),
                border: OutlineInputBorder(),
              ),
              // Bandingkan dengan isi controller password
              validator: (v) => Validators.konfirmasi(v, _passwordController.text),
            ),
            const SizedBox(height: 24),
            FilledButton(
              // null saat mengirim -> tombol disabled, cegah klik ganda
              onPressed: _sedangMengirim ? null : _daftar,
              style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(50)),
              child: _sedangMengirim
                  ? const SizedBox(
                      width: 22,
                      height: 22,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Daftar'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Form registrasi sebelum diisi



Daftar ditekan saat form kosong: semua pesan error tampil



Email salah format dan konfirmasi password tidak sama

Penjelasan kode:

- `validator: Validators.email` bisa ditulis tanpa lambda karena tipe fungsinya sama persis dengan yang diminta (`String? Function(String?)`). Untuk fungsi dengan parameter tambahan, pakai lambda: `(v) => Validators.wajib(v, 'Nama')`.
- Validator konfirmasi membaca `_passwordController.text`, inilah alasan password perlu controller.
- `autovalidateMode: AutovalidateMode.onUserInteraction` membuat pesan error diperbarui sambil mengetik, tetapi hanya setelah field pernah disentuh. Pengguna tidak langsung "dimarahi" saat layar baru dibuka.
- Selama `_sedangMengirim` bernilai `true`, `onPressed` menjadi `null` (tombol nonaktif) dan isi tombol diganti `CircularProgressIndicator`. Ini mencegah data terkirim dua kali.
- `if (!mounted) return;` setelah `await` memastikan layar masih ada sebelum memakai `context` atau `setState`.
- `_formKey.currentState!.reset()` menghapus pesan error; controller dikosongkan dengan `clear()`.

Contoh lengkap kedua: form KRS dengan Dropdown dan Checkbox

Contoh ini memperlihatkan bahwa `DropdownButtonFormField` juga punya `validator` dan ikut diperiksa oleh `validate()`. Checkbox tidak punya validator, sehingga diperiksa manual dengan `if`.

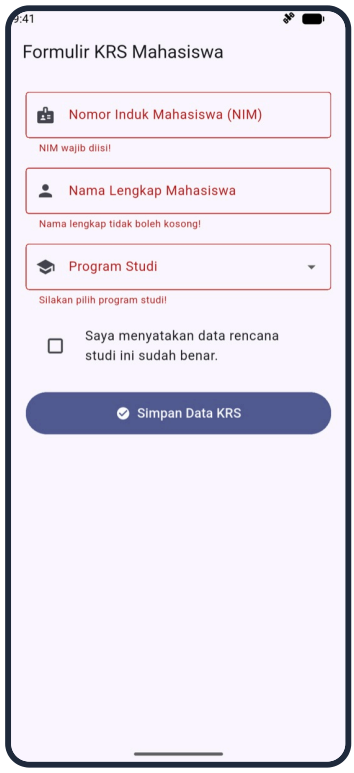
lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: Scaffold(
        appBar: AppBar(title: const Text("Formulir KRS Mahasiswa")),
        // PENTING: TextFormField, Checkbox & SnackBar butuh Scaffold/Material sebagai induk.
        // Jika FormKrsMahasiswa dipasang langsung sebagai home tanpa Scaffold, akan muncul
        // error "No Material widget found" (layar merah / kosong).
        body: const FormKrsMahasiswa(),
      ),
    );
  }
}


class FormKrsMahasiswa extends StatefulWidget {
```

```
  const FormKrsMahasiswa({super.key});

  @override
  State<FormKrsMahasiswa> createState() => _FormKrsMahasiswaState();
}

class _FormKrsMahasiswaState extends State<FormKrsMahasiswa> {
  // 1. Kunci Global untuk mengontrol dan memvalidasi FormState
  final _formKey = GlobalKey<FormState>();

  // 2. Controller untuk membaca teks dari input field
  final TextEditingController _nimController = TextEditingController();
  final TextEditingController _namaController = TextEditingController();

  String? _prodiTerpilih;
  bool _setujuSyarat = false;

  final List<String> _daftarProdi = [
    'Teknik Informatika',
    'Sistem Informasi',
    'Teknologi Rekayasa Perangkat Lunak',
    'Sains Data',
  ];

  // 3. WAJIB dispose controller agar tidak terjadi memory leak!
  @override
  void dispose() {
    _nimController.dispose();
    _namaController.dispose();
    super.dispose();
  }

  void _submitForm() {
    // Mengeksekusi seluruh fungsi validator pada setiap FormField di dalam Form
    if (_formKey.currentState!.validate()) {
      if (!_setujuSyarat) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text("Anda harus menyetujui syarat & ketentuan KRS!"),
            backgroundColor: Colors.red,
          ),
        );
        return;
      }

      // Jika lolos validasi, proses data
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text("Data Mahasiswa ${_namaController.text} (${_nimController.text}) Berhasil
Diverifikasi!"),
          backgroundColor: Colors.green,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Form(
```

```
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          // Input NIM dengan Validasi Regex
          TextFormField(
            controller: _nimController,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
              labelText: "Nomor Induk Mahasiswa (NIM)",
              hintText: "Contoh: 21102001",
              prefixIcon: Icon(Icons.badge),
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return "NIM wajib diisi!";
              }
              if (!RegExp(r'^[0-9]{8,}$').hasMatch(value.trim())) {
                return "NIM harus berupa angka minimal 8 digit!";
              }
              return null; // Nilai null berarti VALID
            },
          ),
          const SizedBox(height: 16),

          // Input Nama Lengkap
          TextFormField(
            controller: _namaController,
            decoration: const InputDecoration(
              labelText: "Nama Lengkap Mahasiswa",
              prefixIcon: Icon(Icons.person),
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return "Nama lengkap tidak boleh kosong!";
              }
              return null;
            },
          ),
          const SizedBox(height: 16),

          // Input Dropdown Pilihan Program Studi
          DropdownButtonFormField<String>(
            initialValue: _prodiTerpilih, // Flutter < 3.35: gunakan `value:`
            isExpanded: true, // Wajib agar nama prodi yang panjang tidak overflow di layar HP kecil
            decoration: const InputDecoration(
              labelText: "Program Studi",
              prefixIcon: Icon(Icons.school),
              border: OutlineInputBorder(),
            ),
            items: _daftarProdi.map((prodi) {
              return DropdownMenuItem(value: prodi, child: Text(prodi));
            }).toList(),
            onChanged: (val) => setState(() => _prodiTerpilih = val),
            validator: (val) => val == null ? "Silakan pilih program studi!" : null,
          ),
          const SizedBox(height: 16),
```

```
          // Checkbox Persetujuan
          CheckboxListTile(
            title: const Text("Saya menyatakan data rencana studi ini sudah benar."),
            value: _setujuSyarat,
            onChanged: (val) => setState(() => _setujuSyarat = val ?? false),
            controlAffinity: ListTileControlAffinity.leading,
          ),
          const SizedBox(height: 24),

          // Tombol Submit
          FilledButton.icon(
            onPressed: _submitForm,
            icon: const Icon(Icons.check_circle),
            label: const Text("Simpan Data KRS", style: TextStyle(fontSize: 16)),
            style: FilledButton.styleFrom(padding: const EdgeInsets.symmetric(vertical: 14)),
          ),
        ],
      ),
    );
  }
}
```



Form KRS setelah tombol Simpan ditekan tanpa isian

ℹ️ onSaved dan save()

Selain `validator`, TextFormField punya `onSaved` yang dipanggil oleh `_formKey.currentState!.save()`. Ini alternatif dari controller: setiap field menyimpan nilainya ke variabel saat `save()` dipanggil. Pada modul ini kita memakai controller karena lebih mudah dibaca dan bisa dipakai untuk mengosongkan field.

⚠️ Kesalahan umum

- **Validator mengembalikan `""` (string kosong) untuk input valid.** String kosong tetap dianggap error. Yang benar `return null;`.
- **Memakai `TextField` di dalam Form.** TextField tidak punya validator. Gunakan `TextFormField`.
- **Membuat `GlobalKey` di dalam `build`.** Key baru di setiap build membuat form kehilangan state. Jadikan field di State.
- **Lupa `trim()`.** Isian berupa spasi saja akan lolos pemeriksaan `isEmpty`.
- **Error "No Material widget found".** Form harus berada di dalam `Scaffold`.

7. Checkbox, Switch, Radio & Slider

Tidak semua input berupa ketikan. Untuk pilihan ya/tidak, satu dari beberapa opsi, atau nilai dalam rentang, Flutter menyediakan Checkbox, Switch, Radio, dan Slider. Semuanya bekerja dengan pola yang sama: **nilai disimpan di State, ditampilkan lewat `value`, dan diubah lewat `onChanged` + `setState`.**

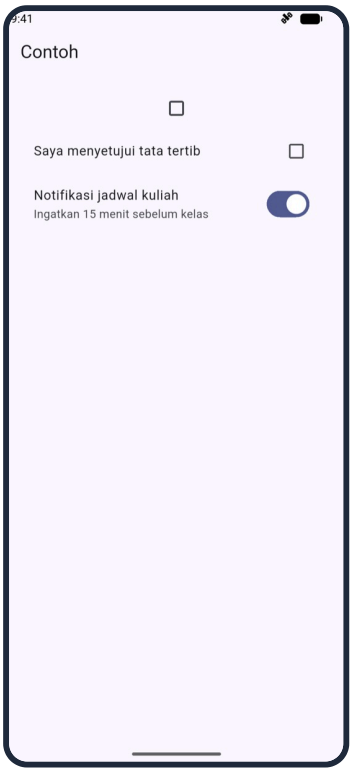
Widget	Tipe nilai	Kapan dipakai
`Checkbox` / `CheckboxListTile`	`bool`	Persetujuan, boleh memilih banyak opsi
`Switch` / `SwitchListTile`	`bool`	Pengaturan on/off yang langsung berlaku
`RadioGroup` + `RadioListTile`	Satu nilai (sering `enum`)	Pilih tepat satu dari beberapa opsi
`Slider` / `RangeSlider`	`double` / `RangeValues`	Nilai atau rentang angka

1. Checkbox dan Switch

Versi `...ListTile` sudah termasuk judul, dan seluruh barisnya bisa diketuk. Hampir selalu lebih nyaman dipakai daripada versi polos.

```
bool _setuju = false;
bool _notifikasi = true;

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      // Checkbox polos: hanya kotaknya saja
      Checkbox(
        value: _setuju,
        onChanged: (baru) => setState(() => _setuju = baru ?? false),
      ),

      // CheckboxListTile: kotak + judul, seluruh baris bisa diketuk
      CheckboxListTile(
        title: const Text('Saya menyetujui tata tertib'),
        value: _setuju,
        onChanged: (baru) => setState(() => _setuju = baru ?? false),
      ),

      // SwitchListTile: saklar on/off + judul
      SwitchListTile(
        title: const Text('Notifikasi jadwal kuliah'),
        subtitle: const Text('Ingatkan 15 menit sebelum kelas'),
```

```
        value: _notifikasi,
        onChanged: (baru) => setState(() => _notifikasi = baru),
      ),
    ],
  );
}
```



Checkbox polos, CheckboxListTile, dan SwitchListTile

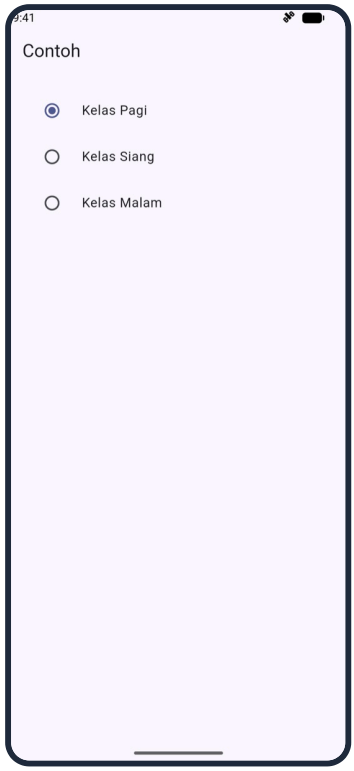
Parameter `onChanged` Checkbox menerima `bool?` (bisa null untuk mode tiga keadaan), sehingga kita menulis `baru ?? false`. Switch menerima `bool` biasa.

2. Radio dengan RadioGroup

Sejak Flutter 3.35, nilai terpilih **tidak lagi** ditulis di setiap Radio (`groupValue` dan `onChanged` di Radio sudah *deprecated*). Sekarang semua radio dibungkus satu `RadioGroup` yang memegang `groupValue` dan `onChanged`, dan setiap radio cukup diberi `value`.

```
enum Kelas { pagi, siang, malam }

class _ContohState extends State<Contoh> {
  Kelas? _kelas = Kelas.pagi;

  @override
  Widget build(BuildContext context) {
    // RadioGroup menyimpan nilai terpilih untuk SEMUA radio di dalamnya
    return RadioGroup<Kelas>(
      groupValue: _kelas,
      onChanged: (baru) => setState(() => _kelas = baru),
      child: const Column(
        children: [
          // Tiap radio cukup diberi value-nya masing-masing
```

```
          RadioListTile<Kelas>(value: Kelas.pagi, title: Text('Kelas Pagi')),
          RadioListTile<Kelas>(value: Kelas.siang, title: Text('Kelas Siang')),
          RadioListTile<Kelas>(value: Kelas.malam, title: Text('Kelas Malam')),
        ],
      ),
    );
  }
}
```



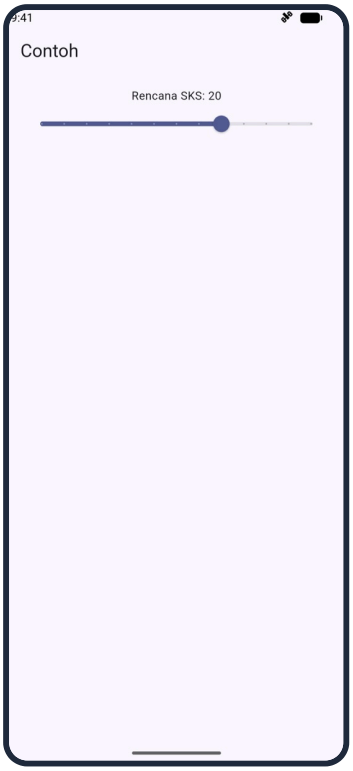
RadioGroup dengan tiga pilihan kelas

> ℹ️ **Kenapa memakai enum?**
>
> Dengan `enum Kelas { pagi, siang, malam }`, salah ketik seperti `'Pagi'` vs `'pagi'` tidak mungkin terjadi, karena compiler langsung memberi tahu jika nilai tidak dikenal. Nama enum bisa ditampilkan dengan `kelas.name`.

3. Slider

```
double _sks = 20;

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text('Rencana SKS: ${_sks.round()}'),
      Slider(
        value: _sks,
```

```
        min: 12,
        max: 24,
        divisions: 12, // 12 langkah -> nilainya bulat 12, 13, ..., 24
        label: '${_sks.round()} SKS', // gelembung saat digeser
        onChanged: (baru) => setState(() => _sks = baru),
      ),
    ],
  );
}
```



Slider rencana SKS

Tanpa `divisions`, nilai Slider berupa pecahan (misalnya 17.3826). Dengan `divisions`, nilai melompat per langkah, dan `label` menampilkan gelembung nilai saat digeser.

Contoh lengkap: halaman preferensi belajar

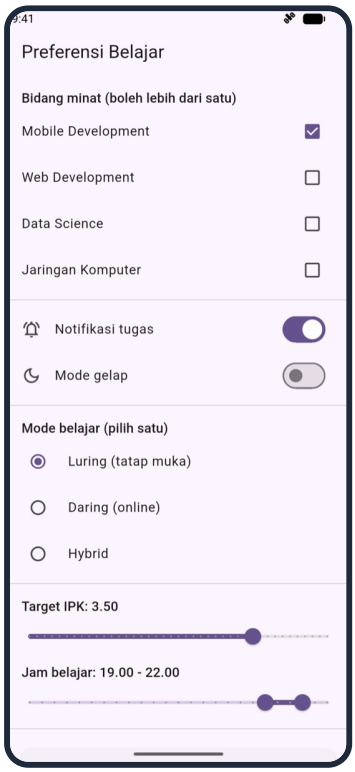
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

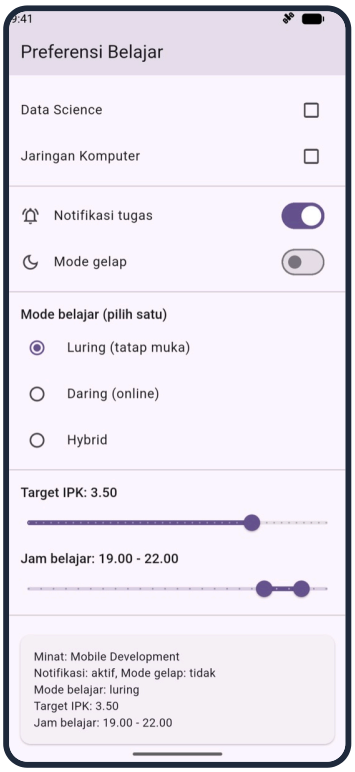
```
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const PreferensiScreen(),
    );
  }
}

enum ModeBelajar { luring, daring, hybrid }

class PreferensiScreen extends StatefulWidget {
  const PreferensiScreen({super.key});

  @override
  State<PreferensiScreen> createState() => _PreferensiScreenState();
}

class _PreferensiScreenState extends State<PreferensiScreen> {
  // Checkbox: boleh memilih lebih dari satu
  final Map<String, bool> _minat = {
    'Mobile Development': true,
    'Web Development': false,
    'Data Science': false,
    'Jaringan Komputer': false,
  };

  // Switch: on/off
  bool _notifikasi = true;
  bool _modeGelap = false;

  // Radio: hanya satu pilihan
  ModeBelajar _mode = ModeBelajar.luring;

  // Slider: nilai dalam rentang
  double _targetIpk = 3.5;
  RangeValues _jamBelajar = const RangeValues(19, 22);

  String get _ringkasan {
    final minatDipilih =
        _minat.entries.where((e) => e.value).map((e) => e.key).join(', ');
    return 'Minat: ${minatDipilih.isEmpty ? '-' : minatDipilih}\n'
        'Notifikasi: ${_notifikasi ? 'aktif' : 'mati'}, '
        'Mode gelap: ${_modeGelap ? 'ya' : 'tidak'}\n'
        'Mode belajar: ${_mode.name}\n'
        'Target IPK: ${_targetIpk.toStringAsFixed(2)}\n'
        'Jam belajar: ${_jamBelajar.start.round()}.00 - ${_jamBelajar.end.round()}.00';
  }

  @override
  Widget build(BuildContext context) {
    final judul = Theme.of(context).textTheme.titleMedium;

    return Scaffold(
      appBar: AppBar(title: const Text('Preferensi Belajar')),
      body: ListView(
        padding: const EdgeInsets.symmetric(vertical: 8),
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Bidang minat (boleh lebih dari satu)', style: judul),
          ),
```

```
          for (final nama in _minat.keys)
            CheckboxListTile(
              title: Text(nama),
              value: _minat[nama],
              onChanged: (baru) => setState(() => _minat[nama] = baru ?? false),
            ),
          const Divider(),

          SwitchListTile(
            secondary: const Icon(Icons.notifications_active_outlined),
            title: const Text('Notifikasi tugas'),
            value: _notifikasi,
            onChanged: (baru) => setState(() => _notifikasi = baru),
          ),
          SwitchListTile(
            secondary: const Icon(Icons.dark_mode_outlined),
            title: const Text('Mode gelap'),
            value: _modeGelap,
            onChanged: (baru) => setState(() => _modeGelap = baru),
          ),
          const Divider(),

          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Mode belajar (pilih satu)', style: judul),
          ),
          RadioGroup<ModeBelajar>(
            groupValue: _mode,
            onChanged: (baru) {
              if (baru != null) setState(() => _mode = baru);
            },
            child: const Column(
              children: [
                RadioListTile(value: ModeBelajar.luring, title: Text('Luring (tatap muka)')),
                RadioListTile(value: ModeBelajar.daring, title: Text('Daring (online)')),
                RadioListTile(value: ModeBelajar.hybrid, title: Text('Hybrid')),
              ],
            ),
          ),
          const Divider(),

          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text('Target IPK: ${_targetIpk.toStringAsFixed(2)}', style: judul),
          ),
          Slider(
            value: _targetIpk,
            min: 2.0,
            max: 4.0,
            divisions: 40, // langkah 0.05
            label: _targetIpk.toStringAsFixed(2),
            onChanged: (baru) => setState(() => _targetIpk = baru),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: Text(
              'Jam belajar: ${_jamBelajar.start.round()}.00 - ${_jamBelajar.end.round()}.00',
              style: judul,
            ),
```

```
          ),
          // RangeSlider: memilih rentang (dua titik)
          RangeSlider(
            values: _jamBelajar,
            min: 0,
            max: 24,
            divisions: 24,
            labels: RangeLabels(
              '${_jamBelajar.start.round()}.00',
              '${_jamBelajar.end.round()}.00',
            ),
            onChanged: (baru) => setState(() => _jamBelajar = baru),
          ),
          const Divider(),

          Card(
            margin: const EdgeInsets.all(16),
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Text(_ringkasan),
            ),
          ),
        ],
      ),
    );
  }
}
```



Bagian atas: daftar Checkbox dan Switch



Setelah digulir: Radio, Slider, RangeSlider, dan ringkasan

Penjelasan kode:

- `Map<String, bool> _minat` menyimpan status setiap checkbox. `for (final nama in _minat.keys)` di dalam `children` membuat satu CheckboxListTile per minat.
- `RadioGroup<ModeBelajar>` membungkus `Column` berisi tiga `RadioListTile`. Karena RadioListTile di dalamnya tidak berubah, Column bisa `const`.
- `divisions: 40` pada rentang 2.0-4.0 menghasilkan langkah 0.05.
- `RangeSlider` memakai `RangeValues(start, end)` untuk dua titik sekaligus.
- Getter `_ringkasan` menyusun semua nilai menjadi teks. Karena dihitung ulang setiap build, ringkasan selalu sesuai pilihan terbaru.

> ⚠️ **Kesalahan umum**
>
> - **Lupa `setState` di `onChanged`.** Nilai variabel berubah tetapi tampilan tidak; checkbox terlihat "macet".
> - **Menulis `Radio(groupValue: ..., onChanged: ...)`.** Cara lama ini menghasilkan peringatan deprecated. Gunakan `RadioGroup`.
> - **Nilai awal Slider di luar `min`-`max`.** Menyebabkan error assertion. Pastikan `min <= value <= max`.
> - **Menyimpan nilai di variabel lokal dalam `build`.** Nilai akan kembali ke awal di setiap build. Simpan sebagai field State.

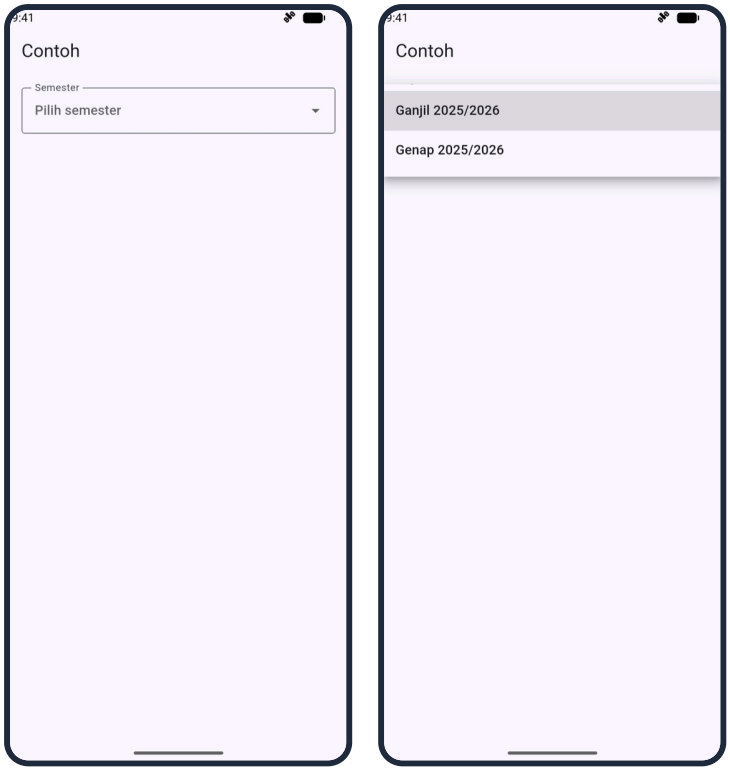
8. Dropdown, DatePicker & TimePicker

Untuk pilihan yang banyak (program studi, dosen) dan untuk tanggal/jam, pengguna sebaiknya memilih, bukan mengetik. Bagian ini membahas `DropdownButtonFormField`, `showDatePicker`, dan `showTimePicker`.

1. DropdownButtonFormField

Versi `...FormField` dipilih karena bisa memakai `InputDecoration` dan `validator` seperti TextFormField.

```
final List<String> _daftarSemester = ['Ganjil 2025/2026', 'Genap 2025/2026'];
String? _semester; // null = belum memilih

@override
Widget build(BuildContext context) {
  return DropdownButtonFormField<String>(
    initialValue: _semester, // nilai awal (Flutter < 3.35: pakai `value:`)
    hint: const Text('Pilih semester'),
    decoration: const InputDecoration(
      labelText: 'Semester',
      border: OutlineInputBorder(),
    ),
    items: _daftarSemester
        .map((s) => DropdownMenuItem(value: s, child: Text(s)))
        .toList(),
    onChanged: (baru) => setState(() => _semester = baru),
    validator: (v) => v == null ? 'Semester wajib dipilih' : null,
  );
}
```



Dropdown sebelum dipilih

Daftar pilihan setelah dropdown diketuk

- `items` adalah daftar `DropdownMenuItem`. Setiap item punya `value` (nilai yang disimpan) dan `child` (yang ditampilkan).
- `.map(...).toList()` mengubah `List<String>` menjadi `List<DropdownMenuItem>`.
- Nilai awal ditulis di `initialValue`. Pada Flutter sebelum 3.35 namanya `value`.

2. showDatePicker dan showTimePicker

Keduanya adalah fungsi `async` yang membuka dialog dan **mengembalikan nilai pilihan** setelah dialog ditutup, atau `null` jika pengguna menekan Batal.

```
DateTime? _tanggal;
TimeOfDay? _jam;

Future<void> _pilihTanggal() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggal ?? DateTime.now(),
    firstDate: DateTime(2020),
    lastDate: DateTime(2030),
    helpText: 'Pilih tanggal seminar',
  );
  // hasil == null jika pengguna menekan Batal
  if (hasil != null) setState(() => _tanggal = hasil);
}

Future<void> _pilihJam() async {
  final hasil = await showTimePicker(
    context: context,
    initialTime: _jam ?? const TimeOfDay(hour: 9, minute: 0),
  );
```

```
  if (hasil != null) setState(() => _jam = hasil);
}
```

`TimeOfDay` hanya berisi jam dan menit (tanpa tanggal). `DateTime` berisi tanggal lengkap. `firstDate` dan `lastDate` membatasi tanggal yang bisa dipilih.

Contoh lengkap: form pengajuan jadwal bimbingan

Trik yang sering dipakai: tanggal dan jam ditampilkan di `TextFormField` dengan `readOnly: true`. Field tidak bisa diketik, tetapi tetap bisa memakai `validator`, dan `onTap` membuka picker.

lib/main.dart

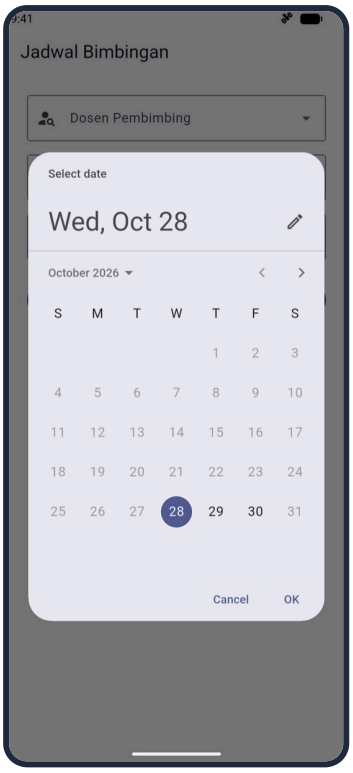
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const JadwalBimbinganScreen(),
    );
  }
}

class JadwalBimbinganScreen extends StatefulWidget {
  const JadwalBimbinganScreen({super.key});

  @override
  State<JadwalBimbinganScreen> createState() => _JadwalBimbinganScreenState();
}

class _JadwalBimbinganScreenState extends State<JadwalBimbinganScreen> {
  final _formKey = GlobalKey<FormState>();
  final _tanggalController = TextEditingController();
  final _jamController = TextEditingController();

  final List<String> _daftarDosen = [
    'Dr. Bambang Susilo, M.Kom.',
    'Nadia Fitri, M.T.',
    'Irham Akbar, M.Cs.',
  ];
  String? _dosen;
  DateTime? _tanggal;
  TimeOfDay? _jam;

  static const _namaBulan = [
    'Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun',
    'Jul', 'Agu', 'Sep', 'Okt', 'Nov', 'Des',
```

```
  ];

  String _formatTanggal(DateTime t) => '${t.day} ${_namaBulan[t.month - 1]} ${t.year}';

  String _formatJam(TimeOfDay j) =>
      '${j.hour.toString().padLeft(2, '0')}:${j.minute.toString().padLeft(2, '0')}';

  @override
  void dispose() {
    _tanggalController.dispose();
    _jamController.dispose();
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final sekarang = DateTime.now();
    final hasil = await showDatePicker(
      context: context,
      initialDate: _tanggal ?? sekarang,
      firstDate: sekarang, // tidak boleh memilih tanggal yang sudah lewat
      lastDate: sekarang.add(const Duration(days: 60)),
      // Hari Sabtu dan Minggu tidak bisa dipilih
      selectableDayPredicate: (hari) =>
          hari.weekday != DateTime.saturday && hari.weekday != DateTime.sunday,
    );
    if (hasil == null) return;
    setState(() {
      _tanggal = hasil;
      _tanggalController.text = _formatTanggal(hasil);
    });
  }

  Future<void> _pilihJam() async {
    final hasil = await showTimePicker(
      context: context,
      initialTime: _jam ?? const TimeOfDay(hour: 9, minute: 0),
    );
    if (hasil == null) return;
    setState(() {
      _jam = hasil;
      _jamController.text = _formatJam(hasil);
    });
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          'Bimbingan dengan $_dosen pada ${_formatTanggal(_tanggal!)} '
          'pukul ${_formatJam(_jam!)}',
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

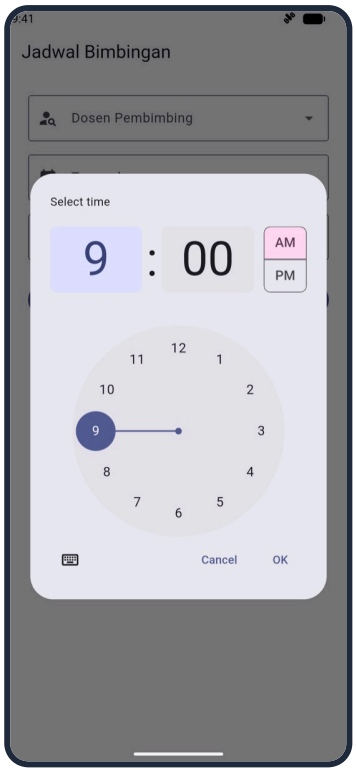
```
      appBar: AppBar(title: const Text('Jadwal Bimbingan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            DropdownButtonFormField<String>(
              initialValue: _dosen,
              isExpanded: true,
              decoration: const InputDecoration(
                labelText: 'Dosen Pembimbing',
                prefixIcon: Icon(Icons.person_search),
                border: OutlineInputBorder(),
              ),
              items: _daftarDosen
                  .map((d) => DropdownMenuItem(value: d, child: Text(d)))
                  .toList(),
              onChanged: (baru) => setState(() => _dosen = baru),
              validator: (v) => v == null ? 'Pilih dosen pembimbing' : null,
            ),
            const SizedBox(height: 16),

            // TextFormField readOnly: tidak bisa diketik, tapi bisa divalidasi
            TextFormField(
              controller: _tanggalController,
              readOnly: true,
              onTap: _pilihTanggal,
              decoration: const InputDecoration(
                labelText: 'Tanggal',
                prefixIcon: Icon(Icons.calendar_month),
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.isEmpty) ? 'Pilih tanggal' : null,
            ),
            const SizedBox(height: 16),

            TextFormField(
              controller: _jamController,
              readOnly: true,
              onTap: _pilihJam,
              decoration: const InputDecoration(
                labelText: 'Jam',
                prefixIcon: Icon(Icons.access_time),
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (_jam == null) return 'Pilih jam';
                if (_jam!.hour < 8 || _jam!.hour >= 16) {
                  return 'Bimbingan hanya pukul 08:00 - 16:00';
                }
                return null;
              },
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.event_available),
              label: const Text('Ajukan Jadwal'),
            ),
```

```
          ],
        ),
      ),
    );
  }
}
```

Ajukan Jadwal ditekan saat form kosong



Dialog DatePicker terbuka saat field Tanggal diketuk



Dialog TimePicker terbuka saat field Jam diketuk

Penjelasan kode:

- `firstDate: sekarang` membuat tanggal yang sudah lewat tidak bisa dipilih. `lastDate` dibatasi 60 hari ke depan.
- `selectableDayPredicate` menerima fungsi yang mengembalikan `false` untuk hari yang tidak boleh dipilih (Sabtu dan Minggu).
- Setelah tanggal dipilih, teksnya ditulis ke `_tanggalController` sehingga tampil di field dan lolos validator.
- Validator jam memeriksa jam kerja (08:00-16:00). Validator tidak harus membaca `v`, bisa juga membaca variabel State seperti `_jam`.
- `_formatTanggal` dan `_formatJam` memformat secara manual. Untuk format yang lebih lengkap (nama hari, bahasa lain), kelak bisa memakai package `intl`.

⚠️ **Kesalahan umum**

- **Error "There should be exactly one item with value ..."** pada dropdown. Terjadi jika nilai terpilih tidak ada di `items` atau ada dua item dengan `value` sama.
- **Teks dropdown panjang overflow.** Tambahkan `isExpanded: true`.
- `initialDate` **di luar rentang** `firstDate` **-** `lastDate`**.** Menyebabkan error. Pastikan nilai awal berada dalam rentang.
- **Tidak memeriksa** `null`**.** Hasil picker bernilai null jika pengguna membatalkan.

9. Umpan Balik: SnackBar, AlertDialog & BottomSheet

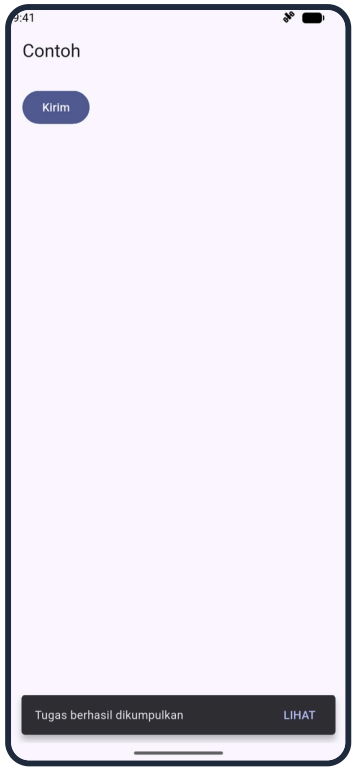
Setelah pengguna menekan tombol, aplikasi harus memberi tanggapan: "berhasil disimpan", "yakin ingin menghapus?", atau menampilkan pilihan tambahan. Tiga alat utamanya adalah **SnackBar**, **AlertDialog**, dan **BottomSheet**.

Widget	Sifat	Kapan dipakai
`SnackBar`	Pesan singkat di bawah layar, hilang sendiri, tidak menghalangi	Konfirmasi berhasil, tombol Urungkan
`AlertDialog`	Kotak di tengah layar, menghentikan interaksi sampai dijawab	Konfirmasi hapus, peringatan penting
`showModalBottomSheet`	Panel dari bawah layar	Daftar pilihan aksi, form singkat

1. SnackBar

SnackBar ditampilkan lewat `ScaffoldMessenger.of(context).showSnackBar(...)`. Tanda `..` (*cascade*) memanggil dua method berturut-turut pada objek yang sama.

```
void _tampilkanSnackBar(BuildContext context) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar() // tutup SnackBar lama agar tidak antre
    ..showSnackBar(
      SnackBar(
        content: const Text('Tugas berhasil dikumpulkan'),
        behavior: SnackBarBehavior.floating, // melayang, sudut membulat
        duration: const Duration(seconds: 3),
        action: SnackBarAction(
          label: 'LIHAT',
          onPressed: () => debugPrint('Buka detail tugas'),
        ),
      ),
    );
}
```



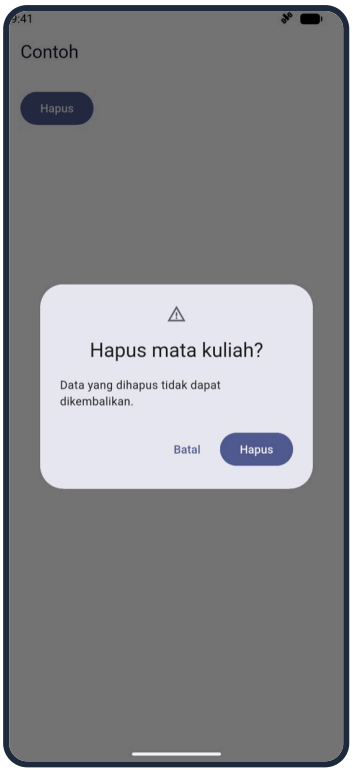
SnackBar mengambang dengan tombol LIHAT

2. AlertDialog yang mengembalikan jawaban

`showDialog<bool>` menunggu sampai dialog ditutup. Nilai yang dikirim lewat `Navigator.pop(dialogContext, nilai)` menjadi hasil `await`.

```
Future<void> _konfirmasiHapus(BuildContext context) async {
  // showDialog mengembalikan nilai yang dikirim lewat Navigator.pop
  final yakin = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      icon: const Icon(Icons.warning_amber),
      title: const Text('Hapus mata kuliah?'),
      content: const Text('Data yang dihapus tidak dapat dikembalikan.'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(dialogContext, true),
          child: const Text('Hapus'),
        ),
      ],
    ),
  );

  // yakin bisa null jika dialog ditutup dengan mengetuk area luar
  if (yakin == true && context.mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Mata kuliah dihapus')),
    );
```

```
  }
}
```



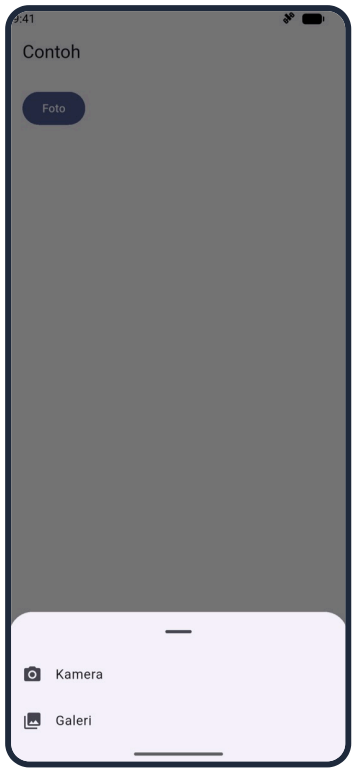
AlertDialog konfirmasi hapus

Perhatikan `context.mounted`: setelah `await`, layar bisa saja sudah ditutup. Memakai `context` yang sudah tidak terpasang akan menimbulkan error, jadi selalu periksa dulu.

3. Modal BottomSheet

```dart
Future<void> _pilihSumberFoto(BuildContext context) async {
  final pilihan = await showModalBottomSheet<String>(
    context: context,
    showDragHandle: true, // garis kecil di atas sheet
    builder: (sheetContext) => SafeArea(
      child: Column(
        mainAxisSize: MainAxisSize.min, // tinggi sheet secukupnya
        children: [
          ListTile(
            leading: const Icon(Icons.camera_alt),
            title: const Text('Kamera'),
            onTap: () => Navigator.pop(sheetContext, 'kamera'),
          ),
          ListTile(
            leading: const Icon(Icons.photo_library),
            title: const Text('Galeri'),
            onTap: () => Navigator.pop(sheetContext, 'galeri'),
          ),
        ],
      ),
    ),
  );
```

```
  debugPrint('Dipilih: $pilihan'); // null jika sheet ditutup tanpa memilih
}
```



BottomSheet berisi pilihan Kamera dan Galeri

`mainAxisSize: MainAxisSize.min` membuat tinggi sheet mengikuti isinya. `SafeArea` mencegah isi tertutup bilah navigasi HP.

Contoh lengkap: catatan tugas dengan tiga jenis umpan balik

`lib/main.dart`

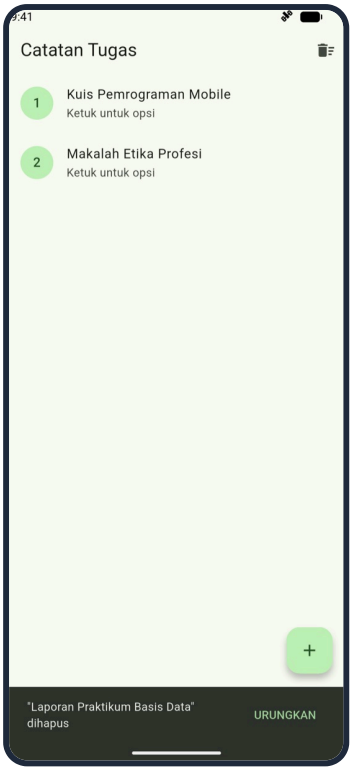
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const CatatanTugasScreen(),
    );
  }
}

class CatatanTugasScreen extends StatefulWidget {
```

```dart
  const CatatanTugasScreen({super.key});

  @override
  State<CatatanTugasScreen> createState() => _CatatanTugasScreenState();
}

class _CatatanTugasScreenState extends State<CatatanTugasScreen> {
  final List<String> _tugas = [
    'Laporan Praktikum Basis Data',
    'Kuis Pemrograman Mobile',
    'Makalah Etika Profesi',
  ];

  // ---------- 1. SnackBar dengan tombol UNDO ----------
  void _hapusDenganUndo(int index) {
    final terhapus = _tugas[index];
    setState(() => _tugas.removeAt(index));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('"$terhapus" dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            onPressed: () => setState(() => _tugas.insert(index, terhapus)),
          ),
        ),
      );
  }

  // ---------- 2. AlertDialog konfirmasi ----------
  Future<void> _konfirmasiHapusSemua() async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus semua tugas?'),
        content: Text('${_tugas.length} tugas akan dihapus.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus Semua'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    setState(() => _tugas.clear());
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Semua tugas dihapus')),
    );
  }

  // ---------- 3. Dialog berisi TextField untuk menambah data ----------
  Future<void> _dialogTambah() async {
```

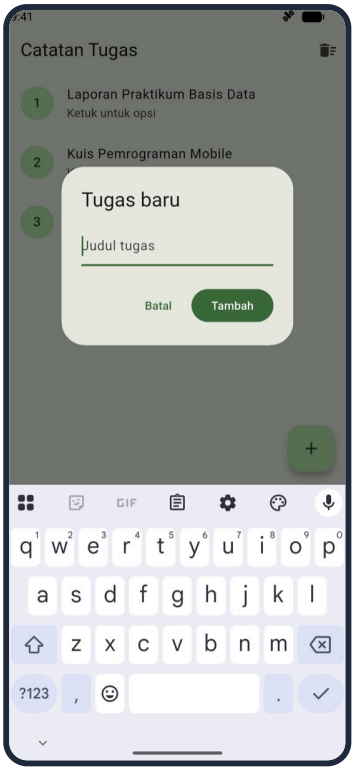
```dart
    // Cukup simpan teks di variabel lokal lewat onChanged.
    // (Membuat TextEditingController di sini berisiko: controller harus
    // di-dispose, padahal TextField masih hidup selama animasi dialog ditutup.)
    var teks = '';
    final judul = await showDialog<String>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Tugas baru'),
        content: TextField(
          autofocus: true,
          decoration: const InputDecoration(hintText: 'Judul tugas'),
          onChanged: (v) => teks = v,
          onSubmitted: (v) => Navigator.pop(dialogContext, v),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, teks),
            child: const Text('Tambah'),
          ),
        ],
      ),
    );

    if (judul == null || judul.trim().isEmpty || !mounted) return;
    setState(() => _tugas.add(judul.trim()));
  }

  // ---------- 4. BottomSheet berisi pilihan aksi ----------
  Future<void> _menuTugas(int index) async {
    final aksi = await showModalBottomSheet<String>(
      context: context,
      showDragHandle: true,
      builder: (sheetContext) => SafeArea(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Padding(
              padding: const EdgeInsets.all(8),
              child: Text(_tugas[index], style: const TextStyle(fontWeight: FontWeight.bold)),
            ),
            ListTile(
              leading: const Icon(Icons.check_circle_outline),
              title: const Text('Tandai selesai'),
              onTap: () => Navigator.pop(sheetContext, 'selesai'),
            ),
            ListTile(
              leading: const Icon(Icons.delete_outline),
              title: const Text('Hapus'),
              onTap: () => Navigator.pop(sheetContext, 'hapus'),
            ),
          ],
        ),
      ),
    );
    if (!mounted) return;
```

```
    if (aksi == 'hapus') _hapusDenganUndo(index);
    if (aksi == 'selesai') {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Mantap! "${_tugas[index]}" selesai'),
          behavior: SnackBarBehavior.floating,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Tugas'),
        actions: [
          IconButton(
            tooltip: 'Hapus semua',
            icon: const Icon(Icons.delete_sweep),
            onPressed: _tugas.isEmpty ? null : _konfirmasiHapusSemua,
          ),
        ],
      ),
      body: _tugas.isEmpty
          ? const Center(child: Text('Tidak ada tugas. Santai dulu!'))
          : ListView(
              children: [
                for (var i = 0; i < _tugas.length; i++)
                  ListTile(
                    leading: CircleAvatar(child: Text('${i + 1}')),
                    title: Text(_tugas[i]),
                    subtitle: const Text('Ketuk untuk opsi'),
                    onTap: () => _menuTugas(i),
                  ),
              ],
            ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah tugas',
        onPressed: _dialogTambah,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Menu bottom sheet saat tugas diketuk



SnackBar dengan tombol URUNGKAN setelah tugas dihapus



Dialog berisi TextField untuk menambah tugas

Penjelasan kode:

- **Urungkan:** sebelum menghapus, item dan posisinya disimpan (`terhapus` , `index`). Tombol URUNGKAN memasukkannya kembali dengan `insert(index, terhapus)` .
- **Hapus semua:** dialog mengembalikan `bool?` . Ditulis `yakin != true` agar nilai `null` (dialog ditutup dengan mengetuk luar) dianggap batal.
- **Tambah:** dialog bisa berisi TextField. Teks disimpan ke variabel lokal lewat `onChanged` lalu dikirim lewat `Navigator.pop` .
- **Menu tugas:** bottom sheet mengembalikan `String` berisi aksi yang dipilih, lalu diproses setelah sheet tertutup.
- Tombol hapus semua di AppBar diberi `onPressed: _tugas.isEmpty ? null : ...` sehingga nonaktif saat daftar kosong.

⚠️ **Kesalahan umum**

- **Memakai `context` milik layar untuk menutup dialog.** Gunakan context dari `builder` dialog (`dialogContext`) di `Navigator.pop`.
- **SnackBar menumpuk dan antre lama.** Panggil `hideCurrentSnackBar()` sebelum menampilkan yang baru.
- **Lupa memeriksa `mounted` setelah `await showDialog`.** Muncul peringatan "use_build_context_synchronously".
- **BottomSheet berisi form tertutup keyboard.** Gunakan `isScrollControlled: true` dan padding sebesar `MediaQuery.viewInsetsOf(context).bottom` (lihat studi kasus Modul 06).

10. Studi Kasus: Formulir Pendaftaran Mahasiswa

Sekarang kita gabungkan semua materi modul ini menjadi satu aplikasi utuh: **formulir pendaftaran mahasiswa baru** yang memakai TextFormField, validasi, password, DatePicker, Radio, Dropdown, Slider, FilterChip, Switch, Checkbox, dialog konfirmasi, dan halaman hasil.

Rancangan aplikasi

1. **Model data** `DataPendaftar`: satu objek yang menampung semua isian. Lebih rapi daripada mengirim sepuluh variabel terpisah.
2. **Validators**: aturan NIM, email, nomor HP, password, dan konfirmasi.
3. **PendaftaranScreen**: form tiga bagian (Data Diri, Kontak & Akun, Data Akademik).
4. **HasilPendaftaranScreen**: menampilkan ringkasan data. Perpindahan halaman memakai `Navigator.push`, yang akan dibahas tuntas di Modul 07.

Alur saat tombol Kirim ditekan

1. Tutup keyboard dengan `unfocus()`.
2. Jalankan `validate()` untuk semua TextFormField dan Dropdown.
3. Periksa manual input yang bukan FormField: jenis kelamin, minat, persetujuan.
4. Kumpulkan isian ke objek `DataPendaftar`.
5. Tampilkan dialog konfirmasi. Jika disetujui, buka halaman hasil.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pendaftaran Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PendaftaranScreen(),
    );
  }
}

// ======================= MODEL DATA =======================
enum JenisKelamin { lakiLaki, perempuan }

class DataPendaftar {
```

```dart
  final String nama;
  final String nim;
  final String email;
  final String noHp;
  final String prodi;
  final JenisKelamin jenisKelamin;
  final DateTime tanggalLahir;
  final List<String> minat;
  final int semester;
  final bool ikutBeasiswa;

  const DataPendaftar({
    required this.nama,
    required this.nim,
    required this.email,
    required this.noHp,
    required this.prodi,
    required this.jenisKelamin,
    required this.tanggalLahir,
    required this.minat,
    required this.semester,
    required this.ikutBeasiswa,
  });
}

// ======================= VALIDATOR =======================
class Validators {
  static final _emailRegex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');

  static String? wajib(String? v, String field) =>
      (v == null || v.trim().isEmpty) ? '$field wajib diisi' : null;

  static String? nim(String? v) {
    if (v == null || v.trim().isEmpty) return 'NIM wajib diisi';
    if (!RegExp(r'^[A-Z]\d[A-Z]\d{6}$').hasMatch(v.trim().toUpperCase())) {
      return 'Format NIM contoh: H1D024001';
    }
    return null;
  }

  static String? email(String? v) {
    if (v == null || v.trim().isEmpty) return 'Email wajib diisi';
    if (!_emailRegex.hasMatch(v.trim())) return 'Format email tidak valid';
    return null;
  }

  static String? noHp(String? v) {
    if (v == null || v.trim().isEmpty) return 'Nomor HP wajib diisi';
    if (!RegExp(r'^08\d{8,11}$').hasMatch(v.trim())) {
      return 'Nomor HP diawali 08, 10-13 digit';
    }
    return null;
  }

  static String? password(String? v) {
    if (v == null || v.isEmpty) return 'Password wajib diisi';
    if (v.length < 8) return 'Password minimal 8 karakter';
    return null;
  }
```

```
  static String? konfirmasi(String? v, String asli) {
    if (v == null || v.isEmpty) return 'Ulangi password';
    if (v != asli) return 'Konfirmasi password tidak sama';
    return null;
  }
}

String formatTanggal(DateTime t) {
  const bulan = [
    'Januari', 'Februari', 'Maret', 'April', 'Mei', 'Juni',
    'Juli', 'Agustus', 'September', 'Oktober', 'November', 'Desember',
  ];
  return '${t.day} ${bulan[t.month - 1]} ${t.year}';
}

// ======================== LAYAR FORM ========================
class PendaftaranScreen extends StatefulWidget {
  const PendaftaranScreen({super.key});

  @override
  State<PendaftaranScreen> createState() => _PendaftaranScreenState();
}

class _PendaftaranScreenState extends State<PendaftaranScreen> {
  final _formKey = GlobalKey<FormState>();

  final _namaC = TextEditingController();
  final _nimC = TextEditingController();
  final _emailC = TextEditingController();
  final _hpC = TextEditingController();
  final _passwordC = TextEditingController();
  final _konfirmasiC = TextEditingController();
  final _tanggalC = TextEditingController();

  static const _daftarProdi = [
    'S1 Informatika',
    'S1 Teknik Elektro',
    'S1 Teknik Sipil',
    'S1 Teknik Industri',
  ];
  static const _daftarMinat = ['Mobile', 'Web', 'AI', 'Jaringan', 'UI/UX'];

  String? _prodi;
  JenisKelamin? _jenisKelamin;
  DateTime? _tanggalLahir;
  final Set<String> _minat = {};
  double _semester = 1;
  bool _ikutBeasiswa = false;
  bool _setuju = false;
  bool _sembunyikan = true;

  @override
  void dispose() {
    for (final c in [_namaC, _nimC, _emailC, _hpC, _passwordC, _konfirmasiC, _tanggalC]) {
      c.dispose();
    }
    super.dispose();
  }
```

```dart
Future<void> _pilihTanggalLahir() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggalLahir ?? DateTime(2005, 1, 1),
    firstDate: DateTime(1990),
    lastDate: DateTime.now(),
    helpText: 'Tanggal lahir',
  );
  if (hasil == null) return;
  setState(() {
    _tanggalLahir = hasil;
    _tanggalC.text = formatTanggal(hasil);
  });
}

void _tampilkanError(String pesan) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(pesan), backgroundColor: Colors.red));
}

Future<void> _kirim() async {
  FocusScope.of(context).unfocus();

  // 1. Validasi semua TextFormField & DropdownButtonFormField
  final formValid = _formKey.currentState!.validate();
  if (!formValid) {
    _tampilkanError('Masih ada isian yang salah');
    return;
  }
  // 2. Validasi input yang tidak memakai FormField
  if (_jenisKelamin == null) {
    _tampilkanError('Pilih jenis kelamin');
    return;
  }
  if (_minat.isEmpty) {
    _tampilkanError('Pilih minimal satu bidang minat');
    return;
  }
  if (!_setuju) {
    _tampilkanError('Centang pernyataan persetujuan');
    return;
  }

  // 3. Kumpulkan data ke satu objek
  final data = DataPendaftar(
    nama: _namaC.text.trim(),
    nim: _nimC.text.trim().toUpperCase(),
    email: _emailC.text.trim(),
    noHp: _hpC.text.trim(),
    prodi: _prodi!,
    jenisKelamin: _jenisKelamin!,
    tanggalLahir: _tanggalLahir!,
    minat: _minat.toList(),
    semester: _semester.round(),
    ikutBeasiswa: _ikutBeasiswa,
  );
```

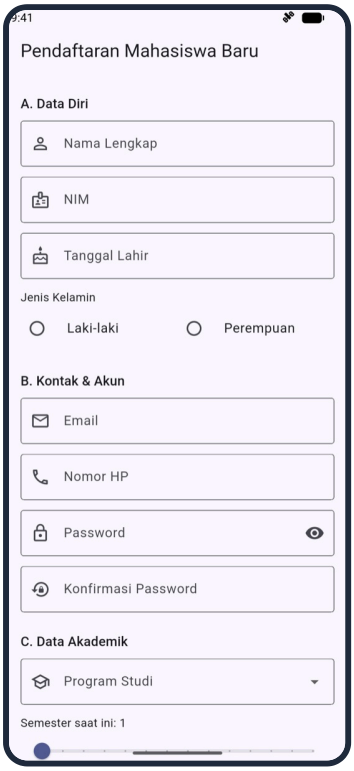
```
    // 4. Konfirmasi lewat dialog
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Kirim pendaftaran?'),
        content: Text('Pastikan data ${data.nama} (${data.nim}) sudah benar.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Periksa lagi'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Ya, kirim'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;

    // 5. Tampilkan hasil di layar baru (detail navigasi dibahas di Modul 07)
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => HasilPendaftaranScreen(data: data)),
    );
  }

  Widget _judulBagian(String teks) => Padding(
        padding: const EdgeInsets.only(top: 24, bottom: 8),
        child: Text(teks, style: Theme.of(context).textTheme.titleMedium),
      );

  @override
  Widget build(BuildContext context) {
    const jarak = SizedBox(height: 12);

    return Scaffold(
      appBar: AppBar(title: const Text('Pendaftaran Mahasiswa Baru')),
      body: Form(
        key: _formKey,
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.fromLTRB(16, 0, 16, 32),
          children: [
            _judulBagian('A. Data Diri'),
            TextFormField(
              controller: _namaC,
              textCapitalization: TextCapitalization.words,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person_outline),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.wajib(v, 'Nama'),
            ),
            jarak,
            TextFormField(
              controller: _nimC,
```

```
            textCapitalization: TextCapitalization.characters,
            textInputAction: TextInputAction.next,
            decoration: const InputDecoration(
              labelText: 'NIM',
              hintText: 'H1D024001',
              prefixIcon: Icon(Icons.badge_outlined),
              border: OutlineInputBorder(),
            ),
            validator: Validators.nim,
          ),
          jarak,
          TextFormField(
            controller: _tanggalC,
            readOnly: true,
            onTap: _pilihTanggalLahir,
            decoration: const InputDecoration(
              labelText: 'Tanggal Lahir',
              prefixIcon: Icon(Icons.cake_outlined),
              border: OutlineInputBorder(),
            ),
            validator: (v) => Validators.wajib(v, 'Tanggal lahir'),
          ),
          jarak,
          const Text('Jenis Kelamin'),
          RadioGroup<JenisKelamin>(
            groupValue: _jenisKelamin,
            onChanged: (v) => setState(() => _jenisKelamin = v),
            child: const Row(
              children: [
                Expanded(
                  child: RadioListTile(
                    value: JenisKelamin.lakiLaki,
                    title: Text('Laki-laki'),
                    contentPadding: EdgeInsets.zero,
                  ),
                ),
                Expanded(
                  child: RadioListTile(
                    value: JenisKelamin.perempuan,
                    title: Text('Perempuan'),
                    contentPadding: EdgeInsets.zero,
                  ),
                ),
              ],
            ),
          ),

          _judulBagian('B. Kontak & Akun'),
          TextFormField(
            controller: _emailC,
            keyboardType: TextInputType.emailAddress,
            textInputAction: TextInputAction.next,
            decoration: const InputDecoration(
              labelText: 'Email',
              prefixIcon: Icon(Icons.email_outlined),
              border: OutlineInputBorder(),
            ),
            validator: Validators.email,
          ),
```

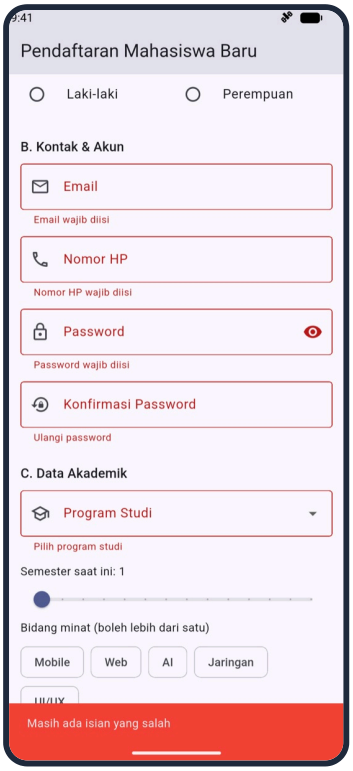
```
            jarak,
            TextFormField(
              controller: _hpC,
              keyboardType: TextInputType.phone,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Nomor HP',
                hintText: '081234567890',
                prefixIcon: Icon(Icons.phone_outlined),
                border: OutlineInputBorder(),
              ),
              validator: Validators.noHp,
            ),
            jarak,
            TextFormField(
              controller: _passwordC,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.next,
              decoration: InputDecoration(
                labelText: 'Password',
                prefixIcon: const Icon(Icons.lock_outline),
                border: const OutlineInputBorder(),
                suffixIcon: IconButton(
                  icon: Icon(_sembunyikan ? Icons.visibility : Icons.visibility_off),
                  onPressed: () => setState(() => _sembunyikan = !_sembunyikan),
                ),
              ),
              validator: Validators.password,
            ),
            jarak,
            TextFormField(
              controller: _konfirmasiC,
              obscureText: _sembunyikan,
              textInputAction: TextInputAction.done,
              decoration: const InputDecoration(
                labelText: 'Konfirmasi Password',
                prefixIcon: Icon(Icons.lock_reset),
                border: OutlineInputBorder(),
              ),
              validator: (v) => Validators.konfirmasi(v, _passwordC.text),
            ),

            _judulBagian('C. Data Akademik'),
            DropdownButtonFormField<String>(
              initialValue: _prodi,
              isExpanded: true,
              decoration: const InputDecoration(
                labelText: 'Program Studi',
                prefixIcon: Icon(Icons.school_outlined),
                border: OutlineInputBorder(),
              ),
              items: _daftarProdi
                  .map((p) => DropdownMenuItem(value: p, child: Text(p)))
                  .toList(),
              onChanged: (v) => setState(() => _prodi = v),
              validator: (v) => v == null ? 'Pilih program studi' : null,
            ),
            jarak,
            Text('Semester saat ini: ${_semester.round()}'),
```

```
            Slider(
              value: _semester,
              min: 1,
              max: 14,
              divisions: 13,
              label: 'Semester ${_semester.round()}',
              onChanged: (v) => setState(() => _semester = v),
            ),
            const Text('Bidang minat (boleh lebih dari satu)'),
            const SizedBox(height: 8),
            Wrap(
              spacing: 8,
              children: [
                for (final m in _daftarMinat)
                  // FilterChip = versi "chip" dari checkbox
                  FilterChip(
                    label: Text(m),
                    selected: _minat.contains(m),
                    onSelected: (pilih) => setState(() {
                      pilih ? _minat.add(m) : _minat.remove(m);
                    }),
                  ),
              ],
            ),
            SwitchListTile(
              contentPadding: EdgeInsets.zero,
              title: const Text('Mengajukan beasiswa'),
              value: _ikutBeasiswa,
              onChanged: (v) => setState(() => _ikutBeasiswa = v),
            ),
            CheckboxListTile(
              contentPadding: EdgeInsets.zero,
              controlAffinity: ListTileControlAffinity.leading,
              title: const Text('Saya menyatakan data di atas benar.'),
              value: _setuju,
              onChanged: (v) => setState(() => _setuju = v ?? false),
            ),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: _kirim,
              icon: const Icon(Icons.send),
              label: const Text('Kirim Pendaftaran'),
              style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(50)),
            ),
          ],
        ),
      ),
    );
  }
}

// ======================== LAYAR HASIL ========================
class HasilPendaftaranScreen extends StatelessWidget {
  final DataPendaftar data;

  const HasilPendaftaranScreen({super.key, required this.data});

  @override
  Widget build(BuildContext context) {
```

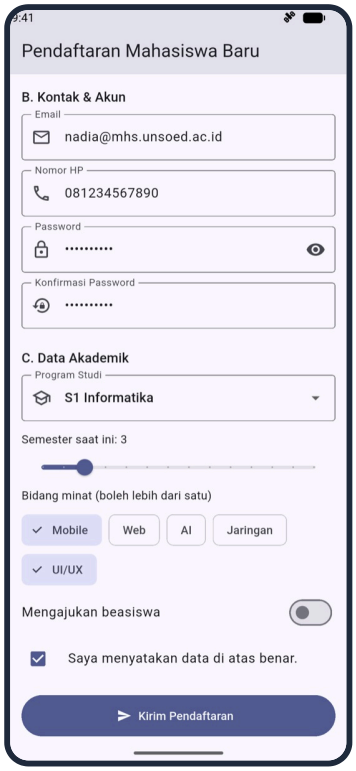
```
    final baris = <(IconData, String, String)>[
      (Icons.person, 'Nama', data.nama),
      (Icons.badge, 'NIM', data.nim),
      (Icons.wc, 'Jenis Kelamin',
          data.jenisKelamin == JenisKelamin.lakiLaki ? 'Laki-laki' : 'Perempuan'),
      (Icons.cake, 'Tanggal Lahir', formatTanggal(data.tanggalLahir)),
      (Icons.email, 'Email', data.email),
      (Icons.phone, 'Nomor HP', data.noHp),
      (Icons.school, 'Program Studi', data.prodi),
      (Icons.timeline, 'Semester', '${data.semester}'),
      (Icons.interests, 'Minat', data.minat.join(', ')),
      (Icons.card_giftcard, 'Beasiswa', data.ikutBeasiswa ? 'Ya' : 'Tidak'),
    ];

    return Scaffold(
      appBar: AppBar(title: const Text('Pendaftaran Berhasil')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Icon(Icons.check_circle, color: Colors.green, size: 80),
          const SizedBox(height: 8),
          Text(
            'Selamat datang, ${data.nama}!',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.titleLarge,
          ),
          const SizedBox(height: 16),
          Card(
            child: Column(
              children: [
                for (final (ikon, label, nilai) in baris)
                  ListTile(
                    leading: Icon(ikon),
                    title: Text(label),
                    subtitle: Text(nilai),
                  ),
              ],
            ),
          ),
          const SizedBox(height: 16),
          OutlinedButton.icon(
            onPressed: () => Navigator.pop(context),
            icon: const Icon(Icons.edit),
            label: const Text('Kembali & ubah data'),
          ),
        ],
      ),
    );
  }
}
```



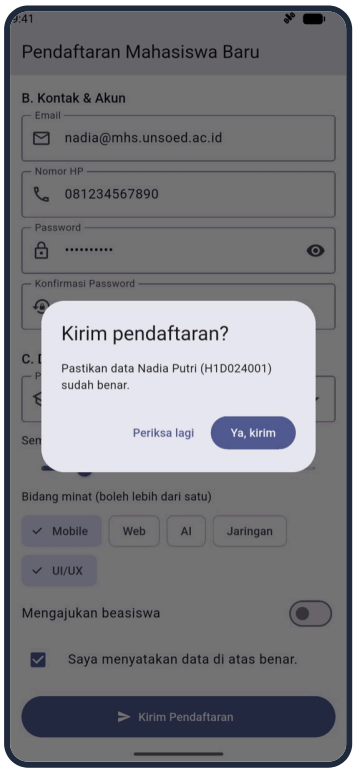
Bagian atas form pendaftaran



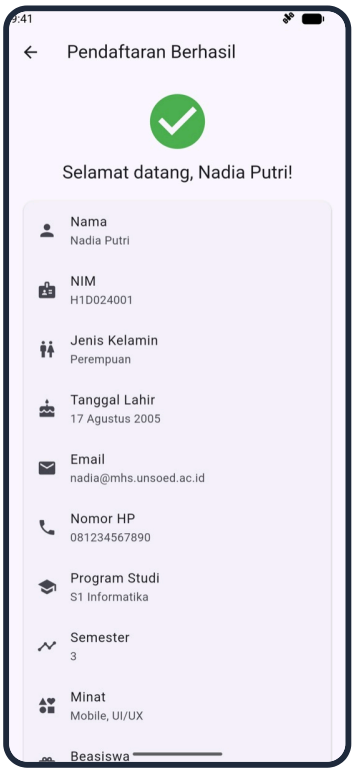
Kirim ditekan saat form kosong: pesan error dan SnackBar merah



Bagian Data Akademik: Dropdown, Slider, FilterChip, Switch, Checkbox



Dialog konfirmasi setelah semua isian valid



Halaman hasil pendaftaran

Penjelasan bagian penting:

- **Validasi NIM** memakai pola `^[A-Z]\d[A-Z]\d{6}$` : huruf, angka, huruf, lalu enam angka (contoh H1D024001). Isian diubah ke huruf besar dulu dengan `toUpperCase()`.

- **Banyak controller** di-dispose sekaligus dengan perulangan `for (final c in [...]) c.dispose();`.
- **FilterChip** berfungsi seperti checkbox dalam bentuk chip. Pilihan disimpan di `Set<String>` agar tidak ada duplikat.
- **Radio dalam Row**: dua `RadioListTile` dibungkus `Expanded` agar berbagi lebar, dan `RadioGroup` membungkus Row tersebut.
- **Record** `(IconData, String, String)` di halaman hasil adalah fitur Dart 3 untuk mengelompokkan beberapa nilai tanpa membuat kelas. `for (final (ikon, label, nilai) in baris)` langsung memecahnya menjadi tiga variabel.
- **Data dikirim ke halaman hasil** lewat konstruktor: `HasilPendaftaranScreen(data: data)`.

> 💡 **Uji form Anda seperti penguji**
>
> - Tekan Kirim tanpa mengisi apa pun: semua pesan error harus muncul.
> - Isi NIM dengan huruf kecil (h1d024001): harus tetap diterima.
> - Isi email `budi@`, nomor HP `0812`, password 5 huruf: masing-masing harus ditolak dengan pesan yang jelas.
> - Isi password dan konfirmasi berbeda satu huruf: harus ditolak.
> - Isi semua dengan benar tetapi tidak memilih minat: SnackBar harus muncul.

> ℹ️ **Dipakai lagi di Modul 13-14**
>
> Kelas `Validators`, pola `GlobalKey<FormState>`, toggle password, dan tombol yang nonaktif saat mengirim akan dipakai ulang untuk layar login dan register pada proyek CRUD + autentikasi. Simpan kode ini baik-baik.

Tugas Praktikum: Praktikum 5: Formulir Pendaftaran Seminar Tugas Akhir

Sasaran Praktikum

Membangun formulir lengkap dengan validasi, input pilihan, dan umpan balik untuk pendaftaran seminar tugas akhir mahasiswa.

Langkah Pengerjaan

1. Buat proyek baru dan file `lib/main.dart` berisi `FormSeminarScreen` (StatefulWidget).
2. Buat kelas `Validators` berisi fungsi static untuk: wajib diisi, NIM (format seperti H1D024001), email, dan panjang minimal.
3. Tambahkan field: NIM, Nama, Email, Judul Tugas Akhir (multi-baris, minimal 20 karakter), dan Ringkasan (multi-baris, maksimal 300 karakter dengan `maxLength`).
4. Atur `keyboardType`, `textInputAction: TextInputAction.next`, dan `FocusNode` agar tombol Next berpindah ke field berikutnya.
5. Tambahkan `DropdownButtonFormField` untuk Dosen Pembimbing 1 dan 2 dengan validator, serta tolak jika keduanya sama.
6. Tambahkan pemilih tanggal seminar dengan `showDatePicker` (hanya hari kerja, mulai besok) dan jam dengan `showTimePicker`, ditampilkan pada TextFormField `readOnly`.
7. Tambahkan `RadioGroup` untuk jenis seminar (Proposal / Hasil / Sidang) dan `CheckboxListTile` persetujuan.
8. Saat Kirim ditekan dan semua valid, tampilkan `AlertDialog` ringkasan data; jika dikonfirmasi tampilkan `SnackBar` sukses dan kosongkan form. Tombol Kirim harus nonaktif selama simulasi pengiriman 2 detik.
9. Pastikan semua `TextEditingController` dan `FocusNode` di-dispose, dan `flutter analyze` tidak menampilkan peringatan.

Output yang Diharapkan

Formulir yang menolak isian salah dengan pesan error yang jelas, berpindah fokus dengan tombol Next, menampilkan dialog ringkasan, dan SnackBar sukses setelah dikirim.

Rubrik Penilaian

Form + GlobalKey<FormState> + kelas Validators (30%), Input pilihan: Dropdown, Radio, Checkbox, Date/TimePicker (25%), Keyboard, FocusNode, dan dispose yang benar (20%), Umpan balik dialog/SnackBar dan tombol nonaktif saat mengirim (15%), Kerapian UI (10%).

Kuis Pemahaman

1. Mengapa kita WAJIB memanggil method controller.dispose() di dalam method dispose() milik StatefulWidget?

A. Untuk mencegah memory leak (kebocoran memori) karena listener controller terus hidup meskipun widget sudah dihancurkan

B. Karena jika tidak ditutup, data form akan otomatis terkirim ke server

C. Untuk mengubah font teks menjadi tebal

D. Agar tombol submit bisa diklik

2. Apa nilai kembalian (return value) dari fungsi validator pada TextFormField jika input pengguna SUDAH BENAR dan VALID?

A. Mengembalikan String 'Valid'

B. Mengembalikan nilai boolean true

C. Mengembalikan null

D. Mengembalikan angka 1

3. Bagaimana cara membuat sebuah FilledButton tampil nonaktif (disabled)?

A. Memberi onPressed: null

B. Memberi child: null

C. Membungkusnya dengan Opacity(opacity: 0.5)

D. Memberi enabled: false pada FilledButton

4. Efek riak (ripple) InkWell tidak terlihat sama sekali. Penyebab yang paling mungkin adalah...

A. InkWell tidak memiliki widget Material di atasnya, atau anaknya Container berwarna yang menutupi riak

B. InkWell hanya bekerja di iOS

C. onTap pada InkWell bernilai fungsi

D. InkWell dibungkus Padding

5. Apa perbedaan utama onChanged dan TextEditingController pada TextField?

A. onChanged hanya membaca saat pengguna mengetik, sedangkan controller dapat membaca, mengubah, dan mengosongkan teks dari kode kapan saja

B. onChanged lebih hemat memori sehingga controller tidak perlu dipakai

C. Controller hanya bisa dipakai pada TextFormField

D. Tidak ada perbedaan

6. Kode manakah yang memindahkan kursor ke field password ketika tombol Next pada keyboard ditekan di field email?

A. textInputAction: TextInputAction.next, onSubmitted: (_) ⇒ _passwordFocus.requestFocus()

B. keyboardType: TextInputType.next

C. obscureText: true pada field email

D. autofocus: true pada field email

7. Validator konfirmasi password yang benar adalah...

A. (v) ⇒ v != _passwordController.text ? 'Konfirmasi password tidak sama' : null

B. (v) ⇒ v == _passwordController.text ? 'Valid' : 'Tidak sama'

C. (v) ⇒ v!.length >= 8

D. (v) ⇒ _formKey.currentState!.validate()

8. Sejak Flutter 3.35, bagaimana cara yang tidak deprecated untuk membuat sekelompok radio button?

A. Membungkus semua RadioListTile dengan RadioGroup yang memegang groupValue dan onChanged, dan setiap radio hanya diberi value

B. Memberi groupValue dan onChanged pada setiap Radio

C. Memakai Checkbox dengan tristate: true

D. Memakai Switch untuk setiap pilihan

Kunci Jawaban & Pembahasan

1. A — TextEditingController memiliki listener internal yang didaftarkan ke sistem operasi. Jika tidak di-dispose saat widget dihancurkan, referensi controller tetap menggantung di memori, menyebabkan kebocoran memori (memory leak).

2. C — Dalam Flutter Form validation, jika fungsi validator mengembalikan nilai null, framework menganggap isian tersebut valid. Jika tidak valid, fungsi mengembalikan String pesan kesalahan yang akan dicetak merah di bawah input field.

3. A — Semua tombol Material menjadi nonaktif secara otomatis ketika onPressed bernilai null. Warnanya memudar dan tombol tidak merespons sentuhan.

4. A — InkWell menggambar riaknya pada widget Material terdekat. Tanpa Material, atau jika anak InkWell adalah Container berwarna, riak tidak terlihat. Berikan warna pada Material atau gunakan Card.

5. A — onChanged adalah callback yang menerima teks terbaru saat pengguna mengetik. TextEditingController memberi akses penuh: .text untuk membaca/mengubah, clear() untuk mengosongkan, dan addListener untuk mendengarkan perubahan.

6. A — textInputAction mengubah tombol Enter menjadi Next, dan onSubmitted dipanggil saat tombol itu ditekan. FocusNode.requestFocus() memindahkan fokus ke field yang memakai FocusNode tersebut.

7. A — Validator harus mengembalikan null jika valid dan String pesan error jika tidak valid. Isian konfirmasi dibandingkan dengan teks pada controller password.

8. A — Parameter groupValue dan onChanged pada Radio/RadioListTile sudah deprecated. RadioGroup<T> kini memegang nilai terpilih dan callback untuk semua radio di dalamnya.

MODUL 06 · LISTS & SLIVERS

Scrollable Views & Optimasi Performa List Data

Menampilkan konten yang bisa digulir dengan SingleChildScrollView, ListView (biasa, builder, separated), ListTile, dan GridView; menambahkan interaksi swipe-to-delete, pull-to-refresh, infinite scroll, serta pencarian; menangani loading/empty state; dan menyusun tampilan lanjutan dengan CustomScrollView dan Slivers.

Durasi Pertemuan 6 (150 Menit) + Belajar Mandiri

Prasyarat Modul 05: Interaktivitas & Form

Capaian Pembelajaran

- Menggunakan SingleChildScrollView untuk halaman berisi konten terbatas dan mengatasi error overflow.
- Membedakan ListView biasa, ListView.builder, dan ListView.separated serta memilihnya sesuai jumlah data.
- Menyusun baris daftar yang informatif dengan ListTile.
- Menerapkan Dismissible (swipe to delete + urungkan) dan RefreshIndicator (pull-to-refresh).
- Membangun grid responsif dengan GridView.count dan GridView.builder.
- Menangani loading, error, dan empty state serta memuat data bertahap dengan ScrollController.
- Membuat pencarian dan filter daftar secara real-time dengan TextField.
- Mengimplementasikan CustomScrollView dengan SliverAppBar, SliverList, dan SliverGrid.

Daftar Isi Modul

1. SingleChildScrollView: Halaman yang Bisa Digulir
2. ListView, ListView.builder & ListView.separated
3. Ragam ListTile
4. Swipe to Delete dengan Dismissible
5. Pull-to-Refresh dengan RefreshIndicator
6. GridView.count & GridView.builder
7. Loading, Error & Empty State
8. Infinite Scroll dengan ScrollController
9. Pencarian & Filter Daftar
10. CustomScrollView, SliverAppBar, SliverList & SliverGrid
11. Studi Kasus: Daftar Mata Kuliah
12. Tugas Praktikum
13. Kuis Pemahaman & Kunci Jawaban

1. SingleChildScrollView: Halaman yang Bisa Digulir

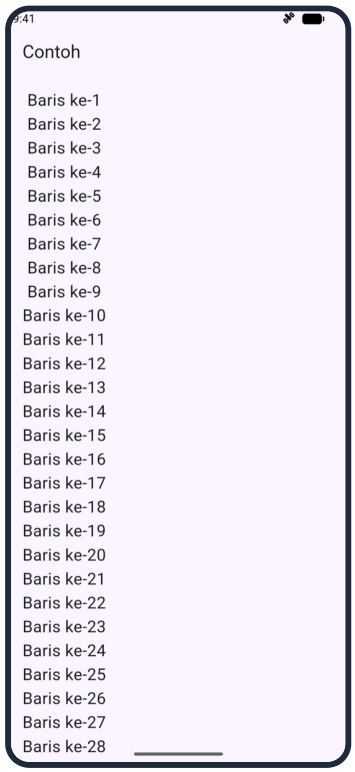
Layar HP itu pendek. Begitu isi halaman lebih tinggi dari layar, `Column` akan menampilkan garis kuning-hitam bertuliskan *"BOTTOM OVERFLOWED BY ... PIXELS"*. Solusi paling sederhana adalah `SingleChildScrollView`.

Konsep

- `SingleChildScrollView` membuat **satu** anak (biasanya `Column`) bisa digulir.
- Semua isi tetap dibuat sekaligus. Karena itu cocok untuk halaman yang isinya **terbatas**: profil, form, detail. Untuk ratusan data, gunakan `ListView.builder` (section berikutnya).
- `scrollDirection: Axis.horizontal` membuatnya bisa digulir ke samping, misalnya untuk baris chip.

Contoh kecil

```
// Column biasa + konten panjang = error "BOTTOM OVERFLOWED BY ... PIXELS"
// Solusi: bungkus Column dengan SingleChildScrollView
body: SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      for (var i = 1; i <= 30; i++) Text('Baris ke-$i', style: const TextStyle(fontSize: 20)),
    ],
  ),
),
```



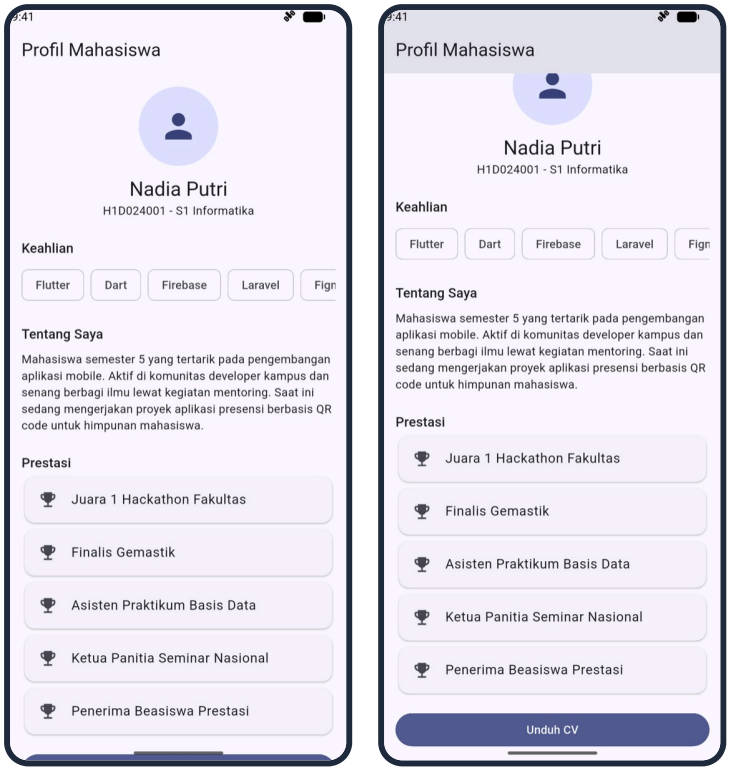
30 baris teks yang bisa digulir tanpa overflow

Contoh lengkap: halaman profil mahasiswa

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ProfilMahasiswaScreen(),
    );
  }
}

class ProfilMahasiswaScreen extends StatelessWidget {
  const ProfilMahasiswaScreen({super.key});

  static const _prestasi = [
    'Juara 1 Hackathon Fakultas',
    'Finalis Gemastik',
    'Asisten Praktikum Basis Data',
    'Ketua Panitia Seminar Nasional',
    'Penerima Beasiswa Prestasi',
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Profil Mahasiswa')),
      // Seluruh halaman bisa digulir vertikal
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Center(
              child: CircleAvatar(radius: 48, child: Icon(Icons.person, size: 48)),
            ),
            const SizedBox(height: 12),
            Center(
              child: Text('Nadia Putri', style: Theme.of(context).textTheme.headlineSmall),
            ),
            const Center(child: Text('H1D024001 - S1 Informatika')),
            const SizedBox(height: 24),

            Text('Keahlian', style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            // Baris chip yang bisa digulir HORIZONTAL
```

```
            SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: Row(
                children: [
                  for (final k in ['Flutter', 'Dart', 'Firebase', 'Laravel', 'Figma', 'Git', 'SQL'])
                    Padding(
                      padding: const EdgeInsets.only(right: 8),
                      child: Chip(label: Text(k)),
                    ),
                ],
              ),
            ),
            const SizedBox(height: 24),

            Text('Tentang Saya', style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            const Text(
              'Mahasiswa semester 5 yang tertarik pada pengembangan aplikasi mobile. '
              'Aktif di komunitas developer kampus dan senang berbagi ilmu lewat '
              'kegiatan mentoring. Saat ini sedang mengerjakan proyek aplikasi '
              'presensi berbasis QR code untuk himpunan mahasiswa.',
            ),
            const SizedBox(height: 24),

            Text('Prestasi', style: Theme.of(context).textTheme.titleMedium),
            for (final p in _prestasi)
              Card(
                child: ListTile(leading: const Icon(Icons.emoji_events), title: Text(p)),
              ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: () {}, child: const Text('Unduh CV')),
            ),
          ],
        ),
      ),
    );
  }
}
```



Bagian atas halaman profil

Setelah digulir ke bawah

Penjelasan kode:

- `SingleChildScrollView` luar membuat seluruh halaman bisa digulir vertikal. `padding` bisa langsung diberikan padanya.
- `SingleChildScrollView` dalam memakai `scrollDirection: Axis.horizontal` untuk baris chip keahlian yang lebih lebar dari layar.
- Daftar prestasi dibuat dengan `for` di dalam `children`. Karena hanya 5 item, ini aman; tidak perlu ListView.

> ⚠️ **Kesalahan umum**
>
> - **Memakai `Expanded` atau `Spacer` di dalam Column yang dibungkus SingleChildScrollView.** Tinggi area gulir tidak terbatas sehingga Expanded tidak tahu harus seberapa besar, dan muncul error.
> - **Menaruh ListView di dalam SingleChildScrollView.** Dua area gulir bertumpuk membuat error "unbounded height". Pilih salah satu.
> - **Memakai SingleChildScrollView untuk ribuan data.** Semua item dibuat sekaligus dan aplikasi menjadi berat.

2. ListView, ListView.builder & ListView.separated

`ListView` adalah widget daftar yang bisa digulir. Ada tiga cara membuatnya, dan memilih yang tepat sangat berpengaruh pada kelancaran aplikasi.

ListView (children)

Membuat **seluruh item sekaligus** saat layar dibuka. Jika ada 1.000 data mahasiswa, Flutter membuat 1.000 widget meskipun hanya 8 yang terlihat. Cocok hanya untuk daftar pendek yang tetap, misalnya menu pengaturan.

ListView.builder (lazy)

Hanya membuat item yang **sedang terlihat di layar** ditambah sedikit cadangan. Item yang keluar layar dibuang. 100.000 data pun tetap ringan. Gunakan untuk data dari database atau API.

1. ListView biasa

```
body: ListView(
  children: const [
    ListTile(title: Text('Senin - Algoritma')),
    ListTile(title: Text('Selasa - Basis Data')),
    ListTile(title: Text('Rabu - Pemrograman Mobile')),
    ListTile(title: Text('Kamis - Jaringan Komputer')),
  ],
),
```

ListView biasa dengan empat ListTile

2. ListView.builder

Dua parameter wajib: `itemCount` (jumlah item) dan `itemBuilder` (fungsi yang membuat widget untuk posisi `index`).

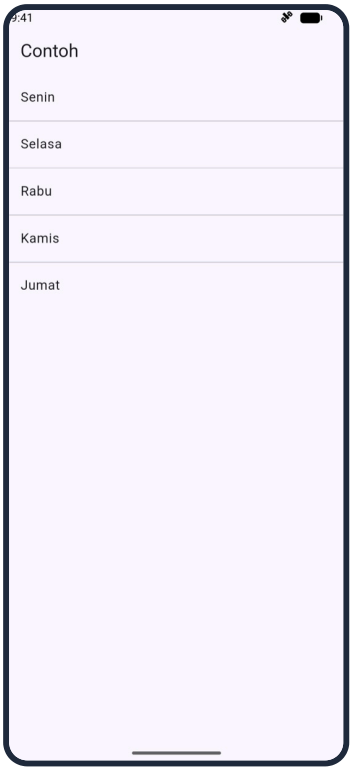
```
final mahasiswa = List.generate(1000, (i) => 'Mahasiswa ke-${i + 1}');

return Scaffold(
  body: ListView.builder(
    itemCount: mahasiswa.length, // jumlah item
    itemBuilder: (context, index) {
      // Dipanggil HANYA untuk item yang akan tampil di layar
      return ListTile(
        leading: CircleAvatar(child: Text('${index + 1}')),
        title: Text(mahasiswa[index]),
      );
    },
  ),
);
```

ListView.builder dengan 1000 item,
hanya yang terlihat yang dibuat

3. ListView.separated

Sama seperti builder, ditambah `separatorBuilder` untuk membuat pemisah di antara item.

```
body: ListView.separated(
  itemCount: hari.length,
  itemBuilder: (context, index) => ListTile(title: Text(hari[index])),
  // Pemisah dibuat DI ANTARA item (jumlahnya itemCount - 1)
  separatorBuilder: (context, index) => const Divider(height: 1),
),
```



Garis pemisah di antara item

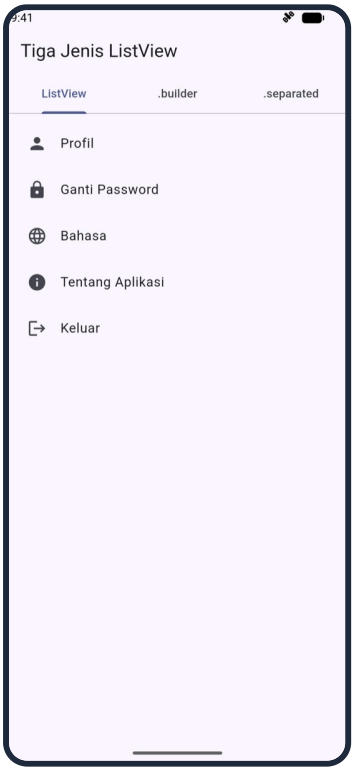
Contoh lengkap: tiga jenis ListView dalam tab

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PerbandinganListScreen(),
    );
  }
}

class PerbandinganListScreen extends StatelessWidget {
  const PerbandinganListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // DefaultTabController: tiga tab untuk tiga jenis ListView
    return DefaultTabController(
      length: 3,
      child: Scaffold(
```

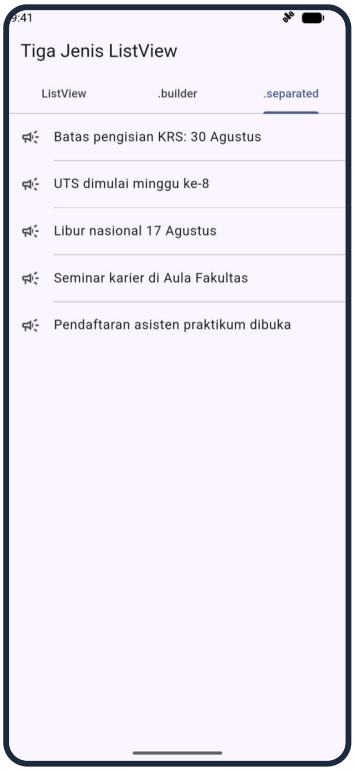
```
        appBar: AppBar(
          title: const Text('Tiga Jenis ListView'),
          bottom: const TabBar(
            tabs: [
              Tab(text: 'ListView'),
              Tab(text: '.builder'),
              Tab(text: '.separated'),
            ],
          ),
        ),
        body: const TabBarView(
          children: [ListBiasa(), ListBuilder(), ListSeparated()],
        ),
      ),
    );
  }
}

// 1. ListView biasa: cocok untuk item SEDIKIT dan TETAP
class ListBiasa extends StatelessWidget {
  const ListBiasa({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(8),
      children: const [
        ListTile(leading: Icon(Icons.person), title: Text('Profil')),
        ListTile(leading: Icon(Icons.lock), title: Text('Ganti Password')),
        ListTile(leading: Icon(Icons.language), title: Text('Bahasa')),
        ListTile(leading: Icon(Icons.info), title: Text('Tentang Aplikasi')),
        ListTile(leading: Icon(Icons.logout), title: Text('Keluar')),
      ],
    );
  }
}

// 2. ListView.builder: cocok untuk data BANYAK / dinamis
class ListBuilder extends StatelessWidget {
  const ListBuilder({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: 10000,
      itemBuilder: (context, index) {
        final nim = 'H1D0240${(index + 1).toString().padLeft(5, '0')}';
        return ListTile(
          leading: CircleAvatar(child: Text('${(index % 99) + 1}')),
          title: Text('Mahasiswa #${index + 1}'),
          subtitle: Text(nim),
        );
      },
    );
  }
}

// 3. ListView.separated: builder + pemisah otomatis
class ListSeparated extends StatelessWidget {
```

```dart
  const ListSeparated({super.key});

  static const _pengumuman = [
    'Batas pengisian KRS: 30 Agustus',
    'UTS dimulai minggu ke-8',
    'Libur nasional 17 Agustus',
    'Seminar karier di Aula Fakultas',
    'Pendaftaran asisten praktikum dibuka',
  ];

  @override
  Widget build(BuildContext context) {
    return ListView.separated(
      itemCount: _pengumuman.length,
      itemBuilder: (context, index) => ListTile(
        leading: const Icon(Icons.campaign_outlined),
        title: Text(_pengumuman[index]),
      ),
      separatorBuilder: (context, index) => const Divider(height: 1, indent: 56),
    );
  }
}
```



Tab ListView: menu pengaturan

Tab .builder: 10.000 mahasiswa tetap lancar



Tab .separated: pengumuman dengan pemisah

Penjelasan kode:

- `DefaultTabController` + `TabBar` + `TabBarView` membuat tiga tab. (Tab dibahas lebih lanjut di Modul 07.)
- Tab **ListView** berisi menu yang tetap dan pendek, jadi `children` biasa sudah tepat.

- Tab **.builder** berisi 10.000 item. Coba gulir cepat: tetap lancar karena hanya sekitar 10-15 ListTile yang benar-benar ada di memori.
- Tab **.separated** memakai `Divider(indent: 56)` agar garis dimulai sejajar teks, bukan ikon.

Situasi	Pilihan
Kurang dari ~20 item tetap (menu, pengaturan)	`ListView(children: ...)`
Data dari list/database/API, jumlah tidak pasti	`ListView.builder`
Seperti builder, perlu garis/jarak di antara item	`ListView.separated`

⚠️ **Kesalahan umum**

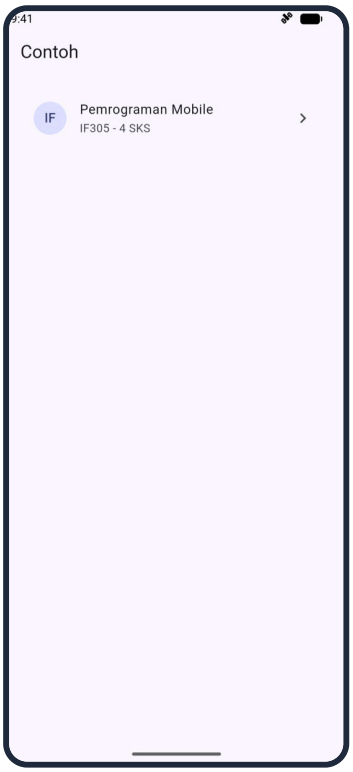
- **Error "Vertical viewport was given unbounded height".** Terjadi saat ListView diletakkan langsung di dalam `Column`. Bungkus ListView dengan `Expanded`.
- **Lupa `itemCount`.** Builder akan meminta item tanpa batas dan terjadi `RangeError` saat index melebihi panjang list.
- **Melakukan pekerjaan berat di `itemBuilder`** (misalnya mengurutkan seluruh list). itemBuilder dipanggil sangat sering saat menggulir; siapkan data di luar.

3. Ragam ListTile

`ListTile` adalah satu baris daftar siap pakai dengan slot `leading` (kiri), `title`, `subtitle`, dan `trailing` (kanan). Hampir semua aplikasi daftar (kontak, chat, pengaturan) bisa dibuat dengannya.

Contoh kecil

```
ListTile(
  leading: const CircleAvatar(child: Text('IF')),   // kiri
  title: const Text('Pemrograman Mobile'),           // judul
  subtitle: const Text('IF305 - 4 SKS'),             // keterangan
  trailing: const Icon(Icons.chevron_right),         // kanan
  onTap: () => debugPrint('Buka detail'),            // seluruh baris bisa ditekan
),
```



ListTile dengan avatar, judul, subjudul, dan ikon panah

Contoh lengkap: sepuluh ragam ListTile

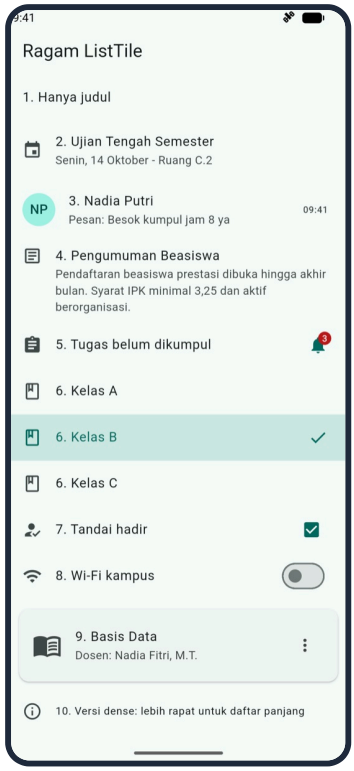
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
```

```dart
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const KatalogListTileScreen(),
    );
  }
}

class KatalogListTileScreen extends StatefulWidget {
  const KatalogListTileScreen({super.key});

  @override
  State<KatalogListTileScreen> createState() => _KatalogListTileScreenState();
}

class _KatalogListTileScreenState extends State<KatalogListTileScreen> {
  int _terpilih = 1;
  bool _hadir = true;
  bool _wifi = false;

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(title: const Text('Ragam ListTile')),
      body: ListView(
        children: [
          // 1. Paling dasar
          const ListTile(title: Text('1. Hanya judul')),

          // 2. Ikon + subjudul
          const ListTile(
            leading: Icon(Icons.event),
            title: Text('2. Ujian Tengah Semester'),
            subtitle: Text('Senin, 14 Oktober - Ruang C.2'),
          ),

          // 3. Avatar + trailing teks + bisa ditekan
          ListTile(
            leading: CircleAvatar(
              backgroundColor: warna.primaryContainer,
              child: const Text('NP'),
            ),
            title: const Text('3. Nadia Putri'),
            subtitle: const Text('Pesan: Besok kumpul jam 8 ya'),
            trailing: const Text('09:41'),
            onTap: () {},
          ),

          // 4. Tiga baris
          const ListTile(
            isThreeLine: true,
            leading: Icon(Icons.article_outlined),
            title: Text('4. Pengumuman Beasiswa'),
```

```dart
    subtitle: Text(
      'Pendaftaran beasiswa prestasi dibuka hingga akhir bulan. '
      'Syarat IPK minimal 3,25 dan aktif berorganisasi.',
    ),
  ),

  // 5. Badge di trailing
  ListTile(
    leading: const Icon(Icons.assignment),
    title: const Text('5. Tugas belum dikumpul'),
    trailing: Badge(
      label: const Text('3'),
      child: Icon(Icons.notifications, color: warna.primary),
    ),
  ),

  // 6. Pilihan aktif (selected)
  for (var i = 0; i < 3; i++)
    ListTile(
      selected: _terpilih == i,
      selectedTileColor: warna.secondaryContainer,
      leading: const Icon(Icons.class_outlined),
      title: Text('6. Kelas ${String.fromCharCode(65 + i)}'),
      trailing: _terpilih == i ? const Icon(Icons.check) : null,
      onTap: () => setState(() => _terpilih = i),
    ),

  // 7. Versi siap pakai dengan checkbox / switch
  CheckboxListTile(
    secondary: const Icon(Icons.how_to_reg),
    title: const Text('7. Tandai hadir'),
    value: _hadir,
    onChanged: (v) => setState(() => _hadir = v ?? false),
  ),
  SwitchListTile(
    secondary: const Icon(Icons.wifi),
    title: const Text('8. Wi-Fi kampus'),
    value: _wifi,
    onChanged: (v) => setState(() => _wifi = v),
  ),

  // 9. Di dalam Card agar terlihat seperti kartu
  Card(
    margin: const EdgeInsets.all(12),
    child: ListTile(
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      leading: const Icon(Icons.menu_book, size: 36),
      title: const Text('9. Basis Data'),
      subtitle: const Text('Dosen: Nadia Fitri, M.T.'),
      trailing: IconButton(
        icon: const Icon(Icons.more_vert),
        onPressed: () {},
      ),
    ),
  ),

  // 10. Rapat (dense)
  const ListTile(
    dense: true,
```

```
            leading: Icon(Icons.info_outline),
            title: Text('10. Versi dense: lebih rapat untuk daftar panjang'),
          ),
        ],
      ),
    );
  }
}
```



Sepuluh ragam ListTile dalam satu layar

Penjelasan kode:

- **Nomor 4:** `isThreeLine: true` memberi ruang untuk subjudul dua baris.
- **Nomor 5:** `Badge` menampilkan angka kecil di atas ikon, cocok untuk jumlah notifikasi.
- **Nomor 6:** `selected` mengubah warna teks/ikon, dan `selectedTileColor` mengubah warna latar baris terpilih.
- **Nomor 7-8:** `CheckboxListTile` dan `SwitchListTile` sudah dibahas di Modul 05. Parameter `secondary` menaruh ikon di sisi berlawanan dari kontrol.
- **Nomor 9:** ListTile di dalam `Card` tampil seperti kartu. `trailing` bisa berisi tombol.
- **Nomor 10:** `dense: true` membuat baris lebih rapat.

⚠️ **Kesalahan umum**

- **Memberi `trailing` berupa `Row` tanpa `mainAxisSize: MainAxisSize.min`.** Row mengambil seluruh lebar dan terjadi error layout.
- **Memasang ListTile di dalam `Row`.** ListTile butuh lebar yang jelas; bungkus dengan `Expanded`.
- **Membungkus ListTile dengan InkWell/GestureDetector.** Tidak perlu, cukup isi `onTap`.

4. Swipe to Delete dengan Dismissible

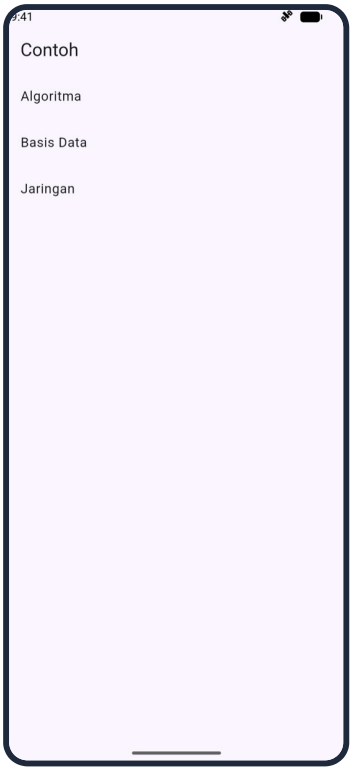
Menggeser baris ke kiri untuk menghapus (*swipe to delete*) adalah gestur yang sudah akrab bagi pengguna HP. Di Flutter, gestur ini dibuat dengan membungkus item daftar memakai `Dismissible`.

Konsep

- `key` : **wajib** dan harus unik untuk setiap item, biasanya dari id data: `ValueKey(item.id)`.
- `background` : tampilan di balik item saat digeser ke kanan (atau ke dua arah jika `secondaryBackground` tidak diisi). `secondaryBackground` : tampilan saat digeser ke kiri.
- `direction` : membatasi arah geser.
- `confirmDismiss` : fungsi `async` yang dipanggil *sebelum* item hilang. Kembalikan `false` untuk membatalkan.
- `onDismissed` : dipanggil setelah item hilang dari layar. Di sini data **wajib** dihapus dari list.

Contoh kecil

```
itemBuilder: (context, index) {
  final item = _items[index];
  return Dismissible(
    key: ValueKey(item), // WAJIB unik dan stabil untuk tiap item
    direction: DismissDirection.endToStart, // hanya geser ke kiri
    background: Container(
      color: Colors.red,
      alignment: Alignment.centerRight,
      padding: const EdgeInsets.only(right: 20),
      child: const Icon(Icons.delete, color: Colors.white),
    ),
    onDismissed: (direction) {
      // WAJIB hapus data dari list, kalau tidak akan error
      setState(() => _items.removeAt(index));
    },
    child: ListTile(title: Text(item)),
  );
},
```



Daftar sederhana; setiap baris bisa digeser ke kiri untuk dihapus

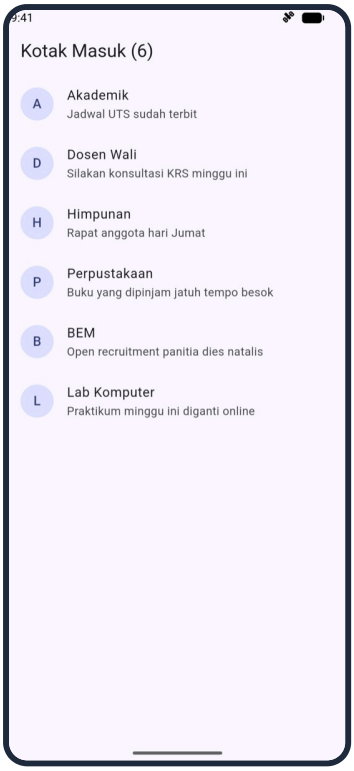
Contoh lengkap: kotak masuk dengan hapus, urungkan, dan arsip

`lib/main.dart`

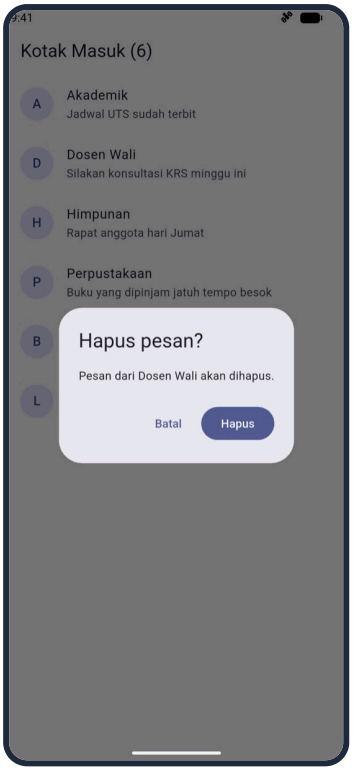
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KotakMasukScreen(),
    );
  }
}

class Pesan {
  final int id;
  final String pengirim;
  final String isi;
  bool diarsipkan;

  Pesan(this.id, this.pengirim, this.isi, {this.diarsipkan = false});
}
```

```dart
class KotakMasukScreen extends StatefulWidget {
  const KotakMasukScreen({super.key});

  @override
  State<KotakMasukScreen> createState() => _KotakMasukScreenState();
}

class _KotakMasukScreenState extends State<KotakMasukScreen> {
  final List<Pesan> _pesan = [
    Pesan(1, 'Akademik', 'Jadwal UTS sudah terbit'),
    Pesan(2, 'Dosen Wali', 'Silakan konsultasi KRS minggu ini'),
    Pesan(3, 'Himpunan', 'Rapat anggota hari Jumat'),
    Pesan(4, 'Perpustakaan', 'Buku yang dipinjam jatuh tempo besok'),
    Pesan(5, 'BEM', 'Open recruitment panitia dies natalis'),
    Pesan(6, 'Lab Komputer', 'Praktikum minggu ini diganti online'),
  ];

  void _hapus(int index) {
    final terhapus = _pesan[index];
    setState(() => _pesan.removeAt(index));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('Pesan dari ${terhapus.pengirim} dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            // Kembalikan ke posisi semula
            onPressed: () => setState(() => _pesan.insert(index, terhapus)),
          ),
        ),
      );
  }

  Future<bool> _konfirmasiHapus(Pesan p) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus pesan?'),
        content: Text('Pesan dari ${p.pengirim} akan dihapus.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );
    return yakin ?? false;
  }

  @override
  Widget build(BuildContext context) {
```

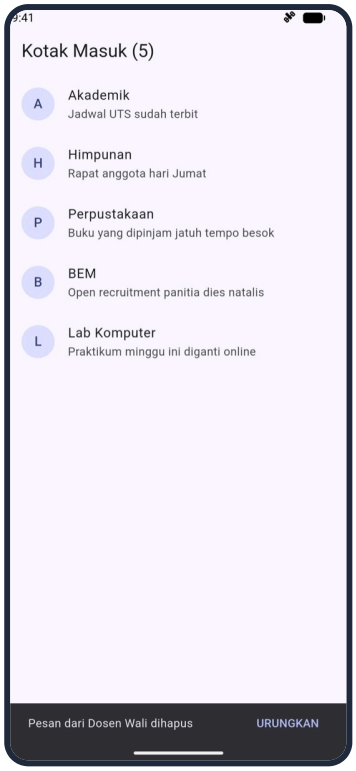
```
    return Scaffold(
      appBar: AppBar(title: Text('Kotak Masuk (${_pesan.length})')),
      body: _pesan.isEmpty
          ? const Center(child: Text('Kotak masuk kosong'))
          : ListView.builder(
              itemCount: _pesan.length,
              itemBuilder: (context, index) {
                final p = _pesan[index];
                return Dismissible(
                  // Pakai id unik dari data, BUKAN index
                  key: ValueKey(p.id),
                  // Geser ke kanan -> arsip (latar hijau)
                  background: Container(
                    color: Colors.green,
                    alignment: Alignment.centerLeft,
                    padding: const EdgeInsets.only(left: 24),
                    child: const Icon(Icons.archive, color: Colors.white),
                  ),
                  // Geser ke kiri -> hapus (latar merah)
                  secondaryBackground: Container(
                    color: Colors.red,
                    alignment: Alignment.centerRight,
                    padding: const EdgeInsets.only(right: 24),
                    child: const Icon(Icons.delete, color: Colors.white),
                  ),
                  // Dipanggil SEBELUM item hilang. Return false = batal.
                  confirmDismiss: (direction) async {
                    if (direction == DismissDirection.startToEnd) {
                      // Arsip: ubah status saja, item tidak dihapus dari list
                      setState(() => p.diarsipkan = !p.diarsipkan);
                      return false;
                    }
                    return _konfirmasiHapus(p);
                  },
                  onDismissed: (_) => _hapus(index),
                  child: ListTile(
                    leading: CircleAvatar(child: Text(p.pengirim[0])),
                    title: Text(p.pengirim),
                    subtitle: Text(p.isi),
                    trailing: p.diarsipkan
                        ? const Icon(Icons.archive, color: Colors.green)
                        : null,
                  ),
                );
              },
            ),
    );
  }
}
```



Daftar pesan awal



Pesan digeser ke kiri: dialog konfirmasi muncul



Setelah dihapus: SnackBar dengan tombol URUNGKAN

Penjelasan kode:

- `key: ValueKey(p.id)` memakai id unik dari data. Jika memakai `index`, setelah satu item dihapus semua index bergeser dan Flutter bisa salah mengenali item.
- Geser ke **kanan** (`startToEnd`) hanya mengubah status arsip lalu `return false`, sehingga item kembali ke tempatnya.
- Geser ke **kiri** memanggil `_konfirmasiHapus` yang menampilkan dialog. Hasil dialog (true/false) langsung menjadi hasil `confirmDismiss`.
- `_hapus` menyimpan item dan posisinya sebelum menghapus, lalu tombol URUNGKAN memasukkannya kembali dengan `insert`.

> ⚠️ **Kesalahan umum**
>
> - **Error "A dismissed Dismissible widget is still part of the tree".** Terjadi jika `onDismissed` tidak menghapus data dari list. Selalu `removeAt` / `remove` lalu `setState`.
> - **Key memakai index atau nama yang bisa kembar.** Gunakan id unik.
> - **Menghapus permanen ke database langsung di `onDismissed` padahal ada tombol Urungkan.** Pada aplikasi nyata, tunda penghapusan permanen sampai SnackBar hilang, atau kembalikan data ke database saat diurungkan.

5. Pull-to-Refresh dengan RefreshIndicator

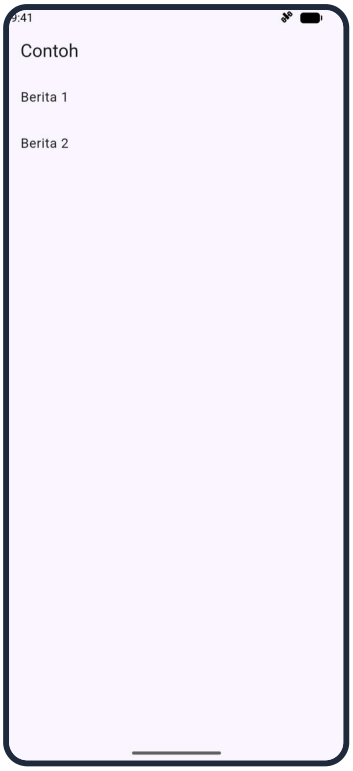
Menarik daftar ke bawah untuk memuat ulang (*pull to refresh*) dibuat dengan membungkus widget gulir memakai `RefreshIndicator`.

Konsep

- `onRefresh` harus berupa fungsi yang mengembalikan `Future`. Lingkaran loading terus berputar sampai Future selesai.
- Anak RefreshIndicator harus widget yang bisa digulir (ListView, GridView, CustomScrollView).
- Jika item sedikit dan tidak memenuhi layar, daftar tidak bisa ditarik. Tambahkan `physics: const AlwaysScrollableScrollPhysics()`.

Contoh kecil

```
// onRefresh WAJIB mengembalikan Future: indikator berputar sampai Future selesai
Future<void> _muatUlang() async {
  await Future.delayed(const Duration(seconds: 1)); // pura-pura ambil dari server
  setState(() => _berita = ['Berita terbaru', ..._berita]);
}

@override
Widget build(BuildContext context) {
  return RefreshIndicator(
    onRefresh: _muatUlang,
    child: ListView.builder(
      // agar tetap bisa ditarik walau item sedikit
      physics: const AlwaysScrollableScrollPhysics(),
      itemCount: _berita.length,
      itemBuilder: (context, index) => ListTile(title: Text(_berita[index])),
    ),
  );
}
```



Daftar berita di dalam RefreshIndicator

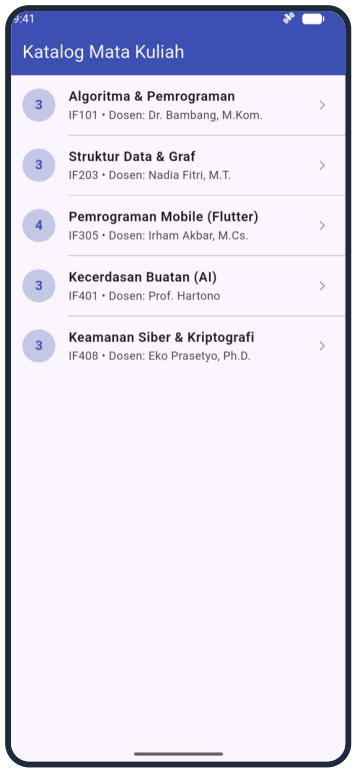
Contoh lengkap: katalog mata kuliah dengan pull-to-refresh

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMataKuliahScreen(),
    );
  }
}


class DaftarMataKuliahScreen extends StatefulWidget {
  const DaftarMataKuliahScreen({super.key});

  @override
  State<DaftarMataKuliahScreen> createState() => _DaftarMataKuliahScreenState();
}

class _DaftarMataKuliahScreenState extends State<DaftarMataKuliahScreen> {
```

```dart
  // Simulasi data dari database kampus
  final List<Map<String, dynamic>> _daftarMatkul = [
    {"kode": "IF101", "nama": "Algoritma & Pemrograman", "sks": 3, "dosen": "Dr. Bambang, M.Kom."},
    {"kode": "IF203", "nama": "Struktur Data & Graf", "sks": 3, "dosen": "Nadia Fitri, M.T."},
    {"kode": "IF305", "nama": "Pemrograman Mobile (Flutter)", "sks": 4, "dosen": "Irham Akbar,
M.Cs."},
    {"kode": "IF401", "nama": "Kecerdasan Buatan (AI)", "sks": 3, "dosen": "Prof. Hartono"},
    {"kode": "IF408", "nama": "Keamanan Siber & Kriptografi", "sks": 3, "dosen": "Eko Prasetyo,
Ph.D."},
  ];

  // Fungsi simulasi refresh data dari server
  Future<void> _refreshData() async {
    await Future.delayed(const Duration(seconds: 2));
    setState(() {
      _daftarMatkul.shuffle(); // Acak urutan sebagai tanda data baru
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Katalog Mata Kuliah"),
        backgroundColor: Colors.indigo,
        foregroundColor: Colors.white,
      ),
      // RefreshIndicator menambahkan gestur tarik ke bawah untuk refresh
      body: RefreshIndicator(
        onRefresh: _refreshData,
        // AlwaysScrollableScrollPhysics: tarik-ke-bawah tetap bisa walau item sedikit
        child: ListView.separated(
          physics: const AlwaysScrollableScrollPhysics(),
          itemCount: _daftarMatkul.length,
          separatorBuilder: (context, index) => const Divider(height: 1, indent: 72),
          itemBuilder: (context, index) {
            final matkul = _daftarMatkul[index];
            return ListTile(
              leading: CircleAvatar(
                backgroundColor: Colors.indigo.shade100,
                child: Text(
                  "${matkul['sks']}",
                  style: const TextStyle(color: Colors.indigo, fontWeight: FontWeight.bold),
                ),
              ),
              title: Text(
                matkul['nama'],
                style: const TextStyle(fontWeight: FontWeight.w600),
              ),
              subtitle: Text("${matkul['kode']} • Dosen: ${matkul['dosen']}"),
              trailing: const Icon(Icons.arrow_forward_ios, size: 14, color: Colors.grey),
              onTap: () {
                // Aksi saat item baris ditekan
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text("Membuka ${matkul['nama']}")),
                );
              },
            );
          },
```

```
        ),
      ),
    );
  }
}
```



Katalog mata kuliah dengan ListView.separated. Tarik ke bawah untuk memuat ulang

Penjelasan kode:

- `_refreshData` menunggu 2 detik (pura-pura mengambil data dari server), lalu mengacak urutan sebagai tanda data berubah.
- `ListView.separated` dipakai agar setiap mata kuliah dipisah garis tipis.
- `CircleAvatar` di `leading` menampilkan jumlah SKS.

> 💡 **Tips**
>
> Pada Modul 09 (REST API), isi `onRefresh` akan diganti dengan pemanggilan `http.get` sungguhan. Polanya sama persis: `await` ambil data, lalu `setState`.

⚠️ **Kesalahan umum**

- **onRefresh tidak `async` / tidak mengembalikan Future.** Indikator langsung hilang atau terjadi error tipe.
- **Lupa memeriksa `mounted`** setelah `await` pada data sungguhan; jika pengguna sudah keluar halaman, `setState` akan error.
- **RefreshIndicator dibungkuskan ke Column.** Ia harus langsung membungkus widget yang bisa digulir.

6. GridView.count & GridView.builder

Jika ListView menyusun item ke bawah, `GridView` menyusunnya dalam baris dan kolom. Cocok untuk menu ikon, galeri foto, atau katalog produk.

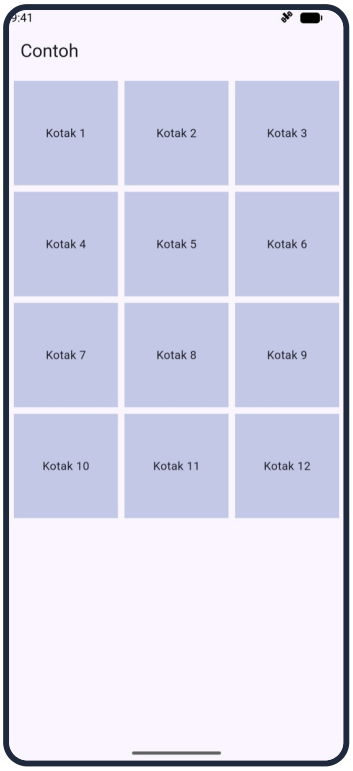
Konsep: dua cara utama

Konstruktor	Cara kerja	Cocok untuk
`GridView.count`	Jumlah kolom tetap (`crossAxisCount`), semua anak dibuat sekaligus	Menu dengan item sedikit
`GridView.builder`	Lazy seperti ListView.builder, tata letak diatur `gridDelegate`	Data banyak / dinamis

`childAspectRatio` adalah perbandingan lebar : tinggi tiap kotak. Nilai 1 = persegi, kurang dari 1 = lebih tinggi, lebih dari 1 = lebih lebar.

Contoh kecil: GridView.count

```
body: GridView.count(
  crossAxisCount: 3,       // 3 kolom
  mainAxisSpacing: 8,      // jarak vertikal antar kotak
  crossAxisSpacing: 8,     // jarak horizontal antar kotak
  padding: const EdgeInsets.all(8),
  children: [
    for (var i = 1; i <= 12; i++)
      Container(
        color: Colors.indigo.shade100,
        alignment: Alignment.center,
        child: Text('Kotak $i'),
      ),
  ],
),
```



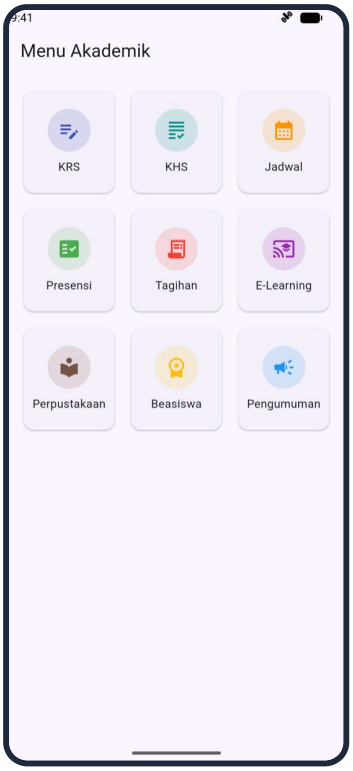
GridView tiga kolom berisi 12 kotak

Contoh lengkap 1: menu akademik dengan GridView.count

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const MenuAkademikScreen(),
    );
  }
}

class MenuItem {
  final String label;
  final IconData ikon;
  final Color warna;

  const MenuItem(this.label, this.ikon, this.warna);
}

class MenuAkademikScreen extends StatelessWidget {
```

```
  const MenuAkademikScreen({super.key});

  static const _menu = [
    MenuItem('KRS', Icons.edit_note, Colors.indigo),
    MenuItem('KHS', Icons.grading, Colors.teal),
    MenuItem('Jadwal', Icons.calendar_month, Colors.orange),
    MenuItem('Presensi', Icons.fact_check, Colors.green),
    MenuItem('Tagihan', Icons.receipt_long, Colors.red),
    MenuItem('E-Learning', Icons.cast_for_education, Colors.purple),
    MenuItem('Perpustakaan', Icons.local_library, Colors.brown),
    MenuItem('Beasiswa', Icons.workspace_premium, Colors.amber),
    MenuItem('Pengumuman', Icons.campaign, Colors.blue),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Menu Akademik')),
      body: GridView.count(
        crossAxisCount: 3,
        padding: const EdgeInsets.all(16),
        mainAxisSpacing: 12,
        crossAxisSpacing: 12,
        childAspectRatio: 0.9, // lebar : tinggi = 0.9 (sedikit lebih tinggi)
        children: [
          for (final m in _menu)
            Card(
              clipBehavior: Clip.antiAlias,
              child: InkWell(
                onTap: () => ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Membuka menu ${m.label}')),
                ),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    CircleAvatar(
                      radius: 26,
                      backgroundColor: m.warna.withValues(alpha: 0.15),
                      child: Icon(m.ikon, color: m.warna, size: 28),
                    ),
                    const SizedBox(height: 8),
                    Text(m.label, textAlign: TextAlign.center),
                  ],
                ),
              ),
            ),
        ],
      ),
    );
  }
}
```



Menu akademik 3 kolom dengan ikon berwarna

- Setiap menu berupa `Card` + `InkWell` sehingga ada efek riak saat ditekan.
- `m.warna.withValues(alpha: 0.15)` membuat latar ikon transparan 15%.

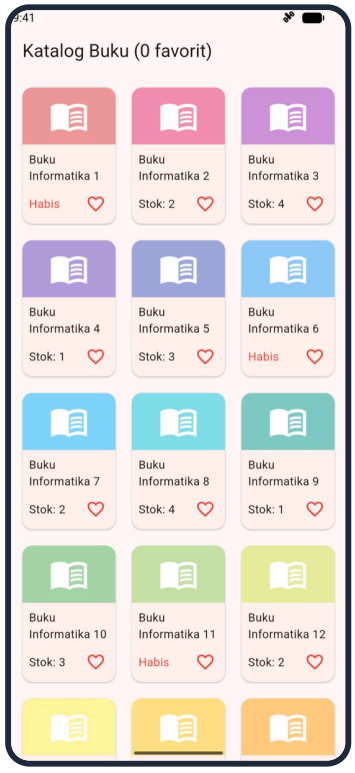
Contoh lengkap 2: katalog buku dengan GridView.builder

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const KatalogBukuScreen(),
    );
  }
}

class KatalogBukuScreen extends StatefulWidget {
  const KatalogBukuScreen({super.key});

  @override
```

```
  State<KatalogBukuScreen> createState() => _KatalogBukuScreenState();
}

class _KatalogBukuScreenState extends State<KatalogBukuScreen> {
  // 60 buku contoh, dibuat otomatis
  final _buku = List.generate(
    60,
    (i) => (judul: 'Buku Informatika ${i + 1}', stok: (i * 7) % 5),
  );
  final Set<int> _favorit = {};

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Katalog Buku (${_favorit.length} favorit)')),
      body: GridView.builder(
        padding: const EdgeInsets.all(12),
        itemCount: _buku.length,
        // Lebar maksimum tiap kotak 180 px: jumlah kolom menyesuaikan lebar layar
        gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
          maxCrossAxisExtent: 180,
          mainAxisSpacing: 12,
          crossAxisSpacing: 12,
          childAspectRatio: 0.7,
        ),
        itemBuilder: (context, index) {
          final b = _buku[index];
          final favorit = _favorit.contains(index);
          return Card(
            clipBehavior: Clip.antiAlias,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Expanded(
                  child: Container(
                    color: Colors.primaries[index % Colors.primaries.length].shade200,
                    child: const Icon(Icons.menu_book, size: 48, color: Colors.white),
                  ),
                ),
                Padding(
                  padding: const EdgeInsets.fromLTRB(8, 8, 8, 0),
                  child: Text(b.judul, maxLines: 2, overflow: TextOverflow.ellipsis),
                ),
                Row(
                  children: [
                    const SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        b.stok == 0 ? 'Habis' : 'Stok: ${b.stok}',
                        style: TextStyle(color: b.stok == 0 ? Colors.red : null),
                      ),
                    ),
                    IconButton(
                      icon: Icon(favorit ? Icons.favorite : Icons.favorite_border),
                      color: Colors.red,
                      onPressed: () => setState(() {
                        favorit ? _favorit.remove(index) : _favorit.add(index);
                      }),
                    ),
```

```
                ],
              ),
            ],
          ),
        );
      },
    ),
  );
 }
}
```



Katalog 60 buku dengan GridView.builder

Penjelasan kode:

- `SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent: 180)` artinya "setiap kotak maksimal 180 piksel". Di HP muncul 2 kolom, di tablet bisa 4-5 kolom secara otomatis. Inilah cara mudah membuat grid responsif.
- Alternatifnya, `SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 2)` memberi jumlah kolom tetap.
- Data buku dibuat dengan `List.generate` berisi *record* `(judul: ..., stok: ...)`.
- Favorit disimpan di `Set<int>` berisi index buku.

⚠️ **Kesalahan umum**

- **Garis kuning overflow di dalam kotak grid.** Tinggi kotak ditentukan oleh `childAspectRatio`, bukan oleh isinya. Kecilkan isi atau ubah rasio (misalnya 0.7).
- **GridView di dalam Column tanpa `Expanded`** : error unbounded height, sama seperti ListView.
- **Memakai GridView.count untuk ratusan item**: semua kotak dibuat sekaligus. Pakai builder.

7. Loading, Error & Empty State

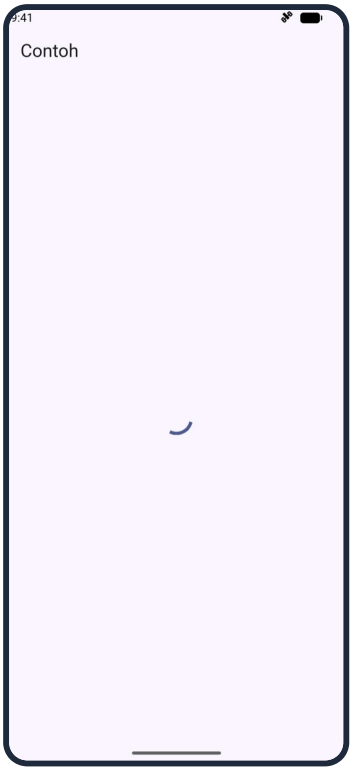
Daftar data jarang langsung tersedia. Data harus dimuat dulu, bisa gagal, atau ternyata kosong. Aplikasi yang baik menampilkan tampilan yang jelas untuk **setiap keadaan**, bukan layar putih kosong.

Konsep: empat keadaan

Keadaan	Yang ditampilkan
Loading	`CircularProgressIndicator` + teks "Memuat..."
Error	Ikon, pesan yang dimengerti pengguna, tombol "Coba Lagi"
Kosong (empty state)	Ikon, penjelasan kenapa kosong, ajakan bertindak
Ada data	ListView / GridView

Contoh kecil: pola operator ternary

```
// Pola umum: periksa keadaan dari yang paling "awal"
return Scaffold(
  body: sedangMemuat
      ? const Center(child: CircularProgressIndicator())       // 1. loading
      : data.isEmpty
          ? const Center(child: Text('Belum ada data'))        // 2. kosong
          : ListView.builder(                                  // 3. ada data
              itemCount: data.length,
              itemBuilder: (context, i) => ListTile(title: Text(data[i])),
            ),
);
```



Keadaan loading: lingkaran berputar di tengah layar

Untuk dua atau tiga keadaan, ternary bersarang masih terbaca. Untuk empat keadaan atau lebih, lebih rapi memakai `enum` + `switch` seperti contoh berikut.

Contoh lengkap: enum StatusMuat

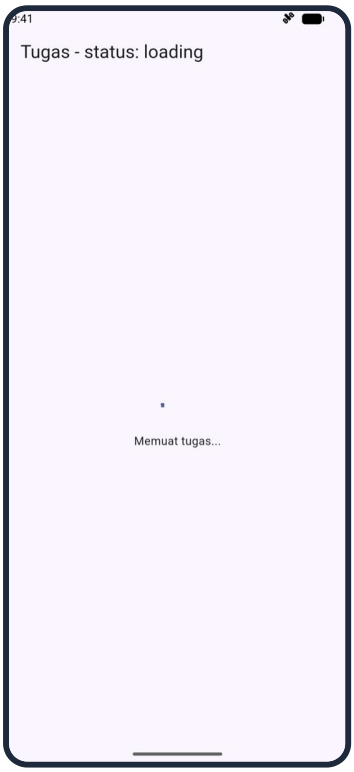
lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarTugasScreen(),
    );
  }
}

// Empat kemungkinan keadaan layar daftar
enum StatusMuat { loading, error, kosong, ada }

class DaftarTugasScreen extends StatefulWidget {
```

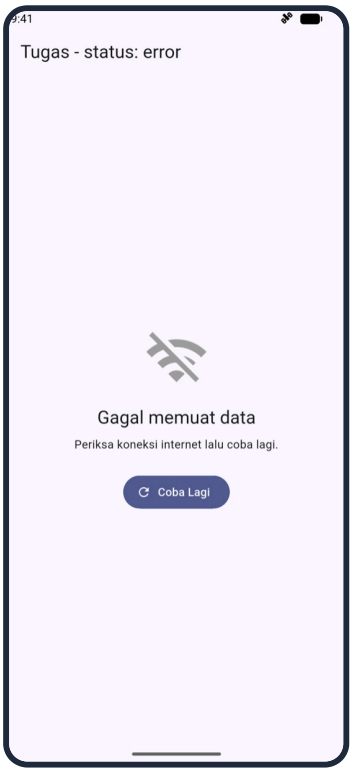
```dart
  const DaftarTugasScreen({super.key});

  @override
  State<DaftarTugasScreen> createState() => _DaftarTugasScreenState();
}

class _DaftarTugasScreenState extends State<DaftarTugasScreen> {
  StatusMuat _status = StatusMuat.loading;
  List<String> _tugas = [];
  int _percobaan = 0;

  @override
  void initState() {
    super.initState();
    _muatData();
  }

  // Simulasi mengambil data: percobaan ke-1 gagal, ke-2 kosong, ke-3 berhasil
  Future<void> _muatData() async {
    setState(() => _status = StatusMuat.loading);
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return;

    _percobaan++;
    setState(() {
      if (_percobaan == 1) {
        _status = StatusMuat.error;
      } else if (_percobaan == 2) {
        _tugas = [];
        _status = StatusMuat.kosong;
      } else {
        _tugas = ['Laporan Praktikum 3', 'Kuis Dart', 'Proyek UI Login'];
        _status = StatusMuat.ada;
      }
    });
  }

  Widget _buildIsi() {
    switch (_status) {
      case StatusMuat.loading:
        return const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              CircularProgressIndicator(),
              SizedBox(height: 16),
              Text('Memuat tugas...'),
            ],
          ),
        );
      case StatusMuat.error:
        return _PesanStatus(
          ikon: Icons.wifi_off,
          judul: 'Gagal memuat data',
          keterangan: 'Periksa koneksi internet lalu coba lagi.',
          tombol: FilledButton.icon(
            onPressed: _muatData,
            icon: const Icon(Icons.refresh),
            label: const Text('Coba Lagi'),
```

```
          ),
        );
      case StatusMuat.kosong:
        return _PesanStatus(
          ikon: Icons.inbox_outlined,
          judul: 'Belum ada tugas',
          keterangan: 'Tugas dari dosen akan muncul di sini.',
          tombol: OutlinedButton(onPressed: _muatData, child: const Text('Muat Ulang')),
        );
      case StatusMuat.ada:
        return ListView.builder(
          itemCount: _tugas.length,
          itemBuilder: (context, i) => ListTile(
            leading: const Icon(Icons.assignment_outlined),
            title: Text(_tugas[i]),
          ),
        );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Tugas - status: ${_status.name}')),
      body: _buildIsi(),
    );
  }
}

// Widget kecil yang bisa dipakai ulang untuk empty state & error state
class _PesanStatus extends StatelessWidget {
  final IconData ikon;
  final String judul;
  final String keterangan;
  final Widget tombol;

  const _PesanStatus({
    required this.ikon,
    required this.judul,
    required this.keterangan,
    required this.tombol,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(ikon, size: 80, color: Colors.grey),
            const SizedBox(height: 16),
            Text(judul, style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 8),
            Text(keterangan, textAlign: TextAlign.center),
            const SizedBox(height: 24),
            tombol,
          ],
```

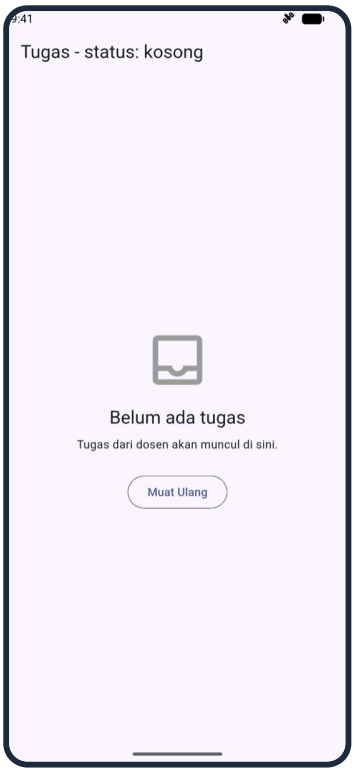
```
        ),
      ),
    );
  }
}
```



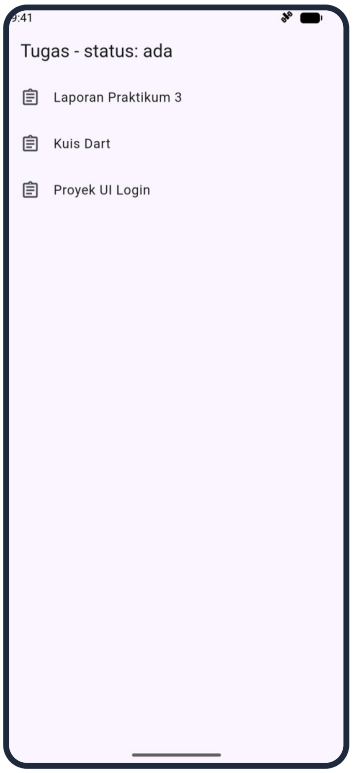
Loading saat pertama dibuka



Percobaan pertama gagal: error state



Setelah Coba Lagi: data kosong



Setelah Muat Ulang: data tampil

Penjelasan kode:

- `enum StatusMuat { loading, error, kosong, ada }` membuat semua keadaan tertulis jelas.
- `switch (_status)` di `_buildIsi` wajib menangani semua nilai enum. Jika kelak ditambah keadaan baru, compiler akan mengingatkan bagian yang belum ditangani.
- Widget `_PesanStatus` dipakai ulang untuk error dan kosong, cukup berbeda ikon, teks, dan tombolnya.
- `_muatData` selalu mengubah status ke loading dulu, lalu menunggu, lalu mengatur hasil. Pola ini juga akan dipakai di Modul 09 saat memanggil API.

> ⚠️ **Kesalahan umum**
>
> - **Hanya memeriksa `list.isEmpty`.** Saat masih loading, list juga kosong, sehingga pengguna sempat melihat tulisan "Belum ada data" padahal data sedang dimuat.
> - **Pesan error teknis** seperti "SocketException: Failed host lookup". Tampilkan bahasa manusia dan berikan tombol coba lagi.
> - **Lupa `if (!mounted) return;`** setelah `await`.

8. Infinite Scroll dengan ScrollController

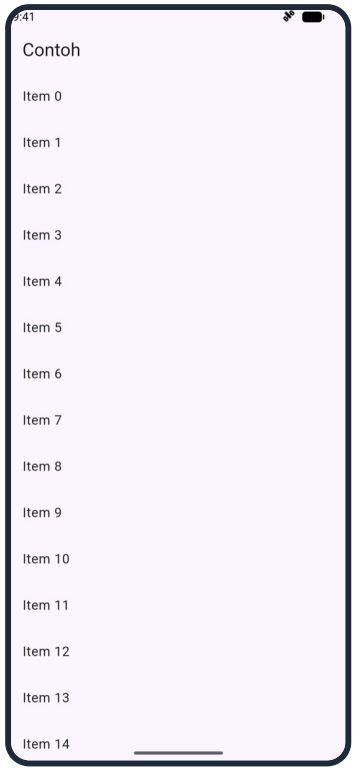
Aplikasi seperti forum atau media sosial tidak memuat ribuan data sekaligus. Data diambil per halaman (misalnya 15 item), dan halaman berikutnya dimuat saat pengguna mendekati akhir daftar. Teknik ini disebut **infinite scroll** atau *load more*.

Konsep: ScrollController

- `ScrollController` dihubungkan ke ListView lewat `controller:`. Sama seperti TextEditingController: dibuat di State, wajib di-dispose.
- `controller.position.pixels` = jarak yang sudah digulir; `maxScrollExtent` = jarak maksimum.
- `addListener` dipanggil setiap kali posisi gulir berubah.
- `animateTo(0, ...)` / `jumpTo(0)` menggulir ke posisi tertentu, misalnya kembali ke atas.

Contoh kecil

```dart
final _scrollController = ScrollController();

@override
void initState() {
  super.initState();
  _scrollController.addListener(() {
    final posisi = _scrollController.position;
    // pixels = jarak yang sudah digulir, maxScrollExtent = jarak maksimum
    if (posisi.pixels >= posisi.maxScrollExtent - 200) {
      debugPrint('Hampir sampai bawah -> muat data berikutnya');
    }
  });
}

@override
void dispose() {
  _scrollController.dispose(); // sama seperti TextEditingController
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return ListView.builder(
    controller: _scrollController, // hubungkan ke list
    itemCount: 50,
    itemBuilder: (context, i) => ListTile(title: Text('Item $i')),
  );
}
```



ListView yang terhubung dengan ScrollController

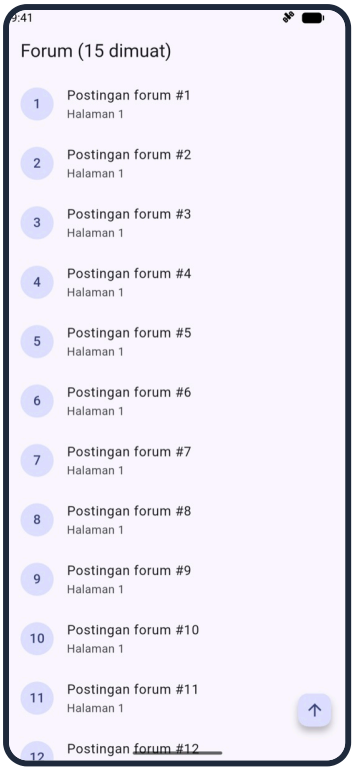
Contoh lengkap: forum dengan load more

lib/main.dart

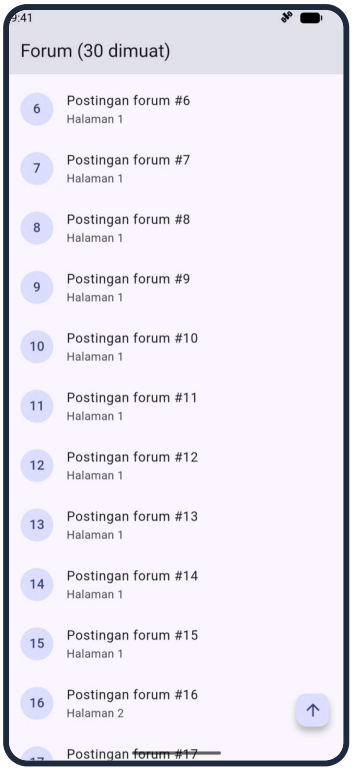
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ForumScreen(),
    );
  }
}

class ForumScreen extends StatefulWidget {
  const ForumScreen({super.key});

  @override
  State<ForumScreen> createState() => _ForumScreenState();
}

class _ForumScreenState extends State<ForumScreen> {
```

```dart
  static const _ukuranHalaman = 15;
  static const _totalDiServer = 70; // anggap server hanya punya 70 postingan

  final _scrollController = ScrollController();
  final List<String> _postingan = [];
  int _halaman = 0;
  bool _sedangMemuat = false;
  bool _masihAda = true;

  @override
  void initState() {
    super.initState();
    _muatHalamanBerikutnya();
    _scrollController.addListener(_cekPosisiScroll);
  }

  @override
  void dispose() {
    _scrollController.removeListener(_cekPosisiScroll);
    _scrollController.dispose();
    super.dispose();
  }

  void _cekPosisiScroll() {
    final posisi = _scrollController.position;
    if (posisi.pixels >= posisi.maxScrollExtent - 300) {
      _muatHalamanBerikutnya();
    }
  }

  // Simulasi API: GET /postingan?page=N
  Future<List<String>> _ambilDariServer(int halaman) async {
    await Future.delayed(const Duration(milliseconds: 1200));
    final mulai = halaman * _ukuranHalaman;
    final akhir = (mulai + _ukuranHalaman).clamp(0, _totalDiServer);
    return [for (var i = mulai; i < akhir; i++) 'Postingan forum #${i + 1}'];
  }

  Future<void> _muatHalamanBerikutnya() async {
    // Penjaga: jangan memuat dua kali bersamaan & berhenti jika data habis
    if (_sedangMemuat || !_masihAda) return;
    setState(() => _sedangMemuat = true);

    final baru = await _ambilDariServer(_halaman);
    if (!mounted) return;

    setState(() {
      _postingan.addAll(baru);
      _halaman++;
      _sedangMemuat = false;
      _masihAda = baru.length == _ukuranHalaman;
    });
  }

  Future<void> _refresh() async {
    setState(() {
      _postingan.clear();
      _halaman = 0;
      _masihAda = true;
```

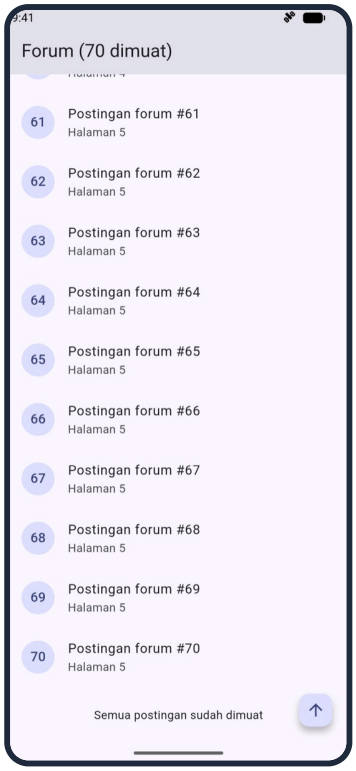
```
    });
    await _muatHalamanBerikutnya();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Forum (${_postingan.length} dimuat)')),
      body: RefreshIndicator(
        onRefresh: _refresh,
        child: ListView.builder(
          controller: _scrollController,
          physics: const AlwaysScrollableScrollPhysics(),
          // +1 untuk baris paling bawah (loading / "sudah habis")
          itemCount: _postingan.length + 1,
          itemBuilder: (context, index) {
            if (index == _postingan.length) {
              return Padding(
                padding: const EdgeInsets.all(24),
                child: Center(
                  child: _masihAda
                      ? const CircularProgressIndicator()
                      : const Text('Semua postingan sudah dimuat'),
                ),
              );
            }
            return ListTile(
              leading: CircleAvatar(child: Text('${index + 1}')),
              title: Text(_postingan[index]),
              subtitle: Text('Halaman ${index ~/ _ukuranHalaman + 1}'),
            );
          },
        ),
      ),
      floatingActionButton: FloatingActionButton.small(
        tooltip: 'Ke atas',
        // animateTo: gulir ke posisi tertentu dengan animasi
        onPressed: () => _scrollController.animateTo(
          0,
          duration: const Duration(milliseconds: 500),
          curve: Curves.easeOut,
        ),
        child: const Icon(Icons.arrow_upward),
      ),
    );
  }
}
```



15 postingan pertama dimuat



Setelah digulir ke bawah: halaman kedua otomatis dimuat (30 postingan)



Semua 70 postingan sudah dimuat

Penjelasan kode:

- `_cekPosisiScroll` memuat halaman berikutnya saat posisi tinggal 300 piksel dari bawah, sehingga data baru sudah siap sebelum pengguna benar-benar sampai di ujung.
- Dua penjaga di awal `_muatHalamanBerikutnya` sangat penting: `_sedangMemuat` mencegah permintaan ganda (listener terpanggil puluhan kali per detik), dan `_masihAda` menghentikan pemuatan jika server sudah tidak punya data.
- `itemCount: _postingan.length + 1` : item terakhir dipakai sebagai baris status (loading atau "sudah habis").
- Data dianggap habis jika halaman yang diterima kurang dari `_ukuranHalaman`.
- FAB memakai `animateTo(0)` untuk kembali ke atas, dan `_refresh` mereset semuanya ke halaman pertama.

> ⚠️ **Kesalahan umum**
>
> - **Tanpa penjaga `_sedangMemuat` :** halaman yang sama dimuat berkali-kali sehingga data dobel.
> - **Lupa `dispose` ScrollController dan `removeListener` .**
> - **Membaca `controller.position` sebelum ListView tampil.** Controller belum terpasang; periksa dengan `controller.hasClients` jika perlu.

9. Pencarian & Filter Daftar

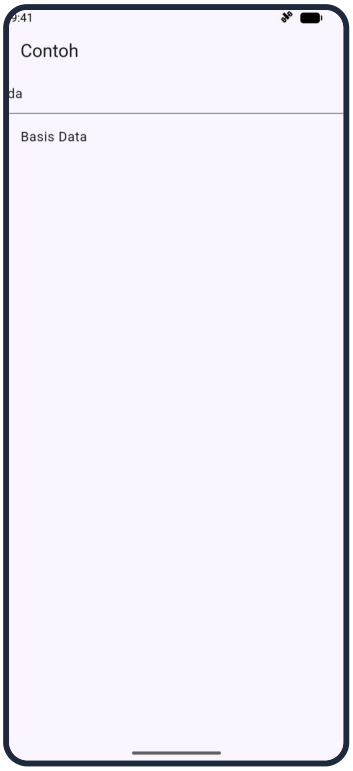
Fitur pencarian membuat daftar panjang mudah dipakai. Kuncinya sederhana: **jangan ubah data asli**. Simpan kata kunci di State, lalu hitung daftar yang ditampilkan dari data asli setiap kali `build`.

Konsep

1. `_semua`: data asli, tidak pernah dikurangi oleh pencarian.
2. `_kataKunci`: diperbarui oleh `onChanged` TextField + `setState`.
3. Daftar tampil = `_semua.where(...)` berdasarkan kata kunci.

Contoh kecil

```
final List<String> _semua = ['Algoritma', 'Basis Data', 'Kalkulus', 'Jaringan'];
String _kataKunci = '';

@override
Widget build(BuildContext context) {
  // Data yang DITAMPILKAN dihitung ulang setiap build dari data asli
  final hasil = _semua
      .where((m) => m.toLowerCase().contains(_kataKunci.toLowerCase()))
      .toList();

  return Column(
    children: [
      TextField(
        decoration: const InputDecoration(hintText: 'Cari...'),
        onChanged: (teks) => setState(() => _kataKunci = teks),
      ),
      Expanded(
        child: ListView.builder(
          itemCount: hasil.length,
          itemBuilder: (context, i) => ListTile(title: Text(hasil[i])),
        ),
      ),
    ],
  );
}
```



Mengetik "da" menyisakan Basis Data

`toLowerCase()` pada kedua sisi membuat pencarian tidak membedakan huruf besar dan kecil.

Contoh lengkap: direktori dosen dengan pencarian dan filter

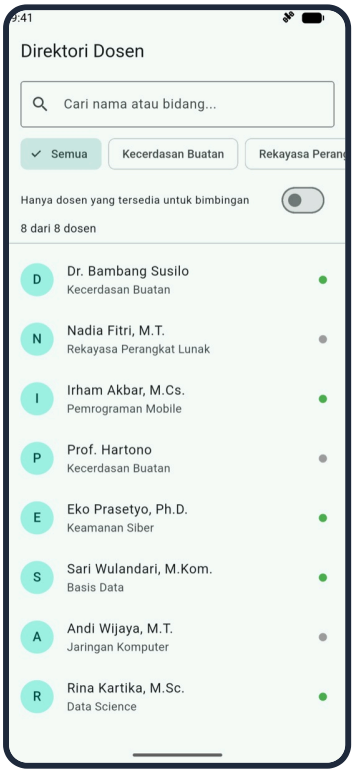
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const DirektoriDosenScreen(),
    );
  }
}

class Dosen {
  final String nama;
  final String bidang;
  final bool tersedia;

  const Dosen(this.nama, this.bidang, this.tersedia);
```

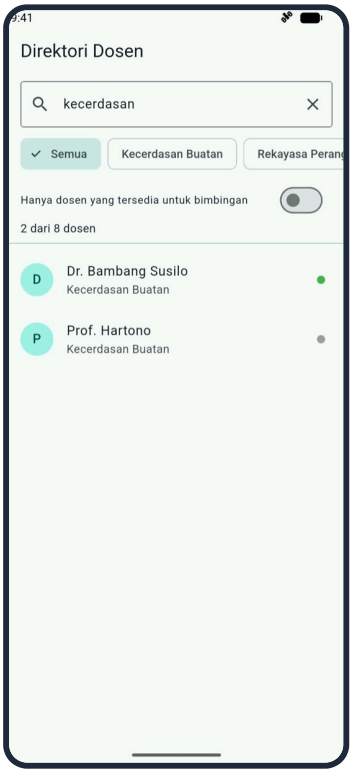
```dart
}

class DirektoriDosenScreen extends StatefulWidget {
  const DirektoriDosenScreen({super.key});

  @override
  State<DirektoriDosenScreen> createState() => _DirektoriDosenScreenState();
}

class _DirektoriDosenScreenState extends State<DirektoriDosenScreen> {
  static const _semuaDosen = [
    Dosen('Dr. Bambang Susilo', 'Kecerdasan Buatan', true),
    Dosen('Nadia Fitri, M.T.', 'Rekayasa Perangkat Lunak', false),
    Dosen('Irham Akbar, M.Cs.', 'Pemrograman Mobile', true),
    Dosen('Prof. Hartono', 'Kecerdasan Buatan', false),
    Dosen('Eko Prasetyo, Ph.D.', 'Keamanan Siber', true),
    Dosen('Sari Wulandari, M.Kom.', 'Basis Data', true),
    Dosen('Andi Wijaya, M.T.', 'Jaringan Komputer', false),
    Dosen('Rina Kartika, M.Sc.', 'Data Science', true),
  ];

  final _cariController = TextEditingController();
  String _kataKunci = '';
  String _bidang = 'Semua';
  bool _hanyaTersedia = false;

  @override
  void dispose() {
    _cariController.dispose();
    super.dispose();
  }

  List<Dosen> get _hasilFilter {
    final kunci = _kataKunci.toLowerCase().trim();
    return _semuaDosen.where((d) {
      final cocokTeks = d.nama.toLowerCase().contains(kunci) ||
          d.bidang.toLowerCase().contains(kunci);
      final cocokBidang = _bidang == 'Semua' || d.bidang == _bidang;
      final cocokStatus = !_hanyaTersedia || d.tersedia;
      return cocokTeks && cocokBidang && cocokStatus;
    }).toList();
  }

  @override
  Widget build(BuildContext context) {
    final hasil = _hasilFilter;
    final daftarBidang = ['Semua', ..._semuaDosen.map((d) => d.bidang).toSet()];

    return Scaffold(
      appBar: AppBar(title: const Text('Direktori Dosen')),
      body: Column(
        children: [
          // 1. Kolom pencarian
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: TextField(
              controller: _cariController,
              decoration: InputDecoration(
                hintText: 'Cari nama atau bidang...',
```

```
            prefixIcon: const Icon(Icons.search),
            border: const OutlineInputBorder(),
            suffixIcon: _kataKunci.isEmpty
                ? null
                : IconButton(
                    icon: const Icon(Icons.clear),
                    onPressed: () {
                      _cariController.clear();
                      setState(() => _kataKunci = '');
                    },
                  ),
          ),
          onChanged: (teks) => setState(() => _kataKunci = teks),
        ),
      ),

      // 2. Filter bidang dengan ChoiceChip (gulir horizontal)
      SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
        child: Row(
          children: [
            for (final b in daftarBidang)
              Padding(
                padding: const EdgeInsets.symmetric(horizontal: 4),
                child: ChoiceChip(
                  label: Text(b),
                  selected: _bidang == b,
                  onSelected: (_) => setState(() => _bidang = b),
                ),
              ),
          ],
        ),
      ),

      // 3. Filter status
      SwitchListTile(
        dense: true,
        title: const Text('Hanya dosen yang tersedia untuk bimbingan'),
        value: _hanyaTersedia,
        onChanged: (v) => setState(() => _hanyaTersedia = v),
      ),
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16),
        child: Align(
          alignment: Alignment.centerLeft,
          child: Text('${hasil.length} dari ${_semuaDosen.length} dosen'),
        ),
      ),
      const Divider(),

      // 4. Hasil (Expanded agar ListView punya tinggi yang jelas)
      Expanded(
        child: hasil.isEmpty
            ? const Center(child: Text('Tidak ada dosen yang cocok'))
            : ListView.builder(
                itemCount: hasil.length,
                itemBuilder: (context, i) {
                  final d = hasil[i];
```

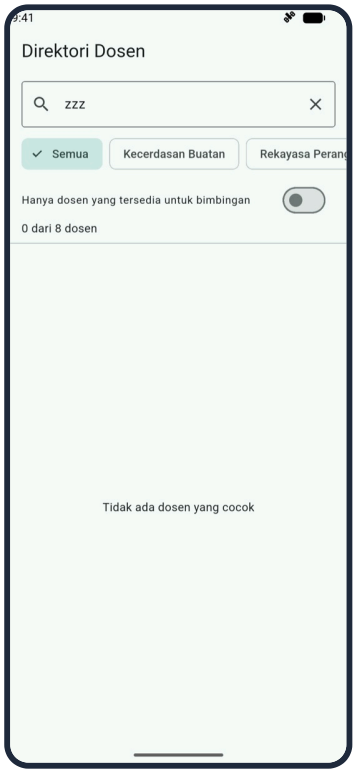
```
                    return ListTile(
                      leading: CircleAvatar(child: Text(d.nama[0])),
                      title: Text(d.nama),
                      subtitle: Text(d.bidang),
                      trailing: Icon(
                        Icons.circle,
                        size: 12,
                        color: d.tersedia ? Colors.green : Colors.grey,
                      ),
                    );
                  },
                ),
            ),
          ],
        ),
      );
    }
}
```



Direktori dosen tanpa filter



Mengetik "kecerdasan" di kolom pencarian



Kata kunci tanpa hasil: pesan kosong

Penjelasan kode:

- Getter `_hasilFilter` menggabungkan tiga syarat dengan `&&`: cocok teks (nama atau bidang), cocok bidang dari chip, dan cocok status ketersediaan.
- `.map((d) => d.bidang).toSet()` menghilangkan bidang yang kembar sebelum dibuat menjadi chip. `['Semua', ...]` menambahkan pilihan "Semua" di depan dengan *spread operator*.
- `ChoiceChip` adalah chip yang hanya bisa dipilih satu (seperti radio).
- Daftar dibungkus `Expanded` karena berada di dalam `Column`.
- Teks "x dari y dosen" memberi tahu pengguna bahwa filter sedang aktif.

⚠️ **Kesalahan umum**

- **Menghapus item dari data asli saat mencari** (`_semua.removeWhere(...)`). Setelah kata kunci dihapus, data tidak bisa kembali.
- **Memakai index hasil filter untuk menghapus dari data asli.** Index keduanya berbeda; hapus berdasarkan objek atau id (lihat studi kasus).
- **Lupa** `setState` di `onChanged`, sehingga daftar tidak berubah saat mengetik.

10. CustomScrollView, SliverAppBar, SliverList & SliverGrid

Sliver adalah potongan area gulir. Dengan `CustomScrollView`, beberapa sliver (app bar, grid, list, widget biasa) digabung menjadi **satu** area gulir yang mulus. Contoh paling terkenal: header bergambar yang mengecil saat digulir (*collapsing header*).

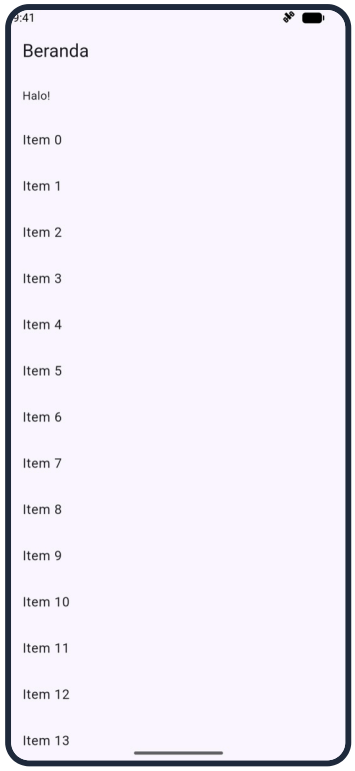
Konsep

Sliver	Padanan widget biasa
`SliverAppBar`	`AppBar` yang bisa membesar/mengecil
`SliverList.builder` / `.separated`	`ListView.builder` / `.separated`
`SliverGrid.count` / `SliverGrid.builder`	`GridView.count` / `.builder`
`SliverToBoxAdapter`	Pembungkus untuk widget biasa (Text, Card, dll.)
`SliverPadding`	`Padding` untuk sliver

Properti penting `SliverAppBar`: `expandedHeight` (tinggi saat terbuka penuh), `pinned` (bagian kecilnya tetap menempel di atas), `floating` (muncul lagi begitu pengguna menggulir sedikit ke atas), `snap` (bersama floating, langsung terbuka penuh), dan `flexibleSpace` (isi area yang membesar).

Contoh kecil

```
body: CustomScrollView(
  // Semua anak CustomScrollView harus berupa SLIVER
  slivers: [
    const SliverAppBar(title: Text('Beranda'), pinned: true),
    // Widget biasa dibungkus SliverToBoxAdapter
    const SliverToBoxAdapter(
      child: Padding(padding: EdgeInsets.all(16), child: Text('Halo!')),
    ),
    SliverList.builder(
      itemCount: 30,
      itemBuilder: (context, i) => ListTile(title: Text('Item $i')),
    ),
  ],
),
```



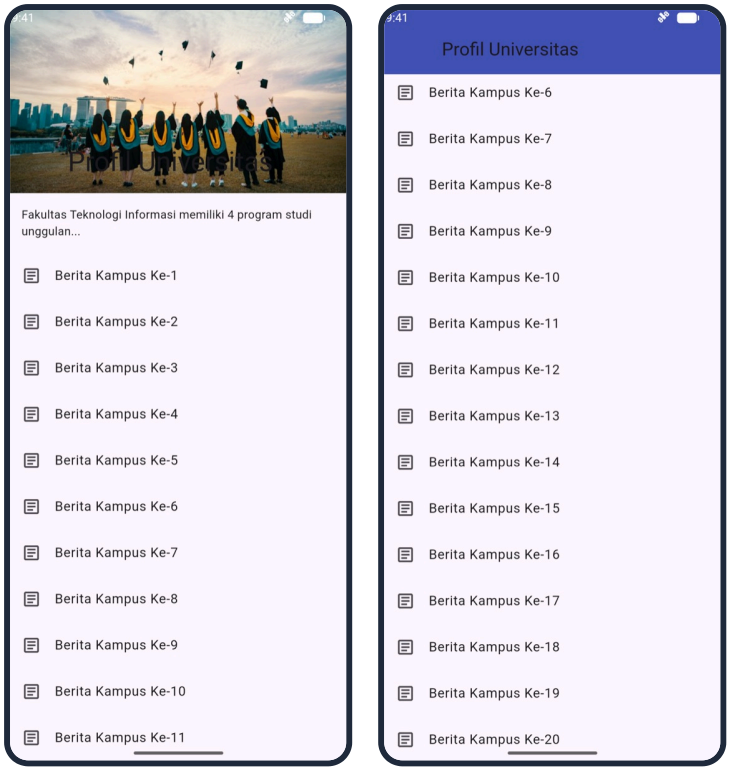
CustomScrollView dengan SliverAppBar, teks, dan SliverList

Contoh lengkap 1: profil universitas dengan gambar header

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: ProfilUniversitasScreen(),
    );
  }
}

class ProfilUniversitasScreen extends StatelessWidget {
  const ProfilUniversitasScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // CustomScrollView TIDAK memakai AppBar biasa: SliverAppBar sudah menjadi app bar-nya
    return Scaffold(
      body: CustomScrollView(
        slivers: [
```

```
          SliverAppBar(
            expandedHeight: 200.0,
            floating: false,
            pinned: true, // Judul tetap menempel di atas saat discroll
            backgroundColor: Colors.indigo,
            foregroundColor: Colors.white,
            flexibleSpace: FlexibleSpaceBar(
              title: const Text("Profil Universitas"),
              background: Image.network(
                "https://images.unsplash.com/photo-1541339907198-e08756dedf3f?w=600",
                fit: BoxFit.cover,
                // Cadangan jika gambar gagal dimuat (offline / URL mati)
                errorBuilder: (context, error, stackTrace) =>
                    Container(color: Colors.indigo.shade300),
              ),
            ),
          ),
          const SliverToBoxAdapter(
            child: Padding(
              padding: EdgeInsets.all(16),
              child: Text("Fakultas Teknologi Informasi memiliki 4 program studi unggulan..."),
            ),
          ),
          SliverList.builder(
            itemCount: 20,
            itemBuilder: (context, index) => ListTile(
              leading: const Icon(Icons.article_outlined),
              title: Text("Berita Kampus Ke-${index + 1}"),
            ),
          ),
        ],
      ),
    );
  }
}
```



Header gambar terbuka penuh

Setelah digulir: header mengecil menjadi app bar biasa

- `FlexibleSpaceBar` mengatur judul yang mengecil dan gambar latar yang memudar.
- `errorBuilder` pada `Image.network` menampilkan warna polos jika gambar gagal dimuat (misalnya saat offline).

Contoh lengkap 2: dashboard dengan grid dan list dalam satu gulir

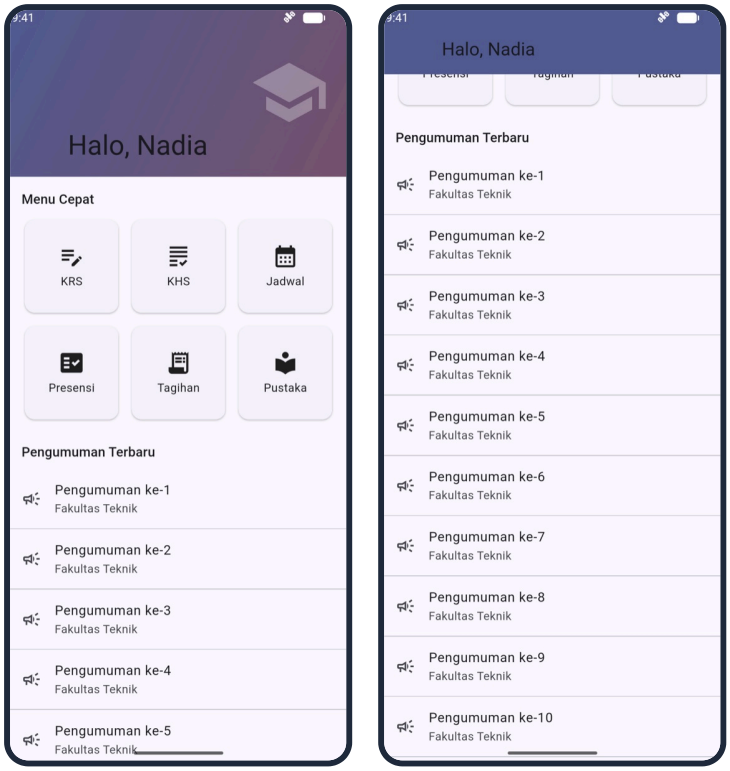
Tanpa sliver, menaruh GridView dan ListView dalam satu halaman yang bisa digulir bersama cukup merepotkan. Dengan `CustomScrollView`, keduanya cukup ditumpuk sebagai sliver.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DashboardScreen(),
    );
  }
}
```

```dart
class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  static const _menu = [
    (Icons.edit_note, 'KRS'),
    (Icons.grading, 'KHS'),
    (Icons.calendar_month, 'Jadwal'),
    (Icons.fact_check, 'Presensi'),
    (Icons.receipt_long, 'Tagihan'),
    (Icons.local_library, 'Pustaka'),
  ];

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      body: CustomScrollView(
        slivers: [
          // 1. App bar yang mengecil saat digulir
          SliverAppBar(
            expandedHeight: 180,
            pinned: true, // bagian kecilnya tetap menempel di atas
            backgroundColor: warna.primary,
            foregroundColor: warna.onPrimary,
            flexibleSpace: FlexibleSpaceBar(
              title: const Text('Halo, Nadia'),
              background: Container(
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    colors: [warna.primary, warna.tertiary],
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                  ),
                ),
                alignment: Alignment.centerRight,
                padding: const EdgeInsets.only(right: 24),
                child: Icon(Icons.school, size: 96, color: warna.onPrimary.withValues(alpha: 0.3)),
              ),
            ),
          ),

          // 2. Widget biasa di antara sliver
          SliverToBoxAdapter(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),
              child: Text('Menu Cepat', style: Theme.of(context).textTheme.titleMedium),
            ),
          ),

          // 3. Grid di dalam CustomScrollView
          SliverPadding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            sliver: SliverGrid.count(
              crossAxisCount: 3,
              mainAxisSpacing: 8,
              crossAxisSpacing: 8,
              children: [
```

```
                for (final (ikon, label) in _menu)
                  Card(
                    child: Column(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [Icon(ikon, size: 32), const SizedBox(height: 4), Text(label)],
                    ),
                  ),
              ],
            ),
          ),

          SliverToBoxAdapter(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16, 24, 16, 8),
              child: Text('Pengumuman Terbaru', style: Theme.of(context).textTheme.titleMedium),
            ),
          ),

          // 4. Daftar panjang di bawahnya: satu scroll untuk semuanya
          SliverList.separated(
            itemCount: 15,
            itemBuilder: (context, i) => ListTile(
              leading: const Icon(Icons.campaign_outlined),
              title: Text('Pengumuman ke-${i + 1}'),
              subtitle: const Text('Fakultas Teknik'),
            ),
            separatorBuilder: (context, i) => const Divider(height: 1),
          ),
        ],
      ),
    );
  }
}
```



Dashboard: header gradasi, grid menu, dan daftar pengumuman

Setelah digulir: header menempel, list berlanjut

Penjelasan kode:

- Header memakai gradasi warna dari `colorScheme`, sehingga ikut berubah jika `colorSchemeSeed` diganti.
- `SliverPadding` memberi jarak kiri-kanan untuk `SliverGrid.count`. `Padding` biasa tidak boleh membungkus sliver.
- Judul bagian ("Menu Cepat", "Pengumuman Terbaru") adalah widget biasa sehingga dibungkus `SliverToBoxAdapter`.
- `SliverList.separated` bekerja persis seperti `ListView.separated`, termasuk lazy loading.

> ⚠️ **Kesalahan umum**
>
> - **Memasukkan widget biasa langsung ke `slivers`**, misalnya `Text` atau `ListView`. Muncul error "A RenderViewport expected a child of type RenderSliver". Bungkus dengan `SliverToBoxAdapter` atau ganti dengan versi sliver-nya.
> - **Memakai `appBar:` Scaffold sekaligus SliverAppBar.** Hasilnya dua app bar bertumpuk.
> - **Memakai `snap: true` tanpa `floating: true`.** Terjadi error assertion; snap hanya berlaku bersama floating.

11. Studi Kasus: Daftar Mata Kuliah

Studi kasus ini menggabungkan materi Modul 05 dan 06 menjadi aplikasi **Daftar Mata Kuliah** dengan fitur: pencarian, hapus dengan geser + urungkan, tambah lewat form di bottom sheet (dengan validasi), ringkasan total SKS, dan empty state.

Rancangan

- **Model** `MataKuliah` (kode, nama, sks, semester). `kode` dipakai sebagai id unik untuk `ValueKey` Dismissible.
- **Data asli** `_semua` dan **data tampil** `_tampil` (hasil filter + urut semester) dipisah.
- **Form tambah** dibuat sebagai StatefulWidget terpisah (`FormMatkulSheet`) agar controller-nya punya `dispose` sendiri. Form mengembalikan objek `MataKuliah` lewat `Navigator.pop`.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Mata Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMatkulScreen(),
    );
  }
}

// ======================= MODEL =======================
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final int semester;

  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.semester,
  });
}

// ======================= LAYAR UTAMA =======================
class DaftarMatkulScreen extends StatefulWidget {
```

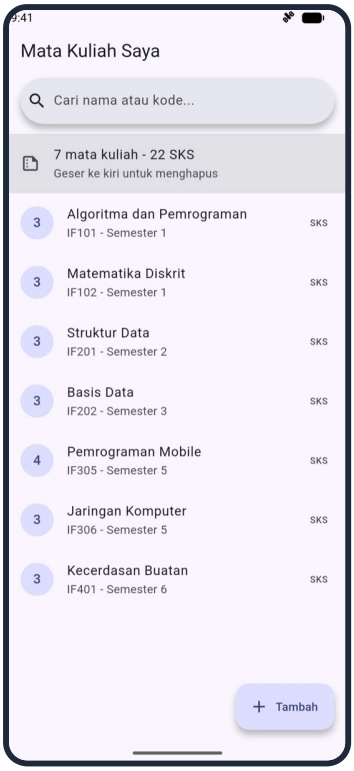
```
  const DaftarMatkulScreen({super.key});

  @override
  State<DaftarMatkulScreen> createState() => _DaftarMatkulScreenState();
}

class _DaftarMatkulScreenState extends State<DaftarMatkulScreen> {
  final List<MataKuliah> _semua = [
    const MataKuliah(kode: 'IF101', nama: 'Algoritma dan Pemrograman', sks: 3, semester: 1),
    const MataKuliah(kode: 'IF102', nama: 'Matematika Diskrit', sks: 3, semester: 1),
    const MataKuliah(kode: 'IF201', nama: 'Struktur Data', sks: 3, semester: 2),
    const MataKuliah(kode: 'IF202', nama: 'Basis Data', sks: 3, semester: 3),
    const MataKuliah(kode: 'IF305', nama: 'Pemrograman Mobile', sks: 4, semester: 5),
    const MataKuliah(kode: 'IF306', nama: 'Jaringan Komputer', sks: 3, semester: 5),
    const MataKuliah(kode: 'IF401', nama: 'Kecerdasan Buatan', sks: 3, semester: 6),
  ];

  final _cariController = TextEditingController();
  String _kataKunci = '';

  @override
  void dispose() {
    _cariController.dispose();
    super.dispose();
  }

  List<MataKuliah> get _tampil {
    final k = _kataKunci.toLowerCase().trim();
    final hasil = _semua
        .where((m) => m.nama.toLowerCase().contains(k) || m.kode.toLowerCase().contains(k))
        .toList();
    hasil.sort((a, b) => a.semester.compareTo(b.semester)); // urut per semester
    return hasil;
  }

  int get _totalSks => _semua.fold(0, (total, m) => total + m.sks);

  // ---------- HAPUS + URUNGKAN ----------
  void _hapus(MataKuliah m) {
    // Simpan posisi di list ASLI (bukan list hasil filter)
    final posisi = _semua.indexOf(m);
    setState(() => _semua.removeAt(posisi));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('${m.nama} dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            onPressed: () => setState(() => _semua.insert(posisi, m)),
          ),
        ),
      );
  }

  // ---------- TAMBAH lewat bottom sheet ----------
  Future<void> _bukaFormTambah() async {
    final baru = await showModalBottomSheet<MataKuliah>(
```

```
      context: context,
      isScrollControlled: true, // agar sheet bisa naik saat keyboard muncul
      showDragHandle: true,
      builder: (_) => FormMatkulSheet(kodeTerpakai: _semua.map((m) => m.kode).toSet()),
    );
    if (baru == null || !mounted) return;

    setState(() => _semua.add(baru));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('${baru.nama} ditambahkan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    final tampil = _tampil;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Mata Kuliah Saya'),
        bottom: PreferredSize(
          preferredSize: const Size.fromHeight(72),
          child: Padding(
            padding: const EdgeInsets.fromLTRB(16, 0, 16, 12),
            child: SearchBar(
              controller: _cariController,
              hintText: 'Cari nama atau kode...',
              leading: const Icon(Icons.search),
              trailing: [
                if (_kataKunci.isNotEmpty)
                  IconButton(
                    icon: const Icon(Icons.clear),
                    onPressed: () {
                      _cariController.clear();
                      setState(() => _kataKunci = '');
                    },
                  ),
              ],
              onChanged: (v) => setState(() => _kataKunci = v),
            ),
          ),
        ),
      ),
      body: Column(
        children: [
          // Ringkasan
          ListTile(
            tileColor: Theme.of(context).colorScheme.surfaceContainerHighest,
            leading: const Icon(Icons.summarize_outlined),
            title: Text('${_semua.length} mata kuliah - $_totalSks SKS'),
            subtitle: Text(_kataKunci.isEmpty
                ? 'Geser ke kiri untuk menghapus'
                : '${tampil.length} hasil untuk "$_kataKunci"'),
          ),
          Expanded(child: _buildDaftar(tampil)),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _bukaFormTambah,
```

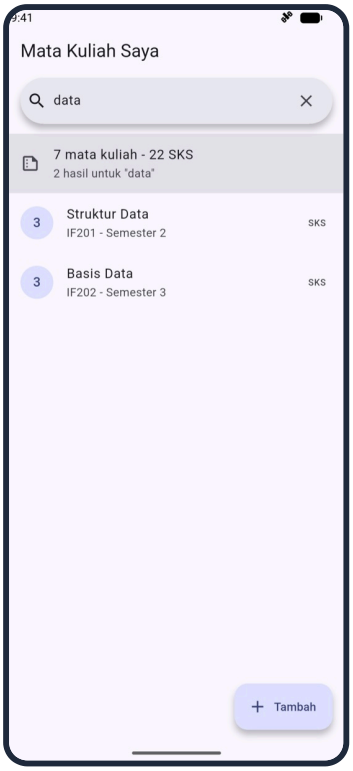
```
        icon: const Icon(Icons.add),
        label: const Text('Tambah'),
      ),
    );
  }

  Widget _buildDaftar(List<MataKuliah> tampil) {
    // Empty state: dua kemungkinan berbeda
    if (_semua.isEmpty) {
      return const _Kosong(ikon: Icons.inbox_outlined, pesan: 'Belum ada mata kuliah.\nTekan
Tambah.');
    }
    if (tampil.isEmpty) {
      return const _Kosong(ikon: Icons.search_off, pesan: 'Tidak ada yang cocok.');
    }

    return ListView.builder(
      padding: const EdgeInsets.only(bottom: 88), // ruang untuk FAB
      itemCount: tampil.length,
      itemBuilder: (context, i) {
        final m = tampil[i];
        return Dismissible(
          key: ValueKey(m.kode), // kode unik
          direction: DismissDirection.endToStart,
          background: Container(
            color: Colors.red,
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 24),
            child: const Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (_) => _hapus(m),
          child: ListTile(
            leading: CircleAvatar(child: Text('${m.sks}')),
            title: Text(m.nama),
            subtitle: Text('${m.kode} - Semester ${m.semester}'),
            trailing: const Text('SKS'),
          ),
        );
      },
    );
  }
}

class _Kosong extends StatelessWidget {
  final IconData ikon;
  final String pesan;

  const _Kosong({required this.ikon, required this.pesan});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(ikon, size: 72, color: Colors.grey),
          const SizedBox(height: 12),
          Text(pesan, textAlign: TextAlign.center),
        ],
```

```
      ),
    );
  }
}

// ======================== FORM TAMBAH (di dalam bottom sheet) ========================
class FormMatkulSheet extends StatefulWidget {
  final Set<String> kodeTerpakai;

  const FormMatkulSheet({super.key, required this.kodeTerpakai});

  @override
  State<FormMatkulSheet> createState() => _FormMatkulSheetState();
}

class _FormMatkulSheetState extends State<FormMatkulSheet> {
  final _formKey = GlobalKey<FormState>();
  final _kodeC = TextEditingController();
  final _namaC = TextEditingController();
  int? _sks;
  double _semester = 1;

  @override
  void dispose() {
    _kodeC.dispose();
    _namaC.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    Navigator.pop(
      context,
      MataKuliah(
        kode: _kodeC.text.trim().toUpperCase(),
        nama: _namaC.text.trim(),
        sks: _sks!,
        semester: _semester.round(),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    // viewInsets.bottom = tinggi keyboard; padding ini mendorong form ke atas
    final tinggiKeyboard = MediaQuery.viewInsetsOf(context).bottom;

    return Padding(
      padding: EdgeInsets.fromLTRB(24, 0, 24, 24 + tinggiKeyboard),
      child: Form(
        key: _formKey,
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Tambah Mata Kuliah', style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 16),
            TextFormField(
              controller: _kodeC,
```

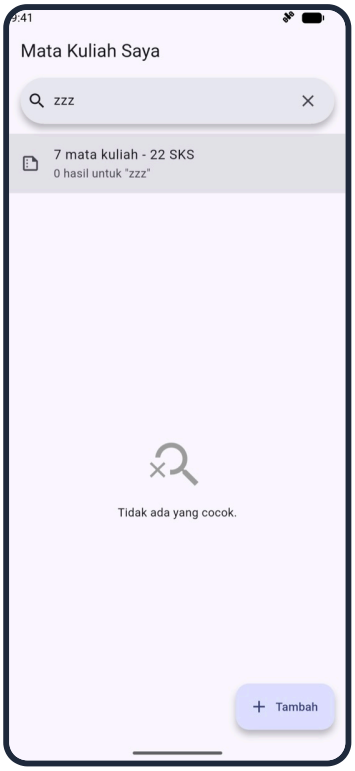
```
              autofocus: true,
              textCapitalization: TextCapitalization.characters,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Kode (contoh: IF307)',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                final kode = (v ?? '').trim().toUpperCase();
                if (kode.isEmpty) return 'Kode wajib diisi';
                if (!RegExp(r'^[A-Z]{2}\d{3}$').hasMatch(kode)) return 'Format: 2 huruf + 3 angka';
                if (widget.kodeTerpakai.contains(kode)) return 'Kode sudah ada';
                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _namaC,
              textCapitalization: TextCapitalization.words,
              decoration: const InputDecoration(
                labelText: 'Nama Mata Kuliah',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().length < 3) ? 'Nama minimal 3 karakter' : null,
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<int>(
              initialValue: _sks,
              decoration: const InputDecoration(labelText: 'SKS', border: OutlineInputBorder()),
              items: [for (final s in [1, 2, 3, 4, 6]) DropdownMenuItem(value: s, child: Text('$s
SKS'))],
              onChanged: (v) => setState(() => _sks = v),
              validator: (v) => v == null ? 'Pilih jumlah SKS' : null,
            ),
            const SizedBox(height: 12),
            Text('Semester: ${_semester.round()}'),
            Slider(
              value: _semester,
              min: 1,
              max: 8,
              divisions: 7,
              label: '${_semester.round()}',
              onChanged: (v) => setState(() => _semester = v),
            ),
            FilledButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```



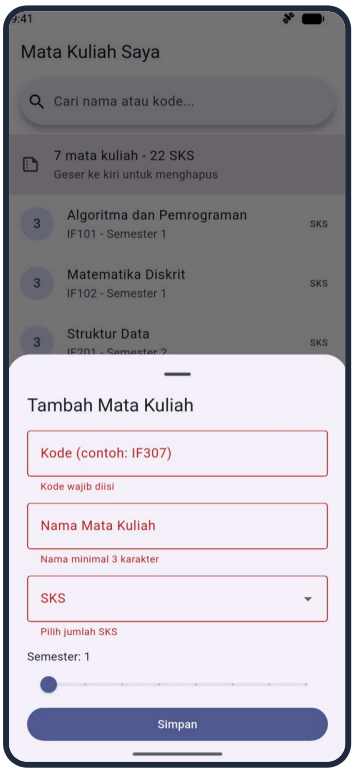
Daftar mata kuliah dengan ringkasan SKS



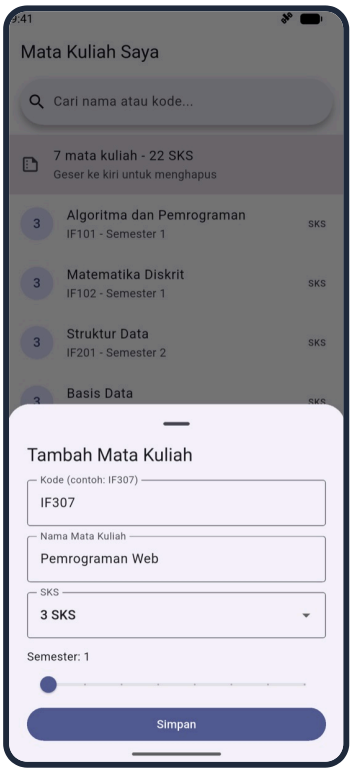
Mencari "data": dua hasil



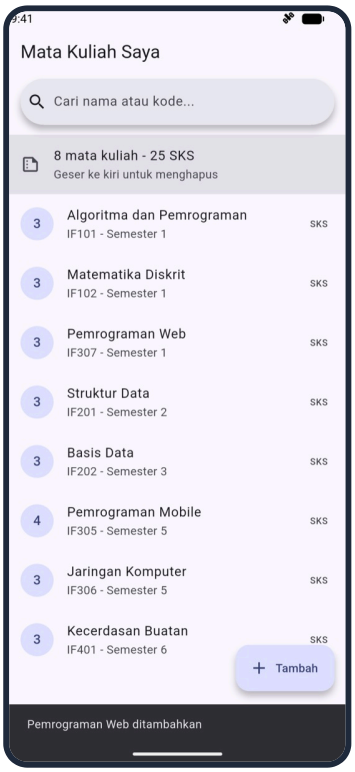
Pencarian tanpa hasil



Form tambah di bottom sheet: Simpan ditekan saat kosong



Form tambah setelah diisi



Mata kuliah baru masuk ke daftar

Penjelasan bagian penting:

- **Hapus dari hasil filter:** `_hapus(m)` menerima objek, lalu mencari posisinya di `_semua` dengan `indexOf`. Dengan begitu penghapusan tetap benar walaupun daftar sedang difilter atau diurutkan.

- **SearchBar** adalah kolom pencarian bergaya Material 3. Diletakkan di `AppBar.bottom` melalui `PreferredSize`.
- **Dua jenis empty state:** "belum ada mata kuliah" (data asli kosong) berbeda dengan "tidak ada yang cocok" (filter tanpa hasil).
- **Bottom sheet berisi form:** `isScrollControlled: true` membolehkan sheet lebih tinggi dari setengah layar, dan padding bawah sebesar `MediaQuery.viewInsetsOf(context).bottom` (tinggi keyboard) mendorong form ke atas keyboard.
- **Validasi kode unik:** daftar kode yang sudah dipakai dikirim ke form sebagai `Set<String>`, lalu diperiksa di validator.
- **Total SKS** dihitung dengan `fold`: mulai dari 0, tambahkan sks setiap mata kuliah.
- `padding: EdgeInsets.only(bottom: 88)` pada ListView memberi ruang agar item terakhir tidak tertutup FAB.

> 💡 **Pengembangan lanjutan**
>
> - Tambahkan fitur edit: ketuk item untuk membuka `FormMatkulSheet` berisi data lama.
> - Kelompokkan daftar per semester dengan judul bagian.
> - Data saat ini hilang saat aplikasi ditutup. Pada Modul 10-12 data akan disimpan ke penyimpanan lokal dan Supabase.

Tugas Praktikum: Praktikum 6: Direktori Dosen dengan Pencarian, Swipe, dan Load More

Sasaran Praktikum

Membangun direktori dosen interaktif yang tetap lancar dengan banyak data, lengkap dengan pencarian, filter, swipe, pull-to-refresh, dan penanganan setiap keadaan layar.

Langkah Pengerjaan

1. Buat model `Dosen` (id, nama, NIDN, bidang keahlian, tersedia untuk bimbingan) dan dataset minimal 40 data (boleh dibuat dengan `List.generate`).
2. Tampilkan data dengan `ListView.builder` atau `ListView.separated`; setiap baris berupa `ListTile` dengan avatar inisial, nama, bidang, dan indikator ketersediaan.
3. Tambahkan `TextField` atau `SearchBar` untuk mencari berdasarkan nama atau bidang tanpa mengubah data asli, plus `ChoiceChip` untuk filter bidang.
4. Tambahkan `Dismissible`: geser ke kanan untuk menandai favorit (item tidak hilang), geser ke kiri untuk menghapus dengan dialog konfirmasi dan SnackBar URUNGKAN. Gunakan id sebagai key.
5. Simulasikan pemuatan bertahap: tampilkan 15 data pertama, lalu muat 15 berikutnya saat mendekati bawah dengan `ScrollController` (beri jeda `Future.delayed`).
6. Tangani keadaan loading awal, daftar kosong, dan hasil pencarian kosong dengan tampilan yang berbeda.
7. Tambahkan `RefreshIndicator` untuk mereset daftar ke halaman pertama.
8. Buat tab kedua (atau layar terpisah) berisi dosen favorit dalam `GridView.builder` dengan `SliverGridDelegateWithMaxCrossAxisExtent`.
9. Pastikan controller di-dispose dan `flutter analyze` bersih.

Output yang Diharapkan

Direktori dosen yang lancar saat digulir, bisa dicari dan difilter secara instan, mendukung swipe favorit/hapus dengan urungkan, memuat data bertahap, dan menampilkan pesan yang jelas saat loading atau kosong.

Rubrik Penilaian

ListView.builder + ListTile yang benar (20%), Pencarian & filter tanpa merusak data asli (20%), Dismissible dengan key unik, konfirmasi, dan urungkan (20%), Infinite scroll dengan penjaga

loading ganda (15%), Loading/empty state + RefreshIndicator (15%), GridView favorit dan kerapian kode (10%).

Kuis Pemahaman

1. Mengapa ListView.builder jauh lebih hemat memori dibanding ListView biasa saat menampilkan 10.000 data?

A. Karena ListView.builder hanya merender item yang saat ini tampak di layar viewport dan mendaur ulang item yang terlewat

B. Karena ListView.builder mengompres resolusi gambar menjadi hitam putih

C. Karena ListView.builder menghapus data dari RAM secara permanen

D. Karena ListView.builder dijalankan di server bukan di HP pengguna

2. Properti SliverAppBar manakah yang jika diatur bernilai 'true' akan membuat judul AppBar tetap menempel di atas layar saat pengguna menggulir konten jauh ke bawah?

A. floating: true

B. pinned: true

C. snap: true

D. stretch: true

3. Kapan SingleChildScrollView lebih tepat dipakai daripada ListView.builder?

A. Untuk halaman dengan isi terbatas seperti form atau profil yang hanya perlu bisa digulir

B. Untuk menampilkan 10.000 data dari API

C. Untuk membuat grid dua kolom

D. Untuk membuat item yang bisa digeser untuk dihapus

4. Muncul error "Vertical viewport was given unbounded height" saat ListView diletakkan di dalam Column. Perbaikan yang tepat adalah...

A. Membungkus ListView dengan Expanded

B. Mengganti Column dengan Row

C. Menghapus itemCount

D. Membungkus Column dengan SingleChildScrollView

5. Mengapa key pada Dismissible sebaiknya memakai id unik data (misalnya ValueKey(item.id)) dan bukan index?

A. Karena index bergeser setelah item dihapus sehingga Flutter bisa salah mengenali item

B. Karena index tidak boleh berupa angka

C. Karena ValueKey membuat animasi lebih cepat

D. Karena Dismissible tidak menerima tipe int

6. Apa syarat fungsi onRefresh pada RefreshIndicator?

A. Harus mengembalikan Future; indikator berputar sampai Future selesai

B. Harus mengembalikan bool

C. Harus memanggil Navigator.pop

D. Harus bersifat synchronous tanpa async

7. Pada infinite scroll, mengapa diperlukan variabel penjaga seperti _sedangMemuat?

A. Agar halaman yang sama tidak dimuat berkali-kali karena listener scroll terpanggil sangat sering

B. Agar ListView tidak bisa digulir

C. Agar ScrollController tidak perlu di-dispose

D. Agar data otomatis tersimpan ke database

8. Widget Text ingin diletakkan di antara SliverAppBar dan SliverList di dalam CustomScrollView. Caranya adalah...

A. Membungkus Text dengan SliverToBoxAdapter

B. Meletakkan Text langsung di daftar slivers

C. Membungkus Text dengan Expanded

D. Memindahkan Text ke parameter appBar Scaffold

Kunci Jawaban & Pembahasan

1. A — ListView.builder menggunakan konsep viewport lazy recycling. Widget anak hanya dibuat saat mendekati layar yang terlihat dan dibuang/digunakan ulang saat keluar layar, menjaga konsumsi memori tetap stabil.

2. B — pinned: true memastikan AppBar tidak akan hilang saat pengguna menggulir ke bawah, melainkan tetap terlihat (menempel) di bagian atas layar.

3. A — SingleChildScrollView membuat seluruh isinya sekaligus, sehingga cocok untuk konten terbatas. Untuk data banyak gunakan ListView.builder yang lazy.

4. A — Column memberi tinggi tak terbatas kepada anaknya, sedangkan ListView ingin mengisi seluruh tinggi yang tersedia. Expanded memberi ListView tinggi sisa yang jelas.

5. A — Setelah item dihapus, index item-item lain berubah. Key berbasis index membuat Flutter mengira item lain adalah item yang sama, memicu error atau tampilan yang salah.

6. A — onRefresh bertipe Future<void> Function(). RefreshIndicator menampilkan lingkaran loading sampai Future tersebut selesai.

7. A — Listener ScrollController dipanggil setiap perubahan posisi gulir. Tanpa penjaga, beberapa permintaan halaman yang sama terkirim bersamaan sehingga data menjadi dobel.

8. A — Semua anak CustomScrollView harus berupa sliver. Widget biasa dibungkus SliverToBoxAdapter agar bisa ditempatkan di antara sliver lain.

MODUL 07 · ROUTING & NAVIGASI

Navigasi & Routing Multi-Halaman

Menguasai prinsip tumpukan navigasi (Navigation Stack), berpindah halaman dengan Navigator.push/pop, mengirim data lewat constructor dan menerima hasil balik, pushReplacement & pushAndRemoveUntil untuk alur login/logout, Named Routes dengan onGenerateRoute, NavigationBar + IndexedStack, Drawer, TabBar, dialog sebagai route, go_router dengan redirect, dan PopScope untuk konfirmasi keluar.

Durasi Pertemuan 7 (150 Menit) + Belajar Mandiri

Prasyarat Modul 06: Scrollable Views

Capaian Pembelajaran

- Memahami konsep tumpukan layar (LIFO: Last-In First-Out) pada Navigator.
- Menggunakan MaterialPageRoute beserta opsi fullscreenDialog dan transisi kustom.
- Mengirim data argumen ke halaman baru dan menerima data balik (return result).
- Memilih push, pushReplacement, pushAndRemoveUntil, dan popUntil sesuai alur login/logout.
- Menerapkan Named Routes dengan onGenerateRoute yang aman dari crash.
- Membangun navigasi multi-tab dengan NavigationBar + IndexedStack, Drawer, dan TabBar.
- Memperlakukan dialog dan bottom sheet sebagai route yang mengembalikan nilai.
- Memahami declarative routing dengan go_router (path parameter, go vs push, redirect).
- Mencegat tombol back dengan PopScope untuk konfirmasi keluar.

Daftar Isi Modul

1. Konsep Tumpukan Navigasi: push & pop
2. MaterialPageRoute & Transisi Halaman
3. Mengirim Data lewat Constructor
4. Menerima Hasil Balik dengan await push & pop
5. pushReplacement & pushAndRemoveUntil (Login/Logout)
6. Named Routes & onGenerateRoute
7. NavigationBar Material 3 & IndexedStack
8. Menu Samping dengan Drawer
9. TabBar & TabBarView
10. Dialog & Bottom Sheet sebagai Route
11. Pengenalan go_router
12. PopScope: Konfirmasi Sebelum Kembali
13. Studi Kasus: Portal Akademik Multi-Halaman
14. Tugas Praktikum
15. Kuis Pemahaman & Kunci Jawaban

1. Konsep Tumpukan Navigasi: push & pop

Navigasi di Flutter bekerja seperti **Tumpukan Piring (Stack - LIFO)**. Saat pengguna membuka halaman baru, halaman tersebut diletakkan di atas tumpukan (`push`). Saat tombol kembali ditekan, halaman paling atas diambil dan disingkirkan (`pop`).

Yang mengatur tumpukan ini adalah widget `Navigator`. Kita tidak perlu membuatnya sendiri: `MaterialApp` sudah menyediakan satu `Navigator` untuk seluruh aplikasi. Setiap "halaman" di dalam tumpukan disebut **route** (rute).

Operasi	Metode Flutter	Efek pada Tumpukan
Masuk Halaman Baru	`Navigator.push(context, route)`	Menambahkan halaman baru di puncak stack. Layar sebelumnya masih hidup di bawahnya.
Kembali	`Navigator.pop(context, [result])`	Menghapus halaman paling atas. Mengembalikan kontrol ke layar di bawahnya.
Ganti Halaman (Replace)	`Navigator.pushReplacement(context, route)`	Menghancurkan halaman saat ini dan menggantinya dengan yang baru (Contoh: dari Layar Login ke Layar Utama).
Bersihkan Seluruh Stack	`Navigator.pushAndRemoveUntil(...)`	Menghapus seluruh riwayat layar masa lalu (Contoh: Logout kembali ke Login).

Ilustrasi tumpukan

```
Awal aplikasi        Setelah push(Tentang)     Setelah pop()
+-----------+        +-----------+             +-----------+
|  Beranda  |        |  Tentang  |  <- atas    |  Beranda  |
+-----------+        +-----------+             +-----------+
                     |  Beranda  |
                     +-----------+
```

Contoh kecil: push dan pop

Dua baris ini adalah inti navigasi. Baris pertama dipanggil di halaman asal, baris kedua di halaman tujuan.

```dart
// MASUK ke halaman baru (push = taruh di atas tumpukan)
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const HalamanB()),
```

```
);

// KEMBALI ke halaman sebelumnya (pop = ambil halaman teratas)
Navigator.pop(context);
```

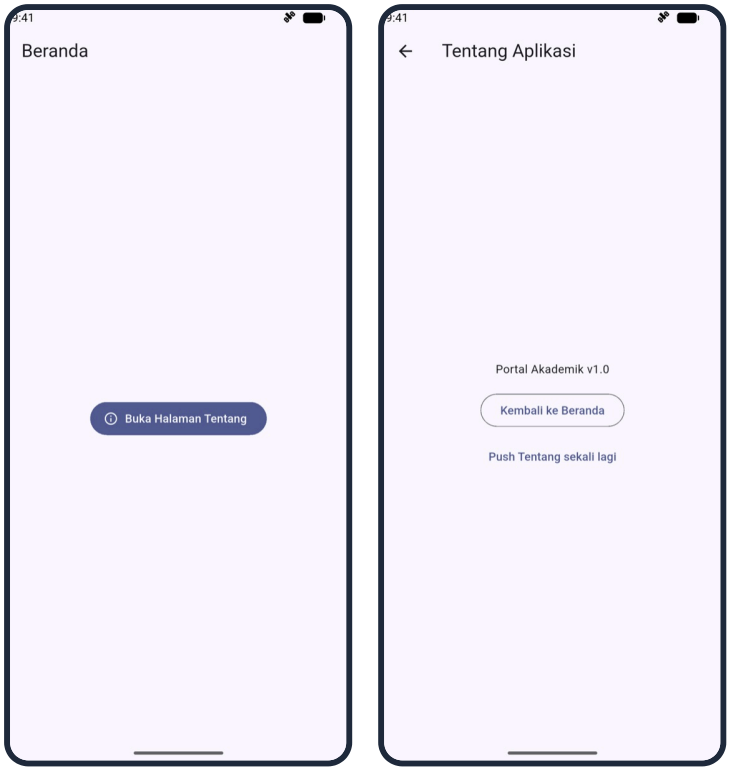
- `context` dipakai untuk mencari `Navigator` terdekat di atas widget ini.
- `MaterialPageRoute` membungkus halaman tujuan dan memberi animasi transisi standar (dibahas di section berikutnya).
- `Navigator.pop(context)` tidak perlu tahu halaman mana yang ada di bawahnya. Ia cukup membuang halaman teratas.

Contoh lengkap: Beranda dan Tentang

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const BerandaScreen(),
    );
  }
}

// --- HALAMAN 1 (paling bawah di tumpukan) ---
class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Center(
        child: FilledButton.icon(
          icon: const Icon(Icons.info_outline),
          label: const Text('Buka Halaman Tentang'),
          onPressed: () {
            // push: TentangScreen diletakkan DI ATAS BerandaScreen
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const TentangScreen()),
            );
          },
        ),
```

```
      ),
    );
  }
}

// --- HALAMAN 2 ---
class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // AppBar otomatis menampilkan tombol panah kembali (←)
      // karena ada halaman lain di bawah tumpukan.
      appBar: AppBar(title: const Text('Tentang Aplikasi')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Portal Akademik v1.0'),
            const SizedBox(height: 16),
            OutlinedButton(
              // pop: buang halaman teratas, kembali ke Beranda
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali ke Beranda'),
            ),
            const SizedBox(height: 8),
            TextButton(
              // push lagi: tumpukan menjadi Beranda -> Tentang -> Tentang
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (context) => const TentangScreen()),
                );
              },
              child: const Text('Push Tentang sekali lagi'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Halaman pertama: Beranda

Setelah push: Tentang di atas Beranda (ada panah kembali)

Penjelasan:

- `home: const BerandaScreen()` menjadi route pertama (paling bawah) di tumpukan.
- Di `TentangScreen`, `AppBar` otomatis menampilkan tombol panah kembali. Tombol itu memanggil `Navigator.maybePop` untuk kita.
- Tombol "Push Tentang sekali lagi" sengaja dibuat untuk membuktikan bahwa halaman yang sama boleh ada dua kali di tumpukan. Tekan back dua kali untuk kembali ke Beranda.
- Tombol back fisik/gestur di Android juga melakukan `pop`.

⚠️ **Kesalahan umum**

- **Memanggil `pop` di halaman pertama.** Jika tumpukan tinggal satu route, `Navigator.pop` membuat layar hitam (tidak ada halaman tersisa). Gunakan `Navigator.canPop(context)` untuk mengecek, atau `Navigator.maybePop(context)` yang aman.
- **Memanggil `Navigator.push` langsung di dalam `build()`.** Navigasi harus dipicu oleh aksi (tombol, timer, hasil login), bukan setiap kali widget di-render.
- **Memakai `context` milik `MyApp` (di atas `MaterialApp`).** Context tersebut belum punya `Navigator`, sehingga muncul error *"Navigator operation requested with a context that does not include a Navigator"*. Lakukan navigasi dari widget halaman (di bawah `MaterialApp`).

2. MaterialPageRoute & Transisi Halaman

`MaterialPageRoute` adalah "bungkus" untuk halaman tujuan. Ia menentukan *bagaimana* halaman muncul: animasi transisi, apakah berupa dialog layar penuh, dan nama rutenya.

Parameter penting

```
MaterialPageRoute(
  builder: (context) => const FormScreen(), // WAJIB: membangun halaman tujuan
  settings: const RouteSettings(name: '/form'), // opsional: nama rute (untuk debug/analytics)
  fullscreenDialog: true, // opsional: animasi dari bawah + tombol X
  maintainState: true, // default true: halaman tetap hidup saat tertutup halaman lain
);
```

Parameter	Fungsi
`builder`	Fungsi yang mengembalikan widget halaman. Dipanggil saat route dibangun.
`settings`	Berisi `name` dan `arguments`. Berguna untuk named routes dan debugging.
`fullscreenDialog`	Jika `true`, halaman masuk dari bawah dan tombol back menjadi ikon X. Cocok untuk form "Tambah ...".
`maintainState`	Jika `true` (default), halaman di bawah tetap disimpan di memori sehingga isian form dan posisi scroll tidak hilang.

Contoh lengkap: tiga gaya transisi

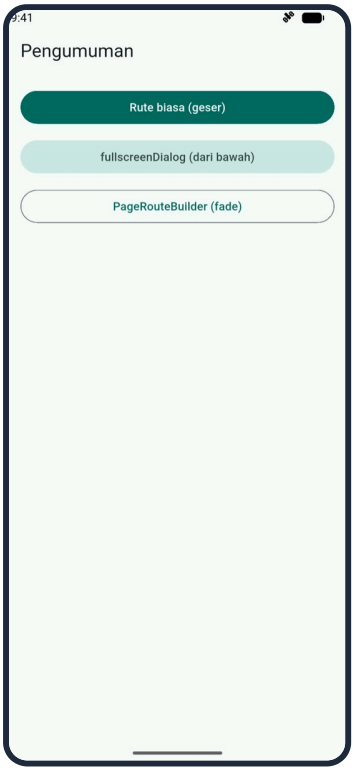
Contoh berikut membandingkan rute biasa, `fullscreenDialog`, dan `PageRouteBuilder` untuk animasi kustom (fade).

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const PengumumanScreen(),
```

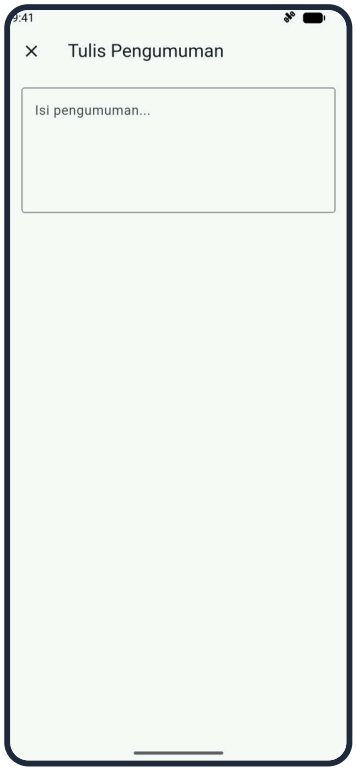
```
    );
  }
}

class PengumumanScreen extends StatelessWidget {
  const PengumumanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengumuman')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // 1. Rute biasa: animasi geser standar platform
          FilledButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  settings: const RouteSettings(name: '/detail'),
                  builder: (context) => const DetailPengumumanScreen(),
                ),
              );
            },
            child: const Text('Rute biasa (geser)'),
          ),
          const SizedBox(height: 12),
          // 2. fullscreenDialog: muncul dari bawah, tombol kembali berubah jadi X
          FilledButton.tonal(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  fullscreenDialog: true,
                  builder: (context) => const TulisPengumumanScreen(),
                ),
              );
            },
            child: const Text('fullscreenDialog (dari bawah)'),
          ),
          const SizedBox(height: 12),
          // 3. PageRouteBuilder: animasi kustom (fade)
          OutlinedButton(
            onPressed: () {
              Navigator.push(
                context,
                PageRouteBuilder(
                  transitionDuration: const Duration(milliseconds: 400),
                  pageBuilder: (context, animation, secondaryAnimation) =>
                      const DetailPengumumanScreen(),
                  transitionsBuilder:
                      (context, animation, secondaryAnimation, child) {
                    return FadeTransition(opacity: animation, child: child);
                  },
                ),
              );
            },
            child: const Text('PageRouteBuilder (fade)'),
```

```
          ),
        ],
      ),
    );
  }
}

class DetailPengumumanScreen extends StatelessWidget {
  const DetailPengumumanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // ModalRoute.of(context) memberi akses ke rute yang sedang aktif
    final namaRute = ModalRoute.of(context)?.settings.name ?? '(tanpa nama)';
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Pengumuman')),
      body: Center(child: Text('Nama rute: $namaRute')),
    );
  }
}

class TulisPengumumanScreen extends StatelessWidget {
  const TulisPengumumanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tulis Pengumuman')),
      body: const Padding(
        padding: EdgeInsets.all(16),
        child: TextField(
          maxLines: 5,
          decoration: InputDecoration(
            border: OutlineInputBorder(),
            hintText: 'Isi pengumuman...',
          ),
        ),
      ),
    );
  }
}
```



Tiga tombol untuk tiga jenis rute

Rute biasa: nama rute terbaca dari settings



fullscreenDialog: tombol kembali berupa ikon X

Penjelasan:

- `MaterialPageRoute` otomatis menyesuaikan animasi dengan platform (Android: zoom/geser, iOS: geser dari kanan).
- `ModalRoute.of(context)?.settings.name` membaca nama rute dari dalam halaman tujuan. Rute ketiga tidak diberi `settings`, jadi namanya `null`.
- `PageRouteBuilder` memberi kendali penuh: `pageBuilder` membangun halaman, `transitionsBuilder` membungkusnya dengan animasi. Nilai `animation` bergerak dari 0.0 ke 1.0.

> 💡 **Tips**
>
> Untuk aplikasi kuliah, `MaterialPageRoute` sudah cukup di 95% kasus. Pakai `PageRouteBuilder` hanya jika desain memang meminta animasi khusus.

⚠️ **Kesalahan umum**

- Menulis `builder: (context) => HalamanB` (tanpa tanda kurung). Yang benar adalah memanggil constructor: `HalamanB()`.
- Mengira `fullscreenDialog` sama dengan `showDialog`. `fullscreenDialog` tetap halaman penuh, hanya gaya animasinya yang berbeda.

3. Mengirim Data lewat Constructor

Cara paling sederhana dan paling aman untuk mengirim data ke halaman baru adalah lewat **constructor**. Halaman tujuan hanyalah sebuah class Dart, jadi data cukup dikirim sebagai argumen.

Pola dasarnya

```
// Halaman ASAL: kirim data lewat argumen constructor
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const ProfilScreen(nama: 'Sinta', angkatan: 2023),
  ),
);


// Halaman TUJUAN: deklarasikan field final + parameter required
class ProfilScreen extends StatelessWidget {
  final String nama;
  final int angkatan;

  const ProfilScreen({super.key, required this.nama, required this.angkatan});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(nama)),
      body: Center(child: Text('Angkatan $angkatan')),
    );
  }
}
```

- Field di halaman tujuan dibuat `final` karena data kiriman tidak berubah selama halaman tampil.
- Parameter `required` membuat compiler protes jika kita lupa mengirim datanya. Kesalahan ketahuan sebelum aplikasi dijalankan.
- Jika halaman tujuan berupa `StatefulWidget`, data dibaca di class State dengan `widget.nama`.

Contoh lengkap: daftar mata kuliah ke halaman detail

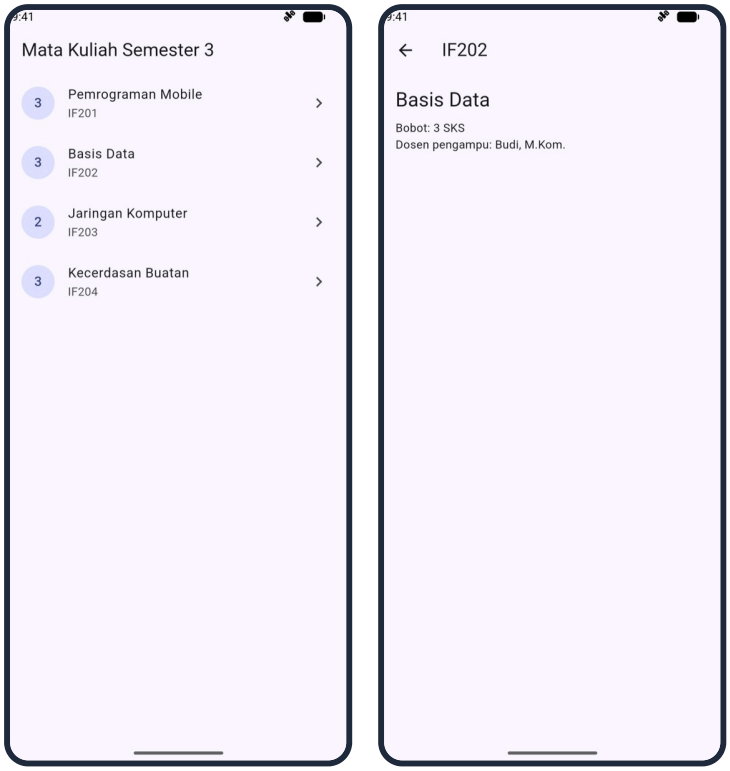
Daripada mengirim `kode`, `nama`, `sks` satu per satu, kirim satu objek model utuh.

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```dart
// Model data sederhana (idealnya di lib/models/mata_kuliah.dart)
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final String dosen;

  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.dosen,
  });
}

const daftarMatkul = [
  MataKuliah(kode: 'IF201', nama: 'Pemrograman Mobile', sks: 3, dosen: 'Dr. Rina'),
  MataKuliah(kode: 'IF202', nama: 'Basis Data', sks: 3, dosen: 'Budi, M.Kom.'),
  MataKuliah(kode: 'IF203', nama: 'Jaringan Komputer', sks: 2, dosen: 'Sari, M.T.'),
  MataKuliah(kode: 'IF204', nama: 'Kecerdasan Buatan', sks: 3, dosen: 'Dr. Agus'),
];

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMatkulScreen(),
    );
  }
}

class DaftarMatkulScreen extends StatelessWidget {
  const DaftarMatkulScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Semester 3')),
      body: ListView.builder(
        itemCount: daftarMatkul.length,
        itemBuilder: (context, index) {
          final mk = daftarMatkul[index];
          return ListTile(
            leading: CircleAvatar(child: Text('${mk.sks}')),
            title: Text(mk.nama),
            subtitle: Text(mk.kode),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              // Kirim SATU objek utuh, bukan field satu per satu
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => DetailMatkulScreen(matkul: mk),
                ),
              );
```

```
            },
          );
        },
      ),
    );
  }
}

class DetailMatkulScreen extends StatelessWidget {
  // Data yang diterima dari halaman sebelumnya
  final MataKuliah matkul;

  const DetailMatkulScreen({super.key, required this.matkul});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(matkul.kode)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(matkul.nama, style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 8),
            Text('Bobot: ${matkul.sks} SKS'),
            Text('Dosen pengampu: ${matkul.dosen}'),
          ],
        ),
      ),
    );
  }
}
```



Daftar mata kuliah

Detail menerima objek MataKuliah lewat constructor

Penjelasan:

- Class `MataKuliah` adalah model data. Di proyek nyata letakkan di `lib/models/`.
- Di `onTap`, objek `mk` dikirim ke `DetailMatkulScreen(matkul: mk)`.
- Halaman detail tidak perlu tahu dari mana datanya berasal. Ia hanya menampilkan `matkul` yang diterima. Pemisahan ini membuat halaman mudah dipakai ulang dan diuji.

> ℹ️ **Constructor vs arguments**
>
> Selain constructor, Flutter juga punya `RouteSettings.arguments` (dipakai di named routes, section 6). Constructor lebih disarankan karena tipe datanya diperiksa compiler, sedangkan `arguments` bertipe `Object?` dan harus dicek manual.

⚠️ **Kesalahan umum**

- Menyimpan data kiriman di variabel global agar "bisa diakses dari halaman lain". Ini membuat data sulit dilacak. Kirim lewat constructor, atau pakai Provider (Modul 08) untuk data yang dipakai banyak halaman.
- Lupa menulis `widget.` saat membaca field di `State`: tulis `widget.matkul.nama`, bukan `matkul.nama`.

4. Menerima Hasil Balik dengan await push & pop

Navigasi bisa dua arah. `Navigator.push` mengembalikan sebuah `Future` yang selesai ketika halaman tujuan ditutup. Nilai yang dikirim lewat `Navigator.pop(context, nilai)` akan menjadi hasil `Future` tersebut.

Pola dasarnya

```
// Halaman ASAL: tunggu hasil dengan await
onPressed: () async {
  final String? kelas = await Navigator.push<String>(
    context,
    MaterialPageRoute(builder: (context) => const PilihKelasScreen()),
  );
  if (kelas == null) return; // pengguna menekan tombol back
  debugPrint('Kelas dipilih: $kelas');
},


// Halaman TUJUAN: tutup sambil membawa nilai
onTap: () => Navigator.pop(context, 'Kelas A'),
```

- `push<String>` memberi tahu Dart bahwa hasilnya bertipe `String?`. Tanda tanya muncul karena pengguna bisa menekan back tanpa memilih apa pun, sehingga hasilnya `null`.
- `await` membuat kode "menunggu" sampai halaman tujuan di-`pop`. Karena itu fungsi `onPressed` harus `async`.

Contoh lengkap 1: verifikasi data mahasiswa

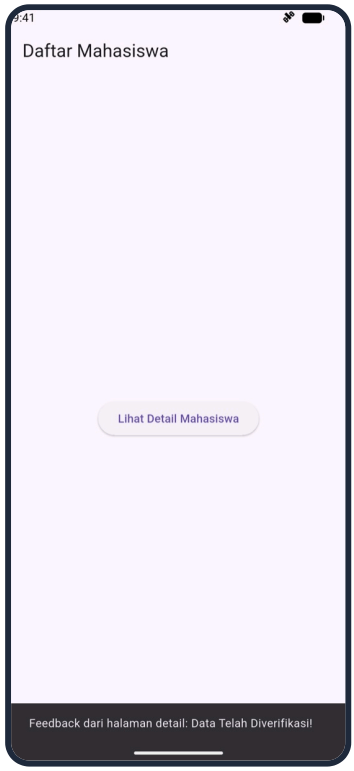
Mari pelajari cara mengirim data objek Mahasiswa ke halaman detail, dan menerima konfirmasi status kembali:

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMahasiswaScreen(),
    );
  }
}
```

```dart
// --- HALAMAN 1: DAFTAR MAHASISWA ---
class DaftarMahasiswaScreen extends StatelessWidget {
  const DaftarMahasiswaScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Daftar Mahasiswa")),
      body: Center(
        child: ElevatedButton(
          onPressed: () async {
            // 1. Mengirim data ke DetailMahasiswaScreen dan MENUNGGU hasil balik (await)
            //    <String> = tipe data yang diharapkan dikirim balik oleh Navigator.pop
            final hasilBalik = await Navigator.push<String>(
              context,
              MaterialPageRoute(
                builder: (context) => const DetailMahasiswaScreen(
                  nama: "Fikri Ramadhan",
                  nim: "21102045",
                ),
              ),
            );

            // 2. Menerima data yang dikirim balik dari Halaman 2.
            //    Cek context.mounted karena ada jeda (await) sebelum memakai context lagi.
            if (hasilBalik != null && context.mounted) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text("Feedback dari halaman detail: $hasilBalik")),
              );
            }
          },
          child: const Text("Lihat Detail Mahasiswa"),
        ),
      ),
    );
  }
}

// --- HALAMAN 2: DETAIL MAHASISWA ---
class DetailMahasiswaScreen extends StatelessWidget {
  final String nama;
  final String nim;

  const DetailMahasiswaScreen({super.key, required this.nama, required this.nim});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Profil $nama")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text("NIM: $nim", style: const TextStyle(fontSize: 18)),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                // Menutup layar DAN mengembalikan data String ke halaman sebelumnya
```

```
                Navigator.pop(context, "Data Telah Diverifikasi!");
              },
              child: const Text("Verifikasi & Kembali"),
            ),
          ],
        ),
      ),
    );
  }
}
```



Hasil balik dari pop tampil di SnackBar

Contoh lengkap 2: form tambah catatan yang mengembalikan objek

Pola ini sangat sering dipakai: halaman daftar membuka halaman form, form mengembalikan objek baru, lalu daftar menambahkannya dengan `setState`.

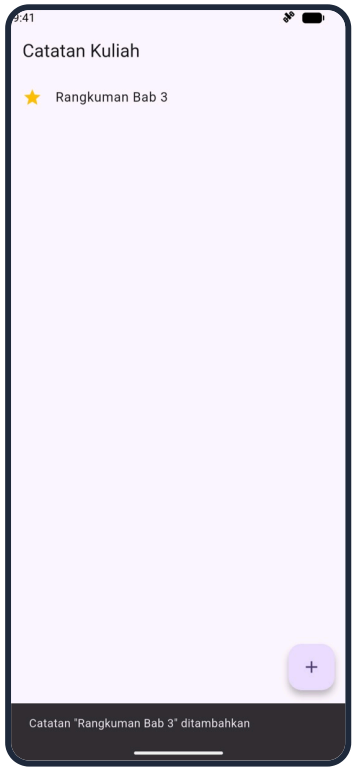
`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class Catatan {
  final String judul;
  final bool penting;

  const Catatan({required this.judul, required this.penting});
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const DaftarCatatanScreen(),
    );
  }
}

// Halaman daftar perlu StatefulWidget karena isinya berubah
class DaftarCatatanScreen extends StatefulWidget {
  const DaftarCatatanScreen({super.key});

  @override
  State<DaftarCatatanScreen> createState() => _DaftarCatatanScreenState();
}

class _DaftarCatatanScreenState extends State<DaftarCatatanScreen> {
  final List<Catatan> _catatan = [];

  Future<void> _bukaForm() async {
    // 1. Buka form dan TUNGGU sampai form ditutup
    final hasil = await Navigator.push<Catatan>(
      context,
      MaterialPageRoute(builder: (context) => const FormCatatanScreen()),
    );

    // 2. hasil == null berarti pengguna batal (tekan back)
    if (hasil == null) return;

    // 3. Setelah await, pastikan State masih terpasang sebelum setState/context
    if (!mounted) return;
    setState(() => _catatan.add(hasil));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Catatan "${hasil.judul}" ditambahkan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Kuliah')),
      body: _catatan.isEmpty
          ? const Center(child: Text('Belum ada catatan. Tekan tombol +'))
          : ListView.builder(
              itemCount: _catatan.length,
              itemBuilder: (context, index) {
                final c = _catatan[index];
                return ListTile(
                  leading: Icon(
                    c.penting ? Icons.star : Icons.notes,
                    color: c.penting ? Colors.amber : null,
                  ),
                  title: Text(c.judul),
```

```
                );
              },
            ),
        floatingActionButton: FloatingActionButton(
          tooltip: 'Tambah catatan',
          onPressed: _bukaForm,
          child: const Icon(Icons.add),
        ),
      );
  }
}

class FormCatatanScreen extends StatefulWidget {
  const FormCatatanScreen({super.key});

  @override
  State<FormCatatanScreen> createState() => _FormCatatanScreenState();
}

class _FormCatatanScreenState extends State<FormCatatanScreen> {
  final _judulController = TextEditingController();
  bool _penting = false;

  @override
  void dispose() {
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _judulController.text.trim();
    if (judul.isEmpty) return;
    // Tutup halaman sambil MENGIRIM objek Catatan ke halaman sebelumnya
    Navigator.pop(context, Catatan(judul: judul, penting: _penting));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Baru')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _judulController,
              autofocus: true,
              decoration: const InputDecoration(
                labelText: 'Judul catatan',
                border: OutlineInputBorder(),
              ),
            ),
            SwitchListTile(
              title: const Text('Tandai penting'),
              value: _penting,
              onChanged: (v) => setState(() => _penting = v),
            ),
            const SizedBox(height: 12),
            SizedBox(
```

```
              width: double.infinity,
              child: FilledButton(onPressed: _simpan, child: const Text('Simpan')),
            ),
          ],
        ),
      ),
    );
  }
}
```



Objek Catatan dikembalikan dan ditambahkan ke daftar

Penjelasan:

- `_bukaForm()` menunggu hasil bertipe `Catatan?`. Jika `null`, berarti pengguna batal sehingga fungsi langsung berhenti.
- `if (!mounted) return;` wajib ada setelah `await`. Selama menunggu, halaman daftar bisa saja sudah ditutup. Memanggil `setState` pada State yang sudah dilepas akan menimbulkan error.
- Di form, `Navigator.pop(context, Catatan(...))` sekaligus menutup halaman dan mengirim objek.
- Di dalam `State` gunakan `mounted`; di dalam fungsi yang hanya punya `BuildContext` (seperti contoh 1) gunakan `context.mounted`.

⚠️ **Kesalahan umum**

- Lupa `await`: kode setelah `push` langsung jalan sebelum pengguna memilih apa pun, dan hasilnya selalu berupa `Future`, bukan nilai.
- Tidak menangani `null`. Pengguna selalu bisa menekan tombol back, jadi hasil *pasti* bisa `null`.
- Tipe hasil tidak cocok, misalnya `push<String>` tetapi halaman tujuan memanggil `pop(context, 123)`. Ini menimbulkan error tipe saat runtime.
- Memakai `context` setelah `await` tanpa cek `mounted`. Analyzer akan memberi peringatan *use_build_context_synchronously*.

5. pushReplacement & pushAndRemoveUntil (Login/Logout)

Tidak semua halaman boleh "dikembalikan" dengan tombol back. Setelah login, pengguna tidak seharusnya kembali ke form login. Setelah logout, pengguna tidak boleh kembali ke halaman berisi data pribadi. Untuk kasus ini kita memakai `pushReplacement` dan `pushAndRemoveUntil`.

Metode	Tumpukan sebelum	Tumpukan sesudah	Kasus
`push(Home)`	[Login]	[Login, Home]	Halaman detail biasa
`pushReplacement(Home)`	[Login]	[Home]	Splash → Login, Login → Home
`pushAndRemoveUntil(Login, (r) => false)`	[Home, Pengaturan]	[Login]	Logout
`popUntil((r) => r.isFirst)`	[Home, A, B]	[Home]	Tombol "Kembali ke Beranda"

Contoh kecil

```
// 1) Ganti halaman saat ini: [Login] menjadi [Home]
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const HomeScreen()),
);


// 2) Push Login lalu hapus SEMUA rute di bawahnya
//    (route) => false artinya "jangan sisakan satu rute pun"
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (context) => const LoginScreen()),
  (route) => false,
);

// 3) Kembali terus sampai ke rute pertama (tanpa push baru)
Navigator.popUntil(context, (route) => route.isFirst);
```

Parameter ketiga `pushAndRemoveUntil` adalah *predicate*: fungsi yang dipanggil untuk setiap route lama, dari atas ke bawah. Route dibuang selama fungsi mengembalikan `false`. Dengan `(route) => false`, semua route lama dibuang.

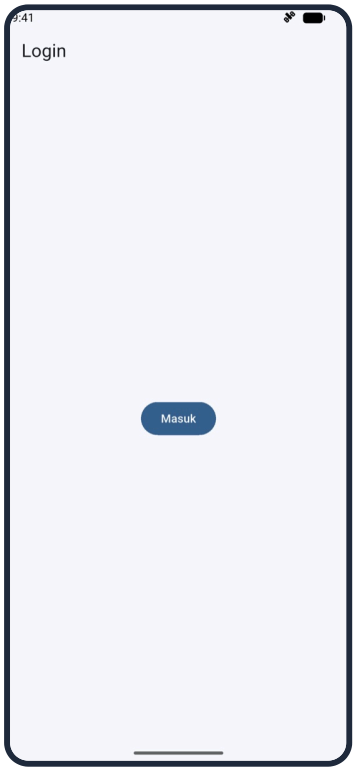
Contoh lengkap: Splash → Login → Home → Pengaturan → Logout

`lib/main.dart`

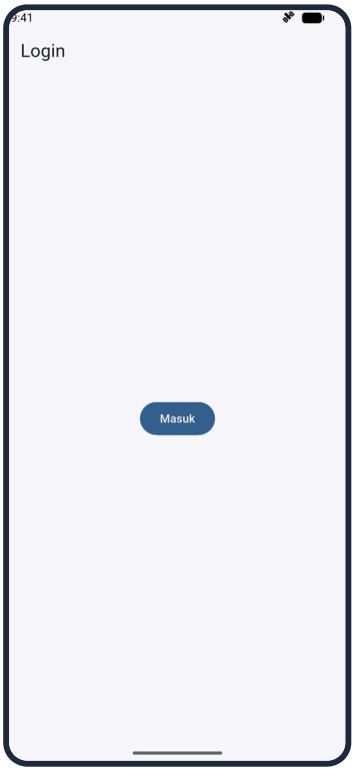
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const SplashScreen(),
    );
  }
}

// ---------------- SPLASH ----------------
class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    _lanjutKeLogin();
  }

  Future<void> _lanjutKeLogin() async {
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return; // halaman bisa saja sudah ditutup selama menunggu
    // Splash diganti Login -> tombol back tidak akan kembali ke splash
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const LoginScreen()),
    );
  }

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            FlutterLogo(size: 96),
            SizedBox(height: 16),
```

```
            CircularProgressIndicator(),
          ],
        ),
      ),
    );
  }
}

// ---------------- LOGIN ----------------
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Center(
        child: FilledButton(
          onPressed: () {
            // Login sukses: GANTI halaman login dengan Home
            Navigator.pushReplacement(
              context,
              MaterialPageRoute(builder: (context) => const HomeScreen()),
            );
          },
          child: const Text('Masuk'),
        ),
      ),
    );
  }
}

// ---------------- HOME ----------------
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Tidak ada panah back di sini karena Home adalah satu-satunya rute
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: FilledButton.tonal(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const PengaturanScreen()),
            );
          },
          child: const Text('Buka Pengaturan'),
        ),
      ),
    );
  }
}

// ---------------- PENGATURAN ----------------
class PengaturanScreen extends StatelessWidget {
  const PengaturanScreen({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Center(
        child: FilledButton.icon(
          style: FilledButton.styleFrom(backgroundColor: Colors.red),
          icon: const Icon(Icons.logout),
          label: const Text('Logout'),
          onPressed: () {
            // Stack saat ini: [Home, Pengaturan]
            // Setelah ini  : [Login]  (Home & Pengaturan dibuang semua)
            Navigator.pushAndRemoveUntil(
              context,
              MaterialPageRoute(builder: (context) => const LoginScreen()),
              (route) => false,
            );
          },
        ),
      ),
    );
  }
}
```



Splash sudah diganti Login (tanpa panah kembali)



Setelah logout: tumpukan hanya [Login]

Penjelasan alur tumpukan:

1. Aplikasi mulai: `[Splash]`. Setelah 2 detik, `pushReplacement` → `[Login]`.
2. Tekan Masuk: `pushReplacement` → `[Home]`. Perhatikan AppBar Home tidak punya panah kembali.

3. Buka Pengaturan: `push` → `[Home, Pengaturan]`.
4. Logout: `pushAndRemoveUntil` → `[Login]`. Tekan back di Android, aplikasi tertutup karena tidak ada halaman lain.

> ℹ️ **Hubungan dengan Modul 13–14**
>
> Di proyek CRUD + Auth nanti, perpindahan login/home tidak lagi memakai `pushReplacement` manual, melainkan sebuah *auth gate*: widget yang memilih menampilkan Login atau Home berdasarkan status sesi di Provider. Konsepnya sama: halaman login tidak boleh tersisa di tumpukan setelah login.

> ⚠️ **Kesalahan umum**
>
> - Memakai `push` biasa setelah login sehingga tombol back membawa pengguna kembali ke form login.
> - Logout dengan `pushReplacement`: hanya halaman teratas yang diganti, halaman Home masih tersisa di bawah.
> - Navigasi di `initState` setelah `Future.delayed` tanpa cek `mounted`.

6. Named Routes & onGenerateRoute

Named routes memberi setiap halaman sebuah nama seperti URL (`'/detail'`). Navigasi cukup menyebut nama, tanpa perlu meng-import class halaman tujuan di setiap file.

Cara 1: tabel `routes` (paling sederhana)

```
return MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const BerandaScreen(),
    '/tentang': (context) => const TentangScreen(),
  },
);


// Di halaman mana pun:
onPressed: () => Navigator.pushNamed(context, '/tentang'),
```

Tabel `routes` cocok untuk halaman tanpa argumen. Kelemahannya: jika halaman butuh data, kita harus membaca `ModalRoute.of(context)!.settings.arguments` di dalam halaman, dan tidak ada penanganan nama rute yang salah.

Cara 2: `onGenerateRoute` (lebih aman)

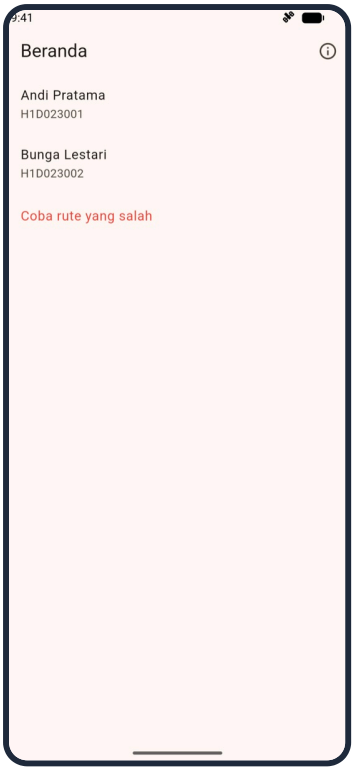
`onGenerateRoute` adalah satu fungsi pusat yang dipanggil setiap kali `pushNamed` dijalankan. Di sana kita bisa memeriksa argumen dan membuat halaman error jika rute tidak dikenal.

`lib/main.dart`

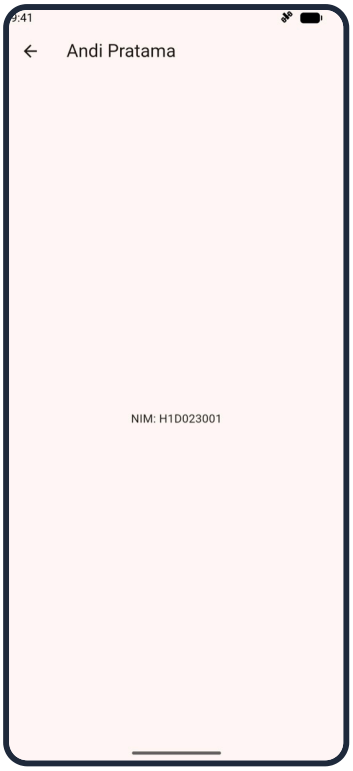
```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class Mahasiswa {
  final String nim;
  final String nama;

  const Mahasiswa(this.nim, this.nama);
}

// Kumpulkan semua nama rute di satu tempat agar tidak salah ketik
class Rute {
  static const beranda = '/';
  static const detail = '/detail';
  static const tentang = '/tentang';
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.orange),
      initialRoute: Rute.beranda,
      onGenerateRoute: generateRoute, // dipanggil setiap pushNamed
    );
  }
}

// Satu fungsi pusat yang menerjemahkan NAMA rute menjadi HALAMAN
Route<dynamic> generateRoute(RouteSettings settings) {
  switch (settings.name) {
    case Rute.beranda:
      return MaterialPageRoute(builder: (_) => const BerandaScreen());

    case Rute.detail:
      // arguments bertipe Object?, jadi WAJIB dicek tipenya
      final args = settings.arguments;
      if (args is Mahasiswa) {
        return MaterialPageRoute(
          settings: settings,
          builder: (_) => DetailScreen(mahasiswa: args),
        );
      }
      return _halamanError('Argumen untuk ${settings.name} tidak valid');

    case Rute.tentang:
      return MaterialPageRoute(builder: (_) => const TentangScreen());

    default:
      return _halamanError('Rute "${settings.name}" tidak ditemukan');
  }
}

// Halaman cadangan agar aplikasi tidak crash ketika rute salah
MaterialPageRoute<dynamic> _halamanError(String pesan) {
  return MaterialPageRoute(
    builder: (_) => Scaffold(
      appBar: AppBar(title: const Text('Oops')),
      body: Center(child: Text(pesan)),
    ),
  );
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  static const data = [
    Mahasiswa('H1D023001', 'Andi Pratama'),
    Mahasiswa('H1D023002', 'Bunga Lestari'),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
```

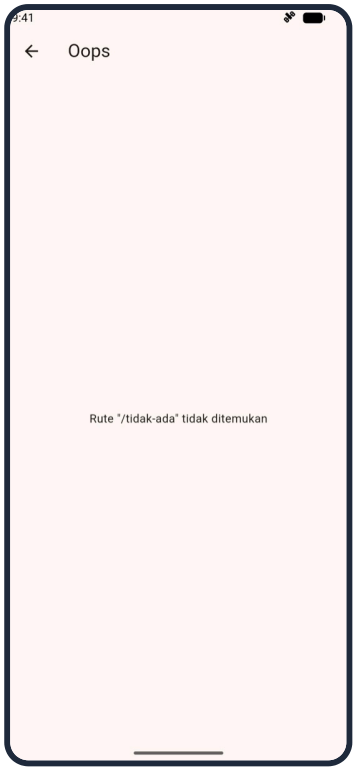
```
        actions: [
          IconButton(
            icon: const Icon(Icons.info_outline),
            onPressed: () => Navigator.pushNamed(context, Rute.tentang),
          ),
        ],
      ),
      body: ListView(
        children: [
          for (final m in data)
            ListTile(
              title: Text(m.nama),
              subtitle: Text(m.nim),
              // Kirim argumen lewat parameter "arguments"
              onTap: () => Navigator.pushNamed(context, Rute.detail, arguments: m),
            ),
          ListTile(
            title: const Text('Coba rute yang salah'),
            textColor: Colors.red,
            onTap: () => Navigator.pushNamed(context, '/tidak-ada'),
          ),
        ],
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  final Mahasiswa mahasiswa;

  const DetailScreen({super.key, required this.mahasiswa});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(mahasiswa.nama)),
      body: Center(child: Text('NIM: ${mahasiswa.nim}')),
    );
  }
}

class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tentang')),
      body: const Center(child: Text('Contoh Named Routes + onGenerateRoute')),
    );
  }
}
```



Beranda dengan named routes



Detail menerima argumen Mahasiswa



Rute tidak dikenal ditangani halaman error

Penjelasan:

- Class `Rute` berisi konstanta nama rute. Salah ketik `Rute.detial` akan langsung ditandai merah oleh editor, sedangkan salah ketik string `'/detial'` baru ketahuan saat aplikasi jalan.
- `Navigator.pushNamed(context, Rute.detail, arguments: m)` mengirim objek lewat `settings.arguments`.
- Di `generateRoute`, `if (args is Mahasiswa)` memeriksa tipe sekaligus melakukan *type promotion*, sehingga `args` bisa langsung dipakai sebagai `Mahasiswa`.
- Cabang `default` mengembalikan halaman error yang ramah, bukan crash.
- Varian lain: `pushReplacementNamed` dan `pushNamedAndRemoveUntil` bekerja sama seperti versi tanpa nama.

> 💡 **Named routes atau go_router?**
>
> Tim Flutter tidak lagi merekomendasikan named routes untuk aplikasi baru yang butuh *deep link* (membuka halaman tertentu dari URL/notifikasi). Untuk kebutuhan itu gunakan `go_router` (section 11). Namun named routes tetap penting dipahami karena banyak proyek lama memakainya.

⚠️ **Kesalahan umum**

- Melakukan cast paksa `settings.arguments as Mahasiswa` tanpa cek. Jika argumen lupa dikirim, aplikasi crash.
- Mengisi `home` sekaligus rute `'/'` di tabel `routes`. Flutter akan memberi error karena keduanya mendefinisikan halaman awal.
- Lupa meneruskan `settings` ke `MaterialPageRoute` sehingga `ModalRoute.of(context)?.settings.name` bernilai `null`.

7. NavigationBar Material 3 & IndexedStack

Aplikasi kampus biasanya punya 3–5 menu utama yang selalu terlihat di bawah layar. Di Material 3, komponennya adalah `NavigationBar` (pengganti `BottomNavigationBar` versi lama).

Konsep

- `NavigationBar` **bukan** Navigator. Berpindah tab tidak melakukan `push`; kita hanya mengganti widget yang ditampilkan di `body` berdasarkan indeks.
- Indeks tab aktif disimpan di `State` dan diubah dengan `setState` di `onDestinationSelected`.
- Setiap item adalah `NavigationDestination` dengan `icon`, `selectedIcon` (opsional), dan `label`.

Contoh lengkap 1: NavigationBar dasar

Struktur aplikasi standar kampus biasanya menggunakan menu tab bawah untuk berpindah antar fitur utama:

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const MainWrapperScreen(),
    );
  }
}

class MainWrapperScreen extends StatefulWidget {
  const MainWrapperScreen({super.key});

  @override
  State<MainWrapperScreen> createState() => _MainWrapperScreenState();
}

class _MainWrapperScreenState extends State<MainWrapperScreen> {
  int _currentIndex = 0;

  // Daftar layar untuk masing-masing tab (urutannya = urutan destinations)
  final List<Widget> _pages = const [
    HalamanTab(judul: "Beranda", ikon: Icons.home),
    HalamanTab(judul: "Jadwal Kuliah", ikon: Icons.calendar_month),
```

```
    HalamanTab(judul: "Kartu Rencana Studi", ikon: Icons.assignment),
    HalamanTab(judul: "Profil Mahasiswa", ikon: Icons.person),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Portal Akademik")),
      body: _pages[_currentIndex], // Menampilkan halaman sesuai tab aktif
      bottomNavigationBar: NavigationBar( // Komponen NavigationBar Material 3
        selectedIndex: _currentIndex,
        onDestinationSelected: (index) {
          setState(() => _currentIndex = index);
        },
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home_outlined), selectedIcon: Icon(Icons.home),
label: "Beranda"),
          NavigationDestination(icon: Icon(Icons.calendar_month_outlined), selectedIcon:
Icon(Icons.calendar_month), label: "Jadwal"),
          NavigationDestination(icon: Icon(Icons.assignment_outlined), selectedIcon:
Icon(Icons.assignment), label: "KRS"),
          NavigationDestination(icon: Icon(Icons.person_outline), selectedIcon: Icon(Icons.person),
label: "Profil"),
        ],
      ),
    );
  }
}

// Halaman sederhana untuk tiap tab. Pada proyek nyata, ganti dengan
// BerandaScreen, JadwalScreen, dst. di file terpisah.
class HalamanTab extends StatelessWidget {
  final String judul;
  final IconData ikon;

  const HalamanTab({super.key, required this.judul, required this.ikon});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(ikon, size: 72, color: Colors.indigo),
          const SizedBox(height: 12),
          Text(judul, style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
        ],
      ),
    );
  }
}
```

NavigationBar Material 3 dengan tab Jadwal aktif

Masalah: state tab hilang saat berpindah

Pada contoh di atas, `body: _pages[_currentIndex]` hanya memasang halaman aktif. Saat pindah tab, halaman lama **dibuang** dari pohon widget. Akibatnya isian form, posisi scroll, dan nilai di `State` tab tersebut hilang. Solusinya adalah `IndexedStack`:

```
// SEBELUM: hanya halaman aktif yang ada di pohon widget
// body: _pages[_currentIndex],

// SESUDAH: semua halaman tetap hidup, hanya satu yang terlihat
body: IndexedStack(
  index: _currentIndex,
  children: _pages,
),
```

`IndexedStack` membangun *semua* anaknya sekali, lalu hanya menampilkan anak pada posisi `index`. Anak yang tersembunyi tetap hidup beserta State-nya.

Contoh lengkap 2: IndexedStack menjaga state tab

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.green),
      home: const MainScreen(),
    );
  }
}

class MainScreen extends StatefulWidget {
  const MainScreen({super.key});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _index = 0;

  static const _judul = ['Presensi', 'Pengumuman', 'Akun'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_judul[_index])),
      // IndexedStack menjaga state setiap tab:
      // angka presensi & posisi scroll tidak hilang saat pindah tab
      body: IndexedStack(
        index: _index,
        children: const [
          PresensiTab(),
          PengumumanTab(),
          Center(child: Text('Halaman Akun')),
        ],
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _index,
        onDestinationSelected: (i) => setState(() => _index = i),
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.how_to_reg_outlined),
            selectedIcon: Icon(Icons.how_to_reg),
            label: 'Presensi',
          ),
          NavigationDestination(
            icon: Badge(label: Text('3'), child: Icon(Icons.campaign_outlined)),
            selectedIcon: Badge(label: Text('3'), child: Icon(Icons.campaign)),
            label: 'Pengumuman',
          ),
          NavigationDestination(
            icon: Icon(Icons.person_outline),
            selectedIcon: Icon(Icons.person),
            label: 'Akun',
          ),
        ],
```

```
      ),
    );
  }
}

// Tab 1: punya state lokal (jumlah kehadiran)
class PresensiTab extends StatefulWidget {
  const PresensiTab({super.key});

  @override
  State<PresensiTab> createState() => _PresensiTabState();
}

class _PresensiTabState extends State<PresensiTab> {
  int _hadir = 0;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text('Hadir: $_hadir pertemuan',
              style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => setState(() => _hadir++),
            child: const Text('Presensi Hari Ini'),
          ),
          const SizedBox(height: 8),
          const Text('Tambah angka, pindah tab, lalu kembali ke sini.'),
        ],
      ),
    );
  }
}

// Tab 2: list panjang (posisi scroll ikut tersimpan)
class PengumumanTab extends StatelessWidget {
  const PengumumanTab({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: 50,
      itemBuilder: (context, i) => ListTile(
        leading: const Icon(Icons.campaign),
        title: Text('Pengumuman ke-${i + 1}'),
      ),
    );
  }
}
```

Tab Presensi setelah tombol ditekan 3 kali

Pindah ke tab Akun lalu kembali: angka tetap 3

Penjelasan:

- `PresensiTab` adalah `StatefulWidget` dengan variabel `_hadir`. Berkat `IndexedStack`, angkanya tidak kembali ke 0 setelah pindah tab.
- `PengumumanTab` berisi 50 item. Gulir ke bawah, pindah tab, lalu kembali: posisi scroll tetap.
- `Badge` menampilkan angka kecil di atas ikon, berguna untuk notifikasi belum dibaca.
- Judul `AppBar` diambil dari list `_judul` sesuai tab aktif.

Pendekatan	Kelebihan	Kekurangan
`_pages[index]`	Hemat memori, tab dibangun saat dibuka	State tab hilang setiap pindah
`IndexedStack`	State tab terjaga	Semua tab dibangun di awal (hati-hati jika tiap tab memanggil API berat)

⚠️ **Kesalahan umum**

- Jumlah `destinations` kurang dari 2. `NavigationBar` butuh minimal dua item.
- Membuat list halaman di dalam `build()` tanpa `const`, lalu heran halaman ter-rebuild. Gunakan `const` jika halaman tidak butuh parameter.
- Memakai `Navigator.push` untuk berpindah tab. Tab bukan tumpukan; cukup ubah indeks.
- Masih memakai `BottomNavigationBar` lama di proyek baru. Untuk tampilan Material 3, pakai `NavigationBar`.

8. Menu Samping dengan Drawer

Drawer adalah panel menu yang muncul dari sisi kiri layar. Cocok untuk menu sekunder yang jumlahnya banyak (Jadwal, Transkrip, Pengaturan, Bantuan) dan tidak muat di `NavigationBar`.

Konsep

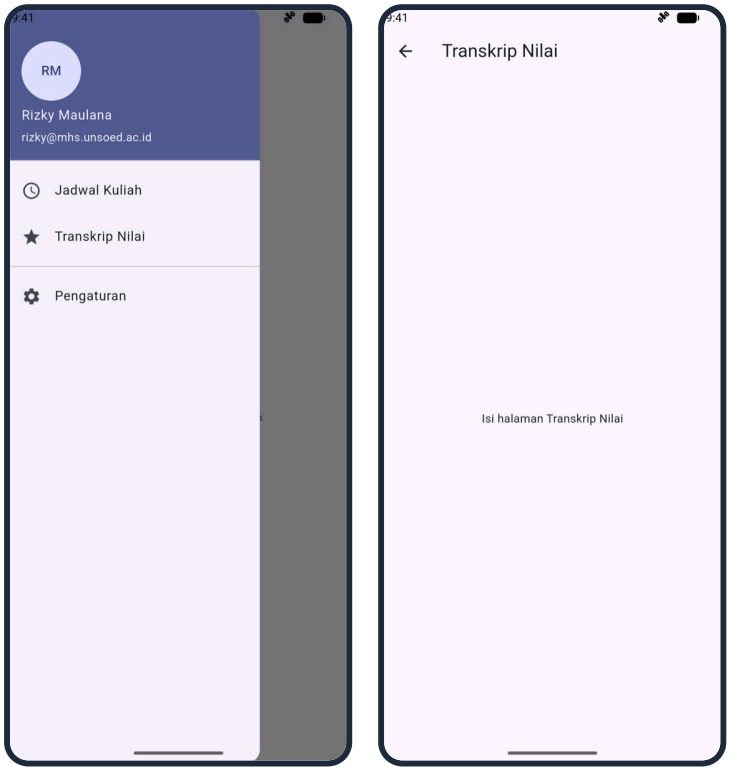
- Pasang drawer di parameter `drawer:` milik `Scaffold`. `AppBar` otomatis menampilkan ikon hamburger (≡).
- Drawer yang terbuka sebenarnya juga sebuah route di tumpukan. Karena itu, menutupnya cukup dengan `Navigator.pop(context)`.
- Saat memilih menu yang membuka halaman baru: **tutup drawer dulu**, baru `push`. Jika tidak, saat pengguna kembali, drawer masih terbuka.

Contoh lengkap: Drawer dengan header akun

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DashboardScreen(),
    );
  }
}

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Ikon hamburger (≡) muncul otomatis di AppBar karena ada drawer
      appBar: AppBar(title: const Text('Dashboard')),
      drawer: const MenuDrawer(),
      body: const Center(child: Text('Geser dari kiri atau tekan ikon ≡')),
    );
  }
}
```

```dart
class MenuDrawer extends StatelessWidget {
  const MenuDrawer({super.key});

  // Tutup drawer DULU, baru buka halaman baru
  void _buka(BuildContext context, Widget halaman) {
    Navigator.pop(context); // menutup drawer (drawer juga sebuah rute!)
    Navigator.push(context, MaterialPageRoute(builder: (_) => halaman));
  }

  @override
  Widget build(BuildContext context) {
    return Drawer(
      child: ListView(
        padding: EdgeInsets.zero,
        children: [
          const UserAccountsDrawerHeader(
            accountName: Text('Rizky Maulana'),
            accountEmail: Text('rizky@mhs.unsoed.ac.id'),
            currentAccountPicture: CircleAvatar(child: Text('RM')),
          ),
          ListTile(
            leading: const Icon(Icons.schedule),
            title: const Text('Jadwal Kuliah'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Jadwal Kuliah')),
          ),
          ListTile(
            leading: const Icon(Icons.grade),
            title: const Text('Transkrip Nilai'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Transkrip Nilai')),
          ),
          const Divider(),
          ListTile(
            leading: const Icon(Icons.settings),
            title: const Text('Pengaturan'),
            onTap: () => _buka(context, const HalamanMenu(judul: 'Pengaturan')),
          ),
        ],
      ),
    );
  }
}

class HalamanMenu extends StatelessWidget {
  final String judul;

  const HalamanMenu({super.key, required this.judul});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(judul)),
      body: Center(child: Text('Isi halaman $judul')),
    );
  }
}
```



Drawer terbuka dengan header akun | Drawer ditutup lalu halaman baru di-push

Penjelasan:

- `UserAccountsDrawerHeader` adalah header siap pakai dengan nama, email, dan foto profil.
- `ListView` dengan `padding: EdgeInsets.zero` agar header menempel sampai atas layar.
- Fungsi `_buka` memanggil `pop` (menutup drawer) lalu `push` halaman tujuan.

Alternatif Material 3: NavigationDrawer

Jika menu drawer dipakai untuk berganti "halaman utama" (seperti tab), gunakan `NavigationDrawer` yang punya penanda menu aktif bawaan:

```
// Alternatif Material 3: NavigationDrawer
drawer: NavigationDrawer(
  selectedIndex: _menu,
  onDestinationSelected: (i) {
    setState(() => _menu = i);
    Navigator.pop(context); // tutup drawer
  },
  children: const [
    Padding(
      padding: EdgeInsets.fromLTRB(28, 16, 16, 10),
      child: Text('Portal Akademik'),
    ),
    NavigationDrawerDestination(icon: Icon(Icons.home), label: Text('Beranda')),
    NavigationDrawerDestination(icon: Icon(Icons.grade), label: Text('Nilai')),
  ],
),
```

💡 **Kapan pakai apa?**

`NavigationBar` untuk 3–5 menu utama yang sering dipakai. `Drawer` untuk menu tambahan. Keduanya boleh dipakai bersamaan dalam satu `Scaffold`.

⚠️ **Kesalahan umum**

- Menulis `Scaffold.of(context).openDrawer()` dengan `context` milik widget yang membuat `Scaffold` itu sendiri. Context tersebut berada di atas `Scaffold`, sehingga error. Bungkus tombol dengan `Builder` atau pisahkan menjadi widget sendiri.
- Lupa menutup drawer sebelum `push`.

9. TabBar & TabBarView

`TabBar` menampilkan deretan tab di bawah `AppBar`, dan `TabBarView` menampilkan isi tab yang bisa digeser kiri-kanan. Cocok untuk mengelompokkan data sejenis, misalnya jadwal per hari atau nilai per semester.

Tiga komponen yang harus selaras

1. `DefaultTabController(length: n)` : pengatur tab aktif.
2. `TabBar(tabs: [...])` : deretan judul tab, biasanya di `AppBar.bottom`.
3. `TabBarView(children: [...])` : isi setiap tab.

Contoh kecil

```
return DefaultTabController(
  length: 2, // HARUS sama dengan jumlah Tab dan jumlah anak TabBarView
  child: Scaffold(
    appBar: AppBar(
      title: const Text('Nilai'),
      bottom: const TabBar(
        tabs: [Tab(text: 'Semester Ganjil'), Tab(text: 'Semester Genap')],
      ),
    ),
    body: const TabBarView(
      children: [
        Center(child: Text('Nilai ganjil')),
        Center(child: Text('Nilai genap')),
      ],
    ),
  ),
);
```

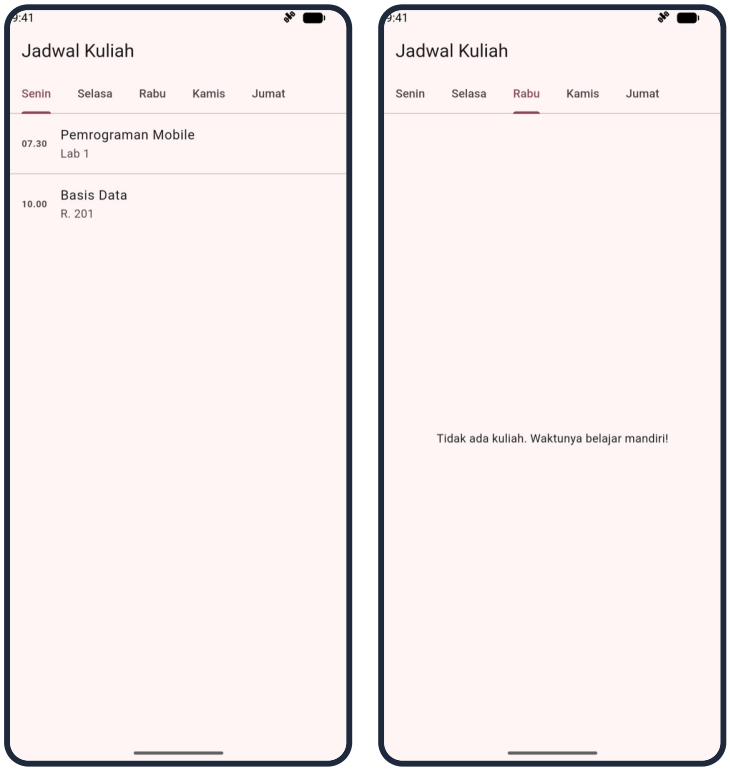
Contoh lengkap: jadwal kuliah per hari

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.pink),
      home: const JadwalScreen(),
    );
```

```
  }
}

class Jadwal {
  final String jam;
  final String matkul;
  final String ruang;

  const Jadwal(this.jam, this.matkul, this.ruang);
}

const jadwalPerHari = {
  'Senin': [
    Jadwal('07.30', 'Pemrograman Mobile', 'Lab 1'),
    Jadwal('10.00', 'Basis Data', 'R. 201'),
  ],
  'Selasa': [Jadwal('08.00', 'Jaringan Komputer', 'Lab 3')],
  'Rabu': <Jadwal>[],
  'Kamis': [Jadwal('13.00', 'Kecerdasan Buatan', 'R. 305')],
  'Jumat': [Jadwal('07.30', 'Etika Profesi', 'Aula')],
};

class JadwalScreen extends StatelessWidget {
  const JadwalScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final hari = jadwalPerHari.keys.toList();

    return DefaultTabController(
      length: hari.length,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('Jadwal Kuliah'),
          bottom: TabBar(
            isScrollable: true, // tab bisa digeser jika banyak
            tabAlignment: TabAlignment.start,
            tabs: [for (final h in hari) Tab(text: h)],
          ),
        ),
        body: TabBarView(
          children: [
            for (final h in hari) DaftarJadwal(items: jadwalPerHari[h]!),
          ],
        ),
      ),
    );
  }
}

class DaftarJadwal extends StatelessWidget {
  final List<Jadwal> items;

  const DaftarJadwal({super.key, required this.items});

  @override
  Widget build(BuildContext context) {
    if (items.isEmpty) {
      return const Center(child: Text('Tidak ada kuliah. Waktunya belajar mandiri!'));
```

```
    }
    return ListView.separated(
      itemCount: items.length,
      separatorBuilder: (context, i) => const Divider(height: 1),
      itemBuilder: (context, i) {
        final j = items[i];
        return ListTile(
          leading: Text(j.jam, style: const TextStyle(fontWeight: FontWeight.bold)),
          title: Text(j.matkul),
          subtitle: Text(j.ruang),
        );
      },
    );
  }
}
```



Tab Senin | Tab Rabu tanpa jadwal

Penjelasan:

- Daftar hari diambil dari `jadwalPerHari.keys`, sehingga `length`, jumlah `Tab`, dan jumlah anak `TabBarView` selalu sama.
- `isScrollable: true` + `tabAlignment: TabAlignment.start` membuat tab bisa digulir dan rata kiri saat jumlahnya banyak.
- Widget `DaftarJadwal` dipakai ulang untuk setiap hari, dengan tampilan khusus jika list kosong.

⚠️ **Kesalahan umum**

- `length` tidak sama dengan jumlah tab atau isi, menghasilkan error *"Controller's length property does not match"*.
- Lupa membungkus dengan `DefaultTabController`, menghasilkan error *"No TabController for TabBar"*.
- Memakai `TabBarView` di dalam `Column` tanpa `Expanded`, sehingga tingginya tidak terbatas.

10. Dialog & Bottom Sheet sebagai Route

Dialog, bottom sheet, dan date picker ternyata juga **route**. `showDialog` melakukan `push` sebuah `DialogRoute` ke tumpukan Navigator. Karena itu dialog ditutup dengan `Navigator.pop`, dan bisa mengembalikan nilai persis seperti halaman biasa.

Contoh kecil: dialog konfirmasi yang mengembalikan bool

```
// showDialog juga melakukan PUSH sebuah rute (DialogRoute)
final bool? yakin = await showDialog<bool>(
  context: context,
  builder: (dialogContext) => AlertDialog(
    title: const Text('Hapus data?'),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(dialogContext, false),
        child: const Text('Batal'),
      ),
      FilledButton(
        onPressed: () => Navigator.pop(dialogContext, true),
        child: const Text('Hapus'),
      ),
    ],
  ),
);
// yakin bisa true, false, atau null (ditutup dengan tap di luar dialog)
if (yakin == true) debugPrint('Data dihapus');
```

- `showDialog<bool>` mengembalikan `Future<bool?>`.
- Nilai `null` muncul ketika pengguna menutup dialog dengan tap di luar dialog atau tombol back.
- Builder dialog memberi `dialogContext` sendiri. Gunakan context ini untuk `pop` agar yang tertutup pasti dialognya.

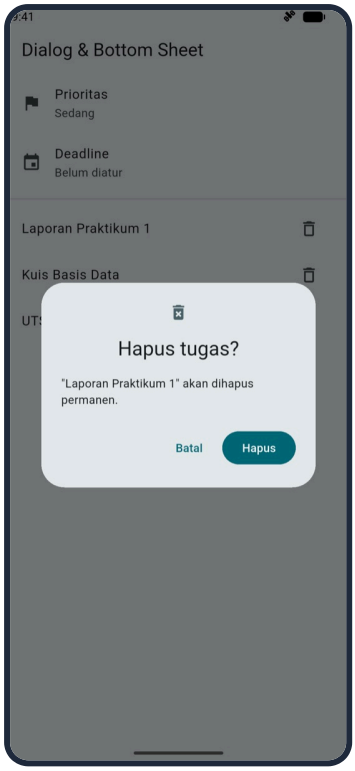
Contoh lengkap: AlertDialog, bottom sheet, dan date picker

`lib/main.dart`

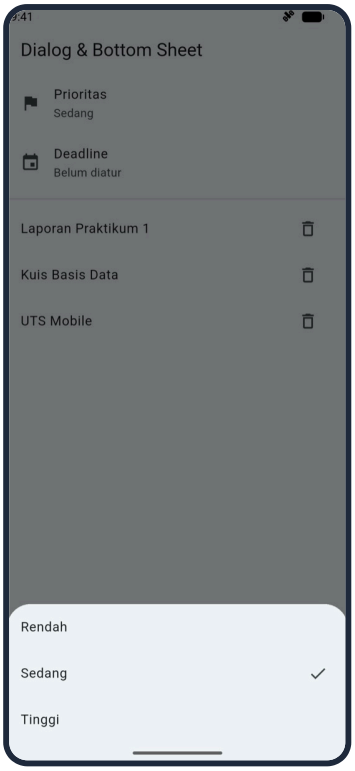
```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      theme: ThemeData(colorSchemeSeed: Colors.cyan),
      home: const DialogDemoScreen(),
    );
  }
}

class DialogDemoScreen extends StatefulWidget {
  const DialogDemoScreen({super.key});

  @override
  State<DialogDemoScreen> createState() => _DialogDemoScreenState();
}

class _DialogDemoScreenState extends State<DialogDemoScreen> {
  final List<String> _tugas = ['Laporan Praktikum 1', 'Kuis Basis Data', 'UTS Mobile'];
  String _prioritas = 'Sedang';
  DateTime? _deadline;

  // 1) AlertDialog yang mengembalikan bool
  Future<void> _hapus(int index) async {
    final yakin = await showDialog<bool>(
      context: context,
      barrierDismissible: false, // wajib pilih tombol, tap di luar tidak menutup
      builder: (dialogContext) => AlertDialog(
        icon: const Icon(Icons.delete_forever),
        title: const Text('Hapus tugas?'),
        content: Text('"${_tugas[index]}" akan dihapus permanen.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    setState(() => _tugas.removeAt(index));
  }

  // 2) Bottom sheet yang mengembalikan String
  Future<void> _pilihPrioritas() async {
    final pilihan = await showModalBottomSheet<String>(
      context: context,
      builder: (sheetContext) => SafeArea(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            for (final p in ['Rendah', 'Sedang', 'Tinggi'])
              ListTile(
                title: Text(p),
                trailing: p == _prioritas ? const Icon(Icons.check) : null,
                onTap: () => Navigator.pop(sheetContext, p),
              ),
          ],
        ),
```

```
      ),
    );
    if (pilihan == null || !mounted) return;
    setState(() => _prioritas = pilihan);
  }

  // 3) Date picker bawaan yang mengembalikan DateTime?
  Future<void> _pilihTanggal() async {
    final sekarang = DateTime.now();
    final tanggal = await showDatePicker(
      context: context,
      firstDate: sekarang,
      lastDate: DateTime(sekarang.year + 1),
      initialDate: _deadline ?? sekarang,
    );
    if (tanggal == null || !mounted) return;
    setState(() => _deadline = tanggal);
  }

  @override
  Widget build(BuildContext context) {
    final teksDeadline = _deadline == null
        ? 'Belum diatur'
        : '${_deadline!.day}/${_deadline!.month}/${_deadline!.year}';

    return Scaffold(
      appBar: AppBar(title: const Text('Dialog & Bottom Sheet')),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.flag),
            title: const Text('Prioritas'),
            subtitle: Text(_prioritas),
            onTap: _pilihPrioritas,
          ),
          ListTile(
            leading: const Icon(Icons.event),
            title: const Text('Deadline'),
            subtitle: Text(teksDeadline),
            onTap: _pilihTanggal,
          ),
          const Divider(),
          for (var i = 0; i < _tugas.length; i++)
            ListTile(
              title: Text(_tugas[i]),
              trailing: IconButton(
                tooltip: 'Hapus tugas',
                icon: const Icon(Icons.delete_outline),
                onPressed: () => _hapus(i),
              ),
            ),
        ],
      ),
    );
  }
}
```



AlertDialog konfirmasi hapus



Bottom sheet pilihan prioritas

Penjelasan:

Fungsi	Tipe hasil	Ditutup dengan
`showDialog<bool>`	`bool?`	`Navigator.pop(dialogContext, true)`
`showModalBottomSheet<String>`	`String?`	`Navigator.pop(sheetContext, p)`
`showDatePicker`	`DateTime?`	Otomatis oleh tombol OK/Cancel

- `barrierDismissible: false` memaksa pengguna memilih tombol.
- Setiap fungsi memeriksa `!mounted` setelah `await` sebelum memanggil `setState`.
- `yakin != true` menangani dua kasus sekaligus: `false` dan `null`.

⚠️ **Kesalahan umum**

- Menulis `if (yakin)` padahal tipenya `bool?`. Dart menolak karena bisa `null`. Tulis `if (yakin == true)`.
- Memanggil `Navigator.pop(context)` dua kali (sekali untuk dialog, sekali "untuk jaga-jaga"), sehingga halaman di bawahnya ikut tertutup.
- Menampilkan SnackBar dengan `dialogContext` setelah dialog tertutup. Gunakan context halaman.

11. Pengenalan go_router

`go_router` adalah package routing resmi dari tim Flutter. Halaman diidentifikasi dengan **URL** (misalnya `/matkul/IF201`), sehingga mendukung deep link, tombol back browser di Flutter Web, dan *auth guard* terpusat.

Instalasi

```
flutter pub add go_router
```

Contoh kecil: rute dan path parameter

```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

// 1) Definisikan semua rute di SATU objek GoRouter
final GoRouter router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const BerandaScreen(),
    ),
    GoRoute(
      path: '/mahasiswa/:nim', // :nim = path parameter
      builder: (context, state) {
        final nim = state.pathParameters['nim']!;
        return ProfilScreen(nim: nim);
      },
    ),
  ],
);

// 2) Pakai MaterialApp.router, bukan MaterialApp biasa
void main() => runApp(MaterialApp.router(routerConfig: router));


// 3) Pindah halaman dengan URL
onPressed: () => context.push('/mahasiswa/H1D023001'),
```

- Semua rute didaftarkan sekali di `GoRouter(routes: [...])`.
- `:nim` pada path adalah **path parameter**. Nilainya dibaca dengan `state.pathParameters['nim']`. Nilainya selalu `String`.
- `MaterialApp.router(routerConfig: router)` menggantikan `MaterialApp(home: ...)`.

context.go vs context.push

Metode	Perilaku	Cocok untuk
`context.go('/x')`	Pindah ke lokasi `/x`. Tumpukan disusun ulang mengikuti struktur rute (sub-rute berada di atas induknya).	Pindah menu utama, setelah login/logout
`context.push('/x')`	Menumpuk `/x` di atas halaman saat ini, seperti `Navigator.push`.	Membuka detail, lalu kembali
`context.pop()`	Kembali satu halaman (bisa membawa hasil).	Tombol kembali/batal

Redirect sebagai auth guard

Fungsi `redirect` dipanggil sebelum setiap perpindahan halaman. Kembalikan path lain untuk membelokkan pengguna, atau `null` untuk membiarkan. Dengan `refreshListenable`, redirect juga dievaluasi ulang setiap kali status login berubah.

Contoh lengkap: login, sub-rute, path & query parameter

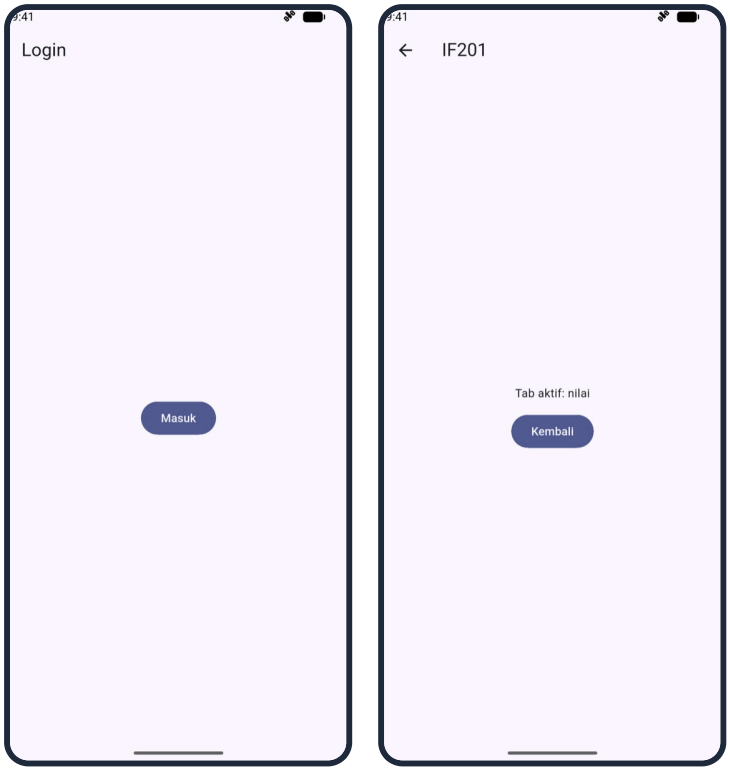
`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah menjalankan: flutter pub add go_router
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

void main() {
  runApp(const MyApp());
}

// Penanda status login sederhana. ChangeNotifier dibahas tuntas di Modul 08;
// di sini cukup tahu bahwa notifyListeners() memberi kabar ke GoRouter.
class AuthState extends ChangeNotifier {
  bool _sudahLogin = false;
  bool get sudahLogin => _sudahLogin;

  void login() {
    _sudahLogin = true;
    notifyListeners();
  }

  void logout() {
    _sudahLogin = false;
    notifyListeners();
  }
}

final authState = AuthState();

final GoRouter router = GoRouter(
```

```
  initialLocation: '/',
  // Router mengevaluasi ulang redirect setiap kali authState berubah
  refreshListenable: authState,
  // AUTH GUARD: dipanggil sebelum setiap perpindahan halaman
  redirect: (context, state) {
    final sedangDiLogin = state.matchedLocation == '/login';
    if (!authState.sudahLogin && !sedangDiLogin) return '/login'; // belum login
    if (authState.sudahLogin && sedangDiLogin) return '/'; // sudah login
    return null; // null = boleh lanjut ke tujuan semula
  },
  routes: [
    GoRoute(
      path: '/login',
      builder: (context, state) => const LoginScreen(),
    ),
    GoRoute(
      path: '/',
      builder: (context, state) => const BerandaScreen(),
      routes: [
        // Sub-rute: URL lengkapnya menjadi /matkul/IF201
        GoRoute(
          path: 'matkul/:kode',
          builder: (context, state) {
            final kode = state.pathParameters['kode']!;
            // Query parameter opsional: /matkul/IF201?tab=nilai
            final tab = state.uri.queryParameters['tab'] ?? 'info';
            return DetailMatkulScreen(kode: kode, tab: tab);
          },
        ),
        GoRoute(
          path: 'tentang',
          builder: (context, state) => const TentangScreen(),
        ),
      ],
    ),
  ],
  // Halaman jika URL tidak dikenal
  errorBuilder: (context, state) => Scaffold(
    body: Center(child: Text('Halaman ${state.uri} tidak ditemukan')),
  ),
);

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      routerConfig: router,
    );
  }
}

class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Center(
        child: FilledButton(
          // Tidak perlu context.go('/'): redirect otomatis memindahkan
          // pengguna ke '/' setelah authState berubah.
          onPressed: authState.login,
          child: const Text('Masuk'),
        ),
      ),
    );
  }
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  static const kodeMatkul = ['IF201', 'IF202', 'IF203'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            icon: const Icon(Icons.logout),
            onPressed: authState.logout, // redirect akan membawa ke /login
          ),
        ],
      ),
      body: ListView(
        children: [
          for (final kode in kodeMatkul)
            ListTile(
              title: Text('Mata kuliah $kode'),
              // push: tumpuk di atas Beranda (ada tombol back)
              onTap: () => context.push('/matkul/$kode'),
            ),
          ListTile(
            title: const Text('Buka nilai IF201 (query parameter)'),
            onTap: () => context.push('/matkul/IF201?tab=nilai'),
          ),
          ListTile(
            title: const Text('Tentang (pakai context.go)'),
            // go: pindah ke lokasi /tentang; tumpukan disusun ulang
            // sesuai struktur rute -> [Beranda, Tentang]
            onTap: () => context.go('/tentang'),
          ),
        ],
      ),
    );
  }
}

class DetailMatkulScreen extends StatelessWidget {
  final String kode;
  final String tab;
```

```
  const DetailMatkulScreen({super.key, required this.kode, required this.tab});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(kode)),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Tab aktif: $tab'),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => context.pop(), // sama seperti Navigator.pop
              child: const Text('Kembali'),
            ),
          ],
        ),
      ),
    );
  }
}

class TentangScreen extends StatelessWidget {
  const TentangScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tentang')),
      body: const Center(child: Text('Contoh go_router')),
    );
  }
}
```



Belum login: redirect ke /login

Path parameter dan query parameter terbaca

Penjelasan:

- Saat aplikasi dibuka, `initialLocation` adalah `/`, tetapi `redirect` melihat pengguna belum login dan membelokkan ke `/login`.
- Tombol Masuk hanya mengubah `authState`. Karena `refreshListenable: authState`, router menjalankan redirect lagi dan memindahkan pengguna ke `/`. Logout bekerja dengan cara yang sama.
- `matkul/:kode` ditulis **tanpa garis miring awal** karena merupakan sub-rute dari `/`. URL lengkapnya `/matkul/IF201`.
- `state.uri.queryParameters['tab']` membaca bagian `?tab=nilai`. Query parameter cocok untuk data opsional.
- `errorBuilder` menampilkan halaman ketika URL tidak cocok dengan rute mana pun.

> ℹ️ **Hubungan dengan Modul 08 dan 13–14**
>
> Pada contoh ini `authState` adalah variabel global agar contoh tetap singkat. Setelah mempelajari Provider, status login akan disimpan di `AuthProvider`. Proyek CRUD + Auth memakai pola yang setara: sebuah auth gate yang menampilkan Login atau Home berdasarkan sesi.

⚠️ **Kesalahan umum**

- Menulis sub-rute dengan garis miring awal (`'/matkul/:kode'` di dalam `routes` milik rute lain). go_router akan menolak konfigurasi tersebut.
- Redirect yang tidak pernah mengembalikan `null`, misalnya selalu `return '/login'`, menyebabkan *redirect loop*.
- Mencampur `MaterialApp(home: ...)` dengan `routerConfig`. Gunakan `MaterialApp.router`.
- Membuat objek `GoRouter` di dalam `build()`. Setiap rebuild akan membuat router baru dan posisi halaman hilang. Buat sekali saja (variabel top-level atau di State).

12. PopScope: Konfirmasi Sebelum Kembali

Kadang tombol back perlu "dicegat", misalnya ketika form sudah diisi tetapi belum disimpan. Widget yang dipakai adalah `PopScope` (pengganti `WillPopScope` yang sudah dihapus dari Flutter).

Dua parameter penting

- `canPop` : `true` berarti back boleh langsung menutup halaman. `false` berarti back diblokir.
- `onPopInvokedWithResult: (didPop, result)` : dipanggil setiap kali ada upaya kembali. `didPop` bernilai `true` jika halaman sudah tertutup, `false` jika diblokir.

Contoh kecil

```
return PopScope(
  canPop: false, // blokir tombol/gestur back
  onPopInvokedWithResult: (bool didPop, Object? result) {
    if (didPop) return; // sudah keluar, tidak perlu apa-apa
    // Di sini: tampilkan dialog konfirmasi, simpan draf, dll.
    debugPrint('Back ditekan tapi diblokir');
  },
  child: const Scaffold(body: Center(child: Text('Form'))),
);
```

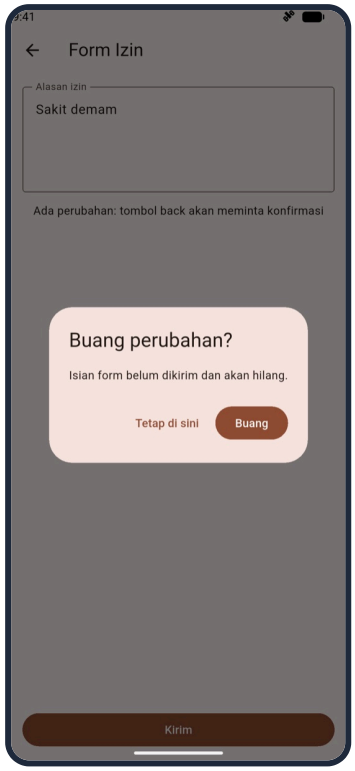
Contoh lengkap: konfirmasi sebelum membuang isian form

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.brown),
      home: const BerandaScreen(),
    );
  }
}

class BerandaScreen extends StatelessWidget {
  const BerandaScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Beranda')),
      body: Center(
        child: FilledButton(
          onPressed: () => Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const FormIzinScreen()),
          ),
          child: const Text('Ajukan Izin Kuliah'),
        ),
      ),
    );
  }
}

class FormIzinScreen extends StatefulWidget {
  const FormIzinScreen({super.key});

  @override
  State<FormIzinScreen> createState() => _FormIzinScreenState();
}

class _FormIzinScreenState extends State<FormIzinScreen> {
  final _alasanController = TextEditingController();
  bool _adaPerubahan = false;

  @override
  void dispose() {
    _alasanController.dispose();
    super.dispose();
  }

  Future<bool> _konfirmasiKeluar() async {
    final keluar = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Buang perubahan?'),
        content: const Text('Isian form belum dikirim dan akan hilang.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Tetap di sini'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Buang'),
          ),
        ],
      ),
    );
    return keluar ?? false;
  }

  void _kirim() {
    // Setelah terkirim, form boleh ditutup tanpa konfirmasi
    setState(() => _adaPerubahan = false);
    Navigator.pop(context);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Pengajuan izin terkirim')),
```

```
    );
  }

  @override
  Widget build(BuildContext context) {
    return PopScope(
      // Jika tidak ada perubahan -> boleh langsung kembali
      canPop: !_adaPerubahan,
      onPopInvokedWithResult: (didPop, result) async {
        if (didPop) return;
        final keluar = await _konfirmasiKeluar();
        // Pakai context milik State, cek mounted setelah await
        if (keluar && context.mounted) {
          Navigator.pop(context);
        }
      },
      child: Scaffold(
        appBar: AppBar(title: const Text('Form Izin')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _alasanController,
                maxLines: 4,
                decoration: const InputDecoration(
                  labelText: 'Alasan izin',
                  border: OutlineInputBorder(),
                ),
                onChanged: (teks) {
                  final berubah = teks.trim().isNotEmpty;
                  if (berubah != _adaPerubahan) {
                    setState(() => _adaPerubahan = berubah);
                  }
                },
              ),
              const SizedBox(height: 12),
              Text(_adaPerubahan
                  ? 'Ada perubahan: tombol back akan meminta konfirmasi'
                  : 'Belum ada perubahan: tombol back langsung kembali'),
              const Spacer(),
              SizedBox(
                width: double.infinity,
                child: FilledButton(onPressed: _kirim, child: const Text('Kirim')),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



Tombol back memicu dialog konfirmasi

Penjelasan:

1. `canPop: !_adaPerubahan`: jika form masih kosong, back berjalan normal.
2. Begitu pengguna mengetik, `_adaPerubahan` menjadi `true`, sehingga `canPop` menjadi `false`.
3. Saat back ditekan, `onPopInvokedWithResult` dipanggil dengan `didPop == false`, lalu dialog konfirmasi ditampilkan.
4. Jika pengguna memilih "Buang", kita menutup halaman sendiri dengan `Navigator.pop(context)`. Pemanggilan `pop` secara langsung dari kode tetap berjalan walaupun `canPop` bernilai `false`.
5. Tombol panah di `AppBar`, tombol back Android, dan gestur back semuanya melewati `PopScope`.

> ⚠️ **Kesalahan umum**
>
> - Memakai `WillPopScope` atau parameter `onPopInvoked` dari tutorial lama. Keduanya sudah usang; gunakan `PopScope` + `onPopInvokedWithResult`.
> - Tidak mengecek `didPop`. Jika halaman sudah tertutup lalu kita memanggil `Navigator.pop` lagi, halaman di bawahnya ikut tertutup.
> - Membuat `canPop: false` permanen tanpa memberi jalan keluar. Pengguna akan "terjebak" di halaman itu.

13. Studi Kasus: Portal Akademik Multi-Halaman

Saatnya menggabungkan semua materi menjadi satu aplikasi **Portal Akademik** multi-halaman: login dengan `pushReplacement`, tab utama dengan `NavigationBar` + `IndexedStack`, detail mata kuliah yang menerima data lewat constructor dan mengembalikan hasil, dialog logout, `pushAndRemoveUntil`, serta `PopScope`.

Struktur folder

```
lib/
├── main.dart
├── models/
|   └── matkul.dart
├── data/
|   └── dummy_data.dart
└── screens/
    ├── login_screen.dart
    ├── main_screen.dart
    └── detail_matkul_screen.dart
```

Peta navigasi

```
LoginScreen --pushReplacement--> MainScreen (tab: Beranda | Mata Kuliah | Profil)
MainScreen  --push<bool>------> DetailMatkulScreen --pop(true)--> MainScreen
MainScreen  --showDialog<bool>--> konfirmasi --pushAndRemoveUntil--> LoginScreen
```

Langkah 1: model dan data

`lib/models/matkul.dart`

```dart
// lib/models/matkul.dart
class Matkul {
  final String kode;
  final String nama;
  final int sks;
  final String dosen;
  final String deskripsi;

  const Matkul({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.dosen,
    required this.deskripsi,
  });
}
```

`lib/data/dummy_data.dart`

```dart
// lib/data/dummy_data.dart
import '../models/matkul.dart';
```

```dart
const List<Matkul> semuaMatkul = [
  Matkul(
    kode: 'IF201',
    nama: 'Pemrograman Mobile',
    sks: 3,
    dosen: 'Dr. Rina Wulandari',
    deskripsi: 'Membangun aplikasi Android & iOS dengan Flutter.',
  ),
  Matkul(
    kode: 'IF202',
    nama: 'Basis Data',
    sks: 3,
    dosen: 'Budi Santoso, M.Kom.',
    deskripsi: 'Perancangan basis data relasional dan SQL.',
  ),
  Matkul(
    kode: 'IF203',
    nama: 'Jaringan Komputer',
    sks: 2,
    dosen: 'Sari Dewi, M.T.',
    deskripsi: 'Model OSI, TCP/IP, dan praktik konfigurasi jaringan.',
  ),
  Matkul(
    kode: 'IF204',
    nama: 'Kecerdasan Buatan',
    sks: 3,
    dosen: 'Dr. Agus Riyadi',
    deskripsi: 'Pencarian, logika, dan dasar machine learning.',
  ),
];
```

Langkah 2: titik masuk aplikasi

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';

import 'screens/login_screen.dart';

void main() {
  runApp(const PortalAkademikApp());
}

class PortalAkademikApp extends StatelessWidget {
  const PortalAkademikApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Portal Akademik',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const LoginScreen(),
    );
  }
}
```

Langkah 3: halaman login

`lib/screens/login_screen.dart`

```dart
// lib/screens/login_screen.dart
import 'package:flutter/material.dart';

import 'main_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _nimController = TextEditingController();

  @override
  void dispose() {
    _nimController.dispose();
    super.dispose();
  }

  void _login() {
    final nim = _nimController.text.trim();
    if (nim.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('NIM wajib diisi')),
      );
      return;
    }
    // Login sukses: ganti Login dengan MainScreen (tidak bisa back ke login)
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => MainScreen(nim: nim)),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Icon(Icons.school, size: 80),
              const SizedBox(height: 16),
              Text('Portal Akademik', style: Theme.of(context).textTheme.headlineMedium),
              const SizedBox(height: 24),
              TextField(
                controller: _nimController,
                decoration: const InputDecoration(
                  labelText: 'NIM',
                  border: OutlineInputBorder(),
```

```
              ),
            ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: _login, child: const Text('Masuk')),
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```

Langkah 4: halaman utama dengan tab

lib/screens/main_screen.dart

```
// lib/screens/main_screen.dart
import 'package:flutter/material.dart';

import '../data/dummy_data.dart';
import '../models/matkul.dart';
import 'detail_matkul_screen.dart';
import 'login_screen.dart';

class MainScreen extends StatefulWidget {
  final String nim;

  const MainScreen({super.key, required this.nim});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _tab = 0;
  final List<Matkul> _diambil = [];

  int get _totalSks => _diambil.fold(0, (total, m) => total + m.sks);

  // Buka detail, tunggu jawaban "ambil" (true) atau tidak
  Future<void> _bukaDetail(Matkul matkul) async {
    final ambil = await Navigator.push<bool>(
      context,
      MaterialPageRoute(
        builder: (_) => DetailMatkulScreen(
          matkul: matkul,
          sudahDiambil: _diambil.contains(matkul),
        ),
      ),
    );
    if (ambil != true || !mounted) return;
    setState(() => _diambil.add(matkul));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        backgroundColor: Colors.green,
```

```
        content: Text('${matkul.nama} masuk KRS'),
      ),
    );
  }

  Future<void> _logout() async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Keluar dari akun?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Logout'),
          ),
        ],
      ),
    );
    if (yakin != true || !mounted) return;
    // Bersihkan SELURUH tumpukan, sisakan Login saja
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (route) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    final tabs = [
      _BerandaTab(nim: widget.nim, totalSks: _totalSks),
      _MatkulTab(diambil: _diambil, onTap: _bukaDetail),
      _ProfilTab(nim: widget.nim, onLogout: _logout),
    ];

    return PopScope(
      // Jika bukan di tab Beranda, tombol back kembali ke Beranda dulu
      canPop: _tab == 0,
      onPopInvokedWithResult: (didPop, result) {
        if (!didPop) setState(() => _tab = 0);
      },
      child: Scaffold(
        appBar: AppBar(title: const Text('Portal Akademik')),
        body: IndexedStack(index: _tab, children: tabs),
        bottomNavigationBar: NavigationBar(
          selectedIndex: _tab,
          onDestinationSelected: (i) => setState(() => _tab = i),
          destinations: [
            const NavigationDestination(icon: Icon(Icons.home), label: 'Beranda'),
            NavigationDestination(
              icon: Badge(
                isLabelVisible: _diambil.isNotEmpty,
                label: Text('${_diambil.length}'),
                child: const Icon(Icons.book),
              ),
```

```
            label: 'Mata Kuliah',
          ),
          const NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    ),
  );
  }
}

class _BerandaTab extends StatelessWidget {
  final String nim;
  final int totalSks;

  const _BerandaTab({required this.nim, required this.totalSks});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text('Selamat datang, $nim'),
          const SizedBox(height: 8),
          Text('Total SKS diambil: $totalSks',
              style: Theme.of(context).textTheme.headlineSmall),
        ],
      ),
    );
  }
}

class _MatkulTab extends StatelessWidget {
  final List<Matkul> diambil;
  final ValueChanged<Matkul> onTap;

  const _MatkulTab({required this.diambil, required this.onTap});

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        for (final m in semuaMatkul)
          ListTile(
            title: Text(m.nama),
            subtitle: Text('${m.kode} • ${m.sks} SKS'),
            trailing: diambil.contains(m)
                ? const Icon(Icons.check_circle, color: Colors.green)
                : const Icon(Icons.chevron_right),
            onTap: () => onTap(m),
          ),
      ],
    );
  }
}

class _ProfilTab extends StatelessWidget {
  final String nim;
  final VoidCallback onLogout;
```

```
  const _ProfilTab({required this.nim, required this.onLogout});

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        ListTile(leading: const Icon(Icons.badge), title: const Text('NIM'), subtitle: Text(nim)),
        ListTile(
          leading: const Icon(Icons.logout, color: Colors.red),
          title: const Text('Logout'),
          onTap: onLogout,
        ),
      ],
    );
  }
}
```

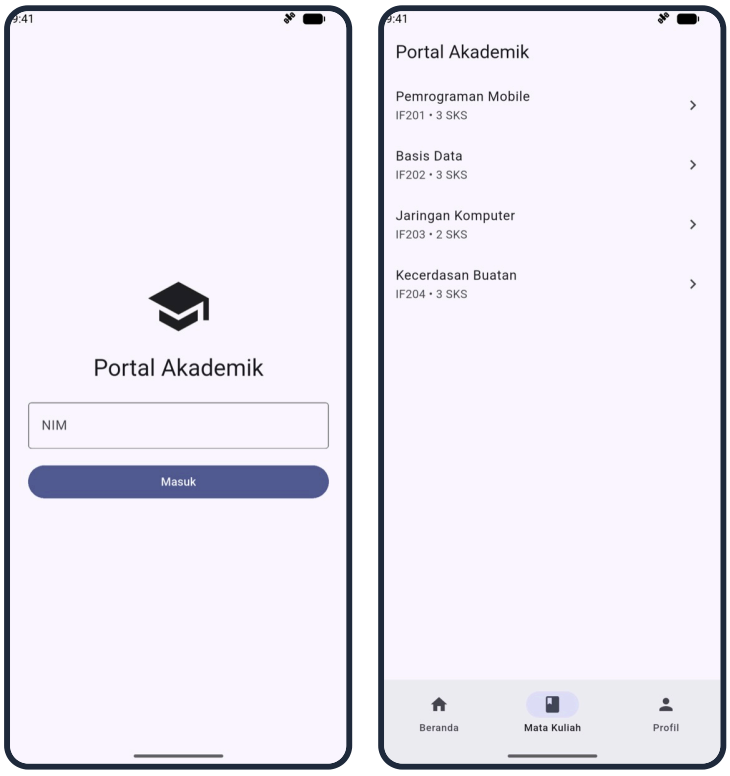
Langkah 5: halaman detail

lib/screens/detail_matkul_screen.dart

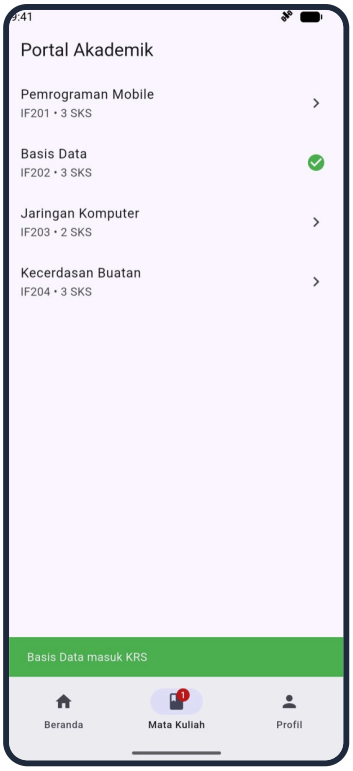
```
// lib/screens/detail_matkul_screen.dart
import 'package:flutter/material.dart';

import '../models/matkul.dart';

class DetailMatkulScreen extends StatelessWidget {
  final Matkul matkul;
  final bool sudahDiambil;

  const DetailMatkulScreen({
    super.key,
    required this.matkul,
    required this.sudahDiambil,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(matkul.kode)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(matkul.nama, style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 8),
            Text('${matkul.sks} SKS • ${matkul.dosen}'),
            const SizedBox(height: 16),
            Text(matkul.deskripsi),
            const Spacer(),
            SizedBox(
              width: double.infinity,
              child: FilledButton.icon(
                icon: const Icon(Icons.add_task),
                label: Text(sudahDiambil ? 'Sudah di KRS' : 'Daftar Matkul Ini'),
                // null = tombol nonaktif
```

```
                onPressed: sudahDiambil ? null : () => Navigator.pop(context, true),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Hasil



Halaman login | Tab Mata Kuliah



Matkul masuk KRS: centang hijau dan badge

Penjelasan:

- **Login** memakai `pushReplacement`, jadi tombol back di MainScreen tidak kembali ke login. NIM dikirim ke `MainScreen` lewat constructor.
- **MainScreen** menyimpan daftar `_diambil` dan mengoper data serta callback ke tiap tab lewat constructor (`_MatkulTab(diambil: ..., onTap: ...)`).
- **Detail** mengembalikan `true` saat tombol "Daftar Matkul Ini" ditekan. MainScreen menunggu hasil itu dengan `await Navigator.push<bool>`.
- **PopScope** di MainScreen: jika pengguna sedang di tab selain Beranda, tombol back memindahkan ke Beranda dulu, bukan menutup aplikasi.
- **Logout** meminta konfirmasi lewat dialog, lalu membersihkan tumpukan dengan `pushAndRemoveUntil`.

> ℹ️ **Menuju Modul 08**
>
> Perhatikan bahwa `_diambil` harus dioper dari `MainScreen` ke tab-tabnya lewat constructor dan callback. Bayangkan jika halaman detail, halaman ringkasan KRS, dan badge di AppBar semuanya butuh data yang sama: constructor akan penuh dengan parameter "titipan". Masalah ini disebut **prop drilling**, dan diselesaikan dengan Provider di modul berikutnya.

💡 **Tantangan tambahan**

- Tambahkan tombol "Batalkan" di halaman detail jika mata kuliah sudah diambil, lalu kembalikan nilai `false` untuk menghapusnya dari `_diambil`.
- Tambahkan `Drawer` berisi menu Tentang Aplikasi.
- Ubah navigasi aplikasi ini menjadi `go_router` dengan rute `/login`, `/`, dan `/matkul/:kode`.

Tugas Praktikum: Praktikum 7: Portal Akademik dengan Navigasi Tab & Halaman Detail

Sasaran Praktikum

Merancang alur navigasi aplikasi kampus yang lengkap mulai dari Splash, Login, Main Tabs, hingga Halaman Detail Berargumen, dialog konfirmasi, dan pencegatan tombol back.

Langkah Pengerjaan

1. Buat proyek baru dan susun folder `lib/models`, `lib/data`, dan `lib/screens` seperti studi kasus section 13.
2. Buat `SplashScreen` yang setelah 2 detik berpindah ke `LoginScreen` dengan `Navigator.pushReplacement` (jangan lupa cek `mounted`).
3. Buat alur layar: `LoginScreen` → `MainNavigationScreen`.
4. Gunakan `Navigator.pushReplacement` dari Login agar mahasiswa tidak bisa menekan tombol back kembali ke layar login setelah login sukses.
5. Di dalam `MainNavigationScreen`, pasang `NavigationBar` Material 3 dengan 3 tab: (1) Beranda Informasi, (2) Daftar Mata Kuliah, (3) Profil Pengguna. Gunakan `IndexedStack` agar posisi scroll tab tidak hilang.
6. Pada tab Daftar Mata Kuliah, ketika salah satu item ditekan, pindah ke `DetailMataKuliahScreen` dengan membawa objek mata kuliah (nama dosen dan deskripsi silabus) lewat constructor.
7. Sediakan tombol 'Daftar Matkul Ini' pada Halaman Detail yang mengembalikan data hasil konfirmasi ke halaman sebelumnya (`await Navigator.push<bool>`) dan menampilkan notifikasi SnackBar hijau.
8. Tambahkan `Drawer` berisi menu Jadwal dan Tentang Aplikasi. Tutup drawer sebelum membuka halaman baru.
9. Halaman Jadwal memakai `TabBar` + `TabBarView` untuk hari Senin sampai Jumat.
10. Tambahkan tombol 'Logout' di tab Profil yang menampilkan `AlertDialog` konfirmasi, lalu mengarahkan kembali ke Login menggunakan `pushAndRemoveUntil`.
11. Pasang `PopScope` di MainNavigationScreen: jika tab aktif bukan Beranda, tombol back memindahkan ke tab Beranda terlebih dahulu.
12. Bonus: implementasikan ulang rute utama dengan `go_router` (rute `/login`, `/`, `/matkul/:kode`) dan `redirect` sebagai auth guard.

Output yang Diharapkan

Alur navigasi aplikasi yang rapi: splash tidak bisa dikembalikan, login tidak tersisa di stack, state tab terjaga, data timbal-balik berfungsi, logout membersihkan seluruh stack, dan tombol back berperilaku sesuai PopScope.

Rubrik Penilaian

Penggunaan push, pop, pushReplacement, dan pushAndRemoveUntil yang tepat (30%), Passing data via constructor dan return result (25%), NavigationBar + IndexedStack, Drawer, dan TabBar yang bersih (25%), Dialog konfirmasi dan PopScope (10%), Struktur folder dan kerapian kode (10%). Bonus go_router +10%.

Kuis Pemahaman

1. Mengapa setelah pengguna berhasil login, kita sebaiknya menggunakan 'Navigator.pushReplacement' alih-alih 'Navigator.push' biasa?

A. Agar layar Login terhapus dari navigation stack, sehingga jika pengguna menekan tombol back di perangkat, aplikasi keluar dan tidak kembali lagi ke layar formulir login

B. Karena push biasa hanya boleh digunakan untuk membuka website di browser

C. Agar warna AppBar otomatis berubah menjadi biru

D. Karena pushReplacement mengompres file gambar lebih cepat

2. Bagaimana cara menerima data balik (return value) yang dikirim oleh sebuah halaman saat halaman tersebut ditutup dengan 'Navigator.pop(context, data)'?

A. Menambahkan keyword 'await' di depan pemanggilan 'Navigator.push'

B. Menyimpan data di dalam file pubspec.yaml

C. Menggunakan fungsi setState() di dalam initState()

D. Mematikan emulator ponsel

3. Apa perbedaan utama context.go() dan context.push() pada go_router?

A. go menyusun ulang tumpukan sesuai struktur rute tujuan, sedangkan push menumpuk halaman baru di atas halaman saat ini

B. go hanya untuk Flutter Web, push hanya untuk Android

C. push menghapus semua halaman, go tidak

D. Keduanya identik, hanya beda nama

4. Pada Navigator.pushAndRemoveUntil(context, route, (route) ⇒ false), apa arti predicate '(route) ⇒ false'?

A. Hapus semua rute lama sehingga hanya rute baru yang tersisa di tumpukan

B. Jangan hapus rute apa pun

C. Hapus hanya rute paling atas

D. Batalkan navigasi

5. Mengapa body: IndexedStack(index: i, children: pages) dipakai alih-alih body: pages[i] pada NavigationBar?

A. Agar state setiap tab (isian, posisi scroll, nilai State) tetap terjaga saat berpindah tab

B. Agar aplikasi lebih hemat memori

C. Karena NavigationBar mewajibkan IndexedStack

D. Agar animasi pindah tab lebih cepat

6. Di dalam onGenerateRoute, bagaimana cara aman membaca settings.arguments yang seharusnya bertipe Mahasiswa?

A. Memeriksa dengan if (args is Mahasiswa) lalu mengembalikan halaman error jika tidak cocok

B. Langsung menulis settings.arguments as Mahasiswa

C. Mengubah arguments menjadi String dengan toString()

D. Menyimpan argumen di variabel global

7. Widget apa yang dipakai di Flutter terbaru untuk mencegat tombol back dan menampilkan konfirmasi keluar?

A. PopScope dengan canPop dan onPopInvokedWithResult

B. WillPopScope dengan onWillPop

C. BackButtonListener di main()

D. GestureDetector dengan onBack

8. showDialog<bool>(...) dipanggil dan pengguna menutup dialog dengan tap di luar kotak dialog. Nilai apa yang diterima?

A. null

B. false

C. true

D. Aplikasi crash

Kunci Jawaban & Pembahasan

1. A — pushReplacement mengganti rute saat ini di stack dengan rute baru. Dengan begitu, rute login tidak tersisa di stack riwayat dan pengguna tidak akan kembali ke login saat menekan tombol back.

2. A — Karena Navigator.push mengembalikan Future<T>, kita dapat menaruh keyword 'await' di depannya: final result = await Navigator.push(...). Ketika layar tujuan ditutup dengan Navigator.pop(context, data), variabel result akan berisi nilai tersebut.

3. A — context.go berpindah ke sebuah lokasi URL dan membangun tumpukan berdasarkan hierarki rute (sub-rute di atas induknya). context.push menumpuk rute di atas halaman saat ini, mirip Navigator.push, sehingga tombol back kembali ke halaman sebelumnya.

4. A — Predicate dipanggil untuk tiap rute lama. Rute dibuang selama predicate mengembalikan false. Karena selalu false, semua rute lama dibuang. Ini pola standar untuk logout.

5. A — IndexedStack membangun semua anak dan hanya menampilkan satu. Anak yang tersembunyi tidak dibuang, sehingga State-nya tetap hidup. Konsekuensinya semua tab dibangun di awal.

6. A — settings.arguments bertipe Object?. Pemeriksaan 'is' mencegah crash ketika argumen lupa dikirim atau tipenya salah, sekaligus melakukan type promotion.

7. A — WillPopScope sudah dihapus. PopScope memakai canPop untuk memblokir back, dan onPopInvokedWithResult(didPop, result) untuk bereaksi, misalnya menampilkan dialog lalu memanggil Navigator.pop sendiri.

8. A — Dialog adalah route. Jika ditutup tanpa Navigator.pop(context, nilai), Future selesai dengan null. Karena itu kondisi ditulis if (yakin == true), bukan if (yakin).

MODUL 08 · STATE MANAGEMENT

Manajemen State Terstruktur dengan Provider

Memahami batasan setState() dan bahaya 'prop drilling' pada aplikasi skala besar. Menguasai Provider: ChangeNotifier, ChangeNotifierProvider, context.watch/read/select, Consumer, Selector, MultiProvider, pola state async (isLoading/error/data), ProxyProvider, struktur folder model/service/provider/UI, auth gate, serta kesalahan umum dan cara memperbaikinya.

Durasi Pertemuan 8 (150 Menit) + Belajar Mandiri

Prasyarat Modul 07: Navigasi & Routing

Capaian Pembelajaran

- Menganalisis perbedaan Ephemeral State (State Lokal) vs App State (State Global).
- Menjelaskan alur reaktif ChangeNotifier dan notifyListeners().
- Menerapkan Provider untuk memisahkan Business Logic dari kode antarmuka UI.
- Mengoptimalkan performa render dengan context.select, Consumer, dan Selector.
- Mengelola beberapa provider dengan MultiProvider dan ChangeNotifierProxyProvider.
- Menerapkan pola state async isLoading/error/data untuk data dari service.
- Menyusun struktur folder model/service/provider/UI dan auth gate berbasis Provider.
- Mengenali dan memperbaiki kesalahan umum: read di build, provider di initState, notify setelah dispose.

Daftar Isi Modul

1. Masalah 'Prop Drilling' & Kapan Butuh State Management?
2. ChangeNotifier dari Nol
3. ChangeNotifierProvider: Menyediakan State ke Pohon Widget
4. watch, read, select, Consumer & Selector
5. Tiga Pilar Utama Arsitektur Provider: Contoh KRS
6. MultiProvider
7. State Async: isLoading, error, data
8. ProxyProvider (Pengenalan)
9. Struktur Folder & Auth Gate
10. Kesalahan Umum & Cara Memperbaikinya
11. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman
12. Tugas Praktikum
13. Kuis Pemahaman & Kunci Jawaban

1. Masalah 'Prop Drilling' & Kapan Butuh State Management?

Ketika aplikasi Flutter tumbuh besar dan memiliki puluhan halaman, mengandalkan `setState()` saja akan menimbulkan masalah serius yang disebut **Prop Drilling**.

Dua jenis state

Jenis	Contoh	Alat yang cocok
Ephemeral state (state lokal)	Tab aktif, isi TextField, checkbox "ingat saya", animasi	`setState` di `StatefulWidget`
App state (state bersama)	User yang login, isi keranjang/KRS, daftar todo, tema gelap	State management, misalnya **Provider**

Aturan praktis: jika data hanya dipakai satu widget, `setState` sudah cukup. Jika data dipakai oleh beberapa widget yang berjauhan atau beberapa halaman, saatnya memakai Provider.

Masalah Prop Drilling vs Solusi Provider

❌ **Tanpa State Management (Prop Drilling)**

Jika Widget Anak di tingkat ke-5 membutuhkan data Mahasiswa yang ada di Root, data tersebut harus dioper satu per satu melewati Widget Tingkat 1 → 2 → 3 → 4 → 5 lewat konstruktor! Widget perantara terpaksa memegang data yang tidak mereka gunakan.

✅ **Dengan Provider (Dependency Injection)**

Data state diletakkan di atas pohon aplikasi (Root/Scope). Widget manapun di kedalaman berapa pun dapat langsung mengakses data dan mendengarkan perubahannya hanya dengan `context.watch()` tanpa melalui widget perantara!

Contoh kecil: data dititipkan dari level ke level

```
// Level 1 punya data
class HomePage extends StatelessWidget {
  final int jumlahKeranjang;
  const HomePage({super.key, required this.jumlahKeranjang});

  @override
  Widget build(BuildContext context) => Header(jumlahKeranjang: jumlahKeranjang);
}
```

```dart
// Level 2 TIDAK memakai data, hanya meneruskan
class Header extends StatelessWidget {
  final int jumlahKeranjang;
  const Header({super.key, required this.jumlahKeranjang});

  @override
  Widget build(BuildContext context) => IkonKeranjang(jumlah: jumlahKeranjang);
}

// Level 3 yang benar-benar butuh data
class IkonKeranjang extends StatelessWidget {
  final int jumlah;
  const IkonKeranjang({super.key, required this.jumlah});

  @override
  Widget build(BuildContext context) => Text('🛒 $jumlah');
}
```

`Header` sama sekali tidak memakai `jumlahKeranjang`, tetapi tetap wajib menerimanya hanya untuk diteruskan ke `IkonKeranjang`.

Contoh lengkap: katalog dan keranjang tanpa Provider

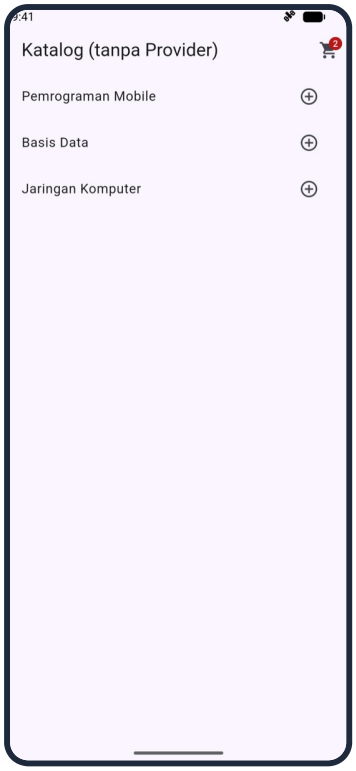
Jalankan kode ini dan hitung berapa constructor yang menerima data "titipan".

lib/main.dart

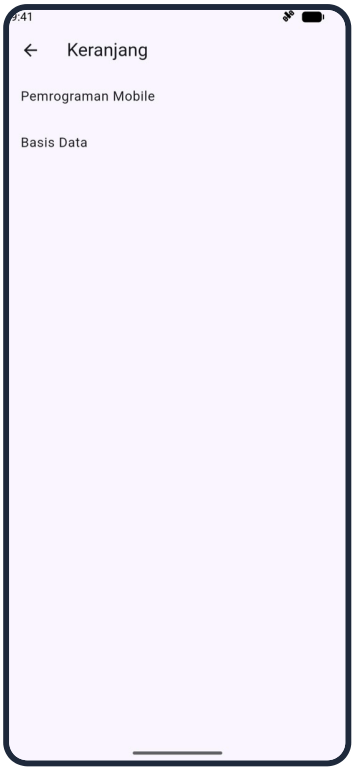
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh TANPA Provider: perhatikan berapa banyak data yang "dititipkan"
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KatalogPage(),
    );
  }
}

// Pemilik state: semua data keranjang ada di sini
class KatalogPage extends StatefulWidget {
  const KatalogPage({super.key});

  @override
  State<KatalogPage> createState() => _KatalogPageState();
}
```

```dart
class _KatalogPageState extends State<KatalogPage> {
  final List<String> _keranjang = [];

  void _tambah(String matkul) => setState(() => _keranjang.add(matkul));

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Titipan 1: jumlah dioper ke AppBarKatalog
      appBar: AppBarKatalog(jumlah: _keranjang.length, isi: _keranjang),
      // Titipan 2: callback dioper ke DaftarMatkul -> ItemMatkul
      body: DaftarMatkul(onTambah: _tambah),
    );
  }
}

class AppBarKatalog extends StatelessWidget implements PreferredSizeWidget {
  final int jumlah;
  final List<String> isi;

  const AppBarKatalog({super.key, required this.jumlah, required this.isi});

  @override
  Size get preferredSize => const Size.fromHeight(kToolbarHeight);

  @override
  Widget build(BuildContext context) {
    return AppBar(
      title: const Text('Katalog (tanpa Provider)'),
      // Titipan 3: data dioper lagi ke TombolKeranjang
      actions: [TombolKeranjang(jumlah: jumlah, isi: isi)],
    );
  }
}

class TombolKeranjang extends StatelessWidget {
  final int jumlah;
  final List<String> isi;

  const TombolKeranjang({super.key, required this.jumlah, required this.isi});

  @override
  Widget build(BuildContext context) {
    return IconButton(
      tooltip: 'Keranjang',
      icon: Badge(label: Text('$jumlah'), child: const Icon(Icons.shopping_cart)),
      onPressed: () {
        // Titipan 4: data dioper ke halaman lain lewat constructor
        Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => KeranjangPage(isi: isi)),
        );
      },
    );
  }
}

class DaftarMatkul extends StatelessWidget {
  final ValueChanged<String> onTambah; // tidak dipakai di sini, hanya diteruskan
```

```dart
  const DaftarMatkul({super.key, required this.onTambah});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: [
        for (final m in matkul) ItemMatkul(nama: m, onTambah: onTambah),
      ],
    );
  }
}

class ItemMatkul extends StatelessWidget {
  final String nama;
  final ValueChanged<String> onTambah;

  const ItemMatkul({super.key, required this.nama, required this.onTambah});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(nama),
      trailing: IconButton(
        tooltip: 'Tambah $nama',
        icon: const Icon(Icons.add_circle_outline),
        onPressed: () => onTambah(nama), // akhirnya dipakai di level terdalam
      ),
    );
  }
}

class KeranjangPage extends StatelessWidget {
  final List<String> isi;

  const KeranjangPage({super.key, required this.isi});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Keranjang')),
      // Masalah tambahan: halaman ini tidak bisa MENGHAPUS item
      // tanpa menitipkan callback lagi dari KatalogPage.
      body: ListView(children: [for (final m in isi) ListTile(title: Text(m))]),
    );
  }
}
```



Katalog: badge menerima jumlah lewat constructor



Keranjang menerima isi lewat constructor

Analisis masalah:

- `AppBarKatalog` dan `DaftarMatkul` hanya menjadi "kurir": menerima data/callback lalu meneruskannya.
- Menambah fitur hapus di `KeranjangPage` berarti menambah callback baru yang harus melewati beberapa widget lagi.
- `KeranjangPage` menerima *salinan referensi* list saat dibuka. Jika list berubah, halaman itu tidak tahu harus rebuild karena tidak ada yang memanggil `setState` di sana.
- Setiap `setState` di `KatalogPage` membangun ulang seluruh halaman, termasuk bagian yang tidak berubah.

> ℹ️ **Apa yang akan kita pelajari**
>
> Di section berikutnya kita membangun solusinya selangkah demi selangkah: `ChangeNotifier` (tempat state), `ChangeNotifierProvider` (cara menyediakan state ke pohon widget), lalu `watch` / `read` / `select` (cara membaca state dari widget mana pun).

⚠️ **Kesalahan umum**

- Langsung memakai Provider untuk *semua* state. Isi TextField atau tab aktif tetap lebih sederhana dengan `setState`.
- Menyimpan state bersama di variabel global biasa. UI tidak akan otomatis rebuild ketika nilainya berubah.

2. ChangeNotifier dari Nol

`ChangeNotifier` adalah class bawaan Flutter (bukan dari package provider) yang bisa "mengabari" siapa saja yang mendengarkan ketika datanya berubah. Inilah pondasi Provider.

Empat langkah membuat ChangeNotifier

1. Buat class yang `extends ChangeNotifier`.
2. Simpan state di field **privat** (diawali `_`).
3. Sediakan **getter** untuk membaca state.
4. Sediakan **method** untuk mengubah state, dan panggil `notifyListeners()` di akhir.

Contoh kecil: tanpa UI sama sekali

```dart
import 'package:flutter/foundation.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0; // 1. state disimpan privat

  int get nilai => _nilai; // 2. dibaca lewat getter

  void tambah() {
    _nilai++; // 3. ubah state
    notifyListeners(); // 4. kabari semua pendengar
  }
}

void main() {
  final counter = CounterModel();

  // Daftarkan "pendengar": fungsi yang dipanggil setiap notifyListeners()
  counter.addListener(() {
    debugPrint('Nilai berubah menjadi ${counter.nilai}');
  });

  counter.tambah(); // Nilai berubah menjadi 1
  counter.tambah(); // Nilai berubah menjadi 2
}
```

- `addListener` mendaftarkan fungsi yang akan dipanggil setiap `notifyListeners()`.
- Dari luar class, `_nilai` tidak bisa diubah langsung. Satu-satunya jalan adalah lewat `tambah()`. Ini menjaga aturan bisnis tetap di satu tempat.

Contoh lengkap: menghubungkan ke UI dengan ListenableBuilder

Sebelum memakai package provider, mari lihat bahwa ChangeNotifier sudah bisa dipakai dengan widget bawaan `ListenableBuilder`.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// ChangeNotifier TANPA package provider, memakai ListenableBuilder bawaan Flutter
import 'package:flutter/material.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }

  void reset() {
    _nilai = 0;
    notifyListeners();
  }
}

// Satu objek model untuk seluruh aplikasi (sementara, sebelum pakai Provider)
final counter = CounterModel();

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const CounterPage(),
    );
  }
}

// Perhatikan: StatelessWidget, tidak ada setState sama sekali
class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('ChangeNotifier dari Nol')),
      body: Center(
        // ListenableBuilder = addListener + rebuild otomatis
        child: ListenableBuilder(
          listenable: counter,
          builder: (context, child) {
            return Text(
              'Jumlah klik: ${counter.nilai}',
              style: Theme.of(context).textTheme.headlineMedium,
            );
          },
```

```
        ),
      ),
      floatingActionButton: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          FloatingActionButton(
            heroTag: 'reset',
            tooltip: 'Reset',
            onPressed: counter.reset,
            child: const Icon(Icons.refresh),
          ),
          const SizedBox(width: 12),
          FloatingActionButton(
            heroTag: 'tambah',
            tooltip: 'Tambah',
            onPressed: counter.tambah,
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```

Tiga kali Tambah tanpa setState

Penjelasan:

- `CounterPage` adalah `StatelessWidget`, tetapi angkanya tetap berubah. Yang memicu rebuild adalah `ListenableBuilder`, bukan `setState`.
- Hanya bagian di dalam `builder` yang di-build ulang. `AppBar` dan tombol tidak ikut.

- Kelemahan contoh ini: `counter` adalah variabel global. Sulit diganti saat testing, dan tidak pernah di-`dispose`. Package provider menyelesaikan kedua masalah ini.

> ⚠️ **Kesalahan umum**
>
> - **Lupa `notifyListeners()`.** Data berubah di memori, tetapi layar tidak berubah sama sekali.
> - Memanggil `notifyListeners()` di dalam getter. Getter hanya membaca; memanggil notify di sana menyebabkan rebuild tanpa henti.
> - Membuat field publik (`int nilai = 0;`) lalu mengubahnya dari UI: `model.nilai++`. Tidak ada yang memanggil notify, dan aturan bisnis tersebar di UI.

3. ChangeNotifierProvider: Menyediakan State ke Pohon Widget

`ChangeNotifierProvider` dari package **provider** "menitipkan" sebuah ChangeNotifier di pohon widget. Semua widget di bawahnya bisa mengambil objek itu lewat `context`, tanpa constructor.

Instalasi

```
flutter pub add provider
```

Contoh kecil: mendaftarkan provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class CounterModel extends ChangeNotifier {
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }
}

void main() {
  runApp(
    // create dipanggil SEKALI; Provider juga otomatis memanggil dispose()
    ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: const MyApp(), // seluruh MyApp & turunannya bisa mengakses CounterModel
    ),
  );
}
```

- `create` dipanggil sekali (secara *lazy*, saat pertama kali dibaca) untuk membuat objek.
- Saat provider dibuang dari pohon widget, ia otomatis memanggil `dispose()` pada objek tersebut.
- Provider di atas `MaterialApp` berarti tersedia di **semua halaman**, termasuk halaman yang dibuka dengan `Navigator.push`.

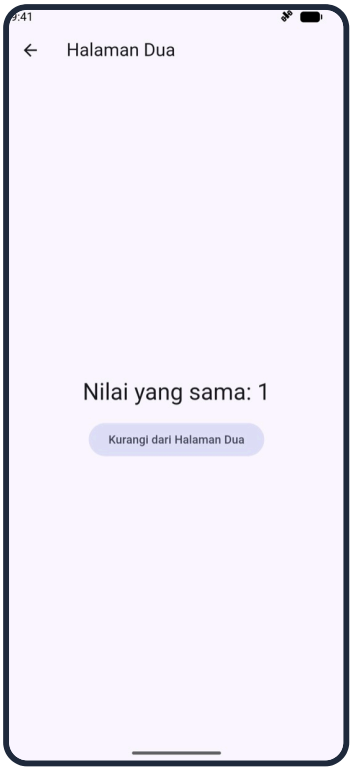
Contoh lengkap: satu state, dua halaman

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class CounterModel extends ChangeNotifier {
```

```
  int _nilai = 0;
  int get nilai => _nilai;

  void tambah() {
    _nilai++;
    notifyListeners();
  }

  void kurang() {
    if (_nilai == 0) return;
    _nilai--;
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const HalamanSatu(),
    );
  }
}

class HalamanSatu extends StatelessWidget {
  const HalamanSatu({super.key});

  @override
  Widget build(BuildContext context) {
    // watch: widget ini rebuild setiap CounterModel memanggil notifyListeners
    final nilai = context.watch<CounterModel>().nilai;

    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Satu')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Nilai: $nilai', style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 16),
            FilledButton(
              // read: hanya memanggil method, tidak berlangganan
              onPressed: () => context.read<CounterModel>().tambah(),
              child: const Text('Tambah'),
            ),
            const SizedBox(height: 8),
```

```
            OutlinedButton(
              // Tidak ada data yang dikirim lewat constructor!
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(builder: (_) => const HalamanDua()),
              ),
              child: const Text('Buka Halaman Dua'),
            ),
          ],
        ),
      ),
    );
  }
}

class HalamanDua extends StatelessWidget {
  const HalamanDua({super.key});

  @override
  Widget build(BuildContext context) {
    final nilai = context.watch<CounterModel>().nilai;

    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Dua')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Nilai yang sama: $nilai',
                style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 16),
            FilledButton.tonal(
              onPressed: () => context.read<CounterModel>().kurang(),
              child: const Text('Kurangi dari Halaman Dua'),
            ),
          ],
        ),
      ),
    );
  }
}
```



Halaman Dua membaca dan mengubah state yang sama

Penjelasan:

- `HalamanDua` dibuka tanpa mengirim data apa pun, tetapi bisa membaca dan mengubah nilai yang sama.
- Saat menekan "Kurangi" di Halaman Dua lalu kembali, Halaman Satu sudah menampilkan nilai terbaru karena ia juga `watch` objek yang sama.
- `context.watch<CounterModel>()` dipakai di `build`; `context.read<CounterModel>()` dipakai di `onPressed`. Bedanya dibahas di section berikutnya.

> ⚠️ **Kesalahan umum**
>
> - **ProviderNotFoundException.** Muncul jika widget mencari provider yang tidak ada di atasnya. Penyebab tersering: provider dipasang di dalam satu halaman, lalu halaman lain (hasil `push`) mencoba membacanya. Pasang provider di atas `MaterialApp` jika dipakai lintas halaman.
> - Lupa menulis tipe generik: `context.watch()` tanpa `<CounterModel>` membuat Dart tidak tahu provider mana yang dicari.
> - Memakai `Provider` biasa (bukan `ChangeNotifierProvider`) untuk ChangeNotifier. Hasilnya UI tidak pernah rebuild.

4. watch, read, select, Consumer & Selector

Ada beberapa cara membaca provider. Perbedaannya ada pada **kapan widget di-build ulang**. Memilih cara yang tepat membuat aplikasi benar sekaligus hemat rebuild.

Ringkasan lima cara

```
// 1) watch: baca + langganan SEMUA perubahan (di dalam build)
final profil = context.watch<ProfilProvider>();

// 2) select: baca + langganan SATU nilai saja (di dalam build)
final nama = context.select((ProfilProvider p) => p.nama);

// 3) read: ambil objek tanpa langganan (di callback/event)
void onTekan() => context.read<ProfilProvider>().tambahPoin();

// 4) Consumer: langganan, tetapi hanya bagian builder yang rebuild
final widgetPoin = Consumer<ProfilProvider>(
  builder: (context, p, child) => Text('Poin: ${p.poin}'),
);

// 5) Selector: seperti Consumer + select
final widgetPoin2 = Selector<ProfilProvider, int>(
  selector: (context, p) => p.poin,
  builder: (context, poin, child) => Text('Poin: $poin'),
);
```

Cara	Rebuild ketika	Dipakai di
`context.watch<T>()`	Setiap `notifyListeners()`	`build()`
`context.select<T, R>(fn)`	Hanya jika nilai hasil `fn` berubah	`build()`
`context.read<T>()`	Tidak pernah	Callback: `onPressed`, `onTap`, `initState` (lewat post-frame)
`Consumer<T>`	Setiap notify, tetapi hanya isi `builder`	Di tengah pohon widget
`Selector<T, R>`	Hanya jika nilai `selector` berubah, hanya isi `builder`	Di tengah pohon widget

Contoh lengkap: membuktikan perbedaan rebuild

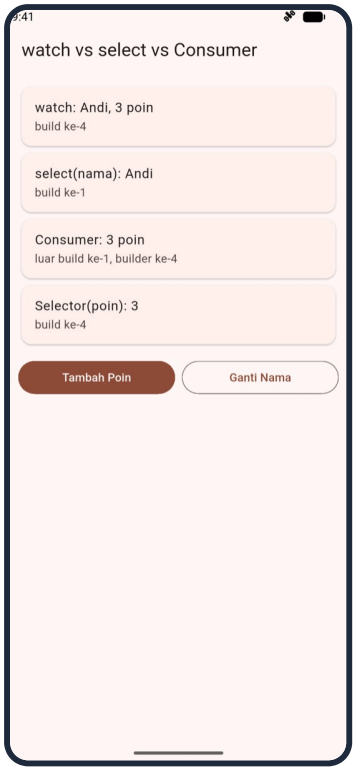
Setiap kartu menampilkan berapa kali ia di-build. Tekan tombol dan amati angka mana yang naik.

lib/main.dart

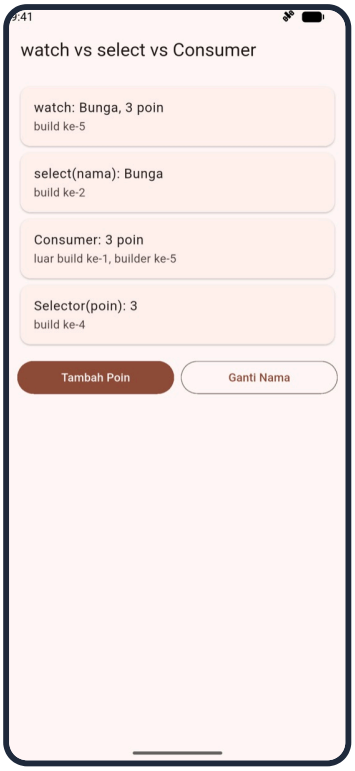
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Demo rebuild: angka "build ke-N" menunjukkan berapa kali tiap kartu di-build
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class ProfilProvider extends ChangeNotifier {
  String _nama = 'Andi';
  int _poin = 0;

  String get nama => _nama;
  int get poin => _poin;

  void tambahPoin() {
    _poin++;
    notifyListeners();
  }

  void gantiNama() {
    _nama = _nama == 'Andi' ? 'Bunga' : 'Andi';
    notifyListeners();
  }
}

// Penghitung build HANYA untuk keperluan demo (jangan ditiru di aplikasi nyata)
final Map<String, int> jumlahBuild = {};
int catatBuild(String nama) => jumlahBuild[nama] = (jumlahBuild[nama] ?? 0) + 1;

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => ProfilProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const DemoPage(),
    );
  }
}

class DemoPage extends StatelessWidget {
  const DemoPage({super.key});

  @override
  Widget build(BuildContext context) {
    // Halaman ini TIDAK watch apa pun, jadi tidak ikut rebuild
    return Scaffold(
      appBar: AppBar(title: const Text('watch vs select vs Consumer')),
```

```
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          const KartuWatch(),
          const KartuSelectNama(),
          const KartuConsumer(),
          const KartuSelectorPoin(),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: FilledButton(
                  onPressed: () => context.read<ProfilProvider>().tambahPoin(),
                  child: const Text('Tambah Poin'),
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: OutlinedButton(
                  onPressed: () => context.read<ProfilProvider>().gantiNama(),
                  child: const Text('Ganti Nama'),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

// 1) watch: rebuild untuk SETIAP perubahan (nama maupun poin)
class KartuWatch extends StatelessWidget {
  const KartuWatch({super.key});

  @override
  Widget build(BuildContext context) {
    final p = context.watch<ProfilProvider>();
    final n = catatBuild('watch');
    return Card(
      child: ListTile(
        title: Text('watch: ${p.nama}, ${p.poin} poin'),
        subtitle: Text('build ke-$n'),
      ),
    );
  }
}

// 2) select: rebuild HANYA jika nama berubah
class KartuSelectNama extends StatelessWidget {
  const KartuSelectNama({super.key});

  @override
  Widget build(BuildContext context) {
    final nama = context.select((ProfilProvider p) => p.nama);
    final n = catatBuild('select');
    return Card(
      child: ListTile(
        title: Text('select(nama): $nama'),
```

```
        subtitle: Text('build ke-$n'),
      ),
    );
  }
}

// 3) Consumer: kartu luar tidak rebuild, hanya isi builder
class KartuConsumer extends StatelessWidget {
  const KartuConsumer({super.key});

  @override
  Widget build(BuildContext context) {
    final nLuar = catatBuild('consumer-luar');
    return Card(
      child: Consumer<ProfilProvider>(
        builder: (context, p, child) {
          final nDalam = catatBuild('consumer-dalam');
          return ListTile(
            title: Text('Consumer: ${p.poin} poin'),
            subtitle: Text('luar build ke-$nLuar, builder ke-$nDalam'),
          );
        },
      ),
    );
  }
}

// 4) Selector: builder rebuild HANYA jika poin berubah
class KartuSelectorPoin extends StatelessWidget {
  const KartuSelectorPoin({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Selector<ProfilProvider, int>(
        selector: (context, p) => p.poin,
        builder: (context, poin, child) {
          final n = catatBuild('selector');
          return ListTile(
            title: Text('Selector(poin): $poin'),
            subtitle: Text('build ke-$n'),
          );
        },
      ),
    );
  }
}
```



Setelah 3x Tambah Poin: kartu select(nama) tidak rebuild



Setelah Ganti Nama: Selector(poin) tidak rebuild

Penjelasan hasil:

- **Tambah Poin**: kartu *watch*, *Consumer (builder)*, dan *Selector* naik. Kartu *select(nama)* tetap karena nama tidak berubah.
- **Ganti Nama**: kartu *watch*, *select(nama)*, dan *Consumer (builder)* naik. *Selector(poin)* tetap.
- Angka "luar" pada kartu Consumer tidak pernah naik. Hanya bagian di dalam `builder` yang di-build ulang.
- `DemoPage` sendiri tidak pernah rebuild karena hanya memakai `read` di tombol.

> 💡 **Cara memilih**
>
> 1. Di callback? Pakai `read`.
> 2. Di `build` dan butuh banyak field? Pakai `watch`.
> 3. Di `build` dan butuh satu nilai saja? Pakai `select`.
> 4. Widget besar tapi hanya sebagian kecil yang berubah? Bungkus bagian kecil itu dengan `Consumer` / `Selector`, atau pecah menjadi widget sendiri.

⚠️ **Kesalahan umum**

- `context.read` di dalam `build`: layar tidak ter-update karena widget tidak berlangganan.
- `context.watch` di dalam `onPressed`: error saat tombol ditekan.
- `select` yang mengembalikan list baru setiap kali, misalnya `select((p) => p.items.where(...).toList())`. List baru selalu dianggap "berbeda", sehingga select tidak menghemat apa pun. Pilih nilai sederhana seperti `int`, `bool`, atau `String`.

5. Tiga Pilar Utama Arsitektur Provider: Contoh KRS

Sekarang kita gabungkan tiga pilar Provider dalam satu contoh nyata: pengisian KRS dengan batas maksimal 24 SKS.

Untuk mengimplementasikan package `provider` (dari pub.dev), kita menggunakan 3 pilar utama:

1. **ChangeNotifier**: tempat state dan logika bisnis.
2. **ChangeNotifierProvider**: menyediakan state ke pohon widget.
3. **watch / read**: membaca state dari widget.

Pasang package terlebih dahulu: `flutter pub add provider`.

Langkah 1: Membuat Model State dengan ChangeNotifier

Buat folder `lib/providers/` lalu file `krs_provider.dart`:

`lib/providers/krs_provider.dart`

```dart
// lib/providers/krs_provider.dart
import 'package:flutter/foundation.dart';

class MataKuliah {
  final String nama;
  final int sks;

  const MataKuliah(this.nama, this.sks);
}

class KrsProvider with ChangeNotifier {
  // Data privat di dalam store
  final List<MataKuliah> _matkulTerpilih = [];
  final int batasMaksimalSks = 24;

  // Getter publik (read-only agar tidak bisa diubah langsung dari luar)
  List<MataKuliah> get matkulTerpilih => List.unmodifiable(_matkulTerpilih);
  int get totalSks => _matkulTerpilih.fold(0, (total, mk) => total + mk.sks);

  // Logika bisnis untuk menambah mata kuliah
  bool tambahMatkul(MataKuliah matkul) {
    if (totalSks + matkul.sks > batasMaksimalSks) {
      return false; // Melebihi batas kuota SKS!
    }

    _matkulTerpilih.add(matkul);

    // SANGAT PENTING: Beritahu seluruh widget yang mendengarkan agar me-rebuild UI!
    notifyListeners();
    return true;
  }

  void hapusMatkul(int index) {
    _matkulTerpilih.removeAt(index);
    notifyListeners();
```

```
  }
}
```

Perhatikan bahwa `tambahMatkul` mengembalikan `bool`. Provider yang memutuskan boleh/tidaknya (aturan bisnis), sedangkan UI hanya menampilkan pesan.

Langkah 2: Mendaftarkan Provider di Root Aplikasi (main.dart)

```dart
// Potongan inti dari lib/main.dart (kode lengkap ada di Langkah 3)
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/krs_provider.dart';

void main() {
  runApp(
    // Membungkus aplikasi dengan ChangeNotifierProvider
    ChangeNotifierProvider(
      create: (context) => KrsProvider(),
      child: const MyApp(),
    ),
  );
}
```

Langkah 3: Mengonsumsi State di Widget (context.watch vs context.read)

Metode Akses	Tujuan & Sifat Reaktif	Tempat Penggunaan
`context.watch<T>()`	Membaca data DAN ikut me-render ulang widget setiap ada perubahan (`notifyListeners`).	Di dalam method `build()` saat menampilkan teks/angka.
`context.read<T>()`	Hanya membaca fungsi/method SATU KALI tanpa mendaftar untuk re-render.	Di dalam event handler seperti tombol `onPressed: () {}`.

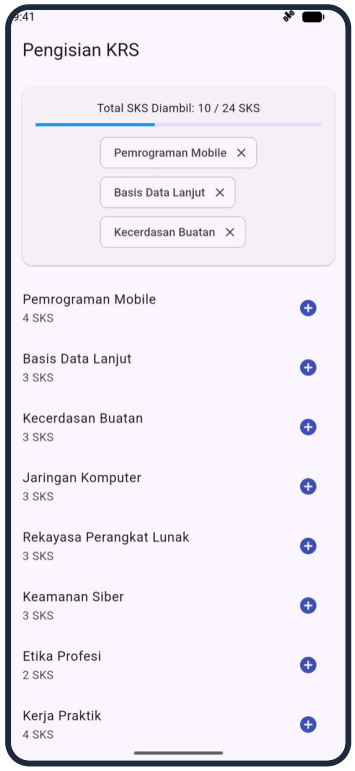
Kode lengkap `lib/main.dart` yang siap dijalankan (katalog mata kuliah + ringkasan SKS):

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: (1) flutter pub add provider, (2) membuat lib/providers/krs_provider.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/krs_provider.dart';

void main() {
  runApp(
    // Membungkus aplikasi dengan ChangeNotifierProvider di root
    ChangeNotifierProvider(
      create: (context) => KrsProvider(),
```

```
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: KatalogKrsScreen(),
    );
  }
}

class KatalogKrsScreen extends StatelessWidget {
  const KatalogKrsScreen({super.key});

  static const List<MataKuliah> katalog = [
    MataKuliah("Pemrograman Mobile", 4),
    MataKuliah("Basis Data Lanjut", 3),
    MataKuliah("Kecerdasan Buatan", 3),
    MataKuliah("Jaringan Komputer", 3),
    MataKuliah("Rekayasa Perangkat Lunak", 3),
    MataKuliah("Keamanan Siber", 3),
    MataKuliah("Etika Profesi", 2),
    MataKuliah("Kerja Praktik", 4),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Pengisian KRS")),
      body: Column(
        children: [
          const RingkasanKrsWidget(),
          Expanded(
            child: ListView.builder(
              itemCount: katalog.length,
              itemBuilder: (context, index) {
                final mk = katalog[index];
                return ListTile(
                  title: Text(mk.nama),
                  subtitle: Text("${mk.sks} SKS"),
                  trailing: IconButton(
                    tooltip: "Ambil ${mk.nama}",
                    icon: const Icon(Icons.add_circle, color: Colors.indigo),
                    onPressed: () {
                      // 2. Di event handler gunakan read: hanya memanggil method, tanpa rebuild
                      final berhasil = context.read<KrsProvider>().tambahMatkul(mk);
                      if (!berhasil) {
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(
                            content: Text("Kuota maksimal 24 SKS terlampaui!"),
                            backgroundColor: Colors.red,
                          ),
                        );
```

```
                    }
                  },
                ),
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}

// Contoh Widget Tampilan KRS Menggunakan Provider
class RingkasanKrsWidget extends StatelessWidget {
  const RingkasanKrsWidget({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. Menggunakan watch untuk mendengarkan perubahan angka SKS
    final krs = context.watch<KrsProvider>();

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Text("Total SKS Diambil: ${krs.totalSks} / ${krs.batasMaksimalSks} SKS"),
            const SizedBox(height: 8),
            LinearProgressIndicator(
              value: krs.totalSks / krs.batasMaksimalSks,
              color: krs.totalSks >= 20 ? Colors.orange : Colors.blue,
            ),
            const SizedBox(height: 8),
            Wrap(
              spacing: 6,
              children: [
                for (var i = 0; i < krs.matkulTerpilih.length; i++)
                  InputChip(
                    label: Text(krs.matkulTerpilih[i].nama),
                    onDeleted: () => context.read<KrsProvider>().hapusMatkul(i),
                  ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```



Ringkasan SKS ikut berubah saat mata kuliah diambil

Penjelasan:

- `KatalogKrsScreen` tidak memakai `watch`, sehingga daftar katalog tidak ikut rebuild ketika SKS bertambah.
- `RingkasanKrsWidget` memakai `watch` sehingga total SKS, progress bar, dan chip selalu sinkron.
- `InputChip.onDeleted` memanggil `hapusMatkul` lewat `read`.
- `List.unmodifiable` di getter mencegah UI menambah item langsung tanpa melewati `tambahMatkul` (yang memeriksa kuota).

> ⚠️ **Kesalahan umum**
>
> - Memeriksa batas SKS di UI, bukan di provider. Jika ada dua tombol tambah di dua halaman, aturannya harus ditulis dua kali.
> - Mengembalikan `_matkulTerpilih` langsung dari getter. UI bisa memanggil `.add()` dan melewati pengecekan kuota.

6. MultiProvider

Aplikasi nyata biasanya punya lebih dari satu provider: sesi login, tema, keranjang, data mata kuliah, dan seterusnya. `MultiProvider` mendaftarkan semuanya sekaligus tanpa nesting yang menjorok ke kanan.

Tanpa vs dengan MultiProvider

```
// Tanpa MultiProvider (sulit dibaca)
ChangeNotifierProvider(
  create: ... TemaProvider,
  child: ChangeNotifierProvider(
    create: ... SesiProvider,
    child: ChangeNotifierProvider(
      create: ... FavoritProvider,
      child: MyApp(),
    ),
  ),
)
```

```
// Dengan MultiProvider
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: ... Tema),
    ChangeNotifierProvider(create: ... Sesi),
    ChangeNotifierProvider(create: ... Favorit),
  ],
  child: MyApp(),
)
```

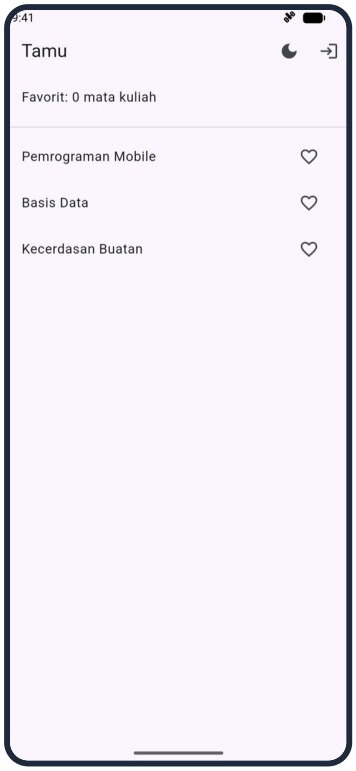
Contoh lengkap: tema, sesi, dan favorit

`lib/main.dart`

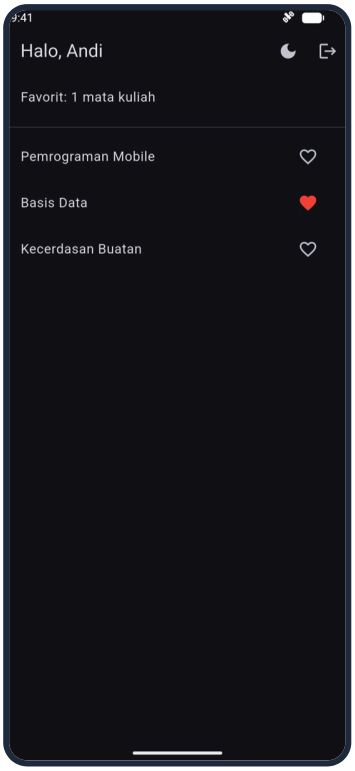
```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

// Provider 1: pengaturan tampilan
class TemaProvider extends ChangeNotifier {
  bool _gelap = false;
  bool get gelap => _gelap;

  void toggle() {
    _gelap = !_gelap;
    notifyListeners();
  }
}

// Provider 2: data pengguna yang sedang login
class SesiProvider extends ChangeNotifier {
  String? _nama;
  String? get nama => _nama;
  bool get sudahLogin => _nama != null;

  void login(String nama) {
    _nama = nama;
    notifyListeners();
  }

  void logout() {
    _nama = null;
    notifyListeners();
  }
```

```dart
}

// Provider 3: daftar favorit
class FavoritProvider extends ChangeNotifier {
  final Set<String> _items = {};
  Set<String> get items => Set.unmodifiable(_items);
  bool isFavorit(String m) => _items.contains(m);

  void toggle(String m) {
    if (!_items.remove(m)) _items.add(m);
    notifyListeners();
  }
}

void main() {
  runApp(
    // MultiProvider = beberapa provider tanpa nesting bertingkat
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => TemaProvider()),
        ChangeNotifierProvider(create: (_) => SesiProvider()),
        ChangeNotifierProvider(create: (_) => FavoritProvider()),
      ],
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // MaterialApp hanya rebuild saat nilai "gelap" berubah
    final gelap = context.select((TemaProvider t) => t.gelap);
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      darkTheme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.dark),
      themeMode: gelap ? ThemeMode.dark : ThemeMode.light,
      home: const BerandaPage(),
    );
  }
}

class BerandaPage extends StatelessWidget {
  const BerandaPage({super.key});

  static const matkul = ['Pemrograman Mobile', 'Basis Data', 'Kecerdasan Buatan'];

  @override
  Widget build(BuildContext context) {
    final sesi = context.watch<SesiProvider>();
    final favorit = context.watch<FavoritProvider>();

    return Scaffold(
      appBar: AppBar(
        title: Text(sesi.sudahLogin ? 'Halo, ${sesi.nama}' : 'Tamu'),
        actions: [
```

```
          IconButton(
            tooltip: 'Ganti tema',
            icon: const Icon(Icons.dark_mode),
            onPressed: () => context.read<TemaProvider>().toggle(),
          ),
          IconButton(
            tooltip: sesi.sudahLogin ? 'Logout' : 'Login',
            icon: Icon(sesi.sudahLogin ? Icons.logout : Icons.login),
            onPressed: () {
              final s = context.read<SesiProvider>();
              sesi.sudahLogin ? s.logout() : s.login('Andi');
            },
          ),
        ],
      ),
      body: ListView(
        children: [
          ListTile(title: Text('Favorit: ${favorit.items.length} mata kuliah')),
          const Divider(),
          for (final m in matkul)
            ListTile(
              title: Text(m),
              trailing: IconButton(
                tooltip: 'Favorit $m',
                icon: Icon(
                  favorit.isFavorit(m) ? Icons.favorite : Icons.favorite_border,
                  color: favorit.isFavorit(m) ? Colors.red : null,
                ),
                onPressed: () => context.read<FavoritProvider>().toggle(m),
              ),
            ),
        ],
      ),
    );
  }
}
```



Tiga provider: tema, sesi, favorit



Setelah login, menambah favorit, dan ganti tema

Penjelasan:

- Tiga provider dengan tanggung jawab berbeda. Masing-masing kecil dan mudah dipahami (prinsip *single responsibility*).
- `MyApp` memakai `context.select((TemaProvider t) => t.gelap)`. `MaterialApp` hanya rebuild ketika tema berganti, bukan ketika favorit bertambah.
- `BerandaPage` me-`watch` dua provider sekaligus. Halaman rebuild jika salah satunya berubah.
- Urutan di `providers: [...]` penting hanya jika satu provider bergantung pada provider lain (lihat ProxyProvider di section 8).

> ⚠️ **Kesalahan umum**
>
> - Membuat satu "GodProvider" raksasa berisi semua state. Setiap perubahan kecil membuat semua pendengar rebuild, dan file-nya sulit dirawat.
> - Mendaftarkan dua provider dengan tipe yang sama. Widget hanya bisa menemukan yang paling dekat; yang lain tertutup.

7. State Async: isLoading, error, data

Data dari internet atau database tidak datang seketika. Selama menunggu, UI harus menampilkan **loading**; jika gagal, menampilkan **error** dengan tombol coba lagi; jika berhasil, menampilkan **data**. Pola tiga serangkai `isLoading` / `error` / `data` ini akan dipakai ulang di Modul 09 (REST API) dan proyek CRUD Modul 13–14.

Pola di sisi UI

```dart
// Urutan pengecekan di UI: loading -> error -> kosong -> data
if (prov.isLoading) return const CircularProgressIndicator();
if (prov.error != null) return Text(prov.error!);
if (prov.data.isEmpty) return const Text('Belum ada data');
return ListView(children: [for (final d in prov.data) Text(d)]);
```

Pola di sisi provider

1. Set `_isLoading = true`, kosongkan `_error`, lalu `notifyListeners()`.
2. `try` : panggil service dan simpan hasilnya.
3. `catch` : simpan pesan error.
4. `finally` : set `_isLoading = false`, lalu `notifyListeners()`. Bagian `finally` selalu dijalankan, sehingga loading tidak "macet".

Contoh lengkap (multi-file)

```
lib/
├── main.dart
├── models/mahasiswa.dart
├── services/mahasiswa_service.dart
└── providers/mahasiswa_provider.dart
```

`lib/models/mahasiswa.dart`

```dart
// lib/models/mahasiswa.dart
class Mahasiswa {
  final String nim;
  final String nama;
  final double ipk;

  const Mahasiswa({required this.nim, required this.nama, required this.ipk});
}
```

`lib/services/mahasiswa_service.dart`

```dart
// lib/services/mahasiswa_service.dart
// Service = satu-satunya tempat yang tahu cara MENGAMBIL data.
// Saat ini datanya palsu; di Modul 09 isi fungsi ini diganti dengan HTTP request.
import '../models/mahasiswa.dart';

class MahasiswaService {
```

```
  Future<List<Mahasiswa>> ambilSemua({bool simulasiGagal = false}) async {
    await Future.delayed(const Duration(seconds: 3)); // pura-pura menunggu server

    if (simulasiGagal) {
      throw Exception('Server tidak dapat dihubungi');
    }

    return const [
      Mahasiswa(nim: 'H1D023001', nama: 'Andi Pratama', ipk: 3.45),
      Mahasiswa(nim: 'H1D023002', nama: 'Bunga Lestari', ipk: 3.81),
      Mahasiswa(nim: 'H1D023003', nama: 'Citra Ayu', ipk: 3.12),
      Mahasiswa(nim: 'H1D023004', nama: 'Dimas Saputra', ipk: 3.67),
    ];
  }
}
```

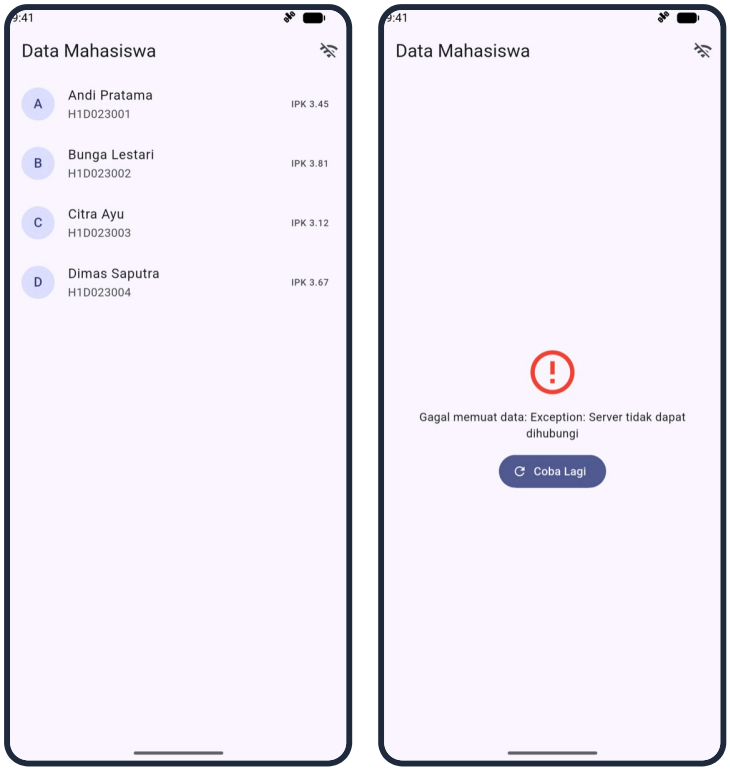
lib/providers/mahasiswa_provider.dart

```
// lib/providers/mahasiswa_provider.dart
import 'package:flutter/foundation.dart';

import '../models/mahasiswa.dart';
import '../services/mahasiswa_service.dart';

class MahasiswaProvider extends ChangeNotifier {
  MahasiswaProvider(this._service);

  final MahasiswaService _service;

  // Tiga serangkai state async
  bool _isLoading = false;
  String? _error;
  List<Mahasiswa> _data = [];

  bool get isLoading => _isLoading;
  String? get error => _error;
  List<Mahasiswa> get data => List.unmodifiable(_data);

  Future<void> muatData({bool simulasiGagal = false}) async {
    if (_isLoading) return; // cegah request ganda saat tombol ditekan berkali-kali

    _isLoading = true;
    _error = null;
    notifyListeners(); // UI menampilkan loading

    try {
      _data = await _service.ambilSemua(simulasiGagal: simulasiGagal);
    } catch (e) {
      _error = 'Gagal memuat data: $e';
    } finally {
      _isLoading = false;
      notifyListeners(); // UI menampilkan data ATAU error
    }
  }
}
```

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/mahasiswa_provider.dart';
import 'services/mahasiswa_service.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      // ..muatData() langsung memulai pemuatan saat provider dibuat
      create: (_) => MahasiswaProvider(MahasiswaService())..muatData(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMahasiswaPage(),
    );
  }
}

class DaftarMahasiswaPage extends StatelessWidget {
  const DaftarMahasiswaPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Mahasiswa'),
        actions: [
          IconButton(
            tooltip: 'Simulasi gagal',
            icon: const Icon(Icons.wifi_off),
            onPressed: () =>
                context.read<MahasiswaProvider>().muatData(simulasiGagal: true),
          ),
        ],
      ),
      body: const _IsiHalaman(),
    );
  }
}

class _IsiHalaman extends StatelessWidget {
  const _IsiHalaman();

  @override
  Widget build(BuildContext context) {
```

```
    final prov = context.watch<MahasiswaProvider>();

    // 1. Loading pertama kali (belum ada data sama sekali)
    if (prov.isLoading && prov.data.isEmpty) {
      return const Center(child: CircularProgressIndicator());
    }

    // 2. Error
    if (prov.error != null) {
      return Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(Icons.error_outline, size: 64, color: Colors.red),
              const SizedBox(height: 12),
              Text(prov.error!, textAlign: TextAlign.center),
              const SizedBox(height: 12),
              FilledButton.icon(
                icon: const Icon(Icons.refresh),
                label: const Text('Coba Lagi'),
                onPressed: () => context.read<MahasiswaProvider>().muatData(),
              ),
            ],
          ),
        ),
      );
    }

    // 3. Data kosong
    if (prov.data.isEmpty) {
      return const Center(child: Text('Belum ada data'));
    }

    // 4. Data tersedia (+ tarik ke bawah untuk memuat ulang)
    return RefreshIndicator(
      onRefresh: () => context.read<MahasiswaProvider>().muatData(),
      child: ListView.builder(
        itemCount: prov.data.length,
        itemBuilder: (context, i) {
          final m = prov.data[i];
          return ListTile(
            leading: CircleAvatar(child: Text(m.nama[0])),
            title: Text(m.nama),
            subtitle: Text(m.nim),
            trailing: Text('IPK ${m.ipk.toStringAsFixed(2)}'),
          );
        },
      ),
    );
  }
}
```



State data | State error dengan tombol Coba Lagi

Penjelasan:

- `MahasiswaService` hanya tahu cara mengambil data. Ia tidak tahu apa pun soal UI atau `notifyListeners`.
- `MahasiswaProvider` menerima service lewat constructor (*dependency injection*). Saat testing (Modul 15), service ini bisa diganti dengan versi palsu.
- `create: (_) => MahasiswaProvider(MahasiswaService())..muatData()` : operator *cascade* `..` memanggil `muatData()` lalu tetap mengembalikan objek provider.
- `if (_isLoading) return;` mencegah request ganda saat tombol ditekan berkali-kali.
- Tombol ikon wifi mensimulasikan kegagalan sehingga tampilan error bisa dicoba tanpa mematikan internet.
- Kondisi loading ditulis `prov.isLoading && prov.data.isEmpty` : saat refresh (data lama masih ada), list tetap tampil dan indikator `RefreshIndicator` yang berputar.

> 💡 **Variasi: enum status**
>
> Sebagian tim memakai `enum Status { awal, loading, sukses, gagal }` sebagai pengganti beberapa boolean. Keduanya sah; yang penting UI selalu menangani semua kemungkinan.

⚠️ **Kesalahan umum**

- Lupa `notifyListeners()` sebelum `await`, sehingga loading tidak pernah terlihat.
- Tidak memakai `finally`: jika terjadi error, `isLoading` tetap `true` selamanya.
- Tidak mengosongkan `_error` saat mencoba lagi, sehingga pesan error lama tetap tampil walaupun request kedua berhasil.
- Memanggil service langsung dari widget (di `build`). Setiap rebuild akan memicu request baru.

8. ProxyProvider (Pengenalan)

Kadang satu provider butuh data dari provider lain. Contoh: daftar catatan bergantung pada siapa yang sedang login. `ChangeNotifierProxyProvider` menyambungkan keduanya: setiap kali provider sumber berubah, provider tujuan diberi tahu.

Bentuk umumnya

```
ChangeNotifierProxyProvider<Sumber, Tujuan>(
  create: (_) => Tujuan(),                                  // dibuat sekali
  update: (_, sumber, tujuan) => tujuan!..perbarui(sumber), // dipanggil tiap Sumber berubah
)
```

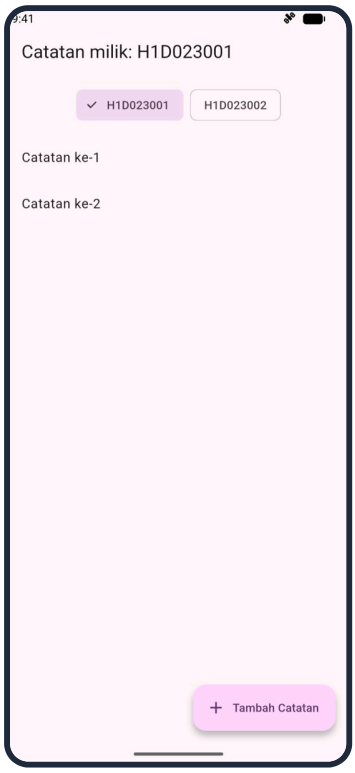
Contoh lengkap: catatan per pengguna

`lib/main.dart`

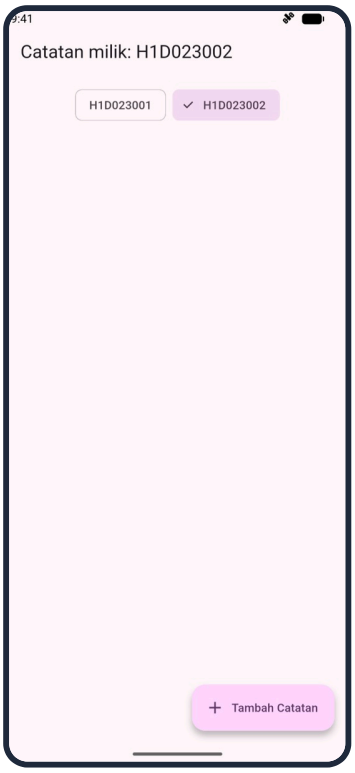
```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class AuthProvider extends ChangeNotifier {
  String? _nim;
  String? get nim => _nim;

  void loginSebagai(String nim) {
    _nim = nim;
    notifyListeners();
  }
}

// CatatanProvider BERGANTUNG pada AuthProvider: catatan milik siapa?
class CatatanProvider extends ChangeNotifier {
  String? _nimAktif;
  final Map<String, List<String>> _catatanPerNim = {};

  List<String> get catatan => _catatanPerNim[_nimAktif] ?? const [];

  // Dipanggil oleh ProxyProvider setiap AuthProvider berubah
  void perbaruiPengguna(String? nim) {
    if (nim == _nimAktif) return;
    _nimAktif = nim;
    notifyListeners();
  }

  void tambah(String teks) {
    final nim = _nimAktif;
    if (nim == null) return;
    _catatanPerNim.putIfAbsent(nim, () => []).add(teks);
    notifyListeners();
  }
}

void main() {
```

```
  runApp(
    MultiProvider(
      providers: [
        // 1. Provider yang menjadi sumber harus didaftarkan LEBIH DULU
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        // 2. ProxyProvider: <Sumber, Hasil>
        ChangeNotifierProxyProvider<AuthProvider, CatatanProvider>(
          create: (_) => CatatanProvider(),
          // update dipanggil di awal dan setiap AuthProvider notifyListeners
          update: (_, auth, catatan) => catatan!..perbaruiPengguna(auth.nim),
        ),
      ],
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.purple),
      home: const CatatanPage(),
    );
  }
}

class CatatanPage extends StatelessWidget {
  const CatatanPage({super.key});

  @override
  Widget build(BuildContext context) {
    final nim = context.watch<AuthProvider>().nim;
    final catatan = context.watch<CatatanProvider>().catatan;

    return Scaffold(
      appBar: AppBar(title: Text('Catatan milik: ${nim ?? '-'}')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Wrap(
              spacing: 8,
              children: [
                for (final n in ['H1D023001', 'H1D023002'])
                  ChoiceChip(
                    label: Text(n),
                    selected: nim == n,
                    onSelected: (_) => context.read<AuthProvider>().loginSebagai(n),
                  ),
              ],
            ),
          ),
          Expanded(
            child: ListView(
              children: [for (final c in catatan) ListTile(title: Text(c))],
```

```
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        icon: const Icon(Icons.add),
        label: const Text('Tambah Catatan'),
        onPressed: nim == null
            ? null
            : () {
                final prov = context.read<CatatanProvider>();
                prov.tambah('Catatan ke-${prov.catatan.length + 1}');
              },
      ),
    );
  }
}
```



Catatan milik H1D023001



Ganti pengguna: CatatanProvider ikut diperbarui

Penjelasan:

- `AuthProvider` wajib didaftarkan **sebelum** ProxyProvider yang membutuhkannya.
- `update` menerima objek `CatatanProvider` yang sudah ada (`catatan!`) dan hanya memperbarui NIM-nya. Objek tidak dibuat ulang, jadi catatan yang tersimpan tidak hilang.
- Di `perbaruiPengguna`, `if (nim == _nimAktif) return;` mencegah notify yang tidak perlu.
- Pilih NIM pertama, tambah dua catatan, lalu pilih NIM kedua: daftar kosong. Kembali ke NIM pertama: dua catatan muncul lagi.

ℹ️ Seberapa sering dipakai?

Untuk proyek kuliah, ProxyProvider jarang wajib. Alternatif yang lebih sederhana: panggil `context.read<CatatanProvider>().perbaruiPengguna(nim)` tepat setelah login berhasil. Namun mengenal ProxyProvider membantu saat membaca kode proyek orang lain.

⚠️ Kesalahan umum

- Membuat objek baru di `update`: `update: (_, auth, __) => CatatanProvider()`. Semua data hilang setiap kali auth berubah.
- Urutan provider terbalik (ProxyProvider di atas AuthProvider) sehingga muncul ProviderNotFoundException.

9. Struktur Folder & Auth Gate

Semakin besar aplikasi, semakin penting memisahkan kode berdasarkan tanggung jawabnya. Struktur berikut sederhana, cukup untuk proyek kuliah, dan dipakai lagi di proyek CRUD + Auth (Modul 13–14).

```
lib/
├── main.dart              # runApp + MultiProvider + MaterialApp
├── models/                # class data murni (User, Mahasiswa, Todo)
├── services/              # akses data: HTTP, database, Supabase
├── providers/             # ChangeNotifier: state + logika bisnis
├── screens/               # halaman penuh (Scaffold)
└── widgets/               # komponen kecil yang dipakai ulang
```

Lapisan	Boleh mengimpor	Tidak boleh
models	Dart murni	Flutter widget, provider, service
services	models, package http/database	widget, `BuildContext`, provider
providers	models, services, `foundation.dart`	widget, `BuildContext`, `Navigator`
screens / widgets	models, providers, widget lain	Memanggil service langsung

Arah ketergantungan: **UI → Provider → Service → Model**. Provider tidak boleh tahu tentang UI; itulah sebabnya provider tidak menampilkan SnackBar atau melakukan navigasi sendiri.

Contoh lengkap: Auth gate dengan Provider

Auth gate adalah widget yang memilih menampilkan `LoginScreen` atau `HomeScreen` berdasarkan status sesi di provider. Login dan logout tidak memerlukan `Navigator.push` sama sekali.

`lib/models/user.dart`

```dart
// lib/models/user.dart
class User {
  final String email;
  final String nama;

  const User({required this.email, required this.nama});
}
```

`lib/services/auth_service.dart`

```dart
// lib/services/auth_service.dart
// Service palsu. Di Modul 12 bagian ini diganti dengan Supabase Auth.
import '../models/user.dart';
```

```dart
class AuthService {
  Future<User> login(String email, String password) async {
    await Future.delayed(const Duration(seconds: 1));
    if (password != '123456') {
      throw Exception('Email atau password salah');
    }
    return User(email: email, nama: email.split('@').first);
  }

  Future<void> logout() async {
    await Future.delayed(const Duration(milliseconds: 300));
  }
}
```

lib/providers/auth_provider.dart

```dart
// lib/providers/auth_provider.dart
import 'package:flutter/foundation.dart';

import '../models/user.dart';
import '../services/auth_service.dart';

class AuthProvider extends ChangeNotifier {
  AuthProvider(this._service);

  final AuthService _service;

  User? _user;
  bool _isLoading = false;
  String? _error;

  User? get user => _user;
  bool get isLoggedIn => _user != null;
  bool get isLoading => _isLoading;
  String? get error => _error;

  Future<bool> login(String email, String password) async {
    _isLoading = true;
    _error = null;
    notifyListeners();
    try {
      _user = await _service.login(email, password);
      return true;
    } catch (e) {
      _error = e.toString().replaceFirst('Exception: ', '');
      return false;
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  Future<void> logout() async {
    await _service.logout();
    _user = null;
    notifyListeners();
```

```
  }
}
```

lib/widgets/auth_gate.dart

```dart
// lib/widgets/auth_gate.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../screens/home_screen.dart';
import '../screens/login_screen.dart';

// Auth gate: memilih halaman berdasarkan status sesi.
// Tidak ada Navigator.push sama sekali untuk login/logout!
class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    final sudahLogin = context.select((AuthProvider a) => a.isLoggedIn);
    return sudahLogin ? const HomeScreen() : const LoginScreen();
  }
}
```

lib/screens/login_screen.dart

```dart
// lib/screens/login_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController(text: 'andi@mhs.unsoed.ac.id');
  final _passwordController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    FocusScope.of(context).unfocus(); // tutup keyboard
    final berhasil = await context.read<AuthProvider>().login(
          _emailController.text.trim(),
          _passwordController.text,
        );
```

```
    // Jika berhasil, AuthGate otomatis menampilkan HomeScreen.
    if (!berhasil && mounted) {
      final pesan = context.read<AuthProvider>().error ?? 'Login gagal';
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(pesan)));
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.select((AuthProvider a) => a.isLoading);

    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            TextField(
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            TextField(
              controller: _passwordController,
              obscureText: true,
              decoration: const InputDecoration(
                labelText: 'Password',
                helperText: 'Petunjuk: 123456',
              ),
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              child: FilledButton(
                onPressed: isLoading ? null : _login,
                child: isLoading
                    ? const SizedBox(
                        width: 20,
                        height: 20,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      )
                    : const Text('Masuk'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/screens/home_screen.dart

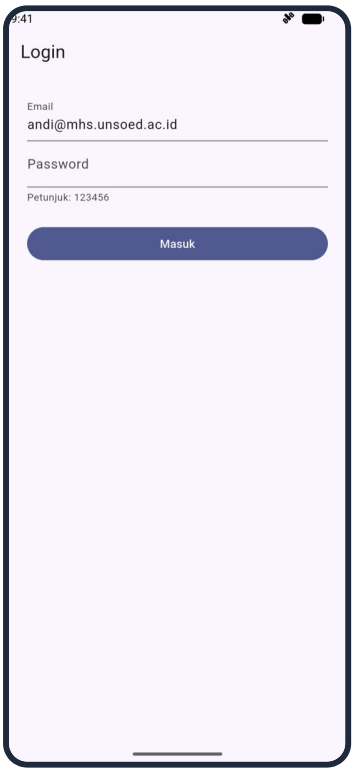
```
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
```

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final user = context.watch<AuthProvider>().user;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            icon: const Icon(Icons.logout),
            // Setelah logout, AuthGate otomatis kembali ke LoginScreen
            onPressed: () => context.read<AuthProvider>().logout(),
          ),
        ],
      ),
      body: Center(
        child: Text(
          'Selamat datang, ${user?.nama}!',
          style: Theme.of(context).textTheme.headlineSmall,
        ),
      ),
    );
  }
}
```

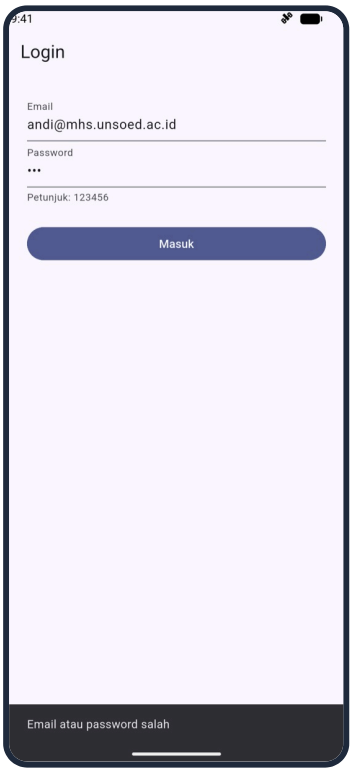
lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/auth_provider.dart';
import 'services/auth_service.dart';
import 'widgets/auth_gate.dart';

void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider(AuthService())),
        // Provider lain (MatkulProvider, TodoProvider, ...) ditambahkan di sini
      ],
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```

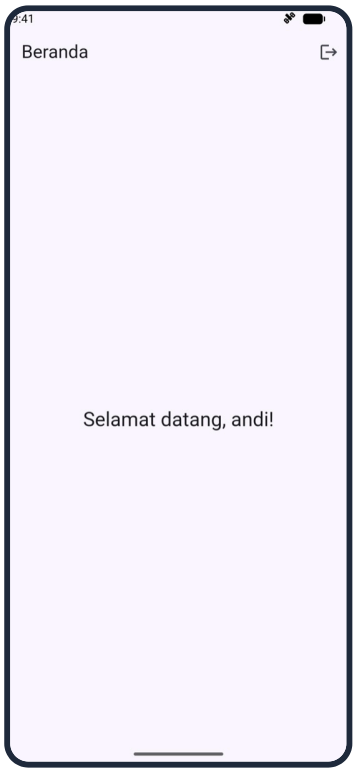
```
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const AuthGate(),
    );
  }
}
```



AuthGate menampilkan LoginScreen



Password salah: pesan error dari provider



Login berhasil: AuthGate menampilkan HomeScreen

Penjelasan:

- `AuthGate` memakai `select` pada `isLoggedIn`. Ia hanya rebuild saat status login berubah, bukan saat `isLoading` berubah.
- `AuthProvider.login` mengembalikan `bool`. Layar login yang memutuskan cara menampilkan error (SnackBar), sedangkan provider hanya menyimpan pesannya.
- Tombol Masuk dinonaktifkan dan diganti spinner selama `isLoading`, mencegah login ganda.
- Logout cukup memanggil `logout()`. Begitu `_user` menjadi `null`, AuthGate menampilkan LoginScreen lagi.

⚠️ **Kesalahan umum**

- Meng-import `material.dart` di provider untuk memanggil `ScaffoldMessenger` atau `Navigator`. Kembalikan hasil (`bool`/pesan) dan biarkan UI yang bereaksi.
- Mencampur AuthGate dengan `pushReplacement` manual setelah login. Pilih satu cara; jika memakai AuthGate, biarkan ia yang menentukan halaman.
- Menaruh semua class di `main.dart` sampai ratusan baris. Pecah sesuai struktur di atas sejak awal.

10. Kesalahan Umum & Cara Memperbaikinya

Bagian ini merangkum kesalahan Provider yang paling sering muncul di praktikum, lengkap dengan pesan error dan perbaikannya.

1. read di build, watch di callback

```
// ❌ SALAH
class TampilanSalah extends StatelessWidget {
  const TampilanSalah({super.key});

  @override
  Widget build(BuildContext context) {
    // read di build: angka TIDAK pernah ter-update di layar
    final nilai = context.read<CounterModel>().nilai;
    return ElevatedButton(
      // watch di callback: error "Tried to listen to a value exposed
      // with provider, from outside of the widget tree"
      onPressed: () => context.watch<CounterModel>().tambah(),
      child: Text('$nilai'),
    );
  }
}

// ✅ BENAR
class TampilanBenar extends StatelessWidget {
  const TampilanBenar({super.key});

  @override
  Widget build(BuildContext context) {
    final nilai = context.watch<CounterModel>().nilai; // watch di build
    return ElevatedButton(
      onPressed: () => context.read<CounterModel>().tambah(), // read di callback
      child: Text('$nilai'),
    );
  }
}
```

2. Memanggil provider di initState

`initState` berjalan saat widget sedang dibangun. Jika method provider langsung memanggil `notifyListeners()`, Flutter menolak karena ada widget lain yang diminta rebuild di tengah proses build.

```
@override
void initState() {
  super.initState();

  // ❌ SALAH: muatData() langsung memanggil notifyListeners() padahal
  // Flutter sedang membangun widget -> error
  // "setState() or markNeedsBuild() called during build"
  // context.read<DataProvider>().muatData();

  // ✅ BENAR: jalankan setelah frame pertama selesai digambar
```

```
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!mounted) return;
    context.read<DataProvider>().muatData();
  });

  // ✅ Setara, gaya lama: Provider.of dengan listen: false
  // WidgetsBinding.instance.addPostFrameCallback((_) {
  //   Provider.of<DataProvider>(context, listen: false).muatData();
  // });
}

// ❌ context.watch TIDAK boleh dipanggil di initState (hanya di build)
```

3. notifyListeners setelah dispose

Provider yang dibuat khusus untuk satu halaman akan di-dispose saat halaman ditutup. Jika request async masih berjalan, `notifyListeners()` di akhir request akan error.

```dart
import 'package:flutter/foundation.dart';

class PencarianProvider extends ChangeNotifier {
  bool _sudahDispose = false;
  List<String> hasil = [];

  Future<void> cari(String kata) async {
    await Future.delayed(const Duration(seconds: 3)); // request lama
    hasil = ['$kata 1', '$kata 2'];
    // Jika halaman sudah ditutup selama 3 detik itu, provider sudah di-dispose.
    // Memanggil notifyListeners() sekarang -> error
    // "A PencarianProvider was used after being disposed".
    if (_sudahDispose) return; // ✅ penjaga
    notifyListeners();
  }

  @override
  void dispose() {
    _sudahDispose = true;
    super.dispose();
  }
}
```

4. Meneruskan provider ke rute baru

```
// ✅ Meneruskan provider yang SUDAH ADA ke rute baru: pakai .value
final keranjang = context.read<KeranjangProvider>();
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => ChangeNotifierProvider.value(
      value: keranjang,
      child: const DetailPage(),
    ),
  ),
);

// ❌ SALAH: create: (_) => keranjang  -> provider akan men-dispose
```

```
//    objek milik orang lain saat DetailPage ditutup.
// ❌ SALAH: ChangeNotifierProvider.value(value: KeranjangProvider())
//    -> objek baru dibuat ulang setiap build dan tidak pernah di-dispose.
```

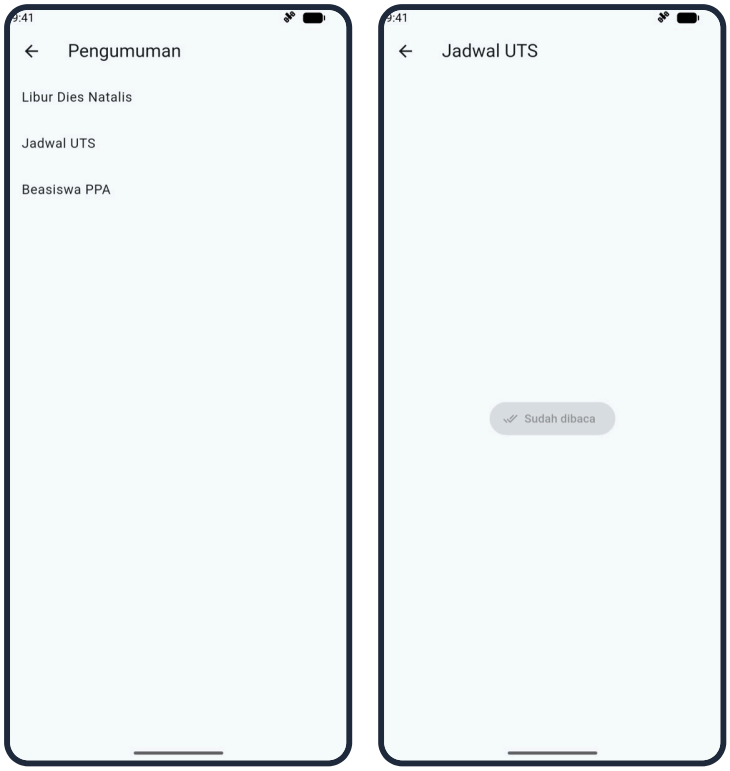
Contoh lengkap: semua perbaikan dalam satu aplikasi

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Contoh BENAR: memuat data di initState + berbagi provider ke halaman lain
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class PengumumanProvider extends ChangeNotifier {
  bool _sudahDispose = false;
  bool _isLoading = false;
  List<String> _items = [];
  final Set<String> _dibaca = {};

  bool get isLoading => _isLoading;
  List<String> get items => List.unmodifiable(_items);
  bool sudahDibaca(String p) => _dibaca.contains(p);

  Future<void> muat() async {
    _isLoading = true;
    _aman();
    await Future.delayed(const Duration(seconds: 1));
    _items = ['Libur Dies Natalis', 'Jadwal UTS', 'Beasiswa PPA'];
    _isLoading = false;
    _aman();
  }

  void tandaiDibaca(String p) {
    _dibaca.add(p);
    _aman();
  }

  // notifyListeners yang tidak crash walau provider sudah di-dispose
  void _aman() {
    if (!_sudahDispose) notifyListeners();
  }

  @override
  void dispose() {
    _sudahDispose = true;
    super.dispose();
  }
}

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blueGrey),
      home: Scaffold(
        appBar: AppBar(title: const Text('Menu')),
        body: Builder(
          builder: (context) => Center(
            child: FilledButton(
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(
                  // Provider khusus halaman: dibuat saat halaman dibuka,
                  // di-dispose otomatis saat halaman ditutup.
                  builder: (_) => ChangeNotifierProvider(
                    create: (_) => PengumumanProvider(),
                    child: const PengumumanPage(),
                  ),
                ),
              ),
              child: const Text('Buka Pengumuman'),
            ),
          ),
        ),
      ),
    );
  }
}

class PengumumanPage extends StatefulWidget {
  const PengumumanPage({super.key});

  @override
  State<PengumumanPage> createState() => _PengumumanPageState();
}

class _PengumumanPageState extends State<PengumumanPage> {
  @override
  void initState() {
    super.initState();
    // Muat data SETELAH frame pertama, bukan langsung di initState
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      context.read<PengumumanProvider>().muat();
    });
  }

  @override
  Widget build(BuildContext context) {
    final prov = context.watch<PengumumanProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Pengumuman')),
      body: prov.isLoading
          ? const Center(child: CircularProgressIndicator())
          : ListView(
              children: [
                for (final p in prov.items)
                  ListTile(
                    title: Text(p),
```

```
                    trailing: prov.sudahDibaca(p)
                        ? const Icon(Icons.done_all, color: Colors.green)
                        : null,
                    onTap: () {
                      // Bagikan provider yang SAMA ke halaman detail
                      final pengumuman = context.read<PengumumanProvider>();
                      Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) => ChangeNotifierProvider.value(
                            value: pengumuman,
                            child: DetailPengumumanPage(judul: p),
                          ),
                        ),
                      );
                    },
                  ),
                ],
              ),
    );
  }
}

class DetailPengumumanPage extends StatelessWidget {
  final String judul;

  const DetailPengumumanPage({super.key, required this.judul});

  @override
  Widget build(BuildContext context) {
    final dibaca = context.select((PengumumanProvider p) => p.sudahDibaca(judul));

    return Scaffold(
      appBar: AppBar(title: Text(judul)),
      body: Center(
        child: FilledButton.icon(
          icon: Icon(dibaca ? Icons.done_all : Icons.mark_email_read),
          label: Text(dibaca ? 'Sudah dibaca' : 'Tandai sudah dibaca'),
          onPressed: dibaca
              ? null
              : () => context.read<PengumumanProvider>().tandaiDibaca(judul),
        ),
      ),
    );
  }
}
```



Data dimuat setelah frame pertama

Halaman detail memakai provider yang sama (.value)

Penjelasan:

- `PengumumanProvider` dibuat di dalam `MaterialPageRoute`, sehingga hidup selama halaman Pengumuman terbuka saja.
- Data dimuat lewat `addPostFrameCallback` di `initState`, dengan cek `mounted`.
- Halaman detail menerima provider yang sama lewat `ChangeNotifierProvider.value`. Status "sudah dibaca" langsung terlihat di daftar saat kembali.
- `_aman()` mencegah error jika halaman ditutup sebelum `muat()` selesai.

Gejala / pesan error	Penyebab	Perbaikan
Angka di layar tidak berubah	Lupa `notifyListeners()` atau memakai `read` di build	Tambahkan notify; ganti dengan `watch` / `select`
Tried to listen to a value exposed with provider, from outside of the widget tree	`watch` di callback	Pakai `read`
Could not find the correct Provider<X> above this Widget	Provider tidak berada di atas widget/halaman	Pindahkan ke atas `MaterialApp` atau pakai `.value`
setState() or markNeedsBuild() called during build	Notify di `initState` / `build`	`addPostFrameCallback`
A X was used after being disposed	Notify setelah halaman ditutup	Penjaga `_sudahDispose`
Data berubah tanpa pengecekan aturan	Getter mengembalikan list asli	`List.unmodifiable`

11. Studi Kasus: Aplikasi Tugas Kuliah Multi-Halaman

Studi kasus penutup: aplikasi **Tugas Kuliah** (todo) dengan fitur tambah, centang selesai, hapus (geser), filter, dan halaman statistik. Satu `TodoProvider` dipakai bersama oleh tiga halaman.

Struktur folder

```
lib/
├── main.dart
├── models/todo.dart
├── providers/todo_provider.dart
└── screens/
    ├── todo_list_screen.dart
    ├── tambah_todo_screen.dart
    └── statistik_screen.dart
```

Langkah 1: model

`lib/models/todo.dart`

```dart
// lib/models/todo.dart
class Todo {
  final String id;
  final String judul;
  final String matkul;
  final bool selesai;

  const Todo({
    required this.id,
    required this.judul,
    required this.matkul,
    this.selesai = false,
  });

  // Membuat salinan dengan sebagian field diubah (objek tetap immutable)
  Todo copyWith({String? judul, String? matkul, bool? selesai}) {
    return Todo(
      id: id,
      judul: judul ?? this.judul,
      matkul: matkul ?? this.matkul,
      selesai: selesai ?? this.selesai,
    );
  }
}

enum FilterTodo { semua, aktif, selesai }
```

`copyWith` membuat salinan objek dengan sebagian field diganti. Objek `Todo` tidak pernah diubah langsung; yang diganti adalah elemen di list.

Langkah 2: provider

`lib/providers/todo_provider.dart`

```dart
// lib/providers/todo_provider.dart
import 'package:flutter/foundation.dart';

import '../models/todo.dart';

class TodoProvider extends ChangeNotifier {
  final List<Todo> _todos = [
    const Todo(id: '1', judul: 'Laporan praktikum Flutter', matkul: 'Pemrograman Mobile'),
    const Todo(id: '2', judul: 'Normalisasi tabel KRS', matkul: 'Basis Data', selesai: true),
  ];
  FilterTodo _filter = FilterTodo.semua;

  // ---------- GETTER (data turunan dihitung, tidak disimpan) ----------
  FilterTodo get filter => _filter;
  int get jumlahTotal => _todos.length;
  int get jumlahSelesai => _todos.where((t) => t.selesai).length;
  int get jumlahAktif => jumlahTotal - jumlahSelesai;
  double get progres => jumlahTotal == 0 ? 0 : jumlahSelesai / jumlahTotal;

  List<Todo> get todosTampil {
    switch (_filter) {
      case FilterTodo.semua:
        return List.unmodifiable(_todos);
      case FilterTodo.aktif:
        return _todos.where((t) => !t.selesai).toList();
      case FilterTodo.selesai:
        return _todos.where((t) => t.selesai).toList();
    }
  }

  Todo? cariById(String id) {
    for (final t in _todos) {
      if (t.id == id) return t;
    }
    return null;
  }

  // ---------- AKSI (mengubah state + notifyListeners) ----------
  void tambah(String judul, String matkul) {
    final id = DateTime.now().microsecondsSinceEpoch.toString();
    _todos.insert(0, Todo(id: id, judul: judul, matkul: matkul));
    notifyListeners();
  }

  void toggle(String id) {
    final i = _todos.indexWhere((t) => t.id == id);
    if (i == -1) return;
    _todos[i] = _todos[i].copyWith(selesai: !_todos[i].selesai);
    notifyListeners();
  }

  void hapus(String id) {
    _todos.removeWhere((t) => t.id == id);
    notifyListeners();
```

```
  }

  void hapusYangSelesai() {
    _todos.removeWhere((t) => t.selesai);
    notifyListeners();
  }

  void ubahFilter(FilterTodo f) {
    if (f == _filter) return;
    _filter = f;
    notifyListeners();
  }
}
```

- Data turunan (`jumlahAktif`, `progres`, `todosTampil`) dihitung dari `_todos`, tidak disimpan terpisah, sehingga tidak mungkin "tidak sinkron".
- Setiap aksi diakhiri `notifyListeners()`.

Langkah 3: main.dart

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/todo_provider.dart';
import 'screens/todo_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TodoProvider(),
      child: const TodoApp(),
    ),
  );
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tugas Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const TodoListScreen(),
    );
  }
}
```

Langkah 4: halaman daftar

`lib/screens/todo_list_screen.dart`

```dart
// lib/screens/todo_list_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/todo.dart';
import '../providers/todo_provider.dart';
import 'statistik_screen.dart';
import 'tambah_todo_screen.dart';

class TodoListScreen extends StatelessWidget {
  const TodoListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tugas Kuliah'),
        actions: [
          // Hanya badge ini yang rebuild saat jumlah aktif berubah
          Selector<TodoProvider, int>(
            selector: (_, p) => p.jumlahAktif,
            builder: (context, aktif, child) => IconButton(
              tooltip: 'Statistik',
              icon: Badge(label: Text('$aktif'), child: const Icon(Icons.insights)),
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute(builder: (_) => const StatistikScreen()),
              ),
            ),
          ),
        ],
      ),
      body: const Column(
        children: [
          _FilterBar(),
          Expanded(child: _DaftarTodo()),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        icon: const Icon(Icons.add),
        label: const Text('Tugas Baru'),
        onPressed: () => Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const TambahTodoScreen()),
        ),
      ),
    );
  }
}

class _FilterBar extends StatelessWidget {
  const _FilterBar();

  @override
```

```
  Widget build(BuildContext context) {
    final filter = context.select((TodoProvider p) => p.filter);

    return Padding(
      padding: const EdgeInsets.all(12),
      child: SegmentedButton<FilterTodo>(
        segments: const [
          ButtonSegment(value: FilterTodo.semua, label: Text('Semua')),
          ButtonSegment(value: FilterTodo.aktif, label: Text('Aktif')),
          ButtonSegment(value: FilterTodo.selesai, label: Text('Selesai')),
        ],
        selected: {filter},
        onSelectionChanged: (pilihan) =>
            context.read<TodoProvider>().ubahFilter(pilihan.first),
      ),
    );
  }
}

class _DaftarTodo extends StatelessWidget {
  const _DaftarTodo();

  @override
  Widget build(BuildContext context) {
    final todos = context.watch<TodoProvider>().todosTampil;

    if (todos.isEmpty) {
      return const Center(child: Text('Tidak ada tugas di filter ini'));
    }

    return ListView.builder(
      itemCount: todos.length,
      itemBuilder: (context, i) {
        final t = todos[i];
        return Dismissible(
          key: ValueKey(t.id),
          direction: DismissDirection.endToStart,
          background: Container(
            color: Colors.red,
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 20),
            child: const Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (_) {
            context.read<TodoProvider>().hapus(t.id);
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('"${t.judul}" dihapus')),
            );
          },
          child: CheckboxListTile(
            value: t.selesai,
            onChanged: (_) => context.read<TodoProvider>().toggle(t.id),
            title: Text(
              t.judul,
              style: TextStyle(
                decoration: t.selesai ? TextDecoration.lineThrough : null,
              ),
            ),
            subtitle: Text(t.matkul),
```

```
          ),
        );
      },
    );
  }
}
```

Langkah 5: halaman tambah

`lib/screens/tambah_todo_screen.dart`

```dart
// lib/screens/tambah_todo_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/todo_provider.dart';

class TambahTodoScreen extends StatefulWidget {
  const TambahTodoScreen({super.key});

  @override
  State<TambahTodoScreen> createState() => _TambahTodoScreenState();
}

class _TambahTodoScreenState extends State<TambahTodoScreen> {
  final _formKey = GlobalKey<FormState>();
  final _judulController = TextEditingController();
  String _matkul = 'Pemrograman Mobile';

  static const daftarMatkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  @override
  void dispose() {
    _judulController.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    // Tidak perlu mengembalikan data lewat pop: provider yang menyimpan
    context.read<TodoProvider>().tambah(_judulController.text.trim(), _matkul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Baru')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judulController,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
```

```
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            DropdownButtonFormField<String>(
              initialValue: _matkul,
              decoration: const InputDecoration(
                labelText: 'Mata kuliah',
                border: OutlineInputBorder(),
              ),
              items: [
                for (final m in daftarMatkul) DropdownMenuItem(value: m, child: Text(m)),
              ],
              onChanged: (v) => setState(() => _matkul = v ?? _matkul),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

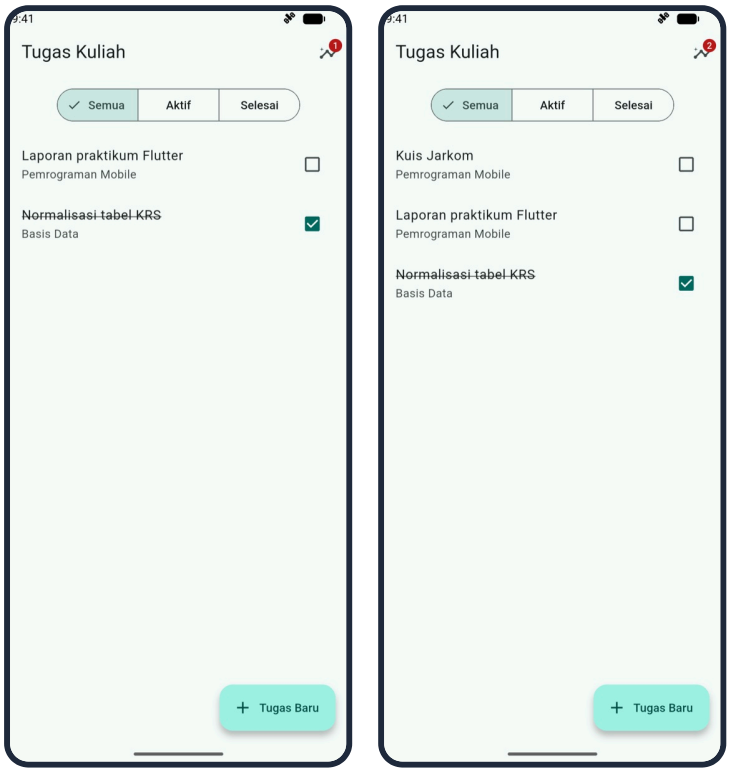
Langkah 6: halaman statistik

lib/screens/statistik_screen.dart

```
// lib/screens/statistik_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/todo_provider.dart';

class StatistikScreen extends StatelessWidget {
  const StatistikScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final p = context.watch<TodoProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Statistik')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('${(p.progres * 100).round()}% selesai',
                style: Theme.of(context).textTheme.headlineMedium),
            const SizedBox(height: 8),
            LinearProgressIndicator(value: p.progres, minHeight: 10),
            const SizedBox(height: 16),
            Card(
              child: Column(
                children: [
                  ListTile(title: const Text('Total tugas'), trailing: Text('${p.jumlahTotal}')),
```

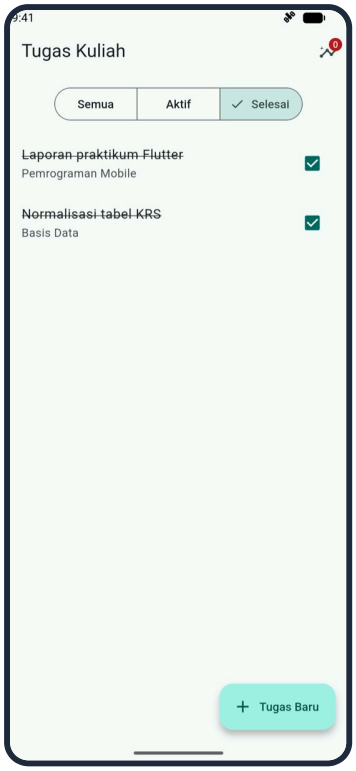
```
                ListTile(title: const Text('Aktif'), trailing: Text('${p.jumlahAktif}')),
                ListTile(title: const Text('Selesai'), trailing: Text('${p.jumlahSelesai}')),
              ],
            ),
          ),
          const SizedBox(height: 16),
          OutlinedButton.icon(
            icon: const Icon(Icons.cleaning_services),
            label: const Text('Hapus semua yang selesai'),
            onPressed: p.jumlahSelesai == 0
                ? null
                : () => context.read<TodoProvider>().hapusYangSelesai(),
          ),
        ],
      ),
    ),
  );
 }
}
```

Hasil

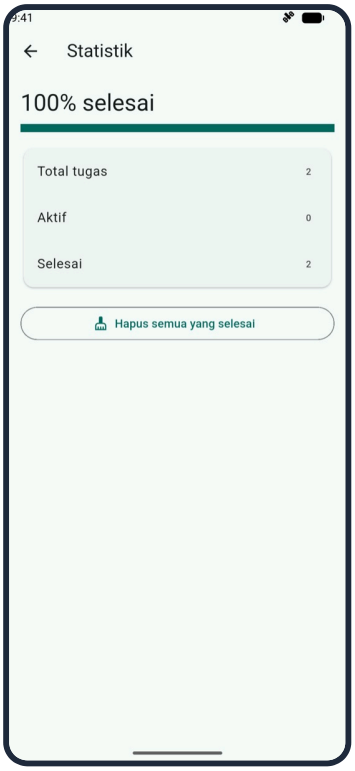


Daftar tugas awal

Tugas baru muncul, badge aktif bertambah



Filter Selesai setelah satu tugas dicentang



Halaman statistik membaca provider yang sama

Ringkasan teknik yang dipakai:

Bagian	Teknik	Alasan
Badge statistik di AppBar	`Selector<TodoProvider, int>`	Hanya ikon yang rebuild saat jumlah aktif berubah
Filter	`context.select` pada `filter`	Tombol filter tidak rebuild saat todo dicentang
Daftar todo	`context.watch`	Butuh seluruh daftar terkini
Tambah tugas	`context.read` lalu `Navigator.pop`	Tidak perlu mengembalikan data lewat pop; provider yang menyimpan
Hapus	`Dismissible` + `ValueKey(t.id)`	Key unik agar Flutter tidak salah menghapus item

ℹ️ Bandingkan dengan Modul 07

Di studi kasus Modul 07, data KRS harus dioper lewat constructor dan callback, dan halaman tambah mengembalikan hasil lewat `Navigator.pop`. Di sini, ketiga halaman cukup membaca `TodoProvider`. Pola yang sama dipakai di proyek CRUD Modul 13–14, dengan tambahan status loading/error dari section 7 dan auth gate dari section 9.

💡 Tantangan tambahan

- Tambahkan halaman edit yang memakai `cariById` dan method baru `ubah(id, judul)`.
- Tambahkan tombol "Urungkan" pada SnackBar hapus.
- Simpan todo ke penyimpanan lokal (Modul 10/11) dengan memanggil service dari provider.

Tugas Praktikum: Praktikum 8: Sistem Pengambilan SKS & Keranjang Rencana Studi

Sasaran Praktikum

Menerapkan Provider untuk sinkronisasi state keranjang KRS antar beberapa layar secara real-time, lengkap dengan pemuatan data async dan struktur folder yang rapi.

Langkah Pengerjaan

1. Tambahkan package `provider` ke dalam proyek menggunakan perintah: `flutter pub add provider`.
2. Susun folder `lib/models`, `lib/services`, `lib/providers`, `lib/screens`, dan `lib/widgets`.
3. Buat model `MataKuliah` dan `MatkulService` yang mengembalikan daftar mata kuliah setelah `Future.delayed` 2 detik.
4. Buat `KatalogProvider` dengan pola `isLoading` / `error` / `data`, lengkap dengan tampilan loading, error + tombol Coba Lagi, dan `RefreshIndicator`.
5. Buat `KrsProvider` yang mencatat mata kuliah yang dipilih, total SKS, dan saldo kuota SKS tersisa. Getter list harus mengembalikan `List.unmodifiable`.
6. Daftarkan kedua provider dengan `MultiProvider` di atas `MaterialApp`.
7. Buat Layar 1 (Katalog Matkul): Menampilkan daftar mata kuliah dengan tombol '+ Ambil Matkul'. Tombol ini memanggil `context.read<KrsProvider>().tambahMatkul(...)`.
8. Cegah penambahan jika kuota melebihi 24 SKS atau mata kuliah sudah diambil, dan tampilkan SnackBar peringatan.
9. Buat Layar 2 (Kartu Rencana Studi): Menampilkan daftar mata kuliah yang sudah diambil, tombol hapus dengan dialog konfirmasi, dan total kalkulasi SKS.
10. Pasang badge angka di ikon tab bar yang otomatis bertambah saat mata kuliah baru diambil. Gunakan `context.select` atau `Selector` agar hanya badge yang rebuild.
11. Muat data katalog dari `initState` memakai `WidgetsBinding.instance.addPostFrameCallback`.
12. Bonus: tambahkan `AuthProvider` sederhana dan `AuthGate` yang menampilkan layar login sebelum katalog.

Output yang Diharapkan

Dua atau lebih halaman yang saling terhubung secara reaktif tanpa perlu restart halaman; saat mata kuliah diambil di halaman 1, angka di halaman 2 dan badge tab langsung bertambah. Katalog menampilkan loading, error, dan data dengan benar.

Rubrik Penilaian

Desain class ChangeNotifier dan penggunaan notifyListeners (25%), Penggunaan context.watch, read, dan select yang tepat (25%), Pola async isLoading/error/data (20%), Pencegahan over-quota 24 SKS dan feedback UI (15%), Struktur folder model/service/provider/UI (15%). Bonus AuthGate +10%.

Kuis Pemahaman

1. Metode pemanggilan Provider manakah yang harus digunakan di dalam fungsi 'onPressed: () {}' sebuah tombol?

A. context.watch<T>()

B. context.read<T>()

C. setState()

D. Provider.of(listen: true)

2. Apa fungsi pemanggilan method 'notifyListeners()' di dalam class yang mewarisi ChangeNotifier?

A. Mengirim sinyal lonceng notifikasi ke server backend

B. Memberitahu framework Flutter bahwa state internal telah berubah, sehingga seluruh widget pendengar (listener) akan di-rebuild secara otomatis

C. Menutup aplikasi secara paksa

D. Menghapus cache memori smartphone

3. Sebuah widget hanya membutuhkan totalSks dari KrsProvider dan tidak ingin rebuild ketika field lain berubah. Cara paling tepat adalah ...

A. context.select((KrsProvider p) ⇒ p.totalSks)

B. context.read<KrsProvider>().totalSks

C. context.watch<KrsProvider>()

D. Membuat KrsProvider baru di widget tersebut

4. Memanggil context.read<DataProvider>().muatData() langsung di initState (dan muatData langsung memanggil notifyListeners) menyebabkan error. Perbaikannya adalah ...

A. Membungkusnya dengan WidgetsBinding.instance.addPostFrameCallback

B. Mengganti read dengan watch

C. Memindahkan panggilan ke dalam build()

D. Menghapus notifyListeners dari provider

5. Pada pola state async, mengapa _isLoading = false sebaiknya diletakkan di blok finally?

A. Agar loading selalu dimatikan, baik request berhasil maupun gagal

B. Agar request berjalan lebih cepat

C. Karena Dart melarang assignment di blok try

D. Agar error tidak pernah terjadi

6. Bagian builder mana yang di-build ulang ketika memakai Consumer<T>?

A. Hanya widget yang dikembalikan oleh builder Consumer

B. Seluruh halaman tempat Consumer berada

C. Seluruh aplikasi mulai dari MaterialApp

D. Tidak ada, Consumer tidak pernah rebuild

7. Halaman A memiliki provider. Halaman B dibuka dengan Navigator.push dan perlu memakai OBJEK provider yang sama. Cara yang benar adalah ...

A. Membungkus halaman B dengan ChangeNotifierProvider.value(value: objekYangSudahAda, ...)

B. Membungkus halaman B dengan ChangeNotifierProvider(create: (_) ⇒ objekYangSudahAda)

C. Membuat objek provider baru di halaman B

D. Mengirim provider melalui Navigator.pop

8. Dalam struktur folder model/service/provider/UI, lapisan mana yang seharusnya memanggil HTTP atau database?

A. services

B. screens

C. models

D. widgets

Kunci Jawaban & Pembahasan

1. B — context.read<T>() digunakan untuk memicu aksi/method tanpa mendaftarkan widget untuk rebuild saat terjadi perubahan nilai. Memanggil context.watch di dalam callback tombol dilarang oleh Flutter karena tombol tidak berada di alur build.

2. B — notifyListeners() adalah jantung dari ChangeNotifier yang memancarkan sinyal ke semua Consumer atau context.watch yang sedang mengamati model tersebut untuk memperbarui tampilannya.

3. A — select hanya memicu rebuild jika nilai yang dipilih berubah. read di build tidak pernah memicu rebuild, sedangkan watch memicu rebuild untuk setiap notifyListeners.

4. A — notifyListeners saat widget sedang dibangun menimbulkan error 'setState() or markNeedsBuild() called during build'. addPostFrameCallback menjalankan kode setelah frame pertama selesai.

5. A — Blok finally selalu dijalankan setelah try/catch. Tanpa finally, jika terjadi exception, isLoading bisa tetap true dan UI terjebak di tampilan loading.

6. A — Consumer membatasi rebuild pada builder-nya saja. Widget di luar Consumer (induknya) tidak ikut di-build ulang.

7. A — .value dipakai untuk objek yang sudah ada dan tidak akan men-dispose objek tersebut. Memakai create dengan objek yang sudah ada menyebabkan objek di-dispose saat halaman B ditutup.

8. A — Service adalah satu-satunya lapisan yang tahu cara mengakses data. Provider memanggil service, UI memanggil provider. Dengan begitu sumber data mudah diganti dan diuji.

MODUL 09 · REST API & HTTP

Integrasi REST API & Asynchronous Networking

Menghubungkan aplikasi Flutter ke server backend melalui HTTP/REST secara bertahap: konsep request-response, JSON dan model Dart (fromJson/toJson), GET/POST/PUT/PATCH/DELETE dengan package http, header & Bearer token, penanganan error dan timeout, lapisan ApiClient-Repository, FutureBuilder, pull-to-refresh, integrasi Provider, hingga studi kasus aplikasi CRUD Posts multi-file.

Durasi Pertemuan 9 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management Provider

Capaian Pembelajaran

- Menjelaskan komponen request/response HTTP: method, URL, header, body, dan status code.
- Mengonversi JSON menjadi objek Dart (jsonDecode, fromJson) termasuk object bersarang dan list, serta sebaliknya (toJson, jsonEncode).
- Melakukan HTTP Request (GET, POST, PUT, PATCH, DELETE) menggunakan package http dan mengirim header Authorization Bearer.
- Menangani kegagalan jaringan (status code, SocketException, ClientException, TimeoutException) dengan pesan yang ramah pengguna.
- Menyusun lapisan ApiClient dan Repository sehingga UI terpisah dari kode jaringan.
- Menampilkan tiga kondisi UI (loading, error, data) dengan FutureBuilder maupun Provider, dilengkapi pull-to-refresh.
- Membangun aplikasi CRUD lengkap (daftar, detail, tambah, edit, hapus) yang terhubung ke REST API.

Daftar Isi Modul

1. Konsep HTTP & REST: Method, Status Code, Header, Body
2. Dasar JSON: jsonDecode & jsonEncode
3. Model Dart: fromJson & toJson (Nested & List)
4. Persiapan: Package http & Izin Internet Android/iOS
5. GET: Mengambil Daftar dan Detail Data
6. POST, PUT, PATCH, dan DELETE
7. Header & Bearer Token
8. Penanganan Error, Timeout, dan Offline
9. Service Layer: ApiClient & Repository
10. Menangani 3 Kondisi UI dengan FutureBuilder
11. Pull-to-Refresh dengan RefreshIndicator
12. Integrasi REST API dengan Provider
13. Studi Kasus: Aplikasi CRUD Posts Multi-File

14. Tugas Praktikum

15. Kuis Pemahaman & Kunci Jawaban

1. Konsep HTTP & REST: Method, Status Code, Header, Body

Hampir semua aplikasi modern (Gojek, Tokopedia, SIA kampus) tidak menyimpan seluruh datanya di HP. Data diambil dari **server** lewat internet. Cara paling umum untuk "mengobrol" dengan server adalah **REST API** yang berjalan di atas protokol **HTTP**. Sebelum menulis kode Flutter, pahami dulu bahasa percakapannya.

Client, Server, Request, Response

Aplikasi Flutter berperan sebagai **client**. Client mengirim **request** (permintaan) ke **server**, lalu server membalas dengan **response** (jawaban). Satu request selalu dibalas satu response.

```
Aplikasi Flutter (client)                        Server (backend)
─────────────────────────                        ────────────────
GET /posts/1   ──────────────── request ──────────>
               <──────────────── response ──────────  200 OK + data JSON
```

Setiap request terdiri dari 4 bagian utama:

1. **Method**: apa yang ingin dilakukan (ambil, buat, ubah, hapus).
2. **URL**: alamat data yang dituju.
3. **Header**: informasi tambahan (format data, token login).
4. **Body**: isi data yang dikirim (biasanya JSON). Request GET dan DELETE umumnya tidak punya body.

Response juga punya **status code** (angka hasil), **header**, dan **body**.

Anatomi URL

```
https://jsonplaceholder.typicode.com/posts/1/comments?_limit=5&_sort=id
└─┬─┘   └──────────────┬─────────────┘└────┬────┘└──────────┬──────────┘
scheme            host / base URL        path          query parameter

- Base URL : https://jsonplaceholder.typicode.com  (alamat server)
- Endpoint : /posts/1/comments                     (sumber data yang dituju)
- Path param: 1                                     (id post yang spesifik)
- Query    : _limit=5 dan _sort=id                  (filter tambahan, dipisah &)
```

HTTP Method (Kata Kerja REST)

REST memakai kata benda untuk URL (`/posts`, `/users`) dan method HTTP sebagai kata kerjanya:

Method	Fungsi	Contoh	Ada body?	Padanan CRUD
`GET`	Mengambil data	`GET /posts`, `GET /posts/1`	Tidak	Read
`POST`	Membuat data baru	`POST /posts`	Ya	Create
`PUT`	Mengganti *seluruh* data	`PUT /posts/1`	Ya (lengkap)	Update
`PATCH`	Mengubah *sebagian* field	`PATCH /posts/1`	Ya (sebagian)	Update
`DELETE`	Menghapus data	`DELETE /posts/1`	Tidak	Delete

Status Code

Status code adalah angka 3 digit yang memberi tahu hasil request. Digit pertama menunjukkan kelompoknya:

Kode	Nama	Arti untuk aplikasi
`200`	OK	Berhasil (GET, PUT, PATCH, DELETE).
`201`	Created	Data baru berhasil dibuat (POST).
`204`	No Content	Berhasil, tetapi body kosong (sering untuk DELETE).
`400`	Bad Request	Data yang dikirim salah format / tidak valid.
`401`	Unauthorized	Belum login atau token salah/kedaluwarsa.
`403`	Forbidden	Sudah login, tetapi tidak punya hak akses.
`404`	Not Found	URL atau data dengan id tersebut tidak ada.
`422`	Unprocessable Entity	Validasi gagal (mis. email sudah dipakai).
`500`	Internal Server Error	Server sedang bermasalah, bukan salah aplikasi.
`503`	Service Unavailable	Server sedang down / maintenance.

> 💡 **Cara Mudah Mengingat**
>
> **2xx** = sukses, **4xx** = kesalahan dari sisi client (aplikasi kita), **5xx** = kesalahan dari sisi server. Di kode, cek sukses dengan `statusCode >= 200 && statusCode < 300`.

Header yang Sering Dipakai

Header	Contoh nilai	Kegunaan
`Content-Type`	`application/json; charset=UTF-8`	Memberi tahu server bahwa body yang kita kirim berformat JSON.
`Accept`	`application/json`	Meminta server membalas dalam format JSON.
`Authorization`	`Bearer eyJhbGciOi...`	Mengirim token login agar server tahu siapa kita.
`x-api-key`	`abc123`	Kunci API khusus yang diminta sebagian layanan.

Contoh Request & Response Lengkap

Beginilah wujud "mentah" satu request POST dan response-nya. Package `http` di Flutter akan membuat teks seperti ini untuk kita.

```
POST /posts HTTP/1.1
Host: jsonplaceholder.typicode.com
Content-Type: application/json; charset=UTF-8
Accept: application/json

{"title": "Belajar REST", "body": "Isi tulisan", "userId": 1}

------------------------------------------------------------

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"title": "Belajar REST", "body": "Isi tulisan", "userId": 1, "id": 101}
```

Contoh Kode: Menyusun URL dengan Kelas `Uri`

Di Dart, URL disimpan sebagai objek `Uri`. Ada dua cara membuatnya. `Uri.parse` cocok untuk URL sederhana, sedangkan `Uri.https` lebih aman untuk query parameter karena karakter khusus (spasi, &) otomatis di-*encode*.

`lib/contoh_uri.dart`

```dart
// Jalankan dengan: dart run lib/contoh_uri.dart
void main() {
  // Cara 1: Uri.parse dari string utuh
  final uri1 = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
  print(uri1.host); // jsonplaceholder.typicode.com
  print(uri1.path); // /posts/1

  // Cara 2: Uri.https(host, path, queryParameters)
  final uri2 = Uri.https(
    'jsonplaceholder.typicode.com',
    '/posts',
    {'userId': '1', '_limit': '5'}, // nilai query WAJIB String
  );
  print(uri2); // https://jsonplaceholder.typicode.com/posts?userId=1&_limit=5

  // Karakter khusus otomatis di-encode
  final uri3 = Uri.https('example.com', '/cari', {'q': 'flutter & dart'});
  print(uri3); // https://example.com/cari?q=flutter+%26+dart

  // Menambah query ke Uri yang sudah ada
  final uri4 = uri1.replace(queryParameters: {'lang': 'id'});
  print(uri4); // https://jsonplaceholder.typicode.com/posts/1?lang=id
}
```

Penjelasan: nilai di `queryParameters` harus bertipe `String`, jadi angka ditulis `'1'` bukan `1`. Bagian `uri3` memperlihatkan alasan memakai `Uri.https`: tanda `&` di dalam kata kunci pencarian berubah menjadi `%26` sehingga tidak dianggap pemisah query.

> ⚠️ **Kesalahan Umum**
>
> - Menulis host dengan `https://` di `Uri.https('https://example.com', ...)`. Argumen pertama hanya host, tanpa scheme.
> - Mengirim angka di `queryParameters` (`{'page': 1}`) sehingga error tipe. Ubah menjadi `'1'` atau `1.toString()`.
> - Menganggap status `404` akan melempar exception. Package `http` **tidak** melempar error untuk 4xx/5xx; kita sendiri yang wajib mengecek `statusCode`.
> - Memakai GET untuk mengirim data sensitif lewat query (mis. password). Data di URL tercatat di log server; kirim lewat body POST.

2. Dasar JSON: jsonDecode & jsonEncode

Server mengirim data dalam bentuk **teks JSON**. Teks ini harus diubah dulu menjadi objek Dart (*decode*) agar bisa dipakai, dan objek Dart harus diubah menjadi teks JSON (*encode*) sebelum dikirim ke server. Keduanya disediakan library bawaan `dart:convert`, tanpa perlu package tambahan.

Wujud JSON

JSON hanya mengenal 6 jenis nilai. Hafalkan padanannya di Dart karena ini sumber error paling sering:

JSON	Contoh	Hasil di Dart setelah `jsonDecode`
Object	`{"nama": "Budi"}`	`Map<String, dynamic>`
Array	`[1, 2, 3]`	`List<dynamic>` (bukan `List<int>`!)
String	`"halo"`	`String`
Number	`10` / `3.5`	`int` / `double`
Boolean	`true`	`bool`
null	`null`	`null`

Contoh JSON satu mahasiswa yang berisi object bersarang (`prodi`) dan array (`hobi`):

```
{
  "nim": "H1D022001",
  "nama": "Budi Santoso",
  "ipk": 3.75,
  "aktif": true,
  "prodi": { "kode": "IF", "nama": "Informatika" },
  "hobi": ["coding", "futsal"]
}
```

Contoh Kecil: `jsonDecode` dan `jsonEncode`

```dart
import 'dart:convert';

void main() {
  // Teks JSON (String) -> objek Dart
  const teks = '{"nama": "Budi", "umur": 20}';
  final Map<String, dynamic> data = jsonDecode(teks);
  print(data['nama']); // Budi
```

```
  // Objek Dart -> teks JSON (String)
  final kirim = {'nama': 'Siti', 'umur': 21};
  final String json = jsonEncode(kirim);
  print(json); // {"nama":"Siti","umur":21}
}
```

`jsonDecode` menerima `String` dan mengembalikan `dynamic`. Karena itu kita menyebutkan tipe tujuannya (`Map<String, dynamic>`) agar editor bisa membantu.

Contoh Lengkap: Membaca Data Bersarang dan Array

`lib/contoh_json.dart`

```
// Jalankan dengan: dart run lib/contoh_json.dart
import 'dart:convert';

const String responseBody = '''
{
  "nim": "H1D022001",
  "nama": "Budi Santoso",
  "ipk": 3.75,
  "aktif": true,
  "prodi": { "kode": "IF", "nama": "Informatika" },
  "hobi": ["coding", "futsal"],
  "nilai": [
    { "matkul": "Pemrograman Mobile", "huruf": "A" },
    { "matkul": "Basis Data", "huruf": "AB" }
  ]
}
''';

void main() {
  // 1. Decode teks menjadi Map
  final Map<String, dynamic> mhs = jsonDecode(responseBody) as Map<String, dynamic>;

  // 2. Ambil nilai sederhana dengan cast "as"
  final String nama = mhs['nama'] as String;
  final double ipk = (mhs['ipk'] as num).toDouble(); // aman untuk 3 maupun 3.75
  final bool aktif = mhs['aktif'] as bool;
  print('$nama, IPK $ipk, aktif: $aktif');

  // 3. Object bersarang adalah Map di dalam Map
  final Map<String, dynamic> prodi = mhs['prodi'] as Map<String, dynamic>;
  print('Prodi: ${prodi['nama']}');

  // 4. Array berisi String: List<dynamic> harus dikonversi
  final List<String> hobi = List<String>.from(mhs['hobi'] as List);
  print('Hobi: ${hobi.join(', ')}');

  // 5. Array berisi object: tiap elemen adalah Map
  final List<dynamic> nilaiList = mhs['nilai'] as List<dynamic>;
  for (final item in nilaiList) {
    final nilai = item as Map<String, dynamic>;
    print('- ${nilai['matkul']}: ${nilai['huruf']}');
  }

  // 6. Field yang tidak ada menghasilkan null, gunakan ?? untuk nilai default
```

```
  final String? email = mhs['email'] as String?;
  print('Email: ${email ?? 'belum diisi'}');

  // 7. Encode kembali; JsonEncoder.withIndent membuat output rapi
  mhs['aktif'] = false;
  const encoder = JsonEncoder.withIndent('  ');
  print(encoder.convert(mhs));
}
```

Penjelasan baris penting:

- **Langkah 2**: angka JSON `3` menjadi `int`, sedangkan `3.75` menjadi `double`. Menulis `as double` akan crash jika server mengirim `3`. Pola `(x as num).toDouble()` aman untuk keduanya.
- **Langkah 4**: hasil decode array selalu `List<dynamic>`. `List<String>.from(...)` menyalin isinya ke list bertipe benar.
- **Langkah 6**: kunci yang tidak ada di Map menghasilkan `null`, bukan error. Pakai tipe nullable (`String?`) lalu beri nilai default.

Mengirim Data: Map Dart ke Body JSON

Saat mengirim POST/PUT, body harus berupa String JSON. Objek seperti `DateTime` tidak bisa langsung di-encode, jadi ubah dulu menjadi String:

```
import 'dart:convert';

void main() {
  final sekarang = DateTime(2026, 9, 1, 8, 30);

  final body = jsonEncode({
    'judul': 'Kuis 1',
    'tanggal': sekarang.toIso8601String(), // DateTime -> "2026-09-01T08:30:00.000"
    'peserta': ['Budi', 'Siti'],
    'selesai': false,
  });
  print(body);

  // Membaca kembali tanggal dari JSON
  final data = jsonDecode(body) as Map<String, dynamic>;
  final tanggal = DateTime.parse(data['tanggal'] as String);
  print(tanggal.day); // 1
}
```

⚠️ Kesalahan Umum

- `type 'List<dynamic>' is not a subtype of type 'List<String>'` : menulis `mhs['hobi'] as List<String>`. Gunakan `List<String>.from(...)` atau `.cast<String>()`.
- `type 'int' is not a subtype of type 'double'` : lihat langkah 2, pakai `num`.
- `FormatException: Unexpected character` : teks yang di-decode bukan JSON, biasanya halaman HTML error dari server. Cek `statusCode` sebelum decode.
- `Converting object to an encodable object failed: Instance of 'DateTime'` : encode objek yang bukan tipe JSON. Ubah ke String/Map dulu.
- Salah huruf kunci: `data['Nama']` berbeda dengan `data['nama']`. Kunci JSON *case-sensitive*. Inilah alasan kita membuat Model Class di section berikutnya.

3. Model Dart: fromJson & toJson (Nested & List)

Jangan pernah menggunakan raw `Map<String, dynamic>` langsung di widget UI karena rentan salah ketik kunci string (typo) dan runtime error. Selalu petakan respon API ke dalam **Model Class** berstruktur kuat. Dengan model, editor akan memberi saran otomatis (*autocomplete*) dan kesalahan nama field langsung terdeteksi sebelum aplikasi dijalankan.

Pola Dasar: `fromJson` dan `toJson`

- `factory NamaModel.fromJson(Map<String, dynamic> json)`: mengubah Map hasil `jsonDecode` menjadi objek.
- `Map<String, dynamic> toJson()`: mengubah objek menjadi Map yang siap di-`jsonEncode`.

Contoh model mahasiswa sederhana:

`lib/models/mahasiswa_model.dart`

```dart
// lib/models/mahasiswa_model.dart
// Model Data Mahasiswa Berbasis JSON
class MahasiswaModel {
  final int id;
  final String nama;
  final String email;
  final String nim;
  final String avatarUrl;

  MahasiswaModel({
    required this.id,
    required this.nama,
    required this.email,
    required this.nim,
    required this.avatarUrl,
  });

  // Factory Constructor untuk mengurai JSON dari API Server
  factory MahasiswaModel.fromJson(Map<String, dynamic> json) {
    return MahasiswaModel(
      id: json['id'] as int,
      nama: json['nama'] as String? ?? 'Mahasiswa Anonim',
      email: json['email'] as String? ?? '-',
      nim: json['nim'] as String? ?? '-',
      avatarUrl: json['avatar'] as String? ?? 'https://i.pravatar.cc/150',
    );
  }

  // Method untuk mengirim data objek kembali ke format JSON payload
  Map<String, dynamic> toJson() {
    return {
      'nama': nama,
      'email': email,
      'nim': nim,
    };
  }
}
```

Penjelasan: `json['nama'] as String? ?? 'Mahasiswa Anonim'` dibaca: "ambil `nama`, boleh null, jika null pakai nilai default". Pola ini membuat aplikasi tidak crash ketika server lupa mengirim satu field. Perhatikan juga `toJson()` tidak menyertakan `id` karena id biasanya dibuat oleh server.

Model untuk JSONPlaceholder: `Post`

Sepanjang modul ini kita memakai API publik gratis jsonplaceholder.typicode.com. Satu post di sana berbentuk:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident",
  "body": "quia et suscipit suscipit recusandae..."
}
```

lib/models/post.dart

```dart
class Post {
  final int? id; // null ketika post belum dikirim ke server
  final int userId;
  final String title;
  final String body;

  const Post({
    this.id,
    required this.userId,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int?,
      userId: json['userId'] as int? ?? 0,
      title: json['title'] as String? ?? '',
      body: json['body'] as String? ?? '',
    );
  }

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id, // "collection if": id hanya ditulis jika ada
      'userId': userId,
      'title': title,
      'body': body,
    };
  }

  // copyWith: membuat salinan dengan sebagian field diganti (berguna saat edit)
  Post copyWith({int? id, int? userId, String? title, String? body}) {
    return Post(
      id: id ?? this.id,
      userId: userId ?? this.userId,
      title: title ?? this.title,
      body: body ?? this.body,
```

```
    );
  }

  @override
  String toString() => 'Post(id: $id, title: $title)';
}
```

Mengubah Array JSON Menjadi `List<Post>`

Endpoint `/posts` mengembalikan array. Polanya selalu sama: decode menjadi `List`, lalu `map` setiap elemen ke `fromJson`.

```
List<Post> parsePosts(String responseBody) {
  final List<dynamic> data = jsonDecode(responseBody) as List<dynamic>;
  return data
      .map((item) => Post.fromJson(item as Map<String, dynamic>))
      .toList();
}
```

Object Bersarang (Nested) dan List di Dalam Model

Endpoint `/users` berisi object di dalam object (`address` berisi `geo`, serta `company`). Buat **satu class untuk setiap object**, lalu panggil `fromJson` milik class anak dari dalam class induk.

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
  "company": { "name": "Romaguera-Crona" }
}
```

`lib/models/user.dart`

```
class Geo {
  final double lat;
  final double lng;

  const Geo({required this.lat, required this.lng});

  // API mengirim koordinat sebagai String, jadi kita parse ke double
  factory Geo.fromJson(Map<String, dynamic> json) {
    return Geo(
      lat: double.tryParse(json['lat']?.toString() ?? '') ?? 0,
      lng: double.tryParse(json['lng']?.toString() ?? '') ?? 0,
    );
  }

  Map<String, dynamic> toJson() => {'lat': lat.toString(), 'lng': lng.toString()};
}
```

```dart
class Address {
  final String street;
  final String city;
  final Geo geo;

  const Address({required this.street, required this.city, required this.geo});

  factory Address.fromJson(Map<String, dynamic> json) {
    return Address(
      street: json['street'] as String? ?? '',
      city: json['city'] as String? ?? '',
      geo: Geo.fromJson(json['geo'] as Map<String, dynamic>? ?? {}),
    );
  }

  Map<String, dynamic> toJson() => {
        'street': street,
        'city': city,
        'geo': geo.toJson(), // anak juga di-toJson
      };
}

class User {
  final int id;
  final String name;
  final String email;
  final Address address;
  final String companyName;
  final List<String> roles; // contoh list String

  const User({
    required this.id,
    required this.name,
    required this.email,
    required this.address,
    required this.companyName,
    this.roles = const [],
  });

  factory User.fromJson(Map<String, dynamic> json) {
    final company = json['company'] as Map<String, dynamic>? ?? {};
    return User(
      id: json['id'] as int,
      name: json['name'] as String? ?? '',
      email: json['email'] as String? ?? '',
      address: Address.fromJson(json['address'] as Map<String, dynamic>),
      companyName: company['name'] as String? ?? '-',
      roles: List<String>.from(json['roles'] as List? ?? const []),
    );
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'name': name,
        'email': email,
        'address': address.toJson(),
        'company': {'name': companyName},
        'roles': roles,
```

```
      };
}
```

Contoh Lengkap yang Bisa Dijalankan

Program berikut memakai model `Post` dan `User` di atas dengan data JSON contoh (belum memakai internet), supaya kamu bisa fokus pada proses parsing.

lib/contoh_model.dart

```dart
// Jalankan dengan: dart run lib/contoh_model.dart
import 'dart:convert';

import 'models/post.dart';
import 'models/user.dart';

const postsJson = '''
[
  {"userId": 1, "id": 1, "title": "Halo Flutter", "body": "Belajar REST API"},
  {"userId": 1, "id": 2, "title": "JSON itu mudah", "body": "Asal pakai model"}
]
''';

const userJson = '''
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "geo": {"lat": "-37.3159", "lng": "81.1496"}
  },
  "company": {"name": "Romaguera-Crona"},
  "roles": ["admin", "editor"]
}
''';

void main() {
  // Array -> List<Post>
  final List<dynamic> rawPosts = jsonDecode(postsJson) as List<dynamic>;
  final posts = rawPosts
      .map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();
  for (final post in posts) {
    print('${post.id}. ${post.title}');
  }

  // Object bersarang -> User
  final user = User.fromJson(jsonDecode(userJson) as Map<String, dynamic>);
  print('${user.name} tinggal di ${user.address.city}');
  print('Koordinat: ${user.address.geo.lat}, ${user.address.geo.lng}');
  print('Peran: ${user.roles.join(', ')}');

  // Model -> JSON (untuk dikirim ke server)
  final baru = Post(userId: 1, title: 'Post baru', body: 'Isi');
  print(jsonEncode(baru.toJson())); // tanpa "id"
```

```dart
  // copyWith untuk edit
  final diedit = posts.first.copyWith(title: 'Judul diganti');
  print(diedit); // Post(id: 1, title: Judul diganti)
}
```

> 💡 **Tips: Generator Otomatis**
>
> Untuk model yang sangat besar, ada package `json_serializable` yang membuat `fromJson` / `toJson` otomatis. Namun pahami dulu cara manual seperti di atas, karena hasil generator pun bentuknya sama.

> ⚠️ **Kesalahan Umum**
>
> - Lupa memanggil `Address.fromJson(...)` sehingga field bertipe `Address` diisi Map, hasilnya error tipe.
> - Lupa memanggil `address.toJson()` di dalam `toJson()` induk sehingga `jsonEncode` gagal mengubah objek `Address`.
> - Memakai `json['id'] as int` untuk field yang kadang dikirim sebagai String (`"1"`). Jika tidak yakin, gunakan `int.tryParse(json['id'].toString())`.
> - Nama kunci di model tidak sama persis dengan JSON (`userid` vs `userId`). Salin kunci langsung dari contoh response.

4. Persiapan: Package http & Izin Internet Android/iOS

Sebelum request pertama, ada dua hal yang perlu disiapkan: memasang package `http` dan memastikan aplikasi **diizinkan** mengakses internet di setiap platform. Banyak mahasiswa mengira kodenya salah, padahal penyebabnya izin yang belum ditambahkan.

1. Memasang Package `http`

```
flutter pub add http
```

Lalu impor dengan awalan `as http` agar fungsi-fungsinya jelas berasal dari package ini (`http.get`, `http.post`, dan seterusnya):

```dart
import 'package:http/http.dart' as http;

Future<void> cobaKoneksi() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
  );
  print('Status: ${response.statusCode}');
  print('Body: ${response.body}');
}
```

2. Android: Izin INTERNET

Mode debug otomatis punya izin internet, tetapi **build release (APK yang dibagikan) tidak**. Tambahkan baris berikut di `android/app/src/main/AndroidManifest.xml`, tepat di atas tag `<application>`:

`android/app/src/main/AndroidManifest.xml`

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="aplikasi_saya"
        ...>
```

3. Android: Server Lokal Tanpa HTTPS

Saat backend dijalankan di laptop sendiri (Laravel, Express, dsb.) alamatnya biasanya `http://` tanpa "s". Android 9 ke atas memblokir HTTP biasa (*cleartext*). Untuk keperluan belajar, izinkan lewat atribut di tag `<application>`:

```
<application
    android:usesCleartextTraffic="true"
    ...>
```

> ℹ️ **localhost di Emulator Android**
>
> Di dalam emulator Android, `localhost` berarti emulator itu sendiri, bukan laptop. Gunakan `http://10.0.2.2:8000` untuk mengakses server di laptop. Jika memakai HP fisik, gunakan IP laptop di jaringan Wi-Fi yang sama, misalnya `http://192.168.1.10:8000`.

4. iOS

iOS tidak membutuhkan izin khusus untuk internet. Namun iOS memblokir `http://` (App Transport Security). Untuk server lokal saat belajar, tambahkan di `ios/Runner/Info.plist` di dalam tag `<dict>` utama:

📄 **`ios/Runner/Info.plist`**

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsLocalNetworking</key>
    <true/>
</dict>
```

Untuk server produksi, selalu gunakan `https://` sehingga pengaturan ini tidak diperlukan.

5. macOS Desktop dan Web

- **macOS**: tambahkan `<key>com.apple.security.network.client</key><true/>` di `macos/Runner/DebugProfile.entitlements` dan `Release.entitlements`.
- **Web (Chrome)**: tidak butuh izin, tetapi server harus mengizinkan **CORS**. JSONPlaceholder sudah mengizinkan. Jika API kampus menolak, error yang muncul adalah `ClientException: XMLHttpRequest error`.

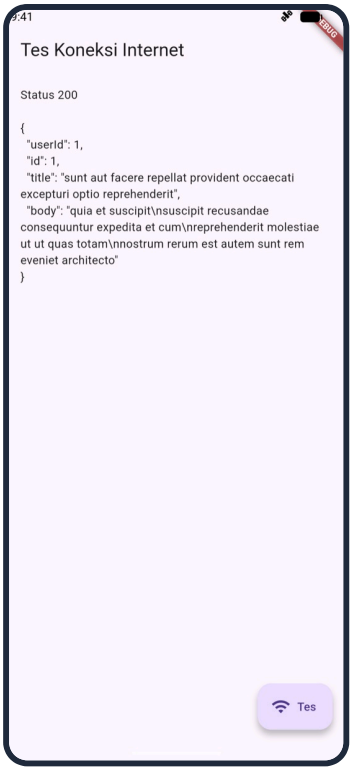
Contoh Lengkap: Tes Koneksi

Gunakan aplikasi kecil ini untuk memastikan semua izin sudah benar sebelum lanjut ke section berikutnya.

📄 **`lib/main.dart`**

```dart
// Pastikan sudah: flutter pub add http
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: TesKoneksiScreen()));

class TesKoneksiScreen extends StatefulWidget {
  const TesKoneksiScreen({super.key});

  @override
  State<TesKoneksiScreen> createState() => _TesKoneksiScreenState();
}
```

```dart
class _TesKoneksiScreenState extends State<TesKoneksiScreen> {
  String _hasil = 'Tekan tombol untuk mengetes koneksi.';
  bool _loading = false;

  Future<void> _tes() async {
    setState(() => _loading = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
      );
      _hasil = 'Status ${response.statusCode}\n\n${response.body}';
    } catch (e) {
      _hasil = 'Gagal terhubung: $e';
    }
    if (!mounted) return;
    setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tes Koneksi Internet')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: _loading
            ? const Center(child: CircularProgressIndicator())
            : SingleChildScrollView(child: Text(_hasil)),
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _loading ? null : _tes,
        icon: const Icon(Icons.wifi),
        label: const Text('Tes'),
      ),
    );
  }
}
```



Setelah tombol Tes ditekan: status 200 dan JSON post #1

Jika muncul `Status 200` beserta JSON, koneksi sudah siap. Jika muncul `Failed host lookup`, cek internet emulator atau izin di atas.

> ⚠️ **Kesalahan Umum**
>
> - Aplikasi jalan saat debug tetapi APK release tidak bisa memuat data: lupa izin INTERNET.
> - Memakai `http://localhost:8000` di emulator Android: gunakan `10.0.2.2`.
> - Menambah izin di `android/app/src/debug/AndroidManifest.xml`, padahal yang dipakai release adalah folder `main`.
> - Mengira hot reload sudah cukup setelah mengubah file native (Manifest/Info.plist). Aplikasi harus **di-stop lalu dijalankan ulang**.

5. GET: Mengambil Daftar dan Detail Data

`GET` adalah request yang paling sering dipakai: mengambil daftar data (list) dan mengambil satu data berdasarkan id (detail). Di section ini kita membaca data post dari JSONPlaceholder lalu menampilkannya di layar.

Langkah-langkah Satu Request GET

1. Susun `Uri`.
2. Panggil `await http.get(uri)` untuk mendapatkan `http.Response`.
3. Cek `response.statusCode`.
4. Decode `response.body` dengan `jsonDecode`.
5. Ubah Map/List menjadi model dengan `fromJson`.

Contoh Kecil: GET Daftar

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

Future<List<Post>> fetchPosts() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/posts'); // 1
  final response = await http.get(uri);                                 // 2

  if (response.statusCode != 200) {                                     // 3
    throw Exception('Gagal memuat post (HTTP ${response.statusCode})');
  }

  final List<dynamic> data = jsonDecode(response.body) as List<dynamic>; // 4
  return data
      .map((e) => Post.fromJson(e as Map<String, dynamic>))              // 5
      .toList();
}
```

Contoh Kecil: GET Detail dan GET dengan Query

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

// GET /posts/{id} -> satu object
Future<Post> fetchPostById(int id) async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts/$id'),
  );
  if (response.statusCode == 404) {
    throw Exception('Post dengan id $id tidak ditemukan');
  }
```

```
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// GET /posts?userId=1 -> filter memakai query parameter
Future<List<Post>> fetchPostsByUser(int userId) async {
  final uri = Uri.https('jsonplaceholder.typicode.com', '/posts', {
    'userId': '$userId',
  });
  final response = await http.get(uri);
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  final data = jsonDecode(response.body) as List<dynamic>;
  return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
}
```

Perbedaannya hanya pada hasil decode: endpoint daftar mengembalikan **array** (`List`), endpoint detail mengembalikan **object** (`Map`).

Contoh Lengkap: Daftar Post dan Halaman Detail

Kita kumpulkan fungsi GET dalam satu file service, lalu memakainya di UI. Model `Post` diambil dari section sebelumnya (`lib/models/post.dart`).

`lib/services/post_service.dart`

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../models/post.dart';

class PostService {
  static const String _baseUrl = 'https://jsonplaceholder.typicode.com';

  Future<List<Post>> fetchPosts({int limit = 20}) async {
    final uri = Uri.parse('$_baseUrl/posts?_limit=$limit');
    final response = await http.get(uri);
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat daftar post (HTTP ${response.statusCode})');
    }
    final data = jsonDecode(response.body) as List<dynamic>;
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }

  Future<Post> fetchPost(int id) async {
    final response = await http.get(Uri.parse('$_baseUrl/posts/$id'));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat post $id (HTTP ${response.statusCode})');
    }
    return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
  }

  // Komentar milik satu post: GET /posts/{id}/comments
```

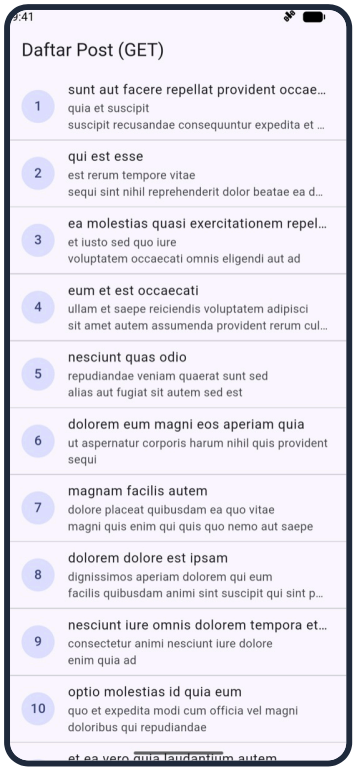
```
  Future<List<String>> fetchCommentBodies(int postId) async {
    final response = await http.get(Uri.parse('$_baseUrl/posts/$postId/comments'));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat komentar (HTTP ${response.statusCode})');
    }
    final data = jsonDecode(response.body) as List<dynamic>;
    return data.map((e) => (e as Map<String, dynamic>)['body'] as String).toList();
  }
}
```

lib/main.dart

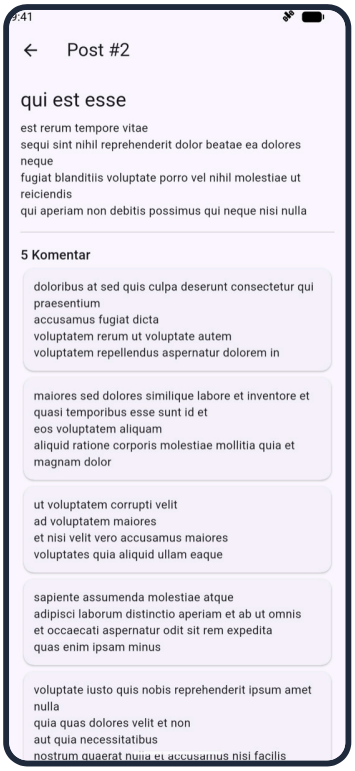
```
// Pastikan sudah: flutter pub add http
// serta membuat lib/models/post.dart dan lib/services/post_service.dart
import 'package:flutter/material.dart';

import 'models/post.dart';
import 'services/post_service.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PostListScreen(),
    );
  }
}

class PostListScreen extends StatefulWidget {
  const PostListScreen({super.key});

  @override
  State<PostListScreen> createState() => _PostListScreenState();
}

class _PostListScreenState extends State<PostListScreen> {
  final _service = PostService();
  List<Post> _posts = [];
  bool _loading = true;
  String? _error;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    setState(() {
      _loading = true;
      _error = null;
    });
```

```
    try {
      final hasil = await _service.fetchPosts();
      if (!mounted) return;
      setState(() => _posts = hasil);
    } catch (e) {
      if (!mounted) return;
      setState(() => _error = e.toString());
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    Widget body;
    if (_loading) {
      body = const Center(child: CircularProgressIndicator());
    } else if (_error != null) {
      body = Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(_error!, textAlign: TextAlign.center),
            const SizedBox(height: 12),
            FilledButton(onPressed: _load, child: const Text('Coba Lagi')),
          ],
        ),
      );
    } else {
      body = ListView.separated(
        itemCount: _posts.length,
        separatorBuilder: (context, index) => const Divider(height: 1),
        itemBuilder: (context, index) {
          final post = _posts[index];
          return ListTile(
            leading: CircleAvatar(child: Text('${post.id}')),
            title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
            subtitle: Text(post.body, maxLines: 2, overflow: TextOverflow.ellipsis),
            onTap: () => Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => PostDetailScreen(postId: post.id!)),
            ),
          );
        },
      );
    }

    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Post (GET)')),
      body: body,
    );
  }
}

class PostDetailScreen extends StatefulWidget {
  final int postId;
  const PostDetailScreen({super.key, required this.postId});

  @override
```

```
  State<PostDetailScreen> createState() => _PostDetailScreenState();
}

class _PostDetailScreenState extends State<PostDetailScreen> {
  final _service = PostService();
  Post? _post;
  List<String> _komentar = [];
  String? _error;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    try {
      // Dua request dijalankan bersamaan dengan Future.wait
      final hasil = await Future.wait([
        _service.fetchPost(widget.postId),
        _service.fetchCommentBodies(widget.postId),
      ]);
      if (!mounted) return;
      setState(() {
        _post = hasil[0] as Post;
        _komentar = hasil[1] as List<String>;
      });
    } catch (e) {
      if (!mounted) return;
      setState(() => _error = e.toString());
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Post #${widget.postId}')),
      body: _error != null
          ? Center(child: Text(_error!))
          : _post == null
              ? const Center(child: CircularProgressIndicator())
              : ListView(
                  padding: const EdgeInsets.all(16),
                  children: [
                    Text(_post!.title, style: Theme.of(context).textTheme.headlineSmall),
                    const SizedBox(height: 8),
                    Text(_post!.body),
                    const Divider(height: 32),
                    Text('${_komentar.length} Komentar',
                        style: Theme.of(context).textTheme.titleMedium),
                    for (final k in _komentar)
                      Card(child: Padding(padding: const EdgeInsets.all(12), child: Text(k))),
                  ],
                ),
    );
  }
}
```



Daftar post hasil GET /posts



Halaman detail: GET /posts/2 dan komentarnya

Penjelasan:

- Layar daftar menyimpan tiga state: `_loading`, `_error`, dan `_posts`. Ketiganya menentukan widget apa yang tampil. Di section FutureBuilder nanti, pola ini dibuat lebih ringkas.
- `if (!mounted) return;` setelah `await` mencegah error *setState() called after dispose()* ketika pengguna sudah keluar dari layar sebelum data datang.
- Halaman detail hanya menerima `postId`, lalu mengambil data sendiri. `Future.wait` menjalankan dua request sekaligus sehingga lebih cepat daripada berurutan.
- `?_limit=20` adalah fitur JSONPlaceholder untuk membatasi jumlah data. Setiap API punya nama parameter sendiri (`limit`, `per_page`, `size`), cek dokumentasinya.

Contoh Lain: Memetakan Nama Field yang Berbeda

Kadang nama field di server berbeda dengan model kita. Service berikut mengambil `/users` lalu memetakannya ke `MahasiswaModel` dari section 3 (field `name` menjadi `nama`, dan NIM dibuat dari id sebagai data contoh).

`lib/services/mahasiswa_api_service.dart`

```dart
// lib/services/mahasiswa_api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../models/mahasiswa_model.dart';

class MahasiswaApiService {
  // Ganti dengan endpoint server API kampus Anda atau mock API
```

```dart
  static const String baseUrl = "https://jsonplaceholder.typicode.com";

  // 1. GET: Mengambil daftar mahasiswa
  Future<List<MahasiswaModel>> fetchMahasiswa() async {
    final response = await http.get(
      Uri.parse('$baseUrl/users'),
      headers: {'Accept': 'application/json'},
    ).timeout(const Duration(seconds: 10));

    if (response.statusCode == 200) {
      final List<dynamic> rawList = json.decode(response.body);
      return rawList.map((item) => MahasiswaModel.fromJson({
        'id': item['id'],
        'nama': item['name'],
        'email': item['email'],
        'nim': '2110${item['id']}',
        'avatar': 'https://i.pravatar.cc/150?u=${item['id']}'
      })).toList();
    } else {
      throw Exception("Gagal memuat data dari server (HTTP ${response.statusCode})");
    }
  }

  // 2. POST: Mengirim data mahasiswa baru ke server
  Future<bool> createMahasiswa(MahasiswaModel mhs) async {
    final response = await http.post(
      Uri.parse('$baseUrl/users'),
      headers: {'Content-Type': 'application/json; charset=UTF-8'},
      body: json.encode(mhs.toJson()),
    );

    return response.statusCode == 201; // 201 Created
  }
}
```

⚠️ Kesalahan Umum

- Lupa `await` sehingga variabel berisi `Future<Response>`, bukan `Response`.
- Decode endpoint detail sebagai `List` (atau sebaliknya). Lihat dulu bentuk JSON-nya di browser.
- Memanggil fungsi GET di dalam `build()`. `build()` bisa dipanggil puluhan kali, sehingga request terkirim berulang. Panggil di `initState()`.
- Teks dengan huruf non-ASCII tampil aneh (mis. `Ã©`). Jika server tidak mengirim charset, gunakan `utf8.decode(response.bodyBytes)` sebagai pengganti `response.body`.

6. POST, PUT, PATCH, dan DELETE

Setelah bisa membaca data, sekarang kita **mengirim** data: membuat (POST), mengubah (PUT/PATCH), dan menghapus (DELETE). Ketiga method ini mirip GET, bedanya ada **body JSON** dan **header** `Content-Type`.

> ℹ️ **JSONPlaceholder Hanya Berpura-pura Menyimpan**
>
> JSONPlaceholder menerima POST/PUT/PATCH/DELETE dan membalas seolah berhasil, tetapi **data tidak benar-benar tersimpan**. POST selalu mengembalikan `id: 101`, dan jika kamu GET lagi, data baru tidak muncul. Ini normal. Untuk latihan, kita perbarui list di aplikasi secara lokal berdasarkan response server. Untuk data yang benar-benar tersimpan, gunakan mockapi.io atau backend sendiri (dan Supabase di Modul 12).

Pola Umum Request dengan Body

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

Future<void> contohPost() async {
  final response = await http.post(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    headers: {'Content-Type': 'application/json; charset=UTF-8'}, // wajib untuk JSON
    body: jsonEncode({'title': 'Halo', 'body': 'Isi', 'userId': 1}), // Map -> String
  );
  print(response.statusCode); // 201
  print(response.body);       // {"title": "Halo", "body": "Isi", "userId": 1, "id": 101}
}
```

PUT vs PATCH

	PUT	PATCH
Isi body	Seluruh field objek	Hanya field yang berubah
Field yang tidak dikirim	Bisa terhapus/di-reset oleh server	Tetap seperti semula
Contoh	`{"id":1,"title":"Baru","body":"...","userId":1}`	`{"title":"Baru"}`
Kapan dipakai	Form edit yang mengirim semua isian	Toggle status, ganti satu field

Contoh Kecil Setiap Method

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'models/post.dart';

const baseUrl = 'https://jsonplaceholder.typicode.com';
const jsonHeaders = {'Content-Type': 'application/json; charset=UTF-8'};

// CREATE: POST /posts -> 201 Created
Future<Post> createPost(Post post) async {
  final response = await http.post(
    Uri.parse('$baseUrl/posts'),
    headers: jsonHeaders,
    body: jsonEncode(post.toJson()),
  );
  if (response.statusCode != 201) {
    throw Exception('Gagal membuat post (HTTP ${response.statusCode})');
  }
  // Server mengembalikan data yang sudah memiliki id
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// UPDATE penuh: PUT /posts/{id} -> 200 OK
Future<Post> updatePost(Post post) async {
  final response = await http.put(
    Uri.parse('$baseUrl/posts/${post.id}'),
    headers: jsonHeaders,
    body: jsonEncode(post.toJson()),
  );
  if (response.statusCode != 200) {
    throw Exception('Gagal mengubah post (HTTP ${response.statusCode})');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}
```

```dart
// UPDATE sebagian: PATCH /posts/{id} -> 200 OK
Future<Post> updateTitle(int id, String title) async {
  final response = await http.patch(
    Uri.parse('$baseUrl/posts/$id'),
    headers: jsonHeaders,
    body: jsonEncode({'title': title}), // hanya field yang berubah
  );
  if (response.statusCode != 200) {
    throw Exception('Gagal mengubah judul (HTTP ${response.statusCode})');
  }
  return Post.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
}

// DELETE /posts/{id} -> 200 OK (beberapa API memakai 204 No Content)
Future<void> deletePost(int id) async {
  final response = await http.delete(Uri.parse('$baseUrl/posts/$id'));
  if (response.statusCode != 200 && response.statusCode != 204) {
    throw Exception('Gagal menghapus post (HTTP ${response.statusCode})');
  }
}
```

Penjelasan: setiap fungsi mengecek status code yang sesuai standar (POST = 201, lainnya = 200). Fungsi create/update mengembalikan `Post` dari response server karena server bisa menambahkan data (misalnya `id` atau waktu dibuat). DELETE tidak perlu mengembalikan apa pun.

Contoh Lengkap: Playground CRUD

Aplikasi berikut punya 5 tombol (GET, POST, PUT, PATCH, DELETE) dan menampilkan status code serta body response. Cara cepat untuk melihat perilaku setiap method.

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add http
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: CrudPlaygroundScreen()));

class CrudPlaygroundScreen extends StatefulWidget {
  const CrudPlaygroundScreen({super.key});

  @override
  State<CrudPlaygroundScreen> createState() => _CrudPlaygroundScreenState();
}

class _CrudPlaygroundScreenState extends State<CrudPlaygroundScreen> {
  static const _base = 'https://jsonplaceholder.typicode.com/posts';
  static const _headers = {'Content-Type': 'application/json; charset=UTF-8'};

  String _output = 'Pilih salah satu method di bawah.';
  bool _loading = false;
```

```
  // Satu fungsi pembantu agar setiap tombol cukup mengirim request-nya
  Future<void> _jalankan(String label, Future<http.Response> Function() request) async {
    setState(() {
      _loading = true;
      _output = 'Mengirim $label ...';
    });
    try {
      final res = await request();
      final pretty = res.body.isEmpty
          ? '(body kosong)'
          : const JsonEncoder.withIndent('  ').convert(jsonDecode(res.body));
      _output = '$label\nStatus: ${res.statusCode}\n\n$pretty';
    } catch (e) {
      _output = '$label gagal: $e';
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    final tombol = <String, Future<http.Response> Function()>{
      'GET /posts/1': () => http.get(Uri.parse('$_base/1')),
      'POST /posts': () => http.post(
            Uri.parse(_base),
            headers: _headers,
            body: jsonEncode({'title': 'Post Baru', 'body': 'Dari Flutter', 'userId': 1}),
          ),
      'PUT /posts/1': () => http.put(
            Uri.parse('$_base/1'),
            headers: _headers,
            body: jsonEncode({'id': 1, 'title': 'Diganti total', 'body': 'Baru', 'userId': 1}),
          ),
      'PATCH /posts/1': () => http.patch(
            Uri.parse('$_base/1'),
            headers: _headers,
            body: jsonEncode({'title': 'Hanya judul berubah'}),
          ),
      'DELETE /posts/1': () => http.delete(Uri.parse('$_base/1')),
    };

    return Scaffold(
      appBar: AppBar(title: const Text('Playground CRUD HTTP')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Wrap(
              spacing: 8,
              runSpacing: 8,
              children: [
                for (final entry in tombol.entries)
                  FilledButton.tonal(
                    onPressed: _loading ? null : () => _jalankan(entry.key, entry.value),
                    child: Text(entry.key),
                  ),
              ],
            ),
          ),
          const Divider(height: 1),
```

```
          Expanded(
            child: SingleChildScrollView(
              padding: const EdgeInsets.all(16),
              child: SelectableText(
                _output,
                style: const TextStyle(fontFamily: 'monospace'),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

POST /posts: status 201 dan id 101

PATCH /posts/1: hanya judul yang berubah

Coba tekan **PATCH**: response berisi seluruh post dengan judul baru, sedangkan **PUT** mengembalikan persis yang kita kirim. **DELETE** mengembalikan `{}` dengan status 200.

⚠️ **Kesalahan Umum**

- **Lupa `jsonEncode`**: jika `body` diisi Map langsung, package `http` mengirimnya sebagai form (`title=Halo&body=Isi`), bukan JSON. Server JSON akan menolak atau mengabaikannya.
- **Lupa header `Content-Type`**: banyak server (Laravel, Express) tidak membaca body sebagai JSON tanpa header ini.
- Mengecek `statusCode == 200` untuk POST, padahal standarnya `201`. Lebih aman cek rentang 2xx.
- Menganggap data JSONPlaceholder tersimpan, lalu bingung karena GET tidak menampilkan data baru.
- PUT ke id yang tidak ada (mis. `/posts/101`) di JSONPlaceholder menghasilkan 500. Tangani error ini, jangan biarkan aplikasi crash.

7. Header & Bearer Token

Kebanyakan API sungguhan (SIA kampus, marketplace) hanya memberikan data kepada pengguna yang sudah login. Setelah login, server memberi **token**. Token itu kemudian dikirim di setiap request melalui **header** `Authorization` dengan format `Bearer <token>`.

Alur Autentikasi Token

```
1. POST /login  {email, password}        -->  server cek akun
                                          <--  200 {"token": "abc.def.ghi"}
2. Simpan token (memori / SharedPreferences, lihat Modul 10)
3. GET /profil
   Authorization: Bearer abc.def.ghi      -->  server cek token
                                          <--  200 {data profil}
4. Token salah / kedaluwarsa              <--  401 Unauthorized  -> arahkan ke login
```

Contoh Kecil: Menambahkan Header

Parameter `headers` menerima `Map<String, String>`. Perhatikan ada **spasi** setelah kata `Bearer`.

```dart
import 'package:http/http.dart' as http;

Future<void> ambilProfil(String token) async {
  final response = await http.get(
    Uri.parse('https://httpbin.org/bearer'),
    headers: {
      'Accept': 'application/json',
      'Authorization': 'Bearer $token',
    },
  );
  print(response.statusCode); // 200 jika ada token, 401 jika tidak
}
```

Kita memakai httpbin.org, layanan uji gratis. Endpoint `/bearer` membalas 200 jika header Bearer ada dan 401 jika tidak, sedangkan `/headers` mengembalikan semua header yang kita kirim.

Fungsi Pembuat Header

Daripada menulis header berulang di setiap request, buat satu fungsi:

```dart
Map<String, String> buildHeaders({String? token, bool json = true}) {
  return {
    'Accept': 'application/json',
    if (json) 'Content-Type': 'application/json; charset=UTF-8',
    if (token != null) 'Authorization': 'Bearer $token',
  };
}
```

Cara Lebih Rapi: `http.BaseClient` yang Menyisipkan Token Otomatis

Kita bisa membuat client sendiri yang otomatis menambahkan header ke **setiap** request. Ini mirip "interceptor" di library lain.

`lib/services/auth_client.dart`

```dart
import 'package:http/http.dart' as http;

class AuthClient extends http.BaseClient {
  final http.Client _inner;
  String? token; // bisa diganti setelah login / logout

  AuthClient({http.Client? inner, this.token}) : _inner = inner ?? http.Client();

  @override
  Future<http.StreamedResponse> send(http.BaseRequest request) {
    request.headers['Accept'] = 'application/json';
    final t = token;
    if (t != null) {
      request.headers['Authorization'] = 'Bearer $t';
    }
    return _inner.send(request);
  }

  @override
  void close() {
    _inner.close();
    super.close();
  }
}
```

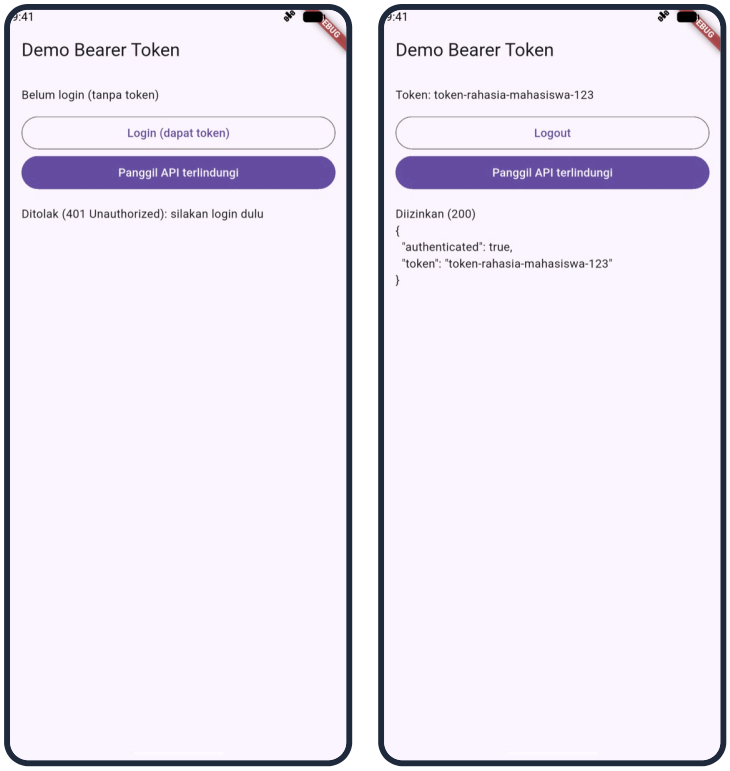
`AuthClient` bisa dipakai persis seperti package `http`: `client.get(...)`, `client.post(...)`, dan seterusnya. Semua method itu pada akhirnya memanggil `send()`, sehingga header pasti ikut.

Contoh Lengkap: Simulasi Login dengan Token

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add http, dan membuat lib/services/auth_client.dart
import 'package:flutter/material.dart';

import 'services/auth_client.dart';

void main() => runApp(const MaterialApp(home: TokenDemoScreen()));

class TokenDemoScreen extends StatefulWidget {
  const TokenDemoScreen({super.key});

  @override
  State<TokenDemoScreen> createState() => _TokenDemoScreenState();
}

class _TokenDemoScreenState extends State<TokenDemoScreen> {
  final _client = AuthClient();
  String _hasil = '-';

  @override
  void dispose() {
    _client.close(); // tutup koneksi saat layar dibuang
    super.dispose();
  }
```

```
  void _login() {
    // Pada aplikasi nyata token didapat dari response POST /login
    setState(() => _client.token = 'token-rahasia-mahasiswa-123');
  }

  void _logout() {
    setState(() => _client.token = null);
  }

  Future<void> _panggilApi() async {
    setState(() => _hasil = 'Memanggil /bearer ...');
    try {
      final res = await _client.get(Uri.parse('https://httpbin.org/bearer'));
      final pesan = switch (res.statusCode) {
        200 => 'Diizinkan (200)\n${res.body}',
        401 => 'Ditolak (401 Unauthorized): silakan login dulu',
        _ => 'Status tak terduga: ${res.statusCode}',
      };
      if (mounted) setState(() => _hasil = pesan);
    } catch (e) {
      if (mounted) setState(() => _hasil = 'Error jaringan: $e');
    }
  }

  @override
  Widget build(BuildContext context) {
    final sudahLogin = _client.token != null;
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Bearer Token')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text(sudahLogin ? 'Token: ${_client.token}' : 'Belum login (tanpa token)'),
            const SizedBox(height: 12),
            OutlinedButton(
              onPressed: sudahLogin ? _logout : _login,
              child: Text(sudahLogin ? 'Logout' : 'Login (dapat token)'),
            ),
            FilledButton(onPressed: _panggilApi, child: const Text('Panggil API terlindungi')),
            const SizedBox(height: 16),
            Expanded(child: SingleChildScrollView(child: Text(_hasil))),
          ],
        ),
      ),
    );
  }
}
```



Memanggil API sebelum login: 401 | Setelah login: token terkirim, status 200

Penjelasan: panggil API sebelum login, hasilnya 401. Tekan Login lalu panggil lagi, hasilnya 200 dan httpbin menampilkan token yang diterimanya. Ekspresi `switch` (Dart 3) membuat penanganan setiap status code ringkas.

⚠️ Kesalahan Umum & Keamanan

- Menulis `'Bearer$token'` tanpa spasi, atau `'bearer'` huruf kecil pada server yang ketat.
- Menyimpan **API key rahasia** (kunci admin, secret key) di kode Flutter. Aplikasi bisa dibongkar; kunci rahasia harus tetap di server.
- Token login yang sensitif sebaiknya disimpan dengan `flutter_secure_storage`. SharedPreferences (Modul 10) cukup untuk latihan, tetapi tidak terenkripsi.
- Tidak menangani 401. Saat token kedaluwarsa, hapus token dan arahkan pengguna ke halaman login.

8. Penanganan Error, Timeout, dan Offline

Jaringan itu tidak bisa diandalkan: sinyal hilang di dalam lift, server kampus down saat KRS-an, atau URL salah ketik. Aplikasi yang baik **tidak crash** dan memberi pesan yang dimengerti pengguna. Di section ini kita mengenali jenis-jenis kegagalan dan cara menanganinya satu per satu.

Dua Jenis Kegagalan

Jenis	Contoh	Bagaimana terdeteksi
Request sampai, tetapi server menolak	404, 401, 500	Tidak ada exception! Harus cek `response.statusCode`.
Tidak ada internet / host salah	Mode pesawat, DNS gagal	`SocketException` / `http.ClientException`
Server terlalu lama	Server lambat, sinyal lemah	`TimeoutException` (jika memakai `.timeout()`)
Response bukan JSON yang diharapkan	Halaman HTML error	`FormatException` / `TypeError`

Langkah 1: Cek Status Code

```dart
import 'package:http/http.dart' as http;

Future<String> ambil() async {
  final response = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/9999'));

  // http TIDAK melempar error untuk 404, jadi kita cek sendiri
  if (response.statusCode >= 200 && response.statusCode < 300) {
    return response.body;
  }
  throw Exception('Server membalas ${response.statusCode}');
}
```

Langkah 2: Tangkap Exception dengan `try/catch` dan `on`

Klausa `on NamaException catch (e)` menangkap jenis error tertentu. Urutannya dari yang paling spesifik ke paling umum.

```dart
import 'dart:async'; // TimeoutException
import 'dart:io'; // SocketException (tidak tersedia di web)

import 'package:http/http.dart' as http;
```

```dart
Future<String> ambilDenganPenanganan() async {
  try {
    final response = await http
        .get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'))
        .timeout(const Duration(seconds: 10)); // batas waktu 10 detik
    return 'Status ${response.statusCode}';
  } on SocketException {
    return 'Tidak ada koneksi internet.';
  } on TimeoutException {
    return 'Server terlalu lama merespons. Coba lagi.';
  } on http.ClientException catch (e) {
    return 'Gagal menghubungi server: ${e.message}';
  } catch (e) {
    return 'Terjadi kesalahan tak terduga: $e';
  }
}
```

> **SocketException vs ClientException**
>
> Di Android/iOS, error koneksi dari package `http` berbentuk `ClientException` yang *juga* merupakan `SocketException`, sehingga kedua klausa di atas bisa menangkapnya. Di Flutter Web, `dart:io` tidak berfungsi dan error koneksi hanya berupa `http.ClientException`. Karena itu selalu sediakan klausa `http.ClientException`.

Langkah 3: Exception Buatan Sendiri

Agar UI cukup menampilkan satu pesan yang ramah, ubah semua jenis error menjadi satu class `ApiException`:

`lib/services/api_exception.dart`

```dart
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  const ApiException(this.message, {this.statusCode});

  // Menerjemahkan status code menjadi pesan yang dimengerti pengguna
  factory ApiException.fromStatus(int code) {
    final pesan = switch (code) {
      400 => 'Data yang dikirim tidak valid.',
      401 => 'Sesi berakhir, silakan login ulang.',
      403 => 'Anda tidak punya akses ke data ini.',
      404 => 'Data tidak ditemukan.',
      422 => 'Validasi gagal, periksa kembali isian.',
      >= 500 => 'Server sedang bermasalah. Coba beberapa saat lagi.',
      _ => 'Terjadi kesalahan (kode $code).',
    };
    return ApiException(pesan, statusCode: code);
  }
```

```
  @override
  String toString() => message; // agar Text('$error') langsung rapi
}
```

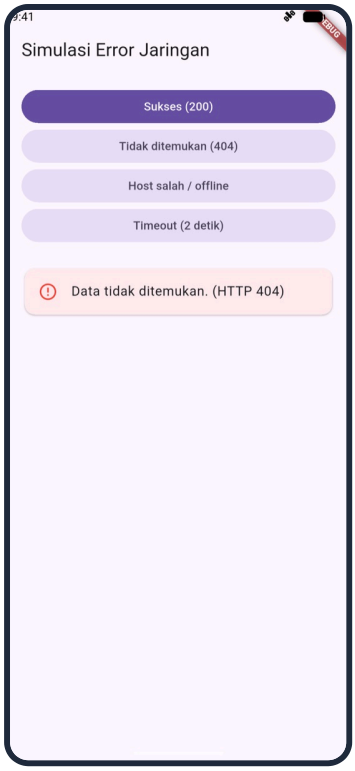
Contoh Lengkap: Simulator Error

Aplikasi ini punya tombol untuk memicu setiap jenis kegagalan: sukses, 404, host tidak ada, dan timeout (endpoint httpbin `/delay/5` butuh 5 detik, sedangkan batas kita 2 detik). Coba juga nyalakan mode pesawat di emulator.

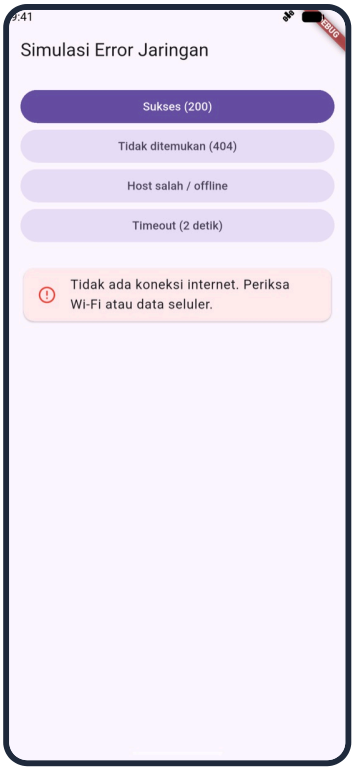
lib/main.dart

```dart
// Pastikan sudah: flutter pub add http, dan membuat lib/services/api_exception.dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

import 'services/api_exception.dart';

/// Satu fungsi GET yang mengubah semua jenis kegagalan menjadi ApiException.
Future<dynamic> getJson(String url, {Duration timeout = const Duration(seconds: 10)}) async {
  try {
    final res = await http.get(Uri.parse(url)).timeout(timeout);
    if (res.statusCode < 200 || res.statusCode >= 300) {
      throw ApiException.fromStatus(res.statusCode);
    }
    return jsonDecode(res.body);
  } on SocketException {
    throw const ApiException('Tidak ada koneksi internet. Periksa Wi-Fi atau data seluler.');
  } on TimeoutException {
    throw const ApiException('Koneksi lambat, server tidak merespons tepat waktu.');
  } on http.ClientException {
    throw const ApiException('Gagal menghubungi server.');
  } on FormatException {
    throw const ApiException('Format data dari server tidak dikenali.');
  }
}

void main() => runApp(const MaterialApp(home: ErrorDemoScreen()));

class ErrorDemoScreen extends StatefulWidget {
  const ErrorDemoScreen({super.key});

  @override
  State<ErrorDemoScreen> createState() => _ErrorDemoScreenState();
}

class _ErrorDemoScreenState extends State<ErrorDemoScreen> {
  String _pesan = 'Pilih skenario.';
  bool _sukses = true;
  bool _loading = false;

  Future<void> _coba(String url, {Duration timeout = const Duration(seconds: 10)}) async {
    setState(() => _loading = true);
```

```
    try {
      final data = await getJson(url, timeout: timeout);
      _sukses = true;
      _pesan = 'Berhasil: ${(data as Map)['title']}';
    } on ApiException catch (e) {
      _sukses = false;
      _pesan = e.statusCode == null ? e.message : '${e.message} (HTTP ${e.statusCode})';
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    const base = 'https://jsonplaceholder.typicode.com';
    return Scaffold(
      appBar: AppBar(title: const Text('Simulasi Error Jaringan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          FilledButton(onPressed: () => _coba('$base/posts/1'), child: const Text('Sukses (200)')),
          FilledButton.tonal(onPressed: () => _coba('$base/posts/9999'), child: const Text('Tidak
ditemukan (404)')),
          FilledButton.tonal(
            onPressed: () => _coba('https://host-tidak-ada.invalid/data'),
            child: const Text('Host salah / offline'),
          ),
          FilledButton.tonal(
            onPressed: () => _coba('https://httpbin.org/delay/5', timeout: const Duration(seconds:
2)),
            child: const Text('Timeout (2 detik)'),
          ),
          const SizedBox(height: 24),
          if (_loading)
            const Center(child: CircularProgressIndicator())
          else
            Card(
              color: _sukses ? Colors.green.shade50 : Colors.red.shade50,
              child: ListTile(
                leading: Icon(
                  _sukses ? Icons.check_circle : Icons.error_outline,
                  color: _sukses ? Colors.green : Colors.red,
                ),
                title: Text(_pesan),
              ),
            ),
        ],
      ),
    );
  }
}
```



Skenario 404 diterjemahkan menjadi pesan ramah



Host tidak ditemukan / offline

Penjelasan: fungsi `getJson` adalah satu-satunya tempat yang tahu soal `SocketException`, `TimeoutException`, dan status code. UI hanya perlu menangkap `ApiException` dan menampilkan `message`. Pola ini akan kita pakai lagi di service/repository layer.

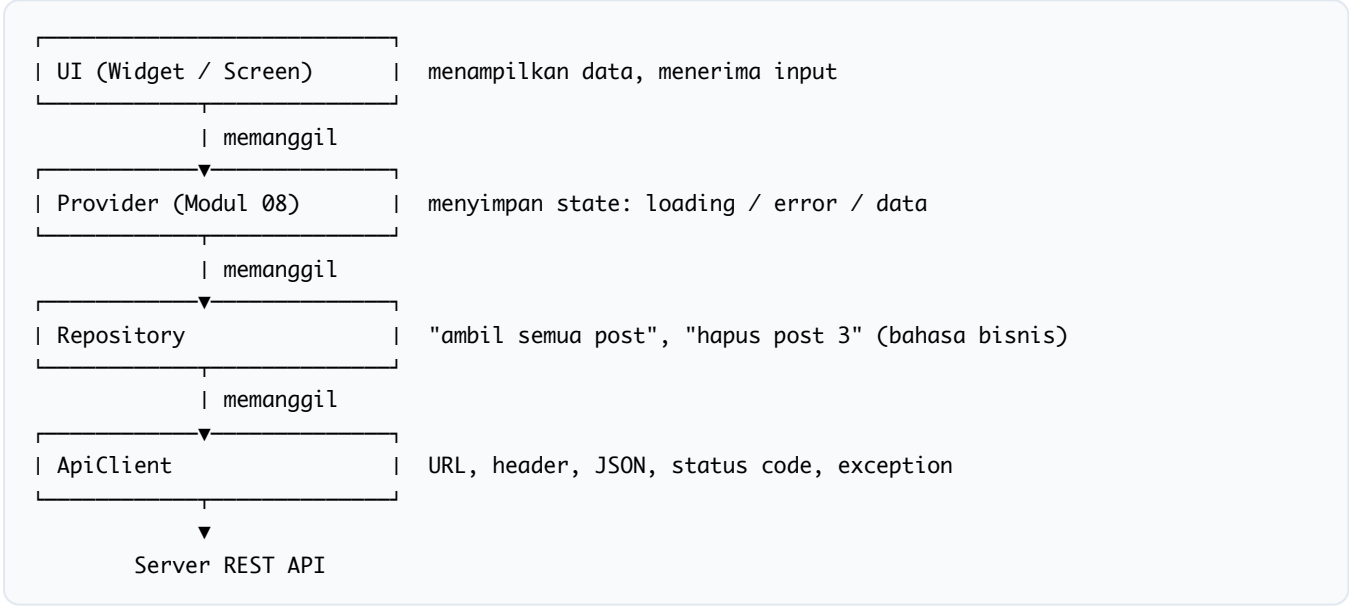
⚠️ Kesalahan Umum

- Hanya memakai `try/catch` tanpa mengecek status code, sehingga response 500 dianggap sukses lalu `jsonDecode` gagal dengan pesan membingungkan.
- Menampilkan `e.toString()` mentah kepada pengguna (`ClientException with SocketException: Failed host lookup...`). Terjemahkan menjadi kalimat yang ramah.
- Tidak memberi `.timeout()`: tanpa batas waktu, spinner bisa berputar sangat lama.
- Menelan error diam-diam: `catch (e) {}` tanpa pesan apa pun membuat pengguna dan programmer sama-sama bingung.

9. Service Layer: ApiClient & Repository

Jika kode `http.get`, `jsonDecode`, dan pengecekan error ditulis langsung di dalam widget, file UI cepat membengkak dan kode yang sama tersebar di mana-mana. Solusinya adalah membagi kode menjadi **lapisan (layer)** yang masing-masing punya satu tugas.

Pembagian Lapisan



Lapisan	Tahu tentang	Tidak boleh tahu tentang
ApiClient	HTTP, header, JSON mentah, status code	Model `Post`, widget
Repository	Endpoint, model, `fromJson` / `toJson`	Widget, `BuildContext`
UI / Provider	Model dan method repository	`http`, `jsonDecode`, URL

1. ApiClient: Satu Pintu untuk Semua Request

File ini memakai `ApiException` dari section sebelumnya (salin ke `lib/core/api_exception.dart`).

`lib/core/api_exception.dart`

```dart
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  const ApiException(this.message, {this.statusCode});

  factory ApiException.fromStatus(int code) {
    final pesan = switch (code) {
      400 => 'Data yang dikirim tidak valid.',
      401 => 'Sesi berakhir, silakan login ulang.',
```

```
      403 => 'Anda tidak punya akses ke data ini.',
      404 => 'Data tidak ditemukan.',
      422 => 'Validasi gagal, periksa kembali isian.',
      >= 500 => 'Server sedang bermasalah. Coba beberapa saat lagi.',
      _ => 'Terjadi kesalahan (kode $code).',
    };
    return ApiException(pesan, statusCode: code);
  }

  @override
  String toString() => message;
}
```

lib/core/api_client.dart

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;

import 'api_exception.dart';

class ApiClient {
  final String baseUrl;
  final http.Client _http;
  final Duration timeout;
  String? token; // diisi setelah login (opsional)

  ApiClient({
    required this.baseUrl,
    http.Client? httpClient,
    this.timeout = const Duration(seconds: 15),
  }) : _http = httpClient ?? http.Client();

  Map<String, String> get _headers => {
        'Accept': 'application/json',
        'Content-Type': 'application/json; charset=UTF-8',
        if (token != null) 'Authorization': 'Bearer $token',
      };

  Uri _uri(String path, [Map<String, String>? query]) =>
      Uri.parse('$baseUrl$path').replace(queryParameters: query);

  Future<dynamic> get(String path, {Map<String, String>? query}) =>
      _send(() => _http.get(_uri(path, query), headers: _headers));

  Future<dynamic> post(String path, Object body) =>
      _send(() => _http.post(_uri(path), headers: _headers, body: jsonEncode(body)));

  Future<dynamic> put(String path, Object body) =>
      _send(() => _http.put(_uri(path), headers: _headers, body: jsonEncode(body)));

  Future<dynamic> patch(String path, Object body) =>
      _send(() => _http.patch(_uri(path), headers: _headers, body: jsonEncode(body)));

  Future<dynamic> delete(String path) =>
      _send(() => _http.delete(_uri(path), headers: _headers));
```

```
  // Semua request lewat sini: timeout, cek status, decode, terjemahkan error
  Future<dynamic> _send(Future<http.Response> Function() request) async {
    try {
      final res = await request().timeout(timeout);
      if (res.statusCode < 200 || res.statusCode >= 300) {
        throw ApiException.fromStatus(res.statusCode);
      }
      if (res.body.isEmpty) return null; // mis. 204 No Content
      return jsonDecode(utf8.decode(res.bodyBytes));
    } on SocketException {
      throw const ApiException('Tidak ada koneksi internet.');
    } on TimeoutException {
      throw const ApiException('Server tidak merespons. Coba lagi.');
    } on http.ClientException {
      throw const ApiException('Gagal menghubungi server.');
    } on FormatException {
      throw const ApiException('Format data dari server tidak valid.');
    }
  }
}
```

Penjelasan: lima method publik hanya berbeda pada fungsi `http` yang dipanggil. Semua logika berat ada di `_send`, sehingga cukup ditulis sekali. `Uri.replace(queryParameters: null)` tidak menambah apa pun jika query kosong.

2. Repository: Bahasa Bisnis

Repository dibuat sebagai `abstract class` (kontrak) plus implementasinya. Keuntungannya: UI bisa diuji dengan repository palsu (*fake*) tanpa internet, dan backend bisa diganti (mis. ke Supabase) tanpa mengubah UI.

`lib/repositories/post_repository.dart`

```
import '../core/api_client.dart';
import '../models/post.dart';

/// Kontrak: apa saja yang bisa dilakukan terhadap data Post.
abstract class PostRepository {
  Future<List<Post>> getAll();
  Future<Post> getById(int id);
  Future<Post> create(Post post);
  Future<Post> update(Post post);
  Future<void> delete(int id);
}

/// Implementasi yang memakai REST API JSONPlaceholder.
class RemotePostRepository implements PostRepository {
  final ApiClient _api;
  RemotePostRepository(this._api);

  @override
  Future<List<Post>> getAll() async {
    final data = await _api.get('/posts', query: {'_limit': '20'}) as List<dynamic>;
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }
```

```dart
  @override
  Future<Post> getById(int id) async {
    final data = await _api.get('/posts/$id') as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<Post> create(Post post) async {
    final data = await _api.post('/posts', post.toJson()) as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<Post> update(Post post) async {
    final data = await _api.put('/posts/${post.id}', post.toJson()) as Map<String, dynamic>;
    return Post.fromJson(data);
  }

  @override
  Future<void> delete(int id) => _api.delete('/posts/$id');
}

/// Implementasi palsu di memori: untuk testing atau saat server belum siap.
class FakePostRepository implements PostRepository {
  final List<Post> _data = [
    const Post(id: 1, userId: 1, title: 'Post palsu pertama', body: 'Tanpa internet'),
    const Post(id: 2, userId: 1, title: 'Post palsu kedua', body: 'Cocok untuk testing'),
  ];

  @override
  Future<List<Post>> getAll() async {
    await Future.delayed(const Duration(milliseconds: 500)); // simulasi jaringan
    return List.of(_data);
  }

  @override
  Future<Post> getById(int id) async => _data.firstWhere((p) => p.id == id);

  @override
  Future<Post> create(Post post) async {
    final baru = post.copyWith(id: _data.length + 1);
    _data.add(baru);
    return baru;
  }

  @override
  Future<Post> update(Post post) async {
    final i = _data.indexWhere((p) => p.id == post.id);
    _data[i] = post;
    return post;
  }

  @override
  Future<void> delete(int id) async => _data.removeWhere((p) => p.id == id);
}
```

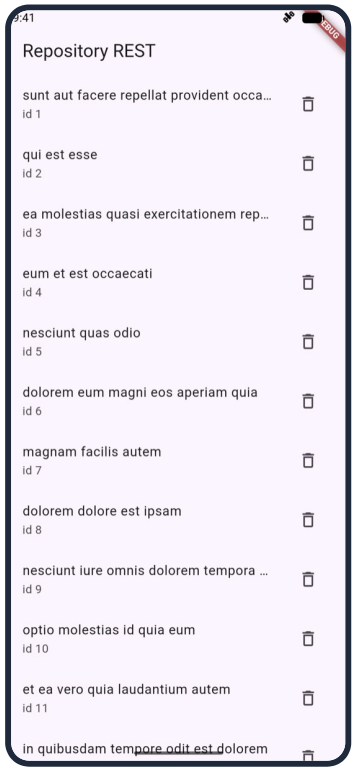
3. Memakai Repository di UI

Perhatikan: file UI berikut **tidak** mengimpor `http` maupun `dart:convert`. Ubah variabel `pakaiDataPalsu` menjadi `true` untuk mencoba tanpa internet.

`lib/main.dart`

```dart
// Butuh: lib/models/post.dart, lib/core/api_exception.dart,
// lib/core/api_client.dart, lib/repositories/post_repository.dart
import 'package:flutter/material.dart';

import 'core/api_client.dart';
import 'models/post.dart';
import 'repositories/post_repository.dart';

const pakaiDataPalsu = false;

void main() {
  final PostRepository repo = pakaiDataPalsu
      ? FakePostRepository()
      : RemotePostRepository(ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com'));
  runApp(MaterialApp(home: PostScreen(repository: repo)));
}

class PostScreen extends StatefulWidget {
  final PostRepository repository;
  const PostScreen({super.key, required this.repository});

  @override
  State<PostScreen> createState() => _PostScreenState();
}

class _PostScreenState extends State<PostScreen> {
  List<Post> _posts = [];
  String? _error;
  bool _loading = true;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    try {
      final posts = await widget.repository.getAll();
      if (mounted) setState(() => _posts = posts);
    } catch (e) {
      if (mounted) setState(() => _error = '$e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _hapus(Post post) async {
    try {
      await widget.repository.delete(post.id!);
      setState(() => _posts.remove(post));
```

```
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('$e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pakaiDataPalsu ? 'Repository Palsu' : 'Repository REST')),
      body: _loading
          ? const Center(child: CircularProgressIndicator())
          : _error != null
              ? Center(child: Text(_error!))
              : ListView(
                  children: [
                    for (final post in _posts)
                      ListTile(
                        title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                        subtitle: Text('id ${post.id}'),
                        trailing: IconButton(
                          icon: const Icon(Icons.delete_outline),
                          onPressed: () => _hapus(post),
                        ),
                      ),
                  ],
                ),
    );
  }
}
```



UI memakai RemotePostRepository tanpa kode HTTP

💡 **Kenapa Repot Membuat Lapisan?**

- **Mudah diganti**: pindah dari REST ke Supabase (Modul 12) cukup membuat `SupabasePostRepository` baru.
- **Mudah diuji**: Modul 15 (Testing) memakai `FakePostRepository` agar tes tidak bergantung internet.
- **Mudah dibaca**: `repository.delete(3)` jauh lebih jelas daripada 10 baris kode HTTP.

⚠️ **Kesalahan Umum**

- Mengimpor `package:flutter/material.dart` di repository atau ApiClient. Lapisan data tidak boleh bergantung pada UI.
- Membuat `http.Client()` baru di setiap method tanpa ditutup. Buat sekali dan gunakan ulang.
- Menangkap error di repository lalu mengembalikan list kosong. UI jadi tidak tahu bahwa terjadi error; biarkan exception naik ke Provider/UI.

10. Menangani 3 Kondisi UI dengan FutureBuilder

Di section GET kita menulis tiga variabel state (`_loading`, `_error`, `_posts`) secara manual. Widget **`FutureBuilder`** melakukan pekerjaan itu untuk kita: ia mengamati sebuah `Future` dan otomatis membangun ulang UI sesuai statusnya.

Konsep: `AsyncSnapshot`

Fungsi `builder` menerima `snapshot` yang berisi keadaan Future saat ini:

Properti	Arti	Tampilkan
`snapshot.connectionState == ConnectionState.waiting`	Future sedang berjalan	Loading (spinner)
`snapshot.hasError` / `snapshot.error`	Future selesai dengan exception	Pesan error + tombol coba lagi
`snapshot.hasData` / `snapshot.data`	Future selesai dengan nilai	Data (list, detail)
`ConnectionState.done`	Future sudah selesai (sukses atau error)	-

Contoh Kecil

```
import 'package:flutter/material.dart';

class JamServer extends StatefulWidget {
  const JamServer({super.key});

  @override
  State<JamServer> createState() => _JamServerState();
}

class _JamServerState extends State<JamServer> {
  // Future disimpan di field, dibuat sekali di initState
  late final Future<String> _future;

  @override
  void initState() {
    super.initState();
    _future = Future.delayed(const Duration(seconds: 2), () => 'Data siap!');
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<String>(
      future: _future,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
```

```
          return const CircularProgressIndicator(); // 1. loading
        }
        if (snapshot.hasError) {
          return Text('Error: ${snapshot.error}'); // 2. error
        }
        return Text(snapshot.data ?? '-'); // 3. data
      },
    );
  }
}
```

Urutan pengecekan selalu sama: **loading → error → data**. Jika urutannya terbalik, layar bisa menampilkan data lama ketika sedang memuat ulang.

Contoh Lengkap: Daftar Mahasiswa dari API

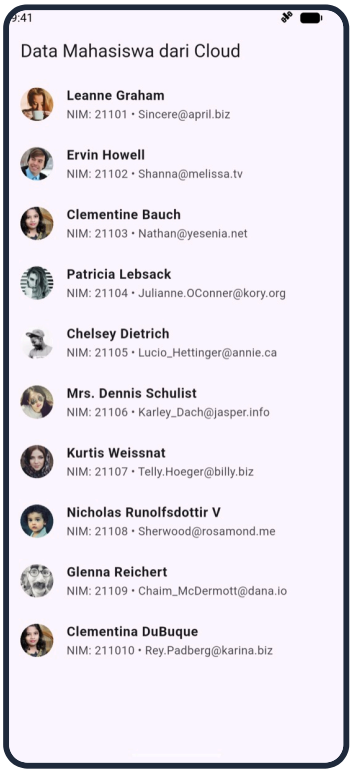
Contoh ini memakai `MahasiswaModel` (section 3) dan `MahasiswaApiService` (section 5). Tombol *Coba Lagi* bekerja dengan **mengganti Future** di dalam `setState`, sehingga FutureBuilder kembali ke status waiting lalu memuat ulang.

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add http, serta membuat file model & service di atas.
import 'package:flutter/material.dart';

import 'models/mahasiswa_model.dart';
import 'services/mahasiswa_api_service.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMahasiswaApiScreen(),
    );
  }
}

class DaftarMahasiswaApiScreen extends StatefulWidget {
  const DaftarMahasiswaApiScreen({super.key});

  @override
  State<DaftarMahasiswaApiScreen> createState() => _DaftarMahasiswaApiScreenState();
}

class _DaftarMahasiswaApiScreenState extends State<DaftarMahasiswaApiScreen> {
  late Future<List<MahasiswaModel>> _futureData;
  final MahasiswaApiService _apiService = MahasiswaApiService();
```

```dart
  @override
  void initState() {
    super.initState();
    // Simpan Future di initState agar tidak terpanggil berulang saat rebuild!
    _futureData = _apiService.fetchMahasiswa();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Data Mahasiswa dari Cloud")),
      body: FutureBuilder<List<MahasiswaModel>>(
        future: _futureData,
        builder: (context, snapshot) {
          // KONDISI 1: Sedang Menunggu Respon Jaringan (Loading)
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  CircularProgressIndicator(),
                  SizedBox(height: 12),
                  Text("Mengunduh data dari server kampus..."),
                ],
              ),
            );
          }

          // KONDISI 2: Terjadi Kesalahan Jaringan / Server Error
          if (snapshot.hasError) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Icon(Icons.cloud_off, size: 64, color: Colors.red),
                    const SizedBox(height: 12),
                    Text("Terjadi Error: ${snapshot.error}"),
                    const SizedBox(height: 16),
                    ElevatedButton(
                      onPressed: () => setState(() => _futureData = _apiService.fetchMahasiswa()),
                      child: const Text("Coba Lagi"),
                    ),
                  ],
                ),
              ),
            );
          }

          // KONDISI 3: Data Berhasil Diterima
          final listMhs = snapshot.data ?? [];
          if (listMhs.isEmpty) {
            return const Center(child: Text("Tidak ada data mahasiswa ditemukan."));
          }

          return ListView.builder(
            itemCount: listMhs.length,
            itemBuilder: (context, index) {
```

```
              final mhs = listMhs[index];
              return ListTile(
                leading: CircleAvatar(
                  // foregroundImage + child: jika foto gagal dimuat, huruf awal nama tetap tampil
                  foregroundImage: NetworkImage(mhs.avatarUrl),
                  onForegroundImageError: (error, stackTrace) {},
                  child: Text(mhs.nama.isNotEmpty ? mhs.nama[0] : '?'),
                ),
                title: Text(mhs.nama, style: const TextStyle(fontWeight: FontWeight.bold)),
                subtitle: Text("NIM: ${mhs.nim} • ${mhs.email}"),
              );
            },
          );
        },
      ),
    );
  }
}
```



Kondisi 3 FutureBuilder: data berhasil dimuat

Penjelasan:

- `late Future<...> _futureData` diisi di `initState`. Jika fungsi API dipanggil langsung di parameter `future:`, setiap rebuild (mis. keyboard muncul) akan mengirim request baru.
- Kondisi 3 juga menangani list kosong. Data kosong bukan error, tetapi tetap perlu pesan agar layar tidak tampak "rusak".
- `foregroundImage` + `child` pada `CircleAvatar` membuat huruf awal tetap tampil jika foto gagal dimuat.

⚠️ **Kesalahan Umum**

- `future: _apiService.fetchMahasiswa()` langsung di `build()`: request berulang tanpa henti.
- Hanya memeriksa `snapshot.hasData` tanpa `hasError`, sehingga saat error spinner berputar selamanya.
- Memakai `snapshot.data!` sebelum memastikan statusnya, menyebabkan *Null check operator used on a null value*.
- Data tidak muncul sama sekali di APK release: cek izin INTERNET (section 4).

11. Pull-to-Refresh dengan RefreshIndicator

Pengguna terbiasa **menarik layar ke bawah** untuk memuat ulang data (Instagram, Gmail). Di Flutter, gestur ini disediakan widget `RefreshIndicator`.

Konsep

- `RefreshIndicator` membungkus widget yang bisa di-scroll (`ListView`, `CustomScrollView`).
- Parameter `onRefresh` wajib mengembalikan `Future`. Spinner tetap tampil sampai Future itu selesai.
- Child harus bisa di-scroll **meskipun isinya sedikit**. Gunakan `physics: const AlwaysScrollableScrollPhysics()`.

Contoh Kecil

```dart
import 'package:flutter/material.dart';

class AngkaAcak extends StatefulWidget {
  const AngkaAcak({super.key});

  @override
  State<AngkaAcak> createState() => _AngkaAcakState();
}

class _AngkaAcakState extends State<AngkaAcak> {
  int _versi = 1;

  Future<void> _refresh() async {
    await Future.delayed(const Duration(seconds: 1)); // pura-pura memanggil API
    setState(() => _versi++);
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      onRefresh: _refresh,
      child: ListView(
        physics: const AlwaysScrollableScrollPhysics(),
        children: [ListTile(title: Text('Data versi $_versi'))],
      ),
    );
  }
}
```

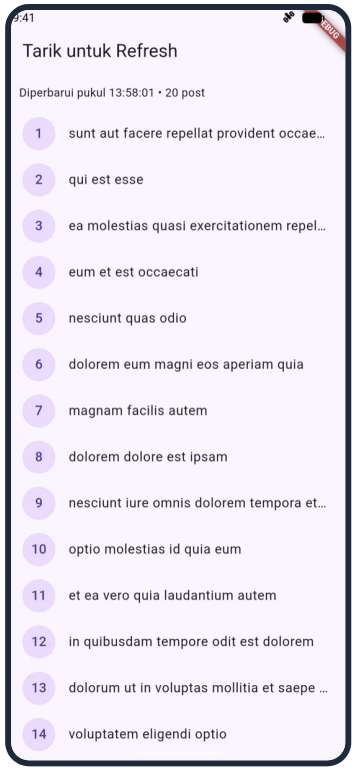
Contoh Lengkap: FutureBuilder + Pull-to-Refresh

Memakai `Post` (section 3) dan `PostService` (section 5). Kuncinya: `_refresh()` membuat Future baru, menyimpannya lewat `setState`, lalu **menunggu** Future yang sama agar spinner RefreshIndicator berhenti tepat ketika data tiba.

lib/main.dart

```dart
// Butuh: flutter pub add http, lib/models/post.dart, lib/services/post_service.dart
import 'package:flutter/material.dart';

import 'models/post.dart';
import 'services/post_service.dart';

void main() => runApp(const MaterialApp(home: RefreshPostScreen()));

class RefreshPostScreen extends StatefulWidget {
  const RefreshPostScreen({super.key});

  @override
  State<RefreshPostScreen> createState() => _RefreshPostScreenState();
}

class _RefreshPostScreenState extends State<RefreshPostScreen> {
  final _service = PostService();
  late Future<List<Post>> _future;
  DateTime _terakhir = DateTime.now();

  @override
  void initState() {
    super.initState();
    _future = _service.fetchPosts();
  }

  Future<void> _refresh() async {
    final future = _service.fetchPosts();
    setState(() {
      _future = future;
      _terakhir = DateTime.now();
    });
    try {
      await future; // spinner RefreshIndicator berhenti setelah ini
    } catch (_) {
      // error sudah ditampilkan oleh FutureBuilder, tidak perlu dilempar lagi
    }
  }

  @override
  Widget build(BuildContext context) {
    final jam = '${_terakhir.hour.toString().padLeft(2, '0')}:'
        '${_terakhir.minute.toString().padLeft(2, '0')}:'
        '${_terakhir.second.toString().padLeft(2, '0')}';
    return Scaffold(
      appBar: AppBar(title: const Text('Tarik untuk Refresh')),
      body: RefreshIndicator(
        onRefresh: _refresh,
        child: FutureBuilder<List<Post>>(
          future: _future,
          builder: (context, snapshot) {
            // Loading pertama kali: belum ada data sama sekali
            if (snapshot.connectionState == ConnectionState.waiting && !snapshot.hasData) {
              return const Center(child: CircularProgressIndicator());
            }
            // Error: tetap pakai ListView agar layar masih bisa ditarik
            if (snapshot.hasError) {
```

```
                return ListView(
                  physics: const AlwaysScrollableScrollPhysics(),
                  children: [
                    const SizedBox(height: 120),
                    const Icon(Icons.cloud_off, size: 64),
                    Center(child: Text('${snapshot.error}')),
                    const Center(child: Text('Tarik ke bawah untuk mencoba lagi')),
                  ],
                );
              }
              final posts = snapshot.data ?? [];
              return ListView.builder(
                physics: const AlwaysScrollableScrollPhysics(),
                itemCount: posts.length + 1,
                itemBuilder: (context, index) {
                  if (index == 0) {
                    return Padding(
                      padding: const EdgeInsets.all(12),
                      child: Text('Diperbarui pukul $jam • ${posts.length} post'),
                    );
                  }
                  final post = posts[index - 1];
                  return ListTile(
                    leading: CircleAvatar(child: Text('${post.id}')),
                    title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                  );
                },
              );
            },
          ),
        ),
      );
    }
}
```



Baris pertama menunjukkan waktu pembaruan terakhir

Penjelasan:

- Kondisi loading memakai `&& !snapshot.hasData`. Saat refresh, Future baru sedang berjalan tetapi kita tidak ingin layar berkedip menjadi spinner besar karena RefreshIndicator sudah menampilkan spinner kecil di atas.
- Tampilan error juga dibuat sebagai `ListView` dengan `AlwaysScrollableScrollPhysics`. Jika memakai `Center` biasa, layar error tidak bisa ditarik untuk mencoba lagi.
- Baris pertama list menampilkan jam pembaruan sehingga efek refresh terlihat jelas.

> ⚠️ **Kesalahan Umum**
>
> - `onRefresh: () { _load(); }` tanpa `async` / `return`: spinner langsung hilang sebelum data datang. Pastikan fungsi mengembalikan Future yang ditunggu.
> - Membungkus `Column` atau `Center` dengan RefreshIndicator: tidak bisa ditarik karena bukan widget scroll.
> - Lupa `AlwaysScrollableScrollPhysics` ketika item sedikit, sehingga gestur tarik tidak berfungsi.

12. Integrasi REST API dengan Provider

`FutureBuilder` cocok untuk satu layar sederhana. Namun jika data yang sama dipakai banyak layar (daftar, detail, form) dan bisa berubah (tambah/hapus), simpan data di **Provider** (Modul 08). Provider menyimpan tiga hal: **status** (loading/error/sukses), **pesan error**, dan **data**.

Konsep: State Hasil Request

```
enum LoadStatus { initial, loading, success, error }
```

Dengan `enum`, UI cukup memakai `switch` pada status. Ini lebih jelas daripada beberapa variabel `bool` yang bisa saling bertentangan (misalnya `isLoading` dan `hasError` sama-sama `true`).

Langkah 1: Provider

Provider memakai `PostRepository` dari section 9, sehingga tidak perlu tahu soal HTTP.

`lib/providers/post_provider.dart`

```
import 'package:flutter/foundation.dart';

import '../models/post.dart';
import '../repositories/post_repository.dart';

enum LoadStatus { initial, loading, success, error }

class PostProvider extends ChangeNotifier {
  final PostRepository _repo;
  PostProvider(this._repo);

  List<Post> _posts = [];
  LoadStatus _status = LoadStatus.initial;
  String? _errorMessage;

  List<Post> get posts => List.unmodifiable(_posts); // UI tidak bisa mengubah langsung
  LoadStatus get status => _status;
  String? get errorMessage => _errorMessage;

  Future<void> fetchPosts() async {
    _status = LoadStatus.loading;
    _errorMessage = null;
    notifyListeners(); // UI menampilkan loading

    try {
      _posts = await _repo.getAll();
      _status = LoadStatus.success;
    } catch (e) {
      _errorMessage = e.toString();
      _status = LoadStatus.error;
    }
    notifyListeners(); // UI menampilkan data atau error
  }

  /// Optimistic delete: hapus dulu dari layar, kembalikan jika server gagal.
```

```
  Future<bool> deletePost(Post post) async {
    final index = _posts.indexOf(post);
    _posts.removeAt(index);
    notifyListeners();
    try {
      await _repo.delete(post.id!);
      return true;
    } catch (e) {
      _posts.insert(index, post); // rollback
      notifyListeners();
      return false;
    }
  }
}
```

Penjelasan: `notifyListeners()` dipanggil dua kali di `fetchPosts`: sekali sebelum request (agar spinner muncul), sekali setelah selesai. Pada `deletePost` kita memakai pola *optimistic update*: item langsung hilang sehingga aplikasi terasa cepat, lalu dikembalikan jika server menolak.

Langkah 2: Daftarkan Provider dan Panggil Data Pertama

Pasang package: `flutter pub add provider`. Operator `..fetchPosts()` (cascade) memanggil fetch tepat saat provider dibuat.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'core/api_client.dart';
import 'providers/post_provider.dart';
import 'repositories/post_repository.dart';

Widget buatApp(Widget home) {
  return ChangeNotifierProvider(
    create: (_) => PostProvider(
      RemotePostRepository(ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com')),
    )..fetchPosts(),
    child: MaterialApp(home: home),
  );
}
```

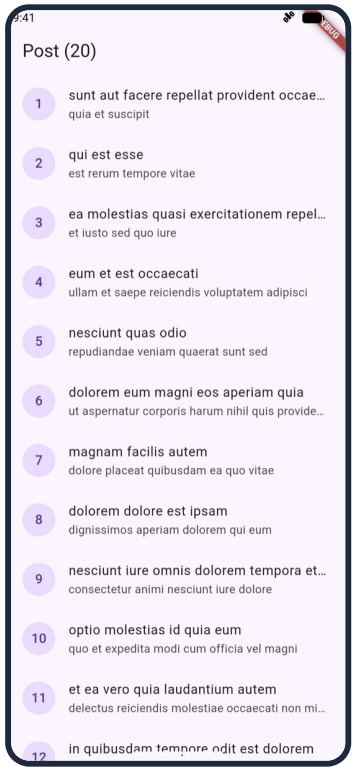
Langkah 3: Contoh Lengkap

`lib/main.dart`

```
// Butuh: flutter pub add http provider
// File: models/post.dart, core/api_exception.dart, core/api_client.dart,
//       repositories/post_repository.dart, providers/post_provider.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'core/api_client.dart';
import 'providers/post_provider.dart';
import 'repositories/post_repository.dart';

void main() {
```

```
  final repo = RemotePostRepository(
    ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com'),
  );
  runApp(
    ChangeNotifierProvider(
      create: (_) => PostProvider(repo)..fetchPosts(),
      child: const MaterialApp(home: PostProviderScreen()),
    ),
  );
}

class PostProviderScreen extends StatelessWidget {
  const PostProviderScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<PostProvider>();

    final Widget body = switch (provider.status) {
      LoadStatus.initial || LoadStatus.loading =>
        const Center(child: CircularProgressIndicator()),
      LoadStatus.error => Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(provider.errorMessage ?? 'Terjadi kesalahan'),
              const SizedBox(height: 12),
              FilledButton(
                onPressed: () => context.read<PostProvider>().fetchPosts(),
                child: const Text('Coba Lagi'),
              ),
            ],
          ),
        ),
      LoadStatus.success => RefreshIndicator(
          onRefresh: () => context.read<PostProvider>().fetchPosts(),
          child: ListView.builder(
            itemCount: provider.posts.length,
            itemBuilder: (context, i) {
              final post = provider.posts[i];
              return Dismissible(
                key: ValueKey(post.id),
                direction: DismissDirection.endToStart,
                background: Container(
                  color: Colors.red,
                  alignment: Alignment.centerRight,
                  padding: const EdgeInsets.only(right: 20),
                  child: const Icon(Icons.delete, color: Colors.white),
                ),
                onDismissed: (_) async {
                  final messenger = ScaffoldMessenger.of(context);
                  final ok = await context.read<PostProvider>().deletePost(post);
                  messenger.showSnackBar(SnackBar(
                    content: Text(ok ? 'Post ${post.id} dihapus' : 'Gagal menghapus, data
dikembalikan'),
                  ));
                },
                child: ListTile(
                  leading: CircleAvatar(child: Text('${post.id}')),
```

```
                  title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                  subtitle: Text(post.body, maxLines: 1, overflow: TextOverflow.ellipsis),
                ),
              );
            },
          ),
        ),
      };

      return Scaffold(
        appBar: AppBar(title: Text('Post (${provider.posts.length})')),
        body: body,
      );
    }
  }
```



Status success: daftar post dari PostProvider

Penjelasan:

- `context.watch` di `build` membuat layar ikut berubah setiap `notifyListeners()`. Di dalam callback tombol, gunakan `context.read`.
- `switch` expression memaksa kita menangani **semua** nilai enum; jika ada status baru yang lupa ditangani, analyzer langsung memberi error.
- `ScaffoldMessenger.of(context)` disimpan ke variabel **sebelum** `await`, sehingga aman dipakai setelahnya.
- Geser item ke kiri untuk menghapus. Karena JSONPlaceholder selalu menjawab sukses, rollback hanya terlihat jika internet dimatikan.

⚠️ **Kesalahan Umum**

- Memanggil `fetchPosts()` di dalam `build()`: memicu loop rebuild tanpa akhir. Panggil di `create` provider atau `initState`.
- Lupa `notifyListeners()` setelah mengubah data, sehingga UI tidak berubah.
- Mengembalikan list internal secara langsung (`get posts => _posts`) lalu UI mengubahnya tanpa `notifyListeners`.
- Memakai `context.watch` di dalam `onPressed`: error, gunakan `context.read`.

13. Studi Kasus: Aplikasi CRUD Posts Multi-File

Saatnya menggabungkan semua yang sudah dipelajari menjadi satu aplikasi utuh: **Aplikasi Posts** dengan fitur daftar, detail, tambah, edit, dan hapus, memakai ApiClient, Repository, dan Provider.

Struktur Folder

```
lib/
├── main.dart
├── core/
|   ├── api_exception.dart        (dari section 9, tanpa perubahan)
|   └── api_client.dart           (dari section 9, tanpa perubahan)
├── models/
|   └── post.dart                 (dari section 3, tanpa perubahan)
├── repositories/
|   └── post_repository.dart      (dari section 9, tanpa perubahan)
├── providers/
|   └── post_crud_provider.dart   (BARU)
└── screens/
    ├── post_list_screen.dart     (BARU)
    ├── post_detail_screen.dart   (BARU)
    └── post_form_screen.dart     (BARU: dipakai untuk tambah dan edit)
```

Persiapan: `flutter pub add http provider`, lalu salin empat file "dari section" di atas. Kita hanya menulis file yang BARU.

> ℹ️ **Menyiasati JSONPlaceholder**
>
> Karena server palsu selalu memberi `id: 101` untuk post baru dan tidak menyimpan perubahan, provider kita (1) memberi id unik lokal untuk post baru, dan (2) tidak mengirim PUT untuk post buatan sendiri (id > 100) karena server akan membalas 500. Di backend sungguhan, dua siasat ini tidak diperlukan.

1. Provider CRUD

📄 **`lib/providers/post_crud_provider.dart`**

```dart
import 'package:flutter/foundation.dart';

import '../models/post.dart';
import '../repositories/post_repository.dart';

enum LoadStatus { initial, loading, success, error }

class PostCrudProvider extends ChangeNotifier {
  final PostRepository _repo;
```

```
  PostCrudProvider(this._repo);

  List<Post> _posts = [];
  LoadStatus _status = LoadStatus.initial;
  String? _error;
  bool _saving = false; // true saat POST/PUT/DELETE berjalan

  List<Post> get posts => List.unmodifiable(_posts);
  LoadStatus get status => _status;
  String? get error => _error;
  bool get saving => _saving;

  Post? findById(int id) {
    for (final p in _posts) {
      if (p.id == id) return p;
    }
    return null;
  }

  // READ
  Future<void> load() async {
    _status = LoadStatus.loading;
    _error = null;
    notifyListeners();
    try {
      _posts = await _repo.getAll();
      _status = LoadStatus.success;
    } catch (e) {
      _error = '$e';
      _status = LoadStatus.error;
    }
    notifyListeners();
  }

  // CREATE
  Future<void> add(String title, String body) async {
    await _runSaving(() async {
      final created = await _repo.create(Post(userId: 1, title: title, body: body));
      // JSONPlaceholder selalu memberi id 101, jadi buat id unik lokal
      final nextId = _posts.fold<int>(100, (max, p) => p.id! > max ? p.id! : max) + 1;
      _posts.insert(0, created.copyWith(id: nextId));
    });
  }

  // UPDATE
  Future<void> edit(Post post) async {
    await _runSaving(() async {
      // Post buatan sendiri (id > 100) tidak dikenal server palsu, cukup ubah lokal
      final saved = post.id! > 100 ? post : await _repo.update(post);
      final i = _posts.indexWhere((p) => p.id == post.id);
      if (i != -1) _posts[i] = saved;
    });
  }

  // DELETE
  Future<void> remove(int id) async {
    await _runSaving(() async {
      if (id <= 100) await _repo.delete(id);
      _posts.removeWhere((p) => p.id == id);
```

```
    });
  }

  // Pembantu: set flag saving, jalankan aksi, lempar ulang error ke UI
  Future<void> _runSaving(Future<void> Function() action) async {
    _saving = true;
    notifyListeners();
    try {
      await action();
    } finally {
      _saving = false;
      notifyListeners();
    }
  }
}
```

Penjelasan: method `add`, `edit`, dan `remove` tidak menangkap error. Error dibiarkan naik ke layar form/detail agar layar itu bisa menampilkan SnackBar dan **tidak menutup** form jika gagal. `try/finally` di `_runSaving` memastikan flag `saving` kembali `false` walaupun terjadi error.

2. Layar Daftar

`lib/screens/post_list_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/post_crud_provider.dart';
import 'post_detail_screen.dart';
import 'post_form_screen.dart';

class PostListScreen extends StatelessWidget {
  const PostListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final prov = context.watch<PostCrudProvider>();

    Widget body;
    switch (prov.status) {
      case LoadStatus.initial:
      case LoadStatus.loading:
        body = const Center(child: CircularProgressIndicator());
      case LoadStatus.error:
        body = Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(Icons.cloud_off, size: 56),
              const SizedBox(height: 8),
              Text(prov.error ?? 'Terjadi kesalahan'),
              TextButton(onPressed: prov.load, child: const Text('Coba Lagi')),
            ],
          ),
        );
      case LoadStatus.success:
        body = prov.posts.isEmpty
```

```
            ? const Center(child: Text('Belum ada post.'))
            : RefreshIndicator(
                onRefresh: prov.load,
                child: ListView.separated(
                  physics: const AlwaysScrollableScrollPhysics(),
                  itemCount: prov.posts.length,
                  separatorBuilder: (_, _) => const Divider(height: 1),
                  itemBuilder: (context, i) {
                    final post = prov.posts[i];
                    return ListTile(
                      leading: CircleAvatar(child: Text('${post.id}')),
                      title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                      subtitle: Text(post.body, maxLines: 2, overflow: TextOverflow.ellipsis),
                      onTap: () => Navigator.push(
                        context,
                        MaterialPageRoute(builder: (_) => PostDetailScreen(postId: post.id!)),
                      ),
                    );
                  },
                ),
              );
    }

    return Scaffold(
      appBar: AppBar(title: const Text('Aplikasi Posts')),
      body: body,
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const PostFormScreen()),
        ),
        icon: const Icon(Icons.add),
        label: const Text('Post Baru'),
      ),
    );
  }
}
```

3. Layar Detail (dengan Edit & Hapus)

lib/screens/post_detail_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/post_crud_provider.dart';
import 'post_form_screen.dart';

class PostDetailScreen extends StatelessWidget {
  final int postId;
  const PostDetailScreen({super.key, required this.postId});

  Future<void> _konfirmasiHapus(BuildContext context) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Hapus post?'),
        content: const Text('Post yang dihapus tidak bisa dikembalikan.'),
```

```
        actions: [
          TextButton(onPressed: () => Navigator.pop(ctx, false), child: const Text('Batal')),
          FilledButton(onPressed: () => Navigator.pop(ctx, true), child: const Text('Hapus')),
        ],
      ),
    );
    if (yakin != true || !context.mounted) return;

    final messenger = ScaffoldMessenger.of(context);
    final navigator = Navigator.of(context);
    try {
      await context.read<PostCrudProvider>().remove(postId);
      navigator.pop(); // kembali ke daftar
      messenger.showSnackBar(const SnackBar(content: Text('Post dihapus')));
    } catch (e) {
      messenger.showSnackBar(SnackBar(content: Text('Gagal menghapus: $e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    final prov = context.watch<PostCrudProvider>();
    final post = prov.findById(postId);

    // Post bisa saja sudah terhapus ketika layar ini masih terbuka
    if (post == null) {
      return Scaffold(appBar: AppBar(), body: const Center(child: Text('Post tidak ditemukan')));
    }

    return Scaffold(
      appBar: AppBar(
        title: Text('Post #${post.id}'),
        actions: [
          IconButton(
            tooltip: 'Edit',
            icon: const Icon(Icons.edit),
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => PostFormScreen(post: post)),
            ),
          ),
          IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: prov.saving ? null : () => _konfirmasiHapus(context),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          Text(post.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 4),
          Text('Penulis: user ${post.userId}', style: Theme.of(context).textTheme.labelMedium),
          const Divider(height: 32),
          Text(post.body, style: Theme.of(context).textTheme.bodyLarge),
        ],
      ),
    );
```

```
  }
}
```

4. Satu Form untuk Tambah dan Edit

Jika parameter `post` bernilai `null`, form berada dalam mode **tambah**; jika diisi, mode **edit** dan field terisi data lama.

`lib/screens/post_form_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/post.dart';
import '../providers/post_crud_provider.dart';

class PostFormScreen extends StatefulWidget {
  final Post? post; // null = tambah, ada isi = edit
  const PostFormScreen({super.key, this.post});

  @override
  State<PostFormScreen> createState() => _PostFormScreenState();
}

class _PostFormScreenState extends State<PostFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleCtrl;
  late final TextEditingController _bodyCtrl;

  bool get _isEdit => widget.post != null;

  @override
  void initState() {
    super.initState();
    _titleCtrl = TextEditingController(text: widget.post?.title ?? '');
    _bodyCtrl = TextEditingController(text: widget.post?.body ?? '');
  }

  @override
  void dispose() {
    _titleCtrl.dispose();
    _bodyCtrl.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final prov = context.read<PostCrudProvider>();
    final title = _titleCtrl.text.trim();
    final body = _bodyCtrl.text.trim();

    try {
      if (_isEdit) {
        await prov.edit(widget.post!.copyWith(title: title, body: body));
      } else {
        await prov.add(title, body);
      }
      if (!mounted) return;
```

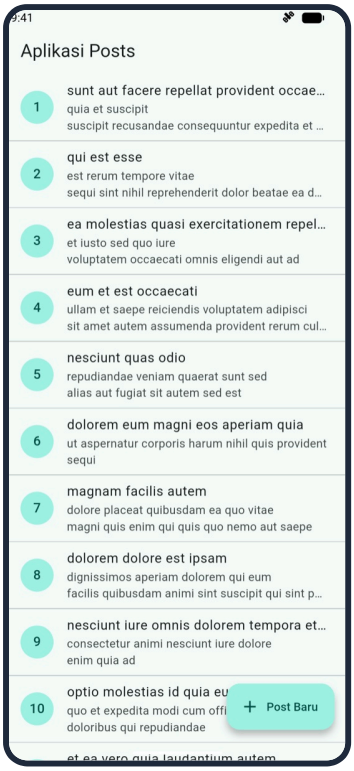
```
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(_isEdit ? 'Perubahan disimpan' : 'Post ditambahkan')),
      );
      Navigator.pop(context);
    } catch (e) {
      if (!mounted) return;
      // Form TIDAK ditutup agar isian pengguna tidak hilang
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Gagal: $e')));
    }
  }

  @override
  Widget build(BuildContext context) {
    final saving = context.watch<PostCrudProvider>().saving;
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Post' : 'Post Baru')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _titleCtrl,
              decoration: const InputDecoration(labelText: 'Judul', border: OutlineInputBorder()),
              validator: (v) => (v == null || v.trim().length < 3) ? 'Judul minimal 3 karakter' :
null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _bodyCtrl,
              maxLines: 5,
              decoration: const InputDecoration(labelText: 'Isi', border: OutlineInputBorder()),
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Isi tidak boleh kosong' : null,
            ),
            const SizedBox(height: 20),
            FilledButton.icon(
              onPressed: saving ? null : _simpan,
              icon: saving
                  ? const SizedBox(width: 18, height: 18, child:
CircularProgressIndicator(strokeWidth: 2))
                  : const Icon(Icons.save),
              label: Text(saving ? 'Menyimpan...' : 'Simpan'),
            ),
          ],
        ),
      ),
    );
  }
}
```

5. `main.dart`

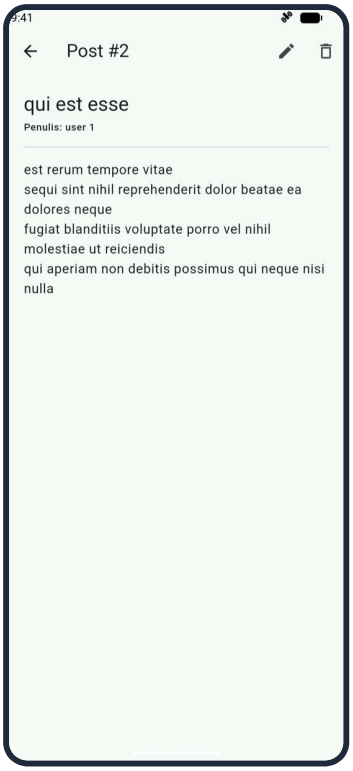
`lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'core/api_client.dart';
```

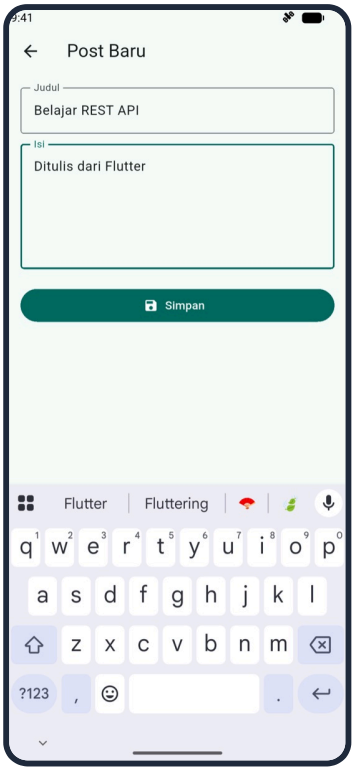
```dart
import 'providers/post_crud_provider.dart';
import 'repositories/post_repository.dart';
import 'screens/post_list_screen.dart';

void main() {
  final api = ApiClient(baseUrl: 'https://jsonplaceholder.typicode.com');
  runApp(
    ChangeNotifierProvider(
      create: (_) => PostCrudProvider(RemotePostRepository(api))..load(),
      child: const PostsApp(),
    ),
  );
}

class PostsApp extends StatelessWidget {
  const PostsApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Aplikasi Posts',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const PostListScreen(),
    );
  }
}
```



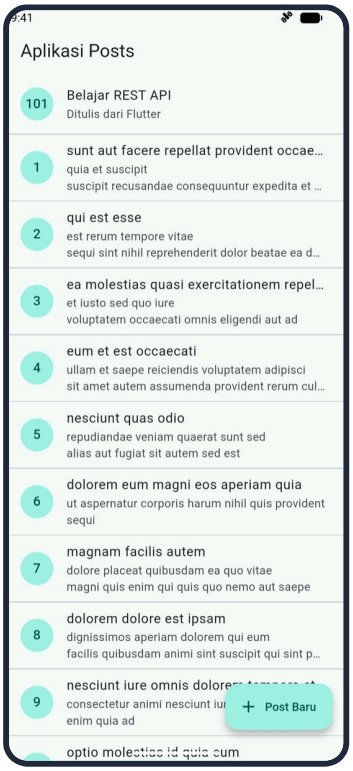
Layar daftar



Layar detail dengan tombol edit dan hapus



Form tambah post



Setelah disimpan: post baru muncul paling atas

Alur Data Saat Menambah Post

```
PostFormScreen._simpan()
   └─> PostCrudProvider.add()          saving = true, notifyListeners (tombol jadi "Menyimpan...")
         └─> RemotePostRepository.create()
               └─> ApiClient.post('/posts', json)   -> 201 Created
```

```
          <- Post baru dimasukkan ke _posts, saving = false, notifyListeners
   <- SnackBar "Post ditambahkan", Navigator.pop
PostListScreen (context.watch) otomatis rebuild dan post baru muncul paling atas
```

⚠️ Kesalahan Umum pada Proyek CRUD

- Menutup form (`Navigator.pop`) sebelum `await` selesai, sehingga error tidak pernah terlihat.
- Mengirim objek yang sama ke halaman detail lalu mengeditnya: detail tidak berubah. Kirim `id` lalu cari data terbaru dari provider (seperti `findById`).
- Menekan tombol Simpan berkali-kali sehingga data dobel. Nonaktifkan tombol saat `saving` .
- Memakai `context` setelah `await` tanpa cek `mounted` / `context.mounted` .

Tugas Praktikum: Praktikum 9: Aplikasi Katalog Tugas Kuliah Berbasis REST API

Sasaran Praktikum

Membangun aplikasi CRUD yang terhubung ke REST API publik dengan arsitektur berlapis (ApiClient, Repository, Provider) dan penanganan error yang ramah pengguna.

Langkah Pengerjaan

1. Gunakan endpoint `https://jsonplaceholder.typicode.com/todos` (atau mock API pribadi seperti mockapi.io agar data benar-benar tersimpan).
2. Buat model `Todo` (id, userId, title, completed) lengkap dengan `fromJson`, `toJson`, dan `copyWith`.
3. Buat `ApiClient` yang menambahkan header JSON, memberi timeout 10 detik, dan mengubah semua kegagalan menjadi `ApiException`.
4. Buat `TodoRepository` (abstract) beserta `RemoteTodoRepository` dan `FakeTodoRepository`.
5. Buat `TodoProvider` dengan status `loading/success/error`; tampilkan daftar todo dengan `?_limit=20` dan pull-to-refresh.
6. Sediakan halaman detail, form tambah/edit (satu form untuk dua mode, dengan validasi), dan hapus dengan dialog konfirmasi.
7. Ubah status selesai todo memakai request `PATCH` yang hanya mengirim field `completed`.
8. Uji mode pesawat dan host yang salah: aplikasi wajib menampilkan pesan error dan tombol coba lagi, bukan crash. Tambahkan izin INTERNET lalu uji juga dengan build release.

Output yang Diharapkan

Aplikasi daftar tugas yang memuat data dari internet, bisa menambah, mengedit, menandai selesai, dan menghapus data dengan umpan balik visual (loading, SnackBar, pesan error) yang jelas.

Rubrik Penilaian

Model & parsing JSON (15%), ApiClient + error handling (20%), Repository & Provider (20%), Operasi CRUD + PATCH berjalan (25%), UX: loading, pull-to-refresh, validasi, konfirmasi hapus (20%).

Kuis Pemahaman

1. Mengapa variabel 'Future' pada FutureBuilder sebaiknya diinisialisasi di dalam method 'initState()', bukan langsung dipanggil di dalam method 'build()'?

A. Agar request HTTP tidak terpanggil berulang kali setiap kali layar berkedip atau di-rebuild oleh framework

B. Karena method build() tidak mendukung internet

C. Agar ukuran aplikasi menjadi lebih kecil

D. Supaya status HTTP otomatis bernilai 200

2. Kode status HTTP (Status Code) manakah yang mengindikasikan bahwa data baru BERHASIL dibuat di server?

A. 200 OK

B. 201 Created

C. 404 Not Found

D. 500 Internal Server Error

3. Apa hasil jsonDecode('["a", "b"]') di Dart?

A. List<dynamic>

B. List<String>

C. Map<String, dynamic>

D. String

4. Request http.get mengembalikan status 404. Apa yang terjadi pada kode yang hanya memakai try/catch tanpa mengecek statusCode?

A. Masuk ke blok catch secara otomatis

B. Tidak ada exception; kode melanjutkan seolah sukses sehingga harus mengecek statusCode sendiri

C. Aplikasi langsung tertutup

D. Package http mengulang request secara otomatis

5. Apa perbedaan utama PUT dan PATCH?

A. PUT untuk membuat data, PATCH untuk menghapus

B. PUT mengganti seluruh data, PATCH hanya mengubah field yang dikirim

C. PATCH lebih aman dari PUT

D. Tidak ada perbedaan

6. Format header yang benar untuk mengirim token login adalah...

A. 'Token': token

B. 'Authorization': 'Bearer $token'

C. 'Authorization': 'Bearer:$token'

D. 'Bearer': token

7. Mengapa body POST harus di-jsonEncode dan diberi header Content-Type: application/json?

A. Agar request lebih cepat

B. Karena tanpa itu package http mengirim Map sebagai form-urlencoded dan server tidak membacanya sebagai JSON

C. Karena Dart tidak mendukung Map

D. Agar status code menjadi 200

8. Pada RefreshIndicator, mengapa daftar perlu physics: AlwaysScrollableScrollPhysics()?

A. Agar scroll lebih cepat

B. Agar gestur tarik tetap bisa dilakukan walaupun isi list sedikit atau sedang menampilkan pesan error

C. Agar list tidak bisa di-scroll

D. Karena RefreshIndicator mewajibkan GridView

Kunci Jawaban & Pembahasan

1. A — Jika memanggil fungsi API langsung di dalam method build(), setiap kali ada interaksi kecil (seperti mengetik di keyboard atau animasi), build() dipanggil ulang, memicu pengiriman request HTTP berulang kali ke server tanpa henti.

2. B — Dalam standar arsitektur REST API, kode 201 Created menandakan bahwa permintaan POST untuk membuat entitas baru berhasil dieksekusi oleh server.

3. A — Array JSON selalu di-decode menjadi List<dynamic>. Untuk mendapatkan List<String> gunakan List<String>.from(...) atau .cast<String>().

4. B — Package http hanya melempar exception untuk kegagalan koneksi. Status 4xx/5xx tetap dianggap response yang valid, sehingga kita wajib mengecek statusCode.

5. B — PUT mengirim representasi lengkap objek sebagai pengganti, sedangkan PATCH hanya mengirim field yang berubah.

6. B — Standar Bearer token memakai header Authorization dengan nilai 'Bearer ' (ada spasi) diikuti token.

7. B — Jika body berupa Map, http mengirimnya sebagai form. jsonEncode mengubahnya menjadi teks JSON dan Content-Type memberi tahu server cara membacanya.

8. B — Jika konten tidak memenuhi layar, ListView tidak bisa di-scroll sehingga gestur pull-to-refresh tidak terpicu. AlwaysScrollableScrollPhysics memaksa list tetap bisa ditarik.

MODUL 10 · LOCAL STORAGE

Penyimpanan Data Lokal (SharedPreferences & SQLite)

Membangun aplikasi yang datanya tetap ada setelah ditutup. Mempelajari SharedPreferences (SharedPreferencesAsync & WithCache) untuk tema, onboarding, dan sesi login, lalu SQLite dengan sqflite: membuka database, model toMap/fromMap, CRUD, migrasi onUpgrade, query aman dengan whereArgs, transaksi & batch, relasi tabel, hingga studi kasus aplikasi Catatan Kuliah multi-file, ditutup perbandingan dengan Hive dan Supabase.

Durasi Pertemuan 10 (150 Menit) + Belajar Mandiri

Prasyarat Modul 09: Integrasi REST API

Capaian Pembelajaran

- Memilih solusi persistensi lokal yang tepat sesuai jenis dan kompleksitas data.
- Menyimpan dan membaca preferensi (tema, onboarding, sesi login) dengan SharedPreferencesAsync dan SharedPreferencesWithCache.
- Merancang skema tabel SQLite, model toMap/fromMap, dan operasi CRUD dengan sqflite.
- Mengelola perubahan skema dengan version, onCreate, dan onUpgrade tanpa menghilangkan data pengguna.
- Menulis query aman (whereArgs) untuk pencarian, filter, agregasi, serta memakai transaksi, batch, dan relasi foreign key.
- Membangun aplikasi CRUD offline multi-file (Model, Database, Provider, Screen) dan menggunakan path_provider untuk file.

Daftar Isi Modul

1. Mengapa Perlu Penyimpanan Lokal & Pilihannya
2. Dasar SharedPreferences (Async & WithCache)
3. Studi Kasus: Menyimpan Tema Terang/Gelap
4. Studi Kasus: Onboarding & Sesi Login
5. Pengenalan SQLite & Membuka Database
6. Model toMap & fromMap
7. CRUD Lengkap dengan sqflite
8. Migrasi Skema: version, onCreate, onUpgrade
9. Query Lanjutan, rawQuery & SQL Injection
10. Transaksi & Batch
11. Relasi Tabel: Foreign Key & JOIN
12. Studi Kasus: Aplikasi Catatan Kuliah Multi-File
13. Menyimpan File dengan path_provider
14. Perbandingan: SharedPreferences, SQLite, Hive, Supabase
15. Tugas Praktikum
16. Kuis Pemahaman & Kunci Jawaban

1. Mengapa Perlu Penyimpanan Lokal & Pilihannya

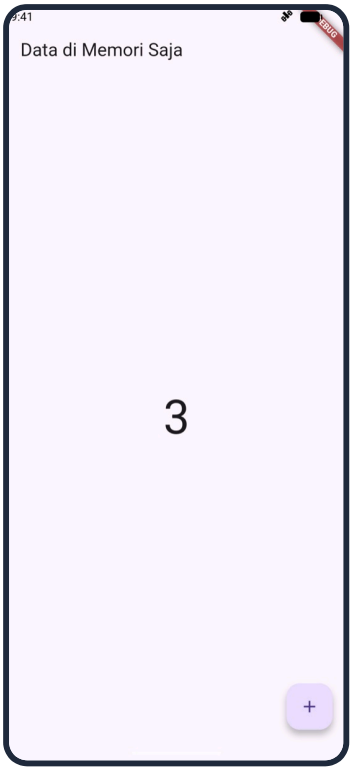
Semua variabel di aplikasi Flutter disimpan di **memori (RAM)**. Begitu aplikasi ditutup, isinya hilang. Agar data bertahan (tema pilihan pengguna, status login, catatan kuliah), data harus ditulis ke **penyimpanan lokal** di HP. Proses ini disebut *persistensi*.

Membuktikan Masalahnya

Jalankan contoh berikut, tekan tombol beberapa kali, lalu tutup aplikasi sepenuhnya (swipe dari recent apps) dan buka lagi. Angkanya kembali ke 0.

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CounterHilang()));

class CounterHilang extends StatefulWidget {
  const CounterHilang({super.key});

  @override
  State<CounterHilang> createState() => _CounterHilangState();
}

class _CounterHilangState extends State<CounterHilang> {
  int _counter = 0; // hanya ada di RAM

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Data di Memori Saja')),
      body: Center(
        child: Text('$_counter', style: Theme.of(context).textTheme.displayLarge),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _counter++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Counter di memori: kembali ke 0
setelah aplikasi ditutup

Di section berikutnya, contoh yang sama kita perbaiki dengan SharedPreferences sehingga angka tetap tersimpan.

Pilihan Penyimpanan Lokal di Flutter

Kategori	Package Populer	Struktur Data	Kasus Penggunaan Ideal
Key-Value Store	`shared_preferences`	Key-Value sederhana (String, int, double, bool, List<String>)	Onboarding status, tema gelap/terang, bahasa, sesi login sederhana
Relational Database	`sqflite`	Tabel SQL, kolom, relasi Foreign Key	Data banyak dan terstruktur: catatan, transaksi keuangan, tugas berelasi mata kuliah
NoSQL Fast Store	`hive_ce` (Modul 11)	Box berisi objek Dart	Objek tanpa relasi rumit, baca-tulis sangat cepat tanpa SQL
Penyimpanan Aman	`flutter_secure_storage`	Key-Value terenkripsi (Keychain/Keystore)	Token akses, refresh token, PIN
File	`path_provider` + `dart:io`	File bebas (teks, JSON, gambar)	Ekspor laporan, cache gambar, file unduhan

Cara Memilih

```
Apakah datanya hanya beberapa nilai pengaturan (tema, flag, nama pengguna)?
 ├─ Ya  ──> SharedPreferences
 └─ Tidak
     Apakah datanya rahasia (token, password)?
      ├─ Ya  ──> flutter_secure_storage
      └─ Tidak
          Apakah datanya banyak, perlu dicari/difilter/diurutkan, atau saling berelasi?
           ├─ Ya  ──> SQLite (sqflite)  atau Hive untuk data tanpa relasi
           └─ Tidak, berupa dokumen/gambar ──> File (path_provider)
Perlu dipakai bersama banyak perangkat/pengguna? ──> database cloud (Supabase, Modul 12)
```

⚠️ **Kesalahan Umum**

- Menyimpan ratusan data (mis. daftar transaksi) di SharedPreferences sebagai satu String JSON raksasa. Lambat dan sulit dicari; gunakan database.
- Menganggap penyimpanan lokal sebagai backup. Data lokal hilang saat aplikasi di-uninstall atau "Clear data".
- Menyimpan password asli di penyimpanan lokal. Jangan pernah; simpan token saja dan gunakan penyimpanan aman.

2. Dasar SharedPreferences (Async & WithCache)

`shared_preferences` menyimpan data kecil berbentuk **pasangan kunci-nilai** (key-value), seperti kamus: kunci `'tema'` bernilai `'gelap'`. Di Android data disimpan di DataStore/SharedPreferences, di iOS di NSUserDefaults, dan di web di LocalStorage.

Pasang package: `flutter pub add shared_preferences`

```
flutter pub add shared_preferences
```

Tipe Data yang Didukung

Tipe	Simpan	Baca (null jika belum ada)
`bool`	`setBool('sudahOnboarding', true)`	`getBool('sudahOnboarding')`
`int`	`setInt('counter', 10)`	`getInt('counter')`
`double`	`setDouble('ukuranFont', 1.2)`	`getDouble('ukuranFont')`
`String`	`setString('nama', 'Budi')`	`getString('nama')`
`List<String>`	`setStringList('riwayat', ['a', 'b'])`	`getStringList('riwayat')`

Tiga API: Mana yang Dipakai?

Sejak versi 2.3.0, package ini menyediakan tiga class. Dokumentasi resminya menyatakan class lama `SharedPreferences` akan di-*deprecate* dan menganjurkan dua class baru untuk kode baru.

Class	Cara membuat	Baca data	Kapan dipaka
`SharedPreferencesAsync`	`SharedPreferencesAsync()`	`await` (selalu dari disk)	Pilihan default Selalu memba nilai terbaru
`SharedPreferencesWithCache`	`await SharedPreferencesWithCache.create(...)`	Sinkron (dari cache memori)	Nilai di sangat sering, di `build` atau sa start aplikas
`SharedPreferences` (lama)	`await SharedPreferences.getInstance()`	Sinkron	Kode lama/tu lama. Hindar untuk proyek baru.

Di modul ini kita memakai `SharedPreferencesAsync` untuk kebanyakan contoh dan `SharedPreferencesWithCache` ketika butuh baca sinkron.

Contoh Kecil: `SharedPreferencesAsync`

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> contohAsync() async {
  final prefs = SharedPreferencesAsync();

  // Simpan
  await prefs.setString('nama', 'Budi');
  await prefs.setInt('semester', 3);
  await prefs.setStringList('matkulFavorit', ['Mobile', 'Basis Data']);

  // Baca: hasilnya nullable, beri nilai default dengan ??
  final nama = await prefs.getString('nama') ?? 'Tamu';
  final semester = await prefs.getInt('semester') ?? 1;
  print('$nama semester $semester');

  // Cek dan hapus
  final ada = await prefs.containsKey('nama');
  await prefs.remove('nama');
```

```
  print('Sebelum dihapus ada: $ada');

  // Hapus beberapa kunci saja (lebih aman daripada clear() tanpa filter)
  await prefs.clear(allowList: {'semester', 'matkulFavorit'});
}
```

Contoh Kecil: `SharedPreferencesWithCache`

Class ini dibuat sekali (async), lalu pembacaan bersifat sinkron tanpa `await`. Parameter `allowList` membatasi kunci yang boleh dipakai; kunci di luar daftar akan memicu error, sehingga typo cepat ketahuan.

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> contohWithCache() async {
  final prefs = await SharedPreferencesWithCache.create(
    cacheOptions: const SharedPreferencesWithCacheOptions(
      allowList: {'tema', 'counter'},
    ),
  );

  await prefs.setInt('counter', 5); // tulis tetap async
  final int counter = prefs.getInt('counter') ?? 0; // baca sinkron, tanpa await
  print(counter);
}
```

Contoh Lengkap: Counter yang Tidak Hilang

Ini perbaikan dari contoh di section 1. Angka dibaca saat layar dibuka dan disimpan setiap kali bertambah.

`lib/main.dart`

```
// Pastikan sudah: flutter pub add shared_preferences
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: CounterAwet()));

class CounterAwet extends StatefulWidget {
  const CounterAwet({super.key});

  @override
  State<CounterAwet> createState() => _CounterAwetState();
}

class _CounterAwetState extends State<CounterAwet> {
  static const _kunci = 'demo_counter';
  final _prefs = SharedPreferencesAsync();
  int? _counter; // null = sedang membaca dari penyimpanan

  @override
  void initState() {
    super.initState();
    _muat();
  }
```

```dart
  Future<void> _muat() async {
    final nilai = await _prefs.getInt(_kunci) ?? 0;
    if (mounted) setState(() => _counter = nilai);
  }

  Future<void> _tambah() async {
    final baru = (_counter ?? 0) + 1;
    setState(() => _counter = baru); // UI langsung berubah
    await _prefs.setInt(_kunci, baru); // lalu disimpan
  }

  Future<void> _reset() async {
    await _prefs.remove(_kunci);
    if (mounted) setState(() => _counter = 0);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter Tersimpan'),
        actions: [IconButton(onPressed: _reset, icon: const Icon(Icons.restart_alt))],
      ),
      body: Center(
        child: _counter == null
            ? const CircularProgressIndicator()
            : Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text('$_counter', style: Theme.of(context).textTheme.displayLarge),
                  const Text('Tutup aplikasi lalu buka lagi, angka tetap ada.'),
                ],
              ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _counter == null ? null : _tambah,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Counter tersimpan dengan SharedPreferencesAsync

Penjelasan:

- `_counter` bertipe `int?`. Nilai `null` berarti data belum selesai dibaca, sehingga kita menampilkan spinner. Membaca storage selalu butuh waktu.
- Kunci disimpan dalam konstanta `_kunci`. Menulis string `'demo_counter'` berulang kali berisiko typo.
- Pada `_tambah`, UI diperbarui dulu baru disimpan, sehingga aplikasi terasa responsif.

⚠️ **Kesalahan Umum**

- **Menyimpan data penting/besar**. Dokumentasi resmi menegaskan SharedPreferences bukan untuk data kritis. Gunakan untuk pengaturan kecil saja.
- **Lupa** `await` pada `SharedPreferencesAsync`, sehingga yang didapat `Future<int?>`, bukan `int?`.
- Membaca dengan tipe berbeda dari saat menyimpan (simpan `setInt`, baca `getString`): hasilnya error tipe.
- Memanggil `clear()` tanpa `allowList` saat logout. Semua data ikut terhapus, termasuk pengaturan tema.
- Data lama dari `SharedPreferences` versi lama tidak otomatis terbaca oleh `SharedPreferencesAsync` di Android. Untuk migrasi, package menyediakan fungsi `migrateLegacySharedPreferencesToSharedPreferencesAsyncIfNecessary`.

3. Studi Kasus: Menyimpan Tema Terang/Gelap

Kasus nyata pertama: pengguna memilih **tema Terang, Gelap, atau Ikuti Sistem**, dan pilihan itu harus tetap berlaku ketika aplikasi dibuka lagi. Kita gabungkan SharedPreferences dengan Provider (Modul 08).

Konsep

1. `MaterialApp` punya parameter `themeMode` (`ThemeMode.light`, `.dark`, `.system`).
2. Enum tidak bisa disimpan langsung, jadi simpan **namanya** (`ThemeMode.dark.name` = `'dark'`), lalu ubah kembali dengan `ThemeMode.values.byName('dark')`.
3. Tema harus sudah diketahui **sebelum** layar pertama tampil agar tidak "berkedip" dari terang ke gelap. Karena itu preferensi dibaca di `main()` sebelum `runApp` memakai `SharedPreferencesWithCache`.

Contoh Kecil: Enum ke String dan Sebaliknya

```dart
import 'package:flutter/material.dart';

void contohEnum() {
  const mode = ThemeMode.dark;
  final String disimpan = mode.name; // 'dark'
  final ThemeMode dibaca = ThemeMode.values.byName(disimpan); // ThemeMode.dark
  print('$disimpan -> $dibaca');
}
```

Langkah 1: ThemeProvider

`lib/providers/theme_provider.dart`

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class ThemeProvider extends ChangeNotifier {
  static const kunciTema = 'theme_mode';

  final SharedPreferencesWithCache _prefs;
  ThemeMode _mode;

  // Nilai awal dibaca sinkron dari cache, jadi tidak perlu loading
  ThemeProvider(this._prefs) : _mode = _baca(_prefs);

  ThemeMode get mode => _mode;

  static ThemeMode _baca(SharedPreferencesWithCache prefs) {
    final nama = prefs.getString(kunciTema);
    if (nama == null) return ThemeMode.system;
    try {
      return ThemeMode.values.byName(nama);
    } on ArgumentError {
      return ThemeMode.system; // nilai rusak/tidak dikenal
    }
```

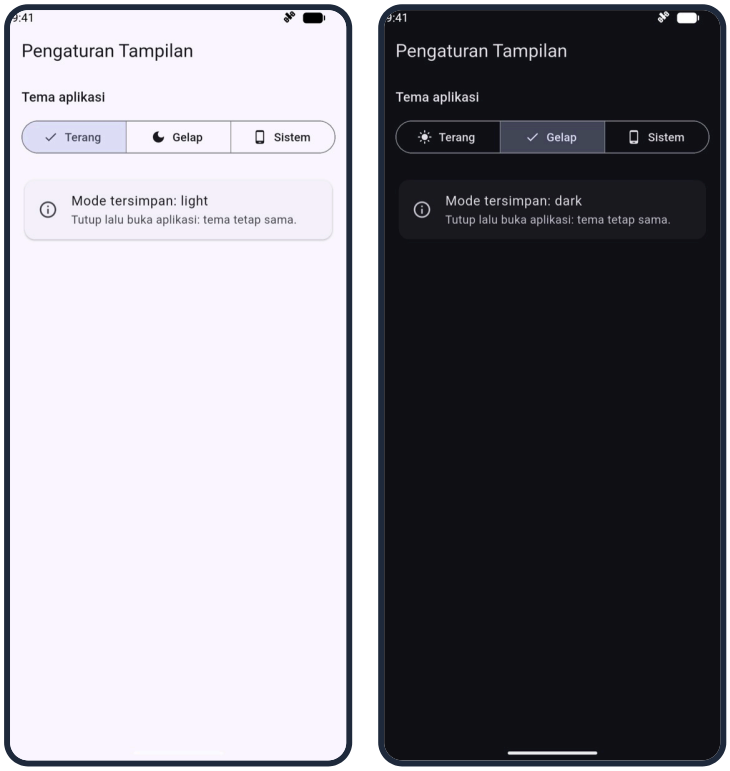
```
  }

  Future<void> setMode(ThemeMode mode) async {
    if (mode == _mode) return;
    _mode = mode;
    notifyListeners(); // UI berubah seketika
    await _prefs.setString(kunciTema, mode.name); // lalu disimpan
  }
}
```

Langkah 2: Baca Preferensi Sebelum `runApp`

`lib/main.dart`

```
// Pastikan sudah: flutter pub add shared_preferences provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'providers/theme_provider.dart';

Future<void> main() async {
  // Wajib sebelum memakai plugin di main()
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferencesWithCache.create(
    cacheOptions: const SharedPreferencesWithCacheOptions(
      allowList: {ThemeProvider.kunciTema},
    ),
  );

  runApp(
    ChangeNotifierProvider(
      create: (_) => ThemeProvider(prefs),
      child: const TemaApp(),
    ),
  );
}

class TemaApp extends StatelessWidget {
  const TemaApp({super.key});

  @override
  Widget build(BuildContext context) {
    final mode = context.watch<ThemeProvider>().mode;
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.light),
      darkTheme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.dark),
      themeMode: mode,
      home: const PengaturanScreen(),
    );
  }
}

class PengaturanScreen extends StatelessWidget {
  const PengaturanScreen({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    final themeProv = context.watch<ThemeProvider>();
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan Tampilan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Tema aplikasi', style: Theme.of(context).textTheme.titleMedium),
          const SizedBox(height: 12),
          SegmentedButton<ThemeMode>(
            segments: const [
              ButtonSegment(value: ThemeMode.light, icon: Icon(Icons.light_mode), label:
Text('Terang')),
              ButtonSegment(value: ThemeMode.dark, icon: Icon(Icons.dark_mode), label:
Text('Gelap')),
              ButtonSegment(value: ThemeMode.system, icon: Icon(Icons.phone_android), label:
Text('Sistem')),
            ],
            selected: {themeProv.mode},
            onSelectionChanged: (pilihan) => themeProv.setMode(pilihan.first),
          ),
          const SizedBox(height: 24),
          Card(
            child: ListTile(
              leading: const Icon(Icons.info_outline),
              title: Text('Mode tersimpan: ${themeProv.mode.name}'),
              subtitle: const Text('Tutup lalu buka aplikasi: tema tetap sama.'),
            ),
          ),
        ],
      ),
    );
  }
}
```



Mode Terang dipilih Mode Gelap dipilih dan tersimpan

Penjelasan:

- `WidgetsFlutterBinding.ensureInitialized()` wajib dipanggil karena kita memakai plugin (`SharedPreferencesWithCache.create`) sebelum `runApp`.
- Objek `prefs` dioper ke `ThemeProvider` lewat constructor. Provider tidak perlu membuat instance sendiri dan mudah diganti saat testing.
- `SegmentedButton` adalah komponen Material 3 untuk memilih satu dari beberapa opsi.

> ⚠️ **Kesalahan Umum**
>
> - Lupa `WidgetsFlutterBinding.ensureInitialized()`: muncul error *Binding has not yet been initialized*.
> - Membaca tema di `initState` layar pertama: tampilan sempat terang sebentar lalu berubah gelap (*flicker*).
> - Menyimpan `mode.toString()` (`'ThemeMode.dark'`) lalu membacanya dengan `byName`: gagal. Pakai `.name`.
> - Lupa menambahkan kunci ke `allowList`: `SharedPreferencesWithCache` melempar error saat kunci dipakai.

4. Studi Kasus: Onboarding & Sesi Login

Dua kebutuhan klasik aplikasi: **layar onboarding** yang hanya tampil sekali saat pertama dipasang, dan **sesi login** agar pengguna tidak perlu login setiap membuka aplikasi. Keduanya cukup memakai beberapa nilai `bool` dan `String` di SharedPreferences.

Alur Aplikasi

```
Buka aplikasi
   └─> SplashGate membaca penyimpanan
         ├─ sudahOnboarding == false ──> OnboardingScreen ──(Mulai)──> simpan true ──> Login
         ├─ token == null            ──> LoginScreen ──(Login)──> simpan token ──> Home
         └─ token ada                ──> HomeScreen ──(Logout)──> hapus token ──> Login
```

Langkah 1: SessionManager

Semua akses SharedPreferences dikumpulkan di satu class. Layar tidak perlu tahu nama kunci.

`lib/services/session_manager.dart`

```dart
// lib/services/session_manager.dart
import 'package:shared_preferences/shared_preferences.dart';

class SessionManager {
  static const String keyOnboarding = 'ONBOARDING_DONE';
  static const String keyToken = 'USER_AUTH_TOKEN';
  static const String keyNama = 'USER_NAME';

  final SharedPreferencesAsync _prefs = SharedPreferencesAsync();

  // ---------- Onboarding ----------
  Future<bool> sudahOnboarding() async => await _prefs.getBool(keyOnboarding) ?? false;

  Future<void> selesaiOnboarding() => _prefs.setBool(keyOnboarding, true);

  // ---------- Sesi login ----------
  Future<void> simpanSesiLogin({required String token, required String nama}) async {
    await _prefs.setString(keyToken, token);
    await _prefs.setString(keyNama, nama);
  }

  Future<String?> ambilToken() => _prefs.getString(keyToken);

  Future<String?> ambilNama() => _prefs.getString(keyNama);

  Future<bool> cekStatusLogin() async => await ambilToken() != null;

  // Hapus HANYA data sesi; flag onboarding dan pengaturan lain tetap ada
  Future<void> logout() => _prefs.clear(allowList: {keyToken, keyNama});
}
```

Penjelasan: status login tidak disimpan sebagai `bool` terpisah, melainkan disimpulkan dari ada/tidaknya token. Dengan begitu tidak mungkin terjadi keadaan "login = true tetapi token

kosong". Method `logout()` memakai `clear(allowList: ...)` agar hanya kunci sesi yang terhapus.

> ℹ️ **Tentang Keamanan Token**
>
> Contoh ini cocok untuk belajar. SharedPreferences tidak terenkripsi, jadi untuk aplikasi produksi simpan token di `flutter_secure_storage`. Polanya sama: ganti isi `SessionManager` saja, layar tidak berubah.

Langkah 2: Contoh Lengkap Onboarding + Login

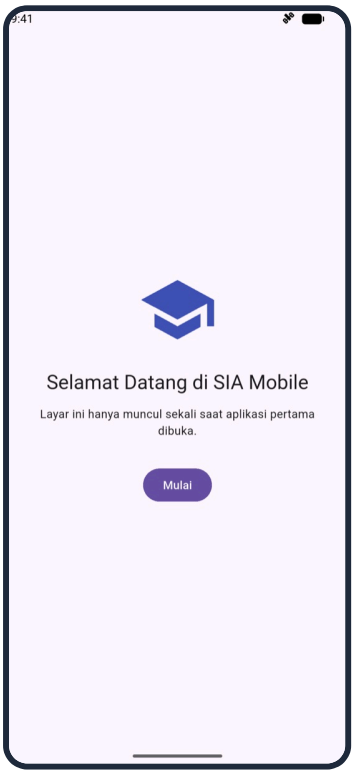
📄 **`lib/main.dart`**

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add shared_preferences, serta membuat
lib/services/session_manager.dart
import 'package:flutter/material.dart';

import 'services/session_manager.dart';

final session = SessionManager();

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: SplashGate()));
}

/// Membaca penyimpanan lalu memutuskan layar pertama.
class SplashGate extends StatefulWidget {
  const SplashGate({super.key});

  @override
  State<SplashGate> createState() => _SplashGateState();
}

class _SplashGateState extends State<SplashGate> {
  @override
  void initState() {
    super.initState();
    _tentukanTujuan();
  }

  Future<void> _tentukanTujuan() async {
    final onboarding = await session.sudahOnboarding();
    final login = await session.cekStatusLogin();
    if (!mounted) return;

    final Widget tujuan = !onboarding
        ? const OnboardingScreen()
        : login
            ? const HomeScreen()
            : const LoginScreen();
    // pushReplacement: SplashGate dibuang agar tombol Back tidak kembali ke sini
```

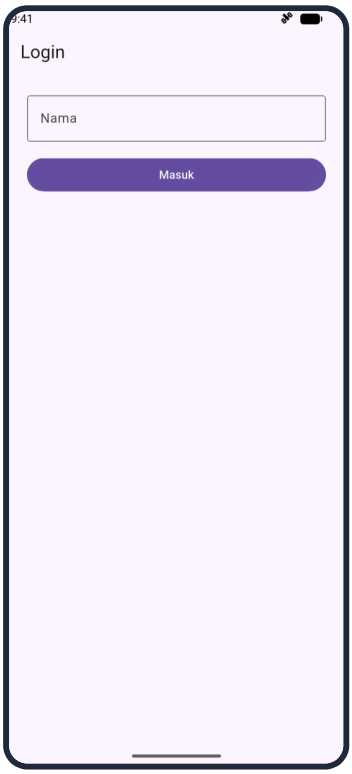
```
    Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => tujuan));
  }

  @override
  Widget build(BuildContext context) {
    return const Scaffold(body: Center(child: CircularProgressIndicator()));
  }
}

class OnboardingScreen extends StatelessWidget {
  const OnboardingScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(32),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Icon(Icons.school, size: 96, color: Colors.indigo),
              const SizedBox(height: 24),
              Text('Selamat Datang di SIA Mobile',
                  style: Theme.of(context).textTheme.headlineSmall, textAlign: TextAlign.center),
              const SizedBox(height: 12),
              const Text('Layar ini hanya muncul sekali saat aplikasi pertama dibuka.',
                  textAlign: TextAlign.center),
              const SizedBox(height: 32),
              FilledButton(
                onPressed: () async {
                  await session.selesaiOnboarding();
                  if (!context.mounted) return;
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(builder: (_) => const LoginScreen()),
                  );
                },
                child: const Text('Mulai'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _namaCtrl = TextEditingController();

  @override
```

```
  void dispose() {
    _namaCtrl.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    final nama = _namaCtrl.text.trim();
    if (nama.isEmpty) return;
    // Pada aplikasi nyata token didapat dari API login (Modul 09)
    await session.simpanSesiLogin(token: 'token-${DateTime.now().millisecondsSinceEpoch}', nama:
nama);
    if (!mounted) return;
    Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const HomeScreen()));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            TextField(
              controller: _namaCtrl,
              decoration: const InputDecoration(labelText: 'Nama', border: OutlineInputBorder()),
            ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: _login, child: const Text('Masuk')),
            ),
          ],
        ),
      ),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            icon: const Icon(Icons.logout),
            onPressed: () async {
              await session.logout();
              if (!context.mounted) return;
              Navigator.pushReplacement(
                context,
                MaterialPageRoute(builder: (_) => const LoginScreen()),
              );
            },
```

```
          ),
        ],
      ),
      body: Center(
        child: FutureBuilder<String?>(
          future: session.ambilNama(),
          builder: (context, snapshot) => Text(
            'Halo, ${snapshot.data ?? '...'}!\nTutup lalu buka aplikasi: kamu tetap login.',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.titleMedium,
          ),
        ),
      ),
    );
  }
}
```



Pembukaan pertama: onboarding



Setelah Mulai: layar login

Setelah login: beranda dengan nama tersimpan

Cara menguji: (1) buka pertama kali, muncul onboarding; (2) tekan Mulai lalu login; (3) tutup aplikasi sepenuhnya dan buka lagi, langsung masuk Beranda; (4) Logout lalu buka lagi, muncul Login (bukan onboarding).

⚠️ **Kesalahan Umum**

- Memakai `Navigator.push` (bukan `pushReplacement`) dari splash/login, sehingga tombol Back kembali ke layar login.
- Logout dengan `clear()` tanpa filter, sehingga onboarding muncul lagi.
- Memakai `context` setelah `await` tanpa cek `mounted` / `context.mounted`.
- Untuk mengulang dari awal saat testing, hapus data aplikasi (Settings → Apps → Clear storage) atau uninstall. Hot restart **tidak** menghapus data tersimpan.

5. Pengenalan SQLite & Membuka Database

Untuk data yang banyak dan terstruktur (daftar catatan, nilai mahasiswa, transaksi), kita memakai **SQLite**: database relasional kecil yang tersimpan sebagai **satu file** di HP. Package `sqflite` menghubungkan Flutter dengan SQLite bawaan Android dan iOS.

```
flutter pub add sqflite path
```

Package `path` dipakai untuk menggabungkan folder dan nama file database dengan benar (`join`).

> ⚠️ **sqflite Tidak Berjalan di Chrome (Web)**
>
> `sqflite` hanya mendukung Android, iOS, dan macOS. Jika dijalankan dengan `flutter run -d chrome`, muncul `MissingPluginException` dan layar kosong. Gunakan emulator Android atau HP fisik.

Kilat SQL: 5 Perintah Dasar

Jika sudah mengambil mata kuliah Basis Data, ini hanya pengingat:

```
-- Membuat tabel
CREATE TABLE mahasiswa (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nim TEXT NOT NULL UNIQUE,
  nama TEXT NOT NULL,
  ipk REAL DEFAULT 0
);

-- Menambah data
INSERT INTO mahasiswa (nim, nama, ipk) VALUES ('H1D022001', 'Budi', 3.5);

-- Membaca data
SELECT * FROM mahasiswa WHERE ipk >= 3.0 ORDER BY nama ASC;

-- Mengubah data
UPDATE mahasiswa SET ipk = 3.8 WHERE id = 1;

-- Menghapus data
DELETE FROM mahasiswa WHERE id = 1;
```

Tipe Data SQLite dan Padanannya di Dart

SQLite	Dart	Catatan
`INTEGER`	`int`	Juga dipakai untuk `bool` (0/1)
`REAL`	`double`	IPK, harga, koordinat
`TEXT`	`String`	Juga untuk `DateTime` (format ISO 8601)
`BLOB`	`Uint8List`	Data biner (jarang dipakai, simpan path file saja)
`NULL`	`null`	Kolom tanpa `NOT NULL` boleh kosong

SQLite **tidak punya** tipe `BOOLEAN` dan `DATETIME` asli. Konversinya kita lakukan sendiri di model (section berikutnya).

Contoh Kecil: Membuka Database

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

Future<Database> bukaDatabase() async {
  final folder = await getDatabasesPath(); // folder khusus database milik aplikasi
  final path = join(folder, 'kampus.db');   // .../databases/kampus.db

  return openDatabase(
    path,
    version: 1,
    // onCreate hanya dipanggil SEKALI: saat file database belum ada
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE mahasiswa (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          nim TEXT NOT NULL UNIQUE,
          nama TEXT NOT NULL,
          ipk REAL DEFAULT 0
        )
      ''');
    },
  );
}
```

- `getDatabasesPath()` mengembalikan folder database aplikasi (di Android: `/data/data/<package>/databases`).
- `version` adalah versi **skema** (struktur tabel), bukan versi aplikasi. Dipakai untuk migrasi di section 8.

- `onCreate` berjalan hanya ketika file database belum ada. Mengubah isi `onCreate` tidak berpengaruh pada database yang sudah terbuat.

Pola Singleton: Satu Koneksi untuk Seluruh Aplikasi

Membuka database itu mahal. Simpan koneksinya dan gunakan ulang:

lib/data/app_database.dart

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  AppDatabase._(); // constructor privat: tidak bisa di-new dari luar
  static final AppDatabase instance = AppDatabase._();

  Database? _db;

  Future<Database> get database async {
    // ??= : buka hanya jika belum pernah dibuka
    return _db ??= await _open();
  }

  Future<Database> _open() async {
    final path = join(await getDatabasesPath(), 'intro_kampus.db');
    return openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE mahasiswa (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nim TEXT NOT NULL UNIQUE,
            nama TEXT NOT NULL,
            ipk REAL DEFAULT 0
          )
        ''');
      },
    );
  }

  Future<void> close() async {
    await _db?.close();
    _db = null;
  }
}
```

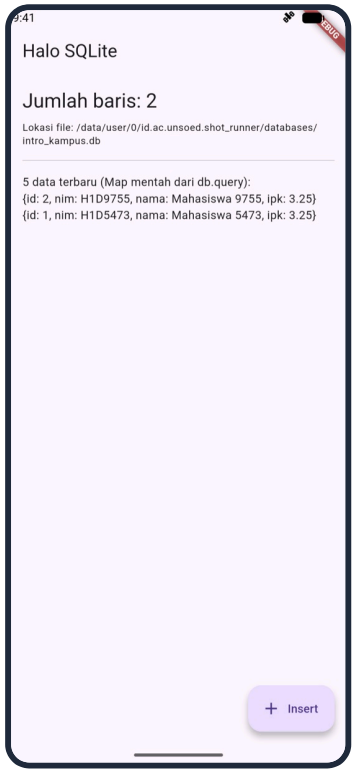
Contoh Lengkap: Insert dan Hitung Baris

lib/main.dart

```
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/app_database.dart
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';

import 'data/app_database.dart';
```

```dart
void main() => runApp(const MaterialApp(home: IntroSqliteScreen()));

class IntroSqliteScreen extends StatefulWidget {
  const IntroSqliteScreen({super.key});

  @override
  State<IntroSqliteScreen> createState() => _IntroSqliteScreenState();
}

class _IntroSqliteScreenState extends State<IntroSqliteScreen> {
  int _jumlah = 0;
  String _path = '';
  List<Map<String, Object?>> _baris = [];

  @override
  void initState() {
    super.initState();
    _refresh();
  }

  Future<void> _refresh() async {
    final db = await AppDatabase.instance.database;
    final jumlah = Sqflite.firstIntValue(await db.rawQuery('SELECT COUNT(*) FROM mahasiswa')) ?? 0;
    final baris = await db.query('mahasiswa', orderBy: 'id DESC', limit: 5);
    if (!mounted) return;
    setState(() {
      _jumlah = jumlah;
      _path = db.path;
      _baris = baris;
    });
  }

  Future<void> _tambah() async {
    final db = await AppDatabase.instance.database;
    final n = DateTime.now().millisecondsSinceEpoch % 100000;
    await db.insert('mahasiswa', {'nim': 'H1D$n', 'nama': 'Mahasiswa $n', 'ipk': 3.25});
    await _refresh();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Halo SQLite')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Jumlah baris: $_jumlah', style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Text('Lokasi file: $_path', style: Theme.of(context).textTheme.bodySmall),
          const Divider(height: 32),
          const Text('5 data terbaru (Map mentah dari db.query):'),
          for (final row in _baris) Text(row.toString()),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _tambah,
        icon: const Icon(Icons.add),
        label: const Text('Insert'),
      ),
```

```
    );
  }
}
```



Setelah dua kali Insert: jumlah baris dan Map hasil query

Penjelasan: setiap baris hasil `db.query` berbentuk `Map<String, Object?>` dengan nama kolom sebagai kunci. `Sqflite.firstIntValue` mengambil angka pertama dari hasil query seperti `COUNT(*)`. Tutup lalu buka aplikasi: jumlah baris tetap.

⚠ Kesalahan Umum

- Membuka database baru (`openDatabase`) di setiap fungsi. Gunakan singleton.
- Salah ketik SQL di `onCreate` (misalnya koma di akhir kolom terakhir). Error baru muncul saat aplikasi dijalankan, jadi baca pesan `DatabaseException` dengan teliti.
- Mengubah `CREATE TABLE` lalu heran kolom baru tidak ada. `onCreate` tidak dijalankan ulang; lihat section Migrasi.
- Menjalankan di Chrome: sqflite tidak mendukung web.

6. Model toMap & fromMap

Seperti di Modul 09 (JSON), data dari database sebaiknya tidak dipakai sebagai `Map` mentah. Kita buat **model** dengan dua method: `toMap()` untuk menyimpan ke tabel dan `fromMap()` untuk membaca dari tabel.

Konsep: Konversi Tipe

Field Dart	Kolom SQLite	`toMap()`	`fromMap()`
`int? id`	`INTEGER PRIMARY KEY`	tidak ditulis jika null (dibuat otomatis)	`map['id'] as int?`
`bool isSelesai`	`INTEGER`	`isSelesai ? 1 : 0`	`map['isSelesai'] == 1`
`DateTime tenggat`	`TEXT`	`tenggat.toIso8601String()`	`DateTime.parse(...)`
`double ipk`	`REAL`	langsung	`(map['ipk'] as num).toDouble()`

Contoh Kecil

```dart
class Matkul {
  final int? id;
  final String nama;
  final int sks;

  const Matkul({this.id, required this.nama, required this.sks});

  Map<String, Object?> toMap() => {
        if (id != null) 'id': id,
        'nama': nama,
        'sks': sks,
      };

  factory Matkul.fromMap(Map<String, Object?> map) => Matkul(
        id: map['id'] as int?,
        nama: map['nama'] as String,
        sks: map['sks'] as int,
      );
}
```

Model Lengkap: Tugas Kuliah

Model ini dipakai di section CRUD berikutnya. Nama kunci Map harus **sama persis** dengan nama kolom tabel.

lib/models/tugas.dart

```dart
class Tugas {
  final int? id;
  final String judul;
  final String matkul;
  final DateTime tenggat;
  final bool isSelesai;

  const Tugas({
    this.id,
    required this.judul,
    required this.matkul,
    required this.tenggat,
    this.isSelesai = false,
  });

  /// Dart -> baris tabel
  Map<String, Object?> toMap() {
    return {
      if (id != null) 'id': id,
      'judul': judul,
      'matkul': matkul,
      'tenggat': tenggat.toIso8601String(), // DateTime -> TEXT
      'isSelesai': isSelesai ? 1 : 0, // bool -> INTEGER
    };
  }

  /// Baris tabel -> Dart
  factory Tugas.fromMap(Map<String, Object?> map) {
    return Tugas(
      id: map['id'] as int?,
      judul: map['judul'] as String,
      matkul: map['matkul'] as String,
      tenggat: DateTime.parse(map['tenggat'] as String),
      isSelesai: map['isSelesai'] == 1,
    );
  }

  Tugas copyWith({int? id, String? judul, String? matkul, DateTime? tenggat, bool? isSelesai}) {
    return Tugas(
      id: id ?? this.id,
      judul: judul ?? this.judul,
      matkul: matkul ?? this.matkul,
      tenggat: tenggat ?? this.tenggat,
      isSelesai: isSelesai ?? this.isSelesai,
    );
  }

  /// Sisa hari menuju tenggat (negatif berarti terlambat)
  int get sisaHari => tenggat.difference(DateTime.now()).inDays;

  @override
  String toString() => 'Tugas(id: $id, judul: $judul, selesai: $isSelesai)';
}
```

Mencoba Konversi Tanpa Database

`lib/contoh_tugas.dart`

```dart
// Jalankan dengan: dart run lib/contoh_tugas.dart
import 'models/tugas.dart';

void main() {
  final tugas = Tugas(
    judul: 'Laporan Praktikum 10',
    matkul: 'Pemrograman Mobile',
    tenggat: DateTime(2026, 11, 20, 23, 59),
  );

  final map = tugas.toMap();
  print(map);
  // {judul: Laporan Praktikum 10, matkul: Pemrograman Mobile,
  //  tenggat: 2026-11-20T23:59:00.000, isSelesai: 0}

  // Seolah-olah baris ini dibaca dari database (sudah punya id)
  final dariDb = Tugas.fromMap({...map, 'id': 7, 'isSelesai': 1});
  print(dariDb); // Tugas(id: 7, judul: Laporan Praktikum 10, selesai: true)
  print(dariDb.tenggat.year); // 2026
}
```

Penjelasan: `{...map, 'id': 7}` adalah *spread operator*: menyalin isi `map` lalu menambah/menimpa kunci tertentu. Perhatikan `toMap()` tidak menulis `id` ketika null, karena kolom `AUTOINCREMENT` akan mengisinya sendiri.

> ⚠️ **Kesalahan Umum**
>
> - Menyimpan `bool` langsung (`'isSelesai': true`). sqflite akan menolak atau menyimpan dengan tidak konsisten; selalu ubah ke 0/1.
> - Menyimpan `DateTime` langsung: error *Unsupported value*. Ubah ke String ISO 8601 (atau `millisecondsSinceEpoch` sebagai INTEGER).
> - Menulis `'id': null` di `toMap()` lalu memakai `ConflictAlgorithm.replace`; lebih aman tidak menulis id sama sekali saat insert.
> - Nama kunci di Map berbeda dengan nama kolom (`is_selesai` vs `isSelesai`): muncul error *no such column*.

7. CRUD Lengkap dengan sqflite

Sekarang kita membuat operasi **CRUD** (Create, Read, Update, Delete) lengkap. sqflite menyediakan method siap pakai sehingga kita jarang menulis SQL mentah: `insert`, `query`, `update`, dan `delete`.

Peta Method sqflite

Operasi	Method	Nilai kembali	SQL yang dihasilkan
Create	`db.insert('tugas', map)`	`int` id baris baru	`INSERT INTO tugas (...) VALUES (...)`
Read	`db.query('tugas', where: ..., orderBy: ...)`	`List<Map>`	`SELECT * FROM tugas WHERE ...`
Update	`db.update('tugas', map, where: 'id = ?', whereArgs: [id])`	`int` jumlah baris berubah	`UPDATE tugas SET ... WHERE id = ?`
Delete	`db.delete('tugas', where: 'id = ?', whereArgs: [id])`	`int` jumlah baris terhapus	`DELETE FROM tugas WHERE id = ?`

Langkah 1: DatabaseHelper (Singleton + Tabel + CRUD Map)

Buat folder `lib/data/` lalu file `database_helper.dart`. Versi ini masih bekerja dengan `Map` agar alurnya mudah dilihat.

`lib/data/database_helper.dart`

```dart
// lib/data/database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('tugas_kampus.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
```

```
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE tugas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            matkul TEXT NOT NULL,
            tenggat TEXT NOT NULL,
            isSelesai INTEGER NOT NULL
          )
        ''');
      },
    );
  }

  // CREATE (Insert)
  Future<int> tambahTugas(Map<String, dynamic> row) async {
    final db = await instance.database;
    return await db.insert('tugas', row);
  }

  // READ (Query All)
  Future<List<Map<String, dynamic>>> ambilSemuaTugas() async {
    final db = await instance.database;
    return await db.query('tugas', orderBy: 'id DESC');
  }

  // UPDATE
  Future<int> updateStatus(int id, bool isSelesai) async {
    final db = await instance.database;
    return await db.update(
      'tugas',
      {'isSelesai': isSelesai ? 1 : 0},
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  // DELETE
  Future<int> hapusTugas(int id) async {
    final db = await instance.database;
    return await db.delete('tugas', where: 'id = ?', whereArgs: [id]);
  }
}
```

Penjelasan: `where: 'id = ?'` berisi tanda tanya sebagai tempat nilai, dan nilainya diberikan lewat `whereArgs`. Jangan pernah menyambung nilai langsung ke string SQL (lihat section Query Lanjutan tentang SQL injection). Tanpa `where`, `update` dan `delete` akan mengenai **semua baris**.

Langkah 2: DAO Bertipe dengan Model `Tugas`

DAO (*Data Access Object*) membungkus helper agar UI bekerja dengan objek `Tugas` (section 6), bukan Map.

`lib/data/tugas_dao.dart`

```dart
import 'package:sqflite/sqflite.dart';

import '../models/tugas.dart';
import 'database_helper.dart';

class TugasDao {
  Future<Database> get _db => DatabaseHelper.instance.database;

  // CREATE: kembalikan objek lengkap dengan id dari database
  Future<Tugas> insert(Tugas tugas) async {
    final db = await _db;
    final id = await db.insert('tugas', tugas.toMap());
    return tugas.copyWith(id: id);
  }

  // READ semua: belum selesai di atas, lalu tenggat terdekat
  Future<List<Tugas>> getAll() async {
    final db = await _db;
    final rows = await db.query('tugas', orderBy: 'isSelesai ASC, tenggat ASC');
    return rows.map(Tugas.fromMap).toList();
  }

  // READ satu
  Future<Tugas?> getById(int id) async {
    final db = await _db;
    final rows = await db.query('tugas', where: 'id = ?', whereArgs: [id], limit: 1);
    return rows.isEmpty ? null : Tugas.fromMap(rows.first);
  }

  // UPDATE seluruh kolom
  Future<int> update(Tugas tugas) async {
    final db = await _db;
    return db.update('tugas', tugas.toMap(), where: 'id = ?', whereArgs: [tugas.id]);
  }

  // DELETE
  Future<int> delete(int id) async {
    final db = await _db;
    return db.delete('tugas', where: 'id = ?', whereArgs: [id]);
  }
}
```

`rows.map(Tugas.fromMap)` adalah *tear-off*: mengoper constructor sebagai fungsi, sama dengan `rows.map((r) => Tugas.fromMap(r))`.

Langkah 3: Contoh Lengkap Aplikasi Tugas Kuliah

Fitur: tambah dan edit lewat dialog, centang selesai (UPDATE), geser untuk hapus (DELETE), dan data tetap ada setelah aplikasi ditutup.

`lib/main.dart`

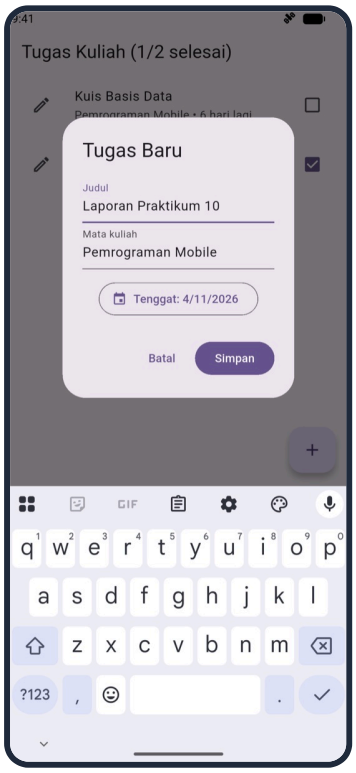
```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add sqflite path, serta membuat
```

```dart
// lib/models/tugas.dart, lib/data/database_helper.dart, lib/data/tugas_dao.dart
// CATATAN: sqflite hanya berjalan di Android, iOS, dan macOS. TIDAK bisa di Chrome (web).
import 'package:flutter/material.dart';

import 'data/tugas_dao.dart';
import 'models/tugas.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
    home: const DaftarTugasScreen(),
  ));
}

class DaftarTugasScreen extends StatefulWidget {
  const DaftarTugasScreen({super.key});

  @override
  State<DaftarTugasScreen> createState() => _DaftarTugasScreenState();
}

class _DaftarTugasScreenState extends State<DaftarTugasScreen> {
  final _dao = TugasDao();
  List<Tugas> _daftar = [];
  bool _loading = true;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final data = await _dao.getAll();
    if (!mounted) return;
    setState(() {
      _daftar = data;
      _loading = false;
    });
  }

  /// Buka dialog tambah (tugas == null) atau edit (tugas != null)
  Future<void> _bukaForm([Tugas? tugas]) async {
    final hasil = await showDialog<Tugas>(
      context: context,
      builder: (_) => TugasDialog(tugas: tugas),
    );
    if (hasil == null) return; // dibatalkan

    if (hasil.id == null) {
      await _dao.insert(hasil);
    } else {
      await _dao.update(hasil);
    }
    await _muat();
  }

  Future<void> _toggle(Tugas t, bool selesai) async {
```

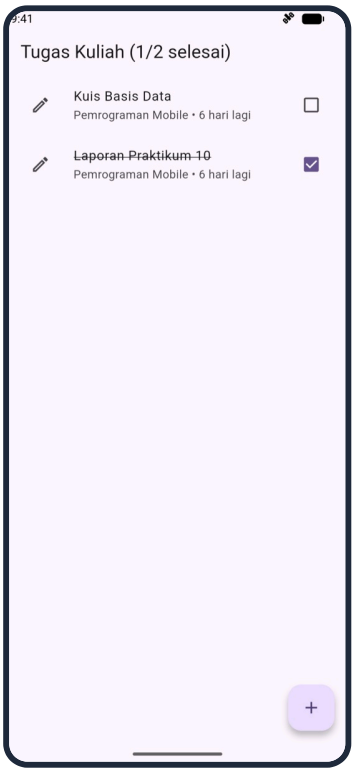
```
    await _dao.update(t.copyWith(isSelesai: selesai));
    await _muat();
  }

  Future<void> _hapus(Tugas t) async {
    await _dao.delete(t.id!);
    setState(() => _daftar.remove(t));
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('"${t.judul}" dihapus'),
        action: SnackBarAction(
          label: 'URUNGKAN',
          onPressed: () async {
            await _dao.insert(t); // id lama ikut disimpan kembali
            await _muat();
          },
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final selesai = _daftar.where((t) => t.isSelesai).length;
    return Scaffold(
      appBar: AppBar(title: Text('Tugas Kuliah ($selesai/${_daftar.length} selesai)')),
      body: _loading
          ? const Center(child: CircularProgressIndicator())
          : _daftar.isEmpty
              ? const Center(child: Text('Belum ada tugas. Tekan tombol + untuk menambah.'))
              : ListView.builder(
                  itemCount: _daftar.length,
                  itemBuilder: (context, index) {
                    final t = _daftar[index];
                    final telat = !t.isSelesai && t.sisaHari < 0;
                    return Dismissible(
                      key: ValueKey(t.id),
                      direction: DismissDirection.endToStart,
                      background: Container(
                        color: Colors.red,
                        alignment: Alignment.centerRight,
                        padding: const EdgeInsets.only(right: 20),
                        child: const Icon(Icons.delete, color: Colors.white),
                      ),
                      onDismissed: (_) => _hapus(t),
                      child: CheckboxListTile(
                        value: t.isSelesai,
                        onChanged: (v) => _toggle(t, v ?? false),
                        title: Text(
                          t.judul,
                          style: TextStyle(
                            decoration: t.isSelesai ? TextDecoration.lineThrough : null,
                          ),
                        ),
                        subtitle: Text(
                          '${t.matkul} • ${telat ? 'TERLAMBAT' : '${t.sisaHari} hari lagi'}',
                          style: TextStyle(color: telat ? Colors.red : null),
                        ),
```

```
                        secondary: IconButton(
                          icon: const Icon(Icons.edit_outlined),
                          onPressed: () => _bukaForm(t),
                        ),
                      ),
                    );
                  },
                ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah tugas',
        onPressed: () => _bukaForm(),
        child: const Icon(Icons.add),
      ),
    );
  }
}

/// Dialog form. Controller dimiliki State dialog sendiri sehingga
/// di-dispose otomatis SETELAH animasi tutup dialog selesai.
class TugasDialog extends StatefulWidget {
  final Tugas? tugas;
  const TugasDialog({super.key, this.tugas});

  @override
  State<TugasDialog> createState() => _TugasDialogState();
}

class _TugasDialogState extends State<TugasDialog> {
  late final _judulCtrl = TextEditingController(text: widget.tugas?.judul);
  late final _matkulCtrl =
      TextEditingController(text: widget.tugas?.matkul ?? 'Pemrograman Mobile');
  late DateTime _tenggat =
      widget.tugas?.tenggat ?? DateTime.now().add(const Duration(days: 7));

  @override
  void dispose() {
    _judulCtrl.dispose();
    _matkulCtrl.dispose();
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final dipilih = await showDatePicker(
      context: context,
      initialDate: _tenggat,
      firstDate: DateTime(2020),
      lastDate: DateTime(2035),
    );
    if (dipilih != null) setState(() => _tenggat = dipilih);
  }

  void _simpan() {
    final judul = _judulCtrl.text.trim();
    if (judul.isEmpty) return;
    final lama = widget.tugas;
    final data = lama == null
        ? Tugas(judul: judul, matkul: _matkulCtrl.text.trim(), tenggat: _tenggat)
        : lama.copyWith(judul: judul, matkul: _matkulCtrl.text.trim(), tenggat: _tenggat);
    Navigator.pop(context, data); // kirim hasil ke pemanggil showDialog
```

```
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(widget.tugas == null ? 'Tugas Baru' : 'Edit Tugas'),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          TextField(controller: _judulCtrl, decoration: const InputDecoration(labelText: 'Judul')),
          TextField(controller: _matkulCtrl, decoration: const InputDecoration(labelText: 'Mata
kuliah')),
          const SizedBox(height: 12),
          OutlinedButton.icon(
            icon: const Icon(Icons.event),
            label: Text('Tenggat: ${_tenggat.day}/${_tenggat.month}/${_tenggat.year}'),
            onPressed: _pilihTanggal,
          ),
        ],
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(context), child: const Text('Batal')),
        FilledButton(onPressed: _simpan, child: const Text('Simpan')),
      ],
    );
  }
}
```



Dialog tambah tugas



Dua tugas tersimpan, satu sudah dicentang selesai

Penjelasan baris penting:

- Setelah setiap perubahan kita memanggil `_muat()` agar list selalu sama dengan isi database (*single source of truth*).
- Form dibuat sebagai widget terpisah `TugasDialog` yang mengembalikan objek `Tugas` lewat `Navigator.pop(context, data)`. Controller dimiliki dialog sehingga di-dispose setelah dialog benar-benar tertutup.
- Fitur URUNGKAN memasukkan kembali objek lama **beserta id-nya**, sehingga posisinya tetap sama.
- Pada `Dismissible`, item dihapus dari list dulu (`_daftar.remove`) karena Flutter mewajibkan item yang sudah di-dismiss segera hilang dari tree.

> ⚠️ **Kesalahan Umum**
>
> - `db.update('tugas', map)` tanpa `where`: seluruh tugas ikut berubah.
> - Lupa memuat ulang data setelah insert/update sehingga layar tidak berubah walaupun database sudah berubah.
> - Membuat `TextEditingController` di method lalu langsung `dispose()` setelah `showDialog` selesai: dialog masih beranimasi menutup sehingga muncul error *_dependents.isEmpty*. Simpan controller di State widget dialog.
> - Error *A dismissed Dismissible widget is still part of the tree*: item belum dihapus dari list setelah digeser.
> - Error *UNIQUE constraint failed*: menyisipkan id/NIM yang sudah ada. Tangani dengan `try/catch` atau `conflictAlgorithm`.

8. Migrasi Skema: version, onCreate, onUpgrade

Aplikasi terus berkembang. Di versi 2 kamu ingin menambah kolom `prioritas` pada tabel tugas. Masalahnya, pengguna lama sudah punya database versi 1 berisi data penting. Kita tidak boleh menghapusnya; kita harus **memigrasi** struktur tabel. Inilah fungsi `version` dan `onUpgrade`.

Konsep: Kapan Callback Dipanggil?

Kondisi di HP pengguna	Callback yang berjalan
Aplikasi baru dipasang (file belum ada)	`onCreate(db, versiTerbaru)`
File ada, versinya lebih kecil dari `version`	`onUpgrade(db, versiLama, versiBaru)`
File ada, versinya lebih besar (aplikasi di-downgrade)	`onDowngrade` (jika diisi)
Setiap kali database dibuka, sebelum yang lain	`onConfigure(db)`
Setiap kali database selesai dibuka	`onOpen(db)`

Contoh Kecil: Menambah Kolom di Versi 2

```
import 'package:sqflite/sqflite.dart';

Future<Database> buka(String path) {
  return openDatabase(
    path,
    version: 2, // naik dari 1 ke 2
    onCreate: (db, version) async {
      // Pengguna BARU langsung mendapat struktur terbaru
      await db.execute('''
        CREATE TABLE tugas (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          judul TEXT NOT NULL,
          prioritas INTEGER NOT NULL DEFAULT 0
        )
      ''');
    },
    onUpgrade: (db, oldVersion, newVersion) async {
      // Pengguna LAMA (versi 1) cukup ditambah kolomnya, data tetap aman
      if (oldVersion < 2) {
        await db.execute('ALTER TABLE tugas ADD COLUMN prioritas INTEGER NOT NULL DEFAULT 0');
      }
    },
  );
}
```

Aturan emas: `onCreate` **selalu membuat struktur terbaru**, sedangkan `onUpgrade` **berisi langkah perubahan dari setiap versi lama**.

Pola Migrasi Bertahap

Pengguna bisa melompat versi (dari 1 langsung ke 3). Karena itu tulis setiap langkah dengan `if (oldVersion < N)` yang berurutan, **bukan** `if (oldVersion == N)`.

`lib/data/migrasi_database.dart`

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

/// Riwayat skema:
///  v1: tabel tugas (id, judul)
///  v2: + kolom prioritas
///  v3: + tabel catatan
class MigrasiDatabase {
  static const int versiSkema = 3;
  static Database? _db;

  static Future<Database> get database async => _db ??= await _open();

  static Future<Database> _open() async {
    final path = join(await getDatabasesPath(), 'migrasi_demo.db');
    return openDatabase(
      path,
      version: versiSkema,
      onConfigure: (db) async {
        // Aktifkan foreign key setiap kali dibuka (dipakai di section Relasi)
        await db.execute('PRAGMA foreign_keys = ON');
      },
      onCreate: (db, version) async {
        // Struktur TERBARU untuk pengguna baru
        await db.execute('''
          CREATE TABLE tugas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            prioritas INTEGER NOT NULL DEFAULT 0
          )
        ''');
        await _buatTabelCatatan(db);
      },
      onUpgrade: (db, oldVersion, newVersion) async {
        if (oldVersion < 2) {
          await db.execute(
            'ALTER TABLE tugas ADD COLUMN prioritas INTEGER NOT NULL DEFAULT 0',
          );
        }
        if (oldVersion < 3) {
          await _buatTabelCatatan(db);
        }
      },
    );
  }

  static Future<void> _buatTabelCatatan(Database db) async {
```

```
    await db.execute('''
      CREATE TABLE catatan (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        isi TEXT NOT NULL,
        dibuat TEXT NOT NULL
      )
    ''');
  }

  /// Daftar kolom suatu tabel, untuk melihat hasil migrasi
  static Future<List<String>> kolom(String tabel) async {
    final db = await database;
    final info = await db.rawQuery('PRAGMA table_info($tabel)');
    return info.map((r) => '${r['name']} (${r['type']})').toList();
  }
}
```

Penjelasan: fungsi `_buatTabelCatatan` dipakai di dua tempat (onCreate dan onUpgrade) sehingga definisi tabel tidak ditulis dua kali. `PRAGMA table_info` adalah perintah SQLite untuk melihat struktur tabel.

Contoh Lengkap: Melihat Struktur Database

lib/main.dart

```
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/migrasi_database.dart
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart'; // untuk db.getVersion()

import 'data/migrasi_database.dart';

void main() => runApp(const MaterialApp(home: StrukturScreen()));

class StrukturScreen extends StatefulWidget {
  const StrukturScreen({super.key});

  @override
  State<StrukturScreen> createState() => _StrukturScreenState();
}

class _StrukturScreenState extends State<StrukturScreen> {
  // Future disimpan sekali agar query tidak diulang setiap rebuild
  late final Future<Map<String, List<String>>> _future = _muat();

  Future<Map<String, List<String>>> _muat() async {
    final db = await MigrasiDatabase.database;
    return {
      'Versi skema': ['${await db.getVersion()}'],
      'Tabel tugas': await MigrasiDatabase.kolom('tugas'),
      'Tabel catatan': await MigrasiDatabase.kolom('catatan'),
    };
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Hasil Migrasi')),
```

```
      body: FutureBuilder(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.hasError) return Center(child: Text('${snapshot.error}'));
          if (!snapshot.hasData) return const Center(child: CircularProgressIndicator());
          return ListView(
            padding: const EdgeInsets.all(16),
            children: [
              for (final entry in snapshot.data!.entries)
                Card(
                  child: ListTile(
                    title: Text(entry.key),
                    subtitle: Text(entry.value.join('\n')),
                  ),
                ),
            ],
          );
        },
      ),
    );
  }
}
```

Versi skema 3 dengan kolom dan tabel terbaru

Coba eksperimen: ubah `versiSkema` menjadi 1 dan hapus bagian yang belum ada di v1, jalankan (setelah uninstall), lalu kembalikan ke 3 tanpa uninstall. Data lama tetap ada, dan kolom/tabel baru muncul.

💡 Fase Belajar vs Produksi

Saat masih belajar dan belum ada pengguna, cara paling cepat setelah mengubah `onCreate` adalah **uninstall aplikasi** (atau Clear storage) lalu jalankan ulang. Setelah aplikasi dirilis, jangan lakukan itu: selalu naikkan `version` dan tulis `onUpgrade`.

⚠️ Kesalahan Umum

- Mengubah `CREATE TABLE` tanpa menaikkan `version`: pengguna lama mendapat error *no such column*.
- Hot reload/hot restart tidak membuat ulang database. Perubahan skema baru terlihat setelah migrasi atau uninstall.
- Menulis `if (oldVersion == 1)`: pengguna yang melompat dari v1 ke v3 tidak mendapat langkah v2 → v3.
- Menurunkan nomor `version`: memicu `onDowngrade` atau error. Nomor versi hanya boleh naik.
- SQLite hanya mendukung sebagian `ALTER TABLE` (tambah/ganti nama kolom). Untuk mengubah tipe kolom, buat tabel baru, salin data, hapus tabel lama.

9. Query Lanjutan, rawQuery & SQL Injection

Kekuatan utama SQLite dibanding SharedPreferences adalah kemampuan **mencari, memfilter, mengurutkan, dan menghitung** data. Di section ini kita mengenal parameter lengkap `db.query()`, `rawQuery()` untuk SQL bebas, dan cara aman memasukkan input pengguna ke query.

Parameter `db.query()`

```
import 'package:sqflite/sqflite.dart';

Future<List<Map<String, Object?>>> contohQuery(Database db) {
  return db.query(
    'mahasiswa',                        // FROM mahasiswa
    columns: ['nim', 'nama', 'ipk'],    // SELECT nim, nama, ipk  (null = semua kolom)
    where: 'prodi = ? AND ipk >= ?',    // WHERE prodi = ? AND ipk >= ?
    whereArgs: ['Informatika', 3.0],    // nilai untuk tanda ? secara berurutan
    orderBy: 'ipk DESC, nama ASC',      // ORDER BY
    limit: 10,                          // LIMIT 10
    offset: 0,                          // OFFSET 0 (untuk halaman berikutnya: 10, 20, ...)
  );
}
```

⚠️ SQL Injection: Mengapa Wajib Memakai `?` dan `whereArgs`

Bayangkan fitur pencarian yang menyambung input pengguna langsung ke SQL:

```
import 'package:sqflite/sqflite.dart';

// ❌ BERBAHAYA: input pengguna disambung langsung ke string SQL
Future<List<Map<String, Object?>>> cariTidakAman(Database db, String input) {
  return db.rawQuery("SELECT * FROM mahasiswa WHERE nama = '$input'");
}

// ✅ AMAN: nilai dikirim terpisah, tidak pernah dianggap perintah SQL
Future<List<Map<String, Object?>>> cariAman(Database db, String input) {
  return db.rawQuery('SELECT * FROM mahasiswa WHERE nama = ?', [input]);
}
```

Jika pengguna mengetik `x' OR '1'='1`, versi tidak aman menjadi `WHERE nama = 'x' OR '1'='1'` yang **selalu benar**, sehingga semua data bocor. Input seperti `'; DROP TABLE mahasiswa; --` bahkan bisa merusak data. Dengan `?`, input tersebut diperlakukan sebagai teks biasa. Tanda kutip di nama seperti `Ma'ruf` pun tidak membuat error.

Pencarian dengan LIKE

```
import 'package:sqflite/sqflite.dart';

Future<List<Map<String, Object?>>> cariNama(Database db, String kata) {
  return db.query(
    'mahasiswa',
```

```
    where: 'nama LIKE ? OR nim LIKE ?',
    whereArgs: ['%$kata%', '%$kata%'], // % = karakter apa saja, ditaruh di whereArgs
  );
}
```

Tanda `%` ditempel pada **nilai** di `whereArgs`, bukan di string `where`.

`rawQuery` untuk Agregasi

Untuk `COUNT`, `AVG`, `GROUP BY`, dan `JOIN`, SQL mentah lebih jelas:

```dart
import 'package:sqflite/sqflite.dart';

Future<void> statistik(Database db) async {
  // Satu angka: gunakan Sqflite.firstIntValue
  final total = Sqflite.firstIntValue(
    await db.rawQuery('SELECT COUNT(*) FROM mahasiswa'),
  ) ?? 0;

  // Rata-rata IPK per prodi
  final rows = await db.rawQuery('''
    SELECT prodi, COUNT(*) AS jumlah, ROUND(AVG(ipk), 2) AS rata
    FROM mahasiswa
    GROUP BY prodi
    ORDER BY rata DESC
  ''');
  print('Total: $total');
  for (final r in rows) {
    print('${r['prodi']}: ${r['jumlah']} mhs, rata-rata ${r['rata']}');
  }
}
```

Contoh Lengkap: Pencarian & Filter Mahasiswa

`lib/data/mahasiswa_query_db.dart`

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class MahasiswaQueryDb {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'query_demo.db'),
        version: 1,
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE mahasiswa (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              nim TEXT NOT NULL UNIQUE,
              nama TEXT NOT NULL,
              prodi TEXT NOT NULL,
              ipk REAL NOT NULL
            )
          ''');
          // Data contoh
```

```
          const contoh = [
            ['H1D022001', 'Budi Santoso', 'Informatika', 3.45],
            ['H1D022002', 'Siti Aminah', 'Informatika', 3.82],
            ['H1D022003', "Ma'ruf Hidayat", 'Informatika', 2.95],
            ['H1A022004', 'Dewi Lestari', 'Teknik Elektro', 3.60],
            ['H1A022005', 'Rizky Pratama', 'Teknik Elektro', 3.10],
            ['H1C022006', 'Andi Wijaya', 'Teknik Sipil', 3.25],
          ];
          for (final m in contoh) {
            await db.insert('mahasiswa', {'nim': m[0], 'nama': m[1], 'prodi': m[2], 'ipk': m[3]});
          }
        },
      );

  /// Gabungan pencarian + filter prodi + filter IPK minimum
  static Future<List<Map<String, Object?>>> cari({
    String kata = '',
    String? prodi,
    double ipkMin = 0,
  }) async {
    final db = await database;
    final kondisi = <String>['ipk >= ?'];
    final args = <Object?>[ipkMin];
    if (kata.isNotEmpty) {
      kondisi.add('(nama LIKE ? OR nim LIKE ?)');
      args.addAll(['%$kata%', '%$kata%']);
    }
    if (prodi != null) {
      kondisi.add('prodi = ?');
      args.add(prodi);
    }
    return db.query(
      'mahasiswa',
      where: kondisi.join(' AND '),
      whereArgs: args,
      orderBy: 'ipk DESC',
    );
  }

  static Future<List<String>> daftarProdi() async {
    final db = await database;
    final rows = await db.rawQuery('SELECT DISTINCT prodi FROM mahasiswa ORDER BY prodi');
    return rows.map((r) => r['prodi'] as String).toList();
  }

  static Future<double> rataIpk() async {
    final db = await database;
    final rows = await db.rawQuery('SELECT AVG(ipk) AS rata FROM mahasiswa');
    return (rows.first['rata'] as num? ?? 0).toDouble();
  }
}
```

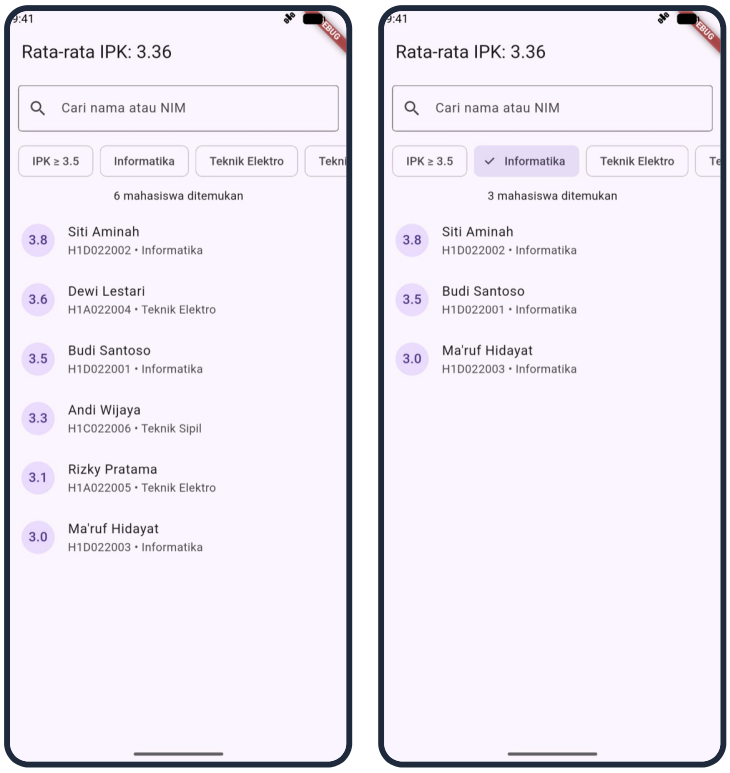
Penjelasan: fungsi `cari` menyusun klausa `WHERE` secara dinamis. String `where` hanya berisi nama kolom dan `?` yang kita tulis sendiri, sedangkan semua nilai dari pengguna masuk ke `args`. Itulah cara aman membuat filter dinamis.

lib/main.dart

```dart
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/mahasiswa_query_db.dart
import 'package:flutter/material.dart';

import 'data/mahasiswa_query_db.dart';

void main() => runApp(const MaterialApp(home: CariMahasiswaScreen()));

class CariMahasiswaScreen extends StatefulWidget {
  const CariMahasiswaScreen({super.key});

  @override
  State<CariMahasiswaScreen> createState() => _CariMahasiswaScreenState();
}

class _CariMahasiswaScreenState extends State<CariMahasiswaScreen> {
  String _kata = '';
  String? _prodi;
  bool _cumlaude = false;
  List<String> _semuaProdi = [];
  List<Map<String, Object?>> _hasil = [];
  double _rata = 0;

  @override
  void initState() {
    super.initState();
    _init();
  }

  Future<void> _init() async {
    final prodi = await MahasiswaQueryDb.daftarProdi();
    final rata = await MahasiswaQueryDb.rataIpk();
    if (!mounted) return;
    setState(() {
      _semuaProdi = prodi;
      _rata = rata;
    });
    await _cari();
  }

  Future<void> _cari() async {
    final hasil = await MahasiswaQueryDb.cari(
      kata: _kata,
      prodi: _prodi,
      ipkMin: _cumlaude ? 3.5 : 0,
    );
    if (mounted) setState(() => _hasil = hasil);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Rata-rata IPK: ${_rata.toStringAsFixed(2)}')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: TextField(
```

```
            decoration: const InputDecoration(
              prefixIcon: Icon(Icons.search),
              hintText: 'Cari nama atau NIM',
              border: OutlineInputBorder(),
            ),
            onChanged: (v) {
              _kata = v.trim();
              _cari();
            },
          ),
        ),
        SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: 12),
          child: Row(
            children: [
              FilterChip(
                label: const Text('IPK ≥ 3.5'),
                selected: _cumlaude,
                onSelected: (v) {
                  setState(() => _cumlaude = v);
                  _cari();
                },
              ),
              for (final p in _semuaProdi)
                Padding(
                  padding: const EdgeInsets.only(left: 8),
                  child: ChoiceChip(
                    label: Text(p),
                    selected: _prodi == p,
                    onSelected: (v) {
                      setState(() => _prodi = v ? p : null);
                      _cari();
                    },
                  ),
                ),
            ],
          ),
        ),
        Padding(
          padding: const EdgeInsets.all(8),
          child: Text('${_hasil.length} mahasiswa ditemukan'),
        ),
        Expanded(
          child: ListView.builder(
            itemCount: _hasil.length,
            itemBuilder: (context, i) {
              final m = _hasil[i];
              return ListTile(
                leading: CircleAvatar(child: Text((m['ipk'] as num).toStringAsFixed(1))),
                title: Text(m['nama'] as String),
                subtitle: Text('${m['nim']} • ${m['prodi']}'),
              );
            },
          ),
        ),
      ],
    ),
  );
```

```
  }
}
```



Tampilan awal diurutkan berdasarkan IPK

Filter IPK di atas 3.5 dan prodi Informatika

Coba ketik `ma'ruf` di kolom pencarian: tetap berjalan normal karena memakai `whereArgs`. Dengan penyambungan string, tanda kutip itu akan membuat SQL error.

> ⚠️ **Kesalahan Umum**
>
> - Menyambung input ke SQL dengan `'$input'` (SQL injection).
> - Jumlah `?` tidak sama dengan jumlah elemen `whereArgs` : error *bind or column index out of range*.
> - Menulis `where: "nama LIKE '%?%'"`. Tanda `?` di dalam kutip tidak dianggap parameter; letakkan `%` di nilai.
> - Mengambil semua data lalu memfilter di Dart dengan `where()` untuk ribuan baris. Biarkan SQLite yang memfilter, jauh lebih cepat.

10. Transaksi & Batch

Beberapa operasi harus berhasil **semua atau tidak sama sekali**. Contoh pengisian KRS: menambah baris KRS *dan* mengurangi kuota kelas. Jika langkah kedua gagal, langkah pertama harus dibatalkan. Inilah **transaksi**. Selain itu, ada **batch** untuk menjalankan banyak perintah sekaligus dengan cepat.

Konsep

	`db.transaction`	`db.batch`
Tujuan	Keutuhan data (atomik)	Kecepatan banyak perintah
Jika ada error	Semua perubahan di dalamnya dibatalkan (rollback)	Jika di-commit tanpa transaksi, perintah sebelumnya bisa tetap tersimpan
Bisa membaca hasil di tengah?	Ya (`await txn.query`)	Tidak, hasil didapat setelah `commit()`
Contoh	Transfer saldo, ambil kuota KRS	Impor 1.000 data awal

Contoh Kecil: Transaksi

```dart
import 'package:sqflite/sqflite.dart';

Future<void> transfer(Database db, int dari, int ke, int jumlah) async {
  await db.transaction((txn) async {
    // PENTING: di dalam transaksi gunakan txn, BUKAN db
    await txn.rawUpdate('UPDATE dompet SET saldo = saldo - ? WHERE id = ?', [jumlah, dari]);
    final rows = await txn.query('dompet', columns: ['saldo'], where: 'id = ?', whereArgs: [dari]);
    if ((rows.first['saldo'] as int) < 0) {
      throw Exception('Saldo tidak cukup'); // melempar = ROLLBACK otomatis
    }
    await txn.rawUpdate('UPDATE dompet SET saldo = saldo + ? WHERE id = ?', [jumlah, ke]);
  });
}
```

Contoh Kecil: Batch

```dart
import 'package:sqflite/sqflite.dart';

Future<void> imporNilai(Database db, List<Map<String, Object?>> data) async {
  await db.transaction((txn) async {
    final batch = txn.batch();
    for (final row in data) {
      batch.insert('nilai', row); // belum dijalankan, hanya dikumpulkan
    }
    await batch.commit(noResult: true); // dijalankan sekaligus
```

```
  });
}
```

Batch di dalam transaksi adalah kombinasi terbaik: cepat **dan** aman. `noResult: true` membuat lebih cepat lagi karena kita tidak butuh id setiap baris.

Contoh Lengkap: Simulasi Pengisian KRS

`lib/data/krs_database.dart`

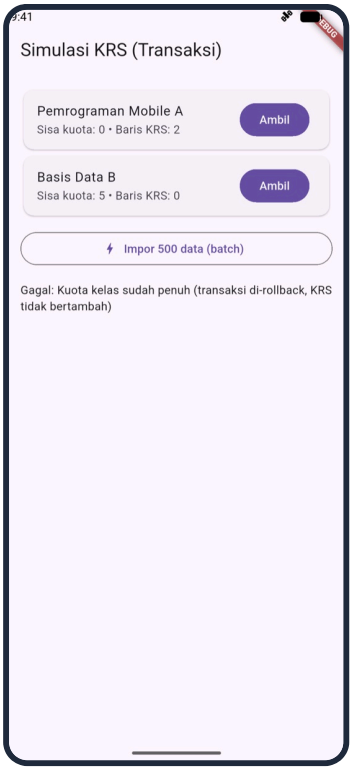
```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class KrsDatabase {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'krs_demo.db'),
        version: 1,
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE kelas (
              id INTEGER PRIMARY KEY,
              nama TEXT NOT NULL,
              kuota INTEGER NOT NULL
            )
          ''');
          await db.execute('''
            CREATE TABLE krs (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              kelas_id INTEGER NOT NULL,
              waktu TEXT NOT NULL
            )
          ''');
          await db.insert('kelas', {'id': 1, 'nama': 'Pemrograman Mobile A', 'kuota': 2});
          await db.insert('kelas', {'id': 2, 'nama': 'Basis Data B', 'kuota': 5});
        },
      );

  /// Ambil kelas: tambah KRS + kurangi kuota, dalam SATU transaksi.
  static Future<void> ambilKelas(int kelasId) async {
    final db = await database;
    await db.transaction((txn) async {
      await txn.insert('krs', {
        'kelas_id': kelasId,
        'waktu': DateTime.now().toIso8601String(),
      });
      final rows = await txn.query('kelas', where: 'id = ?', whereArgs: [kelasId]);
      final kuota = rows.first['kuota'] as int;
      if (kuota <= 0) {
        // Insert KRS di atas ikut dibatalkan karena exception ini
        throw StateError('Kuota kelas sudah penuh');
      }
      await txn.update('kelas', {'kuota': kuota - 1}, where: 'id = ?', whereArgs: [kelasId]);
    });
  }
```

```
  /// Isi 500 baris KRS dummy memakai batch.
  static Future<Duration> imporDummy() async {
    final db = await database;
    final sw = Stopwatch()..start();
    await db.transaction((txn) async {
      final batch = txn.batch();
      for (var i = 0; i < 500; i++) {
        batch.insert('krs', {'kelas_id': 2, 'waktu': 'dummy-$i'});
      }
      await batch.commit(noResult: true);
    });
    return sw.elapsed;
  }

  static Future<List<Map<String, Object?>>> ringkasan() async {
    final db = await database;
    return db.rawQuery('''
      SELECT k.id, k.nama, k.kuota, COUNT(r.id) AS terisi
      FROM kelas k LEFT JOIN krs r ON r.kelas_id = k.id
      GROUP BY k.id
    ''');
  }
}
```

lib/main.dart

```
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/krs_database.dart
import 'package:flutter/material.dart';

import 'data/krs_database.dart';

void main() => runApp(const MaterialApp(home: KrsScreen()));

class KrsScreen extends StatefulWidget {
  const KrsScreen({super.key});

  @override
  State<KrsScreen> createState() => _KrsScreenState();
}

class _KrsScreenState extends State<KrsScreen> {
  List<Map<String, Object?>> _kelas = [];
  String _log = 'Tekan "Ambil" berkali-kali sampai kuota habis.';

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final data = await KrsDatabase.ringkasan();
    if (mounted) setState(() => _kelas = data);
  }

  Future<void> _ambil(int id) async {
    try {
      await KrsDatabase.ambilKelas(id);
```

```
      _log = 'Berhasil mengambil kelas $id';
    } on StateError catch (e) {
      _log = 'Gagal: ${e.message} (transaksi di-rollback, KRS tidak bertambah)';
    }
    await _muat();
  }

  Future<void> _impor() async {
    final durasi = await KrsDatabase.imporDummy();
    _log = '500 baris dimasukkan dengan batch dalam ${durasi.inMilliseconds} ms';
    await _muat();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Simulasi KRS (Transaksi)')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          for (final k in _kelas)
            Card(
              child: ListTile(
                title: Text(k['nama'] as String),
                subtitle: Text('Sisa kuota: ${k['kuota']} • Baris KRS: ${k['terisi']}'),
                trailing: FilledButton(
                  onPressed: () => _ambil(k['id'] as int),
                  child: const Text('Ambil'),
                ),
              ),
            ),
          const SizedBox(height: 12),
          OutlinedButton.icon(
            onPressed: _impor,
            icon: const Icon(Icons.bolt),
            label: const Text('Impor 500 data (batch)'),
          ),
          const SizedBox(height: 16),
          Text(_log),
        ],
      ),
    );
  }
}
```



Setelah Ambil ditekan 3 kali: kuota 0, baris KRS tetap 2

Penjelasan: pada kelas "Pemrograman Mobile A" (kuota 2), tekan Ambil tiga kali. Pada percobaan ketiga, insert ke tabel `krs` sebenarnya sudah dijalankan, tetapi karena exception dilempar sebelum transaksi selesai, SQLite membatalkan insert tersebut. Angka "Baris KRS" tetap 2.

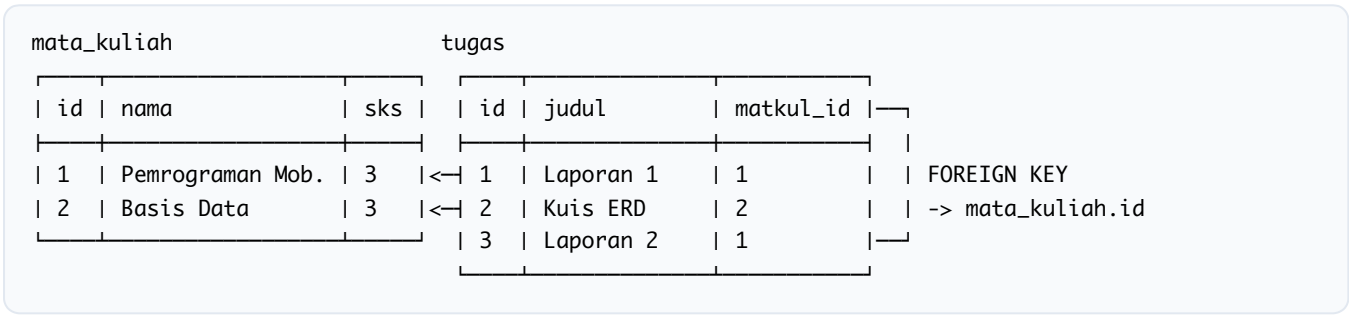
⚠️ Kesalahan Umum

- **Memakai `db` di dalam `db.transaction`**: aplikasi macet (*deadlock*) karena `db` menunggu transaksi selesai. Selalu pakai `txn`.
- Menangkap exception di dalam blok transaksi (try/catch di dalamnya) sehingga rollback tidak terjadi. Biarkan exception keluar dari callback.
- Insert ribuan baris satu per satu dengan `await db.insert` di loop. Gunakan batch.
- Lupa `await batch.commit()`: tidak ada data yang tersimpan.

11. Relasi Tabel: Foreign Key & JOIN

Database relasional bisa menghubungkan tabel. Contoh: satu **mata kuliah** punya banyak **tugas** (relasi *one-to-many*). Daripada menulis nama mata kuliah berulang di setiap tugas, tabel tugas cukup menyimpan `matkul_id` yang menunjuk ke tabel mata kuliah.

Rancangan Tabel



Konsep Penting

- `FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id)` memastikan tugas hanya bisa menunjuk mata kuliah yang ada.
- `ON DELETE CASCADE`: jika mata kuliah dihapus, semua tugasnya ikut terhapus otomatis.
- SQLite **mematikan** foreign key secara default. Aktifkan dengan `PRAGMA foreign_keys = ON` di `onConfigure` (dijalankan setiap database dibuka).
- `JOIN` menggabungkan baris dari dua tabel dalam satu query.

Contoh Kecil: Skema dan JOIN

```
CREATE TABLE mata_kuliah (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nama TEXT NOT NULL,
  sks INTEGER NOT NULL
);

CREATE TABLE tugas (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  judul TEXT NOT NULL,
  matkul_id INTEGER NOT NULL,
  FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id) ON DELETE CASCADE
);

-- Tugas beserta nama mata kuliahnya
SELECT t.id, t.judul, m.nama AS matkul_nama
FROM tugas t
JOIN mata_kuliah m ON m.id = t.matkul_id
ORDER BY m.nama;
```

Model untuk Hasil JOIN

Hasil JOIN berisi kolom dari dua tabel, jadi kita buat model khusus untuk tampilan:

lib/models/relasi_models.dart

```dart
class MataKuliah {
  final int? id;
  final String nama;
  final int sks;
  final int jumlahTugas; // hasil COUNT, bukan kolom tabel

  const MataKuliah({this.id, required this.nama, required this.sks, this.jumlahTugas = 0});

  Map<String, Object?> toMap() => {if (id != null) 'id': id, 'nama': nama, 'sks': sks};

  factory MataKuliah.fromMap(Map<String, Object?> m) => MataKuliah(
        id: m['id'] as int?,
        nama: m['nama'] as String,
        sks: m['sks'] as int,
        jumlahTugas: m['jumlah_tugas'] as int? ?? 0,
      );
}

class TugasDenganMatkul {
  final int id;
  final String judul;
  final String matkulNama;

  const TugasDenganMatkul({required this.id, required this.judul, required this.matkulNama});

  factory TugasDenganMatkul.fromMap(Map<String, Object?> m) => TugasDenganMatkul(
        id: m['id'] as int,
        judul: m['judul'] as String,
        matkulNama: m['matkul_nama'] as String, // alias dari "AS matkul_nama"
      );
}
```

Database Helper dengan Relasi

lib/data/relasi_database.dart

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import '../models/relasi_models.dart';

class RelasiDatabase {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'relasi_demo.db'),
        version: 1,
        onConfigure: (db) => db.execute('PRAGMA foreign_keys = ON'),
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE mata_kuliah (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              nama TEXT NOT NULL,
              sks INTEGER NOT NULL
            )
```

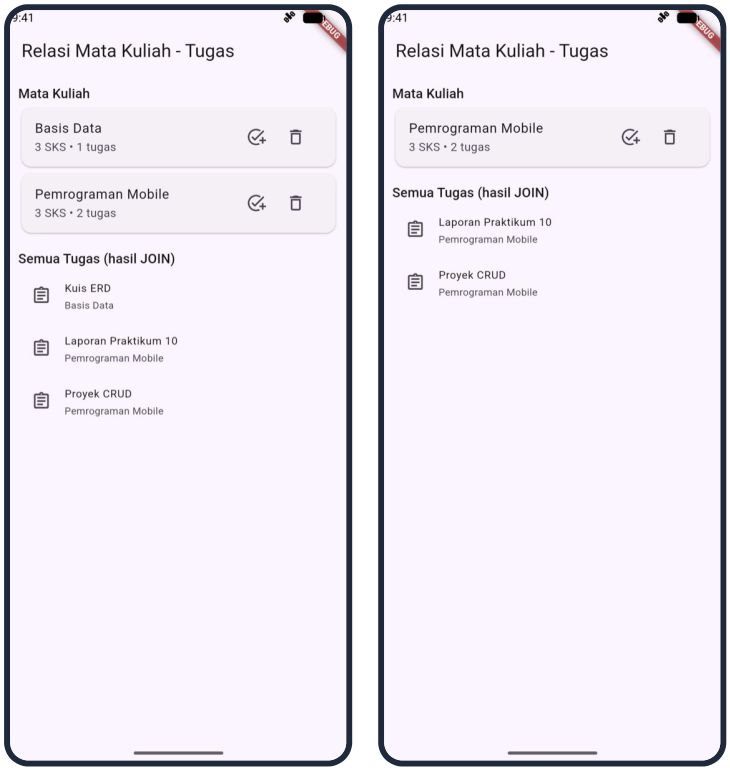
```
          ''');
          await db.execute('''
            CREATE TABLE tugas (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              judul TEXT NOT NULL,
              matkul_id INTEGER NOT NULL,
              FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id) ON DELETE CASCADE
            )
          ''');
          // Data awal
          final mobile = await db.insert('mata_kuliah', {'nama': 'Pemrograman Mobile', 'sks': 3});
          final basdat = await db.insert('mata_kuliah', {'nama': 'Basis Data', 'sks': 3});
          await db.insert('tugas', {'judul': 'Laporan Praktikum 10', 'matkul_id': mobile});
          await db.insert('tugas', {'judul': 'Proyek CRUD', 'matkul_id': mobile});
          await db.insert('tugas', {'judul': 'Kuis ERD', 'matkul_id': basdat});
        },
      );

  /// Mata kuliah + jumlah tugasnya (LEFT JOIN agar matkul tanpa tugas tetap muncul)
  static Future<List<MataKuliah>> semuaMatkul() async {
    final db = await database;
    final rows = await db.rawQuery('''
      SELECT m.id, m.nama, m.sks, COUNT(t.id) AS jumlah_tugas
      FROM mata_kuliah m
      LEFT JOIN tugas t ON t.matkul_id = m.id
      GROUP BY m.id
      ORDER BY m.nama
    ''');
    return rows.map(MataKuliah.fromMap).toList();
  }

  /// Semua tugas beserta nama mata kuliahnya (INNER JOIN)
  static Future<List<TugasDenganMatkul>> semuaTugas() async {
    final db = await database;
    final rows = await db.rawQuery('''
      SELECT t.id, t.judul, m.nama AS matkul_nama
      FROM tugas t
      JOIN mata_kuliah m ON m.id = t.matkul_id
      ORDER BY m.nama, t.id
    ''');
    return rows.map(TugasDenganMatkul.fromMap).toList();
  }

  static Future<void> tambahTugas(String judul, int matkulId) async {
    final db = await database;
    await db.insert('tugas', {'judul': judul, 'matkul_id': matkulId});
  }

  /// Menghapus matkul: tugas-tugasnya ikut terhapus berkat ON DELETE CASCADE
  static Future<void> hapusMatkul(int id) async {
    final db = await database;
    await db.delete('mata_kuliah', where: 'id = ?', whereArgs: [id]);
  }
}
```

Contoh Lengkap: Mata Kuliah dan Tugasnya

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add sqflite path, serta membuat
// lib/models/relasi_models.dart dan lib/data/relasi_database.dart
import 'package:flutter/material.dart';

import 'data/relasi_database.dart';
import 'models/relasi_models.dart';

void main() => runApp(const MaterialApp(home: RelasiScreen()));

class RelasiScreen extends StatefulWidget {
  const RelasiScreen({super.key});

  @override
  State<RelasiScreen> createState() => _RelasiScreenState();
}

class _RelasiScreenState extends State<RelasiScreen> {
  List<MataKuliah> _matkul = [];
  List<TugasDenganMatkul> _tugas = [];

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final matkul = await RelasiDatabase.semuaMatkul();
    final tugas = await RelasiDatabase.semuaTugas();
    if (!mounted) return;
    setState(() {
      _matkul = matkul;
      _tugas = tugas;
    });
  }

  @override
  Widget build(BuildContext context) {
    final judulStyle = Theme.of(context).textTheme.titleMedium;
    return Scaffold(
      appBar: AppBar(title: const Text('Relasi Mata Kuliah - Tugas')),
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          Text('Mata Kuliah', style: judulStyle),
          for (final m in _matkul)
            Card(
              child: ListTile(
                title: Text(m.nama),
                subtitle: Text('${m.sks} SKS • ${m.jumlahTugas} tugas'),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    IconButton(
```

```
                    tooltip: 'Tambah tugas',
                    icon: const Icon(Icons.add_task),
                    onPressed: () async {
                      await RelasiDatabase.tambahTugas('Tugas baru ${m.nama}', m.id!);
                      await _muat();
                    },
                  ),
                  IconButton(
                    tooltip: 'Hapus matkul',
                    icon: const Icon(Icons.delete_outline),
                    onPressed: () async {
                      await RelasiDatabase.hapusMatkul(m.id!);
                      await _muat();
                    },
                  ),
                ],
              ),
            ),
          ),
        ),
        const SizedBox(height: 16),
        Text('Semua Tugas (hasil JOIN)', style: judulStyle),
        for (final t in _tugas)
          ListTile(
            dense: true,
            leading: const Icon(Icons.assignment_outlined),
            title: Text(t.judul),
            subtitle: Text(t.matkulNama),
          ),
      ],
    ),
  );
  }
}
```



Data awal: matkul dan hasil JOIN

Basis Data dihapus: tugas Kuis ERD ikut hilang (CASCADE)

Penjelasan: tekan ikon hapus pada "Basis Data". Matkul hilang **dan** tugas "Kuis ERD" juga hilang dari daftar JOIN, padahal kita hanya menjalankan satu `DELETE` pada tabel `mata_kuliah`. Itulah efek `ON DELETE CASCADE`.

⚠️ Kesalahan Umum

- Lupa `PRAGMA foreign_keys = ON`: CASCADE tidak berjalan dan tugas "yatim" (menunjuk matkul yang sudah tidak ada) tetap tersimpan.
- Memakai `JOIN` biasa saat ingin menampilkan matkul yang belum punya tugas. Gunakan `LEFT JOIN`.
- Dua tabel punya kolom bernama sama (`id`, `nama`). Gunakan alias (`m.nama AS matkul_nama`) agar kunci Map tidak bertabrakan.
- Melakukan query per item di dalam loop (N+1 query). Satu query JOIN jauh lebih efisien.

12. Studi Kasus: Aplikasi Catatan Kuliah Multi-File

Kita gabungkan semuanya menjadi aplikasi **Catatan Kuliah** yang sepenuhnya offline: tambah, edit, hapus (dengan urungkan), sematkan (pin), cari, dan filter kategori. Arsitekturnya sama seperti Modul 09, hanya sumber datanya SQLite: **Model → Database → Provider → Screen**.

Struktur Folder

```
lib/
├── main.dart
├── models/
|   └── catatan.dart
├── data/
|   └── catatan_database.dart
├── providers/
|   └── catatan_provider.dart
└── screens/
    ├── catatan_list_screen.dart
    └── catatan_form_screen.dart
```

```
flutter pub add sqflite path provider
```

1. Model

`lib/models/catatan.dart`

```dart
class Catatan {
  static const kategoriList = ['Kuliah', 'Tugas', 'Organisasi', 'Pribadi'];

  final int? id;
  final String judul;
  final String isi;
  final String kategori;
  final bool disematkan;
  final DateTime diubah;

  const Catatan({
    this.id,
    required this.judul,
    required this.isi,
    this.kategori = 'Kuliah',
    this.disematkan = false,
    required this.diubah,
  });

  Map<String, Object?> toMap() => {
        if (id != null) 'id': id,
        'judul': judul,
        'isi': isi,
        'kategori': kategori,
        'disematkan': disematkan ? 1 : 0,
        'diubah': diubah.toIso8601String(),
```

```
      };

  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
        id: m['id'] as int?,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
        kategori: m['kategori'] as String,
        disematkan: m['disematkan'] == 1,
        diubah: DateTime.parse(m['diubah'] as String),
      );

  Catatan copyWith({String? judul, String? isi, String? kategori, bool? disematkan, DateTime?
diubah}) {
    return Catatan(
      id: id,
      judul: judul ?? this.judul,
      isi: isi ?? this.isi,
      kategori: kategori ?? this.kategori,
      disematkan: disematkan ?? this.disematkan,
      diubah: diubah ?? this.diubah,
    );
  }
}
```

2. Lapisan Database

`lib/data/catatan_database.dart`

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import '../models/catatan.dart';

class CatatanDatabase {
  CatatanDatabase._();
  static final CatatanDatabase instance = CatatanDatabase._();

  static const _tabel = 'catatan';
  Database? _db;

  Future<Database> get _database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'catatan_kuliah.db'),
        version: 1,
        onCreate: (db, version) => db.execute('''
          CREATE TABLE $_tabel (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            isi TEXT NOT NULL,
            kategori TEXT NOT NULL,
            disematkan INTEGER NOT NULL DEFAULT 0,
            diubah TEXT NOT NULL
          )
        '''),
      );

  /// Ambil catatan: yang disematkan di atas, lalu yang terbaru.
  Future<List<Catatan>> getAll({String kata = '', String? kategori}) async {
    final db = await _database;
```

```
    final where = <String>[];
    final args = <Object?>[];
    if (kata.isNotEmpty) {
      where.add('(judul LIKE ? OR isi LIKE ?)');
      args.addAll(['%$kata%', '%$kata%']);
    }
    if (kategori != null) {
      where.add('kategori = ?');
      args.add(kategori);
    }
    final rows = await db.query(
      _tabel,
      where: where.isEmpty ? null : where.join(' AND '),
      whereArgs: args.isEmpty ? null : args,
      orderBy: 'disematkan DESC, diubah DESC',
    );
    return rows.map(Catatan.fromMap).toList();
  }

  Future<int> insert(Catatan c) async => (await _database).insert(_tabel, c.toMap());

  Future<void> update(Catatan c) async {
    final db = await _database;
    await db.update(_tabel, c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }

  Future<void> delete(int id) async {
    final db = await _database;
    await db.delete(_tabel, where: 'id = ?', whereArgs: [id]);
  }
}
```

3. Provider

lib/providers/catatan_provider.dart

```
import 'package:flutter/foundation.dart';

import '../data/catatan_database.dart';
import '../models/catatan.dart';

class CatatanProvider extends ChangeNotifier {
  final CatatanDatabase _db;
  CatatanProvider(this._db);

  List<Catatan> _items = [];
  bool _loading = true;
  String _kata = '';
  String? _kategori;

  List<Catatan> get items => List.unmodifiable(_items);
  bool get loading => _loading;
  String? get kategori => _kategori;
  bool get sedangMemfilter => _kata.isNotEmpty || _kategori != null;

  Future<void> muat() async {
    _items = await _db.getAll(kata: _kata, kategori: _kategori);
    _loading = false;
```

```
    notifyListeners();
  }

  Future<void> cari(String kata) {
    _kata = kata.trim();
    return muat();
  }

  Future<void> pilihKategori(String? kategori) {
    _kategori = kategori;
    return muat();
  }

  Future<void> simpan(Catatan c) async {
    final baru = c.copyWith(diubah: DateTime.now());
    if (c.id == null) {
      await _db.insert(baru);
    } else {
      await _db.update(baru);
    }
    await muat();
  }

  Future<void> toggleSemat(Catatan c) async {
    await _db.update(c.copyWith(disematkan: !c.disematkan));
    await muat();
  }

  Future<void> hapus(Catatan c) async {
    _items.remove(c); // hilang dari layar seketika (dibutuhkan Dismissible)
    notifyListeners();
    await _db.delete(c.id!);
  }

  /// Urungkan hapus: masukkan lagi lengkap dengan id lamanya
  Future<void> pulihkan(Catatan c) async {
    await _db.insert(c);
    await muat();
  }
}
```

4. Layar Daftar

`lib/screens/catatan_list_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';
import 'catatan_form_screen.dart';

class CatatanListScreen extends StatelessWidget {
  const CatatanListScreen({super.key});

  void _bukaForm(BuildContext context, [Catatan? c]) {
    Navigator.push(context, MaterialPageRoute(builder: (_) => CatatanFormScreen(catatan: c)));
  }
```

```dart
@override
Widget build(BuildContext context) {
  final prov = context.watch<CatatanProvider>();

  return Scaffold(
    appBar: AppBar(title: const Text('Catatan Kuliah')),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.fromLTRB(12, 12, 12, 4),
          child: SearchBar(
            hintText: 'Cari judul atau isi catatan',
            leading: const Icon(Icons.search),
            onChanged: context.read<CatatanProvider>().cari,
          ),
        ),
        SizedBox(
          height: 48,
          child: ListView(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: 12),
            children: [
              ChoiceChip(
                label: const Text('Semua'),
                selected: prov.kategori == null,
                onSelected: (_) => prov.pilihKategori(null),
              ),
              for (final k in Catatan.kategoriList)
                Padding(
                  padding: const EdgeInsets.only(left: 8),
                  child: ChoiceChip(
                    label: Text(k),
                    selected: prov.kategori == k,
                    onSelected: (v) => prov.pilihKategori(v ? k : null),
                  ),
                ),
            ],
          ),
        ),
        Expanded(
          child: prov.loading
              ? const Center(child: CircularProgressIndicator())
              : prov.items.isEmpty
                  ? Center(
                      child: Text(prov.sedangMemfilter
                          ? 'Tidak ada catatan yang cocok.'
                          : 'Belum ada catatan. Tekan + untuk menulis.'),
                    )
                  : ListView.builder(
                      padding: const EdgeInsets.only(bottom: 80),
                      itemCount: prov.items.length,
                      itemBuilder: (context, i) => _CatatanTile(
                        catatan: prov.items[i],
                        onTap: () => _bukaForm(context, prov.items[i]),
                      ),
                    ),
        ),
      ],
```

```
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _bukaForm(context),
        icon: const Icon(Icons.edit_note),
        label: const Text('Tulis'),
      ),
    );
  }
}

class _CatatanTile extends StatelessWidget {
  final Catatan catatan;
  final VoidCallback onTap;
  const _CatatanTile({required this.catatan, required this.onTap});

  @override
  Widget build(BuildContext context) {
    final prov = context.read<CatatanProvider>();
    final d = catatan.diubah;
    return Dismissible(
      key: ValueKey(catatan.id),
      direction: DismissDirection.endToStart,
      background: Container(
        color: Colors.red,
        alignment: Alignment.centerRight,
        padding: const EdgeInsets.only(right: 20),
        child: const Icon(Icons.delete, color: Colors.white),
      ),
      onDismissed: (_) {
        final messenger = ScaffoldMessenger.of(context);
        prov.hapus(catatan);
        messenger.showSnackBar(SnackBar(
          content: Text('"${catatan.judul}" dihapus'),
          action: SnackBarAction(label: 'URUNGKAN', onPressed: () => prov.pulihkan(catatan)),
        ));
      },
      child: Card(
        margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
        child: ListTile(
          onTap: onTap,
          title: Text(catatan.judul, maxLines: 1, overflow: TextOverflow.ellipsis),
          subtitle: Text(
            '${catatan.kategori} • ${d.day}/${d.month}/${d.year}\n${catatan.isi}',
            maxLines: 3,
            overflow: TextOverflow.ellipsis,
          ),
          isThreeLine: true,
          trailing: IconButton(
            tooltip: catatan.disematkan ? 'Lepas sematan' : 'Sematkan',
            icon: Icon(catatan.disematkan ? Icons.push_pin : Icons.push_pin_outlined),
            onPressed: () => prov.toggleSemat(catatan),
          ),
        ),
      ),
    );
  }
}
```

5. Layar Form (Tambah & Edit)

`lib/screens/catatan_form_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';

class CatatanFormScreen extends StatefulWidget {
  final Catatan? catatan; // null = catatan baru
  const CatatanFormScreen({super.key, this.catatan});

  @override
  State<CatatanFormScreen> createState() => _CatatanFormScreenState();
}

class _CatatanFormScreenState extends State<CatatanFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul =
      TextEditingController(text: widget.catatan?.judul);
  late final TextEditingController _isi = TextEditingController(text: widget.catatan?.isi);
  late String _kategori = widget.catatan?.kategori ?? Catatan.kategoriList.first;

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final lama = widget.catatan;
    final data = lama == null
        ? Catatan(judul: _judul.text.trim(), isi: _isi.text.trim(), kategori: _kategori, diubah:
DateTime.now())
        : lama.copyWith(judul: _judul.text.trim(), isi: _isi.text.trim(), kategori: _kategori);
    await context.read<CatatanProvider>().simpan(data);
    if (mounted) Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.catatan == null ? 'Catatan Baru' : 'Edit Catatan'),
        actions: [IconButton(tooltip: 'Simpan', onPressed: _simpan, icon: const Icon(Icons.check))],
      ),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(labelText: 'Judul', border: OutlineInputBorder()),
```

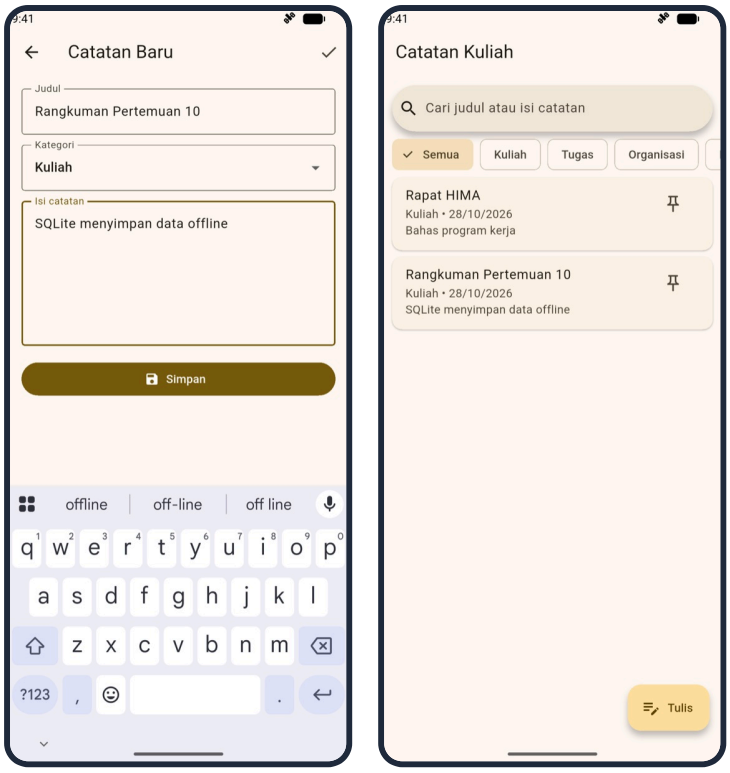
```
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              initialValue: _kategori,
              decoration: const InputDecoration(labelText: 'Kategori', border:
OutlineInputBorder()),
              items: [
                for (final k in Catatan.kategoriList) DropdownMenuItem(value: k, child: Text(k)),
              ],
              onChanged: (v) => setState(() => _kategori = v ?? _kategori),
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              minLines: 6,
              maxLines: 12,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                alignLabelWithHint: true,
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            FilledButton.icon(onPressed: _simpan, icon: const Icon(Icons.save), label: const
Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

6. `main.dart`

`lib/main.dart`

```
// CATATAN: sqflite hanya berjalan di Android, iOS, dan macOS.
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'data/catatan_database.dart';
import 'providers/catatan_provider.dart';
import 'screens/catatan_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CatatanProvider(CatatanDatabase.instance)..muat(),
      child: MaterialApp(
        title: 'Catatan Kuliah',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(colorSchemeSeed: Colors.amber),
        home: const CatatanListScreen(),
      ),
    ),
```

```
    );
  }
```



Form catatan baru | Dua catatan tersimpan di SQLite

Penjelasan alur:

- **Pencarian & filter** tidak dilakukan di Dart, melainkan diteruskan ke SQL (`LIKE` dan `kategori = ?`) lewat `CatatanDatabase.getAll`.
- **Urutan** `disematkan DESC, diubah DESC` membuat catatan yang di-pin selalu di atas, lalu yang paling baru diubah.
- **Hapus** menghapus dari list lebih dulu (kebutuhan `Dismissible`) lalu dari database. URUNGKAN memasukkan kembali objek beserta id lamanya.
- **Form** memakai `late final` controller yang diinisialisasi dari `widget.catatan`, sehingga satu layar melayani tambah dan edit.

> ⚠️ **Kesalahan Umum**
>
> - Lupa `..muat()` saat membuat provider: spinner tidak pernah hilang karena `_loading` tetap `true`.
> - Memanggil `Navigator.pop` sebelum `await simpan(...)` selesai, sehingga daftar belum ter-update saat kembali.
> - Memakai `value:` pada `DropdownButtonFormField`; versi Flutter terbaru memakai `initialValue:`.

13. Menyimpan File dengan path_provider

Kadang data lebih cocok disimpan sebagai **file**: ekspor catatan ke teks, cache JSON dari API, atau foto. Package `path_provider` memberi tahu lokasi folder yang boleh dipakai aplikasi, lalu kita menulis file dengan `dart:io`.

```
flutter pub add path_provider
```

Folder yang Tersedia

Fungsi	Isi yang cocok	Bisa dihapus sistem?
`getApplicationDocumentsDirectory()`	Data buatan pengguna (ekspor, dokumen)	Tidak
`getApplicationSupportDirectory()`	File internal aplikasi yang tidak dilihat pengguna	Tidak
`getTemporaryDirectory()`	Cache sementara (gambar, respons API)	Ya, kapan saja
`getDatabasesPath()` (sqflite)	File database SQLite	Tidak

Contoh Kecil: Tulis dan Baca File Teks

```dart
import 'dart:io';

import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

Future<File> _fileCatatan() async {
  final dir = await getApplicationDocumentsDirectory();
  return File(p.join(dir.path, 'catatan.txt'));
}

Future<void> simpanTeks(String isi) async {
  final file = await _fileCatatan();
  await file.writeAsString(isi); // menimpa isi lama
}

Future<String> bacaTeks() async {
  final file = await _fileCatatan();
  if (!await file.exists()) return ''; // belum pernah disimpan
  return file.readAsString();
}
```

Contoh Lengkap: Buku Harian Sederhana

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add path_provider path
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

void main() => runApp(const MaterialApp(home: BukuHarianScreen()));

class BukuHarianScreen extends StatefulWidget {
  const BukuHarianScreen({super.key});

  @override
  State<BukuHarianScreen> createState() => _BukuHarianScreenState();
}

class _BukuHarianScreenState extends State<BukuHarianScreen> {
  final _ctrl = TextEditingController();
  String _info = 'Memuat...';

  Future<File> get _file async {
    final dir = await getApplicationDocumentsDirectory();
    return File(p.join(dir.path, 'buku_harian.txt'));
  }

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final file = await _file;
    final ada = await file.exists();
    final isi = ada ? await file.readAsString() : '';
    if (!mounted) return;
    setState(() {
      _ctrl.text = isi;
      _info = ada ? 'Dibaca dari ${file.path}' : 'File belum ada';
    });
  }

  Future<void> _simpan() async {
    final file = await _file;
    await file.writeAsString(_ctrl.text);
    final ukuran = await file.length();
    if (!mounted) return;
    setState(() => _info = 'Tersimpan ($ukuran byte) di ${file.path}');
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Buku Harian (File)'),
        actions: [IconButton(tooltip: 'Simpan', onPressed: _simpan, icon: const Icon(Icons.save))],
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Expanded(
              child: TextField(
                controller: _ctrl,
                expands: true,
                maxLines: null,
                textAlignVertical: TextAlignVertical.top,
                decoration: const InputDecoration(
                  hintText: 'Tulis ceritamu hari ini...',
                  border: OutlineInputBorder(),
                ),
              ),
            ),
            const SizedBox(height: 8),
            Text(_info, style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
      ),
    );
  }
}
```

Teks tersimpan ke file di folder dokumen aplikasi

Penjelasan: `File.writeAsString` dan `readAsString` berasal dari `dart:io`. Jalur folder diambil dari `path_provider`, lalu digabung dengan nama file memakai `p.join` agar pemisah folder benar di semua platform.

⚠️ Kesalahan Umum

- Menulis path secara manual (`'/sdcard/...'`): tidak portabel dan sering ditolak izin. Selalu pakai `path_provider`.
- Menyimpan data penting di `getTemporaryDirectory()` : bisa dihapus sistem sewaktu-waktu.
- Membaca file yang belum ada tanpa cek `exists()` : muncul `PathNotFoundException`.
- `dart:io` tidak tersedia di Flutter Web.

14. Perbandingan: SharedPreferences, SQLite, Hive, Supabase

Setelah mencoba SharedPreferences dan SQLite, mari bandingkan dengan dua teknologi yang akan dipelajari berikutnya: **Hive** (Modul 11, NoSQL key-value lokal) dan **Supabase** (Modul 12, database cloud).

Aspek	SharedPreferences	SQLite (sqflite)	Hive (Modul 11)	Supabase (Modul 12)
Lokasi data	Lokal (HP)	Lokal (HP)	Lokal (HP)	Cloud (server PostgreSQL)
Model data	Key-value sederhana	Tabel relasional (SQL)	Key-value NoSQL berisi objek Dart	Tabel relasional (PostgreSQL) via API
Tipe data	bool, int, double, String, List<String>	INTEGER, REAL, TEXT, BLOB	Tipe Dart + objek kustom (adapter)	Semua tipe PostgreSQL, JSON
Query / filter	Tidak ada (ambil per kunci)	Sangat kuat: WHERE, JOIN, GROUP BY	Filter di Dart (`where`)	Kuat: filter, relasi, RPC
Relasi antar data	-	Ya (foreign key)	Manual	Ya (foreign key)
Perlu internet	Tidak	Tidak	Tidak	Ya (untuk sinkron)
Berbagi antar perangkat/pengguna	Tidak	Tidak	Tidak	Ya, plus autentikasi
Dukungan Web	Ya	Tidak (sqflite)	Ya	Ya
Kurva belajar	Sangat mudah	Sedang (perlu SQL & migrasi)	Mudah	Sedang (perlu setup proyek)
Contoh ideal	Tema, onboarding, bahasa	Catatan, transaksi, KRS offline	Cache data, keranjang belanja, favorit	Aplikasi multi-user, login, data bersama

Kombinasi yang Umum di Aplikasi Nyata

- **SharedPreferences** untuk pengaturan + **SQLite/Hive** untuk data offline.
- **Supabase** sebagai sumber data utama + **Hive/SQLite** sebagai cache agar aplikasi tetap bisa dibuka tanpa internet (*offline-first*).

> ℹ️ **Selanjutnya**
>
> **Modul 11 (Hive)** menunjukkan cara menyimpan objek Dart secara langsung tanpa menulis SQL, termasuk di Flutter Web. **Modul 12 (Supabase)** memindahkan data ke cloud sehingga bisa diakses banyak perangkat dan dilengkapi login. Pola Repository yang dipelajari di Modul 09 dan 10 membuat perpindahan ini mudah: cukup ganti implementasi lapisan data.

Tugas Praktikum: Praktikum 10: Aplikasi Tugas Kuliah Offline dengan SQLite & SharedPreferences

Sasaran Praktikum

Membangun aplikasi manajemen tugas kuliah yang sepenuhnya berfungsi tanpa internet, menyimpan pengaturan di SharedPreferences dan data berelasi di SQLite.

Langkah Pengerjaan

1. Buat tabel `mata_kuliah` (id, nama, sks) dan `tugas` (id, judul, matkul_id, tenggat, isSelesai) dengan foreign key `ON DELETE CASCADE` dan `PRAGMA foreign_keys = ON`.
2. Buat model `MataKuliah` dan `Tugas` lengkap dengan `toMap`, `fromMap`, dan `copyWith` (konversi bool dan DateTime).
3. Buat kelas database singleton dengan operasi CRUD bertipe; semua filter wajib memakai `whereArgs`.
4. Tampilkan daftar tugas (hasil JOIN dengan nama mata kuliah) memakai Provider; tambah dan edit melalui satu form dengan validasi dan pemilih tanggal.
5. Centang untuk menandai selesai (UPDATE) dan geser untuk menghapus (DELETE) dengan tombol URUNGKAN.
6. Tambahkan pencarian judul (LIKE) dan filter per mata kuliah, serta ringkasan jumlah tugas per mata kuliah (GROUP BY).
7. Naikkan versi database menjadi 2 dengan menambah kolom `prioritas` melalui `onUpgrade`, lalu buktikan data lama tidak hilang.
8. Simpan pilihan tema (terang/gelap/sistem) dan urutan daftar terakhir di SharedPreferences; tutup aplikasi sepenuhnya lalu buka kembali untuk membuktikan semua data dan pengaturan tetap tersimpan.

Output yang Diharapkan

Aplikasi tugas kuliah offline yang persisten: data tugas dan mata kuliah tersimpan di SQLite (termasuk setelah migrasi versi 2), pengaturan tema tersimpan di SharedPreferences, dan semuanya tetap ada setelah aplikasi di-restart.

Rubrik Penilaian

Desain skema, relasi & migrasi (25%), Model dan lapisan database bertipe (20%), Fungsionalitas CRUD + pencarian/filter aman (30%), SharedPreferences untuk tema (10%), UX: validasi, urungkan, pesan kosong (15%).

Kuis Pemahaman

1. Kapan kita sebaiknya memilih SharedPreferences dibandingkan SQLite?

A. Ketika hanya perlu menyimpan data pengaturan sederhana seperti preferensi tema gelap dan token autentikasi

B. Ketika perlu menyimpan data ribuan transaksi keuangan yang saling berelasi

C. Ketika aplikasi membutuhkan query SELECT yang kompleks dengan filter JOIN

D. SharedPreferences tidak pernah digunakan di industri

2. Mengapa pada SQLite tipe data Boolean biasanya disimpan sebagai tipe INTEGER (0 dan 1)?

A. Karena SQLite tidak memiliki tipe data Boolean terpisah secara natif, sehingga angka 0 mewakili false dan 1 mewakili true

B. Agar data lebih cepat terkirim ke internet

C. Karena Flutter melarang penggunaan kata true di file SQL

D. Karena SQLite hanya menerima angka ganjil

3. Menurut dokumentasi shared_preferences terbaru, API mana yang dianjurkan untuk proyek baru?

A. SharedPreferences.getInstance() (API lama)

B. SharedPreferencesAsync atau SharedPreferencesWithCache

C. localStorage dari dart:html

D. File teks di folder temporary

4. Kamu menambah kolom baru di CREATE TABLE pada onCreate, tetapi di HP yang sudah memasang versi lama muncul error 'no such column'. Mengapa?

A. sqflite tidak mendukung kolom baru

B. onCreate hanya berjalan saat database belum ada; perlu menaikkan version dan menulis ALTER TABLE di onUpgrade

C. Harus memakai hot reload

D. Nama kolom harus huruf kapital

5. Mana cara yang AMAN dari SQL injection untuk mencari mahasiswa berdasarkan input pengguna?

A. db.rawQuery("SELECT * FROM mahasiswa WHERE nama = '$input'")

B. db.query('mahasiswa', where: 'nama = ?', whereArgs: [input])

C. db.execute('SELECT ' + input)

D. db.query('mahasiswa', where: "nama = '$input'")

6. Apa yang terjadi jika sebuah exception dilempar di dalam callback db.transaction?

A. Hanya perintah terakhir yang dibatalkan

B. Semua perubahan di dalam transaksi dibatalkan (rollback)

C. Database terhapus

D. Exception diabaikan dan data tetap tersimpan

7. Mengapa ON DELETE CASCADE tidak berjalan padahal sudah ditulis di CREATE TABLE?

A. SQLite tidak mendukung foreign key sama sekali

B. Foreign key di SQLite nonaktif secara default; harus menjalankan PRAGMA foreign_keys = ON (misalnya di onConfigure)

C. Harus memakai rawDelete

D. CASCADE hanya berjalan di iOS

8. Folder mana yang TIDAK cocok untuk menyimpan data penting pengguna karena bisa dihapus sistem kapan saja?

A. getApplicationDocumentsDirectory()

B. getApplicationSupportDirectory()

C. getTemporaryDirectory()

D. getDatabasesPath()

Kunci Jawaban & Pembahasan

1. A — SharedPreferences dirancang khusus untuk menyimpan pasangan kunci-nilai (key-value) berukuran kecil seperti preferensi pengguna, flags, atau token login. SQLite digunakan saat data berstruktur relasional banyak tabel.

2. A — Engine SQLite tidak memiliki tipe data BOOLEAN khusus tersendiri; nilai boolean disimpan sebagai integer 0 (false) dan 1 (true).

3. B — Sejak versi 2.3.0, SharedPreferences (legacy) direncanakan untuk di-deprecate. Kode baru dianjurkan memakai SharedPreferencesAsync (selalu baca dari disk) atau SharedPreferencesWithCache (baca sinkron dari cache).

4. B — Database yang sudah ada tidak menjalankan onCreate lagi. Perubahan skema untuk pengguna lama dilakukan lewat onUpgrade setelah nomor version dinaikkan.

5. B — Dengan placeholder ? dan whereArgs, nilai dikirim terpisah dari perintah SQL sehingga input seperti x' OR '1'='1 diperlakukan sebagai teks biasa.

6. B — Transaksi bersifat atomik: jika callback melempar exception, sqflite melakukan rollback sehingga tidak ada perubahan yang tersimpan.

7. B — SQLite mematikan penegakan foreign key secara default. PRAGMA foreign_keys = ON perlu dijalankan setiap kali database dibuka, tempat yang tepat adalah onConfigure.

8. C — Folder temporary ditujukan untuk cache; sistem operasi boleh mengosongkannya saat ruang penyimpanan menipis.

MODUL 11 · HIVE NOSQL

Database NoSQL Lokal dengan Hive

Menyimpan data offline dengan Hive Community Edition (hive_ce): box key-value, Map dan List, objek sendiri dengan TypeAdapter manual maupun hasil generate, UI reaktif, pola Repository + Provider, dan enkripsi. Ditutup dengan mini project CRUD Catatan Kuliah Offline lengkap dengan pencarian dan pengurutan.

Durasi Pertemuan 11 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management dengan Provider
Modul 10: Penyimpanan Data Lokal (Storage & SQLite)

Capaian Pembelajaran

- Menjelaskan konsep box key-value pada Hive dan memilih antara SharedPreferences, Hive, dan SQLite sesuai kebutuhan.
- Menginisialisasi Hive dan membuka box dengan benar di main() menggunakan hive_ce_flutter.
- Melakukan operasi put, get, add, delete, keys, values, length, dan clear pada box.
- Menyimpan Map dan List serta mengonversinya dengan aman menjadi objek model (toMap/fromMap).
- Menulis TypeAdapter manual dan membuat adapter otomatis dengan @GenerateAdapters, termasuk aturan typeId dan migrasi field.
- Membangun UI reaktif dengan ValueListenableBuilder serta pola Repository + Provider di atas Hive.
- Mengenkripsi box dengan HiveAesCipher dan menyimpan kuncinya di flutter_secure_storage.
- Membangun aplikasi CRUD offline lengkap (tambah, lihat, edit, hapus, cari, urutkan) dan mengujinya dengan unit test.

Daftar Isi Modul

1. Apa itu Hive? Box, NoSQL, dan Kapan Memakainya
2. Instalasi dan Inisialisasi
3. Operasi Dasar Box
4. Menyimpan Map dan List
5. Objek Sendiri dengan TypeAdapter Manual
6. Generate Adapter Otomatis (Lanjutan)
7. UI Reaktif dengan ValueListenableBuilder
8. Hive + Provider dengan Pola Repository
9. Enkripsi Box
10. Mini Project: Catatan Kuliah Offline (CRUD Lengkap)
11. Kesalahan Umum dan Debugging
12. Tugas Praktikum

13. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Hive? Box, NoSQL, dan Kapan Memakainya

Di Modul 10 kamu sudah memakai **SharedPreferences** untuk data kecil dan **SQLite** untuk data berbentuk tabel. Modul ini mengenalkan pilihan ketiga yang sangat populer di Flutter: **Hive**, database NoSQL yang ringan, cepat, dan ditulis murni dengan Dart.

Hive itu apa?

Hive adalah database *key-value*. Bayangkan sebuah `Map` di Dart yang isinya otomatis tersimpan ke file, sehingga tidak hilang saat aplikasi ditutup. Kamu menyimpan data dengan sebuah **key** (kunci) lalu mengambilnya lagi memakai key yang sama.

```
// Map biasa: hilang saat aplikasi ditutup
final dataSementara = <String, dynamic>{};
dataSementara['nama'] = 'Budi';

// Box Hive: cara pakainya mirip Map, tetapi tersimpan permanen ke file
// box.put('nama', 'Budi');
// box.get('nama'); // 'Budi', bahkan setelah aplikasi dibuka ulang
```

Istilah penting: Box

Data di Hive dikelompokkan ke dalam **box**. Satu box kira-kira setara dengan satu tabel di SQLite atau satu "laci" penyimpanan. Contoh pembagiannya:

- `pengaturan`: tema gelap, bahasa, nama pengguna.
- `catatan_box`: semua catatan kuliah.
- `cache_mahasiswa`: data dari server yang disimpan sementara agar bisa dibuka saat offline.

Setiap box disimpan sebagai file sendiri (`nama_box.hive`) di folder dokumen aplikasi. Box yang sudah dibuka isinya dimuat ke memori, sehingga membaca data dari box **tidak perlu** `await` dan hasilnya langsung didapat.

NoSQL: tidak ada tabel dan kolom

Berbeda dengan SQLite, Hive tidak memakai `CREATE TABLE`, tidak ada skema kolom, dan tidak ada query SQL. Value yang disimpan bisa berupa:

- Tipe dasar: `String`, `int`, `double`, `bool`, `DateTime`, `null`.
- Koleksi: `List` dan `Map` (boleh bersarang).
- Objek buatan sendiri (misalnya `Catatan`), asalkan kamu mendaftarkan **TypeAdapter** (dibahas di bagian 5).

Kapan memakai Hive, SharedPreferences, atau SQLite?

Aspek	SharedPreferences	Hive	SQLite (sqflite)
Model data	Key-value sederhana	Key-value NoSQL, value boleh objek	Relasional (tabel, baris, kolom)
Tipe value	String, int, double, bool, List<String>	Semua tipe dasar, List, Map, objek dengan adapter	TEXT, INTEGER, REAL, BLOB
Cara baca	`await` (async)	Sinkron setelah box dibuka	`await` + query SQL
Query rumit (JOIN, GROUP BY)	Tidak bisa	Tidak ada; filter manual dengan `where()` di Dart	Sangat bisa
Ditulis dengan	Plugin native	Murni Dart (jalan juga di web dan desktop)	Plugin native
Cocok untuk	Flag kecil: tema, sudah onboarding, token	Daftar objek, cache offline, catatan, keranjang belanja	Data besar yang saling berelasi, laporan, agregasi

> 💡 **Aturan praktis**
>
> Cuma beberapa pengaturan? Pakai SharedPreferences. Butuh menyimpan daftar objek dengan cepat dan tanpa SQL? Pakai Hive. Butuh relasi antar tabel dan query kompleks? Pakai SQLite. Ketiganya boleh dipakai bersamaan dalam satu aplikasi.

Kenapa `hive_ce`, bukan `hive`?

Di pub.dev ada package `hive` versi 2.x yang dulu sangat populer. Sayangnya package itu sudah lama **tidak dirawat** lagi oleh pembuatnya. Komunitas lalu melanjutkannya sebagai **Hive Community Edition**: `hive_ce` dan `hive_ce_flutter`. API-nya hampir sama persis (`Hive.openBox`, `box.put`, `TypeAdapter`, dan seterusnya), tetapi rutin diperbarui untuk versi Dart dan Flutter terbaru, serta ada tambahan seperti `@GenerateAdapters` dan rentang `typeId` yang jauh lebih besar. Jadi tutorial lama tentang `hive` umumnya tetap bisa diikuti; yang berbeda hanya nama package dan baris `import`.

ℹ️ **Peta modul ini**

Kita mulai dari operasi dasar box, menyimpan Map dan List, lalu objek sendiri dengan TypeAdapter. Setelah itu UI reaktif, pola repository + Provider, enkripsi, dan diakhiri mini project **Catatan Kuliah Offline** (CRUD lengkap). Teknik menyimpan Map di bagian 4 akan dipakai lagi di Modul 14 sebagai cache offline data Supabase.

2. Instalasi dan Inisialisasi

Sebelum bisa menyimpan data, Hive perlu dipasang, diberi tahu folder penyimpanannya, lalu box yang dibutuhkan dibuka. Semua itu dilakukan sekali di `main()`.

Langkah 1: Pasang package

Buat proyek baru (atau pakai proyek yang ada), lalu jalankan dari folder proyek:

```
flutter create halo_hive
cd halo_hive
flutter pub add hive_ce hive_ce_flutter
```

- `hive_ce`: inti database (murni Dart).
- `hive_ce_flutter`: tambahan khusus Flutter, misalnya `Hive.initFlutter()` dan `box.listenable()`.

Hasilnya di `pubspec.yaml` kurang lebih seperti ini (nomor versi mengikuti yang terbaru saat kamu menjalankan perintah):

```
dependencies:
  flutter:
    sdk: flutter
  hive_ce: ^x.y.z
  hive_ce_flutter: ^x.y.z
```

Langkah 2: Import

Di file Flutter cukup satu baris ini, karena `hive_ce_flutter` sudah ikut meng-export seluruh isi `hive_ce`:

```
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
```

Untuk file Dart murni yang tidak menyentuh Flutter (misalnya model), boleh juga memakai `import 'package:hive_ce/hive_ce.dart';`.

Langkah 3: Inisialisasi di `main()`

Urutannya selalu sama:

1. `WidgetsFlutterBinding.ensureInitialized()`: wajib karena kita memanggil kode async yang memakai plugin (mencari folder dokumen) **sebelum** `runApp`.
2. `await Hive.initFlutter()`: menentukan folder penyimpanan.
3. `Hive.registerAdapter(...)`: jika memakai objek sendiri (bagian 5).
4. `await Hive.openBox(...)`: membuka box yang akan dipakai.
5. `runApp(...)`.

Contoh lengkap: aplikasi yang menghitung sudah berapa kali ia dibuka. Hapus seluruh isi `lib/main.dart` lalu ganti dengan kode berikut.

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> main() async {
  // Wajib dipanggil sebelum await apa pun yang memakai plugin
  WidgetsFlutterBinding.ensureInitialized();

  // Menentukan folder penyimpanan Hive (folder dokumen aplikasi)
  await Hive.initFlutter();

  // Membuka box bernama 'pengaturan'. Jika belum ada, Hive membuatnya.
  final box = await Hive.openBox('pengaturan');

  // Baca hitungan lama (default 0), tambah 1, simpan lagi
  final jumlahBuka = (box.get('jumlah_buka', defaultValue: 0) as int) + 1;
  await box.put('jumlah_buka', jumlahBuka);

  runApp(MyApp(jumlahBuka: jumlahBuka));
}

class MyApp extends StatelessWidget {
  final int jumlahBuka;

  const MyApp({super.key, required this.jumlahBuka});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Halo Hive')),
        body: Center(
          child: Text(
            'Aplikasi ini sudah dibuka $jumlahBuka kali',
            style: const TextStyle(fontSize: 20),
          ),
        ),
      ),
    );
  }
}
```

Angka bertambah setiap aplikasi dijalankan ulang, karena tersimpan di box

Penjelasan:

- `Future<void> main() async` : `main` boleh async agar bisa menunggu Hive siap sebelum UI tampil.
- `Hive.openBox('pengaturan')` membuat file `pengaturan.hive` jika belum ada. Box yang tidak diberi tipe bertipe `Box<dynamic>`, jadi boleh diisi tipe apa saja.
- `box.get('jumlah_buka', defaultValue: 0)` mengembalikan 0 saat aplikasi pertama kali dibuka.
- Coba tutup aplikasi sepenuhnya lalu buka lagi: angkanya bertambah. Itulah bukti data tersimpan permanen.

Mengambil box yang sudah dibuka

Membuka box itu operasi async (membaca file). Setelah terbuka, di mana pun di aplikasi kamu cukup memanggil `Hive.box()` yang sinkron:

```
// Di main(): sekali saja, pakai await
await Hive.openBox('pengaturan');

// Di widget atau kelas lain: langsung ambil, tanpa await
final box = Hive.box('pengaturan');
final tema = box.get('tema_gelap', defaultValue: false);

// Box bertipe: semua value wajib bertipe String
await Hive.openBox<String>('tugas_cepat');
final tugasBox = Hive.box<String>('tugas_cepat');
```

⚠️ **Kesalahan umum**

- Lupa `WidgetsFlutterBinding.ensureInitialized()` menyebabkan error "Binding has not yet been initialized".
- Memanggil `Hive.box('x')` padahal `openBox('x')` belum dipanggil memicu `HiveError: Box not found. Did you forget to call Hive.openBox()?`
- Membuka box yang sama dengan tipe berbeda (`openBox<String>('a')` lalu `openBox('a')`) juga memicu error. Pakai tipe yang sama secara konsisten.

3. Operasi Dasar Box

Box punya method yang mirip `Map`. Di bagian ini kita coba satu per satu, lengkap dengan output yang muncul di Debug Console.

Daftar method penting

Method	Fungsi	Async?
`put(key, value)`	Simpan atau timpa value dengan key tertentu	Ya (`Future`)
`get(key, defaultValue: ...)`	Ambil value; `null` atau `defaultValue` jika key tidak ada	Tidak
`add(value)`	Simpan dengan key angka otomatis (0, 1, 2, ...), mengembalikan key baru	Ya
`delete(key)`	Hapus satu data berdasarkan key	Ya
`containsKey(key)`	Cek apakah key ada	Tidak
`keys`, `values`, `length`	Semua key, semua value, jumlah data	Tidak
`getAt(i)`, `putAt(i, v)`, `deleteAt(i)`	Akses berdasarkan posisi (index), bukan key	`getAt` tidak, lainnya ya
`toMap()`	Salin seluruh isi box menjadi `Map`	Tidak
`clear()`	Hapus semua data di box, mengembalikan jumlah yang terhapus	Ya

Pola mudahnya: **menulis** (put, add, delete, clear) mengembalikan `Future`; **membaca** (get, keys, values, length) langsung dapat hasilnya.

Contoh lengkap semua operasi

Fungsi berikut bisa kamu panggil dari `main()` setelah `await Hive.initFlutter()`, misalnya `await demoOperasiDasar();` sebelum `runApp`.

```
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> demoOperasiDasar() async {
  final box = await Hive.openBox('latihan');
```

```dart
  // ---- CREATE / UPDATE dengan key buatan sendiri ----
  await box.put('nama', 'Budi');
  await box.put('semester', 4);
  await box.put('aktif', true);
  await box.put('semester', 5); // key sama = menimpa (update)

  // ---- READ ----
  debugPrint('${box.get('nama')}'); // Budi
  debugPrint('${box.get('semester')}'); // 5
  debugPrint('${box.get('ipk')}'); // null (key belum ada)
  debugPrint('${box.get('ipk', defaultValue: 0.0)}'); // 0.0
  debugPrint('${box.containsKey('aktif')}'); // true

  // ---- add(): key berupa angka otomatis 0, 1, 2, ... ----
  final k1 = await box.add('Kalkulus');
  final k2 = await box.add('Basis Data');
  debugPrint('key baru: $k1, $k2');

  // ---- melihat isi box ----
  debugPrint('keys   : ${box.keys}');
  debugPrint('values : ${box.values}');
  debugPrint('length : ${box.length}');

  // ---- akses berdasarkan POSISI (index), bukan key ----
  debugPrint('getAt(0): ${box.getAt(0)}');
  await box.putAt(0, 'Kalkulus Lanjut');
  debugPrint('setelah putAt: ${box.get(0)}');

  // ---- DELETE ----
  await box.delete('aktif');
  await box.deleteAt(0);
  debugPrint('setelah delete: ${box.toMap()}');

  final jumlahDihapus = await box.clear();
  debugPrint('clear() menghapus $jumlahDihapus data, sisa ${box.length}');
}
```

Output di Debug Console:

```
Budi
5
null
0.0
true
key baru: 0, 1
keys   : (0, 1, aktif, nama, semester)
values : (Kalkulus, Basis Data, true, Budi, 5)
length : 5
getAt(0): Kalkulus
setelah putAt: Kalkulus Lanjut
setelah delete: {1: Basis Data, nama: Budi, semester: 5}
clear() menghapus 3 data, sisa 0
```

Hal yang perlu diperhatikan dari output:

- `put` **= create sekaligus update.** `put('semester', 5)` menimpa nilai 4 karena key-nya sama. Hive tidak membedakan insert dan update.
- **Key boleh** `String` **atau** `int`**.** `add()` memakai key int otomatis yang terus naik.
- **Urutan key bukan urutan penyimpanan.** Hive mengurutkan key: angka dulu, lalu string secara alfabet. Karena itu `getAt(0)` menghasilkan `Kalkulus` (key 0), bukan `Budi` yang disimpan pertama.
- `deleteAt(0)` **menghapus posisi pertama**, yaitu key 0 (`Kalkulus Lanjut`), sedangkan `delete('aktif')` menghapus berdasarkan key.

> 💡 **Pakai key, hindari index**
>
> Method `...At(index)` mudah salah, karena index bergeser setiap ada data yang dihapus. Untuk data seperti catatan atau tugas, simpan dengan key unik milik datanya sendiri (misalnya `id`) memakai `put(id, data)`, lalu ubah dan hapus memakai key yang sama. Pola ini dipakai di mini project.

Perlu `await` saat menulis?

Data yang di-`put` langsung bisa dibaca dari memori meskipun tidak di-`await`. Namun penulisan ke file berjalan di belakang layar. Jika aplikasi tiba-tiba dimatikan sebelum `Future`-nya selesai, data itu bisa hilang. Biasakan `await` di method async, terutama sebelum pindah halaman atau menampilkan pesan "tersimpan".

```
// Kurang aman: tidak menunggu sampai benar-benar tertulis ke file
box.put('draft', teks);

// Aman
await box.put('draft', teks);
```

4. Menyimpan Map dan List

Tanpa membuat adapter apa pun, Hive sudah bisa menyimpan `List` dan `Map`. Ini cara tercepat menyimpan data terstruktur, dan juga cara yang dipakai untuk cache data JSON dari API atau Supabase.

Menyimpan dan membaca List & Map

```dart
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> demoMapList() async {
  final box = await Hive.openBox('profil_box');

  // Menyimpan List dan Map langsung, tanpa adapter
  await box.put('matkul_favorit', ['Pemrograman Mobile', 'Basis Data', 'AI']);
  await box.put('profil', {
    'nama': 'Siti Aminah',
    'nim': 'H1D023001',
    'ipk': 3.75,
    'hobi': ['membaca', 'coding'],
  });

  final favorit = box.get('matkul_favorit') as List;
  debugPrint('Jumlah favorit: ${favorit.length}');
  debugPrint('Pertama: ${favorit.first}');

  final profil = box.get('profil') as Map;
  debugPrint('Nama: ${profil['nama']}, IPK: ${profil['ipk']}');
}

Future<void> demoMapSetelahDibukaUlang() async {
  final box = await Hive.openBox('profil_box');
  final mentah = box.get('profil');
  debugPrint('Tipe asli: ${mentah.runtimeType}');

  // final salah = mentah as Map<String, dynamic>; // TypeError!
  final profil = Map<String, dynamic>.from(mentah as Map);
  final hobi = List<String>.from(profil['hobi'] as List);
  debugPrint('${profil['nama']} punya ${hobi.length} hobi');
}
```

Output saat `demoMapList()` dijalankan, lalu aplikasi dibuka ulang dan `demoMapSetelahDibukaUlang()` dijalankan:

```
Jumlah favorit: 3
Pertama: Pemrograman Mobile
Nama: Siti Aminah, IPK: 3.75
Tipe asli: _Map<dynamic, dynamic>
Siti Aminah punya 2 hobi
```

> ⚠️ **Jebakan paling sering: Map<dynamic, dynamic>**
>
> Saat disimpan, Map kamu bertipe `Map<String, Object>`. Namun setelah aplikasi dibuka ulang, Hive membacanya kembali dari file sebagai `Map<dynamic, dynamic>`. Akibatnya cast langsung akan gagal:
>
> ```
> type '_Map<dynamic, dynamic>' is not a subtype of type 'Map<String, dynamic>' in type cast
> ```
>
> Yang lebih menjebak, error ini **tidak muncul** saat kamu baru saja menyimpan data (data masih berupa objek asli di memori), tetapi baru muncul setelah aplikasi di-restart. Solusinya: selalu salin dengan `Map<String, dynamic>.from(...)` dan `List<String>.from(...)`, atau terima parameter bertipe `Map` saja.

Pola model + toMap/fromMap

Agar tidak bekerja dengan Map mentah di seluruh aplikasi, buat kelas model yang bisa diubah ke Map dan sebaliknya. Pola ini sama dengan `toJson/fromJson` di Modul 09.

`lib/models/tugas.dart`

```dart
// lib/models/tugas.dart
class Tugas {
  final String id;
  final String judul;
  final DateTime deadline;
  final bool selesai;

  const Tugas({
    required this.id,
    required this.judul,
    required this.deadline,
    this.selesai = false,
  });

  // Objek -> Map (untuk disimpan ke Hive, atau dikirim ke API)
  Map<String, dynamic> toMap() => {
        'id': id,
        'judul': judul,
        'deadline': deadline.toIso8601String(),
        'selesai': selesai,
      };

  // Map -> Objek. Parameter bertipe Map (bukan Map<String, dynamic>)
  // karena Map dari Hive bertipe Map<dynamic, dynamic>.
  factory Tugas.fromMap(Map map) => Tugas(
        id: map['id'] as String,
        judul: map['judul'] as String,
        deadline: DateTime.parse(map['deadline'] as String),
        selesai: map['selesai'] as bool? ?? false,
```

```
      );
}
```

lib/repositories/tugas_repository.dart

```dart
// lib/repositories/tugas_repository.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import '../models/tugas.dart';

class TugasRepository {
  // Box<Map>: setiap value adalah Map
  final Box<Map> _box = Hive.box<Map>('tugas_box');

  Future<void> simpan(Tugas tugas) => _box.put(tugas.id, tugas.toMap());

  List<Tugas> ambilSemua() =>
      _box.values.map((map) => Tugas.fromMap(map)).toList();

  Future<void> hapus(String id) => _box.delete(id);
}
```

Jangan lupa membuka box-nya dengan tipe yang sama di `main()`: `await Hive.openBox<Map>('tugas_box');`

Kenapa `DateTime` diubah menjadi String ISO? Sebenarnya Hive bisa menyimpan `DateTime` langsung di dalam Map. Namun memakai `toIso8601String()` membuat Map yang sama juga bisa dikirim sebagai JSON ke server tanpa perubahan. Itu berguna ketika data lokal harus disinkronkan dengan backend.

> 💡 **Map atau TypeAdapter?**
>
> - **Map (bagian ini):** tanpa adapter, tidak ada `typeId`, mudah ditambah field baru (field lama yang tidak ada cukup diberi nilai default di `fromMap`). Cocok untuk cache data dari API dan untuk data yang strukturnya masih sering berubah.
> - **TypeAdapter (bagian 5 dan 6):** objek tersimpan langsung, `box.get()` sudah mengembalikan objek bertipe tepat, dan file lebih ringkas. Cocok untuk model yang stabil.

5. Objek Sendiri dengan TypeAdapter Manual

Jika kamu mencoba `box.put('a', Mahasiswa(...))` tanpa persiapan, Hive akan melempar `HiveError: Cannot write, unknown type: Mahasiswa. Did you forget to register an adapter?`. Hive tidak tahu cara mengubah objek buatanmu menjadi byte. Tugas itu dikerjakan oleh **TypeAdapter**.

Cara kerja TypeAdapter

TypeAdapter adalah kelas yang berisi tiga hal:

1. `typeId` : nomor unik untuk tipe ini. Hive menulis nomor ini di depan data agar tahu adapter mana yang harus dipakai saat membaca.
2. `write()` : mengubah objek menjadi urutan nilai (serialisasi).
3. `read()` : membaca urutan nilai itu kembali menjadi objek (deserialisasi).

Kita menulis adapter secara manual dulu (tanpa code generator) supaya jelas apa yang sebenarnya terjadi.

Langkah 1: Model dan adapter

`lib/models/mahasiswa.dart`

```dart
// lib/models/mahasiswa.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class Mahasiswa {
  final String nim;
  final String nama;
  final int semester;
  final double ipk;

  const Mahasiswa({
    required this.nim,
    required this.nama,
    required this.semester,
    required this.ipk,
  });

  @override
  String toString() => 'Mahasiswa($nim, $nama, smt $semester, IPK $ipk)';
}

class MahasiswaAdapter extends TypeAdapter<Mahasiswa> {
  @override
  final int typeId = 1; // nomor unik untuk kelas Mahasiswa

  @override
  Mahasiswa read(BinaryReader reader) {
    // Urutan baca = urutan tulis
    final nim = reader.readString();
    final nama = reader.readString();
    final semester = reader.readInt();
```

```
    final ipk = reader.readDouble();
    return Mahasiswa(nim: nim, nama: nama, semester: semester, ipk: ipk);
  }

  @override
  void write(BinaryWriter writer, Mahasiswa obj) {
    writer.writeString(obj.nim);
    writer.writeString(obj.nama);
    writer.writeInt(obj.semester);
    writer.writeDouble(obj.ipk);
  }
}
```

Method yang tersedia di `BinaryWriter` dan `BinaryReader` berpasangan: `writeString/readString`, `writeInt/readInt`, `writeDouble/readDouble`, `writeBool/readBool`, `writeByte/readByte`, `writeList/readList`, `writeMap/readMap`, serta `write/read` yang otomatis memakai tipe yang dikenal Hive.

Langkah 2: Daftarkan adapter lalu pakai box bertipe

```
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'models/mahasiswa.dart';

Future<void> demoMahasiswa() async {
  // Di main(): setelah Hive.initFlutter(), sebelum openBox
  Hive.registerAdapter(MahasiswaAdapter());

  final box = await Hive.openBox<Mahasiswa>('mahasiswa_box');

  await box.put(
    'H1D023001',
    const Mahasiswa(nim: 'H1D023001', nama: 'Siti', semester: 4, ipk: 3.75),
  );
  await box.put(
    'H1D023002',
    const Mahasiswa(nim: 'H1D023002', nama: 'Andi', semester: 4, ipk: 3.20),
  );

  final siti = box.get('H1D023001'); // tipe: Mahasiswa?
  debugPrint('$siti');
  debugPrint('IPK Siti: ${siti?.ipk}');

  final ipkTinggi = box.values.where((m) => m.ipk >= 3.5).toList();
  debugPrint('IPK >= 3.5: ${ipkTinggi.map((m) => m.nama).toList()}');
}
```

Output:

```
Mahasiswa(H1D023001, Siti, smt 4, IPK 3.75)
IPK Siti: 3.75
IPK >= 3.5: [Siti]
```

Perhatikan bahwa `box.get()` langsung mengembalikan `Mahasiswa?`, bukan Map. Filter seperti "IPK minimal 3.5" dikerjakan dengan `where()` biasa di Dart.

Aturan typeId

- **Unik per aplikasi.** Dua adapter dengan `typeId` sama memicu `HiveError: There is already a TypeAdapter for typeId 1`.
- **Jangan pernah diganti** setelah aplikasi dipakai orang. Data lama di file tetap bertanda typeId lama, sehingga tidak bisa dibaca lagi.
- Rentang yang diizinkan `hive_ce` adalah 0 sampai 65439 (package `hive` lama hanya 0 sampai 223). Mulailah dari 0, 1, 2, dan seterusnya.
- `Hive.initFlutter()` sudah mendaftarkan adapter bawaan untuk `Color` (typeId 200) dan `TimeOfDay` (typeId 201). Hindari dua nomor itu.
- Daftarkan adapter **sebelum** `openBox` yang menyimpan tipe tersebut.

Aturan urutan field

Adapter manual membaca data secara berurutan seperti kaset: tidak ada nama field di file, hanya nilai-nilai yang berjajar. Jika di `write()` urutannya nim, nama, semester, ipk, maka `read()` wajib membaca nim, nama, semester, ipk juga. Tertukar sedikit saja (misalnya `readDouble` sebelum `readInt`) akan menghasilkan data aneh atau error `RangeError`.

Hati-hati saat model berubah (migrasi)

Bayangkan aplikasi sudah dipakai, lalu kamu menambah field `email`. Data lama di HP pengguna tidak punya email. Jika `read()` langsung memanggil `readString()` keempat, Hive membaca melewati batas data dan aplikasi crash.

Trik sederhana: tulis **nomor versi format** sebagai byte pertama sejak awal. Saat membaca, field baru hanya dibaca jika versinya mendukung.

```
// Versi 2 dari model: field baru `email`
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class Dosen {
  final String nidn;
  final String nama;
  final String email; // FIELD BARU di versi 2

  const Dosen({required this.nidn, required this.nama, this.email = ''});
}

class DosenAdapter extends TypeAdapter<Dosen> {
  @override
  final int typeId = 2;

  static const int _versiSekarang = 2;

  @override
  Dosen read(BinaryReader reader) {
```

```
    final versi = reader.readByte(); // byte pertama = versi format
    final nidn = reader.readString();
    final nama = reader.readString();
    // Data versi 1 belum punya email, jadi hanya dibaca jika versi >= 2
    final email = versi >= 2 ? reader.readString() : '';
    return Dosen(nidn: nidn, nama: nama, email: email);
  }

  @override
  void write(BinaryWriter writer, Dosen obj) {
    writer.writeByte(_versiSekarang);
    writer.writeString(obj.nidn);
    writer.writeString(obj.nama);
    writer.writeString(obj.email); // field baru selalu DITAMBAH DI AKHIR
  }
}
```

⚠️ **Aturan migrasi adapter manual**

- Field baru selalu **ditambah di akhir**, jangan disisipkan di tengah.
- Jangan menghapus atau mengubah tipe field lama. Jika tidak dipakai lagi, tetap baca lalu abaikan nilainya.
- Selama masih tahap belajar dan data boleh hilang, cara paling cepat adalah menghapus aplikasi dari emulator (uninstall) agar file box ikut terhapus.
- Kode yang dibuat generator (bagian 6) menulis nomor indeks untuk setiap field, sehingga migrasi jauh lebih aman. Itu alasan utama memakai codegen di proyek nyata.

6. Generate Adapter Otomatis (Lanjutan)

Menulis adapter manual untuk banyak model cukup melelahkan dan rawan salah urutan. `hive_ce_generator` bisa membuatkan adapter secara otomatis. Bagian ini bersifat **lanjutan (opsional)**; pastikan kamu sudah paham bagian 5 dulu.

Langkah 1: Pasang generator

```
flutter create rak_buku
cd rak_buku
flutter pub add hive_ce hive_ce_flutter
flutter pub add dev:hive_ce_generator dev:build_runner
```

Awalan `dev:` menaruh package di `dev_dependencies`, karena generator hanya dipakai saat development dan tidak ikut ke APK.

Langkah 2: Model biasa, tanpa anotasi

`lib/models/buku.dart`

```dart
// lib/models/buku.dart  (class biasa, tanpa anotasi apa pun)
class Buku {
  final String isbn;
  final String judul;
  final int tahun;
  final List<String> penulis;

  Buku({
    required this.isbn,
    required this.judul,
    required this.tahun,
    required this.penulis,
  });
}
```

Langkah 3: Daftarkan model di satu file `@GenerateAdapters`

Cara yang direkomendasikan `hive_ce` saat ini adalah anotasi `@GenerateAdapters`. Semua model yang ingin dibuatkan adapter cukup didaftarkan di satu tempat; model tidak perlu ditempeli `@HiveType` dan `@HiveField` seperti di tutorial `hive` lama.

`lib/hive/hive_adapters.dart`

```dart
// lib/hive/hive_adapters.dart
import 'package:hive_ce/hive_ce.dart';

import '../models/buku.dart';

part 'hive_adapters.g.dart';
```

```
@GenerateAdapters([AdapterSpec<Buku>()])
class HiveAdapters {}
```

Langkah 4: Jalankan build_runner

```
dart run build_runner build --delete-conflicting-outputs
```

Tiga file baru akan muncul di folder `lib/hive/`:

File	Isi
`hive_adapters.g.dart`	Kelas `BukuAdapter` hasil generate
`hive_adapters.g.yaml`	Catatan typeId dan indeks tiap field. **Wajib ikut di-commit** ke Git, karena dipakai agar indeks tidak berubah saat model diubah.
`hive_registrar.g.dart`	Extension `Hive.registerAdapters()` untuk mendaftarkan semua adapter sekaligus

Isi `hive_adapters.g.yaml` hasil generate:

```
# Generated by Hive CE
# Manual modifications may be necessary for certain migrations
# Check in to version control
nextTypeId: 1
types:
  Buku:
    typeId: 0
    nextIndex: 4
    fields:
      isbn:
        index: 0
      judul:
        index: 1
      tahun:
        index: 2
      penulis:
        index: 3
```

Dan potongan `BukuAdapter` yang dibuat otomatis. Perhatikan bahwa setiap nilai diawali nomor indeks field (`writeByte(0)`, `writeByte(1)`, ...). Karena itu field baru bisa ditambahkan tanpa merusak data lama.

```
// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'hive_adapters.dart';

// **************************************************************************
// AdaptersGenerator
```

```dart
// **************************************************************************

class BukuAdapter extends TypeAdapter<Buku> {
  @override
  final typeId = 0;

  @override
  Buku read(BinaryReader reader) {
    final numOfFields = reader.readByte();
    final fields = <int, dynamic>{
      for (int i = 0; i < numOfFields; i++) reader.readByte(): reader.read(),
    };
    return Buku(
      isbn: fields[0] as String,
      judul: fields[1] as String,
      tahun: (fields[2] as num).toInt(),
      penulis: (fields[3] as List).cast<String>(),
    );
  }

  @override
  void write(BinaryWriter writer, Buku obj) {
    writer
      ..writeByte(4)
      ..writeByte(0)
      ..write(obj.isbn)
      ..writeByte(1)
      ..write(obj.judul)
      ..writeByte(2)
      ..write(obj.tahun)
      ..writeByte(3)
      ..write(obj.penulis);
  }

  @override
  int get hashCode => typeId.hashCode;

  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is BukuAdapter &&
          runtimeType == other.runtimeType &&
          typeId == other.typeId;
}
```

`lib/hive/hive_registrar.g.dart` (dibuat otomatis, jangan diedit)

```dart
// Generated by Hive CE
// Do not modify
// Check in to version control

import 'package:hive_ce/hive_ce.dart';
import 'package:rak_buku/hive/hive_adapters.dart';

extension HiveRegistrar on HiveInterface {
  void registerAdapters() {
    registerAdapter(BukuAdapter());
  }
```

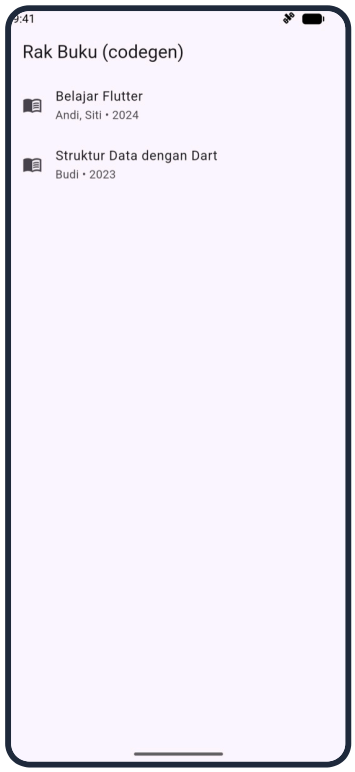
```
}

extension IsolatedHiveRegistrar on IsolatedHiveInterface {
  void registerAdapters() {
    registerAdapter(BukuAdapter());
  }
}
```

Langkah 5: Pakai di `main.dart`

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'models/buku.dart';
import 'hive/hive_registrar.g.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();

  // Satu baris mendaftarkan SEMUA adapter hasil generate
  Hive.registerAdapters();

  final box = await Hive.openBox<Buku>('buku_box');
  if (box.isEmpty) {
    await box.add(Buku(
      isbn: '978-602-1',
      judul: 'Belajar Flutter',
      tahun: 2024,
      penulis: ['Andi', 'Siti'],
    ));
    await box.add(Buku(
      isbn: '978-602-2',
      judul: 'Struktur Data dengan Dart',
      tahun: 2023,
      penulis: ['Budi'],
    ));
  }

  runApp(const BukuApp());
}

class BukuApp extends StatelessWidget {
  const BukuApp({super.key});

  @override
  Widget build(BuildContext context) {
    final daftar = Hive.box<Buku>('buku_box').values.toList();
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Rak Buku (codegen)')),
        body: ListView(
          children: [
            for (final b in daftar)
```

```
              ListTile(
                leading: const Icon(Icons.menu_book),
                title: Text(b.judul),
                subtitle: Text('${b.penulis.join(', ')} • ${b.tahun}'),
              ),
            ],
          ),
        ),
      );
  }
}
```



Objek Buku (termasuk List penulis) tersimpan memakai adapter hasil generate

Saat model berubah

- Tambah field: tambahkan di kelas, jalankan lagi `build_runner`. Field baru mendapat indeks berikutnya (lihat `nextIndex` di file yaml). Untuk data lama, field baru bernilai `null`, jadi buat field baru *nullable* atau beri nilai default di constructor.
- Hapus field: hapus dari kelas lalu generate ulang. Indeksnya tidak akan dipakai ulang.
- Tambah model baru: tambahkan `AdapterSpec<ModelBaru>()` ke daftar `@GenerateAdapters`.

ℹ️ **Bagaimana dengan @HiveType dan @HiveField?**

Gaya lama (anotasi `@HiveType(typeId: 0)` di kelas dan `@HiveField(0)` di setiap field, ditambah `part 'buku.g.dart';`) masih didukung `hive_ce_generator` agar proyek lama tetap jalan. Untuk proyek baru, gunakan `@GenerateAdapters` karena model tetap bersih dan indeks dikelola otomatis lewat file yaml.

7. UI Reaktif dengan ValueListenableBuilder

Kalau data di box berubah, bagaimana UI ikut berubah? Kamu bisa saja memanggil `setState` setelah setiap `put`, tetapi Hive menyediakan cara yang lebih rapi: `box.listenable()` yang dipasangkan dengan widget `ValueListenableBuilder`.

Konsep

- `box.listenable()` (dari `hive_ce_flutter`) menghasilkan `ValueListenable<Box>` yang memberi kabar setiap ada `put`, `add`, `delete`, atau `clear`.
- `ValueListenableBuilder` menjalankan ulang fungsi `builder`-nya setiap kabar itu datang.
- Hasilnya: cukup tulis ke box, dan UI akan menyesuaikan sendiri. Tidak perlu `setState`.

```
ValueListenableBuilder<Box<String>>(
  valueListenable: Hive.box<String>('tugas_cepat').listenable(),
  builder: (context, box, child) {
    return Text('Jumlah tugas: ${box.length}');
  },
)
```

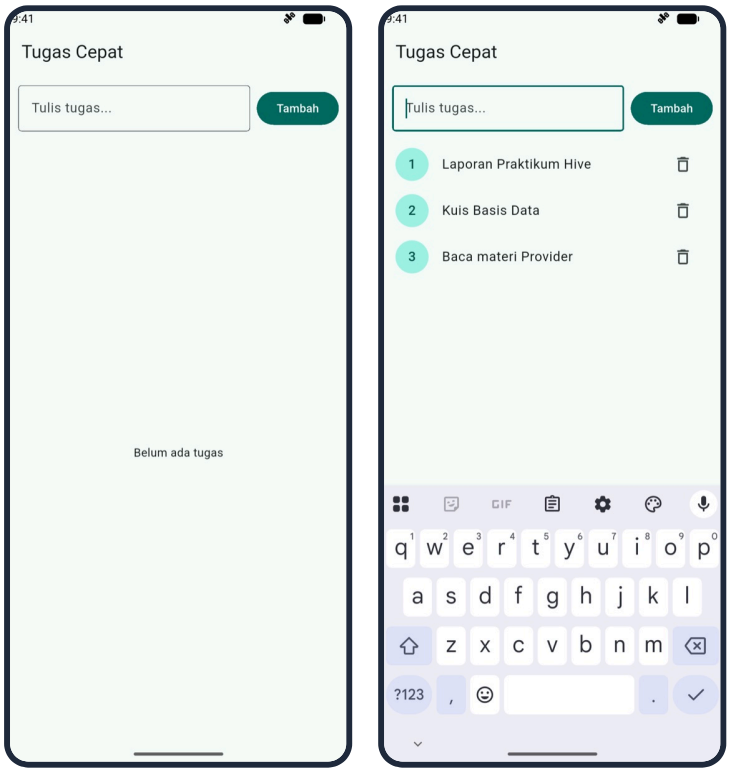
Contoh lengkap: daftar Tugas Cepat

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await Hive.openBox<String>('tugas_cepat');
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const TugasScreen(),
    );
  }
}

class TugasScreen extends StatefulWidget {
  const TugasScreen({super.key});

  @override
  State<TugasScreen> createState() => _TugasScreenState();
```

```dart
}

class _TugasScreenState extends State<TugasScreen> {
  // Box sudah dibuka di main(), di sini cukup diambil
  final Box<String> _box = Hive.box<String>('tugas_cepat');
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _tambah() {
    final teks = _controller.text.trim();
    if (teks.isEmpty) return;
    _box.add(teks); // TIDAK perlu setState: UI ikut berubah sendiri
    _controller.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Cepat')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _controller,
                    decoration: const InputDecoration(
                      hintText: 'Tulis tugas...',
                      border: OutlineInputBorder(),
                    ),
                    onSubmitted: (_) => _tambah(),
                  ),
                ),
                const SizedBox(width: 8),
                FilledButton(onPressed: _tambah, child: const Text('Tambah')),
              ],
            ),
          ),
          Expanded(
            // Builder ini dijalankan ulang setiap isi box berubah
            child: ValueListenableBuilder<Box<String>>(
              valueListenable: _box.listenable(),
              builder: (context, box, _) {
                if (box.isEmpty) {
                  return const Center(child: Text('Belum ada tugas'));
                }
                final keys = box.keys.toList();
                return ListView.builder(
                  itemCount: keys.length,
                  itemBuilder: (context, index) {
                    final key = keys[index];
                    return ListTile(
```

```
                    leading: CircleAvatar(child: Text('${index + 1}')),
                    title: Text(box.get(key) ?? ''),
                    trailing: IconButton(
                      icon: const Icon(Icons.delete_outline),
                      onPressed: () => box.delete(key),
                    ),
                  );
                },
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}
```



Box masih kosong

Setelah menambah tugas: list langsung diperbarui tanpa setState

Penjelasan:

- `_box.add(teks)` di `_tambah()` tidak diikuti `setState`. Builder tetap dijalankan ulang karena mendengarkan box.
- Di dalam builder kita memakai `box.keys.toList()`, lalu menghapus berdasarkan **key** (`box.delete(key)`), bukan index. Ingat bagian 3: key dari `add()` terus naik dan tidak ikut bergeser saat ada yang dihapus.
- `setState` masih wajar dipakai untuk state UI lain yang bukan isi box, misalnya teks di `TextField`.

Hanya mendengarkan key tertentu

Jika widget hanya peduli pada satu atau dua key, berikan parameter `keys` agar widget tidak ikut di-rebuild saat key lain berubah:

```
ValueListenableBuilder<Box>(
  valueListenable: Hive.box('pengaturan').listenable(keys: ['tema_gelap']),
  builder: (context, box, _) {
    final gelap = box.get('tema_gelap', defaultValue: false) as bool;
    return Icon(gelap ? Icons.dark_mode : Icons.light_mode);
  },
)
```

> 💡 **Kapan cukup ValueListenableBuilder, kapan butuh Provider?**
>
> `ValueListenableBuilder` sangat praktis untuk layar sederhana yang langsung menampilkan isi box. Ketika ada logika tambahan (validasi, pencarian, pengurutan, gabungan beberapa sumber data), lebih rapi jika logika itu dipindah ke repository dan Provider. Itu topik bagian berikutnya.

8. Hive + Provider dengan Pola Repository

Di Modul 08 kamu belajar Provider untuk mengelola state. Sekarang kita gabungkan dengan Hive memakai pola yang umum dipakai di industri: **Repository** untuk urusan penyimpanan dan **Provider (ChangeNotifier)** untuk state UI.

Pembagian tugas

Lapisan	Tugas	Boleh tahu tentang
Repository	Membaca dan menulis box Hive	Hive, model
Provider	Logika aplikasi, memanggil repository, lalu `notifyListeners()`	Repository, model
Widget	Menampilkan data, meneruskan aksi pengguna ke provider	Provider, model (tidak menyentuh Hive langsung)

Keuntungannya: jika suatu saat penyimpanan diganti (misalnya ke Supabase di Modul 12), yang diubah cukup repository. Widget tidak perlu disentuh, dan repository mudah diuji tanpa UI.

Contoh: pengaturan tema gelap dan nama pengguna yang tersimpan di Hive. Pasang package dulu: `flutter pub add hive_ce hive_ce_flutter provider`.

Langkah 1: Repository

`lib/repositories/pengaturan_repository.dart`

```dart
// lib/repositories/pengaturan_repository.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class PengaturanRepository {
  static const String namaBox = 'pengaturan';
  static const String _kunciTemaGelap = 'tema_gelap';
  static const String _kunciNama = 'nama_pengguna';

  final Box _box;

  PengaturanRepository(this._box);

  static Future<PengaturanRepository> buka() async {
    final box = await Hive.openBox(namaBox);
    return PengaturanRepository(box);
  }

  bool get temaGelap => _box.get(_kunciTemaGelap, defaultValue: false) as bool;
  Future<void> setTemaGelap(bool nilai) => _box.put(_kunciTemaGelap, nilai);

  String get namaPengguna =>
```

```
      _box.get(_kunciNama, defaultValue: 'Mahasiswa') as String;
  Future<void> setNamaPengguna(String nama) => _box.put(_kunciNama, nama);
}
```

Nama box dan nama key disimpan sebagai konstanta privat. Seluruh aplikasi tidak perlu tahu bahwa tema disimpan dengan key `'tema_gelap'`, sehingga salah ketik key tidak tersebar di banyak file.

Langkah 2: Provider

`lib/providers/pengaturan_provider.dart`

```dart
// lib/providers/pengaturan_provider.dart
import 'package:flutter/foundation.dart';

import '../repositories/pengaturan_repository.dart';

class PengaturanProvider with ChangeNotifier {
  final PengaturanRepository _repo;

  PengaturanProvider(this._repo);

  // Nilai awal langsung dibaca dari Hive (sinkron, tanpa await)
  bool get temaGelap => _repo.temaGelap;
  String get namaPengguna => _repo.namaPengguna;

  Future<void> gantiTema(bool gelap) async {
    await _repo.setTemaGelap(gelap);
    notifyListeners();
  }

  Future<void> gantiNama(String nama) async {
    await _repo.setNamaPengguna(nama);
    notifyListeners();
  }
}
```

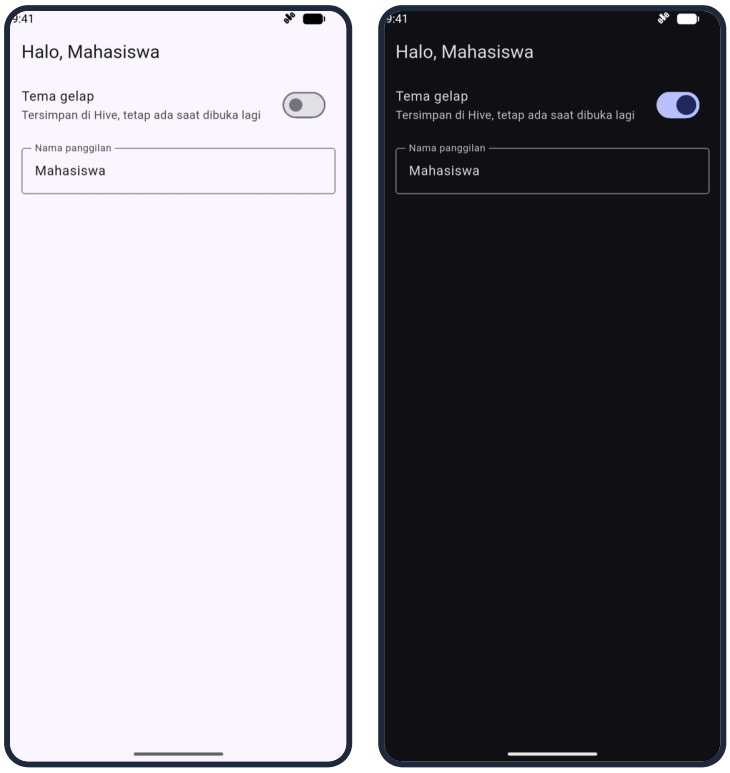
Langkah 3: main.dart

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:provider/provider.dart';

import 'providers/pengaturan_provider.dart';
import 'repositories/pengaturan_repository.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  final repo = await PengaturanRepository.buka();

  runApp(
```

```
    ChangeNotifierProvider(
      create: (_) => PengaturanProvider(repo),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final temaGelap = context.watch<PengaturanProvider>().temaGelap;
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      darkTheme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.dark,
      ),
      themeMode: temaGelap ? ThemeMode.dark : ThemeMode.light,
      home: const PengaturanScreen(),
    );
  }
}

class PengaturanScreen extends StatelessWidget {
  const PengaturanScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final pengaturan = context.watch<PengaturanProvider>();
    return Scaffold(
      appBar: AppBar(title: Text('Halo, ${pengaturan.namaPengguna}')),
      body: ListView(
        children: [
          SwitchListTile(
            title: const Text('Tema gelap'),
            subtitle: const Text('Tersimpan di Hive, tetap ada saat dibuka lagi'),
            value: pengaturan.temaGelap,
            onChanged: (nilai) =>
                context.read<PengaturanProvider>().gantiTema(nilai),
          ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: TextFormField(
              initialValue: pengaturan.namaPengguna,
              decoration: const InputDecoration(
                labelText: 'Nama panggilan',
                border: OutlineInputBorder(),
              ),
              onFieldSubmitted: (nama) =>
                  context.read<PengaturanProvider>().gantiNama(nama.trim()),
            ),
          ),
        ],
      ),
    );
```

```
  }
}
```



Tema terang (nilai awal)

Setelah switch ditekan: tema gelap tersimpan dan tetap aktif saat aplikasi dibuka lagi

Penjelasan:

- Repository dibuka di `main()` (async), lalu dikirim ke provider lewat constructor. Dengan begitu provider tidak perlu berurusan dengan `await openBox`.
- Getter `temaGelap` di provider langsung membaca Hive. Karena membaca box itu sinkron, tidak perlu menyalin data ke variabel lain atau menunggu `Future`.
- `context.watch` di `MyApp` membuat `MaterialApp` dibangun ulang saat tema berubah, sedangkan `context.read` dipakai di `onChanged` (aturan yang sama dengan Modul 08).

> ℹ️ **Bandingkan dengan Modul 10**
>
> Di SharedPreferences, membaca nilai butuh `await SharedPreferences.getInstance()`, sehingga tema awal harus dimuat secara async. Dengan Hive, setelah box dibuka di `main()`, nilai bisa langsung dibaca di `build()`.

9. Enkripsi Box

File box Hive tersimpan sebagai data biner biasa. Siapa pun yang bisa menyalin file aplikasi (misalnya di HP yang di-root) bisa membaca isinya. Untuk data sensitif seperti token atau catatan pribadi, Hive mendukung **enkripsi AES-256** per box.

Bahan yang dibutuhkan

- **Kunci 256-bit** (32 byte). Dibuat sekali dengan `Hive.generateSecureKey()`.
- **Tempat aman untuk kunci.** Kunci tidak boleh disimpan di Hive biasa atau ditulis di kode. Simpan di `flutter_secure_storage`, yang memakai Keystore (Android) dan Keychain (iOS).
- `HiveAesCipher(kunci)` yang diberikan ke `openBox` lewat parameter `encryptionCipher`.

```
flutter pub add flutter_secure_storage
```

Versi terbaru `flutter_secure_storage` membutuhkan Android minimal API 23. Jika build gagal karena `minSdk`, ubah `minSdk` di `android/app/build.gradle.kts` menjadi minimal 23.

`lib/services/box_rahasia.dart`

```dart
// lib/services/box_rahasia.dart
import 'dart:convert';

import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

const _storage = FlutterSecureStorage();
const _namaKunci = 'kunci_enkripsi_hive';

Future<Box<String>> bukaBoxRahasia() async {
  // 1. Coba ambil kunci yang pernah disimpan
  var kunciBase64 = await _storage.read(key: _namaKunci);

  // 2. Belum ada? Buat kunci acak 256-bit lalu simpan ke secure storage
  if (kunciBase64 == null) {
    final kunciBaru = Hive.generateSecureKey(); // List<int> 32 byte
    kunciBase64 = base64UrlEncode(kunciBaru);
    await _storage.write(key: _namaKunci, value: kunciBase64);
  }

  // 3. Buka box dengan cipher AES
  final kunci = base64Url.decode(kunciBase64);
  return Hive.openBox<String>(
    'rahasia',
    encryptionCipher: HiveAesCipher(kunci),
  );
}
```

Pemakaiannya sama seperti box biasa. Enkripsi dan dekripsi terjadi otomatis:

```dart
// Di main(), setelah await Hive.initFlutter();
final rahasia = await bukaBoxRahasia();
await rahasia.put('token', 'eyJhbGciOi...');
debugPrint(rahasia.get('token')); // eyJhbGciOi...
```

Penjelasan:

- Kunci berupa `List<int>`, sedangkan secure storage hanya menyimpan String. Karena itu kunci diubah dengan `base64UrlEncode` saat disimpan dan `base64Url.decode` saat dipakai.
- Kunci dibuat sekali saja. Membuat kunci baru setiap aplikasi dibuka berarti data lama tidak akan bisa dibaca lagi.
- Hanya **value** yang dienkripsi. **Key** box tetap tersimpan tanpa enkripsi, jadi jangan menaruh informasi sensitif di nama key.

> ⚠️ **Perhatian**
>
> - Membuka box terenkripsi dengan kunci yang salah, atau membuka box biasa yang sudah berisi data dengan cipher, akan memicu error. Tentukan sejak awal box mana yang dienkripsi.
> - Jika kunci hilang (misalnya data secure storage terhapus), isi box tidak bisa dipulihkan. Untuk data yang masih ada di server, cukup hapus box lalu unduh ulang.

10. Mini Project: Catatan Kuliah Offline (CRUD Lengkap)

Saatnya merangkai semua materi menjadi satu aplikasi utuh: **Catatan Kuliah Offline**. Aplikasi ini berjalan 100% tanpa internet dan datanya tetap ada walaupun aplikasi ditutup.

Fitur yang dibangun

- **Create:** menambah catatan (judul, mata kuliah, isi) lewat form dengan validasi.
- **Read:** menampilkan daftar catatan dalam `ListView`.
- **Update:** mengetuk catatan untuk membuka form edit yang sudah terisi.
- **Delete:** menghapus catatan setelah konfirmasi dialog.
- **Cari:** berdasarkan judul, mata kuliah, atau isi.
- **Urutkan:** terbaru, terlama, atau judul A-Z.
- **Empty state:** tampilan khusus saat belum ada catatan dan saat pencarian tidak menemukan hasil.

Arsitektur

```
Widget (HomeScreen, FormCatatanScreen)
   |  context.watch / context.read
   ▼
CatatanProvider (ChangeNotifier)  ← cari, urutkan, tambah, ubah, hapus
   |
   ▼
CatatanRepository                 ← satu-satunya yang menyentuh Hive
   |
   ▼
Box<Catatan> 'catatan_box'  +  CatatanAdapter (typeId 0)
```

Struktur folder

```
catatan_kuliah/
├── lib/
|   ├── main.dart
|   ├── models/
|   |   ├── catatan.dart
|   |   └── catatan_adapter.dart
|   ├── repositories/
|   |   └── catatan_repository.dart
|   ├── providers/
|   |   └── catatan_provider.dart
|   ├── screens/
|   |   ├── home_screen.dart
|   |   └── form_catatan_screen.dart
|   └── widgets/
|       ├── catatan_tile.dart
|       └── empty_state.dart
└── test/
    └── catatan_repository_test.dart   (bonus)
```

> 💡 **Urutan membangun**
>
> Kerjakan dari lapisan paling bawah ke atas: model, adapter, repository, provider, widget kecil, layar, lalu `main.dart`. Dengan urutan ini setiap file yang kamu tulis hanya bergantung pada file yang sudah ada, sehingga error merah di editor cepat hilang.

Langkah 0: Buat proyek dan pasang package

```
flutter create catatan_kuliah
cd catatan_kuliah
flutter pub add hive_ce hive_ce_flutter provider
```

Hapus file `test/widget_test.dart` bawaan karena isinya menguji aplikasi counter yang akan kita ganti.

Langkah 1: Model `Catatan`

Model berisi data murni tanpa kode Hive. `copyWith` dipakai saat edit agar kita tidak mengubah objek lama, melainkan membuat salinan dengan field yang diganti. `id` dan `dibuatPada` sengaja tidak bisa diganti.

📄 **`lib/models/catatan.dart`**

```dart
// lib/models/catatan.dart
class Catatan {
  final String id;
  final String judul;
  final String mataKuliah;
  final String isi;
  final DateTime dibuatPada;
  final DateTime diubahPada;

  const Catatan({
    required this.id,
    required this.judul,
    required this.mataKuliah,
    required this.isi,
    required this.dibuatPada,
    required this.diubahPada,
  });

  // Membuat salinan objek dengan sebagian field diganti (dipakai saat edit)
  Catatan copyWith({
    String? judul,
    String? mataKuliah,
    String? isi,
    DateTime? diubahPada,
  }) {
    return Catatan(
      id: id,
```

```
      judul: judul ?? this.judul,
      mataKuliah: mataKuliah ?? this.mataKuliah,
      isi: isi ?? this.isi,
      dibuatPada: dibuatPada,
      diubahPada: diubahPada ?? this.diubahPada,
    );
  }
}
```

Langkah 2: TypeAdapter manual

Adapter ini menerapkan pelajaran dari bagian 5: urutan baca sama dengan urutan tulis, `DateTime` disimpan sebagai angka milidetik, dan byte pertama adalah nomor versi format agar migrasi di masa depan aman.

lib/models/catatan_adapter.dart

```dart
// lib/models/catatan_adapter.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'catatan.dart';

class CatatanAdapter extends TypeAdapter<Catatan> {
  // typeId harus unik di seluruh aplikasi dan TIDAK BOLEH diganti setelah rilis
  @override
  final int typeId = 0;

  // Versi format penyimpanan. Naikkan jika suatu saat menambah field baru.
  static const int _versi = 1;

  @override
  Catatan read(BinaryReader reader) {
    final versi = reader.readByte(); // dibaca dulu, dipakai saat migrasi nanti
    assert(versi <= _versi, 'Data dibuat oleh versi aplikasi yang lebih baru');
    // Urutan baca HARUS sama persis dengan urutan tulis di write()
    return Catatan(
      id: reader.readString(),
      judul: reader.readString(),
      mataKuliah: reader.readString(),
      isi: reader.readString(),
      dibuatPada: DateTime.fromMillisecondsSinceEpoch(reader.readInt()),
      diubahPada: DateTime.fromMillisecondsSinceEpoch(reader.readInt()),
    );
  }

  @override
  void write(BinaryWriter writer, Catatan obj) {
    writer.writeByte(_versi);
    writer.writeString(obj.id);
    writer.writeString(obj.judul);
    writer.writeString(obj.mataKuliah);
    writer.writeString(obj.isi);
    writer.writeInt(obj.dibuatPada.millisecondsSinceEpoch);
    writer.writeInt(obj.diubahPada.millisecondsSinceEpoch);
  }
}
```

Langkah 3: Repository

Setiap catatan disimpan dengan `put(catatan.id, catatan)`. Karena key-nya adalah id catatan, menyimpan catatan yang sudah ada otomatis menjadi update, dan menghapus cukup dengan id. Kita tidak memakai index sama sekali.

`lib/repositories/catatan_repository.dart`

```dart
// lib/repositories/catatan_repository.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import '../models/catatan.dart';

/// Satu-satunya kelas yang boleh menyentuh box Hive catatan.
/// Widget dan provider cukup memanggil method di sini.
class CatatanRepository {
  static const String namaBox = 'catatan_box';

  final Box<Catatan> _box;

  CatatanRepository(this._box);

  /// Membuka box lalu membungkusnya menjadi repository.
  /// Pastikan Hive sudah di-init dan CatatanAdapter sudah didaftarkan.
  static Future<CatatanRepository> buka() async {
    final box = await Hive.openBox<Catatan>(namaBox);
    return CatatanRepository(box);
  }

  /// READ: semua catatan (urutan belum diatur)
  List<Catatan> ambilSemua() => _box.values.toList();

  /// READ: satu catatan berdasarkan id (null jika tidak ada)
  Catatan? ambil(String id) => _box.get(id);

  int get jumlah => _box.length;

  /// CREATE & UPDATE: put() dengan key yang sama akan menimpa data lama
  Future<void> simpan(Catatan catatan) => _box.put(catatan.id, catatan);

  /// DELETE: hapus satu catatan
  Future<void> hapus(String id) => _box.delete(id);

  /// DELETE: kosongkan seluruh catatan
  Future<int> hapusSemua() => _box.clear();
}
```

Langkah 4: Provider

Provider menyimpan state UI (kata kunci pencarian dan pilihan urutan) dan menyediakan getter `daftar` yang sudah difilter dan diurutkan. Setiap operasi tulis memanggil repository, lalu `notifyListeners()`.

lib/providers/catatan_provider.dart

```dart
// lib/providers/catatan_provider.dart
import 'package:flutter/foundation.dart';

import '../models/catatan.dart';
import '../repositories/catatan_repository.dart';

enum UrutanCatatan { terbaru, terlama, judulAZ }

class CatatanProvider with ChangeNotifier {
  final CatatanRepository _repo;

  CatatanProvider(this._repo);

  String _kataKunci = '';
  UrutanCatatan _urutan = UrutanCatatan.terbaru;

  String get kataKunci => _kataKunci;
  UrutanCatatan get urutan => _urutan;

  /// Jumlah seluruh catatan di box (tanpa filter pencarian)
  int get totalCatatan => _repo.jumlah;

  /// Daftar yang siap ditampilkan: sudah difilter lalu diurutkan
  List<Catatan> get daftar {
    final kunci = _kataKunci.trim().toLowerCase();
    final hasil = _repo.ambilSemua().where((c) {
      if (kunci.isEmpty) return true;
      return c.judul.toLowerCase().contains(kunci) ||
          c.mataKuliah.toLowerCase().contains(kunci) ||
          c.isi.toLowerCase().contains(kunci);
    }).toList();

    switch (_urutan) {
      case UrutanCatatan.terbaru:
        hasil.sort((a, b) => b.diubahPada.compareTo(a.diubahPada));
      case UrutanCatatan.terlama:
        hasil.sort((a, b) => a.diubahPada.compareTo(b.diubahPada));
      case UrutanCatatan.judulAZ:
        hasil.sort(
            (a, b) => a.judul.toLowerCase().compareTo(b.judul.toLowerCase()));
    }
    return hasil;
  }

  void cari(String kataKunci) {
    _kataKunci = kataKunci;
    notifyListeners();
  }

  void ubahUrutan(UrutanCatatan urutan) {
    _urutan = urutan;
    notifyListeners();
  }

  Future<void> tambah({
    required String judul,
    required String mataKuliah,
```

```
    required String isi,
  }) async {
    final sekarang = DateTime.now();
    final catatan = Catatan(
      // id unik sederhana berbasis waktu (tanpa package tambahan)
      id: sekarang.microsecondsSinceEpoch.toString(),
      judul: judul,
      mataKuliah: mataKuliah,
      isi: isi,
      dibuatPada: sekarang,
      diubahPada: sekarang,
    );
    await _repo.simpan(catatan);
    notifyListeners();
  }

  Future<void> ubah(
    Catatan lama, {
    required String judul,
    required String mataKuliah,
    required String isi,
  }) async {
    final baru = lama.copyWith(
      judul: judul,
      mataKuliah: mataKuliah,
      isi: isi,
      diubahPada: DateTime.now(),
    );
    await _repo.simpan(baru); // key sama, data lama tertimpa
    notifyListeners();
  }

  Future<void> hapus(String id) async {
    await _repo.hapus(id);
    notifyListeners();
  }
}
```

- `switch` tanpa `break` adalah sintaks Dart 3: setiap `case` otomatis berhenti.
- `totalCatatan` dipisah dari `daftar.length` agar layar bisa membedakan "belum ada catatan sama sekali" dan "pencarian tidak menemukan apa pun".
- Id dibuat dari `microsecondsSinceEpoch` sehingga cukup unik untuk satu perangkat tanpa package tambahan. Di Modul 14 kita akan memakai id dari server.

Langkah 5: Widget kecil yang dipakai ulang

`lib/widgets/empty_state.dart`

```
// lib/widgets/empty_state.dart
import 'package:flutter/material.dart';

class EmptyState extends StatelessWidget {
  final IconData ikon;
  final String judul;
  final String pesan;
```

```
  const EmptyState({
    super.key,
    required this.ikon,
    required this.judul,
    required this.pesan,
  });

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(ikon, size: 72, color: warna.primary.withValues(alpha: 0.5)),
            const SizedBox(height: 16),
            Text(
              judul,
              style: Theme.of(context).textTheme.titleMedium,
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 8),
            Text(
              pesan,
              style: TextStyle(color: warna.onSurfaceVariant),
              textAlign: TextAlign.center,
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/widgets/catatan_tile.dart

```
// lib/widgets/catatan_tile.dart
import 'package:flutter/material.dart';

import '../models/catatan.dart';

class CatatanTile extends StatelessWidget {
  final Catatan catatan;
  final VoidCallback onTap;
  final VoidCallback onHapus;

  const CatatanTile({
    super.key,
    required this.catatan,
    required this.onTap,
    required this.onHapus,
  });

  String _formatTanggal(DateTime t) {
    String duaDigit(int n) => n.toString().padLeft(2, '0');
```

```
    return '${duaDigit(t.day)}/${duaDigit(t.month)}/${t.year} '
        '${duaDigit(t.hour)}:${duaDigit(t.minute)}';
  }

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        onTap: onTap,
        leading: CircleAvatar(
          backgroundColor: warna.primaryContainer,
          child: Text(
            catatan.mataKuliah.isEmpty
                ? '?'
                : catatan.mataKuliah[0].toUpperCase(),
          ),
        ),
        title: Text(
          catatan.judul,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(catatan.mataKuliah, style: TextStyle(color: warna.primary)),
            if (catatan.isi.isNotEmpty)
              Text(catatan.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 4),
            Text(
              'Diubah: ${_formatTanggal(catatan.diubahPada)}',
              style: Theme.of(context).textTheme.bodySmall,
            ),
          ],
        ),
        isThreeLine: true,
        trailing: IconButton(
          icon: const Icon(Icons.delete_outline),
          tooltip: 'Hapus',
          onPressed: onHapus,
        ),
      ),
    );
  }
}
```

`CatatanTile` tidak tahu apa pun tentang Provider atau Hive. Ia hanya menerima data dan dua callback (`onTap`, `onHapus`), sehingga mudah dipakai ulang dan diuji.

Langkah 6: Layar form (tambah & edit)

Satu layar dipakai untuk dua mode. Jika parameter `catatan` bernilai `null`, form berada di mode tambah; jika berisi, form di mode edit dan field langsung terisi data lama.

lib/screens/form_catatan_screen.dart

```dart
// lib/screens/form_catatan_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';

class FormCatatanScreen extends StatefulWidget {
  /// null = mode tambah, berisi = mode edit
  final Catatan? catatan;

  const FormCatatanScreen({super.key, this.catatan});

  @override
  State<FormCatatanScreen> createState() => _FormCatatanScreenState();
}

class _FormCatatanScreenState extends State<FormCatatanScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judulCtrl;
  late final TextEditingController _matkulCtrl;
  late final TextEditingController _isiCtrl;
  bool _menyimpan = false;

  bool get _modeEdit => widget.catatan != null;

  @override
  void initState() {
    super.initState();
    // Saat mode edit, isi form dengan data lama
    _judulCtrl = TextEditingController(text: widget.catatan?.judul ?? '');
    _matkulCtrl = TextEditingController(text: widget.catatan?.mataKuliah ?? '');
    _isiCtrl = TextEditingController(text: widget.catatan?.isi ?? '');
  }

  @override
  void dispose() {
    _judulCtrl.dispose();
    _matkulCtrl.dispose();
    _isiCtrl.dispose();
    super.dispose();
  }

  String? _wajibDiisi(String? nilai) {
    if (nilai == null || nilai.trim().isEmpty) return 'Wajib diisi';
    return null;
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    setState(() => _menyimpan = true);

    final provider = context.read<CatatanProvider>();
    final judul = _judulCtrl.text.trim();
    final matkul = _matkulCtrl.text.trim();
    final isi = _isiCtrl.text.trim();
```

```
    if (_modeEdit) {
      await provider.ubah(
        widget.catatan!,
        judul: judul,
        mataKuliah: matkul,
        isi: isi,
      );
    } else {
      await provider.tambah(judul: judul, mataKuliah: matkul, isi: isi);
    }

    if (!mounted) return; // widget bisa saja sudah ditutup saat menunggu
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(_modeEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
      ),
    );
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(_modeEdit ? 'Edit Catatan' : 'Catatan Baru'),
      ),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judulCtrl,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ),
              textInputAction: TextInputAction.next,
              validator: _wajibDiisi,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _matkulCtrl,
              decoration: const InputDecoration(
                labelText: 'Mata Kuliah',
                hintText: 'mis. Pemrograman Mobile',
                border: OutlineInputBorder(),
              ),
              textInputAction: TextInputAction.next,
              validator: _wajibDiisi,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isiCtrl,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                alignLabelWithHint: true,
                border: OutlineInputBorder(),
              ),
```

```
              minLines: 6,
              maxLines: 12,
            ),
            const SizedBox(height: 20),
            FilledButton.icon(
              onPressed: _menyimpan ? null : _simpan,
              icon: const Icon(Icons.save),
              label: Text(_modeEdit ? 'Simpan Perubahan' : 'Simpan'),
            ),
          ],
        ),
      ),
    );
  }
}
```

- `context.read<CatatanProvider>()` diambil **sebelum** `await`, karena memakai `context` setelah `await` perlu dicek dulu.
- `if (!mounted) return;` setelah `await` mencegah error jika layar sudah ditutup ketika penyimpanan selesai.
- Tombol simpan dinonaktifkan (`onPressed: null`) selama menyimpan agar tidak terjadi simpan ganda.

Langkah 7: Layar utama

`lib/screens/home_screen.dart`

```
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';
import '../widgets/catatan_tile.dart';
import '../widgets/empty_state.dart';
import 'form_catatan_screen.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  void _bukaForm(BuildContext context, [Catatan? catatan]) {
    Navigator.of(context).push(
      MaterialPageRoute(
        builder: (_) => FormCatatanScreen(catatan: catatan),
      ),
    );
  }

  Future<void> _konfirmasiHapus(BuildContext context, Catatan catatan) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus catatan?'),
        content: Text('Catatan "${catatan.judul}" akan dihapus permanen.'),
        actions: [
```

```
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );

    // showDialog mengembalikan null jika dialog ditutup dengan tap di luar
    if (yakin != true) return;
    if (!context.mounted) return;

    await context.read<CatatanProvider>().hapus(catatan.id);
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('"${catatan.judul}" dihapus')),
    );
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<CatatanProvider>();
    final daftar = provider.daftar;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Kuliah'),
        actions: [
          PopupMenuButton<UrutanCatatan>(
            icon: const Icon(Icons.sort),
            tooltip: 'Urutkan',
            initialValue: provider.urutan,
            onSelected: (u) => context.read<CatatanProvider>().ubahUrutan(u),
            itemBuilder: (_) => const [
              PopupMenuItem(
                value: UrutanCatatan.terbaru,
                child: Text('Terbaru dulu'),
              ),
              PopupMenuItem(
                value: UrutanCatatan.terlama,
                child: Text('Terlama dulu'),
              ),
              PopupMenuItem(
                value: UrutanCatatan.judulAZ,
                child: Text('Judul A-Z'),
              ),
            ],
          ),
        ],
      ),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(12, 8, 12, 4),
            child: TextField(
```

```
              decoration: const InputDecoration(
                hintText: 'Cari judul, mata kuliah, atau isi...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
                isDense: true,
              ),
              onChanged: (teks) => context.read<CatatanProvider>().cari(teks),
            ),
          ),
          Expanded(child: _buildDaftar(context, provider, daftar)),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _bukaForm(context),
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
    );
  }

  Widget _buildDaftar(
    BuildContext context,
    CatatanProvider provider,
    List<Catatan> daftar,
  ) {
    if (provider.totalCatatan == 0) {
      return const EmptyState(
        ikon: Icons.note_alt_outlined,
        judul: 'Belum ada catatan',
        pesan: 'Tekan tombol + Catatan untuk menulis catatan kuliah pertamamu.',
      );
    }
    if (daftar.isEmpty) {
      return EmptyState(
        ikon: Icons.search_off,
        judul: 'Tidak ditemukan',
        pesan: 'Tidak ada catatan yang cocok dengan "${provider.kataKunci}".',
      );
    }
    return ListView.builder(
      padding: const EdgeInsets.only(bottom: 88), // agar tidak tertutup FAB
      itemCount: daftar.length,
      itemBuilder: (context, index) {
        final catatan = daftar[index];
        return CatatanTile(
          key: ValueKey(catatan.id),
          catatan: catatan,
          onTap: () => _bukaForm(context, catatan),
          onHapus: () => _konfirmasiHapus(context, catatan),
        );
      },
    );
  }
}
```

- `context.watch` di `build` membuat layar dibangun ulang setiap provider memanggil `notifyListeners()`: setelah tambah, edit, hapus, cari, atau ganti urutan.

- `showDialog<bool>` mengembalikan `true` / `false` dari tombol, atau `null` jika dialog ditutup dengan mengetuk area luar. Karena itu yang dicek adalah `yakin != true`.
- `context.mounted` dicek setelah setiap `await` sebelum `context` dipakai lagi.
- `ValueKey(catatan.id)` membantu Flutter mengenali item yang sama saat urutan list berubah.

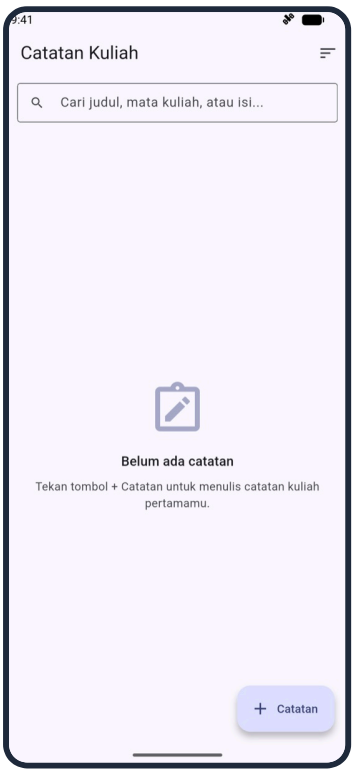
Langkah 8: `main.dart`

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:provider/provider.dart';

import 'models/catatan_adapter.dart';
import 'providers/catatan_provider.dart';
import 'repositories/catatan_repository.dart';
import 'screens/home_screen.dart';

Future<void> main() async {
  // 1. Wajib sebelum memakai plugin (path_provider) sebelum runApp
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Siapkan folder penyimpanan Hive
  await Hive.initFlutter();

  // 3. Daftarkan adapter SEBELUM membuka box yang memakainya
  Hive.registerAdapter(CatatanAdapter());

  // 4. Buka box sekali di awal, lalu bungkus dengan repository
  final repo = await CatatanRepository.buka();

  runApp(
    ChangeNotifierProvider(
      create: (_) => CatatanProvider(repo),
      child: const CatatanApp(),
    ),
  );
}

class CatatanApp extends StatelessWidget {
  const CatatanApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah Offline',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const HomeScreen(),
    );
  }
}
```

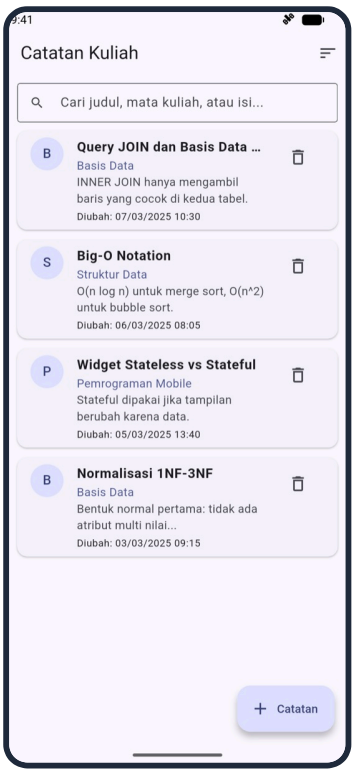
Langkah 9: Jalankan dan uji

```
flutter run
```

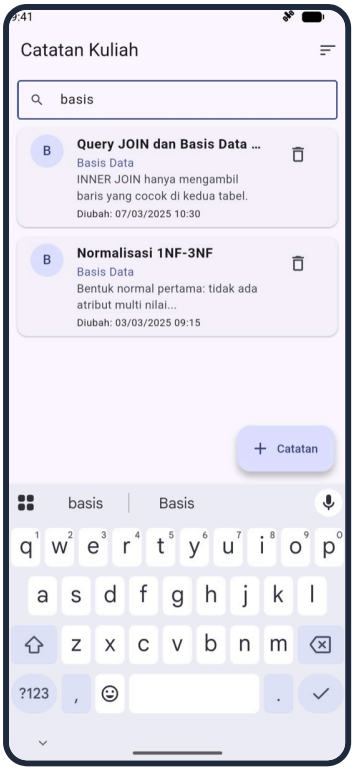


Empty state saat box masih kosong

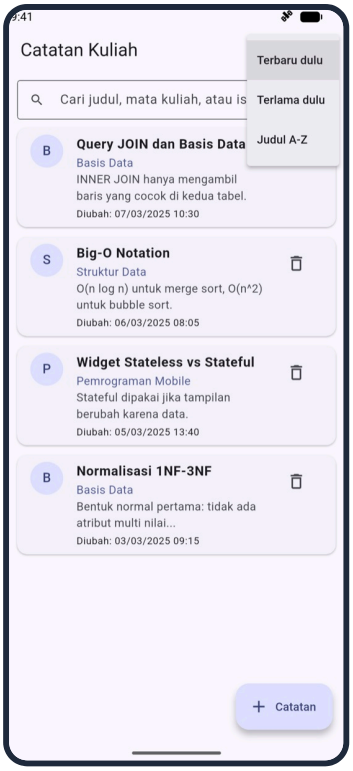
Form tambah catatan



Daftar catatan, terbaru di atas

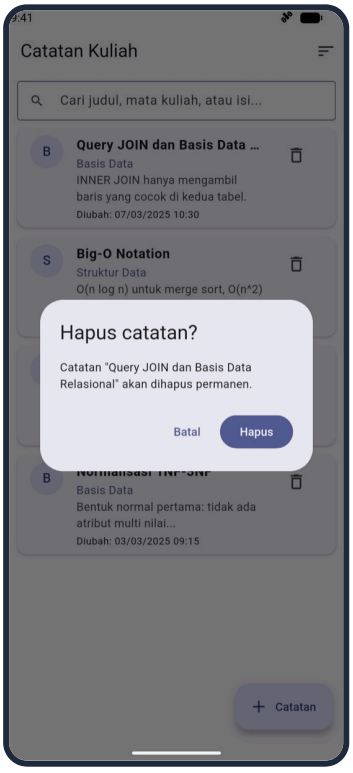


Pencarian kata "basis"



Menu urutkan di AppBar

Mode edit: form terisi data lama



Konfirmasi sebelum menghapus

Daftar uji manual:

1. Buka aplikasi pertama kali: muncul tulisan "Belum ada catatan".
2. Tekan **+ Catatan**, lalu langsung tekan Simpan: muncul pesan "Wajib diisi" pada judul dan mata kuliah.
3. Isi form lalu simpan: catatan muncul di daftar dan SnackBar tampil.

4. Tambah minimal 3 catatan, lalu coba ketiga pilihan urutan.
5. Ketik kata kunci di kolom cari; coba juga kata yang tidak ada untuk melihat empty state "Tidak ditemukan".
6. Ketuk sebuah catatan, ubah judulnya, lalu simpan. Catatan itu naik ke atas karena `diubahPada` diperbarui.
7. Tekan ikon hapus, pilih Batal (data tetap ada), lalu ulangi dan pilih Hapus.
8. **Tutup aplikasi sepenuhnya** (bukan hot reload), lalu buka lagi: semua catatan masih ada.

Bonus: unit test repository

Repository yang terpisah dari UI bisa diuji tanpa emulator. Di unit test tidak ada `path_provider`, sehingga kita memakai `Hive.init(folder)` dengan folder sementara sebagai pengganti `Hive.initFlutter()`. Ganti `catatan_kuliah` pada baris import jika nama proyekmu berbeda (lihat `name:` di `pubspec.yaml`).

`test/catatan_repository_test.dart`

```dart
// test/catatan_repository_test.dart
import 'dart:io';

import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:catatan_kuliah/models/catatan.dart';
import 'package:catatan_kuliah/models/catatan_adapter.dart';
import 'package:catatan_kuliah/repositories/catatan_repository.dart';

void main() {
  late Directory tempDir;
  late CatatanRepository repo;

  setUp(() async {
    // Di unit test tidak ada path_provider, jadi pakai Hive.init + folder sementara
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    if (!Hive.isAdapterRegistered(0)) {
      Hive.registerAdapter(CatatanAdapter());
    }
    repo = await CatatanRepository.buka();
  });

  tearDown(() async {
    await Hive.deleteFromDisk(); // hapus semua box
    await Hive.close();
    await tempDir.delete(recursive: true);
  });

  Catatan buatCatatan(String id, String judul) {
    final t = DateTime(2025, 1, 1, 8, 0);
    return Catatan(
      id: id,
      judul: judul,
      mataKuliah: 'Pemrograman Mobile',
      isi: 'Isi $judul',
```

```
      dibuatPada: t,
      diubahPada: t,
    );
  }

  test('create dan read', () async {
    await repo.simpan(buatCatatan('1', 'Widget'));
    expect(repo.jumlah, 1);
    expect(repo.ambil('1')?.judul, 'Widget');
  });

  test('update menimpa data dengan id yang sama', () async {
    final c = buatCatatan('1', 'Widget');
    await repo.simpan(c);
    await repo.simpan(c.copyWith(judul: 'Widget Dasar'));
    expect(repo.jumlah, 1);
    expect(repo.ambil('1')?.judul, 'Widget Dasar');
  });

  test('delete dan hapusSemua', () async {
    await repo.simpan(buatCatatan('1', 'A'));
    await repo.simpan(buatCatatan('2', 'B'));
    await repo.hapus('1');
    expect(repo.ambil('1'), isNull);
    expect(repo.jumlah, 1);
    await repo.hapusSemua();
    expect(repo.ambilSemua(), isEmpty);
  });

  test('data tetap ada setelah box ditutup dan dibuka lagi', () async {
    await repo.simpan(buatCatatan('1', 'Persisten'));
    await Hive.close(); // seperti aplikasi ditutup
    final repoBaru = await CatatanRepository.buka();
    final c = repoBaru.ambil('1');
    expect(c?.judul, 'Persisten');
    expect(c?.dibuatPada, DateTime(2025, 1, 1, 8, 0)); // adapter bekerja
  });
}
```

```
flutter test
```

```
00:00 +0: loading test/catatan_repository_test.dart
00:00 +0: create dan read
00:00 +1: update menimpa data dengan id yang sama
00:00 +2: delete dan hapusSemua
00:00 +3: data tetap ada setelah box ditutup dan dibuka lagi
00:00 +4: All tests passed!
```

Test terakhir paling penting: ia menutup Hive (seperti aplikasi ditutup), membuka box lagi, lalu memastikan data dan tanggalnya terbaca dengan benar. Itu bukti adapter kita bekerja dua arah.

ℹ️ **Pengembangan lanjutan**

- Tambahkan field `disematkan` (pin) dengan menaikkan versi adapter menjadi 2 seperti contoh `DosenAdapter` di bagian 5.
- Ganti tombol hapus dengan gestur geser (`Dismissible`) dan SnackBar "Batalkan" yang menyimpan ulang catatan.
- Di Modul 14, pola repository yang sama dipakai lagi: data utama dari Supabase, lalu Hive menjadi cache offline.

11. Kesalahan Umum dan Debugging

Kumpulan error yang paling sering ditemui mahasiswa saat memakai Hive, lengkap dengan penyebab dan cara memperbaikinya.

Pesan error / gejala	Penyebab	Solusi
`HiveError: Box not found. Did you forget to call Hive.openBox()?`	`Hive.box('x')` dipanggil sebelum `openBox('x')`, atau nama box salah ketik.	Buka semua box di `main()` s Simpan nama box dalam konst (`CatatanRepository.namaB`
`HiveError: Cannot write, unknown type: Catatan. Did you forget to register an adapter?`	Adapter belum didaftarkan.	Panggil `Hive.registerAdapter(Ca` setelah `initFlutter` dan se
`HiveError: There is already a TypeAdapter for typeId 0.`	Dua adapter memakai typeId sama, atau adapter yang sama didaftarkan dua kali (sering terjadi di test).	Beri typeId berbeda untuk seti cek dulu dengan `if (!Hive.isAdapterRegister`
`Cannot read, unknown typeId: 34. Did you forget to register an adapter?`	File box berisi objek bertipe yang adapternya belum didaftarkan, atau typeId-nya pernah diubah.	Daftarkan adapter yang benar. typeId setelah ada data. Saat c uninstall aplikasi untuk mengh
`RangeError` / data acak saat membaca objek	Urutan `read()` tidak sama dengan `write()`, atau field ditambah tanpa migrasi.	Samakan urutan. Pakai byte ve adapter hasil generate (bagian
`type '_Map<dynamic, dynamic>' is not a subtype of type 'Map<String, dynamic>'`	Map yang dibaca ulang dari file selalu bertipe `Map<dynamic, dynamic>`.	`Map<String, dynamic>.fr` buat `fromMap(Map map)`.
"Binding has not yet been initialized"	`await Hive.initFlutter()` dipanggil sebelum `WidgetsFlutterBinding.ensureInitialized()`.	Jadikan `ensureInitialized` di `main()`.

Pesan error / gejala	Penyebab	Solusi
UI tidak berubah setelah data disimpan	Menulis ke box tanpa `notifyListeners()` / `setState`, atau tidak memakai `listenable()`.	Lewatkan perubahan lewat pro memanggil `notifyListener` UI dengan `ValueListenable`
Data hilang setelah aplikasi ditutup paksa	`put` tidak di-`await`, dan aplikasi mati sebelum data tertulis ke file.	Selalu `await` operasi tulis ya
Data "hilang" setelah ganti nama box atau ganti package	Nama box baru berarti file baru. Box lama masih ada tetapi tidak dibuka.	Jangan mengganti nama box t datanya.

Hot reload vs hot restart

- **Hot reload** tidak menjalankan ulang `main()`. Jika kamu baru menambah `openBox` atau `registerAdapter` di `main()`, perubahannya belum terpakai, sehingga error "Box not found" bisa muncul. Lakukan **hot restart** (tombol ⟳ atau tekan R di terminal).
- **Hot restart** menjalankan ulang `main()` dari awal, tetapi file box tetap ada di perangkat. Data **tidak** ter-reset. Untuk benar-benar mengosongkan data, uninstall aplikasi atau panggil `box.clear()`.
- Setelah mengubah adapter dengan cara yang tidak kompatibel (mengubah urutan field), hot restart akan mencoba membaca data lama dengan adapter baru dan bisa crash. Uninstall aplikasi saat masih tahap development.

Tips debugging

```
// 1. Lihat seluruh isi box di Debug Console
debugPrint(Hive.box('pengaturan').toMap().toString());

// 2. Cek apakah box sudah terbuka sebelum dipakai
if (!Hive.isBoxOpen('catatan_box')) {
  await Hive.openBox<Catatan>('catatan_box');
}

// 3. Hapus satu box beserta filenya (misalnya tombol "Reset data" saat development)
await Hive.deleteBoxFromDisk('catatan_box');

// 4. Tangkap HiveError agar aplikasi tidak langsung crash
try {
  await Hive.openBox<Catatan>('catatan_box');
} on HiveError catch (e) {
  debugPrint('Gagal membuka box: ${e.message}');
}
```

💡 **Ringkasan modul**

- Box = kumpulan key-value yang tersimpan permanen. Buka sekali di `main()`, lalu baca secara sinkron.
- `put` berfungsi untuk create sekaligus update. Pakai key unik (id), hindari index.
- Map dan List bisa langsung disimpan; ingat `Map.from` saat membaca.
- Objek sendiri butuh TypeAdapter dengan `typeId` unik yang tidak boleh berubah; tulis manual atau generate dengan `@GenerateAdapters`.
- UI reaktif lewat `box.listenable()` atau pola Repository + Provider.
- Data sensitif: `HiveAesCipher` dengan kunci di `flutter_secure_storage`.

Tugas Praktikum: Praktikum 11: Aplikasi Catatan Kuliah Offline dengan Hive

Sasaran Praktikum

Membangun aplikasi CRUD offline yang menyimpan objek buatan sendiri di Hive dengan arsitektur Repository + Provider, lalu mengembangkannya dengan fitur tambahan.

Langkah Pengerjaan

1. **Tugas 1 (Wajib, 20 poin):** Bangun ulang mini project Catatan Kuliah Offline dari bagian 10 sampai berjalan di emulator. Buktikan data tetap ada setelah aplikasi ditutup paksa dan dibuka kembali (lampirkan 2 screenshot).
2. **Tugas 2 (Wajib, 20 poin):** Tambahkan field `bool disematkan` (pin) pada model `Catatan`. Naikkan versi `CatatanAdapter` menjadi 2 dan baca field baru hanya jika versi >= 2, sehingga catatan lama dari Tugas 1 tetap terbaca tanpa uninstall.
3. **Tugas 3 (Wajib, 15 poin):** Tampilkan catatan yang disematkan selalu di urutan teratas (apa pun pilihan urutannya) dengan ikon pin, dan sediakan tombol untuk menyematkan atau melepas sematan.
4. **Tugas 4 (Wajib, 15 poin):** Tambahkan filter mata kuliah berupa deretan `FilterChip` di bawah kolom pencarian. Daftar mata kuliah diambil otomatis dari catatan yang ada.
5. **Tugas 5 (Wajib, 15 poin):** Simpan pilihan urutan terakhir pengguna ke box `pengaturan` (gunakan repository terpisah) sehingga urutan tidak kembali ke default saat aplikasi dibuka ulang.
6. **Tugas 6 (Wajib, 15 poin):** Tulis unit test untuk `CatatanRepository` memakai `Hive.init` di folder sementara, minimal mencakup tambah, edit, hapus, dan persistensi setelah `Hive.close()`. Semua test harus lulus dengan `flutter test`.
7. **Bonus (+10 poin):** Ganti tombol hapus dengan gestur geser `Dismissible` dan SnackBar berisi aksi "Batalkan" yang mengembalikan catatan yang terhapus.

Output yang Diharapkan

Aplikasi Catatan Kuliah yang berjalan sepenuhnya offline: daftar catatan dengan pin, pencarian, filter mata kuliah, dan urutan yang diingat; form tambah/edit dengan validasi; konfirmasi hapus; data tetap ada setelah restart. Disertai output flutter test yang semua lulus dan flutter analyze tanpa issue.

Rubrik Penilaian

Tugas 1: fungsionalitas CRUD dan bukti persistensi (20%). Tugas 2: migrasi adapter tanpa kehilangan data lama (20%). Tugas 3: logika dan tampilan pin (15%). Tugas 4: filter mata kuliah (15%). Tugas 5: pengaturan urutan tersimpan dengan repository terpisah (15%). Tugas 6: kualitas dan kelulusan unit test (15%). Bonus Dismissible + Batalkan (+10%). Pengurangan nilai jika widget mengakses Hive secara langsung atau flutter analyze masih menampilkan issue.

Kuis Pemahaman

1. Kenapa modul ini memakai package hive_ce dan hive_ce_flutter, bukan hive?

A. Package hive 2.x sudah tidak dirawat, sedangkan hive_ce adalah lanjutan versi komunitas yang API-nya hampir sama dan terus diperbarui

B. hive_ce adalah database SQL, sedangkan hive adalah NoSQL

C. hive tidak bisa dipakai di Android

D. hive_ce membutuhkan koneksi internet agar lebih cepat

2. Apa urutan yang benar di dalam main() sebelum runApp?

A. openBox, lalu Hive.initFlutter, lalu registerAdapter

B. WidgetsFlutterBinding.ensureInitialized, await Hive.initFlutter, Hive.registerAdapter, await Hive.openBox

C. registerAdapter, lalu runApp, lalu Hive.initFlutter di dalam build()

D. Hive.box, lalu Hive.openBox, lalu ensureInitialized

3. Sebuah box sudah berisi data dengan key 'nama' = 'Budi'. Apa yang terjadi saat kode await box.put('nama', 'Siti') dijalankan?

A. Muncul error karena key sudah ada

B. Box sekarang berisi dua data dengan key 'nama'

C. Nilai lama ditimpa sehingga box.get('nama') menghasilkan 'Siti'

D. Data baru disimpan dengan key angka otomatis

4. Sebuah Map disimpan ke box lalu aplikasi di-restart. Kode final m = box.get('profil') as Map<String, dynamic>; tiba-tiba error. Apa penyebab dan solusinya?

A. Hive tidak bisa menyimpan Map, jadi harus memakai TypeAdapter

B. Map yang dibaca ulang dari file bertipe Map<dynamic, dynamic>, sehingga harus disalin dengan Map<String, dynamic>.from(...)

C. Box harus dibuka dengan enkripsi agar Map terbaca

D. Key Map harus berupa int

5. Manakah aturan typeId yang BENAR pada TypeAdapter?

A. Boleh sama untuk semua model asalkan nama kelasnya berbeda

B. Sebaiknya diganti setiap kali aplikasi di-update

C. Harus unik untuk setiap tipe dan tidak boleh diubah setelah ada data yang tersimpan

D. Harus sama dengan jumlah field pada model

6. Pada adapter manual, kamu menambah field baru 'email' dan langsung memanggil readString() keempat di read(). Aplikasi lama yang sudah berisi data menjadi crash. Cara aman untuk mencegahnya adalah...

A. Menyisipkan field email di posisi pertama

B. Menulis byte versi format sejak awal, menambah field baru di akhir, dan membacanya hanya jika versi data mendukung

C. Mengganti typeId setiap kali menambah field

D. Menghapus await pada box.put

7. Apa fungsi box.listenable() yang dipasangkan dengan ValueListenableBuilder?

A. Mengenkripsi box secara otomatis

B. Membuka box secara async di dalam build()

C. Memberi kabar setiap isi box berubah sehingga builder dijalankan ulang dan UI ikut berubah tanpa setState

D. Mengirim data box ke server

8. Di mana kunci enkripsi untuk HiveAesCipher sebaiknya disimpan?

A. Ditulis langsung sebagai konstanta String di kode Dart

B. Disimpan di box Hive biasa bernama 'kunci'

C. Disimpan di flutter_secure_storage (Keystore/Keychain), dibuat sekali dengan Hive.generateSecureKey()

D. Dibuat ulang secara acak setiap aplikasi dibuka

Kunci Jawaban & Pembahasan

1. A — hive_ce (Community Edition) melanjutkan pengembangan hive yang sudah ditinggalkan pembuatnya. API inti seperti openBox, put, get, dan TypeAdapter tetap sama, sehingga yang berbeda hanya nama package dan import.

2. B — Binding harus siap sebelum memakai plugin, Hive harus tahu folder penyimpanannya, adapter harus terdaftar sebelum box yang memakai tipe tersebut dibuka, dan box dibuka sebelum dipakai di UI.

3. C — put() berfungsi sebagai create sekaligus update. Key yang sama akan menimpa value lama. Key angka otomatis hanya dibuat oleh add().

4. B — Setelah dibaca dari disk, Hive mengembalikan Map<dynamic, dynamic>. Cast langsung ke Map<String, dynamic> gagal; gunakan Map<String, dynamic>.from() atau fromMap yang menerima Map.

5. C — Hive menulis typeId di depan setiap objek untuk mencari adapter saat membaca. typeId ganda memicu HiveError, dan mengganti typeId membuat data lama tidak bisa dibaca.

6. B — Adapter manual membaca data berurutan tanpa nama field. Dengan byte versi, read() tahu apakah data lama memiliki field baru. Adapter hasil generate menyelesaikan masalah ini dengan menulis indeks setiap field.

7. C — listenable() dari hive_ce_flutter menghasilkan ValueListenable yang memicu rebuild ValueListenableBuilder setiap ada put, add, delete, atau clear.

8. C — Kunci di kode atau di box biasa mudah dibaca orang lain. Kunci yang dibuat ulang setiap kali membuat data lama tidak bisa didekripsi. Secure storage menyimpan kunci di penyimpanan aman milik sistem operasi.

MODUL 12 · SUPABASE BAAS

Backend as a Service dengan Supabase

Mengenal Supabase sebagai backend siap pakai berbasis PostgreSQL. Mahasiswa membuat project, menyimpan kunci dengan aman, membuat tabel lewat SQL Editor, mengamankan data dengan Row Level Security, lalu melakukan query CRUD, autentikasi, Realtime, dan Storage dari Flutter.

Durasi Pertemuan 12 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management Provider
Modul 09: REST API & JSON
Modul 11: Hive
Mata kuliah Basis Data (SQL dasar)

Capaian Pembelajaran

- Menjelaskan komponen Supabase (Database, Auth, RLS, Realtime, Storage) dan membandingkannya dengan Firebase.
- Membuat project Supabase, menemukan Project URL dan publishable key, serta membedakannya dengan secret key.
- Mengonfigurasi supabase_flutter dengan Supabase.initialize dan --dart-define-from-file tanpa membocorkan kunci ke repository.
- Membuat tabel dengan SQL (tipe data, default, constraint, uuid, foreign key ke auth.users).
- Menulis policy Row Level Security memakai auth.uid() dan menjelaskan perbedaan using dan with check.
- Melakukan select (filter, order, limit, range), insert, update, delete, upsert, single dan maybeSingle dari Flutter.
- Menangani PostgrestException, AuthException, dan kegagalan jaringan dengan pesan yang ramah pengguna.
- Menggunakan signUp, signInWithPassword, signOut, currentSession, dan onAuthStateChange.

Daftar Isi Modul

1. Apa itu Supabase? (vs Firebase)
2. Membuat Project & Menemukan API Key
3. Instalasi supabase_flutter & Supabase.initialize
4. Keamanan Kunci: env.json & --dart-define

5. Membuat Tabel dengan SQL Editor
6. Row Level Security (RLS) & Policy
7. Query Dasar: Select, Insert, Update, Delete, Upsert
8. Menangani Error Supabase
9. Dasar Supabase Auth
10. Sekilas Realtime & Storage
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Supabase? (vs Firebase)

Sampai Modul 11, semua data aplikasi kita tersimpan di HP (SharedPreferences, SQLite, Hive). Data itu hilang kalau aplikasi di-uninstall dan tidak bisa dibuka dari HP lain. Untuk aplikasi sungguhan kita butuh **backend**: server yang menyimpan data di cloud dan mengatur siapa boleh mengakses apa.

Apa itu Backend as a Service (BaaS)?

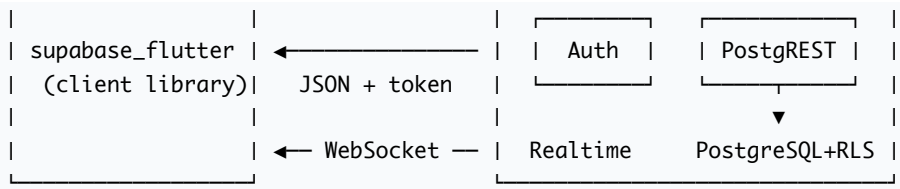
Membuat backend sendiri berarti menyewa server, memasang database, menulis REST API (seperti yang kita konsumsi di Modul 09), membuat sistem login, mengurus keamanan, dan seterusnya. **BaaS** menyediakan semua itu dalam bentuk siap pakai. Kita cukup membuat tabel lewat dashboard, lalu memanggilnya dari Flutter dengan library resmi.

Supabase adalah BaaS open source yang dibangun di atas database **PostgreSQL**. Karena memakai PostgreSQL, semua yang Anda pelajari di mata kuliah Basis Data (tabel, relasi, *primary key*, *foreign key*, SQL) langsung terpakai.

Fitur utama Supabase

Fitur	Fungsi	Dipakai di modul
Database (Postgres)	Tabel relasional sungguhan. Bisa diakses dari Flutter tanpa menulis REST API sendiri, karena Supabase membuat API-nya otomatis (PostgREST).	12, 14
Auth	Register, login email/password, magic link, Google, dan lain-lain. Sesi login disimpan dan diperbarui otomatis.	12, 13
Row Level Security (RLS)	Aturan keamanan per baris di database, misalnya "user hanya boleh melihat catatannya sendiri".	12, 13, 14
Realtime	Aplikasi menerima perubahan data secara langsung (seperti chat) tanpa perlu refresh.	12
Storage	Menyimpan file: foto profil, PDF tugas, dan sejenisnya.	12 (singkat)
Edge Functions	Menjalankan kode server (TypeScript) untuk logika khusus.	Di luar cakupan

Bagaimana Flutter berbicara dengan Supabase?



Kita tidak menulis URL endpoint satu per satu seperti di Modul 09. Cukup panggil `supabase.from('tabel').select()`. Library-lah yang mengubahnya menjadi request HTTP, menyertakan token login, lalu mengembalikan JSON yang sudah di-decode.

Supabase vs Firebase

Firebase (Google) adalah BaaS paling populer di dunia Flutter. Keduanya bagus. Perbedaan utamanya:

Aspek	Supabase	Firebase
Jenis database	Relasional (PostgreSQL), memakai SQL	NoSQL dokumen (Cloud Firestore)
Relasi antar data	Foreign key dan JOIN bawaan	Denormalisasi manual
Aturan keamanan	Row Level Security (policy SQL)	Security Rules (bahasa khusus)
Open source / self-host	Ya, bisa dipasang di server sendiri	Tidak
Setup di Flutter	Cukup URL + key, tanpa file konfigurasi native	Perlu FlutterFire CLI dan file konfigurasi per platform
Paket gratis	Ada (project bisa di-*pause* otomatis bila lama tidak aktif)	Ada (Spark plan)
Cocok untuk	Data terstruktur, mahasiswa yang sudah belajar SQL	Data fleksibel, ekosistem Google

> ℹ️ **Kenapa kita memilih Supabase?**
>
> Kurikulum Informatika sudah membekali Anda dengan SQL dan desain basis data relasional. Dengan Supabase, ilmu itu langsung terpakai, dan konsep keamanan per baris (RLS) adalah praktik yang juga dipakai di dunia industri.

Peta modul berikutnya

- **Modul 12 (ini):** dasar Supabase, yaitu membuat project, tabel, RLS, query CRUD, Auth, dan sekilas Realtime & Storage.
- **Modul 13:** Proyek "Catatan Kuliah" bagian 1, berisi arsitektur aplikasi serta login & register lengkap.
- **Modul 14:** Proyek bagian 2, yaitu CRUD catatan dengan Supabase plus cache offline memakai Hive.

2. Membuat Project & Menemukan API Key

Sebelum menulis kode Flutter, kita siapkan dulu "server"-nya, yaitu sebuah *project* di Supabase. Semua langkah di bawah gratis dan tidak memerlukan kartu kredit.

Langkah 1: Membuat akun

1. Buka supabase.com, klik **Start your project**.
2. Pilih **Continue with GitHub** (paling cepat) atau daftar dengan email.
3. Saat pertama kali masuk, Anda diminta membuat **Organization**. Isi nama (misalnya `praktikum-mobile`), pilih tipe *Personal* dan plan *Free*.

Langkah 2: Membuat project baru

1. Di dashboard klik **New project**.
2. **Project name**: `catatan-kuliah`.
3. **Database Password**: klik *Generate a password*, lalu **simpan** di tempat aman. Password ini adalah password database Postgres (dipakai untuk koneksi langsung ke database), *bukan* password login aplikasi.
4. **Region**: pilih yang paling dekat dengan pengguna, yaitu **Southeast Asia (Singapore)**, agar latensi kecil.
5. Klik **Create new project** dan tunggu 1–2 menit sampai status project siap.

Langkah 3: Menemukan Project URL dan API key

Aplikasi Flutter hanya butuh **dua nilai** untuk terhubung:

1. Klik tombol **Connect** di bagian atas dashboard project, atau buka **Project Settings** (ikon roda gigi) → **API Keys**.
2. **Project URL** berbentuk `https://<project-ref>.supabase.co`. `project-ref` adalah 20 huruf acak, sama dengan yang terlihat di address bar dashboard.
3. **Publishable key** berbentuk `sb_publishable_...`. Inilah kunci yang dipasang di aplikasi.

Jenis kunci	Bentuk	Boleh di aplikasi Flutter?	Keterangan
Publishable key	`sb_publishable_...`	✅ Ya	Kunci "publik". Aman ada di aplikasi *asalkan RLS aktif*. Pengganti anon key.
Anon key (legacy)	JWT panjang `eyJhbGciOi...`	✅ Ya (cara lama)	Ada di tab *Legacy API Keys*. Fungsinya sama dengan publishable key.
Secret key	`sb_secret_...`	❌ TIDAK PERNAH	Melewati semua RLS (akses penuh). Hanya untuk server/backend.
service_role (legacy)	JWT panjang	❌ TIDAK PERNAH	Versi lama dari secret key, sama berbahayanya.

ℹ️ **Publishable key vs anon key**

Supabase memperkenalkan format kunci baru `sb_publishable_...` dan `sb_secret_...`. Package `supabase_flutter` versi terbaru memakai parameter `publishableKey` di `Supabase.initialize()`. Parameter lama `anonKey` masih ada tetapi sudah *deprecated*. Jika Anda menemukan tutorial lama yang menulis `anonKey: 'eyJ...'`, cara itu masih berjalan, tetapi untuk project baru gunakan `publishableKey`.

💡 **Tampilan dashboard berubah?**

Supabase cukup sering memperbarui tampilan dashboard. Jika menu di atas tidak persis sama, cari kata kunci **API Keys** atau tombol **Connect**. Nilai yang dicari tetap sama, yaitu Project URL dan publishable key.

Langkah 4: Mengenal menu dashboard yang akan sering dipakai

Menu	Kegunaan
Table Editor	Melihat dan mengedit isi tabel seperti spreadsheet.
SQL Editor	Menjalankan perintah SQL: membuat tabel, policy, trigger. Kita akan banyak memakai menu ini.
Authentication	Daftar user yang sudah register, pengaturan provider login (Email), dan template email.
Storage	Membuat *bucket* untuk file.
Database → Policies	Melihat policy RLS setiap tabel (juga tersedia lewat Authentication → Policies).
Logs	Melihat request yang masuk dan error, berguna saat debugging.

> ⚠️ **Project gratis bisa di-pause**
>
> Project pada plan Free akan di-*pause* otomatis bila tidak ada aktivitas selama beberapa waktu. Jika aplikasi tiba-tiba tidak bisa terhubung menjelang praktikum, buka dashboard lalu klik **Restore project**.

3. Instalasi supabase_flutter & Supabase.initialize

Supabase menyediakan package resmi `supabase_flutter`. Di dalamnya sudah ada semua yang dibutuhkan: client database (PostgREST), Auth, Realtime, dan Storage, termasuk penyimpanan sesi login otomatis di perangkat.

Langkah 1: Tambahkan package

```
flutter pub add supabase_flutter
```

Perintah ini menambahkan baris `supabase_flutter: ^x.y.z` di `pubspec.yaml` dengan versi terbaru yang kompatibel, lalu menjalankan `flutter pub get`.

Langkah 2: Inisialisasi di `main()`

Supabase harus di-*initialize* **satu kali**, sebelum `runApp()`. Contoh paling sederhana (nilai URL dan key masih ditulis langsung; di section berikutnya kita pindahkan ke file konfigurasi):

```dart
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Future<void> main() async {
  // 1. Wajib: memastikan binding Flutter siap sebelum kode async + plugin.
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Hubungkan aplikasi ke project Supabase (cukup SEKALI).
  await Supabase.initialize(
    url: 'https://abcdefghijklm.supabase.co',
    publishableKey: 'sb_publishable_xxxxxxxxxxxxxxxxxxxx',
  );

  runApp(const MaterialApp(home: Scaffold()));
}

// 3. Ambil client di mana saja setelah initialize.
final supabase = Supabase.instance.client;
```

Penjelasan baris penting:

- `main()` diubah menjadi `async` dan mengembalikan `Future<void>` karena kita perlu `await` sebelum `runApp()`.
- `WidgetsFlutterBinding.ensureInitialized()` wajib dipanggil sebelum memakai plugin (di sini untuk penyimpanan sesi) di dalam `main()`. Lupa baris ini akan memunculkan error *Binding has not yet been initialized*.
- `Supabase.initialize(url:, publishableKey:)` membuat client sekaligus memulihkan sesi login yang tersimpan dari pembukaan aplikasi sebelumnya.
- `Supabase.instance.client` adalah objek `SupabaseClient` yang kita pakai untuk semua operasi. Memanggilnya *sebelum* initialize selesai akan error.

Langkah 3: Izin internet di Android

Proyek Flutter baru hanya memberi izin internet untuk mode *debug*. Agar APK *release* juga bisa mengakses Supabase, tambahkan izin di manifest utama:

`android/app/src/main/AndroidManifest.xml`

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Tambahkan baris ini, di atas tag <application> -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="catatan_kuliah"
        ...
```

> 💡 **Platform lain**
>
> iOS tidak membutuhkan izin tambahan. Untuk macOS, aktifkan `com.apple.security.network.client` di file `macos/Runner/DebugProfile.entitlements` dan `Release.entitlements`. Flutter Web langsung bisa dipakai.

Langkah 4: Uji koneksi

Jalankan aplikasi. Jika tidak ada error merah di konsol, inisialisasi berhasil. Perlu diingat, `initialize` tidak mengirim query ke database, jadi URL atau key yang salah baru ketahuan saat query pertama dijalankan (akan kita coba di section Query).

> ⚠️ **Kesalahan umum**
>
> - **"You must initialize the supabase instance before calling Supabase.instance"**: `Supabase.instance.client` dipanggil sebelum `await Supabase.initialize(...)`, misalnya di variabel global yang dibaca terlalu awal atau karena lupa `await`.
> - **Memanggil initialize di dalam `build()`**: initialize cukup sekali di `main()`.
> - **URL diakhiri `/rest/v1`**: cukup tulis URL dasarnya, `https://xxxx.supabase.co`.

4. Keamanan Kunci: env.json & --dart-define

Menulis URL dan key langsung di kode (*hard-code*) memang praktis, tetapi berisiko: begitu kode di-*push* ke GitHub publik, siapa pun bisa melihat dan memakai project Anda. Di section ini kita belajar memisahkan konfigurasi dari kode.

Aturan emas

1. **Jangan pernah** memasukkan *secret key* (`sb_secret_...`) atau *service_role* ke aplikasi Flutter. Semua isi APK bisa dibongkar (*decompile*), dan kunci itu melewati seluruh RLS, sehingga pemegangnya bisa membaca dan menghapus data semua user.
2. Publishable key **memang boleh** ada di aplikasi. Yang melindungi data adalah **RLS**, bukan kerahasiaan publishable key.
3. Meskipun publishable key tidak rahasia, tetap simpan di file konfigurasi terpisah yang **tidak di-commit**. Dengan begitu setiap anggota tim bisa memakai project masing-masing tanpa mengubah kode, dan repo tidak menjadi sasaran penyalahgunaan kuota.

Cara 1: `--dart-define`

Flutter bisa "menyuntikkan" nilai konstanta saat build. Nilai itu dibaca dengan `String.fromEnvironment`:

`lib/config/env.dart`

```dart
// lib/config/env.dart
class Env {
  // Nilai diisi saat build oleh --dart-define / --dart-define-from-file.
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

Jalankan dengan:

```
flutter run \
  --dart-define=SUPABASE_URL=https://abcdefghijklm.supabase.co \
  --dart-define=SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxxx
```

- `String.fromEnvironment` **harus** dipakai sebagai `const`. Nilainya ditentukan saat kompilasi, bukan saat aplikasi berjalan.
- Jika lupa memberi `--dart-define`, nilainya string kosong `''`. Karena itu kita sediakan `isConfigured` untuk menampilkan pesan yang jelas.

Cara 2 (disarankan): `--dart-define-from-file`

Mengetik dua define setiap kali menjalankan aplikasi cukup merepotkan. Simpan saja di file JSON di root proyek (sejajar `pubspec.yaml`):

`env.json`

```
{
  "SUPABASE_URL": "https://abcdefghijklm.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxxxxxx"
}
```

```
flutter run --dart-define-from-file=env.json
```

Lalu cegah file itu ikut ter-commit dengan menambahkan baris berikut ke `.gitignore`, dan sediakan contoh tanpa isi untuk teman satu tim:

`.gitignore`

```
# Konfigurasi rahasia/pribadi
env.json
```

`env.example.json`

```
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Agar tombol Run di VS Code ikut membaca env.json

Buat file `.vscode/launch.json`:

`.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "catatan_kuliah",
      "request": "launch",
      "type": "dart",
      "args": ["--dart-define-from-file=env.json"]
    }
  ]
}
```

Di Android Studio: *Run* → *Edit Configurations* → isi **Additional run args** dengan `--dart-define-from-file=env.json`.

Inisialisasi memakai Env

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );

  runApp(const MyApp());
}
```

> ⚠️ **Sudah terlanjur push key ke GitHub?**
>
> Hapus dari repo saja tidak cukup karena riwayat commit masih menyimpannya. Buka **Project Settings → API Keys**, buat kunci baru, lalu hapus (*revoke*) kunci lama. Hal ini **wajib** dilakukan segera jika yang bocor adalah secret key.

5. Membuat Tabel dengan SQL Editor

Tabel bisa dibuat lewat **Table Editor** dengan klik-klik, tetapi di praktikum ini kita memakai **SQL Editor**. Perintah SQL bisa disimpan, dibagikan, dan dijalankan ulang persis sama. Kebiasaan ini juga yang dipakai tim profesional (disebut *migration*).

Membuka SQL Editor

1. Di sidebar dashboard klik **SQL Editor**.
2. Klik **New query** (atau ikon +).
3. Tempel SQL, lalu klik **Run** (atau `Ctrl`/`Cmd` + `Enter`).
4. Hasil atau pesan error muncul di panel bawah. "Success. No rows returned" artinya perintah berhasil.

Tabel pertama: `mata_kuliah`

```
create table public.mata_kuliah (
  id bigint generated always as identity primary key,
  kode text not null unique,
  nama text not null,
  sks int not null default 3 check (sks between 1 and 6),
  semester int not null check (semester between 1 and 8),
  created_at timestamptz not null default now()
);
```

Kita bedah baris demi baris:

- `public.mata_kuliah` : tabel dibuat di schema `public`. Hanya tabel di schema ini yang bisa diakses lewat API dari Flutter.
- `bigint generated always as identity primary key` : id berupa angka yang naik otomatis (1, 2, 3, ...). Kita tidak perlu mengirim id saat insert.
- `text not null unique` : kode wajib diisi dan tidak boleh ada yang sama. Insert kode ganda akan ditolak dengan error `23505`.
- `default 3` : jika kolom tidak dikirim saat insert, nilainya otomatis 3.
- `check (sks between 1 and 6)` : database menolak nilai di luar rentang (error `23514`). Validasi di database adalah lapis pertahanan terakhir, meskipun form Flutter juga sudah memvalidasi.
- `timestamptz not null default now()` : waktu dibuat terisi otomatis dalam UTC dengan zona waktu.

Mengisi data awal

```
insert into public.mata_kuliah (kode, nama, sks, semester) values
  ('INF101', 'Algoritma dan Pemrograman', 4, 1),
  ('INF102', 'Matematika Diskrit', 3, 1),
  ('INF201', 'Struktur Data', 3, 2),
  ('INF202', 'Basis Data', 3, 3),
```

```
  ('INF301', 'Pemrograman Mobile', 3, 5),
  ('INF302', 'Jaringan Komputer', 3, 4),
  ('INF303', 'Kecerdasan Buatan', 3, 5),
  ('INF304', 'Rekayasa Perangkat Lunak', 3, 4);

-- Cek hasilnya
select * from public.mata_kuliah order by semester;
```

Tipe data PostgreSQL yang sering dipakai

Tipe Postgres	Tipe di Dart	Contoh penggunaan
`text`	`String`	nama, judul, isi catatan (tanpa batas panjang)
`int` / `bigint`	`int`	sks, jumlah, id identity
`numeric` / `float8`	`num` / `double`	IPK, harga
`boolean`	`bool`	`is_done`, `is_pinned`
`timestamptz`	`String` ISO-8601 → `DateTime.parse()`	`created_at`
`date`	`String` `'2026-10-28'`	tanggal lahir, deadline
`uuid`	`String`	id user (`auth.users.id`), id catatan
`jsonb`	`Map` / `List`	data fleksibel/pengaturan

Identity (angka) atau UUID?

```
-- Contoh tabel ber-id UUID (akan kita pakai untuk catatan di Modul 14)
create table public.contoh_uuid (
  id uuid primary key default gen_random_uuid(),
  judul text not null
);
```

	`bigint identity`	`uuid default gen_random_uuid()`
Bentuk	1, 2, 3, ...	`3f6c2a9e-8b1d-4c7a-...`
Bisa ditebak?	Ya (id berikutnya mudah ditebak)	Tidak
Bisa dibuat di HP sebelum online?	Tidak	Ya (package `uuid`)
Cocok untuk	Data referensi/master	Data milik user, sinkronisasi offline

Relasi ke tabel user (`auth.users`)

Supabase menyimpan akun hasil register di tabel `auth.users` (schema `auth`, dikelola Supabase, tidak bisa diakses langsung lewat API). Tabel milik kita cukup menyimpan id user sebagai *foreign key*:

```
create table public.contoh_milik_user (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  isi text not null
);
```

- `default auth.uid()`: saat insert dari aplikasi, `user_id` otomatis terisi id user yang sedang login. Aplikasi tidak perlu mengirimnya.
- `on delete cascade`: jika akun user dihapus, semua barisnya ikut terhapus.

> 💡 **Membersihkan tabel latihan**
>
> Tabel `contoh_uuid` dan `contoh_milik_user` hanya untuk latihan. Hapus dengan `drop table public.contoh_uuid, public.contoh_milik_user;`

⚠️ **Tabel baru tanpa RLS = terbuka untuk siapa saja**

Tabel yang dibuat lewat SQL tidak otomatis dilindungi. Selama RLS belum aktif, siapa pun yang memegang publishable key bisa membaca *dan mengubah* isinya (dashboard akan memberi label peringatan **UNRESTRICTED**). Sebaliknya, jika RLS aktif tetapi belum ada policy, hasil query dari aplikasi selalu **kosong**. Section berikutnya membahas cara mengatur keduanya dengan benar.

6. Row Level Security (RLS) & Policy

Publishable key ada di dalam aplikasi, sehingga siapa pun yang membongkar APK bisa memakainya untuk memanggil API Supabase. Lalu apa yang mencegah orang iseng membaca catatan milik mahasiswa lain? Jawabannya adalah **Row Level Security (RLS)**.

Konsep: satpam di setiap baris

Bayangkan setiap tabel dijaga seorang satpam. Saat RLS **aktif**, satpam menolak *semua* akses, kecuali yang diizinkan secara eksplisit oleh **policy**. Policy adalah aturan berbentuk kondisi SQL yang dicek untuk *setiap baris*:

```
Request: SELECT * FROM notes   (dikirim oleh user dengan id = A)

 baris  | user_id | policy: auth.uid() = user_id | hasil
--------+---------+------------------------------+---------------
 note 1 |    A    |  A = A  → true               | ✅ dikirim
 note 2 |    B    |  A = B  → false              | ❌ disembunyikan
 note 3 |    A    |  A = A  → true               | ✅ dikirim
```

Aplikasi tidak perlu menulis `.eq('user_id', ...)`. Database sendiri yang memfilter, sehingga tidak bisa diakali dari sisi aplikasi.

Peran (role) dalam request

Role	Kapan	Nilai `auth.uid()`
`anon`	Request dengan publishable key, user **belum login**	`null`
`authenticated`	User **sudah login** (token sesi otomatis dikirim oleh supabase_flutter)	id user tersebut (uuid)
`service_role`	Memakai secret key (khusus server)	Melewati RLS sepenuhnya

Contoh 1: data publik yang boleh dibaca siapa saja

```sql
-- 1. Aktifkan RLS (setelah ini semua akses ditolak)
alter table public.mata_kuliah enable row level security;

-- 2. Izinkan SELECT untuk semua orang
create policy "Semua orang boleh membaca mata kuliah"
on public.mata_kuliah
for select
to anon, authenticated
using (true);

-- 3. Hanya user yang sudah login boleh menambah
```

```sql
create policy "User login boleh menambah mata kuliah"
on public.mata_kuliah
for insert
to authenticated
with check (true);
```

Tanpa policy UPDATE dan DELETE, tidak ada seorang pun (dari aplikasi) yang bisa mengubah atau menghapus mata kuliah. Itulah yang kita mau untuk data master.

Contoh 2: data milik pribadi (pola yang paling sering dipakai)

```sql
create policy "User melihat catatannya sendiri"
on public.notes
for select
to authenticated
using ( (select auth.uid()) = user_id );
```

Kita bedah baris per baris:

Baris	Arti
`create policy "User melihat ..."`	Nama policy, bebas tetapi harus unik per tabel. Tulis nama yang mudah dipahami.
`on public.notes`	Tabel yang diatur.
`for select`	Operasi yang diatur: `select`, `insert`, `update`, `delete`, atau `all`.
`to authenticated`	Policy hanya berlaku untuk user yang sudah login. User anon langsung ditolak.
`using ( ... )`	Kondisi untuk baris yang **sudah ada**: baris mana yang boleh dilihat/diubah/dihapus.
`auth.uid()`	Fungsi bawaan Supabase yang mengembalikan id user dari token login pada request ini.
`(select auth.uid())`	Dibungkus `select` agar Postgres cukup menghitungnya sekali per query, bukan sekali per baris. Hasilnya sama tetapi lebih cepat.
`= user_id`	Baris lolos hanya jika kolom `user_id` sama dengan id user yang login.

`using` vs `with check`

Operasi	`using` (baris lama)	`with check` (baris baru)
SELECT	✅ dipakai	–
INSERT	–	✅ dipakai: data yang akan masuk harus memenuhi syarat
UPDATE	✅ baris mana yang boleh diubah	✅ hasil perubahan harus tetap memenuhi syarat
DELETE	✅ dipakai	–

```
create policy "User menambah catatannya sendiri"
on public.notes for insert to authenticated
with check ( (select auth.uid()) = user_id );

create policy "User mengubah catatannya sendiri"
on public.notes for update to authenticated
using ( (select auth.uid()) = user_id )        -- hanya baris miliknya
with check ( (select auth.uid()) = user_id );  -- tidak boleh "memindahkan" ke user lain

create policy "User menghapus catatannya sendiri"
on public.notes for delete to authenticated
using ( (select auth.uid()) = user_id );
```

Apa yang terjadi jika ditolak RLS?

Operasi	Gejala di Flutter
SELECT	**Tidak error**, hanya mengembalikan list kosong `[]`. Ini yang paling sering membingungkan!
INSERT	`PostgrestException` dengan code `42501`: *new row violates row-level security policy*
UPDATE / DELETE	Tidak error, tetapi tidak ada baris yang berubah (0 baris cocok). Jika memakai `.select().single()`, muncul error karena hasilnya 0 baris.

> 💡 **Melihat dan menguji policy**
>
> Buka **Authentication → Policies** (atau Database → Policies) untuk melihat semua policy per tabel. Di SQL Editor, query dijalankan sebagai admin (`postgres`) sehingga RLS *tidak* berlaku. Jadi data yang terlihat di SQL Editor belum tentu terlihat dari aplikasi.

⚠️ **Kesalahan umum RLS**

- **Lupa** `enable row level security` : tabel terbuka untuk publik. Dashboard menandainya *UNRESTRICTED*.
- **RLS aktif tetapi lupa membuat policy SELECT**: data "hilang", list selalu kosong.
- **Policy** `using (true)` **untuk data pribadi**: semua user bisa melihat data user lain.
- **Insert tanpa** `user_id` **dan tanpa** `default auth.uid()` : `user_id` bernilai null sehingga `with check` gagal dan muncul error 42501.

7. Query Dasar: Select, Insert, Update, Delete, Upsert

Setelah tabel dan policy siap, saatnya mengakses data dari Flutter. Semua query dimulai dengan `supabase.from('nama_tabel')`, lalu disambung method lain seperti rantai (*method chaining*), mirip menyusun kalimat SQL.

Semua contoh di section ini mengasumsikan bagian atas file berisi:

```dart
import 'package:flutter/foundation.dart'; // untuk debugPrint
import 'package:supabase_flutter/supabase_flutter.dart';

final supabase = Supabase.instance.client;
```

Pemetaan SQL ke Dart

SQL	supabase_flutter
`SELECT * FROM t`	`.from('t').select()`
`WHERE a = 1`	`.eq('a', 1)`
`WHERE a <> 1`	`.neq('a', 1)`
`>`, `>=`, `<`, `<=`	`.gt()`, `.gte()`, `.lt()`, `.lte()`
`WHERE nama ILIKE '%x%'`	`.ilike('nama', '%x%')`
`WHERE a IN (1,2)`	`.inFilter('a', [1, 2])`
`ORDER BY a DESC`	`.order('a', ascending: false)`
`LIMIT 10`	`.limit(10)`
`LIMIT 10 OFFSET 20`	`.range(20, 29)`
`INSERT` / `UPDATE` / `DELETE`	`.insert({...})` / `.update({...})` / `.delete()`

1. SELECT: membaca data

Pertama, siapkan model seperti di Modul 09 agar data aman dari salah tipe:

```dart
class MataKuliah {
  const MataKuliah({
    required this.id,
    required this.kode,
```

```
    required this.nama,
    required this.sks,
    required this.semester,
  });

  final int id;
  final String kode;
  final String nama;
  final int sks;
  final int semester;

  // Satu baris hasil query Supabase = Map<String, dynamic>
  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    return MataKuliah(
      id: json['id'] as int,
      kode: json['kode'] as String,
      nama: json['nama'] as String,
      sks: json['sks'] as int,
      semester: json['semester'] as int,
    );
  }
}
```

```
Future<List<MataKuliah>> ambilSemua() async {
  // SELECT * FROM mata_kuliah
  final List<Map<String, dynamic>> rows = await supabase
      .from('mata_kuliah')
      .select();
  return rows.map(MataKuliah.fromJson).toList();
}
```

- Hasil `select()` berupa `List<Map<String, dynamic>>`. Satu Map adalah satu baris, dengan key berupa nama kolom.
- `rows.map(MataKuliah.fromJson).toList()` mengubah setiap Map menjadi objek. Polanya sama persis dengan Modul 09.

Mengambil sebagian kolom saja (lebih hemat kuota dan lebih cepat):

```
Future<void> ambilKolomTertentu() async {
  // SELECT kode, nama FROM mata_kuliah
  final rows = await supabase.from('mata_kuliah').select('kode, nama');
  for (final row in rows) {
    debugPrint('${row['kode']} - ${row['nama']}');
  }
}
```

2. Filter

```
Future<List<Map<String, dynamic>>> contohFilter() async {
  // WHERE semester = 5
  final semester5 = await supabase
      .from('mata_kuliah')
      .select()
      .eq('semester', 5);
```

```dart
  // WHERE sks >= 4
  final sksBesar = await supabase.from('mata_kuliah').select().gte('sks', 4);

  // WHERE nama ILIKE '%data%'  (cari teks, tidak peka huruf besar/kecil)
  final cari = await supabase
      .from('mata_kuliah')
      .select()
      .ilike('nama', '%data%');

  // WHERE semester IN (1, 2)
  final awal = await supabase.from('mata_kuliah').select().inFilter(
    'semester',
    [1, 2],
  );

  return [...semester5, ...sksBesar, ...cari, ...awal];
}
```

3. Urutan dan batas jumlah

```dart
Future<List<Map<String, dynamic>>> urutDanBatasi() async {
  // SELECT * FROM mata_kuliah WHERE semester <= 4
  // ORDER BY semester ASC, nama ASC LIMIT 5
  return supabase
      .from('mata_kuliah')
      .select()
      .lte('semester', 4) // filter dulu
      .order('semester', ascending: true) // lalu urutkan
      .order('nama', ascending: true)
      .limit(5); // terakhir batasi jumlah
}
```

> **Urutan method**
>
> Tulis **filter** (`eq`, `gte`, ...) lebih dulu, baru `order`, `limit` / `range`, dan `single`. Setelah `.order()`, method filter sudah tidak tersedia, dan analyzer akan langsung memberi tahu jika urutannya salah.

4. Pagination dengan `range`

```dart
/// Pagination: halaman dimulai dari 0, setiap halaman berisi [perHalaman] baris.
Future<List<Map<String, dynamic>>> ambilHalaman(
  int halaman, {
  int perHalaman = 3,
}) {
  final dari = halaman * perHalaman; // halaman 0 -> 0, halaman 1 -> 3
  final sampai = dari + perHalaman - 1; // halaman 0 -> 2, halaman 1 -> 5
  return supabase
      .from('mata_kuliah')
```

```
      .select()
      .order('id')
      .range(dari, sampai); // indeks inklusif di kedua sisi
}
```

`range(dari, sampai)` memakai indeks mulai 0 dan **inklusif** di kedua ujung. `range(0, 2)` berarti 3 baris pertama. Panggil `ambilHalaman(0)`, `ambilHalaman(1)`, dan seterusnya saat user menggulir ke bawah (*infinite scroll*, Modul 06).

5. Mengambil satu baris: `single()` dan `maybeSingle()`

```
Future<void> ambilSatu() async {
  // single(): WAJIB tepat 1 baris. 0 atau >1 baris -> PostgrestException.
  final Map<String, dynamic> row = await supabase
      .from('mata_kuliah')
      .select()
      .eq('kode', 'INF301')
      .single();
  debugPrint(row['nama']);

  // maybeSingle(): 0 baris -> null (tidak error).
  final Map<String, dynamic>? mungkin = await supabase
      .from('mata_kuliah')
      .select()
      .eq('kode', 'TIDAK-ADA')
      .maybeSingle();
  if (mungkin == null) debugPrint('Mata kuliah tidak ditemukan');
}
```

Method	Tipe hasil	0 baris	1 baris	>1 baris
(tanpa)	`List<Map>`	`[]`	`[{...}]`	`[{...}, ...]`
`.single()`	`Map`	❌ error	✅ Map	❌ error
`.maybeSingle()`	`Map?`	✅ `null`	✅ Map	❌ error

6. INSERT: menambah data

```
Future<void> tambah() async {
  // Insert tanpa meminta data balik
  await supabase.from('mata_kuliah').insert({
    'kode': 'INF401',
    'nama': 'Komputasi Awan',
    'sks': 3,
    'semester': 7,
  });

  // Insert banyak baris sekaligus: kirim List<Map>
  await supabase.from('mata_kuliah').insert([
    {'kode': 'INF402', 'nama': 'Data Mining', 'semester': 7},
```

```
    {'kode': 'INF403', 'nama': 'Etika Profesi', 'sks': 2, 'semester': 8},
  ]);
}

Future<MataKuliah> tambahDanAmbil() async {
  // .select().single() -> baris yang baru dibuat dikembalikan,
  // lengkap dengan id dan created_at yang diisi database.
  final row = await supabase
      .from('mata_kuliah')
      .insert({'kode': 'INF404', 'nama': 'Keamanan Siber', 'semester': 7})
      .select()
      .single();
  return MataKuliah.fromJson(row);
}
```

- Kolom yang punya `default` (id, sks, created_at) boleh tidak dikirim.
- Secara default insert **tidak** mengembalikan data. Tambahkan `.select()` (dan `.single()` untuk satu baris) jika butuh id yang dibuat database. Pola ini akan kita pakai di Modul 14.

7. UPDATE: mengubah data

```
Future<void> ubah() async {
  // UPDATE mata_kuliah SET sks = 4 WHERE kode = 'INF401'
  await supabase
      .from('mata_kuliah')
      .update({'sks': 4})
      .eq('kode', 'INF401'); // JANGAN lupa filter!
}
```

> ⚠️ **Selalu pakai filter**
>
> `update()` dan `delete()` **wajib** diikuti filter seperti `.eq('id', ...)`. Di project Supabase, update/delete tanpa filter biasanya ditolak dengan pesan *UPDATE/DELETE requires a WHERE clause*. Filter yang salah lebih berbahaya lagi karena bisa mengubah baris yang tidak Anda maksud.

8. DELETE: menghapus data

```
Future<void> hapus() async {
  // DELETE FROM mata_kuliah WHERE kode = 'INF401'
  await supabase.from('mata_kuliah').delete().eq('kode', 'INF401');
}
```

Catatan: dengan policy dari section sebelumnya, tabel `mata_kuliah` tidak punya policy UPDATE/DELETE, sehingga dua contoh di atas tidak mengubah apa pun jika dijalankan dari aplikasi (0 baris). Ini contoh nyata RLS bekerja.

9. UPSERT: insert atau update

```dart
Future<void> simpanAtauPerbarui() async {
  // Jika kode 'INF301' sudah ada -> UPDATE, jika belum -> INSERT.
  // onConflict menunjuk kolom UNIQUE yang dipakai untuk mencocokkan.
  await supabase.from('mata_kuliah').upsert({
    'kode': 'INF301',
    'nama': 'Pemrograman Mobile',
    'sks': 4,
    'semester': 5,
  }, onConflict: 'kode');
}
```

Upsert berguna untuk sinkronisasi, misalnya "simpan profil": jika belum ada dibuat, jika sudah ada diperbarui. Ingat, upsert butuh policy **INSERT dan UPDATE** sekaligus.

10. Menghitung jumlah baris

```dart
Future<int> hitung() async {
  // Hanya menghitung jumlah baris tanpa mengambil datanya.
  return supabase.from('mata_kuliah').count();
}
```

Contoh lengkap: daftar mata kuliah dari Supabase

Gabungkan semuanya menjadi aplikasi kecil yang bisa dijalankan. Pastikan SQL di section 5 dan 6 sudah dijalankan, dan `env.json` sudah dibuat (section 4).

`lib/main.dart`

```dart
// lib/main.dart — contoh lengkap: membaca tabel mata_kuliah dari Supabase.
// Jalankan: flutter run --dart-define-from-file=env.json
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(url: supabaseUrl, publishableKey: supabaseKey);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
      ),
      home: const MataKuliahPage(),
```

```
    );
  }
}

class MataKuliah {
  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.semester,
  });

  final String kode;
  final String nama;
  final int sks;
  final int semester;

  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    return MataKuliah(
      kode: json['kode'] as String,
      nama: json['nama'] as String,
      sks: json['sks'] as int,
      semester: json['semester'] as int,
    );
  }
}

class MataKuliahPage extends StatefulWidget {
  const MataKuliahPage({super.key});

  @override
  State<MataKuliahPage> createState() => _MataKuliahPageState();
}

class _MataKuliahPageState extends State<MataKuliahPage> {
  final _supabase = Supabase.instance.client;
  late Future<List<MataKuliah>> _future;

  @override
  void initState() {
    super.initState();
    _future = _ambilData();
  }

  Future<List<MataKuliah>> _ambilData() async {
    final rows = await _supabase
        .from('mata_kuliah')
        .select()
        .order('semester')
        .order('nama');
    return rows.map(MataKuliah.fromJson).toList();
  }

  Future<void> _refresh() async {
    final future = _ambilData();
    setState(() => _future = future);
    await future;
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Informatika')),
      body: FutureBuilder<List<MataKuliah>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      'Gagal memuat data:\n${snapshot.error}',
                      textAlign: TextAlign.center,
                    ),
                    const SizedBox(height: 12),
                    FilledButton(
                      onPressed: _refresh,
                      child: const Text('Coba lagi'),
                    ),
                  ],
                ),
              ),
            );
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Tabel masih kosong.'));
          }
          return RefreshIndicator(
            onRefresh: _refresh,
            child: ListView.separated(
              itemCount: data.length,
              separatorBuilder: (_, _) => const Divider(height: 1),
              itemBuilder: (context, index) {
                final mk = data[index];
                return ListTile(
                  leading: CircleAvatar(child: Text('${mk.semester}')),
                  title: Text(mk.nama),
                  subtitle: Text(mk.kode),
                  trailing: Text('${mk.sks} SKS'),
                );
              },
            ),
          );
        },
      ),
    );
  }
}
```

Data tabel mata_kuliah diambil dari project Supabase sungguhan

- `_future` disimpan di State dan dibuat di `initState`, **bukan** di `build()`. Jika dibuat di build, setiap rebuild akan memicu query baru (pelajaran yang sama dengan FutureBuilder di Modul 09).
- `_refresh()` membuat Future baru dan memasangnya lewat `setState`, lalu menunggu sampai selesai agar animasi `RefreshIndicator` berhenti tepat waktu.
- `.order('semester').order('nama')` mengurutkan per semester, lalu per nama.

8. Menangani Error Supabase

Request ke internet *pasti* suatu saat gagal, entah karena sinyal hilang, server menolak, atau data tidak valid. Aplikasi yang baik menampilkan pesan yang jelas, bukan layar merah atau loading tanpa akhir.

Tiga jenis kegagalan

Jenis	Exception di Dart	Contoh penyebab
Database menolak	`PostgrestException`	RLS, UNIQUE dilanggar, CHECK gagal, tabel tidak ada
Auth menolak	`AuthException` (dan turunannya)	password salah, email belum dikonfirmasi, email sudah terdaftar
Jaringan gagal	`SocketException`, `ClientException`, `TimeoutException`, `AuthRetryableFetchException`	tidak ada internet, DNS gagal, server terlalu lama

Menangani `PostgrestException`

```
Future<String> tambahDenganPenangananError() async {
  try {
    await supabase.from('mata_kuliah').insert({
      'kode': 'INF301', // sudah ada -> melanggar UNIQUE
      'nama': 'Duplikat',
      'semester': 5,
    });
    return 'Berhasil disimpan';
  } on PostgrestException catch (e) {
    // e.code berisi kode error PostgreSQL / PostgREST
    switch (e.code) {
      case '23505':
        return 'Kode mata kuliah sudah dipakai.';
      case '23514':
        return 'Data melanggar aturan CHECK (misalnya SKS di luar 1-6).';
      case '42501':
        return 'Ditolak oleh Row Level Security. Apakah Anda sudah login?';
      default:
        return 'Kesalahan database: ${e.message}';
    }
  } on TimeoutException {
    return 'Server terlalu lama merespons.';
  } catch (e) {
    // Biasanya SocketException / ClientException: tidak ada internet.
    return 'Tidak dapat terhubung ke server.';
```

```
  }
}
```

- `on PostgrestException catch (e)` hanya menangkap error dari database. Properti pentingnya: `e.message`, `e.code`, `e.details`, dan `e.hint`.
- `catch (e)` paling bawah menangkap sisanya, biasanya masalah jaringan.
- Urutan `on ... catch` penting: yang paling spesifik ditulis di atas.

Kode error yang sering muncul

Code	Arti	Solusi
`42501`	Ditolak RLS (*new row violates row-level security policy*)	Periksa policy INSERT/UPDATE dan pastikan user sudah login.
`23505`	Nilai UNIQUE ganda (*duplicate key value*)	Beri tahu user bahwa data sudah ada.
`23502`	Kolom NOT NULL tidak diisi	Lengkapi data yang dikirim.
`23503`	Foreign key tidak ditemukan	Pastikan baris induk ada.
`23514`	Constraint CHECK gagal	Validasi input di form.
`22P02`	Format salah, misalnya teks dikirim ke kolom uuid/int	Periksa tipe data.
`PGRST116`	`.single()` menerima 0 atau lebih dari 1 baris	Pakai `maybeSingle()` atau perbaiki filter.
`PGRST205` / `42P01`	Tabel tidak ditemukan	Jalankan SQL pembuatan tabel, cek ejaan nama tabel.

Menangani `AuthException`

```
Future<String?> loginDenganPenangananError(
  String email,
  String password,
) async {
  try {
    await supabase.auth.signInWithPassword(email: email, password: password);
    return null; // null = sukses
  } on AuthRetryableFetchException {
    return 'Tidak ada koneksi internet.';
  } on AuthException catch (e) {
    // e.code: 'invalid_credentials', 'email_not_confirmed', dst.
```

```
    if (e.code == 'invalid_credentials') return 'Email atau password salah.';
    if (e.code == 'email_not_confirmed') return 'Konfirmasi email dulu.';
    return e.message;
  }
}
```

`AuthRetryableFetchException` adalah turunan `AuthException` yang dilempar saat request Auth gagal karena jaringan. Karena merupakan turunan, ia harus ditangkap **sebelum** `on AuthException`.

> 💡 **Di mana sebaiknya try-catch ditulis?**
>
> Jangan menyebar try-catch di setiap widget. Di proyek Modul 13–14, error ditangkap di **Provider**, diterjemahkan menjadi pesan Bahasa Indonesia, lalu UI cukup menampilkan pesannya. Dengan begitu tampilan tetap bersih dan logika error terkumpul di satu tempat.

> ⚠️ **Jangan menelan error**
>
> `catch (e) {}` yang kosong membuat bug tidak terlihat: data tidak tersimpan dan user tidak tahu kenapa. Minimal tampilkan SnackBar atau tulis `debugPrint(e.toString())` saat pengembangan.

9. Dasar Supabase Auth

Supabase Auth mengurus register, login, penyimpanan sesi, dan perpanjangan token secara otomatis. Di section ini kita mengenal method-method dasarnya. Implementasi lengkap dengan UI dikerjakan di Modul 13.

Pengaturan di dashboard

1. Buka **Authentication → Sign In / Providers**. Pastikan provider **Email** aktif (default sudah aktif).
2. Perhatikan opsi **Confirm email**:
   - **ON** (default): setelah register, user harus mengeklik link di email sebelum bisa login. `signUp` mengembalikan `session == null`.
   - **OFF**: user langsung login setelah register. `signUp` mengembalikan session yang valid. Cocok untuk latihan di kelas.
3. Daftar user yang sudah register bisa dilihat di **Authentication → Users**.

> ⚠️ **Batas pengiriman email**
>
> Layanan email bawaan Supabase hanya untuk percobaan dan dibatasi beberapa email per jam. Jika satu kelas register bersamaan dengan Confirm email ON, sebagian akan mendapat error `over_email_send_rate_limit`. Untuk praktikum, matikan Confirm email. Untuk aplikasi produksi, pasang SMTP sendiri di **Authentication → Emails → SMTP Settings**.

1. Register: `signUp`

```dart
Future<void> daftar() async {
  final AuthResponse res = await supabase.auth.signUp(
    email: 'budi@mhs.unsoed.ac.id',
    password: 'rahasia123',
    data: {'full_name': 'Budi Santoso'}, // disimpan di user metadata
  );

  final User? user = res.user; // akun yang baru dibuat
  final Session? session = res.session; // null jika wajib konfirmasi email
  debugPrint('User: ${user?.id}, langsung login: ${session != null}');
}
```

- `data` disimpan sebagai **user metadata** (di database bernama kolom `raw_user_meta_data`). Di Modul 13 kita memakai trigger untuk menyalin `full_name` ke tabel `profiles`.

- Password minimal 6 karakter (default Supabase). Jika kurang, muncul `AuthException` dengan code `weak_password`.

2. Login dan logout

```dart
Future<void> masuk() async {
  final res = await supabase.auth.signInWithPassword(
    email: 'budi@mhs.unsoed.ac.id',
    password: 'rahasia123',
  );
  debugPrint('Halo ${res.user?.email}');
}

Future<void> keluar() async {
  await supabase.auth.signOut();
}
```

Setelah login berhasil, **semua** query database berikutnya otomatis membawa token user. RLS pun mengenali siapa yang meminta data melalui `auth.uid()`, tanpa kode tambahan.

3. Mengecek sesi saat ini

```dart
void cekSesi() {
  // Sesi disimpan otomatis di perangkat oleh supabase_flutter,
  // jadi setelah aplikasi ditutup dan dibuka lagi user tetap login.
  final Session? session = supabase.auth.currentSession;
  final User? user = supabase.auth.currentUser;

  if (session == null) {
    debugPrint('Belum login');
  } else {
    debugPrint('Login sebagai ${user!.email}');
    debugPrint('Nama: ${user.userMetadata?['full_name']}');
  }
}
```

4. Mendengarkan perubahan status login

```dart
StreamSubscription<AuthState> dengarkanAuth() {
  return supabase.auth.onAuthStateChange.listen((AuthState data) {
    final AuthChangeEvent event = data.event;
    final Session? session = data.session;

    switch (event) {
      case AuthChangeEvent.initialSession:
        debugPrint('Aplikasi dibuka, sesi awal: ${session != null}');
      case AuthChangeEvent.signedIn:
        debugPrint('Berhasil login');
      case AuthChangeEvent.signedOut:
        debugPrint('Sudah logout');
      case AuthChangeEvent.tokenRefreshed:
        debugPrint('Token diperbarui otomatis');
      default:
        debugPrint('Event lain: ${event.name}');
```

```
    }
  });
}
// Jangan lupa: subscription.cancel() di dispose() jika dipakai di widget.
```

Event	Kapan terjadi
`initialSession`	Selalu menjadi event pertama saat stream didengarkan. Berisi sesi tersimpan (atau null).
`signedIn`	Setelah login/register berhasil (dan sesi aktif).
`signedOut`	Setelah `signOut()` atau sesi tidak valid lagi.
`tokenRefreshed`	Token akses diperpanjang otomatis (kira-kira setiap jam).
`userUpdated`	Data user diubah lewat `updateUser()`.
`passwordRecovery`	User membuka link reset password.

Stream inilah kunci navigasi otomatis: **ada sesi = tampilkan Home, tidak ada sesi = tampilkan Login**. Pola ini disebut *AuthGate* dan dibangun di Modul 13.

> ℹ️ **Di mana sesi disimpan?**
>
> `supabase_flutter` menyimpan sesi di penyimpanan lokal perangkat (berbasis SharedPreferences, Modul 10). Karena itu user tetap login walaupun aplikasi ditutup, dan token diperbarui otomatis selama aplikasi berjalan.

> ⚠️ **Register dengan email yang sudah terdaftar**
>
> Jika Confirm email ON, Supabase **tidak** melempar error saat email sudah terdaftar. Ia mengembalikan user "palsu" agar penyerang tidak bisa menebak email siapa saja yang terdaftar (*user enumeration*). Jika Confirm email OFF, error `user_already_exists` akan muncul.

10. Sekilas Realtime & Storage

Dua fitur tambahan ini tidak wajib untuk proyek Modul 13–14, tetapi sering diminta di tugas besar: data yang berubah *live* (Realtime) dan unggah file (Storage).

Realtime: data yang otomatis ter-update

Langkah 1: aktifkan Realtime untuk tabel. Caranya bisa lewat **Table Editor** → pilih tabel → aktifkan **Enable Realtime**, atau lewat SQL:

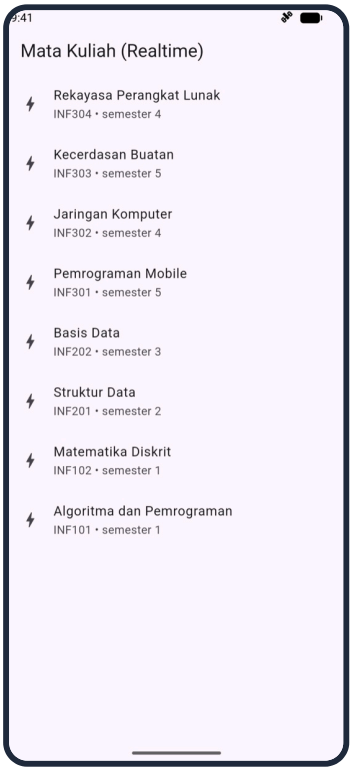
```
alter publication supabase_realtime add table public.mata_kuliah;
```

Langkah 2: ganti `select()` dengan `stream(primaryKey: [...])`, dan `FutureBuilder` dengan `StreamBuilder`:

`lib/main.dart`

```
// lib/main.dart — contoh Realtime: daftar mata kuliah yang otomatis
// berubah ketika tabel diubah dari dashboard atau perangkat lain.
// Jalankan: flutter run --dart-define-from-file=env.json
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(url: supabaseUrl, publishableKey: supabaseKey);
  runApp(
    const MaterialApp(debugShowCheckedModeBanner: false, home: RealtimePage()),
  );
}

class RealtimePage extends StatefulWidget {
  const RealtimePage({super.key});

  @override
  State<RealtimePage> createState() => _RealtimePageState();
}

class _RealtimePageState extends State<RealtimePage> {
  // Stream disimpan di State: JANGAN dibuat di dalam build().
  final Stream<List<Map<String, dynamic>>> _stream = Supabase.instance.client
      .from('mata_kuliah')
      .stream(primaryKey: ['id'])
      .order('kode');

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah (Realtime)')),
      body: StreamBuilder<List<Map<String, dynamic>>>(
        stream: _stream,
```

```
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final rows = snapshot.data!;
          return ListView.builder(
            itemCount: rows.length,
            itemBuilder: (context, index) {
              final row = rows[index];
              return ListTile(
                leading: const Icon(Icons.bolt),
                title: Text(row['nama'] as String),
                subtitle: Text('${row['kode']} • semester ${row['semester']}'),
              );
            },
          );
        },
      ),
    );
  }
}
```



StreamBuilder + stream(): data awal dari Supabase, lalu ikut berubah saat tabel diubah

- `stream(primaryKey: ['id'])` mula-mula memancarkan seluruh data, lalu memancarkan list baru setiap kali ada INSERT/UPDATE/DELETE. `primaryKey` dipakai untuk mencocokkan baris yang berubah.

- Stream disimpan sebagai field State. Membuatnya di dalam `build()` akan membuka koneksi baru di setiap rebuild.
- RLS tetap berlaku: user hanya menerima perubahan baris yang boleh ia lihat.
- Coba buka Table Editor, ubah nama satu mata kuliah, lalu lihat aplikasi berubah sendiri tanpa refresh.

> ℹ️ **Kapan memakai realtime?**
>
> Chat, papan skor, status pesanan, kolaborasi. Untuk daftar yang jarang berubah (seperti catatan pribadi), `select()` biasa plus pull-to-refresh sudah cukup dan lebih hemat kuota koneksi.

Storage: menyimpan file

1. Buka **Storage** → **New bucket**, beri nama `avatars`. Centang **Public bucket** agar file bisa dibuka lewat URL.
2. Storage juga dilindungi RLS. Buat policy agar user hanya boleh mengunggah ke foldernya sendiri:

```
create policy "User mengunggah ke foldernya sendiri"
on storage.objects for insert to authenticated
with check (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

create policy "User menimpa file di foldernya sendiri"
on storage.objects for update to authenticated
using (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);
```

Kode unggah (byte gambar bisa didapat dari package seperti `image_picker` dengan `await file.readAsBytes()`):

```
Future<String> unggahFoto(Uint8List bytes) async {
  final userId = supabase.auth.currentUser!.id;
  // Simpan di folder milik user: avatars/<userId>/foto.jpg
  final path = '$userId/foto.jpg';

  await supabase.storage
      .from('avatars')
      .uploadBinary(
        path,
        bytes,
        fileOptions: const FileOptions(
```

```
          contentType: 'image/jpeg',
          upsert: true, // timpa jika sudah ada
        ),
      );

  // URL publik (bucket harus bertipe Public)
  return supabase.storage.from('avatars').getPublicUrl(path);
}
```

- `uploadBinary` menerima `Uint8List` sehingga bisa dipakai di Android, iOS, maupun Web.
- `storage.foldername(name)[1]` di policy adalah nama folder pertama dari path file, yang harus sama dengan id user.
- `upsert: true` membutuhkan policy UPDATE juga, karena file lama ditimpa.
- Untuk bucket privat, gunakan `createSignedUrl(path, detik)` sebagai pengganti `getPublicUrl`.

Ringkasan Modul 12

- Supabase = PostgreSQL + Auth + RLS + Realtime + Storage, diakses lewat `supabase_flutter`.
- `Supabase.initialize(url:, publishableKey:)` sekali di `main()`. Simpan nilai di `env.json` + `--dart-define-from-file`.
- Secret/service_role key **tidak pernah** masuk ke aplikasi. Data dilindungi oleh **RLS**.
- Query: `from().select()/insert()/update()/delete()/upsert()` plus filter, order, range, single/maybeSingle.
- Error database = `PostgrestException`, error login = `AuthException`, sisanya biasanya jaringan.

Tugas Praktikum: Praktikum 12: Katalog Mata Kuliah di Supabase

Sasaran Praktikum

Membangun aplikasi Flutter yang membaca dan menambah data mata kuliah di Supabase dengan konfigurasi kunci yang aman dan RLS yang benar.

Langkah Pengerjaan

1. Buat project Supabase baru (region Singapore). Catat Project URL dan publishable key ke file `env.json`, lalu tambahkan `env.json` ke `.gitignore` dan buat `env.example.json`.
2. Jalankan SQL pembuatan tabel `mata_kuliah` beserta data awal minimal 8 baris (section 5).
3. Aktifkan RLS dan buat policy: SELECT untuk `anon, authenticated`, INSERT hanya untuk `authenticated` (section 6).
4. Pasang `supabase_flutter` dengan `flutter pub add supabase_flutter`, inisialisasi memakai `Env` dan jalankan dengan `--dart-define-from-file=env.json`.
5. Tampilkan daftar mata kuliah urut per semester dengan FutureBuilder + RefreshIndicator (contoh lengkap section 7).
6. Tambahkan ChoiceChip filter semester (1–8) yang memakai `.eq('semester', n)` saat dipilih.
7. Tambahkan tombol 'Muat lebih banyak' yang memakai `.range()` dengan 3 data per halaman.
8. Tangani error: matikan internet lalu refresh, dan pastikan pesan Bahasa Indonesia muncul (bukan layar merah).
9. Bonus: tambah form insert mata kuliah baru. Uji saat belum login (harus gagal 42501) dan tampilkan pesan yang sesuai.

Output yang Diharapkan

Aplikasi menampilkan data mata kuliah dari Supabase, bisa difilter per semester dan dipaginasi, menampilkan pesan error yang jelas saat offline, dan repository Git tidak memuat env.json.

Rubrik Penilaian

Setup project, env.json & .gitignore (15%), SQL tabel + constraint benar (15%), RLS & policy tepat (20%), Query select/filter/order/range berjalan (25%), Penanganan loading/error/empty state (15%), Kerapian kode & model fromJson (10%). Bonus insert dengan penanganan 42501: +10%.

Kuis Pemahaman

1. Kunci Supabase manakah yang BOLEH dipasang di dalam aplikasi Flutter?

A. Secret key (sb_secret_...)

B. service_role key

C. Publishable key (sb_publishable_...) atau anon key lama

D. Password database Postgres

2. RLS pada tabel notes sudah aktif tetapi belum ada satu pun policy. Apa yang terjadi saat aplikasi menjalankan supabase.from('notes').select()?

A. Muncul error 404

B. Semua baris dikembalikan

C. Hasilnya list kosong [] tanpa error

D. Aplikasi crash

3. Apa fungsi auth.uid() di dalam policy RLS?

A. Membuat UUID acak baru

B. Mengembalikan id user yang sedang login berdasarkan token pada request

C. Mengembalikan id baris yang sedang diproses

D. Mengembalikan email user

4. Pada policy UPDATE, apa beda klausa using dan with check?

A. Keduanya sama saja

B. using mengecek baris lama yang boleh diubah, with check mengecek data hasil perubahan

C. using untuk insert, with check untuk delete

D. with check hanya dipakai di SELECT

5. Query mana yang mengembalikan null (bukan error) jika tidak ada baris yang cocok?

A. .select().single()

B. .select().maybeSingle()

C. .select().limit(1)

D. .select().range(0, 0)

6. Untuk menampilkan halaman ke-2 (indeks halaman 1) dengan 10 data per halaman, pemanggilan range yang benar adalah...

A. .range(10, 20)

B. .range(1, 10)

C. .range(10, 19)

D. .range(11, 20)

7. Insert ke tabel notes menghasilkan PostgrestException code 42501 'new row violates row-level security policy'. Penyebab paling mungkin adalah...

A. Internet mati

B. User belum login atau user_id yang dikirim tidak sama dengan auth.uid() sehingga with check gagal

C. Nama kolom salah ketik

D. Supabase.initialize belum dipanggil

8. Dengan pengaturan 'Confirm email' AKTIF di dashboard, apa hasil signUp yang berhasil?

A. AuthResponse dengan session terisi sehingga user langsung login

B. AuthResponse dengan user terisi tetapi session null sampai user mengeklik link konfirmasi

C. Selalu melempar AuthException

D. User otomatis terhapus setelah 1 jam

Kunci Jawaban & Pembahasan

1. C — Publishable key (atau anon key versi lama) memang dirancang untuk aplikasi klien. Data tetap aman karena dilindungi RLS. Secret/service_role key melewati RLS sehingga tidak boleh ada di aplikasi.

2. C — Saat RLS aktif, akses ditolak secara default. Untuk SELECT, penolakan berupa 'tidak ada baris yang lolos', sehingga hasilnya list kosong tanpa exception.

3. B — auth.uid() membaca token sesi yang dikirim supabase_flutter dan mengembalikan id user tersebut (null untuk anon). Policy seperti (select auth.uid()) = user_id membatasi akses hanya ke baris miliknya.

4. B — using memfilter baris yang sudah ada (mana yang boleh disentuh), sedangkan with check memvalidasi baris baru/hasil update, misalnya agar user_id tidak bisa diganti ke milik user lain.

5. B — maybeSingle() mengembalikan Map? (null jika 0 baris). single() melempar PostgrestException bila jumlah baris bukan tepat 1, sedangkan limit/range mengembalikan list.

6. C — range memakai indeks mulai 0 dan inklusif di kedua ujung. Halaman 1 dimulai dari 1 × 10 = 10 sampai 10 + 10 - 1 = 19.

7. B — 42501 berarti ditolak oleh policy. Pada insert, with check ((select auth.uid()) = user_id) gagal jika user anon (auth.uid() null) atau user_id berisi id lain.

8. B — Saat konfirmasi email wajib, akun dibuat tetapi belum ada sesi. Aplikasi harus memberi tahu user untuk mengecek email, lalu login setelah konfirmasi.

MODUL 13 · PROYEK BAGIAN 1

Proyek Catatan Kuliah (1): Arsitektur & Autentikasi Supabase

Memulai proyek utuh 'Catatan Kuliah': menyusun kebutuhan, arsitektur berlapis (UI, Provider, Service/Repository, Supabase/Hive), struktur folder, tabel profiles dengan trigger, lalu membangun fitur register, login, logout, dan sesi tersimpan menggunakan Supabase Auth, Provider, dan AuthGate.

Durasi Pertemuan 13 (150 Menit) + Belajar Mandiri

Prasyarat Modul 05: Form & Validasi Input
Modul 07: Navigasi
Modul 08: State Management Provider
Modul 12: Supabase

Capaian Pembelajaran

- Menyusun kebutuhan fungsional dan alur layar sebuah aplikasi mobile sebelum menulis kode.
- Menerapkan arsitektur berlapis UI, Provider, Service/Repository dengan dependency injection.
- Membuat tabel profiles beserta trigger handle_new_user dan policy RLS di Supabase.
- Mengimplementasikan AuthService dan AuthProvider dengan status loading dan pesan error berbahasa Indonesia.
- Membangun AuthGate berbasis onAuthStateChange untuk navigasi login/logout otomatis.
- Membuat LoginScreen dan RegisterScreen dengan validasi form, show/hide password, tombol loading, dan SnackBar.
- Menguji alur autentikasi secara sistematis, termasuk mode konfirmasi email dan sesi tersimpan.

Daftar Isi Modul

1. Gambaran Proyek & Kebutuhan
2. Arsitektur, Struktur Folder & Package
3. Setup Supabase: Tabel profiles & Trigger
4. Env, Validator & AuthService
5. AuthProvider & Pesan Error Ramah
6. AuthGate, Home Sementara & main.dart
7. LoginScreen
8. RegisterScreen & Menjalankan Aplikasi
9. Pengujian & Troubleshooting
10. Tugas Praktikum

11. Kuis Pemahaman & Kunci Jawaban

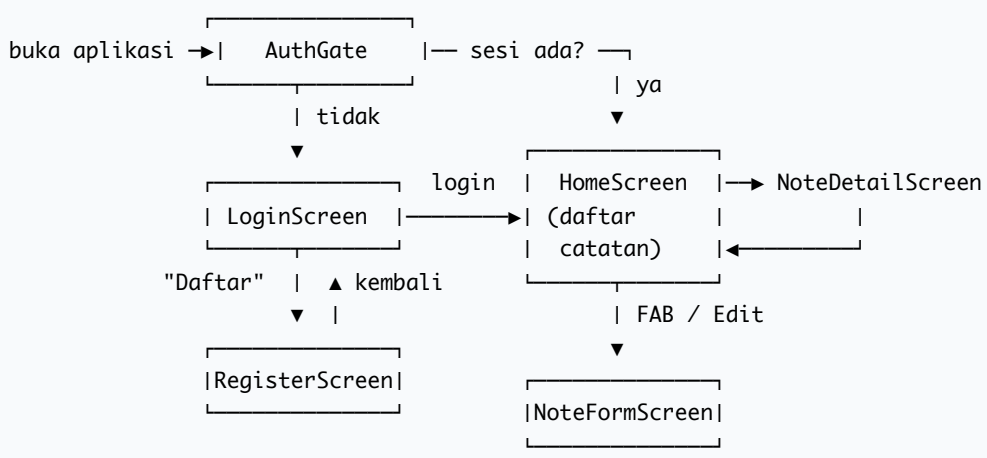
1. Gambaran Proyek & Kebutuhan

Di Modul 13 dan 14 kita membangun satu aplikasi utuh dari proyek kosong sampai siap dipakai: **Catatan Kuliah**. Mahasiswa bisa mendaftar, login, lalu menulis catatan per mata kuliah yang tersimpan di cloud (Supabase) dan tetap bisa dibaca saat offline (cache Hive).

Kebutuhan (requirements)

Kode	Kebutuhan	Dikerjakan di
F-01	User bisa **register** dengan nama lengkap, email, dan password.	Modul 13
F-02	User bisa **login** dan **logout**. Sesi tetap tersimpan saat aplikasi ditutup.	Modul 13
F-03	Pesan error dalam Bahasa Indonesia (password salah, email belum dikonfirmasi, dan lain-lain).	Modul 13
F-04	User bisa **menambah, melihat, mengubah, dan menghapus** catatan (CRUD).	Modul 14
F-05	User **hanya** bisa melihat catatannya sendiri (RLS).	Modul 14
F-06	Pencarian judul/isi dan filter kategori (Kuliah, Tugas, Ujian, Lainnya).	Modul 14
F-07	Saat offline, catatan terakhir tetap tampil dari cache dengan banner "mode offline".	Modul 14
F-08	Logout menghapus cache lokal agar data tidak terbaca user berikutnya.	Modul 14

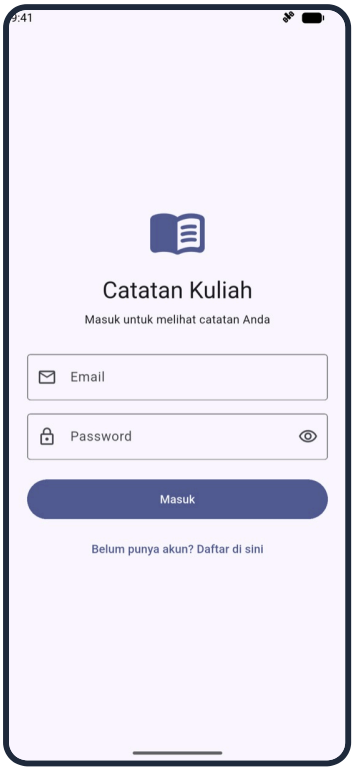
Alur layar



Modul 13 menyelesaikan bagian kiri (AuthGate, Login, Register) beserta `HomeScreen` sementara. Modul 14 mengganti `HomeScreen` dengan daftar catatan lengkap, lalu menambahkan form dan detail.

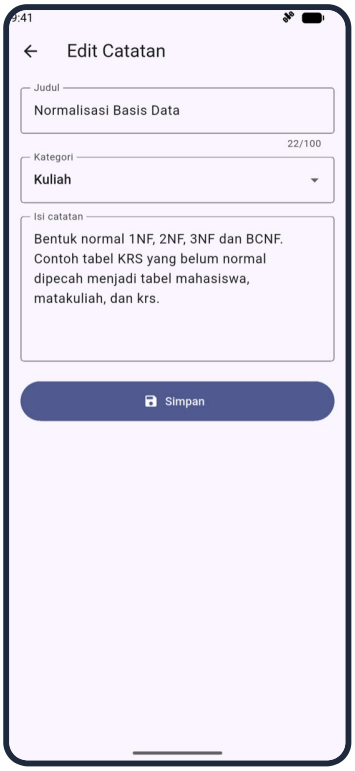
Hasil akhir yang ingin dicapai

Tangkapan layar aplikasi final (akhir Modul 14) yang terhubung ke project Supabase:



Login

Daftar catatan milik user



Form tambah/edit

Mode offline dari cache Hive

Apa yang akan Anda latih

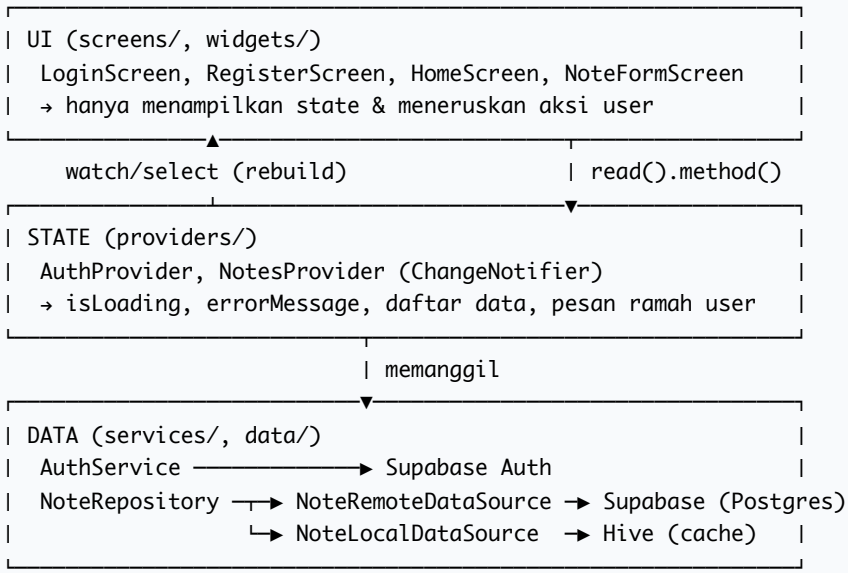
- **Form & validasi** (Modul 05): TextFormField, GlobalKey<FormState>, validator.
- **Navigasi** (Modul 07): push/pop, ditambah navigasi otomatis berbasis status login.
- **Provider** (Modul 08): ChangeNotifier, MultiProvider, context.watch/read/select.
- **JSON & repository** (Modul 09): model fromJson/toJson, pemisahan lapisan data.
- **Hive** (Modul 11): box sebagai cache offline.
- **Supabase** (Modul 12): Auth, SQL, RLS, query CRUD.

> 💡 **Cara mengikuti proyek ini**
>
> Kerjakan **berurutan**. Setiap section menambah satu atau beberapa file, dan di akhir section yang membuat aplikasi bisa dijalankan, ada langkah "jalankan & cek". Jika ada yang error, bandingkan file Anda dengan listing lengkap di akhir Modul 14.

2. Arsitektur, Struktur Folder & Package

Sebelum menulis kode, kita sepakati dulu "denah rumah"-nya. Arsitektur yang rapi membuat kode mudah diuji, mudah dicari saat error, dan mudah dikembangkan.

Lapisan (layer) aplikasi



Lapisan	Boleh tahu tentang	Tidak boleh tahu tentang
UI	Provider, model, widget	Supabase, Hive, SQL
Provider	Service/Repository, model, jenis exception	Widget, BuildContext
Service/Repository	Supabase client, Hive box, model	Provider, UI

Aturan sederhananya: **panah hanya ke bawah**. Karena `NoteRepository` tidak mengenal UI, kita bisa mengujinya dengan unit test tanpa emulator (dilakukan di Modul 14).

Struktur folder akhir

```
catatan_kuliah/
├── env.json                ← URL & publishable key (TIDAK di-commit)
├── env.example.json        ← contoh isi env.json (di-commit)
├── supabase/
│   └── setup.sql           ← semua SQL proyek (disimpan untuk dokumentasi)
├── lib/
│   ├── main.dart           ← inisialisasi Hive + Supabase, MultiProvider
│   ├── config/
│   │   └── env.dart        ← membaca --dart-define            [M13]
│   ├── models/
│   │   └── note.dart       ← model Note + fromJson/toJson/toMap [M14]
```

```
|   ├── services/
|   |   └── auth_service.dart ← signUp/signIn/signOut            [M13]
|   ├── data/
|   |   ├── note_remote_data_source.dart  ← CRUD Supabase        [M14]
|   |   ├── note_local_data_source.dart   ← cache Hive           [M14]
|   |   └── note_repository.dart          ← gabungan + offline   [M14]
|   ├── providers/
|   |   ├── auth_provider.dart            ← state login          [M13]
|   |   └── notes_provider.dart           ← state catatan        [M14]
|   ├── screens/
|   |   ├── auth_gate.dart                ← Login atau Home?     [M13]
|   |   ├── login_screen.dart                                    [M13]
|   |   ├── register_screen.dart                                 [M13]
|   |   ├── home_screen.dart              ← sementara di M13, final di M14
|   |   ├── note_form_screen.dart         ← tambah & edit        [M14]
|   |   └── note_detail_screen.dart                              [M14]
|   ├── widgets/
|   |   ├── confirm_dialog.dart                                  [M14]
|   |   ├── note_card.dart                                       [M14]
|   |   └── offline_banner.dart                                  [M14]
|   └── utils/
|       ├── validators.dart             ← validator form         [M13]
|       └── date_format.dart                                     [M14]
└── test/
    ├── note_local_data_source_test.dart                         [M14]
    └── note_repository_test.dart                                [M14]
```

Langkah 1: Membuat proyek dan memasang package

```
flutter create catatan_kuliah
cd catatan_kuliah

flutter pub add supabase_flutter provider hive_ce hive_ce_flutter
```

Package	Kegunaan di proyek
`supabase_flutter`	Auth dan database cloud (Modul 12)
`provider`	State management (Modul 08)
`hive_ce`	Database key-value lokal untuk cache (Modul 11)
`hive_ce_flutter`	`Hive.initFlutter()` : menentukan folder penyimpanan di HP

Setelah perintah di atas, bagian `dependencies` di `pubspec.yaml` kira-kira seperti ini (nomor versi mengikuti yang terbaru saat Anda menjalankan perintah):

`pubspec.yaml`

```yaml
dependencies:
  flutter:
```

```
    sdk: flutter
  cupertino_icons: ^1.0.8
  supabase_flutter: ^x.y.z
  provider: ^x.y.z
  hive_ce: ^x.y.z
  hive_ce_flutter: ^x.y.z
```

Langkah 2: Siapkan folder dan file konfigurasi

```
mkdir -p lib/config lib/models lib/services lib/data lib/providers lib/screens lib/widgets lib/utils
```

Buat `env.json` di root proyek (sejajar `pubspec.yaml`) dan isi dengan nilai dari dashboard Supabase Anda (Modul 12 section 2):

`env.json`

```
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Tambahkan ke `.gitignore`:

`.gitignore`

```
# Kunci Supabase
env.json
```

Lalu salin `env.json` menjadi `env.example.json` dengan nilai placeholder, agar teman satu tim tahu format yang dibutuhkan.

Terakhir, tambahkan izin internet untuk build release di `android/app/src/main/AndroidManifest.xml` (di atas tag `<application>`):

```
<uses-permission android:name="android.permission.INTERNET"/>
```

3. Setup Supabase: Tabel profiles & Trigger

Aplikasi kita butuh tempat menyimpan data tambahan tentang user, misalnya nama lengkap. Tabel `auth.users` dikelola Supabase dan tidak bisa diakses langsung dari aplikasi, sehingga pola yang lazim adalah membuat tabel `public.profiles` yang barisnya dibuat **otomatis** oleh trigger setiap kali ada user baru.

Alur data saat register

```
Flutter: auth.signUp(email, password, data: {'full_name': 'Fikri Ramadhan'})
   |
   ▼
auth.users  ← baris baru, raw_user_meta_data = {"full_name": "Fikri Ramadhan"}
   |  (trigger on_auth_user_created: AFTER INSERT)
   ▼
public.profiles ← (id = id user baru, full_name = 'Fikri Ramadhan')
```

Langkah 1: Jalankan SQL berikut di SQL Editor

Buka **SQL Editor → New query**, tempel seluruh SQL di bawah, lalu klik **Run**. SQL ini aman dijalankan ulang karena memakai `if not exists`, `drop ... if exists`, dan `create or replace`.

`supabase/setup.sql`

```sql
create table if not exists public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  full_name text,
  created_at timestamptz not null default now()
);

alter table public.profiles enable row level security;

drop policy if exists "User melihat profilnya sendiri" on public.profiles;
create policy "User melihat profilnya sendiri"
on public.profiles for select
to authenticated
using ((select auth.uid()) = id);

drop policy if exists "User mengubah profilnya sendiri" on public.profiles;
create policy "User mengubah profilnya sendiri"
on public.profiles for update
to authenticated
using ((select auth.uid()) = id)
with check ((select auth.uid()) = id);

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, new.raw_user_meta_data ->> 'full_name');
```

```
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();
```

Penjelasan per bagian

Kode	Penjelasan
`id uuid primary key references auth.users (id)`	id profil **sama** dengan id user (relasi satu-satu). `on delete cascade` : profil ikut terhapus jika akun dihapus.
`enable row level security`	Mengunci tabel. Tanpa policy, tidak ada yang bisa membaca.
Policy select/update dengan `(select auth.uid()) = id`	User hanya bisa membaca dan mengubah profilnya sendiri. Tidak ada policy INSERT karena insert dilakukan oleh trigger, bukan oleh aplikasi.
`returns trigger`	Fungsi khusus yang dipanggil oleh trigger. Variabel `new` berisi baris yang baru dimasukkan ke `auth.users`.
`security definer`	Fungsi berjalan dengan hak *pembuat* fungsi (admin), bukan hak user. Ini perlu karena pada saat register user belum login, dan RLS profiles akan menolaknya.
`set search_path = ''`	Praktik keamanan untuk fungsi `security definer` : semua nama tabel harus ditulis lengkap (`public.profiles`) sehingga tidak bisa "dibelokkan" ke tabel lain.
`new.raw_user_meta_data ->> 'full_name'`	Operator `->>` mengambil nilai dari kolom JSON sebagai teks. Isinya berasal dari parameter `data` di `signUp()`.
`after insert on auth.users for each row`	Trigger dijalankan sekali untuk setiap user baru.

> ⚠️ **Trigger error = register gagal**
>
> Jika fungsi trigger error (misalnya salah ketik nama tabel), **seluruh** proses register ikut dibatalkan dan aplikasi menerima pesan *Database error saving new user*. Jika itu terjadi, buka **Logs → Postgres** di dashboard untuk melihat detail error-nya.

Langkah 2: Atur konfirmasi email

1. Buka **Authentication → Sign In / Providers → Email**.
2. Untuk praktikum di kelas, **matikan** *Confirm email* agar register langsung login dan tidak terkena batas pengiriman email.
3. Aplikasi yang kita buat tetap mendukung kedua mode: jika Confirm email aktif, RegisterScreen menampilkan dialog "Cek email Anda".

Langkah 3: Verifikasi

Setelah aplikasi selesai (section 8), register satu akun lalu buka **Table Editor → profiles**. Harus ada satu baris dengan `full_name` sesuai yang Anda isi. Akun yang sama juga terlihat di **Authentication → Users**.

4. Env, Validator & AuthService

Kita mulai dari lapisan paling bawah, lalu naik ke atas: konfigurasi → service → provider → UI. Di section ini kita membuat tiga file yang tidak bergantung pada widget sama sekali.

File 1: `lib/config/env.dart`

Membaca URL dan key yang disuntikkan lewat `--dart-define-from-file=env.json` (Modul 12 section 4).

`lib/config/env.dart`

```dart
/// Konfigurasi Supabase yang dibaca saat build lewat --dart-define.
///
/// Jalankan aplikasi dengan:
///   flutter run --dart-define-from-file=env.json
class Env {
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  /// true jika kedua nilai sudah diisi.
  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

File 2: `lib/utils/validators.dart`

Validator dipakai oleh form Login dan Register. Kita kumpulkan di satu tempat agar aturannya konsisten (misalnya minimal 6 karakter mengikuti aturan default Supabase).

`lib/utils/validators.dart`

```dart
/// Kumpulan validator untuk TextFormField.
/// Setiap fungsi mengembalikan null jika valid, atau pesan error jika tidak.
class Validators {
  static String? fullName(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Nama lengkap wajib diisi';
    if (text.length < 3) return 'Nama minimal 3 karakter';
    return null;
  }

  static String? email(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Email wajib diisi';
    final pola = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');
    if (!pola.hasMatch(text)) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value) {
    final text = value ?? '';
    if (text.isEmpty) return 'Password wajib diisi';
```

```
    if (text.length < 6) return 'Password minimal 6 karakter';
    return null;
  }
}
```

- Setiap validator mengembalikan `null` jika valid, atau `String` pesan error jika tidak valid. Ini kontrak `validator` pada `TextFormField` (Modul 05).
- Method dibuat `static` sehingga bisa langsung dipasang sebagai *tear-off*: `validator: Validators.email`.
- Regex email sengaja dibuat sederhana. Validasi sesungguhnya tetap dilakukan Supabase (error `email_address_invalid`).

File 3: `lib/services/auth_service.dart`

`AuthService` adalah **satu-satunya** class yang memanggil `supabase.auth`. Class lain cukup memakai service ini.

`lib/services/auth_service.dart`

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

/// Lapisan paling bawah untuk urusan autentikasi.
/// Hanya class ini yang berbicara langsung dengan Supabase Auth.
class AuthService {
  AuthService(this._client);

  final SupabaseClient _client;

  GoTrueClient get _auth => _client.auth;

  /// Stream yang memancarkan event setiap status login berubah
  /// (initialSession, signedIn, signedOut, tokenRefreshed, ...).
  Stream<AuthState> get authStateChanges => _auth.onAuthStateChange;

  Session? get currentSession => _auth.currentSession;

  User? get currentUser => _auth.currentUser;

  /// Mendaftarkan akun baru. Nama lengkap dikirim sebagai metadata
  /// (raw_user_meta_data) dan nanti disalin trigger ke tabel profiles.
  Future<AuthResponse> signUp({
    required String fullName,
    required String email,
    required String password,
  }) {
    return _auth.signUp(
      email: email,
      password: password,
      data: {'full_name': fullName},
    );
  }

  Future<AuthResponse> signIn({
    required String email,
    required String password,
```

```
  }) {
    return _auth.signInWithPassword(email: email, password: password);
  }

  Future<void> signOut() => _auth.signOut();
}
```

Penjelasan:

- **Constructor menerima `SupabaseClient`** (*dependency injection*), bukan memanggil `Supabase.instance.client` di dalam class. Dengan begitu kita bisa mengganti client saat testing.
- `authStateChanges` meneruskan stream `onAuthStateChange`. Stream ini yang nanti didengarkan oleh AuthGate.
- `signUp(... data: {'full_name': fullName})`: nama lengkap dikirim sebagai metadata, lalu trigger `handle_new_user` (section 3) menyalinnya ke tabel `profiles`.
- Method mengembalikan `Future<AuthResponse>` apa adanya. Penanganan error *bukan* tugas service, melainkan tugas provider di section berikutnya.

> ℹ️ **Kenapa tidak langsung memanggil Supabase dari widget?**
>
> Bisa saja, dan untuk demo kecil itu wajar. Tetapi jika suatu hari Anda pindah ke Firebase atau backend kampus sendiri, Anda hanya perlu mengganti isi `AuthService`. Provider dan semua layar tidak perlu disentuh.

5. AuthProvider & Pesan Error Ramah

`AuthProvider` menjembatani UI dan `AuthService`. Tugasnya: menyimpan status *loading*, menangkap exception, dan menerjemahkannya menjadi pesan Bahasa Indonesia yang ramah pengguna.

File 4: `lib/providers/auth_provider.dart`

`lib/providers/auth_provider.dart`

```dart
import 'package:flutter/foundation.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';

/// Hasil proses registrasi, dipakai RegisterScreen untuk menentukan
/// pesan yang ditampilkan.
enum RegisterResult { failed, loggedIn, needsEmailConfirmation }

class AuthProvider extends ChangeNotifier {
  AuthProvider(this._authService);

  final AuthService _authService;

  bool _isLoading = false;
  String? _errorMessage;

  bool get isLoading => _isLoading;
  String? get errorMessage => _errorMessage;

  Stream<AuthState> get authStateChanges => _authService.authStateChanges;
  Session? get currentSession => _authService.currentSession;
  User? get currentUser => _authService.currentUser;

  /// Nama yang ditampilkan di AppBar. Diambil dari metadata saat register.
  String get displayName {
    final user = currentUser;
    if (user == null) return '';
    final name = user.userMetadata?['full_name'];
    if (name is String && name.isNotEmpty) return name;
    return user.email ?? '';
  }

  Future<bool> signIn({required String email, required String password}) async {
    _setLoading(true);
    try {
      await _authService.signIn(email: email.trim(), password: password);
      return true;
    } on AuthException catch (e) {
      _errorMessage = _mapAuthError(e);
      return false;
    } catch (e) {
      _errorMessage = 'Terjadi kesalahan tak terduga: $e';
      return false;
    } finally {
      _setLoading(false);
```

```
  }
}

Future<RegisterResult> register({
  required String fullName,
  required String email,
  required String password,
}) async {
  _setLoading(true);
  try {
    final response = await _authService.signUp(
      fullName: fullName.trim(),
      email: email.trim(),
      password: password,
    );
    // Jika "Confirm email" aktif di dashboard, session bernilai null
    // sampai pengguna mengeklik link di email.
    if (response.session == null) {
      return RegisterResult.needsEmailConfirmation;
    }
    return RegisterResult.loggedIn;
  } on AuthException catch (e) {
    _errorMessage = _mapAuthError(e);
    return RegisterResult.failed;
  } catch (e) {
    _errorMessage = 'Terjadi kesalahan tak terduga: $e';
    return RegisterResult.failed;
  } finally {
    _setLoading(false);
  }
}

Future<void> signOut() async {
  try {
    await _authService.signOut();
  } on AuthException catch (e) {
    // signOut tetap menghapus sesi lokal walau request ke server gagal.
    debugPrint('signOut: ${e.message}');
  }
}

void clearError() {
  _errorMessage = null;
  notifyListeners();
}

void _setLoading(bool value) {
  _isLoading = value;
  if (value) _errorMessage = null;
  notifyListeners();
}

/// Menerjemahkan error Supabase Auth menjadi pesan berbahasa Indonesia.
String _mapAuthError(AuthException e) {
  if (e is AuthRetryableFetchException) {
    return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
  }
  switch (e.code) {
    case 'invalid_credentials':
```

```
        return 'Email atau password salah.';
      case 'email_not_confirmed':
        return 'Email belum dikonfirmasi. Buka link verifikasi di inbox Anda.';
      case 'user_already_exists':
      case 'email_exists':
        return 'Email sudah terdaftar. Silakan login.';
      case 'weak_password':
        return 'Password terlalu lemah. Gunakan minimal 6 karakter.';
      case 'email_address_invalid':
        return 'Alamat email tidak valid.';
      case 'over_email_send_rate_limit':
      case 'over_request_rate_limit':
        return 'Terlalu banyak percobaan. Tunggu beberapa menit lalu coba lagi.';
      case 'signup_disabled':
        return 'Pendaftaran akun baru sedang dinonaktifkan.';
    }
    return e.message;
  }
}
```

Membedah AuthProvider

Bagian	Penjelasan
`enum RegisterResult`	Register punya **tiga** kemungkinan hasil: gagal, langsung login, atau perlu konfirmasi email. Enum membuat UI bisa memakai `switch` yang jelas, alih-alih `bool` yang ambigu.
`_isLoading` + `_errorMessage`	State privat dengan getter publik (pola Modul 08). UI membaca `isLoading` untuk menonaktifkan tombol dan menampilkan spinner.
`displayName`	Mengambil `full_name` dari `userMetadata`, dengan email sebagai cadangan jika nama kosong.
`try / on AuthException / catch / finally`	`finally` **selalu** dijalankan, baik sukses maupun gagal, sehingga loading pasti dimatikan. Tanpa `finally`, spinner bisa berputar selamanya jika terjadi error.
`response.session == null`	Tanda bahwa Confirm email aktif: akun dibuat tetapi belum login.
`_setLoading(true)`	Sekaligus menghapus error lama, agar pesan error percobaan sebelumnya tidak muncul lagi.
`signOut()`	Error saat logout hanya dicatat. `signOut` tetap menghapus sesi lokal walaupun request ke server gagal (misalnya offline), jadi user tetap keluar.

Pemetaan error ke pesan ramah

`AuthException` memiliki properti `code` berisi kode error dari server Supabase Auth. Kode-kode yang kita tangani:

`e.code`	Kapan muncul	Pesan untuk user
`invalid_credentials`	Email tidak terdaftar atau password salah	Email atau password salah.
`email_not_confirmed`	Login sebelum klik link konfirmasi	Email belum dikonfirmasi...
`user_already_exists` / `email_exists`	Register dengan email yang sudah dipakai (Confirm email OFF)	Email sudah terdaftar. Silakan login.
`weak_password`	Password kurang dari batas minimal	Password terlalu lemah...
`over_email_send_rate_limit`	Terlalu banyak email konfirmasi terkirim	Terlalu banyak percobaan...
(AuthRetryableFetchException)	Tidak ada internet / server tidak terjangkau	Tidak dapat terhubung ke server...

> 💡 **Kenapa "Email atau password salah", bukan "Email tidak terdaftar"?**
>
> Supabase sengaja tidak membedakan kedua kasus. Pesan yang terlalu spesifik membantu penyerang menebak email mana yang terdaftar. Pesan gabungan seperti ini adalah praktik keamanan standar.

6. AuthGate, Home Sementara & main.dart

Bagaimana aplikasi tahu harus menampilkan Login atau Home? Kita **tidak** memakai `Navigator.pushReplacement` setelah login. Sebagai gantinya, satu widget bernama `AuthGate` mendengarkan stream status login dan memilih layar secara otomatis.

Kenapa AuthGate lebih baik daripada navigasi manual?

Situasi	Navigasi manual	AuthGate (reaktif)
Login berhasil	Harus memanggil `pushReplacement(Home)`	Otomatis pindah ke Home
Logout	Harus `pushAndRemoveUntil(Login)`	Otomatis kembali ke Login
Aplikasi dibuka ulang, sesi masih ada	Perlu cek manual di splash screen	Langsung ke Home (event `initialSession`)
Sesi kedaluwarsa / dicabut	Sering terlupa	Otomatis ke Login (event `signedOut`)

File 5: `lib/screens/auth_gate.dart`

`lib/screens/auth_gate.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../providers/auth_provider.dart';
import 'home_screen.dart';
import 'login_screen.dart';

/// "Penjaga pintu" aplikasi: mendengarkan perubahan status login lalu
/// memilih halaman yang tepat secara otomatis.
class AuthGate extends StatefulWidget {
  const AuthGate({super.key});

  @override
  State<AuthGate> createState() => _AuthGateState();
}

class _AuthGateState extends State<AuthGate> {
  // Simpan stream di State agar tidak dibuat ulang setiap build.
  late final Stream<AuthState> _authStream;

  @override
  void initState() {
    super.initState();
    _authStream = context.read<AuthProvider>().authStateChanges;
```

```
  }

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<AuthState>(
      stream: _authStream,
      builder: (context, snapshot) {
        // Sebelum event pertama datang, pakai sesi yang tersimpan di perangkat.
        final session = snapshot.hasData
            ? snapshot.data!.session
            : context.read<AuthProvider>().currentSession;

        if (session == null) {
          return const LoginScreen();
        }
        return const HomeScreen();
      },
    );
  }
}
```

- **Stream disimpan di `initState`** sebagai `late final`. Jika `stream:` diisi langsung dengan getter di dalam `build()`, setiap rebuild bisa membuat StreamBuilder berlangganan ulang.
- `context.read` boleh dipakai di `initState` karena kita hanya membaca sekali, tanpa mendengarkan perubahan.
- `snapshot.hasData ? ... : currentSession`: sebelum event pertama tiba (beberapa milidetik), kita memakai sesi tersimpan agar layar Login tidak "berkedip" sesaat bagi user yang sudah login.
- Setiap event (`signedIn`, `signedOut`, `tokenRefreshed`, ...) membawa `session` terbaru, jadi cukup cek `session == null`.

File 6 (sementara): `lib/screens/home_screen.dart`

Untuk menguji alur login, kita buat Home sederhana yang menampilkan nama dan email user serta tombol keluar. File ini akan **diganti seluruhnya** di Modul 14.

`lib/screens/home_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';

/// HomeScreen sementara untuk Bagian 1. Di Modul 14 file ini diganti
/// dengan daftar catatan lengkap.
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  Future<void> _logout(BuildContext context) async {
    await context.read<AuthProvider>().signOut();
    // Tidak perlu Navigator: AuthGate otomatis menampilkan LoginScreen.
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    final auth = context.watch<AuthProvider>();
    final user = auth.currentUser;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Kuliah'),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: () => _logout(context),
          ),
        ],
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(Icons.verified_user, size: 72, color: Colors.green),
              const SizedBox(height: 16),
              Text(
                'Halo, ${auth.displayName}!',
                style: Theme.of(context).textTheme.headlineSmall,
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 8),
              Text('Login sebagai ${user?.email ?? '-'}'),
              const SizedBox(height: 4),
              SelectableText(
                'User ID: ${user?.id ?? '-'}',
                style: Theme.of(context).textTheme.bodySmall,
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 24),
              const Text(
                'Autentikasi berhasil. Daftar catatan dibuat di Modul 14.',
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 24),
              OutlinedButton.icon(
                onPressed: () => _logout(context),
                icon: const Icon(Icons.logout),
                label: const Text('Keluar'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Perhatikan `_logout`: setelah `signOut()` tidak ada `Navigator` sama sekali. Stream memancarkan `signedOut`, lalu AuthGate menampilkan LoginScreen.

File 7: `lib/main.dart` (versi Bagian 1)

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'providers/auth_provider.dart';
import 'screens/auth_gate.dart';
import 'services/auth_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }

  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );

  final authService = AuthService(Supabase.instance.client);

  runApp(
    ChangeNotifierProvider(
      create: (_) => AuthProvider(authService),
      child: const CatatanKuliahApp(),
    ),
  );
}

class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

- `if (!Env.isConfigured)`: jika lupa `--dart-define-from-file`, aplikasi menampilkan petunjuk yang jelas, bukan error membingungkan dari Supabase.
- `AuthService(Supabase.instance.client)` dibuat *sesudah* `initialize` selesai, lalu disuntikkan ke `AuthProvider`.
- `ChangeNotifierProvider` di atas `MaterialApp` membuat `AuthProvider` bisa diakses dari semua halaman, termasuk halaman yang di-`push`.

> ⚠️ **Belum bisa dijalankan**
>
> `auth_gate.dart` meng-import `login_screen.dart`, yang baru kita buat di section berikutnya. Wajar jika editor masih menandai error merah. Lanjutkan ke section 7 dan 8.

Jika aplikasi dijalankan tanpa `--dart-define-from-file`, tampilannya seperti ini:

ConfigErrorApp: muncul saat env.json tidak diberikan

7. LoginScreen

LoginScreen menggabungkan materi Form (Modul 05) dan Provider (Modul 08): validasi input, tombol show/hide password, tombol dengan indikator loading, dan SnackBar saat gagal.

File 8: `lib/screens/login_screen.dart`

`lib/screens/login_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../utils/validators.dart';
import 'register_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    // Tutup keyboard lalu validasi form.
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final auth = context.read<AuthProvider>();
    final success = await auth.signIn(
      email: _emailController.text,
      password: _passwordController.text,
    );

    // Jika sukses, AuthGate otomatis pindah ke HomeScreen.
    if (!success && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(auth.errorMessage ?? 'Login gagal'),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
```

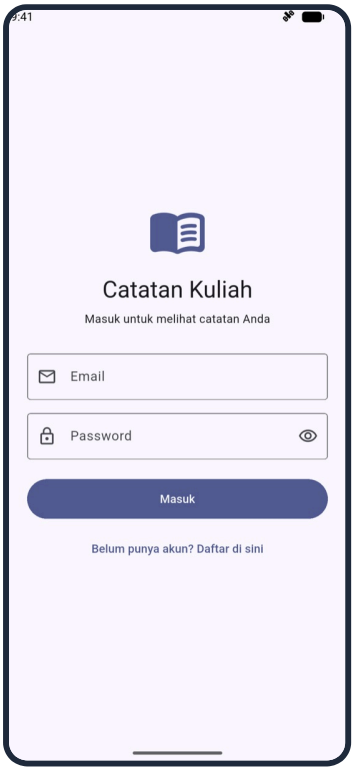
```
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.watch<AuthProvider>().isLoading;

    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Form(
              key: _formKey,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Icon(
                    Icons.menu_book_rounded,
                    size: 72,
                    color: Theme.of(context).colorScheme.primary,
                  ),
                  const SizedBox(height: 16),
                  Text(
                    'Catatan Kuliah',
                    textAlign: TextAlign.center,
                    style: Theme.of(context).textTheme.headlineMedium,
                  ),
                  const SizedBox(height: 8),
                  const Text(
                    'Masuk untuk melihat catatan Anda',
                    textAlign: TextAlign.center,
                  ),
                  const SizedBox(height: 32),
                  TextFormField(
                    controller: _emailController,
                    keyboardType: TextInputType.emailAddress,
                    textInputAction: TextInputAction.next,
                    autofillHints: const [AutofillHints.email],
                    decoration: const InputDecoration(
                      labelText: 'Email',
                      prefixIcon: Icon(Icons.email_outlined),
                      border: OutlineInputBorder(),
                    ),
                    validator: Validators.email,
                  ),
                  const SizedBox(height: 16),
                  TextFormField(
                    controller: _passwordController,
                    obscureText: _obscurePassword,
                    textInputAction: TextInputAction.done,
                    onFieldSubmitted: (_) => _submit(),
                    decoration: InputDecoration(
                      labelText: 'Password',
                      prefixIcon: const Icon(Icons.lock_outline),
                      border: const OutlineInputBorder(),
                      suffixIcon: IconButton(
                        tooltip: _obscurePassword
                            ? 'Tampilkan password'
```

```
                        : 'Sembunyikan password',
                    icon: Icon(
                      _obscurePassword
                          ? Icons.visibility_outlined
                          : Icons.visibility_off_outlined,
                    ),
                    onPressed: () {
                      setState(() => _obscurePassword = !_obscurePassword);
                    },
                  ),
                ),
                validator: Validators.password,
              ),
              const SizedBox(height: 24),
              FilledButton(
                onPressed: isLoading ? null : _submit,
                style: FilledButton.styleFrom(
                  minimumSize: const Size.fromHeight(48),
                ),
                child: isLoading
                    ? const SizedBox(
                        width: 22,
                        height: 22,
                        child: CircularProgressIndicator(strokeWidth: 2.5),
                      )
                    : const Text('Masuk'),
              ),
              const SizedBox(height: 12),
              TextButton(
                onPressed: isLoading
                    ? null
                    : () {
                        Navigator.of(context).push(
                          MaterialPageRoute(
                            builder: (_) => const RegisterScreen(),
                          ),
                        );
                      },
                child: const Text('Belum punya akun? Daftar di sini'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

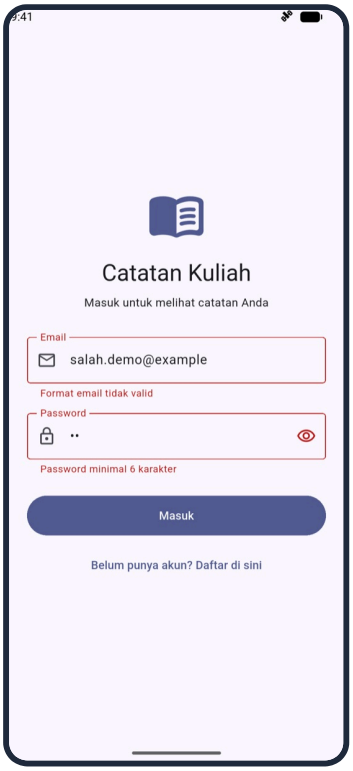
Membedah LoginScreen

Bagian	Penjelasan
`StatefulWidget`	Butuh state lokal: dua controller, `GlobalKey<FormState>`, dan `_obscurePassword`. Ini *ephemeral state*, jadi cukup `setState`, tidak perlu Provider.
`dispose()`	Controller wajib di-dispose agar tidak bocor memori.
`FocusScope.of(context).unfocus()`	Menutup keyboard sebelum proses login, supaya SnackBar tidak tertutup keyboard.
`_formKey.currentState!.validate()`	Menjalankan semua validator. Jika ada yang gagal, pesan merah muncul di bawah field dan proses berhenti.
`context.read<AuthProvider>()` di `_submit`	Di dalam fungsi aksi, pakai `read`, bukan `watch`.
`if (!success && mounted)`	Setelah `await`, widget mungkin sudah tidak ada. Saat login sukses, LoginScreen justru **dibuang** oleh AuthGate, sehingga pengecekan `mounted` wajib sebelum memakai `context`.
`context.watch<AuthProvider>().isLoading`	Di `build` kita `watch` agar tombol otomatis berubah menjadi spinner saat loading.
`onPressed: isLoading ? null : _submit`	`onPressed: null` membuat tombol nonaktif, sehingga mencegah klik ganda yang mengirim dua request login.
`suffixIcon: IconButton(...)`	Tombol mata untuk menampilkan atau menyembunyikan password dengan membalik nilai `obscureText`.
`onFieldSubmitted: (_) => _submit()`	Menekan tombol "selesai" di keyboard langsung memicu login.
`Navigator.push(RegisterScreen)`	Register dibuka di atas Login, dan tombol back mengembalikan user ke Login.

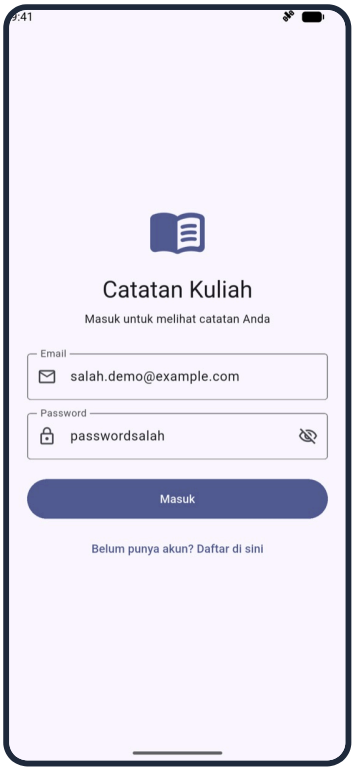
Hasil di emulator dengan kode di atas, terhubung ke project Supabase:



Tampilan awal LoginScreen



Validasi: format email salah dan password kurang dari 6 karakter



Ikon mata ditekan: password terlihat

Password salah: AuthException dari Supabase diterjemahkan menjadi pesan ramah

⚠️ **Kesalahan umum**

- **Memakai `context.watch` di dalam `_submit`** : memicu error *"Tried to listen to a value exposed with provider, from outside of the widget tree"*.
- **Memanggil `Navigator.pushReplacement(HomeScreen)` setelah login**: Home akan muncul dua kali (satu dari Navigator, satu dari AuthGate), dan logout jadi kacau.
- **Lupa cek `mounted`** : muncul warning `use_build_context_synchronously` dan berpotensi crash.

8. RegisterScreen & Menjalankan Aplikasi

RegisterScreen mirip LoginScreen, ditambah field nama lengkap, konfirmasi password, dan penanganan tiga kemungkinan hasil register.

File 9: `lib/screens/register_screen.dart`

`lib/screens/register_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../utils/validators.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  String? _validateConfirm(String? value) {
    if (value == null || value.isEmpty) return 'Ulangi password Anda';
    if (value != _passwordController.text) return 'Password tidak sama';
    return null;
  }

  Future<void> _submit() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final auth = context.read<AuthProvider>();
    final result = await auth.register(
      fullName: _nameController.text,
      email: _emailController.text,
      password: _passwordController.text,
    );
    if (!mounted) return;
```

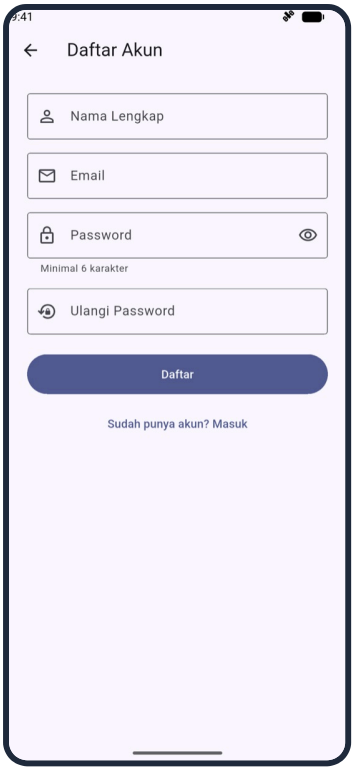
```
    switch (result) {
      case RegisterResult.loggedIn:
        // Sesi langsung aktif. AuthGate di bawah halaman ini sudah berganti
        // ke HomeScreen, jadi cukup tutup halaman register.
        Navigator.of(context).pop();
      case RegisterResult.needsEmailConfirmation:
        await showDialog<void>(
          context: context,
          builder: (dialogContext) => AlertDialog(
            title: const Text('Cek Email Anda'),
            content: Text(
              'Link konfirmasi sudah dikirim ke ${_emailController.text.trim()}. '
              'Klik link tersebut, lalu login dengan akun baru Anda.',
            ),
            actions: [
              FilledButton(
                onPressed: () => Navigator.of(dialogContext).pop(),
                child: const Text('Mengerti'),
              ),
            ],
          ),
        );
        if (mounted) Navigator.of(context).pop();
      case RegisterResult.failed:
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(auth.errorMessage ?? 'Registrasi gagal'),
            backgroundColor: Theme.of(context).colorScheme.error,
          ),
        );
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.watch<AuthProvider>().isLoading;

    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Akun')),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                TextFormField(
                  controller: _nameController,
                  textCapitalization: TextCapitalization.words,
                  textInputAction: TextInputAction.next,
                  decoration: const InputDecoration(
                    labelText: 'Nama Lengkap',
                    prefixIcon: Icon(Icons.person_outline),
                    border: OutlineInputBorder(),
                  ),
                  validator: Validators.fullName,
                ),
```

```
                const SizedBox(height: 16),
                TextFormField(
                  controller: _emailController,
                  keyboardType: TextInputType.emailAddress,
                  textInputAction: TextInputAction.next,
                  decoration: const InputDecoration(
                    labelText: 'Email',
                    prefixIcon: Icon(Icons.email_outlined),
                    border: OutlineInputBorder(),
                  ),
                  validator: Validators.email,
                ),
                const SizedBox(height: 16),
                TextFormField(
                  controller: _passwordController,
                  obscureText: _obscurePassword,
                  textInputAction: TextInputAction.next,
                  decoration: InputDecoration(
                    labelText: 'Password',
                    helperText: 'Minimal 6 karakter',
                    prefixIcon: const Icon(Icons.lock_outline),
                    border: const OutlineInputBorder(),
                    suffixIcon: IconButton(
                      icon: Icon(
                        _obscurePassword
                            ? Icons.visibility_outlined
                            : Icons.visibility_off_outlined,
                      ),
                      onPressed: () {
                        setState(() => _obscurePassword = !_obscurePassword);
                      },
                    ),
                  ),
                  validator: Validators.password,
                ),
                const SizedBox(height: 16),
                TextFormField(
                  controller: _confirmController,
                  obscureText: _obscurePassword,
                  textInputAction: TextInputAction.done,
                  onFieldSubmitted: (_) => _submit(),
                  decoration: const InputDecoration(
                    labelText: 'Ulangi Password',
                    prefixIcon: Icon(Icons.lock_reset_outlined),
                    border: OutlineInputBorder(),
                  ),
                  validator: _validateConfirm,
                ),
                const SizedBox(height: 24),
                FilledButton(
                  onPressed: isLoading ? null : _submit,
                  style: FilledButton.styleFrom(
                    minimumSize: const Size.fromHeight(48),
                  ),
                  child: isLoading
                      ? const SizedBox(
                          width: 22,
                          height: 22,
                          child: CircularProgressIndicator(strokeWidth: 2.5),
```

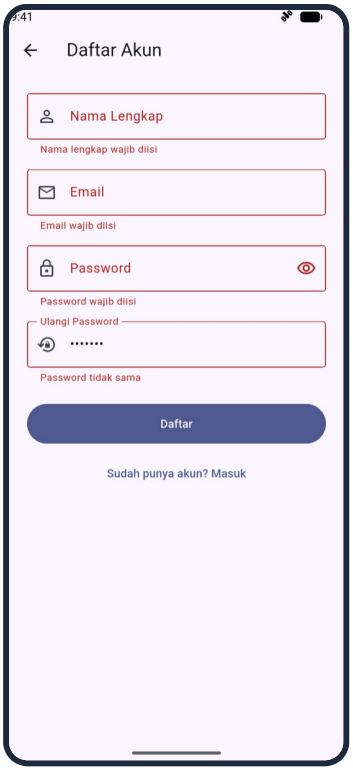
```
                      )
                    : const Text('Daftar'),
              ),
              const SizedBox(height: 12),
              TextButton(
                onPressed: isLoading
                    ? null
                    : () => Navigator.of(context).pop(),
                child: const Text('Sudah punya akun? Masuk'),
              ),
            ],
          ),
        ),
      ),
    ),
  );
  }
}
```

Membedah RegisterScreen

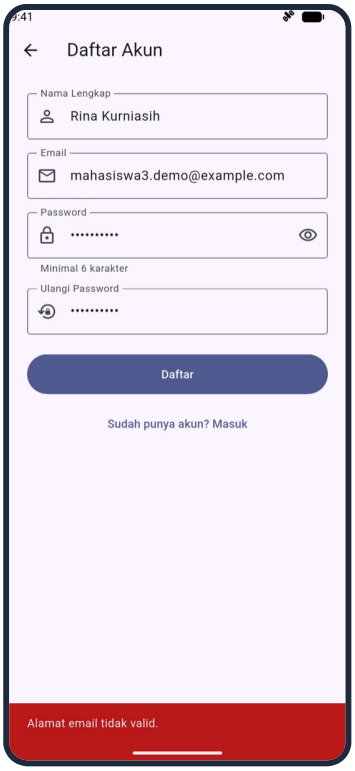
- **`_validateConfirm`** adalah validator yang membandingkan dua field. Ia dibuat sebagai method (bukan di `Validators`) karena butuh akses ke `_passwordController`.
- **`textCapitalization: TextCapitalization.words`** membuat keyboard otomatis menulis huruf kapital di awal setiap kata nama.
- **`switch (result)`** menangani tiga kasus:
  - `loggedIn` (Confirm email OFF): sesi langsung aktif dan AuthGate di *bawah* halaman ini sudah berganti ke Home. Cukup `pop()`, maka user langsung melihat Home.
  - `needsEmailConfirmation` (Confirm email ON): tampilkan dialog, lalu kembali ke Login.
  - `failed`: tampilkan SnackBar berisi pesan dari provider.
- **`if (!mounted) return;`** diletakkan tepat setelah `await auth.register(...)`, sebelum memakai `context`. Setelah `await showDialog` pun kita cek `mounted` lagi.
- Dialog memakai `dialogContext` miliknya sendiri untuk `pop`, agar yang tertutup adalah dialog, bukan halaman register.



RegisterScreen dibuka dari tombol "Daftar di sini"



Validasi: field kosong dan password tidak sama



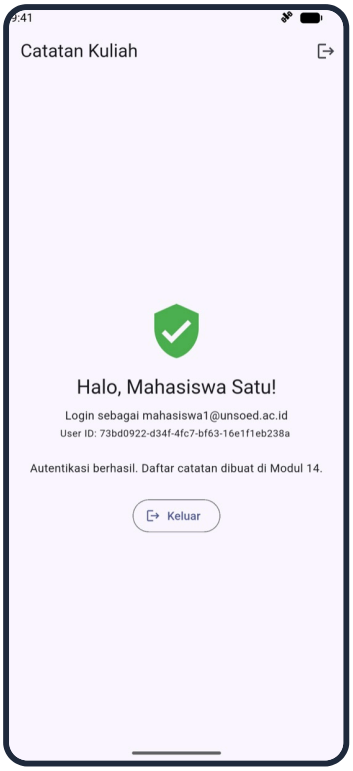
Alamat @example.com ditolak Supabase: pesan error tampil di SnackBar

Jalankan aplikasi!

Semua file Bagian 1 sudah lengkap. Jalankan:

```
flutter analyze
flutter run --dart-define-from-file=env.json
```

`flutter analyze` harus melaporkan *No issues found!*. Lalu coba register, dan jika Confirm email OFF Anda langsung masuk ke Home sementara:



Setelah login: HomeScreen sementara menampilkan nama dari metadata

9. Pengujian & Troubleshooting

Fitur autentikasi wajib diuji secara sistematis karena menyangkut keamanan dan menjadi pintu masuk seluruh aplikasi. Kerjakan checklist berikut dan catat hasilnya di laporan praktikum.

Checklist pengujian manual

No	Skenario	Langkah	Hasil yang diharapkan
1	Validasi form login	Tekan Masuk dengan form kosong, lalu isi email tanpa "@"	Pesan merah di bawah field, tidak ada request ke server
2	Show/hide password	Ketik password, tekan ikon mata	Teks password terlihat/tersembunyi bergantian
3	Validasi register	Isi password dan ulangi password berbeda	"Password tidak sama"
4	Register (Confirm email OFF)	Isi semua field dengan benar, tekan Daftar	Langsung masuk Home dan nama tampil. Baris baru muncul di tabel `profiles`
5	Register (Confirm email ON)	Aktifkan Confirm email di dashboard, register email lain	Dialog "Cek Email Anda", kembali ke Login. Login sebelum konfirmasi menghasilkan "Email belum dikonfirmasi"
6	Register email yang sama (Confirm OFF)	Register ulang dengan email dari skenario 4	SnackBar "Email sudah terdaftar. Silakan login."
7	Login password salah	Email benar, password salah	SnackBar "Email atau password salah."
8	Klik ganda tombol	Tekan Masuk berkali-kali dengan cepat	Tombol nonaktif + spinner, hanya satu request
9	Login tanpa internet	Matikan Wi-Fi/data emulator, coba login	SnackBar "Tidak dapat terhubung ke server..."
10	Logout	Tekan ikon logout di Home	Kembali ke Login tanpa bisa "back" ke Home
11	Sesi tersimpan	Login, tutup aplikasi sepenuhnya (swipe dari recent apps), buka lagi	Langsung tampil Home tanpa login ulang
12	Lupa env.json	Jalankan `flutter run` tanpa `--dart-define-from-file`	Layar petunjuk ConfigErrorApp

> 💡 **Mematikan internet di emulator**
>
> Tarik panel notifikasi emulator lalu matikan Wi-Fi dan Mobile data, atau aktifkan Airplane mode. Di HP fisik cukup aktifkan mode pesawat.

Troubleshooting autentikasi

Gejala	Penyebab	Solusi
Layar "SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi"	Run tanpa `--dart-define-from-file=env.json`	Tambahkan argumen, atau atur `launch.json` (Modul 12 section 4)
Invalid API key / 401	Key salah salin atau memakai key dari project lain	Salin ulang dari Project Settings → API Keys
Database error saving new user	Fungsi trigger `handle_new_user` error	Cek Logs → Postgres. Pastikan tabel `public.profiles` ada dan nama kolom benar
Register sukses tetapi tidak bisa login	Confirm email ON dan link belum diklik	Klik link di email, atau matikan Confirm email untuk praktikum
Register gagal "Alamat email tidak valid"	Supabase menolak domain contoh seperti `@example.com` (kode `email_address_invalid`), dan server email bawaan Supabase hanya mengirim ke alamat anggota tim project	Pakai alamat email sungguhan untuk uji register, atau buat akun uji lewat **Authentication → Users → Add user** dengan **Auto Confirm User**. Untuk produksi, pasang SMTP sendiri di **Authentication → Emails → SMTP Settings**
Email konfirmasi tidak pernah datang	Batas email bawaan Supabase / masuk folder spam	Cek spam, tunggu, atau matikan Confirm email
Kolom `full_name` di profiles kosong	Key metadata berbeda (`fullName` vs `full_name`)	Samakan key di `signUp(data: ...)` dan di SQL trigger
Setelah login, layar tetap di Login	`home:` di MaterialApp bukan `AuthGate`, atau stream dibuat ulang setiap build	Periksa `main.dart` dan `auth_gate.dart`
Setelah register (Confirm OFF), halaman Register tidak tertutup	Lupa `Navigator.pop()` pada kasus `loggedIn`	Lihat `switch` di RegisterScreen
Error *Failed host lookup*	Tidak ada internet, emulator offline, atau URL salah ketik	Cek koneksi emulator dan isi `SUPABASE_URL`

Ringkasan Modul 13

- Arsitektur berlapis: UI → Provider → Service/Repository → Supabase/Hive.
- Tabel `profiles` diisi otomatis oleh trigger `security definer` dari `raw_user_meta_data`.
- `AuthService` membungkus Supabase Auth, sedangkan `AuthProvider` menyimpan loading/error dan menerjemahkan kode error.
- `AuthGate` + `onAuthStateChange` membuat navigasi login/logout otomatis tanpa Navigator manual.
- Login dan Register memakai Form + validator + show/hide password + tombol loading + SnackBar.

Di Modul 14 kita melanjutkan kode yang **sama**: menambah tabel `notes`, cache Hive, dan mengganti HomeScreen sementara dengan daftar catatan lengkap.

Tugas Praktikum: Praktikum 13: Autentikasi Aplikasi Catatan Kuliah

Sasaran Praktikum

Membangun fitur register, login, logout, dan sesi tersimpan yang aman dengan Supabase Auth, mengikuti arsitektur berlapis.

Langkah Pengerjaan

1. Buat proyek `catatan_kuliah`, pasang package dengan `flutter pub add supabase_flutter provider hive_ce hive_ce_flutter`, dan siapkan `env.json` + `.gitignore`.
2. Jalankan SQL tabel `profiles`, fungsi `handle_new_user`, trigger, dan policy RLS di SQL Editor. Simpan SQL di `supabase/setup.sql`.
3. Implementasikan `env.dart`, `validators.dart`, `auth_service.dart`, dan `auth_provider.dart` sesuai section 4–5.
4. Implementasikan `auth_gate.dart`, HomeScreen sementara, dan `main.dart` (section 6).
5. Implementasikan LoginScreen dan RegisterScreen lengkap dengan validasi, show/hide password, tombol loading, dan SnackBar error (section 7–8).
6. Pastikan `flutter analyze` tidak menemukan masalah.
7. Jalankan 12 skenario pada checklist section 9. Lampirkan screenshot minimal untuk skenario 4, 5, 7, 10, dan 11.
8. Tambahan: tampilkan nama dari tabel `profiles` (bukan metadata) di HomeScreen memakai `select().eq('id', userId).maybeSingle()`.

Output yang Diharapkan

Aplikasi dapat register (dengan nama tersimpan di tabel profiles), login, logout, mengingat sesi setelah aplikasi ditutup, dan menampilkan pesan error berbahasa Indonesia untuk password salah, email terdaftar, email belum dikonfirmasi, dan tidak ada koneksi.

Rubrik Penilaian

Setup Supabase: tabel profiles, trigger & RLS benar (20%), Arsitektur berlapis: AuthService & AuthProvider terpisah dari UI (20%), AuthGate & navigasi otomatis (15%), Form login/register: validasi, show/hide, loading, SnackBar (25%), Laporan pengujian 12 skenario (15%), Kerapian kode & flutter analyze bersih (5%).

Kuis Pemahaman

1. Dalam arsitektur proyek ini, class mana yang boleh memanggil supabase.auth secara langsung?

A. LoginScreen

B. AuthProvider

C. AuthService

D. Semua class boleh

2. Mengapa fungsi handle_new_user diberi atribut 'security definer'?

A. Agar fungsi berjalan lebih cepat

B. Agar fungsi berjalan dengan hak pembuatnya sehingga bisa insert ke profiles walaupun RLS aktif dan user belum login

C. Agar password user dienkripsi

D. Agar trigger hanya berjalan sekali seumur hidup

3. Nilai apa yang menandakan bahwa signUp berhasil tetapi user harus mengonfirmasi email terlebih dahulu?

A. response.user == null

B. response.session == null

C. AuthException dengan code invalid_credentials

D. response.session != null

4. Setelah login sukses, bagaimana aplikasi berpindah dari LoginScreen ke HomeScreen?

A. LoginScreen memanggil Navigator.pushReplacement

B. AuthGate menerima event signedIn dari onAuthStateChange lalu membangun HomeScreen

C. AuthProvider memanggil runApp ulang

D. User harus menekan tombol refresh

5. Mengapa stream auth disimpan di initState AuthGate, bukan dipanggil langsung di dalam build()?

A. Karena initState lebih cepat dari build

B. Agar StreamBuilder tidak berlangganan ulang setiap kali widget di-rebuild

C. Karena build() tidak boleh memakai Stream

D. Karena Supabase melarangnya

6. Apa fungsi blok 'finally' pada method signIn di AuthProvider?

A. Menampilkan SnackBar

B. Memastikan isLoading kembali false baik login berhasil maupun gagal

C. Mengulang login jika gagal

D. Menghapus sesi

7. Pada LoginScreen, mengapa onPressed tombol Masuk diisi null ketika isLoading bernilai true?

A. Agar warna tombol berubah saja

B. Untuk menonaktifkan tombol sehingga mencegah request login ganda akibat klik berulang

C. Agar keyboard tertutup

D. Karena Flutter mewajibkannya

8. Mengapa pesan error untuk kode invalid_credentials dibuat 'Email atau password salah' dan bukan 'Email tidak terdaftar'?

A. Agar pesan lebih pendek

B. Supaya penyerang tidak bisa menebak email mana yang terdaftar (user enumeration)

C. Karena Supabase tidak tahu emailnya

D. Karena Bahasa Indonesia tidak punya kata yang tepat

Kunci Jawaban & Pembahasan

1. C — AuthService adalah satu-satunya lapisan yang berbicara dengan Supabase Auth. Provider memanggil service, sedangkan UI memanggil provider. Pemisahan ini memudahkan pengujian dan penggantian backend.

2. B — Saat register, request belum membawa sesi login sehingga RLS menolak insert. security definer membuat fungsi berjalan dengan hak pemiliknya (admin).

3. B — Jika Confirm email aktif, akun dibuat (user terisi) tetapi session bernilai null sampai link konfirmasi diklik.

4. B — AuthGate mendengarkan stream status login. Begitu ada session, ia mengembalikan HomeScreen secara otomatis tanpa Navigator manual.

5. B — Jika objek stream berbeda di setiap build, StreamBuilder akan berhenti berlangganan dan berlangganan ulang. Menyimpannya sekali di State membuat langganan stabil.

6. B — finally selalu dieksekusi setelah try/catch, sehingga spinner pasti berhenti dan tombol aktif kembali dalam semua kondisi.

7. B — Tombol dengan onPressed null otomatis nonaktif. Ini mencegah user mengirim beberapa request login bersamaan.

8. B — Pesan gabungan adalah praktik keamanan standar. Membedakan keduanya membocorkan informasi tentang akun yang ada.

MODUL 14 · PROYEK BAGIAN 2

Proyek Catatan Kuliah (2): CRUD Supabase & Cache Offline Hive

Melanjutkan proyek Catatan Kuliah: membuat tabel notes dengan RLS dan trigger updated_at, model Note, data source remote (Supabase) dan lokal (Hive), repository dengan fallback offline, NotesProvider dengan pencarian dan optimistic delete, halaman daftar/form/detail, logout yang membersihkan cache, unit test, listing lengkap seluruh file, serta troubleshooting.

Durasi Pertemuan 14 (150 Menit) + Belajar Mandiri

Prasyarat Modul 11: Hive
Modul 12: Supabase
Modul 13: Proyek Catatan Kuliah (1)

Capaian Pembelajaran

- Merancang tabel milik user dengan default auth.uid(), constraint, index, empat policy RLS, dan trigger updated_at.
- Membuat model dengan fromJson, toJson (cache), dan toMap (payload insert/update).
- Memisahkan NoteRemoteDataSource (Supabase) dan NoteLocalDataSource (Hive Map) lalu menggabungkannya di NoteRepository.
- Menerapkan strategi network first, cache fallback beserta penanda mode offline.
- Mengelola state daftar, pencarian, filter, dan optimistic delete dengan rollback di NotesProvider.
- Membangun UI daftar dengan RefreshIndicator, empty/loading/error state, Dismissible, serta satu form untuk tambah dan edit.
- Menulis unit test repository dan Hive memakai fake data source dan folder sementara.
- Mendiagnosis error umum seperti 42501 RLS, tabel belum dibuat, dan initialize yang terlewat.

Daftar Isi Modul

1. Tabel notes, RLS & Trigger updated_at
2. Model Note (fromJson, toJson, toMap)
3. NoteRemoteDataSource: CRUD Supabase
4. NoteLocalDataSource: Cache Hive
5. NoteRepository & Mode Offline
6. NotesProvider & Optimistic Delete
7. HomeScreen: Daftar, Cari, Filter, Hapus
8. Form Tambah/Edit & Detail Catatan

9. main.dart Final & Logout yang Aman
10. Unit Test Repository & Hive
11. Listing Lengkap Seluruh File
12. Troubleshooting & Ide Pengembangan
13. Tugas Praktikum
14. Kuis Pemahaman & Kunci Jawaban

1. Tabel notes, RLS & Trigger updated_at

Kita melanjutkan proyek dari Modul 13. Login dan register sudah berjalan, dan sekarang saatnya membuat inti aplikasi: **catatan**. Seperti biasa, kita mulai dari database.

Rencana Modul 14 (urutan kerja)

1. Tabel `notes` + RLS + trigger `updated_at` (section ini)
2. Model `Note`
3. `NoteRemoteDataSource`: CRUD ke Supabase
4. `NoteLocalDataSource`: cache Hive
5. `NoteRepository`: gabungan + fallback offline
6. `NotesProvider`: state, pencarian, optimistic delete
7. HomeScreen final + widget pendukung
8. NoteFormScreen (tambah & edit) + NoteDetailScreen
9. `main.dart` final + logout membersihkan cache
10. Unit test repository & Hive

Desain tabel `notes`

Kolom	Tipe	Keterangan
`id`	uuid	Primary key, dibuat database (`gen_random_uuid()`)
`user_id`	uuid	Pemilik catatan, otomatis `auth.uid()`, FK ke `auth.users`
`title`	text	Judul, wajib, 1–100 karakter
`content`	text	Isi catatan
`category`	text	Salah satu dari Kuliah, Tugas, Ujian, Lainnya
`created_at`	timestamptz	Waktu dibuat, otomatis
`updated_at`	timestamptz	Waktu terakhir diubah, diperbarui oleh trigger

Jalankan SQL berikut di SQL Editor

`supabase/setup.sql`

```
create table if not exists public.notes (
  id uuid primary key default gen_random_uuid(),
```

```
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null check (char_length(title) between 1 and 100),
  content text not null default '',
  category text not null default 'Kuliah'
    check (category in ('Kuliah', 'Tugas', 'Ujian', 'Lainnya')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists notes_user_id_idx on public.notes (user_id);

alter table public.notes enable row level security;

drop policy if exists "User melihat catatannya sendiri" on public.notes;
create policy "User melihat catatannya sendiri"
on public.notes for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "User menambah catatannya sendiri" on public.notes;
create policy "User menambah catatannya sendiri"
on public.notes for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "User mengubah catatannya sendiri" on public.notes;
create policy "User mengubah catatannya sendiri"
on public.notes for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "User menghapus catatannya sendiri" on public.notes;
create policy "User menghapus catatannya sendiri"
on public.notes for delete
to authenticated
using ((select auth.uid()) = user_id);

create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

drop trigger if exists notes_set_updated_at on public.notes;
create trigger notes_set_updated_at
  before update on public.notes
  for each row execute procedure public.set_updated_at();
```

Penjelasan

- **`user_id ... default auth.uid()`** : aplikasi *tidak perlu* mengirim `user_id` saat insert. Database mengisinya dari token login. Ini juga lebih aman karena user tidak bisa "menitipkan" catatan

atas nama orang lain, sebab policy `with check` akan menolaknya.

- `check (category in (...))`: database hanya menerima empat kategori, sama dengan daftar di model Dart.
- `create index ... (user_id)`: setiap query difilter RLS berdasarkan `user_id`. Index membuat filter itu tetap cepat walaupun tabel berisi ribuan catatan.
- **Empat policy** (select, insert, update, delete) semuanya memakai syarat yang sama: `(select auth.uid()) = user_id`.
- **Trigger** `set_updated_at` dijalankan `before update`, sehingga nilai `new.updated_at` diganti `now()` sebelum baris disimpan. Aplikasi tidak perlu (dan tidak bisa salah) mengirim waktu.

Uji RLS dari dua akun

1. Buka **Table Editor → notes** dan pastikan tabel ada dengan label RLS aktif (bukan UNRESTRICTED).
2. Setelah aplikasi selesai, buat dua akun berbeda dan masing-masing menulis catatan.
3. Login bergantian: setiap akun hanya boleh melihat catatannya sendiri. Di Table Editor (mode admin) semua catatan tetap terlihat.

> ⚠️ **Jangan lewati RLS**
>
> Jika lupa `enable row level security`, aplikasi tetap "berjalan normal", tetapi semua user bisa membaca catatan semua orang lewat API. Kesalahan ini tidak terlihat saat diuji dengan satu akun, jadi selalu uji dengan dua akun.

2. Model Note (fromJson, toJson, toMap)

Model `Note` mewakili satu baris tabel `notes`. Model ini dipakai di tiga tempat: data dari Supabase, data dari cache Hive, dan data yang dikirim saat insert/update. Karena itu kita butuh tiga konversi.

File: `lib/models/note.dart`

`lib/models/note.dart`

```dart
/// Model satu catatan kuliah. Satu objek Note = satu baris di tabel `notes`.
class Note {
  const Note({
    required this.id,
    required this.userId,
    required this.title,
    required this.content,
    required this.category,
    required this.createdAt,
    required this.updatedAt,
  });

  /// Pilihan kategori yang tersedia di form.
  static const List<String> categories = [
    'Kuliah',
    'Tugas',
    'Ujian',
    'Lainnya',
  ];

  final String id;
  final String userId;
  final String title;
  final String content;
  final String category;
  final DateTime createdAt;
  final DateTime updatedAt;

  /// Membuat Note dari Map. Dipakai untuk data dari Supabase
  /// (`Map<String, dynamic>`) maupun dari Hive (`Map<dynamic, dynamic>`).
  factory Note.fromJson(Map<dynamic, dynamic> json) {
    return Note(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String? ?? '',
      content: json['content'] as String? ?? '',
      category: json['category'] as String? ?? 'Lainnya',
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }

  /// Seluruh kolom, dipakai untuk menyimpan cache di Hive.
  Map<String, dynamic> toJson() {
    return {
      'id': id,
```

```
      'user_id': userId,
      'title': title,
      'content': content,
      'category': category,
      'created_at': createdAt.toIso8601String(),
      'updated_at': updatedAt.toIso8601String(),
    };
  }

  /// Hanya kolom yang boleh diisi/diubah aplikasi, dipakai untuk
  /// insert/update ke Supabase. id, user_id, created_at, dan updated_at
  /// diisi otomatis oleh database.
  Map<String, dynamic> toMap() {
    return {'title': title, 'content': content, 'category': category};
  }

  Note copyWith({
    String? title,
    String? content,
    String? category,
    DateTime? updatedAt,
  }) {
    return Note(
      id: id,
      userId: userId,
      title: title ?? this.title,
      content: content ?? this.content,
      category: category ?? this.category,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }
}
```

Tiga konversi, tiga tujuan

Method	Arah	Dipakai oleh	Isi
`Note.fromJson(map)`	Map → Note	Remote (Supabase) & Local (Hive)	Semua kolom
`toJson()`	Note → Map	Local (simpan ke Hive)	Semua kolom, tanggal sebagai String ISO
`toMap()`	Note → Map	Remote (update ke Supabase)	Hanya `title`, `content`, `category`

Hal-hal yang perlu diperhatikan

- **Parameter** `Map<dynamic, dynamic>`: Supabase mengembalikan `Map<String, dynamic>`, sedangkan Hive mengembalikan `Map<dynamic, dynamic>` saat membaca Map yang disimpan.

Dengan tipe parameter yang lebih umum, satu factory bisa melayani keduanya tanpa konversi tambahan.

- `json['title'] as String? ?? ''`: aman jika nilai null, tidak langsung crash.
- `DateTime.parse`: Supabase mengirim `timestamptz` sebagai String ISO-8601 (misalnya `2026-10-28T07:15:00.123+00:00`). Kita menyimpannya ke Hive dalam format yang sama, sehingga parsing-nya identik.
- **Kenapa `toMap()` tidak mengirim id, user_id, dan tanggal?** Kolom itu milik database: id dan created_at diisi default, user_id dari `auth.uid()`, dan updated_at dari trigger. Mengirim lebih sedikit data berarti lebih sedikit peluang salah.
- `copyWith` membuat salinan dengan beberapa nilai diganti. Ini dipakai form edit agar objek lama tidak diubah langsung (*immutable*).
- `static const categories` menjadi satu sumber kebenaran untuk dropdown form dan chip filter, dan isinya harus sama dengan `check` di SQL.

Uji cepat di DartPad / main sementara

```
final note = Note.fromJson({
  'id': 'a1',
  'user_id': 'u1',
  'title': 'Basis Data',
  'content': 'Normalisasi 3NF',
  'category': 'Kuliah',
  'created_at': '2026-10-28T07:00:00+00:00',
  'updated_at': '2026-10-28T07:00:00+00:00',
});
debugPrint(note.toMap()); // {title: Basis Data, content: Normalisasi 3NF, category: Kuliah}
debugPrint(Note.fromJson(note.toJson()).title); // Basis Data (bolak-balik aman)
```

3. NoteRemoteDataSource: CRUD Supabase

`NoteRemoteDataSource` berisi semua query Supabase untuk tabel `notes`. Isinya hanyalah penerapan query dasar dari Modul 12 section 7, yang kini dibungkus dalam satu class.

File: `lib/data/note_remote_data_source.dart`

`lib/data/note_remote_data_source.dart`

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';

/// Semua operasi CRUD ke tabel `notes` di Supabase.
/// RLS di database memastikan setiap user hanya melihat barisnya sendiri.
class NoteRemoteDataSource {
  NoteRemoteDataSource(this._client);

  final SupabaseClient _client;

  static const String _table = 'notes';

  Future<List<Note>> fetchNotes() async {
    final rows = await _client
        .from(_table)
        .select()
        .order('updated_at', ascending: false);
    return rows.map(Note.fromJson).toList();
  }

  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final row = await _client
        .from(_table)
        .insert({'title': title, 'content': content, 'category': category})
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<Note> updateNote(Note note) async {
    final row = await _client
        .from(_table)
        .update(note.toMap())
        .eq('id', note.id)
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<void> deleteNote(String id) async {
    await _client.from(_table).delete().eq('id', id);
```

```
  }
}
```

Penjelasan per method

Method	SQL setara	Catatan
`fetchNotes()`	`SELECT * FROM notes ORDER BY updated_at DESC`	Tidak ada `.eq('user_id', ...)` karena RLS sudah memfilter
`createNote()`	`INSERT ... RETURNING *`	`.select().single()` mengembalikan baris baru lengkap dengan id dan tanggal dari database
`updateNote()`	`UPDATE ... WHERE id = ? RETURNING *`	Mengembalikan `updated_at` terbaru dari trigger
`deleteNote()`	`DELETE FROM notes WHERE id = ?`	Filter `id` wajib

> ℹ️ **Kenapa insert tidak mengirim `user_id` ?**
>
> Kolom `user_id` memiliki `default auth.uid()`. Jika Anda tetap ingin mengirimnya secara eksplisit, ambil dari `Supabase.instance.client.auth.currentUser!.id`. Namun mengandalkan default database lebih sederhana dan tidak mungkin salah.

> 💡 **Update jika RLS menolak**
>
> Jika user mencoba mengubah catatan yang bukan miliknya, policy `using` membuat 0 baris cocok. Karena kita memanggil `.single()`, hasil 0 baris berubah menjadi `PostgrestException` (code `PGRST116`), sehingga kegagalan tidak lolos diam-diam.

4. NoteLocalDataSource: Cache Hive

Kita memakai Hive (Modul 11) sebagai **cache**: salinan terakhir catatan dari server yang disimpan di HP. Jika server tidak bisa dihubungi, aplikasi tetap bisa menampilkan catatan dari cache.

Kenapa menyimpan sebagai Map, bukan TypeAdapter?

Di Modul 11 Anda belajar dua cara: menyimpan Map biasa atau membuat `TypeAdapter`. Untuk cache, Map lebih sederhana: model `Note` sudah punya `toJson()` / `fromJson()`, tidak perlu `registerAdapter`, tidak ada `typeId` yang harus dijaga, dan tidak perlu code generation.

```
Box<Map> "notes_cache"
┌──────────────────────────────────────┬──────────────────────────────────────────┐
| key (id catatan)                     | value (Map hasil note.toJson())          |
├──────────────────────────────────────┼──────────────────────────────────────────┤
| "3f6c2a9e-8b1d-4c7a-9e21-..."        | {id: ..., title: "Basis Data", ...}      |
| "a71d0c44-52be-4f0e-8d3a-..."        | {id: ..., title: "Tugas Mobile", ...}    |
└──────────────────────────────────────┴──────────────────────────────────────────┘
```

Memakai **id catatan sebagai key** membuat update dan delete satu catatan menjadi operasi langsung (`put(id)` / `delete(id)`), tanpa perlu mencari indeks.

File: `lib/data/note_local_data_source.dart`

`lib/data/note_local_data_source.dart`

```dart
import 'package:hive_ce/hive_ce.dart';

import '../models/note.dart';

/// Cache catatan di Hive. Setiap catatan disimpan sebagai Map
/// dengan key = id catatan, sehingga tidak perlu TypeAdapter/codegen.
class NoteLocalDataSource {
  NoteLocalDataSource(this._box);

  static const String boxName = 'notes_cache';

  final Box<Map> _box;

  /// Membuka box. Panggil setelah Hive.initFlutter() (atau Hive.init()).
  static Future<NoteLocalDataSource> open() async {
    final box = await Hive.openBox<Map>(boxName);
    return NoteLocalDataSource(box);
  }

  /// Semua catatan di cache, urut dari yang terakhir diubah.
  List<Note> getNotes() {
    final notes = _box.values.map(Note.fromJson).toList();
    notes.sort((a, b) => b.updatedAt.compareTo(a.updatedAt));
    return notes;
  }
```

```
  /// Mengganti seluruh isi cache dengan data terbaru dari server.
  Future<void> saveNotes(List<Note> notes) async {
    await _box.clear();
    await _box.putAll({for (final note in notes) note.id: note.toJson()});
  }

  Future<void> saveNote(Note note) => _box.put(note.id, note.toJson());

  Future<void> deleteNote(String id) => _box.delete(id);

  /// Dipanggil saat logout agar data user lain tidak tertinggal.
  Future<void> clear() => _box.clear();
}
```

Penjelasan

- `import 'package:hive_ce/hive_ce.dart'` : file ini cukup memakai Hive inti, tanpa Flutter. Karena itu ia bisa diuji dengan `Hive.init(folderSementara)` di unit test (section 10).
- `static Future<NoteLocalDataSource> open()` : membuka box bersifat async, sedangkan constructor tidak boleh async. Solusinya adalah *factory method* statis yang membuka box lalu membuat objek.
- `getNotes()` bersifat sinkron karena Hive membaca dari memori setelah box dibuka. Hasilnya diurutkan `updatedAt` terbaru, sama dengan urutan dari server.
- `saveNotes()` mengosongkan box lalu `putAll`. Catatan yang sudah dihapus di server (misalnya dari HP lain) ikut hilang dari cache.
- `clear()` dipanggil saat logout. Tanpa ini, user B yang login di HP yang sama bisa sempat melihat catatan user A ketika offline.

Inisialisasi di `main()`

Box harus dibuka sebelum dipakai. Potongan ini akan menjadi bagian dari `main.dart` final di section 9:

```
await Hive.initFlutter();                // folder penyimpanan di HP
final localDataSource = await NoteLocalDataSource.open(); // buka box
```

5. NoteRepository & Mode Offline

`NoteRepository` adalah "satu pintu" untuk data catatan. Provider cukup memanggil `repository.getNotes()` tanpa perlu tahu apakah data datang dari Supabase atau dari cache Hive.

Strategi: *network first, cache fallback*

```
getNotes()
   |
   ├─► remote.fetchNotes()  (maks 10 detik)
   |       |
   |       ├─ sukses ──────────► local.saveNotes(data) ──► NotesResult(data, fromCache: false)
   |       |
   |       ├─ PostgrestException ──► rethrow  (server MENJAWAB tapi menolak:
   |       |                                    RLS, tabel tidak ada → harus diperbaiki)
   |       |
   |       └─ error lain ─────► local.getNotes() ──► NotesResult(cache, fromCache: true)
   |          (SocketException,                       → UI menampilkan banner "mode offline"
   |           ClientException,
   |           TimeoutException)
```

File: `lib/data/note_repository.dart`

`lib/data/note_repository.dart`

```dart
import 'dart:async';

import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';
import 'note_local_data_source.dart';
import 'note_remote_data_source.dart';

/// Hasil pengambilan daftar catatan.
class NotesResult {
  const NotesResult(this.notes, {required this.fromCache});

  final List<Note> notes;

  /// true jika server tidak bisa dihubungi dan data diambil dari Hive.
  final bool fromCache;
}

/// Satu pintu untuk semua data catatan. Provider tidak perlu tahu
/// apakah data berasal dari Supabase atau dari cache Hive.
class NoteRepository {
  NoteRepository({
    required this.remote,
    required this.local,
    this.timeout = const Duration(seconds: 10),
  });

  final NoteRemoteDataSource remote;
```

```dart
  final NoteLocalDataSource local;
  final Duration timeout;

  /// Ambil dari server lalu simpan ke cache. Jika gagal karena jaringan,
  /// kembalikan isi cache (mode offline).
  Future<NotesResult> getNotes() async {
    try {
      final notes = await remote.fetchNotes().timeout(timeout);
      await local.saveNotes(notes);
      return NotesResult(notes, fromCache: false);
    } on PostgrestException {
      // Server menjawab tapi menolak (misalnya RLS / tabel tidak ada).
      // Ini bukan masalah jaringan, jadi teruskan errornya.
      rethrow;
    } catch (_) {
      // SocketException, ClientException, TimeoutException, dll.
      return NotesResult(local.getNotes(), fromCache: true);
    }
  }

  Future<Note> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final note = await remote
        .createNote(title: title, content: content, category: category)
        .timeout(timeout);
    await local.saveNote(note);
    return note;
  }

  Future<Note> updateNote(Note note) async {
    final updated = await remote.updateNote(note).timeout(timeout);
    await local.saveNote(updated);
    return updated;
  }

  Future<void> deleteNote(String id) async {
    await remote.deleteNote(id).timeout(timeout);
    await local.deleteNote(id);
  }

  Future<void> clearCache() => local.clear();
}
```

Membedah NoteRepository

- `NotesResult` membungkus dua informasi: daftar catatan *dan* asal datanya. Tanpa flag `fromCache`, UI tidak bisa menampilkan banner offline.
- `.timeout(timeout)`: tanpa batas waktu, request di jaringan yang sangat lambat bisa menggantung lama. Setelah 10 detik muncul `TimeoutException` dan kita beralih ke cache.
- `on PostgrestException { rethrow; }` ditulis *sebelum* `catch (_)`. Error dari database bukan masalah koneksi, sehingga tidak boleh disembunyikan dengan data cache. Kalau disembunyikan, Anda tidak akan sadar policy RLS salah.

- **Operasi tulis (`addNote`, `updateNote`, `deleteNote`)** wajib online. Urutannya: server dulu, cache kemudian. Jika server gagal, exception diteruskan dan cache tidak berubah, sehingga cache tidak pernah berisi data yang tidak ada di server.
- **Field `remote` dan `local` bertipe class konkret**, tetapi tetap bisa diganti di test karena Dart mengizinkan `class Palsu implements NoteRemoteDataSource`.

> ℹ️ **Kenapa tidak memakai package connectivity?**
>
> Status "Wi-Fi tersambung" tidak menjamin internet benar-benar bisa dipakai (misalnya Wi-Fi kampus yang belum login portal). Cara paling jujur adalah langsung mencoba request: jika gagal karena jaringan, kita tahu sedang offline.

> 💡 **Batasan desain ini**
>
> Mode offline di proyek ini bersifat **baca saja**. Menambah atau mengubah catatan saat offline akan gagal dengan pesan yang jelas. Antrean penulisan offline (*offline queue*) dibahas sebagai ide pengembangan di section terakhir.

6. NotesProvider & Optimistic Delete

`NotesProvider` memegang semua state halaman catatan: daftar catatan, status loading, mode offline, pesan error, kata kunci pencarian, dan filter kategori.

File: `lib/providers/notes_provider.dart`

`lib/providers/notes_provider.dart`

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

class NotesProvider extends ChangeNotifier {
  NotesProvider(this._repository);

  final NoteRepository _repository;

  List<Note> _notes = [];
  bool _isLoading = false;
  bool _isOffline = false;
  String? _errorMessage;
  String _query = '';
  String? _categoryFilter; // null = semua kategori

  bool get isLoading => _isLoading;
  bool get isOffline => _isOffline;
  String? get errorMessage => _errorMessage;
  String get query => _query;
  String? get categoryFilter => _categoryFilter;
  bool get hasNotes => _notes.isNotEmpty;

  /// Daftar yang ditampilkan di layar setelah difilter pencarian & kategori.
  List<Note> get visibleNotes {
    final q = _query.toLowerCase();
    return _notes.where((note) {
      final cocokKategori =
          _categoryFilter == null || note.category == _categoryFilter;
      final cocokCari =
          q.isEmpty ||
          note.title.toLowerCase().contains(q) ||
          note.content.toLowerCase().contains(q);
      return cocokKategori && cocokCari;
    }).toList();
  }

  Note? findById(String id) {
    for (final note in _notes) {
      if (note.id == id) return note;
    }
    return null;
  }
```

```dart
  /// Memuat catatan (dipanggil saat HomeScreen dibuka & saat pull-to-refresh).
  Future<void> loadNotes() async {
    _isLoading = true;
    _errorMessage = null;
    notifyListeners();

    try {
      final result = await _repository.getNotes();
      _notes = result.notes;
      _isOffline = result.fromCache;
    } catch (e) {
      _errorMessage = _friendlyError(e);
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  /// Mengembalikan null jika berhasil, atau pesan error jika gagal.
  Future<String?> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    try {
      final note = await _repository.addNote(
        title: title,
        content: content,
        category: category,
      );
      _notes.insert(0, note);
      _isOffline = false;
      notifyListeners();
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  Future<String?> updateNote(Note note) async {
    try {
      final updated = await _repository.updateNote(note);
      _notes.removeWhere((n) => n.id == updated.id);
      _notes.insert(0, updated); // yang baru diubah naik ke atas
      _isOffline = false;
      notifyListeners();
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  /// Optimistic delete: hapus dari layar dulu, baru kirim ke server.
  /// Jika server gagal, catatan dikembalikan ke posisi semula (rollback).
  Future<String?> deleteNote(Note note) async {
    final index = _notes.indexWhere((n) => n.id == note.id);
    if (index == -1) return null;
```

```
    _notes.removeAt(index);
    notifyListeners();

    try {
      await _repository.deleteNote(note.id);
      return null;
    } catch (e) {
      _notes.insert(index, note);
      notifyListeners();
      return _friendlyError(e);
    }
  }

  void setQuery(String value) {
    _query = value;
    notifyListeners();
  }

  void setCategoryFilter(String? category) {
    _categoryFilter = category;
    notifyListeners();
  }

  /// Dipanggil saat logout: kosongkan state dan cache Hive.
  Future<void> clear() async {
    _notes = [];
    _query = '';
    _categoryFilter = null;
    _errorMessage = null;
    _isOffline = false;
    await _repository.clearCache();
    notifyListeners();
  }

  String _friendlyError(Object e) {
    if (e is PostgrestException) {
      if (e.code == '42501') {
        return 'Akses ditolak oleh Row Level Security. Periksa policy tabel notes.';
      }
      if (e.code == '42P01' || e.code == 'PGRST205') {
        return 'Tabel notes belum dibuat. Jalankan SQL di Supabase.';
      }
      return 'Kesalahan database: ${e.message}';
    }
    if (e is TimeoutException) {
      return 'Server terlalu lama merespons. Coba lagi.';
    }
    return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
  }
}
```

State yang dikelola

Field	Kegunaan di UI
`_notes`	Semua catatan milik user (sumber data)
`visibleNotes`	Getter: `_notes` setelah difilter pencarian + kategori. Inilah yang ditampilkan ListView
`isLoading`	Spinner saat pertama kali memuat
`isOffline`	Menampilkan `OfflineBanner`
`errorMessage`	Tampilan error + tombol "Coba lagi"
`query`, `categoryFilter`	Isi kolom pencarian dan chip yang terpilih

Pencarian dan filter tanpa query ulang

`visibleNotes` dihitung dari data di memori. Mengetik di kolom pencarian *tidak* mengirim request ke server, sehingga hasilnya instan dan tetap berfungsi saat offline. Untuk ratusan catatan cara ini lebih dari cukup. Untuk data sangat besar, gunakan `.ilike()` di server (Modul 12).

Optimistic delete dengan rollback

```
user geser catatan ──► konfirmasi "Hapus"
                          |
                          ▼
   1. simpan index, hapus dari _notes, notifyListeners()   ← UI langsung hilang (terasa cepat)
                          |
   2. await repository.deleteNote(id)
            |                            |
         sukses                       gagal (offline / RLS)
            |                            |
         selesai            3. _notes.insert(index, note)   ← kembalikan ke posisi semula
                               notifyListeners()            ← catatan muncul lagi
                               return pesan error           ← SnackBar
```

Disebut *optimistic* karena kita "optimis" server akan berhasil, sehingga UI diperbarui duluan. Keuntungannya, aplikasi terasa instan. Konsekuensinya, kita **wajib** menyediakan rollback jika ternyata gagal.

Untuk tambah dan edit kita memakai cara sebaliknya (*pessimistic*): tunggu server, baru perbarui list. Alasannya, id dan tanggal catatan baru berasal dari database.

Kontrak nilai kembali: `Future<String?>`

`addNote`, `updateNote`, dan `deleteNote` mengembalikan `null` jika sukses atau `String` pesan error jika gagal. UI cukup menulis:

```dart
final error = await provider.addNote(
  title: 'Basis Data',
  content: 'Normalisasi sampai 3NF',
  category: 'Kuliah',
);
if (error == null) {
  // sukses
} else {
  // tampilkan error di SnackBar
}
```

Menerjemahkan error

`_friendlyError` mengubah exception menjadi kalimat untuk user: `42501` (RLS menolak), `42P01` / `PGRST205` (tabel belum dibuat), `TimeoutException`, dan selain itu dianggap masalah koneksi.

> ⚠️ **Kesalahan umum**
>
> - **Mengembalikan `_notes` langsung lewat getter**: UI bisa mengubah list tanpa `notifyListeners`. Di sini UI hanya menerima `visibleNotes`, yaitu list baru hasil `where().toList()`.
> - **Lupa `notifyListeners()` setelah rollback**: catatan "hilang" di layar padahal masih ada di server.
> - **Memanggil `loadNotes()` langsung di `initState`**: `notifyListeners` terpanggil saat build pertama dan memicu error *setState() or markNeedsBuild() called during build*. Solusinya ada di HomeScreen (section 7).

7. HomeScreen: Daftar, Cari, Filter, Hapus

Sekarang kita ganti HomeScreen sementara dari Modul 13 dengan halaman daftar catatan lengkap. Sebelum itu, kita siapkan empat file kecil yang dipakai berulang.

File: `lib/utils/date_format.dart`

Format tanggal sederhana tanpa package tambahan. Jika ingin format yang lebih lengkap, Anda bisa memakai package `intl`.

`lib/utils/date_format.dart`

```dart
/// Format tanggal sederhana tanpa package tambahan, contoh: 5 Okt 2026 14:07.
String formatTanggal(DateTime waktu) {
  const bulan = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'Mei',
    'Jun',
    'Jul',
    'Agu',
    'Sep',
    'Okt',
    'Nov',
    'Des',
  ];
  final lokal = waktu.toLocal();
  final jam = lokal.hour.toString().padLeft(2, '0');
  final menit = lokal.minute.toString().padLeft(2, '0');
  return '${lokal.day} ${bulan[lokal.month - 1]} ${lokal.year} $jam:$menit';
}
```

`toLocal()` penting: Supabase menyimpan waktu dalam UTC, sedangkan pengguna di Purwokerto berada di WIB (UTC+7).

File: `lib/widgets/confirm_dialog.dart`

Dialog "Ya/Batal" dipakai untuk logout dan hapus catatan (dua tempat). Daripada menulis `AlertDialog` berkali-kali, kita buat fungsi bersama:

`lib/widgets/confirm_dialog.dart`

```dart
import 'package:flutter/material.dart';

/// Dialog konfirmasi Ya/Batal yang dipakai ulang di beberapa layar.
/// Mengembalikan true jika pengguna menekan tombol konfirmasi.
Future<bool> showConfirmDialog(
  BuildContext context, {
  required String title,
  required String message,
  required String confirmLabel,
}) async {
```

```
  final hasil = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(title),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(dialogContext).pop(false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.of(dialogContext).pop(true),
          child: Text(confirmLabel),
        ),
      ],
    ),
  );
  return hasil ?? false;
}
```

`showDialog<bool>` mengembalikan nilai yang dikirim lewat `Navigator.pop(true/false)`, atau `null` jika user menutup dialog dengan mengetuk area luar. Karena itu ada `?? false`.

File: `lib/widgets/note_card.dart`

`lib/widgets/note_card.dart`

```
import 'package:flutter/material.dart';

import '../models/note.dart';
import '../utils/date_format.dart';

class NoteCard extends StatelessWidget {
  const NoteCard({super.key, required this.note, this.onTap});

  final Note note;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      child: ListTile(
        onTap: onTap,
        contentPadding: const EdgeInsets.fromLTRB(16, 8, 16, 8),
        title: Text(
          note.title,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontWeight: FontWeight.w600),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 4),
```

```
            Text(note.content, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 8),
            Row(
              children: [
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 8,
                    vertical: 2,
                  ),
                  decoration: BoxDecoration(
                    color: scheme.primaryContainer,
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Text(
                    note.category,
                    style: TextStyle(
                      fontSize: 12,
                      color: scheme.onPrimaryContainer,
                    ),
                  ),
                ),
                const Spacer(),
                Text(
                  formatTanggal(note.updatedAt),
                  style: Theme.of(context).textTheme.bodySmall,
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/widgets/offline_banner.dart`

`lib/widgets/offline_banner.dart`

```
import 'package:flutter/material.dart';

/// Pita peringatan yang muncul ketika data diambil dari cache Hive.
class OfflineBanner extends StatelessWidget {
  const OfflineBanner({super.key, required this.onRetry});

  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Material(
      color: scheme.tertiaryContainer,
      child: Padding(
        padding: const EdgeInsets.fromLTRB(16, 4, 8, 4),
        child: Row(
          children: [
            Icon(Icons.cloud_off, color: scheme.onTertiaryContainer),
```

```
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                'Mode offline: menampilkan data tersimpan.',
                style: TextStyle(color: scheme.onTertiaryContainer),
              ),
            ),
            TextButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/screens/home_screen.dart` (versi final)

Hapus seluruh isi HomeScreen sementara dari Modul 13 dan ganti dengan kode berikut:

`lib/screens/home_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/auth_provider.dart';
import '../providers/notes_provider.dart';
import '../widgets/confirm_dialog.dart';
import '../widgets/note_card.dart';
import '../widgets/offline_banner.dart';
import 'note_detail_screen.dart';
import 'note_form_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
    // Tunggu frame pertama selesai, baru muat data (notifyListeners tidak
    // boleh dipanggil ketika widget tree sedang dibangun).
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) context.read<NotesProvider>().loadNotes();
    });
  }

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }
```

```dart
Future<void> _logout() async {
  final yakin = await showConfirmDialog(
    context,
    title: 'Keluar?',
    message: 'Catatan yang tersimpan di perangkat akan dihapus.',
    confirmLabel: 'Keluar',
  );
  if (!yakin || !mounted) return;

  // Ambil referensi provider sebelum await berikutnya.
  final notes = context.read<NotesProvider>();
  final auth = context.read<AuthProvider>();
  await notes.clear(); // hapus cache Hive
  await auth.signOut(); // AuthGate otomatis kembali ke LoginScreen
}

Future<bool> _confirmDelete(Note note) {
  return showConfirmDialog(
    context,
    title: 'Hapus catatan?',
    message: '"${note.title}" akan dihapus permanen.',
    confirmLabel: 'Hapus',
  );
}

Future<void> _delete(Note note) async {
  final messenger = ScaffoldMessenger.of(context);
  final error = await context.read<NotesProvider>().deleteNote(note);
  messenger.showSnackBar(
    SnackBar(
      content: Text(
        error == null
            ? 'Catatan dihapus'
            : 'Gagal menghapus, catatan dikembalikan. $error',
      ),
    ),
  );
}

void _openForm() {
  Navigator.of(context)
      .push(MaterialPageRoute(builder: (_) => const NoteFormScreen()));
}

void _openDetail(Note note) {
  Navigator.of(context).push(
    MaterialPageRoute(builder: (_) => NoteDetailScreen(noteId: note.id)),
  );
}

@override
Widget build(BuildContext context) {
  final provider = context.watch<NotesProvider>();
  final name = context.read<AuthProvider>().displayName;

  return Scaffold(
    appBar: AppBar(
      title: Column(
```

```
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Catatan Kuliah'),
            Text('Halo, $name', style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: _logout,
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _openForm,
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
      body: Column(
        children: [
          if (provider.isOffline) OfflineBanner(onRetry: provider.loadNotes),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
            child: TextField(
              controller: _searchController,
              onChanged: provider.setQuery,
              decoration: InputDecoration(
                hintText: 'Cari judul atau isi catatan...',
                prefixIcon: const Icon(Icons.search),
                border: const OutlineInputBorder(),
                isDense: true,
                suffixIcon: provider.query.isEmpty
                    ? null
                    : IconButton(
                        icon: const Icon(Icons.clear),
                        onPressed: () {
                          _searchController.clear();
                          provider.setQuery('');
                        },
                      ),
              ),
            ),
          ),
          _CategoryFilter(
            selected: provider.categoryFilter,
            onSelected: provider.setCategoryFilter,
          ),
          Expanded(
            child: RefreshIndicator(
              onRefresh: provider.loadNotes,
              child: _buildBody(provider),
            ),
          ),
        ],
      ),
    );
  }
```

```
  Widget _buildBody(NotesProvider provider) {
    // 1. Loading pertama kali (belum ada data sama sekali)
    if (provider.isLoading && !provider.hasNotes) {
      return const Center(child: CircularProgressIndicator());
    }

    // 2. Error dan tidak ada data cadangan
    if (provider.errorMessage != null && !provider.hasNotes) {
      return _MessageView(
        icon: Icons.error_outline,
        message: provider.errorMessage!,
        action: FilledButton.icon(
          onPressed: provider.loadNotes,
          icon: const Icon(Icons.refresh),
          label: const Text('Coba lagi'),
        ),
      );
    }

    final notes = provider.visibleNotes;

    // 3. Kosong
    if (notes.isEmpty) {
      return _MessageView(
        icon: Icons.note_alt_outlined,
        message: provider.hasNotes
            ? 'Tidak ada catatan yang cocok.'
            : 'Belum ada catatan.\nTekan tombol + untuk menambah.',
      );
    }

    // 4. Ada data
    return ListView.builder(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.only(top: 4, bottom: 88),
      itemCount: notes.length,
      itemBuilder: (context, index) {
        final note = notes[index];
        return Dismissible(
          key: ValueKey(note.id),
          direction: DismissDirection.endToStart,
          background: Container(
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 24),
            color: Theme.of(context).colorScheme.errorContainer,
            child: Icon(
              Icons.delete,
              color: Theme.of(context).colorScheme.onErrorContainer,
            ),
          ),
          confirmDismiss: (_) => _confirmDelete(note),
          onDismissed: (_) => _delete(note),
          child: NoteCard(note: note, onTap: () => _openDetail(note)),
        );
      },
    );
  }
}
```

```dart
/// Deretan chip untuk memfilter kategori.
class _CategoryFilter extends StatelessWidget {
  const _CategoryFilter({required this.selected, required this.onSelected});

  final String? selected;
  final ValueChanged<String?> onSelected;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 48,
      child: ListView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        children: [
          Padding(
            padding: const EdgeInsets.only(right: 8),
            child: ChoiceChip(
              label: const Text('Semua'),
              selected: selected == null,
              onSelected: (_) => onSelected(null),
            ),
          ),
          for (final category in Note.categories)
            Padding(
              padding: const EdgeInsets.only(right: 8),
              child: ChoiceChip(
                label: Text(category),
                selected: selected == category,
                onSelected: (_) => onSelected(category),
              ),
            ),
        ],
      ),
    );
  }
}

/// Tampilan pesan di tengah layar yang tetap bisa ditarik (pull-to-refresh).
class _MessageView extends StatelessWidget {
  const _MessageView({required this.icon, required this.message, this.action});

  final IconData icon;
  final String message;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    return ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.all(32),
      children: [
        const SizedBox(height: 48),
        Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
        const SizedBox(height: 16),
        Text(message, textAlign: TextAlign.center),
        if (action != null) ...[
          const SizedBox(height: 16),
          Center(child: action),
```

```
        ],
      ],
    );
  }
}
```

Membedah HomeScreen

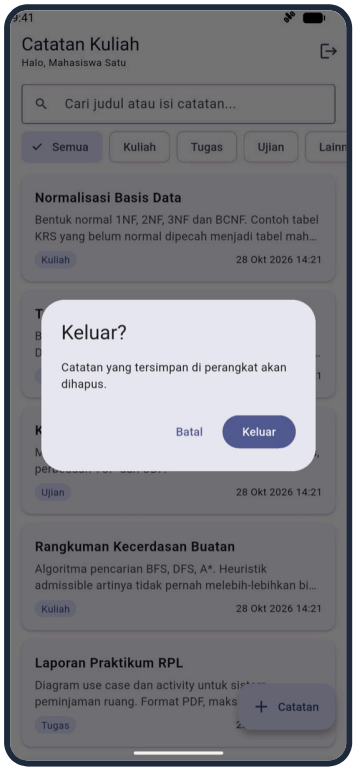
Bagian	Penjelasan
`addPostFrameCallback` di `initState`	Menunda `loadNotes()` sampai frame pertama selesai digambar, karena `loadNotes` langsung memanggil `notifyListeners`.
`_logout()`	Konfirmasi, lalu `notes.clear()` (kosongkan state + Hive), lalu `auth.signOut()`. Provider diambil *sebelum* `await` pertama yang memakai context.
`OfflineBanner(onRetry: provider.loadNotes)`	Banner hanya muncul jika data berasal dari cache. Tombol "Coba lagi" memanggil ulang server.
`TextField(onChanged: provider.setQuery)`	Setiap ketikan memperbarui filter. Ikon ✕ mengosongkan pencarian.
`_CategoryFilter`	Deretan `ChoiceChip`. Memilih "Semua" berarti `null`.
`RefreshIndicator(onRefresh: provider.loadNotes)`	Tarik ke bawah untuk memuat ulang. Child-nya **harus** bisa di-scroll, karena itu tampilan kosong/error juga berupa `ListView` dengan `AlwaysScrollableScrollPhysics`.
`_buildBody` dengan 4 kondisi	Urutan pengecekan: (1) loading pertama, (2) error tanpa data, (3) kosong, (4) daftar. Jika ada data cache, error tidak menutupi data.
`Dismissible`	`key: ValueKey(note.id)` harus unik dan stabil. `confirmDismiss` menampilkan dialog; jika hasilnya `false`, kartu kembali ke tempatnya. `onDismissed` memanggil optimistic delete.
`ScaffoldMessenger.of(context)` sebelum `await`	Di `_delete`, messenger diambil sebelum menunggu server, sehingga aman dipakai setelahnya tanpa melanggar aturan `context` setelah `await`.
`padding: bottom: 88`	Memberi ruang agar kartu terakhir tidak tertutup FAB.

Hasil di emulator, terhubung ke project Supabase dengan akun demo:

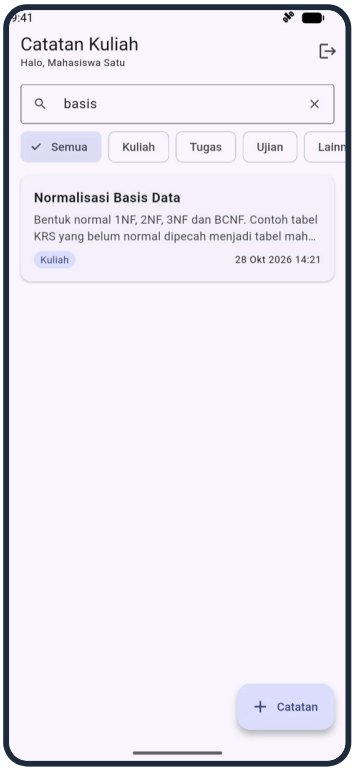
Akun baru: belum ada catatan (empty state)

Catatan dari tabel notes, urut terakhir diubah

Chip "Tugas" dipilih



Konfirmasi sebelum logout



Kata kunci "basis" menyaring daftar catatan

> ⚠️ **Error "A dismissed Dismissible widget is still part of the tree"**
>
> Error ini muncul jika item yang sudah digeser tidak segera dihapus dari list. Optimistic delete di `NotesProvider` menghapusnya secara sinkron sebelum `await`, sehingga error ini tidak terjadi.

8. Form Tambah/Edit & Detail Catatan

Satu layar form untuk dua keperluan, tambah dan edit, adalah pola yang sangat umum. Perbedaannya hanya ditentukan oleh ada atau tidaknya parameter `note`.

File: `lib/screens/note_form_screen.dart`

`lib/screens/note_form_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/notes_provider.dart';

/// Satu layar untuk dua keperluan:
/// - note == null  -> mode TAMBAH
/// - note != null  -> mode EDIT
class NoteFormScreen extends StatefulWidget {
  const NoteFormScreen({super.key, this.note});

  final Note? note;

  @override
  State<NoteFormScreen> createState() => _NoteFormScreenState();
}

class _NoteFormScreenState extends State<NoteFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleController;
  late final TextEditingController _contentController;
  late String _category;
  bool _isSaving = false;

  bool get _isEdit => widget.note != null;

  @override
  void initState() {
    super.initState();
    // Mode edit: isi form dengan data lama. Mode tambah: form kosong.
    _titleController = TextEditingController(text: widget.note?.title ?? '');
    _contentController = TextEditingController(
      text: widget.note?.content ?? '',
    );
    _category = widget.note?.category ?? Note.categories.first;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    FocusScope.of(context).unfocus();
```

```
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
    final provider = context.read<NotesProvider>();
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();

    final String? error;
    if (_isEdit) {
      error = await provider.updateNote(
        widget.note!.copyWith(
          title: title,
          content: content,
          category: _category,
        ),
      );
    } else {
      error = await provider.addNote(
        title: title,
        content: content,
        category: _category,
      );
    }

    if (!mounted) return;
    setState(() => _isSaving = false);

    final messenger = ScaffoldMessenger.of(context);
    if (error == null) {
      messenger.showSnackBar(
        SnackBar(
          content: Text(_isEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
        ),
      );
      Navigator.of(context).pop();
    } else {
      messenger.showSnackBar(
        SnackBar(
          content: Text(error),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Catatan' : 'Catatan Baru')),
      body: SafeArea(
        child: Form(
          key: _formKey,
          child: ListView(
            padding: const EdgeInsets.all(16),
            children: [
              TextFormField(
                controller: _titleController,
                textInputAction: TextInputAction.next,
                maxLength: 100,
```

```
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Judul wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 8),
          DropdownButtonFormField<String>(
            initialValue: _category,
            decoration: const InputDecoration(
              labelText: 'Kategori',
              border: OutlineInputBorder(),
            ),
            items: [
              for (final category in Note.categories)
                DropdownMenuItem(value: category, child: Text(category)),
            ],
            onChanged: (value) {
              if (value != null) setState(() => _category = value);
            },
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _contentController,
            minLines: 6,
            maxLines: 12,
            keyboardType: TextInputType.multiline,
            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              alignLabelWithHint: true,
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Isi catatan wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 24),
          FilledButton.icon(
            onPressed: _isSaving ? null : _save,
            style: FilledButton.styleFrom(
              minimumSize: const Size.fromHeight(48),
            ),
            icon: _isSaving
                ? const SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2.5),
                  )
                : const Icon(Icons.save),
            label: Text(_isSaving ? 'Menyimpan...' : 'Simpan'),
          ),
```

```
            ],
          ),
        ),
      ),
    );
  }
}
```

Membedah NoteFormScreen

- `bool get _isEdit => widget.note != null` adalah satu sumber kebenaran untuk judul AppBar, pesan sukses, dan method provider yang dipanggil.
- **Controller diisi di** `initState` dengan `widget.note?.title ?? ''`. Mode edit langsung menampilkan data lama, sedangkan mode tambah kosong.
- `DropdownButtonFormField` **dengan** `initialValue` memilih kategori awal. Itemnya diambil dari `Note.categories`.
- `maxLength: 100` menampilkan penghitung karakter dan sejalan dengan `check` di SQL.
- `_isSaving` adalah state lokal (bukan di provider) karena hanya relevan untuk layar ini: menonaktifkan tombol dan menampilkan spinner.
- `final String? error;` **diisi di if/else**: Dart mengizinkan variabel `final` diisi tepat sekali di setiap cabang.
- **Setelah** `await` **:** `if (!mounted) return;`, baru `setState`, SnackBar, dan `pop`.
- **Edit memakai** `copyWith`, sehingga id catatan lama tetap dipakai untuk filter `.eq('id', ...)`.

File: `lib/screens/note_detail_screen.dart`

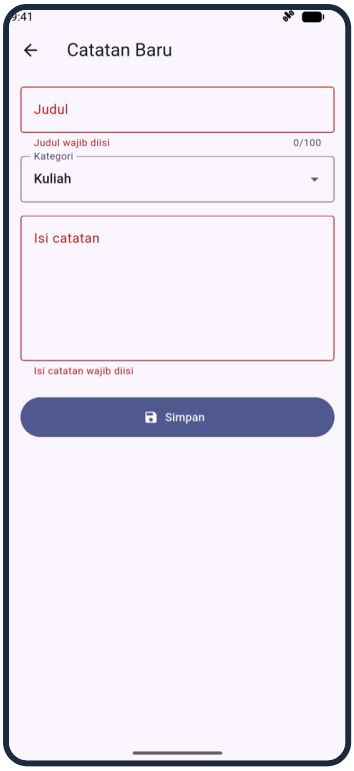
`lib/screens/note_detail_screen.dart`

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/notes_provider.dart';
import '../utils/date_format.dart';
import '../widgets/confirm_dialog.dart';
import 'note_form_screen.dart';

class NoteDetailScreen extends StatelessWidget {
  const NoteDetailScreen({super.key, required this.noteId});

  final String noteId;

  Future<void> _delete(BuildContext context, Note note) async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
    if (!yakin || !context.mounted) return;
```

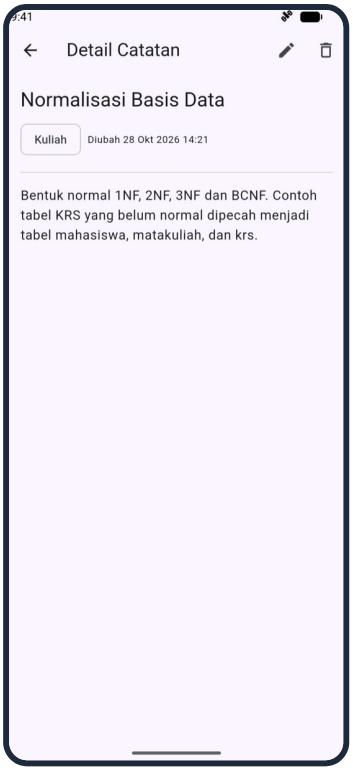
```
    final messenger = ScaffoldMessenger.of(context);
    final provider = context.read<NotesProvider>();
    // Tutup halaman detail dulu, baru hapus (optimistic delete).
    Navigator.of(context).pop();
    final error = await provider.deleteNote(note);
    messenger.showSnackBar(SnackBar(content: Text(error ?? 'Catatan dihapus')));
  }

  @override
  Widget build(BuildContext context) {
    // Ambil versi terbaru catatan dari provider, sehingga setelah diedit
    // halaman ini langsung menampilkan perubahan.
    final note = context.select<NotesProvider, Note?>(
      (provider) => provider.findById(noteId),
    );

    if (note == null) {
      return Scaffold(
        appBar: AppBar(),
        body: const Center(child: Text('Catatan tidak ditemukan.')),
      );
    }

    return Scaffold(
      appBar: AppBar(
        title: const Text('Detail Catatan'),
        actions: [
          IconButton(
            tooltip: 'Edit',
            icon: const Icon(Icons.edit),
            onPressed: () {
              Navigator.of(context).push(
                MaterialPageRoute(builder: (_) => NoteFormScreen(note: note)),
              );
            },
          ),
          IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: () => _delete(context, note),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(note.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              Chip(label: Text(note.category)),
              Text(
                'Diubah ${formatTanggal(note.updatedAt)}',
                style: Theme.of(context).textTheme.bodySmall,
              ),
            ],
          ),
```

```
            const Divider(height: 32),
            SelectableText(
              note.content,
              style: Theme.of(context).textTheme.bodyLarge,
            ),
          ],
        ),
      );
  }
}
```

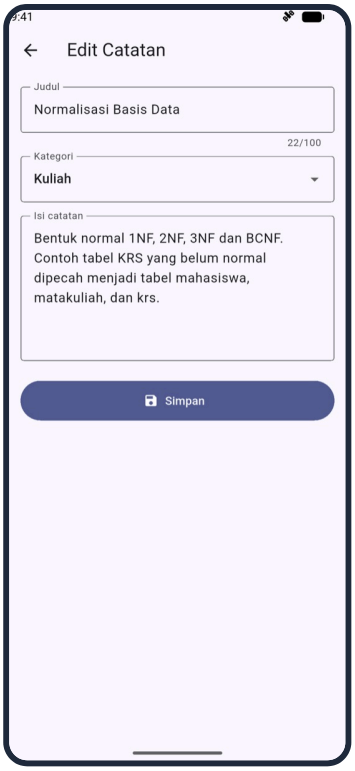
- **Menerima `noteId`, bukan objek `Note`**. Dengan `context.select`, halaman detail selalu menampilkan versi *terbaru* dari provider. Setelah diedit dan form ditutup, detail langsung berubah tanpa refresh.
- `context.select<NotesProvider, Note?>` hanya me-rebuild halaman ini jika catatan dengan id tersebut berubah, bukan setiap kali provider berubah (misalnya saat mengetik pencarian).
- **Hapus dari detail**: halaman ditutup dulu (`pop`), baru optimistic delete berjalan, lalu SnackBar muncul di HomeScreen. Messenger dan provider diambil sebelum `pop`.
- `SelectableText` membuat isi catatan bisa disalin, berguna untuk catatan kuliah.



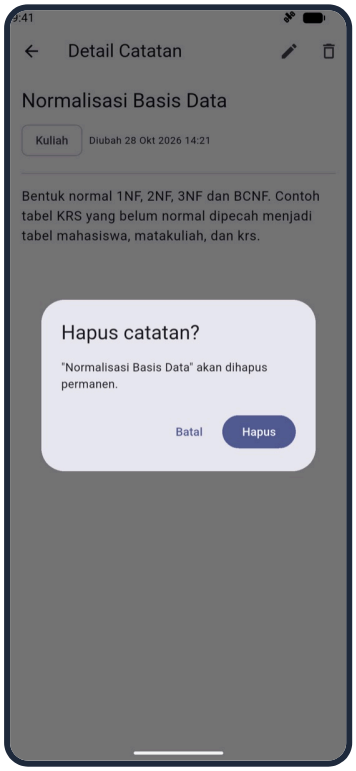
Mode tambah: tombol Simpan ditekan saat form kosong



NoteDetailScreen dengan tombol Edit dan Hapus



Mode edit: form terisi data lama



Dialog konfirmasi sebelum menghapus

Mode tambah: judul, kategori, dan isi siap disimpan

9. main.dart Final & Logout yang Aman

Semua potongan sudah siap. Langkah terakhir adalah merakitnya di `main.dart`: inisialisasi Hive dan Supabase, membuat objek dari lapisan bawah ke atas, lalu mendaftarkan dua provider.

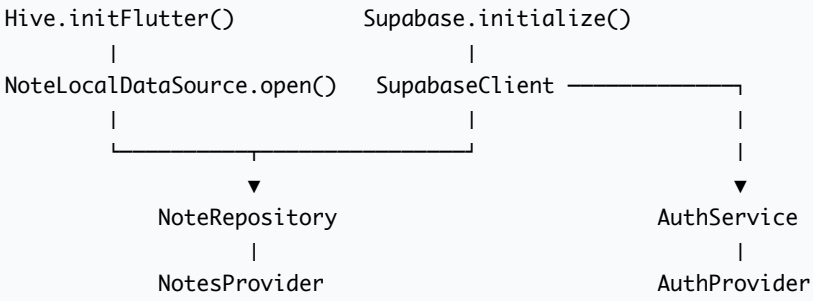
File: `lib/main.dart` (versi final)

`lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:provider/provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'data/note_local_data_source.dart';
import 'data/note_remote_data_source.dart';
import 'data/note_repository.dart';
import 'providers/auth_provider.dart';
import 'providers/notes_provider.dart';
import 'screens/auth_gate.dart';
import 'services/auth_service.dart';

Future<void> main() async {
  // Wajib sebelum memanggil kode async/plugin sebelum runApp.
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }

  // 1. Hive (cache lokal)
  await Hive.initFlutter();
  final localDataSource = await NoteLocalDataSource.open();

  // 2. Supabase (backend)
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );
  final client = Supabase.instance.client;

  // 3. Rakit lapisan data: Service/DataSource -> Repository
  final authService = AuthService(client);
  final repository = NoteRepository(
    remote: NoteRemoteDataSource(client),
    local: localDataSource,
  );

  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider(authService)),
        ChangeNotifierProvider(create: (_) => NotesProvider(repository)),
```

```
      ],
      child: const CatatanKuliahApp(),
    ),
  );
}

class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

Urutan perakitan (dependency graph)



```
                    └──────────────── MultiProvider ────────┘
                                          |
                                  CatatanKuliahApp → AuthGate
```

- **Objek dibuat sekali di `main()`** lalu disuntikkan lewat constructor. Tidak ada class yang memanggil `Supabase.instance` atau `Hive.openBox` sendiri.
- **`MultiProvider`** (Modul 08) mendaftarkan beberapa provider sekaligus tanpa nesting yang dalam.
- Dibanding versi Modul 13, yang **bertambah** hanya bagian Hive, repository, dan `NotesProvider`. Bagian auth tidak berubah sama sekali, dan itulah manfaat arsitektur berlapis.

Alur logout yang aman

```
ikon logout ─► dialog "Keluar?" ─► NotesProvider.clear()
                                       ├─ _notes = [], filter direset
                                       └─ repository.clearCache() → Hive box dikosongkan
                                   ─► AuthProvider.signOut()
                                       └─ onAuthStateChange: signedOut
                                           └─ AuthGate → LoginScreen
```

Urutannya penting: cache dibersihkan **sebelum** sesi dihapus. Jika urutannya dibalik, AuthGate langsung membuang HomeScreen dan state provider masih berisi catatan user lama sampai user berikutnya login.

Jalankan aplikasi

```
flutter analyze
flutter run --dart-define-from-file=env.json
```

Uji mode offline

1. Login dan pastikan catatan tampil (data sudah tersimpan di cache).
2. Aktifkan mode pesawat di emulator.
3. Tarik ke bawah (pull-to-refresh). Setelah request gagal, banner **"Mode offline"** muncul dan catatan tetap tampil dari Hive.
4. Coba tambah catatan. Muncul SnackBar error koneksi dan list tidak berubah.
5. Coba hapus catatan (geser). Catatan hilang sebentar lalu muncul kembali (rollback) dengan SnackBar.
6. Matikan mode pesawat dan tekan **Coba lagi** di banner. Banner hilang.

Emulator dalam mode pesawat: catatan tampil dari cache Hive dengan banner offline

10. Unit Test Repository & Hive

Logika paling penting di proyek ini adalah *fallback* offline di `NoteRepository` dan cache Hive. Logika seperti ini sulit diuji secara manual (harus menyalakan/mematikan internet berulang kali), tetapi mudah diuji dengan unit test karena repository tidak bergantung pada UI.

Ide utama: ganti Supabase dengan "server palsu"

Kita membuat `FakeRemoteDataSource` yang `implements NoteRemoteDataSource`. Class ini menyimpan data di List biasa dan bisa diperintah untuk melempar error (misalnya `SocketException`) agar seolah-olah offline. Hive asli tetap dipakai, tetapi diarahkan ke folder sementara dengan `Hive.init(path)`.

File: `test/note_local_data_source_test.dart`

`test/note_local_data_source_test.dart`

```dart
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

Note buatNote(String id, {String title = 'Judul', int hari = 1}) {
  final waktu = DateTime.utc(2026, 10, hari);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: 'Kuliah',
    createdAt: waktu,
    updatedAt: waktu,
  );
}

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;

  setUp(() async {
    // Hive.init (bukan initFlutter) karena test berjalan tanpa plugin Flutter.
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('saveNotes lalu getNotes mengembalikan data urut terbaru', () async {
```

```
    await local.saveNotes([
      buatNote('a', title: 'Lama', hari: 1),
      buatNote('b', title: 'Baru', hari: 5),
    ]);

    final notes = local.getNotes();

    expect(notes.map((n) => n.id), ['b', 'a']);
    expect(notes.first.title, 'Baru');
  });

  test('saveNotes mengganti seluruh isi cache', () async {
    await local.saveNotes([buatNote('a'), buatNote('b')]);
    await local.saveNotes([buatNote('c')]);

    expect(local.getNotes().map((n) => n.id), ['c']);
  });

  test('saveNote, deleteNote, dan clear', () async {
    await local.saveNote(buatNote('a'));
    await local.saveNote(buatNote('a', title: 'Diubah'));
    expect(local.getNotes().single.title, 'Diubah');

    await local.saveNote(buatNote('b'));
    await local.deleteNote('a');
    expect(local.getNotes().map((n) => n.id), ['b']);

    await local.clear();
    expect(local.getNotes(), isEmpty);
  });
}
```

File: `test/note_repository_test.dart`

`test/note_repository_test.dart`

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_remote_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Note buatNote(String id, {String title = 'Judul'}) {
  final waktu = DateTime.utc(2026, 10, 1);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: 'Kuliah',
    createdAt: waktu,
    updatedAt: waktu,
  );
}
```

```dart
/// Pengganti Supabase untuk test. `implements` membuat kita wajib
/// menulis ulang semua method, sehingga tidak ada request jaringan sungguhan.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  List<Note> serverNotes = [];
  Object? errorToThrow; // jika diisi, setiap method melempar error ini

  void _maybeThrow() {
    final error = errorToThrow;
    if (error != null) throw error;
  }

  @override
  Future<List<Note>> fetchNotes() async {
    _maybeThrow();
    return serverNotes;
  }

  @override
  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    _maybeThrow();
    final note = buatNote('baru', title: title);
    serverNotes.add(note);
    return note;
  }

  @override
  Future<Note> updateNote(Note note) async {
    _maybeThrow();
    return note;
  }

  @override
  Future<void> deleteNote(String id) async {
    _maybeThrow();
    serverNotes.removeWhere((n) => n.id == id);
  }
}

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;
  late FakeRemoteDataSource remote;
  late NoteRepository repository;

  setUp(() async {
    tempDir = await Directory.systemTemp.createTemp('repo_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
    remote = FakeRemoteDataSource();
    repository = NoteRepository(remote: remote, local: local);
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
```

```
    await tempDir.delete(recursive: true);
  });

  test('online: data server dikembalikan dan disimpan ke cache', () async {
    remote.serverNotes = [buatNote('1'), buatNote('2')];

    final result = await repository.getNotes();

    expect(result.fromCache, isFalse);
    expect(result.notes, hasLength(2));
    expect(local.getNotes(), hasLength(2));
  });

  test('offline: jika jaringan gagal, data diambil dari cache', () async {
    await local.saveNotes([buatNote('cache-1', title: 'Dari Hive')]);
    remote.errorToThrow = const SocketException('Tidak ada internet');

    final result = await repository.getNotes();

    expect(result.fromCache, isTrue);
    expect(result.notes.single.title, 'Dari Hive');
  });

  test('error dari server (PostgrestException) tidak disembunyikan', () async {
    remote.errorToThrow = const PostgrestException(
      message: 'permission denied',
      code: '42501',
    );

    expect(repository.getNotes(), throwsA(isA<PostgrestException>()));
  });

  test('addNote dan deleteNote ikut memperbarui cache', () async {
    final note = await repository.addNote(
      title: 'Basis Data',
      content: 'Normalisasi',
      category: 'Kuliah',
    );
    expect(local.getNotes().single.id, note.id);

    await repository.deleteNote(note.id);
    expect(local.getNotes(), isEmpty);
  });

  test('deleteNote gagal: cache tidak berubah', () async {
    await local.saveNotes([buatNote('1')]);
    remote.errorToThrow = const SocketException('offline');

    await expectLater(
      repository.deleteNote('1'),
      throwsA(isA<SocketException>()),
    );
    expect(local.getNotes(), hasLength(1));
  });
}
```

Penjelasan

- `setUp` / `tearDown` dijalankan sebelum/sesudah *setiap* test: buat folder sementara, `Hive.init`, buka box, lalu hapus semuanya. Setiap test mulai dari kondisi bersih.
- `Hive.init(tempDir.path)` dipakai, bukan `Hive.initFlutter()`, karena di unit test tidak ada plugin `path_provider`.
- **Test "offline"** adalah inti modul ini: cache diisi, server dibuat gagal dengan `SocketException`, lalu kita pastikan `fromCache` bernilai `true` dan data cache yang dikembalikan.
- **Test PostgrestException** memastikan error RLS *tidak* disembunyikan oleh cache.
- `expectLater(future, throwsA(...))` menunggu Future selesai lalu memeriksa exception-nya.

Jalankan

```
flutter test
```

Hasil yang diharapkan:

```
00:01 +8: All tests passed!
```

> ℹ️ **Kenapa tidak ada widget test bawaan?**
>
> `flutter create` membuat `test/widget_test.dart` untuk aplikasi counter. File itu tidak relevan lagi dan akan gagal, jadi hapus saja. Widget test untuk layar Login/Home dibahas di modul Testing.

11. Listing Lengkap Seluruh File

Section ini berisi **seluruh** file aplikasi Catatan Kuliah dalam bentuk final dan lengkap, siap disalin. Gunakan untuk membandingkan jika aplikasi Anda error, atau untuk memulai ulang dari nol. Semua kode di bawah sudah diverifikasi dengan `flutter analyze` (tanpa masalah) dan `flutter test` (8 test lulus).

Langkah dari proyek kosong

```
flutter create catatan_kuliah
cd catatan_kuliah
flutter pub add supabase_flutter provider hive_ce hive_ce_flutter
rm test/widget_test.dart
mkdir -p lib/config lib/models lib/services lib/data lib/providers lib/screens lib/widgets lib/utils
supabase
```

Lalu buat setiap file di bawah ini, jalankan SQL di Supabase, isi `env.json`, dan jalankan `flutter run --dart-define-from-file=env.json`.

File non-Dart

`env.example.json`

```
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Salin menjadi `env.json` lalu isi dengan nilai project Anda. Tambahkan ke `.gitignore`:

`.gitignore`

```
# Kunci Supabase
env.json
```

Di `android/app/src/main/AndroidManifest.xml`, tepat di bawah tag pembuka `<manifest ...>`:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

SQL lengkap (jalankan sekali di SQL Editor; bagian `mata_kuliah` adalah tabel demo Modul 12 dan boleh dilewati):

`supabase/setup.sql`

```
-- ============================================================================
-- Setup Supabase untuk Modul 12 (demo) dan Modul 13-14 (Catatan Kuliah)
-- Jalankan seluruh isi file ini sekali di Dashboard > SQL Editor > Run.
-- Aman dijalankan ulang (idempotent).
-- ============================================================================
```

```sql
-- ---------------------------------------------------------------------
-- MODUL 12: tabel demo mata_kuliah
-- ---------------------------------------------------------------------
create table if not exists public.mata_kuliah (
  id bigint generated always as identity primary key,
  kode text not null unique,
  nama text not null,
  sks int not null default 3 check (sks between 1 and 6),
  semester int not null check (semester between 1 and 8),
  created_at timestamptz not null default now()
);

alter table public.mata_kuliah enable row level security;

drop policy if exists "Semua orang boleh membaca mata kuliah" on public.mata_kuliah;
create policy "Semua orang boleh membaca mata kuliah"
on public.mata_kuliah for select
to anon, authenticated
using (true);

drop policy if exists "User login boleh menambah mata kuliah" on public.mata_kuliah;
create policy "User login boleh menambah mata kuliah"
on public.mata_kuliah for insert
to authenticated
with check (true);

insert into public.mata_kuliah (kode, nama, sks, semester) values
  ('INF101', 'Algoritma dan Pemrograman', 4, 1),
  ('INF102', 'Matematika Diskrit', 3, 1),
  ('INF201', 'Struktur Data', 3, 2),
  ('INF202', 'Basis Data', 3, 3),
  ('INF301', 'Pemrograman Mobile', 3, 5),
  ('INF302', 'Jaringan Komputer', 3, 4),
  ('INF303', 'Kecerdasan Buatan', 3, 5),
  ('INF304', 'Rekayasa Perangkat Lunak', 3, 4)
on conflict (kode) do nothing;

-- Aktifkan Realtime untuk tabel mata_kuliah (dipakai contoh .stream()).
do $$
begin
  if not exists (
    select 1 from pg_publication_tables
    where pubname = 'supabase_realtime'
      and schemaname = 'public' and tablename = 'mata_kuliah'
  ) then
    alter publication supabase_realtime add table public.mata_kuliah;
  end if;
end;
$$;

-- ---------------------------------------------------------------------
-- MODUL 13: tabel profiles + trigger handle_new_user
-- ---------------------------------------------------------------------
create table if not exists public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  full_name text,
  created_at timestamptz not null default now()
);
```

```sql
alter table public.profiles enable row level security;

drop policy if exists "User melihat profilnya sendiri" on public.profiles;
create policy "User melihat profilnya sendiri"
on public.profiles for select
to authenticated
using ((select auth.uid()) = id);

drop policy if exists "User mengubah profilnya sendiri" on public.profiles;
create policy "User mengubah profilnya sendiri"
on public.profiles for update
to authenticated
using ((select auth.uid()) = id)
with check ((select auth.uid()) = id);

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, new.raw_user_meta_data ->> 'full_name');
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

-- ------------------------------------------------------------------
-- MODUL 14: tabel notes + RLS + trigger updated_at
-- ------------------------------------------------------------------
create table if not exists public.notes (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null check (char_length(title) between 1 and 100),
  content text not null default '',
  category text not null default 'Kuliah'
    check (category in ('Kuliah', 'Tugas', 'Ujian', 'Lainnya')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists notes_user_id_idx on public.notes (user_id);

alter table public.notes enable row level security;

drop policy if exists "User melihat catatannya sendiri" on public.notes;
create policy "User melihat catatannya sendiri"
on public.notes for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "User menambah catatannya sendiri" on public.notes;
```

```
create policy "User menambah catatannya sendiri"
on public.notes for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "User mengubah catatannya sendiri" on public.notes;
create policy "User mengubah catatannya sendiri"
on public.notes for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "User menghapus catatannya sendiri" on public.notes;
create policy "User menghapus catatannya sendiri"
on public.notes for delete
to authenticated
using ((select auth.uid()) = user_id);

create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

drop trigger if exists notes_set_updated_at on public.notes;
create trigger notes_set_updated_at
  before update on public.notes
  for each row execute procedure public.set_updated_at();
```

Konfigurasi & entry point

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:provider/provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'data/note_local_data_source.dart';
import 'data/note_remote_data_source.dart';
import 'data/note_repository.dart';
import 'providers/auth_provider.dart';
import 'providers/notes_provider.dart';
import 'screens/auth_gate.dart';
import 'services/auth_service.dart';

Future<void> main() async {
  // Wajib sebelum memanggil kode async/plugin sebelum runApp.
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
```

```
    return;
  }

  // 1. Hive (cache lokal)
  await Hive.initFlutter();
  final localDataSource = await NoteLocalDataSource.open();

  // 2. Supabase (backend)
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );
  final client = Supabase.instance.client;

  // 3. Rakit lapisan data: Service/DataSource -> Repository
  final authService = AuthService(client);
  final repository = NoteRepository(
    remote: NoteRemoteDataSource(client),
    local: localDataSource,
  );

  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider(authService)),
        ChangeNotifierProvider(create: (_) => NotesProvider(repository)),
      ],
      child: const CatatanKuliahApp(),
    ),
  );
}

class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
```

```
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/config/env.dart

```
/// Konfigurasi Supabase yang dibaca saat build lewat --dart-define.
///
/// Jalankan aplikasi dengan:
///   flutter run --dart-define-from-file=env.json
class Env {
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  /// true jika kedua nilai sudah diisi.
  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

Model

lib/models/note.dart

```
/// Model satu catatan kuliah. Satu objek Note = satu baris di tabel `notes`.
class Note {
  const Note({
    required this.id,
    required this.userId,
    required this.title,
    required this.content,
    required this.category,
    required this.createdAt,
    required this.updatedAt,
  });

  /// Pilihan kategori yang tersedia di form.
  static const List<String> categories = [
    'Kuliah',
    'Tugas',
    'Ujian',
    'Lainnya',
  ];

  final String id;
  final String userId;
```

```dart
  final String title;
  final String content;
  final String category;
  final DateTime createdAt;
  final DateTime updatedAt;

  /// Membuat Note dari Map. Dipakai untuk data dari Supabase
  /// (`Map<String, dynamic>`) maupun dari Hive (`Map<dynamic, dynamic>`).
  factory Note.fromJson(Map<dynamic, dynamic> json) {
    return Note(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String? ?? '',
      content: json['content'] as String? ?? '',
      category: json['category'] as String? ?? 'Lainnya',
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }

  /// Seluruh kolom, dipakai untuk menyimpan cache di Hive.
  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'user_id': userId,
      'title': title,
      'content': content,
      'category': category,
      'created_at': createdAt.toIso8601String(),
      'updated_at': updatedAt.toIso8601String(),
    };
  }

  /// Hanya kolom yang boleh diisi/diubah aplikasi, dipakai untuk
  /// insert/update ke Supabase. id, user_id, created_at, dan updated_at
  /// diisi otomatis oleh database.
  Map<String, dynamic> toMap() {
    return {'title': title, 'content': content, 'category': category};
  }

  Note copyWith({
    String? title,
    String? content,
    String? category,
    DateTime? updatedAt,
  }) {
    return Note(
      id: id,
      userId: userId,
      title: title ?? this.title,
      content: content ?? this.content,
      category: category ?? this.category,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }
}
```

Service & data

lib/services/auth_service.dart

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

/// Lapisan paling bawah untuk urusan autentikasi.
/// Hanya class ini yang berbicara langsung dengan Supabase Auth.
class AuthService {
  AuthService(this._client);

  final SupabaseClient _client;

  GoTrueClient get _auth => _client.auth;

  /// Stream yang memancarkan event setiap status login berubah
  /// (initialSession, signedIn, signedOut, tokenRefreshed, ...).
  Stream<AuthState> get authStateChanges => _auth.onAuthStateChange;

  Session? get currentSession => _auth.currentSession;

  User? get currentUser => _auth.currentUser;

  /// Mendaftarkan akun baru. Nama lengkap dikirim sebagai metadata
  /// (raw_user_meta_data) dan nanti disalin trigger ke tabel profiles.
  Future<AuthResponse> signUp({
    required String fullName,
    required String email,
    required String password,
  }) {
    return _auth.signUp(
      email: email,
      password: password,
      data: {'full_name': fullName},
    );
  }

  Future<AuthResponse> signIn({
    required String email,
    required String password,
  }) {
    return _auth.signInWithPassword(email: email, password: password);
  }

  Future<void> signOut() => _auth.signOut();
}
```

lib/data/note_remote_data_source.dart

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';

/// Semua operasi CRUD ke tabel `notes` di Supabase.
/// RLS di database memastikan setiap user hanya melihat barisnya sendiri.
class NoteRemoteDataSource {
  NoteRemoteDataSource(this._client);
```

```
  final SupabaseClient _client;

  static const String _table = 'notes';

  Future<List<Note>> fetchNotes() async {
    final rows = await _client
        .from(_table)
        .select()
        .order('updated_at', ascending: false);
    return rows.map(Note.fromJson).toList();
  }

  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final row = await _client
        .from(_table)
        .insert({'title': title, 'content': content, 'category': category})
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<Note> updateNote(Note note) async {
    final row = await _client
        .from(_table)
        .update(note.toMap())
        .eq('id', note.id)
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<void> deleteNote(String id) async {
    await _client.from(_table).delete().eq('id', id);
  }
}
```

lib/data/note_local_data_source.dart

```
import 'package:hive_ce/hive_ce.dart';

import '../models/note.dart';

/// Cache catatan di Hive. Setiap catatan disimpan sebagai Map
/// dengan key = id catatan, sehingga tidak perlu TypeAdapter/codegen.
class NoteLocalDataSource {
  NoteLocalDataSource(this._box);

  static const String boxName = 'notes_cache';

  final Box<Map> _box;

  /// Membuka box. Panggil setelah Hive.initFlutter() (atau Hive.init()).
  static Future<NoteLocalDataSource> open() async {
```

```
    final box = await Hive.openBox<Map>(boxName);
    return NoteLocalDataSource(box);
  }

  /// Semua catatan di cache, urut dari yang terakhir diubah.
  List<Note> getNotes() {
    final notes = _box.values.map(Note.fromJson).toList();
    notes.sort((a, b) => b.updatedAt.compareTo(a.updatedAt));
    return notes;
  }

  /// Mengganti seluruh isi cache dengan data terbaru dari server.
  Future<void> saveNotes(List<Note> notes) async {
    await _box.clear();
    await _box.putAll({for (final note in notes) note.id: note.toJson()});
  }

  Future<void> saveNote(Note note) => _box.put(note.id, note.toJson());

  Future<void> deleteNote(String id) => _box.delete(id);

  /// Dipanggil saat logout agar data user lain tidak tertinggal.
  Future<void> clear() => _box.clear();
}
```

lib/data/note_repository.dart

```
import 'dart:async';

import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';
import 'note_local_data_source.dart';
import 'note_remote_data_source.dart';

/// Hasil pengambilan daftar catatan.
class NotesResult {
  const NotesResult(this.notes, {required this.fromCache});

  final List<Note> notes;

  /// true jika server tidak bisa dihubungi dan data diambil dari Hive.
  final bool fromCache;
}

/// Satu pintu untuk semua data catatan. Provider tidak perlu tahu
/// apakah data berasal dari Supabase atau dari cache Hive.
class NoteRepository {
  NoteRepository({
    required this.remote,
    required this.local,
    this.timeout = const Duration(seconds: 10),
  });

  final NoteRemoteDataSource remote;
  final NoteLocalDataSource local;
  final Duration timeout;
```

```
  /// Ambil dari server lalu simpan ke cache. Jika gagal karena jaringan,
  /// kembalikan isi cache (mode offline).
  Future<NotesResult> getNotes() async {
    try {
      final notes = await remote.fetchNotes().timeout(timeout);
      await local.saveNotes(notes);
      return NotesResult(notes, fromCache: false);
    } on PostgrestException {
      // Server menjawab tapi menolak (misalnya RLS / tabel tidak ada).
      // Ini bukan masalah jaringan, jadi teruskan errornya.
      rethrow;
    } catch (_) {
      // SocketException, ClientException, TimeoutException, dll.
      return NotesResult(local.getNotes(), fromCache: true);
    }
  }

  Future<Note> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final note = await remote
        .createNote(title: title, content: content, category: category)
        .timeout(timeout);
    await local.saveNote(note);
    return note;
  }

  Future<Note> updateNote(Note note) async {
    final updated = await remote.updateNote(note).timeout(timeout);
    await local.saveNote(updated);
    return updated;
  }

  Future<void> deleteNote(String id) async {
    await remote.deleteNote(id).timeout(timeout);
    await local.deleteNote(id);
  }

  Future<void> clearCache() => local.clear();
}
```

Provider

lib/providers/auth_provider.dart

```
import 'package:flutter/foundation.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';

/// Hasil proses registrasi, dipakai RegisterScreen untuk menentukan
/// pesan yang ditampilkan.
enum RegisterResult { failed, loggedIn, needsEmailConfirmation }

class AuthProvider extends ChangeNotifier {
  AuthProvider(this._authService);
```

```dart
  final AuthService _authService;

  bool _isLoading = false;
  String? _errorMessage;

  bool get isLoading => _isLoading;
  String? get errorMessage => _errorMessage;

  Stream<AuthState> get authStateChanges => _authService.authStateChanges;
  Session? get currentSession => _authService.currentSession;
  User? get currentUser => _authService.currentUser;

  /// Nama yang ditampilkan di AppBar. Diambil dari metadata saat register.
  String get displayName {
    final user = currentUser;
    if (user == null) return '';
    final name = user.userMetadata?['full_name'];
    if (name is String && name.isNotEmpty) return name;
    return user.email ?? '';
  }

  Future<bool> signIn({required String email, required String password}) async {
    _setLoading(true);
    try {
      await _authService.signIn(email: email.trim(), password: password);
      return true;
    } on AuthException catch (e) {
      _errorMessage = _mapAuthError(e);
      return false;
    } catch (e) {
      _errorMessage = 'Terjadi kesalahan tak terduga: $e';
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<RegisterResult> register({
    required String fullName,
    required String email,
    required String password,
  }) async {
    _setLoading(true);
    try {
      final response = await _authService.signUp(
        fullName: fullName.trim(),
        email: email.trim(),
        password: password,
      );
      // Jika "Confirm email" aktif di dashboard, session bernilai null
      // sampai pengguna mengeklik link di email.
      if (response.session == null) {
        return RegisterResult.needsEmailConfirmation;
      }
      return RegisterResult.loggedIn;
    } on AuthException catch (e) {
      _errorMessage = _mapAuthError(e);
      return RegisterResult.failed;
```

```
    } catch (e) {
      _errorMessage = 'Terjadi kesalahan tak terduga: $e';
      return RegisterResult.failed;
    } finally {
      _setLoading(false);
    }
  }

  Future<void> signOut() async {
    try {
      await _authService.signOut();
    } on AuthException catch (e) {
      // signOut tetap menghapus sesi lokal walau request ke server gagal.
      debugPrint('signOut: ${e.message}');
    }
  }

  void clearError() {
    _errorMessage = null;
    notifyListeners();
  }

  void _setLoading(bool value) {
    _isLoading = value;
    if (value) _errorMessage = null;
    notifyListeners();
  }

  /// Menerjemahkan error Supabase Auth menjadi pesan berbahasa Indonesia.
  String _mapAuthError(AuthException e) {
    if (e is AuthRetryableFetchException) {
      return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
    }
    switch (e.code) {
      case 'invalid_credentials':
        return 'Email atau password salah.';
      case 'email_not_confirmed':
        return 'Email belum dikonfirmasi. Buka link verifikasi di inbox Anda.';
      case 'user_already_exists':
      case 'email_exists':
        return 'Email sudah terdaftar. Silakan login.';
      case 'weak_password':
        return 'Password terlalu lemah. Gunakan minimal 6 karakter.';
      case 'email_address_invalid':
        return 'Alamat email tidak valid.';
      case 'over_email_send_rate_limit':
      case 'over_request_rate_limit':
        return 'Terlalu banyak percobaan. Tunggu beberapa menit lalu coba lagi.';
      case 'signup_disabled':
        return 'Pendaftaran akun baru sedang dinonaktifkan.';
    }
    return e.message;
  }
}
```

lib/providers/notes_provider.dart

```dart
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

class NotesProvider extends ChangeNotifier {
  NotesProvider(this._repository);

  final NoteRepository _repository;

  List<Note> _notes = [];
  bool _isLoading = false;
  bool _isOffline = false;
  String? _errorMessage;
  String _query = '';
  String? _categoryFilter; // null = semua kategori

  bool get isLoading => _isLoading;
  bool get isOffline => _isOffline;
  String? get errorMessage => _errorMessage;
  String get query => _query;
  String? get categoryFilter => _categoryFilter;
  bool get hasNotes => _notes.isNotEmpty;

  /// Daftar yang ditampilkan di layar setelah difilter pencarian & kategori.
  List<Note> get visibleNotes {
    final q = _query.toLowerCase();
    return _notes.where((note) {
      final cocokKategori =
          _categoryFilter == null || note.category == _categoryFilter;
      final cocokCari =
          q.isEmpty ||
          note.title.toLowerCase().contains(q) ||
          note.content.toLowerCase().contains(q);
      return cocokKategori && cocokCari;
    }).toList();
  }

  Note? findById(String id) {
    for (final note in _notes) {
      if (note.id == id) return note;
    }
    return null;
  }

  /// Memuat catatan (dipanggil saat HomeScreen dibuka & saat pull-to-refresh).
  Future<void> loadNotes() async {
    _isLoading = true;
    _errorMessage = null;
    notifyListeners();

    try {
      final result = await _repository.getNotes();
      _notes = result.notes;
```

```
      _isOffline = result.fromCache;
    } catch (e) {
      _errorMessage = _friendlyError(e);
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }

  /// Mengembalikan null jika berhasil, atau pesan error jika gagal.
  Future<String?> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    try {
      final note = await _repository.addNote(
        title: title,
        content: content,
        category: category,
      );
      _notes.insert(0, note);
      _isOffline = false;
      notifyListeners();
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  Future<String?> updateNote(Note note) async {
    try {
      final updated = await _repository.updateNote(note);
      _notes.removeWhere((n) => n.id == updated.id);
      _notes.insert(0, updated); // yang baru diubah naik ke atas
      _isOffline = false;
      notifyListeners();
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  /// Optimistic delete: hapus dari layar dulu, baru kirim ke server.
  /// Jika server gagal, catatan dikembalikan ke posisi semula (rollback).
  Future<String?> deleteNote(Note note) async {
    final index = _notes.indexWhere((n) => n.id == note.id);
    if (index == -1) return null;

    _notes.removeAt(index);
    notifyListeners();

    try {
      await _repository.deleteNote(note.id);
      return null;
    } catch (e) {
      _notes.insert(index, note);
      notifyListeners();
      return _friendlyError(e);
```

```
    }
  }

  void setQuery(String value) {
    _query = value;
    notifyListeners();
  }

  void setCategoryFilter(String? category) {
    _categoryFilter = category;
    notifyListeners();
  }

  /// Dipanggil saat logout: kosongkan state dan cache Hive.
  Future<void> clear() async {
    _notes = [];
    _query = '';
    _categoryFilter = null;
    _errorMessage = null;
    _isOffline = false;
    await _repository.clearCache();
    notifyListeners();
  }

  String _friendlyError(Object e) {
    if (e is PostgrestException) {
      if (e.code == '42501') {
        return 'Akses ditolak oleh Row Level Security. Periksa policy tabel notes.';
      }
      if (e.code == '42P01' || e.code == 'PGRST205') {
        return 'Tabel notes belum dibuat. Jalankan SQL di Supabase.';
      }
      return 'Kesalahan database: ${e.message}';
    }
    if (e is TimeoutException) {
      return 'Server terlalu lama merespons. Coba lagi.';
    }
    return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
  }
}
```

Screens

lib/screens/auth_gate.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../providers/auth_provider.dart';
import 'home_screen.dart';
import 'login_screen.dart';

/// "Penjaga pintu" aplikasi: mendengarkan perubahan status login lalu
/// memilih halaman yang tepat secara otomatis.
class AuthGate extends StatefulWidget {
  const AuthGate({super.key});
```

```
  @override
  State<AuthGate> createState() => _AuthGateState();
}

class _AuthGateState extends State<AuthGate> {
  // Simpan stream di State agar tidak dibuat ulang setiap build.
  late final Stream<AuthState> _authStream;

  @override
  void initState() {
    super.initState();
    _authStream = context.read<AuthProvider>().authStateChanges;
  }

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<AuthState>(
      stream: _authStream,
      builder: (context, snapshot) {
        // Sebelum event pertama datang, pakai sesi yang tersimpan di perangkat.
        final session = snapshot.hasData
            ? snapshot.data!.session
            : context.read<AuthProvider>().currentSession;

        if (session == null) {
          return const LoginScreen();
        }
        return const HomeScreen();
      },
    );
  }
}
```

lib/screens/login_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../utils/validators.dart';
import 'register_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _emailController.dispose();
```

```
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    // Tutup keyboard lalu validasi form.
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final auth = context.read<AuthProvider>();
    final success = await auth.signIn(
      email: _emailController.text,
      password: _passwordController.text,
    );

    // Jika sukses, AuthGate otomatis pindah ke HomeScreen.
    if (!success && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(auth.errorMessage ?? 'Login gagal'),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.watch<AuthProvider>().isLoading;

    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Form(
              key: _formKey,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Icon(
                    Icons.menu_book_rounded,
                    size: 72,
                    color: Theme.of(context).colorScheme.primary,
                  ),
                  const SizedBox(height: 16),
                  Text(
                    'Catatan Kuliah',
                    textAlign: TextAlign.center,
                    style: Theme.of(context).textTheme.headlineMedium,
                  ),
                  const SizedBox(height: 8),
                  const Text(
                    'Masuk untuk melihat catatan Anda',
                    textAlign: TextAlign.center,
                  ),
                  const SizedBox(height: 32),
                  TextFormField(
                    controller: _emailController,
```

```
    keyboardType: TextInputType.emailAddress,
    textInputAction: TextInputAction.next,
    autofillHints: const [AutofillHints.email],
    decoration: const InputDecoration(
      labelText: 'Email',
      prefixIcon: Icon(Icons.email_outlined),
      border: OutlineInputBorder(),
    ),
    validator: Validators.email,
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _passwordController,
    obscureText: _obscurePassword,
    textInputAction: TextInputAction.done,
    onFieldSubmitted: (_) => _submit(),
    decoration: InputDecoration(
      labelText: 'Password',
      prefixIcon: const Icon(Icons.lock_outline),
      border: const OutlineInputBorder(),
      suffixIcon: IconButton(
        tooltip: _obscurePassword
            ? 'Tampilkan password'
            : 'Sembunyikan password',
        icon: Icon(
          _obscurePassword
              ? Icons.visibility_outlined
              : Icons.visibility_off_outlined,
        ),
        onPressed: () {
          setState(() => _obscurePassword = !_obscurePassword);
        },
      ),
    ),
    validator: Validators.password,
  ),
  const SizedBox(height: 24),
  FilledButton(
    onPressed: isLoading ? null : _submit,
    style: FilledButton.styleFrom(
      minimumSize: const Size.fromHeight(48),
    ),
    child: isLoading
        ? const SizedBox(
            width: 22,
            height: 22,
            child: CircularProgressIndicator(strokeWidth: 2.5),
          )
        : const Text('Masuk'),
  ),
  const SizedBox(height: 12),
  TextButton(
    onPressed: isLoading
        ? null
        : () {
            Navigator.of(context).push(
              MaterialPageRoute(
                builder: (_) => const RegisterScreen(),
              ),
```

```
                          );
                        },
                    child: const Text('Belum punya akun? Daftar di sini'),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/screens/register_screen.dart

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/auth_provider.dart';
import '../utils/validators.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  String? _validateConfirm(String? value) {
    if (value == null || value.isEmpty) return 'Ulangi password Anda';
    if (value != _passwordController.text) return 'Password tidak sama';
    return null;
  }

  Future<void> _submit() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final auth = context.read<AuthProvider>();
    final result = await auth.register(
```

```
      fullName: _nameController.text,
      email: _emailController.text,
      password: _passwordController.text,
    );
    if (!mounted) return;

    switch (result) {
      case RegisterResult.loggedIn:
        // Sesi langsung aktif. AuthGate di bawah halaman ini sudah berganti
        // ke HomeScreen, jadi cukup tutup halaman register.
        Navigator.of(context).pop();
      case RegisterResult.needsEmailConfirmation:
        await showDialog<void>(
          context: context,
          builder: (dialogContext) => AlertDialog(
            title: const Text('Cek Email Anda'),
            content: Text(
              'Link konfirmasi sudah dikirim ke ${_emailController.text.trim()}. '
              'Klik link tersebut, lalu login dengan akun baru Anda.',
            ),
            actions: [
              FilledButton(
                onPressed: () => Navigator.of(dialogContext).pop(),
                child: const Text('Mengerti'),
              ),
            ],
          ),
        );
        if (mounted) Navigator.of(context).pop();
      case RegisterResult.failed:
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(auth.errorMessage ?? 'Registrasi gagal'),
            backgroundColor: Theme.of(context).colorScheme.error,
          ),
        );
    }
  }

  @override
  Widget build(BuildContext context) {
    final isLoading = context.watch<AuthProvider>().isLoading;

    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Akun')),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                TextFormField(
                  controller: _nameController,
                  textCapitalization: TextCapitalization.words,
                  textInputAction: TextInputAction.next,
                  decoration: const InputDecoration(
                    labelText: 'Nama Lengkap',
```

```
      prefixIcon: Icon(Icons.person_outline),
      border: OutlineInputBorder(),
    ),
    validator: Validators.fullName,
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _emailController,
    keyboardType: TextInputType.emailAddress,
    textInputAction: TextInputAction.next,
    decoration: const InputDecoration(
      labelText: 'Email',
      prefixIcon: Icon(Icons.email_outlined),
      border: OutlineInputBorder(),
    ),
    validator: Validators.email,
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _passwordController,
    obscureText: _obscurePassword,
    textInputAction: TextInputAction.next,
    decoration: InputDecoration(
      labelText: 'Password',
      helperText: 'Minimal 6 karakter',
      prefixIcon: const Icon(Icons.lock_outline),
      border: const OutlineInputBorder(),
      suffixIcon: IconButton(
        icon: Icon(
          _obscurePassword
              ? Icons.visibility_outlined
              : Icons.visibility_off_outlined,
        ),
        onPressed: () {
          setState(() => _obscurePassword = !_obscurePassword);
        },
      ),
    ),
    validator: Validators.password,
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _confirmController,
    obscureText: _obscurePassword,
    textInputAction: TextInputAction.done,
    onFieldSubmitted: (_) => _submit(),
    decoration: const InputDecoration(
      labelText: 'Ulangi Password',
      prefixIcon: Icon(Icons.lock_reset_outlined),
      border: OutlineInputBorder(),
    ),
    validator: _validateConfirm,
  ),
  const SizedBox(height: 24),
  FilledButton(
    onPressed: isLoading ? null : _submit,
    style: FilledButton.styleFrom(
      minimumSize: const Size.fromHeight(48),
    ),
```

```
                  child: isLoading
                      ? const SizedBox(
                          width: 22,
                          height: 22,
                          child: CircularProgressIndicator(strokeWidth: 2.5),
                        )
                      : const Text('Daftar'),
                ),
                const SizedBox(height: 12),
                TextButton(
                  onPressed: isLoading
                      ? null
                      : () => Navigator.of(context).pop(),
                  child: const Text('Sudah punya akun? Masuk'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
  }
}
```

lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/auth_provider.dart';
import '../providers/notes_provider.dart';
import '../widgets/confirm_dialog.dart';
import '../widgets/note_card.dart';
import '../widgets/offline_banner.dart';
import 'note_detail_screen.dart';
import 'note_form_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
    // Tunggu frame pertama selesai, baru muat data (notifyListeners tidak
    // boleh dipanggil ketika widget tree sedang dibangun).
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) context.read<NotesProvider>().loadNotes();
    });
  }
```

```dart
  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  Future<void> _logout() async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Keluar?',
      message: 'Catatan yang tersimpan di perangkat akan dihapus.',
      confirmLabel: 'Keluar',
    );
    if (!yakin || !mounted) return;

    // Ambil referensi provider sebelum await berikutnya.
    final notes = context.read<NotesProvider>();
    final auth = context.read<AuthProvider>();
    await notes.clear(); // hapus cache Hive
    await auth.signOut(); // AuthGate otomatis kembali ke LoginScreen
  }

  Future<bool> _confirmDelete(Note note) {
    return showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
  }

  Future<void> _delete(Note note) async {
    final messenger = ScaffoldMessenger.of(context);
    final error = await context.read<NotesProvider>().deleteNote(note);
    messenger.showSnackBar(
      SnackBar(
        content: Text(
          error == null
              ? 'Catatan dihapus'
              : 'Gagal menghapus, catatan dikembalikan. $error',
        ),
      ),
    );
  }

  void _openForm() {
    Navigator.of(context)
        .push(MaterialPageRoute(builder: (_) => const NoteFormScreen()));
  }

  void _openDetail(Note note) {
    Navigator.of(context).push(
      MaterialPageRoute(builder: (_) => NoteDetailScreen(noteId: note.id)),
    );
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<NotesProvider>();
```

```
    final name = context.read<AuthProvider>().displayName;

    return Scaffold(
      appBar: AppBar(
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Catatan Kuliah'),
            Text('Halo, $name', style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: _logout,
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _openForm,
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
      body: Column(
        children: [
          if (provider.isOffline) OfflineBanner(onRetry: provider.loadNotes),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
            child: TextField(
              controller: _searchController,
              onChanged: provider.setQuery,
              decoration: InputDecoration(
                hintText: 'Cari judul atau isi catatan...',
                prefixIcon: const Icon(Icons.search),
                border: const OutlineInputBorder(),
                isDense: true,
                suffixIcon: provider.query.isEmpty
                    ? null
                    : IconButton(
                        icon: const Icon(Icons.clear),
                        onPressed: () {
                          _searchController.clear();
                          provider.setQuery('');
                        },
                      ),
              ),
            ),
          ),
          _CategoryFilter(
            selected: provider.categoryFilter,
            onSelected: provider.setCategoryFilter,
          ),
          Expanded(
            child: RefreshIndicator(
              onRefresh: provider.loadNotes,
              child: _buildBody(provider),
            ),
          ),
```

```
        ],
      ),
    );
  }

  Widget _buildBody(NotesProvider provider) {
    // 1. Loading pertama kali (belum ada data sama sekali)
    if (provider.isLoading && !provider.hasNotes) {
      return const Center(child: CircularProgressIndicator());
    }

    // 2. Error dan tidak ada data cadangan
    if (provider.errorMessage != null && !provider.hasNotes) {
      return _MessageView(
        icon: Icons.error_outline,
        message: provider.errorMessage!,
        action: FilledButton.icon(
          onPressed: provider.loadNotes,
          icon: const Icon(Icons.refresh),
          label: const Text('Coba lagi'),
        ),
      );
    }

    final notes = provider.visibleNotes;

    // 3. Kosong
    if (notes.isEmpty) {
      return _MessageView(
        icon: Icons.note_alt_outlined,
        message: provider.hasNotes
            ? 'Tidak ada catatan yang cocok.'
            : 'Belum ada catatan.\nTekan tombol + untuk menambah.',
      );
    }

    // 4. Ada data
    return ListView.builder(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.only(top: 4, bottom: 88),
      itemCount: notes.length,
      itemBuilder: (context, index) {
        final note = notes[index];
        return Dismissible(
          key: ValueKey(note.id),
          direction: DismissDirection.endToStart,
          background: Container(
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 24),
            color: Theme.of(context).colorScheme.errorContainer,
            child: Icon(
              Icons.delete,
              color: Theme.of(context).colorScheme.onErrorContainer,
            ),
          ),
          confirmDismiss: (_) => _confirmDelete(note),
          onDismissed: (_) => _delete(note),
          child: NoteCard(note: note, onTap: () => _openDetail(note)),
        );
```

```
      },
    );
  }
}

/// Deretan chip untuk memfilter kategori.
class _CategoryFilter extends StatelessWidget {
  const _CategoryFilter({required this.selected, required this.onSelected});

  final String? selected;
  final ValueChanged<String?> onSelected;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 48,
      child: ListView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        children: [
          Padding(
            padding: const EdgeInsets.only(right: 8),
            child: ChoiceChip(
              label: const Text('Semua'),
              selected: selected == null,
              onSelected: (_) => onSelected(null),
            ),
          ),
          for (final category in Note.categories)
            Padding(
              padding: const EdgeInsets.only(right: 8),
              child: ChoiceChip(
                label: Text(category),
                selected: selected == category,
                onSelected: (_) => onSelected(category),
              ),
            ),
        ],
      ),
    );
  }
}

/// Tampilan pesan di tengah layar yang tetap bisa ditarik (pull-to-refresh).
class _MessageView extends StatelessWidget {
  const _MessageView({required this.icon, required this.message, this.action});

  final IconData icon;
  final String message;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    return ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.all(32),
      children: [
        const SizedBox(height: 48),
        Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
```

```
        const SizedBox(height: 16),
        Text(message, textAlign: TextAlign.center),
        if (action != null) ...[
          const SizedBox(height: 16),
          Center(child: action),
        ],
      ],
    );
  }
}
```

lib/screens/note_form_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/notes_provider.dart';

/// Satu layar untuk dua keperluan:
/// - note == null  -> mode TAMBAH
/// - note != null  -> mode EDIT
class NoteFormScreen extends StatefulWidget {
  const NoteFormScreen({super.key, this.note});

  final Note? note;

  @override
  State<NoteFormScreen> createState() => _NoteFormScreenState();
}

class _NoteFormScreenState extends State<NoteFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleController;
  late final TextEditingController _contentController;
  late String _category;
  bool _isSaving = false;

  bool get _isEdit => widget.note != null;

  @override
  void initState() {
    super.initState();
    // Mode edit: isi form dengan data lama. Mode tambah: form kosong.
    _titleController = TextEditingController(text: widget.note?.title ?? '');
    _contentController = TextEditingController(
      text: widget.note?.content ?? '',
    );
    _category = widget.note?.category ?? Note.categories.first;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }
```

```
  Future<void> _save() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
    final provider = context.read<NotesProvider>();
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();

    final String? error;
    if (_isEdit) {
      error = await provider.updateNote(
        widget.note!.copyWith(
          title: title,
          content: content,
          category: _category,
        ),
      );
    } else {
      error = await provider.addNote(
        title: title,
        content: content,
        category: _category,
      );
    }

    if (!mounted) return;
    setState(() => _isSaving = false);

    final messenger = ScaffoldMessenger.of(context);
    if (error == null) {
      messenger.showSnackBar(
        SnackBar(
          content: Text(_isEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
        ),
      );
      Navigator.of(context).pop();
    } else {
      messenger.showSnackBar(
        SnackBar(
          content: Text(error),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Catatan' : 'Catatan Baru')),
      body: SafeArea(
        child: Form(
          key: _formKey,
          child: ListView(
            padding: const EdgeInsets.all(16),
            children: [
              TextFormField(
                controller: _titleController,
```

```
            textInputAction: TextInputAction.next,
            maxLength: 100,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Judul wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 8),
          DropdownButtonFormField<String>(
            initialValue: _category,
            decoration: const InputDecoration(
              labelText: 'Kategori',
              border: OutlineInputBorder(),
            ),
            items: [
              for (final category in Note.categories)
                DropdownMenuItem(value: category, child: Text(category)),
            ],
            onChanged: (value) {
              if (value != null) setState(() => _category = value);
            },
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _contentController,
            minLines: 6,
            maxLines: 12,
            keyboardType: TextInputType.multiline,
            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              alignLabelWithHint: true,
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Isi catatan wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 24),
          FilledButton.icon(
            onPressed: _isSaving ? null : _save,
            style: FilledButton.styleFrom(
              minimumSize: const Size.fromHeight(48),
            ),
            icon: _isSaving
                ? const SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2.5),
                  )
                : const Icon(Icons.save),
```

```
                label: Text(_isSaving ? 'Menyimpan...' : 'Simpan'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

lib/screens/note_detail_screen.dart

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/note.dart';
import '../providers/notes_provider.dart';
import '../utils/date_format.dart';
import '../widgets/confirm_dialog.dart';
import 'note_form_screen.dart';

class NoteDetailScreen extends StatelessWidget {
  const NoteDetailScreen({super.key, required this.noteId});

  final String noteId;

  Future<void> _delete(BuildContext context, Note note) async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
    if (!yakin || !context.mounted) return;

    final messenger = ScaffoldMessenger.of(context);
    final provider = context.read<NotesProvider>();
    // Tutup halaman detail dulu, baru hapus (optimistic delete).
    Navigator.of(context).pop();
    final error = await provider.deleteNote(note);
    messenger.showSnackBar(SnackBar(content: Text(error ?? 'Catatan dihapus')));
  }

  @override
  Widget build(BuildContext context) {
    // Ambil versi terbaru catatan dari provider, sehingga setelah diedit
    // halaman ini langsung menampilkan perubahan.
    final note = context.select<NotesProvider, Note?>(
      (provider) => provider.findById(noteId),
    );

    if (note == null) {
      return Scaffold(
        appBar: AppBar(),
        body: const Center(child: Text('Catatan tidak ditemukan.')),
      );
    }
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Detail Catatan'),
        actions: [
          IconButton(
            tooltip: 'Edit',
            icon: const Icon(Icons.edit),
            onPressed: () {
              Navigator.of(context).push(
                MaterialPageRoute(builder: (_) => NoteFormScreen(note: note)),
              );
            },
          ),
          IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: () => _delete(context, note),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(note.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              Chip(label: Text(note.category)),
              Text(
                'Diubah ${formatTanggal(note.updatedAt)}',
                style: Theme.of(context).textTheme.bodySmall,
              ),
            ],
          ),
          const Divider(height: 32),
          SelectableText(
            note.content,
            style: Theme.of(context).textTheme.bodyLarge,
          ),
        ],
      ),
    );
  }
}
```

Widgets

lib/widgets/confirm_dialog.dart

```
import 'package:flutter/material.dart';

/// Dialog konfirmasi Ya/Batal yang dipakai ulang di beberapa layar.
/// Mengembalikan true jika pengguna menekan tombol konfirmasi.
Future<bool> showConfirmDialog(
  BuildContext context, {
  required String title,
```

```
  required String message,
  required String confirmLabel,
}) async {
  final hasil = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(title),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(dialogContext).pop(false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.of(dialogContext).pop(true),
          child: Text(confirmLabel),
        ),
      ],
    ),
  );
  return hasil ?? false;
}
```

lib/widgets/note_card.dart

```
import 'package:flutter/material.dart';

import '../models/note.dart';
import '../utils/date_format.dart';

class NoteCard extends StatelessWidget {
  const NoteCard({super.key, required this.note, this.onTap});

  final Note note;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      child: ListTile(
        onTap: onTap,
        contentPadding: const EdgeInsets.fromLTRB(16, 8, 16, 8),
        title: Text(
          note.title,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontWeight: FontWeight.w600),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 4),
            Text(note.content, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 8),
            Row(
```

```
              children: [
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 8,
                    vertical: 2,
                  ),
                  decoration: BoxDecoration(
                    color: scheme.primaryContainer,
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Text(
                    note.category,
                    style: TextStyle(
                      fontSize: 12,
                      color: scheme.onPrimaryContainer,
                    ),
                  ),
                ),
                const Spacer(),
                Text(
                  formatTanggal(note.updatedAt),
                  style: Theme.of(context).textTheme.bodySmall,
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/widgets/offline_banner.dart

```
import 'package:flutter/material.dart';

/// Pita peringatan yang muncul ketika data diambil dari cache Hive.
class OfflineBanner extends StatelessWidget {
  const OfflineBanner({super.key, required this.onRetry});

  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Material(
      color: scheme.tertiaryContainer,
      child: Padding(
        padding: const EdgeInsets.fromLTRB(16, 4, 8, 4),
        child: Row(
          children: [
            Icon(Icons.cloud_off, color: scheme.onTertiaryContainer),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                'Mode offline: menampilkan data tersimpan.',
                style: TextStyle(color: scheme.onTertiaryContainer),
```

```
              ),
            ),
            TextButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}
```

Utils

lib/utils/validators.dart

```dart
/// Kumpulan validator untuk TextFormField.
/// Setiap fungsi mengembalikan null jika valid, atau pesan error jika tidak.
class Validators {
  static String? fullName(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Nama lengkap wajib diisi';
    if (text.length < 3) return 'Nama minimal 3 karakter';
    return null;
  }

  static String? email(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Email wajib diisi';
    final pola = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');
    if (!pola.hasMatch(text)) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value) {
    final text = value ?? '';
    if (text.isEmpty) return 'Password wajib diisi';
    if (text.length < 6) return 'Password minimal 6 karakter';
    return null;
  }
}
```

lib/utils/date_format.dart

```dart
/// Format tanggal sederhana tanpa package tambahan, contoh: 5 Okt 2026 14:07.
String formatTanggal(DateTime waktu) {
  const bulan = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'Mei',
    'Jun',
    'Jul',
    'Agu',
    'Sep',
    'Okt',
    'Nov',
    'Des',
```

```
  ];
  final lokal = waktu.toLocal();
  final jam = lokal.hour.toString().padLeft(2, '0');
  final menit = lokal.minute.toString().padLeft(2, '0');
  return '${lokal.day} ${bulan[lokal.month - 1]} ${lokal.year} $jam:$menit';
}
```

Test

`test/note_local_data_source_test.dart`

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

Note buatNote(String id, {String title = 'Judul', int hari = 1}) {
  final waktu = DateTime.utc(2026, 10, hari);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: 'Kuliah',
    createdAt: waktu,
    updatedAt: waktu,
  );
}

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;

  setUp(() async {
    // Hive.init (bukan initFlutter) karena test berjalan tanpa plugin Flutter.
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('saveNotes lalu getNotes mengembalikan data urut terbaru', () async {
    await local.saveNotes([
      buatNote('a', title: 'Lama', hari: 1),
      buatNote('b', title: 'Baru', hari: 5),
    ]);

    final notes = local.getNotes();

    expect(notes.map((n) => n.id), ['b', 'a']);
    expect(notes.first.title, 'Baru');
  });
```

```
  test('saveNotes mengganti seluruh isi cache', () async {
    await local.saveNotes([buatNote('a'), buatNote('b')]);
    await local.saveNotes([buatNote('c')]);

    expect(local.getNotes().map((n) => n.id), ['c']);
  });

  test('saveNote, deleteNote, dan clear', () async {
    await local.saveNote(buatNote('a'));
    await local.saveNote(buatNote('a', title: 'Diubah'));
    expect(local.getNotes().single.title, 'Diubah');

    await local.saveNote(buatNote('b'));
    await local.deleteNote('a');
    expect(local.getNotes().map((n) => n.id), ['b']);

    await local.clear();
    expect(local.getNotes(), isEmpty);
  });
}
```

test/note_repository_test.dart

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_remote_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Note buatNote(String id, {String title = 'Judul'}) {
  final waktu = DateTime.utc(2026, 10, 1);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: 'Kuliah',
    createdAt: waktu,
    updatedAt: waktu,
  );
}

/// Pengganti Supabase untuk test. `implements` membuat kita wajib
/// menulis ulang semua method, sehingga tidak ada request jaringan sungguhan.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  List<Note> serverNotes = [];
  Object? errorToThrow; // jika diisi, setiap method melempar error ini

  void _maybeThrow() {
    final error = errorToThrow;
    if (error != null) throw error;
  }
```

```dart
  @override
  Future<List<Note>> fetchNotes() async {
    _maybeThrow();
    return serverNotes;
  }

  @override
  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    _maybeThrow();
    final note = buatNote('baru', title: title);
    serverNotes.add(note);
    return note;
  }

  @override
  Future<Note> updateNote(Note note) async {
    _maybeThrow();
    return note;
  }

  @override
  Future<void> deleteNote(String id) async {
    _maybeThrow();
    serverNotes.removeWhere((n) => n.id == id);
  }
}

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;
  late FakeRemoteDataSource remote;
  late NoteRepository repository;

  setUp(() async {
    tempDir = await Directory.systemTemp.createTemp('repo_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
    remote = FakeRemoteDataSource();
    repository = NoteRepository(remote: remote, local: local);
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('online: data server dikembalikan dan disimpan ke cache', () async {
    remote.serverNotes = [buatNote('1'), buatNote('2')];

    final result = await repository.getNotes();

    expect(result.fromCache, isFalse);
    expect(result.notes, hasLength(2));
    expect(local.getNotes(), hasLength(2));
  });
```

```
  test('offline: jika jaringan gagal, data diambil dari cache', () async {
    await local.saveNotes([buatNote('cache-1', title: 'Dari Hive')]);
    remote.errorToThrow = const SocketException('Tidak ada internet');

    final result = await repository.getNotes();

    expect(result.fromCache, isTrue);
    expect(result.notes.single.title, 'Dari Hive');
  });

  test('error dari server (PostgrestException) tidak disembunyikan', () async {
    remote.errorToThrow = const PostgrestException(
      message: 'permission denied',
      code: '42501',
    );

    expect(repository.getNotes(), throwsA(isA<PostgrestException>()));
  });

  test('addNote dan deleteNote ikut memperbarui cache', () async {
    final note = await repository.addNote(
      title: 'Basis Data',
      content: 'Normalisasi',
      category: 'Kuliah',
    );
    expect(local.getNotes().single.id, note.id);

    await repository.deleteNote(note.id);
    expect(local.getNotes(), isEmpty);
  });

  test('deleteNote gagal: cache tidak berubah', () async {
    await local.saveNotes([buatNote('1')]);
    remote.errorToThrow = const SocketException('offline');

    await expectLater(
      repository.deleteNote('1'),
      throwsA(isA<SocketException>()),
    );
    expect(local.getNotes(), hasLength(1));
  });
}
```

12. Troubleshooting & Ide Pengembangan

Daftar masalah yang paling sering ditemui mahasiswa saat mengerjakan proyek ini, beserta cara mendiagnosisnya. Biasakan membaca pesan error lengkap di konsol `flutter run` dan di **Logs** dashboard Supabase.

Tabel troubleshooting

Gejala / pesan	Penyebab	Solusi
`PostgrestException(code: 42501, message: new row violates row-level security policy for table "notes")`	Policy INSERT belum dibuat / salah, atau `user_id` tidak sama dengan `auth.uid()`	Jalankan ulang SQL section 1. Pastikan k memiliki `default auth.uid()` dan us
Daftar catatan selalu kosong padahal di Table Editor ada data	Policy SELECT tidak ada, atau data dibuat dengan `user_id` milik akun lain	Cek policy di Authentication → Policies, l `user_id` baris dengan id di Authenticat
"Tabel notes belum dibuat" (code `PGRST205` / `42P01`)	SQL belum dijalankan atau nama tabel salah ketik	Jalankan SQL, lalu cek nama `notes` di `NoteRemoteDataSource._table`
You must initialize the supabase instance before calling Supabase.instance	`Supabase.instance.client` dipakai sebelum `await Supabase.initialize`	Periksa urutan di `main()` dan pastikan
Layar "SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi"	Run tanpa `--dart-define-from-file=env.json`	Tambahkan argumen run atau atur `laun`
Login gagal: *Email belum dikonfirmasi*	Confirm email aktif dan link belum diklik	Klik link di email, atau matikan Confirm e Authentication → Sign In / Providers → E
HiveError: Box not found. Did you forget to call Hive.openBox()?	Box dipakai sebelum dibuka	Pastikan `await NoteLocalDataSourc` dipanggil di `main()` setelah `Hive.ini`
setState() or markNeedsBuild() called during build	`loadNotes()` dipanggil langsung di `initState` / `build`	Bungkus dengan `WidgetsBinding.instance.addPostF`
A dismissed Dismissible widget is still part of the tree	Item tidak dihapus dari list setelah digeser	Pastikan `deleteNote` menghapus dari sebelum `await`
Banner offline tidak pernah muncul walaupun mode pesawat	Semua error ditangkap sebagai error biasa, bukan fallback	Periksa `NoteRepository.getNotes` : `PostgrestException` rethrow, `catch` kembalikan cache

Gejala / pesan	Penyebab	Solusi
Banner offline muncul padahal online (dan data lama)	Error database tersembunyi sebagai "offline", atau URL salah	Jalankan `flutter test` dan periksa lo di `env.json` benar
Setelah logout lalu login akun lain, catatan akun lama sempat terlihat	Cache/state tidak dibersihkan saat logout	Pastikan `_logout` memanggil `NotesProvider.clear()` *sebelum* `si`
`updated_at` tidak berubah setelah edit	Trigger `notes_set_updated_at` belum dibuat	Jalankan bagian trigger di SQL
Insert gagal `23514` (check constraint)	Kategori di Dart berbeda dengan daftar di SQL, atau judul > 100 karakter	Samakan `Note.categories` dengan `c` `(category in (...))`
APK release tidak bisa konek, debug bisa	Izin INTERNET belum ada di manifest utama	Tambahkan `uses-permission INTERN` `android/app/src/main/AndroidMan`
Aplikasi tiba-tiba tidak bisa konek setelah lama tidak dipakai	Project Supabase gratis di-pause	Buka dashboard dan klik **Restore project**

Ide pengembangan (untuk tugas besar)

Ide	Petunjuk implementasi	Tingkat
Sinkronisasi realtime antar perangkat	Aktifkan Realtime untuk tabel `notes`, lalu di `NotesProvider` dengarkan `client.from('notes').stream(primaryKey: ['id'])` dan perbarui `_notes` + cache setiap ada data baru. Batalkan subscription di `dispose`/logout.	Menengah
Antrean penulisan offline (offline queue)	Buat box Hive kedua `pending_ops` berisi `{type: 'insert'|'update'|'delete', data: {...}}`. Saat offline, simpan operasi ke antrean dan perbarui cache lokal. Saat online kembali (refresh sukses), kirim antrean secara berurutan. Gunakan id UUID buatan klien (package `uuid`) agar insert offline punya id sejak awal.	Sulit
Lampiran file (foto papan tulis, PDF)	Buat bucket `attachments` dengan policy folder per user (Modul 12 section 10), tambah kolom `attachment_path text` di `notes`, unggah memakai `uploadBinary`, dan tampilkan dengan `createSignedUrl`.	Menengah
Pin catatan penting	Tambah kolom `is_pinned boolean not null default false`, lalu urutkan `.order('is_pinned', ascending: false).order('updated_at', ascending: false)`.	Mudah
Relasi ke mata kuliah	Ganti `category` dengan `mata_kuliah_id bigint references mata_kuliah(id)` dan ambil nama dengan `select('*, mata_kuliah(nama)')`.	Menengah
Lupa password	`auth.resetPasswordForEmail(email)` + deep link, dengan event `passwordRecovery` di AuthGate.	Menengah
Halaman profil	Baca/ubah tabel `profiles` (`maybeSingle` + `update`), foto profil dari Storage.	Mudah
Pagination	`.range()` + muat lebih banyak saat scroll mendekati bawah.	Menengah

Ringkasan Modul 14

- Tabel `notes` dilindungi empat policy RLS berbasis `auth.uid()`, dengan `user_id default auth.uid()` dan trigger `updated_at`.
- Lapisan data: **Remote** (Supabase CRUD), **Local** (Hive Map dengan key id), dan **Repository** (network first, cache fallback, PostgrestException tidak disembunyikan).
- `NotesProvider`: loading/error/offline, pencarian dan filter di memori, optimistic delete dengan rollback.

- UI: RefreshIndicator, empty/loading/error state, Dismissible + konfirmasi, satu form untuk tambah & edit, detail reaktif dengan `context.select`.
- Logout membersihkan state dan cache sebelum sesi dihapus.
- Logika offline diuji dengan unit test memakai fake data source dan Hive di folder sementara.

Tugas Praktikum: Praktikum 14: CRUD Catatan Kuliah dengan Supabase + Cache Offline

Sasaran Praktikum

Menyelesaikan aplikasi Catatan Kuliah dengan CRUD penuh ke Supabase, keamanan RLS per user, dan kemampuan membaca data saat offline dari cache Hive.

Langkah Pengerjaan

1. Jalankan SQL tabel `notes` + 4 policy RLS + trigger `updated_at`. Simpan di `supabase/setup.sql`.
2. Implementasikan `Note`, `NoteRemoteDataSource`, `NoteLocalDataSource`, dan `NoteRepository` (section 2–5).
3. Implementasikan `NotesProvider` dengan pencarian, filter kategori, dan optimistic delete + rollback (section 6).
4. Ganti HomeScreen sementara dengan HomeScreen final, lalu buat NoteFormScreen (tambah & edit) dan NoteDetailScreen (section 7–8).
5. Perbarui `main.dart` dan pastikan logout membersihkan cache Hive (section 9).
6. Tambahkan dua file unit test, lalu jalankan `flutter test` hingga semua lulus (section 10).
7. Uji RLS dengan dua akun berbeda: catatan akun A tidak boleh terlihat oleh akun B.
8. Uji mode offline: banner muncul, data cache tampil, tambah catatan gagal dengan pesan jelas, hapus catatan di-rollback.
9. Kerjakan minimal satu ide pengembangan dari section 12 (misalnya pin catatan atau realtime).

Output yang Diharapkan

Aplikasi lengkap: login/register, daftar catatan milik sendiri dengan pencarian dan filter, tambah/edit/hapus tersimpan di Supabase, banner mode offline dengan data dari Hive, logout membersihkan cache, flutter analyze bersih, dan flutter test lulus.

Rubrik Penilaian

SQL notes + RLS + trigger benar dan teruji 2 akun (20%), Lapisan data remote/local/repository + fallback offline (25%), NotesProvider: search, filter, optimistic delete + rollback (15%), UI: list, refresh, empty/error/loading, Dismissible, form tambah/edit (20%), Unit test lulus (10%), Fitur pengembangan + laporan (10%).

Kuis Pemahaman

1. Mengapa kolom user_id pada tabel notes diberi 'default auth.uid()'?

A. Agar setiap catatan memiliki id unik

B. Agar database otomatis mengisi pemilik catatan dari token login, sehingga aplikasi tidak perlu mengirim user_id

C. Agar catatan bisa dibaca semua user

D. Agar RLS tidak diperlukan lagi

2. Dalam NoteRepository.getNotes(), apa yang terjadi jika remote melempar PostgrestException?

A. Data diambil dari cache Hive

B. Exception diteruskan (rethrow) karena itu error dari server, bukan masalah koneksi

C. Aplikasi otomatis logout

D. Exception diabaikan dan mengembalikan list kosong

3. Mengapa NoteLocalDataSource menyimpan catatan dengan key berupa id catatan?

A. Karena Hive hanya menerima key String

B. Agar update dan delete satu catatan bisa langsung dengan put(id)/delete(id) tanpa mencari indeks

C. Agar data terenkripsi

D. Agar urutan otomatis berdasarkan tanggal

4. Apa yang dimaksud dengan optimistic delete pada NotesProvider?

A. Menghapus data di server tanpa konfirmasi

B. Menghapus item dari UI lebih dulu, lalu mengirim ke server, dan mengembalikannya (rollback) jika server gagal

C. Menghapus seluruh catatan sekaligus

D. Menunda penghapusan sampai aplikasi ditutup

5. Mengapa loadNotes() dipanggil di dalam WidgetsBinding.instance.addPostFrameCallback pada initState HomeScreen?

A. Agar data dimuat dua kali

B. Karena loadNotes langsung memanggil notifyListeners, dan memanggilnya saat build pertama memicu error 'called during build'

C. Karena Supabase hanya bisa dipanggil setelah frame pertama

D. Agar animasi lebih halus

6. Saat logout, mengapa NotesProvider.clear() dipanggil sebelum AuthProvider.signOut()?

A. Agar logout lebih cepat

B. Agar state dan cache Hive milik user lama sudah bersih sebelum AuthGate berpindah ke LoginScreen, sehingga tidak bocor ke user berikutnya

C. Karena signOut menghapus Hive secara otomatis

D. Urutannya tidak berpengaruh

7. Mengapa NoteDetailScreen menerima noteId lalu memakai context.select, alih-alih menerima objek Note langsung?

A. Karena objek Note tidak bisa dikirim lewat Navigator

B. Agar halaman selalu menampilkan versi terbaru dari provider (misalnya setelah diedit) dan hanya rebuild bila catatan itu berubah

C. Agar memakai lebih sedikit memori

D. Karena context.watch dilarang

8. Di unit test, mengapa dipakai Hive.init(tempDir.path) dan bukan Hive.initFlutter()?

A. Karena Hive.init lebih cepat

B. Karena unit test berjalan tanpa plugin platform (path_provider), jadi folder penyimpanan harus ditentukan manual

C. Karena initFlutter sudah deprecated

D. Karena test tidak boleh menyimpan file

Kunci Jawaban & Pembahasan

1. B — auth.uid() mengembalikan id user dari token request. Dengan default ini, insert cukup mengirim title/content/category, dan policy with check memastikan nilainya cocok.

2. B — PostgrestException berarti server menjawab tetapi menolak (misalnya RLS atau tabel tidak ada). Menyembunyikannya dengan data cache akan menutupi bug yang harus diperbaiki.

3. B — Key yang sama dengan primary key server membuat sinkronisasi cache sederhana: put menimpa versi lama dan delete menghapus tepat satu catatan.

4. B — UI diperbarui seolah-olah server pasti berhasil sehingga terasa instan. Jika gagal, item disisipkan kembali di index semula dan user diberi tahu.

5. B — Menunda sampai frame pertama selesai mencegah perubahan state provider ketika widget tree masih dibangun.

6. B — Begitu signOut selesai, AuthGate langsung membuang HomeScreen. Membersihkan lebih dulu menjamin tidak ada catatan user lama yang tertinggal di memori maupun di disk.

7. B — Objek yang dikirim lewat constructor tidak ikut berubah setelah diedit. Dengan select berdasarkan id, detail reaktif terhadap perubahan catatan tersebut saja.

8. B — initFlutter memakai path_provider untuk mencari folder aplikasi di HP. Di lingkungan test, kita memberi folder sementara secara eksplisit lalu menghapusnya di tearDown.

MODUL 15 · TESTING & DEBUG

Debugging, DevTools & Pengujian Otomatis

Memastikan aplikasi stabil sebelum dirilis. Dimulai dari membaca pesan error dan stack trace, debugPrint, breakpoint, Flutter DevTools, dan flutter analyze, lalu menulis unit test, test dengan fake/mock untuk repository Catatan Kuliah, widget test form login, test Provider, golden test, integration test, dan laporan coverage.

Durasi Pertemuan 15 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management dengan Provider
Modul 14: Proyek Catatan Kuliah Bagian 2 (CRUD & Cache Offline)

Capaian Pembelajaran

- Membaca pesan error dan stack trace Flutter serta memperbaiki lima error yang paling umum (overflow, setState setelah dispose, null check, unbounded height, Provider tidak ditemukan).
- Menggunakan debugPrint, developer.log, breakpoint VS Code, dan Flutter DevTools (Inspector, Performance, Network, Memory) untuk menemukan penyebab bug.
- Menjaga kualitas kode dengan flutter analyze, analysis_options.yaml, dan dart fix.
- Menulis unit test dengan test, expect, group, setUp, matcher, test async, dan test exception.
- Mengisolasi pengujian repository dari Supabase dan Hive menggunakan fake class dan mocktail.
- Menulis widget test (pumpWidget, find, tap, enterText, pump, pumpAndSettle) untuk form login dan halaman yang memakai Provider.
- Menjelaskan fungsi golden test dan integration test, serta mengukur coverage dengan flutter test --coverage.

Daftar Isi Modul

1. Membaca Pesan Error & 5 Error Paling Umum
2. debugPrint, log & Breakpoint di VS Code
3. Flutter DevTools: Inspector, Performance, Network, Memory
4. flutter analyze & Lint
5. Unit Test: test, expect, group, setUp & Matcher
6. Unit Test Lanjutan: Async & Exception
7. Fake & Mock: Menguji Repository Tanpa Supabase
8. Widget Test: Interaksi & Validasi Form Login
9. Menguji Provider (NotesProvider)
10. Golden Test, Integration Test & Coverage
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Membaca Pesan Error & 5 Error Paling Umum

Sebelum belajar alat debugging yang canggih, keterampilan paling penting adalah **membaca pesan error**. Hampir semua error Flutter sebenarnya sudah memberi tahu apa masalahnya, di file mana, dan sering kali juga cara memperbaikinya. Sayangnya, banyak mahasiswa langsung menyalin error ke mesin pencari tanpa membacanya.

Di mana error muncul?

- **Terminal / Debug Console VS Code**: pesan lengkap beserta *stack trace*. Ini sumber informasi utama.
- **Layar aplikasi**: pada mode debug, error saat `build()` menampilkan layar merah dengan teks kuning. Error layout menampilkan garis kuning-hitam.
- **Tab Problems VS Code**: error kompilasi (salah tipe, salah nama) yang muncul sebelum aplikasi dijalankan.

Anatomi pesan error Flutter

Berikut contoh pesan asli yang muncul di Debug Console saat `setState()` dipanggil setelah halaman ditutup:

```
══╡ EXCEPTION CAUGHT BY FLUTTER TEST FRAMEWORK ╞═══════════════════
The following assertion was thrown running a test:
setState() called after dispose(): _ProfilSalahState#7b34c(lifecycle state: defunct, not mounted)
This error happens if you call setState() on a State object for a widget that
no longer appears in the widget tree ...
The preferred solution is to cancel the timer or stop listening to the animation
in the dispose() callback. Another solution is to check the "mounted" property of
this object before calling setState() ...

When the exception was thrown, this was the stack:
#0      State.setState.<anonymous closure> (package:flutter/src/widgets/framework.dart:1163:9)
#1      State.setState (package:flutter/src/widgets/framework.dart:1200:6)
#2      _ProfilSalahState._muat (package:catatan_kuliah/demos/error_demos.dart:71:5)
<asynchronous suspension>
```

Baca dengan urutan berikut:

1. **Judul** (`EXCEPTION CAUGHT BY ...`): bagian Flutter mana yang menangkap error, misalnya *WIDGETS LIBRARY* (saat build) atau *RENDERING LIBRARY* (saat layout).
2. **Kalimat pertama**: inti masalahnya. Di sini: `setState() called after dispose()`.
3. **Paragraf penjelasan**: Flutter sering langsung menyarankan solusi. Di sini: cek properti `mounted`.
4. **Stack trace**: daftar pemanggilan fungsi, paling atas adalah yang terakhir dipanggil. **Cari baris pertama yang berasal dari package proyek Anda** (bukan `package:flutter/`). Pada contoh di atas itu baris `#2` : file `error_demos.dart` baris 71 kolom 5.

> 💡 **Tips**
>
> Di Debug Console VS Code, nama file pada stack trace bisa di-klik (`Cmd/Ctrl` + klik) dan langsung membuka baris yang bermasalah.

Berikut lima error yang paling sering ditemui mahasiswa, lengkap dengan kode penyebab dan perbaikannya.

Error 1: A RenderFlex overflowed by ... pixels

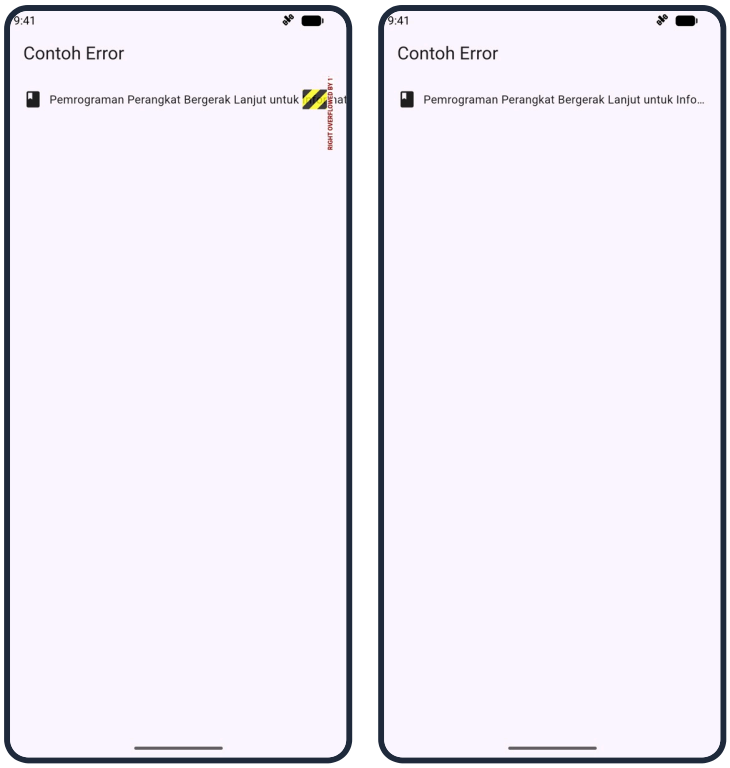
Artinya: anak-anak `Row` / `Column` butuh ruang lebih besar daripada yang tersedia. Muncul garis kuning-hitam di sisi yang meluap.

```
// SALAH: teks panjang di dalam Row tidak punya batas lebar
class KartuMatkulSalah extends StatelessWidget {
  const KartuMatkulSalah({super.key});

  @override
  Widget build(BuildContext context) {
    return const Row(
      children: [
        Icon(Icons.book),
        SizedBox(width: 8),
        Text('Pemrograman Perangkat Bergerak Lanjut untuk Informatika Semester 5'),
      ],
    );
  }
}
```

Pesan di console: `A RenderFlex overflowed by 173 pixels on the right.` Perbaikannya, bungkus anak yang fleksibel dengan `Expanded` (atau `Flexible`) agar ukurannya mengikuti sisa ruang:

```
// BENAR: Expanded memberi Text sisa lebar yang tersedia
class KartuMatkulBenar extends StatelessWidget {
  const KartuMatkulBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return const Row(
      children: [
        Icon(Icons.book),
        SizedBox(width: 8),
        Expanded(
          child: Text(
            'Pemrograman Perangkat Bergerak Lanjut untuk Informatika Semester 5',
            overflow: TextOverflow.ellipsis, // potong dengan "..."
          ),
        ),
      ],
    );
```

```
  }
}
```



Sebelum: teks meluap, muncul garis kuning-hitam di kanan

Sesudah: Expanded + TextOverflow.ellipsis

Untuk `Column` yang terlalu tinggi (misalnya form panjang saat keyboard muncul), solusinya biasanya membungkus `Column` dengan `SingleChildScrollView`.

Error 2: setState() called after dispose()

Artinya: proses async (ambil data, `Future.delayed`, timer) selesai setelah pengguna meninggalkan halaman, lalu kode mencoba memanggil `setState()` pada State yang sudah dibuang.

```dart
class ProfilSalah extends StatefulWidget {
  const ProfilSalah({super.key});

  @override
  State<ProfilSalah> createState() => _ProfilSalahState();
}

class _ProfilSalahState extends State<ProfilSalah> {
  String _nama = 'Memuat...';

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final nama = await ambilProfil(); // butuh 3 detik
```

```
    // Jika pengguna sudah menekan "Back" sebelum 3 detik,
    // State ini sudah di-dispose dan baris berikut memicu error.
    setState(() => _nama = nama);
  }

  @override
  Widget build(BuildContext context) => Text(_nama);
}
```

Perbaikannya: setiap kali ada `await` sebelum `setState()` atau sebelum memakai `context`, cek `mounted` dulu.

```
class ProfilBenar extends StatefulWidget {
  const ProfilBenar({super.key});

  @override
  State<ProfilBenar> createState() => _ProfilBenarState();
}

class _ProfilBenarState extends State<ProfilBenar> {
  String _nama = 'Memuat...';

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final nama = await ambilProfil();
    if (!mounted) return; // widget sudah tidak ada di layar, berhenti
    setState(() => _nama = nama);
  }

  @override
  Widget build(BuildContext context) => Text(_nama);
}
```

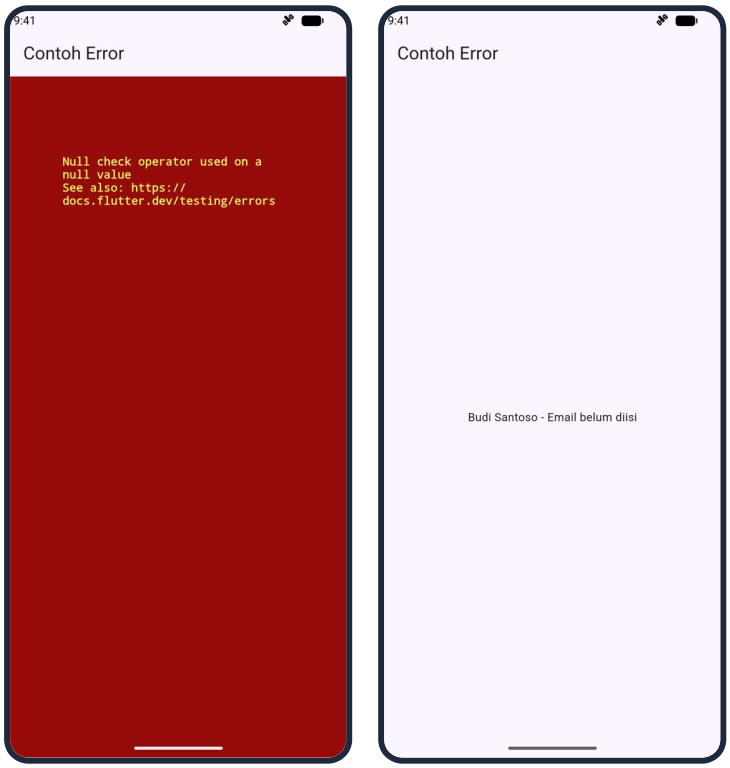
Untuk `Timer` dan `StreamSubscription`, batalkan di `dispose()` dengan `timer.cancel()` atau `subscription.cancel()`.

Error 3: Null check operator used on a null value

Artinya: Anda memakai operator `!` (“saya jamin tidak null”) pada nilai yang ternyata null. Ini error paling sering di aplikasi yang membaca data dari API atau database.

```
class KartuEmail extends StatelessWidget {
  const KartuEmail({super.key, this.mahasiswa});

  final Mahasiswa? mahasiswa;

  @override
  Widget build(BuildContext context) {
    // SALAH: crash jika mahasiswa atau email-nya null
    // return Text(mahasiswa!.email!);
```

```
    // BENAR 1: beri nilai cadangan dengan ?. dan ??
    final email = mahasiswa?.email ?? 'Email belum diisi';

    // BENAR 2: simpan ke variabel lokal, lalu cek null secara eksplisit
    final mhs = mahasiswa;
    if (mhs == null) {
      return const Text('Data mahasiswa tidak ditemukan');
    }
    return Text('${mhs.nama} - $email'); // mhs sudah pasti tidak null
  }
}
```



Versi salah (`mahasiswa!.email!`): layar merah di mode debug

Versi benar: email null diganti teks cadangan

Di build *release*, layar merah ini berubah menjadi area abu-abu kosong. Pengguna tidak melihat pesan apa pun, jadi bug seperti ini harus ditemukan saat debug atau lewat test.

Error 4: Vertical viewport was given unbounded height

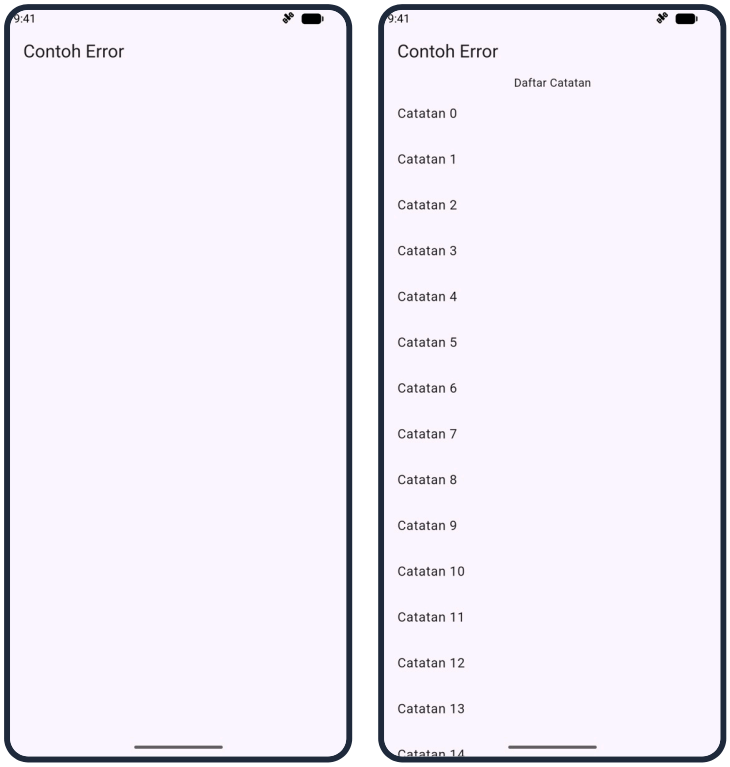
Artinya: widget yang bisa di-scroll (`ListView`, `GridView`) diletakkan di dalam widget yang tidak membatasi tingginya (`Column`, atau `ListView` lain). `ListView` ingin setinggi mungkin, `Column` memberi tinggi tak terbatas, sehingga Flutter bingung.

```
// SALAH: ListView di dalam Column -> "Vertical viewport was given unbounded height"
class DaftarSalah extends StatelessWidget {
  const DaftarSalah({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(
```

```
      children: [
        const Text('Daftar Catatan'),
        ListView.builder(
          itemCount: 20,
          itemBuilder: (context, i) => ListTile(title: Text('Catatan $i')),
        ),
      ],
    );
  }
}
```

Pesan di console menunjuk langsung ke `ListView` yang bermasalah (`The relevant error-causing widget was: ListView ...error_demos.dart:147:18`). Perbaikannya:

```
// BENAR: Expanded memberi ListView tinggi = sisa ruang Column
class DaftarBenar extends StatelessWidget {
  const DaftarBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const Text('Daftar Catatan'),
        Expanded(
          child: ListView.builder(
            itemCount: 20,
            itemBuilder: (context, i) => ListTile(title: Text('Catatan $i')),
          ),
        ),
      ],
    );
  }
}
```



Sebelum: halaman kosong, error hanya terlihat di console

Sesudah: ListView dibungkus Expanded

Alternatif lain adalah `ListView(shrinkWrap: true, physics: NeverScrollableScrollPhysics())`, tetapi ini hanya cocok untuk daftar pendek karena semua item dibangun sekaligus.

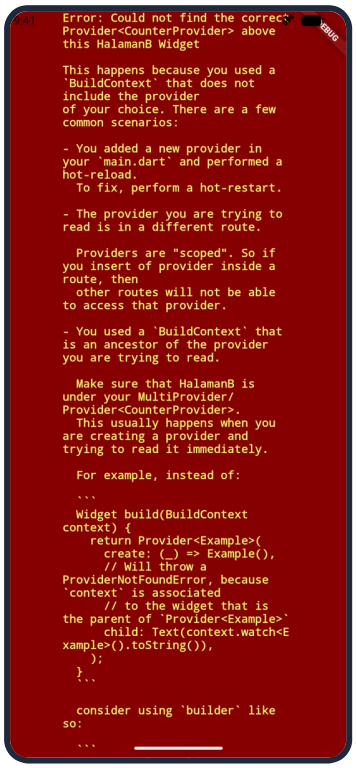
Error 5: Could not find the correct Provider<T> above this Widget

Artinya: widget memanggil `context.watch<T>()` atau `context.read<T>()`, tetapi tidak ada `Provider<T>` di atasnya pada widget tree. Penyebab paling umum: provider dipasang di dalam satu halaman, lalu halaman lain yang dibuka dengan `Navigator.push` mencoba membacanya.

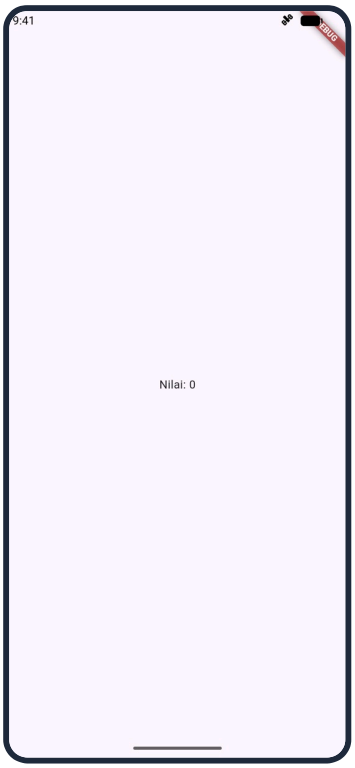
```
// SALAH: provider dipasang DI BAWAH MaterialApp, hanya untuk HalamanA.
// Halaman yang dibuka lewat Navigator.push tidak bisa melihatnya.
class AppSalah extends StatelessWidget {
  const AppSalah({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ChangeNotifierProvider(
        create: (_) => CounterProvider(),
        child: const HalamanA(),
      ),
    );
  }
}
```

Halaman baru dari `Navigator.push` adalah “saudara” `HalamanA` di bawah `MaterialApp`, bukan anaknya, sehingga tidak bisa melihat provider tersebut. Pindahkan provider ke atas `MaterialApp`:

```dart
// BENAR: provider membungkus MaterialApp sehingga semua route bisa membacanya
class AppBenar extends StatelessWidget {
  const AppBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => CounterProvider(),
      child: const MaterialApp(home: HalamanA()),
    );
  }
}
```



AppSalah: setelah membuka Halaman B, pesan error lengkap dengan daftar penyebab



AppBenar: Halaman B bisa membaca CounterProvider

Penyebab lain: lupa memberi tipe generik (`context.read()` tanpa `<CounterProvider>`), atau menambah provider baru di `main.dart` lalu hanya melakukan *hot reload*. Untuk kasus terakhir, lakukan *hot restart* (`Shift+R` atau tombol restart di VS Code).

Ringkasan

Pesan error	Penyebab umum	Perbaikan
RenderFlex overflowed	Isi Row/Column lebih besar dari layar	`Expanded`, `Flexible`, `SingleChildScrollView`
setState() called after dispose()	setState setelah await, halaman sudah ditutup	`if (!mounted) return;`, cancel timer di dispose
Null check operator used on a null value	Operator `!` pada nilai null	`?.`, `??`, cek null dengan variabel lokal
Vertical viewport was given unbounded height	ListView di dalam Column	Bungkus dengan `Expanded`
Could not find the correct Provider	Provider dipasang terlalu bawah	Pindahkan ke atas `MaterialApp`, hot restart

⚠️ **Kesalahan umum**

- Hanya membaca baris terakhir error, padahal inti masalah ada di kalimat pertama.
- Mencari penyebab di file `framework.dart`. Baris dari `package:flutter/` hampir tidak pernah salah; cari baris pertama dari proyek Anda.
- Menambahkan `!` di mana-mana agar error merah di editor hilang. Ini hanya memindahkan error dari waktu kompilasi ke waktu aplikasi berjalan.
- Memperbaiki overflow dengan mengecilkan font. Di HP lain dengan layar lebih kecil, error akan muncul lagi.

2. debugPrint, log & Breakpoint di VS Code

Setelah tahu letak error, langkah berikutnya adalah memahami *kenapa* kode berperilaku salah. Ada dua cara: mencetak isi variabel ke console (logging), atau menghentikan aplikasi di baris tertentu lalu memeriksa semua variabel (breakpoint).

1. debugPrint: cara paling sederhana

`print()` memang bisa dipakai, tetapi lint bawaan Flutter (`avoid_print`) akan memberi peringatan. Gunakan `debugPrint()` dari `package:flutter/foundation.dart`. Bedanya, `debugPrint` membatasi kecepatan output sehingga log panjang tidak terpotong oleh Android.

```dart
import 'dart:developer' as developer;

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

class KeranjangDebug {
  final List<int> sks = [];

  void tambah(int nilaiSks) {
    debugPrint('tambah() dipanggil dengan sks=$nilaiSks');
    sks.add(nilaiSks);
    debugPrint('isi sekarang: $sks, total=${sks.fold(0, (a, b) => a + b)}');
  }
}
```

Output di Debug Console saat `tambah(3)` lalu `tambah(4)` dipanggil:

```
I/flutter ( 5123): tambah() dipanggil dengan sks=3
I/flutter ( 5123): isi sekarang: [3], total=3
I/flutter ( 5123): tambah() dipanggil dengan sks=4
I/flutter ( 5123): isi sekarang: [3, 4], total=7
```

Tulis log yang **informatif**: sebutkan nama fungsi dan nilai variabelnya, bukan sekadar `debugPrint('masuk')`.

2. log dari dart:developer: log yang lebih terstruktur

`developer.log()` mendukung nama kategori (`name`), objek error, dan stack trace. Log ini tampil rapi di tab *Logging* DevTools dan bisa difilter berdasarkan nama.

```dart
Future<void> muatCatatan() async {
  developer.log('Mulai memuat catatan', name: 'catatan.repo');
  try {
    await Future.delayed(const Duration(seconds: 1));
    throw Exception('Timeout ke server');
  } catch (e, stackTrace) {
    developer.log(
      'Gagal memuat catatan',
      name: 'catatan.repo',
      error: e,
```

```
      stackTrace: stackTrace,
      level: 1000, // 1000 = SEVERE
    );
  }
}
```

Poin penting:

- `catch (e, stackTrace)`: parameter kedua menangkap stack trace, sehingga Anda tahu dari mana error berasal.
- `name: 'catatan.repo'`: kategori log. Di DevTools, ketik `catatan` di kolom filter untuk hanya melihat log ini.
- `level: 1000`: tingkat keparahan (800 = INFO, 900 = WARNING, 1000 = SEVERE).

3. kDebugMode dan assert

Log dan pemeriksaan tambahan sebaiknya hanya aktif saat pengembangan. Flutter menyediakan konstanta `kDebugMode` dan pernyataan `assert`:

```dart
void catatDebug(String pesan) {
  // kDebugMode bernilai false di build release,
  // sehingga blok ini dibuang total oleh compiler.
  if (kDebugMode) {
    debugPrint('[DEBUG] $pesan');
  }
}

double hitungRataRata(List<int> nilai) {
  // assert hanya diperiksa di mode debug
  assert(nilai.isNotEmpty, 'List nilai tidak boleh kosong');
  return nilai.reduce((a, b) => a + b) / nilai.length;
}
```

`assert` sangat berguna untuk menangkap "hal yang seharusnya tidak pernah terjadi". Di mode debug, `hitungRataRata([])` langsung berhenti dengan pesan jelas, bukan menghasilkan `NaN` diam-diam.

4. Breakpoint di VS Code

Breakpoint menghentikan aplikasi tepat di satu baris sehingga Anda bisa melihat nilai *semua* variabel tanpa menulis `debugPrint`. Langkahnya:

1. Jalankan aplikasi dengan mode debug: tekan F5 (atau menu *Run* → *Start Debugging*). Jangan pakai `flutter run` di terminal jika ingin breakpoint VS Code bekerja.
2. Klik di sebelah kiri nomor baris sampai muncul titik merah.
3. Lakukan aksi di aplikasi yang menjalankan baris tersebut. Aplikasi akan berhenti dan baris tersebut disorot kuning.
4. Lihat panel **Variables** (nilai variabel lokal), **Watch** (ekspresi yang Anda pantau), dan **Call Stack**.

```dart
int hitungTotalSks(List<int> daftarSks) {
  var total = 0;
  for (final sks in daftarSks) {
    total += sks; // <- pasang breakpoint di baris ini (klik kiri nomor baris)
  }
  return total;
}

class TombolHitung extends StatelessWidget {
  const TombolHitung({super.key});

  @override
  Widget build(BuildContext context) {
    return FilledButton(
      onPressed: () {
        final total = hitungTotalSks([3, 4, 2, 3]);
        // Bisa juga berhenti lewat kode (hanya jika debugger terhubung)
        if (total > 10) developer.debugger(message: 'Total SKS > 10');
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Total SKS: $total')),
        );
      },
      child: const Text('Hitung SKS'),
    );
  }
}
```

Saat berhenti di baris `total += sks;`, gunakan tombol kontrol debug:

Tombol	Pintasan	Fungsi
Continue	F5	Lanjut sampai breakpoint berikutnya
Step Over	F10	Jalankan baris ini, berhenti di baris berikutnya
Step Into	F11	Masuk ke dalam fungsi yang dipanggil di baris ini
Step Out	Shift+F11	Selesaikan fungsi saat ini, kembali ke pemanggil
Restart	Shift+Cmd/Ctrl+F5	Hot restart aplikasi

Breakpoint bersyarat dan logpoint

Klik kanan titik merah lalu pilih *Edit Breakpoint*:

- **Expression**: berhenti hanya jika kondisi benar, misalnya `sks > 3`. Berguna di dalam loop panjang.
- **Log Message** (logpoint): tidak berhenti, hanya mencetak pesan seperti `total sekarang {total}`. Ini seperti `debugPrint` tanpa mengubah kode.

Konfigurasi launch.json

Jika aplikasi butuh argumen tambahan (misalnya file konfigurasi Supabase di Modul 16), buat `.vscode/launch.json` lewat menu *Run* → *Add Configuration*:

`.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Catatan Kuliah (debug)",
      "request": "launch",
      "type": "dart",
      "program": "lib/main.dart",
      "args": ["--dart-define-from-file=config/dev.json"]
    },
    {
      "name": "Catatan Kuliah (profile)",
      "request": "launch",
      "type": "dart",
      "flutterMode": "profile",
      "args": ["--dart-define-from-file=config/dev.json"]
    }
  ]
}
```

> ⚠️ **Kesalahan umum**
>
> - Memasang breakpoint lalu menjalankan aplikasi dengan `flutter run` di terminal biasa. Debugger VS Code tidak terhubung, jadi aplikasi tidak pernah berhenti.
> - Meninggalkan puluhan `debugPrint` di kode yang dikumpulkan. Hapus atau bungkus dengan `if (kDebugMode)`.
> - Mencetak data sensitif (password, token Supabase) ke log. Log Android bisa dibaca oleh siapa pun yang memegang HP lewat `adb logcat`.
> - Menulis `catch (e) {}` kosong. Error “tertelan” dan Anda tidak akan pernah tahu ada masalah. Minimal log errornya.

3. Flutter DevTools: Inspector, Performance, Network, Memory

Flutter DevTools adalah kumpulan alat diagnostik berbasis web yang terhubung ke aplikasi yang sedang berjalan. Jika `debugPrint` dan breakpoint adalah "kaca pembesar", DevTools adalah "ruang kontrol" yang menampilkan widget tree, waktu render tiap frame, lalu lintas jaringan, dan pemakaian memori.

Membuka DevTools

```
# Cara 1: dari VS Code
# Jalankan aplikasi dengan F5, lalu Cmd/Ctrl+Shift+P
# ketik "Flutter: Open DevTools" dan pilih tab yang diinginkan

# Cara 2: dari terminal
flutter run
# setelah aplikasi berjalan, terminal menampilkan alamat DevTools, contoh:
# The Flutter DevTools debugger and profiler on sdk gphone64 arm64 is available at:
# http://127.0.0.1:9100?uri=http://127.0.0.1:52341/abc123=/
# Klik/salin alamat tersebut ke browser

# Cara 3: jalankan DevTools mandiri lalu tempel alamat VM service
dart devtools
```

Tab	Dipakai untuk	Mode aplikasi
Inspector	Melihat widget tree, ukuran, padding, constraints	debug
Performance	Mencari frame lambat (jank)	**profile**
CPU Profiler	Fungsi mana yang paling banyak makan waktu CPU	profile
Memory	Pemakaian RAM, mencari memory leak	debug/profile
Network	Request HTTP: URL, status, header, body, durasi	debug/profile
Logging	Output `debugPrint`, `developer.log`, event framework	debug

1. Inspector: memeriksa widget tree dan layout

Inspector menampilkan pohon widget yang *benar-benar* sedang dirender. Fitur yang paling sering dipakai:

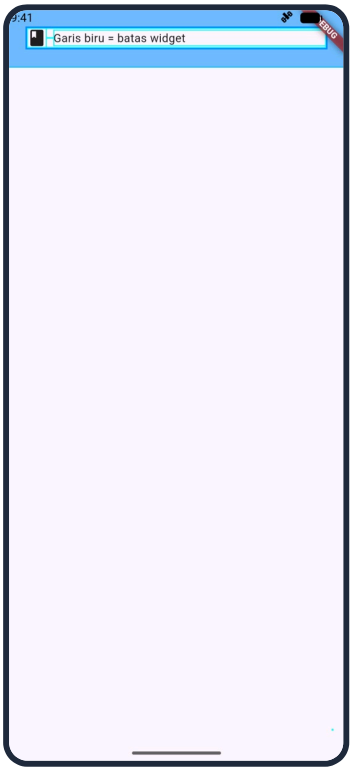
- **Select Widget Mode**: klik ikon panah, lalu sentuh elemen di emulator. Inspector melompat ke widget tersebut dan VS Code membuka baris kodenya.
- **Layout Explorer**: untuk `Row` / `Column`, menampilkan ukuran setiap anak, `flex`, dan alignment. Sangat membantu saat memperbaiki RenderFlex overflow.

- **Show Guidelines** (debug paint): menggambar garis batas, padding, dan baseline semua widget di layar.

Debug paint juga bisa diaktifkan lewat kode, berguna jika DevTools belum terbuka:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true; // tampilkan garis batas semua widget
  runApp(
    const MaterialApp(
      home: Scaffold(
        body: Padding(
          padding: EdgeInsets.all(24),
          child: Row(
            children: [
              Icon(Icons.book),
              SizedBox(width: 8),
              Expanded(child: Text('Garis biru = batas widget')),
            ],
          ),
        ),
      ),
    ),
  );
}
```



debugPaintSizeEnabled: setiap widget digambar dengan garis batas, padding, dan arah layout

Jangan lupa menghapus baris `debugPaintSizeEnabled = true` setelah selesai.

2. Performance: mencari jank

Layar 60 Hz menggambar 60 frame per detik, artinya setiap frame punya jatah sekitar **16 ms**. Jika satu frame butuh lebih lama, animasi terlihat patah-patah (*jank*). Penyebab paling umum: pekerjaan berat di dalam `build()`.

```
// SALAH: perhitungan berat di dalam build() -> frame macet (jank)
int hitungPrimaSampai(int batas) {
  var jumlah = 0;
  for (var n = 2; n <= batas; n++) {
    var prima = true;
    for (var d = 2; d * d <= n; d++) {
      if (n % d == 0) {
        prima = false;
        break;
      }
    }
    if (prima) jumlah++;
  }
  return jumlah;
}

class StatistikSalah extends StatelessWidget {
  const StatistikSalah({super.key});

  @override
  Widget build(BuildContext context) {
    final jumlah = hitungPrimaSampai(5000000); // dihitung ulang tiap rebuild!
    return Text('Ada $jumlah bilangan prima');
  }
}
```

Di tab Performance, frame-frame yang lambat tampil sebagai batang merah. Klik batang tersebut untuk melihat bahwa `hitungPrimaSampai` menghabiskan ratusan milidetik di thread UI. Perbaikannya, pindahkan perhitungan ke isolate lain dengan `Isolate.run` dan simpan Future-nya di `initState`:

```
// BENAR: jalankan di isolate lain, hasilnya ditampilkan lewat FutureBuilder
class StatistikBenar extends StatefulWidget {
  const StatistikBenar({super.key});

  @override
  State<StatistikBenar> createState() => _StatistikBenarState();
}

class _StatistikBenarState extends State<StatistikBenar> {
  // Future dibuat SEKALI di initState, bukan di build()
  late final Future<int> _hasil;

  @override
  void initState() {
    super.initState();
    _hasil = Isolate.run(() => hitungPrimaSampai(5000000));
```

```
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<int>(
      future: _hasil,
      builder: (context, snapshot) {
        if (!snapshot.hasData) return const CircularProgressIndicator();
        return Text('Ada ${snapshot.data} bilangan prima');
      },
    );
  }
}
```

Ukur performa dengan mode **profile**, bukan debug. Mode debug sengaja lambat karena banyak pemeriksaan tambahan, sehingga hasilnya menyesatkan.

```
# Mode profile hanya berjalan di HP fisik atau emulator, tidak di web debug
flutter run --profile
```

Sebagai alternatif cepat tanpa DevTools, aktifkan grafik performa langsung di atas aplikasi:

```
class AppDenganOverlay extends StatelessWidget {
  const AppDenganOverlay({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      // Menampilkan grafik waktu frame (UI & raster) di atas aplikasi
      showPerformanceOverlay: true,
      home: Scaffold(body: Center(child: Text('Cek grafik di atas'))),
    );
  }
}
```

Performance overlay: grafik atas = thread raster (GPU), grafik bawah = thread UI. Garis horizontal menandai kelipatan 16 ms; penanda frame terbaru hijau jika di bawah batas dan merah jika melewatinya.

3. Network: memeriksa request HTTP

Tab Network mencatat setiap request yang dibuat dengan `package:http`, `dio`, maupun client Supabase. Untuk tiap request Anda bisa melihat:

- Method dan URL lengkap (apakah query `?select=*` sudah benar?).
- Status code: `200` sukses, `401` belum login / token kedaluwarsa, `404` URL salah, `500` error server.
- Header request (apakah header `Authorization` terkirim?) dan body response (JSON yang benar-benar dikirim server).
- Durasi: request yang butuh 3 detik akan terasa lambat bagi pengguna.

Saat Catatan Kuliah gagal memuat data, buka tab ini *sebelum* menebak-nebak. Jika status `200` tetapi daftar tetap kosong, masalahnya ada di parsing JSON atau RLS, bukan koneksi.

4. Memory: mencari memory leak

Memory leak terjadi jika objek yang sudah tidak dipakai tetap tersimpan di RAM. Di Flutter, penyebab paling umum adalah controller yang tidak di-`dispose()`:

```
class HalamanCari extends StatefulWidget {
  const HalamanCari({super.key});

  @override
  State<HalamanCari> createState() => _HalamanCariState();
```

```dart
}

class _HalamanCariState extends State<HalamanCari> {
  final _controller = TextEditingController();
  final _scrollController = ScrollController();

  @override
  void dispose() {
    // Tanpa dua baris ini, controller tetap tinggal di memori
    // setiap kali halaman dibuka-tutup (memory leak).
    _controller.dispose();
    _scrollController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: TextField(controller: _controller)),
      body: ListView.builder(
        controller: _scrollController,
        itemCount: 100,
        itemBuilder: (context, i) => ListTile(title: Text('Hasil $i')),
      ),
    );
  }
}
```

Cara memeriksanya di tab Memory:

1. Buka tab Memory, klik *Take heap snapshot* (snapshot A).
2. Buka lalu tutup `HalamanCari` sebanyak 10 kali.
3. Klik *GC* (paksa garbage collection), lalu ambil snapshot B.
4. Bandingkan (*Diff*) snapshot B dengan A dan cari `TextEditingController`. Jika jumlahnya bertambah 10, ada leak.

Objek yang wajib di-dispose: `TextEditingController`, `ScrollController`, `AnimationController`, `FocusNode`, `TabController`, `StreamSubscription` (cancel), dan `Timer` (cancel).

⚠️ **Kesalahan umum**

- Mengukur performa di mode debug lalu menyimpulkan “Flutter lambat”. Selalu ukur dengan `flutter run --profile`.
- Membuat `Future` langsung di parameter `FutureBuilder(future: ambilData())`. Setiap rebuild memicu request baru; simpan Future di `initState`.
- Lupa memanggil `super.dispose()` di akhir method `dispose()`.
- Membiarkan `showPerformanceOverlay: true` atau `debugPaintSizeEnabled = true` sampai build rilis.

4. flutter analyze & Lint

Bug yang paling murah adalah bug yang tertangkap *sebelum* aplikasi dijalankan. Dart menyediakan **analyzer** yang membaca kode Anda dan melaporkan error, peringatan, dan saran gaya penulisan (lint). Biasakan menjalankan `flutter analyze` sebelum commit dan sebelum mengumpulkan tugas.

Menjalankan flutter analyze

```
# Dari folder root proyek
flutter analyze

# Hanya folder tertentu
flutter analyze lib/pages
```

Misalkan ada kode "gaya lama" seperti ini:

```dart
import 'package:flutter/material.dart';

class Contoh extends StatelessWidget {
  Contoh({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    var judul = 'Catatan';
    print(judul);
    return Container(
      child: Text(judul),
    );
  }
}
```

Hasil `flutter analyze` (output asli):

```
info • Constructors in '@immutable' classes should be declared as 'const' • lib/contoh.dart:4:3 •
prefer_const_constructors_in_immutables
info • Parameter 'key' could be a super parameter • lib/contoh.dart:4:3 • use_super_parameters
info • Don't invoke 'print' in production code. Try using a logging framework • lib/contoh.dart:9:5
• avoid_print
info • Unnecessary instance of 'Container' • lib/contoh.dart:10:12 • avoid_unnecessary_containers

4 issues found.
```

Setiap baris berisi: **tingkat** (`error`, `warning`, `info`), **pesan**, **lokasi** (file:baris:kolom), dan **nama aturan**. Nama aturan bisa dicari di dart.dev/tools/linter-rules untuk penjelasan lengkap dan contoh.

Tingkat	Artinya	Wajib diperbaiki?
`error`	Kode tidak bisa dikompilasi (salah tipe, nama tidak ada)	Ya, aplikasi tidak akan jalan
`warning`	Kode jalan tetapi hampir pasti salah (variabel tidak dipakai, import tidak terpakai)	Ya
`info`	Saran gaya dan praktik terbaik dari lint	Sangat disarankan; target tugas: 0 issue

Memperbaiki otomatis dengan dart fix

Banyak lint bisa diperbaiki otomatis. Lihat dulu apa yang akan diubah, baru terapkan:

```
# Pratinjau perubahan
dart fix --dry-run

# Terapkan semua perbaikan
dart fix --apply

# Rapikan format (indentasi, spasi, koma)
dart format lib test
```

Pada contoh di atas, `dart fix --apply` mengubah konstruktor menjadi `const Contoh({super.key});`. Lint `avoid_print` dan `avoid_unnecessary_containers` harus Anda perbaiki sendiri (ganti dengan `debugPrint` dan hapus `Container`).

Mengatur aturan di analysis_options.yaml

Proyek baru dari `flutter create` sudah memakai paket `flutter_lints`. Anda bisa menambah aturan yang lebih ketat atau menaikkan tingkat suatu aturan:

`analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml

analyzer:
  errors:
    # Naikkan avoid_print dari info menjadi error
    avoid_print: error
  exclude:
    - "**/*.g.dart" # file hasil generator (misalnya adapter Hive)
    - build/**
    - android/**
    - ios/**
    - web/**
    - windows/**
    - macos/**
    - linux/**

linter:
  rules:
```

```
    prefer_single_quotes: true      # pakai 'kutip satu'
    prefer_final_locals: true       # variabel lokal yang tidak diubah wajib final
    always_declare_return_types: true
    unawaited_futures: true         # Future yang lupa di-await akan ditandai
```

Aturan `unawaited_futures` sangat berguna untuk aplikasi Supabase/Hive: ia menandai pemanggilan seperti `repository.addNote(...)` yang lupa diberi `await`, sehingga error-nya tidak tertangkap oleh `try/catch`.

Mengabaikan lint secara sengaja

Kadang ada alasan kuat untuk melanggar aturan. Abaikan hanya di baris itu dan tulis alasannya:

```dart
// Skrip seeding data hanya dijalankan developer dari terminal
// ignore: avoid_print
void cetakRingkasan(int jumlah) => print('Berhasil membuat $jumlah catatan');
```

Jangan menulis `// ignore_for_file:` di atas file hanya agar daftar issue terlihat bersih.

> ⚠️ **Kesalahan umum**
>
> - Mengabaikan semua `info` karena “cuma saran”. Lint seperti `use_build_context_synchronously` menandai bug sungguhan (memakai `context` setelah `await` tanpa cek `mounted`).
> - Menghapus baris `include: package:flutter_lints/flutter.yaml` agar warning hilang.
> - Menjalankan `dart fix --apply` tanpa commit terlebih dahulu. Jika hasilnya tidak sesuai, Anda tidak bisa kembali dengan mudah.

5. Unit Test: test, expect, group, setUp & Matcher

Debugging mencari bug yang *sudah* terjadi. Testing mencegah bug yang *akan* terjadi. Dengan test otomatis, setiap kali Anda mengubah kode, cukup jalankan satu perintah untuk memastikan fitur lama tidak rusak (regresi).

Piramida pengujian Flutter

Jenis	Yang diuji	Kecepatan	Butuh emulator?	Jumlah ideal
Unit test	Satu fungsi/class: validator, perhitungan, repository, provider	Milidetik	Tidak	Paling banyak
Widget test	Satu halaman/widget: tampilan dan interaksi	Ratusan ms	Tidak	Sedang
Integration test	Aplikasi utuh di perangkat sungguhan	Menit	Ya	Sedikit, untuk alur penting

Struktur folder test

Semua test berada di folder `test/` dan nama filenya **wajib** diakhiri `_test.dart`. Biasakan struktur folder test meniru folder `lib/`:

```
catatan_kuliah/
├── lib/
|   ├── utils/ipk.dart
|   ├── utils/validators.dart
|   └── models/keranjang_krs.dart
└── test/
    ├── unit/ipk_test.dart
    ├── unit/validators_test.dart
    └── unit/keranjang_krs_test.dart
```

Package `flutter_test` sudah otomatis ada di `dev_dependencies` proyek baru, jadi tidak perlu dipasang.

Test pertama: fungsi hitungIpk

Agar mudah diuji, letakkan logika di file terpisah di `lib/`, bukan di dalam widget:

`lib/utils/ipk.dart`

```dart
// lib/utils/ipk.dart
double hitungIpk(List<int> bobot, List<int> sks) {
  if (bobot.length != sks.length) {
    throw ArgumentError('Jumlah bobot dan SKS harus sama');
```

```
  }
  if (bobot.isEmpty) return 0.0;

  var totalBobot = 0;
  var totalSks = 0;
  for (var i = 0; i < bobot.length; i++) {
    totalBobot += bobot[i] * sks[i];
    totalSks += sks[i];
  }
  return double.parse((totalBobot / totalSks).toStringAsFixed(2));
}

String predikat(double ipk) {
  if (ipk >= 3.51) return 'Dengan Pujian';
  if (ipk >= 3.01) return 'Sangat Memuaskan';
  if (ipk >= 2.76) return 'Memuaskan';
  return 'Cukup';
}
```

Lalu tulis test-nya. Ganti `catatan_kuliah` pada import dengan nama proyek Anda (baris `name:` di `pubspec.yaml`).

`test/unit/ipk_test.dart`

```
// test/unit/ipk_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/utils/ipk.dart';

void main() {
  group('hitungIpk', () {
    test('menghitung IPK normal dengan benar', () {
      // Arrange: siapkan data
      final bobot = [4, 4, 3, 4]; // A, A, B, A
      final sks = [3, 3, 3, 3];

      // Act: jalankan fungsi yang diuji
      final hasil = hitungIpk(bobot, sks);

      // Assert: periksa hasilnya
      expect(hasil, 3.75);
    });

    test('daftar kosong menghasilkan 0.0', () {
      expect(hitungIpk([], []), 0.0);
    });

    test('SKS berbeda ikut menjadi bobot', () {
      // (4 x 4 + 2 x 2) / 6 = 3.33
      expect(hitungIpk([4, 2], [4, 2]), 3.33);
    });
  });

  group('predikat', () {
    test('IPK 3.75 berpredikat Dengan Pujian', () {
      expect(predikat(3.75), 'Dengan Pujian');
    });
```

```
    test('batas bawah tiap predikat', () {
      expect(predikat(3.51), 'Dengan Pujian');
      expect(predikat(3.01), 'Sangat Memuaskan');
      expect(predikat(2.76), 'Memuaskan');
      expect(predikat(2.0), 'Cukup');
    });
  });
}
```

Penjelasan:

- `test('deskripsi', () { ... })` mendefinisikan satu kasus uji. Tulis deskripsi sebagai kalimat yang menjelaskan perilaku, karena kalimat inilah yang muncul saat test gagal.
- `expect(nilaiAktual, nilaiHarapan)` adalah inti test. Jika keduanya berbeda, test gagal.
- `group()` mengelompokkan test yang berkaitan. Nama grup ditambahkan di depan nama test pada laporan.
- Pola **Arrange-Act-Assert**: siapkan data, jalankan fungsi, periksa hasil. Pola ini membuat test mudah dibaca.

Menjalankan test

```
# Semua test di folder test/
flutter test

# Satu file saja
flutter test test/unit/ipk_test.dart

# Hanya test yang namanya mengandung kata tertentu
flutter test --plain-name "predikat"

# Output lebih rinci (satu baris per test)
flutter test --reporter expanded
```

Contoh output saat semua lulus dan saat ada yang gagal:

```
# Semua lulus
00:01 +5: All tests passed!

# Ada yang gagal (expect(hasil, 3.8) padahal hasilnya 3.75)
00:00 +0 -1: hitungIpk menghitung IPK normal dengan benar [E]
  Expected: <3.8>
    Actual: <3.75>

  package:matcher                                     expect
  package:flutter_test/src/widget_tester.dart 473:18  expect
  test/unit/ipk_test.dart 16:7                        main.<fn>.<fn>
...
00:00 +4 -1: Some tests failed.
```

`+4 -1` berarti 4 lulus, 1 gagal. Laporan menunjukkan nilai yang diharapkan, nilai aktual, dan baris test yang gagal. Di VS Code, Anda juga bisa menekan tombol ▶ *Run* yang muncul di atas setiap

`test()` atau membuka panel *Testing* (ikon tabung kimia).

setUp dan tearDown

Jika beberapa test membutuhkan objek yang sama, jangan membuatnya sekali di luar lalu dipakai bersama, karena perubahan di satu test akan bocor ke test lain. Gunakan `setUp()` yang dijalankan ulang **sebelum setiap test**.

lib/models/keranjang_krs.dart

```dart
// lib/models/keranjang_krs.dart
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;

  const MataKuliah(this.kode, this.nama, this.sks);
}

class KeranjangKrs {
  static const batasSks = 24;
  final List<MataKuliah> _items = [];

  List<MataKuliah> get items => List.unmodifiable(_items);
  int get totalSks => _items.fold(0, (total, mk) => total + mk.sks);

  void tambah(MataKuliah mk) {
    if (_items.any((item) => item.kode == mk.kode)) {
      throw StateError('${mk.nama} sudah ada di KRS');
    }
    if (totalSks + mk.sks > batasSks) {
      throw StateError('Total SKS melebihi $batasSks');
    }
    _items.add(mk);
  }

  void hapus(String kode) => _items.removeWhere((mk) => mk.kode == kode);

  void kosongkan() => _items.clear();
}
```

test/unit/keranjang_krs_test.dart

```dart
// test/unit/keranjang_krs_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/models/keranjang_krs.dart';

void main() {
  late KeranjangKrs keranjang;

  const mobile = MataKuliah('IF301', 'Pemrograman Mobile', 3);
  const basdat = MataKuliah('IF205', 'Basis Data', 4);

  // Dijalankan SEBELUM setiap test: keranjang selalu baru dan kosong
  setUp(() {
    keranjang = KeranjangKrs();
```

```
  });

  test('keranjang baru kosong', () {
    expect(keranjang.items, isEmpty);
    expect(keranjang.totalSks, 0);
  });

  test('menambah mata kuliah menambah total SKS', () {
    keranjang.tambah(mobile);
    keranjang.tambah(basdat);

    expect(keranjang.items, hasLength(2));
    expect(keranjang.totalSks, 7);
  });

  test('menghapus mata kuliah berdasarkan kode', () {
    keranjang.tambah(mobile);
    keranjang.hapus('IF301');

    expect(keranjang.items, isEmpty);
  });

  test('daftar items tidak bisa diubah dari luar', () {
    expect(() => keranjang.items.add(mobile), throwsUnsupportedError);
  });
}
```

- `late KeranjangKrs keranjang;` dideklarasikan di luar, lalu diisi ulang di `setUp`. Test kedua tidak terpengaruh mata kuliah yang ditambahkan di test lain.
- `tearDown()` adalah kebalikannya: dijalankan setelah setiap test, misalnya untuk menutup box Hive atau menghapus file sementara.
- `setUpAll()` / `tearDownAll()` dijalankan sekali saja untuk seluruh file, cocok untuk pekerjaan mahal seperti membuka database tes.

Matcher: cara membandingkan yang lebih ekspresif

Argumen kedua `expect` bisa berupa nilai biasa atau **matcher**. Matcher membuat test lebih mudah dibaca dan pesan gagalnya lebih jelas.

`test/unit/matchers_test.dart`

```dart
// test/unit/matchers_test.dart
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('contoh matcher yang sering dipakai', () {
    final nilai = [80, 90, 75];
    const nama = 'Pemrograman Mobile';

    expect(nilai.length, equals(3)); // sama dengan 3
    expect(nilai, contains(90)); // list berisi 90
    expect(nilai, hasLength(3)); // panjang list 3
    expect(nilai, isNot(contains(100))); // TIDAK berisi 100
    expect(nama, startsWith('Pemrograman'));
    expect(nama, contains('Mobile'));
```

```
    expect(nama, isA<String>()); // bertipe String
    expect(0.1 + 0.2, closeTo(0.3, 0.0001)); // untuk angka desimal
    expect(85, greaterThan(80));
    expect(85, inInclusiveRange(80, 100));
    expect(<String>[], isEmpty);
    expect(nilai, everyElement(greaterThanOrEqualTo(75)));
    expect(nilai, orderedEquals([80, 90, 75]));
    expect(nilai, unorderedEquals([75, 80, 90]));
  });
}
```

Matcher	Lulus jika
`equals(x)` atau langsung `x`	Nilai sama dengan `x` (list dibandingkan isinya)
`isTrue`, `isFalse`, `isNull`, `isNotNull`	Sesuai namanya
`isEmpty`, `isNotEmpty`, `hasLength(n)`	Untuk String, List, Map
`contains(x)`	String memuat teks / List memuat elemen
`isA<T>()`	Objek bertipe `T`
`closeTo(x, delta)`	Angka desimal mendekati `x`
`greaterThan`, `lessThan`, `inInclusiveRange`	Perbandingan angka
`isNot(m)`	Kebalikan matcher `m`
`throwsA(m)`, `throwsArgumentError`, `throwsStateError`	Fungsi melempar error (bagian berikutnya)

Contoh nyata: menguji validator form login

Validator yang dipakai form login di Modul 13 sebaiknya diletakkan di file sendiri agar bisa diuji tanpa UI:

`lib/utils/validators.dart`

```
// lib/utils/validators.dart
final _polaEmail = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');

String? validasiEmail(String? value) {
  if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
  if (!_polaEmail.hasMatch(value.trim())) return 'Format email tidak valid';
  return null;
}
```

```
String? validasiPassword(String? value) {
  if (value == null || value.isEmpty) return 'Password wajib diisi';
  if (value.length < 8) return 'Password minimal 8 karakter';
  return null;
}
```

test/unit/validators_test.dart

```
// test/unit/validators_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/utils/validators.dart';

void main() {
  group('validasiEmail', () {
    test('email kosong ditolak', () {
      expect(validasiEmail(''), 'Email wajib diisi');
      expect(validasiEmail(null), 'Email wajib diisi');
      expect(validasiEmail('   '), 'Email wajib diisi');
    });

    test('format salah ditolak', () {
      expect(validasiEmail('budi'), 'Format email tidak valid');
      expect(validasiEmail('budi@kampus'), 'Format email tidak valid');
    });

    test('email benar lolos (mengembalikan null)', () {
      expect(validasiEmail('budi@mhs.unsoed.ac.id'), isNull);
    });
  });

  group('validasiPassword', () {
    test('kurang dari 8 karakter ditolak', () {
      expect(validasiPassword('abc123'), contains('minimal 8'));
    });

    test('8 karakter atau lebih lolos', () {
      expect(validasiPassword('rahasia123'), isNull);
    });
  });
}
```

Perhatikan bahwa setiap aturan diuji dari dua sisi: **kasus positif** (input benar lolos) dan **kasus negatif** (input salah ditolak). Juga uji **kasus batas**: string berisi spasi saja, nilai `null`, panjang tepat 8 karakter.

⚠️ **Kesalahan umum**

- Nama file tidak diakhiri `_test.dart`, sehingga `flutter test` tidak menemukannya.
- Import memakai path relatif ke `lib/` (`import '../../lib/utils/ipk.dart'`). Selalu gunakan `package:nama_proyek/...`.
- Membandingkan `double` dengan `==`: `0.1 + 0.2` tidak sama persis dengan `0.3`. Gunakan `closeTo`.
- Satu test menguji terlalu banyak hal. Jika gagal, Anda sulit tahu bagian mana yang salah. Lebih baik banyak test kecil.
- Test saling bergantung (test B hanya lulus jika test A dijalankan dulu). Gunakan `setUp` agar setiap test mulai dari kondisi bersih.

6. Unit Test Lanjutan: Async & Exception

Kode aplikasi nyata penuh dengan `Future` (ambil data dari Supabase, baca Hive) dan error (input tidak valid, koneksi putus). Bagian ini membahas cara menguji keduanya.

1. Menguji Future

Cara paling mudah: jadikan fungsi test `async`, lalu `await` hasilnya seperti di kode biasa. Alternatifnya, gunakan matcher `completion()`.

`test/unit/async_test.dart`

```dart
// test/unit/async_test.dart
import 'package:flutter_test/flutter_test.dart';

Future<String> ambilNamaMahasiswa(String nim) async {
  await Future.delayed(const Duration(milliseconds: 100));
  if (nim.isEmpty) throw ArgumentError('NIM kosong');
  return 'Budi Santoso';
}

Stream<int> hitungMundur(int dari) async* {
  for (var i = dari; i >= 0; i--) {
    yield i;
  }
}

void main() {
  test('cara 1: async/await di dalam test', () async {
    final nama = await ambilNamaMahasiswa('H1D021001');
    expect(nama, 'Budi Santoso');
  });

  test('cara 2: matcher completion', () {
    expect(ambilNamaMahasiswa('H1D021001'), completion('Budi Santoso'));
  });

  test('Future yang gagal diuji dengan expectLater + throwsA', () async {
    await expectLater(
      ambilNamaMahasiswa(''),
      throwsA(isA<ArgumentError>()),
    );
  });

  test('Stream mengeluarkan nilai berurutan', () {
    expect(hitungMundur(3), emitsInOrder([3, 2, 1, 0, emitsDone]));
  });
}
```

Penjelasan:

- **Cara 1** (`async` / `await`) paling mudah dibaca dan paling sering dipakai.
- **Cara 2** `completion(x)`: test menunggu Future selesai lalu membandingkan hasilnya dengan `x`.

- Untuk Future yang **gagal**, gunakan `await expectLater(future, throwsA(...))`. Kata `await` penting: tanpa itu test bisa selesai sebelum Future-nya gagal.
- `emitsInOrder` menguji Stream: nilai apa saja yang keluar dan urutannya. `emitsDone` memastikan Stream sudah ditutup.

2. Menguji exception pada fungsi biasa

Test juga harus memastikan kode *menolak* input yang salah. Kuncinya: bungkus pemanggilan dengan `() =>` agar `expect` yang menjalankan fungsinya dan menangkap error.

`test/unit/exception_test.dart`

```dart
// test/unit/exception_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/models/keranjang_krs.dart';
import 'package:catatan_kuliah/utils/ipk.dart';

void main() {
  test('hitungIpk melempar ArgumentError jika panjang list beda', () {
    // Bungkus pemanggilan dengan () => supaya error ditangkap oleh expect
    expect(() => hitungIpk([4, 3], [3]), throwsArgumentError);
  });

  test('menambah matkul yang sama dua kali melempar StateError', () {
    final keranjang = KeranjangKrs();
    const mk = MataKuliah('IF301', 'Pemrograman Mobile', 3);
    keranjang.tambah(mk);

    expect(() => keranjang.tambah(mk), throwsStateError);
  });

  test('pesan error melebihi 24 SKS bisa diperiksa', () {
    final keranjang = KeranjangKrs();
    for (var i = 0; i < 6; i++) {
      keranjang.tambah(MataKuliah('MK$i', 'Matkul $i', 4)); // 24 SKS
    }

    expect(
      () => keranjang.tambah(const MataKuliah('MK9', 'Matkul 9', 2)),
      throwsA(
        isA<StateError>().having(
          (e) => e.message,
          'message',
          contains('melebihi 24'),
        ),
      ),
    );
  });
}
```

- `throwsArgumentError`, `throwsStateError`, `throwsUnsupportedError`, `throwsException` adalah matcher siap pakai untuk tipe error umum.
- `throwsA(isA<StateError>())` adalah bentuk umumnya, bisa dipakai untuk class exception buatan sendiri, misalnya `throwsA(isA<AuthException>())`.

- `.having((e) => e.message, 'message', contains(...))` memeriksa isi properti error. Berguna jika satu fungsi bisa melempar beberapa pesan berbeda.

Perbedaan kecil yang sering membuat bingung

```
// SALAH: hitungIpk dijalankan SEBELUM expect, error tidak tertangkap
// dan test gagal dengan pesan "ArgumentError" mentah.
// expect(hitungIpk([4], []), throwsArgumentError);

// BENAR: expect yang menjalankan fungsi di dalam closure
// expect(() => hitungIpk([4], []), throwsArgumentError);

// BENAR untuk fungsi async: future langsung, tanpa closure, dan di-await
// await expectLater(repository.addNote('', 'isi'), throwsArgumentError);
```

3. Ringkasan pola test async dan error

Yang diuji	Pola
Future berhasil	`expect(await f(), hasil)`
Future gagal	`await expectLater(f(), throwsA(isA<E>()))`
Fungsi biasa melempar error	`expect(() => f(), throwsA(isA<E>()))`
Stream	`expect(stream, emitsInOrder([a, b, emitsDone]))`
Fungsi TIDAK melempar error	`expect(() => f(), returnsNormally)`

> ⚠️ **Kesalahan umum**
>
> - Lupa `async` pada fungsi test padahal di dalamnya ada `await` (error kompilasi), atau lupa `await` pada `expectLater` (test lulus palsu).
> - Memakai `Future.delayed` berdetik-detik di kode yang diuji. Test jadi lambat. Pada widget test, waktu bisa “dipercepat” dengan `tester.pump(Duration)`.
> - Menguji fungsi yang langsung memanggil Supabase. Test menjadi lambat, butuh internet, dan mengotori database. Solusinya ada di bagian berikutnya: *fake* dan *mock*.

7. Fake & Mock: Menguji Repository Tanpa Supabase

Di Modul 14, `NoteRepository` mengambil catatan dari Supabase dan menyimpannya ke cache Hive. Bagaimana mengujinya tanpa internet, tanpa akun Supabase, dan tanpa mengotori database? Jawabannya: ganti sumber data asli dengan **pengganti palsu** yang kita kendalikan penuh.

> ℹ️ **Versi ringkas, bukan salinan kode proyek**
>
> Agar setiap contoh di modul ini bisa dijalankan sendiri, class `Note`, `NoteRepository`, dan data source di bawah sengaja dibuat lebih sederhana daripada kode proyek Modul 14 (nama method dan field bisa berbeda). Konsepnya sama persis. Test untuk kode proyek yang sesungguhnya ada di Modul 14 section *Unit Test* (`test/note_repository_test.dart` dan `test/note_local_data_source_test.dart`).

Kenapa butuh pengganti?

- **Cepat**: tidak ada request jaringan, ratusan test selesai dalam hitungan detik.
- **Stabil**: hasil test tidak bergantung pada koneksi atau isi database hari ini.
- **Bisa mensimulasikan kegagalan**: kondisi "offline" sulit dibuat dengan server sungguhan, tetapi mudah dengan pengganti palsu.

Istilah	Arti	Contoh
Fake	Implementasi sederhana yang benar-benar berfungsi, ditulis tangan	Data disimpan di `List`, bukan di Supabase
Mock	Objek yang perilakunya diatur per test dan bisa diperiksa pemanggilannya	`when(...).thenAnswer(...)`, `verify(...)` dengan mocktail

Langkah 1: kode harus bergantung pada abstraksi

Pengganti hanya bisa dipasang jika repository tidak membuat koneksi Supabase/Hive sendiri, melainkan **menerimanya dari luar** lewat konstruktor (*dependency injection*). Berikut versi sederhana arsitektur Modul 14:

`lib/models/note.dart`

```dart
// lib/models/note.dart
class Note {
  final String id;
  final String title;
```

```
  final String content;

  const Note({required this.id, required this.title, required this.content});
}
```

lib/data/note_data_source.dart

```
// lib/data/note_data_source.dart
import '../models/note.dart';

/// Sumber data online (di Modul 14: tabel notes di Supabase).
abstract class NoteRemoteDataSource {
  Future<List<Note>> fetchNotes();
  Future<Note> createNote(String title, String content);
}

/// Cache offline (di Modul 14: box Hive).
abstract class NoteLocalCache {
  Future<void> saveNotes(List<Note> notes);
  Future<List<Note>> readNotes();
}
```

lib/data/note_repository.dart

```
// lib/data/note_repository.dart
import '../models/note.dart';
import 'note_data_source.dart';

class NoteRepository {
  NoteRepository({required this.remote, required this.cache});

  final NoteRemoteDataSource remote;
  final NoteLocalCache cache;

  /// true jika data terakhir diambil dari cache (sedang offline).
  bool lastFromCache = false;

  Future<List<Note>> getNotes() async {
    try {
      final notes = await remote.fetchNotes();
      await cache.saveNotes(notes);
      lastFromCache = false;
      return notes;
    } catch (_) {
      lastFromCache = true;
      return cache.readNotes();
    }
  }

  Future<Note> addNote(String title, String content) async {
    if (title.trim().isEmpty) {
      throw ArgumentError('Judul tidak boleh kosong');
    }
    return remote.createNote(title.trim(), content);
  }
}
```

`NoteRepository` hanya mengenal `NoteRemoteDataSource` dan `NoteLocalCache` (abstract class). Di aplikasi asli, kita memasang implementasi Supabase dan Hive. Di test, kita memasang versi palsu.

Langkah 2: menulis fake class

Fake ditulis dengan `implements`, sehingga compiler memaksa kita mengisi semua method. Tambahkan "tombol" untuk mengatur perilaku (`shouldFail`) dan "penghitung" untuk diperiksa (`fetchCount`).

`test/mock/fakes.dart`

```dart
// test/mock/fakes.dart
import 'package:catatan_kuliah/data/note_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';

/// Pengganti Supabase: datanya disimpan di list biasa.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  FakeRemoteDataSource({List<Note>? awal}) : notes = [...?awal];

  final List<Note> notes;
  bool shouldFail = false; // ubah ke true untuk mensimulasikan offline
  int fetchCount = 0; // berapa kali fetchNotes dipanggil

  @override
  Future<List<Note>> fetchNotes() async {
    fetchCount++;
    if (shouldFail) throw Exception('Tidak ada koneksi internet');
    return List.of(notes);
  }

  @override
  Future<Note> createNote(String title, String content) async {
    if (shouldFail) throw Exception('Tidak ada koneksi internet');
    final note = Note(id: 'n${notes.length + 1}', title: title, content: content);
    notes.add(note);
    return note;
  }
}

/// Pengganti Hive: cache disimpan di memori.
class FakeLocalCache implements NoteLocalCache {
  List<Note> saved = [];

  @override
  Future<void> saveNotes(List<Note> notes) async => saved = List.of(notes);

  @override
  Future<List<Note>> readNotes() async => List.of(saved);
}
```

File ini diletakkan di folder `test/` karena hanya dipakai untuk pengujian. Namanya tidak diakhiri `_test.dart`, jadi tidak dijalankan sebagai test.

Langkah 3: menguji repository dengan fake

`test/mock/note_repository_test.dart`

```dart
// test/mock/note_repository_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';

import 'fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late FakeLocalCache cache;
  late NoteRepository repository;

  const catatan1 = Note(id: 'n1', title: 'Pertemuan 1', content: 'Pengenalan Dart');

  setUp(() {
    remote = FakeRemoteDataSource(awal: [catatan1]);
    cache = FakeLocalCache();
    repository = NoteRepository(remote: remote, cache: cache);
  });

  test('online: data diambil dari remote lalu disimpan ke cache', () async {
    final notes = await repository.getNotes();

    expect(notes, hasLength(1));
    expect(notes.first.title, 'Pertemuan 1');
    expect(cache.saved, hasLength(1)); // cache ikut terisi
    expect(repository.lastFromCache, isFalse);
  });

  test('offline: data diambil dari cache', () async {
    cache.saved = [catatan1];
    remote.shouldFail = true;

    final notes = await repository.getNotes();

    expect(notes.single.id, 'n1');
    expect(repository.lastFromCache, isTrue);
  });

  test('offline dan cache kosong: hasilnya list kosong, bukan crash', () async {
    remote.shouldFail = true;

    expect(await repository.getNotes(), isEmpty);
  });

  test('addNote menolak judul kosong tanpa memanggil remote', () async {
    await expectLater(repository.addNote('   ', 'isi'), throwsArgumentError);
    expect(remote.notes, hasLength(1)); // tidak bertambah
  });

  test('addNote merapikan spasi judul', () async {
    final note = await repository.addNote('  Kuis 1  ', 'Bab 1-3');

    expect(note.title, 'Kuis 1');
```

```
    expect(remote.notes, hasLength(2));
  });
}
```

Lima test ini mendokumentasikan perilaku penting repository: online menyimpan cache, offline membaca cache, offline tanpa cache tidak crash, dan validasi judul. Jika suatu hari ada yang mengubah `getNotes()` dan lupa menyimpan cache, test pertama langsung gagal.

Alternatif: mocktail

Menulis fake butuh waktu jika interface-nya besar. Package `mocktail` membuat mock tanpa menulis method satu per satu. Pasang sebagai *dev dependency*:

```
flutter pub add dev:mocktail
```

`test/mock/note_repository_mocktail_test.dart`

```
// test/mock/note_repository_mocktail_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:catatan_kuliah/data/note_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';

// Cukup satu baris per class, tanpa menulis method apa pun
class MockRemote extends Mock implements NoteRemoteDataSource {}

class MockCache extends Mock implements NoteLocalCache {}

void main() {
  late MockRemote remote;
  late MockCache cache;
  late NoteRepository repository;

  const catatan = Note(id: 'n1', title: 'UTS', content: 'Bab 1-6');

  setUp(() {
    remote = MockRemote();
    cache = MockCache();
    repository = NoteRepository(remote: remote, cache: cache);
  });

  test('online: hasil remote disimpan ke cache', () async {
    // Atur perilaku mock (stub)
    when(() => remote.fetchNotes()).thenAnswer((_) async => [catatan]);
    when(() => cache.saveNotes(any())).thenAnswer((_) async {});

    final notes = await repository.getNotes();

    expect(notes, [catatan]);
    // Pastikan saveNotes dipanggil tepat satu kali dengan data yang benar
    verify(() => cache.saveNotes([catatan])).called(1);
    verifyNever(() => cache.readNotes());
  });
```

```
  test('offline: remote melempar error, cache yang dibaca', () async {
    when(() => remote.fetchNotes()).thenThrow(Exception('offline'));
    when(() => cache.readNotes()).thenAnswer((_) async => [catatan]);

    final notes = await repository.getNotes();

    expect(notes, [catatan]);
    expect(repository.lastFromCache, isTrue);
    verifyNever(() => cache.saveNotes(any()));
  });
}
```

API mocktail	Fungsi
`class MockX extends Mock implements X {}`	Membuat class mock
`when(() => mock.f()).thenReturn(nilai)`	Method biasa mengembalikan nilai
`when(() => mock.f()).thenAnswer((_) async => nilai)`	Method yang mengembalikan Future
`when(() => mock.f()).thenThrow(error)`	Method melempar error
`any()`	Argumen apa saja
`verify(() => mock.f()).called(n)`	Memastikan method dipanggil n kali
`verifyNever(() => mock.f())`	Memastikan method tidak pernah dipanggil

Jika memakai `any()` untuk argumen bertipe class buatan sendiri (misalnya `Note`), mocktail meminta Anda mendaftarkan nilai cadangan sekali di `setUpAll(() => registerFallbackValue(const Note(id: '', title: '', content: '')));`. Untuk `List<Note>` seperti contoh di atas, hal ini tidak diperlukan.

Fake atau mock?

- Pakai **fake** jika pengganti yang sama dipakai di banyak file test (repository, provider, widget test). Contoh di modul ini memakai fake yang sama di tiga tempat.
- Pakai **mock** jika Anda ingin memeriksa *interaksi*: method apa yang dipanggil, berapa kali, dengan argumen apa.

⚠️ **Kesalahan umum**

- Membuat koneksi di dalam class (`final client = Supabase.instance.client;` di dalam repository). Class seperti ini tidak bisa diuji tanpa Supabase sungguhan. Terima dependensi lewat konstruktor.
- Lupa melakukan stub: memanggil method mock yang belum diatur dengan `when` menghasilkan error `type 'Null' is not a subtype of type 'Future<...>'`.
- Memakai `thenReturn` untuk method yang mengembalikan Future. Gunakan `thenAnswer((_) async => ...)`.
- Hanya menguji jalur sukses. Bagian paling berharga dari fake/mock justru kemampuannya mensimulasikan kegagalan.

8. Widget Test: Interaksi & Validasi Form Login

Widget test menjalankan widget di lingkungan virtual (tanpa emulator), lalu mensimulasikan interaksi pengguna: mengetik, menekan tombol, menggulir. Hasilnya diperiksa dengan cara yang sama seperti unit test, yaitu `expect`.

Kosakata dasar WidgetTester

Perintah	Fungsi
`testWidgets('...', (tester) async { })`	Mendefinisikan widget test; `tester` adalah "robot" yang mengendalikan UI
`await tester.pumpWidget(widget)`	Merender widget untuk pertama kali
`find.text('...')`, `find.byKey(...)`, `find.byType(...)`, `find.byIcon(...)`	Mencari widget di layar (finder)
`await tester.tap(finder)`	Menekan widget
`await tester.enterText(finder, 'teks')`	Mengetik ke `TextField` / `TextFormField`
`await tester.pump()`	Menggambar satu frame berikutnya (setelah setState)
`await tester.pump(Duration)`	Memajukan waktu virtual lalu menggambar frame
`await tester.pumpAndSettle()`	Terus menggambar frame sampai tidak ada animasi/timer yang tersisa
`findsOneWidget`, `findsNothing`, `findsNWidgets(n)`, `findsWidgets`	Matcher jumlah widget yang ditemukan

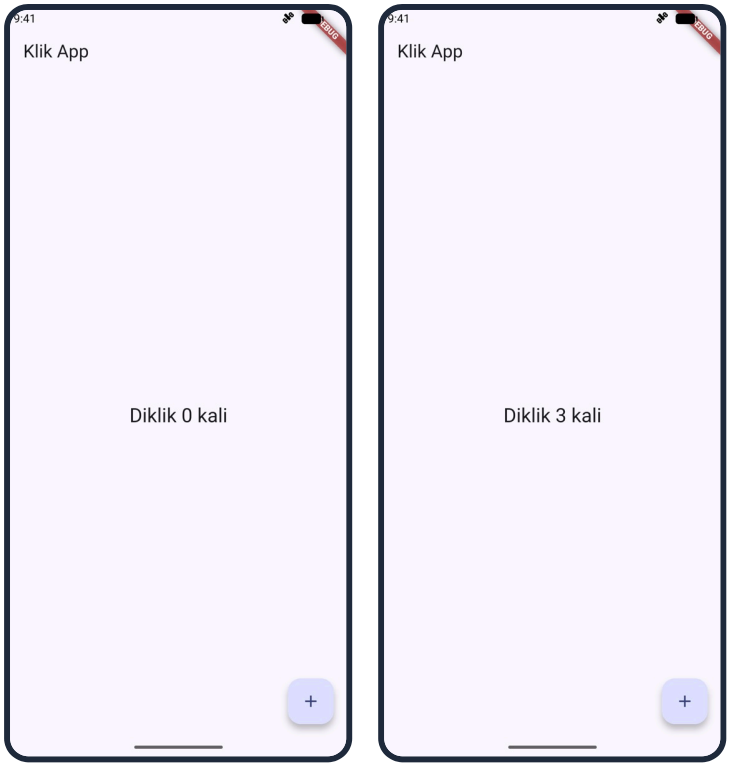
Contoh 1: aplikasi penghitung klik

Kita mulai dari aplikasi paling sederhana. Simpan sebagai `lib/main.dart`:

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const KlikApp());
}
```

```dart
class KlikApp extends StatelessWidget {
  const KlikApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Klik App',
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KlikPage(),
    );
  }
}

class KlikPage extends StatefulWidget {
  const KlikPage({super.key});

  @override
  State<KlikPage> createState() => _KlikPageState();
}

class _KlikPageState extends State<KlikPage> {
  int _jumlah = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Klik App')),
      body: Center(
        child: Text(
          'Diklik $_jumlah kali',
          style: Theme.of(context).textTheme.headlineSmall,
        ),
      ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah',
        onPressed: () => setState(() => _jumlah++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Kondisi awal: "Diklik 0 kali" Setelah tombol + ditekan tiga kali

Test berikut melakukan hal yang sama dengan yang Anda lakukan secara manual di emulator, tetapi otomatis:

test/widget/klik_app_test.dart

```
// test/widget/klik_app_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/main.dart';

void main() {
  testWidgets('angka bertambah saat tombol + ditekan', (tester) async {
    // 1. Render widget
    await tester.pumpWidget(const KlikApp());

    // 2. Periksa kondisi awal
    expect(find.text('Diklik 0 kali'), findsOneWidget);
    expect(find.text('Diklik 1 kali'), findsNothing);

    // 3. Tekan tombol, lalu minta Flutter menggambar frame baru
    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // 4. Periksa hasilnya
    expect(find.text('Diklik 1 kali'), findsOneWidget);
  });

  testWidgets('menekan tombol tiga kali', (tester) async {
    await tester.pumpWidget(const KlikApp());

    for (var i = 0; i < 3; i++) {
      await tester.tap(find.byTooltip('Tambah'));
    }
```

```
    await tester.pump();

    expect(find.text('Diklik 3 kali'), findsOneWidget);
  });
}
```

Poin penting:

- `tester.tap()` hanya mengirim sentuhan. Layar **belum** berubah sampai Anda memanggil `pump()`. Ini kesalahan nomor satu pemula.
- `find.byTooltip('Tambah')` mencari tombol lewat tooltip-nya. Ini sekaligus memastikan tombol punya label aksesibilitas.
- Test dimulai dengan memeriksa kondisi awal. Tanpa itu, test bisa lulus walaupun teks awal sudah salah.

Contoh 2: berbagai cara mencari widget (finder)

test/widget/finder_test.dart

```
// test/widget/finder_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  testWidgets('berbagai cara mencari widget', (tester) async {
    await tester.pumpWidget(
      MaterialApp(
        home: Scaffold(
          appBar: AppBar(title: const Text('Profil')),
          body: Column(
            children: [
              const Text('Budi Santoso', key: Key('nama')),
              const Icon(Icons.verified),
              ElevatedButton(onPressed: () {}, child: const Text('Edit')),
              const Text('Edit'),
            ],
          ),
        ),
      ),
    );

    expect(find.text('Profil'), findsOneWidget); // berdasarkan teks
    expect(find.byKey(const Key('nama')), findsOneWidget); // berdasarkan Key
    expect(find.byIcon(Icons.verified), findsOneWidget); // berdasarkan ikon
    expect(find.byType(ElevatedButton), findsOneWidget); // berdasarkan tipe
    expect(find.text('Edit'), findsNWidgets(2)); // ada 2 teks "Edit"
    // Teks "Edit" yang berada DI DALAM ElevatedButton saja
    expect(find.widgetWithText(ElevatedButton, 'Edit'), findsOneWidget);
    expect(find.text('Hapus'), findsNothing);

    // Membaca properti widget yang ditemukan
    final teksNama = tester.widget<Text>(find.byKey(const Key('nama')));
    expect(teksNama.data, 'Budi Santoso');
```

```
  });
}
```

Urutan prioritas memilih finder: `find.byKey` paling stabil (tidak terpengaruh perubahan teks), `find.text` paling mudah dibaca, `find.byType` jika hanya ada satu widget sejenis. `tester.widget<T>(finder)` mengambil objek widget-nya untuk memeriksa properti.

pump() vs pumpAndSettle()

	`pump()`	`pumpAndSettle()`
Jumlah frame	Tepat satu	Sebanyak yang dibutuhkan sampai tenang
Cocok untuk	setState sederhana, memeriksa kondisi "di tengah" (loading)	Navigasi, dialog, SnackBar, Future yang sudah selesai
Bahaya	Animasi belum selesai saat diperiksa	**Timeout** jika ada animasi tanpa akhir (misalnya `CircularProgressIndicator` yang terus berputar)

Contoh 3: menguji validasi form login (Modul 13)

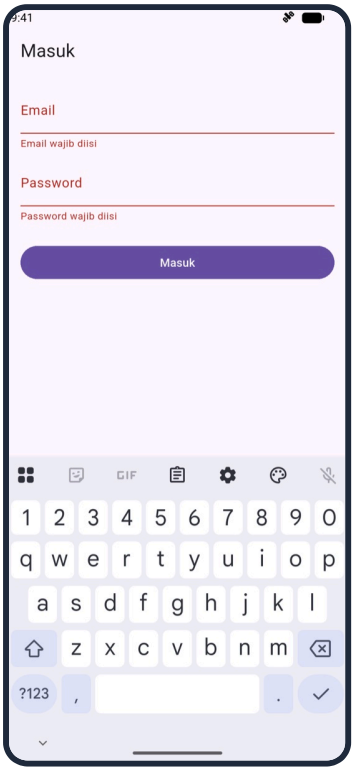
Form login Catatan Kuliah adalah kandidat ideal untuk widget test: banyak aturan validasi, dan bug di sini membuat pengguna tidak bisa masuk sama sekali. Agar mudah diuji, proses login **tidak** dipanggil langsung di dalam halaman, melainkan diterima lewat parameter `onLogin`. Di aplikasi asli isinya `context.read<AuthProvider>().signIn`; di test isinya fungsi palsu.

`lib/pages/login_page.dart`

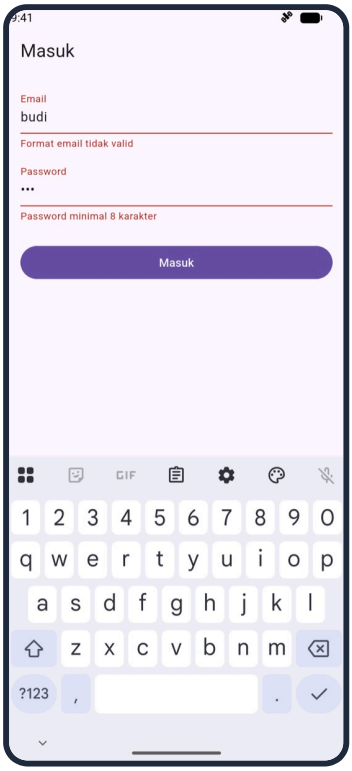
```dart
// lib/pages/login_page.dart
import 'package:flutter/material.dart';

import '../utils/validators.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key, required this.onLogin});

  /// Dipanggil saat form valid. Di Modul 13 isinya AuthProvider.signIn.
  final Future<void> Function(String email, String password) onLogin;

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _loading = false;
```

```dart
@override
void dispose() {
  _emailController.dispose();
  _passwordController.dispose();
  super.dispose();
}

Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() => _loading = true);
  try {
    await widget.onLogin(
      _emailController.text.trim(),
      _passwordController.text,
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Login gagal, periksa email/password')),
    );
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Masuk')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Form(
        key: _formKey,
        child: Column(
          children: [
            TextFormField(
              key: const Key('field-email'),
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
              keyboardType: TextInputType.emailAddress,
              validator: validasiEmail,
            ),
            const SizedBox(height: 12),
            TextFormField(
              key: const Key('field-password'),
              controller: _passwordController,
              decoration: const InputDecoration(labelText: 'Password'),
              obscureText: true,
              validator: validasiPassword,
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              child: FilledButton(
                onPressed: _loading ? null : _submit,
                child: _loading
                    ? const SizedBox(
```

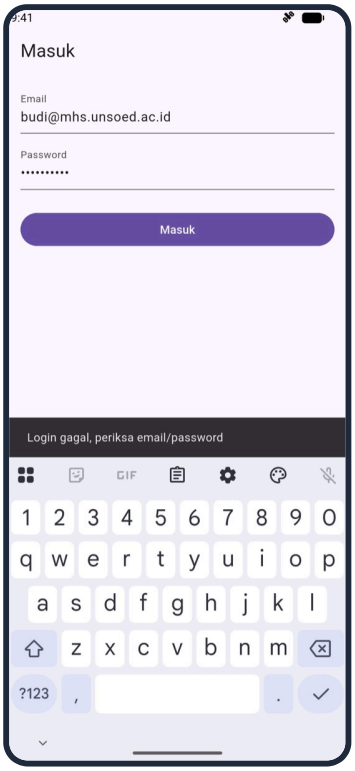
```
                        width: 20,
                        height: 20,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      )
                    : const Text('Masuk'),
              ),
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```



Tombol Masuk ditekan saat form kosong



Email “budi” dan password “123”



Form valid tetapi onLogin melempar error: SnackBar muncul

Ketiga kondisi pada screenshot di atas kini kita uji secara otomatis, ditambah dua skenario lain:

`test/widget/login_page_test.dart`

```
// test/widget/login_page_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/pages/login_page.dart';

void main() {
  // Helper agar setiap test tidak menulis MaterialApp berulang kali
  Future<void> pumpLogin(
    WidgetTester tester, {
    Future<void> Function(String, String)? onLogin,
  }) async {
    await tester.pumpWidget(
```

```
    MaterialApp(
      home: LoginPage(onLogin: onLogin ?? (email, password) async {}),
    ),
  );
}

final fieldEmail = find.byKey(const Key('field-email'));
final fieldPassword = find.byKey(const Key('field-password'));
final tombolMasuk = find.widgetWithText(FilledButton, 'Masuk');

testWidgets('form kosong menampilkan dua pesan error', (tester) async {
  await pumpLogin(tester);

  await tester.tap(tombolMasuk);
  await tester.pump();

  expect(find.text('Email wajib diisi'), findsOneWidget);
  expect(find.text('Password wajib diisi'), findsOneWidget);
});

testWidgets('email salah format dan password pendek', (tester) async {
  await pumpLogin(tester);

  await tester.enterText(fieldEmail, 'budi');
  await tester.enterText(fieldPassword, '123');
  await tester.tap(tombolMasuk);
  await tester.pump();

  expect(find.text('Format email tidak valid'), findsOneWidget);
  expect(find.text('Password minimal 8 karakter'), findsOneWidget);
});

testWidgets('form valid memanggil onLogin dengan data yang diketik',
    (tester) async {
  String? emailDikirim;
  String? passwordDikirim;

  await pumpLogin(
    tester,
    onLogin: (email, password) async {
      emailDikirim = email;
      passwordDikirim = password;
    },
  );

  await tester.enterText(fieldEmail, '  budi@mhs.unsoed.ac.id ');
  await tester.enterText(fieldPassword, 'rahasia123');
  await tester.tap(tombolMasuk);
  await tester.pumpAndSettle();

  expect(emailDikirim, 'budi@mhs.unsoed.ac.id'); // sudah di-trim
  expect(passwordDikirim, 'rahasia123');
  expect(find.text('Email wajib diisi'), findsNothing);
});

testWidgets('login gagal menampilkan SnackBar', (tester) async {
  await pumpLogin(
    tester,
    onLogin: (email, password) async => throw Exception('401'),
```

```
    );

    await tester.enterText(fieldEmail, 'budi@mhs.unsoed.ac.id');
    await tester.enterText(fieldPassword, 'salahsalah');
    await tester.tap(tombolMasuk);
    await tester.pumpAndSettle();

    expect(find.text('Login gagal, periksa email/password'), findsOneWidget);
  });

  testWidgets('tombol menampilkan loading selama proses login',
      (tester) async {
    await pumpLogin(
      tester,
      onLogin: (email, password) =>
          Future.delayed(const Duration(seconds: 2)),
    );

    await tester.enterText(fieldEmail, 'budi@mhs.unsoed.ac.id');
    await tester.enterText(fieldPassword, 'rahasia123');
    await tester.tap(tombolMasuk);
    await tester.pump(); // satu frame: setState(_loading = true) terlihat

    expect(find.byType(CircularProgressIndicator), findsOneWidget);

    await tester.pump(const Duration(seconds: 2)); // majukan waktu 2 detik
    await tester.pump(); // frame setelah loading selesai

    expect(find.byType(CircularProgressIndicator), findsNothing);
  });
}
```

Penjelasan bagian penting:

- **Helper `pumpLogin`**: membungkus `LoginPage` dengan `MaterialApp` (wajib, karena `Scaffold`, `TextFormField`, dan `SnackBar` butuh tema dan `ScaffoldMessenger`).
- **`Key('field-email')`**: diberikan di `LoginPage` agar test bisa menemukan field dengan pasti. Mencari lewat `find.text('Email')` rawan karena label bisa berubah.
- **`find.widgetWithText(FilledButton, 'Masuk')`**: teks “Masuk” juga ada di judul AppBar, jadi kita batasi pencarian ke tombol saja.
- **Menangkap argumen**: variabel `emailDikirim` diisi oleh fungsi palsu, lalu diperiksa. Ini cara sederhana memastikan form mengirim data yang benar (termasuk `trim()`).
- **Test loading**: `pump()` sekali untuk melihat kondisi tengah (indikator loading tampil), lalu `pump(Duration(seconds: 2))` memajukan waktu virtual. Test tetap selesai dalam milidetik walaupun “menunggu” 2 detik. Di sini `pumpAndSettle()` tidak bisa dipakai di tengah, karena indikator loading berputar terus.

Menggulir sampai widget terlihat

Jika widget berada di bawah layar dalam `ListView`, ia mungkin belum dibangun sehingga `find` tidak menemukannya. Gulir dulu:

```dart
// di dalam testWidgets, setelah pumpWidget sebuah ListView berisi 100 item
await tester.scrollUntilVisible(
  find.text('Catatan 80'),
  300, // jarak gulir per langkah (pixel)
  scrollable: find.byType(Scrollable).first,
);
expect(find.text('Catatan 80'), findsOneWidget);
```

⚠️ Kesalahan umum

- Lupa `await` di depan `tester.tap`, `tester.pump`, atau `tester.enterText`.
- Lupa memanggil `pump()` setelah `tap()`, lalu bingung kenapa teks belum berubah.
- Memanggil `pumpAndSettle()` saat ada animasi tak berujung, sehingga test gagal dengan pesan `pumpAndSettle timed out`.
- Me-render halaman tanpa `MaterialApp`, menghasilkan error `No MediaQuery widget ancestor found` atau `No Material widget found`.
- Halaman langsung memanggil `Supabase.instance.client`. Di test, Supabase belum diinisialisasi sehingga test crash. Pisahkan aksi lewat parameter atau provider (bagian berikutnya).

9. Menguji Provider (NotesProvider)

Di Modul 14, logika halaman catatan dipindahkan ke `NotesProvider`. Kabar baiknya: `ChangeNotifier` adalah class Dart biasa, sehingga bisa diuji dengan **unit test** tanpa widget sama sekali. Setelah itu, kita uji halaman yang memakai provider tersebut dengan **widget test**.

Provider yang akan diuji

`lib/providers/notes_provider.dart`

```dart
// lib/providers/notes_provider.dart
import 'package:flutter/foundation.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

enum NotesStatus { initial, loading, loaded, error }

class NotesProvider extends ChangeNotifier {
  NotesProvider(this._repository);

  final NoteRepository _repository;

  NotesStatus status = NotesStatus.initial;
  List<Note> notes = [];
  String? errorMessage;
  bool get isOffline => _repository.lastFromCache;

  Future<void> loadNotes() async {
    status = NotesStatus.loading;
    notifyListeners();
    try {
      notes = await _repository.getNotes();
      status = NotesStatus.loaded;
    } catch (e) {
      errorMessage = 'Gagal memuat catatan';
      status = NotesStatus.error;
    }
    notifyListeners();
  }

  Future<bool> addNote(String title, String content) async {
    try {
      final note = await _repository.addNote(title, content);
      notes = [note, ...notes];
      notifyListeners();
      return true;
    } catch (e) {
      errorMessage = 'Gagal menyimpan catatan';
      notifyListeners();
      return false;
    }
  }
}
```

Langkah 1: unit test untuk ChangeNotifier

Provider dibuat dengan repository sungguhan yang diberi data source palsu dari bagian sebelumnya (`test/mock/fakes.dart`). Dengan begitu kita sekaligus menguji kerja sama provider dan repository.

`test/unit/notes_provider_test.dart`

```dart
// test/unit/notes_provider_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:catatan_kuliah/providers/notes_provider.dart';

import '../mock/fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late NotesProvider provider;

  setUp(() {
    remote = FakeRemoteDataSource(
      awal: [const Note(id: 'n1', title: 'Pertemuan 1', content: 'Dart')],
    );
    final repository = NoteRepository(remote: remote, cache: FakeLocalCache());
    provider = NotesProvider(repository);
  });

  test('status awal initial dan list kosong', () {
    expect(provider.status, NotesStatus.initial);
    expect(provider.notes, isEmpty);
  });

  test('loadNotes: loading lalu loaded', () async {
    final riwayatStatus = <NotesStatus>[];
    provider.addListener(() => riwayatStatus.add(provider.status));

    await provider.loadNotes();

    expect(riwayatStatus, [NotesStatus.loading, NotesStatus.loaded]);
    expect(provider.notes, hasLength(1));
  });

  test('addNote menaruh catatan baru di urutan teratas', () async {
    await provider.loadNotes();

    final berhasil = await provider.addNote('Kuis', 'Bab 2');

    expect(berhasil, isTrue);
    expect(provider.notes.first.title, 'Kuis');
    expect(provider.notes, hasLength(2));
  });

  test('addNote gagal saat offline, errorMessage terisi', () async {
    remote.shouldFail = true;

    final berhasil = await provider.addNote('Kuis', 'Bab 2');
```

```
    expect(berhasil, isFalse);
    expect(provider.errorMessage, 'Gagal menyimpan catatan');
  });

  test('provider memanggil notifyListeners saat addNote', () async {
    var jumlahNotifikasi = 0;
    provider.addListener(() => jumlahNotifikasi++);

    await provider.addNote('Kuis', 'Bab 2');

    expect(jumlahNotifikasi, 1);
  });
}
```

Teknik yang perlu diperhatikan:

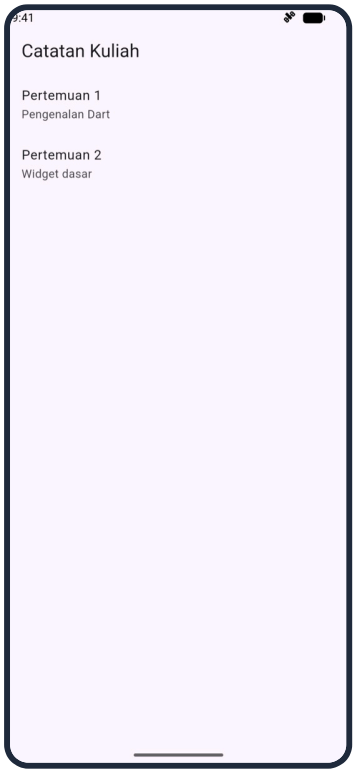
- `provider.addListener(...)`: inilah cara widget "mendengar" provider. Di test, kita memakainya untuk merekam urutan status (`loading` lalu `loaded`) dan menghitung berapa kali `notifyListeners()` dipanggil.
- **Menguji jumlah notifikasi** mencegah dua bug yang sering terjadi: lupa memanggil `notifyListeners()` (UI tidak berubah) dan memanggilnya berlebihan (UI rebuild tanpa perlu).
- `remote.shouldFail = true`: mensimulasikan kondisi offline cukup dengan satu baris.

Langkah 2: halaman yang memakai provider

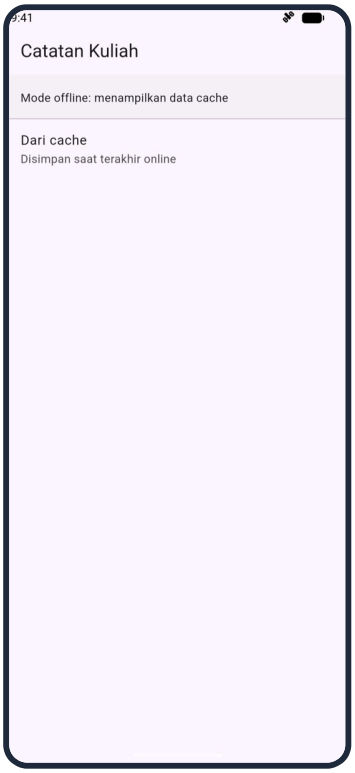
`lib/pages/notes_page.dart`

```
// lib/pages/notes_page.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/notes_provider.dart';

class NotesPage extends StatelessWidget {
  const NotesPage({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<NotesProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Kuliah')),
      body: switch (provider.status) {
        NotesStatus.initial ||
        NotesStatus.loading => const Center(child: CircularProgressIndicator()),
        NotesStatus.error => Center(child: Text(provider.errorMessage ?? 'Error')),
        NotesStatus.loaded => Column(
            children: [
              if (provider.isOffline)
                const MaterialBanner(
                  content: Text('Mode offline: menampilkan data cache'),
                  actions: [SizedBox.shrink()],
                ),
              Expanded(
                child: provider.notes.isEmpty
```

```
                    ? const Center(child: Text('Belum ada catatan'))
                    : ListView.builder(
                        itemCount: provider.notes.length,
                        itemBuilder: (context, index) {
                          final note = provider.notes[index];
                          return ListTile(
                            title: Text(note.title),
                            subtitle: Text(note.content),
                          );
                        },
                      ),
                ),
              ],
            ),
          },
        );
      }
    }
```



Fake remote berhasil: daftar catatan tampil



`shouldFail = true`: data cache dan banner mode offline

Screenshot di atas diambil dengan memasang `FakeRemoteDataSource` dan `FakeLocalCache` di `main()`. Ini trik yang berguna untuk mendesain tampilan offline tanpa harus mematikan internet.

Langkah 3: widget test dengan ChangeNotifierProvider

Pola kuncinya: bungkus halaman dengan `ChangeNotifierProvider` yang berisi provider dengan dependensi palsu, **di atas** `MaterialApp`, persis seperti di `main.dart`.

test/widget/notes_page_test.dart

```dart
// test/widget/notes_page_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:catatan_kuliah/pages/notes_page.dart';
import 'package:catatan_kuliah/providers/notes_provider.dart';

import '../mock/fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late FakeLocalCache cache;

  setUp(() {
    remote = FakeRemoteDataSource(
      awal: [const Note(id: 'n1', title: 'Pertemuan 1', content: 'Dart')],
    );
    cache = FakeLocalCache();
  });

  Widget buatApp() {
    final repository = NoteRepository(remote: remote, cache: cache);
    return ChangeNotifierProvider(
      create: (_) => NotesProvider(repository)..loadNotes(),
      child: const MaterialApp(home: NotesPage()),
    );
  }

  testWidgets('menampilkan loading lalu daftar catatan', (tester) async {
    await tester.pumpWidget(buatApp());

    // Frame pertama: data belum datang
    expect(find.byType(CircularProgressIndicator), findsOneWidget);

    await tester.pumpAndSettle(); // tunggu Future selesai

    expect(find.text('Pertemuan 1'), findsOneWidget);
    expect(find.byType(CircularProgressIndicator), findsNothing);
  });

  testWidgets('offline dengan cache: tampil banner mode offline',
      (tester) async {
    remote.shouldFail = true;
    cache.saved = [const Note(id: 'c1', title: 'Dari cache', content: '-')];

    await tester.pumpWidget(buatApp());
    await tester.pumpAndSettle();

    expect(find.text('Dari cache'), findsOneWidget);
    expect(find.textContaining('Mode offline'), findsOneWidget);
  });

  testWidgets('tanpa data sama sekali: tampil pesan kosong', (tester) async {
    remote.notes.clear();
```

```
    await tester.pumpWidget(buatApp());
    await tester.pumpAndSettle();

    expect(find.text('Belum ada catatan'), findsOneWidget);
  });
}
```

- `NotesProvider(repository)..loadNotes()` : operator kaskade (`..`) memanggil `loadNotes()` lalu mengembalikan provider-nya.
- Frame pertama setelah `pumpWidget` menampilkan loading karena Future belum selesai. `pumpAndSettle()` menunggu Future fake selesai lalu menggambar ulang.
- Fungsi `buatApp()` dipanggil *setelah* test mengubah `remote` / `cache`, sehingga setiap test bisa menyiapkan skenario berbeda.

Alternatif: memakai ChangeNotifierProvider.value

Jika test perlu memegang objek provider untuk dipanggil langsung, buat provider di luar lalu berikan dengan `.value`:

```
testWidgets('catatan baru langsung tampil di daftar', (tester) async {
  final provider = NotesProvider(
    NoteRepository(remote: FakeRemoteDataSource(), cache: FakeLocalCache()),
  );
  await provider.loadNotes();

  await tester.pumpWidget(
    ChangeNotifierProvider.value(
      value: provider,
      child: const MaterialApp(home: NotesPage()),
    ),
  );
  expect(find.text('Belum ada catatan'), findsOneWidget);

  await provider.addNote('Tugas Besar', 'Deadline minggu 16');
  await tester.pump();

  expect(find.text('Tugas Besar'), findsOneWidget);
});
```

⚠️ **Kesalahan umum**

- Membuat provider yang di dalamnya langsung memanggil `Supabase.instance.client` atau `Hive.box()`. Provider seperti itu tidak bisa diuji; terima repository lewat konstruktor.
- Meletakkan `ChangeNotifierProvider` di bawah `MaterialApp` di test, lalu test navigasi gagal dengan *ProviderNotFoundException* (lihat bagian 1).
- Hanya menguji provider lewat UI. Unit test provider jauh lebih cepat dan pesan gagalnya lebih jelas; gunakan widget test untuk memastikan UI bereaksi terhadap state.

10. Golden Test, Integration Test & Coverage

Bagian terakhir membahas tiga alat pelengkap: **golden test** untuk menjaga tampilan tidak berubah tanpa sengaja, **integration test** untuk menguji aplikasi utuh di perangkat, dan **coverage** untuk mengukur seberapa banyak kode yang sudah diuji.

1. Golden test (sekilas)

Golden test merender widget menjadi gambar PNG, lalu membandingkannya dengan gambar acuan (“golden file”) yang disimpan di repo. Jika ada satu pixel yang berbeda, test gagal. Cocok untuk widget yang tampilannya penting dan jarang berubah, seperti kartu atau komponen desain.

`lib/widgets/kartu_nilai.dart`

```dart
// lib/widgets/kartu_nilai.dart
import 'package:flutter/material.dart';

class KartuNilai extends StatelessWidget {
  const KartuNilai({super.key, required this.matkul, required this.nilai});

  final String matkul;
  final String nilai;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        leading: const Icon(Icons.school),
        title: Text(matkul),
        trailing: CircleAvatar(child: Text(nilai)),
      ),
    );
  }
}
```

`test/golden/kartu_nilai_golden_test.dart`

```dart
// test/golden/kartu_nilai_golden_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/widgets/kartu_nilai.dart';

void main() {
  testWidgets('tampilan KartuNilai sesuai golden', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        debugShowCheckedModeBanner: false,
        home: Scaffold(
          body: Center(
            child: SizedBox(
              width: 300,
              child: KartuNilai(matkul: 'Pemrograman Mobile', nilai: 'A'),
            ),
          ),
```

```
        ),
      ),
    );

    await expectLater(
      find.byType(KartuNilai),
      matchesGoldenFile('goldens/kartu_nilai.png'),
    );
  });
}
```

```
# Pertama kali: buat gambar acuan (test/golden/goldens/kartu_nilai.png)
flutter test --update-goldens test/golden

# Selanjutnya: bandingkan dengan acuan
flutter test test/golden
```

Jika test golden gagal, Flutter menyimpan gambar perbedaan di folder `test/golden/failures/`. Lihat gambarnya: jika perubahan memang disengaja (misalnya Anda mengganti warna), jalankan lagi `--update-goldens` dan commit gambar barunya.

> ℹ️ **Catatan**
>
> Di lingkungan test, semua teks dirender dengan font khusus berbentuk kotak-kotak (Ahem) agar hasilnya sama di semua komputer. Jadi jangan kaget jika gambar golden tidak menampilkan huruf yang terbaca. Golden file juga bisa sedikit berbeda antara macOS, Windows, dan Linux, sehingga tim biasanya membuat golden di satu OS yang sama (misalnya di CI).

2. Integration test (pengenalan)

Unit dan widget test berjalan di lingkungan virtual. Integration test menjalankan **aplikasi asli** di emulator atau HP, termasuk plugin native (kamera, Hive, Supabase). Gunanya untuk menguji alur penting dari awal sampai akhir, misalnya “register → login → tambah catatan → logout”.

Tambahkan package `integration_test` (bagian dari Flutter SDK) sebagai dev dependency:

```
flutter pub add 'dev:integration_test:{"sdk":"flutter"}'
```

Hasilnya di `pubspec.yaml`:

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  integration_test:
    sdk: flutter
```

Buat folder `integration_test/` di root proyek (sejajar dengan `lib/` dan `test/`):

`integration_test/app_test.dart`

```dart
// integration_test/app_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:catatan_kuliah/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('alur lengkap: buka aplikasi dan tekan tombol', (tester) async {
    app.main(); // jalankan aplikasi ASLI
    await tester.pumpAndSettle();

    expect(find.text('Diklik 0 kali'), findsOneWidget);

    await tester.tap(find.byIcon(Icons.add));
    await tester.pumpAndSettle();

    expect(find.text('Diklik 1 kali'), findsOneWidget);
  });
}
```

```bash
# Pastikan emulator menyala atau HP terhubung
flutter devices

# Jalankan integration test di perangkat
flutter test integration_test/app_test.dart

# Pilih perangkat tertentu
flutter test integration_test -d emulator-5554
```

Perhatikan perbedaannya dengan widget test:

- `IntegrationTestWidgetsFlutterBinding.ensureInitialized()` wajib dipanggil di awal.
- `app.main()` menjalankan fungsi `main()` asli, termasuk inisialisasi Supabase dan Hive. Karena memakai backend sungguhan, gunakan akun dan project Supabase khusus pengujian.
- Integration test jauh lebih lambat (build APK + install), jadi tulis sedikit saja untuk alur yang paling penting.

3. Coverage: seberapa banyak kode yang diuji?

Coverage menghitung persentase baris kode di `lib/` yang dijalankan oleh test.

```bash
# Jalankan semua test sambil merekam coverage
flutter test --coverage

# Hasilnya: coverage/lcov.info (format teks LCOV)
head coverage/lcov.info
```

Isi `lcov.info` berupa daftar file (`SF:`) dan jumlah eksekusi tiap baris (`DA:nomor_baris,jumlah`):

```
SF:lib/utils/ipk.dart
DA:2,2
DA:3,6
DA:4,1
DA:6,1
...
```

Agar mudah dibaca, ubah menjadi laporan HTML dengan `genhtml` dari paket `lcov`:

```
# macOS
brew install lcov
# Ubuntu/Debian
sudo apt install lcov

genhtml coverage/lcov.info -o coverage/html
open coverage/html/index.html        # macOS
xdg-open coverage/html/index.html    # Linux
```

Pengguna Windows atau yang tidak ingin memasang lcov bisa memakai ekstensi VS Code **Coverage Gutters**: ia membaca `coverage/lcov.info` dan mewarnai baris di editor (hijau = teruji, merah = belum).

> 💡 **Tips**
>
> Jangan mengejar 100%. Target realistis untuk tugas besar: logika bisnis (`utils/`, `data/`, `providers/`) di atas 70%, ditambah widget test untuk halaman-halaman utama. Coverage tinggi dengan test yang tidak memeriksa apa-apa (tanpa `expect`) tidak ada artinya.

Ringkasan perintah

Perintah	Fungsi
`flutter analyze`	Cek error dan lint tanpa menjalankan aplikasi
`flutter test`	Jalankan semua unit, widget, dan golden test
`flutter test --update-goldens`	Perbarui gambar acuan golden
`flutter test integration_test`	Jalankan integration test di perangkat
`flutter test --coverage`	Rekam coverage ke `coverage/lcov.info`

> ⚠️ **Kesalahan umum**
>
> - Menaruh file integration test di folder `test/`. File tersebut akan ikut dijalankan oleh `flutter test` biasa dan gagal karena tidak ada perangkat.
> - Lupa meng-commit folder `goldens/`, sehingga test golden gagal di komputer teman satu kelompok.
> - Meng-commit folder `coverage/`. Folder ini hasil generate dan sudah ada di `.gitignore` bawaan Flutter.

Tugas Praktikum: Praktikum 15: Debugging & Suite Pengujian Otomatis Catatan Kuliah

Sasaran Praktikum

Menemukan dan memperbaiki bug dengan alat debugging, lalu menulis rangkaian test otomatis (unit, fake/mock, widget, provider) untuk proyek Catatan Kuliah dari Modul 13-14 sehingga perubahan kode berikutnya tidak merusak fitur lama.

Langkah Pengerjaan

1. **Bagian A - Debugging.** Buat proyek latihan berisi kelima contoh kode “SALAH” dari bagian 1. Jalankan, salin kalimat pertama setiap pesan error dan baris stack trace yang menunjuk ke kode Anda ke dalam file `DEBUGGING.md`, lalu perbaiki satu per satu.
2. Pasang breakpoint di dalam fungsi `getNotes()` pada repository Anda, jalankan dengan F5, dan catat nilai variabel `notes` yang terlihat di panel Variables. Lampirkan screenshot-nya.
3. Buka DevTools tab Network saat halaman catatan dimuat. Catat URL, status code, dan durasi request ke Supabase.
4. Jalankan `flutter analyze` pada proyek Catatan Kuliah, tambahkan aturan `prefer_final_locals` dan `unawaited_futures` ke `analysis_options.yaml`, lalu perbaiki hingga hasilnya `No issues found!`.
5. **Bagian B - Unit test.** Pindahkan validator email/password ke `lib/utils/validators.dart` dan buat `test/unit/validators_test.dart` dengan minimal 6 test (positif, negatif, dan kasus batas).
6. **Bagian C - Fake.** Pastikan `NoteRepository` menerima data source remote dan cache lewat konstruktor. Buat `test/mock/fakes.dart` dan `test/mock/note_repository_test.dart` yang menguji skenario online, offline dengan cache, offline tanpa cache, dan validasi judul kosong.
7. **Bagian D - Provider.** Buat `test/unit/notes_provider_test.dart` yang memeriksa urutan status (loading lalu loaded), penambahan catatan, dan pesan error saat offline.
8. **Bagian E - Widget test.** Buat `test/widget/login_page_test.dart` yang menguji: form kosong, format email salah, password kurang dari 8 karakter, form valid memanggil fungsi login dengan data yang benar, dan SnackBar saat login gagal.
9. Buat `test/widget/notes_page_test.dart` yang memastikan halaman catatan menampilkan loading, daftar catatan, dan banner mode offline.
10. Jalankan `flutter test --coverage`, buat laporan HTML (genhtml atau Coverage Gutters), dan ambil screenshot persentase coverage folder `lib/data` dan `lib/providers`.

Output yang Diharapkan

Repositori berisi file DEBUGGING.md, folder test/ dengan minimal 20 test yang semuanya lulus (output “All tests passed!”), flutter analyze tanpa issue, dan screenshot laporan coverage dengan coverage lib/data dan lib/providers minimal 70%.

Rubrik Penilaian

Laporan debugging lengkap dan perbaikan benar (20%), unit test validator dan provider (20%), test repository dengan fake/mock termasuk skenario gagal (25%), widget test form login dan halaman catatan (25%), flutter analyze bersih dan laporan coverage (10%).

Kuis Pemahaman

1. Saat membaca stack trace error Flutter, baris mana yang paling perlu Anda periksa terlebih dahulu?

A. Baris paling bawah stack trace

B. Baris pertama yang berasal dari package proyek Anda sendiri (bukan package:flutter/)

C. Semua baris yang berasal dari framework.dart

D. Baris yang berisi kata `<asynchronous suspension>`

2. Sebuah ListView.builder diletakkan langsung di dalam Column dan muncul error “Vertical viewport was given unbounded height”. Perbaikan yang paling tepat adalah...

A. Mengganti Column dengan Row

B. Membungkus ListView dengan Expanded agar tingginya mengikuti sisa ruang Column

C. Menambahkan operator ! pada itemCount

D. Memanggil setState setelah ListView dibangun

3. Kode berikut sering memicu error “setState() called after dispose()”: final data = await ambilData(); setState(() ⇒ _data = data);. Apa perbaikannya?

A. Menghapus kata await

B. Menambahkan if (!mounted) return; setelah await dan sebelum setState

C. Mengganti StatefulWidget menjadi StatelessWidget

D. Memanggil dispose() secara manual sebelum setState

4. Mengapa pengukuran performa (mencari jank) di DevTools sebaiknya dilakukan dengan flutter run --profile, bukan mode debug?

A. Karena mode debug tidak bisa membuka DevTools

B. Karena mode debug sengaja lebih lambat akibat banyak pemeriksaan tambahan, sehingga hasil pengukurannya menyesatkan

C. Karena mode profile otomatis memperbaiki jank

D. Karena mode profile tidak membutuhkan perangkat

5. Manakah cara yang BENAR untuk menguji bahwa fungsi biasa hitungIpk melempar ArgumentError?

A. expect(hitungIpk([4], []), throwsArgumentError);

B. expect(() ⇒ hitungIpk([4], []), throwsArgumentError);

C. expect(hitungIpk, ArgumentError);

D. expect(throwsArgumentError, hitungIpk([4], []));

6. Apa tujuan utama membuat FakeRemoteDataSource saat menguji NoteRepository?

A. Agar aplikasi rilis lebih cepat

B. Agar test tidak bergantung pada Supabase/internet dan kondisi seperti offline bisa disimulasikan dengan mudah

C. Agar data di Supabase terhapus setelah test

D. Agar tidak perlu menulis expect

7. Dalam widget test, setelah await tester.tap(find.byIcon(Icons.add)); teks di layar belum berubah. Apa yang terlupa?

A. Memanggil await tester.pump() agar Flutter menggambar frame berikutnya

B. Memanggil setState dari dalam test

C. Menjalankan emulator

D. Menambahkan Key pada Text

8. Perintah apa yang menghasilkan file coverage/lcov.info berisi data baris kode yang dijalankan oleh test?

A. flutter analyze --coverage

B. flutter test --coverage

C. flutter build apk --coverage

D. dart fix --coverage

Kunci Jawaban & Pembahasan

1. B — Stack trace diurutkan dari pemanggilan terakhir. Baris dari package:flutter/ hampir tidak pernah menjadi penyebab; baris pertama dari kode Anda menunjukkan file dan nomor baris tempat masalah dimulai.

2. B — Column memberi tinggi tak terbatas kepada anaknya, sedangkan ListView ingin setinggi mungkin. Expanded memberi ListView batas tinggi berupa sisa ruang Column.

3. B — Selama menunggu Future, pengguna bisa saja meninggalkan halaman sehingga State sudah di-dispose. Properti mounted bernilai false pada kondisi itu, jadi setState tidak boleh dipanggil.

4. B — Mode debug memakai JIT dan banyak assertion sehingga lebih lambat dari aplikasi rilis. Mode profile mendekati performa rilis tetapi tetap bisa diprofil oleh DevTools.

5. B — Fungsi harus dibungkus closure () ⇒ agar expect yang menjalankannya dan bisa menangkap error. Tanpa closure, error terjadi sebelum expect dipanggil sehingga test gagal.

6. B — Fake menggantikan sumber data asli dengan implementasi di memori yang kita kendalikan. Test menjadi cepat, stabil, dan skenario gagal seperti koneksi putus bisa dibuat cukup dengan shouldFail = true.

7. A — tap hanya mengirim sentuhan. Perubahan state baru terlihat setelah frame berikutnya digambar dengan pump() (atau pumpAndSettle() jika ada animasi).

8. B — flutter test --coverage menjalankan semua test sambil merekam baris yang dieksekusi ke coverage/lcov.info, yang kemudian bisa diubah menjadi laporan HTML dengan genhtml atau dibaca ekstensi Coverage Gutters.

MODUL 16 · PRODUKSI & CAPSTONE

Build Rilis APK/AAB & Panduan Capstone Project

Mempersiapkan aplikasi untuk dirilis: versi aplikasi, nama dan application ID, ikon dan splash screen, permission, konfigurasi Supabase dengan --dart-define-from-file, keystore dan signing Gradle Kotlin DSL, obfuscation, build APK split-per-abi dan App Bundle, pengujian build rilis, gambaran Play Console dan iOS, serta panduan lengkap Capstone Project.

Durasi Pertemuan 16 (150 Menit) + Belajar Mandiri

Prasyarat Modul 14: Proyek Catatan Kuliah Bagian 2 (CRUD & Cache Offline)
Modul 15: Debugging & Testing

Capaian Pembelajaran

- Mengatur versi aplikasi (version: x.y.z+build), nama aplikasi, dan application ID yang unik.
- Mengonfigurasi launcher icon (termasuk adaptive icon) dan native splash screen dengan generator.
- Mendeklarasikan permission yang dibutuhkan di AndroidManifest.xml dan Info.plist, termasuk INTERNET untuk build rilis.
- Memisahkan konfigurasi rahasia dari kode dengan --dart-define-from-file dan .gitignore.
- Membuat keystore, key.properties, dan konfigurasi signing di build.gradle.kts, lalu memverifikasi tanda tangan APK.
- Mengompilasi APK split-per-abi dan App Bundle dengan obfuscation, lalu mengujinya di perangkat sungguhan.
- Menjelaskan alur internal testing di Google Play Console dan gambaran rilis iOS.
- Menyelesaikan Capstone Project sesuai fitur wajib, rubrik, timeline, dan deliverables.

Daftar Isi Modul

1. Mode Build & Versi Aplikasi
2. Nama Aplikasi & Application ID
3. App Icon & Splash Screen
4. Permission di AndroidManifest & Info.plist
5. Konfigurasi Lingkungan dengan --dart-define-from-file
6. Keystore & Konfigurasi Signing
7. Build APK & AAB, Obfuscation, Uji di Perangkat
8. Google Play Console & Gambaran iOS

9. Panduan Capstone Project
10. Tugas Praktikum
11. Kuis Pemahaman & Kunci Jawaban

1. Mode Build & Versi Aplikasi

Selama ini aplikasi selalu dijalankan dalam mode *debug*. Pengguna sungguhan tidak pernah melihat mode itu: mereka memasang build **release** yang lebih kecil, lebih cepat, dan sudah ditandatangani. Modul ini membahas semua langkah dari kode yang "jalan di laptop" sampai APK/AAB yang siap dibagikan, lalu ditutup dengan panduan Capstone Project.

Tiga mode build Flutter

Mode	Perintah	Ciri	Dipakai untuk
Debug	`flutter run`	Hot reload, assert aktif, pita DEBUG, lambat, ukuran besar	Pengembangan sehari-hari
Profile	`flutter run --profile`	Performa mendekati rilis, DevTools tetap bisa dipakai	Mengukur performa (Modul 15)
Release	`flutter run --release`, `flutter build apk`	Kode dikompilasi AOT, assert dan `kDebugMode` mati, tanpa hot reload	Dibagikan ke pengguna

Peta jalan rilis

1. Atur nomor versi (bagian ini).
2. Atur nama aplikasi dan package id (bagian 2).
3. Pasang ikon dan splash screen (bagian 3).
4. Periksa permission di `AndroidManifest.xml` (bagian 4).
5. Keluarkan kunci Supabase dari kode dengan `--dart-define-from-file` (bagian 5).
6. Buat keystore dan konfigurasi signing (bagian 6).
7. Build APK/AAB dengan obfuscation lalu uji di HP sungguhan (bagian 7).
8. Unggah ke Play Console untuk internal testing (bagian 8).

Versi aplikasi: version: x.y.z+build

Nomor versi diatur di satu tempat saja, yaitu baris `version` di `pubspec.yaml`:

`pubspec.yaml` (potongan)

```yaml
name: catatan_kuliah
description: "Aplikasi catatan kuliah dengan Supabase dan cache offline."
publish_to: 'none'

# format: versionName+versionCode
version: 1.2.0+7
```

Bagian	Contoh	Di Android menjadi	Dilihat oleh	Aturan
Nama versi (sebelum `+`)	`1.2.0`	`versionName`	Pengguna (di Play Store dan pengaturan HP)	Bebas, biasanya mengikuti semantic versioning
Nomor build (setelah `+`)	`7`	`versionCode`	Sistem Android dan Play Store	Bilangan bulat, **wajib naik** setiap unggahan baru

Semantic versioning (`MAYOR.MINOR.PATCH`) memudahkan pengguna dan tim memahami besarnya perubahan:

- **PATCH** (`1.2.0 → 1.2.1`): perbaikan bug tanpa fitur baru.
- **MINOR** (`1.2.1 → 1.3.0`): fitur baru, misalnya fitur pencarian catatan.
- **MAYOR** (`1.3.0 → 2.0.0`): perubahan besar, misalnya struktur database berubah total.

Nilai `versionCode` dan `versionName` otomatis dibaca oleh Gradle dari `pubspec.yaml`. Di `android/app/build.gradle.kts` hasil `flutter create` sudah ada baris berikut, jadi **jangan** mengubah angka di sana:

`android/app/build.gradle.kts` **(potongan bawaan)**

```
defaultConfig {
    applicationId = "id.ac.unsoed.catatan_kuliah"
    minSdk = flutter.minSdkVersion
    targetSdk = flutter.targetSdkVersion
    versionCode = flutter.versionCode
    versionName = flutter.versionName
}
```

Versi juga bisa ditimpa sekali jalan dari terminal, misalnya oleh CI:

```
flutter build apk --build-name=1.2.1 --build-number=8
```

Menampilkan versi di dalam aplikasi

Halaman “Tentang” atau footer halaman Profil biasanya menampilkan versi aplikasi. Ini membantu saat pengguna melapor bug (“saya pakai versi berapa?”). Gunakan package `package_info_plus`:

```
flutter pub add package_info_plus
```

`lib/utils/versi.dart`

```dart
// lib/utils/versi.dart
import 'package:package_info_plus/package_info_plus.dart';

/// Contoh hasil: "1.2.0 (build 7)"
Future<String> versiAplikasi() async {
  final info = await PackageInfo.fromPlatform();
  return '${info.version} (build ${info.buildNumber})';
}
```

Panggil `versiAplikasi()` di `initState` halaman Profil lalu tampilkan hasilnya dengan `FutureBuilder` atau `setState`, seperti pola di Modul 09.

> ⚠️ **Kesalahan umum**
>
> - Mengunggah build baru ke Play Console dengan nomor build yang sama. Play Console menolak: *"Version code 7 has already been used"*. Naikkan angka setelah `+`.
> - Menurunkan nomor build. Android menolak memasang APK dengan `versionCode` lebih kecil di atas versi yang sudah terpasang.
> - Mengubah `versionCode = 7` langsung di `build.gradle.kts`. Hapus angka manual itu; pubspec adalah satu-satunya sumber versi.

2. Nama Aplikasi & Application ID

Setiap aplikasi Android punya dua identitas: **nama tampilan** yang dilihat pengguna di bawah ikon, dan **application ID** (package id) yang unik di seluruh Play Store. Nama tampilan boleh diganti kapan saja; application ID **tidak boleh berubah** setelah aplikasi dipublikasikan.

1. Nama aplikasi (label)

Secara bawaan, nama aplikasi sama dengan nama proyek (`catatan_kuliah`), lengkap dengan garis bawah. Ganti atribut `android:label`:

`android/app/src/main/AndroidManifest.xml` **(potongan)**

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:label="Catatan Kuliah"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        ...
    </application>
</manifest>
```

Untuk iOS, ubah nilai `CFBundleDisplayName`:

`ios/Runner/Info.plist` **(potongan)**

```
<key>CFBundleDisplayName</key>
<string>Catatan Kuliah</string>
```

Nama yang tampil di *task switcher* Android saat aplikasi berjalan diambil dari properti `title` pada `MaterialApp`, jadi samakan juga: `MaterialApp(title: 'Catatan Kuliah', ...)`.

2. Application ID

Application ID ditulis seperti nama domain terbalik. Untuk tugas kuliah, gunakan pola `id.ac.unsoed.<nim_atau_kelompok>.<nama_app>` agar tidak bentrok dengan teman sekelas.

- Hanya huruf kecil, angka, dan garis bawah; dipisah titik; minimal dua segmen.
- Setiap segmen harus diawali huruf: `id.ac.unsoed.h1d022001.catatan` boleh, `id.ac.unsoed.2022.catatan` tidak.
- Hindari `com.example`: Play Console menolak application ID yang diawali `com.example`.

Cara paling mudah adalah menentukannya **saat membuat proyek** dengan opsi `--org`:

```
flutter create --org id.ac.unsoed.h1d022001 catatan_kuliah
# hasil: applicationId = "id.ac.unsoed.h1d022001.catatan_kuliah"
```

Untuk proyek yang sudah terlanjur dibuat, ubah `applicationId` di Gradle:

`android/app/build.gradle.kts` (potongan)

```
android {
    // namespace = package kode Kotlin/Java (lokasi MainActivity)
    namespace = "com.example.catatan_kuliah"

    defaultConfig {
        // applicationId = identitas unik di HP dan Play Store
        applicationId = "id.ac.unsoed.h1d022001.catatan_kuliah"
        // ...
    }
}
```

	`applicationId`	`namespace`
Fungsi	Identitas aplikasi di HP dan Play Store	Nama package kode Kotlin/Java dan class `R`
Boleh berbeda satu sama lain?	Ya. Mengubah `applicationId` saja sudah cukup untuk mengganti identitas aplikasi.	
Jika diubah	Dianggap aplikasi yang berbeda (data lama tidak ikut)	Harus memindahkan `MainActivity.kt` ke folder dan `package` yang sesuai

Jika Anda juga ingin mengubah `namespace`, sesuaikan baris `package` di `MainActivity.kt` dan pindahkan file tersebut ke folder yang cocok:

`android/app/src/main/kotlin/id/ac/unsoed/h1d022001/catatan_kuliah/MainActivity.kt`

```
package id.ac.unsoed.h1d022001.catatan_kuliah

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

Setelah mengubah application ID, lakukan `flutter clean` lalu jalankan ulang. Untuk iOS, bundle identifier diatur dari Xcode (Runner → Signing & Capabilities).

⚠️ **Kesalahan umum**

- Mengubah `namespace` tanpa memindahkan `MainActivity.kt`. Build berhasil, tetapi aplikasi langsung tertutup dengan error `ClassNotFoundException: ...MainActivity`.
- Mengubah application ID setelah aplikasi diunggah ke Play Console. Play Store akan menganggapnya aplikasi baru; pengguna lama tidak mendapat update.
- Satu kelas memakai `com.example.catatan_kuliah` yang sama. Saat dosen memasang beberapa APK, APK berikutnya menimpa APK sebelumnya (atau gagal dipasang karena tanda tangan berbeda).

3. App Icon & Splash Screen

Aplikasi profesional tidak boleh menampilkan ikon bawaan Flutter atau layar putih kosong saat pertama dibuka. Dua package generator membuat semua ukuran ikon dan splash screen untuk Android dan iOS secara otomatis dari satu gambar.

Menyiapkan aset gambar

```
catatan_kuliah/
├── assets/
|   ├── icon/
|   |   ├── icon.png              # 1024x1024, latar penuh (untuk iOS dan Android lama)
|   |   └── icon_foreground.png   # 1024x1024, latar TRANSPARAN, logo di tengah
|   └── splash/
|       └── logo.png              # logo transparan untuk splash
└── pubspec.yaml
```

- Gunakan PNG persegi minimal 1024×1024 piksel. Jangan memberi sudut membulat sendiri; Android dan iOS akan memotongnya sesuai bentuk masing-masing.
- Untuk *adaptive icon* Android, logo pada `icon_foreground.png` harus berada di area tengah (sekitar 66% bagian dalam), karena pinggirannya bisa terpotong menjadi lingkaran, kotak membulat, atau bentuk lain tergantung merek HP.

Langkah 1: ikon aplikasi dengan flutter_launcher_icons

```
flutter pub add dev:flutter_launcher_icons
```

Konfigurasi bisa ditulis di `pubspec.yaml` atau di file terpisah. File terpisah membuat `pubspec.yaml` tetap rapi:

`flutter_launcher_icons.yaml` (di root proyek)

```yaml
flutter_launcher_icons:
  # Android: ganti ikon bawaan (ic_launcher) dengan ikon baru
  android: true
  ios: true
  image_path: "assets/icon/icon.png"           # 1024x1024, tanpa sudut membulat
  min_sdk_android: 21
  # Adaptive icon (Android 8+): background warna + foreground transparan
  adaptive_icon_background: "#1E3A8A"
  adaptive_icon_foreground: "assets/icon/icon_foreground.png"
  adaptive_icon_monochrome: "assets/icon/icon_foreground.png" # ikon bertema Android 13+
  # iOS menolak ikon transparan di App Store
  remove_alpha_ios: true
  background_color_ios: "#1E3A8A"
```

```
dart run flutter_launcher_icons
```

Output yang diharapkan:

```
• Creating colors.xml file and adding it to your Android project
• Overwriting the default Android launcher icon with a new icon
• Creating mipmap xml file Android
• Overwriting default iOS launcher icon with new icon

✓ Successfully generated launcher icons
```

Opsi	Fungsi
`android: true`	Timpa ikon bawaan `ic_launcher`. Isi dengan nama lain (misalnya `"launcher_icon"`) jika ingin membuat ikon baru tanpa menghapus yang lama.
`image_path`	Gambar sumber untuk semua platform
`adaptive_icon_background` / `_foreground`	Dua lapisan adaptive icon Android 8+. Keduanya wajib diisi agar adaptive icon dibuat.
`adaptive_icon_monochrome`	Versi satu warna untuk fitur *themed icons* Android 13+
`remove_alpha_ios`	Hapus transparansi ikon iOS (syarat App Store)

Langkah 2: splash screen dengan flutter_native_splash

Splash screen native tampil *sebelum* Flutter selesai dimuat, menggantikan layar putih kosong.

```
flutter pub add dev:flutter_native_splash
```

`flutter_native_splash.yaml` **(di root proyek)**

```yaml
flutter_native_splash:
  # Android 11 ke bawah dan iOS
  color: "#1E3A8A"
  image: assets/splash/logo.png
  color_dark: "#0F172A"
  image_dark: assets/splash/logo.png

  # Android 12+ punya aturan splash sendiri
  android_12:
    color: "#1E3A8A"
    image: assets/splash/logo.png
    color_dark: "#0F172A"
    image_dark: assets/splash/logo.png

  android: true
  ios: true
  web: false
```

```
dart run flutter_native_splash:create

# Mengembalikan ke splash bawaan jika ingin membatalkan
dart run flutter_native_splash:remove
```

Kenapa ada blok `android_12` terpisah? Mulai Android 12, sistem selalu menampilkan splash berupa **ikon di tengah lingkaran** di atas satu warna latar. Gambar penuh layar tidak didukung. Siapkan logo yang tetap terlihat baik jika dipotong menjadi lingkaran (sekitar 1152×1152 px dengan logo di dalam lingkaran berdiameter 768 px).

Langkah 3 (opsional): menahan splash selama inisialisasi

Aplikasi Catatan Kuliah perlu menginisialisasi Supabase dan Hive sebelum halaman pertama tampil. Splash bisa ditahan sampai proses itu selesai. Untuk fitur ini, package harus dipindah ke `dependencies` biasa (bukan `dev`), karena dipakai di kode aplikasi:

```
flutter pub remove flutter_native_splash
flutter pub add flutter_native_splash
```

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:flutter_native_splash/flutter_native_splash.dart';

Future<void> main() async {
  final binding = WidgetsFlutterBinding.ensureInitialized();
  // Tahan splash selama inisialisasi (Supabase, Hive, dll.)
  FlutterNativeSplash.preserve(widgetsBinding: binding);

  await Future.delayed(const Duration(seconds: 1)); // ganti dengan inisialisasi asli

  FlutterNativeSplash.remove(); // splash hilang, frame pertama tampil
  runApp(const MaterialApp(home: Scaffold(body: Center(child: Text('Siap!')))));
}
```

`preserve()` menahan splash native, `remove()` menghilangkannya. Jangan lupa memanggil `remove()` juga pada jalur error (misalnya di blok `catch`), agar aplikasi tidak “macet” di splash selamanya.

Melihat hasilnya

Ikon dan splash adalah bagian native, jadi **hot reload tidak cukup**. Hentikan aplikasi, hapus aplikasi lama dari emulator (beberapa launcher menyimpan cache ikon), lalu jalankan ulang `flutter run`.

⚠️ **Kesalahan umum**

- Logo foreground mengisi seluruh gambar 1024×1024, sehingga terpotong di HP dengan ikon berbentuk lingkaran.
- Menulis konfigurasi di dua tempat (di `pubspec.yaml` dan file terpisah). Pilih salah satu.
- Lupa menjalankan ulang perintah generator setelah mengganti gambar.
- Memakai `FlutterNativeSplash.preserve` sementara package masih di `dev_dependencies`. `flutter analyze` memberi peringatan `depend_on_referenced_packages`.

4. Permission di AndroidManifest & Info.plist

Android melindungi fitur sensitif (internet, kamera, lokasi, notifikasi) dengan **permission**. Aplikasi harus mendeklarasikan permission di `AndroidManifest.xml`; sebagian juga harus diminta langsung kepada pengguna saat aplikasi berjalan.

Jebakan nomor satu: permission INTERNET

Aplikasi Catatan Kuliah berjalan lancar di emulator, tetapi setelah APK release dipasang di HP, login Supabase selalu gagal. Kenapa? Proyek Flutter punya **tiga** file manifest:

```
android/app/src/
├── debug/AndroidManifest.xml     # dipakai saat flutter run (debug)
├── profile/AndroidManifest.xml   # dipakai saat --profile
└── main/AndroidManifest.xml      # dipakai SEMUA mode, termasuk release
```

Manifest `debug` bawaan sudah berisi permission INTERNET (dibutuhkan untuk hot reload), sedangkan manifest `main` **tidak**. Jadi aplikasi bisa mengakses internet saat debug, tetapi tidak saat release. Isi manifest debug bawaan:

`android/app/src/debug/AndroidManifest.xml` (bawaan flutter create)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- The INTERNET permission is required for development. Specifically,
         the Flutter tool needs it to communicate with the running application
         to allow setting breakpoints, to provide hot reload, etc.
    -->
    <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

Solusinya, tambahkan permission di manifest `main`, **di luar** tag `<application>`:

`android/app/src/main/AndroidManifest.xml` (potongan)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Wajib untuk Supabase, REST API, gambar dari internet -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="Catatan Kuliah"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        ...
    </application>
</manifest>
```

Permission lain yang sering dipakai

Fitur	Permission	Perlu diminta saat runtime?
Internet	`android.permission.INTERNET`	Tidak
Cek status koneksi	`android.permission.ACCESS_NETWORK_STATE`	Tidak
Kamera	`android.permission.CAMERA`	Ya
Lokasi	`android.permission.ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`	Ya
Notifikasi (Android 13+)	`android.permission.POST_NOTIFICATIONS`	Ya
Getar	`android.permission.VIBRATE`	Tidak

Contoh manifest untuk aplikasi capstone yang memakai kamera dan lokasi (misalnya aplikasi pengaduan sarana kampus):

`android/app/src/main/AndroidManifest.xml` (potongan)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

    <!-- Kamera tidak wajib: aplikasi tetap bisa dipasang di perangkat tanpa kamera -->
    <uses-feature android:name="android.hardware.camera" android:required="false"/>

    <application ...>
        ...
    </application>
</manifest>
```

Permission “berbahaya” (kamera, lokasi, notifikasi) juga harus diminta saat runtime. Package seperti `image_picker` dan `geolocator` biasanya sudah menampilkan dialog izin sendiri. Untuk kontrol penuh, gunakan package `permission_handler` dan ikuti petunjuk konfigurasinya di pub.dev.

iOS: alasan penggunaan di Info.plist

iOS tidak memakai daftar permission, tetapi mewajibkan **kalimat alasan** untuk setiap fitur sensitif. Tanpa kalimat ini aplikasi langsung tertutup saat meminta izin, dan App Store menolak aplikasi.

`ios/Runner/Info.plist` (potongan)

```
<key>NSCameraUsageDescription</key>
<string>Kamera dipakai untuk memotret fasilitas yang rusak.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Lokasi dipakai untuk menandai posisi laporan di peta kampus.</string>
```

> ⚠️ **Kesalahan umum**
>
> - Menambahkan INTERNET hanya di manifest `debug`, atau menaruh `<uses-permission>` di dalam tag `<application>`.
> - Meminta permission yang tidak dipakai. Play Console akan menanyakan alasannya, dan pengguna jadi curiga.
> - Tidak menangani kasus pengguna menolak izin. Tampilkan pesan dan jalan alternatif, jangan biarkan aplikasi crash.

5. Konfigurasi Lingkungan dengan --dart-define-from-file

Contoh singkat sering menulis URL dan key Supabase langsung di `main.dart`. Proyek Modul 13-14 sudah memakai cara yang dibahas di sini, jadi bagian ini merangkum dan memperdalamnya. Begitu kode di-push ke GitHub publik, kunci itu ikut terbaca semua orang. Bagian ini memindahkan konfigurasi ke file terpisah yang tidak masuk git, lalu menyuntikkannya saat build dengan `--dart-define-from-file`.

Konsep: konstanta saat kompilasi

Flag `--dart-define` mengirim pasangan kunci-nilai ke compiler. Kode Dart membacanya dengan `String.fromEnvironment`:

```
flutter run --dart-define=APP_ENV=dev --dart-define=SUPABASE_URL=https://abcdefghijkl.supabase.co
```

Menulis banyak `--dart-define` tentu merepotkan. Flutter menyediakan `--dart-define-from-file` yang membaca semua nilai dari satu file JSON (atau `.env`).

Langkah 1: file konfigurasi per lingkungan

`config/dev.json` (TIDAK di-commit)

```
{
  "SUPABASE_URL": "https://abcdefghijkl.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxx",
  "APP_ENV": "dev",
  "ENABLE_LOG": true
}
```

`config/prod.json` (TIDAK di-commit)

```
{
  "SUPABASE_URL": "https://abcdefghijkl.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxx",
  "APP_ENV": "prod",
  "ENABLE_LOG": false
}
```

Agar anggota kelompok tahu kunci apa saja yang dibutuhkan, commit file **contoh** tanpa nilai asli:

`config/example.json` (di-commit)

```
{
  "SUPABASE_URL": "https://ISI_PROJECT_ID.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "ISI_PUBLISHABLE_KEY_DARI_DASHBOARD",
  "APP_ENV": "dev",
  "ENABLE_LOG": true
}
```

Lalu tambahkan ke `.gitignore` di root proyek:

`.gitignore` (tambahkan di bagian bawah)

```
# Konfigurasi rahasia
/config/*.json
!/config/example.json
```

Baris `!` berarti pengecualian: semua file JSON di `config/` diabaikan kecuali `example.json`.

Langkah 2: class Env

`lib/config/env.dart`

```dart
// lib/config/env.dart
class Env {
  // Nilai diisi saat build lewat --dart-define atau --dart-define-from-file.
  // WAJIB const, karena nilainya ditanam ke aplikasi saat kompilasi.
  static const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');
  static const appEnv = String.fromEnvironment('APP_ENV', defaultValue: 'dev');
  static const enableLog = bool.fromEnvironment('ENABLE_LOG');

  static bool get isProduction => appEnv == 'prod';

  /// false jika aplikasi dijalankan tanpa --dart-define-from-file.
  static bool get isConfigured => supabaseUrl.isNotEmpty && supabaseKey.isNotEmpty;
}
```

- `String.fromEnvironment` dan `bool.fromEnvironment` **harus** dipanggil sebagai `const`. Dokumentasi Dart hanya menjamin nilainya terbaca dalam konteks `const`; tanpa `const`, nilainya bisa kosong, terutama di build release.
- Nilai JSON `true` / `false` dibaca dengan `bool.fromEnvironment`; angka dengan `int.fromEnvironment`.
- `defaultValue` dipakai jika kunci tidak ada di file.

Langkah 3: memakai Env di main.dart

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Lupa --dart-define-from-file? Tampilkan pesan yang jelas, jangan crash.
  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }
```

```
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabaseKey,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah',
      // Pita "DEBUG" hanya muncul jika bukan build produksi
      debugShowCheckedModeBanner: !Env.isProduction,
      home: Scaffold(
        appBar: AppBar(title: const Text('Catatan Kuliah')),
        body: Center(child: Text('Lingkungan: ${Env.appEnv}')),
      ),
    );
  }
}

class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                Icon(Icons.settings_suggest, size: 64, color: Colors.orange),
                SizedBox(height: 16),
                Text(
                  'Konfigurasi Supabase belum diisi',
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                ),
                SizedBox(height: 8),
                Text(
                  'Jalankan dengan:\n'
                  'flutter run --dart-define-from-file=config/dev.json',
                  textAlign: TextAlign.center,
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Jika aplikasi dijalankan tanpa file konfigurasi (misalnya menekan tombol Run tanpa `launch.json`), pengguna melihat pesan jelas, bukan crash misterius dari Supabase:

Dijalankan tanpa `--dart-define-from-file`: ConfigErrorApp tampil

Langkah 4: menjalankan dan mem-build

```
# Pengembangan
flutter run --dart-define-from-file=config/dev.json

# Build rilis dengan konfigurasi produksi
flutter build apk --split-per-abi --dart-define-from-file=config/prod.json

# Juga bisa memakai file .env (format KUNCI=nilai)
flutter run --dart-define-from-file=.env
```

Agar tombol Run/Debug di VS Code otomatis memakai file konfigurasi, isi `.vscode/launch.json` seperti di Modul 15 bagian 2 (`"args": ["--dart-define-from-file=config/dev.json"]`).

ℹ️ Apakah key Supabase jadi rahasia?

Tidak sepenuhnya. Nilai `--dart-define` ditanam ke dalam APK, sehingga orang yang membongkar APK masih bisa menemukannya. Tujuan langkah ini adalah **tidak menyebarkan kunci lewat GitHub** dan memudahkan berganti project dev/prod. Keamanan data Supabase tetap bergantung pada **Row Level Security** (Modul 12). Publishable/anon key memang dirancang boleh berada di aplikasi klien; yang **tidak boleh sama sekali** masuk ke aplikasi adalah `service_role` /secret key.

⚠️ Kesalahan umum

- Menulis `final url = String.fromEnvironment('SUPABASE_URL');` tanpa `const`. Di build release, nilainya bisa kosong.
- Lupa menambahkan flag saat `flutter build`. APK jadi tidak bisa login. Simpan perintah build di README agar tidak lupa.
- Sudah terlanjur commit key ke GitHub, lalu hanya menghapus file di commit berikutnya. Key tetap ada di riwayat git. Buat ulang (rotate) key di dashboard Supabase.
- Memasukkan `service_role` key ke aplikasi agar “RLS tidak mengganggu”. Siapa pun yang membongkar APK bisa membaca dan menghapus seluruh database.

6. Keystore & Konfigurasi Signing

Android hanya mau memasang APK yang **ditandatangani secara digital**. Selama ini Flutter diam-diam memakai *debug key* yang sama untuk semua developer di dunia. Untuk rilis, Anda butuh kunci sendiri (**keystore**) yang menjadi bukti bahwa update aplikasi benar-benar berasal dari Anda.

Konsep: kenapa tanda tangan penting?

- Android hanya mengizinkan update jika APK baru ditandatangani dengan kunci yang **sama** dengan APK yang sudah terpasang.
- Jika keystore hilang, Anda tidak bisa lagi memperbarui aplikasi yang sudah dibagikan (kecuali memakai Play App Signing, lihat bagian 8).
- Jika keystore bocor, orang lain bisa membuat "update" palsu atas nama Anda.

Langkah 1: membuat keystore dengan keytool

`keytool` sudah tersedia bersama JDK yang dipasang Android Studio. Jalankan dari folder root proyek:

```
# macOS / Linux: simpan di android/app/
keytool -genkey -v -keystore android/app/upload-keystore.jks \
  -keyalg RSA -keysize 2048 -validity 10000 -alias upload

# Windows (PowerShell), satu baris:
keytool -genkey -v -keystore android\app\upload-keystore.jks -keyalg RSA -keysize 2048 -validity
10000 -alias upload
```

Jika muncul `keytool: command not found`, pakai keytool bawaan Android Studio. Lokasinya bisa dilihat dengan `flutter doctor -v` pada baris *Java binary at*, lalu ganti `java` dengan `keytool` pada path tersebut.

keytool akan menanyakan password dan identitas Anda:

```
Enter keystore password:  ********
Re-enter new password:    ********
What is your first and last name?
  [Unknown]:  Budi Santoso
What is the name of your organizational unit?
  [Unknown]:  Informatika
What is the name of your organization?
  [Unknown]:  UNSOED
What is the name of your City or Locality?
  [Unknown]:  Purwokerto
What is the name of your State or Province?
  [Unknown]:  Jawa Tengah
What is the two-letter country code for this unit?
  [Unknown]:  ID
Is CN=Budi Santoso, OU=Informatika, O=UNSOED, L=Purwokerto, ST=Jawa Tengah, C=ID correct?
  [no]:  yes
```

```
...
[Storing android/app/upload-keystore.jks]
```

Opsi	Arti
`-keystore`	Lokasi file keystore yang dibuat
`-alias upload`	Nama kunci di dalam keystore (satu keystore bisa berisi beberapa kunci)
`-validity 10000`	Berlaku 10.000 hari (sekitar 27 tahun). Play Store mensyaratkan masa berlaku panjang.
`-keyalg RSA -keysize 2048`	Algoritma dan panjang kunci

Langkah 2: file android/key.properties

Gradle membaca password dari file terpisah agar tidak tertulis di `build.gradle.kts`:

`android/key.properties`

```
storePassword=password_keystore_anda
keyPassword=password_key_anda
keyAlias=upload
# Relatif terhadap folder android/app/
storeFile=upload-keystore.jks
```

Periksa `android/.gitignore`. Proyek dari `flutter create` versi terbaru sudah mengabaikan file-file ini:

`android/.gitignore` **(bawaan, bagian bawah)**

```
# Remember to never publicly share your keystore.
# See https://flutter.dev/to/reference-keystore
key.properties
**/*.keystore
**/*.jks
```

Pastikan dengan perintah berikut. Jika file muncul di daftar “Untracked files”, berarti belum diabaikan:

```
git status --ignored android/ | grep -E "key.properties|jks"
# hasil yang benar: kedua file muncul di bagian "Ignored files"
```

Langkah 3: konfigurasi signing di build.gradle.kts

Proyek Flutter baru memakai Kotlin DSL (`build.gradle.kts`). Berikut file lengkap hasil `flutter create` versi terbaru yang sudah ditambah konfigurasi signing. Bagian yang Anda tambahkan:

dua baris `import` di paling atas, blok pembacaan `key.properties`, blok `signingConfigs`, dan isi `buildTypes`.

`android/app/build.gradle.kts`

```
import java.io.FileInputStream
import java.util.Properties

plugins {
    id("com.android.application")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-plugin")
}

// Baca android/key.properties (jika ada)
val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

android {
    namespace = "id.ac.unsoed.catatan_kuliah"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    defaultConfig {
        // TODO: Specify your own unique Application ID
(https://developer.android.com/studio/build/application-id.html).
        applicationId = "id.ac.unsoed.catatan_kuliah"
        // You can update the following values to match your application needs.
        // For more information, see: https://flutter.dev/to/review-gradle-config.
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        // Uses the version code from pubspec.yaml. When using split APKs, 1000 * ABI_VERSION
        // is added automatically by Flutter. (https://developer.android.com/studio/build/configure-
apk-splits#configure-APK-versions)
        // You can force using the value of versionCode by specifying the `-P force-version-code-
ignoring-abi=true`
        // flag during build.
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }

    signingConfigs {
        create("release") {
            if (keystorePropertiesFile.exists()) {
                keyAlias = keystoreProperties["keyAlias"] as String
                keyPassword = keystoreProperties["keyPassword"] as String
                storeFile = file(keystoreProperties["storeFile"] as String)
                storePassword = keystoreProperties["storePassword"] as String
            }
        }
```

```
    }

    buildTypes {
        release {
            signingConfig = if (keystorePropertiesFile.exists()) {
                signingConfigs.getByName("release")
            } else {
                // Belum ada key.properties: pakai debug key agar
                // `flutter run --release` tetap bisa dicoba
                signingConfigs.getByName("debug")
            }
        }
    }
}

kotlin {
    compilerOptions {
        jvmTarget = org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17
    }
}

flutter {
    source = "../.."
}
```

Penjelasan:

- `import` **harus di baris paling atas**, sebelum blok `plugins`. Di Kotlin script, import di tengah file menyebabkan error kompilasi Gradle.
- `rootProject.file("key.properties")` menunjuk ke `android/key.properties`, sedangkan `file(...)` di dalam blok `android` relatif terhadap `android/app/`. Itu sebabnya `storeFile=upload-keystore.jks` cukup ditulis nama filenya.
- Pemeriksaan `keystorePropertiesFile.exists()` membuat proyek tetap bisa di-build oleh anggota kelompok yang tidak memegang keystore (APK-nya memakai debug key). Tanpa pemeriksaan ini, `flutter run` biasa pun gagal di komputer mereka.
- Bagian lain (`compileSdk`, `versionCode`, blok `kotlin`) biarkan sesuai bawaan proyek Anda. Versi Flutter yang berbeda bisa menghasilkan isi bawaan yang sedikit berbeda.

Jika proyek Anda masih memakai `build.gradle` (Groovy, proyek lama), ikuti panduan resmi: docs.flutter.dev/deployment/android.

Langkah 4: memastikan APK benar-benar memakai kunci rilis

Setelah build (bagian 7), periksa sertifikat APK dengan `apksigner` dari Android SDK build-tools:

```
# macOS (sesuaikan versi build-tools di komputer Anda)
~/Library/Android/sdk/build-tools/<versi>/apksigner verify --print-certs \
  build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

```
# Benar: memakai keystore Anda
V2 Signer: certificate DN: CN=Budi Santoso, OU=Informatika, O=UNSOED, L=Purwokerto, ST=Jawa Tengah,
```

```
C=ID

# Salah: masih debug key (key.properties tidak terbaca)
V2 Signer: certificate DN: C=US, O=Android, CN=Android Debug
```

> 💡 **Backup keystore**
>
> Simpan salinan `upload-keystore.jks` beserta password-nya di tempat aman di luar laptop (password manager, Google Drive pribadi yang tidak dibagikan). Laptop rusak tanpa backup keystore berarti aplikasi Anda tidak bisa di-update lagi di luar Play App Signing.

> ⚠️ **Kesalahan umum**
>
> - Meng-commit `key.properties` atau file `.jks` ke GitHub publik.
> - Menulis path Windows dengan backslash tunggal di `key.properties` (`C:\Users\budi\key.jks`). Gunakan garis miring biasa (`C:/Users/budi/key.jks`) atau simpan keystore di `android/app/` dan tulis nama filenya saja.
> - Lupa mengganti `signingConfig` di `buildTypes.release`, sehingga APK rilis tetap memakai debug key.
> - Membuat keystore baru setiap kali build. Satu aplikasi = satu keystore selamanya.

7. Build APK & AAB, Obfuscation, Uji di Perangkat

Semua persiapan sudah selesai. Saatnya mengompilasi aplikasi menjadi file yang bisa dipasang: **APK** untuk dibagikan langsung, atau **AAB** (Android App Bundle) untuk diunggah ke Google Play.

APK atau AAB?

	APK	AAB
Bisa langsung dipasang di HP?	Ya	Tidak
Untuk Google Play	Tidak diterima untuk aplikasi baru	**Wajib**
Cara kerja	Satu file berisi kode untuk CPU tertentu (atau semua CPU)	Play Store membuat APK yang pas untuk setiap HP pengunduh
Cocok untuk	Dibagikan ke dosen/teman, pengumpulan tugas	Publikasi di Play Store

1. Membersihkan dan memeriksa sebelum build

```
# Pastikan kode bersih dan semua test lulus (Modul 15)
flutter analyze
flutter test

# Opsional: hapus hasil build lama jika ada perilaku aneh
flutter clean
flutter pub get
```

2. Build APK per arsitektur CPU

```
flutter build apk --split-per-abi --dart-define-from-file=config/prod.json
```

Contoh output asli untuk aplikasi yang memakai Supabase dan beberapa package lain:

```
Running Gradle task 'assembleRelease'...                          91.4s
✓ Built build/app/outputs/flutter-apk/app-armeabi-v7a-release.apk (13.2MB)
✓ Built build/app/outputs/flutter-apk/app-arm64-v8a-release.apk (16.1MB)
✓ Built build/app/outputs/flutter-apk/app-x86_64-release.apk (17.6MB)
```

Bandingkan dengan `flutter build apk` tanpa `--split-per-abi` pada proyek yang sama, yang menghasilkan satu *fat APK* berisi kode untuk semua CPU:

```
✓ Built build/app/outputs/flutter-apk/app-release.apk (47.2MB)
```

File	Untuk perangkat
`app-arm64-v8a-release.apk`	Hampir semua HP Android modern (pilih ini jika ragu)
`app-armeabi-v7a-release.apk`	HP Android lama/murah dengan CPU 32-bit
`app-x86_64-release.apk`	Emulator di laptop Intel/AMD, Chromebook
`app-release.apk` (fat APK)	Semua perangkat, tetapi ukurannya hampir tiga kali lipat

Flutter juga otomatis menambahkan `1000 × nomor ABI` ke `versionCode` setiap APK split, sehingga ketiga APK tidak bentrok nomor versinya.

3. Obfuscation dan split-debug-info

Kode Dart di APK bisa dibongkar untuk melihat nama class dan fungsi. `--obfuscate` mengacak nama-nama tersebut, dan `--split-debug-info` memisahkan simbol debug ke folder di laptop Anda sehingga APK lebih kecil. Keduanya selalu dipakai bersamaan.

```
flutter build apk --split-per-abi \
  --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
```

Folder `build/symbols/` berisi file simbol per arsitektur:

```
build/symbols/
├── app.android-arm.symbols
├── app.android-arm64.symbols
└── app.android-x64.symbols
```

Stack trace dari aplikasi yang di-obfuscate tidak bisa dibaca langsung. Simpan folder simbol untuk setiap versi yang dirilis, lalu terjemahkan stack trace dengan:

```
# stack_trace.txt = isi error yang disalin dari logcat/laporan crash
flutter symbolize -i stack_trace.txt -d build/symbols/app.android-arm64.symbols
```

> ⚠️ **Efek samping obfuscation**
>
> Nama class ikut diacak, sehingga kode yang bergantung pada nama tipe akan berubah perilaku, misalnya `runtimeType.toString()` dan `toString()` pada enum. Jangan menyimpan atau membandingkan data berdasarkan nilai-nilai tersebut; gunakan string tetap yang Anda tulis sendiri.

4. Build App Bundle untuk Google Play

```
flutter build appbundle \
  --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
```

```
Running Gradle task 'bundleRelease'...                             19.8s
✓ Built build/app/outputs/bundle/release/app-release.aab (45.1MB)
```

Ukuran AAB terlihat besar karena berisi semua arsitektur. Pengguna Play Store hanya mengunduh bagian yang sesuai dengan HP-nya, sehingga ukuran unduhannya mirip APK split.

5. Menguji build rilis di HP sungguhan

Build rilis **wajib** diuji sebelum dikumpulkan, karena beberapa bug hanya muncul di mode release (permission INTERNET, konfigurasi kosong, kode di dalam `assert`).

```
# Cara 1: jalankan langsung dalam mode release (HP tersambung USB, USB debugging aktif)
flutter run --release --dart-define-from-file=config/prod.json

# Cara 2: pasang APK yang sudah di-build ke perangkat yang tersambung
flutter install --use-application-binary=build/app/outputs/flutter-apk/app-arm64-v8a-release.apk

# Cara 3: lewat adb
adb install -r build/app/outputs/flutter-apk/app-arm64-v8a-release.apk

# Melihat log aplikasi yang sedang berjalan
flutter logs
```

Cara 4: kirim file APK ke HP (Google Drive, Telegram, kabel USB), buka dari aplikasi File, lalu izinkan “Instal dari sumber tidak dikenal” bila diminta.

Daftar uji minimal untuk build rilis Catatan Kuliah:

1. Aplikasi terbuka tanpa layar abu-abu; ikon, nama, dan splash sesuai.
2. Register, login, dan logout berhasil (membuktikan INTERNET dan konfigurasi Supabase benar).
3. Tambah, ubah, dan hapus catatan berhasil.

4. Matikan internet (mode pesawat): catatan dari cache Hive tetap tampil.
5. Tutup paksa lalu buka lagi: sesi login dan data cache masih ada.
6. Putar layar dan ubah ukuran font sistem ke terbesar: tidak ada overflow.

> ⚠️ **Kesalahan umum**
>
> - Mengirim `app-x86_64-release.apk` ke teman yang memakai HP biasa. Aplikasi tidak bisa dipasang (“App not installed”). Kirim `arm64-v8a`.
> - Memasang APK rilis di atas aplikasi debug yang sudah terpasang. Tanda tangannya berbeda sehingga gagal; hapus dulu aplikasi lama.
> - Mengumpulkan APK debug (`build/app/outputs/flutter-apk/app-debug.apk`). Ukurannya besar, lambat, dan menampilkan pita DEBUG.
> - Membuang folder `build/symbols` dengan `flutter clean` sebelum menyalinnya. Simpan di luar folder `build/` (misalnya `symbols/1.2.0+7/`, tetapi jangan di-commit).

8. Google Play Console & Gambaran iOS

Bagian ini memberi gambaran alur publikasi. Untuk mata kuliah ini, publikasi ke Play Store **tidak wajib**, tetapi memahami alurnya penting jika suatu hari Anda merilis aplikasi sungguhan atau bekerja di perusahaan.

Google Play Console

Akun developer Google Play dibuat di play.google.com/console dengan biaya pendaftaran satu kali dan verifikasi identitas. Setelah akun aktif, alurnya:

1. **Create app**: isi nama aplikasi, bahasa default, jenis (app/game), gratis atau berbayar.
2. **Lengkapi "Set up your app"**: kebijakan privasi (URL), akses aplikasi (jika butuh login, berikan akun demo untuk reviewer), iklan, rating konten, target usia, dan formulir **Data safety** (data apa yang dikumpulkan: email, catatan pengguna, dll.).
3. **Store listing**: deskripsi singkat dan lengkap, ikon 512×512, feature graphic 1024×500, minimal beberapa screenshot HP.
4. **Testing → Internal testing**: buat rilis baru, unggah file `app-release.aab`, isi catatan rilis.
5. **Tambahkan penguji**: daftar email (maksimal 100 orang). Penguji membuka tautan undangan, menerima, lalu memasang dari Play Store.
6. Setelah stabil, naikkan ke *closed testing*, lalu *production*. Akun developer pribadi baru biasanya diwajibkan menjalankan closed testing dengan sejumlah penguji selama periode tertentu sebelum boleh rilis production; cek syarat terbaru di Play Console.

Jalur	Penguji	Review Google	Kegunaan
Internal testing	Maks. 100 email	Sangat cepat / tanpa menunggu lama	Tim sendiri, dosen, asisten
Closed testing	Daftar email atau Google Group	Ya	Beta tester terbatas
Open testing	Siapa saja yang bergabung	Ya	Beta publik
Production	Semua pengguna Play Store	Ya	Rilis resmi

Play App Signing

Saat mengunggah AAB pertama, Play Console menawarkan **Play App Signing**: Google menyimpan kunci penandatanganan aplikasi yang sesungguhnya, sedangkan keystore Anda

menjadi *upload key* (itulah sebabnya aliasnya `upload`). Keuntungannya, jika upload key hilang, Anda bisa meminta reset ke Google tanpa kehilangan aplikasi.

Jika memakai fitur Google Sign-In atau API yang meminta sidik jari SHA-1/SHA-256, ambil nilainya dari halaman *App integrity* di Play Console (kunci aplikasi), bukan hanya dari keystore lokal. Sidik jari keystore lokal bisa dilihat dengan:

```
keytool -list -v -keystore android/app/upload-keystore.jks -alias upload
```

Setiap update

```
# 1. Naikkan versi di pubspec.yaml, misalnya 1.2.0+7 menjadi 1.2.1+8
# 2. Build ulang
flutter build appbundle --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
# 3. Unggah app-release.aab ke jalur testing, isi catatan rilis
# 4. Simpan folder build/symbols untuk versi ini
```

iOS secara singkat

Build iOS hanya bisa dilakukan di **macOS dengan Xcode**. Gambaran alurnya:

1. Daftar **Apple Developer Program** (berbayar tahunan). Tanpa akun berbayar, aplikasi hanya bisa dipasang ke iPhone sendiri lewat kabel dan berlaku beberapa hari.
2. Buka `ios/Runner.xcworkspace` di Xcode, atur *Bundle Identifier* dan *Team* di tab *Signing & Capabilities*.
3. Isi alasan permission di `Info.plist` (bagian 4).
4. Build arsip: `flutter build ipa --dart-define-from-file=config/prod.json`.
5. Unggah ke **App Store Connect** (lewat aplikasi Transporter atau Xcode), lalu bagikan ke penguji lewat **TestFlight**.
6. Kirim untuk review App Store saat siap rilis.

Android	Padanan iOS
Play Console	App Store Connect
Internal testing	TestFlight
applicationId	Bundle Identifier
Keystore + key.properties	Certificate + Provisioning Profile (dikelola Xcode)
`flutter build appbundle`	`flutter build ipa`
AndroidManifest permission	Info.plist usage description

⚠️ **Kesalahan umum**

- Mengunggah APK ke Play Console. Aplikasi baru wajib memakai AAB.
- Aplikasi butuh login tetapi tidak memberi akun demo ke reviewer, sehingga rilis ditolak.
- Formulir Data safety tidak sesuai kenyataan (misalnya menyatakan tidak mengumpulkan email padahal ada fitur register).
- Tidak punya kebijakan privasi. Untuk aplikasi yang menyimpan data pengguna, URL kebijakan privasi wajib ada (boleh berupa halaman GitHub Pages sederhana).

9. Panduan Capstone Project

> 🏆 **Ketentuan Capstone Project (Bobot 35% Nilai Akhir)**
>
> Mahasiswa secara berkelompok (2-3 orang) atau mandiri membangun satu aplikasi mobile lengkap yang menyelesaikan permasalahan nyata di lingkungan kampus atau masyarakat. Capstone adalah tempat menggabungkan semua materi: widget dan layout (Modul 03-06), navigasi (07), Provider (08), API dan penyimpanan (09-12), arsitektur proyek (13-14), testing (15), dan rilis (16).

Boleh memakai arsitektur Catatan Kuliah sebagai dasar

Anda **diizinkan dan disarankan** memakai struktur proyek Modul 13-14 sebagai kerangka: `AuthProvider` + `AuthGate` untuk login Supabase, pola `Repository` dengan remote data source (Supabase) dan cache Hive, serta Provider untuk state halaman. Syaratnya:

- Domain aplikasi **harus berbeda** dari catatan kuliah (tabel, model, dan halaman baru, bukan sekadar mengganti judul).
- Minimal **dua entitas data yang saling berhubungan** (misalnya `laporan` dan `kategori`, atau `produk` dan `pesanan`), bukan hanya satu tabel.
- Sebutkan di README bagian mana yang diadaptasi dari modul.

Struktur folder yang direkomendasikan:

```
lib/
├── main.dart              # inisialisasi Env, Supabase, Hive, MultiProvider
├── config/env.dart        # konfigurasi --dart-define-from-file
├── models/                # class data + fromJson/toJson
├── data/                  # data source (Supabase, Hive) dan repository
├── providers/             # AuthProvider dan provider fitur
├── pages/                 # satu file per halaman
├── widgets/               # widget yang dipakai ulang
└── utils/                 # validator, formatter, konstanta
test/
├── unit/                  # validator, model, provider
├── mock/                  # fake data source
└── widget/                # halaman login, daftar, form
```

Pilihan tema proyek

1. **Sistem Pengaduan Sarana Kampus:** laporan fasilitas rusak dengan foto kamera, kategori, dan status penanganan (baru, diproses, selesai).
2. **Marketplace UMKM Mahasiswa:** katalog produk, keranjang belanja (Provider), riwayat pesanan per pengguna.

3. **Perpustakaan & Peminjaman Buku:** pencarian buku via REST API publik, daftar pinjaman di Supabase, bookmark offline.
4. **Layanan Akademik:** jadwal kuliah, pengajuan surat izin online, pengingat tugas.
5. **Tema bebas** dengan persetujuan dosen paling lambat minggu ke-2 periode capstone.

Fitur wajib

No	Fitur	Keterangan / modul acuan
1	Autentikasi	Register, login, logout, sesi tetap tersimpan saat aplikasi dibuka ulang (Modul 13)
2	CRUD data online	Minimal 2 tabel berelasi di Supabase dengan Row Level Security aktif (Modul 12, 14)
3	Penyimpanan lokal	Cache offline (Hive/SQLite) atau preferensi pengguna (SharedPreferences) yang benar-benar dipakai (Modul 10, 11, 14)
4	State management	Provider dengan pemisahan UI, provider, dan repository (Modul 08, 14)
5	Navigasi	Minimal 5 halaman, ada passing data antarhalaman, ada bottom navigation atau drawer (Modul 07)
6	Form tervalidasi	Minimal 2 form dengan validasi dan pesan error yang jelas (Modul 05)
7	State UI lengkap	Setiap daftar menangani loading, kosong, error, dan offline
8	Testing	Minimal 10 test: unit test (validator/provider), test repository dengan fake, dan 2 widget test (Modul 15)
9	Siap rilis	Ikon dan splash sendiri, nama dan application ID sendiri, key Supabase lewat `--dart-define-from-file`, APK release bertanda tangan (Modul 16)

Nilai tambah (bonus maksimal 10 poin): fitur kamera/galeri dengan Supabase Storage, pencarian dan filter, mode gelap, realtime Supabase, notifikasi lokal, integration test, atau dipublikasikan di jalur internal testing Play Console.

Rubrik penilaian

Kriteria	Bobot	Sangat baik (85-100)	Cukup (60-75)	Kurang (< 60)
Fungsionalitas fitur wajib	25%	Semua fitur wajib berjalan di APK release tanpa crash	Sebagian kecil fitur belum lengkap atau ada bug minor	Beberapa fitur wajib tidak ada atau crash
Arsitektur & kualitas kode	20%	Lapisan UI/provider/repository jelas, `flutter analyze` 0 issue, nama dan struktur rapi	Ada pemisahan tetapi masih ada logika data di widget, beberapa lint	Semua logika di dalam widget, banyak warning
Integrasi data (online & offline)	15%	CRUD berelasi lancar, RLS benar, cache offline berfungsi	CRUD berjalan, offline terbatas	Data hanya lokal atau RLS dimatikan
UI/UX	15%	Material 3 konsisten, bebas overflow, state loading/kosong/error rapi, responsif	Tampilan cukup, ada overflow kecil atau state yang terlewat	Tampilan tidak konsisten, sering overflow
Testing	10%	≥ 10 test bermakna lulus, termasuk fake repository dan widget test	Beberapa test, sebagian hanya kasus sukses	Tidak ada test atau test gagal
Rilis & dokumentasi	10%	APK bertanda tangan, README lengkap sesuai template, tidak ada rahasia di repo	README kurang lengkap, APK debug	Tanpa README atau key Supabase ter-commit
Presentasi & demo	5%	Demo lancar, semua anggota menguasai kode saat tanya jawab	Demo tersendat, sebagian anggota kurang menguasai	Tidak bisa mendemonstrasikan aplikasi

Nilai individu dapat disesuaikan berdasarkan kontribusi yang terlihat di riwayat commit dan jawaban saat tanya jawab.

Timeline (6 minggu)

Minggu	Kegiatan	Luaran (dicek dosen/asisten)
1	Pembentukan kelompok, pemilihan tema, analisis kebutuhan	Proposal 1-2 halaman: masalah, fitur, sketsa layar
2	Desain data dan UI	Skema tabel Supabase + RLS, wireframe semua halaman, repo GitHub dibuat
3	Fondasi: auth, navigasi, model, repository	Login/register berjalan, kerangka halaman
4	Fitur inti: CRUD berelasi, cache offline	Demo progres 50% kepada asisten
5	Penyempurnaan UI, testing, perbaikan bug	Minimal 10 test lulus, `flutter analyze` bersih
6	Rilis dan dokumentasi	APK release, README, video demo, presentasi akhir

Deliverables (yang dikumpulkan)

1. **Repositori GitHub** (publik atau undang dosen): riwayat commit dari semua anggota, tanpa `key.properties`, `*.jks`, dan `config/*.json` berisi key asli.
2. **README.md** sesuai template di bawah.
3. **APK release** `app-arm64-v8a-release.apk` yang sudah ditandatangani, diunggah di bagian *Releases* GitHub.
4. **Video demo** 3-5 menit (YouTube unlisted/Google Drive): alur utama dari register sampai fitur inti, termasuk kondisi offline.
5. **Slide presentasi** (maksimal 10 slide): masalah, solusi, arsitektur, demo, tantangan, pembagian tugas.
6. **Akun demo** (email dan password) untuk penguji, ditulis di README atau formulir pengumpulan.

Template README.md

`README.md`

```
# Lapor Kampus

Aplikasi pelaporan kerusakan fasilitas kampus UNSOED. Mahasiswa memotret
fasilitas yang rusak, memilih kategori, dan memantau status penanganannya.

## Anggota Kelompok
| Nama          | NIM         | Peran                        |
```

```
|---------------|-------------|------------------------------|
| Budi Santoso  | H1D022001   | Auth, repository, testing    |
| Siti Aminah   | H1D022002   | UI halaman laporan, navigasi |
| Rizky Pratama | H1D022003   | Supabase (tabel, RLS), rilis |

## Screenshot
| Login | Daftar Laporan | Detail | Mode Offline |
|-------|----------------|--------|--------------|
| ![](docs/login.png) | ![](docs/list.png) | ![](docs/detail.png) | ![](docs/offline.png) |

## Fitur
- Register, login, logout (Supabase Auth), sesi tersimpan
- CRUD laporan + kategori (tabel berelasi, RLS aktif)
- Cache offline dengan Hive
- Unggah foto ke Supabase Storage (bonus)

## Arsitektur
UI (pages, widgets) -> Provider -> Repository -> Data source (Supabase / Hive).
Diadaptasi dari proyek Catatan Kuliah Modul 13-14: AuthGate, AuthProvider,
pola NoteRepository (diganti menjadi LaporanRepository).

## Skema Database
| Tabel    | Kolom penting                                |
|----------|----------------------------------------------|
| kategori | id, nama                                     |
| laporan  | id, user_id, kategori_id, judul, status, foto |

## Menjalankan Proyek
1. `flutter pub get`
2. Salin `config/example.json` menjadi `config/dev.json`, isi URL dan key Supabase
3. `flutter run --dart-define-from-file=config/dev.json`

## Testing
```bash
flutter test            # 14 test, semua lulus
flutter test --coverage
```

Build Rilis
```bash
flutter build apk --split-per-abi --obfuscate \
  --split-debug-info=build/symbols --dart-define-from-file=config/prod.json
```

Unduh APK
Lihat halaman Releases: app-arm64-v8a-release.apk (v1.0.0+1)

Video Demo
https://youtu.be/xxxxxxxx

Akun Demo
Email: demo@laporkampus.id / Password: diberikan di formulir pengumpulan
```

Panduan presentasi dan demo

- Demo memakai **APK release di HP sungguhan**, bukan emulator dalam mode debug.
- Tunjukkan satu skenario offline: matikan internet, data cache tetap tampil.

- Siapkan diri untuk pertanyaan seperti “tunjukkan di mana data laporan diambil dari Supabase” atau “apa yang terjadi jika token kedaluwarsa”. Setiap anggota harus bisa menjelaskan kode yang ia tulis.
- Boleh memakai bantuan AI atau referensi daring, tetapi sebutkan di README dan pastikan Anda memahami setiap baris kode yang dikumpulkan.

> ⚠️ **Kesalahan umum capstone**
>
> - Satu anggota mengerjakan semuanya dan anggota lain hanya membuat slide. Riwayat commit akan terlihat.
> - Mematikan RLS di Supabase agar “tidak error”. Ini mengurangi nilai integrasi data secara signifikan.
> - Hanya menguji di emulator debug, lalu APK release gagal login saat presentasi (lupa permission INTERNET atau lupa `--dart-define-from-file`).
> - Meng-commit `config/dev.json` berisi key asli. Rotasi key di dashboard Supabase jika sudah terlanjur.
> - Mulai mengerjakan di minggu ke-5. Ikuti timeline; demo progres minggu ke-4 adalah titik pemeriksaan wajib.

Tugas Praktikum: Praktikum 16: Rilis APK Catatan Kuliah & Persiapan Repositori Capstone

Sasaran Praktikum

Mengubah proyek Catatan Kuliah (Modul 13-14) menjadi APK release yang bertanda tangan, aman dari kebocoran key, dan siap dipasang di HP orang lain, sekaligus menyiapkan repositori capstone sesuai standar.

Langkah Pengerjaan

1. Ubah `version` di `pubspec.yaml` menjadi `1.0.0+1`, lalu tampilkan versi aplikasi di halaman Profil memakai `package_info_plus`.
2. Ganti `android:label` menjadi "Catatan Kuliah" dan `applicationId` menjadi `id.ac.unsoed.<nim>.catatan_kuliah`.
3. Buat ikon 1024×1024 (dengan versi foreground transparan) dan logo splash, lalu jalankan `dart run flutter_launcher_icons` dan `dart run flutter_native_splash:create`.
4. Tambahkan permission INTERNET di `android/app/src/main/AndroidManifest.xml`.
5. Pindahkan URL dan publishable key Supabase ke `config/dev.json` dan `config/prod.json`, buat `config/example.json`, perbarui `.gitignore`, dan gunakan class `Env`. Pastikan `git status` tidak menampilkan file konfigurasi rahasia.
6. Buat keystore dengan `keytool`, isi `android/key.properties`, dan tambahkan konfigurasi signing ke `android/app/build.gradle.kts`.
7. Build dengan `flutter build apk --split-per-abi --obfuscate --split-debug-info=build/symbols --dart-define-from-file=config/prod.json`, lalu verifikasi tanda tangan dengan `apksigner verify --print-certs` (screenshot hasilnya).
8. Pasang `app-arm64-v8a-release.apk` di HP fisik (milik sendiri atau teman) dan jalankan 6 butir daftar uji build rilis dari bagian 7. Catat hasilnya dalam tabel di README.
9. Unggah APK ke bagian Releases GitHub dan lengkapi README sesuai template capstone (screenshot, fitur, cara menjalankan, cara build).
10. Bonus: build `flutter build appbundle` dan bandingkan ukurannya dengan APK split dan fat APK.

Output yang Diharapkan

APK release bertanda tangan keystore sendiri (bukan Android Debug) yang terpasang di HP fisik dengan ikon, nama, dan splash kustom; login dan CRUD Supabase berjalan; repositori GitHub tanpa key/keystore, dengan README lengkap dan APK di halaman Releases.

Rubrik Penilaian

APK release berhasil dibuat dan bertanda tangan kunci rilis (30%), branding: ikon, splash, nama, application ID (20%), konfigurasi aman: dart-define-from-file dan .gitignore benar (20%), hasil uji di HP fisik terdokumentasi (15%), README dan Releases GitHub (15%).

Kuis Pemahaman

1. Pada baris version: 1.4.2+15 di pubspec.yaml, angka 15 berfungsi sebagai...

A. versionName yang dilihat pengguna di Play Store

B. versionCode, bilangan bulat yang wajib naik setiap kali mengunggah build baru

C. minSdk Android

D. Jumlah halaman aplikasi

2. Aplikasi Catatan Kuliah bisa login saat flutter run, tetapi APK release yang dipasang di HP selalu gagal terhubung ke Supabase. Penyebab yang paling mungkin adalah...

A. Supabase tidak mendukung Android

B. Permission INTERNET hanya ada di manifest debug, belum ditambahkan ke android/app/src/main/AndroidManifest.xml

C. APK release tidak bisa memakai package http

D. Ikon aplikasi belum diganti

3. Mengapa kode berikut bisa menghasilkan nilai kosong di build release? final url = String.fromEnvironment('SUPABASE_URL');

A. Karena nama kunci harus huruf kecil

B. Karena String.fromEnvironment hanya dijamin bekerja jika dipanggil dalam konteks const

C. Karena --dart-define hanya berlaku di iOS

D. Karena nilai harus dibaca dengan SharedPreferences

4. Apa tujuan utama memakai --dart-define-from-file=config/prod.json dengan config/*.json di .gitignore?

A. Membuat key Supabase mustahil ditemukan oleh siapa pun

B. Agar key tidak tersebar lewat repositori git dan konfigurasi dev/prod mudah ditukar tanpa mengubah kode

C. Agar APK lebih kecil

D. Agar RLS Supabase tidak diperlukan lagi

5. Mengapa file key.properties dan keystore (.jks) harus masuk .gitignore?

A. Karena GitHub tidak bisa menyimpan file biner

B. Karena siapa pun yang memegang keystore beserta password-nya bisa menandatangani update palsu atas nama Anda

C. Agar ukuran repositori di bawah 1 GB

D. Karena keystore hanya bisa dibaca Windows

6. Apa keuntungan flutter build apk --split-per-abi dibandingkan flutter build apk biasa?

A. Menghasilkan APK terpisah per arsitektur CPU sehingga setiap file jauh lebih kecil daripada fat APK

B. Membuat aplikasi bisa berjalan di iOS

C. Menghapus kebutuhan keystore

D. Mengubah APK menjadi AAB

7. Anda mem-build dengan --obfuscate --split-debug-info=build/symbols. Apa yang harus dilakukan agar stack trace crash dari pengguna bisa dibaca?

A. Tidak perlu apa-apa, stack trace selalu terbaca

B. Menyimpan folder simbol untuk versi tersebut dan menerjemahkan stack trace dengan flutter symbolize

C. Build ulang tanpa obfuscate lalu meminta pengguna mengulang crash

D. Menghapus folder build/symbols agar APK lebih aman

8. Format apa yang wajib diunggah ke Google Play Console untuk aplikasi baru, dan jalur apa yang paling cocok untuk membagikan ke dosen dan anggota kelompok?

A. APK, jalur production

B. AAB (Android App Bundle), jalur internal testing

C. IPA, jalur TestFlight

D. ZIP berisi kode sumber, jalur open testing

Kunci Jawaban & Pembahasan

1. B — Bagian sebelum + adalah versionName (1.4.2), bagian setelahnya adalah versionCode (15). Play Console menolak unggahan dengan versionCode yang sudah pernah dipakai, dan Android menolak downgrade.

2. B — flutter create menaruh permission INTERNET di manifest debug (untuk hot reload). Build release hanya memakai manifest main, sehingga permission harus ditambahkan di sana.

3. B — Nilai --dart-define ditanam saat kompilasi. Dokumentasi Dart hanya menjamin String.fromEnvironment terbaca dalam konteks const, sehingga tulis static const url = String.fromEnvironment('SUPABASE_URL');.

4. B — Nilai dart-define tetap tertanam di APK, jadi bukan rahasia mutlak. Manfaatnya adalah key tidak ikut ter-commit dan lingkungan mudah diganti. Keamanan data tetap bergantung pada RLS, dan service_role key tidak boleh ada di aplikasi.

5. B — Keystore adalah identitas kriptografis pemilik aplikasi. Android hanya menerima update yang ditandatangani dengan kunci yang sama, sehingga kebocoran keystore memungkinkan pemalsuan update.

6. A — APK biasa berisi kode native untuk semua CPU (pada contoh modul 47,2 MB). Split per ABI menghasilkan file terpisah seperti arm64-v8a (16,1 MB) yang cukup untuk hampir semua HP modern.

7. B — Obfuscation mengacak nama fungsi. File .symbols menyimpan pemetaan aslinya, sehingga flutter symbolize -i stack.txt -d build/symbols/app.android-arm64.symbols bisa mengembalikan nama yang terbaca.

8. B — Aplikasi baru di Play Store wajib memakai AAB. Internal testing memungkinkan hingga 100 penguji berdasarkan email dan prosesnya paling cepat, cocok untuk tim dan dosen.